

```

1  ; *****
2  ; RETRODOS.SYS (PCDOS 7.1 Kernel) - RETRO DOS v5.0 by ERDOGAN TAN - 12/09/2023
3  ; -----
4  ; Last Update: 04/02/2026 - Retro DOS v5.0 (Modified PCDOS 7.1)
5  ; -----
6  ; Beginning: 26/12/2018 (Retro DOS 4.0), 01/10/2022 (Retro DOS 4.2)
7  ; -----
8  ; Assembler: NASM version 2.15
9  ; -----
10 ; ((nasm retrodos5.s -l retrodos5.txt -o PCDOS.SYS -Z error.txt))
11 ; -----
12 ; Included binary file: IBMDOS7.BIN (Retro DOS 5.0 - PCDOS 7.1 IBMDOS.COM)
13 ; *****
14 ;
15 ; 12/09/2023 - Retro DOS v5.0 Kernel -dosbios- ('ibmbio7.s')
16 ; Modified from 'iosys6.s' (11/09/2023, Retro DOS v4.2 Kernel's IO.SYS) file
17 ; as below:
18 ;
19 ; 1) Retro DOS v4.2 IO.SYS is based on disassembled source code
20 ; of MSDOS 6.21 IO.SYS, derived using MSDOS 6.0 source code.
21 ;
22 ; 2) Labels, names, comments, explanations and structure definitions
23 ; about procedures and code details are almost entirely taken from
24 ; the original MSDOS 6.0 source code, except for the details that
25 ; Erdogan Tan personally experienced. Some of them are incompatible
26 ; with PCDOS 7.1 code. But they have not been deleted to preserve
27 ; the originality of the descriptions.)
28 ;
29 ; 3) 'ibmbio7.s' contains the BIOSLOADER (MSLOADER) section located in
30 ; the 1st 4 sectors of the IBMBIO.COM file on disk. This is a method
31 ; from older DOS versions (3 sectors for MSDOS 6.22).
32 ; The MSDOS/PCDOS boot sector code only reads these MSLOADER/BIOSLOADER
33 ; sectors and transfers control to the MSLOADER/BIOSLOADER code.
34 ; BUT!!! The Retro DOS v3 (& v5) boot sector code loads the entire
35 ; MSDOS.SYS/PCDOS.SYS -combined- kernel file into memory at once.
36 ; So, hence the Retro DOS boot sector code, 'retrodos5.s' file
37 ; contains slightly different IO.SYS/IBMBIO.COM Initialization code
38 ; than the original PCDOS/MSDOS. It does not include
39 ; the MSLOADER/BIOSLOADER section. The 'retrodos5.s' and 'ibmbio7.s'
40 ; files are almost identical except their INIT codes.)
41 ;
42 ; ('iosys6.s' has been converted to 'ibmbio7.s' and 'retrodos42.s' has been
43 ; converted to 'retrodos5.s'. 'ibmbio7.s' is IBMBIO.COM source code file
44 ; while 'retrodos5.s' is source code of Retro DOS v5 kernel file 'PCDOS.SYS'.
45 ; 'retrodos5.s' includes 'ibmdos7.bin' or IBMDOS.COM as binary file.)
46 ;
47 ; -----
48 ;
49 ; 20/12/2022 - Modifications for initiating IO.SYS by Retro DOS v2 boot sector
50 ;
51 ; (Retro DOS v2 BS loads IO.SYS & MSDOS.SYS as single kernel file
52 ; with name of 'MSDOS.SYS'. Retro DOS init code -for IO.SYS init-
53 ; is different than original MSDOS IO.SYS LOADER and INIT code.)
54 ;
55 ; ((RETRODOS.SYS/MSDOS.SYS can be loaded by a fake IO.SYS for
56 ; using it with MSDOS 5.0 boot sector & as bootable MSDOS disk.
57 ; For that, fake IO.SYS must load 'MSDOS.SYS' at 1000h:0000h.))
58 ;
59 ; 18/12/2022 - Modified MSDOS 5.0 IO.SYS (for using with MSDOS 5 boot sector)
60 ; 09/12/2022 - Multisection binary file format (BIOSDATA & BIOSCODE sections)
61 ; 01/10/2022 - Erdogan Tan (Istanbul)
62 ;
63 ; Note: This code is a part of Retro DOS 4.0 kernel source code
64 ; (as included binary, 'IOSYS5.BIN')
65 ; Equivalent of MSDOS 5.0 IO.SYS, BIOSCODE and BIOSDATA and SYSINIT
66 ; (except MSLOAD code)
67 ;
68 ; -----
69 ; Retro DOS v2 (v3) boot sector loads RETRODOS.SYS (MSDOS.SYS)
70 ; at 1000h:0000h and loader (initialization) part of RETRODOS kernel
71 ; moves IO.SYS (DOSBIOSCODE & DOSBIOSDATA, 'IOSYS5.BIN') to 70h:0000h.
72 ; Then SYSINIT code to the next segment (46Dh for original MSDOS 5.0)..
73 ; SYSINIT code relocates itself and DOSBIOSCODE and MSDOS.SYS
74 ; (MSDOS5.BIN) according to request/setting in 'config.sys' file.
75 ; -----
76 ;
77 ; =====
78 ; Modified from 'retrodos3.s', Retro DOS v3.0 Kernel (IBMBIO.COM) Source code
79 ; by Erdogan Tan, 10/09/2018
80 ; =====
81 ;
82 ; MSBIO (IO.SYS 6.0) source files:
83 ; MSBIO1.ASM,MSCHAR.ASM,MSDISK.ASM,MSDIOCTL.ASM,MSINT13.ASM,MSBIO2.ASM
84 ; MSINIT.ASM,SYSINIT1.ASM,SYSCONF.ASM,SYSPRE.ASM,SYSINIT2.ASM
85 ; SYSIMES.ASM,POWER.ASM,PTIME.ASM,MSEND.ASM
86 ;
87 ; =====
88 ; MSBIO
89 ; =====
90 ; msbio1+mschar+msdisk+msdioclt+msint13+msbio2+
91 ; msinit+sysinit1+sysconf+syspre+sysinit2+sysimes+power+ptime+
92 ; msend,msbio,msbio;
93 ;
94 ; =====
95 ; RETRO DOS kernel versions by Erdogan Tan (2018-2022)
96 ; =====
97 ;
98 ; Retro DOS v1.0 == MSDOS 1.25 -- derived from MSDOS 1.25 source code
99 ; Retro DOS v2.0 == MSDOS 2.11 -- derived from MSDOS 2.11 source code
100 ; Retro DOS v3.0 == MSDOS 3.30 -- derived from MSDOS 3.3 & 6.0 source code
101 ; Retro DOS v4.0 == MSDOS 6.21 -- derived from MSDOS 6.0 source code (2019) (*)
102 ; Retro DOS v4.0 == MSDOS 5.0+ -- derived from MSDOS 6.0 source code (2022) (***)
103 ; Retro DOS v4.1 == MSDOS 5.0+ -- will be optimized -shortened- version (2023)
104 ; Retro DOS v4.2 == MSDOS 6.21 -- will be MSDOS 6.21 (6.22) compatible (2023)(?)
105 ; Retro DOS v5.0 == PCDOS 7.10 -- will be derived from IBM PCDOS 7.1 source code
106 ;
107 ; (*) unfinished, draft, canceled (failed in 2019)
108 ; (**) MSDOS 5.0 IO.SYS & SYSINIT, MSDOS 5.0-6.22 mixed MSDOS.SYS (succeeded)
109 ; (?) MSDOS 6.21 IO.SYS & SYSINIT, MSDOS 6.21 MSDOS.SYS except doublespace
110 ;
111 ; Disassembly: (reverse engineering via IDA Pro Free)
112 ;
113 ; Retro DOS v1.0 <-- IBM PCDOS 1.1
114 ; Retro DOS v2.0 <-- IBM PCDOS 2.1 & MSDOS 2.11
115 ; Retro DOS v3.0 <-- IBM PCDOS 3.3 & MSDOS 3.3
116 ; Retro DOS v4.0 <-- MSDOS 6.21 ; 2018-2019 (*)
117 ; Retro DOS v4.0 <-- MSDOS 5.0 ; 2022 (**)
118 ; Retro DOS v5.0 <-- IBM PCDOS 7.1
119 ;
120 ; -----
121 ; MSDOS 6.21 IO.SYS (13/02/1994)
122 ; -----
123
124 SECTOR_SIZE equ 0200h ; size of a sector

```

```

125 PAUSE_KEY equ 7200h ; scancode + charcode of PAUSE key
126 KEYBUF_NEXT equ 041Ah ; next character in keyboard buffer
127 KEYBUF_FREE equ 041Ch ; next free slot in keyboard buffer
128 KEYBUF equ 041Eh ; keyboard buffer data
129 LOGICAL_DRIVE equ 0504h ; linear address of logical drive byte
130 ;DOS_SEGMENT equ 00BFh ; v1.1 ; segment in which DOS will run
131 DOS_SEGMENT equ 00C4h ; Retro DOS v1.0 - 13/02/2018
132 BIO_SEGMENT equ 0060h ; segment in which BIO is running
133
134 ; 24/02/2018 (Retro DOS 2.0 - MSDOS 3.3 "DISKPRM.INC" - 24/07/1987)
135 ; The following structure defines the disk parameter table
136 ; pointed to by Interrupt vector 1EH (location 0:78H)
137
138 struc DISK_PARMS
139 00000000 ?? .DISK_SPECIFY_1: resb 1
140 00000001 ?? .DISK_SPECIFY_2: resb 1
141 00000002 ?? .DISK_MOTOR_WAIT: resb 1 ; wait till motor off
142 00000003 ?? .DISK_SECTOR_SIZE: resb 1 ; Bytes/Sector (2 = 512)
143 00000004 ?? .DISK_EOT: resb 1 ; Sectors per track (MAX)
144 00000005 ?? .DISK_RW_GAP: resb 1 ; Read Write Gap
145 00000006 ?? .DISK_DTL: resb 1
146 00000007 ?? .DISK_FORMAT_GAP: resb 1 ; Format Gap Length
147 00000008 ?? .DISK_FILL: resb 1 ; Format Fill Byte
148 00000009 ?? .DISK_HEAD_STTL: resb 1 ; Head Settle Time (MSec)
149 0000000A ?? .DISK_MOTOR_STRT: resb 1 ; Motor start delay
150 .size:
151 endstruc
152
153 ; 09/03/2019 - Retro DOS v4.0
154 ; -----
155 ; MSEQU.INC, MSDOS 6.0, 1991
156
157 ftoobig equ 80h
158 fbig equ 40h
159 ; 12/09/2023
160 fbigbig equ 20h ; Retro DOS 5.0 ; PCDOS 7.1 ; FAT32 FS flag
161 romstatus equ 1
162 romread equ 2
163 romwrite equ 3
164 romverify equ 4
165 romformat equ 5
166
167 ; 26/12/2018 (Retro DOS 4.0 - MSDOS 6.0 "MSBDS.INC" - 1991)
168 ; -----
169 ; 24/02/2018 (Retro DOS 2.0 - MSDOS 3.3 "MSBDS.INC" - 24/07/1987)
170 ;
171 ; BDS is the Bios Data Structure.
172 ;
173 ; There is one BDS for each logical drive in the system. All the BDS's
174 ; are linked together in a list with the pointer to the first BDS being
175 ; found in START_BDS. The BDS hold various values important to the disk
176 ; drive. For example there is a field for last time accesses. As actions
177 ; take place in the system the BDS are update to reflect the actions.
178 ; For example is there is a read to a disk the last access field for the
179 ; BDS for that drive is update to the current time.
180 ;
181 ; Values for various flags in BDS.flags.
182 ;
183
184 fnon_removable equ 01h ;For non-removable media
185 fchangeline equ 02h ;If changeline supported on drive
186 return_fake_bpb equ 04h ; When set, don't do a build BPB
187 ; just return the fake one
188 good_tracklayout equ 08h ; The track layout has no funny sectors
189 fi_am_mult equ 10h ;If more than one logical for this physical
190 fi_own_physical equ 20h ;Signify logical owner of this physical
191 fchanged equ 40h ;Indicates media changed
192 set_dasd_true equ 80h ; Set DASD before next format
193 fchanged_by_format equ 100h ;Media changed by format
194 ; MSDOS 6.0
195 unformatted_media equ 200h ;Fixed disk only
196
197 ;
198 ; Various form factors to describe media
199 ;
200
201 ff48tpi equ 0
202 ff96tpi equ 1
203 ffSmall equ 2
204 ffHardFile equ 5
205 ffOther equ 7
206 ; MSDOS 6.0 ("MSBDS.INC", 1991)
207 ff288 equ 9 ; 2.88 MB drive
208 ; Retro DOS v4.0 feature only !
209 ;ff144 equ 10 ; 1.44 MB drive
210
211 ; 12/09/2023
212 ; Retro DOS v4 (MDOS 5.0-6.22) BDS structure
213 ; -----
214 ; 100 bytes
215
216 %if 0
217
218 ; 26/05/2019
219
220 struc BDS ; BDS_Type
221 .link: resd 1 ; Link to next BDS
222 .drivenum: resb 1 ; Physical drive number
223 .drivelet: resb 1 ; DOS drive number
224
225 ;we want to embed a BPB declaration here, but we can't initialize
226 ;it properly if we do, so we duplicate the byte/word/dword architecture
227 ;of the BPB declaration.
228 .BPB:
229 .bytespersec: resw 1 ; bytes per sectors ; def = 512
230 .secpclus: resb 1 ; sectors per cluster
231 .resectors: resw 1 ; reserved sectors
232 .fats: resb 1 ; number of fats
233 .direntries: resw 1 ; number of root directory entries
234 .totalsecs16: resw 1 ; total sectors on medium
235 .media: resb 1 ; media descriptor byte ; def = 0F8h
236 .fatsecs: resw 1 ; number of fat sectors
237 .secptrack: resw 1 ; sectors per track
238 .heads: resw 1 ; number of heads
239 .hiddensecs: resw 1 ; hidden sectors
240 ; MSDOS 6.0
241 .hiddensecs: resd 1 ; hidden sectors
242 .totalsecs32: resd 1 ; big total sectors
243 ;
244 .fatsiz: resb 1 ; flags...
245 .opcnt: resw 1 ; open ref. count
246 ;.valid: resb 12 ; volume ID of medium
247 .formfactor: resb 1 ; form factor index
248 .flags: resw 1 ; various flags ; def: 0020h

```

```

249 .cylinders: resw 1          ; number of cylinders
250 ;
251 .R_BPB:                    ; recommended BPB
252 .rbytespersec: resw 1
253 .rsecperclus: resb 1
254 .rsectors: resw 1
255 .rfats: resb 1
256 .rdirentries: resw 1
257 .rtotalsecs16: resw 1
258 .rmedia: resb 1
259 .rfatsecs: resw 1
260 .rsecpertrack: resw 1
261 .rheads: resw 1
262 .rhidsecs: resd 1
263 .rtotalsecs32: resd 1
264 .rreserved: resb 6          ; not used (reserved)
265 ;
266 .track: resb 1              ; last track accessed on drive
267 .bdsm_ismini:
268 .tim_lo: resw 1              ; time of last access. keep
269 .bdsm_hidden_trks:
270 .tim_hi: resw 1              ; these contiguous.
271 .valid: resb 12             ; volume id of medium
272 ;db "NO NAME",0
273 .vol_serial: resd 1          ; current volume serial number from boot record
274 .filesys_id: resb 9          ; current file system id from boot record
275 ;db "FAT12",0
276 .size:
277 endstruc
278
279 %endif
280
281 ; 12/09/2023 - Retro DOS 5.0 - PCDOS 7.1 (FAT32 compatible) BDS structure
282 ; -----
283 ; 150 bytes
284
285 %if 1
286
287 struc BDS ; BDS_Type
288 .link: resd 1 ; Link to next BDS
289 .drivenum: resb 1 ; Physical drive number
290 .drivelet: resb 1 ; DOS drive number
291
292 ;We want to embed a BPB declaration here, but we can't initialize
293 ;it properly if we do, so we duplicate the byte/word/dword architecture
294 ;of the BPB declaration.
295 .BPB:
296 .bytespersec: resw 1 ; bytes per sectors ; def = 512
297 .secperclus: resb 1 ; sectors per cluster
298 .resectors: resw 1 ; reserved sectors
299 .fats: resb 1 ; number of fats
300 .direntries: resw 1 ; number of root directory entries
301 .totalsecs16: resw 1 ; total sectors on medium
302 .media: resb 1 ; media descriptor byte ; def = 0F8h
303 .fatsecs16: resw 1 ; number of fat sectors
304 .secpertrack: resw 1 ; sectors per track
305 .heads: resw 1 ; number of heads
306 .hidensecs: resd 1 ; hidden sectors
307 .totalsecs32: resd 1 ; big total sectors
308 ; ----- FAT32 extensions to BDS ----- Retro DOS 5.0 -----
309 .fatsecs32: resd 1 ; BPB_FATSz32 ; FAT32 FAT size in sectors
310 .extflags: resw 1 ; BPB_ExtFlags ; FAT32 Extended Flags
311 .fsver: resw 1 ; BPB_FSVer ; FAT32 volume version number
312 .rootdirclust: resd 1 ; BPB_RootClus ; FAT32 root dir's 1st clust num
313 .fsinfo: resw 1 ; BPB_FSInfo ; FAT32 FSINFO sector number
314 .bkbootsec: resw 1 ; BPB_BkBootSec ; FAT32 backup boot sector number
315 .reserved: resb 12 ; BPB_Reserved ; FAT32 reserved field = 0, 12 bytes
316 ; -----
317 .fatsiz: resb 1 ; flags...
318 .opcnt: resw 1 ; open ref. count
319 .formfactor: resb 1 ; form factor index
320 .flags: resw 1 ; various flags ; def: 0020h
321 .cylinders: resw 1 ; number of cylinders
322 ;
323 .R_BPB:                    ; recommended BPB
324 .rbytespersec: resw 1
325 .rsecperclus: resb 1
326 .rsectors: resw 1
327 .rfats: resb 1
328 .rdirentries: resw 1
329 .rtotalsecs16: resw 1
330 .rmedia: resb 1
331 .rfatsecs: resw 1
332 .rsecpertrack: resw 1
333 .rheads: resw 1
334 .rhidsecs: resd 1
335 .rtotalsecs32: resd 1
336 ; ----- FAT32 extensions to BDS ----- Retro DOS 5.0
337 .rfatsecs32: resd 1 ;
338 .rextflags: resw 1 ;
339 .rfsver: resw 1 ;
340 .rrootdirclust: resd 1 ;
341 .rfsinfo: resw 1 ; default/initial value = -1
342 .rbkbootsec: resw 1 ; default/initial value = -1
343 .rreserved: resb 12 ; default value = 0
344 ; -----
345 ;
346 .track: resb 1 ; last track accessed on drive (def=-1)
347 .bdsm_ismini:
348 .tim_lo: resw 1 ; time of last access. keep
349 .bdsm_hidden_trks:
350 .tim_hi: resw 1 ; these contiguous.
351 .valid: resb 12 ; volume id of medium
352 ;db "NO NAME",0
353 .vol_serial: resd 1 ; current volume serial number from boot record
354 .filesys_id: resb 9 ; current file system id from boot record
355 ;db "FAT12",0
356 .size:
357 endstruc
358
359 %endif
360 ; -----
361
362 ;The assembler will generate bad data for "size bds_valid",
363 ;so we'll define an equate here.
364
365 VOLID_SIZ equ 12
366
367 ;bdsm_ismini equ bds_tim_lo ; overlapping bds_tim_lo
368 ;bdsm_hidden_trks equ bds_tim_hi ; overlapping bds_tim_hi
369
370 max_mini_dsk_num equ 23 ; max # of mini disk ibmbio can support
371
372 ; 29/12/2018

```

```

373 ; Retro DOS v4.0
374 ;
375 ; MSDOS 6.0 - BOOTFORM.INC
376
377 BOOT_SIZE EQU 512
378 EXT_BOOT_SIGNATURE EQU 29h ; 41 ; Extended boot signature
379
380 %define BOOT_SIGNATURE [BOOT_SIZE-2]
381
382 struct EBPB ; EXT_BPB_INFO
383 00000000 ???? .BYTESPERSECTOR: resw 1
384 00000002 ?? .SECTORS PER CLUSTER: resb 1
385 00000003 ???? .RESERVED SECTORS: resw 1
386 00000005 ?? .NUMBER OF FATs: resb 1
387 00000006 ???? .ROOT ENTRIES: resw 1
388 00000008 ???? .TOTAL SECTORS: resw 1
389 0000000A ?? .MEDIAD ESCRIPTOR: resb 1
390 0000000B ???? .SECTORS PER FAT: resw 1
391 0000000D ???? .SECTORS PER TRACK: resw 1
392 0000000F ???? .HEADS: resw 1
393 00000011 ???????? .HIDDEN SECTORS: resd 1
394 00000015 ???????? .BIG TOTAL SECTORS: resd 1
395 .size:
396 endstruc
397
398 ;EXT_PHYDRV, EXT_CURHD included in the header for OS2.
399 struct EXT_BOOT ; EXT_IBMBOOT_HEADER
400 00000000 ?????? .JUMP: resb 3
401 00000003 ?????????????? .OEM: resb 8
402 0000000B <res 19h> .BPB: resb EBPB.size ; 25 bytes
403 00000024 ?? .PHYDRV: resb 1
404 00000025 ?? .CURHD: resb 1
405 00000026 ?? .SIG: resb 1
406 00000027 ???????? .SERIAL: resd 1
407 0000002B <res Bh> .VOL_LABEL: resb 11
408 00000036 ?????????????? .SYSTEM_ID: resb 8
409 .size:
410 endstruc
411
412 ; 12/09/2023
413 ; -----
414 ; Retro DOS v5.0 (PCDOS 7.1) - FAT32 Boot Sector Parameters
415
416 struct XBPB ; FAT32_BPB_INFO ; 12/09/2023
417 00000000 ???? .BYTESPERSECTOR: resw 1
418 00000002 ?? .SECTORS PER CLUSTER: resb 1
419 00000003 ???? .RESERVED SECTORS: resw 1
420 00000005 ?? .NUMBER OF FATs: resb 1
421 00000006 ???? .ROOT ENTRIES: resw 1
422 00000008 ???? .TOTAL SECTORS: resw 1
423 0000000A ?? .MEDIAD ESCRIPTOR: resb 1
424 0000000B ???? .SECTORS PER FAT: resw 1
425 0000000D ???? .SECTORS PER TRACK: resw 1
426 0000000F ???? .HEADS: resw 1
427 00000011 ???????? .HIDDEN SECTORS: resd 1
428 00000015 ???????? .BIG TOTAL SECTORS: resd 1
429 ;..... FAT32 ..... + 28
430 00000019 ???????? .FATSIZE32: resd 1
431 0000001D ???? .EXTFLAGS: resw 1
432 0000001F ???? .FSVER: resw 1
433 00000021 ???????? .ROOTDIRCLUSTER: resd 1
434 00000025 ???? .FSINFOSECTOR: resw 1 ; (offset from FAT32 bs)
435 00000027 ???? .BACKUPBOOTSECTOR: resw 1 ; (offset from FAT32 bs)
436 00000029 <res Ch> .RESERVEDBYTES: resb 12 ; (zero bytes)
437 .size:
438 endstruc
439
440 struct FAT32_EXT_BOOT ; FAT32_IBMBOOT_HEADER ; 12/09/2023
441 00000000 ?????? .JUMP: resb 3
442 00000003 ?????????????? .OEM: resb 8
443 0000000B <res 35h> .BPB: resb XBPB.size ; 53 bytes (25+28)
444 00000040 ?? .PHYDRV: resb 1
445 00000041 ?? .CURHD: resb 1
446 00000042 ?? .SIG: resb 1
447 00000043 ???????? .SERIAL: resd 1
448 00000047 <res Bh> .VOL_LABEL: resb 11
449 00000052 ?????????????? .SYSTEM_ID: resb 8
450 .size:
451 endstruc
452
453 ; -----
454
455 ; 23/03/2018
456
457 ;STATIC REQUEST HEADER (DEVSYS.INC, MSDOS 6.0, 1991)
458 STRUC SRHEAD
459 00000000 ?? .REQLEN: resb 1 ;LENGTH IN BYTES OF REQUEST BLOCK
460 00000001 ?? .REQUNIT: resb 1 ;DEVICE UNIT NUMBER
461 00000002 ?? .REQFUNC: resb 1 ;TYPE OF REQUEST
462 00000003 ???? .REQSTAT: resw 1 ;STATUS WORD
463 00000005 ?????????????? resb 8 ;RESERVED FOR QUEUE LINKS
464 .size:
465 endstruc
466
467 ; GENERIC IOCTL REQUEST STRUCTURE (DEVSYS.INC, MSDOS 6.0, 1991)
468 ; SEE THE DOS 4.0 DEVICE DRIVER SPEC FOR FURTHER ELABORATION.
469 ;
470 struct IOCTL_REQ
471 00000000 <res Dh> resb SRHEAD.size
472 ;GENERIC IOCTL ADDITION.
473 0000000D ?? .MAJORFUNCTION: resb 1 ;FUNCTION CODE
474 0000000E ?? .MINORFUNCTION: resb 1 ;FUNCTION CATEGORY
475 0000000F ???? .REG_SI: resw 1
476 00000011 ???? .REG_DI: resw 1
477 00000013 ???????? .GENERICIOCTL_PACKET: resd 1 ; POINTER TO DATA BUFFER
478 endstruc
479
480 ; GENERIC IOCTL CATEGORY CODES (IOCTL.INC, MSDOS 6.0, 1991)
481 IOC_OTHER EQU 0 ; Other device control J.K. 4/29/86
482 IOC_SE EQU 1 ; SERIAL DEVICE CONTROL
483 IOC_TC EQU 2 ; TERMINAL CONTROL
484 IOC_SC EQU 3 ; SCREEN CONTROL
485 IOC_KC EQU 4 ; KEYBOARD CONTROL
486 IOC_PC EQU 5 ; PRINTER CONTROL
487 IOC_DC EQU 8 ; DISK CONTROL (SAME AS RAWIO)
488
489 ; DEFINITIONS FOR IOCTL_REQ.MINORFUNCTION
490 GEN_IOCTL_WRT_TRK EQU 40H
491 GEN_IOCTL_RD_TRK EQU 60H
492 GEN_IOCTL_FN_TST EQU 20H ; USED TO DIFF. BET READS AND WRTS
493
494 ;struct A_RETRYCOUNT ; (IOCTL.INC, MSDOS 6.0, 1991)
495 ;.RC_COUNT: resw 1
496 ;endstruc

```

```

497
498 ; 29/05/2019 - Retro DOS v4.0 (DEVSYS.INC, MSDOS 6.0, 1991)
499
500 ; THE DEVICE TABLE LIST HAS THE FORM:
501
502 ;struc SYSDEV
503 ; .NEXT: resd 1 ;POINTER TO NEXT DEVICE HEADER
504 ; .ATT: resw 1 ;ATTRIBUTES OF THE DEVICE
505 ; .STRAT: resw 1 ;STRATEGY ENTRY POINT
506 ; .INT: resw 1 ;INTERRUPT ENTRY POINT
507 ; .NAME: resb 8 ;NAME OF DEVICE (ONLY FIRST BYTE USED FOR BLOCK)
508 ; .size:
509 ;endstruc
510
511 ; 27/03/2018 - DEVSYS.INC - MSDOS 3.3 - 24/07/1987
512
513 ;
514 ; ATTRIBUTE BIT MASKS
515 ;
516 ; CHARACTER DEVICES:
517
518 ; BIT 15 -> MUST BE 1
519 ; 14 -> 1 IF THE DEVICE UNDERSTANDS IOCTL CONTROL STRINGS
520 ; 13 -> 1 IF THE DEVICE SUPPORTS OUTPUT-UNTIL-BUSY
521 ; 12 -> UNUSED
522 ; 11 -> 1 IF THE DEVICE UNDERSTANDS OPEN/CLOSE
523 ; 10 -> MUST BE 0
524 ; 9 -> MUST BE 0
525 ; 8 -> UNUSED
526 ; 7 -> UNUSED
527 ; 6 -> UNUSED
528 ; 5 -> UNUSED
529 ; 4 -> 1 IF DEVICE IS RECIPIENT OF INT 29H
530 ; 3 -> 1 IF DEVICE IS CLOCK DEVICE
531 ; 2 -> 1 IF DEVICE IS NULL DEVICE
532 ; 1 -> 1 IF DEVICE IS CONSOLE OUTPUT
533 ; 0 -> 1 IF DEVICE IS CONSOLE INPUT
534
535 ; BLOCK DEVICES:
536
537 ; BIT 15 -> MUST BE 0
538 ; 14 -> 1 IF THE DEVICE UNDERSTANDS IOCTL CONTROL STRINGS
539 ; 13 -> 1 IF THE DEVICE DETERMINES MEDIA BY EXAMINING THE FAT ID BYTE.
540 ; THIS REQUIRES THE FIRST SECTOR OF THE FAT TO *ALWAYS* RESIDE IN
541 ; THE SAME PLACE.
542 ; 12 -> UNUSED
543 ; 11 -> 1 IF THE DEVICE UNDERSTANDS OPEN/CLOSE/REMOVABLE MEDIA
544 ; 10 -> MUST BE 0
545 ; 9 -> MUST BE 0
546 ; 8 -> UNUSED
547 ; 7 -> UNUSED
548 ; 6 -> IF DEVICE HAS SUPPORT FOR GETMAP/SETMAP OF LOGICAL DRIVES.
549 ; IF THE DEVICE UNDERSTANDS GENERIC IOCTL FUNCTION CALLS.
550 ; 5 -> UNUSED
551 ; 4 -> UNUSED
552 ; 3 -> UNUSED
553 ; 2 -> UNUSED
554 ; 1 -> UNUSED
555 ; 0 -> UNUSED
556
557
558 DEVTYP EQU 8000H ; BIT 15 - 1 IF CHAR, 0 IF BLOCK
559 CHARDEV EQU 8000H
560 DEVIOCTL EQU 4000H ; BIT 14 - CONTROL MODE BIT
561 ISFATBYDEV EQU 2000H ; BIT 13 - DEVICE USES FAT ID BYTES,
562 ; COMP MEDIA.
563 OUTTILBUSY EQU 2000H ; OUTPUT UNTIL BUSY IS ENABLED
564 ISNET EQU 1000H ; BIT 12 - 1 IF A NET DEVICE, 0 IF
565 ; NOT. CURRENTLY BLOCK ONLY.
566 DEVOPCL EQU 0800H ; BIT 11 - 1 IF THIS DEVICE HAS
567 ; OPEN,CLOSE AND REMOVABLE MEDIA
568 ; ENTRY POINTS, 0 IF NOT
569
570 EXTENTBIT EQU 0400H ; BIT 10 - CURRENTLY 0 ON ALL DEVS
571 ; THIS BIT IS RESERVED FOR FUTURE USE
572 ; TO EXTEND THE DEVICE HEADER BEYOND
573 ; ITS CURRENT FORM.
574
575 ; NOTE BIT 9 IS CURRENTLY USED ON IBM SYSTEMS TO INDICATE "DRIVE IS SHARED".
576 ; SEE IOCTL FUNCTION 9. THIS USE IS NOT DOCUMENTED, IT IS USED BY SOME
577 ; OF THE UTILITIES WHICH ARE SUPPOSED TO FAIL ON SHARED DRIVES ON SERVER
578 ; MACHINES (FORMAT,CHKDSK,RECOVER,..).
579
580 ; 18/03/2019 - Retro DOS v4.0
581 IOQUERY EQU 0080H ;Bit 7 - supports generic IOCTL query M017
582
583 DEV320 EQU 0040H ;BIT 6 - FOR BLOCK DEVICES, THIS
584 ;DEVICE SUPPORTS SET/GET MAP OF
585 ;LOGICAL DRIVES, AND SUPPORTS
586 ;GENERIC IOCTL CALLS.
587 ;FOR CHARACTER DEVICES, THIS
588 ;DEVICE SUPPORTS GENERIC IOCTL.
589 ;THIS IS A DOS 3.2 DEVICE DRIVER.
590
591 ISSPEC EQU 0010H ;BIT 4 - THIS DEVICE IS SPECIAL
592 ISLOCK EQU 0008H ;BIT 3 - THIS DEVICE IS THE CLOCK DEVICE.
593 ISNULL EQU 0004H ;BIT 2 - THIS DEVICE IS THE NULL DEVICE.
594 ISOUT EQU 0002H ;BIT 1 - THIS DEVICE IS THE CONSOLE OUTPUT.
595 ISIN EQU 0001H ;BIT 0 - THIS DEVICE IS THE CONSOLE INPUT.
596
597 ; 23/07/2019 - Retro DOS v4.0
598 EXTDRVR EQU 0002h ; (MSDOS 6.0, DEVSYS.INC, 1991)
599
600 ; 27/05/2018 - Retro DOS v3.0
601 ; [MSDOS 3.3, MSDISK.ASM]
602
603 struc INT13FRAME
604 .oldbp: resw 1
605 .oldax: resw 1
606 .oldbx: resw 1
607 .oldcx: resw 1
608 .olddx: resw 1
609 .olddd: resd 1
610 .oldf: resw 1
611 .size:
612 ;endstruc
613
614 ; 02/06/2018 - Retro DOS v3.0
615 ; [MSDOS 3.3, BIOSTRUC.INC]
616
617 struc ROMBIOS_DESC ; BIOS_SYSTEM_DESCRIPTOR
618 .bios_sd_leng: resw 1
619 .bios_sd_modelbyte: resb 1
620 .bios_sd_scnd_modelbyte:
621 resb 1
622 resb 1

```

```

621 00000005 ??      .bios_sd_featurebyte1: resb 1
622 00000006 ????????      resb 4
623 endstruc
624
625 ;-----
626 ; MSDIOCTL.ASM - MSDOS 6.0 - 1991
627 ;-----
628 ; 11/03/2019 - Retro DOS v4.0
629
630 ; 18/03/2019
631 DSK_TIMEOUT_ERR EQU 80h ; Time out error (no media present).
632 DSK_CHANGELINE_ERR EQU 06h ; Change line error
633 DSK_ILLEGAL_COMBINATION EQU 0Ch ; Return code of ah=18h function.
634 MULTI_TRK_ON EQU 10000000b ; User specified multitrack=on,
635 ; or system turns
636 ; IOCTL.INC - MSDOS 6.0 - 1991
637 ; .....
638
639 ;*** J.K.
640 ;General Guide -
641 ;Category Code:
642 ; 0... .... DOS Defined
643 ; 1... .... User defined
644 ; .xxx xxxx Code
645
646 ;Function Code:
647 ; 0... .... Return error if unsupported
648 ; 1... .... Ignore if unsupported
649 ; .0.. .... Intercepted by DOS
650 ; .1.. .... Passed to driver
651 ; ..0. .... Sends data/commands to device
652 ; ..1. .... Queries data/info from device
653 ; ...x .... Subfunction
654 ;
655 ; Note that "Sends/queries" data bit is intended only to regularize the
656 ; function set. It plays no critical role; some functions may contain both
657 ; command and query elements. The convention is that such commands are
658 ; defined as "sends data".
659
660 ;*****
661 ; BLOCK DRIVERS ;*
662 ;*****
663
664 ; IOCTL SUB-FUNCTIONS
665 IOCTL_GET_DEVICE_INFO EQU 0
666 IOCTL_SET_DEVICE_INFO EQU 1
667 IOCTL_READ_HANDLE EQU 2
668 IOCTL_WRITE_HANDLE EQU 3
669 IOCTL_READ_DRIVE EQU 4
670 IOCTL_WRITE_DRIVE EQU 5
671 IOCTL_GET_INPUT_STATUS EQU 6
672 IOCTL_GET_OUTPUT_STATUS EQU 7
673 IOCTL_CHANGEABLE? EQU 8
674 IOCTL_DeviceLocOrRem? EQU 9
675 IOCTL_HandleLocOrRem? EQU 0Ah ;10
676 IOCTL_SHARING_RETRY EQU 0Bh ;11
677 GENERIC_IOCTL_HANDLE EQU 0Ch ;12
678 GENERIC_IOCTL EQU 0Dh ;13
679 IOCTL_GET_DRIVE_MAP EQU 0Eh ;14
680 IOCTL_SET_DRIVE_MAP EQU 0Fh ;15
681 IOCTL_QUERY_HANDLE EQU 10h ;16
682 IOCTL_QUERY_BLOCK EQU 11h ;17
683
684 ; GENERIC IOCTL SUB-FUNCTIONS
685 RAWIO EQU 8
686
687 ; RAWIO SUB-FUNCTIONS
688 GET_DEVICE_PARAMETERS EQU 60H
689 SET_DEVICE_PARAMETERS EQU 40H
690 READ_TRACK EQU 61H
691 WRITE_TRACK EQU 41H
692 VERIFY_TRACK EQU 62H
693 FORMAT_TRACK EQU 42H
694 GET_MEDIA_ID EQU 66h ;AN000;AN003;changed from 63h
695 SET_MEDIA_ID EQU 46h ;AN000;AN003;changed from 43h
696 GET_ACCESS_FLAG EQU 67h ;AN002;AN003;Unpublished function.Changed from 64h
697 SET_ACCESS_FLAG EQU 47h ;AN002;AN003;Unpublished function.Changed from 44h
698 SENSE_MEDIA_TYPE EQU 68H ;Added for 5.00
699
700
701 ; SPECIAL FUNCTION FOR GET DEVICE PARAMETERS
702 BUILD_DEVICE_BPB EQU 000000001B
703
704 ; SPECIAL FUNCTIONS FOR SET DEVICE PARAMETERS
705 INSTALL_FAKE_BPB EQU 000000001B
706 ONLY_SET_TRACKLAYOUT EQU 000000010B
707 TRACKLAYOUT_IS_GOOD EQU 000000100B
708
709 ; SPECIAL FUNCTION FOR FORMAT TRACK
710 STATUS_FOR_FORMAT EQU 000000001B
711 DO_FAST_FORMAT EQU 000000010B ;AN001;
712 ; CODES RETURNED FROM FORMAT STATUS CALL
713 FORMAT_NO_ROM_SUPPORT EQU 000000001B
714 FORMAT_COMB_NOT_SUPPORTED EQU 000000010B
715
716 ; DEVICETYPE VALUES
717 MAX_SECTORS_IN_TRACK EQU 63 ; MAXIMUM SECTORS ON A DISK.(was 40 in DOS 3.2)
718 DEV_5INCH EQU 0
719 DEV_5INCH96TPI EQU 1
720 DEV_3INCH720KB EQU 2
721 DEV_8INCHSS EQU 3
722 DEV_8INCHDS EQU 4
723 DEV_HARDDISK EQU 5
724 DEV_OTHER EQU 7
725 ;DEV_3INCH1440KB EQU 7
726 DEV_3INCH2880KB EQU 9
727 ; Retro DOS v2.0 - 26/03/2018
728 ;DEV_TAPE EQU 6
729 ;DEV_ERIMO EQU 8
730 ;DEV_3INCH2880KB EQU 9
731 DEV_3INCH1440KB EQU 10
732
733 ;MAX_DEV_TYPE EQU 9 ; MAXIMUM DEVICE TYPE THAT WE
734 ; CURRENTLY SUPPORT.
735 MAX_DEV_TYPE EQU 10
736
737 struc A_SECTORTABLE
738 .ST_SECTORNUMBER: resw 1
739 .ST_SECTORSIZE: resw 1
740 .size:
741 endstruc
742
743 ; 25/03/2019 - Retro DOS v4.0 (MSDOS 6.0, BPB.INC, IOCTL.INC)
744

```



```

745 ; MSDOS 6.0 - BPB.INC - 1991
746 ; #####
747 ; ** BIOS PARAMETER BLOCK DEFINITION
748 ;
749 ; The BPB contains information about the disk structure. It dates
750 ; back to the earliest FAT systems and so FAT information is
751 ; intermingled with physical driver information.
752 ;
753 ; A boot sector contains a BPB for its device; for other disks
754 ; the driver creates a BPB. DOS keeps copies of some of this
755 ; information in the DPB.
756 ;
757 ; The BDS structure contains a BPB within it.
758 ;
759 ; 01/01/2024
760 %if 0
761
762 struc A_BPB
763 .BPB_BYTESPERSECTOR: resw 1
764 .BPB_SECTORS PERCLUSTER: resb 1
765 .BPB_RESERVEDSECTORS: resw 1
766 .BPB_NUMBEROFFATS: resb 1
767 .BPB_ROOTENTRIES: resw 1
768 .BPB_TOTALSECTORS: resw 1
769 .BPB_MEDIADSCRIPTOR: resb 1
770 .BPB_SECTORS PERFAT: resw 1
771 .BPB_SECTORS PERTRACK: resw 1
772 .BPB_HEADS: resw 1
773 .BPB_HIDDENSECTORS: resw 1
774 resw 1
775 .BPB_BIGTOTALSECTORS: resw 1
776 resw 1
777 resb 6 ; NOTE: many times these
778 ; 6 bytes are omitted
779 ; when BPB manipulations
780 ; are performed!
781 .size:
782 endstruc
783
784 %else
785 ; 14/04/2024
786 ; 01/01/2024 - Retro DOS v5.0
787
788 struc A_BPB
789 .BYTESPERSECTOR: resw 1
790 .SECTORS PERCLUSTER: resb 1
791 .RESERVEDSECTORS: resw 1
792 .NUMBEROFFATS: resb 1
793 .ROOTENTRIES: resw 1
794 .TOTALSECTORS: resw 1
795 .MEDIADSCRIPTOR: resb 1
796 .SECTORS PERFAT: resw 1
797 .SECTORS PERTRACK: resw 1
798 .HEADS: resw 1
799 .HIDDENSECTORS: resd 1
800 .BIGTOTALSECTORS: resd 1
801 ;..... FAT32 ..... + 28
802 .FATSIZE32: resd 1
803 .EXTFLAGS: resw 1
804 .FSVER: resw 1
805 .ROOTDIRCLUSTER: resd 1
806 .FSINFOSECTOR: resw 1 ; (offset from FAT32 bs)
807 .BACKUPBOOTSECTOR: resw 1 ; (offset from FAT32 bs)
808 .RESERVEDBYTES: resb 12 ; (zero bytes)
809 <res Ch>
810 ; 14/04/2024
811 00000035 ???????????? resb 6 ; A_BPB.size must be 59
812 .size:
813 endstruc
814
815 %endif
816
817 struc A_DEVICEPARAMETERS
818 .DP_SPECIALFUNCTIONS: resb 1
819 .DP_DEVICE TYPE: resb 1
820 .DP_DEVICEATTRIBUTES: resw 1
821 .DP_CYLINDERS: resw 1
822 .DP_MEDIATYPE: resb 1
823 .DP_BPB: resb A_BPB.size
824 .DP_TRACKTABLEENTRIES: resw 1
825 .DP_SECTORTABLE: resb MAX_SECTORS_IN_TRACK * A_SECTORTABLE.size
826 endstruc
827
828 struc A_TRACKREADWRITEPACKET
829 .TRWP_SPECIALFUNCTIONS: resb 1
830 .TRWP_HEAD: resw 1
831 .TRWP_CYLINDER: resw 1
832 .TRWP_FIRSTSECTOR: resw 1
833 .TRWP_SECTORSTOREADWRITE: resw 1
834 .TRWP_TRANSFERADDRESS: resd 1
835 endstruc
836
837 ;AN001; - FP_TRACKCOUNT is only meaningful when FP_SPECIALFUNCTIONS bit 1 = 1.
838 struc A_FORMATPACKET
839 .FP_SPECIALFUNCTIONS: resb 1 ; db ?
840 .FP_HEAD: resw 1 ; dw ?
841 .FP_CYLINDER: resw 1 ; dw ?
842 .FP_TRACKCOUNT: resw 1 ; dw 1 ; !
843 endstruc
844
845 struc A_VERIFYPACKET
846 .VP_SPECIALFUNCTIONS: resb 1
847 .VP_HEAD: resw 1
848 .VP_CYLINDER: resw 1
849 endstruc
850
851 struc A_MEDIA_ID_INFO
852 .MI_LEVEL: resw 1 ; dw 0 ; ! ;J.K. 87 Info. level
853 .MI_SERIAL: resd 1 ; dd ? ;J.K. 87 Serial #
854 .MI_LABEL: resb 11 ; db 11 DUP ( ' ' ) ;! ;J.K. 87 volume label
855 .MI_SYSTEM: resb 8 ; db 8 DUP ( ' ' ) ;! ;J.K. 87 File system type
856 endstruc
857
858 struc A_DISKACCESS_CONTROL ;AN002; Unpublished function. Only for Hard file.
859 .DAC_SPECIALFUNCTIONS: resb 1 ; db 0 ; ! ;AN002; Always 0
860 .DAC_ACCESS_FLAG: resb 1 ; db 0 ; !
861 ; Non Zero - allow disk I/O to unformatted hard file
862 endstruc ; 0 - Disallow disk I/O to unformatted hard file
863
864 struc A_MEDIA_SENSE
865 .MS_ISDEFAULT: resb 1 ; Media sense structure added 5.00
866 .MS_DEVICE TYPE: resb 1 ; If 1 type returned is drv default
867 .MS_RESERVED1: resb 1 ; Drive type
868 .MS_RESERVED1: resb 1 ; RESERVED

```

```

869 00000003 ??      .MS_RESERVED2:      resb 1      ; RESERVED
870 endstruc
871
872 ;*****;*
873 ; CHARACTER DEVICES (PRINTERS) ;*
874 ;*****;*
875
876 ;RAWIO SUB-FUNCTIONS
877 GET_RETRY_COUNT EQU 65H
878 SET_RETRY_COUNT EQU 45H
879
880
881 00000000 ????      struc A_RETRYCOUNT
882 .RC_COUNT:      resw 1
883 endstruc
884
885 ;*****;* ;J.K. 4/29/86
886 ; CHARACTER DEVICES (SCREEN) ;*
887 ;*****;* ;J.K. 4/29/86
888
889 ;SC_MODE_INFO      struc
890 ;SC_INFO_LENGTH    DW 9
891 ;SC_MODE            DB 0
892 ;SC_COLORS          DW 0
893 ;SC_WIDTH           DW 0
894 ;SC_LENGTH          DW 0
895 ;SC_MODE_INFO      ends
896
897 ;SC_INFO_PACKET_LENGTH EQU 9 ;LENGTH OF THE INFO PACKET.
898
899 ;SUBFUNCTIONS FOR CON$GENIOCTL
900 GET_SC_MODE EQU 60h
901 SET_SC_MODE EQU 40h
902 ;The following subfunctions are reserved for installable CODE PAGE switch
903 ;console devices. - J.K. 4/29/86
904 Get_active_codepage equ 6Ah
905 Invoke_active_codepage equ 4Ah
906 Start_designate_codepage equ 4Ch
907 End_designate_codepage equ 4Dh
908 Get_list_of_designated_codepage equ 6Bh
909 ;J.K. 4/29/86 *** End of Con$genioctl equates & structures
910
911 -----
912 ; MULT.INC - MSDOS 6.0 - 1991
913 -----
914 ; 18/03/2019
915
916 ; The current set of defined multiplex channels is (* means documented):
917
918 ; Channel(h) Issuer Receiver Function
919 ; 00 server PSPRINT print job control
920 ; *01 print/apps PRINT Queueing of files
921 ; 02 BIOS REDIR signal open/close of printers
922
923 ; 05 command REDIR obtain text of net int 24 message
924 ; *06 server/assign ASSIGN Install check
925
926 ; 08 external driver IBMBIO interface to internal routines
927
928 ; 10 sharer/server Sharer install check
929 ; 11 DOS/server Redir install check/redirection funcs
930 ; 12 sharer/redir DOS dos functions and structure maint
931 ; 13 MSNET MSNET movement of NCBS
932 ; 13 external driver IBMBIO Reset_Int_13, allows installation
933 ; of alternative INT_13 drivers after
934 ; boot_up
935 ; 14 (IBM) DOS NLSFUNC down load NLS country info,DOS 3.3
936 ; 14 (MS) APPS POPUP MSDOS 4 popup screen functions
937 ; 15 APPS MSCDEX CD-ROM extensions interface
938 ; 16 WIN386 WIN386 windows communications
939 ; 17 Clipboard WINDOWS Clipboard interface
940 ; *18 Applications MS-Manger Toggle interface to manager
941 ; 19 Shell
942 ; 1A Ansi.sys
943 ; 1B Fastopen,Vdisk IBMBIO EMS INT 67H stub handler
944
945 ; 40h OS/2
946 ; 41h Lanman
947 ; 42h Lanman
948 ; 43h Himem
949 ; AL = 20h reserved for Mach 20 Himem support
950 ; AL = 30h reserved for Himem external A20 code
951
952 ; 44h Dosextender
953 ; 45h Windows profiler
954 ; 46h Windows/286 DOS extender
955 ; 47h Basic Compiler Vn. 7.0
956 ; 48h Doskey
957 ; 49h DOS 5.x install
958 ; 4Ah Multi Purpose
959 ; multMULTSWPDSK 0 - Swap Disk in drive A (BIOS)
960 ; multMULTGETHMAPTR 1 - Get available HMA & ptr
961 ; multMULTALLOCHMA 2 - Allocate HMA (bx == no of bytes)
962 ; multMULTTASKSHELL 5 - Shell/switcher API
963 ; multMULTRPLTOM 6 - Top Of Memory for RPL support
964
965 ; multSmartdrv 10h
966 ; multMagicdrv 11h
967 ; 4Bh Task Switcher API
968
969 ; 4Ch APPS APM Advanced power management
970 ; 4Dh Kana Kanji Converter, MSKK
971
972 ; 51h ODI real mode support driver (for Chicago)
973
974 ; 53h POWER.EXE - used for broadcasting APM events ; M036
975 ; 54h POWER.EXE - used for POWER API ; M036
976
977 ; 55h COMMAND.COM
978 ; multCOMFIRST 0 - API to determine whether 1st
979 ; instance of command.com
980 ; multCOMFIRSTROM 1 - API to determine whether 1st
981 ; instance of ROM COMMAND
982
983 ; 56h Sewell Development
984 ; INTERLNK
985
986 ; 57h Iomega Corp.
987
988 ; AB Unspecified IBM use
989 ; AC Graphics
990 ; AD NLS (toronto)
991 ; AE
992 ; AF Mode
993 ; B0 GRAFTABL GRAFTABL
994
995 ; D7 Banyan VINES

```



```

993 multMULT      equ 4Ah
994
995 multMULTSWPDSK      equ 0      ; Swap Disk in drive A (BIOS)
996 multMULTGETHMAPTR  equ 1      ; Get available HMA & ptr
997 multMULTALLOCHMA   equ 2      ; Allocate HMA (bx == no of bytes)
998 multMULTTASKSHELL  equ 5      ; Shell/switcher API
999 multMULTRPLTOM      equ 6      ; Top Of Memory for RPL support
1000
1001 ;-----
1002 ; WIN386.INC - MSDOS 6.0 - 1991
1003 ;-----
1004 ; 18/03/2019
1005
1006 ; WIN386.INC
1007 ;
1008 ; Symbols and structures relating to WIN386 support.
1009 ;
1010 ; Used by files in both the DOS and the BIOS.
1011 ;
1012 ; Created: 7-13-89 by MRW
1013 ;
1014 ;
1015 ; WIN386 broadcast int 2fh multiplex number and subfunction numbers
1016
1017 Multwin386      equ 16h      ; Int 2f multiplex number
1018
1019 win386_Init      equ 05h      ; win386 initialization
1020 win386_Exit      equ 06h      ; win386 exit
1021 win386_Devcall   equ 07h      ; win386 device call out
1022 win386_InitDone  equ 08h      ; win386 initialization is complete
1023
1024 ; =====
1025 ;-----
1026 ;
1027 ;-----+-----+
1028 ; | This file has been generated by The Interactive Disassembler (IDA) |
1029 ; | Copyright (c) 2013 Hex-Rays, <support@hex-rays.com> |
1030 ; | Licensed to: Freeware version |
1031 ; |-----+-----+
1032 ;
1033 ;
1034 ;
1035 ;-----
1036 ;
1037 ; .386
1038 ; .model flat
1039
1040 ; =====
1041 ; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
1042 ; 10/12/2022
1043 ; 09/12/2022
1044 ; 21/10/2022
1045 ; 19/10/2022
1046 ; 17/10/2022, 18/10/2022
1047 ; 15/10/2022, 16/10/2022
1048 ; 03/10/2022
1049 ; 02/10/2022
1050 ; 01/10/2022 - Erdogan Tan
1051
1052 ; [[ Most of comments here are from the original MSDOS 6.0 source code ]]
1053
1054 ;-----
1055 ; Start of PC-DOS 7.1 IBMBIO.COM (IO.SYS)
1056 ;-----
1057 ;
1058 ; [ORG 0] ; segment 0x0070h
1059
1060 ;=====
1061 ; DOS BIOS (IO.SYS) DATA SEGMENT
1062 ;=====
1063 ; 09/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
1064 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
1065 ; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
1066
1067 section .BIOSDATA vstart=0
1068
1069 ;--- DOSBIOS data segment -----
1070 ;-----
1071 ;Bios_Data segment
1072
1073 BData_start:
1074 hdrv_pat: jmp init ; MSBIOS.ASM, MSSBDATA.INC
1075 ;-----
1076 00000000 E9951B
1077
1078 DosDataSg: dw 0
1079
1080 ; DOS's int 2f handler will exit via a jump through here.
1081 ; This is how the BIOS hooks int2f
1082
1083 ;BIOSDATA:0005h: ; 10/05/2023 (Note the 'bios_i2f equ 5' in 'msdos6.s')
1084
1085 bios_i2f: db 0EAh ; far jump to int_2f (segment may not be at 70h)
1086 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
1087 ; PCDOS 7.1 IBMBIO.COM - BIODATA:0006h
1088 ;dw int_2f
1089 ;dw 70h ; BIOSDATA segment (KERNEL_SEGMENT)
1090 ;dw i2f_handler
1091 bios_i2f_seg: ; 10/08/2023
1092 dw DOSBIOCODESEG ; 02Cch for MSDOS 6.21 IO.SYS (25ch+070h)
1093 ; 0364h PCDOS 7.1 IBMBIO.COM (2F4h+070h)
1094
1095 romstartaddr: dw 0 ; The start address for the romfind routines
1096 ; This is to maintain binary compatibility
1097 ; with DISK based DOS 5.0
1098
1099 ; This is a byte used for special key handling in the resident
1100 ; console device driver. It must be here so that it can be included
1101 ; in the WIN386 instance table (in INC\LMSTUB.ASM).
1102
1103 altah: db 0 ; special key handling
1104
1105 inhMA: db 0 ; flag indicates we're running from HMA
1106 xms: dd 0 ; entry point to xms if above is true
1107
1108 ; PTRSAV - pointer save
1109 ;
1110 ; This variable holds the pointer to the Request Header passed by a program
1111 ; wishing to use a device driver. When the strategy routine is called it
1112 ; puts the address of the Request header in this variable and returns.
1113
1114 ptrsav: dd 0
1115 auxbuf: db 4 dup(0) ; set of 1 byte buffers for com 1,2,3, and 4
1116

```

```

1117 00000016 00000000          db 0, 0, 0, 0 ; 19/10/2022
1118 0000001A 0000          zeroseg: dw 0          ; easy way to load segment registers with zero
1119 0000001C 0000          i13_ds:  dw 0          ; ds register for int13 call through
1120 0000001E 0000          prevoper: dw 0          ; holds int 13 request (i.e. register ax).
1121 00000020 00          number_of_sec: db 0          ; holds number of secs. to read on an ecc error
1122 00000021 0000          auxnum: dw 0          ; which aux device was requested
1123
1124          ;-----
1125
1126          res_dev_list:
1127
1128          ; Device Header for the CON Device Driver
1129
1130          CONHeader:          ; HEADER FOR DEVICE "CON"
1131 00000023 [3500]          dw auxdev2
1132 00000025 7000          dw 70h
1133 00000027 1380          word_727: dw 8013h
1134 00000029 [1506]          dw strategy
1135 0000002B [2006]          dw con_entry
1136 0000002D 434F4E2020202020          aCon: db 'CON'
1137 00000035 [4700]          auxdev2: dw prndev2          ; HEADER FOR DEVICE "AUX"
1138 00000037 7000          dw 70h
1139 00000039 0080          dw 8000h
1140 0000003B [1506]          dw strategy
1141 0000003D [4106]          dw aux0_entry
1142 0000003F 4155582020202020          aAux: db 'AUX'
1143 00000047 [5900]          prndev2: dw timdev          ; HEADER FOR DEVICE "PRN"
1144 00000049 7000          dw 70h
1145 0000004B C0A0          word_74B: dw 0A0C0h
1146 0000004D [1506]          dw strategy
1147 0000004F [2506]          dw prn0_entry
1148 00000051 50524E2020202020          aPrn: db 'PRN'          ; HEADER FOR DEVICE "CLOCK$"
1149 00000059 [6800]          timdev: dw dskdev
1150 0000005B 7000          dw 70h
1151 0000005D 0880          dw 8008h
1152 0000005F [1506]          dw strategy
1153 00000061 [5906]          dw tim_entry
1154 00000063 434C4F434B242020          aClock: db 'CLOCK$'
1155 0000006B [7800]          dskdev: dw com1dev          ; HEADER FOR DISK DEVICES
1156 0000006D 7000          dw 70h
1157          ;dw 8C2h
1158          ; 02/10/2023 - Retro DOS v5.0
1159 0000006F C248          dw 48C2h          ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:006Fh
1160          ;dw offset strategy
1161          ;dw offset dsk_entry
1162          ; 19/10/2022
1163 00000071 [1506]          dw strategy
1164 00000073 [5E06]          dw dsk_entry
1165
1166          ; maximum number of drives
1167
1168 00000075 04          drvmax: db 4
1169 00000076 FE          step_drv: db 0FEh ; -2          ; last drive accessed
1170 00000077 00          fhav96: db 0          ; flag to indicate presence of
1171          ; 96tpi support
1172 00000078 00          single: db 0          ; used to detect single drive systems
1173 00000079 00          fhav96: db 0          ; indicates if this is a k09 or not
1174          ; used by console driver.
1175 0000007A 00          fsetowner: db 0          ; = 1 if we are setting the owner of a
1176          ; drive. (examined by checksingle).
1177
1178 0000007B [8D00]          com1dev: dw lpt1dev          ; Device Header for device "COM1"
1179 0000007D 7000          dw 70h
1180 0000007F 0080          dw 8000h
1181 00000081 [1506]          dw strategy
1182 00000083 [4106]          dw aux0_entry
1183 00000085 434F4D3120202020          aCom1: db 'COM1'
1184 0000008D [9F00]          lpt1dev: dw lpt2dev          ; Device Header for device LPT1
1185 0000008F 7000          dw 70h
1186 00000091 C0A0          dw 0A0C0h
1187 00000093 [1506]          dw strategy
1188 00000095 [2C06]          dw prn1_entry
1189 00000097 4C50543120202020          aLpt1: db 'LPT1'
1190 0000009F [B800]          lpt2dev: dw lpt3dev          ; Device Header for device LPT2
1191 000000A1 7000          dw 70h
1192 000000A3 C0A0          dw 0A0C0h
1193 000000A5 [1506]          dw strategy
1194 000000A7 [3306]          dw prn2_entry
1195 000000A9 4C5054322020202000-          aLpt2: db 'LPT2' ',0,0,0'
1196 000000B2 0000
1197
1198          ;M058; Start of changes
1199          ; Orig13 needs to be at offset 0B4h for the CMS floppy driver to work.
1200          ;These guys patch Orig13 with their own int 13h hook and so this offset
1201          ;cannot change for them to work. Even ProComm does this.
1202 000000B4 00000000          orig13: dd 0          ; to make Orig13 offset 0B4h
1203
1204 000000B8 [CA00]          lpt3dev: dw com2dev          ; Device Header for device LPT3
1205 000000BA 7000          dw 70h
1206 000000BC C0A0          dw 0A0C0h
1207 000000BE [1506]          dw strategy
1208 000000C0 [3A06]          dw prn3_entry
1209 000000C2 4C50543320202020          aLpt3: db 'LPT3'
1210 000000CA [DC00]          com2dev: dw com3dev          ; Device Header for device "COM2"
1211 000000CC 7000          dw 70h
1212 000000CE 0080          dw 8000h
1213 000000D0 [1506]          dw strategy
1214 000000D2 [4706]          dw aux1_entry
1215          ; 19/10/2022
1216 000000D4 434F4D3220202020          aCom2: db 'COM2'
1217          ;dw offset com4dev          ; Device Header for device "COM3"
1218 000000DC [EE00]          com3dev: dw com4dev
1219 000000DE 7000          dw 70h
1220 000000E0 0080          dw 8000h
1221          ;dw offset strategy
1222          ;dw offset aux2_entry
1223 000000E2 [1506]          dw strategy
1224 000000E4 [4D06]          dw aux2_entry
1225 000000E6 434F4D3320202020          aCom3: db 'COM3'
1226 000000EE FFFF          com4dev: dw 0FFFFh          ; Device Header for device "COM4"
1227 000000F0 7000          dw 70h
1228 000000F2 0080          dw 8000h
1229 000000F4 [1506]          dw strategy
1230 000000F6 [5306]          dw aux3_entry
1231 000000F8 434F4D3420202020          db 'COM4'
1232
1233          ;-----
1234
1235 00000100 10          RomVectors: db 10h
1236 00000101 00000000          old10: dd 0
1237 00000105 13          db 13h
1238 00000106 00000000          old13: dd 0
1239 0000010A 15          db 15h

```

```

1240 0000010B 00000000      old15:      dd 0
1241 0000010F 19             db 19h
1242 00000110 00000000      old19:      dd 0
1243 00000114 1B             db 1Bh
1244 00000115 00000000      old1B:      dd 0
1245
1246 ;EndRomVectors      equ $
1247
1248 ;NUMROMVECTORS      equ ((EndRomVectors - RomVectors)/5)
1249
1250 ;-----
1251
1252 00000119 [5203]      start_bds: dw bds1      ; Start of linked list of BDS's
1253 0000011B 7000             dw 70h      ; KERNEL_SEGMENT
1254
1255 ; (MSDOS 3.3) NOTE:
1256 ; Some floppy drives do not have changeline support. The result is a
1257 ; large amount of inefficiency in the code. A media-check always returns
1258 ; "I don't know". This cause DOS to reread the FAT on every access and
1259 ; always discard any cached data.
1260 ; We get around this inefficiency by implementing a "Logical Door Latch".
1261 ; The following three items are used to do this. The logical door latch is
1262 ; based on the premise that it is not physically possible to change floppy
1263 ; disks in a drive in under two seconds (most people take about 10). The
1264 ; logical door latch is implemented by saving the time of the last successful
1265 ; disk operation (in the value TIM_DRV). When a new request is made the
1266 ; current time is compared to the saved time. If less than two seconds have
1267 ; passed then the value "No Change" is returned. If more than two seconds
1268 ; have passed the value "Don't Know" is returned.
1269 ; There is one complication to this algorithm. Some programs change the
1270 ; value of the timer. In this unfortunate case we have an invalid timer.
1271 ; This possibility is detected by counting the number of disk operations
1272 ; which occur without any time passing. If this count exceeds the value of
1273 ; "AccessMax" we assume the counter is invalid and always return "Don't
1274 ; know". The variable "AccessCount" is used to keep track of the number
1275 ; of disk operation which occur without the time changing.
1276
1277 0000011D 00      accesscount: db 0
1278 0000011E FF      tim_drv:      db 0FFh
1279 0000011F 00      medbyt:      db 0
1280
1281 00000120 02      wrtverify: ; 15/10/2022
1282 00000121 00      rflag:      db 2      ; 2 for read, 3 for write
1283 00000122 0000      verify:      db 0      ; 1 if verify after write
1284 00000124 00      secnt:      dw 0
1285 00000125 01      dsctnum:      db 1      ; -- pad where hardnum was
1286 ;                                     ; number of diskette drives
1287
1288 ; (MSDOS 3.3) NOTE:
1289 ; Some of the older versions of the IBM rom-bios always assumed a seek would
1290 ; have to be made to read the diskette. Consequently a large head settle
1291 ; time was always used in the I/O operations. To get around this problem
1292 ; we need to continually adjust the head settle time. The following
1293 ; algorithm is used:
1294 ;
1295 ;   Get the current head settle value.
1296 ;   If it is 1, then
1297 ;     set slow = 15
1298 ;   else
1299 ;     set slow = value
1300 ;   ...
1301 ;   if we are seeking and writing then
1302 ;     use slow
1303 ;   else
1304 ;     use fast
1305 ;   ...
1306 ;   restore current head settle value
1307
1307 00000126 00      motorstartup: db 0      ; value from table
1308 00000127 00      settlecurrent: db 0      ; value from table
1309 00000128 00      settleslow: db 0      ; slow settle value
1310 00000129 00      nextspeed: db 0      ; value of speed to be used
1311 0000012A 00      save_head_sttl: db 0      ; used by read_sector routine
1312 0000012B 00      save_eot: db 0      ; saved eot from the default DPT
1313 0000012C 09      eot: db 9
1314 0000012D 00000000      dpt: dd 0      ; pointer to Disk Parameter Table
1315 00000131 00      cursec: db 0      ; current sector
1316 00000132 00      curhd: db 0      ; current head
1317 00000133 0000      curtrk: dw 0      ; current track
1318 00000135 0000      spsav: dw 0      ; save the stack pointer
1319 00000137 08      format_eot: db 8      ; eot used for format
1320 00000138 00      hdnum: db 0      ; head number
1321 00000139 0000      trknum: dw 0      ; track being manipulated
1322 0000013B 50      gap_patch: db 50h      ; format gap patched into dpt
1323
1324 ;-----
1325
1326 ; disk errors returned from the IBM rom
1327
1328 0000013C CC      errin: db 0Cch      ; write fault (hard disk)
1329 0000013D 80             db 80h      ; write fault (hard disk)
1330 0000013E 40             db 40h      ; seek failed
1331 0000013F 10             db 10h      ; uncorrectable CRC or ECC error on read
1332 00000140 08             db 8       ; dma overrun
1333 00000141 06             db 6       ; disk changed (floppy)
1334 00000142 04             db 4       ; sector not found/read error
1335 00000143 03             db 3       ; disk write-protected
1336 ; 02/10/2023 - Retro DOS 5.0 (PCDOS 7.1 IBMBIO.COM)
1337 00000144 01             db 1       ; invalid function in AH or invalid parameter
1338 00000145 B2             db 0B2h    ; volume not removable
1339 ;
1340 00000146 00      lsterr: db 0      ; all other errors
1341
1342 ; returned error codes corresponding to above
1343
1344 00000147 0A      errout: db 10      ; write fault error
1345 00000148 02             db 2       ; no response (timeout)
1346 00000149 06             db 6       ; seek failure
1347 0000014A 04             db 4       ; bad crc
1348 0000014B 04             db 4       ; dma overrun
1349 0000014C 0F             db 15      ; invalid media change
1350 0000014D 08             db 8       ; sector not found
1351 0000014E 00             db 0       ; write attempt to write-protect disk
1352 ; 02/10/2023
1353 0000014F 03             db 3       ; unknown command error
1354 00000150 03             db 3       ; unknown command error
1355 ;
1356 00000151 0C             db 12      ; general error
1357
1358 ;-----
1359 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0152h
1360
1361 ; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
1362 %if 1
1363 disksector: times 174 db 0

```

```

1364 NUM174 equ 512-$
1365 00000152 00<rep AEh> times NUM174 db 0
1366 JB_sign : ;dw 424Ah ; 'BJ' (nasm) ; 'JB' (masm)
1367 00000200 4A dec dx
1368 00000201 42 inc dx
1369 00000202 E9FBFD jmp BData_start ; db 0E9h, 0FBh, 0FDh
1370
1371 00000205 402349424D3A31322E- IBMBIOCOM$: db '@#IBM:12.01.2003.build_1.32#@ IBMIO.COM(USA)',0
1371 0000020E 30312E323030332E62-
1371 00000217 75696C645F312E3332-
1371 00000220 23402049424D42494F-
1371 00000229 2E434F4D2855534129-
1371 00000232 00
1372
1373 ;times 287 db 0
1374 00000233 00<rep 11Fh> times (disksector+512-$) db 0 ; 287
1375 %endif
1376
1377 ;-----
1378 ; 30/12/2018 - Retro DOS v4.0
1379
1380 ; read in boot sector here, read done in readboot.
1381 ; also read sector for dma check for hard disk.
1382 ;
1383 ;
1384 ; This buffer is word aligned because certain AMI BIOSs have a bug
1385 ; in them which causes the byte after the buffer to be trashed
1386 ; on floppy reads to odd-byte boundaries. Although no general effort
1387 ; is made to enforce this in the bigger picture, this one small sacrifice
1388 ; makes that system more-or-less work.
1389
1390 ; 02/10/2023
1391 %if 0
1392
1393 disksector: ;db 512 dup(0) ; read in boot sector here
1394 ; 19/10/2022
1395 times 512 db 0
1396 %endif
1397
1398 ;-----
1399
1400 ; 02/10/2023 - Retro DOS v5.0
1401 ; 30/12/2018 - Retro DOS v4.0
1402 ;-----
1403 ; 25/05/2018 (04/04/2018)
1404 *****
1405 ; "bds" contains information for each drive in the system.
1406 ; various values are patched whenever actions are performed.
1407 ; sectors/alloc. unit in bpb initially set to -1 to signify that
1408 ; the bpb has not been filled. link also set to -1 to signify end
1409 ; of list. # of cylinders in maxparms initialized to -1 to indicate
1410 ; that the parameters have not been set.
1411
1412 bds1: ;dw offset bds2
1413 00000352 [E803] dw bds2 ; 19/10/2022
1414 00000354 7000 dw 70h ; dwordlink to next structure
1415 00000356 00 db 0 ; int 13h drive number
1416 00000357 00 db 0 ; logical drive letter
1417 00000358 0002 fdrive1: dw 512
1418 ; physical sector size in bytes
1419 0000035A FF db 0FFh ; sectors/allocation unit
1420 0000035B 0100 dw 1 ; reserved sectors for dos
1421 0000035D 02 db 2 ; no of file allocation tables
1422 0000035E 4000 dw 64 ; number of root directory entries
1423 00000360 6801 dw 360 ; number sectors (at 512 bytes each)
1424 00000362 00 db 0 ; mediadescriptor, initially 0
1425 00000363 0200 dw 2 ; number of fat sectors
1426 00000365 0900 dw 9 ; sector limit (sectors per track)
1427 00000367 0100 dw 1 ; head limit (number of heads - 1)
1428 ;
1429 ; 02/10/2023
1430 ; MSDOS 5.0-6.22 (& PCDOS 7.0)
1431 ;dw 0 ; hidden sector count (low word)
1432 ;dw 0 ; hidden sector (high)
1433 ;dw 0 ; number sectors (low)
1434 ;dw 0 ; number sectors (high)
1435 ;db 0 ; true => large fats
1436 ; 02/10/2023
1437 ; PCDOS 7.1 (FAT32 support)
1438 00000369 00000000 dd 0 ; hidden sector count
1439 0000036D 00000000 dd 0 ; number of sectors (32 bit)
1440 00000371 00000000 dd 0 ; BPB_FATSz32 ; FAT32 FAT size in sectors ; 4 bytes
1441 ; BS_DrvNum ; FAT INT 13h drive number ; 1 byte
1442 ; BS_Reserved1 ; FAT reserved byte = 0 ; 1 byte
1443 ; BS_BootSig ; FAT Extended boot signature = 29h ; 1 byte
1444 ; BS_VolID ; FAT Volume serial number ; 4 bytes
1445 00000375 0000 dw 0 ; BPB_ExtFlags ; FAT32 Extended Flags
1446 00000377 0000 dw 0 ; BPB_FSVer ; FAT32 fs/volume version
1447 00000379 00000000 dd 0 ; BPB_RootClus ; FAT32 root directory's first cluster number
1448 0000037D FFFF dw 0FFFFh ; BPB_FSInfo ; FAT32 FSINFO sector number = -1 (initial)
1449 0000037F FFFF dw 0FFFFh ; BPB_BkBootSec ; FAT32 backup boot sector number = -1 (initial)
1450 00000381 00<rep Ch> times 12 db 0 ; BPB_Reserved ; FAT32 reserved field = 0, 12 bytes
1451 0000038D 00 db 0 ; true => large fats
1452 ;
1453 0000038E 0000 dw 0 ; open ref. count
1454 00000390 03 db 3 ; form factor
1455 00000391 2000 dw 20h ; various flags
1456 00000393 2800 dw 40 ; number of cylinders
1457 00000395 0002 recommended_bps: dw 512 ; recommended bps for this drive
1458 00000397 01 db 1
1459 00000398 0100 dw 1
1460 0000039A 02 db 2
1461 0000039B E000 dw 224 ; number of root directory entries
1462 0000039D 6801 dw 360
1463 0000039F F0 db 0F0h ; mediadescriptor, initially 0F0h
1464 000003A0 0200 dw 2
1465 000003A2 0900 dw 9
1466 000003A4 0200 dw 2
1467 ;
1468 ; 02/10/2023
1469 ;dw 0
1470 ;dw 0
1471 ;dw 0
1472 ;dw 0
1473 ;db 6 dup(0)
1474 times 6 db 0 ; 19/10/2022
1475 000003A6 00000000 dd 0
1476 000003AA 00000000 dd 0
1477 000003AE 00000000 dd 0
1478 000003B2 0000 dw 0
1479 000003B4 0000 dw 0
1480 000003B6 00000000 dd 0
1481 000003BA FFFF dw 0FFFFh
1482 000003BC FFFF dw 0FFFFh

```

```

1483                                     ;db 12 dup(0)
1484 000003BE 00<rep Ch>                 times 12 db 0          ; 02/10/2023
1485                                     ;
1486 000003CA FF                         db 0FFh                ; last track accessed on this drive
1487 000003CB FFFF                      dw 0FFFFh              ; keep these two contiguous (?)
1488 000003CD FFFF                      dw 0FFFFh
1489 000003CF 4E4F204E414D452020-      db 'NO NAME' ',0'      ; volume id for this disk
1489 000003D8 202000
1490 000003DB 00000000                 dd 0                  ; current volume serial from boot record
1491 000003DF 464154313220202000       db 'FAT12' ',0'       ; current file system id from boot record
1492                                     ; ----
1493                                     ; 02/10/2023
1494                                     ; PCDOS 7.1
1495                                     %if 1
1496
1497 bds2:                                ; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
1498                                     dw 0FFFFh ; -1
1499 000003E8 FFFF                      dw 70h
1500 000003EA 7000                      db 0
1501 000003EC 00                        db 0
1502 000003ED 00                        dw 512
1503 000003EE 0002                     fdrive2:                db 0FFh
1504 000003F0 FF                        dw 1
1505 000003F1 0100                      db 2
1506 000003F3 02                        dw 64
1507 000003F4 4000                      dw 360
1508 000003F6 6801                      db 0
1509 000003F8 00                        dw 2
1510 000003F9 0200                      dw 9
1511 000003FB 0900                      dw 1
1512 000003FD 0100                      times 5 dd 0
1513 000003FF 00000000<rep 5h>         dd 0FFFFFFFFh
1514 00000413 FFFFFFFF
1515 00000417 00000000<rep 3h>         times 3 dd 0
1516 00000423 00                        db 0
1517 00000424 0000                      dw 0
1518 00000426 03                        db 3
1519 00000427 2000                      dw 20h
1520 00000429 2800                      dw 40
1521
1522 0000042B 0002                     recbpb2:                dw 512
1523 0000042D 01                        db 1
1524 0000042E 0100                      dw 1
1525 00000430 02                        db 2
1526 00000431 E000                      dw 224
1527 00000433 6801                      dw 360
1528 00000435 F0                        db 0F0h
1529 00000436 0200                      dw 2
1530 00000438 0900                      dw 9
1531 0000043A 0200                      dw 2
1532 0000043C 00000000<rep 5h>         times 5 dd 0
1533 00000450 FFFFFFFF                  dd 0FFFFFFFFh
1534 00000454 00000000<rep 3h>         times 3 dd 0
1535 00000460 FF                        db 0FFh
1536 00000461 FFFFFFFF                  dd 0FFFFFFFFh
1537 00000465 4E4F204E414D452020-      db 'NO NAME' ',0'
1537 0000046E 202000
1538 00000471 00000000                 dd 0
1539 00000475 464154313220202000       db 'FAT12' ',0'
1540                                     %endif
1541                                     ; ----
1542                                     ; 02/10/2023
1543                                     ; MSDOS 5.0 - 6.22 (& PCDOS 7.0)
1544                                     %if 0
1545
1546 bds2:                                dw bds3
1547                                     dw 70h
1548                                     db 0
1549                                     db 0
1550 fdrive2:                             dw 512
1551                                     db 0FFh, 1, 0, 2, 40h, 0, 68h, 1, 0, 2, 0, 9, 0, 1, 0
1552                                     db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h, 0
1553                                     db 0, 2, 1, 1, 0, 2, 0E0h, 0, 68h, 1, 0F0h, 2, 0, 9, 0
1554                                     db 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh
1555                                     db 0FFh, 0FFh, 0FFh, 0FFh, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh
1556                                     db 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h, 54h
1557                                     db 31h, 32h, 20h, 20h, 20h, 0
1558
1559 bds3:                                dw bds4
1560                                     dw 70h
1561                                     db 0
1562                                     db 0
1563 fdrive3:                             dw 512
1564                                     db 0FFh, 1, 0, 2, 40h, 0, 68h, 1, 0, 2, 0, 9, 0, 1, 0
1565                                     db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h, 0
1566                                     db 0, 2, 1, 1, 0, 2, 0E0h, 0, 68h, 1, 0F0h, 2, 0, 9, 0
1567                                     db 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh
1568                                     db 0FFh, 0FFh, 0FFh, 0FFh, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh
1569                                     db 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h, 54h
1570                                     db 31h, 32h, 20h, 20h, 20h, 0
1571
1572                                     ; ----
1573
1574 bds4:                                dw 0FFFFh
1575                                     dw 70h
1576                                     db 0
1577                                     db 0
1578 fdrive4:                             dw 512
1579                                     db 0FFh, 1, 0, 2, 40h, 0, 68h, 1, 0, 2, 0, 9, 0, 1, 0
1580                                     db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h, 0
1581                                     db 0, 2, 1, 1, 0, 2, 0E0h, 0, 68h, 1, 0F0h, 2, 0, 9, 0
1582                                     db 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh
1583                                     db 0FFh, 0FFh, 0FFh, 0FFh, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh
1584                                     db 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h, 54h
1585                                     db 31h, 32h, 20h, 20h, 20h, 0
1586
1587 -----
1588
1589 sm92:                                db 3                                ; .spf
1590                                     db 9                                ; .spt
1591                                     dw 112 ; 70h                      ; .cdire
1592                                     dw 1440 ; 2*9*80                  ; .csec
1593                                     db 2                                ; .spau
1594                                     db 2                                ; .thead
1595
1596 %endif
1597
1598 keyrd_func: db 0
1599 0000047E 00
1600 0000047F 01                        keysts_func: db 1
1601 00000480 00                        printdev: db 0          ; printer device index
1602
1603 wait_count: ;dw 4 dup(50h)          ; retry counts for printers
1604 00000481 5000<rep 4h>             times 4 dw 50h        ; 19/10/2022

```

```

1605
1606 00000489 0000      daycnt:          dw 0
1607 0000048B 00      t_switch:   db 0          ; flag for updating daycnt
1608 0000048C 00      havecmosclock: db 0
1609 0000048D 13      base_century: db 19
1610 0000048E 50      base_year:  db 80
1611
1612 0000048F 1F      month_tab:  db 31
1613 00000490 1C      february:   db 28 ; 08/08/2023
1614 00000491 1F1E1F1E1F1E1F1E- db 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
1614 0000049A 1F
1615
1616      ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
1617      %if 0
1618      bintobcd:   dw bin_to_bcd      ; points to bin_to_bcd proc in msinit
1619      dw 70h ; 17/10/2022
1620      daycnttoday: dw daycnt_to_day      ; points to daycnt_to_day in msinit
1621      dw 70h ; 17/10/2022
1622      %endif
1623
1624 0000049B 00      set_id_flag:      db 0          ; flag for getbp routine
1625
1626      ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
1627      ;fat_12_id: db 'FAT12' ,0
1628      ;fat_16_id: db 'FAT16' ,0
1629      ;vol_no_name: db 'NO NAME' ,0
1630      ;temp_h:     dw 0          ; temporary for      32 bit calculation
1631
1632 0000049C 0000      start_sec_h:     dw 0          ; starting sector number high word
1633 0000049E 0000      saved_word: dw 0          ; tempory saving place for a word
1634 000004A0 0000      multrk_flag:     dw 0
1635 000004A2 00      ec35flag:   db 0          ; flags for 3.5 inch disk drives
1636 000004A3 0000      vretry_cnt: dw 0
1637 000004A5 0000      soft_ecc_cnt:   dw 0
1638 000004A7 00      multitrk_format_flag: db 0      ; multitrack format request flag
1639 000004A8 0000      xfer_seg:     dw 0          ; temp for transfer segment
1640
1641      ; variables for msdioc1.asm module
1642
1643      ; tracktable contains a 4-tuples (c,h,r,n) for each sector in a track
1644      ; c = cylinder number,h = head number,r = sector id,n = bytes per sector
1645      ; n      bytes per sector
1646      ; --- -----
1647      ; 0          128
1648      ; 1          256
1649      ; 2          512
1650      ; 3          1024
1651
1652      ;max_sectors_curr_sup equ 63          ; current maximum sec/trk that
1653      ;                          ; we support (was 40 in dos 3.2)
1654
1655 000004AA 2400      sectorspertrack: dw 36
1656 000004AC 00000102 tracktable: db 0, 0, 1, 2
1657 000004B0 00000202      db 0, 0, 2, 2
1658 000004B4 00000302      db 0, 0, 3, 2
1659 000004B8 00000402      db 0, 0, 4, 2
1660 000004BC 00000502      db 0, 0, 5, 2
1661 000004C0 00000602      db 0, 0, 6, 2
1662 000004C4 00000702      db 0, 0, 7, 2
1663 000004C8 00000802      db 0, 0, 8, 2
1664 000004CC 00000902      db 0, 0, 9, 2
1665 000004D0 00000A02      db 0, 0, 10, 2
1666 000004D4 00000B02      db 0, 0, 11, 2
1667 000004D8 00000C02      db 0, 0, 12, 2
1668 000004DC 00000D02      db 0, 0, 13, 2
1669 000004E0 00000E02      db 0, 0, 14, 2
1670 000004E4 00000F02      db 0, 0, 15, 2
1671 000004E8 00001002      db 0, 0, 16, 2
1672 000004EC 00001102      db 0, 0, 17, 2
1673 000004F0 00001202      db 0, 0, 18, 2
1674 000004F4 00001302      db 0, 0, 19, 2
1675 000004F8 00001402      db 0, 0, 20, 2
1676 000004FC 00001502      db 0, 0, 21, 2
1677 00000500 00001602      db 0, 0, 22, 2
1678 00000504 00001702      db 0, 0, 23, 2
1679 00000508 00001802      db 0, 0, 24, 2
1680 0000050C 00001902      db 0, 0, 25, 2
1681 00000510 00001A02      db 0, 0, 26, 2
1682 00000514 00001B02      db 0, 0, 27, 2
1683 00000518 00001C02      db 0, 0, 28, 2
1684 0000051C 00001D02      db 0, 0, 29, 2
1685 00000520 00001E02      db 0, 0, 30, 2
1686 00000524 00001F02      db 0, 0, 31, 2
1687 00000528 00002002      db 0, 0, 32, 2
1688 0000052C 00002102      db 0, 0, 33, 2
1689 00000530 00002202      db 0, 0, 34, 2
1690 00000534 00002302      db 0, 0, 35, 2
1691 00000538 00002402      db 0, 0, 36, 2
1692
1693      ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
1694      ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:053Ch
1695
1696      ;times 108 db 0          ; 19/10/2022
1697      ;;db 108 dup(0)          ; 4*max_sectors_curr_sup - ($ - tracktable) dup      (0)
1698      ; times ((4*63) - 144) db 0
1699
1700 0000053C [5803]      dskdrvs:      dw fdrive1
1701 0000053E [EE03]      dw fdrive2
1702
1703      ;dw 52 dup(0)
1704 00000540 00<rep 68h> times 104 db 0      ; times (((4*63)-144)-4) db 0
1705      ; 4*max_sectors_curr_sup-($-tracktable)-4 dup (0)
1706
1707      ;-----
1708
1709      ; this is a real ugly place to put this
1710      ; it should really go in the bds
1711
1712 000005A8 00      mediatype: db 0
1713 000005A9 00      media_set_for_format: db 0      ; 1 if we have done an int 13h set media
1714      ; type for format call
1715 000005AA 00      had_format_error: db 0      ; 1 if the previous format operation
1716      ; failed.
1717
1718      ; temp disk base table. it holds the the current dpt which is then replaced by
1719      ; the one passed by "new roms" before we perform a format operation. the old
1720      ; dpt is restored in restoreolddpt. the first entry (disk_specify_1) is -1 if
1721      ; this table does not contain the previously saved dpt.
1722
1723 000005AB FFFFFFFF      tempdpt:   dd 0FFFFFFFFh ; -1      ; temp disk base table
1724 000005AF FF      model_byte: db 0FFh      ; model byte set at init time
1725 000005B0 00      secondary_model_byte: db 0
1726
1727 000005B1 00      int19sem:  db 0          ; indicate that all int 19h

```



```

1728                                     ; initialization is complete
1729
1730 ;: we assume the following remain contiguous and their order doesn't change
1731 ;i19_lst:
1732 ;   irp   aa,<02,08,09,0a,0b,0c,0d,0e,70,72,73,74,76,77>
1733 ;   public int19old&aa
1734 ;   db     aa&h      ; store the number as a byte
1735 ;int19old&aa      dd     -1      ; original hardware int. vectors for int 19h.
1736 ;   endm
1737
1738 ; 21/10/2022
1739
1740 000005B2 02      i19_lst:      db 2
1741                                     ; Int19old&aa
1742 000005B3 FFFFFFFF int19old02: dd 0FFFFFFFh ; -1
1743 000005B7 08      db 8
1744 000005B8 FFFFFFFF int19old08: dd 0FFFFFFFh      ; original hardware int. vectors for int 19h
1745 000005BC 09      db 9
1746 000005BD FFFFFFFF int19old09: dd 0FFFFFFFh
1747 000005C1 0A      db 0Ah
1748 000005C2 FFFFFFFF int19old0A: dd 0FFFFFFFh
1749 000005C6 0B      db 0Bh
1750 000005C7 FFFFFFFF int19old0B: dd 0FFFFFFFh
1751 000005CB 0C      db 0Ch
1752 000005CC FFFFFFFF int19old0C: dd 0FFFFFFFh
1753 000005D0 0D      db 0Dh
1754 000005D1 FFFFFFFF int19old0D: dd 0FFFFFFFh
1755 000005D5 0E      db 0Eh
1756 000005D6 FFFFFFFF int19old0E: dd 0FFFFFFFh
1757 000005DA 70      db 70h
1758 000005DB FFFFFFFF int19old70: dd 0FFFFFFFh
1759 000005DF 72      db 72h
1760 000005E0 FFFFFFFF int19old72: dd 0FFFFFFFh
1761 000005E4 73      db 73h
1762 000005E5 FFFFFFFF int19old73: dd 0FFFFFFFh
1763 000005E9 74      db 74h
1764 000005EA FFFFFFFF int19old74: dd 0FFFFFFFh
1765 000005EE 76      db 76h
1766 000005EF FFFFFFFF int19old76: dd 0FFFFFFFh
1767 000005F3 77      db 77h
1768 000005F4 FFFFFFFF int19old77: dd 0FFFFFFFh
1769
1770 ;num_i19      equ ($ - i19_lst)/5 ; 18/03/2019
1771
1772 ;-----
1773
1774 ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
1775 ;
1776 ;dskdrvs:      dw fdrive1
1777 ;              dw fdrive2
1778 ;              dw fdrive3
1779 ;              dw fdrive4
1780 ;
1781 ;;M011 -- made all hard drive stuff variable
1782 ;;dw 22 dup(0) ; up to 26 drives for mini disks
1783 ;;times 22 dw 0 ; 19/10/2022
1784 ;
1785 ;-----
1786
1787 ; 01/10/2022 - Retro DOS v4.0 (MSDOS v5.0 -actual-)
1788 ; 30/12/2018 - Retro DOS v4.0 (MSDOS v6.21 -draft-)
1789 ; 01/06/2018 - Retro DOS v3.0 (MSDOS v3.3)
1790
1791 ;variables for dynamic relocatable modules
1792 ;these should be stay resident.
1793
1794 000005F8 00000000 int6c_ret_addr: dd 0      ; return address from int 6ch
1795                                     ; for p12 machine
1796
1797 ; data structures for real-time date and time
1798
1799 000005FC 00000000 bin_date_time: db 0, 0, 0, 0      ; century, year, month, day
1800
1801 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
1802 %if 0
1803 month_table:      dw 0      ; january
1804                  dw 31      ; february
1805                  dw 59
1806                  dw 90
1807                  dw 120
1808                  dw 151
1809                  dw 181
1810                  dw 212
1811                  dw 243
1812                  dw 273
1813                  dw 304
1814                  dw 334      ; december
1815 %endif
1816
1817 00000600 0000      daycnt2: dw 0
1818 ; 08/08/2023
1819 ;feb29:          db 0      ; february 29 in a leap year flag
1820
1821 ;-----
1822 ;
1823 ; 01/10/2022 - (New/Actual) Retro DOS v4.0 (will run as MSDOS 5.0)
1824 ; by Erdogan Tan (Istanbul) ! free source code !
1825 ; 31/12/2018 - (old/draft) Retro DOS v4.0 (will/would run as MSDOS 6.21)
1826 ;
1827 ; -----
1828 ;
1829 ;*****
1830 ;*
1831 ;* Entry points into Bios_Code routines. The segment values *
1832 ;* are plugged in by seg_reinit. *
1833 ;*
1834 ;*****
1835
1836 ; 01/10/2022 - Retro DOS v4.0 - IO.SYS (MSDOS v5.0)
1837 ; BIOSCODE_SEGMENT equ 2C7h
1838 ; BIOSDATA_SEGMENT equ 70h ; KERNEL_SEGMENT equ 70h
1839
1840 ; 01/10/2022 - Erdogan Tan
1841 ; (disassembled MSDOS 5.0 IO.SYS code here with fixed function/routine
1842 ; addresses, they will be changed to table labels later)
1843
1844 ; 09/12/2022
1845 %if 0
1846 cdev:          dw 43h, 2C7h      ; chardev_entry
1847                                     ; at 2C7h:43h = 70h:25B3h
1848 tticks:        dw 396h, 2C7h      ; time_to_ticks
1849                                     ; at 2C7h:396h = 70h:2906h
1850 bcode_i2f:     dw 1302h, 2C7h      ; i2f_handler
1851                                     ; at 2C7h:1302h = 70h:3872h

```

```

1852      i13x:          dw 154Bh, 2C7h          ; i13z
1853                                     ; at 2C7h:154Bh      = 70h:3ABh
1854 %endif
1855
1856 ; 30/12/2022
1857 ; (IOSYSCODESEG is 2CCh for MSDOS 6.21 IO.SYS)
1858
1859 ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
1860 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0602h
1861 ; (IOSYSCODESEG is 364h for PCDOS 7.1 IBMBIO.COM)
1862
1863 ; 09/12/2022
1864 cdev:          dw chardev_entry, IOSYSCODESEG
1865 tticks:       dw time_to_ticks, IOSYSCODESEG
1866 ; 07/08/2023
1867 bcode_i2f:    dw i2f_handler, IOSYSCODESEG
1868 i13x:         dw i13z, IOSYSCODESEG
1869
1870 end_BC_entries: ; 15/10/2022
1871
1872 ;*****
1873 ;*
1874 ;* cbreak - break key handling - simply set altah=3 and iret *
1875 ;*
1876 ;*****
1877
1878 cbreak:
1879     mov     byte [cs:altah], 3 ; break key handling
1880                                     ; indicate break key set
1881
1882 intret:
1883     iret
1884
1885 ; ===== S U B   R O U T I N E =====
1886
1887 ;*****
1888 ;*
1889 ;* strategy - store es:bx (device driver request packet) *
1890 ;*          away at [ptrsav] for next driver function call *
1891 ;*
1892 ;*****
1893
1894 strategy: ; proc far
1895     mov     [cs:ptrsav], bx ; store es:bx (device driver request packet)
1896                                     ; away at [ptrsav] for next driver function call
1897     mov     [cs:ptrsav+2], es
1898     retf
1899
1900 ; -----
1901
1902 ;*****
1903 ;*
1904 ;* device driver entry points. these are the initial *
1905 ;* 'interrupt' hooks out of the device driver chain. *
1906 ;* in the case of our resident drivers, they'll just *
1907 ;* stick a fake return address on the stack which *
1908 ;* points to dispatch tables and possibly some unit *
1909 ;* numbers, and then call through a common entry point *
1910 ;* which can take care of a20 switching *
1911 ;*
1912 ;*****
1913
1914 ; 01/10/2022 - Erdogan Tan
1915 ; (disassembled MSDOS 5.0 IO.SYS code here with fixed table
1916 ; addresses, they will be changed to table labels later)
1917
1918 ; 09/12/2022
1919
1920 ; 02/10/2023 - Retro DOS v5.0
1921 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:0620h, BIOSCODE = 0364h)
1922
1923 con_entry:
1924     call    cdev_entry
1925 ; -----
1926     ;dw 0E4h          ; con_table
1927     dw con_table      ; 364h:0E4h (PCDOS 7.1)
1928                                     ; 2C7h:0E4h = 70h:2654h
1929 ; -----
1930
1931 prn0_entry:
1932     call    cdev_entry
1933 ; -----
1934     ;dw 0FBh          ; prn_table
1935     dw prn_table      ; 2C7h:0FBh = 70h:266Bh
1936
1937     db 0, 0
1938 ; -----
1939
1940 prn1_entry:
1941     call    cdev_entry
1942 ; -----
1943     ;dw 0FBh          ; prn_table
1944     dw prn_table      ; 2C7h:0FBh = 70h:266Bh
1945
1946     db 0, 1
1947 ; -----
1948
1949 prn2_entry:
1950     call    cdev_entry
1951 ; -----
1952     ;dw 0FBh          ; prn_table
1953     dw prn_table      ; 2C7h:0FBh = 70h:266Bh
1954
1955     db 1, 2
1956 ; -----
1957
1958 prn3_entry:
1959     call    cdev_entry
1960 ; -----
1961     ;dw 0FBh          ; prn_table
1962     dw prn_table      ; 2C7h:0FBh = 70h:266Bh
1963
1964     db 2, 3
1965 ; -----
1966
1967 aux0_entry:
1968     call    cdev_entry
1969 ; -----
1970     ;dw 130h          ; aux_table
1971     dw aux_table      ; 2C7h:130h = 70h:26A0h
1972
1973     db 0
1974 ; -----
1975

```



```

1976 aux1_entry: call cdev_entry
1977 00000647 E81900 ; -----
1978 ; dw 130h ; aux_table
1979 dw aux_table ; 364h:130h = 70h:3070h (PCDOS 7.1)
1980 0000064A [3001] db 1 ; 2C7h:130h = 70h:26A0h
1981
1982 0000064C 01 ; -----
1983
1984 aux2_entry: call cdev_entry
1985 ; -----
1986 0000064D E81300 ; dw 130h ; aux_table
1987 dw aux_table ; 2C7h:130h = 70h:26A0h
1988
1989 00000650 [3001] db 2
1990
1991 00000652 02 ; -----
1992
1993 aux3_entry: call cdev_entry
1994 ; -----
1995 00000653 E80D00 ; dw 130h ; aux_table
1996 dw aux_table ; 2C7h:130h = 70h:26A0h
1997
1998 00000656 [3001] db 3
1999
2000 00000658 03 ; -----
2001
2002 tim_entry: call cdev_entry
2003 ; -----
2004 00000659 E80700 ; dw 147h ; tim_table
2005 dw tim_table ; 364h:147h = 70h:3087h (PCDOS 7.1)
2006
2007 0000065C [4701] ; 2C7h:147h = 70h:26B7h
2008
2009 ; -----
2010 ; 15/10/2022
2011 ; DSKTBL equ dsktbl - DOSBIOSEG_2C7h ; dsktbl - 2C70h
2012 ; 09/12/2022
2013 DSKTBL equ dsktbl
2014
2015 dsk_entry: call cdev_entry
2016 ; -----
2017 0000065E E80200 ; dw 4A2h ; dsktbl
2018 dw DSKTBL ; 09/12/2022
2019 ; 2C7h:4A2h = 70h:2A12h
2020 00000661 [6F05] ; 02/10/2023 (PCDOS 7.1 IBMBIO.COM)
2021 ; 364h:579h = 70h:34B9h
2022
2023 ; ===== S U B R O U T I N E =====
2024
2025 ;*****
2026 ;*
2027 ;* Ensure A20 is enabled before jumping into code in HMA. *
2028 ;* This code assumes that if Segment of Device request packet is *
2029 ;* DOS DATA segment then the Device request came from DOS & that *
2030 ;* A20 is already on. *
2031 ;*
2032 ;*****
2033
2034 cdev_entry: proc near
2035 cmp byte [cs:inHMA],0
2036 jz short ce_enter_codesseg
2037 ; optimized for DOS in HMA
2038
2039 push ax
2040 mov ax,[cs:DosDataSg]
2041 cmp [cs:ptrsav+2],ax
2042 pop ax
2043 jnz short not_from_dos
2044 ; jump is coded this way to fall thru
2045 ; in 99.99% of the cases
2046
2047 ce_enter_codesseg:
2048 jmp far [cs:cdev]
2049 jmp dword ptr cs:cdev
2050 ;-----
2051
2052 not_from_dos:
2053 call EnsureA20on
2054 jmp short ce_enter_codesseg
2055
2056 ;*****
2057 ;*
2058 ;* outchr - this is our int 29h handler. it writes the *
2059 ;* character in al on the display using int 10h ttywrite *
2060 ;*
2061 ;*****
2062
2063 ; 17/07/2024
2064 ; 02/10/2023
2065 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0682h
2066
2067 outchr: push ax ; int 29h handler
2068 push si
2069 push di
2070 push bp
2071 push bx
2072 ;;;
2073 ; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
2074 ;mov ah,0Eh
2075 ;mov bx,7
2076 ;int 10h ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
2077 ; ; AL = character, BH = display page (alpha modes)
2078 ; ; BL = foreground color (graphics modes)
2079 ; 17/07/2024
2080 ; 02/10/2023
2081 push ds ; *
2082 xor bx,bx ; 0
2083 mov ds,bx ; 0
2084 mov ah,0Eh
2085 mov bl,7
2086 cmp [cs:Iswin386],bl ; (are we in) windows ?
2087 ; 17/07/2024
2088 ;jnz short win_outchr ; *
2089 ;push ds ; *
2090 ;mov ds,bx ; 0
2091 ;mov ah,0Eh
2092 ;mov bl,7
2093 jnz short win_outchr ; Running on windows
2094 pushf ; far call (simulate INT)
2095 cli ; disable interrupts
2096 call far [40h] ; far call to INT 10h vector
2097 ; 17/07/2024
2098 ;pop ds ; *
2099 jmp short outchr_ok

```

```

2100 win_outchr:
2101 0000069F CD10      int     10h
2102 outchr_ok:
2103                ; 17/07/2024
2104 000006A1 1F        pop     ds ; *
2105                ;;;
2106 000006A2 5B        pop     bx
2107 000006A3 5D        pop     bp
2108 000006A4 5F        pop     di
2109 000006A5 5E        pop     si
2110 000006A6 58        pop     ax
2111 000006A7 CF        ired
2112
2113 ;-----
2114
2115                ; 02/10/2023 - Retro DOS v5.0
2116                ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:06A8h
2117
2118 000006A8 50        db 50h ; P                ; 'PCI' signature
2119 000006A9 43        db 43h ; C
2120 000006AA 49        db 49h ; I
2121
2122 000006AB 00000000   orig1A:      dd 0
2123
2124                ; ===== S U B R O U T I N E =====
2125
2126                ; 02/10/2023 - Retro DOS v5.0
2127                ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:06AFh
2128
2129 int1A:
2130 000006AF 80FC04     cmp     ah,4      ; (Y2K-fix)
2131 000006B2 7405     je      short int1a_1 ; Read the date from the computer's real-time clock
2132 000006B4 2EFF2E[AB06] jmp     far [cs:orig1A]
2133
2134 000006B9 55        int1a_1:   push    bp
2135
2136 000006BA 89E5     int1a_2:   mov     bp,sp
2137 000006BC 55        push    bp
2138 000006BD 9C        pushf
2139 000006BE 2EFF1E[AB06] call   far [cs:orig1A]
2140 000006C3 7220     jc      short int1a_4
2141
2142                ; cmp     cl,0      ; Year (BCD)
2143                ; 02/10/2023
2144 000006C5 08C9     or      cl,cl
2145 000006C7 7515     jnz     short int1a_3
2146 000006C9 80FD19     cmp     ch,19h    ; Century (BCD)
2147 000006CC 7510     jne     short int1a_3
2148 000006CE B520     mov     ch,20h
2149 000006D0 B405     mov     ah,5      ; Set the date on the computer's real-time clock
2150 000006D2 51        push    cx
2151 000006D3 52        push    dx      ; dh = Month (BCD), dl = Day (BCD)
2152 000006D4 9C        pushf
2153 000006D5 2EFF1E[AB06] call   far [cs:orig1A]
2154 000006DA 5A        pop     dx
2155 000006DB 59        pop     cx
2156 000006DC 7207     jc      short int1a_4
2157
2158 000006DE 5D        int1a_3:   pop     bp
2159 000006DF 806606FE and     byte [bp+6],0FEh ; clear carry flag
2160 000006E3 EB05     jmp     short int1a_5
2161
2162 000006E5 5D        int1a_4:   pop     bp
2163 000006E6 804E0601 or      byte [bp+6],1 ; set carry flag
2164
2165 000006EA 5D        int1a_5:   pop     bp
2166 000006EB CF        ired
2167
2168                ; 02/10/2023
2169 000006EC 90        nop      ; (not necessary, i have used this 'nop' to locate 'block13:'
2170                ; at BIOSDATA:06EDh, just as in the original PCDOS 7.1 IBMBIO.COM)
2171
2172 ;-----
2173
2174                ; *****
2175                ; *
2176                ; * block13 - our int13 hooker
2177                ; *
2178                ; *****
2179
2180                ; 02/10/2023 - Retro DOS v5.0
2181                ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:06EDh
2182
2183 block13:
2184 000006ED 2E803E[0D00]00 cmp     byte [cs:inHMA],0
2185 000006F3 7403     jz      short skipa20
2186
2187                ; call     IsA20Off      ; A20 off?
2188                ; jnz     short skipa20
2189                ; call     EnableA20     ; assure a20 enabled
2190                ; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
2191 000006F5 E83200     call    EnsureA20On ; assure a20 enabled
2192
2193 000006F8 2E8C1E[1C00] skipa20: mov     [cs:i13_ds],ds ; save caller's ds for call-through
2194 000006FD 9C        pushf      ; fake interrupt
2195 000006FE 2EFF1E[0A06] call   far [cs:i13x]
2196                ; call     dword ptr cs:i13x
2197                ; call through Bios_Code entry table
2198 00000703 2E8E1E[1C00] mov     ds,[cs:i13_ds]
2199 00000708 CA0200     retf     2
2200
2201                ; ===== S U B R O U T I N E =====
2202
2203                ; the int13 hook calls back here to call-through to the ROM
2204                ; this is necessary because some people have extended their
2205                ; ROM BIOSs to use ds as a parameter/result register and
2206                ; our int13 hook relies heavily on ds to access Bios_Data
2207
2208 call_orig13:
2209 0000070B 8E1E[1C00] mov     ds,[i13_ds] ; get caller's ds register
2210 0000070F 9C        pushf      ; simulate an int13
2211 00000710 2EFF1E[B400] call   far [cs:orig13]
2212                ; call     cs:orig13
2213 00000715 2E8C1E[1C00] mov     [cs:i13_ds],ds
2214 0000071A 0E        push     cs
2215 0000071B 1F        pop      ds      ; restore ds -> Bios_Data before return
2216
2217 0000071C 9C        pushf
2218                ; 10/12/2022
2219                ; ds = cs
2220 0000071D 803E[0D00]00 cmp     byte [inHMA],0 ; 16/10/2022
2221                ; cmp     byte [cs:inHMA],0
2222 00000722 7403     jz      short corig13_popf_ret
2223                ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)

```

```

2224 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0725h
2225 ;call IsA200ff
2226 ;jnz short corig13_popf_ret
2227 ;call EnableA20
2228 00000724 E80300 call EnsureA200n ; 07/08/2023
2229 corig13_popf_ret:
2230 00000727 9D popf
2231 ; 20/09/2023
2232 re_init: ; 07/08/2023
2233 00000728 CB retf
2234
2235 ; 02/10/2023
2236 00000729 90 nop ; (not necessary, i have used this 'nop' to locate 'EnsureA200n:'
2237 ; at BIOSDATA:072Ah, just as in the original PCDOS 7.1 IBMBIO.COM)
2238
2239 ; -----
2240
2241 ; BIOSDATA:07BBh (MSDOS 6.21, IO.SYS)
2242 ; BIOSDATA:07BBh (MSDOS 5.0, IO.SYS) ; 16/10/2022
2243
2244 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2245 ;HiMem: dd 0FFFF0090h
2246 ;LoMem: dd 80h
2247
2248 ; -----
2249
2250 ; ===== S U B R O U T I N E =====
2251
2252 ; *****
2253 ; *
2254 ; *
2255 ; * EnsureA200n - ensure that a20 is enabled if we're running *
2256 ; * in the HMA before interrupt entry points into Bios_Code *
2257 ; *
2258 ; *****
2259
2260 EnsureA200n: ; proc near
2261 0000072A E80E00 call IsA200ff
2262 ;jz short EnableA20
2263 ;ret
2264 ; 18/12/2022
2265 0000072D 750B jnz short A200n_retn
2266
2267 ; ===== S U B R O U T I N E =====
2268
2269 EnableA20: ; proc near
2270 push ax
2271 0000072F 50 push bx
2272 00000730 53 push cx
2273 00000731 B405 mov ah,5 ; local enable a20
2274 ;call cs:xms
2275 00000733 2EFF1E[0E00] call far [cs:xms] ; 16/10/2022
2276 00000738 5B pop bx
2277 00000739 58 pop ax
2278 A200n_retn: ; 18/12/2022
2279 0000073A C3 ret
2280
2281 ; ===== S U B R O U T I N E =====
2282
2283 IsA200ff: ; proc near
2284 push ds
2285 0000073B 1E push es
2286 0000073C 06 push cx
2287 0000073D 51 push si
2288 0000073E 56 push di
2289 0000073F 57 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2290 ;lds si,[cs:HiMem]
2291 ;les di,[cs:LoMem]
2292 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0740h
2293 xor di,di
2294 00000740 31FF mov es,di
2295 00000742 8EC7 dec di
2296 00000744 4F mov si,90h ; 0FFFFh:0090h ; HiMem
2297 00000745 BE9000 mov ds,di
2298 00000748 8EDF mov di,80h ; 0000h:0080h ; LoMem
2299 0000074A BF8000 ; 02/10/2023 - Retro DOS v5.0 IBMBIO.COM (PCDOS 7.1)
2300 ; (following cpu instructions will be modified by 'SYSIN'
2301 ; if the cpu is a 386/32bit, for checking A20 line faster)
2302 cpu386_cmpsd:
2303 nop
2304 0000074D 90 mov cx,8
2305 0000074E B90800 repe cmpsw
2306 00000751 F3A7 ; zf = 0 -> A20 line is ON
2307 ; zf = 1 -> A20 line is OFF
2308
2309 pop di
2310 00000753 5F pop si
2311 00000754 5E pop cx
2312 00000755 59 pop es
2313 00000756 07 pop ds
2314 00000758 C3 ret
2315
2316 ; -----
2317
2318 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2319 %if 0
2320 DisableA20:
2321 push ax
2322 push bx
2323 mov ah,6 ; local disable A20
2324 call far [cs:xms]
2325 ;call cs:xms
2326 pop bx
2327 pop ax
2328 ret
2329 %endif
2330
2331 ; -----
2332
2333 ; *****
2334 ; *
2335 ; * int19 - bootstrap interrupt -- we must restore a bunch of the *
2336 ; * interrupt vectors before resuming the original int19 code *
2337 ; *
2338 ; *****
2339
2340 ; 02/10/2023 - Retro DOS v5.0
2341 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0759h
2342 int19:
2343 00000759 0E push cs
2344 0000075A 1F pop ds
2345 ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2346 ;mov es,[zeroseg] ; 16/10/2022
2347 ;mov cx,5 ; NUMROMVECTORS

```

```

2348 0000075B 31C9      xor     cx,cx
2349 0000075D 8EC1      mov     es,cx
2350 0000075F B105      mov     cl,5
2351      ;mov     si,offset RomVectors
2352 00000761 BE[0001]    mov     si,RomVectors ; 19/10/2022
2353
2354 00000764 AC      next_int: lodsb             ; get int number
2355 00000765 98      cbw             ; assume < 128
2356 00000766 D1E0      shl     ax,1
2357 00000768 D1E0      shl     ax,1      ; int * 4
2358      ; 07/08/2023
2359      ;mov     di,ax
2360      ;lodsw
2361      ;stosw
2362      ;lodsw
2363      ;stosw      ; install the saved vector
2364      ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:076Ah
2365 0000076A 97      xchg     ax,di
2366 0000076B A5      movsw
2367 0000076C A5      movsw
2368 0000076D E2F5      loop    next_int
2369      ;cmp     byte [int19sem], 0 ; 19/10/2022
2370 0000076F 380E[B105]  cmp     [int19sem], cl ; 0 ; 07/08/2023
2371 00000773 7419      jz      short doint19
2372 00000775 BE[B205]    mov     si,i19_1st      ; stacks code has changed these hardware interrupt vectors
2373      ;                               ; stkit in sysinit1 will initialize int19oldxx values
2374      ;                               ; num_i19
2375      ;mov     cx,14
2376 00000778 B10E      ; 07/08/2023
2377      mov     cl,14
2378 0000077A AC      i19_restore_loop: lodsb             ; get interrupt      number
2379 0000077B 98      cbw             ; assume < 128
2380      ;mov     di,ax
2381      ;lodsw
2382      ;mov     bx,ax      ; get original vector offset
2383      ;lodsw      ; save it
2384      ; 07/08/2023
2385 0000077C 97      xchg     ax,di
2386 0000077D AD      lodsw
2387 0000077E 93      xchg     ax,bx
2388 0000077F AD      lodsw
2389      ;cmp     bx,0FFFFh      ; check for 0ffffh (unlikely segment)
2390 00000780 43      inc     bx ; 07/08/2023
2391 00000781 7409      jz      short i19_restor_1 ; opt no need to check selector too
2392      ;cmp     ax,0FFFFh      ; opt 0ffffh is      unlikely offset
2393      ;jz      short i19_restor_1
2394 00000783 4B      dec     bx ; 07/08/2023
2395 00000784 01FF      add     di,di
2396 00000786 01FF      add     di,di
2397 00000788 93      xchg     ax,bx
2398 00000789 AB      stosw
2399 0000078A 93      xchg     ax,bx
2400 0000078B AB      stosw      ; put the vector back
2401
2402 0000078C E2EC      i19_restor_1: loop    i19_restore_loop
2403
2404      doint19:      ;cmp     byte [inhMA],0 ; ; Is dos running from      HMA
2405 0000078E 380E[0D00]  cmp     [inhMA],cl ; 0 ; 07/08/2023
2406 00000792 7403      jz      short SkipVDisk
2407 00000794 E82A00      call    EraseVDiskHead ; Then erase our VDISK header at 1MB boundary
2408      ;                               ; Some m/c's (AST 386 & HP QS/16 do not clear
2409      ;                               ; the memory above 1MB during a      warm boot.
2410
2411 00000797 CD19      skipVDisk:      int     19h      ; DISK BOOT
2412      ;                               ; causes reboot      of disk system
2413
2414      ; ===== S U B   R O U T I N E =====
2415
2416      ;-----
2417      ;
2418      ; procedure : int15
2419      ;
2420      ;      Int15 handler for recognizing ctrl-alt-del seq
2421      ;      If it recognizes ctrl-alt-del and if DOS was
2422      ;      is running high, it Erases the VDISK header
2423      ;      present at 1MB boundary
2424      ;
2425      ;-----
2426
2427      ; 16/10/2022
2428      ;DELKEY      equ     53h
2429      ;ROMDATASEG equ     40h
2430      ;KBFLAG      equ     17h
2431      ;CTRLSTATE  equ     04h
2432      ;ALTSTATE   equ     08h
2433
2434      ; 02/10/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
2435
2436      Int15:
2437 00000799 3D534F      ;cmp     ax,4F00h+DELKEY
2438      cmp     ax,4F53h      ; del keystroke ?
2439      ; 02/10/2023 - Retro DOS v5.0
2440 0000079C 7405      ; 07/08/2023
2441      jz      short int15_1
2442      ;jnz     short old15_j ; 07/08/2023
2443 0000079E 2EFF2E[0B01]  old15_j:      jmp     far [cs:old15] ; 16/10/2022
2444
2445      ;-----
2446      ;int15_1:
2447 000007A3 1E      push     ds
2448 000007A4 50      push     ax
2449      ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2450      ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:07A5h
2451      ;mov     ax,40h      ; ROMDATASEG
2452      ;mov     ds,ax
2453      ;mov     al,ds:17h      ; [KBFLAG]
2454      ; 16/10/2022
2455      ;mov     al,[KBFLAG]
2456 000007A5 31C0      xor     ax,ax
2457 000007A7 8ED8      mov     ds,ax
2458 000007A9 A01704      mov     al,[0417h]      ; KBFLAG = 0417h (PCDOS 7.1 IBMBIO.COM)
2459 000007AC 240C      and     al,0Ch      ; (CTRLSTATE | ALTSTATE)
2460 000007AE 3C0C      cmp     al,0Ch      ; (CTRLSTATE | ALTSTATE)
2461 000007B0 750A      jnz     short int15_2
2462      ; 07/08/2023
2463      ;push     cs
2464      ;pop      ds
2465      ;cmp     byte [inhMA],0 ; is DOS running from HMA
2466 000007B2 2E3826[0D00]  cmp     byte [cs:inhMA],ah ; 0
2467 000007B7 7403      jz      short int15_2
2468 000007B9 E80500      call    EraseVDiskHead
2469
2470 000007BC 58      int15_2:      pop     ax
2471 000007BD 1F      pop     ds

```

```

2472 000007BE F9          stc
2473                      ; 02/10/2023 - Retro DOS v5.0
2474 000007BF EBDD      jmp     short Old15_j
2475                      ; 02/10/2023
2476                      ; 07/08/2023
2477 Old15_j:             ;
2478                      jmp     far [cs:old15] ; 16/10/2022
2479                      ;
2480                      jmp     cs:old15
2481                      ;
2482                      ; ===== S U B   R O U T I N E =====
2483                      ;
2484                      ; -----
2485                      ; procedure : EraseVDiskHead
2486                      ;
2487                      ; Erases the VDisk Header present in the 1MB boundary
2488                      ;
2489                      ; -----
2490
2491 EraseVDiskHead:        ; proc near
2492                      ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2493                      ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:07C1h
2494                      ; push ax
2495 000007C1 51          push  cx
2496 000007C2 57          push  di
2497 000007C3 06          push  es
2498 000007C4 E863FF     call   EnsureA20on
2499                      ; mov ax,0FFFFh ; HMA seg
2500                      ; mov es,ax
2501                      ; 03/10/2023 - Retro DOS v5.0
2502 000007C7 6AFF       push  0FFFFh
2503 000007C9 07          pop    es
2504 000007CA BF1000     mov     di,10h ; point to VDISK header
2505                      ; 07/08/2023
2506                      ; mov cx,10h ; size of vdisk header
2507 000007CD 89F9       mov     cx,di ; 16
2508                      ; 03/10/2023
2509 000007CF 31C0       xor     ax,ax
2510                      ; inc ax ; ax = 0
2511 000007D1 F3AB       rep     stosw ; clear it
2512 000007D3 07          pop    es
2513 000007D4 5F          pop    di
2514 000007D5 59          pop    cx
2515                      ; pop ax ; 07/08/2023
2516 000007D6 C3          retn
2517                      ;
2518                      ; -----
2519                      ; 03/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
2520                      ; 17/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
2521                      ;
2522                      ; 09/12/2022
2523                      ; SYSINITSEG equ 46Dh ; SYSINIT segment
2524                      ; DOSLOADSEG equ 83Fh ; MSDOS.SYS (kernel) loading segment
2525                      ; (followings are in sysinit segment)
2526                      ; FTRYTOmovDOSHI equ 0A84h ; (procedure in SYSINIT segment)
2527                      FTRYTOmovDOSHI equ FTRYTOmovDOSHI ; SYSINIT section
2528                      ; DEVICELIST equ 273h
2529                      ;
2530                      DEVICELIST equ DEVICE_LIST ; SYSINIT section
2531                      ; MEMORYSIZE equ 292h
2532                      ;
2533                      MEMORYSIZE equ MEMORY_SIZE ; SYSINIT section
2534                      ; DEFAULTDRIVE equ 296h
2535                      ;
2536                      DEFAULTDRIVE equ DEFAULT_DRIVE ; SYSINIT section
2537                      ; currentdoslocation equ 271h
2538                      ;
2539                      CURRENTDOSLOCATION equ 271h
2540                      ;
2541                      CURRENTDOSLOCATION equ CURRENT_DOS_LOCATION ; SYSINIT section
2542                      ; SYSINITSTART equ 267h
2543                      ;
2544                      SYSINITSTART equ SYSINIT ; SYSINIT section
2545                      ; 18/10/2022
2546                      ; toomanydrivesflag equ 3FFh
2547                      ;
2548                      TOOMANYDRIVESFLAG equ toomanydrivesflag ; SYSINIT section
2549                      ;
2550                      ; -----
2551                      ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
2552                      ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:07D7h
2553                      ;
2554                      %if 1
2555                      FreeHMAPtr: dw 0FFFFh
2556                      ; MoveDOSIntoHMA: dd 46D0A84h ; FTRYTOmovDOSHI
2557                      ; (procedure in SYSINIT segment)
2558                      ; 17/10/2022
2559                      MoveDOSIntoHMA: dw FTRYTOmovDOSHI ; 09/12/2022
2560                      dw SYSINITSEG ; 08/08/2023
2561                      ; 0544h for PCDOS 7.1 IBMBIO.COM
2562                      ; 0473h for MSDOS 6.21 IO.SYS
2563                      ; SR;
2564                      ; A communication block has been setup between the DOS and the BIOS. All
2565                      ; the data starting from SysinitPresent will be part of the data block.
2566                      ; Right now, this is the only data being communicated. It can be expanded
2567                      ; later to add more stuff
2568                      ;
2569                      SysinitPresent: db 0
2570                      ;
2571                      %endif
2572                      ; -----
2573                      ; *****
2574                      ; *
2575                      ; * the int2f handler chains up to Bios_Code through here. *
2576                      ; * it returns through one of the three functions that follow. *
2577                      ; * notice that we'll assume we're being entered from DOS, so *
2578                      ; * that we're guaranteed to be A20 enabled if needed *
2579                      ; *
2580                      ; *****
2581                      ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
2582                      %if 0 ; 20/09/2023
2583                      int_2f:
2584                      jmp     far [cs:bcode_i2f] ; 16/10/2022
2585                      jmp     dword ptr cs:bcode_i2f ; far [cs:bcode_i2f]
2586                      ; -----
2587                      ;
2588                      ; re-enter here to transition out of hma mode and jmp to dsk_entry
2589                      ; note: is it really necessary to transition out and then back in?
2590                      ; It's not as if this is a really speed critical function.
2591                      ; might as well do whatever's most compact.
2592                      ;
2593                      i2f_dskentry:
2594                      jmp     dsk_entry
2595

```

```

2596 ; -----
2597 ;
2598 ;*****
2599 ;*
2600 ;* re_init - called back by sysinit after a bunch of stuff *
2601 ;* is done. presently does nothing. affects no *
2602 ;* registers! *
2603 ;* *
2604 ;*****
2605 ;
2606 ; 09/12/2022
2607 ; re_init:
2608 re_init: ; called back by sysinit after
2609 ; retf ; a bunch of stuff is done.
2610 ; ; presently does nothing
2611 %endif
2612 ; -----
2613 ;
2614 ;SR; WIN386 support
2615 ;
2616 ; WIN386 instance data structure
2617 ;
2618 ; Here is a win386 startup info structure which we set up and to which
2619 ; we return a pointer when win386 initializes.
2620 ;
2621 ;
2622 win386_SI: db 3,0 ; SI_Version
2623 ; ; Startup Info for win386
2624 SI_Next: dd 0 ; pointer to next info structure
2625 ; dd 0 ; a field we don't need
2626 ; dd 0 ; another field we don't need
2627 SI_Instance: dw Instance_Table
2628 ; dw 70h ; Bios_Data ; far pointer to instance table
2629 ;
2630 ; This table gives win386 the instance data in the BIOS and ROM-BIOS data
2631 ; areas. Note that the address and size of the hardware stacks must
2632 ; be calculated and inserted at boot time.
2633 ;
2634 Instance_Table: dw 0,50h ; printscreen status...
2635 ; dw 2 ; ... 2 bytes
2636 ; dw 0Eh,50h ; ROM Basic data...
2637 ; dw 14h ; ... 14H bytes
2638 ; dw altah ; a condevice buffer...
2639 ; dw 70h ; Bios_Data segment
2640 ; dw 1 ; ... 1 byte
2641 ;
2642 NextStack:
2643 ;
2644 ; NOTE: If stacks are disabled by STACKS=0,0, the following
2645 ; instance items WILL NOT be filled in by SYSINIT.
2646 ; That's just fine as long as these are the last items.
2647 ; in the instance list since the first item is initialized
2648 ; to 0000 at load time.
2649 ;
2650 ; dw 0,0 ; pointer to next stack to be used...
2651 ; dw 2 ; ... 2 bytes
2652 IT_StackLoc: dd 0 ; location of hardware stacks
2653 IT_StackSize: dw 0 ; size of hardware stacks
2654 ; dd 0 ; terminate the instance table
2655 ;
2656 ;SR;
2657 Iswin386: db 0 ; Flag to indicate whether
2658 ; ; win386 is running or not
2659 ; -----
2660 ;
2661 ; 09/12/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
2662 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:0813h
2663 ;
2664 ;This routine was originally in BIOS_CODE but this causes a lot of problems
2665 ;when we call it including checking of A20. The code being only about
2666 ;30 bytes, we might as well put it in BIOS_DATA
2667 ;
2668 V86_Crit_SetFocus:
2669 ; push di
2670 ; push es
2671 ; push bx
2672 ; push ax
2673 ; xor di,di
2674 ; mov es,di
2675 ; mov bx,15h ; Device ID of DOSMGR device
2676 ; mov ax,1684h ; Get API entry point
2677 ; int 2Fh ; - Multiplex - MS WINDOWS - GET DEVICE API ENTRY POINT
2678 ; ; BX = virtual device (VxD) ID, ES:DI =0000h:0000h
2679 ; ; Return: ES:DI -> VxD API entry point, or 0:0 if the VxD does not
2680 ; support an API
2681 ; mov ax, es
2682 ; or ax, di
2683 ; jz short Skip ; Here, es:di is address of API routine.
2684 ; ; Set up stack frame to simulate a call.
2685 ; push cs
2686 ; mov ax,offset Skip
2687 ; mov ax,Skip
2688 ; push ax
2689 ; 03/10/2023 - Retro DOS v5.0
2690 ; push Skip
2691 ; push es ; API far call address
2692 ; push di ; SetFocus function number
2693 ; mov ax,1 ; do the call
2694 ; retf
2695 ; -----
2696 ;
2697 Skip:
2698 ; pop ax
2699 ; pop bx
2700 ; pop es
2701 ; pop di
2702 ; retf
2703 ;
2704 ;End WIN386 support
2705 ; -----
2706 ;
2707 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
2708 %if 0
2709 ;
2710 FreeHMAptr: dw 0FFFFh
2711 ;MoveDOSIntoHMA: dd 46D0A84h ; FTryToMovDOSHi
2712 ; ; (procedure in SYSINIT segment)
2713 ; 17/10/2022
2714 MoveDOSIntoHMA: dw FTRYTOMOVDOshi ; 09/12/2022
2715 ; dw SYSINITSEG ; 08/08/2023
2716 ; ; 0544h for PCDOS 7.1 IBMBIO.COM
2717 ; ; 0473h for MSDOS 6.21 IO.SYS
2718 ;SR;

```



```

2719 ; A communication block has been setup between the DOS and the BIOS. All
2720 ; the data starting from SysinitPresent will be part of the data block.
2721 ; Right now, this is the only data being communicated. It can be expanded
2722 ; later to add more stuff
2723
2724 SysinitPresent:    db 0
2725
2726 endfloppy: db 0, 0
2727
2728 %endif
2729
2730 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM
2731
2732 endfloppy:
2733 db 0
2734
2735 ; 03/10/2023
2736
2737 numxdiv    equ ($-BData_start)
2738 numxmod    equ (numxdiv % 16)
2739
2740 %if (numxmod>0) & (numxmod<16)
2741 00000839 00<rep 7h>    times (16-numxmod) db 0
2742 %endif
2743
2744 ; -----
2745
2746 ; Bios_Data ends
2747
2748 ; Possibly disposable BIOS data
2749 ; This data follows the regular BIOS data,
2750 ; and is part of the same group.
2751
2752 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM
2753 ; nul_vid: db 'NO NAME', 0 ; null volume id
2754 ; tmp_vid: db 'NO NAME', 0 ; vid scratch buffer
2755
2756 ; 03/10/2023
2757 00000840 4E4F204E414D452020- tmp_vid: db 'NO NAME'
2758 00000849 2020
2759
2760 harddrv:    db 80h
2761
2762 end96tpi:
2763
2764 ; 03/10/2023 - Retro DOS v5.0 IBMBIO.COM (Modified PCDOS 7.1)
2765 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:084Ch ('bdss:' address)
2766
2767 ; *****
2768 ; memory allocation for bdss
2769 ; *****
2770 ; max_mini_dsk_num equ 23 ; max # of mini disk ibmbio can support
2771 ;
2772 ; bdss BDS_STRUC (2+max_mini_dsk_num) dup (<>) ; currently max. 25
2773 ;
2774 ; bdss: times BDS.size*(2+max_mini_dsk_num) db 0
2775
2776
2777 ; 09/12/2023
2778 %if 1
2779 ; Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM BDS structure (FAT32 adaptation)
2780
2781 0000084C FFFF bdss: dw 0FFFFh ;
2782 ; max_mini_dsk_num equ 23
2783 ; BDS_STRUC (2+max_mini_dsk_num) dup (<>)
2784 ; currently max. 25
2785 ; (MSDOS 6 BDS structure size = 100 bytes)
2786 ; (PCDOS 7.1 BDS structure size = 150 bytes)
2787 ; BDS.link
2788 0000084E 0000 dw 0
2789 00000850 50 db 80 ; BDS.drivenum
2790 00000851 03 db 3 ; BDS.drivelet
2791 00000852 0002 dw 512 ; BDS.BPB (BDS offset 6)
2792 ; 53 bytes BPB for FAT32 fs
2793 ; 25 bytes BPB for FAT16 and FAT12 fs
2794 ; .bytespersec
2795 00000854 01 db 1 ; .secpersclus
2796 00000855 0100 dw 1 ; .resectors
2797 00000857 02 db 2 ; .fats
2798 00000858 1000 dw 16 ; .direntries
2799 0000085A 0000 dw 0 ; .totalsec16
2800 0000085C F8 db 0F8h ; .media
2801 0000085D 0100 dw 1 ; .fatsecs16
2802 0000085F 0000 dw 0 ; .secpctrack
2803 00000861 0000 dw 0 ; .heads
2804 00000863 00000000 dd 0 ; .hiddensectors
2805 00000867 00000000 dd 0 ; .totalsecs32
2806 ; (End of FAT12/FAT16 BPB)
2807 ;
2808 ; FAT32 extensions to BDS
2809 0000086B 00000000 dd 0 ; .fatsecs32 ; BPB_FATSz32 (BDS offset 31)
2810 0000086F 0000 dw 0 ; .extflags ; BPB_ExtFlags
2811 00000871 0000 dw 0 ; .fsver ; BPB_FSVer
2812 00000873 00000000 dd 0 ; .rootdirclust ; BPB_RootClus (BDS offset 39)
2813 00000877 FFFF dw 0FFFFh ; .fsinfo ; BPB_FSInfo ; initialized to -1
2814 00000879 FFFF dw 0FFFFh ; .bkbootsec ; BPB_BkBootSec ; initialized to -1
2815 0000087B 00<rep Ch> times 12 db 0 ; .reserved ; BPB_Reserved (12 zero bytes)
2816 00000887 00 db 0 ; BDS.fatsiz (BDS offset 59)
2817 00000888 0000 dw 0 ; BDS.opcnt
2818 0000088A 03 db 3
2819 0000088B 2000 dw 20h ; BDS.flags (BDS offset 63)
2820 0000088D 2800 dw 40
2821 0000088F 00<rep 25h> times 37 db 0
2822 000008B4 FFFFFFFF dd 0FFFFFFFh
2823 000008B8 00<rep Ch> times 12 db 0
2824 000008C4 FF db -1 ; BDS.track (BDS offset 120)
2825 000008C5 0100 dw 1 ; BDS.tim_lo ; BDS.bds_m_ismini
2826 000008C7 0000 dw 0 ; BDS.tim_hi
2827 000008C9 4E4F204E414D452020- db 'NO NAME', 0 ; BDS.vol_id
2828 000008D2 202000
2829 000008D5 00000000 dd 0 ; BDS.vol_serial (BDS offset 137)
2830 000008D9 464154313220202000 db 'FAT12', 0 ; BDS.filesys_id
2831 000008E2 FFFF dw 0FFFFh
2832 000008E4 000050030002010100- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
2833 000008ED 02100000000F8
2834 000008F3 010000000000000000- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2835 000008FC 000000000000000000
2836 00000905 0000000000000000FF- db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
2837 0000090E FFFFFFFF0000
2838 00000913 00000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
2839 0000091C 00000000003200028
2840 00000924 00000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2841 0000092D 00000000000000000000

```

2836	00000936	00000000000000000000-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2836	0000093F	00000000000000000000		
2837	00000948	0000FFFFFFF0000000-	db	0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
2837	00000951	00000000000		
2838	00000956	000000000FF01000000-	db	0, 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
2838	0000095F	4E4F204E41		
2839	00000964	4D4520202020000000-	db	4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
2839	0000096D	00004641		
2840	00000971	54313220202000	db	54h, 31h, 32h, 20h, 20h, 20h, 0
2841	00000978	FFFF	bds_2:	dw 0FFFFh
2842	0000097A	000050030002010100-	db	0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0, 0F8h
2842	00000983	02100000000F8		
2843	00000989	01000000000000000000-	db	1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2843	00000992	00000000000000000000		
2844	0000099B	000000000000000000FF-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
2844	000009A4	FFFFFF0000		
2845	000009A9	00000000000000000000-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
2845	000009B2	00000000003200028		
2846	000009BA	00000000000000000000-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2846	000009C3	00000000000000000000		
2847	000009CC	00000000000000000000-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2847	000009D5	00000000000000000000		
2848	000009DE	0000FFFFFFF0000000-	db	0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2848	000009E7	00000000000		
2849	000009EC	000000000FF01000000-	db	0, 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
2849	000009F5	4E4F204E41		
2850	000009FA	4D4520202020000000-	db	4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
2850	00000A03	00004641		
2851	00000A07	54313220202000	db	54h, 31h, 32h, 20h, 20h, 20h, 0
2852	00000A0E	FFFF	bds_3:	dw 0FFFFh
2853	00000A10	000050030002010100-	db	0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0, 0F8h
2853	00000A19	02100000000F8		
2854	00000A1F	01000000000000000000-	db	1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2854	00000A28	00000000000000000000		
2855	00000A31	000000000000000000FF-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
2855	00000A3A	FFFFFF0000		
2856	00000A3F	00000000000000000000-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
2856	00000A48	00000000003200028		
2857	00000A50	00000000000000000000-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2857	00000A59	00000000000000000000		
2858	00000A62	00000000000000000000-	db	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2858	00000A6B	00000000000000000000		
2859	00000A74	0000FFFFFFF0000000-	db	0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2859	00000A7D	00000000000		
2860	00000A82	000000000FF01000000-	db	0, 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
2860	00000A8B	4E4F204E41		
2861	00000A90	4D4520202020000000-	db	4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
2861	00000A99	00004641		
2862	00000A9D	54313220202000	db	54h, 31h, 32h, 20h, 20h, 20h, 0
2863	00000AA4	FFFF	bds_4:	dw


```

2904 00000CDA 00000000FF01000000-
2904 00000CE3 4E4F204E41
2905 00000CE8 4D4520202020000000-
2905 00000CF1 00004641
2906 00000CF5 54313220202000
2907 00000CFC FFFF
2908 00000CFE 000050030002010100-
2908 00000D07 02100000000F8
2909 00000D0D 010000000000000000-
2909 00000D16 000000000000000000
2910 00000D1F 0000000000000000FF-
2910 00000D28 FFFFFFF0000
2911 00000D2D 000000000000000000-
2911 00000D36 0000000003200028
2912 00000D3E 000000000000000000-
2912 00000D47 000000000000000000
2913 00000D50 000000000000000000-
2913 00000D59 000000000000000000
2914 00000D62 0000FFFFFFFFF000000-
2914 00000D6B 00000000000
2915 00000D70 00000000FF01000000-
2915 00000D79 4E4F204E41
2916 00000D7E 4D4520202020000000-
2916 00000D87 00004641
2917 00000D8B 54313220202000
2918 00000D92 FFFF
2919 00000D94 000050030002010100-
2919 00000D9D 02100000000F8
2920 00000DA3 010000000000000000-
2920 00000DAC 000000000000000000
2921 00000DB5 0000000000000000FF-
2921 00000DBE FFFFFFF0000
2922 00000DC3 000000000000000000-
2922 00000DCC 0000000003200028
2923 00000DD4 000000000000000000-
2923 00000DDD 000000000000000000
2924 00000DE6 000000000000000000-
2924 00000DEF 000000000000000000
2925 00000DF8 0000FFFFFFFFF000000-
2925 00000E01 00000000000
2926 00000E06 00000000FF01000000-
2926 00000E0F 4E4F204E41
2927 00000E14 4D4520202020000000-
2927 00000E1D 00004641
2928 00000E21 54313220202000
2929 00000E28 FFFF
2930 00000E2A 000050030002010100-
2930 00000E33 02100000000F8
2931 00000E39 010000000000000000-
2931 00000E42 000000000000000000
2932 00000E4B 0000000000000000FF-
2932 00000E54 FFFFFFF0000
2933 00000E59 000000000000000000-
2933 00000E62 0000000003200028
2934 00000E6A 000000000000000000-
2934 00000E73 000000000000000000
2935 00000E7C 000000000000000000-
2935 00000E85 000000000000000000
2936 00000E8E 0000FFFFFFFFF000000-
2936 00000E97 00000000000
2937 00000E9C 00000000FF01000000-
2937 00000EA5 4E4F204E41
2938 00000EAA 4D4520202020000000-
2938 00000EB3 00004641
2939 00000EB7 54313220202000
2940 00000EBE FFFF
2941 00000EC0 000050030002010100-
2941 00000EC9 02100000000F8
2942 00000ECF 010000000000000000-
2942 00000ED8 000000000000000000
2943 00000EE1 0000000000000000FF-
2943 00000EEA FFFFFFF0000
2944 00000EEF 000000000000000000-
2944 00000EF8 0000000003200028
2945 00000F00 000000000000000000-
2945 00000F09 000000000000000000
2946 00000F12 000000000000000000-
2946 00000F1B 000000000000000000
2947 00000F24 0000FFFFFFFFF000000-
2947 00000F2D 00000000000
2948 00000F32 00000000FF01000000-
2948 00000F3B 4E4F204E41
2949 00000F40 4D4520202020000000-
2949 00000F49 00004641
2950 00000F4D 54313220202000
2951 00000F54 FFFF
2952 00000F56 000050030002010100-
2952 00000F5F 02100000000F8
2953 00000F65 010000000000000000-
2953 00000F6E 000000000000000000
2954 00000F77 0000000000000000FF-
2954 00000F80 FFFFFFF0000
2955 00000F85 000000000000000000-
2955 00000F8E 0000000003200028
2956 00000F96 000000000000000000-
2956 00000F9F 000000000000000000
2957 00000FA8 000000000000000000-
2957 00000FB1 000000000000000000
2958 00000FBA 0000FFFFFFFFF000000-
2958 00000FC3 00000000000
2959 00000FC8 00000000FF01000000-
2959 00000FD1 4E4F204E41
2960 00000FD6 4D4520202020000000-
2960 00000FDF 00004641
2961 00000FE3 54313220202000
2962 00000FEA FFFF
2963 00000FEC 000050030002010100-
2963 00000FF5 02100000000F8
2964 00000FFB 010000000000000000-
2964 00001004 000000000000000000
2965 0000100D 0000000000000000FF-
2965 00001016 FFFFFFF0000
2966 0000101B 000000000000000000-
2966 00001024 0000000003200028
2967 0000102C 000000000000000000-
2967 00001035 000000000000000000
2968 0000103E 000000000000000000-
2968 00001047 000000000000000000
2969 00001050 0000FFFFFFFFF000000-
2969 00001059 00000000000
2970 0000105E 00000000FF01000000-
2970 00001067 4E4F204E41
2971 0000106C 4D4520202020000000-
2971 00001075 00004641
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h

```

```

2972 00001079 54313220202000      db 54h, 31h, 32h, 20h, 20h, 20h, 0
2973 00001080 FFFF             dw 0FFFFh
2974 00001082 000050030002010100- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
2974 0000108B 02100000000F8
2975 00001091 010000000000000000- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2975 0000109A 000000000000000000
2976 000010A3 0000000000000000FF- db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
2976 000010AC FFFFFFF0000
2977 000010B1 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
2977 000010BA 0000000003200028
2978 000010C2 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2978 000010CB 000000000000000000
2979 000010D4 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2979 000010DD 000000000000000000
2980 000010E6 0000FFFFFFFFF000000- db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
2980 000010EF 00000000000
2981 000010F4 00000000FF01000000- db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
2981 000010FD 4E4F204E41
2982 00001102 4D4520202020000000- db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
2982 0000110B 00004641
2983 0000110F 54313220202000
2984 00001116 FFFF
2985 00001118 000050030002010100- db 54h, 31h, 32h, 20h, 20h, 20h, 0
2985 00001121 02100000000F8      dw 0FFFFh
2986 00001127 010000000000000000- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
2986 00001130 000000000000000000
2987 00001139 0000000000000000FF- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2987 00001142 FFFFFFF0000
2988 00001147 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
2988 00001150 0000000003200028
2989 00001158 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
2989 00001161 000000000000000000
2990 0000116A 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2990 00001173 000000000000000000
2991 0000117C 0000FFFFFFFFF000000- db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
2991 00001185 00000000000
2992 0000118A 00000000FF01000000- db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
2992 00001193 4E4F204E41
2993 00001198 4D4520202020000000- db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
2993 000011A1 00004641
2994 000011A5 54313220202000
2995 000011AC FFFF
2996 000011AE 000050030002010100- db 54h, 31h, 32h, 20h, 20h, 20h, 0
2996 000011B7 02100000000F8      dw 0FFFFh
2997 000011BD 010000000000000000- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
2997 000011C6 000000000000000000
2998 000011CF 0000000000000000FF- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2998 000011D8 FFFFFFF0000
2999 000011DD 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
2999 000011E6 0000000003200028
3000 000011EE 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
3000 000011F7 000000000000000000
3001 00001200 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3001 00001209 000000000000000000
3002 00001212 0000FFFFFFFFF000000- db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
3002 0000121B 00000000000
3003 00001220 00000000FF01000000- db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
3003 00001229 4E4F204E41
3004 0000122E 4D4520202020000000- db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
3004 00001237 00004641
3005 0000123B 54313220202000
3006 00001242 FFFF
3007 00001244 000050030002010100- db 54h, 31h, 32h, 20h, 20h, 20h, 0
3007 0000124D 02100000000F8      dw 0FFFFh
3008 00001253 010000000000000000- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3008 0000125C 000000000000000000
3009 00001265 0000000000000000FF- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3009 0000126E FFFFFFF0000
3010 00001273 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
3010 0000127C 0000000003200028
3011 00001284 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
3011 0000128D 000000000000000000
3012 00001296 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3012 0000129F 000000000000000000
3013 000012A8 0000FFFFFFFFF000000- db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
3013 000012B1 00000000000
3014 000012B6 00000000FF01000000- db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
3014 000012BF 4E4F204E41
3015 000012C4 4D4520202020000000- db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
3015 000012CD 00004641
3016 000012D1 54313220202000
3017 000012D8 FFFF
3018 000012DA 000050030002010100- db 54h, 31h, 32h, 20h, 20h, 20h, 0
3018 000012E3 02100000000F8      dw 0FFFFh
3019 000012E9 010000000000000000- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3019 000012F2 000000000000000000
3020 000012FB 0000000000000000FF- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3020 00001304 FFFFFFF0000
3021 00001309 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
3021 00001312 0000000003200028
3022 0000131A 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
3022 00001323 000000000000000000
3023 0000132C 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3023 00001335 000000000000000000
3024 0000133E 0000FFFFFFFFF000000- db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
3024 00001347 00000000000
3025 0000134C 00000000FF01000000- db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
3025 00001355 4E4F204E41
3026 0000135A 4D4520202020000000- db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
3026 00001363 00004641
3027 00001367 54313220202000
3028 0000136E FFFF
3029 00001370 000050030002010100- db 54h, 31h, 32h, 20h, 20h, 20h, 0
3029 00001379 02100000000F8      dw 0FFFFh
3030 0000137F 010000000000000000- db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3030 00001388 000000000000000000
3031 00001391 0000000000000000FF- db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3031 0000139A FFFFFFF0000
3032 0000139F 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
3032 000013A8 0000000003200028
3033 000013B0 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
3033 000013B9 000000000000000000
3034 000013C2 000000000000000000- db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3034 000013CB 000000000000000000
3035 000013D4 0000FFFFFFFFF000000- db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0, 0
3035 000013DD 00000000000
3036 000013E2 00000000FF01000000- db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
3036 000013EB 4E4F204E41
3037 000013F0 4D4520202020000000- db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 46h, 41h
3037 000013F9 00004641
3038 000013FD 54313220202000
3039 00001404 FFFF
3040 00001406 000050030002010100- db 54h, 31h, 32h, 20h, 20h, 20h, 0
3040 0000140F 02100000000F8      dw 0FFFFh

```

3041 00001415 01000000000000000000-
3041 0000141E 00000000000000000000
3042 00001427 0000000000000000FF-
3042 00001430 FFFFFFF0000
3043 00001435 00000000000000000000-
3043 0000143E 0000000003200028
3044 00001446 00000000000000000000-
3044 0000144F 00000000000000000000
3045 00001458 00000000000000000000-
3045 00001461 00000000000000000000
3046 0000146A 0000FFFFFFFFF0000000-
3046 00001473 00000000000
3047 00001478 00000000FF01000000-
3047 00001481 4E4F204E41
3048 00001486 4D4520202020000000-
3048 0000148F 00004641
3049 00001493 54313220202000
3050 0000149A FFFF
3051 0000149C 000050030002010100-
3051 000014A5 02100000000F8
3052 000014AB 01000000000000000000-
3052 000014B4 00000000000000000000
3053 000014BD 0000000000000000FF-
3053 000014C6 FFFFFFF0000
3054 000014CB 00000000000000000000-
3054 000014D4 0000000003200028
3055 000014DC 00000000000000000000-
3055 000014E5 00000000000000000000
3056 000014EE 00000000000000000000-
3056 000014F7 00000000000000000000
3057 00001500 0000FFFFFFFFF0000000-
3057 00001509 00000000000
3058 0000150E 00000000FF01000000-
3058 00001517 4E4F204E41
3059 0000151C 4D4520202020000000-
3059 00001525 00004641
3060 00001529 54313220202000
3061 00001530 FFFF
3062 00001532 000050030002010100-
3062 0000153B 02100000000F8
3063 00001541 01000000000000000000-
3063 0000154A 00000000000000000000
3064 00001553 0000000000000000FF-
3064 0000155C FFFFFFF0000
3065 00001561 00000000000000000000-
3065 0000156A 0000000003200028
3066 00001572 00000000000000000000-
3066 0000157B 00000000000000000000
3067 00001584 00000000000000000000-
3067 0000158D 00000000000000000000
3068 00001596 0000FFFFFFFFF0000000-
3068 0000159F 00000000000
3069 000015A4 00000000FF01000000-
3069 000015AD 4E4F204E41
3070 000015B2 4D4520202020000000-
3070 000015BB 00004641
3071 000015BF 54313220202000
3072 000015C6 FFFF
3073 000015C8 000050030002010100-
3073 000015D1 02100000000F8
3074 000015D7 01000000000000000000-
3074 000015E0 00000000000000000000
3075 000015E9 0000000000000000FF-
3075 000015F2 FFFFFFF0000
3076 000015F7 00000000000000000000-
3076 00001600 0000000003200028
3077 00001608 00000000000000000000-
3077 00001611 00000000000000000000
3078 0000161A 00000000000000000000-
3078 00001623 00000000000000000000
3079 0000162C 0000FFFFFFFFF0000000-
3079 00001635 00000000000
3080 0000163A 00000000FF01000000-
3080 00001643 4E4F204E41
3081 00001648 4D4520202020000000-
3081 00001651 00004641
3082 00001655 54313220202000
3083 0000165C FFFF
3084 0000165E 000050030002010100-
3084 00001667 02100000000F8
3085 0000166D 01000000000000000000-
3085 00001676 00000000000000000000
3086 0000167F 0000000000000000FF-
3086 00001688 FFFFFFF0000
3087 0000168D 00000000000000000000-
3087 00001696 0000000003200028
3088 0000169E 00000000000000000000-
3088 000016A7 00000000000000000000
3089 000016B0 00000000000000000000-
3089 000016B9 00000000000000000000
3090 000016C2 0000FFFFFFFFF0000000-
3090 000016CB 00000000000
3091 000016D0 00000000FF01000000-
3091 000016D9 4E4F204E41
3092 000016DE 4D4520202020000000-
3092 000016E7 00004641
3093 000016EB 54313220202000
3094
3095
3096
3097
3098
3099
3100
3101
3102
3103
3104
3105
3106
3107
3108
3109
3110
3111
3112
3113
3114
3115
3116
3117
3118
3119
3120

db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 20h, 0, 28h
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0FFh, 0FFh, 0FFh, 0FFh, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h
db 4Dh, 45h, 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h
db 54h, 31h, 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
db 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
db 0, 0FFh, 1, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
db 32h, 20h, 20h, 20h, 0
dw 0FFFFh
db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

bds_24:

%endif

; 09/12/2023
%if 0
; Retro DOS v4.2 (MSDOS 6.22) IO.SYS BDS structure

bdss:

3121	db	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0
3122	db	0, 0FFh, 1, 0, 0, 0,	4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh,	45h
3123	db	20h, 20h, 20h, 20h, 0, 0, 0,	0, 0, 46h, 41h,	54h, 31h
3124	db	32h, 20h, 20h, 20h, 0		
3125	dw	0FFFFh		
3126	db	0, 0, 50h, 3, 0, 2, 1, 1, 0,	2, 10h, 0, 0, 0, 0F8h	
3127	db	1, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 3
3128	db	20h, 0, 28h, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0	
3129	db	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0
3130	db	0, 0FFh, 1, 0, 0, 0,	4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh,	45h
3131	db	20h, 20h, 20h, 20h, 0, 0, 0,	0, 0, 46h, 41h,	54h, 31h
3132	db	32h, 20h, 20h, 20h, 0		
3133	dw	0FFFFh		
3134	db	0, 0, 50h, 3, 0, 2, 1, 1, 0,	2, 10h, 0, 0, 0, 0F8h	
3135	db	1, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 3
3136	db	20h, 0, 28h, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0	
3137	db	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0
3138	db	0, 0FFh, 1, 0, 0, 0,	4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh,	45h
3139	db	20h, 20h, 20h, 20h, 0, 0, 0,	0, 0, 46h, 41h,	54h, 31h
3140	db	32h, 20h, 20h, 20h, 0		
3141	dw	0FFFFh		
3142	db	0, 0, 50h, 3, 0, 2, 1, 1, 0,	2, 10h, 0, 0, 0, 0F8h	
3143	db	1, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 3
3144	db	20h, 0, 28h, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0	
3145	db	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0
3146	db	0, 0FFh, 1, 0, 0, 0,	4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh,	45h
3147	db	20h, 20h, 20h, 20h, 0, 0, 0,	0, 0, 46h, 41h,	54h, 31h
3148	db	32h, 20h, 20h, 20h, 0		
3149	dw	0FFFFh		
3150	db	0, 0, 50h, 3, 0, 2, 1, 1, 0,	2, 10h, 0, 0, 0, 0F8h	
3151	db	1, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 3
3152	db	20h, 0, 28h, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0	
3153	db	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0
3154	db	0, 0FFh, 1, 0, 0, 0,	4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh,	45h
3155	db	20h, 20h, 20h, 20h, 0, 0, 0,	0, 0, 46h, 41h,	54h, 31h
3156	db	32h, 20h, 20h, 20h, 0		
3157	dw	0FFFFh		
3158	db	0, 0, 50h, 3, 0, 2, 1, 1, 0,	2, 10h, 0, 0, 0, 0F8h	
3159	db	1, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 3
3160	db	20h, 0, 28h, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0	
3161	db	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0
3162	db	0, 0FFh, 1, 0, 0, 0,	4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh,	45h
3163	db	20h, 20h, 20h, 20h, 0, 0, 0,	0, 0, 46h, 41h,	54h, 31h
3164	db	32h, 20h, 20h, 20h, 0		
3165	dw	0FFFFh		
3166	db	0, 0, 50h, 3, 0, 2, 1, 1, 0,	2, 10h, 0, 0, 0, 0F8h	
3167	db	1, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 3
3168	db	20h, 0, 28h, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0	
3169	db	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0
3170	db	0, 0FFh, 1, 0, 0, 0,	4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh,	45h
3171	db	20h, 20h, 20h, 20h, 0, 0, 0,	0, 0, 46h, 41h,	54h, 31h
3172	db	32h, 20h, 20h, 20h, 0		
3173	dw	0FFFFh		
3174	db	0, 0, 50h, 3, 0, 2, 1, 1, 0,	2, 10h, 0, 0, 0, 0F8h	
3175	db	1, 0, 0, 0, 0, 0, 0, 0,	0, 0, 0, 0, 0, 0, 0, 0,	0, 0, 3
3176	db	20h, 0, 28h, 0, 0, 0, 0, 0, 0,	0,	

```

3245 dw 0FFFFh
3246 db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3247 db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
3248 db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3249 db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3250 db 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
3251 db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
3252 db 32h, 20h, 20h, 20h, 0
3253 dw 0FFFFh
3254 db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3255 db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
3256 db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3257 db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3258 db 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
3259 db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
3260 db 32h, 20h, 20h, 20h, 0
3261 dw 0FFFFh
3262 db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3263 db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
3264 db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3265 db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3266 db 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
3267 db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
3268 db 32h, 20h, 20h, 20h, 0
3269 dw 0FFFFh
3270 db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3271 db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
3272 db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3273 db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3274 db 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
3275 db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
3276 db 32h, 20h, 20h, 20h, 0
3277 dw 0FFFFh
3278 db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3279 db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
3280 db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3281 db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3282 db 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
3283 db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
3284 db 32h, 20h, 20h, 20h, 0
3285 dw 0FFFFh
3286 db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3287 db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
3288 db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3289 db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3290 db 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
3291 db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
3292 db 32h, 20h, 20h, 20h, 0
3293 dw 0FFFFh
3294 db 0, 0, 50h, 3, 0, 2, 1, 1, 0, 2, 10h, 0, 0, 0, 0F8h
3295 db 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3
3296 db 20h, 0, 28h, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3297 db 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3298 db 0, 0FFh, 1, 0, 0, 0, 0, 4Eh, 4Fh, 20h, 4Eh, 41h, 4Dh, 45h
3299 db 20h, 20h, 20h, 20h, 0, 0, 0, 0, 0, 0, 0, 46h, 41h, 54h, 31h
3300 db 32h, 20h, 20h, 20h, 0
3301 db 0
3302 %endif
3303
3304 ;-----
3305 ; Possibly disposable data, goes at end of data group
3306 ;*****
3307
3308 ; Possibly disposable data, goes at end of data group
3309
3310 ;***ibm_disk_io - main routine, fixes at rom bug
3311 ;
3312 ; entry: (ah) = function, 02 or 0a for read.
3313 ;        (dl) = drive number (80h or 81h).
3314 ;        (dh) = head number.
3315 ;        (ch) = cylinder number.
3316 ;        (cl) = sector number (high 2 bits has cylinder number).
3317 ;        (al) = number of sectors.
3318 ;        (es:bx) = address of read buffer.
3319 ;        for more on register contents see rom bios listing.
3320 ;        stack set up for return by an iret.
3321 ;
3322 ; exit:  (ah) = status of current operation.
3323 ;        (cy) = 1 if failed, 0 if successful.
3324 ;        for other register contents see rom bios listing.
3325 ;
3326 ; uses:
3327 ;
3328 ;
3329 ; warning: uses old13 vector for non-read calls.
3330 ;         does direct calls to the at rom.
3331 ;         does segment arithmetic.
3332 ;
3333 ; effects: performs disk i/o operation.
3334 ;
3335 ; 16/10/2022
3336 ; 28/05/2019
3337 cmd_block equ 42h ; ROMBIOS DATA segment (40h) offset 42h ; 13/12/2022
3338
3339 ;* offsets into cmd_block for registers
3340
3341 pre_comp equ 0 ;write pre-compensation
3342 sec_cnt equ 1 ;sector count
3343 sec_num equ 2 ;sector number
3344 cyl_low equ 3 ;cylinder number, low part
3345 cyl_high equ 4 ;cylinder number, high part
3346 drv_head equ 5 ;drive/head (bit 7 = ecc mode, bit 5 = 512 byte sectors,
3347 ; bit 4 = drive number, bits 3-0 have head number)
3348 cmd_reg equ 6 ;command register
3349
3350 ; 01/10/2022
3351 disk_status1 equ 74h
3352 hf_num equ 75h
3353 control_byte equ 76h
3354
3355 ibm_disk_io:
3356 cmp dl, 80h ; main routine, fixes at rom bug
3357 jb short atd1 ; pass through floppy disk calls.
3358 cmp ah, 2
3359 jz short atd2 ; intercept call 02 (read sectors).
3360 cmp ah, 0Ah
3361 jz short atd2 ; and call 0Ah (read long).
3362
3363 atd1: jmp far [cs:old13]
3364 ;jmp cs:old13 ; use rom int 13h handler
3365 ;-----
3366
3367 atd2: push bx
3368 00001706 53

```

```

3369 00001707 51          push    cx
3370 00001708 52          push    dx
3371 00001709 57          push    di
3372 0000170A 1E          push    ds
3373 0000170B 06          push    es
3374 0000170C 50          push    ax
3375 0000170D B84000      mov     ax, 40h          ; bioseg (rombios data segment)
3376                                ; establish bios segment addressing
3377 00001710 8ED8      mov     ds, ax
3378                                ; 16/10/2022
3379 00001712 C606740000  mov     byte [disk_status1], 0
3380                                ;mov     byte ptr ds:74h, 0 ; [disk_status1]
3381                                ; initially no error code.
3382 00001717 80E27F      and     dl, 7Fh          ; mask to hard disk number
3383 0000171A 3A167500      cmp     dl, [hf_num]
3384                                ;cmp     dl, ds:75h          ; [hf_num] ; 40h:75h
3385 0000171E 7207      jb      short atd3      ; disk number in range
3386                                ;mov     byte ptr ds:74h, 1 ; bad_disk
3387 00001720 C606740001  mov     byte [disk_status1], 1
3388 00001725 EB20      jmp     short atd4      ; disk number out of range error,
3389                                ; return
3390                                ; -----
3391
3392
3393 00001727 53          push    bx
3394 00001728 8CC0      mov     ax, es
3395 0000172A C1EB04      shr     bx, 4          ; make es:bx to seg:000x form.
3396 0000172D 01D8      add     ax, bx
3397 0000172F 8EC0      mov     es, ax
3398 00001731 5B          pop     bx
3399 00001732 83E30F      and     bx, 0Fh
3400 00001735 0E          push    cs
3401 00001736 E8DF00      call    check_dma
3402 00001739 720C      jb      short atd4      ; abort if dma across segment boundary
3403 0000173B 5B          pop     ax
3404 0000173C 50          push    ax
3405 0000173D E81A00      call    setcmd          ; set up command block for disk op
3406 00001740 BAF603      mov     dx, 3F6h        ; hf_reg_port
3407 00001743 EE          out     dx, al          ; write out command modifier
3408 00001744 E86B00      call    docmd           ; carry out command
3409                                ; -----
3410
3411
3412
3413
3414
3415
3416 00001747 58          pop     ax
3417                                ;mov     ah, ds:74h          ; [disk_status1]
3418 00001748 8A267400  mov     ah, [disk_status1]
3419 0000174C 08E4      or      ah, ah
3420 0000174E 7401      jz      short atd5
3421 00001750 F9          stc
3422
3423 00001751 07          pop     es
3424 00001752 1F          pop     ds
3425 00001753 5F          pop     di
3426 00001754 5A          pop     dx
3427 00001755 59          pop     cx
3428 00001756 5B          pop     bx
3429 00001757 CA0200      retf     2              ; far return, dropping flags
3430
3431                                ; ===== S U B   R O U T I N E =====
3432
3433
3434
3435
3436
3437
3438
3439
3440
3441
3442
3443
3444
3445
3446
3447
3448
3449
3450
3451
3452 0000175A A24300      ;mov     ds:43h, al          ; [cmd_block+sec_cnt]
3453                                ; 16/10/2022
3454 0000175D C606480020  mov     [cmd_block+sec_cnt], al
3455 00001762 80FC02      mov     byte ptr ds:48h, 20h ; [cmd_block+cmd_reg]
3456 00001765 7405      cmp     ah, 2
3457 00001767 C606480022  jz      short setc1      ; cmd_reg = 20h          if function 02h          (read)
3458                                mov     byte [cmd_block+cmd_reg], 22h
3459                                ;mov     byte ptr ds:48h, 22h ; [cmd_block+cmd_reg]
3460                                ; cmd_reg = 22h          if function 0Ah          (read long)
3461
3462 0000176C 88C8      mov     al, cl
3463 0000176E 243F      and     al, 3Fh          ; mask sector number
3464                                ;mov     ds:44h, al          ; [cmd_block+sec_num]
3465                                ;mov     ds:45h, ch          ; [cmd_block+cyl_low]
3466 00001770 A24400      mov     [cmd_block+sec_num], al ; mov [44h],al
3467 00001773 882E4500  mov     [cmd_block+cyl_low], ch ; mov [45h],ch
3468 00001777 88C8      mov     al, cl
3469 00001779 C0E806      shr     al, 6            ; get two high bits of cylinder          number
3470                                ;mov     ds:46h, al          ; [cmd_block+cyl_high]
3471 0000177F 89D0      mov     [cmd_block+cyl_high], al ; mov [46h],al
3472 00001781 C0E004      mov     ax, dx
3473 00001784 80E40F      shl     al, 4            ; drive number
3474 00001787 08E0      and     ah, 0Fh
3475 00001789 0CA0      or      al, ah          ; head number
3476                                or      al, 0A0h          ; set ecc and 512 bytes          per sector
3477 0000178B A24700      mov     ds:47h, al          ; [cmd_block+drv_head]
3478 0000178E 06          mov     [cmd_block+drv_head], al ; mov [47h],al
3479 0000178F 53          push    es
3480 00001790 0E          push    bx
3481 00001791 E85C00      push    cs
3482 00001794 268B4705  call    get_vec
3483                                mov     ax, [es:bx+5] ; [es:bx+fdp_precomp]
3484                                ; write pre-comp from disk parameters
3485 00001798 C1E802      shr     ax, 2
3486 0000179B A24200      mov     ds:42h, al          ; [cmd_block+pre_comp]
3487                                [cmd_block+pre_comp], al ; mov [42h],al
3488 0000179E 268A4708  mov     al, [es:bx+8] ; only use low part
3489                                ; [es:bx+fdp_control]
3490                                ; control byte modifier
3491 000017A2 5B          pop     bx
3492 000017A3 07          pop     es
3493                                mov     ah, ds:76h          ; [control_byte]

```



```

3493 000017A4 8A267600      mov     ah, [control_byte] ; mov ah,[76h]
3494 000017A8 80E4C0      and     ah, 0C0h          ; keep disable retry bits
3495 000017AB 08C4      or      ah, al
3496      ;mov     ds:76h, ah
3497 000017AD 88267600      mov     [control_byte], ah ; mov [76h],al
3498 000017B1 C3      retn
3499
3500      ; ===== S U B   R O U T I N E =====
3501
3502      ;***docmd - carry out read operation to at hard disk
3503      ;
3504      ; entry: (es:bx) = address for read in data.
3505      ;        cmd_block set up for disk read.
3506      ;
3507      ; exit:  buffer at (es:bx) contains data read.
3508      ;        disk_status1 set to error code (0 if success).
3509      ;
3510      ;
3511      ;
3512      ; warning: (ax), (bl), (cx), (dx), (di) destroyed.
3513      ;          no check is made for dma boundary overrun.
3514      ;
3515      ; effects: programs disk controller.
3516      ;          performs disk input.
3517
3518 docmd:      ; proc near
3519      mov     di, bx
3520      push    cs
3521      call    command
3522      jnz     short doc3
3523
3524 doc1:      push    cs
3525      call    waitt          ; wait for controller to complete read
3526      jnz     short doc3
3527      mov     cx, 256        ; 256 words per sector
3528      mov     dx, 1F0h       ; hf_port
3529      cld                ; string op goes up
3530      cli                ; disable interrupts
3531      ; (bug was forgetting this)
3532
3533      ; M062 -- some of these old machines have intermittent failures
3534      ;          when the read is done at full speed. Instead of using
3535      ;          a string rep instruction, we'll use a loop. There is
3536      ;          a slight performance hit, but it only affects these
3537      ;          very old machines with an exact date code match, and
3538      ;          it makes said machines more reliable
3539      ;
3540      ;M062      repz     insw          ;read in sector
3541
3542 rsct_loop: insw
3543      loop    rsct_loop
3544      sti
3545      ; 16/10/2022
3546      test    byte [cmd_block+cmd_reg], 02h
3547      ;test    byte ptr ds:48h, 2 ; [cmd_block+cmd_reg]
3548      ;          ; (ds = 40h)
3549      jz      short doc2      ; no ecc bytes to read.
3550      push    cs
3551      call    wait_drq        ; wait for controller to complete read
3552      jnb     short doc3
3553      mov     cx, 4           ; 4 bytes of ecc
3554      mov     dx, 1F0h       ; hf_port
3555      cli
3556      rep     insb           ; read in ecc
3557      sti
3558
3559 doc2:      push    cs
3560      call    check_status
3561      jnz     short doc3      ; operation failed
3562      ;dec     byte ptr ds:43h ; [cmd_block+sec_cnt]
3563      dec     byte [cmd_block+sec_cnt]
3564      jnz     short doc1      ; loop while more sectors to read
3565
3566 doc3:      retn
3567
3568      ; ===== S U B   R O U T I N E =====
3569
3570      ;***define where the rom routines are actually located
3571      ;
3572      ; in the buggy old AT BIOS that we might need to
3573      ; install a special level of int13 handler for
3574      ;
3575      ; 16/10/2022
3576
3577 romsegment equ 0F000h ; segment
3578 romcommand equ 2E1Eh ; offset in romsegment
3579 romwait     equ 2E7Fh ; offset in romsegment
3580 romwait_drq equ 2EE2h ; offset in romsegment
3581 romcheck_status equ 2EF8h ; offset in romsegment
3582 romcheck_dma  equ 2F69h ; offset in romsegment
3583 romget_vec    equ 2F8Eh ; offset in romsegment
3584 romfret       equ 0FF65h ; far return in rom
3585
3586 ;***get_vec - get pointer to hard disk parameters.
3587 ;
3588 ; entry: (dl) = low bit has hard disk number (0 or 1).
3589 ;
3590 ; exit:  (es:bx) = address of disk parameters table.
3591 ;
3592 ; uses:  ax for segment computation.
3593 ;
3594 ; loads es:bx from interrupt table in low memory, vector 46h (disk 0)
3595 ; or 70h (disk 1).
3596 ;
3597 ; warning: (ax) destroyed.
3598 ;          this does a direct call to the at rom.
3599
3600 get_vec:    ; proc near
3601      ;push    0FF65h          ; romfret ; far          return in rom
3602      ;jmp     far ptr 0F000h:2F8Eh
3603      ; 16/10/2022
3604      push    romfret          ; far return in rom
3605      jmp     romsegment:romget_vec
3606
3607      ; ===== S U B   R O U T I N E =====
3608
3609      ;***command - send contents of cmd_block to disk controller.
3610      ;
3611      ; entry: control_byte
3612      ;        cmd_block - set up with values for hard disk controller.
3613      ;
3614      ; exit:  disk_status1 = error code.
3615      ;        nz if error, zr for no error.
3616      ;

```

```

3617 ;
3618 ; warning: (ax), (cx), (dx) destroyed.
3619 ; does a direct call to the at rom.
3620 ;
3621 ; effects: programs disk controller.
3622 ;
3623 command: ; proc near
3624 ; push 0FF65h ; romfret ; far return in rom
3625 ; jmp far ptr0F000h:2E1Eh
3626 ; 16/10/2022
3627 000017F8 6865FF push romfret ; far return in rom
3628 000017FB EA1E2E00F0 jmp romsegment:romcommand
3629 ;
3630 ; ===== S U B R O U T I N E =====
3631 ;
3632 ; ***waitt - wait for disk interrupt
3633 ;
3634 ; entry: nothing.
3635 ;
3636 ; exit: disk_status1 = error code.
3637 ; nz if error, zr if no error.
3638 ;
3639 ;
3640 ; warning: (ax), (bx), (cx) destroyed.
3641 ; does a direct call to the at rom.
3642 ;
3643 ; effects: calls int 15h, function 9000h.
3644 ;
3645 waitt: ; proc near
3646 ; push 0FF65h ; romfret ; far return in rom
3647 ; jmp far ptr0F000h:2E7Fh
3648 ; 16/10/2022
3649 00001800 6865FF push romfret ; far return in rom
3650 00001803 EA7F2E00F0 jmp romsegment:romwait
3651 ;
3652 ; ===== S U B R O U T I N E =====
3653 ;
3654 ; ***wait_drq - wait for data request.
3655 ;
3656 ; entry: nothing.
3657 ;
3658 ; exit: disk_status1 = error code.
3659 ; cy if error, nc if no error.
3660 ;
3661 ; warning: (ax), (cx), (dx) destroyed.
3662 ; does a direct call to the at rom.
3663 ;
3664 wait_drq: ; proc near
3665 ; push 0FF65h ; romfret ; far return in rom
3666 ; jmp far ptr0F000h:2EE2h
3667 ; 16/10/2022
3668 00001808 6865FF push romfret ; far return in rom
3669 0000180B EAE22E00F0 jmp romsegment:romwait_drq
3670 ;
3671 ; ===== S U B R O U T I N E =====
3672 ;
3673 ; ***check_status - check hard disk status.
3674 ;
3675 ; entry: nothing.
3676 ;
3677 ; exit: disk_status1 = error code.
3678 ; nz if error, zr if no error.
3679 ;
3680 ; warning: (ax), (cx), (dx) destroyed.
3681 ; does a direct call to the at rom.
3682 ;
3683 check_status: ; proc near
3684 ; push 0FF65h ; romfret ; far return in rom
3685 ; jmp far ptr0F000h:2EF8h
3686 ; 16/10/2022
3687 00001810 6865FF push romfret ; far return in rom
3688 00001813 EAF82E00F0 jmp romsegment:romcheck_status
3689 ;
3690 ; ===== S U B R O U T I N E =====
3691 ;
3692 ; ***check_dma - check for dma overrun 64k segment.
3693 ;
3694 ; entry: (es:bx) = addr. of memory buffer in seg:000x form.
3695 ; cmd_block set up for operation.
3696 ;
3697 ; exit: disk_status1 - error code.
3698 ; cy if error, nc if no error.
3699 ;
3700 ; warning: does a direct call to the at rom.
3701 ;
3702 check_dma: ; proc near
3703 ; push 0FF65h ; romfret ; far return in rom
3704 ; jmp far ptr0F000h:2F69h
3705 ; 16/10/2022
3706 00001818 6865FF push romfret ; far return in rom
3707 0000181B EA692F00F0 jmp romsegment:romcheck_dma
3708 ;
3709 ; -----
3710 ;
3711 ; endatrom:
3712 ;
3713 ; -----
3714 ;
3715 ; M015 -- begin changes
3716 ;
3717 ; Certain old COMPAQ '286 machines have a bug in their ROM BIOS.
3718 ; When Int13 is done with AH > 15h and DL >= 80h, they trash
3719 ; the byte at DS:74h, assuming that DS points to ROM_DATA.
3720 ; If our init code detects this error, it will install this
3721 ; special Int13 hook through the same mechanism that was set
3722 ; up for the IBM patch above. This code is also dynamically
3723 ; relocated by MSINIT.
3724 ;
3725 compaq_disk_io:
3726 00001820 80FC15 cmp ah, 15h ; compaq_disk_io proc far
3727 ;
3728 ; the following label defines the end of the at rom patch.
3729 ; this is used at configuration time.
3730 ;
3731 ; warning!!!
3732 ; this code will be dynamically relocated by msinit
3733 00001823 7705 ja short mebbe_hookit ; only deal with functions > 15h
3734 no_hookit:
3735 ; jmp cs:01d13
3736 ; 16/10/2022
3737 00001825 2EFF2E[0601] jmp far [cs:01d13]
3738 ;
3739 ; -----
3740 ;

```



```

3741 mebbe_hookit:
3742     cmp     dl, 80h
3743     jb      short no_hookit
3744     push    ds
3745
3746     ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3747     ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:1830h
3748     ; push    ax
3749     ; mov     ax, 40h
3750     ; mov     ds, ax
3751     ; pop     ax
3752     push    40h
3753     pop     ds
3754
3755     pushf
3756     ; call    cs:old13
3757     ; 16/10/2022
3758     call    far [cs:old13]
3759     pop     ds
3760     retf    2
3761
3762 ; -----
3763
3764 end_compaq_i13hook: db 0
3765
3766 ; ===== S U B   R O U T I N E =====
3767
3768 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3769 %if 0
3770
3771 ; CMOS clock setting support routines used by MSCLOCK.
3772 ; Warning!!! This code will be dynamically relocated by MSINIT.
3773
3774 daycnt_to_day:    ; proc far
3775
3776 ; entry: [daycnt] = number of days since 1-1-80
3777 ;
3778 ; return: ch - century in bcd
3779 ;         cl - year in bcd
3780 ;         dh - month in bcd
3781 ;         dl - day in bcd
3782
3783     ; 16/10/2022
3784     push    word [cs:daycnt] ; save daycnt
3785     cmp     word [cs:daycnt], 7305 ; (365*20+(20/4))
3786             ; # days from 1-1-1980 to 1-1-2000
3787     jnb     short century20
3788     mov     byte [cs:base_century], 19
3789     mov     byte [cs:base_year], 80
3790     jmp     short years
3791
3792 ; -----
3793
3794 century20:
3795     mov     byte [cs:base_century], 20
3796     mov     byte [cs:base_year], 0
3797     sub     word [cs:daycnt], 7305 ; (365*20+(20/4))
3798             ; adjust daycnt
3799
3800 years:
3801     xor     dx, dx
3802     mov     ax, [cs:daycnt]
3803     mov     bx, 1461 ; (366+365*3)
3804             ; # of days in a Leap year block
3805             ; AX = # of leap block, DX = daycnt
3806     div     bx
3807     mov     [cs:daycnt], dx ; save daycnt left
3808     mov     bl, 4
3809     mul     bl ; AX = # of years. Less than 100
3810     add     [cs:base_year], al ; So, ah = 0. Adjust year
3811     inc     word [cs:daycnt] ; set daycnt to 1 base
3812     cmp     word [cs:daycnt], 366 ; daycnt=remainder of leap year bk
3813     jbe     short leapyear ; within 366+355+355+355 days.
3814     inc     byte [cs:base_year] ; if daycnt <= 366, then leap year
3815     sub     word [cs:daycnt], 366 ; else daycnt--, base_year++ ;
3816     mov     cx, 3 ; And next three years are normal
3817
3818 regularyear:
3819     cmp     word [cs:daycnt], 365 ; for(i=1; i>3 or daycnt <=365; i++)
3820     jbe     short yeardone ; {if (daycnt > 365)
3821     inc     byte [cs:base_year] ; { daycnt -= 365
3822     sub     word [cs:daycnt], 365 ; }
3823     loop    regularyear ; }
3824             ; should never fall through loop
3825
3826 leapyear:
3827     mov     byte [cs:month_tab+1], 29 ; leap year.
3828             ; change month table.
3829
3830 yeardone:
3831     xor     bx, bx
3832     xor     dx, dx
3833     mov     ax, [cs:daycnt]
3834     ; mov     si, offset month_tab
3835     mov     si, month_tab ; 19/10/2022
3836     mov     cx, 12
3837
3838 months:
3839     inc     bl
3840
3841     ; !!! -- 16/10/2022 -- (if DS=CS, what for CS: prefixes are used !?)
3842     ; mov     dl, [cs:si]
3843     ; !!! -- 16/10/2022 -- (may be to keep code addrs as unchanged/fix!?)
3844     ; ds = cs !? (ofcourse ds must be same with cs here)
3845     ; mov     dl, [si] ; 20/03/2019 (MSDOS 6.21 IO.SYS, BIOSDATA:14C0h)
3846     ; mov     dl, [si] ; 16/10/2022 (MSDOS 5.0 IO.SYS, BIOSDATA:14C0h)
3847
3848     mov     dl, [si] ; ? ; mov dl, [cs:si]
3849     cmp     ax, dx ; cmp daycnt for each month till fit
3850             ; dh=0
3851     jbe     short month_done
3852     inc     si ; next month
3853     sub     ax, dx ; adjust daycnt
3854     loop    months ; should never fall through loop
3855
3856 month_done:
3857     mov     byte [cs:month_tab+1], 28
3858             ; restore month table value
3859     mov     dl, bl
3860     mov     dh, [cs:base_year]
3861     mov     cl, [cs:base_century] ; al=day,dl=month,dh=year,cl=cntry
3862     call    far [cs:bintobcd]
3863     ; call    cs:bintobcd ; convert "day" to bcd
3864             ; dl = bcd day, al = month
3865
3866     xchg    dl, al
3867     call    far [cs:bintobcd]
3868     ; call    cs:bintobcd ; dh = bcd month, al = year
3869     xchg    dh, al
3870     call    far [cs:bintobcd]
3871     ; call    cs:bintobcd ; cl = bcd year, al = century

```

```

3865             xchg     cl, al
3866             call    far [cs:bintobcd]
3867             ;call    cs:bintobcd      ; ch = bcd century
3868             mov     ch, al
3869             pop     word [cs:daycnt] ; restore original value
3870             retf
3871
3872     enddaycnttoday:
3873
3874 %endif
3875
3876 ; ===== S U B   R O U T I N E =====
3877
3878 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3879 %if 0
3880
3881 bin_to_bcd: ; proc far                ; real time clock support
3882
3883 ;convert a binary input in al (less than 63h or 99 decimal)
3884 ;into a bcd value in al. ah destroyed.
3885
3886             push     cx
3887             aam                     ; al=high digit      bcd, ah=low digit bcd
3888             mov     cl, 4
3889             shl     ah, cl          ; mov the high digit to      high nibble
3890             or      al, ah
3891             pop     cx
3892             retf
3893
3894 %endif
3895
3896 ; -----
3897
3898 ; the k09 requires the routines for reading the clock because of the suspend/
3899 ; resume facility. the system clock needs to be reset after resume.
3900
3901 ; the following routine is executed at resume time when the system
3902 ; powered on after suspension. it reads the real time clock and
3903 ; resets the system time and date, and then irets.
3904
3905 ; warning!!! this code will be dynamically relocated by msinit.
3906
3907             ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3908             ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:183Eh
3909 int_6Ch:
3910             push     cs
3911             pop     ds
3912             ;cmp     byte [cs:inHMA], 0
3913             cmp     byte [inHMA], 0
3914             jz      short int6c
3915             mov     bx, EnsureA20on
3916             call    bx
3917
3918 int6c:
3919             ;push     cs
3920             ;pop     ds
3921             pop     word [int6c_ret_addr] ; pop off return address
3922             pop     word [int6c_ret_addr+2]
3923             popf
3924             call    read_real_date ; get the date from the clock
3925             cli
3926             [daycnt], si ; update dos copy of date
3927             sti
3928             call    read_real_time ; get the time from the      rtc
3929             cli
3930             mov     ah, 1
3931             int     1Ah          ; CLOCK - SET TIME OF DAY
3932                                     ; CX:DX = clock count
3933                                     ; Return: time of day set
3934             sti
3935             jmp     int6c_ret_addr ; long jump
3936             ; 16/10/2022
3937             jmp     far [int6c_ret_addr] ; long jump
3938
3939 ; ===== S U B   R O U T I N E =====
3940
3941 ; read_real_date reads real-time clock for date and returns the number
3942 ; of days elapsed since 1-1-80 in si
3943
3944 read_real_date: ; proc near
3945             push     ax
3946             push     cx
3947             push     dx
3948             xor     ah, ah          ; throwaway clock roll      over
3949             int     1Ah          ; CLOCK - GET TIME OF DAY
3950                                     ; Return: CX:DX = clock count
3951                                     ; AL = 00h if clock was      read or written (via AH=0,1) since the previous
3952                                     ; midnight
3953                                     ; Otherwise, AL      > 0
3954             pop     dx
3955             pop     cx
3956             pop     ax
3957             push     ax
3958             push     bx
3959             push     cx
3960             push     dx
3961             mov     word [cs:daycnt2], 1
3962             ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
3963             ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:187Ah
3964             mov     word [daycnt2], 1
3965                                     ; REAL TIME CLOCK ERROR      FLAG (+1 DAY)
3966             mov     ah, 4
3967             int     1Ah          ; CLOCK - READ DATE FROM REAL TIME CLOCK (AT,XT286,CONV,PS)
3968                                     ; Return: DL = day in BCD
3969                                     ; DH = month in      BCD
3970                                     ; CL = year in BCD
3971                                     ; CH = century (19h or 20h)
3972             jnb     short read_ok
3973             jmp     r_d_ret
3974
3975 ;-----
3976
3977 read_ok:
3978             mov     [bin_date_time], ch
3979             mov     [bin_date_time+1], cl
3980             mov     [bin_date_time+2], dh
3981             mov     [bin_date_time+3], dl
3982             mov     word [cs:daycnt2], 2 ; READ OF R-T CLOCK SUCCESSFUL
3983             ; 08/08/2023
3984             mov     byte [daycnt2], 2
3985             inc     byte [daycnt2] ; 2
3986             call    bcd_verify ; verify bcd values in range
3987             jb      short r_d_ret ; some value out of range
3988             mov     word [cs:daycnt2], 3
3989             ; 08/08/2023
3990             mov     byte [daycnt2], 3
3991             inc     byte [daycnt2] ; 3

```

```

3989 000018A5 E8DD00      call    date_verify
3990 000018A8 7263        jb     short r_d_ret
3991                                ;mov    word [cs:daycnt2], 0
3992                                ; 08/08/2023
3993 000018AA C606[0006]00  mov     byte [daycnt2], 0
3994 000018AF E8A300      call    in_bin
3995 000018B2 A0[FD05]      mov     al, [bin_date_time+1]
3996 000018B5 98          cbw
3997 000018B6 803E[FC05]14  cmp     byte [bin_date_time], 20 ; 20th century?
3998 000018B8 7503        jnz     short century_19 ; no
3999 000018BD 83C064      add     ax, 100 ; add in a century
4000 century_19:
4001 000018C0 83E850      sub     ax, 80 ; subtract off 1-1-80
4002 000018C3 B104        mov     cl, 4 ; leap year every 4
4003 000018C5 F6F1        div     cl ; al= #leap year blocks, ah= remainder
4004 000018C7 88E3        mov     bl, ah ; save odd years
4005 000018C9 98          cbw ; zero ah
4006 000018CA B9B505      mov     cx, 1461 ; 366+(3*365)
4007                                ; # of days in leap year blocks
4008 000018CD F7E1        mul     cx
4009                                ;mov    [cs:daycnt2], ax ; SAVE COUNT OF DAYS
4010                                ; 08/08/2023
4011 000018CF A3[0006]      mov     [daycnt2], ax
4012 000018D2 88D8        mov     al, bl ; get odd years count
4013 000018D4 98          cbw
4014 000018D5 09C0        or      ax, ax
4015 000018D7 740B        jz      short leap_year
4016 000018D9 B96D01      mov     cx, 365 ; days in year
4017 000018DC F7E1        mul     cx
4018                                ;add    [cs:daycnt2], ax ; ADD ON DAYS IN ODD YEARS
4019                                ; 08/08/2023
4020 000018DE 0106[0006]  add     [daycnt2], ax
4021 000018E2 EB07        jmp     short leap_adjustment ; account for leap year
4022                                ; possibly account for a leap day
4023 ;-----
4024
4025 leap_year:
4026 000018E4 803E[FE05]02  cmp     byte [bin_date_time+2], 2 ; is month february?
4027 000018E9 7604        jbe     short no_leap_adjustment ; jan or feb. no leap day yet.
4028 leap_adjustment:
4029                                ;inc    word [cs:daycnt2] ; account for leap day
4030                                ; 08/08/2023
4031 000018EB FF06[0006]  inc     word [daycnt2]
4032 no_leap_adjustment:
4033 000018EF 8A0E[FF05]      mov     cl, [bin_date_time+3] ; get days of month
4034 000018F3 30ED        xor     ch, ch
4035 000018F5 49          dec     cx ; because of offset from day 1, not day 0
4036                                ;add    [cs:daycnt2], cx ; GET DAYS IN MONTHS PRECEEDING
4037                                ; 08/08/2023
4038 000018F6 010E[0006]  add     [daycnt2], cx
4039 000018FA 8A0E[FE05]      mov     cl, [bin_date_time+2] ; get month
4040                                ; 08/08/2023
4041                                ;xor    ch, ch
4042 000018FE 49          dec     cx ; january starts at offset 0
4043                                ; 19/01/2026 (BugFix)
4044 000018FF 740C        jz      short r_d_ret
4045
4046                                ; 08/08/2023
4047                                ;shl    cx, 1 ; word offset
4048                                ;;mov   si, month_table
4049                                ;add    si, cx
4050                                ;; 16/10/2022
4051                                ;; ds must be same with cs here, if so..
4052                                ;; what for cs: prefixes are used !?)
4053                                ;; mov ax, [cs:si]
4054                                ;; mov ax, [si] ; 16/10/2022 (MSDOS 5.0 IO.SYS - BIOSDATA:15D5h)
4055                                ;mov    ax, [si] ; mov ax, [cs:si]
4056                                ; ; get #days in previous months
4057                                ;add    [cs:daycnt2], ax
4058
4059                                ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
4060                                ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:1907h
4061 00001901 B400        mov     ah, 0
4062 00001903 BE[8F04]      mov     si, month_tab
4063 r_d_sum_loop:
4064 00001906 AC          lodsb
4065 00001907 0106[0006]  add     [daycnt2], ax
4066 0000190B E2F9        loop    r_d_sum_loop
4067 r_d_ret:
4068                                ;mov    si, [cs:daycnt2]
4069                                ; 08/08/2023
4070 0000190D 8B36[0006]  mov     si, [daycnt2]
4071 00001911 5A          pop     dx
4072 00001912 59          pop     cx
4073 00001913 5B          pop     bx
4074 00001914 58          pop     ax
4075 00001915 C3          retn
4076
4077 ;-----
4078
4079 r_t_retj:
4080 00001916 31C9        xor     cx, cx
4081 00001918 31D2        xor     dx, dx
4082 0000191A EB38        jmp     short r_t_ret
4083
4084 ; ===== S U B R O U T I N E =====
4085
4086 ; read_real_time reads the time from the rtc. on exit, it has the number of
4087 ; ticks (at 18.2 ticks per sec.) in cx:dx.
4088
4089 read_real_time: ; proc near
4090 mov     ah, 2
4091 int     1Ah ; CLOCK - READ REAL TIME CLOCK (AT,XT286,CONV,PS)
4092                                ; Return: CH = hours in BCD
4093                                ; CL = minutes in BCD
4094                                ; DH = seconds in BCD
4095 00001920 72F4        jb     short r_t_retj
4096 00001922 882E[FC05]      mov     [bin_date_time], ch ; hours
4097 00001926 880E[FD05]      mov     [bin_date_time+1], cl ; minutes
4098 0000192A 8836[FE05]      mov     [bin_date_time+2], dh ; seconds
4099 0000192E C606[FF05]00  mov     byte [bin_date_time+3], 0 ; unused for time
4100 00001933 E89F00      call    bcd_verify
4101 00001936 72DE        jb     short r_t_retj
4102 00001938 E88500      call    time_verify
4103 0000193B 72D9        jb     short r_t_retj
4104 0000193D E81500      call    in_bin ; from bcd to bin
4105 00001940 8A2E[FC05]      mov     ch, [bin_date_time]
4106 00001944 8A0E[FD05]      mov     cl, [bin_date_time+1]
4107 00001948 8A36[FE05]      mov     dh, [bin_date_time+2]
4108 0000194C 8A16[FF05]      mov     dl, [bin_date_time+3]
4109                                ; 16/10/2022
4110                                ; 17/09/2022
4111                                ; 31/05/2019
4112 00001950 FF1E[0606]  call    far [ttticks]

```

```

4113             ;call dword ptr tticks ; note: indirect far call
4114             ; cx:dx= number of ticks
4115             ; (at 18.2 ticks per sec.)
4116 r_t_ret:
4117 00001954 C3             retn
4118
4119 ; ===== S U B   R O U T I N E =====
4120
4121 ;   in_bin converts bin_date_time values from bcd to bin
4122
4123 in_bin:
4124 00001955 A0[FC05]        ; proc near
4125 00001958 E81F00        mov     al, [bin_date_time] ; century or hours
4126 0000195B A2[FC05]        call   bcd_to_bin
4127 0000195E A0[FD05]        mov     [bin_date_time], al
4128 00001961 E81600        mov     al, [bin_date_time+1] ; years or minutes
4129 00001964 A2[FD05]        call   bcd_to_bin
4130 00001967 A0[FE05]        mov     [bin_date_time+1], al
4131 0000196A E80D00        mov     al, [bin_date_time+2] ; months or seconds
4132 0000196D A2[FE05]        call   bcd_to_bin
4133 00001970 A0[FF05]        mov     [bin_date_time+2], al
4134 00001973 E80400        mov     al, [bin_date_time+3] ; days (not used for time)
4135 00001976 A2[FF05]        call   bcd_to_bin
4136 00001979 C3             mov     [bin_date_time+3], al
4137                         retn
4138
4139 ; ===== S U B   R O U T I N E =====
4140
4141 ;   bcd_to_bin converts two bcd nibbles in al (value <= 99.) to
4142 ;   a binary representation in al
4143 ;   ah is destroyed
4144
4145 bcd_to_bin: ; proc near
4146 0000197A 88C4        mov     ah, al
4147 0000197C 240F        and     al, 0Fh
4148 0000197E B104        mov     cl, 4
4149 00001980 D2EC        shr     ah, cl
4150 00001984 C3             aad
4151                         retn
4152
4153 ; ===== S U B   R O U T I N E =====
4154
4155 ;   date_verify loosely checks bcd date values to be in range
4156 ;   in bin_date_time
4157
4158 date_verify: ; proc near
4159 00001985 803E[FC05]20  cmp     byte [bin_date_time], 20h ; century check
4160 0000198A 7732        ja      short date_error
4161 0000198C 740E        jz      short century_20 ; jmp in 21th century
4162 0000198E 803E[FC05]19  cmp     byte [bin_date_time], 19h ; century check
4163                         ;jb     short date_error
4164                         ; 12/12/2022
4165 00001993 722A        jb      short date_err2
4166 00001995 803E[FD05]80  cmp     byte [bin_date_time+1], 80h ; year check
4167                         ;jb     short date_error
4168                         ; 12/12/2022
4169 0000199A 7223        jb      short date_err2
4170
4171 century_20:
4172 0000199C 803E[FD05]99  cmp     byte [bin_date_time+1], 99h ; year check
4173 000019A1 771B        ja      short date_error
4174 000019A3 803E[FE05]12  cmp     byte [bin_date_time+2], 12h ; month check
4175 000019A8 7714        ja      short date_error
4176 000019AA 803E[FE05]00  cmp     byte [bin_date_time+2], 0
4177                         ;jbe    short date_error
4178 000019AF 760D        jna     short date_error
4179 000019B1 803E[FF05]31  cmp     byte [bin_date_time+3], 31h ; day check
4180 000019B6 7706        ja      short date_error
4181                         ;cmp     byte [bin_date_time+3], 0 ; day check
4182                         ;jbe     short date_error
4183                         ;jna     short date_error
4184                         ; 12/12/2022
4185                         ; cf=0
4186                         ;clc
4187 000019B8 803E[FF05]01  cmp     byte [bin_date_time+3], 1 ; day check
4188 000019BD C3             ; 12/12/2022
4189                         cmp     byte [bin_date_time+3], 1 ; day check
4190                         retn
4191
4192 date_error:
4193 000019BE F9             stc
4194
4195 date_err2:
4196 000019BF C3             retn
4197
4198 ; ===== S U B   R O U T I N E =====
4199
4200 ;   time_verify very loosely checks bcd date values to be in range
4201 ;   in bin_date_time
4202
4203 time_verify: ; proc near
4204 000019C0 803E[FC05]24  cmp     byte [bin_date_time], 24h ; hour check
4205 000019C5 770C        ja      short time_error
4206 000019C7 803E[FD05]59  cmp     byte [bin_date_time+1], 59h ; minute check
4207 000019CC 7705        ja      short time_error
4208                         ; 12/12/2022h
4209 000019CE 803E[FE05]5A  cmp     byte [bin_date_time+2], 59h ; second check
4210                         ;ja      short time_error
4211                         ;clc
4212                         ;retn
4213                         ; 12/12/2022
4214 000019D3 F5             cmp     byte [bin_date_time+2], 5Ah
4215 000019D4 C3             time_error:
4216                         bvc     bv_error
4217                         cmc     ; cf=0 -> cf=1, cf=1 -> cf=0
4218                         retn
4219
4220 ; -----
4221
4222 time_error:
4223 000019D5 B90400        ;stc
4224 000019D8 BB[FC05]        ;retn
4225
4226 ; ===== S U B   R O U T I N E =====
4227
4228 ;   bcd_verify checks values in bin_date_time to be valid
4229 ;   bcd numerals. carry set if any nibble out of range
4230
4231 bcd_verify: ; proc near
4232 000019D5 B90400        mov     cx, 4 ; 4 bytes to check
4233 000019D8 BB[FC05]        mov     bx, bin_date_time
4234
4235 bv_loop:
4236 000019DB 8A07        mov     al, [bx] ; get abcd number (0..99)
4237 000019DD 88C4        mov     ah, al
4238 000019DF 250FF0        and     ax, 0F00Fh ; 10's place in high ah, 1's in al
4239                         ; is 1's place in range?
4240 000019E2 3C0A        cmp     al, 10

```

```

4237 000019E4 77ED          ja      short bv_error ; jmp out of range
4238 000019E6 D0EC          shr     ah, 1
4239 000019E8 D0EC          shr     ah, 1
4240 000019EA D0EC          shr     ah, 1
4241 000019EC D0EC          shr     ah, 1
4242 000019EE 80E40F        and     ah, 0Fh          ; get rid of any erroneous bits
4243 000019F1 80FC0A        cmp     ah, 10          ; is 10's place in range
4244 000019F4 77DD          ja      short bv_error ; jmp out of range
4245 000019F6 43              inc     bx              ; next byte
4246 000019F7 49              dec     cx
4247 000019F8 75E1          jnz     short bv_loop
4248 000019FA F8              cld
4249 000019FB C3              retn
4250
4251
4252          ; 12/12/2022
4253 ;bv_error:
4254          ;stc              ; set error flag
4255          ;retn
4256
4257
4258
4259 endk09:
4260
4261
4262
4263
4264
4265          ;
4266          ; System initialization
4267          ;
4268          ; The entry conditions are established by the bootstrap
4269          ; loader and are considered unknown. The following jobs
4270          ; will be performed by this module:
4271          ;
4272          ; 1. All device initialization is performed
4273          ; 2. A local stack is set up and DS:SI are set
4274          ; to point to an initialization table. Then
4275          ; an inter-segment call is made to the first
4276          ; byte of the dos
4277          ; 3. Once the dos returns from this call the ds
4278          ; register has been set up to point to the start
4279          ; of free memory. The initialization will then
4280          ; load the command program into this area
4281          ; beginning at 100 hex and transfer control to
4282          ; this program.
4283          ;
4284
4285          ; 01/10/2022
4286          ; 08/01/2018 - Retro DOS v4.0
4287
4288          ; drvfat must be the first location of freeable space!
4289
4290 align 2
4291          ;db 90h
4292
4293          ; 09/12/2023 - Retro DOS v5.0 IO.SYS (IBMBIO.COM 7.1)
4294          ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1A0Ch)
4295
4296          ; 30/12/2022
4297          ; (MSDOS 6.21 IO.SYS, BIOSDATA:16D6h)
4298
4299 000019FC 0000        drvfat:      dw 0          ; drive and fat id of dos
4300
4301          ; 09/12/2023
4302          ; bios_l:      dw 0          ; first sector of data (low word)
4303          ; bios_h:      dw 0          ; first sector of data (high word)
4304          ; First_Data_Sector:
4305          ; dw 0
4306          ; dw 0
4307          ; doscnt:      dw 0          ; how many sectors to read
4308          ; fbigfat:      db 0          ; flags for drive
4309          ; fatloc:      dw 0          ; seg addr of fat sector
4310          ; init_bootseg: dw 0          ; seg addr of buffer for reading boot record
4311          ; 09/12/2023
4312          ; fbigfat:      db 0          ; flags for drive
4313          ; rom_drv_num:   dw 80h       ; rom drive number
4314          ; md_sectorsize: dw 200h      ; used by get_fat_sector proc.
4315          ; 12/12/2023
4316          ; temp_cluster: dw 0          ; used by get_fat_sector proc.
4317          ; last_fat_sec_num: dw 0FFFFh ; used by get_fat_sector proc.
4318
4319          ; the following two bytes are used to save the info returned by int 13, ah = 8
4320          ; call to determine drive parameters.
4321          ; num_heads:      db 2          ; dw 2          ; number of heads returned by rom
4322          ; db 0          ; 09/12/2023
4323          ; sec_trk:      db 9          ; dw 40          ; sec/trk returned by rom
4324          ; num_cylin:     db 40         ; dw 40          ; number of cylinders returned by rom
4325          ; db 0          ; 09/12/2023
4326          ; 09/12/2023
4327          ; sec_trk:      db 9          ; dw 40          ; sec/trk returned by rom
4328          ; fakefloppydrv: db 0          ; if 1, then no diskette drives in the system.
4329
4330          ; 09/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
4331          ; Orig_Int1Eh_Table:
4332          ; dw 0
4333          ; dw 0
4334
4335          ; -----
4336
4337          ; 09/12/2023
4338          %if 0
4339
4340          disktable:      dw 512, 0100h, 64, 0 ; warning !!! old values
4341          dw 2048, 0201h, 112, 0
4342          dw 8192, 0402h, 256, 0
4343          dw 32680, 0803h, 512, 0 ; warning !!! old values
4344          dw 65535, 1004h, 1024, 0
4345          ; default disktable under
4346          ; the assumption of total fat size <= 128 kb,
4347          ; and the maximum size of fat entry = 16 bit.
4348          %endif
4349
4350          ; 09/12/2023 - Retro DOS v5.0 IO.SYS (IBMBIO.COM 7.1)
4351          ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1A2Ah)
4352
4353          ; 09/12/2023
4354          ; 08/08/2023
4355          ; disktable.totalsectors: resw 1 ; high word
4356          ; resw 1 ; low word
4357          ; disktable.shiftcount:      resb 1
4358          ; disktable.secpersclus:      resb 1
4359          ; disktable.rdiretries:        resw 1
4360          ; disktable.bigflag:          resw 1

```

```

4361 00001A18 0000A87F0308000200-   disktable2: dw 0, 32680, 0803h, 512, 0 ; for compatibility.
4361 00001A21 00
4362
4363 00001A22 0400000000204000240-   dw 4, 0, 0402h, 512, 40h ; (32680 sectors, 16340 KB)
4363 00001A2B 00 ; covers upto 134 mb media.
4364
4365 00001A2C 0800000000308000240-   dw 8, 0, 0803h, 512, 40h ; fbig = 40h ; (40000h sectors = 128 MB)
4365 00001A35 00 ; upto 268 mb ; (80000h sectors = 256 MB)
4366 00001A36 1000000000410000240-   dw 16, 0, 1004h, 512, 40h ; upto 536 mb ; (100000h sectors = 512 MB)
4366 00001A3F 00
4367 00001A40 2000000000520000240-   dw 32, 0, 2005h, 512, 40h ; upto 1072 mb ; (200000h sectors = 1024 MB)
4367 00001A49 00
4368 00001A4A 4000000000640000240-   dw 64, 0, 4006h, 512, 40h ; upto 2144 mb ; (400000h sectors = 2048 MB)
4368 00001A53 00
4369 ; 09/12/2023
4370 ; dw 128, 0, 8007h, 512, 40h ; upto 4288 mb ; (800000h sectors = 4096 MB)
4371 00001A54 FFFFFFFF0308000060-   dw 0FFFFh, 0FFFFh, 0803h, 0, 60h ; FAT32 (> 2144MB)
4371 00001A5D 00
4372 ; (fbig and fbigbig flags are set)
4373
4374 ; -----
4375 ; *****
4376 ; variables for mini disk initialization
4377 ; *****
4378
4379 ; 01/10/2022
4380 ; [ Note: Minidisk == logical dos drive (in extended dos partition) ]
4381
4382 rom_minidisk_num: db 0 ; temp variable for phys unit
4383 00001A5E 00 hnum: db 0 ; real number of hardfiles
4384 00001A5F 00 last_dskdrv_table: dw dskdrvs ; index into dskdrv table
4385 00001A60 [3C05] end_of_bdss: dw bdss ; offset value of the ending address
4386 00001A62 [4C08] ; of bds table. needed to figure out
4387 ; the dosdatasg address.
4388
4389 00001A64 0000 mini_hdlim: dw 0
4390 00001A66 0000 mini_seclim: dw 0
4391
4392 ; 19/12/2023
4393 ; 09/12/2023
4394 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1A7Ah)
4395 ; ld_p_number: dw 2BADh ; (for 'find_mini_partition' proc)
4396
4397 ; end of mini disk init variables *****
4398
4399 ; -----
4400
4401 00001A68 30312F31302F383400 bios_date: db '01/10/84',0 ; used for checking at rom bios date.
4402
4403 ; 13/12/2022
4404 %if 0
4405
4406 ; align 2
4407 db 90h
4408
4409 ; the following are the recommended bpbs for the media that we know of so far.
4410
4411 ; struc bpbx
4412 ; resw 1 ; 512
4413 ; resb 1
4414 ; resw 1 ; 1
4415 ; resb 1 ; 2
4416 ; resw 1
4417 ; resw 1
4418 ; resb 1
4419 ; resw 1
4420 ; resw 1
4421 ; resw 1 ; 2
4422 ; resw 1
4423 ; resw 1 ; hidden sector high
4424 ; resd 1 ; extended total sectors
4425 ; .size:
4426 ; endstruc
4427
4428 ; 08/01/2019 - Retro DOS v4.0
4429
4430 ; 20/04/2019
4431
4432 ; 01/10/2022 - Retro DOS v4.0 (MSDOS 5.0) IO.SYS
4433
4434 ; 09/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
4435 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1A86h)
4436
4437 ; 09/12/2022
4438 BPB48T:
4439 ; bpb48t: ; bpbx <512, 2, 1, 2, 112, 720, 0FDh, 2, 9, 2, 0, 0, 0, 0>
4440 ; 48 tpi diskettes ;
4441 dw 512 ; physical sector size in bytes
4442 db 2 ; sectors/allocation unit
4443 dw 1 ; reserved sectors for dos
4444 db 2 ; number of allocation tables
4445 dw 112 ; number of directory entries
4446 dw 720 ; 2*9*40 ; number of sectors (at 512 bytes each)
4447 db 0FDh ; media descriptor
4448 dw 2 ; number of fat sectors
4449 dw 9 ; sectors per track
4450 dw 2 ; heads
4451 dw 0 ; hidden sector count (low word)
4452 dw 0 ; hidden sector (high)
4453 dw 0 ; number of sectors (low)
4454 dw 0 ; number of sectors (high)
4455 ; 09/12/2023
4456 ; FAT32 extensions (to BDS)
4457 times 28 db 0
4458 ;
4459 db 90h
4460
4461 ; align 2
4462 BPB96T:
4463 ; bpb96t: ; bpbx <512, 1, 1, 2, 224, 2400, 0F9h, 7, 15, 2, 0, 0, 0, 0>
4464 ; 96 tpi diskettes ;
4465 dw 512 ; physical sector size in bytes
4466 db 1 ; sectors/allocation unit
4467 dw 1 ; reserved sectors for dos
4468 db 2 ; number of allocation tables
4469 dw 224 ; number of directory entries
4470 dw 2400 ; 2*15*80 ; number of sectors (at 512 bytes each)
4471 db 0F9h ; media descriptor
4472 dw 7 ; number of fat sectors
4473 dw 15 ; sectors per track
4474 dw 2 ; heads
4475 dw 0 ; hidden sector count (low word)
4476 dw 0 ; hidden sector (high)
4477 dw 0 ; number of sectors (low)
4478 dw 0 ; number of sectors (high)

```



```

4478 ; 09/12/2023
4479 ; FAT32 extensions (to BDS)
4480 times 28 db 0
4481 ;
4482 db 90h
4483 ;align 2
4484 BPB35:
4485 ;bpbx <512, 2, 1, 2, 112, 1440, 0F9h, 3, 9, 2, 0, 0, 0, 0>
4486 ; 3.5" diskettes - 720 KB ;
4487 dw 512 ; physical sector size in bytes
4488 db 2 ; sectors/allocation unit
4489 dw 1 ; reserved sectors for dos
4490 db 2 ; number of allocation tables
4491 dw 112 ; number of directory entries
4492 dw 1440 ; 2*9*80 ; number of sectors (at 512 bytes each)
4493 db 0F9h ; media descriptor
4494 dw 3 ; number of fat sectors
4495 dw 9 ; sectors per track
4496 dw 2 ; heads
4497 dw 0 ; hidden sector count (low word)
4498 dw 0 ; hidden sector (high)
4499 dw 0 ; number of sectors (low)
4500 dw 0 ; number of sectors (high)
4501 ; 09/12/2023
4502 ; FAT32 extensions (to BDS)
4503 times 28 db 0
4504 ;
4505 db 90h
4506 ;align 2
4507
4508 ;align 2
4509 ;BPB144:
4510 ;bpb144: ; Retro DOS v4.0 feature only ! ; 1.44MB diskettes
4511 ;
4512 dw 512 ; physical sector size in bytes
4513 db 1 ; sectors/allocation unit
4514 dw 1 ; reserved sectors for dos
4515 db 2 ; number of allocation tables
4516 dw 224 ; number of directory entries
4517 dw 2880 ; 2*18*80 ; number of sectors (at 512 bytes each)
4518 db 0F0h ; media descriptor
4519 dw 9 ; number of fat sectors
4520 dw 18 ; sectors per track
4521 dw 2 ; heads
4522 dw 0 ; hidden sector count (low word)
4523 dw 0 ; hidden sector (high)
4524 dw 0 ; number of sectors (low)
4525 dw 0 ; number of sectors (high)
4526 ;
4527 db 90h
4528 ;align 2
4529
4530 BPB288:
4531 ;bpbx <512, 2, 1, 2, 240, 5760, 0F0h, 9, 36, 2, 0, 0, 0, 0>
4532 ; 3.5" diskettes - 2.88 MB ;
4533 dw 512 ; physical sector size in bytes
4534 db 2 ; sectors/allocation unit
4535 dw 1 ; reserved sectors for dos
4536 db 2 ; number of allocation tables
4537 dw 240 ; number of directory entries
4538 dw 5760 ; 2*36*80 ; number of sectors (at 512 bytes each)
4539 db 0F0h ; media descriptor
4540 dw 3 ; number of fat sectors
4541 dw 9 ; sectors per track
4542 dw 2 ; heads
4543 dw 0 ; hidden sector count (low word)
4544 dw 0 ; hidden sector (high)
4545 dw 0 ; number of sectors (low)
4546 dw 0 ; number of sectors (high)
4547 ; 09/12/2023
4548 ; FAT32 extensions (to BDS)
4549 times 28 db 0
4550 ;
4551 db 90h
4552 ;align 2
4553
4554 %endif
4555
4556 ; -----
4557 ; align 2
4558 ; 09/12/2022
4559 %if 0
4560 bphtable: dw bpb48t ; 48tpi drives
4561 dw bpb96t ; 96tpi drives
4562 dw bpb35 ; 3.5" drives
4563 dw bpb35 ; unused 8" diskette
4564 dw bpb35 ; unused 8" diskette
4565 dw bpb35 ; used for hard disk
4566 dw bpb35 ; used for tape drive
4567 dw bpb35 ; FFOTHER
4568 dw bpb35 ; ERIMO
4569 dw bpb288 ; 2.88MB drive
4570 ;
4571 ;dw bpb144 ; 1.44MB drive - Retro DOS v4.0 feature !
4572 %endif
4573
4574 ; 13/12/2022
4575 %if 0
4576 BPBTABLE: dw BPB48T ; 48tpi drives
4577 dw BPB96T ; 96tpi drives
4578 dw BPB35 ; 3.5" drives
4579 dw BPB35 ; unused 8" diskette
4580 dw BPB35 ; unused 8" diskette
4581 dw BPB35 ; used for hard disk
4582 dw BPB35 ; used for tape drive
4583 dw BPB35 ; FFOTHER
4584 dw BPB35 ; ERIMO
4585 dw BPB288 ; 2.88MB drive
4586 ;
4587 ;dw BPB144 ; 1.44MB drive - Retro DOS v4.0 feature !
4588 %endif
4589
4590 ; -----
4591 ;
4592 ; entry point to call utility functions in Bios_Code. At this time,
4593 ; we aren't doing any A20 switching. During MSINIT time Bios_Code
4594 ; will not yet be moved to its final resting place, so we know
4595 ; it'll be low.
4596 ;
4597 ;
4598 ; to use this function, do a "push cs" and load bp with the offset of
4599 ; the function you want to call in Bios_Code. This routine will
4600 ; push the address of a retf in Bios_Code onto the stack which
4601 ; will get executed when the utility function finishes. It will

```

```

4602 ; then transfer control to Bios_Code:bp using a couple of pushes
4603 ; and a retf
4604
4605 ; 16/10/2022
4606 ;BC_RETf equ bc_ret - DOSBIOSEG_2C7h
4607 ; 09/12/2022
4608 BC_RETf equ bc_ret
4609
4610 ; 09/12/2023
4611 ;PCDOS 7.1 IBMBIO.COM bc_ret offset = 0CAh (in BIOSCODE segment = 364h)
4612
4613 addr_of_bcretf: ;dw 0C8h ; dw bc_ret
4614 ; 2C7h:0C8h = 70h:2638h
4615 ; 09/12/2023
4616 ; 364h:0CAh = 70h:300Ah ; PCDOS 7.1
4617 00001A71 [CA00] dw BC_RETf ; dw 0CAh
4618
4619 ; -----
4620
4621 call_bios_code: ; proc far
4622 00001A73 2EFF36[711A] push word [cs:addr_of_bcretf]
4623 ; set up near return to far return
4624 00001A78 2EFF36[0406] push word [cs:cdev+2] ; push Bios_Code segment
4625 00001A7D 55 push bp ; save offset of utility function
4626 00001A7E CB retf ; far jump to (DOS)BIOS code
4627
4628 ; -----
4629
4630 ; 09/12/2023 - Retro DOS v5.0 IBMBIO.COM/IO.SYS
4631 ; 20/12/2022
4632 00001A7F 00 flp_drvs: db 0
4633 ; 11/12/2023
4634 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:1B81h)
4635 firstcluster_hw:
4636 00001A80 0000 dw 0 ; 06/04/2024
4637 00001A82 00 Boot_Drv: db 0
4638
4639 ; -----
4640
4641 ; 09/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
4642 ; -----
4643 ; PCDOS 7.1 CD BOOT option code
4644 ; -----
4645 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1B84h)
4646
4647 cd_boot_option:
4648 00001A83 50 push ax
4649 00001A84 1E push ds
4650 00001A85 06 push es
4651 00001A86 52 push dx
4652
4653 00001A87 B401 cdbo_1: mov ah, 1
4654 00001A89 CD16 int 16h ; KEYBOARD - status
4655 00001A8B 7406 jz short cdbo_2
4656 00001A8D 30E4 xor ah, ah
4657 00001A8F CD16 int 16h ; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
4658 ; Return: AH = scan code, AL = character
4659 00001A91 EBF4 jmp short cdbo_1
4660
4661 00001A93 0E cdbo_2: push cs
4662 00001A94 1F pop ds
4663 00001A95 BE[6D1B] mov si, cd_boot_msg ; "Press the ENTER key to boot from CD"...
4664 00001A98 AC lodsb
4665
4666 00001A99 BB0700 cdbo_3: mov bx, 7
4667 00001A9C B40E mov ah, 0Eh
4668 00001A9E CD10 int 10h ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
4669 ; AL = character, BH = display page (alpha modes)
4670 ; BL = foreground color (graphics modes)
4671 00001AA0 AC lodsb
4672 00001AA1 08C0 or al, al
4673 00001AA3 75F4 jnz short cdbo_3
4674 00001AA5 B84000 mov ax, 40h
4675 00001AA8 8ED8 mov ds, ax
4676 ;mov bx, [6Ch] ; 0:46Ch = Daily timer counter (4 bytes)
4677 ; 09/12/2023
4678 00001AAA 8B166C00 mov dx, [6Ch]
4679 00001AAE 8B366E00 mov si, [6Eh]
4680
4681 wait_for_key:
4682 ;push bx
4683 ;mov bx, 7
4684 ; bx = 7
4685 00001AB2 B8080E mov ax, 0E08h
4686 00001AB5 CD10 int 10h ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
4687 ; AL = character, BH = display page (alpha modes)
4688 ; BL = foreground color (graphics modes)
4689 00001AB7 B8200E mov ax, 0E20h
4690 00001ABA CD10 int 10h ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
4691 ; AL = character, BH = display page (alpha modes)
4692 ; BL = foreground color (graphics modes)
4693 00001ABC B8080E mov ax, 0E08h
4694 00001ABF CD10 int 10h ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
4695 ; AL = character, BH = display page (alpha modes)
4696 ; BL = foreground color (graphics modes)
4697 ;pop bx
4698 ;add bx, 18 ; 18.2 ticks per second
4699 ; 09/12/2023
4700 00001AC4 83D600 add dx, 18
4701 ;adc si, 0 ; next second (if carry flag is 1)
4702
4703 continue_to_wait:
4704 00001AC7 B401 mov ah, 1
4705 00001AC9 CD16 int 16h ; KEYBOARD - status
4706 00001ACB 741B jz short cdbo_5
4707 00001ACD B400 mov ah, 0
4708 00001ACF CD16 int 16h ; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
4709 ; Return: AH = scan code, AL = character
4710
4711 ; 09/12/2023
4712 ;cmp ax, 11Bh ; ESC key
4713 ;jz short cdb0_7
4714
4715 ;cdb0_4:
4716 ;push ax ; *
4717 ;mov dx, ax ; *
4718 ; CRLF (next line)
4719 ;mov bx, 7
4720 ; bx = 7
4721 00001AD3 B80D0E mov ax, 0E0Dh
4722 00001AD6 CD10 int 10h ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
4723 ; AL = character, BH = display page (alpha modes)
4724 ; BL = foreground color (graphics modes)
4725 00001AD8 B80A0E mov ax, 0E0Ah
4726 00001ADB CD10 int 10h ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
4727 ; AL = character, BH = display page (alpha modes)

```



```

4726                                     ; BL = foreground color (graphics modes)
4727                                     ; 09/12/2023
4728                                     ; pop ax ; *
4729
4730 00001ADD 81FA1B01                   cmp     dx, 11Bh
4731                                     ; cmp ax, 11Bh ; ESC key (to cancel CD/DVD boot)
4732 00001AE1 7418                       je      short cdbo_7
4733
4734                                     ; 10/12/2023
4735 00001AE3 5A                         pop     dx
4736 00001AE4 07                         pop     es
4737 00001AE5 1F                         pop     ds
4738 00001AE6 58                         pop     ax
4739 00001AE7 C3                         retn
4740
4741 00001AE8 3B366E00                   cmp     si, [6Eh]
4742 00001AEC 7504                       jnz     short cdbo_6
4743                                     ; 09/12/2023
4744 00001AEE 3B166C00                   cmp     dx, [6Ch]
4745                                     ; cmp bx, [6Ch]
4746
4747 00001AF2 73D3                       jnb     short continue_to_wait
4748 00001AF4 2EFE0E[6C1B]              dec     byte [cs:time_counter]
4749 00001AF9 75B7                       jnz     short wait_for_key
4750
4751                                     ; 09/12/2023
4752                                     ; CRLF (next line)
4753                                     ;
4754                                     ; mov bx, 7
4755                                     ; mov ax, 0E0Dh
4756                                     ; int 10h                                     ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
4757                                     ;                                     ; AL = character, BH = display page (alpha modes)
4758                                     ;                                     ; BL = foreground color (graphics modes)
4759                                     ;
4760                                     ; mov ax, 0E0Ah
4761                                     ; int 10h                                     ; - VIDEO - WRITE CHARACTER AND ADVANCE CURSOR (TTY WRITE)
4762                                     ;                                     ; AL = character, BH = display page (alpha modes)
4763                                     ;                                     ; BL = foreground color (graphics modes)
4764 00001AFB 0E                         push    cs
4765 00001AFC 1F                         pop     ds
4766                                     ; 09/12/2023
4767 00001AFD 1E                         push    ds
4768 00001AFE 07                         pop     es
4769                                     ; es = ds = cs
4770
4771 00001AFF B8004B                     mov     ax, 4B00h
4772                                     ; xor dl, dl
4773                                     ; 09/12/2023
4774 00001B02 31D2                       xor     dx, dx
4775                                     ; dl = disk drive = 0 ; fd
4776                                     ; mov si, 1C93h
4777 00001B04 BE[591B]                   mov     si, empty_dap_buff
4778 00001B07 CD13                       int     13h                                     ; DISK - Bootable CD-ROM - AL = TERMINATE DISK EMULATION
4779                                     ; DS:SI = Specification packet filled
4780
4781                                     ; mov dx, 80h
4782                                     ; xor ax, ax
4783                                     ; 09/12/2023
4784 00001B09 B81300                     mov     ax, 19
4785 00001B0C 89F7                       mov     di, si
4786                                     ; mov byte [si], 13h
4787                                     ; mov [si+1], al
4788 00001B0E AB                         stosw
4789                                     ; mov [si+2], dx
4790 00001B0F B080                       mov     al, 80h
4791 00001B11 AB                         stosw
4792 00001B12 89C2                       mov     dx, ax
4793                                     ; mov [si+4], ax
4794                                     ; mov [si+6], ax
4795                                     ; mov [si+8], ax
4796                                     ; mov [si+0Ah], ax
4797                                     ; mov [si+0Ch], ax
4798                                     ; mov [si+0Eh], ax
4799                                     ; mov [si+10h], al
4800                                     ; mov [si+11h], al
4801                                     ; mov [si+12h], al
4802 00001B14 B90F00                     mov     cx, 15
4803 00001B17 F3AA                       rep     stosb
4804                                     ; dl = disk drive = 80h ; hd
4805 00001B19 B8004B                     mov     ax, 4B00h
4806 00001B1C CD13                       int     13h                                     ; DISK - Bootable CD-ROM - AL = TERMINATE DISK EMULATION
4807 00001B1E 31C0                       xor     ax, ax
4808                                     ; 09/12/2023
4809                                     ; mov dx, 80h
4810                                     ; dx = 80h
4811 00001B20 CD13                       int     13h                                     ; DISK - RESET DISK SYSTEM
4812                                     ; DL = drive (if bit 7 is set both hard disks and floppy disks reset)
4813                                     ; 09/12/2023
4814                                     ; push cs
4815                                     ; pop es
4816                                     ; es = ds = cs
4817
4818 00001B22 B80102                     mov     ax, 201h
4819                                     ; mov bx, 152h
4820 00001B25 BB[5201]                   mov     bx, disksector
4821                                     ; mov cx, 1
4822                                     ; 09/12/2023
4823 00001B28 41                         inc     cx ; cx = 1
4824                                     ; mov dx, 80h
4825                                     ; dx = 80h
4826 00001B29 CD13                       int     13h                                     ; DISK - READ SECTORS INTO MEMORY
4827                                     ; AL = number of sectors to read, CH = track, CL = sector
4828                                     ; DH = head, DL = drive, ES:BX -> buffer to fill
4829                                     ; Return: CF set on error, AH = status, AL = number of sectors read
4830                                     ; jc short cdbo_8
4831                                     ; 10/12/2023
4832 00001B2B 72B6                       jc      short cdbo_4
4833
4834 00001B2D 2681BFFE0155AA             cmp     word [es:bx+1FEh], 0AA55h
4835                                     ; jz short cdbo_9
4836                                     ; 10/12/2023
4837 00001B34 75AD                       jnz     short cdbo_4
4838
4839                                     ; jmp short cdbo_4
4840
4841                                     ; 10/12/2023
4842                                     ; (stack clearing -pop- is not necessary here,
4843                                     ; PCDOS 7.1 boot sector will set stack pointer again)
4844                                     ; pop ax ; near call return address
4845                                     ; pop cx ; +++ ; ch = [MediaByte]
4846
4847                                     ; 09/12/2023
4848                                     ; push cs
4849                                     ; pop ds

```

```

4850                ; ds = cs
4851 00001B36 31C0    xor     ax, ax ; 0
4852 00001B38 BF007C  mov     di, 7C00h
4853 00001B3B 8EC0    mov     es, ax
4854 00001B3D 89DE    mov     si, bx
4855 00001B3F 06      push    es
4856 00001B40 57      push    di
4857 00001B41 B90001    mov     cx, 100h ; 256
4858                ; 10/12/2023
4859                ; cld ; not necessary (direction flag is already cleared)
4860 00001B44 F3A5    rep movsw
4861 00001B46 8ED8    mov     ds, ax
4862 00001B48 BE7800    mov     si, 78h
4863 00001B4B 2EA1[141A]  mov     ax, [cs:Orig_Int1Eh_Table]
4864 00001B4F 8904    mov     [si], ax
4865 00001B51 2EA1[161A]  mov     ax, [cs:Orig_Int1Eh_Table+2]
4866 00001B55 894402    mov     [si+2], ax
4867 00001B58 CB      retf
4868
4869                ; -----
4870 dap_buffer: ; 16/12/2023
4871
4872 00001B59 13      empty_dap_buff: db 19
4873                ; db 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0 ; db 18 dup(0)
4874 00001B5A 00<rep 12h>    times 18 db 0
4875 00001B6C 05      time_counter: db 5 ; 5 seconds
4876 00001B6D 0D0A    cd_boot_msg: db 0Dh,0Ah
4877                ; db 'Press the ENTER key to boot from CD or DVD.....',0
4878                ; 09/12/2023
4879 00001B6F 507265737320616E79- db 'Press any key to boot from CD or DVD ...',0
4879 00001B78 206B657920746F2062-
4879 00001B81 6F6F742066726F6D20-
4879 00001B8A 4344206F7220445644-
4879 00001B93 202E2E2E00
4880
4881                ; -----
4882
4883                ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1)
4884
4885                ; 01/10/2022 - Retro DOS v4.0 (MSDOS 5.0, classic/old MICROSOFT DOS method)
4886                ; 20/12/2022 - Retro DOS v4.0 (combined kernel files, original/new method) (*)
4887                ; (*) (for using Retro DOS kernel 'MSDOS.SYS' with Retro DOS boot sector)
4888
4889                ; -----
4890                ; entry point from boot sector
4891                ; -----
4892
4893 init:
4894                ; 27/12/2018
4895                ; MSDOS 6.0 (MSINIT.ASM)
4896                ; =====
4897                ; entry from boot sector. the register contents are:
4898                ;
4899                ; dl = int 13 drive number we booted from
4900                ; ch = media byte
4901                ; bx = first data sector on disk.
4902                ; ax = first data sector (high)
4903                ; di = sectors/fat for the boot media.
4904
4905                ; 10/12/2023
4906                ; Retro DOS v5.0 (IBMBIO.COM)
4907                ; =====
4908                ; PCDOS 7.1 IBMBIO.COM - registers from MSLOAD section
4909                ; DL = [BootDrive]
4910                ; CH = [MediaByte]
4911                ; AX:BX = First data Sector
4912                ; DS:SI = Original INT 1Eh table address
4913                ;
4914                ; Stack: INT 1Eh vector (0:78h) !not used! (dword [sp])
4915                ; INT 1Eh table address !not used! (dword [sp+4])
4916                ; DI = 78h !not used!
4917
4918                ; 07/04/2018
4919                ; =====
4920                ; Retro DOS v2.0 - registers from FD Boot Sector
4921                ; DL = [bsDriveNumber]
4922                ; DH = [bsMedia]
4923                ; AX = [bsSectors] ; Total sectors
4924                ; DS = 0, SS = 0
4925                ; BP = 7C00h
4926
4927                ; 29/09/2023
4928                ; SP = 0FFFEh (for Retro DOS v2&v3 boot sector)
4929                ; = 07C00h (for MSDOS 5.0 boot sector)
4930
4931                ; 10/12/2023 - Retro DOS v5.0
4932                ; -----
4933                ; INPUT (registers from Retro DOS v4-v5 boot sector):
4934                ; DL = [bsDriveNumber]
4935                ; DH = [bsMedia]
4936                ; SS = 0
4937                ; BP = 7C00h (boot sector address)
4938                ;
4939                ; If the boot drive is a CD (CDROM) or DVD
4940                ; and CD boot option is enabled/requested:
4941                ; AX = 'CD'
4942                ; If the boot drive is a FD or HD
4943                ; or CD boot option is not enabled/requested:
4944                ; AX <> 'CD'
4945
4946                ; 20/12/2022
4947                ; Changing original MSDOS 5.0 IO.SYS init code with Retro DOS v4.0 init code.
4948                %if 0
4949                cli
4950
4951                push    ax
4952                xor     ax, ax
4953                mov     ds, ax
4954                pop     ax
4955                %endif
4956
4957                ; 20/12/2022 - Retro DOS v4.0 (combined kernel)
4958                ; 10/12/2023 - Retro DOS v5.0 (combined kernel)
4959
4960 KERNEL_SEGMENT equ 70h ; (DOS BIOSDATA SEGMENT)
4961 BSSECPERTRACK equ 18h ; boot sector offset 18h (for Retro DOS & MSDOS)
4962
4963                ; -----
4964                ; initialization - stage 1
4965                ; -----
4966                ; 02/06/2018 - Retro DOS v3.0
4967
4968                ; 10/12/2023
4969 00001B98 FC      cld ; may not be necessary

```

```

4970
4971 ; 21/12/2022
4972 ; Move Retro DOS v2.0 boot sector parameters to 0060h:0
4973 ;mov bx, 60h
4974 ;mov es, bx
4975 ;mov si, bp
4976 ;sub di, di
4977 ;mov cx, 35 ; 70 bytes, 35 words
4978 ;;mov cl, 35
4979 ;rep movsw
4980
4981 ; 10/12/2023 - Retro DOS v5.0
4982 00001B99 3D4344 cmp ax, 'CD' ; is CD boot option enabled or not ?
4983 00001B9C 7503 jne short init0
4984
4985 00001B9E E8E2FE call cd_boot_option
4986
4987 00001BA1 0E init0: push cs
4988 00001BA2 1F pop ds
4989
4990 ; 10/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
4991 ;mov [Boot_Drv], dl
4992
4993 ; 20/03/2019 - Retro DOS v4.0
4994 ;cli ; turn interrupts off while manipulating stack
4995 ;mov ss, cx ; set stack segment register
4996
4997 00001BA3 BC0007 mov sp, 0700h ; move stack pointer to safe place
4998
4999 ;sti ; turn interrupts on
5000
5001 ; 27/03/2018
5002 ;mov cx, KERNEL_SIZE ; words !
5003
5004 ; 20/03/2019
5005 00001BA6 B90080 mov cx, 32768 ; 65536 bytes
5006
5007 ; 21/12/2022
5008 ; 07/04/2018
5009 00001BA9 BB7000 mov bx, KERNEL_SEGMENT ; 0070h
5010 ;mov bl, KERNEL_SEGMENT
5011 00001BAC 8EC3 mov es, bx
5012 00001BAE 31FF xor di, di
5013 00001BB0 89FE mov si, di
5014
5015 ; Move KERNEL file from 1000h:0 to 0070h:0
5016 ; (Retro DOS v2 BS loads 'MSDOS.SYS' at 1000h:0000h)
5017 00001BB2 F3A5 rep movsw
5018
5019 ; 20/03/2019 - Retro DOS v4.0
5020 00001BB4 53 push bx
5021 ;push init0
5022 00001BB5 68[B91B] push init1 ; 10/12/2023
5023 00001BB8 CB retf
5024
5025 ;init0: ; 10/12/2023 - Retro DOS 5.0
5026
5027 init1: ; 20/12/2022
5028 ; (combined kernel file > 64KB)
5029
5030 ; 20/03/2019
5031 00001BB9 B520 mov ch, 20h
5032 00001BBB 8ED9 mov ds, cx ; 2000h
5033 ;mov cx, 1070h
5034 00001BBD B97010 mov cx, KERNEL_SEGMENT+1000h ; 20/12/2022
5035 00001BC0 8EC1 mov es, cx
5036
5037 ; 21/12/2022
5038 ;KERNEL_SIZE equ END_OF_KERNEL - BData_start
5039 ; 28/09/2023
5040 NXWORDCOUNT equ ((KERNEL_SIZE+1)>>1)-32768
5041
5042 ;mov cx, KERNEL_SIZE - 32768
5043 ; 28/09/2023 (BugFix)
5044 00001BC2 B9E31E mov cx, NXWORDCOUNT
5045 ;mov cx, NXBYTECOUNT
5046 ;shr cx, 1 ; 28/09/2023
5047 ;xor si, si
5048 ;xor di, di
5049 00001BC5 F3A5 rep movsw
5050
5051 ; 28/09/2023
5052 ;; 17/06/2018
5053 ;mov ds, bx
5054 ;; 21/03/2019
5055 ;mov es, bx
5056
5057 ;init0:
5058 ;
5059 ;push es
5060 ;push bx ; 20/03/2019
5061 ;push init1 ; 07/04/2018
5062 ;retf ; jump to 0070h:init1
5063
5064 ;init:
5065 ;init1:
5066
5067 init2: ; 10/12/2023
5068 ; 20/12/2022
5069 ; Change INT 1Eh diskette parameters table and INT 1Eh address
5070 ; for full MSDOS compatibility.
5071
5072 ; 10/12/2023
5073 ;cli ; not necessary for INT 1Eh
5074
5075 00001BC7 8EC1 mov es, cx ; 0
5076 00001BC9 8ED9 mov ds, cx ; 0
5077
5078 00001BCB B82205 mov ax, SEC9
5079
5080 ;mov bx, 1Eh*4 ; [0078h] ; INT 1Eh vector/pointer
5081 ;mov bl, 1Eh*4
5082 ; ; INT 1Eh points to diskette parms table
5083
5084 ; check if the table is already at 0:SEC9 (0:0522h)
5085 ; (do not move the DPT if is not original ROMBIOS table)
5086
5087 ;;or [bx+2],cx [(1Eh*4)+2] ; [007Ah] ; segment
5088 ;;jnz short mov_dpt
5089
5090 ;cmp ax, [bx] ; [1Eh*4] = 0522h ?
5091 ;je short dont_mov_dpt
5092
5093 ;mov si, [bx] ; [1Eh*4]
5094 ;mov_dpt:
5095 ;mov ds, [bx+2] ; [(1Eh*4)+2] ; [007Ah] ; segment
5096 ;lds si, [bx]
5097
5098 00001BD0 C537

```

```

5094
5095
5096 ; 10/12/2023 - Retro DOS v5.0
5097 ;mov [cs:Orig_Int1Eh_Table+2], ds
5098 ;mov [cs:Orig_Int1Eh_Table], si
5099 00001BD2 89C7 mov di, ax ; SEC9
5100 00001BD4 B10B mov cl, 11
5101 ;cld
5102 00001BD6 F3A4 rep movsb
5103
5104 ; Set INT 1Eh vector/pointer to the new DPT address
5105 00001BD8 8ED9 mov ds, cx ; 0
5106 00001BDA 8907 mov [bx], ax ; SEC9 ; [007Eh] ; 1Eh*4 ; offset
5107 00001BDC 894F02 mov [bx+2], cx ; 0 ; [007Ah] ; 1Eh*4+2 ; segment
5108 ;dont_mov_dpt:
5109
5110 ; 20/12/2022 - Retro DOS v4.0
5111 %if 0
5112 ; 27/12/2018 - Retro DOS v4.0
5113 ; 'Starting MS-DOS...' message
5114 ; (MSDOS 6.21, IO.SYS segment: 423h, Offset: 5673h)
5115 ; (0070h:96A3h)
5116
5117 mov si, SYSINIT_START+StartMsg ; 18/03/2019
5118 mov ah, 0Eh
5119 ;bh = 0
5120 mov bl, 7 ; "normal" attribute and page
5121 startmsg_nxt_chr:
5122 lodsb
5123 or al, al
5124 jz short startmsg_ok
5125
5126 int 10h ; video write
5127 jmp short startmsg_nxt_chr
5128
5129 ;flp_drvs: db 0 ; 27/12/2018 - Retro DOS v4.0
5130
5131 startmsg_ok:
5132
5133 %endif
5134
5135 ;-----
5136 ; initialization - stage 2
5137 ;-----
5138 ; 20/12/2022 - Retro DOS v4.0 (combined kernel)
5139
5140
5141 ; 19/03/2018
5142 ; Retro DOS v2.0 (24/02/2018)
5143 ; [REF: MSDOS 3.3, MSBIO, "MSINIT.ASM" (24/07/1987)]
5144
5145 ;-----
5146 ;
5147 ; System initialization
5148 ;
5149 ; The entry conditions are established by the bootstrap
5150 ; loader and are considered unknown. The following jobs
5151 ; will be performed by this module:
5152 ;
5153 ; 1. All device initialization is performed
5154 ; 2. A local stack is set up and DS:SI are set
5155 ; to point to an initialization table. Then
5156 ; an inter-segment call is made to the first
5157 ; byte of the dos
5158 ; 3. Once the dos returns from this call the ds
5159 ; register has been set up to point to the start
5160 ; of free memory. The initialization will then
5161 ; load the command program into this area
5162 ; beginning at 100 hex and transfer control to
5163 ; this program.
5164 ;
5165 ;-----
5166
5167 ; 20/12/2022
5168 ;-----
5169 ; Registers
5170 ;-----
5171 ; DL = [bsDriveNumber]
5172 ; DH = [bsMedia]
5173 ; DS = 0, ES = 0, SS = 0
5174 ; BP = 7C00h
5175 ; SP = 700h
5176 ;-----
5177 ; CX = 0
5178
5179 ; 02/10/2022 - 20/12/2022
5180 ;-----
5181 ; Note: Retro DOS v4.0 Kernel does not use/contain MSLOAD part of IO.SYS (5.0)
5182 ; Because, Retro DOS v2 boot sector loads complete/entire MSDOS.SYS
5183 ; (RETRODOS.SYS) kernel file (IO.SYS & MSDOS.SYS together).
5184 ; As result of boot sector ve init differences, Retro DOS init code (here)
5185 ; moves kernel to segment 70h at first, then sets diskette parameters
5186 ; at segment 50h (while MSDOS 5.0 boot sector and then MSLOAD sets this).
5187 ;-----
5188
5189 ; msload will check the extended boot record and set ax, bx accordingly.
5190 ;
5191 ; msload passes a 32 bit sector number hi word in ax and low in bx
5192 ; save this in cs:bios_h and cs:bios_l. this is for the start of
5193 ; data sector of the bios.
5194 ;
5195 mov [cs:bios_h], ax ; (start of) dos bios (IO.SYS) data sector
5196 mov [cs:bios_l], bx
5197
5198 ; with the following information from msload, we don't need the
5199 ; boot sector any more.-> this will solve the problem of 29 kb size
5200 ; limitation of msbio.com file.
5201
5202 ; 10/12/2023
5203 ; 21/12/2022
5204 ;cli
5205
5206 00001BDF 0E push cs ; Save a peck of interrupt vectors...
5207 00001BE0 07 pop es
5208 ;push cx
5209 ;push di
5210
5211 ; 20/12/2022
5212 00001BE1 B105 mov cl, 5
5213 ;mov cx, 5 ; NUMROMVECTORS
5214 ; no. of rom vectors to be saved
5215 ;mov si, offset RomVectors ; point to list of int vectors
5216 00001BE3 BE[0001] mov si, RomVectors
5217

```

```

5218 ; 10/12/2023
5219 00001BE6 FA cli
5220 next_int_:
5221 cs ; 16/10/2022
5222 lodsb
5223 ;lods byte ptr cs:[si] ; cs lodsb
5224 00001BE9 98 cbw ; ax = interrupt number
5225 00001BEA D1E0 shl ax, 1
5226 00001BEC D1E0 shl ax, 1 ; int no * 4
5227 00001BEE 89C7 mov di, ax ; interrupt vector address
5228 00001BF0 87F7 xchg si, di ; rombios interrupt vector address in si
5229 ; saving address in di
5230 ;lodsw ; movsw
5231 ;stosw
5232 ;lodsw ; movsw
5233 ;stosw ; save the vector
5234 ; 20/12/2022
5235 00001BF2 A5 movsw
5236 00001BF3 A5 movsw
5237
5238 00001BF4 87F7 xchg si, di
5239 00001BF6 E2EF loop next_int_
5240
5241 ;pop di
5242 ;pop cx
5243
5244 ; we need to save int13 in two places in case we are running on an at.
5245 ; on ats we install the ibm supplied rom_bios patch which hooks
5246 ; int13 ahead of orig13. since int19 must unhook int13 to point to the
5247 ; rom int13 routine, we must have that rom address also stored away.
5248
5249 ; 20/12/2022
5250 mov ax, [cs:old13] ; save old13 in orig13 also
5251 mov [cs:orig13], ax
5252 mov ax, [cs:old13+2]
5253 mov [cs:orig13+2], ax
5254
5255 ; 10/12/2023
5256 cli
5257
5258 ; 16/10/2022
5259 00001BF8 C7064C00[ED06] mov word [13h*4], block13
5260 ;mov word ptr ds:4Ch, offset block13; 13h*4
5261 ; set up int 13 for newaction
5262 00001BFE 8C0E4E00 mov [13h*4+2], cs
5263 ;mov word ptr ds:4Eh, cs ; 13h*4+2
5264 00001C02 C7065400[9907] mov word [15h*4], int15
5265 ;mov word ptr ds:54h, offset int15 ; 15h*4
5266 ; set up int 15 for newaction
5267 00001C08 8C0E5600 mov [15h*4+2], cs
5268 ;mov word ptr ds:56h, cs ; 15h*4+2
5269 00001C0C C7066400[5907] mov word [19h*4], int19
5270 ;mov word ptr ds:64h, offset int19 ; 19h*4
5271 ; set up int 19 for newaction
5272 00001C12 8C0E6600 mov [19h*4+2], cs
5273 ;mov word ptr ds:66h, cs ; 19h*4+2
5274
5275 ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
5276 00001C16 A16800 mov ax, [68h] ; 1Ah*4
5277 00001C19 8B3E6A00 mov di, [6Ah] ; 1Ah*4+2
5278 00001C1D C7066800[AF06] mov word [68h], int1A
5279 00001C23 8C0E6A00 mov [6Ah], cs
5280
5281 ; 20/12/2022
5282 00001C27 0E push cs
5283 00001C28 1F pop ds
5284
5285 ; 10/12/2023
5286 00001C29 A3[AB06] mov [orig1A], ax
5287 00001C2C 893E[AD06] mov [orig1A+2], di
5288
5289 00001C30 A1[0601] mov ax, [old13] ; save old13 in orig13 also
5290 00001C33 A3[B400] mov [orig13], ax
5291 00001C36 A1[0801] mov ax, [old13+2]
5292 00001C39 A3[B600] mov [orig13+2], ax
5293 ; ;
5294 00001C3C FB sti
5295 00001C3D CD11 int 11h ; EQUIPMENT DETERMINATION
5296 ; Return: AX = equipment flag bits
5297
5298 ; 10/12/2023
5299 jmp short chk_fd_count
5300 ; (PCDOS 7.1 IBMBIO.COM, BIOSDATA:1DF7h) ; *!*
5301 ; ((signature))
5302 ;push dx ; 52h ; 'R'
5303 ;push ax ; 50h ; 'P'
5304 ;push bx ; 53h ; 'S'
5305
5306 ; we have to support a system that does not have any diskette
5307 ; drives but only hardfiles. this system will ip1 from the hardfile.
5308 ; if the equipment flag bit 0 is 1, then the system has diskette drive(s).
5309 ; otherwise, the system has only hardfiles.
5310 ;
5311 ; important thing is that still, for compatibility reason, the drive letter
5312 ; for the hardfiles start from "c". so, we still need to allocate dummy bds
5313 ; drive a and drive b. at sysinit time, we are going to set cds table entry
5314 ; of dpb pointer for these drives to 0, so any user attempt to access this
5315 ; drives will get "invalid drive letter ..." message. we are going to
5316 ; establish "fakefloppydrv" flag. ***sysinit module should call int 11h to
5317 ; determine whether there are any diskette drivers in the system or not.!!!***
5318
5319 ; check the register returned by the equipment determination interrupt
5320 ; we have to handle the case of no diskettes in the system by faking
5321 ; two dummy drives.
5322 ;
5323 ; if the register indicates that we do have floppy drives we don't need
5324 ; to do anything special.
5325 ;
5326 ; if the register indicates that we don't have any floppy drives then
5327 ; what we need to do is set the fakefloppydrv variable, change the
5328 ; register to say that we do have floppy drives and then go to execute
5329 ; the code which starts at notsingle. this is because we can skip the
5330 ; code given below which tries to find if there are one or two drives
5331 ; since we already know about this.
5332 chk_fd_count: ; 10/12/2023
5333 ;or ax, 1 ; *!*
5334
5335 ; 06/05/2019 - Retro DOS v4.0
5336 00001C3F 88C1 mov cl, al
5337
5338 ; 12/12/2022
5339 00001C41 A801 test al, 1
5340 ;test ax, 1 ; floppy drives present?
5341 00001C43 751E jnz short normalfloppydrv ; yes.

```

```

5342
5343 ; Some ROM BIOSs lie that there are no floppy drives. Lets find out
5344 ; whether it is an old ROM BIOS or a new one
5345 ;
5346 ; WARNING !!!
5347 ;
5348 ; This sequence of code is present in SYSINIT1.ASM also. Any modification
5349 ; here will require an equivalent modification in SYSINIT1.ASM also
5350
5351 ; 20/12/2022
5352 ;push ax
5353 ;push bx
5354 ;push cx
5355 00001C45 52 push dx
5356 ;push di
5357 00001C46 06 push es
5358
5359 00001C47 B408 mov ah, 8
5360 00001C49 B200 mov dl, 0
5361 00001C4B CD13 int 13h ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
5362 ; DL = drive number
5363 ; Return: CF set on error, AH = status code, BL = drivetype
5364 ; DL = number of consecutive drives
5365 ; DH = maximum value for head number, ES:DI -> drive parameter
5366 00001C4D 7202 jc short _gdskp_error
5367 ;mov [cs:flp_drvs], dl
5368 ; 20/12/2022
5369 ; ds = cs
5370 ;mov [flp_drvs], dl
5371 00001C4F 88D1 mov cl, dl
5372 _gdskp_error:
5373 ; 20/12/2022
5374 00001C51 07 pop es
5375 ;pop di
5376 00001C52 5A pop dx
5377 ;pop cx
5378 ;pop bx
5379 ;pop ax
5380
5381 00001C53 720E jc short normalfloppydrv
5382 ; if error it is an old ROM BIOS
5383 ; so, lets assume that ROM BIOS lied
5384 ; 20/12/2022
5385 ; ds = cs
5386 ;cmp byte [flp_drvs], 0
5387 ;cmp byte [cs:flp_drvs], 0 ; number of drvs == 0?
5388 ;jz short _set_fake_flpdrv
5389 ;mov al, [cs:flp_drvs]
5390 ;mov al, [flp_drvs]
5391 ;dec al ; make it zero based
5392 ; 18/12/2022
5393 ;dec ax
5394 ;jmp short got_num_flp_drvs
5395
5396 ; 20/12/2022
5397 00001C55 08C9 or cl, cl ; [flp_drvs]
5398 00001C57 7403 jz short _set_fake_flpdrv
5399 00001C59 49 dec cx
5400 00001C5A EB0B jmp short got_num_flp_drvs
5401 ; -----
5402
5403 _set_fake_flpdrv:
5404 ; 20/12/2022
5405 ; ds = cs
5406 ;inc cl ; cl = 1
5407 ; 10/12/2023
5408 00001C5C 41 inc cx ; cl = 1
5409 00001C5D 880E[131A] mov [fakefloppydrv], cl ; 1
5410 ;mov byte [fakefloppydrv], 1
5411 ;mov byte [cs:fakefloppydrv], 1
5412 ; we don't have any floppy drives.
5413 ; 20/12/2022
5414 ;mov ax, 1
5415 00001C61 EB0A jmp short settwodrive ; well then set it for two drives!
5416 ; -----
5417
5418 normalfloppydrv: ; yes, bit 0 is 1.
5419 ; 20/12/2022
5420 ;rol al, 1 ; there exist floppy drives.
5421 ;rol al, 1 ; put bits 6 & 7 into bits 0 & 1
5422 00001C63 D0C1 rol cl, 1
5423 00001C65 D0C1 rol cl, 1
5424
5425 got_num_flp_drvs:
5426 ;and ax, 3 ; only look at bits 0 & 1
5427 ; 18/12/2022
5428 ;and al, 3
5429 00001C67 80E103 and cl, 3
5430 00001C6A 7505 jnz short notsingle ; zero means single drive system
5431 ; 20/12/2022
5432 00001C6C 41 inc cx
5433 ;inc ax ; pretend it's a two drive system
5434 ; set this to two fakedrives
5435 settwodrive:
5436 ; 20/12/2022
5437 ; ds = cs
5438 00001C6D FE06[7800] inc byte [single]
5439 ;inc byte [cs:single] ; remember this
5440 notsingle:
5441 ; 20/12/2022
5442 ;inc ax ; ax has number of drives, 2-4
5443 ; ; is also 0 indexed boot drive if we
5444 ;booted off hard file
5445 ;mov cl, al ; ch is fat id, cl # floppies
5446 ; 20/12/2022
5447 ;inc cl ; cl >= 2
5448 ; 10/12/2023
5449 00001C71 41 inc cx ; cl >= 2
5450
5451 ; 16/10/2022
5452 ; MSDOS 3.3 - "MSEQU.INC" (24/07/1987)
5453 INITSPOT EQU 534h ; IBM wants 4 zeros here
5454 BRKADR EQU 1BH * 4 ; 6CH, 1BH break vector address
5455 TIMADR EQU 1CH * 4 ; 70H, 1CH timer interrupt
5456 DSKADR EQU 1EH * 4 ; address of ptr to disk parameters
5457 SEC9EQU 522h ; address of disk parameters
5458 CHROUT EQU 29h
5459 LSTDRV EQU 504h
5460
5461 ; determine whether we booted from floppy or hard disk...
5462
5463 ; 20/12/2022
5464 00001C72 88C8 mov al, cl ; 26/05/2019
5465

```



```

5466 00001C74 F6C280      test    dl, 80h      ; boot from floppy ?
5467 00001C77 7502      jnz     short gothrd ; no.
5468 00001C79 31C0      xor     ax, ax      ; indicate boot      from drive a
5469                      ; 10/12/2023
5470                      ;mov    [Boot_Drv], al
5471 gothrd:
5472
5473 ; MSDOS 6.0
5474 ; ax = 0-based drive we booted from
5475 ; bios_l, bios_h set.
5476 ; cl = number of floppies including fake one
5477 ; ch = media byte
5478
5479 ; Retro DOS 4.0 - 27/12/2018
5480 ; (from Retro DOS v2.0 boot sector)
5481 ; dl = int 13 drive number we booted from
5482 ; dh = media byte
5483
5484                      ; 20/12/2022
5485 00001C7B 88F5      mov     ch, dh      ; 01/07/2018
5486
5487                      ; cl = number of floppies
5488                      ; ch = media byte
5489
5490                      ; set up local stack
5491
5492                      ; 20/12/2022
5493 ;xor     dx, dx      ; ax = 0-based drive we      booted from
5494                      ; bios_l, bios_h set.
5495                      ; cl = number of floppies including fake one
5496                      ; ch = media byte
5497
5498                      ; 20/12/2022
5499                      ; es = ds = cs
5500                      ; ss = 0
5501                      ; sp = 700h
5502
5503                      ; 20/12/2022
5504 ;cli
5505 ;mov     ss, dx      ; set stack segment and stack pointer
5506 ;mov     sp, 700h
5507 ;sti
5508 00001C7D 51      push    cx ; (***)      ; save number of floppies and media byte
5509
5510 00001C7E 88EC      mov     ah, ch      ; FAT ID to AH
5511 00001C80 50      push    ax ; (***)      ; save boot drive number and media byte
5512
5513 ; let model_byte, secondary_model_byte be set here!!!
5514
5515 00001C81 84C0      mov     ah, 0C0h
5516 00001C83 CD15      int     15h      ; SYSTEM - GET CONFIGURATION (XT after 1/10/86, AT mdl 3x9, CONV, XT286, PS)
5517 00001C85 7215      jb     short no_rom_system_conf ; just      use Model_Byte
5518 00001C87 80FC00     cmp     ah, 0
5519 00001C8A 7510      jnz     short no_rom_system_conf
5520
5521 ;
5522 ; 20/12/2022
5523 ; (Programmer's Guide to the AMIBIOS, page 268)
5524 ; (https://stanislavs.org/helppc/int_15-c0.html)
5525 ;
5526 ;
5527 ; INT 15h, ah = C0h - Return System Configuration Parameters (PS/2 only)
5528 ;
5529 ; on return:
5530 ; CF = 0 if successful
5531 ;       = 1 if error
5532 ; AH = when CF set, 80h for PC & PCjr, 86h for XT
5533 ;       (BIOS after 11/8/82) and AT (BIOS after 1/10/84)
5534 ;
5535 ; ES:BX = pointer to system descriptor table in ROM of the format:
5536 ;
5537 ; Offset Size      Description
5538 ; 00 word length of descriptor (8 minimum)
5539 ; 02 byte model byte (same as F000:FFFE, not reliable)
5540 ; 03 byte secondary model byte
5541 ; 04 byte BIOS revision level (zero based)
5542 ; 05 byte feature information, see below
5543 ; 06 dword reserved
5544 ;
5545 ; 20/12/2022
5546 ; ds = cs
5547 00001C8C 268A4702     mov     al, [es:bx+2] ; [es:bx+ROMBIOS_DESC.bios_sd_modelbyte]
5548 00001C90 A2[AF05]     mov     [model_byte], al
5549 ;mov     [cs:model_byte], al
5550 00001C93 268A4703     mov     al, [es:bx+3] ; [es:bx+ROMBIOS_DESC.bios_sd_scnd_modelbyte]
5551 00001C97 A2[B005]     mov     [secondary_model_byte], al
5552 ;mov     [cs:secondary_model_byte], al
5553 ;mov     [cs:secondary_model_byte], al ; get/save secondary model byte
5554 00001C9A EB0C      jmp     short turn_timer_on
5555 ;-----
5556
5557 no_rom_system_conf:
5558 00001C9C BEFFFF     mov     si, 0FFFFh
5559 00001C9F 8EC6      mov     es, si
5560 ; 20/12/2022
5561 00001CA1 26A00E00     mov     al, [es:0Eh] ; get model byte (from 0FFFFh:0Eh)
5562 00001CA5 A2[AF05]     mov     [model_byte], al
5563 ;mov     [cs:model_byte], al ; save model byte
5564
5565 00001CA8 B020      turn_timer_on:
5566 00001CAA E620      mov     al, 20h ; ' ' ; turn on the timer
5567                      out     20h, al ; Interrupt controller,      8259A.
5568                      ; AKPORT
5569
5570 ; some Olivetti m24 machines have an 8530 serial communications
5571 ; chip installed at io address 50h and 52h. if we're running
5572 ; on one of those, we must inhibit the normal aux port initialization
5573
5574 ; 20/12/2022
5575 00001CAC 803E[AF05]00     ; ds = cs
5576 cmp     byte [model_byte], 0
5577 ;cmp     byte [cs:model_byte], 0 ; next to last      byte in rom bios
5578 jnz     short not_olivetti_m24 ; skip for all other machines
5579                      ; (except Olivetti m24)
5580 in     al, 66h      ; is 8530 installed?
5581 test    al, 20h
5582 jz     short not_olivetti_m24 ; we're done if not
5583 mov     al, 0Fh      ; double check
5584 out     50h, al
5585 in     al, 50h
5586 test    al, 1      ; this test was      copied from Olivetti
5587 jz     short skip_aux_port_init ; take      this branch if 8530 installed
5588
5589 not_olivetti_m24:
5590 00001CC3 B003      mov     al, 3      ; init com4

```

```

5590 00001CC5 E8DB09      call    aux_init
5591 00001CC8 B002      mov     al, 2           ; init com3
5592 00001CCA E8D609      call    aux_init
5593 00001CCD B001      mov     al, 1           ; init com2
5594 00001CCF E8D109      call    aux_init
5595 00001CD2 30C0      xor     al, al           ; init com1
5596 00001CD4 E8CC09      call    aux_init
5597
5598 skip_aux_port_init:
5599 00001CD7 B002      mov     al, 2           ; init lpt3
5600 00001CD9 E8BF09      call    print_init
5601 00001CDC B001      mov     al, 1           ; init lpt2
5602 00001CDE E8BA09      call    print_init
5603 00001CE1 30C0      xor     al, al           ; init lpt1
5604 00001CE3 E8B509      call    print_init
5605
5606      ; 11/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
5607      ;mov     di, 534h      ; offset INITSPOT
5608      ;mov     di, INITSPOT ; 0534h
5609      ;        ; IBMDOS.COM's first cluster - high word
5610      ;        ; 520h (the 2nd entry of root dir) + 14h
5611      ;mov     ax, [di]
5612      ;mov     [firstcluster_hw], ax
5613
5614 00001CE6 31D2      xor     dx, dx ; 0
5615 00001CE8 8EDA      mov     ds, dx          ; to initialize      print screen vector
5616 00001CEA 8EC2      mov     es, dx
5617 00001CEC 31C0      xor     ax, ax
5618      ; 16/10/2022
5619 00001CEE BF3405      mov     di, INITSPOT    ; 0534h
5620      ;mov     di, 534h      ; INITSPOT (0000h:0534h)
5621      ;        ; IBM wants 4 zeros here
5622 00001CF1 AB      stosw
5623 00001CF2 AB      stosw
5624 00001CF3 8CC8      mov     ax, cs          ; fetch segment
5625 00001CF5 C7066C00[0E06] mov     word [BRKADR], cbreak
5626      ;mov     word ptr ds:6Ch, offset      cbreak ; [BRKADR]
5627      ;        ; breakentry point
5628 00001CFB A3E00      mov     [BRKADR+2], ax
5629      ;mov     ds:6Eh, ax      ; vector for break
5630 00001CFE C706A400[8206] mov     word [CHROUT*4], outchr
5631      ;mov     word ptr ds:0A4h, offset outchr ; [CHROUT*4]
5632 00001D04 A3A600      mov     [CHROUT*4+2], ax
5633      ;mov     ds:0A6h, ax      ; [CHROUT*4+2]
5634
5635 00001D07 BF0400      mov     di, 4
5636 00001D0A BB[1406]    mov     bx, intret ; 19/10/2022
5637      ;mov     bx, offset intret ; intret (cs:intret)
5638      ;        ; will initialize rest of interrupts
5639 00001D0D 93      xchg     ax, bx
5640 00001D0E AB      stosw
5641 00001D0F 93      xchg     ax, bx          ; location 4
5642 00001D10 AB      stosw
5643 00001D11 83C704      add     di, 4
5644 00001D14 93      xchg     ax, bx          ; cs:
5645 00001D15 AB      stosw
5646 00001D16 93      xchg     ax, bx          ; int 1; location 6
5647 00001D17 AB      stosw
5648 00001D18 93      xchg     ax, bx          ; location 12
5649 00001D19 AB      stosw
5650 00001D1A 93      xchg     ax, bx          ; cs:
5651 00001D1B AB      stosw
5652      ;        ; int 3; location 14
5653      ;        ; location 16
5654      ;        ; cs:
5655      ;        ; int 4; location 18
5656      ;
5657      ; 20/12/2022
5658      ; (https://stanislavs.org/helppc/bios_data_area.html)
5659      ; Address Size Description (BIOS/DOS Data Area)
5660      ;
5661      ; 50:00 byte Print screen status byte
5662      ; 00 = PrtSc not active,
5663      ; 01 = PrtSc in progress
5664      ; FF = error
5665      ;
5666      ; 50:01 3 bytes Used by BASIC
5667      ; 50:04 byte DOS single diskette mode flag, 0=A:, 1=B:
5668      ; 50:05 10bytes POST work area
5669      ; 50:0F byte BASIC shell flag; set to 2 if current shell
5670      ; 50:10 word BASICs default DS value (DEF SEG)
5671      ; 50:12 dword Pointer to BASIC INT 1C interrupt handler
5672      ; 50:16 dword Pointer to BASIC INT 23 interrupt handler
5673      ; 50:1A dword Pointer to BASIC INT 24 disk error handler
5674      ; 50:20 word DOS dynamic storage
5675      ; 50:22 14bytes DOS diskette initialization table (INT 1E)
5676      ; 50:30 4bytes MODE command
5677      ; 70:00 I/O drivers from IO.SYS/IBMBIO.COM
5678
5679 00001D1C 89160005      mov     [0500h], dx ; 0
5680      ;mov     ds:500h, dx ; set print screen & break = 0
5681 00001D20 89160405      mov     [LSTDRV], dx ; [0504h]
5682      ;mov     ds:504h, dx ; cleanout last drive spec
5683
5684      ; we need to initialize the cs:motorstartup variable from the disk
5685      ; parameter table at sec9. the offsets in this table are defined in
5686      ; the disk_parms struc in msdkprm.inc. 2 locs
5687
5688 00001D24 A02C05      mov     al, [SEC9+0Ah] ; 16/10/2022
5689      ;mov     al, ds:52Ch ; [SEC9+DISK_PARMS.DISK_MOTOR_STRT]
5690      ;        ; [522h+0Ah]
5691      ; 20/12/2022
5692      ; ds = 0
5693
5694 00001D27 2EA2[2601]    mov     [cs:motorstartup], al
5695 00001D2B 2E803E[AF05]FD cmp     byte [cs:model_byte], 0FDh ; is this an old rom?
5696 00001D31 720B      jnb     short no_diddle ; no
5697 00001D33 C7062B050F02 mov     word [SEC9+09h], 20Fh
5698      ;mov     word ptr ds:52Bh, 20Fh ; [SEC9+DISK_PARMS.DISK_HEAD_STTL], 0200h+NORMSETTLE
5699      ;        ; set head settle and motor start on pc-1 pc-2 pc-xt hal0
5700 00001D39 C6062205DF mov     byte [SEC9+0], 0DFh
5701      ;mov     byte ptr ds:522h, 0DFh ; [SEC9+DISK_PARMS.DISK_SPECIFY_1]
5702      ;        ; set 1st specify byte on pc-1pc-2 pc-xt hal0
5703 no_diddle:
5704      int     12h          ; MEMORY SIZE -
5705      ; Return: AX = number of contiguous 1K blocks of memory
5706 00001D40 B106      mov     cl, 6
5707 00001D42 D3E0      shl     ax, cl          ; convert memory size to 16-byte blocks (segment no.)
5708
5709      ; 20/12/2022
5710      ; 03/07/2018 - 27/12/2018
5711      ;pop     cx ; (**)
5712      ;mov     [cs:drvfat], cx
5713 00001D44 50      push    ax ; (*) ; save real top of memory
5714
5715      ; 27/12/2018 - (MSDOS 6.0, 6.21)
5716
5717 ;M068 - BEGIN

```



```

5714 ;----- Check if an RPL program is present at TOM and do not tromp over it
5715 ;
5716 ; 20/12/2022
5717 ; ds = 0
5718
5719 ;push ds
5720 ;push bx ; pushes not required but since this
5721 ; happens to be a last minute change
5722 ; & since it is only init code.
5723
5724 ;xor bx, bx
5725 ;mov ds, bx
5726
5727 ;;mov bx, ds:0BCh ; [2Fh*4]
5728 ;mov bx, [2Fh*4]
5729 ;;mov ds, word ptr ds:0BEh ; [2Fh*4+2]
5730 ;mov ds, [2Fh*4+2]
5731 ; 29/09/2023
5732 00001D45 C51EBC00 1ds bx, [2Fh*4]
5733
5734 00001D49 817F035250 cmp word [bx+3], 'RP' ; 'RPL'
5735 00001D4E 750F ;cmp word ptr [bx+3], 'PR' ; 'RPL'
5736 00001D50 807F054C jnz short SkipRPL
5737 cmp byte [bx+5], 'L'
5738 00001D54 7509 ;cmp byte ptr [bx+5], 'L'
5739 00001D56 89C2 jnz short SkipRPL
5740 00001D58 B8064A mov dx, ax ; get TOM into DX
5741 00001D5B CD2F mov ax, 4A06h ; (multMULT shl 8) + multMULTRPLTOM
5742 00001D5D 89D0 int 2Fh ; Get new TOM from any RPL
5743 mov ax, dx
5744 SkipRPL: ; 20/12/2022
5745 ;pop bx
5746 ;pop ds
5747
5748 ;M068 - END
5749 ; 20/12/2022
5750 ; 27/12/2018
5751 00001D5F 0E push cs
5752 00001D60 1F pop ds
5753
5754 ; 18/03/2019 - Retro DOS v4.0
5755 ;sub ax, 64 ; room for fatloc segment. (1 kb buffer)
5756 ;mov [cs:fatloc], ax ; location to read fat
5757
5758 ; 01/07/2018
5759 ; 08/04/2018
5760 ; 28/03/2018
5761 ; MSDOS 6.0 - MSINIT.ASM, 1991
5762 00001D61 83E840 sub ax, 64
5763 00001D64 A3[061A] mov [init_bootseg], ax ; 20/12/2022
5764 ;mov [cs:init_bootseg], ax
5765
5766 ; 27/12/2018 - Retro DOS v4.0
5767 ;;pop ax ; (*) ; get back real top of memory
5768 ;pop dx ; (*)
5769 ; 29/09/2023 - Retro DOS v4.2 (BugFix)
5770 00001D67 58 pop ax ; (*) ; get back real top of memory
5771
5772 ; 20/12/2022
5773 ; 27/12/2018
5774 00001D68 59 pop cx ; (**)
5775 00001D69 890E[FC19] mov [drvfat], cx ; save drive to load dos, and fat id
5776
5777 ; 20/12/2022
5778
5779 ;mov dx, 46Dh ; SYSINIT segment
5780 ;mov dx, 544h ; 10/12/2023 (PCDOS 7.1 IBMBIO.COM)
5781 00001D6D BAD904 mov dx, SYSINITSEG ; 17/10/2022
5782 00001D70 8EDA mov ds, dx
5783
5784 ; set pointer to resident device driver chain
5785 ; 17/10/2022
5786 mov word [DEVICELIST], res_dev_list
5787 ;mov word [273h], res_dev_list
5788 00001D72 C706[7502][2300] ;mov word ptr ds:273h, offset res_dev_list
5789 ; ; [SYSINIT+DEVICE_LIST]
5790 ;mov [DEVICELIST+2], cs
5791 ;mov [275h], cs
5792 00001D78 8C0E[7702] ;mov word ptr ds:275h, cs ; [SYSINIT+DEVICE_LIST+2]
5793
5794 ;mov [MEMORYSIZE], ax
5795 ;mov [292h], ax
5796 00001D7C A3[9402] ;mov ds:292h, ax ; [SYSINIT+MEMORY_SIZE]
5797
5798 inc cl
5799 mov [DEFAULTDRIVE], cl
5800 00001D7F FEC1 ;mov [296h], cl
5801 00001D81 880E[9802] ;mov ds:296h, cl ; [SYSINIT+DEFAULT_DRIVE]
5802
5803 ;mov word [CURRENTDOSLOCATION], 0AF8h ; 10/12/2023
5804 00001D85 C706[7302]1A0A mov word [CURRENTDOSLOCATION], DOSLOADSEG
5805 ;mov word [271h], 83Fh ; (MSDOS.SYS segment)
5806 ;mov word ptr ds:271h, 83Fh ; [SYSINIT+CURRENT_DOS_LOCATION]
5807 ; dos_load_seg
5808
5809 ; important: some old ibm hardware generates spurious int 0F's due to bogus
5810 ; printer cards. we initialize this value to point to an iret only if
5811 ;
5812 ; 1) the original segment points to storage inside valid ram.
5813 ;
5814 ; 2) the original segment is 0F000:xxxx
5815
5816 ;;mov ax, 46Dh ; SYSINIT segment
5817 ;;mov ax, 544h ; 10/12/2023
5818 ;mov ax, SYSINITSEG ; 17/10/2022
5819 ;mov es, ax
5820 ; 20/12/2022
5821 ;push ds ; SYSINITSEG
5822 ;pop es
5823 mov es, dx ; SYSINITSEG
5824 xor ax, ax ; 0
5825 00001D8B 8EC2 mov ds, ax ; segment 0
5826 00001D8D 31C0 ;mov ax, ds:3Eh ; [0Fh*4+2]
5827 00001D8F 8ED8 mov ax, [0Fh*4+2] ; segment for INT 0Fh
5828
5829 00001D91 A13E00 ; 18/10/2022
5830 cmp ax, [es:MEMORYSIZE] ; es:292h
5831 00001D94 263B06[9402] ;cmp ax, es:292h ; [ES:MEMORY_SIZE] ; (condition 1)
5832 jbe short resetintf
5833 00001D99 7605 cmp ax, 0F000h ; (condition 2)
5834 00001D9B 3D00F0 jnz short keepintf
5835 00001D9E 750A
5836 resetintf:
5837 00001DA0 C7063C00[1406] mov word [0Fh*4], intret

```

```

5838 ;mov word ptr ds:3Ch, offset intret ; [0Fh*4]
5839 00001DA6 8C0E3E00 mov word [0Fh*4+2], cs
5840 ;mov word ptr ds:3Eh, cs ; [0Fh*4+2]
5841 keepintf:
5842 ; end important
5843
5844 ; 17/10/2022
5845 ; 28/12/2018 - Retro DOS v4.0
5846
5847 ; (MSDOS 6.0, MSINIT.ASM, 1991)
5848 ;
5849 ; we will check if the system has ibm extended keyboard by
5850 ; looking at a byte at 40:96. if bit 4 is set, then extended keyboard
5851 ; is installed, and we are going to set keyrd_func to 10h, keysts_func to 11h
5852 ; for the extended keyboard function. use cx as the temporary register.
5853
5854 ; 20/12/2022
5855 ; ds = 0
5856 ;xor cx, cx
5857 ;mov ds, cx
5858
5859 00001DAA 8A0E9604 mov cl, [496h] ; get keyboard flag
5860
5861 ; 20/12/2022
5862 ; 20/03/2019
5863 00001DAE 0E push cs
5864 00001DAF 1F pop ds
5865
5866 ;test cl, 00010000b ; 10h
5867 00001DB0 F6C110 test cl, 10h ; extended keyboard ?
5868 00001DB3 740A jz short org_key ; no, original keyboard
5869
5870 ; 20/12/2022
5871 ; ds = cs
5872 00001DB5 C606[7E04]10 mov byte [keyrd_func], 10h ; extended keyboard
5873 00001DBA C606[7F04]11 mov byte [keysts_func], 11h
5874 ;mov byte [cs:keyrd_func], 10h ; extended keyboard
5875 ;mov byte [cs:keysts_func], 11h
5876 ; change for extended keyboard functions
5877 org_key:
5878
5879 ; 02/06/2018 - Retro DOS v3.0
5880
5881 ;*****
5882 ; will initialize the number of drives
5883 ; after the equipment call (int 11h) bits 6&7 will tell
5884 ; the indications are as follows:
5885 ;
5886 ; bits 7 6 drives
5887 ; 0 0 1
5888 ; 0 1 2
5889 ; 1 0 3
5890 ; 1 1 4
5891 ;*****
5892
5893 ; 20/12/2022
5894 ; ds = cs
5895 ;push cs
5896 ;pop ds
5897 ; 21/12/2022
5898 ;push cs
5899 ;pop es
5900
5901 00001DBF E8C20A call cmos_clock_read ; If cmos clock exists,
5902 ; then set the system time according to that.
5903 ; also, reset the cmos clock rate.
5904
5905 ; 18/10/2022
5906 ;mov word ptr BData_start, offset harddrv ;
5907 ; set up pointer to hdrive
5908 00001DC2 C706[0000][4B08] mov word [hdrv_pat], harddrv
5909
5910 ; 20/12/2022
5911 ; 28/12/2018 - Retro DOS v4.0 (MSDOS 6.21)
5912 00001DC8 58 pop ax ; (***) ; number of floppies and FAT ID
5913
5914 00001DC9 30E4 xor ah, ah ; chuck fat id byte
5915 00001DCB A2[7500] mov [drvmax], al ; remember which drive is hard disk
5916 00001DCE A2[2501] mov [dsktnum], al ; and set initial number of drives
5917 00001DD1 D1E0 shl ax, 1
5918 00001DD3 0106[601A] add [last_dskdrv_table], ax
5919
5920 ; 10/12/2023 - Retro DOS v5.0 IO.SYS (IBMBIO.COM)
5921 ; ((MSDOS 6.22 IO.SYS & PCDOS 7.1 IBMBIO.COM))
5922 ; .....
5923 00001DD7 1E push ds
5924 00001DD8 B800F0 mov ax, 0F000h
5925 00001ddb 8ED8 mov ds, ax
5926
5927 00001DDD 813EEAFF434F cmp word [0FFEAh], 'CO' ; 'COMPAQ'
5928 00001DE3 751F jne short skip_mode2
5929 00001DE5 813EECF4D50 cmp word [0FFECCh], 'MP'
5930 00001DEB 7517 jne short skip_mode2
5931 00001DED 813EEEF4151 cmp word [0FFEEh], 'AQ'
5932 00001DF3 750F jne short skip_mode2
5933
5934 00001DF5 B800E4 mov ax, 0E400h ; get advanced system info (COMPAQ ROMBIOS)
5935 00001DF8 CD15 int 15h
5936 00001DFA 7208 jc short skip_mode2
5937 ; 10/12/2023
5938 ; PCDOS 7.1 IBMBIO.COM
5939 ; or bx, 0 ; or bx,40h ; enable mode 2
5940 ; (MSDOS 6.0)
5941 ; MSDOS 6.22 IO.SYS
5942 00001DFC 83CB40 or bx, 40h ; enable mode 2 (dual harddisk controller)
5943 00001DFF B880E4 mov ax, 0E480h ; set advanced system info (COMPAQ ROMBIOS)
5944 00001E02 CD15 int 15h
5945 skip_mode2:
5946 00001E04 1F pop ds
5947 ; .....
5948
5949 00001E05 B280 mov dl, 80h
5950 00001E07 B408 mov ah, 8
5951 00001E09 CD13 int 13h ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
5952 ; DL = drive number
5953 ; Return: CF set on error, AH = status code, BL = drivetype
5954 ; DL = number of consecutive drives
5955 ; DH = maximum value for head number, ES:DI -> drive parameter
5956 00001E0B 7204 jc short enddrv
5957 00001E0D 8816[5F1A] mov [hnum], dl ; save number of hard disk drives
5958 enddrv:
5959 ; 21/12/2022
5960 00001E11 0E push cs
5961 00001E12 07 pop es

```

```

5962
5963 ; scan the list of drives to determine their type. we have three flavors of
5964 ; diskette drives:
5965 ;
5966 ; 48tpi drives we do nothing special for them
5967 ; 96tpi drives mark the fact that they have changeline support.
5968 ; 3.5" drives mark changeline support and small.
5969
5970 ; the following code uses registers for certain values:
5971 ;
5972 ; dl - physical drive
5973 ; ds:di - points to current bds
5974 ; cx - flag bits for bds
5975 ; dh - form factor for the drive (1 - 48tpi, 2 - 96tpi, 3 - 3.5" medium)
5976
5977 00001E13 30D2          xor     dl, dl
5978
5979 ; 20/12/2022
5980 ; ds = cs
5981 ; 17/06/2018
5982 ; push cs
5983 ; pop ds
5984
5985 00001E15 C606[2C01]09   mov     byte [eot], 9
5986 00001E1A BF[1901]      mov     di, start_bds ; if we are faking floppy drives we need
5987                                     ; to set aside two bdss for the two fake floppy drives
5988
5989 ; 02/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS)
5990 ; 28/12/2018 - Retro DOS v4.0 (MSDOS 6.0, MSINIT.ASM)
5991
5992 ; check to see if we are faking floppy drives. if not we don't
5993 ; do anything special. if we are faking floppy drives we need
5994 ; to set aside two bdss for the two fake floppy drives. we
5995 ; don't need to initialise any fields though. so starting at start_bds
5996 ; use the link field in the bds structure to go to the second bds
5997 ; in the list and initialise it's link field to -1 to set the end of
5998 ; the list. then jump to the routine at dohard to allocate/initialise
5999 ; the bds for harddrives.
6000
6001 00001E1D 803E[131A]01   cmp     byte [fakefloppydrv], 1
6002 00001E22 750B          jnz     short loop_drive
6003 00001E24 8B3D          mov     di, [di] ; [di+BDS.link]
6004                                     ; di <- first bds link
6005 00001E26 8B3D          mov     di, [di] ; [di+BDS.link]
6006                                     ; di <- second bds link
6007 00001E28 C705FFFF      mov     word [di], 0FFFFh ; -1 ; set end of link
6008 00001E2C E98801      jmp     dohard ; allocate/initialise bds for harddrives
6009
6010 -----
6011 loop_drive:
6012 00001E2F 3A16[7500]     cmp     dl, [drvmax]
6013 00001E33 7203          jb      short got_more
6014 00001E35 E97B01      jmp     done_drives
6015 -----
6016
6017 got_more:
6018 ; 12/12/2023
6019 ; xor cx, cx ; zero all flags
6020 00001E38 8B3D          mov     di, [di] ; [di+BDS.link]
6021                                     ; get next bds
6022 ;
6023 ; .....
6024 ; 10/12/2023 - Retro DOS v5.0
6025 ; (PCDOS 7.1 IBMBIO.COM BIOSDATA:2046h)
6026 00001E3A 83FFFF      cmp     di, 0FFFFh ; end of link ?
6027 00001E3D 7516          jne     short not_last_bds
6028 00001E3F 88D0          mov     al, dl ; drive number (0 based)
6029 00001E41 98          cbw
6030 00001E42 01C0          add     ax, ax
6031 00001E44 05[3C05]     add     ax, dskdrvs
6032 00001E47 A3[601A]      mov     [last_dskdrv_table], ax
6033 00001E4A 8B3E[621A]     mov     di, [end_of_bdss]
6034 00001E4E E8FD09      call    xinstall_bds
6035 00001E51 FE0E[7500]     dec     byte [drvmax]
6036
6037 not_last_bds:
6038 ; .....
6039
6040 00001E55 B600          mov     dh, 0 ; ff48tpi
6041                                     ; set form factor to 48 tpi
6042 00001E57 C606[101A]28   mov     byte [num_cyls], 40 ; 40 tracks per side
6043
6044 ; 20/12/2022
6045 ; push ds ; 11/05/2019
6046 ; push di
6047 ; push dx
6048 ; push cx ; not necessary (10/12/2023)
6049 ; push es ; ((*)) ; 20/12/2022
6050 ;
6051 ; .....
6052 ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
6053 00001E5F 52          xor     bx, bx
6054                                     ; dx ; dl = drive number
6055 00001E60 B408          mov     ah, 8
6056 00001E62 CD13          int     13h ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
6057                                     ; DL = drive number
6058                                     ; Return: CF set on error, AH = status code, BL = drivetype
6059                                     ; DL = number of consecutive drives
6060                                     ; DH = maximum value for head number, ES:DI -> drive parameter
6061 ; jc short noparmsfromrom
6062 ; 10/12/2023
6063 00001E64 58          pop     ax ; al = drive number
6064 00001E65 7303          jnc     short chk_drv_type
6065 00001E67 E9E600      jmp     noparmsfromrom
6066
6067 chk_drv_type:
6068 ; 10/12/2023
6069 ; ch = low eight bits of maximum cylinder number
6070 ; cl = maximum sector number (bits 5-0)
6071 ; high two bits of maximum cylinder number (bits 7-6)
6072 ;
6073 00001E6A 80FB10      cmp     bl, 10h ; ATAPI Removable Media Device
6074 00001E6D 7554          jne     short not_atapi_removable
6075
6076 ; save ds:si
6077 00001E6F 1E          push    ds
6078                                     ; not necessary (10/12/2023)
6079
6080 00001E70 88C2          mov     dl, al
6081 00001E72 83EC1A      sub     sp, 26
6082 00001E75 31C0          xor     ax, ax ; 0
6083 00001E77 50          push    ax
6084 00001E78 B81E00      mov     ax, 30
6085 00001E7B 50          push    ax

```

```

6086 00001E7C 89E6      mov     si, sp          ; DS:SI = segment:offset pointer to Result Buffer
6087 00001E7E 16        push    ss
6088 00001E7F 1F        pop     ds
6089 00001E80 B448      mov     ah, 48h
6090 00001E82 CD13      int     13h            ; DISK - IBM/MS Extension
6091                                ; GET DRIVE PARAMETERS (DL - drive, DS:SI - buffer)
6092 00001E84 7239      jb     short ext_gdp_err
6093 00001E86 8B4408    mov     ax, [si+8]      ; physical number of heads
6094 00001E89 A3[0E1A]   mov     [num_heads], ax
6095 00001E8C 8B4404    mov     ax, [si+4]      ; physical number of cylinders
6096 00001E8F A3[101A]   mov     [num_cyl], ax
6097 00001E92 8A440C    mov     al, [si+0Ch]    ; physical number of sectors per track
6098 00001E95 A2[121A]   mov     [sec_trk], al
6099 00001E98 3A06[2C01] cmp     al, [eot]
6100 00001E9C 7603      jbe     short _eotok
6101 00001E9E A2[2C01]   mov     [eot], al
6102
6103 _eotok:      ; 10/12/2023
6104            ;xor     al, al
6105 00001EA1 31C9      xor     cx, cx ; 0
6106 00001EA3 F6440210 test    byte [si+2], 10h ; information flags
6107                                ; bit 4 = Device has change line support
6108 00001EA7 7403      jz     short not_chgline_sup
6109            ;or     al, 2 ; change line support
6110 00001EA9 80C902    or     cl, 2
6111 not_chgline_sup:
6112 00001EAC 83C41E    add     sp, 30
6113            ;pop     si ; (10/12/2023)
6114 00001EAF 1F        pop     ds
6115            ;
6116 00001EB0 07        pop     es ; es=cs=ds (21/12/2022)
6117            ;pop     cx ; (10/12/2023)
6118 00001EB1 5A        pop     dx
6119 00001EB2 5F        pop     di
6120            ;pop     ds ; (21/12/2022)
6121
6122            ; 10/12/2023
6123 00001EB3 F6C102    test    cl, 2
6124            ;test    al, 2
6125            ;jz     short gotother_j
6126 00001EB6 7450      jz     short gotother
6127            ;or     cl, al
6128 00001EB8 C606[7700]01 mov     byte [fhav96], 1 ; Device has change line support
6129 gotother_j:
6130 00001EBD EB49      jmp     short gotother
6131 ext_gdp_err:
6132 00001EBF 83C41E    add     sp, 30
6133            ;pop     si ; (10/12/2023)
6134 00001EC2 1F        pop     ds
6135
6136            ; 10/12/2023
6137 not_atapi_removable:
6138            ; .....
6139
6140 ; if cmos is bad, it gives es,ax,bx,cx,dh,di=0. cy=0.
6141 ; in this case, we are going to put bogus informations to bds table.
6142 ; we are going to set ch=39,cl=9,dh=1 to avoid divide overflow when
6143 ; they are calculated at the later time. this is just for the diagnostic
6144 ; diskette which need msbio,msdos to boot up before it sets cmos.
6145 ; this should only happen with drive b.
6146
6147
6148 00001EC3 80FD00    cmp     ch, 0 ; if ch=0, then cl,dh=0 too.
6149 00001EC6 7505      jnz     short pfr_ok
6150
6151            ;mov     ch, 39 ; rom gave wrong info.
6152            ;mov     cl, 9 ; let's default to 360k.
6153            ; 20/12/2022
6154 00001EC8 B90927    mov     cx, 2709h
6155
6156 00001ECB B601      mov     dh, 1
6157 pfr_ok:
6158            ;inc     dh ; make number of heads 1-based
6159            ;mov     [num_heads], dh ; save parms returned by rom
6160            ; 10/12/2023
6161 00001ECD 86D6      xchg     dl, dh
6162 00001ECF 30F6      xor     dh, dh
6163 00001ED1 42        inc     dx ; make number of heads 1-based
6164 00001ED2 8916[0E1A] mov     [num_heads], dx
6165
6166            ;inc     ch ; make number of cylinders 1-based
6167            ;and     cl, 3Fh
6168            ;mov     [sec_trk], cl
6169            ;mov     [num_cyl], ch ; assume less than 256 cylinders!!
6170            ; 10/12/2023
6171 00001ED6 88CA      mov     dl, cl
6172 00001ED8 80E23F    and     dl, 3Fh
6173 00001EDB 8816[121A] mov     [sec_trk], dl
6174 00001EDF 86CD      xchg     cl, ch
6175 00001EE1 D0C5      rol     ch, 1
6176 00001EE3 D0C5      rol     ch, 1
6177 00001EE5 80E503    and     ch, 3
6178 00001EE8 41        inc     cx ; make number of cylinders 1-based
6179 00001EE9 890E[101A] mov     [num_cyl], cx
6180
6181 ; make sure that eot contains the max number of sec/trk in system of floppies
6182
6183            ;mov     cl, [sec_trk] ; 10/12/2023
6184            ;cmp     cl, [eot] ; may set carry
6185            ;jbe     short eot_ok
6186            ; 09/12/2022
6187            ;jne     short eotok ; wrong ! 14/08/2023
6188            ; 14/08/2023
6189            ;jbe     short eotok
6190            ;mov     [eot], cl
6191            ; 10/12/2023
6192 00001EED 3A16[2C01] cmp     dl, [eot] ; dl = [sec_trk]
6193 00001EF1 7604      jbe     short eotok
6194 00001EF3 8816[2C01] mov     [eot], dl
6195 ;eot_ok:
6196 eotok:
6197            ; 10/12/2023
6198            ; !!!
6199            ; (following pops are moved to 'chk_changeline' procedure)
6200            ;
6201            ; 20/12/2022
6202            ;pop     es ; ((*)) es = cs = ds
6203            ;pop     cx ; 10/12/2023
6204            ;pop     dx
6205            ;pop     di
6206
6207            ; 20/12/2022
6208            ;pop     ds
6209

```

```

6210 ; Check for presence of changeline
6211 ; 10/12/2023
6212 %if 0
6213 ; 10/12/2023
6214 ;xor cx, cx ; 0
6215 ;push cx
6216 ;push dx
6217
6218 mov ah, 15h
6219 int 13h ; DISK - DISK - GET TYPE (AT,XT2,XT286,CONV,PS)
6220 ; DL = drive ID
6221 ; Return: CF set on error, AH = disk type (3 = hard drive)
6222 ; CX:DX = number of sectors on the media
6223
6224 ; 10/12/2023
6225 pop dx
6226 ;pop cx
6227 mov cx, 0 ; 12/12/2023
6228 jc short changeline_done
6229 cmp ah, 2 ; check for presence of changeline
6230 jnz short changeline_done
6231
6232 ; we have a drive with change line support.
6233
6234 or cl, 2 ; fchangeline
6235 ; signal type
6236 mov byte [fhav96], 1 ; remember that we have 96tpi disks
6237 %endif
6238 ; 10/12/2023
6239 call chk_changeline
6240 ;jc short changeline_done
6241
6242 ; we now try to set up the form factor for the types of media that we know
6243 ; and can recognise. for the rest, we set the form factor as "other".
6244 changeline_done:
6245 ; 40 cylinders and 9 or less sec/trk, treat as 48 tpi medium.
6246
6247 cmp byte [num_cyl], 40
6248 jnz short try_80
6249 cmp byte [sec_trk], 9
6250 jbe short nextdrive
6251
6252 gotother:
6253 ; 10/12/2023
6254 ; ch = 0, cl = 2 or 0
6255
6256 mov dh, 7 ; ffother
6257 ; we have a "strange" medium
6258 jmp short nextdrive
6259
6260 ;-----
6261 ; 80 cylinders and 9 sectors/track => 720 kb device
6262 ; 80 cylinders and 15 sec/trk => 96 tpi medium
6263
6264 try_80:
6265 cmp byte [num_cyl], 80
6266 jnz short gotother
6267 mov dh, 9 ; ff288 ; assume 2.88 MB drive
6268 cmp byte [sec_trk], 36 ; is it ?
6269 jz short nextdrive ; yeah, go update
6270
6271 ; 12/05/2019 (ff144 type will not be used -compatibility problem-)
6272 ; 08/01/2018 - Retro DOS v4.0 feature only ! for 1.44MB diskettes
6273 ;mov dh, ff144
6274 ;cmp byte [sec_trk], 18
6275 ;je short nextdrive
6276
6277 cmp byte [sec_trk], 15
6278 jz short got96
6279
6280 cmp byte [sec_trk], 9
6281 jnz short gotother
6282
6283 mov dh, 2 ; ffSmall
6284 jmp short nextdrive
6285
6286 ; -----
6287 got96:
6288 mov dh, 1 ; ff96tpi
6289 jmp short nextdrive
6290
6291 ; -----
6292
6293 ; 10/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
6294 ; check change line feature (and set fhav96 if there is)
6295 ; (common procedure for 'eotok:' and 'noparmsfromrom:')
6296
6297 chk_changeline:
6298 pop cx ; near call return address
6299
6300 ; (pop es, dx, di for 'eotok' and 'noparmsfromrom' procs)
6301 pop es ; es=cs=ds ; 21/12/2022
6302 ;pop cx ; (10/12/2023)
6303 pop dx
6304 pop di ; BDS address/offset
6305
6306 push cx ; near call return address
6307
6308 ;xor cx, cx ; 0
6309 ;push cx
6310 ;push dx
6311
6312 mov ah, 15h
6313 int 13h ; DISK - DISK - GET TYPE (AT,XT2,XT286,CONV,PS)
6314 ; DL = drive ID
6315 ; Return: CF set on error, AH = disk type (3 = hard drive)
6316 ; CX:DX = number of sectors on the media
6317
6318 pop dx
6319 ;pop cx
6320 mov cx, 0
6321 jc short chk_chgl_1
6322
6323 cmp ah, 2 ; is there changeline?
6324 jne short chk_chgl_2 ; *
6325
6326 or cl, 2
6327 ;or cl, ah ; 2
6328 mov byte [fhav96], 1 ; fchangeline
6329 ; cf = 0
6330
6331 chk_chgl_1:
6332 chk_chgl_2:
6333 retn
;chk_chgl_2: ; *

```

```

6334 ; ; 10/12/2023
6335 ; ; ah = 1 ; harddisk type (ah = 3) return not possible here for floppies
6336 ; ; stc
6337 ; ; cf = 1
6338 ; ; retn
6339 ;
6340 ; -----
6341 ;
6342 ; we have an old rom, so we either have a 48tpi or 96tpi drive. if the drive
6343 ; has changeline, we assume it is a 96tpi, otherwise we treat it as a 48tpi.
6344 ;
6345 noparmsfromrom:
6346 ; 10/12/2023
6347 ; !!!
6348 ; (following pops are moved to 'chk_changeline' procedure)
6349 ;
6350 ; 20/12/2022
6351 ; pop es ; ((*))
6352 ; pop cx ; (10/12/2023)
6353 ; pop dx
6354 ; pop di
6355 ;
6356 ; 20/12/2022
6357 ; pop ds
6358 ; 10/12/2023
6359 %if 0
6360 ; 10/12/2023
6361 ; xor cx, cx ; 0
6362 ; push cx
6363 ; push dx
6364 ;
6365 ;
6366 mov ah, 15h
6367 int 13h ; DISK - DISK - GET TYPE (AT,XT2,XT286,CONV,PS)
6368 ; DL = drive ID
6369 ; Return: CF set on error, AH = disk type (3 = hard drive)
6370 ; CX:DX = number of sectors on the media
6371 ; 10/12/2023
6372 pop dx
6373 ; pop cx
6374 mov cx, 0 ; 12/12/2023
6375 jc short nextdrive
6376 ;
6377 cmp ah, 2 ; is there changeline?
6378 jnz short nextdrive
6379 ;
6380 or cl, 2
6381 mov byte [fhav96], 1 ; fchangeline
6382 %endif
6383 ; 10/12/2023
6384 call chk_changeline
6385 jc short nextdrive
6386 ;
6387 ; change line support, [fhav96] = 1
6388 ;
6389 mov byte [num_cyl], 80
6390 mov dh, 1 ; ff96tpi
6391 mov al, 15
6392 cmp al, [eot]
6393 jbe short nextdrive
6394 mov [eot], al
6395 ; -----
6396 ;
6397 ; eot_ok2:
6398 nextdrive:
6399 ; 10/12/2023
6400 ; ch = 0, cl = 2 or 0
6401 ;
6402 or cl, 20h ; fi_own_physical
6403 ; set this true for all drives
6404 mov bh, dl ; save int13 drive number
6405 ;
6406 ; we need to do special things if we have a single drive system and are setting
6407 ; up a logical drive. it needs to have the same int13 drive number as its
6408 ; counterpart, but the next drive letter. also reset ownership flag.
6409 ; we detect the presence of this situation by examining the flag single for the
6410 ; value 2.
6411 cmp byte [single], 2
6412 jnz short not_special
6413 dec bh ; int13 drive number same for logical drive
6414 xor cl, 20h ; fi_own_physical
6415 ; reset ownership flag for logical drive
6416 not_special:
6417 ;
6418 ; the values that we put in for BDS_RBPB.BPB_HEADS and
6419 ; BDS_RBPB.BPB_SECTORS PER TRACK will only remain if the
6420 ; form factor is of type "ffother".
6421 ;
6422 xor ax, ax ; fill BDS for drive
6423 mov al, [num_heads]
6424 ; 10/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM) ; *
6425 mov ax, [num_heads]
6426 mov [di+36h], ax ; [di+BDS.rheads]
6427 mov [di+52h], ax ; [di+BDS.rheads] ; *
6428 xor ax, ax ; *
6429 mov al, [sec_trk]
6430 mov [di+34h], ax ; [di+BDS.rsecptrack]
6431 mov [di+50h], ax ; [di+BDS.rsecptrack] ; *
6432 mov [di+23h], cx ; [di+BDS.flags]
6433 mov [di+3Fh], cx ; [di+BDS.flags] ; *
6434 mov [di+22h], dh ; [di+BDS.formfactor]
6435 mov [di+3Eh], dh ; [di+BDS.formfactor] ; *
6436 mov [di+5], dl ; [di+BDS.drivelet]
6437 mov [di+4], bh ; [di+BDS.drivenum]
6438 mov bl, [num_cyl]
6439 mov [di+25h], bl ; [di+BDS.cylinders]
6440 ; 10/12/2023
6441 mov ax, [num_cyl]
6442 mov [di+41h], ax ; [di+BDS.cylinders] ; *
6443 ;
6444 cmp byte [single], 1 ; Special case for single drive system
6445 jnz short no_single
6446 mov byte [single], 2 ; Don't forget we have
6447 ; single drive system
6448 ; 10/12/2023
6449 inc byte [single] ; [single] = 2
6450 ; 18/12/2022
6451 or cl, 10h
6452 ; or cx, 10h ; fi_am_mult
6453 ; set that this is one of several drives
6454 ; or [di+23h], cx ; [di+BDS.flags]
6455 ; or [di+3Fh], cx ; [di+BDS.flags] ; *
6456 ; save flags
6457 mov di, [di] ; [di+BDS.link]

```



```

6458                                     ; move to next BDS in list
6459 00001FAB FEC2                     inc    dl      ; add a number
6460 00001FAD EBB8                     jmp     short nextdrive ; Use same info      for BDS as previous
6461                                     ; -----
6462
6463 no_single:
6464                                     ; inc    dl
6465                                     ; 18/12/2022
6466 00001FAF 42                        inc     dx
6467 00001FB0 E97CFE                     jmp     loop_drive
6468                                     ; -----
6469
6470                                     ; 11/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
6471                                     ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:21E8h)
6472
6473 done_drives:
6474 00001FB3 C705FFFF                     mov     word [di+BDS.link], -1
6475                                     mov     word [di], -1 ; set link to null
6476
6477 ; set up all the hard drives in the system
6478
6479                                     ; 20/12/2022
6480                                     ; 28/12/2018 - Retro DOS v4.0 (MSDOS 6.21)
6481
6482 dohard:
6483 00001FB7 8A36[5F1A]                   mov     dh, [hnum]
6484 00001FB8 08F6                         or      dh, dh ; done if no hardfiles
6485 00001FB9 7459                         jz      short static_configure
6486 00001FBA B280                         mov     dl, 80h
6487
6488 dohard1:
6489 00001FC1 52                           push    dx
6490 00001FC2 8B3E[621A]                   mov     di, [end_of_bdss]
6491 00001FC3 8A1E[7500]                   mov     bl, [drvmax]
6492 00001FC4 B700                         mov     bh, 0 ; first primary partition (or active)
6493 00001FC5 E89A01                       call    sethard
6494 00001FC6 7208                         jb      short hardfile_err
6495 00001FC7 E86508                       call    dmax_check ; error if already 26 drives
6496 00001FC8 7303                         jnb     short hardfile_err
6497 00001FC9 E87508                       call    xinstall_bds ; insert new bds into linked list
6498
6499 hardfile_err:
6500 00001FD9 5A                           pop     dx
6501                                     inc     dl ; next hard drive
6502                                     ; 12/12/2023
6503                                     inc     dx
6504 00001FDA 42                           dec     dh
6505 00001FDB FECE                         jnz     short dohard1
6506
6507 ; end of physical drive initialization
6508
6509 ; *** do not change the position of the following statement.
6510 ; *** domini routine will use [drvmax] value for the start of the logical
6511 ; *** drive number of mini disk(s).
6512
6513 00001FDF E8CA06                       call    domini ; for setting up mini disks, if found
6514
6515 ; -- begin added section
6516
6517 dohardx1:
6518 00001FE2 8A36[5F1A]                   mov     dh, [hnum] ; we already know this is >0
6519 00001FE3 B280                         mov     dl, 80h
6520
6521 dohardx2:
6522 00001FE8 B701                         mov     bh, 1 ; do all subsequent primary partitions
6523
6524 dohardx4:
6525 00001FEA 52                           push    dx
6526 00001FEB 53                           push    bx
6527 00001FEC 8B3E[621A]                   mov     di, [end_of_bdss]
6528 00001FED 8A1E[7500]                   mov     bl, [drvmax]
6529 00001FEE E87201                       call    sethard
6530 00001FEF 720E                         jb      short dohardx4 ; move to next hardfile if error
6531 00001FF0 E83D08                       call    dmax_check ; make sure <=26 drives
6532 00001FF1 7309                         jnb     short dohardx4 ; skip if error
6533 00001FF2 E84D08                       call    xinstall_bds ; insert new bds into linked list
6534 00001FF3 5B                           pop     bx ; get partition number
6535 00001FF4 5A                           pop     dx ; restore physical drive counts
6536 00001FF5 FEC7                         inc     bh
6537 00001FF6 EBE3                         jmp     short dohardx2 ; keep looping until we fail
6538                                     ; -----
6539
6540 dohardx4:
6541 00002007 5B                           pop     bx ; unjunk partition number from stack
6542 00002008 5A                           pop     dx ; restore physical drive counts
6543                                     inc     dl ; next hard drive
6544                                     ; 12/12/2023
6545                                     inc     dx
6546 00002009 42                           dec     dh
6547 0000200A FECE                         jnz     short dohardx1
6548
6549 ; -- end changed section
6550
6551 ; *****
6552 ; if more than 2 diskette drives on the system, then it is necessary to remap
6553 ; the bds chain to adjust the logical drive num (drive letter) with greater
6554 ; than two diskette drives
6555 ;
6556 ; new scheme: if more than 2 diskette drives, first map the bds structure
6557 ; as usual and then rescan the bds chain to adjust the drive
6558 ; letters. to do this, scan for disk drives and assign logical
6559 ; drive number starting from 2 and then rescan diskette drives
6560 ; and assign next to the last logical drive number of last disk
6561 ; drive to the 3rd and 4th diskette drives.
6562 ; *****
6563
6564 0000200E 803E[2501]02                   cmp     byte [dsktnum], 2 ; >2 diskette drives
6565                                     jbe     short static_configure ; no - no need for remapping
6566 00002013 7603                         jbe     short no_remap
6567 00002015 E8D500                       call    remap ; remap bds chain to adjust driver letters
6568
6569 no_remap:
6570
6571 ; End of drive initialization.
6572
6573 ; -----
6574
6575 ; we now decide, based on the configurations available so far, what
6576 ; code or data we need to keep as a stay resident code. the following table
6577 ; shows the configurations under consideration. they are listed in the order
6578 ; of their current position memory.
6579 ;
6580 ; configuration will be done in two ways:
6581 ;
6582 ; first, we are going to set "static configuration". static configuration will
6583 ; consider from basic configuration to endof96tpi configuration. the result
6584 ; of static configuration will be the address the dynamic configuration will
6585 ; use to start with.
6586 ;
6587 ; secondly, "dynamic configuration" will be performed. dynamic configuration
6588 ; involves possible relocation of code or data. dynamic configuration routine
6589 ; will take care of bds tables and at rom fix module thru k09 suspend/resume

```

```

6582 ;code individually. after these operation, [dosdatasg] will be set.
6583 ;this will be the place sysinit routine will relocate msdos module for good.
6584
6585 ; -- begin changed section
6586 ;
6587 ; 1. basic configuration for msbio (endfloppy)
6588 ; 2. end96tpi ; a system that supports "change line error"
6589 ; 3. end of bdss ; end of bdss for hard disks
6590 ; 4. endatrom ;some of at rom fix module.
6591 ; 5. endcmosclockset;supporting program for cmos clock write.
6592 ; 6. endk09 ;k09 cmos clock module to handle suspend/resume operation.
6593 ;
6594
6595 ; 02/10/2022 - Retro DOS v4.0 (MSDOS v5.0 IO.SYS)
6596
6597 static_configure:
6598 00002018 8B3E[621A] mov di, [end_of_bdss]
6599 0000201C 81FF[4C08] cmp di, bdss ; 19/10/2022
6600 ;cmp di, offset bdss ; did we allocate any hard drive bdss?
6601 00002020 750D jnz short dynamic_configure ; that's the end, then
6602 ; 18/10/2022
6603 00002022 BF[4C08] mov di, end96tpi
6604 ;mov di, offset harddrv ; end96tpi
6605 ;keep everything up to end96tpi
6606 00002025 803E[7700]00 cmp byte [fhave96], 0
6607 0000202A 7503 jnz short dynamic_configure
6608
6609 0000202C BF[3808] mov di, endfloppy
6610 dynamic_configure:
6611 ; 20/12/2022
6612 ;push cs
6613 ;pop es
6614
6615 ; 10/12/2023
6616 0000202F FC cld ; clear direction flag is not necessary here !?
6617 ; because there will not be a running program
6618 ; which will set direction flag as backward (std)
6619
6620 ; -- end changed section
6621
6622 ; 20/12/2022
6623 ; ds = cs <> es
6624 ; ss = 0
6625 ; sp = 700h
6626
6627 ; 13/12/2023
6628 00002030 BE00F0 mov si, 0F000h
6629 00002033 8EC6 mov es, si ; ES -> ROM BIOS segment
6630
6631 00002035 803E[AF05]FC cmp byte [model_byte], 0Fch ; AT ?
6632 ;jnz short checkcmosclock
6633 ; 10/12/2023
6634 0000203A 751E jnz short checkcompaqbug ; no
6635 0000203C 803E[5F1A]00 cmp byte [hnum], 0 ; No hard file?
6636 ;jz short checkcmosclock
6637 00002041 7417 jz short checkcompaqbug
6638 00002043 97 xchg ax, di ; save allocation pointer in ax
6639 ; 13/12/2023
6640 ;mov si, 0F000h
6641 ;mov es, si ; ES -> ROM BIOS segment
6642 00002044 BE[681A] mov si, bios_date ; "01/10/84"
6643 00002047 BFF5FF mov di, 0FFF5h ; ROM BIOS string is at F000:FFF5
6644 0000204A B90900 mov cx, 9 ; bdate_l
6645 ; Only patch ROM for bios 01/10/84
6646 0000204D F3A6 repe cmpsb ; check for date + zero on end
6647 0000204F 97 xchg ax, di ; restore allocation pointer
6648
6649 ; M015 -- begin changes
6650
6651 ;jnz short checkcmosclock
6652 ; 02/10/2022
6653 00002050 7508 jnz short checkcompaqbug
6654
6655 ; install at rom fix
6656
6657 ; 19/10/2022
6658 ;mov cx, offset endatrom
6659 00002052 B9[2018] mov cx, endatrom
6660 ;mov si, offset ibm_disk_io
6661 00002055 BE[F216] mov si, ibm_disk_io
6662 00002058 EB46 jmp short install_int13_patch
6663 ; -----
6664
6665 ; M065 -- begin changes
6666 ;
6667 ; On certain systems with western Digital disk controllers, the
6668 ; following detection scheme caused an unpredictable and serious
6669 ; failure. In particular, they've implemented a nonstandard
6670 ; int13(ah=16h) which reconfigures the hard drive, depending on
6671 ; what happens to be at es:[bx] and other memory locations indexed
6672 ; off of it.
6673 ;
6674 ; Compaq was unable to tell us exactly which kind of systems have
6675 ; the bug, except that they guarantee that the bug was fixed in
6676 ; ROM BIOSs dated 08/04/86 and later. We'll check for the COMPAQ
6677 ; string, and then look for date codes before 08/04/86 to decide
6678 ; when to install the hook.
6679
6680 ;checkcmosclock:
6681 ; 02/10/2022
6682 checkcompaqbug:
6683 ; 20/12/2022
6684 ; es = 0F000h
6685 ;mov ax, 0F000h ; point to ROM BIOS
6686 ;mov es, ax
6687
6688 ; 19/10/2022
6689 0000205A 26813EEAFF434F cmp word [es:0FFEAh], 'CO'
6690 ;cmp word ptr es:0FFEAh, 'OC' ; look for COMPAQ
6691 00002061 754B jnz short not_compaq_patch
6692 00002063 26813EECFF4D50 cmp word [es:0FFEC], 'MP'
6693 ;cmp word ptr es:0FFEC, 'PM'
6694 0000206A 7542 jnz short not_compaq_patch
6695 0000206C 26813EEEFF4151 cmp word [es:0FFEEh], 'AQ'
6696 ;cmp word ptr es:0FFEEh, 'QA'
6697 00002073 7539 jnz short not_compaq_patch
6698
6699 ; We're running on a COMPAQ. Now look at the date code.
6700
6701 00002075 26A1FBFF mov ax, [es:0FFFBh] ; get year
6702 00002079 86E0 xchg ah, al
6703
6704 ; 11/12/2023
6705 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:22B9h)

```



```

6706      %if 0
6707          cmp     ax, 3836h      ; '68' (NASM syntax) (('86' in MASM syntax))
6708          ja      short not_compaq_patch
6709          jz      short chkcompaqbug1
6710          cmp     ax, 3739h      ; '97'
6711          jbe     short not_compaq_patch
6712          stc
6713      chkcompaqbug1:
6714          jb      short do_compaq_patch
6715          mov     ax, [es:0FFF5h]
6716          xchg    ah, al
6717          cmp     ax, 3038h      ; '80'
6718          ja      short not_compaq_patch
6719          jb      short do_compaq_patch
6720          mov     ax, [es:0FFF8h]
6721          xchg    ah, al
6722          cmp     ax, 3034h      ; '40'
6723          jnb     short not_compaq_patch
6724      do_compaq_patch:
6725      %endif
6726          ; 11/12/2023
6727          ; (MSDOS 6.22 IO.SYS - BIOSDATA:1C85h)
6728
6729      0000207B 3D3638          cmp     ax, 3836h ; 02/10/2022 (NASM syntax)
6730          ;cmp     ax, '86'      ; 3836h
6731          ; is it 86?
6732          ja      short not_compaq_patch
6733          jb      short do_compaq_patch
6734      00002082 26A1F5FF      mov     ax, [es:0FFF5h] ; get month
6735      00002086 86E0          xchg    ah, al
6736      00002088 3D3830          cmp     ax, 3038h ; 02/10/2022 (NASM syntax)
6737          ;cmp     ax, '08'      ; 3038h
6738          ; is it 08?
6739      0000208B 7721          ja      short not_compaq_patch
6740      0000208D 720B          jb      short do_compaq_patch
6741      0000208F 26A1F8FF      mov     ax, [es:0FFF8h] ; get day
6742      00002093 86E0          xchg    ah, al
6743      00002095 3D3430          cmp     ax, 3034h ; 02/10/2022 (NASM syntax)
6744          ;cmp     ax, '04'      ; 3034h
6745          ; is it 04?
6746          jnb     short not_compaq_patch
6747
6748      do_compaq_patch:
6749      0000209A B9[3D18]      mov     cx, end_compaq_i13hook
6750          ;mov     si, endatrom
6751          ; 11/12/2023
6752      0000209D BE[2018]      mov     si, compaq_disk_io ; endatrom
6753
6754      install_int13_patch:
6755      000020A0 0E          push    cs
6756      000020A1 07          pop     es
6757          ; 18/10/2022
6758      000020A2 893E[B400]      mov     [Orig13], di ; set new rom bios int 13 vector
6759      000020A6 8C0E[B600]      mov     [Orig13+2], cs
6760      000020AA 29F1          sub     cx, si ; size of rom fix module
6761      000020AC F3A4          rep movsb ; relocate it
6762
6763          ; M065 -- end changes
6764
6765          ; -----
6766      not_compaq_patch:      ; M065
6767          ; 17/10/2022
6768      checkcmosclock:
6769          ; 18/10/2022
6770      000020AE 0E          push    cs
6771      000020AF 07          pop     es
6772
6773          ; 20/12/2022
6774          ; ds = cs = es
6775          ; ss = 0
6776          ; sp = 700h
6777
6778          ; 09/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
6779      %if 0
6780          cmp     byte [havecmosclock], 1 ; cmos clock exists?
6781          jnz     short checkk09 ; no
6782
6783          mov     word [daycnttoday], di
6784          ;mov     word ptr ds:daycnttoday, di ; set the address for mschar
6785          mov     cx, 209 ; enddaycnttoday - daycnt_to_day
6786          mov     si, daycnt_to_day
6787          rep movsb
6788          mov     word [bintobcd], di
6789          ;mov     word ptr ds:bintobcd, di ; set the address for msclock
6790          ; let original segment stay
6791          ;mov     cx, 11 ; endcmosclockset - bin_to_bcd
6792          ; 08/08/2023
6793          mov     cl, 11
6794          mov     si, bin_to_bcd
6795          rep movsb
6796      %endif
6797
6798          ; 09/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
6799          ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:22F4h
6800          ;push     cs
6801          ;pop      es
6802      checkk09:
6803      000020B0 57          push    di ; ? ; save ? ; 21/12/2022
6804
6805          ; 13/12/2023 - Retro DOS v4.2 IO.SYS
6806          ; (MSDOS 6.22 IO.SYS - BIOSDATA:1CDAh)
6807      %if 0
6808
6809          mov     ax, 4101h      ; wait for bh=es:[di]
6810          mov     bl, 1          ; wait for 1 clock tick
6811          mov     bh, [es:di]
6812          stc
6813          int     15h            ; Assume we will fail
6814          ; SYSTEM - WAIT ON EXTERNAL EVENT (CONVERTIBLE)
6815          ; AL = condition type, BH = condition compare or mask value
6816          ; BL = timeout value times 55 milliseconds, 00h means no timeout
6817          ; DX = I/O port address if AL bit 4 set
6818          ; 11/12/2023
6819          ; ES:DI = user byte if AL bit 4 clear
6820      %endif
6821
6822          ; 13/12/2023 - Retro DOS v5.0 IBMBIO.COM/IO.SYS
6823          ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:1CDAh)
6824          ; .....
6825          mov     ax, 4100h      ; wait for any external event (al=0)
6826          mov     bl, 4          ; wait for 4 clock ticks
6827          stc
6828          int     15h            ; Assume we will fail
6829          ; SYSTEM - WAIT ON EXTERNAL EVENT (CONVERTIBLE)
6830          ; AL = condition type, BH = condition compare or mask value

```

```

6830                                     ; BL = timeout value times 55 milliseconds, 00h means no timeout
6831                                     ; DX = I/O port address if AL bit 4 set
6832                                     ; .....
6833
6834 000020B9 5F          pop     di ; ?
6835 000020BA 721B       jc      short configdone ; 21/12/2022
6836
6837 000020BC C606[7900]01 mov     byte [fhavsek09], 1
6838                                     ; remember we have a k09 type
6839 000020C1 1E          push    ds
6840 000020C2 31C0       xor     ax, ax
6841 000020C4 8ED8       mov     ds, ax
6842
6843 000020C6 893EB001   mov     [6Ch*4], di
6844                                     ;mov     ds:1B0h, di ; [6Ch*4]
6845                                     ; new int 6Ch handler
6846                                     ;mov     word ptr ds:1B2h, cs ; [6Ch*4+2]
6847 000020CA 8C0EB201   mov     word [6Ch*4+2], cs
6848 000020CE 1F          pop     ds
6849                                     ; 20/12/2022
6850                                     ; ds = cs = es
6851                                     ;mov     si, int6c
6852                                     ;mov     cx, endk09-int6c ; 459
6853                                     ;mov     cx, 459 ; endk09 - int6c
6854                                     ; size of k09 routine
6855                                     ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
6856                                     ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:2315h
6857 000020CF BE[3E18]   mov     si, int_6Ch
6858 000020D2 B9BE01     mov     cx, endk09-int_6Ch ; 461 in PCDOS 7.1 IBMBIO.COM
6859 000020D5 F3A4       rep movsb
6860                                     ; set up config stuff for sysinit
6861 ; -----
6862 ; Set up config stuff for SYSINIT
6863
6864 ; 17/10/2022
6865 ; SETDRIVE equ SetDrive - DOSBIOSEG_2C7h ; (4D7h for MSDOS 5.0 IO.SYS)
6866 ; GETBP equ GetBp - DOSBIOSEG_2C7h ; (606h for MSDOS 5.0 IO.SYS)
6867 ; 09/12/2022
6868 SETDRIVE equ SetDrive
6869 GETBP equ GetBp
6870
6871 ; 17/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
6872 configdone:
6873 ; 21/12/2022
6874 ; 20/03/2019
6875 ;push    cs ; di is final ending address of msbio.
6876 ;pop     ds
6877
6878 000020D7 83C70F     add     di, 15 ; round (up) to paragraph
6879 ; 10/12/2022
6880 ;shr     di, 1
6881 ;shr     di, 1
6882 ;shr     di, 1
6883 ;shr     di, 1
6884 000020DA B104     mov     cl, 4
6885 000020DC D3EF     shr     di, cl
6886 ; 10/12/2022
6887 000020DE 83C770     add     di, 70h ; KERNEL_SEGMENT (in fact: IO.SYS loading segment)
6888 ; 19/10/2022 - Temporary !
6889 ;db      81h, 0C7h, 70h, 0 ; add di, 0070h
6890 000020E1 893E[0300] mov     [DosDataSg], di ; where the dosdata segment will be
6891
6892 ; 21/12/2022 - Retro DOS v4.0 (MSDOS 5.0 combined/single kernel file)
6893
6894 ; 19/03/2018 - No need to read remain clusters of MSDOS kernel because
6895 ; Retro DOS v2.0 boot sector has loaded all of the kernel file before.
6896
6897 ; ("MSINIT.ASM" contains kernel file reading code here, below...)
6898
6899 ; 11/12/2023 - Retro DOS v5.0 (PCDOS 7.1 combined/single kernel file)
6900
6901 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2332h)
6902
6903 ; (("IBMDOS.COM" kernel file reading code here, below...))
6904
6905 ; -----
6906 ; -----
6907 %if 0
6908 mov     ax, [drvfat] ; get drive and fat id
6909 ; 22/12/2022
6910 ; Note: SETDRIVES uses AL (drive number) only
6911 mov     bp, SETDRIVE
6912 ;mov     bp, 5AEh ; 11/12/2023 (PCDOS 7.1 IBMBIO.COM)
6913 ;mov     bp, 4D7h ; set_drive (in dosbioscode segment)
6914                                     ; at 2C7h:4D7h = 70h:2A47h
6915                                     ; simulate far call
6916 push    cs
6917 call    call_bios_code ; get bds for drive
6918 mov     bp, GETBP ; ensure valid bpb is present
6919 ;mov     bp, 6E4h ; 11/12/2023 (PCDOS 7.1 IBMBIO.COM)
6920 ;mov     bp, 606h ; GetBp (2C7h:606h = 70h:2B76h)
6921 push    cs
6922 call    call_bios_code
6923
6924 ; resort to funky old segment definitions for now
6925
6926 ; 22/12/2022
6927 ;push    es ; copy bds to ds:di
6928 ;pop     ds
6929
6930 ; the following read of es:0000 was spurious anyway. Should look into it.
6931 ;
6932 ; hmmmmmm. j.k. took out a call to getfat right here a while
6933 ; back. Apparently it was what actually setup es: for the following
6934 ; cas----
6935
6936 ; 22/12/2022
6937 ;xor     di, di
6938 ;mov     al, [es:di] ; get fat id byte
6939 ;mov     byte ptr es:drvfat+1, al ; save fat byte
6940 ;mov     [es:drvfat+1], al
6941 ;mov     ax, [es:drvfat]
6942
6943 ; 22/12/2022
6944 ; ds = cs
6945 ;;; mov     al, [drvfat]
6946
6947 ; cas -- why do a SECOND setdrive here???
6948
6949 ; 22/12/2022
6950 ;push    es ; save whatever's in es
6951 ;push    ds ; copy bds to es:di
6952 ;pop     es
6953 ;push    cs ; copy Bios_Data to ds
6954 ;pop     ds

```

```

6954
6955 ; 22/12/2022
6956 ;;; mov bp, SETDRIVE
6957 ;;; ;mov bp, 4D7h ; SetDrive (2C7h:47Dh = 70h:2A47h)
6958 ;;; push cs ; simulate far call
6959 ;;; call call_bios_code ; get correct bds for this drive
6960
6961 ; 22/12/2022
6962 ;push es ; copy bds back to ds:di
6963 ;pop ds
6964 ;pop es ; pop whatever was in es
6965
6966 ; Now we load in the MSDOS.SYS file
6967
6968 ; 22/12/2022
6969 ; ----
6970 ; mov bx, [di+6] ; [di+BDS.BDS_BPB.BPB_BYTESPERSECTOR]
6971 ; mov [cs:md_sectorsize], bx ; used by get_fat_sector proc.
6972 ; mov bl, [di+1Fh] ; [di+BDS.fatsiz]
6973 ; ; get size of fat on media
6974 ; ;mov es:16DEh, bl
6975 ; mov [es:fbigfat], bl
6976 ; mov cl, [di+8]
6977 ; mov ax, [di+17h] ; [di+BDS.BDS_BPB.BPB_HIDDENSECTORS]
6978 ; sub es:16D8h, ax
6979 ; sub [es:bios_l], ax ; subtract hidden sectors since we
6980 ; ; need a logical sector number that will
6981 ; ; be used by getclus(diskrd procedure)
6982 ; mov ax, [di+19h] ; [di+BDS.BDS_BPB.BPB_HIDDENSECTORS+2]
6983 ; sbb es:16DAh, ax
6984 ; sbb [es:bios_h], ax ; subtract upper 16 bits of sector num
6985 ; ----
6986
6987 ; 11/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
6988 ; ----; 22/12/2022
6989 ; mov bx, [es:di+6] ; [di+BDS.BDS_BPB.BPB_BYTESPERSECTOR]
6990 ; mov [md_sectorsize], bx ; used by get_fat_sector proc.
6991 ; 11/12/2023 ; *
6992 ; mov bl, [es:di+3Bh] ; [di+BDS.fatsiz] ; *
6993 ; mov bl, [es:di+1Fh] ; [di+BDS.fatsiz]
6994 ; ; get size of fat on media
6995 ; mov [fbigfat], bl
6996 ; mov cl, [es:di+8]
6997 ; mov ax, [es:di+17h] ; [di+BDS.BDS_BPB.BPB_HIDDENSECTORS]
6998 ; sub [First_Data_Sector], ax ; *
6999 ; sub [bios_l], ax ; subtract hidden sectors since we
7000 ; ; need a logical sector number that will
7001 ; ; be used by getclus(diskrd procedure)
7002 ; mov ax, [es:di+19h] ; [di+BDS.BDS_BPB.BPB_HIDDENSECTORS+2]
7003 ; sbb [First_Data_Sector+2], ax ; *
7004 ; sbb [bios_h], ax ; subtract upper 16 bits of sector num
7005 ; ----
7006
7007 xor ch, ch ; cx = sectors/cluster
7008
7009 ; the boot program has left the directory at 0:500h
7010
7011 ; 11/12/2023 - - Retro DOS v5.0 IBMBIO.COM/IO.SYS
7012 ; push di
7013 ; push ds
7014 ; xor di, di
7015 ; mov ds, di
7016 xor bx, bx ; 0
7017 mov ds, bx
7018 mov bx, [53Ah]
7019 ; mov bx, ds:53Ah ; (First cluster of the 2nd dir entry
7020 ; ; of root directory in the buffer at 500h)
7021 pop ds
7022 mov si, [firstcluster_hw] ; 11/12/2023
7023 ; ; (32 bit cluster number for FAT32 fs)
7024 ; pop ds
7025 ; pop di
7026
7027 ; 12/12/2023
7028 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2397h)
7029 ; .....
7030 ; ds = cs
7031 mov al, [fbigfat]
7032 push ax ; (*) save fbigfat flags
7033 mov al, [drvfat]
7034 or al, [Boot_Drv]
7035 jnz short boot_drv_fixed ; hard disk
7036 boot_drv_removable: ; calculate cluster count and set fbig or fbigbig flag
7037 push bx ; for removable drives
7038 push cx
7039 ; 28/12/2023
7040 ; push dx ; (not necessary)
7041
7042 ; 12/12/2023
7043 push es
7044 pop ds
7045
7046 mov ax, [di+0Eh] ; [di+BDS.totalsecs16]
7047 xor dx, dx
7048 or ax, ax
7049 jnz short prep_totalsecs_ok
7050 mov ax, [di+1Bh] ; [di+BDS.totalsecs32]
7051 mov dx, [di+1Dh]
7052 prep_totalsecs_ok:
7053 sub ax, [di+9] ; [di+BDS.resectors]
7054 sbb dx, 0
7055 push ax
7056 push dx
7057 mov bx, [di+11h] ; [di+BDS.fatsecs16]
7058 xor ax, ax
7059 or bx, bx
7060 jnz short prep_fatsecs_ok
7061 mov bx, [di+1Fh] ; [di+BDS.fatsecs32]
7062 mov ax, [di+21h]
7063 prep_fatsecs_ok:
7064 mov cl, [di+0Bh] ; ax:bx = 32 bit count of FAT sectors
7065 ; ; [di+BDS.fats]
7066 xor ch, ch
7067 mul cx
7068 xchg ax, cx
7069 mul bx
7070 add cx, dx
7071 mov bx, ax ; cx:bx = total (2*) fat sectors
7072 pop dx
7073 pop ax ; dx:ax = totals sectors - reserved sectors
7074 sub ax, bx
7075 sbb dx, cx ; dx:ax = data sectors (includes root dir sectors)
7076 mov bx, [di+0Ch] ; [di+BDS.direntries]
7077 add bx, 15 ; 16 directory entries per sector

```

```

7078                                     ; (round up sector count by adding 15)
7079     mov     cl, 4                     ; (rounded) dir entries / 16
7080     shr     bx, cl
7081     xor     cx, cx
7082     sub     ax, bx
7083     sbb     dx, cx                     ; dx:ax = data sectors (except root directory sectors)
7084                                     ; (will be used for cluster count calculation)
7085     mov     cl, [di+8]                 ; [di+BDS.secpersclus]
7086
7087     ; 12/12/2023
7088     push    cs
7089     pop     ds
7090
7091     push    ax                         ; 32 bit division (data sectors / sector per cluster)
7092     mov     ax, dx
7093     xor     dx, dx
7094     div     cx
7095     mov     bx, ax
7096     pop     ax
7097     div     cx
7098     or      bx, bx                     ; 32 bit cluster count if bx > 0
7099     jnz     short set_fbigbig_flag ; too big cluster number
7100     cmp     ax, 0FFF6h
7101     jb      short set_fbig_flag
7102 set_fbigbig_flag:
7103     or      byte [fbigfat], 20h ; FAT32 ; fbigbig
7104     jmp     short set_fbig_flag_ok
7105 ; -----
7106
7107 set_fbig_flag:
7108     cmp     ax, 0FF6h                 ; 4096-10
7109                                     ; is this 16-bit fat?
7110     jb      short set_fbig_flag_ok ; no, small fat
7111     or      byte [fbigfat], 40h ; FAT16 ; fbig
7112 set_fbig_flag_ok:
7113     ; 28/12/2023
7114     ; pop     dx
7115     pop     cx
7116     pop     bx
7117 boot_drv_fixed:
7118     xor     di, di
7119
7120     ; cx = sectors/cluster
7121     ; si:bx = first cluster
7122     ; di = 0
7123
7124     ; .....
7125 loadit:
7126     mov     ax, SYSINITSEG ; 46Dh
7127     ; mov     ax, 544h         ; 11/12/2023 - PC DOS 7.1 IBMBIO.COM
7128     ; mov     ax, 46Dh         ; sysinit segment
7129     mov     es, ax
7130     mov     es, [es:CURRENTDOSLOCATION] ; 09/12/2022
7131     ; mov     es, [es:271h]
7132
7133     call    getclus                    ; read cluster at ES:DI (DI is updated)
7134
7135 ; -----
7136
7137     ; 13/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
7138     ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2431h)
7139
7140     ; test    byte [cs:fbigfat], 20h ; fbigbig ; FAT32 fs flag
7141     test    byte [fbigfat], 20h ; fbigbig ; FAT32 fs flag
7142     jz      short iseof
7143
7144 eofbigbig: ; si:bx = 32 bit cluster number
7145     cmp     si, 0FFFh
7146     jnz     short iseofx
7147     cmp     bx, 0FFF7h
7148     jmp     short iseofx
7149
7150 ; -----
7151
7152     ; 13/12/2023
7153 iseof:
7154     ;; test   byte [cs:fbigfat], fbig
7155     ; test    byte [cs:fbigfat], 40h ; fbig
7156     ; 12/12/2023
7157     ; ds = cs
7158     test    byte [fbigfat], 40h ; fbig
7159     jnz     short eofbig
7160     cmp     bx, 0FF7h
7161     jmp     short iseofx
7162
7163 ; -----
7164
7165 eofbig:
7166     cmp     bx, 0FFF7h
7167
7168 iseofx:
7169     jb      short loadit ; keep loading until cluster = eof
7170
7171 ; -----
7172
7173     ; 19/04/2024
7174     ; 28/12/2023
7175     pop     ax                         ; (*) restore fbigfat flags
7176                                     ; (after loading DOS kernel)
7177     ; 06/04/2024
7178     ; mov     [cs:fbigfat], al
7179     mov     [fbigfat], al
7180 %endif
7181 ; -----
7182
7183     ; 19/04/2024
7184     call    setdrvparms
7185
7186     ;; jmp    far ptr 46Dh:267h ; jmp     SYSINIT_SEG:SYSINIT_START
7187     ; jmp     far 46Dh:267h
7188     ; 12/12/2023
7189     jmp     far 544h:269h ; (PCDOS 7.1 IBMBIO.COM)
7190
7191     jmp     SYSINITSEG:SYSINITSTART
7192
7193 ; ===== S U B   R O U T I N E =====
7194
7195 ; Following are subroutines to support resident device driver initialization
7196 ;
7197 ; M011 -- note: deleted setup_bdsms and reset_bdsms here
7198
7199 ; M035 -- begin changed section
7200
7201 ; *****
7202 ; module name: remap

```

```

7202 ;
7203 ; descriptive name: all the code for himem that could be separated from msbio
7204 ;
7205 ; function: remap the bds chain to adjusted logical drive numbers (drive
7206 ; letters) if more than two diskette drives on the system.
7207 ;
7208 ; scheme: if more than 2 diskette drives, first map the bds structure
7209 ; as usual and then rescan the bds chain to adjust the drive
7210 ; letters. to do this, scan for disk drives and assign logical
7211 ; drive number starting from 2 and then rescan diskette drives
7212 ; and assign next to the last logical drive number of last disk
7213 ; drive to the 3rd and 4th diskette drives.
7214 ;
7215 ; input: none
7216 ; exit: drive letters have been remapped in bds chain
7217 ; exit error: none
7218 ; called from: msinit
7219 ;
7220 ; notes: this function will be called only if more than 2 diskettes are
7221 ; found in the system
7222 ; this function assumes that there are no more than 26 drives assigned
7223 ; this is guaranteed by the code that creates bdss for partitions
7224 ; this function assumes that the first entries in the chain are
7225 ; floppy drives, and all the rest are hard drives
7226 ; will alter the boot drive if necessary to reflect remapping
7227 ;
7228 ;*****
7229 ;
7230 ; 17/10/2022
7231 ; 02/10/2022
7232 ; 15/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
7233 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2464h)
7234 ; (MSDOS 6.22 IO.SYS - BIOSDATA:1D9Eh)
7235 ;
7236 remap: ; proc near
7237 ;
7238 ; 15/12/2023
7239 ; ds = cs
7240 ;mov di, [cs:start_bds] ; get first bds
7241 mov di, [start_bds]
7242 ;
7243 ; search for 1st fixed disk physical drive num
7244 ;
7245 drive_loop:
7246 cmp byte [di+4], 80h ; [di+BDS.drivenum]
7247 ; first hard disk??
7248 jz short fdrv_found ; yes, continue
7249 mov di, [di] ; [di+BDS.link]
7250 ; get next bds, assume segment
7251 cmp di, -1 ; 0FFFFh ; last bds?
7252 jnz short drive_loop ; loop if not
7253 jmp short rmap_exit ; yes, no hard drive on system
7254 ;
7255 ;-----
7256 ;first disk drive bds, now change the logical drive num to 2 and the subsequent
7257 ;logical drive nums to 3, 4, 5 etc.
7258 ;-----
7259 ;
7260 fdrv_found:
7261 mov al, 2 ; startwith logical drv num=2
7262 fdrv_loop:
7263 mov [di+5], al ; [di+BDS.drivelet]
7264 mov di, [di] ; [di+BDS.link]
7265 ; ds:di--> next bds
7266 ; inc al ; set num for next drive
7267 ; 18/12/2022
7268 inc ax
7269 cmp di, 0FFFFh ; last hard drive ?
7270 jnz short fdrv_loop ; no - assign more disk drives
7271 ;
7272 ;-----
7273 ; now, rescan and find bds of 3rd floppy drive and assign next drive letter
7274 ; in al to 3rd. if the current drive letter is past z, then do not allocate
7275 ; any more.
7276 ;-----
7277 ;
7278 ;mov di, [cs:start_bds] ; [start_bds]
7279 ; 15/12/2023
7280 mov di, [start_bds] ; get first bds
7281 mov di, [di] ; [di+BDS.link]
7282 ; ds:di-->bds2
7283 ;mov ah, [cs:dsktnum] ; get number of floppies to remap
7284 mov ah, [dsktnum]
7285 sub ah, 2 ; adjust for a: & b:
7286 remap_loop1:
7287 mov di, [di] ; [di+BDS.link]
7288 ; set new num to next floppy
7289 mov [di+5], al ; [di+BDS.drivelet]
7290 inc al ; new number for next floppy
7291 dec ah ; count down extra floppies
7292 jnz short remap_loop1
7293 ;
7294 ; now we've got to adjust the boot drive if we reassigned it
7295 ;
7296 ; 15/12/2023
7297 ;mov al, [cs:drvfat]
7298 mov al, [drvfat]
7299 cmp al, 2 ; is ita: or b: ?
7300 jb short rmap_exit
7301 ;sub al, [cs:dsktnum]
7302 sub al, [dsktnum] ; is it one of the other floppies?
7303 jb short remap_boot_flop ; brif so
7304 ;
7305 ; we've got to remap the boot hard drive
7306 ; subtract the number of EXTRA floppies from it
7307 ;
7308 add al, 2 ; bootdrv -= (dsktnum-2)
7309 jmp short remap_change_boot_drv
7310 ; -----
7311 ;
7312 ; we've got to remap the boot floppy.
7313 ; add the number of hard drive partitions to it
7314 ;
7315 remap_boot_flop:
7316 ;add al, [cs:drvmax] ; bootdrv += (drvmax-dsktnum)
7317 ; 15/12/2023
7318 add al, [drvmax]
7319 remap_change_boot_drv:
7320 ;mov [cs:drvfat], al ; alter msdos.sys load drive
7321 mov [drvfat], al
7322 inc al
7323 push ds
7324 mov di, SYSINITSEG ; 46Dh
7325 ;mov di, 544h ; PCDOS 7.1 IBMBIO.COM

```

```

7326             ;;mov di, 46Dh      ; SYSINIT segment
7327 00002143 8EDF      mov ds, di
7328 00002145 A2[9802]  mov [DEFAULTDRIVE], al
7329             ;mov ds:296h, al    ; [SYSINIT+DEFAULT_DRIVE]
7330             ; pass it to sysinit as well
7331 00002148 1F        pop ds ; ds = cs
7332 rmap_exit:
7333 00002149 C3        retn
7334
7335 ; ===== S U B   R O U T I N E =====
7336
7337 ; 17/10/2022
7338 ; 02/10/2022 - Retro DOS v4.0 (MSDOS 5.0 -actual-)
7339 ; 28/12/2018 - Retro DOS v4.0 (MSDOS 6.21 -draft-)
7340 ; 02/06/2018 - Retro DOS v3.0 (MSDOS 3.3)
7341 ; 19/03/2018 - Retro DOS v2.0 (MSDOS 2.11)
7342 ; *****
7343 ; getboot - get the boot sector for a hard disk
7344 ;
7345 ; Reads the boot sector from a specified drive into
7346 ; a buffer at the top of memory.
7347 ;
7348 ; dl = int13 drive number to read boot sector for
7349 ; *****
7350
7351 ; 17/10/2022
7352 bootbias equ 200h
7353
7354 getboot:      ; proc near
7355
7356             ; 15/12/2023 - Retro DOS v5.0
7357             ; (Modified PCDOS 7.1) IBMBIO.COM/IO.SYS
7358             ; ds = cs
7359
7360             ; 08/04/2018
7361             ; Retro DOS v2.0 (IBMBIO.COM, IBMDOS 2.1)
7362             ; 28/03/2018 - MSDOS 6.0 - MSINIT.ASM, 1991
7363             ; 02/10/2022 - Retro DOS v4.0
7364             ; (disassembled IO.SYS code of MSDOS 5.0)
7365
7366             ;mov ax, [cs:init_bootseg] ; 17/10/2022
7367             ; 15/12/2023
7368 0000214A A1[061A]  mov ax, [init_bootseg]
7369 0000214D 8EC0      mov es, ax
7370
7371             ; 17/10/2022
7372 0000214F BB0002    mov bx, bootbias ; 200h
7373             ;mov bx, 200h      ; bootbias
7374             ; load BX, ES:BX is where sector goes
7375 00002152 B80102    mov ax, 201h
7376 00002155 30F6      xor dh, dh
7377 00002157 B90100    mov cx, 1
7378 0000215A CD13      int 13h
7379             ; DISK - READ SECTORS INTO MEMORY
7380             ; AL = number of sectors to read, CH = track, CL = sector
7381             ; DH = head, DL = drive, ES:BX -> buffer to fill
7382             ; Return: CF set on error, AH = status, AL = number of sectors read
7383             jc short erret
7384             ; 17/10/2022
7385             cmp word [es:bootbias+1FEh], 0AA55h
7386             ;cmp word ptr es:3FEh, 0AA55h ; [es:bootbias+1FEh]
7387             ; Dave Litton magic word?
7388             jz short norm_ret ; yes
7389 erret:      stc
7390 norm_ret:   retn
7391 00002168 C3
7392
7393 ; ===== S U B   R O U T I N E =====
7394
7395 ; 28/12/2018 - Retro DOS v4.0
7396
7397 ; *****
7398 ; sethard - generate bpb for a variable sized hard file. ibm has a
7399 ; partitioned hard file; we must read physical sector 0 to determine where
7400 ; our own logical sectors start. we also read in our boot sector to
7401 ; determine version number
7402 ;
7403 ; inputs: dl is rom drive number (80...)
7404 ; bh is partition number (0...)
7405 ; ds:di points to bds
7406 ; outputs: carry clear -> bpb is filled in
7407 ; carry set -> bpb is left uninitialized due to error
7408 ; trashes (at least) si, cx
7409 ; MUST PRESERVE ES:!!!!
7410 ; *****
7411
7412             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
7413             ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:24E9h)
7414
7415 sethard:     ; proc near
7416             ; 12/08/2023
7417             ; ds = cs = BIOSDATA
7418 00002169 57        push di
7419 0000216A 53        push bx
7420             ;push ds ; ds = cs = BIOSDATA ; 12/08/2023
7421 0000216B 06        push es
7422 0000216C 885D05    mov [di+5], bl ; [di+BDS.drivelet]
7423 0000216F 885504    mov [di+4], dl ; [di+BDS.drivenum]
7424             ; 16/12/2023
7425 00002172 804D3F01  or byte [di+3Fh], 1 ; PCDOS 7.1
7426             ;or byte [di+23h], 1 ; [di+BDS.flags]
7427             ; fnon_removable
7428 00002176 C6453E05  mov byte [di+3Eh], 5 ; PCDOS 7.1
7429             ;mov byte [di+22h], 5 ; [di+BDS.formfactor]
7430             ; fHardFile
7431 0000217A C606[081A]00 mov byte [fbigfat], 0 ; assume 12 bit FAT
7432 0000217F 88FE      mov dh, bh ; partition number
7433 00002181 52        push dx
7434 00002182 B408      mov ah, 8
7435 00002184 CD13      int 13h
7436             ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
7437             ; DL = drive number
7438             ; Return: CF set on error, AH = status code, BL = drivetype
7439             ; DL = number of consecutive drives
7440             ; DH = maximum value for head number, ES:DI -> drive parameter
7441             ;inc dh
7442             ; 16/12/2023 - Retro DOS v5.0
7443 00002186 88F2      mov dl, dh
7444 00002188 B600      mov dh, 0
7445 0000218A 42        inc dx
7446 0000218B 895515    ;mov [di+15h], dh ; [di+BDS.heads] ; get number of heads
7447 0000218E 5A        mov [di+15h], dx
7448 0000218F 7253      pop dx
7449             jc short setret ; error if no hard disk
7450             ; 16/12/2023

```



```

7450             ;jc      short setret_j
7451
7452             and      cl, 3Fh
7453             mov      [di+13h], cl ; [di+BDS.secptrack]
7454             push     dx           ; save partition number
7455             call     getboot
7456             pop      dx           ; restore partition number
7457             jc       short setret
7458             ; 16/12/2023
7459             jnc      short chk_act_part
7460 ;setret_j:
7461             jmp      setret
7462
7463             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
7464 chk_act_part:
7465             xor      bx, bx ; 0
7466             ;mov     [cs:ep_start_sector], bx
7467             ;mov     [cs:ep_start_sector+2], bx
7468             ;mov     [cs:ep_hidden_secs], bx
7469             ;mov     [cs:ep_hidden_secs+2], bx
7470             ; 16/12/2023
7471             ; ds = cs
7472             ; 20/12/2023
7473             ;mov     [ep_start_sector], bx
7474             ;mov     [ep_start_sector+2], bx
7475             mov     [ep_hidden_secs], bx
7476             mov     [ep_hidden_secs+2], bx
7477
7478             mov     bx, 3C2h ; 1C2h+bootbias
7479
7480             ; The first 'active' partition is 00, the second is 01....
7481             ; then the remainder of the 'primary' but non-active partitions
7482
7483 act_part:
7484             test     byte [es:bx-4], 80h ; is the partition active?
7485             jz       short no_act ; no
7486             ; 16/12/2023
7487             %if 0
7488             ; 16/12/2023
7489             ; reject if partitiontype != 1, 4, 6, 0Bh, 0Ch, 0Eh
7490             cmp     byte [es:bx], 1 ; FAT12
7491             jz       short got_good_act
7492             cmp     byte [es:bx], 4 ; FAT16 CHS (<= 32MB)
7493             jz       short got_good_act
7494
7495             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
7496             cmp     byte [es:bx], 0Bh ; FAT32 CHS
7497             jz       short got_good_act
7498             cmp     byte [es:bx], 0Ch ; FAT32 LBA
7499             jz       short got_good_act
7500             cmp     byte [es:bx], 0Eh ; FAT16 LBA
7501             jz       short got_good_act
7502
7503             cmp     byte [es:bx], 6 ; FAT16 BIG CHS (> 32MB)
7504             jnz      short no_act
7505             ;%else
7506             ; 16/12/2023
7507             mov     al, [es:bx] ; partition type
7508
7509             ; reject if partitiontype != 1, 4, 6, 0Bh, 0Ch, 0Eh
7510             cmp     al, 1 ; FAT12
7511             je       short got_good_act
7512             cmp     al, 4 ; FAT16 CHS (<= 32MB)
7513             je       short got_good_act
7514
7515             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
7516             cmp     al, 0Bh ; FAT32 CHS
7517             je       short got_good_act
7518             cmp     al, 0Ch ; FAT32 LBA
7519             je       short got_good_act
7520             cmp     al, 0Eh ; FAT16 LBA
7521             je       short got_good_act
7522
7523             cmp     al, 6 ; FAT16 BIG CHS (> 32MB)
7524             jne      short no_act
7525             %endif
7526             ; 16/12/2023 - Retro DOS v5.0 (Modified PC DOS 7.1)
7527             ; check if it is a primary dos partition
7528
7529             call     chk_partition_type
7530             jne      short no_act
7531
7532 got_good_act:
7533             ; 11/08/2023
7534             or       dh, dh ; is this our target partition #?
7535             ; (0 = first primary dos or active partition)
7536             jz       short set2 ; WE GOT THE ONE WANTED!!
7537             dec      dh ; count down
7538
7539 no_act:
7540             add     bx, 16
7541             cmp     bx, 402h ; 202h+bootbias
7542             ; last entry done?
7543             jnz      short act_part ; no, process next entry
7544             mov     bx, 3C2h ; 1C2h+bootbias
7545             ; restore original value of bx
7546
7547             ; Now scan the non-active partitions
7548
7549 get_primary:
7550             test     byte [es:bx-4], 80h
7551             jnz      short not_prim ; we've already scanned
7552             ; the ACTIVE ones
7553             ; 16/12/2023
7554             %if 0
7555             ; 16/12/2023
7556             ; see if partitiontype == 1, 4, 6, 0Bh, 0Ch, 0Eh
7557             cmp     byte [es:bx], 1 ; FAT12
7558             jz       short got_prim
7559             cmp     byte [es:bx], 4 ; FAT16 CHS (<= 32MB)
7560             jz       short got_prim
7561
7562             ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
7563             cmp     byte [es:bx], 0Bh ; FAT32 CHS
7564             jz       short got_prim
7565             cmp     byte [es:bx], 0Ch ; FAT32 LBA
7566             jz       short got_prim
7567             cmp     byte [es:bx], 0Eh ; FAT16 LBA
7568             jz       short got_prim
7569
7570             cmp     byte [es:bx], 6 ; FAT16 BIG CHS (> 32MB)
7571             jnz      short not_prim
7572             ;%else
7573             ; 16/12/2023
7574             mov     al, [es:bx] ; partition type

```

```

7574
7575 ; see if partitiontype == 1, 4, 6, 0Bh, 0Ch, 0Eh
7576 cmp     al, 1 ; FAT12
7577 je      short got_prim
7578 cmp     al, 4 ; FAT16 CHS (<= 32MB)
7579 je      short got_prim
7580
7581 ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
7582 cmp     al, 0Bh ; FAT32 CHS
7583 je      short got_prim
7584 cmp     al, 0Ch ; FAT32 LBA
7585 je      short got_prim
7586 cmp     al, 0Eh ; FAT16 LBA
7587 je      short got_prim
7588
7589 cmp     al, 6 ; FAT16 BIG CHS (> 32MB)
7590 jne     short not_prim
7591 %endif
7592 ; 16/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
7593 ; check if it is a primary dos partition
7594
7595 000021D0 E81500 call     chk_partition_type
7596 000021D3 7506 jne     short not_prim
7597
7598 got_prim:
7599 000021D5 08F6 or      dh, dh ; is this our target partition?
7600 000021D7 7431 jz      short set2
7601 000021D9 FCEC dec     dh
7602
7603 000021DB 83C310 add     bx, 16
7604 000021DE 81FB0204 cmp     bx, 402h ; 202h+bootbias
7605 000021E2 75E5 jnz     short get_primary ; loop till we've gone through table
7606
7607 000021E4 F9 stc ; error return
7608 000021E5 E9C703 jmp     ret_hard_err
7609
7610 ; -----
7611 ; 16/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
7612
7613 chk_partition_type:
7614 ; 16/12/2023
7615 mov     al, [es:bx] ; partition type
7616 000021E8 268A07
7617
7618 ; see if partitiontype == 1, 4, 6, 0Bh, 0Ch, 0Eh
7619 000021EB 3C01 cmp     al, 1 ; FAT12
7620 000021ED 7412 je      short chk_ptype_retn
7621 000021EF 3C04 cmp     al, 4 ; FAT16 CHS (<= 32MB)
7622 000021F1 740E je      short chk_ptype_retn
7623
7624 ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
7625 000021F3 3C0B cmp     al, 0Bh ; FAT32 CHS
7626 000021F5 740A je      short chk_ptype_retn
7627 000021F7 3C0C cmp     al, 0Ch ; FAT32 LBA
7628 000021F9 7406 je      short chk_ptype_retn
7629 000021FB 3C0E cmp     al, 0Eh ; FAT16 LBA
7630 000021FD 7402 je      short chk_ptype_retn
7631
7632 000021FF 3C06 cmp     al, 6 ; FAT16 BIG CHS (> 32MB)
7633 chk_ptype_retn:
7634 ; zf = 1 -> primary DOS partition
7635 ; zf = 0 -> not a primary DOS partition
7636 00002201 C3 retn
7637
7638 ; -----
7639 ; 16/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
7640 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:25B6h)
7641
7642 ep_start_sector:
7643 00002202 00000000 dd 0
7644 00002206 00000000 ep_hidden_secs: dd 0
7645
7646 ; -----
7647 ; until we get the real logical boot record and get the bpb,
7648 ; BDS_BPB.BPB_BIGTOTALSECTORS will be used instead of BDS_BPB.BPB_TOTALSECTORS
7649 ; for the convenience of the computation.
7650 ;
7651 ; at the end of this procedure, if a bpb information is gotten from
7652 ; the valid boot record, then we are going to use those bpb information
7653 ; without change.
7654 ;
7655 ; otherwise, if (hidden sectors + total sectors) <= a word, then we will move
7656 ; BDS_BPB.BPB_BIGTOTALSECTORS (low) to BDS_BPB.BPB_TOTALSECTORS and zero out
7657 ; BDS_BPB.BPB_BIGTOTALSECTORS entry to make it a conventional bpb format.
7658
7659 set2:
7660 ; 12/08/2023
7661 ; ds = cs = BIOSDATA segment (0070h)
7662 0000220A 8816[091A] mov     [rom_drv_num], dl
7663 ;mov     [cs:rom_drv_num], dl
7664 ; save the rom bios drive number we are handling now.
7665 0000220E 268B4704 mov     ax, [es:bx+4] ; hidden sectors (start sector)
7666 00002212 268B5706 mov     dx, [es:bx+6]
7667
7668 ; decrement the sector count by 1 to make it zero based. exactly 64k
7669 ; sectors should be allowed
7670
7671 sub     ax, 1
7672 sbb     dx, 0
7673 add     ax, [es:bx+8] ; sectors in partition
7674 adc     dx, [es:bx+10]
7675 jnc     short okdrive
7676 or      byte [fbigfat], 80h ; ftoobig
7677
7678 ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
7679 ;;;
7680
7681 okdrive:
7682 ;add     ax, [cs:ep_hidden_secs]
7683 ;adc     dx, [cs:ep_hidden_secs+2]
7684 ; ds = cs
7685 0000222B 0306[0622] add     ax, [ep_hidden_secs]
7686 0000222F 1316[0822] adc     dx, [ep_hidden_secs+2]
7687 00002233 7305 jnc     short okdrive_1
7688 00002235 800E[081A]80 or      byte [fbigfat], 80h ; ftoobig
7689
7690 okdrive_1:
7691 0000223A 26803F0C cmp     byte [es:bx], 0Ch ; FAT32 LBA partition ID
7692 0000223E 7418 je      short set_lba_flag
7693 00002240 26803F0E cmp     byte [es:bx], 0Eh ; FAT16 LBA partition ID
7694 00002244 7412 je      short set_lba_flag
7695 00002246 3B5513 cmp     dx, [di+13h] ; if dx > [di+BDS.secptrack] then
7696 00002249 730D jnb     short set_lba_flag ; set LBA r/w flag
7697 0000224E 31D2 div     word [di+13h]
xor     dx, dx

```



```

7698 00002250 F77515      div    word [di+15h]
7699 00002253 3D0004      cmp    ax, 400h          ; if ax (cylinder number) >= 1024
7700                                ; set LBA r/w flag
7701 00002256 7204      jb     short set3
7702 set_lba_flag:
7703 00002258 804D4004    or     byte [di+40h], 4 ; fLBArw ; LBA r/w flag
7704                                ;;;
7705 ;okdrive:
7706                                ; 16/12/2023
7707 set3:
7708                                ;mov    ax, [es:bx+4]
7709                                ;mov    [di+17h], ax ; [di+BDS.hiddensecs]
7710                                ;                                ; BPB_HIDDESECTORS = p->partitionbegin
7711                                ;mov    ax, [es:bx+6]
7712                                ;mov    [di+19h], ax ; [di+BDS.hiddensecs+2]
7713                                ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
7714                                ;;;
7715                                mov    ax, [es:bx+4] ; start sector (LBA) of the partition
7716 0000225C 268B4704    mov    dx, [es:bx+6]
7717 00002260 268B5706    ;add    ax, [cs:ep_hidden_secs]
7718                                ;adc    dx, [cs:ep_hidden_secs+2]
7719                                ; ds = cs
7720                                add    ax, [ep_hidden_secs]
7721 00002264 0306[0622]    ;                                ; + hidden secs of the extd dos partion
7722                                adc    dx, [ep_hidden_secs+2]
7723 00002268 1316[0822]    mov    [di+17h], ax ; [di+BDS.hiddensecs]
7724 0000226C 894517      mov    [di+19h], dx ; [di+BDS.hiddensecs+2]
7725 0000226F 895519      xor    ax, ax ; 0
7726 00002272 31C0      mov    [di+7Bh], ax ; [di+BDS.bds_hidden_trks]
7727 00002274 89457B      mov    [di+0Eh], ax ; [di+BDS.totalsec16]
7728 00002277 89450E      ;;;
7729
7730                                mov    dx, [es:bx+10] ; # of sectors (high)
7731 0000227A 268B570A    mov    ax, [es:bx+8] ; # of sectors (low)
7732 0000227E 268B4708    mov    [di+1Dh], dx ; [di+BDS.totalsecs32+2]
7733 00002282 89551D      mov    [di+1Bh], ax ; [di+BDS.totalsecs32]
7734 00002285 89451B      ;                                ; bpb->maxsec = p->partitionlength
7735                                ;cmp    dx, 0
7736                                ;ja     short okdrive_1
7737                                ; 16/12/2023
7738                                or     dx, dx
7739 00002288 09D2      jnz    short set3_read
7740 0000228A 7505      cmp    ax, 64          ; if (p->partitionlength < 64)
7741 0000228C 83F840    ;jb     short setret    ; return -1;
7742                                ;jb     short set3_err
7743 0000228F 7264      ;okdrive_1:
7744                                ; 16/12/2023
7745                                set3_read:
7746                                mov    dx, [di+19h] ; [di+BDS.hiddensecs+2]
7747 00002291 8B5519      mov    ax, [di+17h] ; [di+BDS.hiddensecs]
7748 00002294 8B4517      xor    bx, bx          ; boot sector number - for mini disk
7749 00002297 31DB      ;                                ; usually equal to the # of sec/trk.
7750                                mov    bl, [di+13h] ; [di+BDS.secpertrack]
7751 00002299 8A5D13      push   ax
7752 0000229C 50      mov    ax, dx
7753 0000229D 89D0      xor    dx, dx
7754 0000229F 31D2      div    bx
7755 000022A1 F7F3      ; (sectors)dx:ax / (BDS.secpertrack)bx =
7756                                ; (track)temp_h:ax + (sector)dx
7757                                ; 16/12/2023
7758                                %if 0
7759                                ; 17/10/2022
7760                                ;mov    [cs:temp_h], ax
7761                                ; 12/08/2023 (ds=cs)
7762                                mov    [temp_h], ax
7763                                pop     ax
7764                                div     bx
7765                                mov     cl, dl
7766                                inc     cl
7767                                xor     bx, bx
7768                                mov     bl, [di+15h] ; [di+BDS.heads]
7769                                push    ax
7770                                xor     dx, dx
7771                                ;mov    ax, [cs:temp_h]
7772                                mov     ax, [temp_h] ; 12/08/2023
7773                                div     bx
7774                                ;mov    [cs:temp_h], ax
7775                                mov     [temp_h], ax ; 12/08/2023
7776                                pop     ax
7777                                div     bx          ; dl is head, ax is cylinder
7778                                ; 12/08/2023 (ds=cs)
7779                                cmp     word [temp_h], 0
7780                                ;cmp    word [cs:temp_h], 0
7781                                ja     short setret_brdg ; exceeds the limit of int 13h
7782                                cmp     ax, 1024
7783                                ja     short setret_brdg ; exceeds the limit of int 13h
7784                                ; Retro DOS v3.2 note by Erdogan Tan - 28/07/2019
7785                                ; **MSDOS code accepts if ax = 1024 but it is nonsense here
7786                                ; ('ja' must be 'jnb')
7787                                okdrive_2:
7788                                ; 28/07/2019
7789                                ; dl is head.
7790                                ; ax is cylinder
7791                                ; cl is sector number (assume less than 2**6 = 64 for int 13h)
7792                                ;*** for mini disks ***
7793                                cmp     word [di+47h], 1 ; [di+BDS.bds_hidden_trks]
7794                                ;                                ; check for mini disk
7795                                jnz     short oknotmini ; not mini disk.
7796                                add     ax, [di+49h] ; [di+BDS.bds_hidden_trks]
7797                                ;                                ; set the physical track number
7798                                oknotmini:
7799                                %endif
7800                                ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
7801                                ;;;
7802                                ;mov    [cs:saved_word], ax
7803                                mov     [saved_word], ax
7804                                pop     ax
7805 000022A3 A3[9E04]    div     bx
7806 000022A6 58      mov     cl, dl
7807 000022A7 F7F3      inc     cl
7808 000022A9 88D1      mov     bx, [di+15h] ; [di+BDS.heads]
7809 000022AB FEC1      push   ax
7810 000022AD 8B5D15      xor     dx, dx
7811 000022B0 50      ;mov    ax, [cs:saved_word]
7812 000022B1 31D2      mov     ax, [saved_word]
7813                                div     bx
7814                                ;mov    [cs:saved_word], ax
7815 000022B3 A1[9E04]    mov     [saved_word], ax ; not necessary !? (ax must be 0)
7816 000022B6 F7F3      pop     ax
7817 000022B8 A3[9E04]    div     bx          ; dl is head, ax is cylinder
7818 000022BB 58      ; 16/12/2023
7819 000022BC F7F3      push    cs
7820
7821 000022BE 0E

```

```

7822 000022BF 07                pop     es ; (*)
7823 000022C0 BB[5201]          mov     bx, disksector ; (**)
7824                                ;
7825 000022C3 F6454004          test    byte [di+40h], 4 ; fLBArw ; LBA read/write flag
7826 000022C7 742F              jz      short set3_chs_read
7827 set3_lba_read:
7828                                ; 16/12/2023
7829                                %if 0
7830                                ;push    cs
7831                                ;pop     es ; (*)
7832                                ;mov     bx, disksector ; (**)
7833
7834                                ;push    ds
7835                                ;push    si
7836                                xor     ax, ax ; 0
7837                                push    ax
7838                                push    ax
7839                                mov     ax, [di+19h] ; [di+BDS.hiddensectors+2]
7840                                push    ax
7841                                mov     ax, [di+17h] ; [di+BDS.hiddensectors]
7842                                push    ax
7843                                push    es ; buffer address
7844                                push    bx
7845                                mov     ax, 1 ; sector (read) count
7846                                push    ax
7847                                ;mov     ax, 16 ; DAP size
7848                                mov     al, 16
7849                                push    ax
7850                                mov     dl, [rom_drv_num] ; ds = cs
7851                                mov     ax, ss
7852                                mov     ds, ax ; ds = ss
7853                                mov     si, sp
7854                                ;mov     dl, [cs:rom_drv_num]
7855                                mov     ah, 42h
7856                                int     13h ; DISK - IBM/MS Extension
7857                                ; EXTENDED READ (DL - drive, DS:SI - disk address packet)
7858
7859                                ;pop     si
7860                                ;pop     ds
7861                                jnc     short set3_lba_read_ok
7862                                add     sp, 16
7863                                ;pop     si
7864                                ;pop     ds
7865 set3_err:
7866                                ;jmp     setret
7867                                jmp     ret_hard_err
7868
7869 set3_lba_read_ok
7870                                add     sp, 16
7871                                ;pop     si
7872                                ;pop     ds
7873                                jmp     short set3_read_ok
7874 %else
7875                                ; 16/12/2023
7876                                ;push    si ; * ; (not necessary)
7877                                ;mov     si, empty_dap_buff ; dap_buffer
7878 000022C9 BE[591B]          mov     si, dap_buffer ; empty_dap_buff
7879 000022CC 56                push    si
7880 000022CD 87F7          xchg    si, di
7881                                ; si = BDS
7882                                ; di = DAP buffer
7883 000022CF B81000          mov     ax, 16 ; DAP size
7884 000022D2 AB                stosw    ; DAP size
7885 000022D3 B001          mov     al, 1 ; sector (read) count
7886 000022D5 AB                stosw    ; buffer address
7887                                ; DAP size
7888 000022D6 89D8          mov     ax, bx ; offset disksector
7889 000022D8 AB                stosw
7890 000022D9 8CC0          mov     ax, es ; es=ds=cs = BIOSDATA segment
7891 000022DB AB                stosw
7892                                ; sector address (bits 0 to 31)
7893 000022DC 8B4417          mov     ax, [si+17h] ; [di+BDS.hiddensectors]
7894 000022DF AB                stosw
7895 000022E0 8B4419          mov     ax, [si+19h] ; [di+BDS.hiddensectors+2]
7896 000022E3 AB                stosw
7897                                ; sector address bits 32 to 63 (0)
7898 000022E4 31C0          xor     ax, ax ; 0
7899 000022E6 AB                stosw
7900 000022E7 AB                stosw
7901                                ;xchg    di, si
7902 000022E8 89F7          mov     di, si
7903                                ; di = BDS
7904 000022EA 5E                pop     si ; DAP buffer address
7905
7906 000022EB 8A16[091A]        mov     dl, [rom_drv_num] ; ds = cs
7907 000022EF B442          mov     ah, 42h
7908 000022F1 CD13          int     13h ; DISK - IBM/MS Extension
7909                                ; EXTENDED READ (DL - drive, DS:SI - disk address packet)
7910
7911 000022F3 7324          jnc     short set3_read_ok
7912 set3_err:
7913                                ;jmp     setret
7914 000022F5 E9B702          jmp     ret_hard_err
7915 %endif
7916
7917 set3_chs_read:
7918 000022F8 837D7901        cmp     word [di+79h], 1 ; [di+BDS.bdsmini] ; check for mini disk
7919 000022FC 7503          jnz     short oknotmini
7920 000022FE 03457B          add     ax, [di+7Bh] ; [di+BDS.bdsmini_hidden_trks]
7921                                ;;
7922
7923 oknotmini:
7924 ;*** end of added logic for mini disk
7925
7926 00002301 D0CC          ror     ah, 1 ; move high two bits of cyl to high
7927 00002303 D0CC          ror     ah, 1 ; two bits of upper byte
7928 00002305 80E4C0          and     ah, 0C0h ; turn off remainder of bits
7929 00002308 08E1          or      cl, ah ; move two bits to correct spot
7930 0000230A 88C5          mov     ch, al ; ch is cylinder (low 8 bits)
7931                                ; cl is sector + 2 high bits of cylinder
7932 0000230C 88D6          mov     dh, dl ; dh is head
7933
7934                                ; 12/08/2023 (ds=cs)
7935 0000230E 8A16[091A]        mov     dl, [rom_drv_num]
7936                                ;mov     dl, [cs:rom_drv_num] ; dl is drive number
7937
7938                                ; cl is sector + 2 high bits of cylinder
7939                                ; ch is low 8 bits of cylinder
7940                                ; dh is head
7941                                ; dl is drive
7942
7943                                ; for convenience, we are going to read the logical boot sector
7944                                ; into cs:disksector area.
7945

```

```

7946 ; read in boot sector using bios disk interrupt. the buffer where it
7947 ; is to be read in is cs:disksector.
7948
7949 ; 16/12/2023
7950 ; es=ds=cs = BIOSDATA segment
7951 ; bx = disksector ; (**)
7952
7953 ;push cs
7954 ;pop es ; (*)
7955
7956 ;mov bx, disksector ; for convenience,
7957 ; we are going to read the logical boot sector
7958 ; into cs:disksector area.
7959 00002312 B80102 mov ax, 201h
7960 00002315 CD13 int 13h ; DISK - READ SECTORS INTO MEMORY
7961 ; AL = number of sectors to read, CH = track, CL = sector
7962 ; DH = head, DL = drive, ES:BX -> buffer to fill
7963 ; Return: CF set on error, AH = status, AL = number of sectors read
7964
7965 00002317 72DC ; 16/12/2023
7966 jc short set3_err
7967
7968 ; cs:disksec contains the boot sector. in theory, (ha ha) the bpb in this thing
7969 ; is correct. we can, therefore, suck out all the relevant statistics on the
7970 ; media if we recognize the version number.
7971
7972 set3_read_ok:
7973 ; 11/08/2023
7974 ;mov bx, disksector ; BIOSDATA:014Eh ; MSDOS 6.21 ; 11/08/2023
7975 ; BIOSDATA:0152h ; PCDOS 7.1 IBMBIO.COM
7976 ; 18/12/2023
7977 ;push bx ; +
7978 ;push ax ; (not necessary)
7979
7980 ; 16/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
7981 ;;;
7982 00002319 81BFFE0155AA cmp word [bx+1FEh], 0AA55h
7983 0000231F 7541 jne short invalid_boot_record
7984 ; 16/12/2023
7985 ; 12/08/2023
7986 ; ds = cs = BIOSDATA segment ('disksector:' is in BIOSDATA)
7987 00002321 803FE9 cmp byte [bx], 0E9h ; is it a near jump?
7988 00002324 740B je short check_1_ok ; yes
7989 00002326 803FEB cmp byte [bx], 0EBh ; is it a short jump?
7990 00002329 7537 jne short invalid_boot_record ; no
7991 0000232B 807F0290 cmp byte [bx+2], 90h ; yes, is the next one a nop?
7992 0000232F 7531 jne short invalid_boot_record ; no, invalid bs ; 11/08/2023
7993
7994 check_1_ok:
7995 00002331 837F1600 cmp word [bx+16h], 0 ; [bx+BPB_FATSz16]
7996 00002335 740E jz short check_1 ; 16 bit FAT size is 0 if it is FAT32 bs
7997 ; 16/12/2023
7998 jz short check_2 ; FAT32 bs
7999
8000 ; FAT16 or FAT12 bs
8001
8002 ;push ds
8003 ;push si ; (not necessary)
8004 00002337 57 push di
8005 ; es=ds=cs = BIOSDATA segment
8006 ;push es
8007 ;pop ds
8008
8009 ;mov cx, 28
8010 00002338 B90E00 mov cx, 14 ; *
8011 0000233B 8D7724 lea si, [bx+24h] ; move offset 36 to 63
8012 ; to offset 64 (28 bytes)
8013 0000233E 8D7F40 lea di, [bx+40h] ; boot sector offset 64
8014 00002341 FC cld ; (not necessary, 'std' is not used before here)
8015 ;rep movsb
8016 00002342 F3A5 rep movsw ; *
8017 00002344 5F pop di
8018 ;pop si
8019 ;pop ds
8020 ;;;
8021 ; 16/12/2023
8022 %if 0
8023 ;check_1:
8024 ; 12/08/2023
8025 ; ds = cs = BIOSDATA segment ('disksector:' is in BIOSDATA)
8026 00002348 803FE9 cmp byte [bx], 0E9h ; is it a near jump?
8027 0000234B 740B je short check_1_ok ; yes
8028 0000234D 803FEB cmp byte [bx], 0EBh ; is it a short jump?
8029 00002350 7537 jne short invalid_boot_record ; no
8030 00002353 807F0290 cmp byte [bx+2], 90h ; yes, is the next one a nop?
8031 00002356 7531 jne short invalid_boot_record ; no, invalid bs ; 11/08/2023
8032
8033 check_1_ok:
8034 %endif
8035
8036 ; 18/12/2023
8037 %if 0
8038 ; 14/08/2023
8039 check_2:
8040 mov bx, disksector+11 ; disksector+EXT_BOOT.BPB
8041 ;mov bx, 159h ; disksector+EXT_BOOT.BPB
8042 ; point to the bpb in the boot record
8043 ;mov al, [cs:bx+10] ; [bx+EBPB.MEDIADSCRIPTOR]
8044 mov al, [bx+10] ; 12/08/2023
8045 ; get the mediadescriptor byte
8046 and al, 0F0h ; mask off low nibble
8047 cmp al, 0F0h ; is high nibble = 0Fh?
8048 jne short invalid_boot_record ; no, invalid boot record
8049 ;cmp word [cs:bx], 512 ; [bx+EBPB.BYTESPERSECTOR]
8050 cmp word [bx], 512 ; 12/08/2023
8051 jne short invalid_boot_record ; invalidate non 512 byte sectors
8052
8053 check2_ok:
8054 ; yes, mediadescriptor ok.
8055 mov al, [bx+2] ; 12/08/2023
8056 ;mov al, [cs:bx+2] ; now make sure that
8057 ; the sectorspercluster is
8058 ; a power of 2
8059 ;
8060 ; [bx+EBPB.SECTORSPERCLUSTER]
8061 ; get the sectorspercluster
8062 %endif
8063 ;;;
8064 check_2:
8065 ; 18/12/2023
8066 00002345 8A4715 ; bx = disksector
8067 mov al, [bx+21] ; [bx+EXT_BOOT.BPB+EBPB.MEDIADSCRIPTOR]
8068 ; get the mediadescriptor byte
8069 and al, 0F0h ; mask off low nibble
8070 cmp al, 0F0h ; is high nibble = 0Fh?

```

```

8070 0000234C 7514                jne     short invalid_boot_record ; no, invalid boot record
8071 0000234E 817F0B0002          cmp     word [bx+11], 512 ; [bx+EXT_BOOT.BPB+EBPB.BYTESPERSECTOR]
8072 00002353 750D                jne     short invalid_boot_record ; invalidate non 512 byte sectors
8073
8074                                check2_ok: ; yes, mediadescriptor ok.
8075 00002355 8A470D          mov     al, [bx+13] ; now make sure that
8076                                ; the sectorspercluster is
8077                                ; a power of 2
8078                                ;
8079                                ; [bx++EXT_BOOT.BPB+EBPB.SECTORSPERCLUSTER]
8080                                ; get the sectorspercluster
8081                                ;
8082                                ;;;
8083 00002358 08C0          or      al, al ; is it zero?
8084 0000235A 7406          jz      short invalid_boot_record ; yes, invalid boot record
8085
8086                                ck_power_of_two:
8087 0000235C D0E8          shr     al, 1 ; shift until first bit emerges
8088 0000235E 73FC          jnc     short ck_power_of_two
8089 00002360 7406          jz      short valid_boot_record
8090
8091                                invalid_boot_record:
8092                                ; 18/12/2023
8093                                ; pop ax
8094                                ; pop bx ; +
8095 00002362 E96001          jmp     unknown ; jump to invalid boot record
8096                                ; unformatted or illegal media.
8097                                ; 16/12/2023
8098                                ; -----
8099                                ; 12/08/2023
8100                                ; setret_brdg:
8101                                ; jmp setret
8102                                ; -----
8103
8104                                unknown3_0_j:
8105 00002365 E96101          jmp     unknown3_0 ; legally formatted media,
8106                                ; although, content might be bad.
8107                                ; -----
8108
8109                                valid_boot_record:
8110                                ; 18/12/2023
8111                                ; pop ax
8112                                ; pop bx ; +
8113                                ;
8114                                ; 18/12/2023
8115                                ; bx = offset disksector ; +
8116
8117                                ; Signature found. Now check version.
8118                                ;
8119                                ; 14/08/2023
8120 00002368 817F08322E          cmp     word [bx+8], '2.'
8121                                ; cmp word [cs:bx+8], '2.' ; 03/10/2022 (NASM syntax)
8122                                ; cmp word ptr cs:[bx+8], 2E32h ; '2.'
8123 0000236D 7506          jne     short try5
8124 0000236F 807F0A30          cmp     byte [bx+10], '0'
8125                                ; cmp byte [cs:bx+0Ah], '0' ; 03/10/2022 (NASM syntax)
8126                                ; cmp byte ptr cs:[bx+0Ah], 30h ; '0'
8127                                ; 12/08/2023
8128                                ; jnz short try5
8129                                ; jmp short copybpb
8130 00002373 7425          je      short copybpb
8131
8132                                ; -----
8133                                ; 12/08/2023
8134                                ; setret_brdg:
8135                                ; jmp setret
8136                                ; -----
8137
8138                                ; unknown3_0_j:
8139                                ; jmp unknown3_0 ; legally formatted media,
8140                                ; although, content might be bad.
8141                                ; -----
8142
8143                                try5:
8144 00002375 E83B02          call    cover_fdisk_bug
8145
8146                                ; see if it is an os2 signature
8147                                ;
8148                                ; 12/08/2023
8149                                ; ds = cs = BIOSDATA segment
8150 00002378 817F08302E          cmp     word [bx+8], '0.'
8151                                ; cmp word [cs:bx+8], '0.' ; 03/10/2022 (NASM syntax)
8152                                ; cmp word ptr cs:[bx+8], 2E30h ; '0.'
8153 0000237D 750C          jne     short no_os2
8154 0000237F 8A4707          mov     al, [bx+7] ; 12/08/2023
8155                                ; mov al, [cs:bx+7] ; 17/10/2022 (NASM syntax)
8156 00002382 2C31          sub     al, '1'
8157                                ; sub al, 31h ; '1'
8158 00002384 24FE          and     al, 0FEh
8159 00002386 7412          jz      short copybpb ; accept either '1' or '2'
8160 00002388 E93A01          jmp     unknown
8161                                ; -----
8162
8163                                ; no os2 signature, this is to check for real dos versions
8164
8165                                no_os2:
8166                                ; 12/08/2023
8167                                ; ds = cs = BIOSDATA
8168 0000238B 817F08332E          cmp     word [bx+8], '3.'
8169                                ; cmp word [cs:bx+8], '3.' ; 03/10/2022 (NASM syntax)
8170                                ; cmp word ptr cs:[bx+8], 2E33h ; '3.'
8171 00002390 72D3          jb      short unknown3_0_j ; must be 2.1 boot record.
8172                                ; do not trust it, but still legal.
8173 00002392 7506          jnz     short copybpb ; honor os2 boot record
8174                                ; or dos 4.0 version
8175 00002394 807F0A31          cmp     byte [bx+10], '1' ; 12/08/2023
8176                                ; cmp byte [cs:bx+10], '1'
8177                                ; cmp byte ptr cs:[bx+0Ah], 31h ; '1'
8178 00002398 72CB          jb      short unknown3_0_j ; if version >= 3.1, then o.k.
8179
8180                                copybpb:
8181                                ; 03/10/2022
8182
8183                                ; we have a valid boot sector. use the bpb in it to build the
8184                                ; bpb in bios. it is assumed that only
8185                                ; BDS_BPB.BPB_SECTORSPERCLUSTER
8186                                ; BDS_BPB.BPB_ROOTENTRIES, and
8187                                ; BDS_BPB.BPB_SECTORSPERFAT
8188                                ; need to be set (all other values in already). fbigfat is also set.
8189
8190                                ; if it is non fat based system, then just copy the bpb from the boot sector
8191                                ; into the bpb in bds table, and also set the boot serial number, volume id,
8192                                ; and system id according to the boot record.
8193                                ; for the non_fat system, don't need to set the other value. so just do goodret.

```

```

8194
8195 ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
8196 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2787h)
8197 ;;;
8198 ; 17/12/2023
8199 0000239A BE[5D01] mov si, disksector+11
8200 ;sub ch, ch ; (ch may be > 0)
8201 0000239D 29C9 sub cx, cx ; 0
8202 ;mov cl, [disksector+16] ; BPB_NumFATS
8203 0000239F 8A4C05 mov cl, [si+5] ; number of FATS
8204
8205 ; NOTE: This check is not proper for FAT32 boot sector (standard spec)
8206 ; (after PCDOS 7.1). So, it is not existing in Windows ME IO.SYS
8207 ; Erdogan Tan - 01/09/2023 ((IBMBIO.COM 7.1 disassembly note))
8208
8209 ;cmp word ptr cs:disksector+4Dh, 0 ; ???
8210 ;cmp word [disksector+4Dh], 0
8211 ;jnz short check_3
8212
8213 ; 17/12/2023
8214 ; check extended boot signature (0x29)
8215 ;
8216 ; (***) NOTE: 28 bytes of FAT16/FAT12 boot sector from offset 36
8217 ; have been moved to offset 64 (see label 'check_1_ok:' above)
8218 ; ((now, BS_BootSig is at offset 66 even if it was at offset 38))
8219
8220 ;cmp cs:disksector+42h, 29h ; BS_BootSig (FAT32)
8221 000023A2 803E[9401]29 cmp byte [disksector+42h], 29h ; BS_BootSig (***)
8222 ;jmp short check_4
8223
8224 ;cmp cs:disksector+26h, 29h ; BS_BootSig (FAT16/FAT12)
8225 ;cmp byte [disksector+26h], 29h ; (***)
8226
8227 000023A7 7538 jnz short copybpb_fat ; conventional fat system
8228
8229 ; 17/12/2023
8230 %if 0
8231 ; 10/12/2022
8232 ; (number of FATS optimization)
8233 mov si, disksector+11 ; disksector+0Bh
8234 ;mov cl, [cs:disksector+10h] ; Number of FATS (may be 2 or 1)
8235 ;mov cl, [cs:si+05h]
8236 ; 12/08/2023
8237 ; ds = cs = BIOSDATA segment (0070h)
8238 mov cl, [si+05h] ; number of FATS
8239
8240 cmp byte [si+1Bh], 29h ; 12/08/2023
8241 ;cmp byte [cs:si+1Bh], 29h ; 10/12/2022
8242 ;cmp byte [cs:disksector+26h], 29h ; 17/10/2022
8243 ; [disksector+EXT_BOOT.SIG]
8244 ; EXT_BOOT_SIGNAATURE
8245 jnz short copybpb_fat ; conventional fat system
8246
8247 ; 03/10/2022
8248 ; 29/12/2018 - Retro DOS v4.0 modification note:
8249 ; Regarding 'fat_big_small' part of this (MSDOS 6.0) code
8250 ; number of FATS must be 2 ; =*?=
8251 ; (Otherwise, '# of data sectors' would be calculated as wrong!!!)
8252 ;
8253 ;cmp byte [disksector+EXT_BOOT.BPB+EBPB.NUMBEROFFATS], 2 ; =*?=
8254
8255 ; 10/12/2022
8256 ;cmp byte [cs:disksector+10h], 0
8257 ; [disksector+EXT_BOOT.BPB+EBPB.NUMBEROFFATS]
8258 ;jnz short copybpb_fat ; a fat system.
8259 or cl, cl ; [cs:disksector+10h]
8260 jnz short copybpb_fat ; a fat system.
8261
8262 %endif
8263
8264 ; 17/12/2023 - Retro DOS v5.0
8265 ;cmp byte [cs:disksector+10h], 0 ; BPB.fats
8266 ;cmp byte [disksector+10h], 0 ; BPB_NumFATS
8267 ;jnz short copybpb_fat ; a fat system
8268 ; 17/12/2023
8269 000023A9 20C9 and cl, cl ; 0 ?
8270 000023AB 7534 jnz short copybpb_fat ; a fat system
8271
8272 ; non fat based media.
8273
8274 000023AD 57 push di ; BDS
8275 ; 12/08/2023
8276 ;push ds ; ds = cs = BIOSDATA segment
8277
8278 ; 17/12/2023
8279 ; es = ds = cs
8280 ;push ds
8281 ;pop es
8282
8283 ; 12/08/2023
8284 ; ds = cs
8285 ;push cs
8286 ;pop ds
8287
8288 ; 10/12/2022
8289 ; (number of FATS optimization)
8290 ; SI = disksector+11
8291 ; 17/10/2022
8292 ;mov si, 159h ; disksector+EXT_BOOT.BPB
8293 ;mov si, disksector+11
8294 000023AE 83C706 add di, 6 ; add di,BDS.BPB
8295
8296 ; just for completeness, we'll make sure that total_sectors and
8297 ; big_total_sectors aren't both zero. I've seen examples of
8298 ; this on DOS 3.30 boot records. I don't know exactly how it
8299 ; got that way. If it occurs, then use the values from the
8300 ; partition table.
8301
8302 ; 17/12/2023
8303 ; cx = 0
8304 ; 18/12/2022
8305 ;sub cx, cx
8306
8307 ;cmp word [cs:si+8], 0 ; [cs:si+EBPB.TOTALSECTORS]
8308 ;jnz short already_nonz
8309 ; how about big_total?
8310 ;cmp word [cs:si+15h], 0 ; [cs:si+EBPB.BIGTOTALSECTORS]
8311 ;jnz short already_nonz ; we're okay if any are != 0
8312 ;cmp word [cs:si+17h], 0 ; [cs:si+EBPB.BIGTOTALSECTORS+2]
8313 ;jnz short already_nonz
8314
8315 ; 12/08/2023
8316 ; ds = cs = BIOSDATA segment (0070h)
8317

```

```

8318 ; 17/12/2023
8319 ; 12/08/2023
8320 000023B1 394C08 cmp [si+8], cx ; 0 ; [si+EBPB.TOTALSECTORS]
8321 000023B4 751C jnz short already_nonz ; how about big_total?
8322 ;
8323 000023B6 394C15 cmp [si+15h], cx ; 0 ; [si+EBPB.BIGTOTALSECTORS]
8324 000023B9 7517 jnz short already_nonz ; we're okay if any are != 0
8325 000023BB 394C17 cmp [si+17h], cx ; 0 ; [si+EBPB.BIGTOTALSECTORS+2]
8326 000023BE 7512 jnz short already_nonz
8327
8328 ; now let's copy the values from the partition table (now in the BDS)
8329 ; into the BPB in the boot sector buffer, before they get copied back.
8330
8331 000023C0 8B4508 mov ax, [di+8] ; [di+BDS.totalsecs16]
8332 ; 12/08/2023
8333 ;mov [cs:si+8], ax ; [cs:si+EBPB.TOTALSECTORS]
8334 000023C3 894408 mov [si+8], ax
8335 000023C6 8B4515 mov ax, [di+15h] ; [di+BDS.totalsecs32]
8336 ;mov [cs:si+15h], ax ; [cs:si+EBPB.BIGTOTALSECTORS]
8337 000023C9 894415 mov [si+15h], ax
8338 000023CC 8B4517 mov ax, [di+17h] ; [di+BDS.totalsecs32+2]
8339 ;mov [cs:si+17h], ax ; [cs:si+EBPB.BIGTOTALSECTORS+2]
8340 000023CF 894417 mov [si+17h], ax
8341
8342 already_nonz:
8343 ; 18/12/2022
8344 ; cx = 0
8345 ;mov cl, 25
8346 ;mov cx, 25 ; A_BPB.size - 6 ; Use SMALL version!
8347 ; 17/12/2023 - Retro DOS v5.0
8348 000023D2 B135 mov cl, 53 ; PC DOS 7.1 IBMBIO.COM
8349 ; BDS.BPB size (25 + 28 for FAT32 parms)
8350 000023D4 F3A4 rep movsb
8351 ;pop ds
8352 ; 12/08/2023
8353 ; ds = cs
8354 ;pop bp ; ds (on top of stack) = BIOSDATA
8355 000023D6 5F pop di ; BDS
8356 ;push es
8357 ;push ds
8358 ;pop es
8359 ;push cs
8360 ;pop ds
8361 ; 12/08/2023
8362 ;mov es, bp
8363 ; ds = cs = es
8364
8365 ; 14/08/2023
8366 000023D7 BD[4F08] mov bp, MOV MEDIAIDS ; mov_media_ids
8367 ; 18/12/2022
8368 ;mov bp, mov_media_ids
8369 ;;mov bp, 751h ; mov_media_ids
8370 ; at 2C7h:751h = 70h:2CC1h
8371 ; set volume id, systemid, serial.
8372 000023DA 0E push cs ; simulate far call
8373 000023DB E895F6 call call_bios_code
8374 ; 12/08/2023
8375 ; ds = cs = es
8376 ;push es
8377 ;pop ds
8378 ;pop es
8379 000023DE E9C701 jmp goodret
8380
8381 ; -----
8382
8383 ; ***** cas ---
8384 ; IBM DOS 3.30 doesn't seem to mind that the TOTAL_SECTORS and
8385 ; BIG_TOTAL_SECTORS field in the boot sector are 0000. This
8386 ; happens with some frequency -- perhaps through some OS/2 setup
8387 ; program. We haven't actually been COPYING the TOTAL_SECTORS
8388 ; from the boot sector into the DPB anyway, we've just been using
8389 ; it for calculating the fat size. Pretty scary, huh? For now,
8390 ; we'll go ahead and copy it into the DPB, except in the case
8391 ; that it equals zero, in which case we just use the values in
8392 ; the DPB from the partition table.
8393
8394 ; 17/10/2022
8395 ;MOV MEDIAIDS equ mov_media_ids - DOSBIOSEG_2C7h ; (751h for MSDOS 5.0 IO.SYS)
8396 ;CLEARIDS equ clear_ids - DOSBIOSEG_2C7h ; (5D9h for MSDOS 5.0 IO.SYS)
8397 ; 09/12/2022
8398 ;MOV MEDIAIDS equ mov_media_ids
8399 ;CLEARIDS equ clear_ids
8400 ; 11/09/2023
8401 ;CLEARIDS_x equ clear_ids_x
8402
8403 copybpb_fat:
8404 ; 17/12/2023
8405 ; ch = 0, cl = number of FATs
8406 ; 10/12/2022
8407 ; (number of FATs optimization)
8408 ; SI = disksector+11
8409 ; 17/10/2022
8410 ;mov si, disksector+11
8411 ;;mov si, 159h ; disksector+EXT_BOOT.BPB
8412 ; ; cs:si -> bpb in boot
8413
8414 ; 17/12/2023
8415 ; dx = 0
8416 ; 08/05/2024
8417 000023E1 31D2 xor dx, dx
8418
8419 ; 12/08/2023
8420 ; ds = cs = BIOSDATA segment (0070h)
8421 000023E3 8B4408 mov ax, [si+8]
8422 ;mov ax, [cs:si+8] ; [cs:si+EBPB.TOTALSECTORS]
8423 ; ; get totsec from boot sec
8424 000023E6 09C0 or ax, ax
8425 000023E8 7514 jnz short copy_totsec ; if non zero, use that
8426 000023EA 8B4415 mov ax, [si+15h] ; 12/08/2023
8427 ;mov ax, [cs:si+15h] ; [cs:si+EBPB.BIGTOTALSECTORS]
8428 ; ; get the big version
8429 ; ; (32 bit total sectors)
8430 000023ED 8B5417 mov dx, [si+17h] ; 12/08/2023
8431 ;mov dx, [cs:si+17h] ; [cs:si+EBPB.BIGTOTALSECTORS+2]
8432 ; 10/12/2022
8433 ; (number of FATs optimization)
8434 ; CL = number of FATs (2 or 1)
8435 000023F0 89D3 mov bx, dx
8436 000023F2 09C3 or bx, ax
8437 000023F4 7508 jnz short copy_totsec
8438 ; screw it. it was bogus.
8439 000023F6 8B4518 mov ax, [di+18h] ; [di+BDS.totalsecs32]
8440 000023F9 8B551D mov dx, [di+1Dh] ; [di+BDS.totalsecs32+2]
8441 000023FC EB06 jmp short fat_big_small

```



```

8442
8443
8444
8445
8446
8447 000023FE 89451B
8448
8449 00002401 89551D
8450
8451
8452
8453
8454
8455
8456
8457
8458
8459
8460
8461
8462
8463
8464
8465
8466
8467
8468 00002404 8B5C03
8469
8470
8471 00002407 895D09
8472
8473 0000240A 29D8
8474 0000240C 83DA00
8475
8476 0000240F 8B5C0B
8477
8478
8479 00002412 895D11
8480
8481
8482
8483 00002415 53
8484 00002416 09DB
8485 00002418 753A
8486
8487
8488
8489
8490
8491
8492
8493
8494
8495
8496
8497
8498
8499
8500
8501
8502
8503
8504
8505
8506
8507
8508
8509
8510
8511
8512
8513
8514
8515
8516
8517
8518
8519
8520
8521
8522 0000241A 8B5C19
8523
8524 0000241D 29D8
8525 0000241F 1B541B
8526
8527 00002422 49
8528 00002423 7FF8
8529
8530 00002425 895D1F
8531 00002428 8B5C1B
8532 0000242B 895D21
8533
8534 0000242E 8B5C1D
8535 00002431 895D23
8536 00002434 8B5C1F
8537 00002437 895D25
8538 0000243A 8B5C21
8539 0000243D 895D27
8540 00002440 8B5C23
8541 00002443 895D29
8542 00002446 8B5C25
8543 00002449 895D2B
8544 0000244C 8B5C27
8545 0000244F 895D2D
8546 00002452 EB08
8547
8548
8549
8550
8551
8552
8553
8554
8555
8556
8557 00002454 49
8558 00002455 D3E3
8559
8560
8561 00002457 29D8
8562 00002459 83DA00
8563
8564
8565 0000245C 8B5C06

;mov cx, dx
;or cx, ax ; see if it is a big zero
;jz short totsec_already_set ; screw it. it was bogus.
copy_totsec:
mov [di+1Bh], ax ; [di+BDS.totalsecs32]
; make DPB match boot sec
mov [di+1Dh], dx ; [di+BDS.totalsecs32+2]
; 10/12/2022
;totsec_already_set:
;mov ax, [di+1Bh] ; [di+BDS.totalsecs32]
;mov dx, [di+1Dh] ; [di+BDS.totalsecs32+2]
; determine fat entry size.
fat_big_small:
;at this moment dx;ax = total sector number
;Do not assume 1 reserved sector. Update the reserved sector field in BDS
;from the BPB on the disk
; 12/08/2023
; ds = cs = BIOSDATA segment (0070h)
mov bx, [si+3]
;mov bx, [cs:si+3] ; [cs:si+EBPB.RESERVEDSECTORS]
; get #reserved_sectors from BPB
mov [di+9], bx ; [di+BDS.reseectors]
; update BDS field
sub ax, bx
sbb dx, 0 ; update the count
; 12/08/2023
mov bx, [si+0Bh]
;mov bx, [cs:si+0Bh] ; [cs:si+EBPB.SECTORSSECFAT]
; bx = sectors/fat
mov [di+11h], bx ; [di+BDS.fatsecs]
; set in bds bpb
; 17/12/2023 - Retro DOS v5.0
; (PCDOS 7.1 IBMBIO.COM)
push bx ; FAT sectors
or bx, bx
jnz short fat_16bit
; 17/12/2023
;if 0
sub ax, [si+19h] ; FAT32 file system (PCDOS 7.1 BUG!)
; BPB.FATSz32
sbb dx, [si+1Bh] ; BPB.FATSz32+2 (PCDOS 7.1 BUG!)
; dx:ax = partition size - (one FAT sectors + reserved sects)
mov bx, [si+19h] ; BPB.FATSz32
mov [di+1Fh], bx ; [di+BDS.fatsecs32]
mov bx, [si+1Bh] ; BPB.FATSz32+2
mov [di+21h], bx ; [di+BDS.fatsecs32+2]
mov bx, [si+1Dh] ; BPB.BPB_ExtFlags
mov [di+23h], bx ; [di+BDS.extflags]
mov bx, [si+1Fh] ; BPB.FSver
mov [di+25h], bx ; [di+BDS.fsver]
mov bx, [si+21h] ; BPB.RootClus
mov [di+27h], bx ; [di+BDS.rootdirclust]
mov bx, [si+23h] ; BPB.RootClus+2
mov [di+29h], bx ; [di+BDS.rootdirclust+2]
mov bx, [si+25h] ; BPB.FSInfo
mov [di+2Bh], bx ; [di+BDS.fsinfo]
mov bx, [si+27h] ; BPB.FSInfo+2
mov [di+2Dh], bx ; [di+BDS.fsinfo+2]
jmp short fat_32bit ; PCDOS 7.1 BUG! Erdogan Tan - 8/8/2023
; correct code (would be):
; mov cl, [cs:si+05h] ; BPB_NumFATS
; sub_fat32_size:
; sub ax, [cs:si+19h] ; BPB_FATSz32
; sbb dx, [cs:si+1Bh] ; BPB_FATSz32+2
; dec cl
; jg short sub_fat32_size
; jmp short fat_32bit
;endif
; 17/12/2023
; cl = BPB_NumFATS (2 or 1)
; ch = 0
mov bx, [si+19h] ; BPB.FATSz32
sub_fat32_size:
sub ax, bx
sbb dx, [si+1Bh] ; BPB.FATSz32+2
dec cl
dec cx
jg short sub_fat32_size
mov [di+1Fh], bx ; [di+BDS.fatsecs32]
mov bx, [si+1Bh] ; BPB.FATSz32+2
mov [di+21h], bx ; [di+BDS.fatsecs32+2]
mov bx, [si+1Dh] ; BPB.BPB_ExtFlags
mov [di+23h], bx ; [di+BDS.extflags]
mov bx, [si+1Fh] ; BPB.FSver
mov [di+25h], bx ; [di+BDS.fsver]
mov bx, [si+21h] ; BPB.RootClus
mov [di+27h], bx ; [di+BDS.rootdirclust]
mov bx, [si+23h] ; BPB.RootClus+2
mov [di+29h], bx ; [di+BDS.rootdirclust+2]
mov bx, [si+25h] ; BPB.FSInfo
mov [di+2Bh], bx ; [di+BDS.fsinfo]
mov bx, [si+27h] ; BPB.FSInfo+2
mov [di+2Dh], bx ; [di+BDS.fsinfo+2]
jmp short fat_32bit
fat_16bit:
; 17/12/2023 - Retro DOS v5.0
; (PCDOS 7.1 IBMBIO.COM)
; 10/12/2022
; (number of FATs optimization)
; CL = number of FATs (2 or 1)
; CH = 0 ; 17/12/2023
;dec cl ; *
; 18/12/2022
dec cx ; *
shl bx, cl
;shl bx, 1 ; *=? ; always 2 fats
sub ax, bx ; sub # fat sectors
sbb dx, 0
fat_32bit:
; 17/12/2023
mov bx, [si+6] ; 12/08/2023

```

```

8566      ;mov     bx, [cs:si+6] ; [cs:si+EBPB.ROOTENTRIES]
8567      ;         ; # root entries
8568 0000245F 895D0C      mov     [di+0Ch], bx ; [di+BDS.direntries]
8569      ;         ; set in bds bpb
8570 00002462 B104      mov     cl, 4
8571 00002464 D3EB      shr     bx, cl ; div by 16 ents/sector
8572 00002466 29D8      sub     ax, bx ; sub #dir sectors
8573 00002468 83DA00      sbb     dx, 0 ;
8574      ;         ; dx:ax now contains the
8575      ;         ; # of data sectors
8576      ; 17/12/2023
8577      ; ch = 0
8578      ;xor     cx, cx ; *
8579 0000246B 8A4C02      mov     cl, [si+2] ; 12/08/2023
8580      ;mov     cl, [cs:si+2] ; [cs:si+EBPB.SECTORSPERCLUSTER]
8581      ;         ; sectors per cluster
8582 0000246E 884D08      mov     [di+8], cl ; [di+BDS.secpclus]
8583      ;         ; set in bios bpb
8584 00002471 50          push    ax
8585 00002472 89D0      mov     ax, dx
8586 00002474 31D2      xor     dx, dx
8587 00002476 F7F1      div     cx ; cx = sectors per cluster
8588      ; 12/08/2023 (ds=cs)
8589      ;mov     [temp_h], ax
8590      ;mov     [cs:temp_h], ax ; [temp_h]:ax now contains the
8591      ;         ; # clusters.
8592      ; 17/12/2023
8593 00002478 A3[9E04]      mov     [saved_word], ax ; hw of cluster number
8594 0000247B 58          pop     ax
8595 0000247C F7F1      div     cx
8596      ; 17/12/2023
8597      ;cmp     word [cs:temp_h], 0
8598      ;cmp     word [temp_h], 0 ; 12/08/2023
8599      ;cmp     word [saved_word], 0 ; (*)
8600      ;ja     short toobig_ret ; too big cluster number
8601
8602      ; 17/12/2023
8603      ;;;
8604 0000247E 5B          pop     bx ; FAT sectors (16 bit)
8605      ;and     bx, bx ; 0 ?
8606 0000247F 09DB      or      bx, bx ; 0 ?
8607 00002481 751F      jnz     short chk_clnum_hw
8608      ;         ; 16 bit fat sectors > 0 ; FAT12 or FAT16 fs
8609
8610 00002483 813E[9E04]FF0F      cmp     word [saved_word], 0FFFFh
8611 00002489 7503      jne     short fat32_clust_limit
8612 0000248B 83F8F6      cmp     ax, 0FFF6h ; FAT32 cluster number limit: 0FFFFFF6h
8613      fat32_clust_limit:
8614      ja     short short toobig_ret ; too big cluster number
8615      cmp     [saved_word], bx ; 0 ?
8616      jnz     short fat16_clust_limit
8617 00002494 7505      jnz     short set_fbigbig_flag ; 17/12/2023
8618      fat16_clust_limit: ; 17/12/2023
8619 00002496 83F8F6      cmp     ax, 0FFF6h ; FAT16 cluster number limit: 0FFF6h
8620      ;fat16_clust_limit:
8621 00002499 760E      jna     short fat12_clust_limit ; jbe
8622      set_fbigbig_flag: ; 17/12/2023
8623 0000249B 800E[081A]20      or      byte [fbigfat], 20h ; fbigbig ; FAT32 fs
8624 000024A0 EB11      jmp     short copymediaid
8625      chk_clnum_hw:
8626 000024A2 833E[9E04]00      cmp     word [saved_word], 0 ; (*)
8627 000024A7 7714      ja     short toobig_ret ; too big cluster number
8628      ;;;
8629      fat12_clust_limit:
8630      cmp     ax, 0FF6h ; 4096-10
8631      ;         ; is this 16-bit fat?
8632 000024AC 7205      jb     short copymediaid ; no, small fat
8633      ; 17/10/2022
8634 000024AE 800E[081A]40      or      byte [fbigfat], 40h ; fbig ; FAT16 fs
8635      ;or      ds:fbigfat, 40h ; fbig
8636      ;         ; 16 bit fat
8637      copymediaid:
8638      ; 17/12/2023
8639      ; es = ds = cs
8640
8641      ;push     es
8642      ;push     ds
8643      ;pop      es
8644
8645      ; 12/08/2023
8646      ; ds = cs = BIOSDATA
8647      ;push     cs
8648      ;pop      ds
8649      ; 17/10/2022
8650 000024B3 BD[4F08]      mov     bp, MOVMEDIAIDS
8651      ;mov     bp, 865h ; (PCDOS 7.1 IBMBIO.COM)
8652      ;mov     bp, 751h ; mov_media_ids
8653      ;         ; at 2C7h:751h = 70h:2CC1h
8654      ;         ; copy filesys_id, volume label
8655      ;         ; simulate far call
8656 000024B7 E8B9F5      push    cs
8657      call    call_bios_code
8658
8659      ; 12/08/2023
8660      ;push     es
8661      ;pop      ds
8662      ; 17/12/2023
8663      ;pop      es
8664 000024BA E9CD00      jmp     message_bpb ; now final check for bpb info
8665      ;         ; and return.
8666      ; -----
8667
8668      toobig_ret:
8669      ; 12/08/2023 (ds=cs=BIOSDATA)
8670 000024BD 800E[081A]80      or      byte [fbigfat], 80h ; ftoobig
8671      ;or      byte [cs:fbigfat], 80h ; ftoobig
8672      ;         ; too big (32 bit clust #) for FAT16
8673 000024C2 E9E300      jmp     goodret ; still drive letter is assigned
8674      ;         ; but useless. to big for
8675      ;         ; current pc dos fat file system
8676      ; -----
8677
8678      unknown:
8679      ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8680 000024C5 804D4002      or      byte [di+40h], 2 ; [di+BDS.flags+1]
8681      ;         ; unformatted_media
8682      ; 12/12/2022
8683      ;or      byte [di+24h], 02h
8684      ;or      word [di+23h], 200h ; [di+BDS.flags]
8685      ;         ; unformatted_media
8686      ;         ; Set unformatted media flag.
8687
8688      ; the boot signature may not be recognizable,
8689      ; but we should try and read it anyway.

```



```

8690
8691
8692 000024C9 8B551D      unknown3_0:  mov     dx, [di+1Dh]    ; skip setting unformatted_media bit
8693                                     ; [di+BDS.totalsecs32+2]
8694 000024CC 8B451B      mov     ax, [di+1Bh]    ; [di+BDS.totalsecs32]
8695 000024CF BE[181A]    mov     si, disktable2
8696                                     ; 08/08/2023
8697                                     ; cmp     dx, [cs:si]    ; total sectors hw
8698                                     ; 12/08/2023 (ds=cs)
8699 000024D2 3B14      cmp     dx, [si]
8700 000024D4 720C      jb      short gotparm
8701 000024D6 7705      ja      short scan_next
8702                                     ; cmp     ax, [cs:si+2]    ; total sectors lw
8703 000024D8 3B4402    cmp     ax, [si+2]
8704 000024DB 7605      jbe     short gotparm
8705
8706 000024DD 83C60A    scan_next:  add     si, 10          ; 5*2
8707 000024E0 EBF0      jmp     short scan      ; covers upto 512 mb media
8708
8709 ; -----
8710
8711 000024E2 8A4C08    gotparm:    mov     cl, [si+8]    ; fat size for fbigfat flag
8712                                     ; or      ds:fbigfat, cl
8713                                     ; 17/10/2022
8714 000024E5 080E[081A]  or      [fbigfat], cl  ; (fbig flag, 40h or 0) ; 08/08/2023
8715                                     ; 12/08/2023
8716                                     ; ds = cs = BIOSDATA
8717 000024E9 8B4C04    mov     cx, [si+4]
8718                                     ; mov     cx, [cs:si+4]    ; ch = number of sectors per cluster
8719                                     ; cl = log base 2 of ch
8720 000024EC 8B5406    mov     dx, [si+6]
8721                                     ; mov     dx, [cs:si+6]    ; dx = number of root dir entries
8722
8723 ; now calculate size of fat table
8724
8725 000024EF 89550C    mov     [di+0Ch], dx    ; [di+BDS.direntries]
8726                                     ; save number of (root)      dir entries
8727 000024F2 8B551D    mov     dx, [di+1Dh]    ; [di+BDS.totalsecs32+2]
8728 000024F5 8B451B    mov     ax, [di+1Bh]    ; [di+BDS.totalsecs32]
8729 000024F8 8B6D08    mov     [di+8], ch      ; [di+BDS.secpclus]
8730                                     ; save sectors per cluster
8731
8732                                     ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8733 000024FB F606[081A]60  test    byte [fbigfat], 60h ; fbig+fbigbig ; FAT16 or FAT32
8734                                     ; 11/09/2023
8735                                     ; 17/10/2022
8736                                     ; test    byte [fbigfat], 40h
8737                                     ; test    ds:fbigfat, 40h    ; fbig
8738                                     ; if (fbigfat)
8739 00002500 751E      jnz     short dobig     ; goto dobig; (16 bit fat)
8740
8741 ; we don't need to change "small fat" logic since it is guaranteed
8742 ; that double word total sector will not use 12 bit fat (unless
8743 ; it's sectors/cluster >= 16 which will never be in this case.)
8744 ; so in this case we assume dx = 0 !!
8745
8746 00002502 31DB      xor     bx, bx          ; 12 bit fat (FAT12 fs)
8747 00002504 88EB      mov     bl, ch
8748 00002506 4B        dec     bx
8749 00002507 01C3      add     bx, ax          ; dx=0
8750 00002509 D3EB      shr     bx, cl          ; bx = 1+(bpb->maxsec+BDS.secpclus-1)/
8751 0000250B 43        inc     bx              ; BDS.secpclus
8752 0000250C 80E3FE    and     bl, 0FEh        ; bx &= ~1; (=number of clusters)
8753 0000250F 89DE      mov     si, bx
8754 00002511 D1EB      shr     bx, 1
8755 00002513 01F3      add     bx, si          ; number of FAT bytes ; 08/08/2023
8756 00002515 81C3FF01  add     bx, 511          ; bx += 511 + bx/2
8757 00002519 D0EF      shr     bh, 1           ; bh >= 1; (=bx/512)
8758 0000251B 887D11    mov     [di+11h], bh     ; [di+BDS.fatsecs]
8759                                     ; save number of fat sectors
8760 0000251E EB6A      jmp     short message_bpb
8761
8762 ; -----
8763
8764 ; for bigfat we do need to extend this logic to 32 bit sector calculation.
8765
8766 00002520 B104      dobig:      mov     cl, 4           ; 16 (2^4) directory entries per sector
8767 00002522 52        push    dx             ; save total sectors (high)
8768 00002523 8B550C    mov     dx, [di+0Ch]    ; [di+BDS.direntries]
8769 00002526 D3EA      shr     dx, cl          ; root dir sectors = BDS.direntries / 16;
8770 00002528 29D0      sub     ax, dx
8771 0000252A 5A        pop     dx
8772 0000252B 83DA00    sbb     dx, 0           ; dx:ax= total sectors - root dir sectors
8773 0000252E 83E801    sub     ax, 1
8774 00002531 83DA00    sbb     dx, 0           ; dx:ax= t - r - d
8775                                     ; total secs - reserved      secs - root dir      secs
8776 00002534 B302      mov     bl, 2
8777 00002536 8A7D08    mov     bh, [di+8]      ; [di+BDS.secpclus]
8778                                     ; bx = 256 * BDS.secpclus + 2
8779
8780 ; I don't understand why to add bx here!!!
8781
8782                                     ; 29/12/2018 - Erdogan Tan (Retro DOS v4.0)
8783                                     ; 27/09/2022
8784                                     ; (Microsoft FAT32 File      System Specification,
8785                                     ; December 2000, Page 21)
8786                                     ; TmpVal1 = DskSize - (BPB_ResvdSecCnt+RootDirSectors)
8787                                     ; TmpVal2 = (256*BPB_SecPerClus)+BPB_NumFATS
8788                                     ; 8/8/2023 (Retro DOS v5.0)
8789                                     ; If(FATType == FAT32)
8790                                     ;     TmpVal2 = TmpVal2 / 2;
8791                                     ; FATSz = (TmpVal1+(TmpVal2-1))/TmpVal2
8792                                     ; 8/8/2023 (Retro DOS v5.0)
8793                                     ; If(FATType == FAT32) {
8794                                     ;     BPB_FATSz16 = 0;
8795                                     ;     BPB_FATSz32 = FATSz;
8796                                     ; } else {
8797                                     ;     BPB_FATSz16 = LOWORD(FATSz);
8798                                     ; /* there is no BPB_FATSz32 in a FAT16 BPB */
8799                                     ; }
8800                                     ; dx:ax = TmpVal1, bx = TmpVal2
8801 00002539 01D8      add     ax, bx
8802 0000253B 83D200    adc     dx, 0           ; dx:ax = TmpVal1+TmpVal2
8803 0000253E 83E801    sub     ax, 1
8804 00002541 83DA00    sbb     dx, 0           ; dx:ax = TmpVal1+TmpVal2-1
8805
8806 ;;;
8807                                     ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8808 00002544 F606[081A]20  test    byte [fbigfat], 20h ; fbigbig (FAT32) flag
8809 00002549 740D      jz      short dobig1
8810
8811 0000254B D1EB      shr     bx, 1           ; TmpVal2 = TmpVal2 / 2
8812                                     ; dx:ax = TmpVal1+(2*TmpVal2)-1
8813 0000254D 83E81F    sub     ax, 31          ; reserved sectors = 32 (for FAT32 fs) /// 1+31 = 32

```

```

8814 00002550 83DA00          sbb     dx, 0
8815 00002553 29D8          sub     ax, bx
8816 00002555 83DA00          sbb     dx, 0          ; dx:ax = Tmpval1+(2*Tmpval2)-Tmpval2-1
8817                                     ;      = Tmpval1+(Tmpval2-1)
8818
8819 00002558 50          dobig1:      push    ax          ; save 1w of dividend
8820 00002559 89D0          mov     ax, dx          ; divide hw of dx:ax at first (as 1st stage)
8821 0000255B 31D2          xor     dx, dx
8822 0000255D F7F3          div     bx          ; 32 bit division, dx:ax/bx
8823                                     ; remainder in dx is hw of 2nd stage dividend
8824 0000255F 89C5          mov     bp, ax          ; hw of quotient
8825 00002561 58          pop     ax          ; restore 1w of dividend (of 1st stage)
8826                                     ;;;
8827
8828                                     ; assuming dx in the table will never be bigger than bx.
8829
8830 00002562 F7F3          div     bx          ; BDS.fatsecs =
8831                                     ; ceil((total-dir-res)/(256*BDS.secpclus+2))
8832 00002564 894511      mov     [di+11h], ax    ; [di+BDS.fatsecs]
8833                                     ; number of fat          sectors
8834                                     ;;;
8835
8836                                     ; 18/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8837 00002567 8A1E[081A]  mov     bl, [fbigfat]
8838 0000256B 885D3B      mov     [di+3Bh], bl    ; [di+BDS.fatsiz] ; fat size flag
8839
8840 0000256E F6C320      test    bl, 20h        ; fbigbig (FAT32) flag
8841 00002571 7410          jz      short dobig2    ; not FAT32
8842
8843 00002573 89451F      mov     [di+1Fh], ax    ; [di+BDS.fatsecs32]
8844 00002576 896D21      mov     [di+21h], bp    ; [di+BDS.fatsecs32+2]
8845 00002579 C745110000  mov     word [di+11h], 0 ; [di+BDS.fatsecs] = 0
8846                                     ; clear 16 bit FAT size field
8847 0000257E C745092000  mov     word [di+9], 32  ; [di+BDS.resectors]
8848                                     ; set reserved sectors to 32 (FAT32 de facto)
8849
8850 dobig2:
8851                                     ;;;
8852
8853                                     ; now, set the default filesystem_id, volume label, serial number
8854                                     ; 05/08/2023
8855                                     ; [di+1Fh] = [fbigfat]
8856                                     ;
8857                                     ; mov     bl, ds:fbigfat
8858                                     ; 17/10/2022
8859                                     ; mov     bl, [fbigfat]
8860                                     ; mov     [di+1Fh], bl    ; [di+BDS.fatsiz] ; fat          size flag
8861
8862                                     ; 12/08/2023
8863                                     ; push     ds ; ds = cs = BIOSDATA
8864
8865                                     ; 17/12/2023
8866                                     ; es = ds = cs
8867                                     ; push     ds
8868                                     ; pop      es
8869
8870                                     ; 12/08/2023
8871                                     ; ds = cs = BIOSDATA
8872                                     ; push     cs
8873                                     ; pop      ds
8874
8875                                     ; 18/12/2023 - Retro DOS v5.0
8876                                     ; bl = [fbigfat] (clear_ids_x uses bl value here)
8877                                     ; 11/09/2023
8878                                     ; mov     al, [fbigfat]
8879 00002583 BD[A106]  mov     bp, CLEARIDS_X ; clear_ids_x (uses AL value here)
8880                                     ; 17/10/2022
8881                                     ; mov     bp, CLEARIDS
8882                                     ; mov     bp, 5D9h          ; clear_ids
8883                                     ; at 2C7h:5D9h = 70h:2B49h
8884                                     ; at BIOSCODE:06ABh
8885                                     ; in PCDOS 7.1 IBMBIO.COM
8886 00002586 0E          push     cs
8887 00002587 E8E9F4      call    call_bios_code
8888
8889                                     ; 12/08/2023
8890                                     ; pop      ds ; ds = cs = BIOSDATA
8891
8892                                     ; at this point, in bpb of bds table, BDS_BPB.BPB_BIGTOTALSECTORS which is
8893                                     ; set according to the partition information. we are going to
8894                                     ; see if (hidden sectors + total sectors) > a word. if it is true,
8895                                     ; then no change. otherwise, BDS_BPB.BPB_BIGTOTALSECTORS will be moved
8896                                     ; to BDS_BPB.BPB_TOTALSECTORS and BDS_BPB.BPB_BIGTOTALSECTORS will be set to 0.
8897                                     ; we don't do this for the bpb information from the boot record. we
8898                                     ; are not going to change the bpb information from the boot record.
8899
8900 message_bpb:
8901                                     ; 05/08/2023
8902                                     ; [di+1Fh] = [fbigfat]
8903                                     ;
8904                                     ; 12/12/2022
8905                                     ; mov     bl, [fbigfat]
8906                                     ; mov     [di+1Fh], bl    ; [di+BDS.fatsiz]
8907                                     ; set size of fat on media
8908                                     ;
8909 0000258A 8B551D      mov     dx, [di+1Dh]    ; [di+BDS.totalsecs32+2]
8910 0000258D 8B451B      mov     ax, [di+1Bh]    ; [di+BDS.totalsecs32]
8911                                     ; 11/09/2023
8912 00002590 09D2          or      dx, dx
8913 00002592 7514          jnz     short goodret
8914                                     ; cmp     dx, 0          ; double word total sectors?
8915                                     ; ja      short goodret ; don't have to change it.
8916                                     ; 12/12/2022
8917                                     ; ja      short short goodret2
8918                                     ; cmp     word [di+19h], 0 ; [di+BDS.hiddensecs+2]
8919                                     ; ja      short goodret ; don't have to change it.
8920                                     ; 12/12/2022
8921 00002594 395519      cmp     [di+19h], dx ; 0
8922                                     ; ja      short goodret2
8923 00002597 770F          ja      short goodret ; 11/09/2023
8924 00002599 034517      add     ax, [di+17h]    ; [di+BDS.hiddensecs]
8925                                     ; jb      short goodret
8926                                     ; 12/12/2022
8927                                     ; jc      short goodret
8928 0000259C 7209          jc      short goodret_c1c ; 11/09/2023
8929 0000259E 8B451B      mov     ax, [di+1Bh]    ; [di+BDS.totalsecs32]
8930 000025A1 89450E      mov     [di+0Eh], ax    ; [di+BDS.totalsecs16]
8931                                     ; mov     word [di+1Bh], 0 ; [di+BDS.totalsecs32]
8932                                     ; 12/12/2022
8933 000025A4 89551B      mov     [di+1Bh], dx ; 0
8934
8935 goodret_c1c:
8936 000025A7 F8          ; 11/09/2023
8937                                     c1c
goodret:

```

```

8938             ;mov     bl, ds:fbigfat
8939             ; 11/09/2023
8940             ; 12/12/2022
8941             ; 17/10/2022
8942 000025A8 8A1E[081A]    mov     bl, [fbigfat]
8943             ; 17/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8944 000025AC 885D3B        mov     [di+3Bh], bl ; [di+BDS.fatsiz]
8945             ;mov     [di+1Fh], bl ; [di+BDS.fatsiz]
8946             ; set size of fat on media
8947             ; 11/09/2023
8948             ;clic
8949 ret_hard_err:
8950             ; 12/12/2022
8951 goodret2:
8952 000025AF 07            pop     es
8953             ;pop     ds ; ds = cs = BIOSDATA ; 14/08/2023
8954 000025B0 5B            pop     bx
8955 000025B1 5F            pop     di
8956 000025B2 C3            retn
8957
8958 ; ===== S U B   R O U T I N E =====
8959
8960 ; 15/10/2022
8961
8962 ;fdisk of pc dos 3.3 and below, os2 1.0 has a bug. the maximum number of
8963 ;sector that can be handled by pc dos 3.3 ibmbio should be 0ffffh.
8964 ;instead, sometimes fdisk use 10000h to calculate the maximum number.
8965 ;so, we are going to check that if BPB_TOTALSECTORS + hidden sector = 10000h
8966 ;then subtract 1 from BPB_TOTALSECTORS.
8967
8968             ; 17/10/2022
8969 cover_fdisk_bug:
8970             ; 12/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
8971             ; ds = cs
8972
8973             ; 18/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
8974             ; (optimization)
8975             ;push    ax
8976             ;push    dx
8977             ;push    si
8978
8979             ; 18/12/2023
8980             ; bx = offset disksector
8981
8982             ; 18/12/2023
8983 000025B3 807F2629      cmp     byte [bx+26h], 29h
8984             ; 12/08/2023
8985             ;cmp     byte [disksector+26h], 29h
8986             ;cmp     byte [cs:disksector+26h], 29h
8987             ;             ; [disksector+EXT_BOOT.SIG],
8988             ;             ; EXT_BOOT_SIGNATURE
8989 000025B7 7426            je      short cfb_retit ; if extended bpb, then     >= pc dos 4.00
8990
8991 000025B9 817F073130     cmp     word [bx+7], 3031h
8992             ;cmp     word [cs:bx+7], 3031h ; '10' ; os2 1.0 = ibm 10.0
8993 000025BE 7506            jne     short cfb_chk_totalsecs ; 11/08/2023
8994 000025C0 807F0A30     cmp     byte [bx+10], '0'
8995             ;cmp     byte [cs:bx+10], '0'
8996 000025C4 7519            jne     short cfb_retit
8997
8998 cfb_chk_totalsecs:
8999             ; 11/08/2023
9000             ; 18/12/2023
9001 %if 0
9002             ; 17/10/2022
9003             mov     si, disksector+11 ; 14Eh+0Bh
9004             ;mov     si, 159h ; disksector+EXT_BOOT.BPB
9005             ; 12/08/2023
9006             cmp     word [si+8], 0
9007             ;cmp     word [cs:si+8], 0 ; [cs:si+EBPB.TOTALSECTORS]
9008             ; just to make sure.
9009             jz      short cfb_retit
9010             ;mov     ax, [cs:si+8] ; [cs:si+EBPB.TOTALSECTORS]
9011             ;add     ax, [cs:si+11h] ; [cs:si+EBPB.HIDDENSECTORS]
9012             ; 12/08/2023
9013             mov     ax, [si+8]
9014             add     ax, [si+11h]
9015
9016             jnb     short cfb_retit
9017             jnz     short cfb_retit
9018             ; if carry set and ax=0
9019             dec     word [si+8]
9020             ;dec     word [cs:si+8] ; 0 -> 0FFFFh
9021             ; then decrease BPB_TOTALSECTORS by 1
9022 %endif
9023
9024             ; 18/12/2023
9025 000025C6 8B4713      cmp     word [bx+19], 0
9026 000025C9 21C0        mov     ax, [bx+19] ; [bx+EBPB.TOTALSECTORS]
9027 000025CB 7412            and     ax, ax ; 0 ?
9028             jz      short cfb_retit
9029
9030             ;mov     ax, [bx+19]
9031             add     ax, [bx+28] ; [bx+EBPB.HIDDENSECTORS]
9032 000025D0 730D        jnc     short cfb_retit
9033 000025D2 750B        jnz     short cfb_retit
9034             ; ax = 0
9035             dec     word [bx+19] ; 0 -> 0FFFFh
9036             ; then decrease BPB_TOTALSECTORS by 1
9037 000025D4 FF4F13      sub     word [di+1Bh], 1 ; [di+BDS.totalsecs32]
9038 000025DB 835D1D00     sbb     word [di+1Dh], 0 ; [di+BDS.totalsecs32+2]
9039
9040 cfb_retit:
9041             ; 18/12/2023
9042             ;pop     si
9043             ;pop     dx
9044             ;pop     ax
9045
9046             retn
9047
9048 ; -----
9049 word2:             dw 2
9050 word3:             dw 3
9051 word512:           dw 512
9052
9053 ; ===== S U B   R O U T I N E =====
9054
9055 ; 15/10/2022
9056
9057 ; setdrvparms sets up the recommended bpb in each bds in the system based on
9058 ; the form factor. it is assumed that the bpbs for the various form factors
9059 ; are present in the bphtable. for hard files, the recommended bpb is the same
9060 ; as the bpb on the drive.
9061 ;
9062 ; no attempt is made to preserve registers since we are going to jump to

```

```

9062 ; sysinit straight after this routine.
9063
9064 ; 18/12/2023 - Retro DOS v5.0
9065 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2A43h)
9066
9067 setdrvparms:
9068 ; 12/12/2023
9069 ; ds = cs
9069 000025E6 31DB xor bx, bx
9070 ; 18/10/2022
9071 000025E8 C43E[1901] les di, [start_bds] ; get first bds in list
9072
9073 _next_bds:
9073 000025EC 06 push es
9074 000025ED 57 push di
9075
9076 ; 18/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
9077 000025EE 268A5D3E mov bl, [es:di+3Eh] ; [es:di+BDS.formfactor]
9078 ;mov bl, [es:di+22h] ; [es:di+BDS.formfactor]
9079
9080 000025F2 80FB05 cmp bl, 5 ; ffHardFile
9081 000025F5 753A jnz short nothardff
9082 000025F7 31D2 xor dx, dx
9083 000025F9 268B450E mov ax, [es:di+0Eh] ; [es:di+BDS.totalsecs16]
9084 000025FD 09C0 or ax, ax
9085 000025FF 7508 jnz short get_ccyl
9086 00002601 268B551D mov dx, [es:di+1Dh] ; [es:di+BDS.totalsecs32+2]
9087 00002605 268B451B mov ax, [es:di+1Bh] ; [es:di+BDS.totalsecs32]
9088
9089 get_ccyl:
9089 00002609 52 push dx
9090 0000260A 50 push ax
9091 0000260B 268B4515 mov ax, [es:di+15h] ; [es:di+BDS.heads]
9092 0000260F 26F76513 mul word [es:di+13h] ; [es:di+BDS.secpertrack]
9093 ; assume sectors per cyl. < 64k.
9094 00002613 89C1 mov cx, ax ; cx has # sectors per cylinder
9095 00002615 58 pop ax
9096 00002616 5A pop dx ; dx:ax = total sectors
9097 00002617 50 push ax
9098 00002618 89D0 mov ax, dx
9099 0000261A 31D2 xor dx, dx
9100 0000261C F7F1 div cx
9101 ; 12/12/2023 ; !!
9102 ; (data segment may not be same with code segment here)
9103 ;mov [cs:temp_h], ax ; ax be0 here.
9104 ; 18/12/2023 - Retro DOS v5.0
9105 ;mov [cs:saved_word], ax
9106 0000261E 58 pop ax
9107 0000261F F7F1 div cx ; div #sec by sec/cyl to get # cyl.
9108 00002621 09D2 or dx, dx
9109 00002623 7401 jz short no_cyl_rnd ; came out even
9110 00002625 40 inc ax ; roundup
9111
9112 no_cyl_rnd:
9113 00002626 26894541 ; 18/12/2023 - Retro DOS v5.0
9114 mov [es:di+41h], ax ; [es:di+BDS.cylinders]
9115 ;mov [es:di+25h], ax ; [es:di+BDS.cylinders]
9116
9117 push es
9117 0000262B 1F pop ds ; !! ; 12/12/2023
9118
9119 0000262C 8D7506 lea si, [di+6] ; [di+BDS.bytespersec]
9120 ; ds:si -> bpb for hard file
9121 0000262F EB55 jmp short set_recbbp
9122
9123 ; -----
9124
9125 nothardff:
9125 00002631 0E push cs
9126 00002632 1F pop ds
9127
9128 ; if fake floppy drive variable is set then we don't have to handle this bds.
9129 ; we can just go and deal with the next bds at label go_to_next_bds.
9130
9131 ; 10/12/2022
9132 ; ds = cs
9133 ; 17/10/2022 (ds=cs)
9134 00002633 803E[131A]01 cmp byte [fakefloppydrv], 1
9135 ;cmp byte [cs:fakefloppydrv], 1
9136 00002638 7454 jz short go_to_next_bds
9137 0000263A 80FB07 cmp bl, 7 ; ffother
9138 ; special case "other" type of medium
9139 0000263D 753D jnz short not_process_other
9140
9141 process_other:
9141 0000263F 31D2 xor dx, dx
9142
9143 ;mov ax, [di+25h] ; [di+BDS.cylinders]
9144 ;mul word [di+36h] ; [di+BDS.rheads]
9145 ;mul word [di+34h] ; [di+BDS.rsecpertrack]
9146 ;mov [di+2Fh], ax ; [di+BDS.rtotalsecs16]
9147 ; have the total number of sectors
9148 ; 18/12/2023 - Retro DOS v5.0
9149 00002641 8B4541 mov ax, [di+41h] ; [di+BDS.cylinders]
9150 00002644 F76552 mul word [di+52h] ; [di+BDS.rheads]
9151 00002647 F76550 mul word [di+50h] ; [di+BDS.rsecpertrack]
9152 0000264A 89454B mov [di+4Bh], ax ; [di+BDS.rtotalsecs16]
9153 ; have the total number of sectors
9154 0000264D 48 dec ax
9155 0000264E B201 mov dl, 1
9156
9157 _again:
9157 00002650 3DF60F cmp ax, 0FF6h ; 4096-10
9158 00002653 7206 jb short _@@
9159 00002655 D1E8 shr ax, 1
9160 00002657 D0E2 shl dl, 1
9161 00002659 EBF5 jmp short _again
9162
9163 ; -----
9164
9165 _@@:
9165 0000265B 80FA01 cmp dl, 1 ; is it a small disk ?
9166 0000265E 7405 jz short _@@ ; yes, 224 root entries is enuf
9167
9168 ; 18/12/2023 - Retro DOS v5.0
9169 00002660 C74549F000 mov word [di+49h], 240 ; [di+BDS.rdirentries]
9170 ;mov word [di+2Dh], 240 ; [di+BDS.rdirentries]
9171
9172 _@@@:
9173 00002665 885545 ; 18/12/2023 - Retro DOS v5.0
9174 mov [di+45h], dl ; [di+BDS.rsecperclus]
9175 ;mov [di+29h], dl ; [di+BDS.rsecperclus]
9176
9177 ; logic to get the sectors/fat area.
9178 ; fat entry is assumed to be 1.5 bytes!!!
9179
9180 ; 10/12/2022
9181 ; ds = cs
9182 ; 17/10/2022 (ds=cs)
9182 00002668 F726[E225] mul word [word3] ; * 3
9183 0000266C F736[E025] div word [word2] ; / 2
9184 00002670 31D2 xor dx, dx
9185 00002672 F736[E425] div word [word512] ; / 512

```

```

9186 ;
9187 ; 10/12/2022
9188 ;mul word [cs:word3] ; * 3
9189 ;div word [cs:word2] ; / 2
9190 ;xor dx, dx
9191 ;div word [cs:word512] ; / 512
9192 ;
9193 00002676 40 inc ax ; + 1
9194 no_round_up:
9195 ; 18/12/2023 - Retro DOS v5.0
9196 00002677 89454E mov [di+4Eh], ax ; [di+BDS.rfatsecs]
9197 ;mov [di+32h], ax ; [di+BDS.rfatsecs]
9198 ;
9199 0000267A EB12 jmp short go_to_next_bds
9200 ; -----
9201
9202 not_process_other:
9203 0000267C D1E3 shl bx, 1 ; bx is word index into table of bpbs
9204 ;
9205 ;mov si, bpbtable
9206 ;mov si, [bpbtable+bx] ; 15/10/2022
9207 ; 09/12/2022
9208 ;mov si, BPBTABLE
9209 ;mov si, [bx+si] ; get address of bpb
9210 ; 10/12/2022
9211 ;mov si, [BPBTABLE+bx]
9212 ; 13/12/2022
9213 ;mov si, [SYSINITOFFSET+bpbtable+bx] ; wrong ! 14/08/2023
9214 ;
9215 ; 14/08/2023
9216 SYSINIT_OFFSET equ (SYSINITSEG-DOSBIODATASEG<<4)
9217 ; correct offset
9218 0000267E 8BB7[BC96] mov si, [bx+SYSINIT_OFFSET+bpbtable]
9219 ;
9220 ; 18/12/2023
9221 ; si = address of the requested disk(ette) parameter block
9222 ; ! as offset from SYSINIT segment !
9223 ;
9224 ; 28/08/2023
9225 00002682 81C69046 add si, SYSINIT_OFFSET
9226 ; + displacement from BIOSDATA segment ; 18/12/2023
9227
9228 set_recbpb:
9229 ; 18/12/2023
9230 ;lea di, [di+27h] ; [di+BDS.R_BPB]
9231 ; ; es:di -> recbpb
9232 ;mov cx, 25 ; bpbx.size
9233 ;rep movsb ; move (size bpbx) bytes
9234 ;
9235 ; 18/12/2023 - Retro DOS v5.0
9236 ;lea di, [di+43h] ; [di+BDS.R_BPB]
9237 ; ; es:di -> recbpb
9238 00002689 B93500 mov cx, 53 ; bpbx.size
9239 0000268C F3A4 rep movsb ; move (size bpbx) byte
9240 go_to_next_bds:
9241 pop di
9242 pop es ; restore pointer to bds
9243 les di, [es:di] ; [es:di+BDS.link]
9244 cmp di, 0FFFFh ; -1
9245 00002696 740A jz short got_end_of_bds_chain
9246 00002698 E951FF jmp _next_bds
9247 ; -----
9248
9249 ; 18/12/2022
9250 ;got_end_of_bds_chain:
9251 ;retn
9252
9253 ; ===== S U B R O U T I N E =====
9254
9255 ; 15/10/2022
9256 ; 30/12/2018 - Retro DOS v4.0
9257
9258 ; al = device number
9259
9260 print_init:
9261 0000269B 98 cbw
9262 0000269C 89C2 mov dx, ax
9263 0000269E B401 mov ah, 1
9264 000026A0 CD17 int 17h ; PRINTER - INITIALIZE
9265 ; DX = printer port (0-3)
9266 ; Return: AH = status
9267
9268 got_end_of_bds_chain: ; 18/12/2022
9269 retn
9270
9271 ; ===== S U B R O U T I N E =====
9272
9273 ; al = device number
9274
9275 aux_init:
9276 000026A3 98 cbw
9277 000026A4 89C2 mov dx, ax
9278 ;mov al, 0A3h ; RSINIT ; 0A3h
9279 ; ; 2400,n,1,8 (msequ.inc)
9280 ; 10/12/2022
9281 000026A6 B8A300 mov ax, 00A3h
9282 000026A9 CD14 int 14h ; SERIAL I/O - INITIALIZE USART
9283 ; ; AL = initializing parameters,
9284 ; ; DX = port number (0-3)
9285 ; ; Return: AH = RS-232 status code bits,
9286 ; ; AL = modem status bits
9287 000026AB C3 retn
9288
9289 ; ===== S U B R O U T I N E =====
9290
9291 ; 18/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
9292 ; 08/08/2023 - Retro DOS v4.2 (Modified MSDOS 6.22 IO.SYS)
9293 ; 15/10/2022 (Modified MSDOS 5.0 IO.SYS) -Retro DOS v4 2022- (MSDOS 5.0-6.21)
9294 ; 30/12/2018 - Retro DOS v4.0
9295 ; 03/06/2018 - Retro DOS v3.0
9296 ; (19/03/2018 - Retro DOS v2.0)
9297
9298 ; domini *****
9299 ;
9300 ;mini disk initialization routine. called right after dohard
9301 ;modified for >2 hardfile support
9302 ;
9303 ; **cs=ds=es=datagrp
9304 ;
9305 ; **domini will search for every extended partition in the system, and
9306 ; initialize it.
9307 ;
9308 ; **bdsbm stands for bds table for mini disk and located right after the label
9309 ; end96tpi. end_of_bdsbm will have the offset value of the ending

```

```

9310 ; address of bdsm table.
9311 ;
9312 ; **bdsm is the same as usual bds structure except that tim_lo, tim_hi entries
9313 ; are overlapped and used to identify mini disk and the number of hidden_trks.
9314 ; right now, they are called as ismini, hidden_trks respectively.
9315 ;
9316 ; **domini will use the same routine in sethard routine after label set2 to
9317 ; save coding.
9318 ;
9319 ; **drvmax determined in dohard routine will be used for the next
9320 ; available logical mini disk drive number.
9321 ;
9322 ; input: drvmax, dskdrvs
9323 ;
9324 ; output: minidisk installed. bdsm table established and installed to bds.
9325 ; end_of_bdsm - ending offset address of bdsm.
9326 ;
9327 ; called modules:
9328 ; getboot
9329 ; find_mini_partition (new), xinstall_bds (new), M038
9330 ;
9331 ; setmini (new, it will use set2 routine)
9332 ;
9333 ; variables used: end_of_bdsm
9334 ; rom_minidisk_num
9335 ; mini_hdlim, mini_seclim
9336 ; BDS_STRUC, start_bds
9337 ;
9338 ;*****
9339 ; 18/12/2023 - Retro DOS v5.0 IO.SYS/IBMBIO.COM
9340 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2B10h)
9341 ;
9342 ; 19/10/2022
9343 ;
9344 000026AC 8A36[5F1A] domini: mov dh, [hnum] ; get number of hardfiles
9345 ; 10/12/2022
9346 000026B0 20F6 and dh, dh
9347 ; cmp dh, 0
9348 000026B2 743C jz short dominiret ; no hard file? then exit.
9349 000026B4 B280 mov dl, 80h ; startwith hardfile 80h
9350 ;
9351 ; 18/12/2023 - Retro DOS v5.0
9352 000026B6 31C0 domini_loop: xor ax, ax ; 0
9353 ; ds = cs
9354 ; mov [cs:ep_start_sector], ax
9355 ; mov [cs:ep_start_sector+2], ax
9356 ; mov [cs:ep_hidden_secs], ax
9357 ; mov [cs:ep_hidden_secs+2], ax
9358 000026B8 A3[0222] mov [ep_start_sector], ax
9359 000026BB A3[0422] mov [ep_start_sector+2], ax
9360 000026BE A3[0622] mov [ep_hidden_secs], ax
9361 000026C1 A3[0822] mov [ep_hidden_secs+2], ax
9362 ;
9363 000026C4 52 push dx
9364 000026C5 8816[5E1A] mov [rom_minidisk_num], dl
9365 000026C9 B408 mov ah, 8
9366 000026CB CD13 int 13h ; DISK - DISK - GET CURRENT DRIVE PARAMETERS (XT,AT,XT286,CONV,PS)
9367 ; DL = drive number
9368 ; Return: CF set on error, AH = status code, BL = drivetype
9369 ; DL = number of consecutive drives
9370 ; DH = maximum value for head number, ES:DI -> drive parameter
9371 ;
9372 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
9373 ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:2B36h
9374 ; inc dh
9375 ; xor ax, ax
9376 ; mov al, dh
9377 000026CD 31C0 xor ax, ax
9378 000026CF 88F0 mov al, dh ; <= 255
9379 000026D1 40 inc ax ; (0FFh -> 100h)
9380 000026D2 A3[641A] mov [mini_hdlim], ax ; # of heads
9381 ; and cl, 3Fh
9382 ; mov al, cl
9383 ; 08/08/2023
9384 000026D5 88C8 mov al, cl
9385 000026D7 83E03F and ax, 3Fh
9386 000026DA A3[661A] mov [mini_seclim], ax ; # of sectors/track
9387 ;
9388 ; 18/12/2023
9389 ; push es ; * ; not necessary
9390 000026DD 8A16[5E1A] mov dl, [rom_minidisk_num]
9391 000026E1 E866FA call getboot ; read master boot record into
9392 ; initbootsegment:bootbias
9393 000026E4 7203 jc short domininext
9394 000026E6 E80800 call find_mini_partition
9395 ;
9396 ; domininext:
9397 000026E9 5A pop es ; *
9398 000026EA FEC2 pop dx
9399 000026EC FECE inc dl ; next hard file
9400 000026EE 75C6 dec dh
9401 ; jnz short domini_loop
9402 000026F0 C3 dominiret: retn
9403 ;
9404 ; ===== S U B R O U T I N E =====
9405 ;
9406 ; 15/10/2022 (Modified MSDOS 5.0 IO.SYS)
9407 ; 30/12/2018 - Retro DOS v4.0
9408 ;
9409 ; find_mini_partition tries to find every extended partition on a disk.
9410 ; at entry: di -> bdsm entry
9411 ; es:bx -> 07c0:bootbias - master boot record
9412 ; rom_minidisk_num - rom drive number
9413 ; drvmax - logical drive number
9414 ; mini_hdlim, mini_seclim
9415 ;
9416 ; called routine: setmini which uses set2 (in sethard routine)
9417 ; variables & equates used from original bios - flags, fnon_removable, fbigfat
9418 ;
9419 ; 19/12/2023 - Retro DOS v5.0
9420 ; (Modified PCDOS 7.1 IBMBIO.COM)
9421 ; (PCDOS 7.1 IBMBIO.COM - BIOSADATA:2BFCh)
9422 ;
9423 ; find_mini_partition:
9424 000026F1 81C3C201 add bx, 1C2h ; bx -> file system id
9425 ;
9426 ; 19/12/2023
9427 ; PCDOS 7.1 IBMBIO.COM
9428 ; mov word [ld_p_number], 26
9429 ;
9430 ; fmpnext:
9431 ; add word [ld_p_number], 16
9432 ; cmp word [ld_p_number], 4122
9433 ; ; 64 logical disk partitions (64 EBRs)

```



```

9434 ; ; (64*4 = 256 pte's, 256*16 = 4096, + 26 = 4122)
9435 ;jg short fmpnextfound
9436
9437 000026F5 26803F05 cmp byte [es:bx], 5 ; 05h = extended partition id.
9438 000026F9 7410 je short fmpgot ; Extended DOS CHS
9439
9440 ; 19/12/2023 - Retro DOS v5.0
9441 000026FB 26803F0F cmp byte [es:bx], 0Fh ; Extended DOS LBA
9442 000026FF 740A je short fmpgot
9443
9444 00002701 83C310 add bx, 16
9445 00002704 81FB0204 cmp bx, 402h ; 202h+bootbias
9446 00002708 75EB jnz short fmpnext
9447 ;jmp short fmpnextfound ; extended partition not found
9448 ; 18/12/2022
9449 fmpnextfound:
9450 0000270A C3 retn
9451
9452 ; ; 30/07/2019 - Retro DOS v3.2
9453 ; ;
9454 ;fmpret: jb short fmpnext
9455 ; ;
9456 ; ;
9457 ; -----
9458
9459 ; 19/10/2022
9460 fmpgot: ; found my partition.
9461 0000270B E82B01 call dmax_check ; check for drvmax already 26
9462 0000270E 73FA jnb short fmpnextfound ; done if too many
9463
9464 00002710 8B3E[621A] mov di, [end_of_bdss] ; get next free bds
9465
9466 ; 19/12/2023
9467 ;mov word [di+47h], 1 ; [di+BDS.bdsm_ismini]
9468 ; 10/12/2022
9469 ;or byte [di+23h], 1
9470 ;or word [di+23h], 1 ; [di+BDS.flags]
9471 ; ; fNon_Removable
9472 ;mov byte [di+22h], 5 ; [di+BDS.formfactor]
9473 ; ; ffHardFile
9474 ; 19/12/2023 - Retro DOS v5.0
9475 00002714 C745790100 mov word [di+79h], 1 ; [di+BDS.bdsm_ismini]
9476 00002719 804D3F01 or byte [di+3Fh], 1 ; [di+BDS.flags], fNon_Removable
9477 0000271D C6453E05 mov byte [di+3Eh], 5 ; [di+BDS.formfactor], ffHardFile
9478
9479 00002721 C606[081A]00 mov byte [fbigfat], 0 ; assume 12 bit fat.
9480 00002726 A1[641A] mov ax, [mini_hdlim]
9481 00002729 894515 mov [di+15h], ax ; [di+BDS.heads]
9482 0000272C A1[661A] mov ax, [mini_seclim]
9483 0000272F 894513 mov [di+13h], ax ; [di+BDS.secpertrack]
9484 00002732 A0[5E1A] mov al, [rom_minidisk_num]
9485 00002735 884504 mov [di+4], al ; [di+BDS.drivenum]
9486 ; ; set physical number
9487 00002738 A0[7500] mov al, [drvmax]
9488 0000273B 884505 mov [di+5], al ; [di+BDS.drivelet]
9489 ; ; set logical number
9490 0000273E 26837F0A00 cmp word [es:bx+10], 0
9491 ;ja short fmpgot_cont
9492 00002743 7707 ja short fmpgot1 ; 19/12/2023
9493 00002745 26837F0840 cmp word [es:bx+8], 64 ; with current bpb,
9494 ; ; only lower word is meaningful.
9495 0000274A 72BE jb short fmpnextfound
9496 ; ; should be bigger than 64 sectors at least
9497 fmpgot1: ; 19/12/2023
9498 ;fmpgot_cont:
9499 0000274C 83EB04 sub bx, 4 ; let bx point to the start of the entry
9500 0000274F 268A7702 mov dh, [es:bx+2] ; cylinder
9501 00002753 80E6C0 and dh, 0C0h ; get higher bits of cyl
9502 00002756 D0C6 rol dh, 1
9503 00002758 D0C6 rol dh, 1
9504 0000275A 268A5703 mov dl, [es:bx+3] ; cyl byte
9505 ; 19/12/2023 - Retro DOS v5.0
9506 0000275E 89557B mov [di+7Bh], dx ; [di+BDS.bdsm_hidden_trks]
9507 ;mov [di+49h], dx ; [di+BDS.bdsm_hidden_trks]
9508 ; ; set hidden trks
9509 ; 19/12/2023
9510 ;push bx ; * ; PCDOS 7.1
9511 ;;;
9512 00002761 268B4F08 mov cx, [es:bx+8] ; partition size, lw
9513 00002765 268B470A mov ax, [es:bx+10] ; partition size, hw
9514 00002769 030E[0222] add cx, [ep_start_sector]
9515 0000276D 1306[0422] adc ax, [ep_start_sector+2]
9516 00002771 31D2 xor dx, dx ; 19/12/2023
9517 00002773 3916[0222] cmp [ep_start_sector], dx ; 0
9518 ;cmp word [ep_start_sector], 0
9519 00002777 750D jnz short fmpgot2
9520 00002779 3916[0422] cmp [ep_start_sector+2], dx ; 0
9521 ;cmp word [ep_start_sector+2], 0
9522 0000277D 7507 jnz short fmpgot2
9523 0000277F 890E[0222] mov [ep_start_sector], cx
9524 00002783 A3[0422] mov [ep_start_sector+2], ax
9525 fmpgot2:
9526 00002786 890E[0622] mov [ep_hidden_secs], cx
9527 0000278A A3[0822] mov [ep_hidden_secs+2], ax
9528
9529 ; convert start sector address to CHS
9530
9531 ; 19/12/2023
9532 ; dx = 0
9533 ;push bx ; * ; not necessary
9534
9535 ;mov bx, [di+13h] ; [di+BDS.secpertrack]
9536 0000278D 8B7513 mov si, [di+13h] ; [di+BDS.secpertrack]
9537 ;xor dx, dx ; dx = 0
9538 ;div bx
9539 00002790 F7F6 div si
9540 00002792 91 xchg ax, cx
9541 ;div bx
9542 00002793 F7F6 div si
9543 ;mov bx, [di+15h] ; [di+BDS.heads]
9544 ; 07/05/2024
9545 ; 17/04/2024 (BugFix)
9546 00002795 8B7515 mov si, [di+15h] ; [di+BDS.heads]
9547 00002798 91 xchg ax, cx
9548 00002799 31D2 xor dx, dx
9549 ;div bx
9550 0000279B F7F6 div si
9551 0000279D 91 xchg ax, cx
9552 ;div bx
9553 0000279E F7F6 div si
9554
9555 ;pop bx ; *
9556
9557 000027A0 09C9 or cx, cx

```

```

9558 000027A2 7505          jnz     short fmpgot_lba_rd
9559 000027A4 3D0004        cmp     ax, 1024          ; cylinder number < 1024, CHS read is proper
9560 000027A7 7235          jb      short fmpgot_chs_rd
9561                                fmpgot_lba_rd:
9562 000027A9 804D4004        or      byte [di+40h], 4 ; set fLBArw flag ; LBA read/write ok/ready
9563 000027AD 8A16[5E1A]    mov     dl, [rom_minidisk_num]
9564 000027B1 1E          push    ds
9565                                ; 19/12/2023
9566                                ;push    si ; ** ; not necessary
9567 000027B2 31C0        xor     ax, ax          ; push bp
9568                                ; mov bp, sp ; (*)
9569 000027B4 50          push    ax ; 0
9570 000027B5 50          push    ax ; 0
9571 000027B6 FF36[0822]    push    word [ep_hidden_secs+2]
9572 000027BA FF36[0622]    push    word [ep_hidden_secs]
9573 000027BE B80002        mov     ax, bootbias ; 200h
9574                                ;mov     ax, 200h ; bootbias (buffer offset)
9575 000027C1 06          push    es ; buffer segment
9576 000027C2 50          push    ax
9577 000027C3 B80100        mov     ax, 1
9578 000027C6 50          push    ax ; read count
9579 000027C7 B81000        mov     ax, 10h ; DAP size = 16
9580 000027CA 50          push    ax
9581 000027CB 8CD0        mov     ax, ss
9582 000027CD 8ED8        mov     ds, ax
9583 000027CF 89E6        mov     si, sp ; ds:si = Disk Address Packet
9584                                ;
9585 000027D1 B442        mov     ah, 42h ; LBA read
9586 000027D3 CD13        int     13h ; DISK - IBM/MS Extension
9587                                ; EXTENDED READ (DL - drive, DS:SI - disk address packet)
9588                                ; 19/12/2023
9589                                ;pushf ; PCDOS 7.1 IBMBIO.COM BUG! Erdogan Tan - 08/08/2023
9590                                ;add     sp, 16
9591                                ;popf ; BUG!
9592                                ; mov sp, bp ; (*)
9593                                ; pop bp
9594                                ; 19/12/2023
9595 000027D5 9F          lahf     ; load status flags into AH
9596 000027D6 83C410    add     sp, 16
9597 000027D9 9E          sahf     ; store AH into flags
9598                                ;
9599                                ;pop     si ; ** ; 19/12/2023
9600 000027DA 1F          pop     ds
9601 000027DB 7317        jnc     short fmpgot3
9602                                ; 19/12/2023
9603 000027DD C3          retn
9604                                ;jmp     short fmpgot3
9605                                ;;;
9606                                ;
9607                                ; 19/12/2023
9608                                fmpgot_chs_rd:
9609 000027DE 268B4F02    mov     cx, [es:bx+2] ; cylinder,cylinder/sector
9610 000027E2 268A7701    mov     dh, [es:bx+1] ; head
9611 000027E6 8A16[5E1A]    mov     dl, [rom_minidisk_num]
9612 000027EA B80002        mov     bx, 200h ; bootbias
9613 000027ED B80102        mov     ax, 201h
9614 000027F0 CD13        int     13h ; DISK - READ SECTORS INTO MEMORY
9615                                ; AL = number of sectors to read, CH = track, CL = sector
9616                                ; DH = head, DL = drive, ES:BX -> buffer to fill
9617                                ; Return: CF set on error, AH = status, AL = number of sectors read
9618                                ; 19/12/2023
9619                                ;jc      short fmpnextfound
9620 000027F2 72E9        jc      short fmpnotfound
9621                                ;
9622 000027F4 BBC203    mov     bx, 3C2h ; 1C2h+bootbias
9623                                ;
9624                                ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
9625                                ; PCDOS 7.1 IBMBIO.COM - BIOSDATA:2C7Ch
9626 000027F7 26817F3C55AA    cmp     word [es:bx+3Ch], 0AA55h ; 03C2h+03Ch = 3FEh
9627                                ;jne     short fmpnextfound ; not a valid boot sector !
9628                                ; 19/12/2023
9629 000027FD 75DE        jne     short fmpnotfound ; not a valid boot sector !
9630                                ;
9631                                ; 13/08/2023
9632                                ;push    es
9633 000027FF E80800    call    setmini ; install a mini disk.
9634                                ; bx value saved.
9635                                ;pop     es ; 13/08/2023
9636 00002802 7203        jc      short fmpnextchain
9637 00002804 E84700    call    xinstall_bds ; -- install the bdsm into table
9638                                fmpnextchain:
9639 00002807 E9EBFE        jmp     fmpnext ; let's find out
9640                                ; if we have any chained partition
9641                                ; -----
9642                                ;
9643                                ; 18/12/2022
9644                                ;fmpnextfound:
9645                                ;retn
9646                                ;
9647                                ; ===== S U B R O U T I N E =====
9648                                ;
9649                                ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
9650                                ; 28/12/2018 - Retro DOS v4.0 (MSDOS 6.21)
9651                                ;
9652                                ; 19/12/2022 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
9653                                ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2C92h)
9654                                ;
9655                                setmini: ; 'setmini' is called from 'find_mini_partition' procedure
9656                                ;
9657 0000280A 57          push    di
9658 0000280B 53          push    bx
9659                                ; 12/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
9660                                ; ds = cs = BIOSDATA segment
9661                                ;push    ds
9662 0000280C 06          push    es
9663                                setmini_1:
9664                                ;cmp     byte [es:bx], 1 ; FAT12 partition
9665                                ;je      short setmini_2
9666                                ;cmp     byte [es:bx], 4 ; FAT16 (CHS) partition
9667                                ;je      short setmini_2
9668                                ;cmp     byte [es:bx], 6 ; FAT16 BIG (CHS) partition
9669                                ;je      short setmini_2
9670                                ;
9671                                ; 19/12/2023 - Retro DOS v5.0
9672                                ;cmp     byte [es:bx], 0Bh ; FAT32 (CHS) partition
9673                                ;je      short setmini_2
9674                                ;cmp     byte [es:bx], 0Ch ; FAT32 (LBA) partition
9675                                ;je      short setmini_2
9676                                ;cmp     byte [es:bx], 0Eh ; FAT16 (LBA) partition
9677                                ;je      short setmini_2
9678                                ;
9679                                ; 19/12/2023
9680 0000280D 268A07    mov     al, [es:bx]
9681 00002810 3C01        cmp     al, 1 ; FAT12 partition

```



```

9682 00002812 7422      je      short setmini_2
9683 00002814 3C04      cmp     al, 4      ; FAT16 (CHS) partition
9684 00002816 741E      je      short setmini_2
9685 00002818 3C06      cmp     al, 6      ; FAT16 BIG (CHS) partition
9686 0000281A 741A      je      short setmini_2
9687 0000281C 3C0B      cmp     al, 0Bh     ; FAT32 (CHS) partition
9688 0000281E 7416      je      short setmini_2
9689 00002820 3C0C      cmp     al, 0Ch     ; FAT32 (LBA) partition
9690 00002822 7412      je      short setmini_2
9691 00002824 3C0E      cmp     al, 0Eh     ; FAT16 (LBA) partition
9692 00002826 740E      je      short setmini_2
9693
9694 00002828 83C310     add     bx, 16
9695 0000282B 81FB0204    cmp     bx, 402h     ; 202h+bootbias
9696                                ;jne     short setmini_1
9697 0000282F 72DC      jb      short setmini_1 ; 19/12/2023
9698 00002831 F9          stc
9699 00002832 07          pop     es
9700                                ; 12/08/2023
9701                                ;pop     ds
9702 00002833 5B          pop     bx
9703 00002834 5F          pop     di
9704 00002835 C3          retn
9705
9706 ; -----
9707 setmini_2:
9708 00002836 E9D1F9     jmp     set2      ; branch into middle of sethard
9709
9710 ; ===== S U B   R O U T I N E =====
9711
9712 ; 30/12/2022 - Retro DOS v4.2
9713 ; (SYSINITSEG is 473h for MSDOS 6.21 IO.SYS)
9714
9715 ; 15/10/2022
9716 ; 28/12/2018 - Retro DOS v4.0
9717
9718 ; dmax_check -- call this when we want to install a new drive.
9719 ; it checks for drvmax < 26 to see if there is
9720 ; a drive letter left.
9721
9722 ; drvmax < 26 : carry SET!
9723 ; drvmax >=26 : carry RESET!, error flag set for message later
9724 ; trash ax
9725
9726 ; 19/12/2023 - Retro DOS v5.0
9727 dmax_check:
9728 00002839 803E[7500]1A     cmp     byte [drvmax], 26 ; checks for drvmax < 26
9729 0000283E 720D     jb      short dmax_ok ; return with carry if okay
9730 00002840 06          push    es
9731                                ;mov     ax, 46Dh     ; SYSINIT_SEG (SYSINIT segment)
9732                                ;mov     ax, 544h     ; 19/12/2023 (PCDOS 7.1)
9733 00002841 B8D904     mov     ax, SYSINITSEG ; 17/10/2022
9734 00002844 8EC0     mov     es, ax
9735                                ; 18/10/2022
9736 00002846 26C606[8803]01  mov     byte [es:TOOMANYDRIVESFLAG], 1 ; 09/12/2022
9737                                ;mov     byte ptr es:3FFh, 1 ; [es:toomanydrivesflag]
9738                                ; set message flag
9739                                ; [SYSINIT+toomanydrivesflag]
9740 0000284C 07          pop     es
9741
9742                                ;push    es
9743                                ;mov     ax, SYSINIT_SEG
9744                                ;mov     es, ax
9745                                ;mov     byte [es:toomanydrivesflag], 1
9746                                ; set message flag
9747                                ;pop     es
9748
9749                                ;mov     byte [SYSINIT+toomanydrivesflag], 1
9750 dmax_ok:
9751 0000284D C3          retn
9752
9753 ; ===== S U B   R O U T I N E =====
9754
9755 ; 18/10/2022
9756 ; 15/10/2022
9757 ; 28/12/2018 - Retro DOS v4.0
9758
9759 ; link next bds (at ds:di) into the chain. assume that the
9760 ; chain is entirely within ds == datagrp. also update drvmax,
9761 ; dskdrv_table, and end_of_bdss.
9762
9763 ; 19/12/2023 - Retro DOS v5.0
9764 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2CE1h)
9765 xinstall_bds:
9766 0000284E 56          push    si
9767 0000284F 53          push    bx
9768 00002850 8B36[1901]    mov     si, [start_bds]      ; get first bds
9769
9770 xinstall_bds_1:
9771 00002854 833CFF     cmp     word [si], 0FFFFh ; is this the last one?
9772 00002857 7404     jz      short xinstall_bds_2 ; skip ahead if so
9773                                ;mov     si, [si+BDS.link]
9774                                ;mov     si, [si]      ; chain through list
9775                                jmp     short xinstall_bds_1
9776
9777 xinstall_bds_2:
9778                                ;mov     [si+BDS.link], di
9779                                mov     [si], di
9780                                ;mov     [si+BDS.link+2], ds
9781                                mov     [si+2], ds
9782                                ;mov     word [di+BDS.link], -1
9783                                mov     word [di], 0FFFFh ; make sure it is a null ptr.
9784                                ;mov     [di+BDS.link+2], ds
9785                                mov     [di+2], ds ; might as well plug segment
9786                                ; 20/03/2019 - Retro DOS v4.0
9787                                lea     bx, [di+BDS.BPB]
9788                                lea     bx, [di+6]
9789                                mov     si, [last_dskdrv_table]
9790                                mov     [si], bx
9791                                add     word [last_dskdrv_table], 2
9792                                inc     byte [drvmax]
9793                                ;add     word [end_of_bdss], 100 ; BDS.size = 100
9794                                ; 19/12/2023 - Retro DOS v5.0
9795                                add     word [end_of_bdss], 150 ; BDS.size = 150
9796                                pop     bx
9797                                pop     si
9798                                retn
9799
9800 ; ===== S U B   R O U T I N E =====
9801
9802 ; 17/10/2022
9803 ; 15/10/2022
9804 ; 28/12/2018 - Retro DOS v4.0
9805 ; 03/06/2018 - Retro DOS v3.0

```

```

9806 ; 19/12/2023 - Retro DOS v5.0
9807 cmos_clock_read:
9808     push    ax
9809     push    cx
9810     push    dx
9811     push    bp
9812     xor     bp, bp
9813 loop_clock:
9814     xor     cx, cx
9815     xor     dx, dx
9816     mov     ah, 2
9817     int     1Ah ; CLOCK - READ REAL TIME CLOCK (AT,XT286,CONV,PS)
9818 ; Return: CH = hours in BCD
9819 ; CL = minutes in BCD
9820 ; DH = seconds in BCD
9821 ; 19/12/2023
9822     cmp     cx, 0
9823     and     cx, cx
9824     jnz     short clock_present
9825     cmp     dx, 0
9826     or      dx, dx
9827     jnz     short clock_present
9828     cmp     bp, 1 ; read again after a slight delay, in case clock
9829     ;je     short no_readdate ; was at zero setting.
9830     and     bp, bp
9831     jnz     short no_readdate
9832     inc     bp ; only perform delay once.
9833     mov     cx, 4000h ; 16384
9834     ; 19/12/2023
9835     mov     ch, 40h ; cx = 4000h ; 16384
9836 delay:
9837     loop    delay
9838     jmp     short loop_clock
9839 ; -----
9840 clock_present:
9841     mov     byte [cs:havecmosclock], 1 ; set the flag for cmos clock
9842     ; 19/12/2023
9843     ; ds = cs
9844     mov     byte [havecmosclock], 1 ; set the flag for cmos clock
9845 000028A5 C606[8C04]01
9846
9847 000028AA E81000
9848     call    cmoscck ; reset cmos clock rate that may be
9849 ; possibly destroyed by cp dos and
9850 ; post routine did not restore that.
9851 000028AD 56
9852 000028AE E8BAEF
9853 000028B1 FA
9854 000028B2 8936[8904]
9855 000028B6 FB
9856 000028B7 5E
9857 no_readdate:
9858     pop     bp
9859     pop     dx
9860     pop     cx
9861     pop     ax
9862
9863 cmoscck9: ; 19/12/2023
9864 000028BC C3
9865     retn
9866 ; -----
9867 ; the following code is written by jack gulley in engineering group.
9868 ; cp dos (CP/DOS, OS/2) is changing cmos clock rate for its own purposes
9869 ; and if the use cold boot the system to use pc dos while running cp dos,
9870 ; the cmos clock rate are still slow which slow down disk operations
9871 ; of pc dos which uses cmos clock. pc dos is put this code in msinit
9872 ; to fix this problem at the request of cp dos.
9873 ;
9874 ; the program is modified to be run on msinit. equates are defined
9875 ; in cmosequ.inc. this program will be called by cmos_clock_read procedure.
9876 ;
9877 ; the following code cmoscck is used to insure that the cmos has not
9878 ; had its rate controls left in an invalid state on older at's.
9879 ;
9880 ; it checks for an at model byte "fc" with a submodel type of
9881 ; 00, 01, 02, 03 or 06 and resets the periodic interrupt rate
9882 ; bits in case post has not done it. this initialization routine
9883 ; is only needed once when dos loads. it should be run as soon
9884 ; as possible to prevent slow diskette access.
9885 ;
9886 ; this code exposes one to dos clearing cmos setup done by a
9887 ; resident program that hides and re-boots the system.
9888 ;
9889 cmoscck: ; check and reset rtc rate bits
9890 ; model byte and submodel byte were already determined in msinit.
9891 ; 16/06/2018 - Retro DOS v3.0
9892 ; 19/03/2018 (Model: 0FCh, Sub Model: 01h, REF: AMIBIOS Prog. Guide)
9893 ; 19/12/2023 - Retro DOS v5.0
9894 ; 19/12/2023
9895 ; ds = cs
9896 ; push ax ; not necessary ; 19/12/2023
9897 ;
9898 000028BD 803E[AF05]FC
9899     cmp     byte [model_byte], 0FCh
9900     cmp     byte [cs:model_byte], 0FCh
9901     jnz     short cmoscck9 ; Exit if not an AT model
9902     cmp     byte [secondary_model_byte], 6 ; 21/04/2024
9903     cmp     byte [cs:secondary_model_byte], 6
9904     ; Is it 06 for the industrial AT ?
9905     jz      short cmoscck4 ; Go reset CMOS periodic rate if 06
9906     cmp     byte [secondary_model_byte], 4
9907     cmp     byte [cs:secondary_model_byte], 4
9908     ; Is it 00, 01, 02, or 03 ?
9909     jnb     short cmoscck9 ; EXIT if problem fixed by POST
9910     ; Also, Secondary_model_byte = 0
9911     ; when AH=0C0h, int 15h failed.
9912     ; RESET THE CMOS PERIODIC RATE
9913     ; Model=FC submodel=00,01,02,03 or 06
9914 cmoscck4:
9915     mov     al, 8Ah ; cmos_reg_a|nmi
9916     ; NMI disabled on return
9917     mov     ah, 26h ; 00100110b
9918     ; Set divider & rate selection
9919     call    cmosc_write
9920     mov     al, 8Bh ; cmos_reg_b|nmi
9921     ; NMI disabled on return
9922     call    cmosc_read
9923     and     al, 7 ; 00000111b
9924     ; clear SET,PIE,AIE,UIE,SQWE
9925     mov     ah, al
9926 000028D2 B08A
9927 000028D4 B426
9928 000028D6 E80B00
9929 000028D9 B08B
9930 000028DB E82000
9931 000028DE 2407
9932 000028E0 88C4

```

```

9930 000028E2 B00B      mov     al, 0Bh      ; cmos_reg_b
9931                                     ; NMI enabled on return
9932                                     ; 19/12/2023
9933                                     ; call cmos_write
9934 ;cmosck9:
9935                                     ; pop ax ; 19/12/2023
9936                                     ; retn
9937
9938                                     ; 19/12/2023
9939                                     ; jmp short cmos_write
9940
9941 ; ===== S U B R O U T I N E =====
9942
9943 ;--- cmos_write -----
9944 ; write byte to cmos system clock configuration table :
9945 ; :
9946 ; input: (al)= cmos table address to be written to :
9947 ; bit 7 = 0 for nmi enabled and 1 for nmi disabled on exit :
9948 ; bits 6-0 = address of table location to write :
9949 ; (ah)= new value to be placed in the addressed table location :
9950 ; :
9951 ; output: value in (ah) placed in location (al) with nmi left disabled :
9952 ; if bit 7 of (al) is on. during the cmos update both nmi and :
9953 ; normal interrupts are disabled to protect cmos data integrity. :
9954 ; the cmos address register is pointed to a default value and :
9955 ; the interrupt flag restored to the entry state on return. :
9956 ; only the cmos location and the nmi state is changed. :
9957 ;-----
9958
9959 cmos_write:                                     ; write (ah) to location (al)
9960 000028E4 9C      pushf                                     ;
9961 000028E5 50      push     ax                               ; save work register values
9962 000028E6 FA      cli                                     ;
9963 000028E7 50      push     ax                               ; save user nmi state
9964 000028E8 0C80   or      al, 80h                           ; disable nmi for us
9965 000028EA E670   out      70h, al                          ; CMOS Memory/RTC Index Register:
9966                                     ; RTC Seconds
9967 000028EC 90      nop
9968 000028ED 88E0   mov     al, ah
9969 000028EF E671   out      71h, al                          ; CMOS Memory/RTC Data Register
9970 000028F1 58      pop      ax                             ; get user nmi
9971 000028F2 2480   and     al, 80h
9972 000028F4 0C0F   or      al, 0Fh
9973 000028F6 E670   out      70h, al                          ; CMOS Memory/RTC Index Register:
9974                                     ; RTC Seconds
9975 000028F8 90      nop
9976 000028F9 E471   in      al, 71h                          ; CMOS Memory/RTC Data Register
9977 000028FB 58      pop      ax                             ; restore work registers
9978
9979                                     ; 19/12/2023
9980 ;push cs                                     ; *place code segment in stack and
9981 ;call cmos_popf                             ; *handle popf for b- level 80286
9982 ;retn
9983 000028FC EB18   jmp     short cmos_rw_popf
9984
9985 ; ===== S U B R O U T I N E =====
9986
9987 ;--- CMOS_READ -----
9988 ; read byte from cmos system clock configuration table :
9989 ; :
9990 ; input: (al)= cmos table address to be read :
9991 ; bit 7 = 0 for nmi enabled and 1 for nmi disabled on exit :
9992 ; bits 6-0 = address of table location to read :
9993 ; :
9994 ; output: (al) value at location (al) moved into (al). if bit 7 of (al) was :
9995 ; on then nmi left disabled. during the cmos read both nmi and :
9996 ; normal interrupts are disabled to protect cmos data integrity. :
9997 ; the cmos address register is pointed to a default value and :
9998 ; the interrupt flag restored to the entry state on return. :
9999 ; only the (al) register and the nmi state is changed. :
10000 ;-----
10001
10002 cmos_read:                                     ; read location (al) into (al)
10003 000028FE 9C      pushf
10004 000028FF FA      cli
10005 00002900 53      push     bx
10006                                     ; push ax ; *
10007                                     ; 19/12/2023
10008 00002901 89C3   mov     bx, ax ; * ; input
10009 00002903 0C80   or      al, 80h
10010 00002905 E670   out      70h, al                          ; CMOS Memory/RTC Index Register:
10011                                     ; RTC Seconds
10012 00002907 90      nop
10013 00002908 E471   in      al, 71h                          ; CMOS Memory/RTC Data Register
10014
10015 ;mov bx, ax ; output
10016 ;pop ax ; * ; input
10017
10018 ; 19/12/2023
10019 ; al = output, bl = input
10020 0000290A 93      xchg     ax, bx ; *
10021                                     ; bl = output, al = input
10022
10023 0000290B 2480   and     al, 80h
10024 0000290D 0C0F   or      al, 0Fh
10025 0000290F E670   out      70h, al                          ; CMOS Memory/RTC Index Register:
10026                                     ; RTC Seconds
10027 00002911 90      nop
10028 00002912 E471   in      al, 71h                          ; CMOS Memory/RTC Data Register
10029 ;mov ax, bx ; * ; output
10030 ; 19/12/2023
10031 00002914 93      xchg     ax, bx
10032 00002915 5B      pop      bx
10033
10034 ; 19/12/2023
10035 cmos_rw_popf:
10036 00002916 0E      push     cs
10037 00002917 E80100  call     cmos_popf
10038 0000291A C3      retn                                     ; return with flags restored
10039
10040 ; -----
10041
10042 cmos_popf:
10043 0000291B CF      iret                                     ; popf for level b- parts
10044                                     ; return far and restore flags
10045
10046 ; 21/12/2022
10047 ; -----
10048 ; -----
10049 %if 0
10050
10051 ; -----
10052 ; MSINIT.ASM (MSDOS 6.0, 1991)
10053 ; -----

```

```

10054 ; The following routines provide support for reading in the file MSDOS.SYS.
10055 ; -----
10056 ;
10057 ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
10058 ;
10059 ; (For Retro DOS, 'IO.SYS' and 'MSDOS.SYS' are already loaded together
10060 ; at once -as single kernel file- by the Retro DOS boot sector code.
10061 ; So, following disk reads -MSDOS.SYS loading- is not needed!
10062 ; Only needing is to move MSDOS Kernel to it's final memory location.)
10063 ;
10064 ; ===== S U B   R O U T I N E =====
10065 ;
10066 ; GetClus, read in a cluster at a specified address
10067 ;
10068 ; bx = cluster to read
10069 ; cx = sectors per cluster
10070 ; es:di = load location
10071 ;
10072 ; 17/10/2022
10073 ;DISKRD equ diskrd - DOSBIOSEG_2C7h      ; (8E5h for MSDOS 5.0 IO.SYS)
10074 ; 09/12/2022
10075 DISKRD equ diskrd
10076 ;
10077 ; 29/12/2023
10078 ; 20/12/2023 - Retro DOS v5.0
10079 ; (PCDOS 7.1 IBMBIO.COM - BIOSDATA:2DC4h)
10080 ;
10081 ; si:bx = (32 bit) cluster to read
10082 ; cx = sectors per cluster
10083 ; es:di = load location
10084 ;
10085 ; 17/10/2022
10086 getclus:
10087 ; 12/12/2023
10088 ; ds = cs
10089 ;
10090 push    cx ; 1*
10091 push    di ; 2*
10092 ;mov     [cs:doscnt], cx
10093 mov     [doscnt], cx ; 12/12/2023
10094 ;
10095 ; 20/12/2023
10096 ;;mov     [cs:ClusterH], si ; high word of cluster number
10097 ;mov     [ClusterH], si ; high word of cluster number
10098 mov     bp, si
10099 ;
10100 mov     ax, bx
10101 ;
10102 ;dec     ax
10103 ;dec     ax
10104 ; 20/12/2023
10105 sub     ax, 2
10106 ;
10107 ;;sbb     [cs:ClusterH], 0
10108 ;sbb     [ClusterH], 0
10109 sbb     bp, 0
10110 ;
10111 ; 20/12/2023
10112 ;;xchg    ax, [cs:ClusterH]
10113 ;xchg    ax, [ClusterH]
10114 xchg    ax, bp
10115 ;
10116 mul     cx
10117 ;
10118 ;;xchg    ax, [cs:ClusterH]
10119 ;xchg    ax, [ClusterH]
10120 xchg    ax, bp ; (+)
10121 ;
10122 mul     cx                ;; convert to logical sector
10123                        ;; dx:ax = matching logical sector number
10124                        ;; starting from the data sector
10125 ;;add     ax, [cs:bios_l]
10126 ;;adc     dx, [cs:bios_h]      ; dx:ax= first logical sector to read
10127 ; 12/12/2023
10128 ;add     ax, [bios_l]
10129 ;adc     dx, [bios_h]      ; dx:ax= first logical sector to read
10130 ;
10131 ; 20/12/2023
10132 ;;add     dx, [cs:ClusterH]
10133 ;add     ax, [cs:First_Data_Sector]
10134 ;adc     dx, [cs:First_Data_Sector+2]
10135 add     dx, bp ; (+)
10136 ;add     dx, [ClusterH] ; convert to logical sector
10137                        ;; dx:ax= matching logical sector number
10138                        ;; starting from the data sector
10139 add     ax, [First_Data_Sector]
10140 adc     dx, [First_Data_Sector+2]
10141                        ; dx:ax = first logical sector to read
10142 unpack:
10143 ; 20/12/2023
10144 push    ds ; 3* ; ds = cs ; 12/12/2023
10145 push    dx ; 4* ; * ; 12/12/2023
10146 push    ax ; 5*
10147 ; 29/12/2023
10148 push    si ; 6*
10149 push    bx ; 7*
10150 ;
10151 ;;mov     si, [cs:fatloc]
10152 ;mov     si, [fatloc] ; 12/12/2023
10153 ;mov     ds, si
10154 ; 20/12/2023
10155 ;mov     ax, [fatloc]
10156 ;mov     ds, ax
10157 push    bx ; 8*
10158 push    word [fatloc] ; 9*
10159 ;
10160 ;test     byte [cs:fbigfat], 20h
10161 test     byte [fbigfat], 20h ; fbigbig FAT32 ?
10162 pop     ds ; 9* ; ds = [fatloc]
10163 jz       short not_32bit_cluster ; no
10164 unpack32:
10165 ;push     dx
10166 mov     dx, si
10167 ;mov     si, bx
10168 pop     si ; 8* ; si = bx
10169 add     si, si
10170 adc     dx, dx
10171 add     si, si
10172 adc     dx, dx
10173 ; dx:si = 4*(si:bx) ; clust num offset from FAT entry 0
10174 call    get_fat_sector
10175 mov     si, [bx+2] ; high word of the FAT32 cluster number
10176 mov     bx, [bx] ; low word of the FAT32 cluster number
10177 ;pop     dx

```

```

10178             jmp     short getc11
10179
10180 not_32bit_cluster:
10181             ;mov     si, bx             ; next cluster
10182             pop     si ; 8* ; si = bx
10183             test    byte [cs:fbigfat], 40h; fbig
10184             ; 16 bit fat?
10185             jnz     short unpack16 ; yes
10186
10187 unpack12:
10188             shr     si, 1             ; 12 bit fat. si = si/2
10189             add     si, bx             ; si = clus + clus/2
10190             ; (si = byte offset of the cluster in the FAT)
10191             ;push    dx ; 12/12/2023
10192             xor     dx, dx
10193             ; 12/12/2023
10194             ; ds = FAT buffer segment
10195             call    get_fat_sector
10196             ;pop     dx ; 12/12/2023
10197
10198             mov     ax, [bx]           ; save it into ax
10199             jnz     short even_odd ; if not a splitted fat, check even-odd.
10200             ; 25/06/2023
10201             ;mov     al, [bx]         ; splitted fat
10202
10203             ; 12/12/2023
10204             ;mov     [cs:temp_cluster], al
10205             push    ax ; ** ; al = low 8 bits of 12 bits cluster number
10206
10207             inc     si                 ; (next byte)
10208
10209             ;push    dx ; 12/12/2023
10210             xor     dx, dx
10211             call    get_fat_sector
10212             ;pop     dx ; 12/12/2023
10213
10214             ;mov     al, ds:0
10215             ; 12/12/2023
10216             ; ds = FAT buffer segment
10217             ;mov     al, [0] ; 19/10/2022
10218             ;mov     [cs:temp_cluster+1], al
10219             ;mov     ax, [cs:temp_cluster]
10220             ; 12/12/2023
10221             ;mov     al, [cs:temp_cluster]
10222             pop     ax ; ** ; al = low 8 bits of 12 bits cluster number
10223             mov     ah, [0] ; high 4 bits (bits 7 to 11) of 12 bits cluster num
10224
10225 even_odd:
10226             ; 29/12/2023
10227             pop     bx ; 7*           ; restore old fat entry value
10228             push    bx               ; save it right away.
10229             shr     bx, 1             ; was it even or odd?
10230             jnc     short havclus    ; it was even.
10231             shr     ax, 1             ; odd. massage fat value and keep
10232             ; the highest 12 bits.
10233             shr     ax, 1
10234             shr     ax, 1
10235
10236 havclus:
10237             mov     bx, ax             ; now bx = new fat entry.
10238             and     bx, 0FFFh         ; keep low 12 bits.
10239             jmp     short unpackx
10240
10241 ; -----
10242 unpack16:
10243             ;push    dx ; 12/12/2023
10244             xor     dx, dx ; 0
10245             shl     si, 1             ; extend to 32 bit offset
10246             ;adc     dx, 0
10247             ; 12/12/2023
10248             rcl     dx, 1
10249
10250             ; 12/12/2023
10251             ; ds = FAT buffer segment
10252             call    get_fat_sector
10253             ;pop     dx ; 12/12/2023
10254             mov     bx, [bx]
10255             ; bx = new fat entry.
10256
10257 unpackx:
10258             ; 20/12/2023
10259             xor     si, si             ; high word of cluster number = 0
10260             ; (FAT12 or FAT16)
10261
10262 getc11:
10263             ; 29/12/2023
10264             pop     ax ; 7* - cluster number lw
10265             ;pop     word [cs:ClusterH]
10266             pop     dx ; 6* - cluster number hw
10267
10268             ; 20/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
10269             ; (this is a fast kernel loading method by the MSDOS programmer)
10270             ; ((consecutive clusters --> consecutive sectors))
10271
10272             sub     ax, bx ; previous - current (or current - new)
10273             ;sbb     [cs:ClusterH], si
10274             sbb     dx, si
10275             ;cmp     [cs:ClusterH], -1 ; one apart? (current = previous+1)
10276             ;cmp     dx, -1
10277             ; 29/12/2023
10278             inc     dx ; -1 -> 0
10279             jnz     short not_consequential
10280             ;cmp     ax, -1 ; 0FFFFh ; is [ClusterH]:ax = -1 ?
10281             inc     ax ; -1 -> 0
10282
10283 not_consequential:
10284             pop     ax ; 5*           ; restore logical sector (low)
10285             pop     dx ; 4* ; * ; 12/12/2023
10286             pop     ds ; 3*
10287
10288             ; 12/12/2023
10289             ; (this is a fast kernel loading method by the MSDOS programmer)
10290             ; ((consecutive clusters --> consecutive sectors))
10291             ; ds = cs
10292             ;sub     si, bx
10293             ;cmp     si, -1           ; one apart? (consecutive?)
10294             ; (current = previous+1)
10295             jnz     short getc12 ; no, read [doscnt] sectors
10296
10297             ;add     [cs:doscnt], cx ; (cx = sectors per cluster)
10298             add     [doscnt], cx ; 12/12/2023 ; add to read count
10299             jmp     unpack
10300
10301 ; -----
10302 getc12:
10303             push    si ; 20/12/2023
10304             push    bx

```

```

10302 ; bx = low word of the new cluster number
10303 ; 20/12/2023 - Retro DOS v5.0 (32 bit cluster numbers)
10304 ; si = high word of the new cluster number
10305 push dx ; sector to read (high word)
10306 push ax ; sector to read (low word)
10307
10308 ; 12/12/2023
10309 ; ds = cs
10310 ;mov ax, [cs:drvfat] ; get drive and fat spec
10311 ;mov cx, [cs:doscnt]
10312 mov ax, [drvfat] ; get drive and fat spec
10313
10314 ;;;
10315 ; 20/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
10316 ;;;
10317 ; dma and segment (64K boundary) overrun precaution
10318 ; (sector count will be decreased if it is required)
10319 mov cx, di
10320 not cx ; cx = 65535 - cx
10321 shr cx, 1 ; cx = cx/2
10322 xor cl, cl
10323 xchg cl, ch ; cx = cx/256
10324
10325 ;cmp cx, [cs:doscnt]
10326 ; ; if sector read count > cx, decrease it to cx
10327 cmp cx, [doscnt]
10328 jbe short getc13
10329 ;;;
10330 ;mov cx, [cs:doscnt]
10331 mov cx, [doscnt]
10332
10333 getc13: pop dx ; sector to read for diskrd (low)
10334 ;pop word [cs:start_sec_h]
10335 ; 12/12/2023
10336 pop word [start_sec_h]
10337 ; sector to read for diskrd (high)
10338 ; 12/12/2023
10339 ; ds = cs
10340 ;push ds
10341 ;push cs
10342 ;pop ds
10343
10344 push cs ; simulate far call
10345
10346 ; 20/12/2023
10347 ; 17/10/2022
10348 mov bp, DISKRD ; offset diskrd
10349 ;mov bp, 0A2Bh ; 20/12/2023
10350 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0A2Bh ; 364h:0A2Bh)
10351 ;mov bp, 8E5h ; 17/10/2022
10352 ; 2C7h:8E5h = 70h:2E55h
10353
10354 call call_bios_code ; read the clusters
10355
10356 ;pop ds
10357 ; 12/12/2023
10358 ; ds = cs
10359 pop bx ; lw of the new cluster number
10360 pop si ; 20/12/2023 ; hw of the new cluster number
10361
10362 pop di ; 2* - (kernel) load location (es:di)
10363
10364 ;mov ax, [cs:doscnt] ; get number of sectors read
10365 ; 12/12/2023
10366 mov ax, [doscnt]
10367 xchg ah, al ; multiply by 256
10368 shl ax, 1 ; times 2 equal 512
10369 add di, ax ; update load location
10370
10371 pop cx ; 1* ; restore sectors/cluster
10372
10373 retn
10374
10375 ; ===== S U B R O U T I N E =====
10376
10377 ;function: find and read the corresponding fat sector into ds:0
10378 ;
10379 ;in). dx:si - offset value (starting from fat entry 0) of fat entry to find. M054
10380 ; ds - fatloc segment
10381 ; cs:drvfat - logical drive number, fat id
10382 ; cs:md_sectorsize
10383 ; cs:last_fat_secnum - last fat sector number read in.
10384 ;
10385 ;out). corresponding fat sector read in.
10386 ; bx = offset value from fatlog segment.
10387 ; other registers are saved.
10388 ; zero flag set if the fat entry is splitted, i.e., when 12 bit fat entry
10389 ; starts at the last byte of the fat sector. in this case, the caller
10390 ; should save this byte, and read the next fat byte sector to get the rest
10391 ; of the fat entry value. (this will only happen with the 12 bit fat.)
10392
10393 ; 17/10/2022
10394 get_fat_sector:
10395 ; 12/12/2023
10396 ; ds = fat buffer segment
10397
10398 ; 12/12/2023
10399 ;push ax ; (not necessary)
10400 push cx ; read count (sectors per cluster)
10401 push di ; IBMDOS.COM/MSDOS.SYS load offset
10402 push si ; FAT offset value (from fat entry 0)
10403 push es ; IBMDOS.COM/MSDOS.SYS load segment
10404 push ds ; FAT buffer segment
10405
10406 ; 12/12/2023
10407 push cs
10408 pop ds
10409
10410 mov ax, si
10411 ;mov cx, [cs:md_sectorsize] ; 512
10412 ; 12/12/2023
10413 ;mov cx, [md_sectorsize] ; 512
10414 ;div cx ; ax = sector number, dx = offset
10415 ; 12/12/2023
10416 ;nop
10417
10418 ; 12/12/2023
10419 div word [md_sectorsize] ; 512
10420
10421 ; ax = FAT sector (sequence/index) number
10422 ; dx = cluster number offset
10423
10424 ; Get rid of the assumption that
10425 ; there is only one reserved sector

```



```

10426 ; 12/12/2023 ; *
10427 ;push es ; *
10428 ;push ds ; *
10429 ;push di ; *
10430 push ax
10431 ;push cs ; *
10432 ;pop ds ; *
10433
10434 ;mov ax, [cs:drvfat] ; get drive # and FAT id
10435 ; 12/12/2023
10436 mov ax, [drvfat] ; get drive # and FAT id
10437 mov bp, SETDRIVE
10438 ;mov bp, 5AEh ; PCDOS 7.1 IBMBIO.COM
10439 ;mov bp, 4D7h ; setdrive
10440 ; at 2C7h:4D7h = 70h:2A47h
10441 push cs ; simulate far call
10442 call call_bios_code ; get bds for drive
10443 pop ax ; (sector number -without reserved and hidden sectors-)
10444 add ax, [es:di+9] ; [es:di+BDS.resectors]
10445 ; add #reserved_sectors
10446
10447 ; 12/12/2023
10448 ;pop di ; *
10449 ;pop ds ; *
10450 ;pop es ; *
10451
10452 ; 12/12/2023
10453 ; ds = cs
10454 cmp ax, [last_fat_sec_num]
10455 ;cmp ax, [cs:last_fat_sec_num]
10456 jz short gfs_split_chk ; don't need to read it again.
10457 mov [last_fat_sec_num], ax
10458 ;mov [cs:last_fat_sec_num], ax
10459 ; sector number
10460 ; (in the partition, without hidden sectors)
10461
10462 ; 13/12/2023
10463 pop es ; FAT buffer segment (DS on top of the stack)
10464 push es ; (put it on top of the stack again)
10465
10466 push dx ; cluster number offset
10467
10468 ; 12/12/2023
10469 xor cx, cx
10470 mov [start_sec_h], cx ; 0
10471 ;mov word [cs:start_sec_h], 0
10472 ; prepare to read the fat sector
10473 ; start_sec_h is always 0 for fat sector.
10474 mov dx, ax
10475 ; 12/12/2023
10476 inc cx ; cx = 1
10477 ;mov cx, 1 ; 1 sector read
10478 ;mov ax, [cs:drvfat]
10479 mov ax, [drvfat]
10480 ;push ds
10481 ;pop es
10482
10483 xor di, di ; 0 ; es:di -> fatloc segment:0
10484
10485 ; 12/12/2023
10486 ;push ds
10487 ;push cs
10488 ;pop ds
10489
10490 push cs ; simulate far call
10491 mov bp, DISKRD ; 8E5h
10492 ;mov bp, 8E5h ; offset diskrd
10493 ; 2C7h:8E5h = 70h:2E55h
10494 call call_bios_code
10495
10496 ; 12/12/2023
10497 ;pop ds
10498 ; ds = cs = biosdata segment
10499
10500 pop dx ; cluster number offset
10501
10502 gfs_split_chk:
10503 ; 13/12/2023
10504 ;mov cx, [cs:md_sectorsize] ; 512
10505 mov cx, [md_sectorsize]
10506 ;gfs_split_chk:
10507 dec cx ; 511
10508 cmp dx, cx ; if offset points to the
10509 ; last byte of this sector,
10510 ; then splitted entry.
10511 mov bx, dx ; set bx to dx
10512
10513 ; 12/12/2023
10514 ; bx = dx = cluster number offset in the FAT buffer
10515 pop ds ; FAT buffer segment
10516 pop es ; IBMDOS.COM/MSDOS.SYS load segment
10517 pop si ; FAT offset value (from fat entry 0)
10518 pop di ; IBMDOS.COM/MSDOS.SYS load offset
10519 pop cx ; read count (sectors per cluster)
10520 ;pop ax
10521 retn
10522 ; 15/10/2022
10523 ;Bios_Data_Init ends
10524
10525 %endif
10526 ; -----
10527 ; -----
10528
10529 ; 09/12/2022
10530 ;db 0
10531
10532 numbertodiv equ ($-BData_start)
10533 numbertomod equ (numbertodiv % 16)
10534
10535 %if (numbertomod>0) & (numbertomod<16) ; 17/09/2023
10536 0000291C 00<rep 4h> times (16-numbertomod) db 0
10537 %endif
10538
10539 ;align 16
10540
10541 ; 09/12/2022
10542 IOSYSCODESEGOFF equ $ - BData_start
10543 ; 29/09/2023
10544 ;IOSYSCODESEGOFF equ $-$$
10545 IOSYSCODESEGE equ (IOSYSCODESEGOFF>>4)+(700h>>4)
10546
10547 ; 28/09/2023
10548 S1SIZE equ $-$$
10549

```

```

10550 ;--- End of DOSBIOS data segment -----
10551 ; -----
10552 ;db 4 dup(0)
10553 ; 09/12/2022
10554 ; times 4 db 0 ; 19/10/2022
10555 ; -----
10556
10557 ;=====
10558 ; DOS BIOS (IO.SYS) CODE SEGMENT
10559 ;=====
10560 ; 09/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
10561 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
10562
10563 section .BIOSCODE vstart=0
10564
10565 ; 30/12/2022
10566 ; (BIOSCODE SEGMENT is 2CCh for MSDOS 6.21 IO.SYS) -- ((25C0h+700h)>>4) --
10567
10568 BCode_start: ; 09/12/2022
10569
10570 ; 02/10/2022
10571
10572 ;--- DOSBIOS code segment -----
10573 ; -----
10574 ; MSBI01.ASM (MSDOS 6.0, 1991)
10575 ; -----
10576
10577 DOSBIOSEG_2C7h: ;db 30h dup(0) ; SEGMENT 2C7h (2C70h-700h=2570h)
10578 times 48 db 0 ; 19/10/2022
10579 BiosDataWord: dw 70h
10580
10581 ; 15/10/2022
10582 ;BIOSDATAWORD equ BiosDataWord - DOSBIOSEG_2C7h
10583 ; 09/12/2022
10584 BIOSDATAWORD equ BiosDataWord
10585
10586 ; -----
10587
10588 ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
10589 ; 20/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
10590
10591 ;*****
10592 ;*
10593 ;* seg_reinit is called with ax = our new code segment value, *
10594 ;* trashes di, cx, es *
10595 ;* *
10596 ;* cas -- should be made disposable! *
10597 ;* *
10598 ;*****
10599
10600 ; 20/09/2023
10601 ; 10/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
10602 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0032h
10603
10604 _seg_reinit:
10605 mov es, [cs:BIOSDATAWORD]
10606 ; at 2C7h:30h or 70h:25A0h
10607 ;mov di, (offset cdev+2)
10608 mov di, cdev+2 ; 19/10/2022
10609 ;mov cx, 4 ; (end_BC_entries - cdev)/4
10610 ; 10/08/2023
10611 mov cx, 3 ; (PCDOS 7.1)
10612
10613 _seg_reinit_1:
10614 stosw ; modify Bios_Code entry points
10615 inc di
10616 inc di
10617 loop _seg_reinit_1
10618 ; 10/08/2023 (PCDOS 7.1)
10619 ; (direct jump to i2f_handler from BIOSDATA:bios_i2f)
10620 ; (instead of 'bcode_i2f: dw i2f_handler, IOSYSCODESEG')
10621 mov [es:bios_i2f_seg], ax ; actual BIOSCODE segment
10622 retf
10623
10624 ; -----
10625
10626 ; 15/10/2022
10627
10628 ;*****
10629 ;*
10630 ;* chardev_entry - main device driver dispatch routine *
10631 ;* called with a dummy parameter block on the stack *
10632 ;* dw dispatch_table, dw prn/aux numbers (optional) *
10633 ;* *
10634 ;* will eventually take care of doing the transitions in *
10635 ;* out of Bios_Code *
10636 ;* *
10637 ;*****
10638
10639 ; 20/09/2023
10640 chardev_entry: ; 0070h:25B3h = 02C7h:0043h
10641 push si
10642 push ax
10643 push cx
10644 push dx
10645 push di
10646 push bp
10647 push ds
10648 push es
10649 push bx
10650 mov bp, sp
10651 mov si, [bp+18] ; get return address (dispatch table)
10652 ;mov ds, word [cs:0030h]
10653 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:0030h]
10654 mov ds, [cs:BIOSDATAWORD] ; 17/10/2022
10655 ; 20/09/2023 (PCDOS 7.1 - IBMBIO.COM - BIOSCODE:005Ah)
10656 les si, [si]
10657 ;mov ax, [si+2] ; get the device number if present
10658 mov ax, es
10659 mov [auxnum], al
10660 mov [printdev], ah
10661 ;mov si, [si] ; point to the device dispatch table
10662 les bx, [ptrsav] ; get pointer to i/o packet
10663 mov al, [es:bx+1] ; [es:bx+unit] ; al = unit code
10664 mov ah, [es:bx+13] ; [es:bx+media] ; ah = media descrip
10665 mov cx, [es:bx+18] ; [es:bx+count] ; cx = count
10666 mov dx, [es:bx+20] ; [es:bx+start] ; dx = start sector
10667 ; 17/10/2022
10668 cmp si, DSKTBL
10669 ;cmp si, 579h ; (PCDOS 7.1 IBMBIO.COM)
10670 ;cmp si, 4A2h ; dsktbl
10671 ; at 2C7h:4A2h = 70h:2A12h
10672 jnz short no_sector32_mapping
10673

```



```

10674 ; Special case for 32-bit start sector number:
10675 ; if (si==dsktbl) /* if this is a disk device call */
10676 ; set high 16 bits of secnum to 0
10677 ; if (secnum == 0xffff) fetch 32 bit sector number
10678 ;
10679 ; pass high word of sector number in start_sec_h, low word in dx
10680 ;
10681 ; note: start_l and start_h are the offsets within the io_request packet
10682 ; which contain the low and hi words of the 32 bit start sector if
10683 ; it has been used.
10684 ;
10685 ; note: remember not to destroy the registers which have been set up before
10686 ;
10687 ; 20/09/2023
10688 ;mov ds:start_sec_h, 0 ; initialize to 0
10689 0000007F C706[9C04]0000 mov word [start_sec_h], 0
10690 00000085 83FAFF cmp dx, 0FFFFh
10691 00000088 750C jnz short no_sector32_mapping
10692 0000008A 268B571C mov dx, [es:bx+28] ; [es:bx+start_h]
10693 ; ; 32 bits dsk req
10694 ;mov ds:start_sec_h, dx ; start_sec_h = packet.start_h
10695 0000008E 8916[9C04] mov [start_sec_h], dx
10696 00000092 268B571A mov dx, [es:bx+26] ; [es:bx+start_l]
10697 ; dx = packet.start_l
10698 no_sector32_mapping:
10699 00000096 97 xchg ax, di
10700 00000097 268A4702 mov al, [es:bx+2] ; [es:bx+cmd]
10701 0000009B 2E3A04 cmp al, [cs:si]
10702 0000009E 732B jnb short command_error
10703 000000A0 98 cbw ; note that al <= 15 means ok
10704 000000A1 D1E0 shl ax, 1
10705 000000A3 01C6 add si, ax
10706 000000A5 97 xchg ax, di
10707 000000A6 26C47F0E les di, [es:bx+14] ; [es:bx+trans]
10708 000000AA FC cld
10709 ; 17/10/2022
10710 000000AB 2EFF5401 call near [cs:si+1]
10711 ;call word ptr cs:si+1
10712 000000AF 7202 jb short already_got_ah_status
10713 000000B1 B401 mov ah, 1
10714 already_got_ah_status:
10715 ;mov ds, [cs:0030h] ; 15/10/2022
10716 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:0030h]
10717 ; cas note: shouldn't be needed!
10718 000000B3 2E8E1E[3000] mov ds, [cs:BIOSDATAWORD] ; 17/10/2022
10719 ;lds bx, ds:ptrsav
10720 000000B8 C51E[1200] lds bx, [ptrsav]
10721 000000BC 894703 mov [bx+3], ax ; [bx+status]
10722 ; mark operation complete
10723 000000BF 5B pop bx
10724 000000C0 07 pop es
10725 000000C1 1F pop ds
10726 000000C2 5D pop bp
10727 000000C3 5F pop di
10728 000000C4 5A pop dx
10729 000000C5 59 pop cx
10730 000000C6 58 pop ax
10731 000000C7 5E pop si
10732 ;add sp, 2 ; get rid of fake return address
10733 ; 20/09/2023 (PCDOS 7.1 - IBMBIO.COM - BIOSCODE:00C8h)
10734 000000C8 44 inc sp
10735 000000C9 44 inc sp
10736 ;
10737 ; fall through into bc_retfn
10738 ; -----
10739 bc_retfn:
10740 000000CA CB retf
10741 ; -----
10742 ;
10743 command_error:
10744 000000CB E80700 call bc_cmderr
10745 000000CE EBE3 jmp short already_got_ah_status
10746 ; 15/10/2022
10747 ; 01/05/2019
10748 ; -----
10749 ; The following piece of hack is for supporting CP/M compatibility
10750 ; Basically at offset 5 we have a far call into 0:c0. But this does not call
10751 ; 0:c0 directly instead it call f01d:feF0, because it needs to support 'lhld 6'
10752 ; The following hack has to reside at ffff:d0 (= f01d:feF0) if BIOS is loaded
10753 ; high.
10754 ; -----
10755 ;
10756 ;db 7 dup(0)
10757 ;
10758 ; 20/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
10759 ; (PCDOS 7.1 - IBMBIO.COM - BIOSCODE:0D0h)
10760 ; 15/10/2022
10761 ;dw 0 ; pad to bring offset to 0D0h
10762 ;
10763 off_d0: times 5 db 0 ; 5 bytes from 0:c0 will be copied onto here
10764 000000D0 00<rep 5h> ; which is the CP/M call 5 entry point
10765 ;
10766 ; -----
10767 ;
10768 ; exit - all routines return through this path
10769 ;
10770 ;
10771 bc_cmderr:
10772 000000D5 B003 mov al, 3 ; 2C7h:D5h = 70h:2645h
10773 ; unknown command error
10774 ;
10775 ; ===== S U B R O U T I N E =====
10776 ;
10777 ; now zero the count field by subtracting its current value,
10778 ; which is still in cx, from itself.
10779 ;
10780 ; subtract the number of i/o's NOT YET COMPLETED from total
10781 ; in order to return the number actually complete
10782 ;
10783 bc_err_cnt:
10784 ;les bx, ds:ptrsav
10785 ; 19/10/2022
10786 000000D7 C41E[1200] les bx, [ptrsav]
10787 000000DB 26294F12 sub [es:bx+18], cx ; [es:bx+count]
10788 ; # of successful i/o's
10789 000000DF B481 mov ah, 81h ; mark error return
10790 000000E1 F9 stc ; indicate abnormal end
10791 000000E2 C3 retn
10792 ;
10793 ; 15/10/2022
10794 ;Bios_Code ends
10795
10796
10797

```

```

10798 ;-----
10799 ; MSCHAR.ASM - MSDOS 6.0 - 1991
10800 ;-----
10801 ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
10802 ; 10/01/2019 - Retro DOS v4.0
10803
10804 ; 30/04/2019
10805
10806 ;title      mschar - character and clock devices
10807
10808 ;MODE_CTRLBRK      equ      0FFh
10809
10810 ; BIOSCODE:00E4h (MSDOS 6.21, IO.SYS)
10811
10812 ;*****
10813 ;*
10814 ;* device driver dispatch tables
10815 ;*
10816 ;* each table starts with a byte which lists the number of
10817 ;* legal functions, followed by that number of words. Each
10818 ;* word represents an offset of a routine in Bios_Code which
10819 ;* handles the function. The functions are terminated with
10820 ;* a near return. If carry is reset, a 'done' code is returned
10821 ;* to the caller. If carry is set, the ah/al registers are
10822 ;* returned as abnormal completion status. Notice that ds
10823 ;* is assumed to point to the Bios_Data segment throughout.
10824 ;*
10825 ;*****
10826
10827 ; 20/09/2023
10828 ; (PCDOS 7.1 - IBMBIO.COM - BIOSCODE:00E3h)
10829 ; 13/12/2022
10830 000000E3 00      db 0
10831
10832 ; 13/12/2022
10833 000000E4 0B      con_table: db ((con_table_end - con_table)-1)/2 ; 11
10834 ; 2C7h:0E4h = 70h:2654h
10835 000000E5 [FA01]   dw bc_exvec ; 1FBh ; bc_exvec at 2C7h:1FBh = 70h:276Bh
10836 ; 00 init
10837 000000E7 [FA01]   dw bc_exvec ; 1FBh ; 01
10838 000000E9 [FA01]   dw bc_exvec ; 1FBh ; 02
10839 000000EB [D500]   dw bc_cmderr ; 0D5h ; bc_exvec at 2C7h:D5h = 70h:2645h
10840 ; 03
10841 000000ED [5C01]   dw con_read ; 15Ch ; con_read at 2C7h:15Ch = 70h:26CCh
10842 ; 04
10843 000000EF [9F01]   dw con_rdnd ; 19Fh ; con_rdnd at 2C7h:19Fh = 70h:270Fh
10844 ; 05
10845 000000F1 [FA01]   dw bc_exvec ; 1FBh ; 06
10846 000000F3 [0802]   dw con_flush ; 209h ; con_flush at 2C7h:209h = 70h:2779h
10847 ; 07
10848 000000F5 [FC01]   dw con_writ ; 1FDh ; con_writ at 2C7h:1FDh = 70h:276Dh
10849 ; 08
10850 000000F7 [FC01]   dw con_writ ; 1FDh ; 09
10851 000000F9 [FA01]   dw bc_exvec ; 1FBh ; 0A
10852
10853 000000FB 1A      con_table_end:
10854 prn_table: db ((prn_table_end - prn_table)-1)/2 ; 26
10855 ; 2C7h:0FBh = 70h:266Bh
10856 000000FC [FA01]   dw bc_exvec ; 1FBh ; bc_exvec
10857 000000FE [FA01]   dw bc_exvec ; 1FBh ; 01
10858 00000100 [FA01]   dw bc_exvec ; 1FBh ; 02
10859 00000102 [D500]   dw bc_cmderr ; 0D5h ; bc_cmderr
10860 00000104 [1902]   dw prn_input ; 21Ah ; prn_input
10861 00000106 [C701]   dw z_bus_exit ; 1C8h ; 04 indicate zero chars read
10862 ; z_bus_exit
10863 00000108 [FA01]   dw bc_exvec ; 1FBh ; 05 read non-destructive
10864 0000010A [FA01]   dw bc_exvec ; 1FBh ; 06
10865 0000010C [1E02]   dw prn_writ ; 21Fh ; prn_writ
10866 0000010E [1E02]   dw prn_writ ; 21Fh ; 09
10867 00000110 [4F02]   dw prn_stat ; 251h ; prn_stat
10868 00000112 [FA01]   dw bc_exvec ; 1FBh ; 0B
10869 00000114 [FA01]   dw bc_exvec ; 1FBh ; 0C
10870 00000116 [FA01]   dw bc_exvec ; 1FBh ; 0D
10871 00000118 [FA01]   dw bc_exvec ; 1FBh ; 0E
10872 0000011A [FA01]   dw bc_exvec ; 1FBh ; 0F
10873 0000011C [9402]   dw prn_tilbusy ; 28Bh ; prn_tilbusy
10874 0000011E [FA01]   dw bc_exvec ; 1FBh ; 11
10875 00000120 [FA01]   dw bc_exvec ; 1FBh ; 12
10876 00000122 [C202]   dw prn_genioctl ; 2BAh ; prn_genioctl
10877 00000124 [FA01]   dw bc_exvec ; 1FBh ; 14
10878 00000126 [FA01]   dw bc_exvec ; 1FBh ; 15
10879 00000128 [FA01]   dw bc_exvec ; 1FBh ; 16
10880 0000012A [FA01]   dw bc_exvec ; 1FBh ; 17
10881 0000012C [FA01]   dw bc_exvec ; 1FBh ; 18
10882 0000012E [F702]   dw prn_ioctl_query ; 2F0h ; prn_ioctl_query
10883
10884 00000130 0B      prn_table_end:
10885 aux_table: db ((aux_table_end - aux_table)-1)/2 ; 11
10886 ; 2C7h:130h = 70h:26A0h
10887 00000131 [FA01]   dw bc_exvec ; 1FBh ; 00 - init
10888 00000133 [FA01]   dw bc_exvec ; 1FBh ; 01
10889 00000135 [FA01]   dw bc_exvec ; 1FBh ; 02
10890 00000137 [D500]   dw bc_cmderr ; 0D5h ; 03
10891 00000139 [1203]   dw aux_read ; 30Dh ; aux_read ; 04 - read
10892 0000013B [3703]   dw aux_rdnd ; 335h ; aux_rdnd - 05 - read non-destructive
10893 0000013D [FA01]   dw bc_exvec ; 1FBh ; 06
10894 0000013F [7803]   dw aux_flush ; 36Ch ; aux_flush
10895 00000141 [7F03]   dw aux_writ ; 374h ; aux_writ
10896 00000143 [7F03]   dw aux_writ ; 374h ; 09
10897 00000145 [5703]   dw aux_wrst ; 355h ; aux_wrst
10898
10899 00000147 0A      aux_table_end:
10900 tim_table: db ((tim_table_end - tim_table)-1)/2 ; 10
10901 ; 2C7h:147h = 70h:26B7h
10902 00000148 [FA01]   dw bc_exvec ; 1FBh ; 00
10903 0000014A [FA01]   dw bc_exvec ; 1FBh ; 01
10904 0000014C [FA01]   dw bc_exvec ; 1FBh ; 02
10905 0000014E [D500]   dw bc_cmderr ; 0D5h ; 03
10906 00000150 [E404]   dw tim_read ; 435h ; tim_read
10907 00000152 [C701]   dw z_bus_exit ; 1C8h ; z_bus_exit
10908 00000154 [FA01]   dw bc_exvec ; 1FBh ; 06
10909 00000156 [FA01]   dw bc_exvec ; 1FBh ; 07
10910 00000158 [E503]   dw tim_writ ; 3DBh ; tim_writ
10911 0000015A [E503]   dw tim_writ ; 3DBh ; 09
10912
10913 ; -----
10914 ;*****
10915 ;*
10916 ;* con_read - read cx bytes from keyboard into buffer at es:di
10917 ;*
10918 ;*****
10919
10920 con_read: ; 2C7h:15Ch = 70h:26CCh
10921 ;jcxz short con_exit ; read cx bytes from keyboard into buffer

```

```

10922 0000015C E306
10923
10924 0000015E E80500
10925 00000161 AA
10926 00000162 E2FA
10927
10928 00000164 F8
10929 00000165 C3
10930
10931
10932
10933
10934
10935
10936
10937
10938
10939
10940
10941
10942
10943
10944
10945
10946
10947
10948
10949
10950
10951
10952
10953
10954
10955
10956
10957
10958
10959
10960
10961 00000166 8A26[7E04]
10962 0000016A 30C0
10963 0000016C 8606[0C00]
10964 00000170 08C0
10965 00000172 752A
10966 00000174 CD16
10967 00000176 09C0
10968 00000178 74EC
10969 0000017A 3D0072
10970 0000017D 7504
10971 0000017F B010
10972 00000181 EB1B
10973
10974
10975
10976
10977
10978
10979
10980
10981
10982
10983
10984
10985
10986
10987 00000183 803E[7E04]00
10988 00000188 740C
10989 0000018A 3CE0
10990 0000018C 7508
10991 0000018E 08E4
10992 00000190 740C
10993 00000192 30C0
10994 00000194 EB04
10995
10996
10997
10998 00000196 08C0
10999 00000198 7504
11000
11001 0000019A 8826[0C00]
11002
11003 0000019E C3
11004
11005
11006
11007
11008
11009
11010
11011
11012
11013
11014
11015
11016
11017
11018
11019
11020
11021 0000019F A0[0C00]
11022 000001A2 08C0
11023 000001A4 754C
11024 000001A6 8A26[7F04]
11025 000001AA CD16
11026 000001AC 751D
11027 000001AE 803E[7900]00
11028 000001B3 7412
11029 000001B5 C41E[1200]
11030
11031 000001B9 26F6470404
11032
11033 000001BE 7407
11034 000001C0 B80041
11035 000001C3 30DB
11036 000001C5 CD15
11037
11038
11039
11040
11041 000001C7 F9
11042 000001C8 B403
11043 000001CA C3
11044
11045

con_loop:    jcxz    con_exit    ; 19/10/2022
             call    chrin       ; get char in al
             stosb    ; store char at es:di
             loop    con_loop

con_exit:    cld
             retn

; ===== S U B   R O U T I N E =====
; *****
; *
; * chrin - input single char from keyboard into al
; *
; *
; * we are going to issue extended keyboard function, if
; * supported. the returning value of the extended keystroke
; * of the extended keyboard function uses 0E0h in al
; * instead of 00h as in the conventional keyboard function.
; * this creates a conflict when the user entered real
; * greek alpha character (= 0E0h) to distinguish the extended
; * keystroke and the greek alpha. this case will be handled
; * in the following manner:
; *
; *      ah = 16h
; *      int 16h
; *      if al == 0, then extended code (in ah)
; *      else if al == 0E0h, then
; *      if ah < 0, then extended code (in ah)
; *      else greek_alpha character.
; *
; * also, for compatibility reason, if an extended code is
; * detected, then we are going to change the value in al
; * from 0E0h to 00h.
; *
; *****

chrin:    ; 19/10/2022
          mov     ah, [keyrd_func] ; set by msinit. 0 or 10h
          xor     al, al
          xchg    al, [altah]      ; get character      & zero altah
          or      al, al
          jnz     short keyret
          int     16h              ; KEYBOARD -
          or      ax, ax
          jz      short chrin
          cmp     ax, 7200h        ; check for ctrl-prtsc
          jnz     short alt_ext_chk
          mov     al, 10h
          jmp     short keyret

; -----
; if operation was extended function (i.e. keyrd_func != 0) then
; if character read was 0E0h then
; if extended byte was zero (i.e. ah == 0) then
; goto keyret
; else
; set al to zero
; goto alt_save
; endif
; endif
; endif

alt_ext_chk:
          cmp     byte [keyrd_func], 0
          jz      short not_ext
          cmp     al, 0E0h
          jnz     short not_ext
          or      ah, ah
          jz      short keyret
          xor     al, al
          jmp     short alt_save

; -----

not_ext:
          or      al, al          ; special case?
          jnz     short keyret

alt_save:
          mov     [altah], ah    ; store special key

keyret:
          retn

; -----
; *****
; *
; * con_rndnd - keyboard non destructive read, no wait
; *
; *
; * pc-convertible-type machine: if bit 10 is set by the dos
; * in the status word of the request packet, and there is no
; * character in the input buffer, the driver issues a system
; * wait request to the rom. on return from the rom, it returns
; * a 'char-not-found' to the dos.
; *
; *****

con_rndnd:    ; 19/10/2022
             mov     al, [altah]
             or      al, al
             jnz     short rdexit
             mov     ah, [keysts_func]
             int     16h          ; KEYBOARD -
             jnz     short gotchr
             cmp     byte [fhavok09], 0
             jz      short z_bus_exit
             les     bx, [ptrsav]
             ; 12/12/2022
             test    byte [es:bx+4], 04h
             ; test word [es:bx+3], 400h ; [es:bx+status]
             jz      short z_bus_exit
             mov     ax, 4100h
             xor     bl, bl
             int     15h          ; SYSTEM - WAIT ON EXTERNAL EVENT (CONVERTIBLE)
             ; AL = condition type, BH = condition compare or mask value
             ; BL = timeout value times 55 milliseconds, 00h means no timeout
             ; DX = I/O port address if AL bit 4 set

z_bus_exit:
             stc
             mov     ah, 3        ; 2c7h:1c8h = 70h:2738h
             ; indicate busy status
             retn

; -----

```

```

11046
11047 000001C8 09C0
11048 000001CD 7508
11049 000001CF 8A26[7E04]
11050 000001D3 CD16
11051 000001D5 EBC8
11052
11053
11054
11055 000001D7 3D0072
11056 000001DA 7504
11057 000001DC B010
11058 000001DE EB12
11059
11060
11061
11062 000001E0 803E[7E04]00
11063 000001E5 740B
11064 000001E7 3CE0
11065 000001E9 7507
11066 000001EB 80FC00
11067 000001EE 7402
11068 000001F0 B000
11069
11070
11071 000001F2 C41E[1200]
11072 000001F6 2688470D
11073
11074
11075 000001FA F8
11076
11077 000001FB C3
11078
11079
11080
11081
11082
11083
11084
11085
11086
11087
11088
11089
11090
11091 000001FC E3FC
11092
11093
11094
11095 000001FE 268A05
11096 00000201 47
11097 00000202 CD29
11098
11099 00000204 E2F8
11100
11101 00000206 F8
11102 00000207 C3
11103
11104
11105
11106
11107
11108
11109
11110
11111
11112
11113 00000208 C606[0C00]00
11114
11115 0000020D B401
11116 0000020F CD16
11117
11118
11119
11120 00000211 74F3
11121 00000213 30E4
11122 00000215 CD16
11123
11124 00000217 EBF4
11125
11126
11127
11128
11129
11130
11131
11132
11133
11134
11135
11136
11137
11138
11139
11140
11141
11142
11143
11144
11145
11146
11147
11148
11149
11150
11151
11152
11153
11154
11155
11156
11157
11158
11159
11160
11161
11162 00000219 E8BBFE
11163
11164
11165
11166 0000021C F8
11167 0000021D C3
11168
11169

gotchr:
        or      ax, ax
        jnz     short notbrk ; check for null after break
        mov     ah, [keyrd_func] ; issue keyboard read function
        int     16h           ; KEYBOARD -
        jmp     short con_rdnd ; get areal status
; -----

notbrk:
        cmp     ax, 7200h     ; check for ctrl-prtsc
        jnz     short rd_ext_chk
        mov     al, 10h       ; ('P' & 1Fh) ; return control p
        jmp     short rdxexit
; -----

rd_ext_chk:
        cmp     byte [keyrd_func], 0 ; extended keyboard function?
        jz      short rdxexit
        cmp     al, 0E0h      ; extended key value or      greek alpha?
        jnz     short rdxexit
        cmp     ah, 0         ; scan code exist?
        jz      short rdxexit ; yes. greek alpha char.
        mov     al, 0         ; no. extended key stroke.
                                ; change it for      compatibility

rdxexit:
        les     bx, [ptrsav]
        mov     [es:bx+13], al ; [es:bx+media]
                                ; return keyboard character here

bc_exvec:
        cld
                                ; bc_exvec at 2C7h:1FBh      = 70h:276Bh
                                ; indicate normal termination
        ret

; -----
; *****
; *
; * con_write - console write routine
; *
; * entry: es:di -> buffer
; *          CX = count
; *
; *****

con_writ:
        jcxz    short bc_exvec
        jcxz    bc_exvec      ; 19/10/2022
        jcxz    cc_ret        ; 12/12/2022
        jcxz    cc_ret

con_lp:
        mov     al, [es:di]
        inc     di
        int     29h           ; DOS 2+ internal - FAST PUTCHAR
                                ; AL = character to display
        loop    con_lp

cc_ret:
        cld
        ret

; ===== S U B   R O U T I N E =====
; *****
; *
; * con_flush - flush out keyboard queue
; *
; *****

con_flush:
        mov     byte [altah], 0 ; clearout holding buffer
floop:
        mov     ah, 1
        int     16h           ; KEYBOARD - CHECK BUFFER, DO NOT CLEAR
                                ; Return: ZF clear if character in buffer
                                ; AH = scan code, AL = character
                                ; ZF set if no character in buffer
        jz      short cc_ret
        xor     ah, ah
        int     16h           ; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
                                ; Return: AH = scan code, AL = character
        jmp     short flloop
; -----

; 15/10/2022
; *****
; *
; * some equates for rom bios printer i/o
; *
; *****

; ibm rom status bits (i don't trust them, neither should you)
; warning!!! the ibm rom does not return just one bit. it returns a
; whole slew of bits, only one of which is correct.

;notbusystatus equ 10000000b ; not busy
;nopaperstatus equ 00100000b ; no more paper
;prnselected equ 00010000b ; printer selected
;ierrstatus equ 00001000b ; some kinda error
;timeoutstatus equ 00000001b ; time out.
;
;noprnter equ 00110000b ; no printer attached

; 18/03/2019 - Retro DOS v4.0
;error_I24_out_of_paper equ 9 ; MSDOS 6.0, ERR.INC, 1991
; -----
; *****
; *
; * prn_input - return with no error but zero chars read
; *
; * enter with cx = number of characters requested
; *
; *****

prn_input:
        call    bc_err_cnt ; 2C7h:21Ah = 70h:278Ah
                                ; reset count to zero
                                ; (sub reqpkt.count,cx)
        ; 12/12/2022

prn_done:
        cld
                                ; but return with carry      reset for no error
        ret
; -----

```

```

11170
11171
11172
11173
11174
11175
11176
11177
11178
11179
11180 0000021E E3FC
11181
11182 00000220 BB0200
11183
11184 00000223 E83600
11185 00000226 751D
11186 00000228 268A05
11187 0000022B 30E4
11188 0000022D E82E00
11189 00000230 7419
11190 00000232 80FCFF
11191 00000235 7509
11192 00000237 B00C
11193 00000239 C606[0C00]00
11194 0000023E EB08
11195
11196
11197
11198 00000240 F6C401
11199 00000243 7406
11200
11201 00000245 4B
11202 00000246 75DB
11203
11204 00000248 E98CFE
11205
11206
11207
11208 0000024B 47
11209 0000024C E2D2
11210
11211
11212
11213
11214
11215 0000024E C3
11216
11217
11218
11219
11220
11221
11222
11223
11224
11225 0000024F E80A00
11226 00000252 75F4
11227 00000254 F6C480
11228
11229
11230 00000257 75F5
11231 00000259 E96BFF
11232
11233
11234
11235
11236
11237
11238
11239
11240
11241
11242 0000025C B402
11243
11244
11245
11246
11247
11248
11249
11250
11251
11252
11253
11254
11255
11256
11257
11258
11259
11260
11261
11262
11263
11264 0000025E 1E
11265 0000025F FF36[2100]
11266 00000263 31D2
11267 00000265 8EDA
11268 00000267 5A
11269 00000268 9C
11270 00000269 FA
11271
11272 0000026A FF1E5C00
11273 0000026E 1F
11274
11275
11276
11277
11278
11279 0000026F 50
11280 00000270 80E430
11281 00000273 80FC30
11282 00000276 58
11283 00000277 7506
11284 00000279 80E4DF
11285 0000027C 80CC08
11286
11287
11288
11289
11290
11291
11292 0000027F F6C428
11293

;*****
;
;* prn_writ - write cx bytes from es:di to printer device *
;
;* auxnum has printer number *
;
;*****

prn_writ:
; jcxz short prn_done ; 2C7h:21Fh = 70h:278Fh
; jcxz prn_done ; no chars to output
; ; 19/10/2022
prn_loop:
mov bx, 2 ; retry count
prn_out:
call prnstat ; get status
jnz short TestPrnError
mov al, [es:di] ; get character to print
xor ah, ah
call prnop ; print to printer
jz short prn_con ; no error - continue
cmp ah, 0FFh ; MODE_CTRLBRK
jnz short _prnwf
mov al, 0Ch ; error_I24_gen_failure
mov byte [altah], 0
jmp short pmsgg
; -----
_prnwf:
test ah, 1 ; timeoutstatus
jz short prn_con
TestPrnError:
dec bx ; retryuntil count is exhausted.
jnz short prn_out
pmsgg:
jmp bc_err_cnt
; -----
prn_con:
inc di ; point to nextchar and continue
loop prn_loop
;prn_done:
; ; 12/12/2022
prn_done2:
;clc
; cf=0
;retn
; -----
;*****
;
;* prn_stat - device driver entry to return printer status *
;
;*****

prn_stat:
; 2C7h:251h = 70h:27C1h
call prnstat ; device in dx
jnz short pmsgg
test ah, 80h ; notbusystatus
; jnz short prn_done
; ; 12/12/2022
jnz short prn_done2 ; cf=0
jmp z_bus_exit
; -----
;*****
;
;* prnstat - utility function to call ROM BIOS to check *
;* printer status. Return meaningful error code *
;
;*****

prnstat:
mov ah, 2 ; set command for get status
; ; PRINTER - GET STATUS
; ; DX = printer port (0-3)
; ; Return: AH = status
; ===== S U B R O U T I N E =====
;*****
;
;* prnop - call ROM BIOS printer function in ah *
;* return zero true if no error *
;* return zero false if error, al = error code *
;
;*****

prnop:
; 20/12/2023 - Retro DOS v5.0
; PCDOS 7.1 IBMBIO.COM
;mov dx, [auxnum] ; get printer number
;int 17h
push ds
push word [auxnum]
xor dx, dx ; 0
mov ds, dx
pop dx
pushf ; simulate int 17h
cli
call dword ptr ds:5ghghCh
call far [17h*4] ; 0:5Ch = INT 17h vector
pop ds
; This check was added to see if this is a case of no
; printer being installed. This tests checks to be sure
; the error is noprinter (30h)
push ax
and ah, 30h
cmp ah, 30h ; noprinter
pop ax
jnz short NextTest
and ah, 0DFh ; ~nopaperstatus
or ah, 8 ; ioerrstatus
; examine the status bits to see if an error occurred. unfortunately, several
; of the bits are set so we have to pick and choose. we must be extremely
; careful about breaking basic.
NextTest:
test ah, 28h ; (ioerrstatus+nopaperstatus)
; i/o error?

```

```

11294 00000282 740A          jz      short checknotready ; no, try not ready
11295
11296          ; at this point, we know we have an error. the converse is not true
11297
11298 00000284 B009          mov     al, 9          ; error_I24_out_of_paper
11299          ; first, assume out of paper
11300 00000286 F6C420        test    ah, 20h        ; out of paper set?
11301 00000289 7502          jnz     short ret1      ; yes, error is set
11302 0000028B FEC0          inc     al          ; return al=10 (i/o error)
11303
11304 0000028D C3            retn
11305
11306          ; -----
11307
11308 0000028E B002          checknotready:
11309 00000290 F6C401        mov     al, 2          ; assume not-ready
11310 00000293 C3            test    ah, 1
11311          retn
11312
11313          ; -----
11314          ; *****
11315          ; *
11316          ; * prn_tilbusy - output until busy. Used by print spooler. *
11317          ; * this entry point should never block waiting for *
11318          ; * device to come ready. *
11319          ; *
11320          ; * inputs: cx = count, es:di -> buffer *
11321          ; * outputs: set the number of bytes transferred in the *
11322          ; * device driver request packet *
11323          ; *
11324          ; *****
11325
11326          ; 19/10/2022
11327          prn_tilbusy:          ; 2C7h:28Bh = 70h:27FBh
11328 00000294 89FE          mov     si, di          ; everything is set for lodsb
11329          prn_tilbloop:
11330          push    cx
11331 00000297 53            push    bx
11332 00000298 30FF          xor     bh, bh
11333 0000029A 8A1E[8004]    mov     bl, [printdev]
11334 0000029E D1E3          shl     bx, 1
11335          ;mov    cx, ds:wait_count[bx] ; wait count times to come ready
11336 000002A0 8B8F[8104]    mov     cx, [wait_count+bx]
11337 000002A4 5B            pop     bx
11338          prn_getstat:
11339          call    prnstat      ; get status
11340 000002A8 7514          jnz     short prn_bperr      ; error
11341 000002AA F6C480        test    ah, 80h        ; ready yet?
11342 000002AD E1F6          loope   prn_getstat      ; no, go for more
11343 000002AF 59            pop     cx          ; get original count
11344 000002B0 740D          jz      short prn_berr      ; still not ready => done
11345 000002B2 26            es
11346 000002B3 AC          lodsb
11347          ;lods    byte ptr es:[si] ; es
11348          ;lods    ; lodsb
11349          xor     ah, ah
11350 000002B6 E8A5FF        call    prnop
11351 000002B9 7504          jnz     short prn_berr      ; error
11352 000002BB E2D9          loop   prn_tilbloop
11353          ; 12/12/2022
11354          ; cf=0 (prnop)
11355          ;clc
11356 000002BD C3            retn          ; normal no-error return
11357          ; from device driver
11358
11359          ; -----
11360          prn_bperr:
11361 000002BE 59            pop     cx          ; restore transfer count from stack
11362          prn_berr:
11363 000002BF E915FE        jmp     bc_err_cnt
11364          ; -----
11365
11366          ; 15/10/2022
11367          ; *****
11368          ; *
11369          ; * prn_genioctl - get/set printer retry count *
11370          ; *
11371          ; *****
11372
11373          ; IOCTL.INC (MSDOS 6.0, 1991)
11374          ; 11/01/2019
11375
11376          ; *****;
11377          ; CHARACTER DEVICES (PRINTERS) ;
11378          ; *****;
11379          ; *****;
11380
11381          ;;RAWIO SUB-FUNCTIONS
11382          ;;get_retry_count equ 65h
11383          ;;set_retry_count equ 45h
11384
11385          ;;struc A_RETRYCOUNT
11386          ;;rc_count: resw 1
11387          ;;endstruc
11388
11389          ;ioc_pc equ 5
11390
11391          ; -----
11392
11393          ; 19/10/2022
11394          prn_genioctl:          ; 2C7h:2BAh = 70h:282Ah
11395          les     di, [ptrsav]
11396 000002C6 26807D0D05    cmp     byte [es:di+13], 5 ; [es:di+IOCTL_REQ.MAJORFUNCTION]
11397          ; ioc_pc
11398 000002CB 7403          jz      short prnfunc_ok
11399
11400          prnfuncerr:
11401 000002CD E905FE        jmp     bc_cmderr
11402          ; -----
11403
11404          prnfunc_ok:
11405          mov     al, [es:di+14] ; [es:di+IOCTL_REQ.MINORFUNCTION]
11406          les     di, [es:di+19] ; [es:di+IOCTL_REQ.GENERICIOCTL_PACKET]
11407          xor     bh, bh
11408          ;mov    bl, ds:printdev ; get index into retry counts
11409          mov     bl, [printdev]
11410          shl     bx, 1
11411          ;mov    cx, ds:wait_count[bx] ; pull out retry count for device
11412          mov     cx, [wait_count+bx]
11413          cmp     al, 65h        ; get_retry_count
11414          jz      short prngetcount
11415          cmp     al, 45h        ; set_retry_count
11416          jnz     short prnfuncerr
11417          mov     cx, [es:di]

```



```

11418
11419
11420 000002EF 898F[8104]
11421 000002F3 26890D
11422
11423
11424
11425
11426 000002F6 C3
11427
11428
11429
11430
11431
11432
11433
11434
11435
11436
11437 000002F7 C43E[1200]
11438 000002FB 26807D0D05
11439
11440 00000300 750D
11441 00000302 268A450E
11442 00000306 3C65
11443 00000308 7404
11444 0000030A 3C45
11445 0000030C 7501
11446
11447
11448
11449
11450 0000030E C3
11451
11452
11453
11454
11455
11456 0000030F E9C3FD
11457
11458
11459
11460
11461
11462
11463
11464
11465
11466
11467
11468
11469
11470
11471
11472
11473
11474
11475
11476
11477
11478
11479
11480
11481
11482
11483
11484
11485
11486
11487
11488
11489
11490
11491
11492
11493
11494
11495
11496
11497
11498
11499
11500
11501 00000312 E311
11502 00000314 E88000
11503 00000317 30C0
11504 00000319 8607
11505 0000031B 08C0
11506 0000031D 7503
11507
11508 0000031F E80500
11509
11510
11511 00000322 AA
11512 00000323 E2FA
11513
11514 00000325 F8
11515
11516 00000326 C3
11517
11518
11519
11520
11521
11522
11523
11524
11525
11526
11527
11528
11529 00000327 B402
11530 00000329 E83A00
11531 0000032C F6C40E
11532
11533
11534
11535 0000032F 74F5
11536
11537
11538
11539 00000331 58
11540
11541

prngetcount:
    mov     ds:wait_count[bx], cx
    mov     [wait_count+bx], cx
    mov     [es:di], cx      ; [es:di+A_RETRYCOUNT.RC_COUNT]
                                ; return current retry count
    ; 12/12/2022
    ; cf=0
    cld
    retn

; -----
; *****
; *
; * prn_ioctl_query
; *
; * Added for 5.00
; *****
prn_ioctl_query:
    ; 2C7h:2F0h = 70h:2860h
    les     di, [ptrsav]
    cmp     byte [es:di+13], 5 ; [es:di+IOCTL_REQ.MAJORFUNCTION]
    ; ioc_pc
    jnz     short prn_query_err
    mov     al, [es:di+14] ; [es:di+IOCTL_REQ.MINORFUNCTION]
    cmp     al, 65h      ; GET_RETRY_COUNT
    jz      short IOCTLSupported
    cmp     al, 45h      ; SET_RETRY_COUNT
    jnz     short prn_query_err
IOCTLSupported:
    ; 12/12/2022
    ; cf=0
    cld
    retn

; -----
prn_query_err:
    ; 12/12/2022
    stc
    jmp     bc_cmderr ; (bc_cmderr sets cf to 1)

; -----
; *****
; *
; * aux port driver code -- "aux" == "com1"
; *
; * the device driver entry/dispatch code sets up auxnum to
; * give the com port number to use (0=com1, 1=com2, 2=com3...)
; *****
; values in ah, requesting function of int 14h in rom bios

;auxfunc_send     equ 1      ;transmit
;auxfunc_receive  equ 2      ;read
;auxfunc_status   equ 3      ;request status

; error flags, reported by int 14h, reported in ah:

;flag_data_ready equ 01h    ;data ready
;flag_overnrun   equ 02h    ;overrun error
;flag_parity      equ 04h    ;parity error
;flag_frame      equ 08h    ;framing error
;flag_break      equ 10h    ;break detect
;flag_tranhol_empt equ 20h  ;transmit holding register empty
;flag_timeout    equ 80h    ;timeout

; these flags reported in al:

;flag_cts equ 10h    ;clear to send
;flag_dsr equ 20h    ;data set ready
;flag_rec_sig equ 80h ;receive line signal detect

; -----
; *****
; *
; * aux_read - read cx bytes from [auxnum] aux port to buffer
; * at es:di
; *
; *****
aux_read:
    ; 2C7h:30Dh = 70h:287Dh
    jcxz    short exvec2
    jcxz    exvec2      ; 19/10/2022
    call    getbx       ; put address of auxbuf in bx
    xor     al, al
    xchg    al, [bx]
    or      al, al
    jnz     short aux2

aux1:
    call    auxin        ; get character from port
                        ; won't return if error

aux2:
    stosb
    loop    aux1         ; if more characters, go around again

exvec2:
    cld
    ; all done, successful exit
auxin_retn: ; 18/12/2022
    retn

; -----
; *****
; *
; * auxin - call rom bios to read character from aux port
; * if error occurs, map the error and return one
; * level up to device driver exit code, setting
; * the number of bytes transferred appropriately
; *
; *****
auxin:
    mov     ah, 2        ; auxfunc_receive
    call    auxop
    test    ah, 0Eh      ; flag_frame|flag_parity|flag_overnrun
    jnz     short arbad   ; skip if any error bits set
    retn
    ; 25/06/2023 (BugFix)
    jz      short auxin_retn

; -----
arbad:
    pop     ax           ; remove return address (near call)
    xor     al, al
    or      al, 0B0h     ; flag_rec_sig|flag_dsr|flag_cts

```

```

11542 ; 11/08/2023
11543 00000332 B0B0 mov al, 0B0h ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0334h
11544 00000334 E9A0FD jmp bc_err_cnt
11545 ; -----
11546 ;
11547 ;
11548 ;
11549 ;
11550 ; * aux_rdnnd - non-destructive aux port read *
11551 ; * *
11552 ; * *
11553 ; * *
11554 ; * *
11555 aux_rdnnd: ; 2C7h:335h = 70h:28A5h
11556 call getbx ; have bx point to auxbuf
11557 mov al, [bx] ; if al is non-zero (char in buffer)
11558 or al, al ; then return character
11559 jnz short auxdrx ; if not, get status of aux device
11560 call auxstat ; flag_data_ready - test data ready
11561 test ah, 1 ; then device is busy (not ready)
11562 jz short auxbus ; flag_dsr - test data set ready
11563 test al, 20h ; then device is busy (not ready)
11564 jz short auxbus ; else aux is ready, get character
11565 call auxin
11566 mov [bx], al
11567 auxdrx: jmp rdxexit ; return busy status
11568 ; -----
11569 ;
11570 ;
11571 auxbus: jmp z_bus_exit
11572 ; -----
11573 ;
11574 ;
11575 ; *
11576 ; * aux_wrst - return aux port write status *
11577 ; * *
11578 ; * *
11579 ; * *
11580 ; * *
11581 aux_wrst: ; 2C7h:355h = 70h:28C5h
11582 call auxstat ; get status of aux in ax
11583 test al, 20h ; test data set ready
11584 jz short auxbus ; then device is busy (not ready)
11585 test ah, 20h ; flag_tranhol_emp - test transmit hold reg empty
11586 jz short auxbus ; then device is busy (not ready)
11587 ; 12/12/2022
11588 ; cf=0 ; (test instruction resets cf)
11589 ; cfc
11590 retn
11591 ; -----
11592 ;
11593 ;
11594 ; *
11595 ; * auxstat - call rom bios to determine aux port status *
11596 ; * *
11597 ; * exit: ax = status *
11598 ; * dx = [auxnum] *
11599 ; * *
11600 ; * *
11601 ; * *
11602 00000364 B403 auxstat: mov ah, 3 ; auxfunc_status
11603 ; fall into auxop
11604 ;
11605 ;
11606 ; ===== S U B R O U T I N E =====
11607 ; -----
11608 ;
11609 ;
11610 ; *
11611 ; * auxop - perform rom-biox aux port interrupt *
11612 ; * *
11613 ; * entry: ah = int 14h function number *
11614 ; * exit: ax = results *
11615 ; * dx = [auxnum] *
11616 ; * *
11617 ; * *
11618 ; * *
11619 ; * *
11620 ; * *
11621 ; * *
11622 ; * *
11623 ; * *
11624 ; * *
11625 ; * *
11626 ; * *
11627 ; * *
11628 00000366 1E ; (PCDOS 7.1 IBMBIO.COM)
11629 00000367 FF36[2100] push ds
11630 00000368 31D2 push word [auxnum]
11631 0000036D 8EDA xor dx, dx ; 0
11632 0000036F 5A mov ds, dx
11633 00000370 9C pop dx
11634 00000371 FA pushf ; simulate INT 14h
11635 cli
11636 00000372 FF1E5000 ; call dword ptr ds:50h
11637 00000376 1F call far [14h*4] ; INT 14h vector (14h*4 = 50h)
11638 00000377 C3 pop ds
11639 retn
11640 ; -----
11641 ;
11642 ;
11643 ; *
11644 ; * aux_f1sh - flush aux input buffer - set contents of *
11645 ; * auxbuf [auxnum] to zero *
11646 ; * *
11647 ; * *
11648 ; * cas - shouldn't this code call the rom bios input function *
11649 ; * repeatedly until it isn't ready? to flush out any *
11650 ; * pending serial input queue if there's a tsr like MODE *
11651 ; * which is providing interrupt-buffering of aux port? *
11652 ; * *
11653 ; * *
11654 ; * *
11655 00000378 E81C00 aux_f1sh: ; 2C7h:36Ch = 70h:28DCh
11656 0000037B C60700 call getbx ; flush aux input buffer
11657 mov byte [bx], 0 ; get bx to point to auxbuf
11658 ; cfc ; zero out buffer
11659 ; 12/12/2022 ; all done, successful return
11660 ; cf=0 ('add' instruction in 'getbx')
11661 0000037E C3 retn
11662 ; -----
11663 ;
11664 ;
11665 ; *

```



```

11666      ;* aux_writ - write to aux device                                     *
11667      ;*                                                                 *
11668      ;*****
11669
11670      aux_writ:      ; 2C7h:374h = 70h:28E4h
11671      ;jcxz short exvec2 ; write to aux device (if cx > 0)
11672      0000037F E3A4      jcxz exvec2 ; 19/10/2022
11673
11674      aux_loop:      mov     al, [es:di] ; get character to be written
11675      ;                                     ; move di pointer to next character
11676      inc     di
11677      mov     ah, 1 ; auxfunc_send - indicates a write
11678      call    auxop ; send character over aux port
11679      test    ah, 80h ; check for error
11680      jz      short awok ; then no error
11681      mov     al, 10 ; else indicate write fault
11682      00000391 E943FD      jmp     bc_err_cnt ; call error routines
11683      ; -----
11684
11685      awok:      loop     aux_loop ; if cx is non-zero,
11686      00000394 E2EB      ; still more character to print
11687      ;cld ; all done, successful return
11688      ; 12/12/2022
11689      ; cf=0 (test instruction above)
11690      00000396 C3      retn
11691
11692      ; ===== S U B R O U T I N E =====
11693
11694      ;*****
11695      ;*
11696      ;* getbx - return bx -> single byte input buffer for
11697      ;* selected aux port ([auxnum])
11698      ;*
11699      ;*
11700      ;*****
11701
11702      getbx:      mov     bx, [auxnum] ; return bx -> single byte input buffer
11703      00000397 8B1E[2100] ; for selected aux port ([auxnum])
11704      ;add     bx, offset auxbuf
11705      add     bx, auxbuf ; 19/10/2022
11706      0000039B 81C3[1600] ; 12/12/2022
11707      ; cf=0 (if [auxnum] is valid number)
11708      0000039F C3      retn
11709
11710      ; -----
11711
11712      ; 15/10/2022
11713
11714      ;-----
11715      ;
11716      ; clock device driver
11717      ;
11718      ; this file contains the clock device driver.
11719      ;
11720      ; the routines in this files are:
11721      ;
11722      ; routine function
11723      ;-----
11724      ; tim_writ set the current time
11725      ; tim_read read the current time
11726      ; time_to_ticks convert time to corresponding
11727      ; number of clock ticks
11728      ;
11729      ; the clock ticks at the rate of:
11730      ;
11731      ; 1193180/65536 ticks/second (about 18.2 ticks per second):
11732      ; see each routine for information on the use.
11733      ;
11734      ;-----
11735
11736      ; convert time to ticks
11737      ; input : time in cx and dx
11738      ; ticks returned in cx:dx
11739
11740      ;19/07/2019
11741      ;09/03/2019
11742
11743      time_to_ticks:      ; 0070h:2906h = 02C7h:0396h
11744
11745      ; first convert from hour,min,sec,hund. to
11746      ; total number of 100th of seconds
11747
11748      mov     al, 60
11749      mul     ch ; hours to minutes
11750      000003A0 B03C      mov     ch, 0
11751      000003A2 F6E5      add     ax, cx ; total minutes
11752      000003A4 B500      mov     cx, 6000 ; 60*100
11753      000003A6 01C8      mov     bx, dx ; get out of the way of the multiply
11754      000003A8 B97017      mul     cx ; convert to 1/100 sec
11755      000003AB 89D3      mov     cx, ax
11756      000003AD F7E1      mov     al, 100
11757      000003AF 89C1      mul     bh ; convert seconds to 1/100 sec
11758      000003B1 B064      add     cx, ax ; combine seconds with hours and min
11759      000003B3 F6E7      adc     dx, 0 ; ripple carry
11760      000003B5 01C1      mov     bh, 0
11761      000003B7 83D200      add     cx, bx ; combine 1/100 sec
11762      000003BA B700      adc     dx, 0
11763      000003BC 01D9
11764      000003BE 83D200
11765
11766      ; dx:cx is time in 1/100 sec
11767
11768      xchg     ax, dx
11769      000003C1 92      xchg     ax, cx ; now time is in cx:ax
11770      000003C2 91      mov     bx, 59659
11771      000003C3 BB0BE9      mul     bx ; multiply low half
11772      000003C6 F7E3      xchg     dx, cx
11773      000003C8 87D1      xchg     ax, dx ; cx->ax, ax->dx, dx->cx
11774      000003CA 92      mul     bx ; multiply high half
11775      000003CB F7E3      add     ax, cx ; combine overlapping products
11776      000003CD 01C8      adc     dx, 0
11777      000003CF 83D200      xchg     ax, dx ; ax:dx=time*59659
11778      000003D2 92      mov     bx, 5
11779      000003D3 BB0500      div     bl ; divide high half by 5
11780      000003D6 F6F3      mov     cl, al
11781      000003D8 88C1      mov     ch, 0
11782      000003DA B500      mov     al, ah ; remainder of divide-by-5
11783      000003DC 88E0      cbw
11784      000003DE 98      xchg     ax, dx ; use it to extend low half
11785      000003DF 92      div     bx ; divide low half by 5
11786      000003E0 F7F3      mov     dx, ax ; cx:dx is now number of ticks in time
11787      000003E2 89C2      retf ; far return
11788      000003E4 CB
11789
11790      ; -----

```

```

11790
11791 ; 17/10/2022
11792 ; 15/10/2022
11793
11794
11795
11796 ; tim_writ sets the current time
11797
11798 ; on entry es:[di] has the current time:
11799
11800 ; number of days since 1-1-80 (word)
11801 ; minutes (0-59) (byte)
11802 ; hours (0-23) (byte)
11803 ; hundredths of seconds (0-99) (byte)
11804 ; seconds (0-59) (byte)
11805
11806 ; each number has been checked for the correct range.
11807
11808 ; NOTE: Any changes in this routine probably require corresponding
11809 ; changes in the version that is built with the power manager driver.
11810 ; See ptime.asm.
11811
11812
11813
11814 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
11815 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:03EAh
11816 tim_writ: ; 2C7h:3DBh = 70h:294Bh
11817 mov ax, [es:di]
11818 push ax ; daycnt. we need to set this at the very
11819 ; end to avoid tick windows.
11820 cmp byte [havecmosclock], 0
11821 ; cmp ds:havecmosclock, 0
11822 jz short no_cmos_1
11823 mov al, [es:di+3] ; near indirect calls
11824 ; get binary hours
11825 ; convert to bcd
11826 ; call far [bintobcd]
11827 ; call ds:bintobcd ; call far [bintobcd]
11828 ; 08/08/2023
11829 call bintobcd
11830 mov ch, al ; ch = bcd hours
11831 mov al, [es:di+2] ; get binary minutes
11832 ; call far [bintobcd]
11833 ; call ds:bintobcd ; convert to bcd
11834 call bintobcd
11835 mov cl, al ; cl = bcd minutes
11836 mov al, [es:di+5] ; get binary seconds
11837 ; call far [bintobcd]
11838 ; call ds:bintobcd
11839 call bintobcd
11840
11841 mov dh, al ; dh = bcd seconds
11842 mov dl, 0 ; dl = 0 (st) or 1 (dst)
11843 cli
11844 mov ah, 3
11845 int 1Ah ; CLOCK - SET REAL TIME CLOCK (AT,XT286,CONV,PS)
11846 ; CH = hours in BCD, CL = minutes in BCD
11847 ; DH = seconds in BCD, DL = 01h if daylight savings, 00h if standard
11848
11849 ; Return: CMOS clock set
11850 sti
11851 no_cmos_1:
11852 mov cx, [es:di+2]
11853 mov dx, [es:di+4]
11854 ; 17/10/2022
11855 call far [ttticks]
11856 ; call dword ptr ds:ttticks ; call far [ttticks]
11857 ; convert time to ticks
11858 ; cx:dx now has time in ticks
11859 ; turn off timer
11860 cli
11861 mov ah, 1
11862 int 1Ah ; CLOCK - SET TIME OF DAY
11863 ; CX:DX = clock count
11864 ; Return: time of day set
11865 ; pop ds:daycnt
11866 pop word [daycnt]
11867 sti
11868 ; cmp ds:havecmosclock, 0
11869 cmp byte [havecmosclock], 0
11870 jz short no_cmos_2
11871 ; 08/08/2023
11872 call far [daycnttoday]
11873 ; call ds:daycnttoday ; call far [daycnttoday]
11874 ; convert to bcd format
11875 call daycnttoday
11876 cli
11877 mov ah, 5
11878 int 1Ah ; CLOCK - SET DATE IN REAL TIME CLOCK (AT,XT286,CONV,PS)
11879 ; DL = day in BCD, DH = month in BCD, CL = year in BCD
11880 ; CH = century (19h or 20h)
11881 ; Return: CMOS clock set
11882 sti
11883 no_cmos_2:
11884 ; 12/12/2022
11885 ; cf=0
11886 ; clc
11887 ret
11888
11889 ; -----
11890
11891 ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
11892 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0440h
11893 %if 1
11894
11895 ; CMOS clock setting support routines used by MSCLOCK.
11896 ; Warning!!! This code will be dynamically relocated by MSINIT.
11897
11898 daycnttoday: ; proc near
11899
11900 ; entry: [daycnt] = number of days since 1-1-80
11901 ;
11902 ; return: ch - century in bcd
11903 ; cl - year in bcd
11904 ; dh - month in bcd
11905 ; dl - day in bcd
11906
11907 ; 20/12/2023 - Retro DOS v5.0
11908
11909 ; 08/08/2023 (ds:) (near proc)
11910 ; 16/10/2022 (cs:) (far proc)
11911 push word [daycnt] ; save daycnt
11912 cmp word [daycnt], 7305 ; (365*20+(20/4))

```

```

11913                                     ; # days from 1-1-1980 to 1-1-2000
11914 00000444 7308                     jnb     short century20
11915                                     ;mov     byte [base_century], 19
11916                                     ;mov     byte [base_year], 80
11917                                     ; 08/08/2023
11918 00000446 C706[8D04]1350          mov     word [base_century], 5013h
11919 0000044C EB0C                     jmp     short years
11920                                     ; -----
11921
11922 century20:
11923                                     ;mov     byte [base_century], 20
11924                                     ;mov     byte [base_year], 0
11925                                     ; 08/08/2023
11926 0000044E C706[8D04]1400          mov     word [base_century], 20
11927 00000454 812E[8904]891C          sub     word [daycnt], 7305 ; (365*20+(20/4))
11928                                     ; adjust daycnt
11929
11930 0000045A 31D2                     years:  xor     dx, dx
11931 0000045C A1[8904]                 mov     ax, [daycnt]
11932 0000045F BB8505                 mov     bx, 1461 ; (366+365*3)
11933                                     ; # of days in a Leap year block
11934 00000462 F7F3                     div     bx ; AX = # of leap block, DX = daycnt
11935 00000464 8916[8904]                 mov     [daycnt], dx ; save daycnt left
11936 00000468 B304                     mov     bl, 4
11937 0000046A F6E3                     mul     bl ; AX = # of years. Less than 100
11938 0000046C 0006[8E04]                 add     [base_year], al ; So, ah = 0. Adjust year
11939 00000470 FF06[8904]                 inc     word [daycnt] ; set daycnt to 1 base
11940                                     ; 08/08/2023
11941 00000474 BB6E01                 mov     bx, 366
11942 00000477 B90300                 mov     cx, 3
11943                                     ;cmp     word [daycnt], 366 ; daycnt=remainder of leap year
11944 0000047A 391E[8904]                 cmp     [daycnt], bx ; 366
11945 0000047E 7619                     jbe     short leapyear ; within 366+355+355+355 days.
11946 00000480 FE06[8E04]                 inc     byte [base_year] ; if daycnt <= 366, then leap year
11947                                     ;sub     word [daycnt], 366 ; else daycnt--, base_year++ ;
11948 00000484 291E[8904]                 sub     [daycnt], bx ; 366 ; 08/08/2023
11949                                     ;mov     cx, 3 ; And next three years are normal
11950                                     ; 08/08/2023
11951 00000488 4B                     dec     bx ; 365
11952                                     ; 20/12/2023
11953                                     ;cmp     word [daycnt], 365 ; for(i=1; i>3 or daycnt <=365; i++)
11954 00000489 391E[8904]                 cmp     [daycnt], bx ; 365 ; 08/08/2023
11955 0000048D 760F                     jbe     short yeardone ; {if (daycnt > 365)
11956 0000048F FE06[8E04]                 inc     byte [base_year] ; { daycnt -= 365
11957                                     ;sub     word [daycnt], 365 ; }
11958 00000493 291E[8904]                 sub     [daycnt], bx ; 365 ; 08/08/2023
11959 00000497 E2F0                     loop    regularyear ; }
11960                                     ;
11961                                     ; should never fall through loop
11962
11963 00000499 C606[9004]1D             leapyear: mov     byte [february], 29 ; 08/08/2023
11964                                     ;mov     byte [month_tab+1], 29 ; leap year.
11965                                     ; change month table.
11966
11967 0000049E 31DB                     yeardone: xor     bx, bx
11968 000004A0 31D2                     xor     dx, dx
11969 000004A2 A1[8904]                 mov     ax, [daycnt]
11970                                     ;mov     si, offset month_tab
11971 000004A5 BE[8F04]                 mov     si, month_tab ; 19/10/2022
11972                                     ;mov     cx, 12
11973                                     ; 08/08/2023
11974 000004A8 B10C                     months:  mov     cl, 12
11975
11976 000004AA FEC3                     inc     bl
11977                                     ; 08/08/2023
11978 000004AC 8A14                     mov     dl, [si] ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:04B7h
11979 000004AE 39D0                     cmp     ax, dx ; cmp daycnt for each month till fit
11980                                     ; dh=0
11981 000004B0 7605                     jbe     short month_done
11982 000004B2 46                     inc     si ; next month
11983 000004B3 29D0                     sub     ax, dx ; adjust daycnt
11984 000004B5 E2F3                     loop    months ;
11985                                     ; should never fall through loop
11986
11987 000004B7 C606[9004]1C             month_done: mov     byte [february], 28 ; 08/08/2023
11988                                     ;mov     byte [month_tab+1], 28
11989                                     ; restore month table value
11990 000004BC 88DA                     mov     dl, bl
11991 000004BE 8A36[8E04]                 mov     dh, [base_year]
11992 000004C2 8A0E[8D04]                 mov     cl, [base_century] ; al=day,dl=month,dh=year,cl=cntry
11993 000004C6 E81600                 call    bintobcd ; convert "day" to bcd
11994                                     ; dl = bcd day, al = month
11995 000004C9 86D0                     xchg     dl, al
11996 000004CB E81100                 call    bintobcd ; dh = bcd month, al = year
11997 000004CE 86F0                     xchg     dh, al
11998 000004D0 E80C00                 call    bintobcd ; cl = bcd year, al = century
11999 000004D3 86C8                     xchg     cl, al
12000 000004D5 E80700                 call    bintobcd ; ch = bcd century
12001 000004D8 88C5                     mov     ch, al
12002 000004DA 8F06[8904]                 pop     word [daycnt] ; restore original value
12003 000004DE C3                     retn
12004
12005                                     ; -----
12006
12007 bintobcd: ; proc near ; real time clock support
12008
12009 ;convert a binary input in al (less than 63h or 99 decimal)
12010 ;into a bcd value in al. ah destroyed.
12011
12012 000004DF D40A                     aam ; AH = AL/10, AL = AL MOD 10
12013 000004E1 D510                     aad 10h ; db 0D5h,10h
12014                                     ; AL = (AH*10H)+AL, AH = 0
12015 000004E3 C3                     retn
12016 %endif
12017
12018                                     ; -----
12019
12020 ; 15/10/2022
12021
12022                                     ; -----
12023 ; gettime reads date and time
12024 ; and returns the following information:
12025 ;
12026 ; es:[di] =count of days since 1-1-80
12027 ; es:[di+2]=hours
12028 ; es:[di+3]=minutes
12029 ; es:[di+4]=seconds
12030 ; es:[di+5]=hundredths of seconds
12031 ;
12032 ; NOTE: Any changes in this routine probably require corresponding
12033 ; changes in the version that is built with the power manager driver.
12034 ; See ptime.asm.
12035                                     ; -----
12036

```

```

12037 tim_read:          ; 2C7h:435h = 70h:29A5h
12038          call    GetTickCnt
12039          mov     si, [daycnt]
12040
12041      ; we now need to convert the time in tick to the time in 100th of
12042      ; seconds. the relation between tick and seconds is:
12043      ;
12044      ;         65,536 seconds
12045      ; -----
12046      ;        1,193,180 tick
12047
12048      ; to get to 100th of second we need to multiply by 100. the equation is:
12049      ;
12050      ; ticks from clock * 65,536 * 100
12051      ; ----- = time in 100th of seconds
12052      ;        1,193,180
12053
12054      ; fortunately this formula simplifies to:
12055      ;
12056      ; ticks from clock * 5 * 65,536
12057      ; ----- = time in 100th of seconds
12058      ;        59,659
12059
12060      ; the calculation is done by first multiplying tick by 5. next we divide by
12061      ; 59,659. in this division we multiply by 65,536 by shifting the dividend
12062      ; my 16 bits to the left.
12063
12064      ; start with ticks in cx:dx
12065      ; multiply by 5
12066
12067          mov     ax, cx
12068          mov     bx, dx          ; startwith ticks in cx:dx
12069                                  ; multiply by 5
12070          shl     dx, 1
12071          rcl     cx, 1          ; times 2
12072          shl     dx, 1
12073          rcl     cx, 1          ; times 4
12074          add     dx, bx
12075          adc     ax, cx          ; times 5
12076          xchg    ax, dx
12077
12078      ; now have ticks * 5 in dx:ax
12079      ; we now need to multiply by 65536 and divide by 59659 d.
12080
12081          mov     cx, 59659      ; get divisor
12082          div     cx              ; dx now has remainder
12083                                  ; ax has high word of final quotient
12084
12085      ; 08/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
12086      ; mov     bx, ax          ; put high word in safeplace
12087      ; xchg    bx, ax
12088      ; xor     ax, ax          ; this is the multiply by 65536
12089      ; div     cx              ; bx:ax now has time in 100th of seconds
12090
12091      ; rounding based on the remainder may be added here
12092      ; the result in bx:ax is time in 1/100 second.
12093
12094          mov     dx, bx
12095          mov     cx, 200        ; extract 1/100's
12096
12097      ; division by 200 is necessary to ensure no overflow--max result
12098      ; is number of seconds in a day/2 = 43200.
12099
12100          div     cx
12101          cmp     dl, 100        ; remainder over 100?
12102          jnb     short noadj
12103          sub     dl, 100        ; keep 1/100's less than 100
12104      noadj:
12105          cmc
12106          mov     bl, dl          ; if we subtracted 100, carry is now set
12107                                  ; save 1/100's
12108
12109      ; to compensate for dividing by 200 instead of 100, we now multiply
12110      ; by two, shifting a one in if the remainder had exceeded 100.
12111          rcl     ax, 1
12112          mov     dl, 0
12113          rcl     dx, 1
12114          ; mov     cx, 60          ; divide out seconds
12115          ; 20/12/2023
12116          mov     cl, 60
12117          div     cx
12118          mov     bh, dl          ; save the seconds
12119          div     cl              ; break into hours and minutes
12120          xchg    al, ah
12121
12122      ; time is now in ax:bx (hours, minutes, seconds, 1/100 sec)
12123
12124      ; 08/08/2023
12125      ; push     ax
12126      ; mov     ax, si          ; daycnt
12127      ; xchg    ax, si
12128      ; stosw
12129      ; pop     ax
12130      ; xchg    ax, si          ; al = hours, ah = minutes
12131      ; stosw
12132      ; mov     ax, bx
12133      ; stosw
12134      ; cld
12135      ; [es:di] = count of days since 1-1-80
12136      ; [es:di+2] = hours
12137      ; [es:di+3] = minutes
12138      ; [es:di+4] = seconds
12139      ; [es:di+5] = hundredths of seconds
12140      ;
12141      ; ===== S U B   R O U T I N E =====
12142
12143      ; 15/10/2022
12144
12145      ; -----
12146      ;
12147      ; procedure : GetTickCnt
12148      ;
12149      ; Returns the tick count in CX:DX. Takes care of DayCnt in case
12150      ; of rollover [except when power management driver is in use].
12151      ; Uses the following logic for updating Daycnt
12152      ;
12153      ; if ( rollover ) {
12154      ;     if ( t_switch )
12155      ;         daycnt++;
12156      ;     else
12157      ;         daycnt += rollover ;
12158      ; }
12159
12160      ; USES : AX

```

```

12161 ;
12162 ; RETURNS : CX:DX - tick count
12163 ; MODIFIES : daycnt
12164 ;
12165 ;-----
12166 ;
12167 ; 17/10/2022
12168 GetTickCnt:
12169     xor     ah, ah
12170     int     1Ah
12171 ; CLOCK - GET TIME OF DAY
12172 ; Return: CX:DX = clock count
12173 ; AL = 00h if clock was read or written (via AH=0,1) since the previous
12174 ; midnight
12175 ; Otherwise, AL > 0
12176 ; 20/12/2023
12177     xor     ah, ah
12178     cmp     byte [t_switch], ah ; 0
12179     ;cmp     byte [t_switch], 0 ; use old method ? (>0 is yes)
12180     jnz     short inc_case ; old method assumes that Int 1Ah returns rollover flag
12181     ;xor     ah, ah ; new method assumes that Int 1Ah returns roll over count
12182     ;and not flag
12183     add     [daycnt], ax
12184     retn
12185 ;-----
12186 inc_case:
12187     or      al, al
12188     jz      short no_rollover
12189     inc     word [daycnt]
12190 no_rollover:
12191     retn
12192 ;-----
12193 ;-----
12194 ; 03/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
12195 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0556h
12196
12197 %if 1
12198 fat_12_id: db 'FAT12 '
12199 fat_16_id: db 'FAT16 '
12200 fat_32_id: db 'FAT32 '
12201 nul_vid:  db 'NO NAME '
12202
12203 %endif
12204
12205 ;-----
12206 ; MSDISK.ASM - MSDOS 6.0 - 1991
12207 ;-----
12208 ; 15/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
12209 ; 09/03/2019 - Retro DOS v4.0
12210
12211 ; MSDISK.ASM - MSDOS 3.3 - 02/02/1988
12212 ; 26/05/2018 - Retro DOS v3.0
12213 ; 23/03/2018 - Retro DOS v2.0
12214
12215 ;error_unknown_media equ 7 ; for use in BUILD BPB call
12216
12217 ;struc BPB_TYPE
12218 ;.SECSIZE: resw 1
12219 ;.SECALL:  resb 1
12220 ;.RESNUM:  resw 1
12221 ;.FATNUM:  resb 1
12222 ;.DIRNUM:  resw 1
12223 ;.SECNUM:  resw 1
12224 ;.FATID:   resb 1
12225 ;.FATSIZE: resw 1
12226 ;.SLIM:    resw 1
12227 ;.HLIM:    resw 1
12228 ;.HIDDEN:  resw 1
12229 ;.size:    resw 1
12230 ;endstruc
12231
12232 ;-----
12233 ; disk interface routines
12234 ;-----
12235
12236 ; device attribute bits:
12237 ; bit 6 - get/set map for logical drives and generic ioctl.
12238
12239 ;MAXERR equ 5
12240 ;MAX_HD_FMT_ERR equ 2
12241
12242 ;LSTDRV equ 504h
12243
12244 ; some floppies do not have changeline. as a result, media-check would
12245 ; normally return i-don't-know, the dos would continually reread the fat and
12246 ; discard cached data. we optimize this by implementing a logical door-latch:
12247 ; it is physically impossible to change a disk in under 2 seconds. we retain
12248 ; the time of the last successful disk operation and compare it with the current
12249 ; time during media-check. if < 2 seconds and at least 1 timer tick has passed,
12250 ; the we say no change. if > 2 seconds then we say i-don't-know. finally,
12251 ; since we cannot trust the timer to be always available, we record the number
12252 ; of media checks that have occurred when no apparent time has elapsed. while
12253 ; this number is < a given threshold, we say no change. when it exceeds that
12254 ; threshold, we say i-don't-know and reset the counter to 0. when we store
12255 ; the time of last successful access, if we see that time has passed too,
12256 ; we reset the counter.
12257
12258 accessmax equ 5
12259
12260 ; due to various bogosities, we need to continually adjust what the head
12261 ; settle time is. the following algorithm is used:
12262 ;
12263 ; get the current head settle value.
12264 ; if it is 0, then
12265 ; set slow = 15
12266 ; else
12267 ; set slow = value
12268 ;
12269 ;
12270 ;-----
12271 ;***** old algorithm *****
12272 ;* if we are seeking and writing then
12273 ;* use slow
12274 ;* else
12275 ;* use fast
12276 ;*****
12277 ;***** ibm's requested logic *****
12278 ;* if we are seeking and writing and not on an at then
12279 ;* use slow
12280 ;* else
12281 ;* use fast
12282 ;*
12283 ;

```

```

12284 ; restore current head settle value
12285 ;
12286 ;
12287 ;
12288 multrk_on equ 10000000b ;user spcified mutitrack=on, or system turns
12289 ; it on after handling config.sys file as a
12290 ; default value, if multrk_flag = multrk_off1.
12291 multrk_off1 equ 00000000b ;initial value. no "multitrack=" command entered.
12292 multrk_off2 equ 00000001b ;user specified multitrack=off.
12293
12294 ; close data segment, open Bios_Code segment
12295 ;
12296 ; 15/10/2022
12297
12298 ; BIOSCODE:04A2h (MSDOS 6.21, IO.SYS)
12299
12300 ;-----
12301 ; command jump table
12302 ;-----
12303
12304 0000056E 00 db 0
12305
12306 ; 11/12/2022
12307 %if 0
12308
12309 dsktbl: db 26 ; 2C7h:4A2h = 70h:2A12h
12310 ; ((dtbl_siz-1)/2) ; this is the size of the table ; 26
12311 dw 1742h ; dsk_init
12312 dw 4EBh ; media_chk
12313 dw 592h ; get_bpb
12314 dw 0D5h ; bc_cmderr
12315 dw 857h ; dsk_read
12316 dw 83Dh ; x_bus_exit
12317 dw 558h ; ret_carry_clear
12318 dw 558h ; ret_carry_clear
12319 dw 849h ; dsk_writ
12320 dw 841h ; dsk_writv
12321 dw 558h ; ret_carry_clear
12322 dw 558h ; ret_carry_clear
12323 dw 0D5h ; bc_cmderr
12324 dw 80Ah ; dsk_open
12325 dw 81Ah ; dsk_close
12326 dw 831h ; dsk_rem
12327 dw 558h ; ret_carry_clear
12328 dw 558h ; ret_carry_clear
12329 dw 558h ; ret_carry_clear
12330 dw 0C6Bh ; do_generic_ioctl
12331 dw 558h ; ret_carry_clear
12332 dw 558h ; ret_carry_clear
12333 dw 558h ; ret_carry_clear
12334 dw 1124h ; ioctl_getown
12335 dw 1142h ; ioctl_setown
12336 dw 129Ah ; ioctl_support_query
12337
12338 ;dtbl_siz equ $-dsktbl
12339
12340 %endif
12341
12342 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
12343 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0579h
12344
12345 ; 21/12/2023 - Retro DOS v5.0
12346 ; 11/12/2022
12347 0000056F 1A dsktbl: db (dtbl_siz-1)/2 ; 26 ; this is the size of the table
12348 00000570 [4A1A] dw dsk_init
12349 00000572 [B805] dw media_chk
12350 00000574 [5706] dw get_bpb
12351 ;dw bc_cmderr
12352 00000576 [4A0E] dw ioctl_input ; PCDOS 7 ; 21/12/2023
12353 00000578 [7209] dw dsk_read
12354 0000057A [5809] dw x_bus_exit
12355 0000057C [2206] dw ret_carry_clear
12356 0000057E [2206] dw ret_carry_clear
12357 00000580 [6409] dw dsk_writ
12358 00000582 [5C09] dw dsk_writv
12359 00000584 [2206] dw ret_carry_clear
12360 00000586 [2206] dw ret_carry_clear
12361 ;dw bc_cmderr
12362 00000588 [F70D] dw ioctl_output ; PCDOS 7 ; 21/12/2023
12363 0000058A [2909] dw dsk_open
12364 0000058C [3809] dw dsk_close
12365 0000058E [4E09] dw dsk_rem
12366 00000590 [2206] dw ret_carry_clear
12367 00000592 [2206] dw ret_carry_clear
12368 00000594 [2206] dw ret_carry_clear
12369 00000596 [CC0E] dw do_generic_ioctl
12370 00000598 [2206] dw ret_carry_clear
12371 0000059A [2206] dw ret_carry_clear
12372 0000059C [2206] dw ret_carry_clear
12373 0000059E [A713] dw ioctl_getown
12374 000005A0 [C413] dw ioctl_setown
12375 000005A2 [1C15] dw ioctl_support_query
12376
12377 dtbl_siz equ $-dsktbl
12378
12379 ; ===== S U B R O U T I N E =====
12380
12381 ; -----
12382 ; setdrive scans through the data structure of bdss, and returns a pointer to
12383 ; the one that belongs to the drive specified. carry is set if none exists
12384 ; for the drive. Pointer is returned in es:[di]
12385 ;
12386 ; AL contains the logical drive number.
12387 ; -----
12388
12389 SetDrive:
12390 ;les di, dword ptr ds:start_bds ; Point es:di to first bds
12391 000005A4 C43E[1901] les di, [start_bds] ; 19/10/2022
12392
12393 X_Scan_Loop:
12394 000005A8 26384505 cmp [es:di+5], al
12395 000005AC 7409 jz short X_SetDrv
12396 000005AE 26C43D les di, [es:di] ; [es:di+BDS.link] ; Go to next bds
12397 000005B1 83FFFF cmp di, 0FFFFh
12398 000005B4 75F2 jnz short X_Scan_Loop
12399 000005B6 F9 stc
12400 000005B7 C3 X_SetDrv: retn
12401
12402 ; -----
12403
12404 ; 15/10/2022
12405
12406 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
12407 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:05C2h

```



```

12408
12409
12410 ; if id is f9, have a 96tpi disk else
12411 ; if bit 2 is 0 then media is not removable and could not have changed
12412 ; otherwise if within 2 secs of last disk operation media could not
12413 ; have changed, otherwise dont know if media has changed
12414 ; -----
12415
12416 media_chk: ; 2C7h:4EBh = 70h:2A5Bh
12417 call SetDrive
12418 mov si, 1
12419 ; 21/12/2023
12420 test byte [es:di+40h], 1
12421 ;test byte [es:di+24h], 1 ; [es:di+BDS.flags+1]
12422 ; fchanged_by_format
12423 jz short weAreNotFakingIt
12424 ; 21/12/2023
12425 and byte [es:di+40h], 0FEh
12426 ; 12/12/2022
12427 ;and byte [es:di+24h], 0FEh ; ~fchanged_by_format
12428 ;and word [es:di+23h], 0FEFFh ; [es:di+BDS.flags]
12429 ; ~fchanged_by_format ; reset flag
12430 mov byte [tim_drv], 0FFh ; -1
12431 ; Ensure that we ask the rom if media has changed
12432 ; 21/12/2023
12433 test byte [es:di+3Fh], 1
12434 ;test byte [es:di+23h], 1 ; [es:di+BDS.flags]
12435 ; fnon_removable
12436 jz short wehaveafloppy
12437 ;mov si, 0FFFFh ; Indicate media changed
12438 ; 11/08/2023
12439 neg si ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:05E0h
12440 jmp short Media_Done ; Media_Done
12441 ; -----
12442
12443 weAreNotFakingIt:
12444 ; 21/12/2023
12445 test byte [es:di+3Fh], 1
12446 ;test byte [es:di+BDS.flags], fnon_removable
12447 ;test byte [es:di+23h], 1
12448 jnz short Media_Done
12449
12450 wehaveafloppy:
12451 ;xor si, si ; 0 ; Presume "I don't know"
12452 ; 11/08/2023
12453 dec si ; 0 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:05EBh
12454 ; If we have a floppy with changeline support, we ask the ROM
12455 ; to determine if media has changed. We do not perform the
12456 ; 2 second check for these drives.
12457
12458 cmp byte [fhav96], 0 ; Do we have changeline support?
12459 jz short mChk_NoChangeline ; Brif not
12460 call mediacheck ; Call into removable routine
12461 jb short err_exitj
12462 call haschange
12463 jnz short Media_Done
12464
12465 mChk_NoChangeline:
12466 ; If we come here, we have a floppy with no changeline support
12467
12468 mov si, 1 ; Presume no change
12469 mov al, [tim_drv] ; Last drive accessed
12470 cmp al, [es:di+4] ; [es:di+BDS.drivenum]
12471 ; Is drive of last access the same?
12472 jnz short Media_Unk ; No, then "i don't know"
12473 call Check_Time_Of_Access
12474 jmp short Media_Done
12475 ; -----
12476
12477 Media_Unk:
12478 dec si ; 0 ; Return "I don't know"
12479 ; SI now contains the correct value for media change.
12480 ; Clean up the left overs
12481
12482 Media_Done:
12483 ; 19/10/2022
12484 push es
12485 les bx, [ptrsav]
12486 mov [es:bx+0Eh], si ; [es:bx+trans]
12487 pop es
12488 or si, si
12489 jns short ret_carry_clear ; volidok
12490 cmp byte [fhav96], 0
12491 jz short mChk1_NoChangeline ; Brif no changeline support
12492 call media_set_vid
12493
12494 mChk1_NoChangeline:
12495 mov byte [tim_drv], 0FFh ; -1
12496 ; Make sure we ask rom for media check
12497
12498 ret_carry_clear:
12499 cld ; volidok
12500 retn
12501 ; -----
12502
12503 err_exitj:
12504 call maperror ; guaranteed to set carry
12505
12506 ret81:
12507 mov ah, 81h ; return error status
12508 retn ; return with carry set
12509
12510 ; ===== S U B R O U T I N E =====
12511 ; -----
12512 ; perform a check on the time passed since the last access for this physical
12513 ; drive.
12514 ; we are accessing the same drive. if the time of last successful access was
12515 ; less than 2 seconds ago, then we may presume that the disk was not changed.
12516 ; returns in si:
12517 ; 0 - if time of last access was >= 2 seconds
12518 ; 1 - if time was < 2 seconds (i.e no media change assumed)
12519 ; registers affected ax,cx,dx, flags.
12520 ;
12521 ; assume es:di -> bds, ds->Bios_Data
12522 ; -----
12523
12524 ; 21/12/2023 - Retro DOS v5.0 IBMBIO.COM
12525 ; 19/10/2022
12526 Check_Time_Of_Access:
12527 mov si, 1 ; presume no change.
12528 call GetTickCnt ; cx:dx is the elapsed time
12529 ; 21/12/2023
12530 mov ax, [es:di+79h]
12531 ;mov ax, [es:di+47h] ; [es:di+BDS.tim_lo]
12532 ; get stored time
12533 sub dx, ax
12534 ; 21/12/2023

```

```

12532 00000636 268B457B      mov     ax, [es:di+7Bh]
12533                        ;mov     ax, [es:di+49h]          ; [es:di+BDS.tim_hi]
12534 0000063A 19C1         sbb     cx, ax
12535                        ; 11/08/2023
12536                        ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0646h
12537                        ;mov     al, [accesscount]
12538 0000063C 7515         jnz     short timecheck_unk ; cx<=>0 => >1 hour
12539 0000063E 09D2         or      dx, dx          ; time must pass
12540 00000640 750C         jnz     short timepassed ; yes, examine max value
12541                        ; 11/08/2023
12542                        ;inc     al
12543                        ;cmp     al, 5
12544                        ;inc     byte [accesscount]
12545                        ;cmp     byte [accesscount], 5
12546                        ;          ; if count is less than threshold, ok
12547                        ;jb      short timecheck_ret
12548                        ;dec     byte [accesscount] ; don't let the count wrap
12549                        ; 11/08/2023
12550                        ;dec     al
12551                        ;jmp     short timecheck_unk ; "i don't know" if media changed
12552                        ; 11/08/2023
12553 00000642 803E[1D01]04   cmp     byte [accesscount], 4
12554 00000647 730A         jnb     short timecheck_unk
12555 00000649 FE06[1D01]     inc     byte [accesscount]
12556 0000064D C3           retn
12557
12558                        ; -----
12559
12560                        timepassed:
12561 0000064E 83FA24         cmp     dx, 36          ; 18*2 ; 18.2 tics per second.
12562                        ;          ; min elapsed time? (2 seconds)
12563 00000651 7601         jbe     short timecheck_ret ; yes, presume no change
12564                        ;
12565                        ; everything indicates that we do not know what has happened.
12566                        timecheck_unk:
12567 00000653 4E           dec     si          ; presume i don't know
12568                        timecheck_ret:
12569                        ; 11/08/2023
12570                        ;mov     [accesscount], al
12571 00000654 C3           retn
12572
12573                        ; -----
12574                        ; 15/10/2022
12575                        Err_Exitj2:
12576 00000655 EB CD         jmp     short err_exitj
12577
12578                        ; -----
12579
12580                        ; 15/10/2022
12581
12582                        ; =====
12583                        ; Build a valid bpb for the disk in the drive.
12584                        ; =====
12585
12586                        ; 21/12/2023 - Retro DOS v5.0 IBMBIO.COM
12587                        ; 19/10/2022
12588
12589 00000657 268A25         get_bpb: mov     ah, [es:di]          ; 2C7h:592h = 70h:2B02h
12590 0000065A E847FF         call    SetDrive          ; get fat id byte read by dos
12591                        ; 21/12/2023
12592 0000065D 26F6453F01     test     byte [es:di+3Fh], 1
12593                        ;test     byte [es:di+23h], 1 ; [es:di+BDS.flags]
12594                        ;          ; fnon_removable
12595 00000662 7523         jnz     short already_gotbpb ; no need to build      for fixed disks
12596
12597                        ; let's set the default value for volid,vol_serial,
12598                        ; filesys_id in bds table
12599
12600 00000664 E83600         call    clear_ids
12601                        ;mov     ds:set_id_flag, 1 ; indicate to      set system id in bds
12602 00000667 C606[9B04]01   mov     byte [set_id_flag], 1
12603 0000066C E86700         call    GetBp              ; build a bpb if necessary
12604 0000066F 72B6         jb      short ret81
12605                        ;cmp     ds:set_id_flag, 2 ; already, volume_label set from boot
12606 00000671 803E[9B04]02   cmp     byte [set_id_flag], 2
12607                        ;mov     ds:set_id_flag, 0 ; record to bds table?
12608 00000676 C606[9B04]00   mov     byte [set_id_flag], 0
12609 0000067B 740A         jz      short already_gotbpb ; do not set it again from      root dir
12610                        ;          ; otherwise, conventional boot record
12611                        ;cmp     ds:fhave96, 0 ; do we have changeline      support?
12612 0000067D 803E[7700]00   cmp     byte [fhave96], 0
12613 00000682 7403         jz      short already_gotbpb ; brief not
12614 00000684 E80A16         call    set_volume_id
12615                        already_gotbpb:
12616 00000687 83C706         add     di, 6          ; BDS.BPB
12617                        ;          ; return the bpb from the current bds
12618
12619                        ; fall into setptrsav, es:di -> result
12620
12621                        ; -----
12622
12623                        ; 15/10/2022
12624
12625                        ; =====
12626                        ; Setptrsav is also jumped to from dsk_init (msbio2.asm). In both cases, the
12627                        ; pointer to be returned is in es:di. We were incorrectly returning ds:di.
12628                        ; Note that this works in most cases because most pointers are in Bios_Data.
12629                        ; It fails, for instance, when we install an external drive using driver.sys
12630                        ; because then the BDS segment is no longer Bios_Data.
12631                        ; NB: It is fine to corrupt cx because this is not a return value and anyway
12632                        ; this returns to Chardev_entry (msbio1.asm) where all registers are
12633                        ; restored before returning to the caller.
12634                        ; =====
12635
12636                        ; 21/12/2023
12637                        %if 0
12638                        ; 19/10/2022
12639                        SetPtrSav: ; return point for dsk_init
12640                        mov     cx, es          ; save es
12641                        les     bx, ds:ptrsav
12642                        les     bx, [ptrsav]
12643                        mov     [es:bx+0Dh], ah ; [es:bx+media]
12644                        mov     [es:bx+12h], di ; [es:bx+count]
12645                        mov     [es:bx+14h], cx ; [es:bx+count+2]
12646                        cld
12647                        retn
12648
12649                        %endif
12650                        ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
12651                        ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:0698h
12652                        SetPtrSav: ; return point for dsk_init
12653 0000068A 1E           push    ds
12654                        lds     bx, ds:ptrsav
12655 0000068B C51E[1200]     lds     bx, [ptrsav]

```



```

12656 0000068F 88670D      mov     [bx+0Dh], ah    ; [bx+media]
12657 00000692 897F12      mov     [bx+12h], di    ; [bx+count]
12658 00000695 8C4714      mov     [bx+14h], es    ; [bx+count+2]
12659 00000698 1E                push    ds
12660 00000699 07                pop     es
12661 0000069A 1F                pop     ds
12662 0000069B F8                clc
12663 0000069C C3                retn
12664
12665 ; ===== S U B   R O U T I N E =====
12666
12667 ; 15/10/2022
12668
12669 ; -----
12670 ; clear ids in bds table. only applied for floppies.
12671 ; input: es:di -> bds table
12672 ; assumes ds: -> Bios_Data
12673 ; output: volid set to "NO NAME "
12674 ; vol_serial set to 0.
12675 ; filesys_id set to "FAT12 " or "FAT16 "
12676 ; depending on the flag fatsize in bds.
12677 ;
12678 ; trashes si, cx
12679 ; -----
12680
12681 ;size_of_EXT_BOOT_VOL_LABEL equ 11
12682 ;size_of_EXT_SYSTEM_ID equ 8
12683
12684 ; 11/09/2023
12685 ; 14/08/2023
12686 ;BDS.fatsiz equ 1Fh
12687 ; 21/12/2023
12688 ;BDS.fatsiz equ 59
12689
12690 ; 22/12/2023
12691 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
12692
12693 clear_ids:
12694 ;mov     al, [es:di+1Fh] ; mov al,[es:di+BDS.fatsiz]
12695 ; 21/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM BugFix)
12696 ;mov     bl, [es:di+3Bh] ; mov bl,[es:di+BDS.fatsiz]; *+
12697
12698 clear_ids_x:
12699 ; 21/12/2023
12700 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:06ABh)
12701 ; 11/09/2023
12702 ; (MSDOS 5.0 IO.SYS - BIOSCODE:05D9h)
12703 push     di
12704 xor      cx, cx          ; no serial number
12705 ; 21/12/2023
12706 mov     [es:di+89h], cx    ; [es:di+BDS.vol_serial]
12707 mov     [es:di+8Bh], cx    ; [es:di+BDS.vol_serial+2]
12708 ;mov     [es:di+57h], cx    ; [es:di+BDS.vol_serial]
12709 ;mov     [es:di+59h], cx    ; [es:di+BDS.vol_serial+2]
12710
12711 ; BUGBUG - there's a lot in common here and with
12712 ; mov_media_ids.. see if we can save some space by
12713 ; merging them... jgl
12714
12715 ;mov     cx, 11          ; size_of_EXT_BOOT_VOL_LABEL
12716 ; 10/12/2022
12717 mov     cl, 11 ; cx = 11
12718
12719 ;;mov     si, offset vol_no_name ; "NO NAME "
12720 ;mov     si, vol_no_name    ; 19/10/2022
12721 ; 22/12/2023
12722 ;mov     si, offset nul_vid ; "NO NAME "
12723 mov     si, nul_vid
12724
12725 ; 21/12/2023
12726 add     di, 125
12727 ;add     di, 75          ; BDS.volid
12728
12729 ;rep movsb
12730 ; 21/12/2023
12731 ;rep movs byte ptr es:[di], byte ptr cs:[si] ; cs rep movsb
12732 ; 26/12/2023
12733 ;cs      ; vol_no_name is in BIOSCODE segment
12734 ;rep movsb
12735 rep
12736 cs
12737 movsb
12738
12739 ; 11/09/2023 (BugFix, DI is not start addr of BDS structure here)
12740 ;;test byte [es:di+BDS.fatsiz], fbig
12741 ;; (MSDOS 5.0 IO.SYS - BIOSCODE:05EFh)
12742 ;test byte [es:di+1Fh], 40h
12743 ; 21/12/2023 - Retro DOS v5.0
12744 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:06C3h)
12745 ;test byte [es:di+59], 20h
12746 ; (here, es:di points to the BDS offset +136)
12747 ; purpose: test byte [es:di+BDS.fatsiz], fbigbig
12748 ; applied: test byte [es:BDS.fatsiz+136], fbigbig -BUG!-
12749
12750 ; (PCDOS 7.1 BUG note: 26/06/2023 - Erdogan Tan)
12751 ;; ! NOTE - 11/08/2023 - Erdogan Tan (Retro DOS v4.2 IO.SYS bugfix)
12752 ; Microsoft/IBM code has a bug here because the BDS's
12753 ; .volid and .filesys_id fields will be reset
12754 ; (to their default text) according to 'BDS.fatsiz' flags
12755 ; at the BDS offset 59 but current (this) code checks flags
12756 ; at ES:DI+59 while DI points the BDS offset 136!? ; (PCDOS 7.1)
12757 ;; at the BDS offset 31 but current (this) code checks flags
12758 ;; at ES:DI+31 while DI points the BDS offset 86!? ; (MSDOS 6.22)
12759 ;
12760 ; Correct Code:
12761 ; ;test byte [ES:59],20h or [ES:BDS.fatsiz],fbigbig ; (PCDOS 7.1)
12762 ; ;test byte [ES:31],40h or [ES:BDS.fatsiz],fbig ; (MSDOS 6.22)
12763 ; 11/09/2023
12764 ; (before 'rep movsb') 'mov al,[es:di+BDS.Fatsiz]' and then
12765 ; (after 'rep movsb') 'test al,fbig' (AL is free/proper to use here)
12766 ;
12767 ; Same BUG is existing in MSDOS 6.22 IO.SYS - BIOSCODE:05EFh
12768 ; and in Windows ME IO.SYS - BIOSCODE:0E1Ah as 'test byte [es:di+59],20h'
12769 ;
12770 ; (why this bug did not affect MSDOS and PCDOS 7.x applications:
12771 ; 'clear_ids' is used for floppy disks only and the default
12772 ; option of 'clear_ids' is FAT12 volid and filesys_id text
12773 ; when the flag bit has wrong value for FAT16/40h or FAT32/20h.)
12774
12775 ; 21/12/2023 - Retro DOS v5.0
12776 ;mov     si, offset fat_32_id ; "FAT32 "
12777 mov     si, fat_32_id
12778
12779 ; 21/12/2023
12780 ; BugFix (of the PCDOS 7.1 IBMBIO.COM BUG) ; *+

```

```

12780 ;test bl, fbigbig ; FAT32 flag
12781 000006BC F6C320 test bl, 20h ; * ; BL = [es:BDS.fatsiz] = [es:59]
12782 000006BF 750B jnz short ci_bigfat
12783
12784 ;mov si, offset fat_16_id ; "FAT16"
12785 000006C1 BE[5305] mov si, fat_16_id ; 19/10/2022
12786
12787 ; 21/12/2023
12788 ; !BUG! (PCDOS 7.1 IBMBIO.COM BIOSCODE:06CDh)
12789 ;test byte [es:di+59], 40h ; [es:di+BDS.fatsiz], fbig
12790 ; BugFix ; *+
12791 ;test bl, fbig ; FAT16 flag
12792 000006C4 F6C340 test bl, 40h ; * ; Retro DOS v5.0
12793 ;test al, 40h ; * ; Retro DOS v4.2
12794 000006C7 7503 jnz short ci_bigfat
12795
12796 ;mov si, offset fat_12_id ; "FAT12"
12797 000006C9 BE[4B05] mov si, fat_12_id ; 19/10/2022
12798 ci_bigfat:
12799 ;mov cx, 8 ; size_of_EXT_SYSTEM_ID
12800 ; 10/12/2022
12801 000006CC B108 mov cl, 8 ; cx = 8
12802 000006CE 83C705 add di, 5 ; (BDS.filesys_id-BDS.volid)-size_of_EXT_BOOT_VOL_LABEL
12803 ; filesys_id field
12804 ;rep movsb
12805 ; 21/12/2023 - Retro DOS v5.0
12806 ;rep movs byte ptr es:[di], byte ptr cs:[si] ; 0F3h,2Eh,0A4h
12807 ; 26/12/2023
12808 ;cs ; fat32_id, fat16_id and fat12_id are in BIOSCODE segment
12809 ;rep movsb
12810 000006D1 F3 rep
12811 000006D2 2E cs
12812 000006D3 A4 movsb
12813
12814 000006D4 5F pop di ; restore bds pointer
12815 getret_exit: ; 21/12/2023
12816 000006D5 C3 retn
12817
12818 ; ===== S U B R O U T I N E =====
12819
12820 ; 15/10/2022
12821
12822 ; -----
12823 ; getbp - return bpb from the drive specified by the bds.
12824 ; if the return_fake_bpb flag is set, then it does nothing.
12825 ; note that we never come here for fixed disks.
12826 ; for all other cases,
12827 ; - it reads boot sector to pull out the bpb
12828 ; - if no valid bpb is found, it then reads the fat sector,
12829 ; to get the fat id byte to build the bpb from there.
12830
12831 ; inputs: es:di point to correct bds.
12832
12833 ; outputs: fills in bpb in current bds if valid bpb or fat id on disk.
12834 ; carry set, and al=7 if invalid disk.
12835 ; carry set and error code in al if other error.
12836 ; if failed to recognize the boot record, then will set the
12837 ; set_id_flag to 0.
12838 ; this routine will only work for a floppy diskette.
12839 ; for a fixed disk, it will just return.
12840
12841 ; ***** Note: getbp is a clone of getbp which uses the newer
12842 ; segment definitions. It should be migrated towards.
12843 ; now es:di has the bds, ds: has Bios_Data
12844 ; -----
12845
12846 ; 29/12/2023
12847 ; 21/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
12848 GetBp:
12849 ; if returning fake bpb then return bpb as is.
12850 ; 21/12/2023
12851 000006D6 26F6453F05 test byte [es:di+3Fh], 5 ; PCDOS 7.1
12852 test byte [es:di+BDS.flags], return_fake_bpb|fnon_removable
12853 ;test byte [es:di+23h], 5 ; MSDOS 6.22 (& MSDOS 5.0)
12854 ;jz short getbp1 ; getbp1
12855 ;jmp getret_exit
12856 ; 21/12/2023
12857 000006DB 75F8 jnz short getret_exit
12858
12859 ; -----
12860 getbp1:
12861 000006DD 51 push cx
12862 000006DE 52 push dx
12863 000006DF 53 push bx
12864
12865 ; attempt to read in boot sector and determine bpb.
12866 ; we assume that the 2.x and greater dos disks all
12867 ; have a valid boot sector.
12868 call readbootsec
12869 000006E0 E8CF00 jb short getbp_err_ret_brdg ; carry set if there was error.
12870 000006E3 720A or bx, bx ; bx is 0 if boot sector is valid.
12871 000006E5 09DB jnz short dofatbpb
12872 000006E7 7509 call movbpb ; move bpb into registers
12873 000006E9 E81401 ;jmp short Has1
12874 ; 21/12/2023 - Retro DOS v5.0 (PCDOS 7.1) IBMBIO.COM
12875 000006EC E9B500 jmp getret
12876
12877 ; -----
12878 getbp_err_ret_brdg:
12879 000006EF E9B600 jmp getbp_err_ret
12880
12881 ; -----
12882 ; we have a 1.x diskette. In this case read in the fat ID byte
12883 ; and fill in bpb from there.
12884 dofatbpb:
12885 000006F2 E8B401 call readfat ; puts media descriptor byte inah
12886 000006F5 72F8 jb short getbp_err_ret_brdg
12887 ;cmp ds:fhave96, 0 ; changeline support available?
12888 000006F7 803E[7700]00 cmp byte [fhave96], 0 ; 19/10/2022
12889 000006FC 7403 jz short bpb_nochangeline ; brif not
12890 000006FE E83115 call hidensity ; may not return! May add sp, 2 and
12891 ; ; jump to has1!!!!!! or has720K
12892 bpb_nochangeline: ; test for a valid 3.5" medium
12893 ; 21/12/2023 - Retro DOS v5.0
12894 00000701 26807D3E02 cmp byte [es:di+3Eh], 2
12895 ;cmp byte [es:di+22h], 2 ; [es:di+BDS.formfactor]
12896 ; ; ffsmall
12897 00000706 7512 jnz short is_floppy
12898 00000708 80FCF9 cmp ah, 0F9h ; is it a valid fat id byte for 3.5" ?
12899 0000070B 7512 jnz short got_unknown_medium
12900 Has720K:
12901 ; 21/12/2023
12902 ;mov bx, offset sm92 ; pointer to correct bpb
12903 ;mov bx, sm92 ; 19/10/2022

```

```

12904
12905 ; es points to segment of bds. the following should be modified
12906 ; to get spf,csec,spau,spt correctly. it had been wrong if
12907 ; driver.sys is loaded since the bds is inside the driver.sys.
12908
12909 ; 21/12/2023
12910 ; 10/12/2022
12911 ; mov al, [bx+0] ; [bx+bpbtype.spf]
12912 ; 21/12/2022
12913 ; mov al, [bx]
12914 ; mov cx, [bx+3] ; [bx+bpbtype.csec]
12915 ; mov dx, [bx+5] ; [bx+bpbtype.spau]
12916 ; mov bx, [bx+1] ; [bx+bpbtype.spt]
12917 ; 19/10/2022 - Temporary !
12918 ; db 8Ah, 87h, 0, 0 ; mov al, [bx+0]
12919 ; db 8Bh, 8Fh, 3, 0 ; mov cx, [bx+3]
12920 ; db 8Bh, 97h, 5, 0 ; mov dx, [bx+5]
12921 ; db 8Bh, 9Fh, 1, 0 ; mov bx, [bx+1]
12922
12923 ; 21/12/2023 - Retro DOS v5.0
12924 0000070D B003 mov al, 3 ; bpbtype.sbf = 3
12925 0000070F B9A005 mov cx, 1440 ; bpbtype.csec = 1440
12926 00000712 BA0202 mov dx, 202h ; dl = bpbtype.spau = 2
12927 ; dh = bpbtype.chead = 2
12928 00000715 BB0970 mov bx, 7009h ; bl = bpbtype.spt = 9
12929 ; bh = bpbtype.dire = 112
12930 00000718 EB30 jmp short Has1
12931 ; -----
12932 is_floppy: ; must be a 5.25" floppy if we come here
12933 ; cmp ah, 0F8h ; valid media?? (0F8h-0FFh)
12934 0000071A 80FCF8 ; jb short got_unknown_medium
12935 ; 21/12/2023
12936 ; jnb short chk_160K
12937 0000071D 730A
12938 ; -----
12939 ; 21/12/2023
12940 ; we have a 3.5" diskette for which we cannot build a bpb.
12941 ; we do not assume any type of bpb for this medium.
12942 got_unknown_medium:
12943 ; mov ds:set_id_flag, 0
12944 0000071F C606[9B04]00 mov byte [set_id_flag], 0
12945 00000724 B007 mov al, 7
12946 00000726 F9 stc
12947 00000727 EB7B jmp short getret
12948 ; -----
12949 chk_160K:
12950 00000729 B001 mov al, 1 ; set number of fat sectors
12951 0000072B BB0840 mov bx, 16392 ; 64*256+8
12952 ; set dir entries and sector max
12953 0000072E B94001 mov cx, 320 ; 40*8
12954 ; set size of drive
12955 00000731 BA0101 mov dx, 257 ; 01*256+1
12956 ; set head limit and sec/all unit
12957 ; 21/12/2023
12958 ; mov al, 1 ; bpbtype.sbf = 1
12959 ; mov bx, 4008h ; bl = bpbtype.spt = 8
12960 ; bh = bpbtype.dire = 64
12961 ; mov cx, 140h ; bpbtype.csec = 320
12962 ; mov dx, 101h ; dl = bpbtype.spau = 1
12963 ; dh = bpbtype.chead = 1
12964
12965 00000734 F6C402 test ah, 2 ; test for 8 or 9 sector
12966 00000737 7505 jnz short has8 ; nz = has 8 sectors
12967
12968 ; 29/12/2023
12969 ; inc al ; 2 ; inc number of fat sectors
12970 ; inc bl ; 9 ; inc sector max
12971 00000739 40 inc ax
12972 0000073A 43 inc bx
12973
12974 ; add cx, 40 ; increase size (to 360)
12975 ; 18/12/2022
12976 0000073B 80C128 add cl, 40 ; 28h ; 180K (360 sectors)
12977
12978 has8: test ah, 1 ; test for 1 or 2 heads
12979 00000741 7407 jz short Has1 ; jz = 1 head
12980 00000743 01C9 add cx, cx ; double size of disk
12981 00000745 8770 mov bh, 112 ; increase number of directory entries
12982 00000747 FEC6 inc dh ; 2 ; inc sec/all unit
12983 ; 29/12/2023
12984 ; inc dl ; 2 ; inc head limit
12985 00000749 42 inc dx
12986
12987 Has1: ; 02/09/2023 (PCDOS 7.1, IBMBIO.COM - BIOSCODE:0754h)
12988 0000074A 1E push ds
12989 0000074B 06 push es
12990 0000074C 1F pop ds
12991
12992 ; mov [es:di+8], dh ; [es:di+BDS.secperclus]
12993 ; mov [es:di+0Ch], bh ; [es:di+BDS.direntries]
12994 ; mov [es:di+0Eh], cx ; [es:di+BDS.totalsecs16]
12995 ; mov [es:di+10h], ah ; [es:di+BDS.media]
12996 ; mov [es:di+11h], al ; [es:di+BDS.fatsecs]
12997 ; mov [es:di+13h], bl ; [es:di+BDS.secpertrack]
12998 ; mov [es:di+15h], dl ; [es:di+BDS.heads]
12999
13000 mov [di+8], dh ; [di+BDS.secperclus]
13001 00000750 30F6 xor dh, dh
13002 00000752 895515 mov [di+15h], dx ; [di+BDS.heads]
13003 00000755 88FA mov dl, bh
13004 00000757 89550C mov [di+0Ch], dx ; [di+BDS.direntries]
13005 0000075A 894D0E mov [di+0Eh], cx ; [di+BDS.totalsecs16]
13006 0000075D 894D1B mov [di+1Bh], cx ; [di+BDS.totalsecs32]
13007 00000760 886510 mov [di+10h], ah ; [di+BDS.media]
13008 00000763 88C2 mov dl, al
13009 00000765 895511 mov [di+11h], dx ; [di+BDS.fatsecs]
13010 00000768 88DA mov dl, bl
13011 0000076A 895513 mov [di+13h], dx ; [di+BDS.secpertrack]
13012
13013 ; the BDS_BPB.BPB_HIDDENSECTORS+2 field and the
13014 ; BDS_BPB.BPB_BIGTOTALSECTORS field need to be set
13015 ; to 0 since this code is for floppies
13016
13017 ; 18/12/2022
13018 ; mov word [es:di+19h], 0 ; [es:di+BDS.hiddensecs+2]
13019 ; mov word [es:di+17h], 0 ; [es:di+BDS.hiddensecs]
13020 ; mov word [es:di+1Dh], 0 ; [es:di+BDS.totalsecs32+2]
13021 ; 18/12/2022
13022 0000076D 29C9 sub cx, cx ; 0
13023 ; mov [es:di+19h], cx ; 0 ; [es:di+BDS.hiddensecs+2]
13024 ; mov [es:di+17h], cx ; 0 ; [es:di+BDS.hiddensecs]
13025 ; mov [es:di+1Dh], cx ; 0 ; [es:di+BDS.totalsecs32+2]
13026
13027 ; 02/09/2023

```

```

13028 0000076F 894D19      mov     [di+19h], cx ; 0 ; [di+BDS.hiddensecs+2]
13029 00000772 894D17      mov     [di+17h], cx ; 0 ; [di+BDS.hiddensecs]
13030 00000775 894D1D      mov     [di+1Dh], cx ; 0 ; [di+BDS.totalsecs32+2]
13031
13032      ; 21/12/2023 - Retro DOS v5.0
13033 00000778 894D1F      mov     [di+1Fh], cx ; [di+BDS.fatsecs32] ; BPB_FATSz32
13034 0000077B 894D21      mov     [di+21h], cx ; [di+BDS.fatsecs32+2]
13035 0000077E 894D27      mov     [di+27h], cx ; [di+BDS.rootdirclust]
13036 00000781 894D29      mov     [di+29h], cx ; [di+BDS.rootdirclust+2]
13037 00000784 894D2F      mov     [di+2Fh], cx ; [di+BDS.reserved]
13038      ; BPB_Reserved (12 zero bytes)
13039 00000787 894D31      mov     [di+31h], cx
13040 0000078A 894D33      mov     [di+33h], cx
13041 0000078D 894D35      mov     [di+35h], cx
13042 00000790 894D37      mov     [di+37h], cx
13043 00000793 894D39      mov     [di+39h], cx
13044 00000796 894D23      mov     [di+23h], cx ; [di+BDS.extflags] ; BPB_ExtFlags
13045 00000799 894D25      mov     [di+25h], cx ; [di+BDS.fsver] ; BPB_FSVer
13046
13047 0000079C 49          dec     cx ; -1 ; 0FFFFFFFh
13048 0000079D 894D2B      mov     [di+2Bh], cx ; [di+BDS.fsinfo] ; BPB_FSInfo
13049 000007A0 894D2D      mov     [di+2Dh], cx ; [di+BDS.bkbootsec] ; BPB_BkBootSec
13050
13051 000007A3 1F          pop     ds ; 02/09/2023
13052      getret:
13053 000007A4 5B          pop     bx
13054 000007A5 5A          pop     dx
13055 000007A6 59          pop     cx
13056      ;getret_exit:
13057 000007A7 C3          retn     ; 21/12/2023
13058      ; -----
13059
13060      getbp_err_ret:      ; before doing anything else, set set_id_flag to 0.
13061      ;mov     ds:set_id_flag, 0
13062      ; 19/10/2022
13063 000007A8 C606[9B04]00      mov     byte [set_id_flag], 0
13064 000007AD E8F905      call    maperror
13065 000007B0 EBF2      jmp     short getret
13066      ; -----
13067      ; 21/12/2023
13068      ; we have a 3.5" diskette for which we cannot build a bpb.
13069      ; we do not assume any type of bpb for this medium.
13070
13071      got_unknown_medium:
13072      ;mov     ds:set_id_flag, 0
13073      ;mov     byte [set_id_flag], 0
13074      ;mov     al, 7
13075      ;stc
13076      ;jmp     short getret
13077
13078      ; ===== S U B   R O U T I N E =====
13079
13080      ; 15/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
13081
13082      ; -----
13083      ; read in the boot sector. set carry if error in reading sector.
13084      ; bx is set to 1 if the boot sector is invalid, otherwise it is 0.
13085      ;
13086      ; assumes es:di -> bds, ds-> Bios_Data
13087      ; -----
13088
13089      ; 10/03/2019 - Retro DOS v4.0
13090
13091      ; 30/12/2022 - Retro DOS v4.2
13092      ; (MSDOS 6.21 IO.SYS, BIOSCODE:06C3h)
13093      ; ((MSDOS 6.22 IO.SYS, BIOSCODE:06C3h)) ; 22/12/2023
13094
13095      ; 22/12/2023 - Retro DOS v5.0
13096      ; (PCDOS 7.1 IBMBIO.COM, BIOSCODE:07C6h)
13097
13098      readbootsec:
13099 000007B2 B600      mov     dh, 0 ; head 0
13100 000007B4 B90100      mov     cx, 1 ; cylinder 0, sector 1
13101 000007B7 E8FC00      call    read_sector
13102 000007BA 7243      jb     short err_ret
13103 000007BC 31DB      xor     bx, bx ; assume valid boot sector
13104
13105      ; put a sanity check for the boot sector in here to detect
13106      ; boot sectors that do not have valid bpb. we examine the
13107      ; first two bytes - they must contain a long jump (69h) or a
13108      ; short jump (EBh) followed by a nop (90h), or a short jump
13109      ; (E9h). if this test is passed, we further check by examining
13110      ; the signature at the end of the boot sector for the word
13111      ; AA55h. if the signature is not present, we examine the media
13112      ; descriptor byte to see if it is valid. for dos 3.3, this
13113      ; logic is modified a little bit. we are not going to check
13114      ; signature. instead we are going to sanity check the media
13115      ; byte in bpb regardless of the validity of signature. this is
13116      ; to save the already developed commercial products that have
13117      ; good jump instruction and signature but with the false bpb
13118      ; informations
13119
13120      ; that will crash the diskette drive operation. (for example, symphony diskette).
13121
13122      ; 02/09/2023
13123      ; 19/10/2022
13124      ;cmp     byte [disksector], 69h ; is it a direct jump?
13125      ;jz     short check_bpb_mediabyte ; don't need to find a nop
13126      ;cmp     byte [disksector], 0E9h ; dos 2.0 jump?
13127      ;jz     short check_bpb_mediabyte ; no need for nop
13128      ;cmp     byte [disksector], 0EBh ; how about a short jump?
13129      ;jnz     short invalidbootsec
13130      ;cmp     byte [disksector+2], 90h ; is next one a nop?
13131      ;jnz     short invalidbootsec
13132
13133      ; 02/09/2023 (PCDOS 7.1)
13134 000007BE A0[5201]      mov     al, [disksector]
13135 000007C1 3C69      cmp     al, 69h ; is it a direct jump?
13136 000007C3 740F      je     short check_bpb_mediabyte
13137      ; don't need to find a nop
13138 000007C5 3CE9      cmp     al, 0E9h ; dos 2.0 jump?
13139 000007C7 740B      je     short check_bpb_mediabyte
13140      ; no need for nop
13141 000007C9 3CEB      cmp     al, 0EBh ; how about a short jump?
13142 000007CB 7530      jne     short invalidbootsec
13143 000007CD 803E[5401]90      cmp     byte [disksector+2], 90h ; is next one a nop?
13144 000007D2 7529      jne     short invalidbootsec
13145
13146      ; 15/10/5022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
13147      ;
13148      ; 10/03/2019
13149      ; (MSDOS 3.3, MSDISK.ASM, 1988)
13150      ;
13151      ; Don't have to perform the following signature check since

```

```

13152 ;; we need to check the media byte even with the good signed diskette.
13153 ;;
13154 ;; check_signature:
13155 ;;     cmp     word [cs:disksector+1FEh],0AA55h ; see if non-ibm
13156 ;;                                     ; disk or 1.x media.
13157 ;;     jz      short checksinglesided ; go see if singled sided medium.
13158 ;;                                     ; may need some special handling
13159 ;;
13160 ; check for non-ibm disks which do not have the signature AA55h at the
13161 ; end of the boot sector, but still have a valid boot sector. this is done
13162 ; by examining the media descriptor in the boot sector.
13163 ;
13164 ; 19/10/2022
13165 check_bpb_mediabyte:
13166 000007D4 A0[6701]     mov     al, [disksector+15h]
13167                                     ; [disksector+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
13168 000007D7 50           push    ax ; 02/09/2023
13169 000007D8 24F0         and     al, 0F0h
13170 000007DA 3CF0         cmp     al, 0F0h ; allow for strange media
13171 000007DC 58           pop     ax ; 02/09/2023
13172 000007DD 751E         jnz     short invalidbootsec
13173 ;
13174 ; there were some (apparently a lot of them) diskettes that had been formatted
13175 ; under dos 3.1 and earlier versions which have invalid bpbs in their boot
13176 ; sectors. these are specifically diskettes that were formatted in drives
13177 ; with one head, or whose side 0 was bad. these contain bpbs in the boot
13178 ; sector that have the sec/clus field set to 2 instead of 1, as is standard
13179 ; in dos. in order to support them, we have to introduce a "hack" that will
13180 ; help our build bpb routine to recognise these specific cases, and to
13181 ; set up out copy of the bpb accordingly.
13182 ; we do this by checking to see if the boot sector is off a diskette that
13183 ; is single-sided and is a pre-dos 3.20 diskette. if it is, we set the
13184 ; sec/clus field to 1. if not, we carry on as normal.
13185 ;
13186 checksinglesided:
13187     mov     al, [disksector+15h]
13188     ; 02/09/2023
13189     ; al = [disksector+15h]
13190 000007DF 3CF0         cmp     al, 0F0h
13191 000007E1 741B         jz      short gooddsk
13192 000007E3 A801         test    al, 1
13193 000007E5 7517         jnz     short gooddsk
13194 000007E7 813E[5A01]332E     cmp     word [disksector+8], 2E33h ; "3."
13195 000007ED 7507         jnz     short mustbeearlier
13196 000007EF 803E[5C01]32     cmp     byte [disksector+0Ah], 32h ; "2"
13197 000007F4 7308         jnb     short gooddsk
13198 ;
13199 ; we must have a pre-3.20 diskette. set the sec/clus field to 1
13200 ;
13201 mustbeearlier:
13202 000007F6 C606[5F01]01     mov     byte [disksector+0Dh], 1
13203                                     ; [disksector+EXT_BOOT.BPB+EBPB.SECTORSPERCLUSTER]
13204 000007FB EB01         jmp     short gooddsk
13205 ; -----
13206 ;
13207 invalidbootsec:
13208 000007FD 43           inc     bx ; indicate that boot sector invalid
13209                                     ; 10/12/2022
13210 movbpb_ret:
13211 gooddsk:
13212 000007FE F8           clc
13213 err_ret:
13214 000007FF C3           retn
13215 ; -----
13216 ;
13217 ; 10/12/2022
13218 ;err_ret:
13219 ;retn
13220 ;
13221 ; ===== S U B R O U T I N E =====
13222 ;
13223 ; 15/10/2022
13224 ; -----
13225 ; 'movbpb' moves the bpb read from the boot sector into registers for use by
13226 ; getbp routine at hasl
13227 ;
13228 ; if the set_id_flag is 1, and if an extended boot record, then set volume
13229 ; serial number, volume label, file system id in bds according to
13230 ; the boot record. after that, this routine will set the set_id_flag to 2
13231 ; to signal that volume label is set already from the extended boot record
13232 ; (so, don't set it again by calling "set_volume_id" routine which uses
13233 ; the volume label in the root directory.)
13234 ; -----
13235 ;
13236 ; 10/03/2019 - Retro DOS v4.0
13237 ;
13238 ; 22/12/2023
13239 %if 0
13240 ; 19/10/2022
13241 movbpb:
13242     mov     dh, [disksector+0Dh]
13243                                     ; disksector+EXT_BOOT.BPB+EBPB.SECTORSPERCLUSTER]
13244                                     ; sectors per unit
13245     mov     bh, [disksector+11h]
13246                                     ; [disksector+EXT_BOOT.BPB+EBPB.ROOTENTRIES]
13247                                     ; number of directory entries
13248     mov     cx, [disksector+13h]
13249                                     ; [disksector+EXT_BOOT.BPB+EBPB.TOTALSECTORS]
13250                                     ; size of drive
13251     mov     ah, [disksector+15h]
13252                                     ; [disksector+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
13253                                     ; media descriptor
13254     mov     al, [disksector+16h]
13255                                     ; [disksector+EXT_BOOT.BPB+EBPB.SECTORSPERFAT]
13256                                     ; number of fat sectors
13257     mov     bl, [disksector+18h]
13258                                     ; [disksector+EXT_BOOT.BPB+EBPB.SECTORSPERTRACK]
13259                                     ; sectors per track
13260     mov     dl, [disksector+1Ah]
13261                                     ; [disksector+EXT_BOOT.BPB+EBPB.HEADS]
13262                                     ; number of heads
13263 %else
13264 ; 29/12/2023
13265 ; 22/12/2023 - Retro DOS v5.0
13266 ; (PCDOS 7.1 IBMBIO.COM, BIOSCODE:0814h)
13267 ;;;
13268 movbpb:
13269 00000800 57           push    di
13270 00000801 83C706         add     di, 6 ; BDS+6 = BDS.BPB
13271 00000804 8D36[5D01]     lea     si, [disksector+0Bh]
13272 00000808 B93500         mov     cx, 53 ; copy bios parameters block
13273                                     ; from BPB_BytsPerSec to (FAT32) BS_DrvNum (excluded)
13274 0000080B FC           cld
13275 0000080C F3A4         rep     movsb

```

```

13276 0000080E 8B4CD3          mov     cx, [si-45]      ; si = disksector+64 -> 64-45 = 19
13277                                ; disksektor+19 = BPB_TotSec16
13278 00000811 31C0          xor     ax, ax
13279 00000813 E308          jcxz    movbpb_bigdisk
13280 00000815 26894DE0        mov     [es:di-32], cx ; write 16 bit total sectors
13281                                ; to 32 bit total sectors field
13282 00000819 268945E2        mov     [es:di-30], ax ; BPB_TotalSec32+2 (BDS offset 29, BPB offset 23)
13283 movbpb_bigdisk:
13284 0000081D 3944D6        cmp     [si-42], ax     ; BPB_FATSz16 = disksector+22
13285 00000820 7410          jz      short movbpb_fat32
13286 movbpb_fat:
13287 00000822 83EF1C        sub     di, 28          ; di = BDS offset 31 (BPB offset 25)
13288                                ; 29/12/2023
13289 00000825 B90C00        mov     cx, 12          ; clear 12 byte extended BDS (FAT32) fields
13290                                ; (which are used only for FAT32 disks)
13291 00000828 F3AA          rep stosb
13292 0000082A 48          dec     ax              ; -1 ; 0FFFFh
13293 0000082B AB          stosw                ; set BDS offset 43 (dword) to -1
13294                                ; dword [BDS.BPB_FSInfo] = 0FFFFFFFh
13295 0000082C AB          stosw
13296 0000082D 40          inc     ax              ; ax = 0
13297 0000082E B10C        mov     cl, 12
13298                                ;mov     cx, 12          ; clear BDS offset 47 to 59
13299                                ; (BPB offset 41 to 53) (disksector offset 52 to 64)
13300 00000830 F3AA          rep stosb
13301 movbpb_fat32:
13302 00000832 5F          pop     di
13303 %endif
13304                ;;;
13305
13306 00000833 803E[9B04]01        cmp     byte [set_id_flag], 1 ; called by get_bpb?
13307 00000838 75C4          jnz     short movbpb_ret
13308 0000083A E81200        call    mov_media_ids
13309 0000083D 7205          jb      short movbpb_conv ; conventional boot record?
13310 0000083F C606[9B04]02        mov     byte [set_id_flag], 2 ; signals that volume id is set
13311 movbpb_conv:
13312 00000844 803E[7700]01        cmp     byte [fhav96], 1
13313 00000849 75B3          jnz     short movbpb_ret
13314 0000084B E83714        call    resetchanged ; reset flags in bds to      not fchanged.
13315                                ; 10/12/2022
13316                                ; cf = 0
13317 movbpb_ret:
13318                                ;clic
13319 0000084E C3          retn
13320
13321 ; ===== S U B   R O U T I N E =====
13322
13323 ;copy the boot_serial number, volume id, and filesystem id from the
13324 ;***extended boot record*** in ds:disksector to the bds table pointed
13325 ;by es:di.
13326
13327 ;in.) es:di -> bds
13328 ;      ds:disksector = valid extended boot record.
13329 ;out.) vol_serial, bds_volid and bds_system_id in bds are set according to
13330 ;      the boot record information.
13331 ;      carry flag set if not an extended bpb.
13332 ;      all registers saved except the flag.
13333
13334 ; 22/12/2023
13335 %if 0
13336 ; 19/10/2022
13337 mov_media_ids:
13338 cmp     byte [disksector+26h], 29h
13339                                ; [disksector+EXT_BOOT.SIG],
13340                                ; EXT_BOOT_SIGNATURE
13341 jnz     short mmi_not_ext
13342 push    cx
13343 mov     cx, [disksector+27h]
13344                                ; [disksector+EXT_BOOT.SERIAL]
13345 mov     [es:di+57h], cx      ; [es:di+BDS.vol_serial]
13346 mov     cx, [disksector+29h]
13347                                ; [disksector+EXT_BOOT.SERIAL+2]
13348 mov     [es:di+59h], cx      ; [es:di+BDS.vol_serial+2]
13349 push    di
13350 push    si
13351 mov     cx, 11              ; size_of_EXT_BOOT_VOL_LABEL
13352 mov     si, disksector+2Bh
13353 ;mov     si, (offset disksector+2Bh) ;
13354                                ; disksector+EXT_BOOT.VOL_LABEL
13355 add     di, 75              ; BDS.volid
13356 rep movsb
13357 ;mov     cx, 8              ; size_of_EXT_SYSTEM_ID
13358 ; 10/12/2022
13359 mov     cl, 8 ; cx = 8
13360 mov     si, disksector+36h
13361 ;mov     si, (offset disksector+36h) ; disksector+EXT_BOOT.SYSTEM_ID
13362 add     di, 5              ; (BDS.filesys_id-BDS.volid)-size_of_EXT_BOOT_VOL_LABEL
13363 rep movsb
13364 pop     si
13365 pop     di
13366 pop     cx
13367 ; 10/12/2022
13368 ; cf = 0
13369 ;clic                      ; this clic is not required (16/06/2019 - Erdogan Tan)
13370                                ; (20/09/2022)
13371 retn
13372 %else
13373 ; 22/12/2023 - Retro DOS v5.0
13374 ; (PCDOS 7.1 IBMBIO.COM, BIOSCODE:0865h)
13375                ;;;
13376 mov_media_ids:
13377 0000084F 833E[6801]00        cmp     word [disksector+16h], 0 ; BPB.FATSz16
13378 00000854 7507          jnz     short mmi_chk_fat
13379 00000856 803E[9401]29        cmp     byte [disksector+42h], 29h
13380                                ; [disksector+FAT32_EXT_BOOT.SIG],
13381                                ; EXT_BOOT_SIGNATURE
13382 0000085B EB05          jmp     short mmi_chk_fat32
13383 mmi_chk_fat:
13384 0000085D 803E[7801]29        cmp     byte [disksector+26h], 29h
13385                                ; [disksector+EXT_BOOT.SIG],EXT_BOOT_SIGNATURE
13386 mmi_chk_fat32:
13387 00000862 7543          jnz     short mmi_not_ext
13388 00000864 51          push    cx
13389 00000865 50          push    ax
13390 00000866 57          push    di
13391 00000867 56          push    si
13392 00000868 1E          push    ds
13393 00000869 833E[6801]00        cmp     word [disksector+16h], 0 ; BPB.FATSz16
13394 0000086E 750C          jnz     short mmi_fat
13395 mmi_fat32:
13396                                ; FAT32 file system
13397 ;lds     cx, dword ptr ds:disksector+43h
13398 00000870 C50E[9501]        lds     cx, [disksector+43h] ; BS_FAT32_Volid
13399 00000874 BE[9901]        mov     si, disksector+47h ; BS_FAT32_VolLab

```



```

13400 00000877 B8[A401]      mov     ax, disksector+52h    ; BS_FAT32_FilSysType
13401 0000087A EB0A          jmp     short mmi_do
13402
13403 mmi_fat:
13404      ;lds     cx, dword ptr ds:disksector+27h
13405      lds     cx, [disksector+27h] ; BS_VolID
13406      mov     si, disksector+2Bh   ; BS_VolLab
13407      mov     ax, disksector+36h   ; BS_FilSysType
13408 mmi_do:
13409      mov     [es:di+89h], cx      ; [es:di+BDS.vol_serial]
13410      ; (BDS offset 137)
13411      mov     [es:di+8Bh], ds      ; [es:di+BDS.vol_serial+2]
13412      pop     ds
13413      mov     cx, 11
13414      add     di, 125              ; di = di+125 = BDS.volid
13415      rep movsb
13416      mov     cl, 8                ; di = di+136
13417      mov     si, ax               ; BS_FilSysType or BS_FAT32_FilSysType
13418      add     di, 5                ; di = di+141 = BDS.filesys_id
13419      rep movsb
13420      pop     si
13421      pop     di
13422      pop     ax
13423      pop     cx
13424      ;clc      ; this clc is not required (16/06/2019 - Erdogan Tan)
13425      ; (20/09/2022 - 27/06/2023) MSDOS 6.21 .. PCDOS 7.1
13426      retn
13427 %endif
13428      ;;;
13429
13430 ; -----
13431 mmi_not_ext:
13432      stc
13433      retn
13434
13435 ; ===== S U B   R O U T I N E =====
13436
13437 ; 15/10/2022
13438 ; -----
13439 ; read in the fat sector and get the media byte from it.
13440 ; input : es:di -> bds
13441 ; output:
13442 ;         carry set if an error occurs, ax contains error code.
13443 ;         otherwise, ah contains media byte on exit
13444 ; -----
13445
13446 readfat:
13447      ;mov     dh, 0
13448      ; 10/12/2022
13449      xor     dh, dh
13450      mov     cx, 2                ; head 0
13451      ; cylinder 0, sector 2
13452      call    read_sector
13453      jnb     short bad_fat_ret
13454      mov     ah, [bx]             ; media byte
13455 bad_fat_ret:
13456      retn
13457
13458 ; ===== S U B   R O U T I N E =====
13459
13460 ; 15/10/2022
13461 ; -----
13462 ; read a single sector into the temp buffer.
13463 ; perform three retries in case of error.
13464 ; inputs: es:[di].bds_drivenum has physical drive to use
13465 ;         cx has sector and cylinder
13466 ;         dh has head
13467 ;         es:di has bds
13468 ;         ds has Bios_Data
13469 ;
13470 ; outputs: carry clear
13471 ;         Bios_Data:bx point to sector
13472 ;         (note: some callers assume location of buffer)
13473 ;
13474 ;         carry set
13475 ;         ax has rom error code
13476 ;
13477 ; register bp is preserved.
13478 ; -----
13479 ; 10/03/2019 - Retro DOS v4.0
13480 ; 22/12/2023 - Retro DOS v5.0
13481
13482 ; 19/10/2022
13483 read_sector:
13484      push     bp
13485      mov     bp, 3                ; make 3 attempts
13486      mov     dl, [es:di+4]        ; [es:di+BDS.drivenum]
13487      mov     bx, disksector       ; get es:bx to point to buffer
13488 rd_ret:
13489      push     es
13490      push     ds
13491      pop      es
13492      mov     ax, 201h
13493      int     13h                  ; DISK - READ SECTORS INTO MEMORY
13494      ; AL = number of sectors to read, CH = track, CL = sector
13495      ; DH = head, DL = drive, ES:BX -> buffer to fill
13496      ; Return: CF set on error, AH = status, AL = number of sectors read
13497      pop     es
13498      jnb     short okret2
13499 rd_rty:
13500      call    again                ; resetdisk, decrement bp, preserve ax
13501      jz      short err_rd_ret
13502
13503 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
13504 test     byte [es:di+3Fh], 1
13505 ;test     byte [es:di+23h], 1
13506 ;;test     byte ptr [es:di+23h], 1 ; [es:di+BDS.flags]
13507 ; ;fnon_removable
13508 jnz      short rd_ret
13509 cmp     byte [media_set_for_format], 0
13510 jnz      short rd_skip1_dpt
13511 push     ax
13512 push     ds                      ; for retry, set the head settle time to 0Fh
13513 lds     si, [dpt]
13514 ;mov     al, [si+9]              ; [si+DISK_PARMS.DISK_HEAD_STTL]
13515 ;mov     byte [si+9], 15 ; [si+DISK_PARMS.DISK_HEAD_STTL]
13516 ; ;NORMSETTLE
13517 ; 12/12/2022
13518 mov     al, 15
13519 xchg     al, [si+9]
13520
13521 000008E5 B00F
13522 000008E7 864409
13523

```

```

13524 000008EA 1F          pop     ds
13525 000008EB A2[2A01]      mov     [save_head_sttl], al
13526 000008EE 58          pop     ax
13527          rd_skip1_dpt:
13528 000008EF 06          push    es
13529 000008F0 1E          push    ds
13530 000008F1 07          pop     es
13531 000008F2 B80102      mov     ax, 201h
13532 000008F5 CD13          int     13h
13533                                     ; DISK - READ SECTORS INTO MEMORY
13534                                     ; AL = number of sectors to read, CH = track, CL = sector
13535                                     ; DH = head, DL = drive, ES:BX -> buffer to fill
13536                                     ; Return: CF set on error, AH = status, AL = number of sectors read
13536 000008F7 07          pop     es
13537 000008F8 9C          pushf
13538 000008F9 803E[A905]00      cmp     byte [media_set_for_format], 0
13539 000008FE 750E      jnz     short rd_skip2_dpt
13540 00000900 50          push    ax
13541 00000901 A0[2A01]      mov     al, [save_head_sttl]
13542 00000904 1E          push    ds
13543 00000905 C536[2D01]      lds     si, [dpt]
13544 00000909 884409      mov     [si+9], al ; [si+DISK_PARMS.DISK_HEAD_STTL]
13545 0000090C 1F          pop     ds
13546 0000090D 58          pop     ax
13547          rd_skip2_dpt:
13548 0000090E 9D          popf
13549 0000090F 7305      jnb     short okret2
13550 00000911 EBB9      jmp     short rd_rty
13551          ; -----
13552          err_rd_ret:
13553          mov     dl, 0FFh ; make sure we ask rom if media has changed
13554 00000913 B2FF          stc ; return error
13555          ;
13556 00000915 F9          ; update information pertaining to last drive accessed, time of access, last
13557          ; track accessed in that drive.
13558          ;
13559          okret2:
13560          mov     [step_drv], dl ; set up for head settle logic in disk
13561          mov     [tim_drv], dl ; save drive last accessed
13562          ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
13563 00000916 8816[7600]      mov     [es:di+78h], ch
13564          ;mov     [es:di+46h], ch ; [es:di+BDS.track]
13565          ; save last track accessed on this drive
13566 0000091E 26886D78      ; preserve flags in case error occurred
13567          pushf
13568          call    set_tim
13569          popf
13570 00000922 9C          ; restore flags
13571 00000923 E89B04      pop     bp
13572 00000926 9D          retn
13573 00000927 5D
13574 00000928 C3
13575          ; -----
13576          ; disk open/close routines
13577          ; -----
13578          ;
13579          dsk_open:
13580          ; 2C7h:80Ah = 70h:2D7Ah
13581 00000929 803E[7700]00      cmp     byte [fhave96], 0
13582 0000092E 7407      jz     short dsk_open_exit ; done if no changeline support
13583 00000930 E871FC      call    SetDrive ; get bds for drive
13584          ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
13585 00000933 26FF453C      inc     word [es:di+3Ch] ; [es:di+BDS.opcnt] ; BDS offset 60
13586          ;inc     word [es:di+20h] ; [es:di+BDS.opcnt]
13587          dsk_open_exit:
13588          ; 10/12/2022
13589          ; cf = 0
13590          ;clc
13591          ; CF is already ZERO here (18/09/2022, MSDOS 5.0 IO.SYS)
13592          ; (19/07/2019 - Erdogan Tan - MSDOS 6.0 IO.SYS - retrodos4.s)
13593          retn
13594          ; -----
13595          ;
13596          dsk_close:
13597          ; 2C7h:81Ah = 70h:2D8Ah
13598 00000938 803E[7700]00      cmp     byte [fhave96], 0
13599 0000093D 740E      jz     short exitjx ; done if no changeline support
13600 0000093F E862FC      call    SetDrive ; get bds for drive
13601          ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
13602 00000942 26837D3C00      cmp     word [es:di+3Ch], 0 ; [es:di+BDS.opcnt] ; BDS off 60
13603          ;cmp     word [es:di+20h], 0 ; [es:di+BDS.opcnt]
13604          jz     short exitjx ; watch out for wrap
13605          ; 22/12/2023
13606          dec     word [es:di+3Ch]
13607          ;dec     word [es:di+20h]
13608          exitjx:
13609          ; 10/12/2022
13610          ; cf = 0
13611          ;clc
13612          ; CF is already ZERO here (18/09/2022, MSDOS 5.0 IO.SYS)
13613          ; (19/07/2019 - Erdogan Tan - MSDOS 6.0 IO.SYS - retrodos4.s)
13614          retn
13615          ; -----
13616          ; disk removable routine
13617          ; -----
13618          ;
13619          dsk_rem:
13620          ; al is unit #
13621          ; 2C7h:831h = 70h:2DA1h
13622 0000094E E853FC      call    SetDrive ; get bds for this drive
13623          ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
13624          ;test     byte [es:di+BDS.flags], fnon_removable
13625          test     byte [es:di+3Fh], 1 ; [es:di+BDS.flags], fnon_removable
13626          jz     short exitjx
13627          ;test     byte [es:di+23h], 1 ; [es:di+BDS.flags], fnon_removable
13628          ;jnz     short x_bus_exit ; non_rem
13629          ;jnz     short non_rem ; 15/10/2022
13630          ; 10/12/2022
13631          ; cf = 0
13632          ;clc
13633          ; CF is already ZERO here
13634          ; 15/10/2022
13635          retn
13636          ; -----
13637          non_rem:
13638          x_bus_exit:
13639          mov     ah, 3 ; 2C7h:83Dh = 0070h:2DADh
13640          ; return busy status
13641          stc
13642          dsk_ret:
13643          retn
13644          ; -----
13645          ; disk i/o routines
13646          ; -----
13647          ;
13648          dsk_writv:
13649          ; 2C7h:841h = 70h:2DB1h
13650          mov     word [wrtverify], 103h

```



```

13648 ; 19/10/2022
13649 0000095C C706[2001]0301 mov word [rflag], 103h
13650 ;mov word ptr ds:rflag, 103h ; write and verify
13651 00000962 EB06 jmp short dsk_cl
13652 ; -----
13653
13654 dsk_writ: ; 2C7h:849h = 70h:2DB9h
13655 ;mov word [wrtverify], 3
13656 ; 19/10/2022
13657 00000964 C706[2001]0300 mov word [rflag], 3
13658 ;mov word ptr ds:rflag, 3 ; romwrite
13659 dsk_cl: call diskio ; romwrite
13660 0000096A E8A400 ; -----
13661
13662 dsk_io:
13663 jnb short dsk_ret
13664 0000096D 73EC jmp bc_err_cnt
13665 0000096F E965F7 ; -----
13666
13667 dsk_read: call diskrd ; ; 2C7h:857h = 70h:2DC7h
13668 jmp short dsk_io
13669 00000972 E89700
13670 00000975 EBF6
13671
13672 ; ===== S U B R O U T I N E =====
13673
13674 ; 15/10/2022
13675 ; 10/03/2019 - Retro DOS v4.0
13676 ; 22/12/2023 - Retro DOS v5.0
13677
13678 ; -----
13679 ; miscellaneous odd jump routines.
13680 ; moved out of mainline for speed.
13681
13682 ; if we have a system where we have virtual drives, we need
13683 ; to prompt the user to place the correct disk in the drive.
13684 ;
13685 ; assume es:di -> bds, ds:->Bios_Data
13686 ; -----
13687
13688 ; 19/10/2022
13689 checksingle:
13690 00000977 50 push ax
13691 00000978 53 push bx
13692 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
13693 00000979 268B5D3F mov bx, [es:di+3Fh] ; [es:di+BDS.flags]
13694 ;mov bx, [es:di+23h] ; [es:di+BDS.flags]
13695
13696 ; if hard drive, cannot change disk.
13697 ; if current owner of physical drive, no need to change diskette.
13698
13699 0000097D F6C321 test bl, 21h ; fnon_removable|fi_own_physical
13700 00000980 7573 jnz short singleret
13701 00000982 F6C310 test bl, 10h ; fi_am_mult
13702 ; is there a drive sharing this physical drive?
13703 00000985 746E jz short singleret
13704
13705 ; look for the previous owner of this physical drive
13706 ; and reset its ownership flag.
13707
13708 00000987 268A4504 mov al, [es:di+4] ; [es:di+BDS.drivenum]
13709 ; get physical drive number
13710 0000098B 06 push es ; preserve pointer to current bds
13711 0000098C 57 push di
13712 0000098D C43E[1901] les di, [start_bds] ; get first bds
13713
13714 scan_list: cmp [es:di+4], al
13715 jnz short scan_skip ; Not our drive. Try next bds.
13716 00000995 7553 mov bl, 20h ; fi_own_physical ; test ownership flag
13717 00000997 B320 ; 22/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
13718 00000999 26845D3F test [es:di+3Fh], bl ; [es:di+BDS.flags]
13719 ;test [es:di+23h], bl
13720 0000099D 744B jz short scan_skip ; he doesn't own it either. continue
13721 0000099F 26305D3F xor [es:di+3Fh], bl
13722 ;xor [es:di+23h], bl ; reset ownership flag
13723 000009A3 5F pop di ; restore pointer to current bds
13724 000009A4 07 pop es
13725 000009A5 26085D3F or [es:di+3Fh], bl ; [es:di+BDS.flags]
13726 ;or [es:di+23h], bl ; ; set ownership flag
13727
13728 ; we examine the fsetowner flag. if it is set, then we are using the code in
13729 ; checksingle to just set the owner of a drive. we must not issue the prompt
13730 ; in this case.
13731 000009A9 803E[7A00]01 cmp byte [fsetowner], 1
13732 000009AE 7517 jnz short not_fsetowner
13733 ;cmp byte ptr es:[di+4], 0 ; are we handling drive number 0 ?
13734 000009B0 26807D0400 cmp byte [es:di+4], 0
13735 000009B5 753E jnz short singleret
13736 000009B7 268A4505 mov al, [es:di+5]
13737 ;mov al, es:[di+5] ; [es:di+BDS.drivelet]
13738 ; get the DOS drive letter
13739 000009BB 06 push es
13740 000009BC 8E06[1A00] mov es, [zeroseg]
13741 000009C0 26A20405 mov [es:LSTDRV], al
13742 ;mov es:504h, al ; [es:LSTDRV]
13743 ; set up sdsb
13744 000009C4 07 pop es ; restore bds pointer
13745 000009C5 EB2E jmp short singleret
13746 ; -----
13747
13748 ; to support "backward" compatibility with ibm's "single drive status byte"
13749 ; we now check to see if we are in a single drive system and the application
13750 ; has "cleverly" diddled the sdsb
13751
13752 not_fsetowner:
13753 000009C7 803E[7800]02 cmp byte [single], 2 ; if (single_drive_system)
13754 000009CC 7517 jnz short ignore_sdsb
13755 000009CE 50 push ax
13756 000009CF 268A4505 mov al, [es:di+5] ; if (curr_drv == req_drv)
13757 000009D3 88C4 mov ah, al
13758 000009D5 06 push es
13759 000009D6 8E06[1A00] mov es, [zeroseg]
13760 000009DA 2686060405 xchg al, [es:LSTDRV]
13761 ;xchg al, es:504h ; [es:LSTDRV]
13762 ; then swap(curr_drv, req_drv)
13763 000009DF 07 pop es
13764 000009E0 38C4 cmp ah, al ; else
13765 000009E2 58 pop ax ; swap(curr_drv, req_drv)
13766 000009E3 7410 jz short singleret ; issue swap_dsk_msg
13767
13768 ignore_sdsb: call swpdsk
13769 000009E8 EB0B jmp short singleret
13770 ; -----
13771

```

```

13772 scan_skip: les di, [es:di]
13773 000009EA 26C43D ;les di, es:[di] ; [es:di+BDS.link]
13774 ; go to next bds
13775 cmp di, 0FFFFh ; -1 ; end of list?
13776 000009ED 83FFFF jnz short scan_list ; continue until hit end of list
13777 000009F0 759F stc
13778 000009F2 F9 pop di ; restore current bds
13779 000009F3 5F pop es
13780 000009F4 07 singleret:
13781 pop bx
13782 000009F5 5B pop ax
13783 000009F6 58 retn
13784 000009F7 C3
13785
13786 ; 22/12/2023
13787 %if 0
13788 ; -----
13789
13790 baddrive: mov al, 8 ; sector not found
13791 jmp short baddrive_ret
13792
13793 %endif
13794 ; -----
13795
13796 unformatteddrive: mov al, 7 ; unknown media
13797 ;baddrive_ret: stc
13798 000009F8 B007
13799 ; -----
13800 000009FA F9
13801
13802 ioret: retn
13803
13804 000009FB C3
13805
13806 ; -----
13807
13808 ; 22/12/2023 - Retro DOS v5.0
13809 ; PC DOS 7.1 IBMBIO.COM - BIOS CODE:0A1Bh
13810
13811 000009FC 10 LBA_Packet: db 16 ; ...
13812 ; DAP buffer
13813 000009FD 00 db 0
13814 000009FE 0000 dap_block_cnt: dw 0 ; ...
13815 00000A00 00000000 dap_trans_buf: dd 0 ; ...
13816 00000A04 00000000 dap_lba_value: dd 0 ; ...
13817 00000A08 00000000 dd 0
13818
13819 ; -----
13820
13821 ; 15/10/2022
13822 ; -----
13823 ; disk i/o handler
13824 ;
13825 ; al = drive number (0-6)
13826 ; ah = media descriptor
13827 ; cx = sector count
13828 ; dx = first sector (low)
13829 ; [start_sec_h] = first sector (high) 32 bit calculation.
13830 ; ds = cs
13831 ; es:di = transfer address
13832 ; [rflag]=operation (2=read, 3=write)
13833 ; [verify]=1 for verify after write
13834 ;
13835 ; if successful carry flag = 0
13836 ; else cf=1 and al contains error code
13837 ; -----
13838
13839 ; 12/12/2023
13840 ; ds = biosdata segment (cs = bioscode segment)
13841
13842 diskrd: mov ds:rflag, 2 ; romread
13843 ; 19/10/2022
13844 mov byte [rflag], 2 ; romread
13845 00000A0C C606[2001]02
13846
13847 ; ===== S U B R O U T I N E =====
13848
13849 ; 22/12/2023 - Retro DOS v5.0
13850 ; PC DOS 7.1 IBMBIO.COM - BIOS CODE:0A30h
13851 ; 22/12/2023
13852 %if 0
13853 ; 19/10/2022
13854
13855 diskio: mov bx, di ; es:bx= transfer address
13856 mov [xfer_seg], es ; save transfer segment
13857 call SetDrive
13858 mov al, [es:di+10h] ; [es:di+BDS.media]
13859 mov [medbyt], al
13860 jcxz short ioret
13861 jcxz ioret
13862
13863 ; see if the media is formatted or not by checking the flags field in
13864 ; in the bds. if it is unformatted we cannot allow i/o, so we should
13865 ; go to the error exit at label unformatteddrive.
13866 test byte [es:di+24h], 2
13867 ;test byte ptr es:[di+24h], 2 ; [es:di+BDS.flags+1]
13868 ; unformatted_media
13869 jnz short unformatteddrive
13870 mov [seccnt], cx ; save sector count
13871 mov [spsav], sp ; save sp
13872
13873 ; ensure that we are trying to access valid sectors on the drive
13874
13875 mov ax, dx
13876 xor si, si ; 0
13877 add dx, cx
13878 adc si, 0
13879 ; 02/09/2023 (PCDOS 7.1)
13880 rcl si, 1
13881 cmp word [es:di+0Eh], 0 ; [es:di+BDS.totalsecs16]
13882 ; 32 bit sector ?
13883 jz short sanity32
13884 ;cmp si, 0
13885 ; 02/09/2023
13886 or si, si
13887 jnz short baddrive
13888 cmp dx, [es:di+0Eh] ; [es:di+BDS.totalsecs16]
13889 ja short baddrive
13890 jmp short sanityok
13891
13892 ; -----
13893
13894 sanity32: add si, [start_sec_h]
13895

```

```

13896             cmp     si, [es:di+1Dh]          ; [es:di+BDS.totalsecs32+2]
13897             jb      short sanityok
13898             ja      short baddrive
13899             cmp     dx, [es:di+1Bh]          ; [es:di+BDS.totalsecs32]
13900             ja      short baddrive
13901 sanityok:
13902             mov     dx, [start_sec_h]
13903             add     ax, [es:di+17h]          ; [es:di+BDS.hiddensecs]
13904             adc     dx, [es:di+19h]          ; [es:di+BDS.hiddensecs+2]
13905
13906             ; now dx;ax have the physical first sector.
13907             ; since the following procedures is going to destroy ax, let's
13908             ; save it temporarily to saved_word.
13909
13910             mov     [saved_word], ax ; save the sector number (low)
13911
13912             ; set up pointer to disk base table in [dpt]. we cannot assume that iosetup
13913             ; will do it because we will skip the set up stuff with hard disks.
13914
13915             push    es
13916             ;mov     es, [zeroseg]
13917             ; 02/09/2023
13918             xor     si, si ; 0
13919             mov     es, si
13920             les     si, [es:DSKADR]
13921             ;les     si, es:78h          ; [es:DSKADR]
13922             ; current disk parm table
13923             mov     [dpt], si
13924             mov     [dpt+2], es
13925             pop     es
13926             test    byte [es:di+23h], 1 ; [es:di+BDS.flags]
13927             ; fnon_removable
13928             jnz     short skip_setup
13929             call    checksingle
13930
13931             ; check to see if we have previously noted a change line. the routine
13932             ; returns if everything is ok. otherwise, it pops off the stack and returns
13933             ; the proper error code.
13934
13935             cmp     byte [fhav96], 0 ; do we have changeline support?
13936             jz      short diskio_nochangeline ; brif not
13937             call    checklatchio ; will do a sneaky pop stack return
13938             ; if a disk error occurs
13939 diskio_nochangeline:
13940             call    iosetup ; set up tables and variables for i/o
13941
13942             ; now the settle values are correct for the following code
13943
13944 skip_setup:
13945
13946             ; 32 bit sector calculation.
13947             ; dx:[saved_word] = starting sector number.
13948
13949             mov     ax, dx
13950             xor     dx, dx
13951             ;div     word [es:di+13h] ; [es:di+BDS.secperttrack]
13952             ; divide by sec per track
13953             ; 02/09/2023
13954             mov     cx, [es:di+13h]
13955             div     cx
13956             mov     [temp_h], ax
13957             mov     ax, [saved_word]
13958             div     cx ; 02/09/2023
13959             ;div     word [es:di+13h] ; [es:di+BDS.secperttrack]
13960             ; now, [temp_h]:ax = track #, dx = sector
13961             ; sector number is 1 based.
13962             ;inc     dl
13963             ; 18/12/2022
13964             inc     dx
13965             mov     [cursec], dl ; save current sector
13966             mov     cx, [es:di+15h] ; es:di+BDS.heads]
13967             ; get number of heads
13968             push    ax
13969             xor     dx, dx
13970             mov     ax, [temp_h] ; divide tracks by heads per cylinder
13971             div     cx
13972             mov     [temp_h], ax
13973             pop     ax
13974             div     cx ; now, [temp_h]:ax = cylinder #, dx = head
13975             cmp     word [temp_h], 0
13976             ja      short baddrive_brdg
13977             cmp     ax, 1024 ; 2^10 currently maxium for track #.
13978             ja      short baddrive_brdg
13979             mov     [curhd], dl ; save current head
13980             mov     [curtrk], ax ; save current track
13981
13982             ; we are now set up for the i/o. normally, we consider the dma boundary
13983             ; violations here. not true. we perform the operation as if everything is
13984             ; symmetric; let the int 13 handler worry about the dma violations.
13985
13986             mov     ax, [secCnt]
13987             call    block ; (cas - call/ret)
13988             ;call    done
13989             ;retn
13990             ; 18/12/2022
13991             jmp     done
13992 %else
13993             ;; ; 22/12/2023
13994 diskio:
13995             mov     bx, di ; al = drive number
13996             ; cx = sector count
13997             ; dx = first sector (low)
13998             ; [start_sec_h] = first sector (high)
13999             ; es:bx = transfer address
14000             mov     [xfer_seg], es ; save transfer segment
14001             call    SetDrive
14002             mov     al, [es:di+10h] ; [es:di+BDS.media]
14003             mov     [medbyt], al
14004             jcxz    ioret
14005
14006             ; see if the media is formatted or not by checking the flags field in
14007             ; in the bds. if it is unformatted we cannot allow i/o, so we should
14008             ; go to the error exit at label unformatteddrive.
14009
14010             test    byte [es:di+40h], 2 ; [es:di+BDS.flags+1]
14011             ; unformatted_media
14012             jnz     short unformatteddrive
14013             mov     [secCnt], cx ; save sector count
14014             mov     [spsav], sp ; save sp
14015
14016             ; ensure that we are trying to access valid sectors on the drive
14017
14018             mov     ax, dx
14019             xor     si, si ; 0

```

```

14020 00000A36 01CA          add    dx, cx
14021 00000A38 D1D6          rcl    si, 1
14022 00000A3A 26837D0E00        cmp    word [es:di+0Eh], 0 ; [es:di+BDS.totalsecs16]
14023                                     ; > 32 bit sector ?
14024 00000A3F 740E          jz     short sanity32
14025 00000A41 09F6          or     si, si
14026 00000A43 7506          jnz    short baddrive
14027 00000A45 263B550E        cmp    dx, [es:di+0Eh] ; [es:di+BDS.totalsecs16]
14028                                     ; ja     short baddrive
14029                                     ; jmp    short sanityok
14030                                     ; 22/12/2023
14031 00000A49 7616          jna     short sanityok
14032                                     ; 29/12/2023
14033                                     ; 22/12/2023
14034                                     ; %if 1
14035                                     ; -----
14036
14037 baddrive:
14038 00000A4B B008          mov     al, 8 ; sector not found
14039                                     ; jmp    short baddrive_ret
14040                                     ; -----
14041 ;unformatteddrive:
14042                                     ; mov     al, 7 ; unknown media
14043 baddrive_ret:
14044                                     stc
14045 ;ioret:
14046 00000A4E C3          retn
14047                                     ; %endif
14048                                     ; -----
14049
14050
14051 sanity32:
14052 00000A4F 0336[9C04]        add     si, [start_sec_h]
14053 00000A53 263B751D        cmp     si, [es:di+1Dh] ; [es:di+BDS.totalsecs32+2]
14054 00000A57 7208          jb     short sanityok
14055 00000A59 77F0          ja     short baddrive
14056 00000A5B 263B551B        cmp     dx, [es:di+1Bh] ; [es:di+BDS.totalsecs32]
14057 00000A5F 77EA          ja     short baddrive
14058
14059 00000A61 8B16[9C04]        mov     dx, [start_sec_h]
14060 00000A65 26034517        add     ax, [es:di+17h] ; [es:di+BDS.hiddensecs]
14061 00000A69 26135519        adc     dx, [es:di+19h] ; [es:di+BDS.hiddensecs+2]
14062
14063                                     ; now dx;ax have the physical first sector.
14064                                     ; since the following procedures is going to destroy ax, let's
14065                                     ; save it temporarily to saved_word.
14066
14067 00000A6D A3[9E04]        mov     [saved_word], ax ; save the sector number (low)
14068
14069                                     ; set up pointer to disk base table in [dpt]. we cannot assume that iosetup
14070                                     ; will do it because we will skip the set up stuff with hard disks.
14071
14072 00000A70 06          push    es
14073 00000A71 31F6          xor     si, si ; 0
14074 00000A73 8EC6          mov     es, si
14075                                     ; les     si, dword ptr es:78h
14076 00000A75 26C4367800        les     si, [es:78h] ; INT 1Eh vector address
14077                                     ; [es:DSKADR] - current disk parm table
14078 00000A7A 8936[2D01]        mov     [dpt], si
14079 00000A7E 8C06[2F01]        mov     [dpt+2], es
14080 00000A82 07          pop     es
14081 00000A83 26F6453F01        test    byte [es:di+3Fh], 1 ; [es:di+BDS.flags], fnon_removable
14082 00000A88 7510          jnz     short chk_13h_ext_flag
14083 00000A8A E8EAFE          call    checksingle
14084
14085                                     ; check to see if we have previously noted a change line. the routine
14086                                     ; returns if everything is ok. otherwise, it pops off the stack and returns
14087                                     ; the proper error code.
14088
14089 00000A8D 803E[7700]00        cmp     byte [fhave96], 0 ; do we have changeline support?
14090 00000A92 7403          jz     short diskio_nochangeline ; brif not
14091 00000A94 E8D210        call    checklatchio ; will do a sneaky pop stack return
14092                                     ; if a disk error occurs
14093
14094 00000A97 E8E000        call    iosetup ; set up tables and variables for i/o
14095
14096
14097 00000A9A 26F6454004        test    byte [es:di+40h], 4 ; [es:di+BDS.flags+1], fLBArw
14098                                     ; LBA read/write flag
14099 00000A9F 7539          jnz     short set_lbarw_1
14100                                     ; jmp    skip_setup
14101                                     ; 22/12/2023
14102                                     ; -----
14103
14104                                     ; now the settle values are correct for the following code
14105
14106 skip_setup:
14107
14108                                     ; 32 bit sector calculation.
14109                                     ; dx:[saved_word] = starting sector number.
14110
14111                                     ; push    bp ; ! (not necessary) ; 22/12/2023
14112 00000AA1 92          xchg     ax, dx ; mov ax, dx
14113 00000AA2 31D2          xor     dx, dx
14114 00000AA4 268B4D13        mov     cx, [es:di+13h] ; [es:di+BDS.secptrack]
14115                                     ; divide by sec per track
14116 00000AA8 F7F1          div     cx
14117 00000AAA 95          xchg     ax, bp ; mov bp, ax
14118 00000AAB A1[9E04]        mov     ax, [saved_word]
14119 00000AAE F7F1          div     cx ; [es:di+BDS.secptrack]
14120                                     ; now, bp:ax = track #, dx = sector
14121                                     ; sector number is 1 based.
14122 00000AB0 42          inc     dx
14123 00000AB1 8B16[3101]        mov     [cursec], dl ; save current sector
14124 00000AB5 268B4D15        mov     cx, [es:di+15h] ; [es:di+BDS.heads]
14125                                     ; get number of heads
14126                                     ; 22/12/2023
14127                                     ; push    ax ; *
14128 00000AB9 31D2          xor     dx, dx
14129 00000ABB 95          xchg     ax, bp ; bp = * ; divide tracks by heads per cylinder
14130 00000ABC F7F1          div     cx
14131 00000ABE 95          xchg     ax, bp ; ax = *, bp = **
14132                                     ; pop     ax ; *
14133 00000ABF F7F1          div     cx ; now, bp:ax = cylinder #, dx = head
14134 00000AC1 09ED          or     bp, bp ; ** = 0 ?
14135                                     ; pop     bp ; ! ; 22/12/2023
14136                                     ; jnz     short baddrive_brdg
14137                                     ; 22/12/2023
14138 00000AC3 7586          jnz     short baddrive
14139
14140                                     ; cmp     ax, 1024 ; 2^10 currently maximum for track #.
14141                                     ; jnb     short baddrive_brdg
14142                                     ; 22/12/2023
14143 00000AC5 80FC04        cmp     ah, 4 ; if ax >= 4*256 (1024)

```

```

14144 00000AC8 7381                jnb     short baddrive
14145
14146 00000ACA 8816[3201]             mov     [curhd], dl      ; save current head
14147 00000ACE A3[3301]             mov     [curtrk], ax   ; save current track
14148
14149                                ; we are now set up for the i/o. normally, we consider the dma boundary
14150                                ; violations here. not true. we perform the operation as if everything is
14151                                ; symmetric; let the int 13 handler worry about the dma violations.
14152
14153 00000AD1 A1[2201]             mov     ax, [seccnt]
14154 00000AD4 E81F01             call    block
14155                                ;call    done
14156                                ;retn
14157                                ; 22/12/2023
14158 00000AD7 E9E500             jmp     done
14159
14160                                ; -----
14161
14162 set_lbarw_1:
14163 00000ADA A1[9E04]             mov     ax, [saved_word] ; check for mini disk
14164                                ; (logical dos drive/partition)
14165 00000ADD 26837D7901          cmp     word [es:di+79h], 1 ; [di+BDS.bdsminimini]
14166                                ; logical dos partition
14167 00000AE2 750F             jnz     short set_lbarw_2 ; not a logical dos partition/drive
14168 00000AE4 26837D7B00          cmp     word [es:di+7Bh], 0 ; [di+BDS.bdsmin_hidden_trks] (> 0)
14169 00000AE9 7408             jz      short set_lbarw_2
14170 00000AEB 26034517          add     ax, [es:di+17h]   ; [es:di+BDS.hiddensecs]
14171 00000AEF 26135519          adc     dx, [es:di+19h]   ; [es:di+BDS.hiddensecs+2]
14172
14173 set_lbarw_2:
14174 00000AF3 2EA3[040A]          mov     [cs:dap_lba_value], ax
14175 00000AF7 2E8916[060A]          mov     [cs:dap_lba_value+2], dx
14176 00000AFC 2E891E[000A]          mov     [cs:dap_trans_buf], bx
14177 00000B01 A1[A804]             mov     ax, [xfer_seg]
14178 00000B04 2EA3[020A]          mov     [cs:dap_trans_buf+2], ax
14179 00000B08 A1[2201]             mov     ax, [seccnt]
14180 00000B0B 2EA3[FE09]          mov     [cs:dap_block_cnt], ax
14181 00000B0F BD0500             mov     bp, 5
14182 00000B12 892E[A304]          mov     [vretry_cnt], bp ; verify op. retry cnt for write-verify
14183 00000B16 892E[A504]          mov     [soft_ecc_cnt], bp ; soft ecc error retry count
14184
14185 set_lbarw_3:
14186 00000B1A 268A5504          mov     dl, [es:di+4]   ; [es:di+BDS.drivenum]
14187 00000B1E 8A26[2001]          mov     ah, [rflag]    ; get read/write indicator
14188 00000B22 80C440             add     ah, 40h
14189 00000B25 30C0             xor     al, al
14190 00000B27 1E             push    ds
14191 00000B28 0E             push    cs
14192 00000B29 1F             pop     ds
14193 00000B2A BE[FC09]           mov     si, LBA_Packet
14194 00000B2D CD13             int     13h            ; LBA read/write
14195 00000B2F 1F             pop     ds
14196 00000B30 731A             jnc     short set_lbarw_7
14197 00000B32 E8AC02             call    again
14198
14199 00000B35 7503             jnz     short set_lbarw_4
14200 00000B37 E92B02             jmp     harderr
14201
14202                                ; -----
14203
14204 set_lbarw_4:
14205 00000B3A 80FCCC             cmp     ah, 0Cch        ; 22/12/2023
14206 00000B3D 7505             jnz     short set_lbarw_5 ; write fault (hard disk)
14207 00000B3F BD0100             mov     bp, 1
14208                                ; jmp     short set_lbarw_6
14209                                ; 17/04/2024
14210 00000B42 EBD6             jmp     short set_lbarw_3
14211
14212                                ; -----
14213
14214 set_lbarw_5:
14215 00000B44 C706[A504]0500      mov     word [soft_ecc_cnt], 5 ; soft ecc error retry count
14216
14217 set_lbarw_6:
14218 00000B4A EBCE             jmp     short set_lbarw_3
14219
14220                                ; -----
14221
14222 set_lbarw_7:
14223 00000B4C 813E[2001]0301      cmp     word [rflag], 103h
14224 00000B52 7523             jnz     short set_lbarw_12
14225 00000B54 B444             mov     ah, 44h
14226 00000B56 1E             push    ds
14227 00000B57 0E             push    cs
14228 00000B58 1F             pop     ds
14229 00000B59 CD13             int     13h            ; DISK - IBM/MS Extension - VERIFY SECTORS
14230                                ; (DL - drive, [SI - disk address packet])
14231 00000B5B 1F             pop     ds
14232 00000B5C 7319             jnc     short set_lbarw_12
14233 00000B5E 80FC11          cmp     ah, 11h        ; ECC corrected data error (soft error - retried OK )
14234 00000B61 7506             jnz     short set_lbarw_8
14235 00000B63 FF0E[A504]          dec     word [soft_ecc_cnt]
14236
14237 ;set_lbarw_8:
14238 00000B67 740E             jz      short set_lbarw_12
14239 00000B69 E8CF07             call    ResetDisk
14240 00000B6C 80FC11          cmp     ah, 11h
14241 00000B6F 74D9             jz      short set_lbarw_11
14242 00000B71 FF0E[A304]          dec     word [vretry_cnt]
14243                                ; jnz     short set_lbarw_9
14244                                ; jmp     harderr
14245                                ; 22/12/2023
14246 00000B75 EBBE             jmp     short set_lbarw_9
14247
14248                                ; -----
14249                                ; 22/12/2023
14250 ;set_lbarw_9:
14251                                ;
14252                                ; cmp     ah, 0Cch
14253                                ; jnz     short set_lbarw_10
14254                                ; mov     bp, 1
14255                                ; jmp     short set_lbarw_11
14256                                ; -----
14257                                ; 22/12/2023
14258 ;set_lbarw_10:
14259                                ;
14260                                ; mov     word [soft_ecc_cnt], 5 ; soft ecc error retry count
14261 ;set_lbarw_11:
14262                                ;
14263                                ; jmp     short set_lbarw_3
14264                                ; -----
14265
14266 set_lbarw_12:
14267 00000B77 31C0             xor     ax, ax
14268 skip_dpt_setting:
14269 00000B79 C3             retn
14270                                ; 23/12/2023
14271                                ;
14272 %endif

```

```

14268
14269
14270 ; -----
14271 ; 22/12/2023
14272 ;baddrive_brdg:
14273 ;jmp baddrive
14274
14275 ; ===== S U B R O U T I N E =====
14276
14277 ; -----
14278 ; set the drive-last-accessed flag for diskette only.
14279 ; we know that the hard disk will not be removed.
14280 ; es:di -> current bds.
14281 ; ds -> Bios_Data
14282 ; ax,cx,si are destroyed.
14283 ; -----
14284
14285 ; 23/12/2023 - Retro DOS v5.0
14286
14287 ; 19/10/2022
14288
14289 iosetup: mov al, [es:di+4] ; [es:di+BDS.drivenum]
14290 mov [tim_drv], al ; save drive letter
14291
14292 ; determine proper head settle values
14293
14294 cmp byte [media_set_for_format], 0
14295 jnz short skip_dpt_setting
14296 mov al, [eot] ; fetchup eot before changing ds
14297 push ds
14298 lds si, [dpt] ; get pointer to disk base table
14299 mov [si+4], al
14300
14301 ;; 23/12/2023
14302 ;mov ah, al
14303 ;mov al, [si+10] ; [si+DISK_PARMS.DISK_MOTOR_STRT]
14304 ;mov ah, [si+4] ; [si+DISK_PARMS.DISK_EOT]
14305 ;pop ds
14306 ;mov [motorstartup], al
14307 ;mov [save_eot], ah
14308 ; 06/04/2024
14309 mov ah, [si+10]
14310 pop ds
14311 mov [motorstartup], ah
14312 mov [save_eot], al
14313
14314 ; for 3.5" drives, both external as well as on the k09, we need to set the
14315 ; motor start time to 4. this checking for every i/o is going to affect
14316 ; performance across the board, but is necessary!!
14317
14318 push ds
14319 lds si, [dpt] ; get pointer to disk base table
14320 ; 23/12/2023 - Retro DOS v5.0
14321 cmp byte [es:di+3Eh], 2 ; (PCDOS 7.1 IBMBIO.COM)
14322 ;cmp byte [es:di+22h], 2 ; [es:di+BDS.formfactor]
14323 ; ; ffsma11
14324 jnz short motor_start_ok
14325 mov al, 4
14326 xchg al, [si+10] ; [si+DISK_PARMS.DISK_MOTOR_STRT]
14327 motor_start_ok:
14328
14329 ; ds:si now points to disk parameter table.
14330 ; get current settle and set fast settle
14331
14332 ;xor al, al
14333 ;inc al ; ibm wants fast settle to be 1
14334 ; 18/12/2022
14335 xor ax, ax
14336 inc ax
14337 xchg al, [si+9] ; [si+DISK_PARMS.DISK_HEAD_STTL]
14338 ; get settle and set up for fast
14339 pop ds
14340 mov [settlecurrent], al
14341 mov al, 15 ; NORMSETTLE
14342 ; someone has diddled the settle
14343 mov [settlelow], al
14344 ; 23/12/2023
14345 ;skip_dpt_setting:
14346 retn
14347
14348 ; ===== S U B R O U T I N E =====
14349
14350 ; -----
14351 ; set time of last access, and reset default values in the dpt.
14352 ;
14353 ; note: trashes (at least) si
14354 ; -----
14355
14356 ; 23/12/2023 - Retro DOS v5.0
14357
14358 ; 19/10/2022
14359
14360 done: ;test byte [es:di+23h], 1 ; [es:di+BDS.flags]
14361 ; fnon_removable
14362 ; 23/12/2023
14363 test byte [es:di+3Fh], 1 ; (PCDOS 7.1 IBMBIO.COM)
14364 jnz short ddbx ; do not set for non-removable media
14365 call set_tim
14366 ;diddleback:
14367 ; 09/12/2022
14368 diddle_back:
14369 pushf
14370 cmp byte [media_set_for_format], 0
14371 jnz short nodiddleback
14372 push ax
14373 push es
14374 les si, [dpt]
14375 mov al, [save_eot]
14376 mov [es:si+4], al ; [es:si+DISK_PARMS.DISK_EOT]
14377 mov al, [settlecurrent]
14378 mov ah, [motorstartup]
14379 mov [es:si+9], al ; [es:si+DISK_PARMS.DISK_HEAD_STTL]
14380 mov byte [es:si+3], 2 ; [es:si+DISK_PARMS.DISK_SECTOR_SIZ]
14381 mov [es:si+0Ah], ah ; [es:si+DISK_PARMS.DISK_MOTOR_STRT]
14382 pop es
14383 pop ax
14384 nodiddleback:
14385 popf
14386 ddbx:
14387 retn
14388
14389 ; ===== S U B R O U T I N E =====
14390
14391 ; -----

```



```

14392 ;read the number of sectors specified in ax,
14393 ;handling track boundaries
14394 ;es:di -> bds for this drive
14395 ;-----
14396
14397 ; 23/12/2023 - Retro DOS v5.0
14398
14399 ; 19/10/2022
14400
14401 block: or ax, ax
14402 jz short ddbx
14403 ; 23/12/2023
14404 test byte [es:di+3Fh], 1 ; (PCDOS 7.1 IBMBIO.COM)
14405 ;test byte [es:di+23h], 1 ; [es:di+BDS.flags]
14406 ; fnon_removable
14407 jz short block_floppy
14408
14409 ; check to see if multi track operation is allowed. if not
14410 ; we have to go to the block_floppy below to break up the operation.
14411
14412 test byte [multkr_flag], 80h
14413 ;test byte ptr ds:multkr_flag, 80h ; multkr_on
14414 jz short block_floppy
14415 call Disk
14416 xor ax, ax
14417 ret
14418 ; -----
14419
14420 block_floppy:
14421 ; read at most 1 track worth. perform minimization at sector / track
14422
14423 mov cl, [es:di+19] ; [es:di+BDS.secpertrack]
14424 ;inc cl
14425 ; 23/12/2023
14426 inc cx
14427 sub cl, [cursec]
14428 xor ch, ch
14429 cmp ax, cx
14430 jnb short gotmin
14431 mov cx, ax
14432 gotmin:
14433
14434 ; ax is the requested number of sectors to read
14435 ; cx is the number that we can do on this track
14436
14437 push ax
14438 push cx
14439 mov ax, cx
14440 call Disk
14441 pop cx
14442 pop ax
14443
14444 ; cx is the number of sectors just transferred
14445
14446 sub ax, cx ; reduce sectors-remaining by last i/o
14447 shl cl, 1
14448 add bh, cl ; adjust transfer address
14449 jmp short block
14450 dskerr_brdg: jmp dskerr
14451
14452 ; ===== S U B R O U T I N E =====
14453
14454 ; 15/10/2022
14455
14456 ;-----
14457 ;perform disk i/o with retries
14458 ; al = number of sectors (1-8, all on one track)
14459 ; es:di point to drive parameters
14460 ; xfer_seg:bx = transfer address
14461 ; (must not cross a 64k physical boundary)
14462 ; [rflag] = 2 if read, 3 if write
14463 ; [verify] = 0 for normal, 1 for verify after write
14464 ;-----
14465
14466 ; 18/04/2024
14467 ; 23/12/2023 - Retro DOS v5.0
14468 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0C74h)
14469
14470 ; 19/10/2022
14471
14472 Disk:
14473 ; Check for hard disk format and
14474 ; if TRUE then set max error count to 2
14475
14476 mov bp, 5 ; MAXERR
14477 ; set up retry count
14478
14479 ; 18/04/2024
14480 ; 23/12/2023
14481 ;mov cl, [es:di+3Fh]
14482 ;and cx, 1
14483 test byte [es:di+3Fh], 1
14484 ;test byte [es:di+23h], 1 ; [es:di+BDS.flags], fnon_removable
14485 jz short GetRdwrInd
14486 cmp ah, 4 ; romverify ; Is this a track verify?
14487 jz short GetRdwrInd
14488 mov bp, 2 ; This is not verify so only 1 retry
14489 GetRdwrInd:
14490 mov [vretry_cnt], bp ; verify op. retry cnt for write-verify
14491 mov [soft_ecc_cnt], bp ; soft ecc error retry count.
14492 mov ah, [rflag] ; get read/write indicator
14493
14494 ;retry:
14495 ; 09/12/2022
14496 _retry:
14497 push ax
14498 mov dx, [curtrk]
14499 ; 23/12/2023
14500 ;jcxz disk_not_mini
14501 ; 18/04/2024
14502 test byte [es:di+3Fh], 1
14503 ;test byte [es:di+23h], 1
14504 jz short disk_not_mini
14505
14506 ; 23/12/2023
14507 cmp word [es:di+79h], 1
14508 ;cmp word [es:di+47h], 1 ; [es:di+BDS.bdsbm_ismini]
14509 ; is this a mini disk? ((logical dos partition))
14510 jnz short disk_not_mini ; no. continue to next.
14511 ; 23/12/2023
14512 add dx, [es:di+7Bh]
14513 ;add dx, [es:di+49h] ; [es:di+BDS.bdsbm_hidden_trks]
14514 ; add hidden trks.
14515

```

```

14516 disk_not_mini:
14517     ror     dh, 1
14518     ror     dh, 1
14519     or      dh, [cursec]
14520     mov     cx, dx
14521     xchg    ch, cl      ; cl = sector, ch = cylinder
14522     mov     dh, [curhd] ; load current head number and
14523     mov     dl, [es:di+4] ; physical drive number
14524                        ; [es:di+BDS.drivenum]
14525                        ; 23/12/2023
14526     cmp     byte [es:di+3Eh], 5
14527     ;cmp     byte [es:di+22h], 5 ; [es:di+BDS.formfactor], ffHardFile
14528     jz      short do_fast ; hard files use fast speed
14529
14530     ; if we have [step_drv] set to -1, we use the slow settle time.
14531     ; this helps when we have just done a reset disk operation and the head has
14532     ; been moved to another cylinder - the problem crops up with 3.5" drives.
14533
14534     cmp     byte [step_drv], 0FFh ; -1
14535     ;jz      short do_writej
14536     ; 23/12/2023
14537     jz      short do_write
14538     cmp     ah, 2 ; romread
14539     jz      short do_fast
14540     cmp     ah, 4 ; romverify
14541     ;jz      short do_fast
14542     ; 23/12/2023
14543     jnz     short do_write
14544 do_writej:
14545     ; reads always fast, unless we have just done a disk reset operation
14546
14547     ;jmp     short do_write ; reads always fast
14548
14549     ; -----
14550 do_fast:
14551     call    fastspeed ; change settle mode
14552     testerr:
14553     jb      short dskerr_brdg
14554
14555     ; 23/12/2023 Retro DOS v5.0
14556     ; (PCDOS 7.1 IBMBIO.COM)
14557     cmp     bp, 5 ; is there retry ?
14558     jnz     short testerror ; yes
14559     cmp     ah, 0BBh ; Undefined error (hard disk)
14560     jz      short dskerr_brdg
14561 testerror:
14562     ; set drive and track of last access
14563
14564     mov     [step_drv], dl
14565     ; 23/12/2023
14566     mov     [es:di+78h], ch
14567     ;mov     [es:di+46h], ch ; [es:di+BDS.track]
14568 no_set:
14569     ;cmp     word [wrtverify], 103h
14570     cmp     word [rflag], 103h ; check for write and verify
14571     jz      short doverify
14572 noverify:
14573     pop     ax
14574
14575     ; check the flags word in the bds to see if the drive is non removable
14576     ; if not we needn't do anything special
14577     ; if it is a hard disk then check to see if multi-track operation
14578     ; is specified. if specified we don't have to calculate for the next
14579     ; track since we are already done. so we can go to the exit of this routine.
14580
14581     ; 23/12/2023
14582     test     byte [es:di+3Fh], 1
14583     ;test     byte [es:di+23h], 1 ; [es:di+BDS.flags]
14584     ; ; fnon_removable
14585     jz      short its_removable
14586     test     byte [multtrk_flag], 80h ; multtrk_on
14587     jnz     short disk_ret
14588 its_removable:
14589     and     cl, 3Fh ; eliminate cylinder bits from sector
14590     xor     ah, ah
14591     sub     [secnt], ax ; reduce count of sectors to go next sector
14592     add     cl, al
14593     mov     [cursec], cl
14594     cmp     cl, [es:di+13h] ; [es:di+BDS.secpertack]
14595     ; ; see if sector/track limit reached
14596     jbe     short disk_ret
14597     mov     byte [cursec], 1 ; start with first sector of next track
14598     mov     dh, [curhd]
14599     inc     dh
14600     cmp     dh, [es:di+15h] ; [es:di+BDS.heads]
14601     jb      short noxor
14602     xor     dh, dh
14603     inc     word [curtrk]
14604 noxor:
14605     mov     [curhd], dh
14606 disk_ret:
14607     cld
14608     ret
14609
14610     ; -----
14611     ; 15/10/2022
14612
14613     ; 24/12/2023 - Retro DOS v5.0
14614
14615     ; -----
14616     ; the request is for write. determine if we are talking about
14617     ; the same track and drive. if so, use the fast speed.
14618     ; -----
14619 do_write:
14620     cmp     dl, [step_drv]
14621     jnz     short do_norm ; we have changed drives
14622     ; 24/12/2023
14623     cmp     ch, [es:di+78h]
14624     ;cmp     ch, [es:di+46h] ; [es:di+BDS.track]
14625     jz      short do_fast ; we are still on the same track
14626 do_norm:
14627     call    normspeed
14628     jmp     short testerr
14629
14630     ; -----
14631     ; we have a verify request also. get state info and go verify
14632     ; -----
14633 doverify:
14634     pop     ax
14635
14636
14637
14638
14639     0000D05 58

```



```

14640 0000D06 50          push    ax
14641 0000D07 B404        mov     ah, 4
14642 0000D09 E89000    call    fastspeed
14643 0000D0C 73A5        jnb     short noverify
14644
14645 ; check the error returned in ah to see if it is a soft ecc error.
14646 ; if it is not we needn't do anything special. if it is a soft
14647 ; ecc error then decrement the soft_ecc_cnt error retry count. if
14648 ; this retry count becomes 0 then we just ignore the error and go to
14649 ; no_verify but if we can still try then we call the routine to reset
14650 ; the disk and go to dskerr1 to retry the operation.
14651
14652 0000D0E 80FC11      cmp     ah, 11h ; soft ecc error ?
14653 0000D11 750B        jnz     short not_softecc_err
14654 0000D13 FF0E[A504] dec     word [soft_ecc_cnt]
14655 0000D17 749A        jz      short noverify ; no more retry
14656 0000D19 E81F06      call    ResetDisk ; reset disk
14657 0000D1C EB3E        jmp     short dskerr1 ; retry
14658
14659 ; -----
14660 not_softecc_err: ; other error.
14661 0000D1E E81A06      call    ResetDisk
14662 0000D21 FF0E[A304] dec     word [vretry_cnt]
14663 0000D25 EB1C        jmp     short dskerr0
14664
14665 ; -----
14666 ; need to special case the change-line error ah=06h.
14667 ; if we get this, we need to return it.
14668 ; -----
14669
14670
14671 dskerr:
14672 0000D27 803E[7700]00 cmp     byte [fhave96], 0 ; do we have changeline support?
14673 0000D2C 7403        jz      short dskerr_nochangeline ; brief not
14674 0000D2E E8BE0E      call    checkio
14675 dskerr_nochangeline:
14676 0000D31 803E[A704]01 cmp     byte [multitrk_format_flag], 1 ; multi trk format request?
14677 0000D36 7508        jnz     short dochkagain ; no more retry.
14678 0000D38 BD0100    mov     bp, 1
14679 0000D3B C606[A704]00 mov     byte [multitrk_format_flag], 0 ; clear the flag.
14680 dochkagain:
14681 0000D40 E89E00      call    again
14682 dskerr0:
14683 0000D43 7420        jz      short harderr
14684 ; 24/12/2023
14685 0000D45 26F6453F01 test    byte [es:di+3Fh], 1
14686 ; test byte [es:di+23h], 1 ; [es:di+BDS.flags]
14687 ; fnon_removable
14688 0000D4A 7505        jnz     short skip_timeout_chk
14689 0000D4C 80FC80      cmp     ah, 80h ; timeout?
14690 0000D4F 7414        jz      short harderr
14691 skip_timeout_chk:
14692 0000D51 80FCCC      cmp     ah, 0CCh ; write fault error?
14693 0000D54 740A        jz      short write_fault_err ; then, don't retry.
14694 0000D56 C706[A504]0500 mov     word [soft_ecc_cnt], 5 ; MAXERR
14695 ; set soft_ecc_cnt back to maxerr
14696 dskerr1:
14697 0000D5C 58          pop     ax ; restore sector count
14698 ; jmp retry
14699 ; 09/12/2022
14700 0000D5D E9F1FE      jmp     _retry
14701
14702 ; -----
14703 write_fault_err:
14704 0000D60 BD0100    mov     bp, 1 ; just retry only once
14705 ; for write fault error.
14706 0000D63 EBF7        jmp     short dskerr1
14707 ; fall into harderr
14708
14709 ; -----
14710 ; entry point for routines that call maperror themselves
14711
14712
14713 harderr:
14714 0000D65 E84100      call    maperror
14715 harderr2:
14716 0000D68 C606[1E01]FF mov     byte [tim_drv], 0FFh
14717 ; force a media check through rom
14718 0000D6D 8B0E[2201] mov     cx, [seccnt] ; get count of sectors to go
14719 0000D71 8B26[3501] mov     sp, [spsav] ; recover entry stack pointer
14720
14721 ; since we are performing a non-local goto, restore the disk parameters
14722 ; jmp diddleback
14723 ; 09/12/2022
14724 0000D75 E951FE      jmp     diddle_back
14725
14726 ; ===== S U B R O U T I N E =====
14727
14728 ; change settle value from settlecurrent to whatever is appropriate
14729 ; note that this routine is never called for a fixed disk.
14730
14731 ; 19/10/2022
14732
14733 normspeed:
14734 0000D78 803E[A905]00 cmp     byte [media_set_for_format], 0
14735 0000D7D 751D        jnz     short fastspeed
14736 0000D7F 06          push    es
14737 0000D80 50          push    ax
14738 0000D81 A0[2801]    mov     al, [settle_low]
14739 0000D84 C436[2D01] les     si, [dpt] ; current disk parm table
14740 0000D88 26884409    mov     [es:si+9], al ; [es:si+DISK_PARMS.DISK_HEAD_STTL]
14741 0000D8C 58          pop     ax
14742 0000D8D 07          pop     es
14743 0000D8E E80B00      call    fastspeed
14744 ; 24/12/2023
14745 ; push es
14746 ; les si, [dpt]
14747 ; mov byte [es:si+9], 1 ; [es:si+DISK_PARMS.DISK_HEAD_STTL]
14748 ; ; 1 is fast settle value
14749 ; pop es
14750 0000D91 1E          push    ds
14751 0000D92 C536[2D01] lds     si, [dpt]
14752 0000D96 C6440901    mov     byte [si+9], 1
14753 0000D9A 1F          pop     ds
14754
14755 0000D9B C3          retn
14756
14757 ; ===== S U B R O U T I N E =====
14758
14759 ; if the drive has been marked as too big (i.e. starting sector of the
14760 ; partition is > 16 bits, then always return drive not ready.
14761
14762 ; 24/12/2023 - Retro DOS v5.0
14763 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0DDdh)

```

```

14764 fastspeed:
14765 ;;test byte [es:di+3Bh], 80h ; [es:di+BDS.fatsiz]
14766 ;;test byte [es:di+1Fh], 80h ; [es:di+BDS.fatsiz]
14767 ;
14768 ;jnz short notready
14769 push es
14770 mov es, [xfer_seg]
14771 int 13h ; DISK -
14772 mov [xfer_seg], es
14773 pop es
14774 ret
14775 ; -----
14776 ; ; 24/12/2023
14777 ;notready:
14778 ;stc
14779 ;mov ah, 80h
14780 ;ret
14781
14782 ; ===== S U B R O U T I N E =====
14783
14784 ; map error returned by rom in ah into corresponding code to be returned to
14785 ; dos in al. trashes di. guaranteed to set carry.
14786
14787 maperror:
14788 push cx
14789 push es
14790 push ds ; set es=Bios_Data
14791 pop es
14792 mov al, ah ; put error code in al
14793 mov [lsterr], al ; terminate list with error code
14794 ; 24/12/2023
14795 mov cx, 11 ; PCDOS 7.1 ; 02/09/2023
14796 ;mov cx, 9 ; numerr (= errout-errin)
14797 ; ; number of possible error conditions
14798 mov di, errin ; point to error conditions
14799 repne scasb
14800
14801 ; 24/12/2023
14802 ; 02/09/2023
14803 mov al, [di+10] ; PCDOS 7.1 IBMBIO.COM
14804 ; 10/12/2022
14805 ;mov al, [di+8] ; [di+numerr-1]
14806 ; ; get translation
14807 ; 19/10/2022 - Temporary !
14808 ;db 8Ah, 85h, 8, 0 ; mov al, [di+8]
14809 pop es
14810 pop cx
14811 stc ; flag error condition
14812 ret
14813
14814 ; ===== S U B R O U T I N E =====
14815
14816 ; set the time of last access for this drive.
14817 ; this is done only for removable media. es:di -> bds
14818
14819 set_tim:
14820 push ax
14821 call GetTickCnt ; Does INT 1A ah=0 & updates daycnt
14822
14823 ; we have the new time. if we see that the time has passed,
14824 ; then we reset the threshold counter...
14825
14826 ; 24/12/2023 - Retro DOS v5.0
14827 cmp dx, [es:di+79h] ; PCDOS 7.1 IBMBIO.COM
14828 cmp dx, [es:di+47h] ; [es:di+BDS.tim_lo]
14829 jne short setaccess
14830 ; 24/12/2023
14831 cmp cx, [es:di+7Bh] ; PCDOS 7.1 IBMBIO.COM
14832 cmp cx, [es:di+49h] ; [es:di+BDS.tim_hi]
14833 jz short done_set
14834 ; 12/12/2022
14835 je short done_set2
14836
14837 setaccess:
14838 mov byte [accesscount], 0
14839
14840 ; 24/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
14841 mov [es:di+79h], dx
14842 mov [es:di+7Bh], cx
14843 ;mov [es:di+47h], dx ; [es:di+BDS.tim_lo]
14844 ;mov [es:di+49h], cx ; [es:di+BDS.tim_hi]
14845
14846 done_set:
14847 cll
14848
14849 done_set2:
14850 ; 12/12/2022
14851 pop ax
14852 ret
14853
14854 ; ===== S U B R O U T I N E =====
14855
14856 ; this routine is called if an error occurs while formatting or verifying.
14857 ; it resets the drive, and decrements the retry count.
14858 ; on entry - ds:di - points to bds for the drive
14859 ; bp - contains retry count
14860 ; on exit flags indicate result of decrementing retry count
14861
14862 again:
14863 call ResetDisk
14864 cmp ah, 6
14865 jz short dont_dec_retry_count ; If it is a media change error
14866 ; do not decrement retry count.
14867 dec bp ; decrement retry count
14868 ret
14869 ; -----
14870
14871 dont_dec_retry_count:
14872 or ah, ah
14873 ret
14874
14875 ; -----
14876 ; Retro DOS v5.0 - PCDOS 7.1 IBMBIO.COM - BIOSCODE:0E30h
14877 ; -----
14878 ; 24/12/2023 - Retro DOS v5.0
14879 ;;;;
14880
14881 ioctl_drvnum: db 0
14882
14883 ; 24/12/2023
14884
14885 ; ===== S U B R O U T I N E =====
14886
14887 get_phy_drv_num:
14888 call SetDrive ; get physical drive number
14889 ; INPUT: al = logical drive number (BDS.drivelet)
14890 ; OUTPUT: physical drive number (BDS.drivenum)
14891 mov dl, [es:di+4] ; [es:di+BDS.drivenum]

```

```

14888 00000DF6 C3                                retn
14889
14890 ; ===== S U B R O U T I N E =====
14891
14892 ; 24/12/2023
14893 ioctl_output:
14894 00000DF7 E8F5FF      call    get_phy_drv_num
14895 00000DFA 2E8816[EE0D]  mov     [cs:ioctl_drvnum], dl
14896 00000DFF B441      mov     ah, 41h
14897 00000E01 B8AA55      mov     bx, 55AAh
14898 00000E04 CD13      int      13h
14899                                ; DISK - Check for INT 13h Extensions
14900                                ; BX = 55AAh, DL = drive number
14901                                ; Return: CF set if not supported
14902                                ; AH = extensions version
14903                                ; BX = AA55h
14904                                ; CX = Interface support bit map
14904 00000E06 7235      jc      short int13h_exts_err
14905 ioctl_input_1:
14906 00000E08 C43E[1200]  les     di, [ptrsav]
14907 00000E0C 26C47D0E  les     di, [es:di+14] ; [es:di+IOCTL_REQ.MINORFUNCTION]
14908 00000E10 723E      jc      short ioctl_input_2
14909 00000E12 B80046      mov     ax, 4600h      ; Eject removable media
14910 00000E15 263805      cmp     [es:di], al      ; al = 0 ; disk ioctl function = 0
14911 00000E18 7417      je      short ioctl_output_1
14912 00000E1A 26803D01  cmp     byte [es:di], 1 ; al = 1 ; disk ioctl function = 1
14913 00000E1E 751B      jne     short ioctl_output_2
14914 00000E20 B80145      mov     ax, 4501h      ; Lock/unlock media
14915                                ; (al, 0 = lock, 1 = unlock)
14916 00000E23 26807D0100  cmp     byte [es:di+1], 0 ; unlock (reverse of INT 13h ah=45h)
14917 00000E28 7407      jz      short ioctl_output_1
14918 00000E2A 26384501  cmp     [es:di+1], al      ; lock (reverse of INT 13h ah=45h)
14919 00000E2E 750B      jne     short ioctl_output_2
14920 00000E30 48      dec     ax
14921 ioctl_output_1:
14922 00000E31 2E8A16[EE0D]  mov     dl, [cs:ioctl_drvnum]
14923 00000E36 CD13      int      13h
14924                                ; DISK - IBM/MS Extension - LOCK/UNLOCK DRIVE (DL - drive, [SI - disk address
packet)
14924 00000E38 7203      jc      short int13h_exts_err
14925 ioctl_lock_err:
14926                                ; cf=0
14927 ioctl_output_ret:
14928                                ; cld
14929                                retn
14930 ; -----
14931
14932 ioctl_output_2:
14933 00000E3B B401      mov     ah, 1
14934 int13h_exts_err:
14935 00000E3D 80FCB0      cmp     ah, 0B0h      ; volume not locked in drive
14936 00000E40 74F8      je      short ioctl_lock_err
14937 00000E42 80FCB4      cmp     ah, 0B4h      ; lock count exceeded
14938 00000E45 74F3      je      short ioctl_lock_err
14939 00000E47 E9DAF7      jmp     err_exitj
14940
14941 ; ===== S U B R O U T I N E =====
14942
14943 ; 24/12/2023
14944 ioctl_input:
14945 00000E4A E8A2FF      call    get_phy_drv_num
14946 00000E4D F9      stc
14947 00000E4E EBB8      jmp     short ioctl_input_1
14948 ioctl_input_2:
14949 00000E50 26803D06  cmp     byte [es:di], 6      ; disk ioctl function = 6
14950 00000E54 75E5      jne     short ioctl_output_2
14951 00000E56 B80245      mov     ax, 4502h      ; get lock status
14952 00000E59 CD13      int      13h
14953                                ; DISK - IBM/MS Extension - LOCK/UNLOCK DRIVE (DL - drive, [SI - disk address
packet)
14953 00000E5B 72E0      jc      short int13h_exts_err
14954 00000E5D B80C00      mov     bx, 0Ch      ; bit 1 lock bit
14955 00000E60 3C00      cmp     al, 0      ; not locked
14956 00000E62 7402      jz      short ioctl_input_3
14957 00000E64 B30E      mov     bl, 0Eh
14958 ioctl_input_3:
14959 00000E66 53      push    bx
14960 00000E67 B404      mov     ah, 4
14961 00000E69 B90101      mov     cx, 101h
14962 00000E6C B601      mov     dh, 1
14963 00000E6E CD13      int      13h
14964                                ; DISK - VERIFY SECTORS
14965                                ; AL = number of sectors to verify, CH = track, CL = sector
14966                                ; DH = head, DL = drive
14967                                ; Return: CF set on error, AH = status
14968                                ; AL = number of sectors verified
14968 00000E70 5B      pop     bx
14969 00000E71 80FC31      cmp     ah, 31h      ; no media in drive (IBM/MS INT 13 extensions)
14970 00000E74 740B      je      short ioctl_input_5
14971 00000E76 80FC80      cmp     ah, 80h      ; timeout (not ready)
14972 00000E79 7406      je      short ioctl_input_5
14973 ioctl_input_4:
14974 00000E7B 26895D01  mov     [es:di+1], bx
14975 00000E7F EBB9      jmp     short ioctl_lock_err
14976 ioctl_input_5:
14977 00000E81 81CB0108  or      bx, 801h      ; bit 0 error bit (1 = error, 31h or 80h)
14978                                ; bit 11 (not ready -removable media error- bit)
14979                                ; if bit 11 = 0, another error (except 31h and 80h)
14980 00000E85 EBF4      jmp     short ioctl_input_4
14981
14982 ; -----
14983 ;;;;
14984
14985 ; 16/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
14986
14987 ; -----
14988 ; MSDIOCTL.ASM - MSDOS 6.0 - 1991
14989 ; -----
14990 ; 11/03/2019 - Retro DOS v4.0
14991
14992 ; 18/03/2019
14993
14994 ; =====
14995 ;
14996 ; NOTE: GetAccessFlag/SetAccessFlag is unpublished function.
14997 ;
14998 ; This function is intended to give the user to control the
14999 ; bds table flags of unformatted_media bit.
15000 ; GetAccessFlag will show the status -
15001 ; a_DiskAccess_Control.dac_access_flag = 0 disk i/o not allowed
15002 ; 1 disk i/o allowed
15003 ; SetAccessFlag will set/reset the unformatted_media bit in flags -
15004 ; a_DiskAccess_Control.dac_access_flag = 0 allow disk i/o
15005 ; 1 disallow disk i/o
15006 ; =====
15007
15008 ; generic ioctl dispatch tables
15009

```

```

15010 ; BIOSCODE:0C3Ch (MSDOS 6.21, IO.SYS)
15011
15012 ; 24/12/2023
15013 ; BIOSCODE:0ECAh (PCDOS 7.1, IBMBIO.COM)
15014
15015 ; -----
15016 ; 24/12/2023
15017 ;db 0
15018 ; 09/12/2022
15019 %if 0
15020
15021 IoReadJumpTable: db 8 ; ((IowriteJumpTable-IoReadJumpTable)-1)/2
15022 dw 0CA7h ; 60h ; GetDeviceParameters
15023 dw 0EE8h ; 61h ; ReadTrack
15024 dw 0E86h ; 62h ; VerifyTrack
15025 dw 0CA3h ; Cmd_Error_Proc
15026 dw 0CA3h ; Cmd_Error_Proc
15027 dw 0CA3h ; Cmd_Error_Proc
15028 dw 119Ah ; 66h ; GetMediaId
15029 dw 1269h ; 67h ; GetAccessFlag ; unpublished function
15030 dw 12C1h ; 68h ; SenseMediaType
15031
15032 IowriteJumpTable: db 7 ; ((IOC_DC_Table-IowriteJumpTable)-1)/2
15033 dw 0CF3h ; 40h ; SetDeviceParameters
15034 dw 0EEFh ; 41h ; WriteTrack
15035 dw 0DC1h ; 42h ; FormatTrack
15036 dw 0CA3h ; Cmd_Error_Proc
15037 dw 0CA3h ; Cmd_Error_Proc
15038 dw 0CA3h ; Cmd_Error_Proc
15039 dw 11D2h ; 46h ; SetMediaId
15040 dw 1280h ; 47h ; SetAccessFlag ; unpublished function
15041
15042 %endif
15043 ; 24/12/2023 - Retro DOS v5.0
15044 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0ECAh)
15045
15046 ; 09/12/2022
15047 IoReadJumpTable:
15048 db ((IowriteJumpTable-IoReadJumpTable)-1)/2 ; 15
15049 dw GetDeviceParameters ; 60h
15050 dw ReadTrack ; 61h
15051 dw VerifyTrack ; 62h
15052 dw Cmd_Error_Proc
15053 dw Cmd_Error_Proc
15054 dw Cmd_Error_Proc
15055 dw GetMediaId ; 66h
15056 dw GetAccessFlag ; 67h ; unpublished function
15057 dw SenseMediaType ; 68h
15058 ; 24/12/2023
15059 ; Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
15060 dw Cmd_Error_Proc ; 69h
15061 dw Cmd_Error_Proc ; 6Ah
15062 dw Cmd_Error_Proc
15063 dw Cmd_Error_Proc
15064 dw Cmd_Error_Proc
15065 dw Cmd_Error_Proc ; 6Eh
15066 dw Cmd_Error_Proc ; 6Fh
15067
15068 IowriteJumpTable:
15069 db ((IOC_DC_Table-IowriteJumpTable)-1)/2 ; 9
15070 dw SetDeviceParameters ; 40h
15071 dw WriteTrack ; 41h
15072 dw FormatTrack ; 42h
15073 dw Cmd_Error_Proc
15074 dw Cmd_Error_Proc
15075 dw Cmd_Error_Proc
15076 dw SetMediaId ; 46h
15077 dw SetAccessFlag ; 47h ; unpublished function
15078 ; 24/12/2023
15079 ; Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
15080 dw SetLockState ; 48h
15081 dw EjectMedia ; 49h
15082
15083 ; =====
15084 ; IOC_DC_Table
15085 ;
15086 ; This table contains all of the valid generic IOCTL Minor codes for
15087 ; major function 08 to be used by the IOCTL_Support_Query function.
15088 ; Added for 5.00
15089 ; =====
15090
15091 ; 24/12/2023 - Retro DOS v5.0
15092 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0F00h)
15093
15094 IOC_DC_Table: db 60h ; GET_DEVICE_PARAMETERS
15095 db 40h ; SET_DEVICE_PARAMETERS
15096 db 61h ; READ_TRACK
15097 db 41h ; WRITE_TRACK
15098 db 62h ; VERIFY_TRACK
15099 db 42h ; FORMAT_TRACK
15100 db 66h ; GET_MEDIA_ID
15101 db 46h ; SET_MEDIA_ID
15102 db 67h ; GET_ACCESS_FLAG
15103 db 47h ; SET_ACCESS_FLAG
15104 db 68h ; SENSE_MEDIA_TYPE
15105 ; 24/12/2023
15106 ; Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
15107 db 48h ; SET_LOCK_STATE
15108 db 49h ; EJECT_MEDIA
15109 db 6Fh ; GET_DRV_MAP_INFO
15110
15111 ;IOC_DC_TABLE_LEN EQU $ - IOC_DC_Table
15112
15113 ; 24/12/2023 - Retro DOS v5.0
15114 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0F0Eh)
15115
15116 new_genioctl: db 0
15117
15118 ; -----
15119
15120 ; 16/10/2022
15121
15122 ; =====
15123 ; Do_Generic_IOCTL: perform generic ioctl request
15124 ;
15125 ; input: AL contains logical drive
15126 ;
15127 ; functions are dispatched through a call. On return, carry indicates
15128 ; error code in al. Note::BES:b& ds undefined on return from
15129 ; subfunctions.
15130 ;
15131 ; =====
15132
15133 ; 11/03/2019

```

```

15134 ; 24/12/2023 - Retro DOS v5.0
15135 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0F0Fh)
15136
15137 ; 19/10/2022
15138 do_generic_ioctl: ; 2C7h:0C6Bh = 70h:31DBh
15139 00000ECC E8D5F6 call SetDrive ; ES:DI Points to bds for drive
15140
15141 ; 24/12/2023
15142 ;;;
15143 00000ECF 2EC606[CB0E]00 mov byte [cs:new_genioctl], 0
15144 ;;; ; 0, old generic ioctl function
15145 00000ED5 06 push es
15146 00000ED6 C41E[1200] les bx, [ptrsav] ; ES:BX Points to request header
15147 00000EDA 26807F0D08 cmp byte [es:bx+0Dh], 8 ; [es:bx+IOCTL_REQ.MAJORFUNCTION]
15148 ; ; RAWIO
15149 ; 24/12/2023
15150 ;mov al, [es:bx+0Eh] ; [es:bx+IOCTL_REQ.MINORFUNCTION]
15151 ;pop es
15152 ;jnz short IoctlFuncErr
15153 00000EDF 740A jz short chk_genioctl_minor
15154 00000EE1 2EFE06[CB0E] inc byte [cs:new_genioctl]
15155 ; ; 1, new generic ioctl function (FAT32)
15156 00000EE6 26807F0D48 cmp byte [es:bx+0Dh], 48h ; Generic IOCTL Request support
15157 ; ; (called only if bit 6 of attribute is set to 1)
15158 chk_genioctl_minor:
15159 00000EEB 268A470E mov al, [es:bx+0Eh] ; [es:bx+IOCTL_REQ.MINORFUNCTION]
15160 00000EEF 07 pop es
15161 00000EF0 7525 jnz short IoctlFuncErr
15162 ;;;
15163 ; cas note: Could do the above two blocks in reverse order.
15164 ; would have to preserve al for SetDrive
15165
15166 ; 10/12/2022
15167 00000EF2 BE[870E] mov si, IoReadJumpTable
15168 ;mov si, 0C3Ch ; IoReadJumpTable
15169 ; ; at 2C7h:0C3Ch = 70h:31ACh
15170 00000EF5 A820 test al, 20h ; GEN_IOCTL_FN_TST ; test of req. function
15171 00000EF7 7503 jnz short NotGenericwrite ; function is a read.
15172 ; 10/12/2022
15173 00000EF9 BE[A80E] mov si, IoWriteJumpTable
15174 ;mov si, 0C4Fh ; IoWriteJumpTable
15175 ; ; at 2C7h:0C4Fh = 70h:31BFh
15176 NotGenericwrite:
15177 and al, 0DFh ; ~GEN_IOCTL_FN_TST ; get rid of read/write bit
15178 00000EFC 24DF sub al, 40h ; offset for base function
15179 00000EFE 2C40 cmp al, [cs:si]
15180 00000F00 2E3A04 ja short IoctlFuncErr
15181 00000F03 7712 cbw
15182 00000F05 98 ; 24/12/2023
15183 ;shl ax, 1
15184 add ax, ax
15185 00000F06 01C0 inc si
15186 00000F08 46 add si, ax
15187 00000F09 01C6 call near [cs:si]
15188 00000F0B 2EFF14 call word ptr cs:[si]
15189 ;mov ds, [cs:BIOSDATAWORD]
15190 00000F0E 2E8E1E[3000] mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
15191 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
15192 ; 2C7h:30h = 70h:25A0h
15193 00000F13 B481 mov ah, 81h ; Return this status in case of carry
15194 00000F15 C3 retn ; Pass carry flag through to exit code
15195 ; -----
15196 ; Cmd_Error_Proc is called as a procedure and also use
15197 ; as a fall through from above
15198 Cmd_Error_Proc: ; 2C7h:0CA3h = 70h:3213h
15199 pop dx
15200 00000F16 5A IoctlFuncErr:
15201 jmp bc_cmderr
15202 00000F17 E9BBF1 ; -----
15203
15204 ; 16/10/2022
15205
15206 ; =====
15207 ; ** GetDeviceParameters:
15208 ;
15209 ; GetDeviceParameters implements the generic ioctl function:
15210 ; majorcode=RAWIO, minorcode=GetDeviceParameters (60h)
15211 ;
15212 ; ENTRY (ES:di) = BDS for drive
15213 ; PtrSav = long pointer to request header
15214 ; EXIT ??? BUGBUG
15215 ; USES ??? BUGBUG
15216 ; =====
15217
15218 ; 24/12/2023 - Retro DOS v5.0
15219 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0F5Dh)
15220
15221 ; 19/10/2022
15222 GetDeviceParameters:
15223 ; Copy info from bds to the device parameters packet
15224
15225 00000F1A C51E[1200] lds bx, [ptrsav] ; DS:BX points to request header
15226 00000F1E C5F13 lds bx, [bx+19] ; [bx+IOCTL_REQ.GENERIC_IOCTL_PACKET]
15227 ; ; (DS:BX) = return buffer
15228 ; 24/12/2023
15229 mov al, [es:di+3Eh]
15230 ;mov al, [es:di+34] ; [es:di+BDS.formfactor]
15231 mov [bx+1], al ; [bx+A_DEVICEPARAMETERS.DP_DEVICE_TYPE]
15232 00000F21 268A453E ; 24/12/2023
15233 mov ax, [es:di+3Fh]
15234 00000F28 268B453F mov ax, [es:di+35] ; [es:di+BDS.flags]
15235 and ax, 3 ; fnon_removable+fchangeline
15236 00000F2C 83E003 ; Mask off other bits
15237 mov [bx+2], ax ; [bx+A_DEVICEPARAMETERS.DP_DEVICE_ATTRIBUTES]
15238 00000F2F 894702 ; 24/12/2023
15239 mov ax, [es:di+41h]
15240 00000F32 268B4541 mov ax, [es:di+37] ; [es:di+BDS.cylinders]
15241 ;mov [bx+4], ax ; [bx+A_DEVICEPARAMETERS.DP_CYLINDERS]
15242 00000F36 894704 xor al, al ; Set media type to default
15243 00000F39 30C0 mov [bx+6], al ; [bx+A_DEVICEPARAMETERS.DP_MEDIATYPE]
15244 00000F3B 884706
15245 ; copy recommended bpb
15246 ; 24/12/2023
15247 lea si, [di+43h]
15248 ;lea si, [di+39] ; [di+BDS.rbytespersec] = [di+BDS.R_BPB]
15249 00000F3E 8D7543 test byte [bx], 1 ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
15250 ; ; BUILD_DEVICE_BPB
15251 00000F41 F60701 jz short UseBpbPresent
15252 ; Save request packet segment
15253 00000F44 7412 ds
15254 00000F46 1E mov ds, [cs:BIOSDATAWORD]
15255 00000F47 2E8E1E[3000] ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
15256 ; 2C7h:30h = 70h:25A0h
15257

```

```

15258                                     ; Point back to Bios_Data
15259 00000F4C E828FA      call    checksingle
15260 00000F4F E884F7      call    GetBp      ; Build the bpb from scratch
15261 00000F52 1F          pop     ds      ; Restore request packet segment
15262 00000F53 7224        jb     short GetParmRet
15263 00000F55 8D7506      lea     si, [di+6]      ; [di+BDS.bytespersec] = [di+BDS.DP_BPB]
15264                                     ; Use this subfield of bds instead
15265
15266 00000F58 8D7F07      UseBpbPresent: lea     di, [bx+7]      ; [bx+A_DEVICEPARAMETERS.DP_BPB]
15267                                     ; This is where the result goes
15268                                     ; 24/12/2023
15269 00000F5B 31D2        xor     dx, dx ; 0
15270
15271                                     ; 24/12/2023
15272 00000F5D B91F00      mov     cx, 31      ; A_BPB.size = 31
15273                                     ;mov    cx, 25      ; A_BPB.size - 6
15274                                     ; 24/12/2023      ; For now use 'small' bpb
15275                                     ;;;
15276                                     ;;;
15277 00000F60 2E3816[CB0E] cmp     [cs:new_genioct1], dl ; 0 ? ; *
15278 00000F65 7404        jz     short gdp_1      ; old type (FAT12 & FAT16) structure
15279                                     ;mov    cx, 53      ; FAT32 BPB size
15280                                     ;mov    dx, 32      ; 53+32 = 85 bytes (A_BPB_FAT32.size)
15281 00000F67 B135        mov     cl, 53
15282 00000F69 B220        mov     dl, 32
15283
15284 gdp_1:                                     ;;;
15285 00000F6B 1E          push    ds      ; reverse segments for copy
15286 00000F6C 06          push    es
15287 00000F6D 1F          pop     ds
15288 00000F6E 07          pop     es
15289 00000F6F F3A4        rep movsb
15290
15291                                     ; 24/12/2023
15292                                     ;;;
15293 00000F71 89D1        mov     cx, dx      ; 0 or 32
15294 00000F73 E304        jcxz    gdp_2
15295 00000F75 30C0        xor     al, al      ; 32 zeros
15296 00000F77 F3AA        rep stosb
15297
15298 gdp_2:                                     ;clc    ; cf is already 0 ; * ; 24/12/2023
15299                                     ;;;
15300                                     ; 12/12/2022
15301                                     ; cf=0 (cmp instruction -above- resets cf)
15302                                     ;clc
15303
15304 GetParmRet:
15305 00000F79 C3          retn
15306
15307 ; -----
15308 ; 17/10/2022
15309 ; 16/10/2022
15310
15311 ; =====
15312 ; SetDeviceParameters:
15313 ;
15314 ; input: ES:di points to bds for drive
15315 ; =====
15316
15317                                     ; 24/12/2023 - Retro DOS v5.0
15318                                     ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:0FC0h)
15319
15320                                     ; 19/10/2022
15321 SetDeviceParameters:                                     ; 2C7h:0CF3h = 70h:3263h
15322 00000F7A C51E[1200] lds     bx, [ptrsav] ; DS:BX points to request header
15323 00000F7E C55F13      lds     bx, [bx+19] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
15324                                     ; 24/12/2023
15325 00000F81 26814D3F4001 or     word [es:di+3Fh], 140h
15326                                     ;or     word [es:di+23h], 140h ; [es:di+BDS.flags]
15327                                     ; fchanged_by_format|fchanged
15328 00000F87 F60702      test    byte [bx], 2 ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
15329                                     ; ONLY_SET_TRACKLAYOUT
15330                                     ;jnz    short setTrackTable
15331                                     ; 24/12/2023
15332 00000F8A 7403        jz     short sdp_1
15333 00000F8C E98000      jmp     setTrackTable
15334
15335 sdp_1: mov     al, [bx+1] ; [bx+A_DEVICEPARAMETERS.DP_DEVICEYPE]
15336                                     ; 24/12/2023
15337 00000F92 2688453E      mov     [es:di+3Eh], al
15338                                     ;mov    [es:di+34], al ; [es:di+BDS.formfactor]
15339 00000F96 8B4704      mov     ax, [bx+4] ; [bx+A_DEVICEPARAMETERS.DP_CYLINDERS]
15340                                     ; 24/12/2023
15341 00000F99 26894541      mov     [es:di+41h], ax
15342                                     ;mov    [es:di+37], ax ; [es:di+BDS.cylinders]
15343 00000F9D 8B4702      mov     ax, [bx+2] ; [bx+A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES]
15344 00000FA0 1E          push    ds
15345                                     ; 17/10/2022
15346 00000FA1 2E8E1E[3000] mov     ds, [cs:BIOSDATAWORD]
15347                                     ;mov    ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_word]
15348                                     ; 2C7h:30h = 70h:25A0h
15349                                     ;cmp    byte [fhave96], 0
15350 00000FA6 803E[7700]00 cmp     byte [fhave96], 0
15351 00000FAB 1F          pop     ds
15352 00000FAC 7502        jnz     short HaveChange ; we have changeline support
15353                                     ; 10/12/2022
15354 00000FAE 24FD        and     al, 0FDh
15355                                     ;and    ax, 0FFFDh ; ~fchangeline
15356
15357                                     ; Ignore all bits except non_removable and changeline
15358 HaveChange:
15359 00000FB0 83E003      and     ax, 3 ; fnon_removable|fchangeline
15360                                     ; 24/12/2023
15361 00000FB3 268B4D3F      mov     cx, [es:di+3Fh]
15362                                     ;mov    cx, [es:di+35] ; [es:di+BDS.flags]
15363 00000FB7 81E1F4FD      and     cx, 0FDF4h ; ~(fnon_removable|fchangeline|good_tracklayout|unformatted_media)
15364 00000FBB 09C8        or     ax, cx
15365                                     ; 24/12/2023
15366 00000FBD 2689453F      mov     [es:di+3Fh], ax
15367                                     ;mov    [es:di+35], ax ; [es:di+BDS.flags]
15368 00000FC1 8A4706      mov     al, [bx+6] ; [bx+A_DEVICEPARAMETERS.DP_MEDIATYPE]
15369                                     ; Set media type
15370 00000FC4 1E          push    ds
15371 00000FC5 2E8E1E[3000] mov     ds, [cs:BIOSDATAWORD]
15372                                     ;mov    ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_word]
15373 00000FCA A2[A805]      mov     [mediatype], al
15374                                     ;mov    ds:mediatype, al
15375
15376                                     ; 24/12/2023
15377                                     ;;;
15378 00000FCD B93500      mov     cx, 53 ; FAT32 BPB size
15379 00000FD0 2E803E[CB0E]00 cmp     byte [cs:new_genioct1], 0
15380 00000FD6 7502        jnz     short sdp_2 ; new type (FAT32) structure
15381                                     ;mov    cx, 31 ; A_BPB.size = 31

```



```

15382 00000FD8 B11F      mov     cl, 31
15383
15384
15385 00000FDA 1F      sdp_2:  pop     ds
15386
15387                ; The media changed (maybe) so we will have to do a set dasd
15388                ; the next time we format a track
15389
15390                ; 24/12/2023
15391 00000FDB 26804D3F80    or      byte [es:di+3Fh], 80h
15392                ; 10/12/2022
15393                ;or     byte [es:di+35], 80h
15394                ;;or    word [es:di+35], 80h ; [es:di+BDS.flags]
15395                ;         ; set_dasd_true
15396 00000FE0 57      push    di          ; Save bds pointer
15397
15398                ; Figure out what we are supposed to do with the bpb
15399                ; were we asked to install a fake bpb?
15400
15401 00000FE1 F60701    test    byte [bx], 1    ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
15402                ;         ; INSTALL_FAKE_BPB
15403 00000FE4 7511      jnz     short InstallFakeBpb
15404
15405                ; were we returning a fake bpb when asked to build a bpb?
15406
15407                ; 24/12/2023
15408 00000FE6 26F6453F04    test    byte [es:di+3Fh], 4
15409                ; 10/12/2022
15410                ;test   byte [es:di+35], 4
15411                ;;test  word [es:di+35], 4 ; [es:di+BDS.flags]
15412                ;         ; return_fake_bpb
15413 00000FEB 7405      jz      short InstallRecommendedBpb
15414
15415                ; we were returning a fake bpb but we can stop now
15416
15417                ; 24/12/2023
15418 00000FED 2680653FFB    and     byte [es:di+3Fh], 0FBh
15419                ; 10/12/2022
15420                ;and    byte [es:di+35], 0FBh
15421                ;;and   word [es:di+35], 0FFFBh ; [es:di+BDS.flags]
15422                ;         ; ~return_fake_bpb
15423
15424 InstallRecommendedBpb:
15425                ; 24/12/2023
15426                ;mov     cx, 31          ; A_BPB.size
15427                ;lea     di, [di+27h]    ; [di+BDS.R_BPB] = [di+BDS.rbytespersec]
15428                ;         ; cx = 53 or 31
15429                ;         ; (PCDOS 7.1 IBMBIO.COM)
15430                jmp     short CopyTheBpb
15431
15432                ; -----
15433
15434 00000FF7 26804D3F04    InstallFakeBpb:
15435                ; 24/12/2023
15436                ;or      byte [es:di+3Fh], 4
15437                ; 10/12/2022
15438                ;or      byte [es:di+35], 4
15439                ;;or     word [es:di+35], 4 ; byte [es:di+BDS.flags]
15440                ;         ; return_fake_bpb
15441                ; 24/12/2023
15442                ;         ; cx = 53 or 31
15443                ;mov     cx, 25          ; A_BPB.size - 6
15444                ;         ; move 'smaller' bpb
15445                ;         ; [es:di+BDS.BPB] = [es:di+BDS.bytespersec]
15446                ;         ;
15447                ;         ;
15448                ;         ;
15449                ;         ;
15450                ;         ;
15451                ;         ;
15452                ;         ;
15453                ;         ;
15454                ;         ;
15455                ;         ;
15456                ;         ;
15457                ;         ;
15458                ;         ;
15459                ;         ;
15460                ;         ;
15461                ;         ;
15462                ;         ;
15463                ;         ;
15464                ;         ;
15465                ;         ;
15466                ;         ;
15467                ;         ;
15468                ;         ;
15469                ;         ;
15470                ;         ;
15471                ;         ;
15472                ;         ;
15473                ;         ;
15474                ;         ;
15475                ;         ;
15476                ;         ;
15477                ;         ;
15478                ;         ;
15479                ;         ;
15480                ;         ;
15481                ;         ;
15482                ;         ;
15483                ;         ;
15484                ;         ;
15485                ;         ;
15486                ;         ;
15487                ;         ;
15488                ;         ;
15489                ;         ;
15490                ;         ;
15491                ;         ;
15492                ;         ;
15493                ;         ;
15494                ;         ;
15495                ;         ;
15496                ;         ;
15497                ;         ;
15498                ;         ;
15499                ;         ;
15500                ;         ;
15501                ;         ;
15502                ;         ;
15503                ;         ;
15504                ;         ;
15505                ;         ;

```

```

15506             ;;
15507
15508             ; 17/10/2022
15509 0000104F 2E8E06[3000]    mov     es, [cs:BIOSDATAWORD]
15510             ;mov     es, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
15511             ; Trash our bds pointer
15512 StoreSectorInfo:
15513 00001054 47             inc     di
15514 00001055 47             inc     di             ; Skip over cylinder and head
15515 00001056 AD             lodsw
15516 00001057 AA             stosb             ; Get sector id
15517 00001058 AD             lodsw             ; Copy it
15518             ; Get sector size
15519
15520             ; 24/12/2023
15521             ; 02/09/2023 (PCDOS 7.1)
15522             ; call SectSizeToSectIndex
15523             ; 18/04/2024
15524 00001059 80FC02        cmp     ah, 3 ; 02/09/2023
15525             cmp     ah, 2             ; (0=>128,1=>256,2=>512,3=>1024)
15526             ; examine upper byte only
15527 0000105C 7704        ja     short OneK
15528 0000105E 88E0        mov     al, ah             ; value in AH is the index!
15529 00001060 EB02        jmp     short sdp_s
15530 OneK:
15531             mov     al, 3             ; 1024 bytes per sector
15532 sdp_s:
15533             stosb             ; Store sector SIZE index
15534             loop     StoreSectorInfo
15535 SectorInfoSaved:
15536             cld
15537             retn
15538
15539 ; -----
15540 TooManyPerTrack:
15541             mov     al, 0Ch
15542             stc
15543             retn
15544
15545 ; -----
15546 ; 16/10/2022
15547
15548 ; =====
15549 ; FormatTrack:
15550 ; if specialfunction byte is 1, then this is a status call to see if there is
15551 ; rom support for the combination of sec/trk and # of cyl, and if the
15552 ; combination is legal. if specialfunction byte is 0, then format the track.
15553 ;
15554 ; input: ES:di points to bds for drive
15555 ;
15556 ; output:
15557 ; for status call:
15558 ; specialfunction byte set to:
15559 ; 0 - rom support + legal combination
15560 ; 1 - no rom support
15561 ; 2 - illegal combination
15562 ; 3 - no media present
15563 ; carry cleared.
15564 ;
15565 ; for format track:
15566 ; carry set if error
15567 ;
15568 ; =====
15569
15570 ; 16/03/2019
15571             ; 24/12/2023 - Retro DOS 5.0
15572             ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:10B7h)
15573
15574 ; 19/10/2022
15575 FormatTrack:
15576             lds     bx, [ptrsav]
15577             lds     bx, [bx+19]             ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET
15578             test    byte [bx], 1             ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
15579             ; STATUS_FOR_FORMAT
15580             jz     short DoFormatTrack
15581             push    ds
15582             ; 17/10/2022
15583             mov     ds, [cs:BIOSDATAWORD]
15584             ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
15585             call    SetMediaForFormat ; Also moves current Dpt to TempDpt
15586             pop     ds
15587             mov     [bx], al             ; [bx+A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS]
15588             cld
15589             retn
15590
15591 ; -----
15592 DoFormatTrack:
15593             ; 24/12/2023 - Retro DOS 5.0
15594             cmp     byte [es:di+3Eh], 5
15595             ;cmp     byte [es:di+34], 5 ; [es:di+BDS.formfactor]
15596             ; DEV_HARDDISK
15597             jnz     short DoFormatDiskette
15598             ; 17/10/2022
15599             mov     ds, [cs:BIOSDATAWORD]
15600             ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
15601             ; Point to Bios_Data (at 2C7h:30h or 70h:25A0h)
15602             jmp     VerifyTrack
15603
15604 ; -----
15605 DoFormatDiskette:
15606             mov     cx, [bx+1]
15607             mov     dx, [bx+3]
15608             test    byte [bx], 2
15609             ; 17/10/2022
15610             mov     ds, [cs:BIOSDATAWORD]
15611             ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
15612             ; Setup ds-> Bios_Data for verify
15613             jz     short DoFormatDiskette_1
15614             jmp     VerifyTrack_Err
15615
15616 ; -----
15617 DoFormatDiskette_1:
15618             call    SetMediaForFormat ; Also moves current Dpt to TempDpt
15619             cmp     al, 1             ; ROM support for sec/trk, # trks comb?
15620             jz     short NeedToSetDasd ; Old rom
15621             cmp     al, 3             ; Time out error?
15622             jnz     short NoSetDasd ; No, fine. (at this point, don't care
15623             ; about the illegal combination)
15624             jmp     short FormatFailed
15625
15626 ; -----
15627 NeedToSetDasd:
15628             push    dx
15629             call    SetDasd             ; INT 13h, AH=17h
15630             pop     dx

```



```

15630      NoSetDasd:
15631      0000108B E8B9F8      call    checksingle    ; Do any needed      diskette swapping
15632      0000108E 89D0      mov     ax, dx          ; Get track from packet
15633      000010C0 A3[3901]      mov     [trknum], ax
15634      000010C3 880E[3801]  mov     [hdnum], cl     ; Store head from packet
15635      000010C7 88CC      mov     ah, cl
15636      000010C9 BB[AC04]      mov     bx, tracktable
15637      000010CC 8B0E[AA04]  mov     cx, [sectorspertrack]
15638      ; 24/12/2023 - Retro DOS 5.0
15639      000010D0 E307      jcxz    set_fmt_retry_count
15640      StoreCylinderHead:
15641      000010D2 8907      mov     [bx], ax        ; Store into TrackTable
15642      000010D4 83C304      add     bx, 4           ; Skip to next sector field
15643      000010D7 E2F9      loop    StoreCylinderHead
15644      set_fmt_retry_count:      ; 24/12/2023
15645      ;mov     cx, 5           ; MAXERR - Set up retry      count
15646      ; 02/09/2023
15647      000010D9 B105      mov     cl, 5
15648      FormatRetry:
15649      000010DB 51      push    cx
15650      000010DC BB[AC04]      mov     bx, tracktable
15651      000010DF A0[AA04]      mov     al, [sectorspertrack]
15652      000010E2 B405      mov     ah, 5           ; romformat
15653      000010E4 8C1E[A804]  mov     [xfer_seg], ds
15654      000010E8 E86602      call    ToRom
15655      000010EB 59      pop     cx
15656      000010EC 7216      jb     short FormatError
15657      000010EE 51      push    cx
15658      ; Now verify the sectors just formatted.
15659      ; NOTE: because of bug in some BIOSes we have to
15660      ; set ES:BX to 00:00
15661      000010EF 53      push    bx
15662      000010F0 31DB      xor     bx, bx
15663      000010F2 891E[A804]  mov     [xfer_seg], bx
15664      000010F6 A0[AA04]      mov     al, [sectorspertrack]
15665      000010F9 B404      mov     ah, 4           ; romverify
15666      000010FB B101      mov     cl, 1
15667      000010FD E85102      call    ToRom
15668      00001100 5B      pop     bx
15669      00001101 59      pop     cx
15670      00001102 7329      jnb     short FormatOk
15671      00001104 E83402      call    ResetDisk
15672      00001107 C606[AA05]01  mov     byte [had_format_error], 1
15673      0000110C 50      push    ax
15674      0000110D 51      push    cx
15675      0000110E 52      push    dx
15676      0000110F E89501      call    SetMediaForFormat
15677      00001112 3C01      cmp     al, 1
15678      00001114 7503      jnz     short whileErr
15679      00001116 E83101      call    SetDasd
15680      whileErr:
15681      00001119 5A      pop     dx
15682      0000111A 59      pop     cx
15683      0000111B 58      pop     ax
15684      0000111C E2BD      loop    FormatRetry
15685      FormatFailed:
15686      0000111E C606[AA05]01  mov     byte [had_format_error], 1
15687      ; Set the format error flag
15688      00001123 80FC06      cmp     ah, 6           ; DSK_CHANGE_LINE_ERR - convert change line
15689      00001126 7502      jnz     short DoMapIt   ; Error to timeout error
15690      00001128 B480      mov     ah, 80h        ; DSK_TIMEOUT_ERR
15691      DoMapIt:
15692      0000112A E97CFC      jmp     maperror
15693      ; -----
15694      FormatOk:
15695      0000112D C606[AA05]00  mov     byte [had_format_error], 0 ; reset the format error flag
15696      00001132 C3      retn
15697      ; -----
15698      ; 16/10/2022
15699      ; =====
15700      ;
15701      ;
15702      ; VerifyTrack:
15703      ;
15704      ; input: ES:di points to bds for drive
15705      ; =====
15706      ; 24/12/2023 - Retro DOS 5.0
15707      VerifyTrack:
15708      00001133 1E      push    ds
15709      00001134 C51E[1200]  lds     bx, [ptrsav]    ; DS:BX points to request header.
15710      00001138 C55F13      lds     bx, [bx+19]     ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
15711      ; Come here with DS:[BX] -> packet, ES:[DI] -&; bds
15712      0000113B 8B4F03      mov     cx, [bx+3]      ; [bx+A_VERIFYPACKET.VP_CYLINDER]
15713      0000113E 8B4701      mov     ax, [bx+1]      ; [bx+A_VERIFYPACKET.VP_HEAD]
15714      00001141 8B5705      mov     dx, [bx+5]      ; [bx+A_FORMATPACKET.FP_TRACKCOUNT]
15715      00001144 8A1F      mov     bl, [bx]        ; [bx+A_FORMATPACKET.FP_SPECIALFUNCTIONS]
15716      ; Get option flag word
15717      00001146 1F      pop     ds
15718      00001147 C606[2001]04  mov     byte [rflag], 4 ; romverify
15719      0000114C 890E[3301]  mov     [curtrk], cx
15720      00001150 A2[3201]      mov     [curhd], al     ; ASSUME heads < 256
15721      00001153 8B0E[AA04]  mov     cx, [sectorspertrack]
15722      ; Check specialfunctions to see if DO_FAST_FORMAT has been
15723      ; specified if not we should go to the normal track verification
15724      ; routine. If fast format has been specified we should get the
15725      ; number of tracks to be verified and check it to see if it is
15726      ; > 255. If it is then it is an error and we should go to
15727      ; VerifyTrack_Err. If not multiply the number of tracks by the
15728      ; sectors per track to get the total number of sectors to be
15729      ; verified. This should also be less than equal to 255
15730      ; otherwise we go to same error exit. If everything is okay
15731      ; we initialise cx to the total sectors. use ax as a temporary
15732      ; register.
15733      00001157 F6C302      test    bl, 2           ; Special function requested?
15734      0000115A 7421      jz     short NormVerifyTrack
15735      0000115C 89D0      mov     ax, dx          ; Get ax = number of trks to verify
15736      0000115E 08E4      or      ah, ah
15737      00001160 752C      jnz     short VerifyTrack_Err ; #tracks > 255
15738      00001162 F6E1      mul     cl
15739      00001164 08E4      or      ah, ah
15740      00001166 7526      jnz     short VerifyTrack_Err ; #sectors > 255
15741      00001168 89C1      mov     cx, ax
15742      ; 24/12/2023
15743      0000116A 26F6453F01  test    byte [es:di+3Fh], 1 ; PCDOS 7.1 IBMBIO.COM
15744      ; 10/12/2022
15745      ; test    byte [es:di+35], 1

```

```

15754                ; ;test word [es:di+35], 1 ; [es:di+BDS.flags]
15755                ; ;fnon_removable
15756 0000116F 740C    jz      short NormVerifyTrack
15757                ; ; Multitrack operation = on?
15758                ; ; 10/12/2022
15759                ; ; 19/10/2022
15760 00001171 F606[A004]80 test    byte [multirk_flag], 80h
15761                ; ;test word [multirk_flag], 80h ; MULTI_TRK_ON
15762                ; ;test ds:multirk_flag, 80h ; MULTI_TRK_ON
15763 00001176 7405    jz      short NormVerifyTrack
15764 00001178 C606[A704]01 mov     byte [multitrk_format_flag], 1
15765 NormVerifyTrack:
15766 0000117D 31C0    xor     ax, ax          ; 1st sector
15767 0000117F 31DB    xor     bx, bx
15768 00001181 891E[A804] mov     [xfer_seg], bx ; Use 0:0 as the transfer address for verify
15769 00001185 E83F00    call    TrackIo
15770 00001188 C606[A704]00 mov     byte [multitrk_format_flag], 0
15771 0000118D C3      retn
15772                ; -----
15773
15774 VerifyTrack_Err:
15775 0000118E B401    mov     ah, 1
15776 00001190 E916FC    jmp     maperror
15777                ; -----
15778
15779                ; 16/10/2022
15780
15781                ; =====
15782                ;
15783                ; ReadTrack:
15784                ;
15785                ; input: ES:di points to bds for drive
15786                ;
15787                ; =====
15788
15789 ReadTrack:
15790 00001193 C606[2001]02 mov     byte [rflag], 2          ; romread
15791 00001198 EB05    jmp     short ReadWriteTrack
15792                ; -----
15793
15794 WriteTrack:
15795                ; =====
15796                ;
15797                ; WriteTrack:
15798                ;
15799                ; input: ES:di points to bds for drive
15800                ;
15801                ; =====
15802
15803                mov     byte [rflag], 3          ; romwrite
15804 0000119A C606[2001]03                ; Fall into ReadWriteTrack
15805
15806                ; =====
15807                ;
15808                ; readwriteTrack:
15809                ;
15810                ; input:
15811                ; ES:di points to bds for drive
15812                ; rFlag - 2 for read, 3 for write
15813                ;
15814                ; =====
15815
15816 ReadWriteTrack:
15817                ; save bds pointer segment so we can use it to access
15818                ; our packet. Notice that this is not the standard register
15819                ; assignment for accessing packets
15820                ;
15821                ; 19/10/2022
15822                push    es
15823 0000119F 06      les     bx, [ptrsav]          ; ES:BX -> to request header
15824 000011A0 C41E[1200] les     bx, [es:bx+19]          ; [es:bx+IOCTL_REQ.GENERICIOCTL_PACKET]
15825 000011A4 26C45F13 mov     ax, [es:bx+3]          ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_CYLINDER]
15826 000011A8 268B4703 mov     [curtrk], ax
15827 000011AC A3[3301] mov     ax, [es:bx+1]          ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_HEAD]
15828 000011AF 268B4701 mov     [curhd], al           ; Assume heads < 256!!!
15829 000011B3 A2[3201] mov     ax, [es:bx+5]          ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_FIRSTSECTOR]
15830 000011B6 268B4705 mov     cx, [es:bx+7]          ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_SECTORSTOREADWRITE]
15831 000011BA 268B4F07 mov     bx, [es:bx+9]          ; [es:bx+A_TRACKREADWRITEPACKET.TRWP_TRANSFERADDRESS]
15832 000011BE 26C45F09                ; Get transfer address
15833
15834                ; we just trashed our packet address, but we no longer care
15835
15836                mov     [xfer_seg], es ; Pass transfer segment
15837 000011C2 8C06[A804] pop     es
15838 000011C6 07      ; Fall into TrackIo
15839
15840                ; ===== S U B R O U T I N E =====
15841
15842                ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
15843
15844                ; =====
15845                ;
15846                ; TrackIo:
15847                ; performs track read/write/verify
15848                ;
15849                ; input:
15850                ; rFlag - 2 = read
15851                ; 3 = write
15852                ; 4 = verify
15853                ; AX - Index into track table of first sector to io
15854                ; CX - Number of sectors to io
15855                ; Xfer_Seg:BX - Transfer address
15856                ; ES:DI - Pointer to bds
15857                ; CurTrk - Current cylinder
15858                ; CurHd - Current head
15859                ; =====
15860
15861                ; 16/03/2019 - Retro DOS v4.0
15862                ;
15863                ; 24/12/2023 - Retro DOS 5.0
15864                ;
15865                ; 19/10/2022
15866
15867 TrackIo:
15868                ; Procedure `disk' will pop stack to
15869                ; [spsav], sp ; SpSav and return if error
15870 000011C7 8926[3501] mov     [spsav], sp
15871 000011CB E8A9F7    call    checksingle          ; Ensure correct disk is in drv
15872 000011CE 803E[A905]01 cmp     byte [media_set_for_format], 1
15873                ; See if we have already set disk
15874 000011D3 7407    jz      short Dptalreadyset ; base table
15875 000011D5 50      push    ax                ; set up tables and variables for i/o

```

```

15878 000011D6 51          push    cx
15879 000011D7 E8A0F9      call   iosetup
15880 000011DA 59          pop     cx
15881 000011DB 58          pop     ax
15882          Dptalreadys:      ; Point si at the table entry of the
15883 000011DC BE[AC04]      mov     si, tracktable ; first sector to be io'd
15884          ; 24/12/2023
15885          ;add    ax, ax      ; PCDOS 7.1 IBMBIO.COM
15886          ;add    ax, ax
15887 000011DF D1E0      shl     ax, 1
15888 000011E1 D1E0      shl     ax, 1
15889 000011E3 01C6      add     si, ax
15890
15891          ; WE WANT:
15892          ; CX to be the number of times we have to loop
15893          ; DX to be the number of sectors we read on each iteration
15894
15895 000011E5 BA0100      mov     dx, 1
15896
15897          ; 24/12/2023
15898 000011E8 26F6453F08    test    byte [es:di+3Fh], 8 ; PCDOS 7.1 IBMBIO.COM
15899          ; 12/12/2022
15900          ;test    byte [es:di+23h], 8
15901          ;;test    word [es:di+35], 8 ; [es:di+BDS.flags]
15902          ; good_tracklayout
15903 000011ED 7402      jz      short ionextsector
15904
15905 000011EF 87D1      xchg    dx, cx      ; HEY! We can read all secs in one blow
15906          ionextsector:
15907          push    cx
15908          push    dx
15909          inc     si
15910          inc     si      ; Skip over the      cylinder and head in
15911          ; the track table
15912          lodsb    ; Get sector ID      from track table
15913          mov     [cursec], al
15914
15915          ; assumptions for a fixed disk multi-track disk      i/o
15916          ; 1). In the input CX (# of sectors to go) to TrackIo,
15917          ; only CL is valid.
15918          ; 2). Sector size should be set      to 512 bytes.
15919          ; 3). Good track layout
15920
15921          ; 24/12/2023
15922 000011F9 26F6453F01    test    byte [es:di+3Fh], 1 ; PCDOS 7.1 IBMBIO.COM
15923          ; 12/12/2022
15924          ;test    byte [es:di+23h], 1
15925          ;;test    word [es:di+35], 1 ; [es:di+BDS.flags]
15926          ; fnon_removable ; Fixed disk?
15927 000011FE 7414      jz      short IoRemovable ; No
15928
15929          ; 12/12/2022
15930 00001200 F606[A004]80    test    byte [multrk_flag], 80h
15931          ;test    word [multrk_flag], 80h ; MULTI_TRK_ON
15932          ; Allow multi-track operation?
15933          jz      short IoRemovable ; No,don't do that.
15934          mov     [seccnt], dx
15935          mov     ax, dx
15936          call    Disk
15937          pop     dx
15938          pop     cx
15939          cld
15940          retfn
15941
15942          ; -----
15943          IoRemovable:
15944          lodsb      ; Get sector size index      from track
15945          ; table and save it
15946          push    ax
15947          push    si
15948          push    ds      ; Save Bios_Data
15949          push    ax
15950          mov     ah, [eot] ; Preserve whatever might be in      ah
15951          ; Fetch EOT while ds-> Bios_Data
15952          lds     si, [dpt]
15953          mov     [si+3], al ; [si+DISK_PARMS.DISK_SECTOR_SIZ]
15954          mov     [si+4], ah ; [si+DISK_PARMS.DISK_EOT]
15955          pop     ax
15956          pop     ds
15957          mov     al, dl
15958          mov     [seccnt], ax
15959          call    Disk
15960          pop     si      ; Advance buffer pointer by adding
15961          ; sector size
15962          ;pop    ax
15963          ; 24/12/2023
15964          pop     cx
15965
15966          ; 02/09/2023 (PCDOS 7.1)
15967          ;call    SectorSizeIndexToSectorSize
15968          ;mov     cl, al ; 24/12/2023
15969          mov     ax, 128
15970          shl     ax, cl
15971
15972          add     bx, ax
15973          pop     dx
15974          pop     cx
15975          loop    ionextsector
15976          cmp     byte [media_set_for_format], 1
15977          ;jz      short NoNeedDone
15978          ; 12/12/2022
15979          je      short NoNeedDone2
15980          call    done      ; set time of last access, and reset
15981          ; entries in Dpt.
15982          NoNeedDone:
15983          cld      ; not necessary ('done' clears cf) ; 24/12/2023
15984          NoNeedDone2:
15985          retfn
15986
15987          ; ===== S U B R O U T I N E =====
15988          ; -----
15989          ;
15990          ; The sector size in bytes needs to be converted to an index value for the ibm
15991          ; rom. (0=>128,1=>256,2=>512,3=>1024). It is assumed that only these values
15992          ; are permissible.
15993          ;
15994          ;
15995          ; On Input  AX contains sector size in bytes
15996          ; On Output AL Contains index
15997          ; All other registers preserved
15998          ;
15999          ; -----
16000          ; 02/09/2023

```

```

16002 ;SectSizeToSectIndex:
16003 ;      cmp     ah, 2          ; (0=>128,1=>256,2=>512,3=>1024)
16004 ;      ;      ; examine upper      byte only
16005 ;      ja      short OneK
16006 ;      mov     al, ah        ; value in AH is the index!
16007 ;      retn
16008 ;
16009 ; -----
16010 ;
16011 ;OneK:
16012 ;      mov     al, 3
16013 ;      retn
16014 ;
16015 ; ===== S U B   R O U T I N E =====
16016 ;
16017 ; 02/09/2023
16018 ;SectorSizeIndexToSectorSize:
16019 ;      mov     cl, al
16020 ;      mov     ax, 128
16021 ;      shl     ax, cl
16022 ;      retn
16023 ;
16024 ; ===== S U B   R O U T I N E =====
16025 ;
16026 ; 16/10/2022
16027 ;
16028 ; -----
16029 ;
16030 ; SetDASD
16031 ;
16032 ; Set up the rom for formatting.
16033 ; we have to tell the rom bios what type of disk is in the drive.
16034 ;
16035 ; On Input  - ES:di - Points to bds
16036 ;
16037 ; -----
16038 ;
16039 ;      ; 24/12/2023 - Retro DOS 5.0
16040 ;      ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:129Bh)
16041 ;
16042 ;      ; 19/10/2022
16043 ;
16044 0000124A 803E[AA05]01 SetDasd:
16045 ;      cmp     byte [had_format_error], 1 ;
16046 ;      ;      ; See if we've previously set dasd type
16047 ;      jz      short DoSetDasd
16048 ;      ; 24/12/2023
16049 ;      test    byte [es:di+3Fh], 80h
16050 ;      ; 10/12/2022
16051 ;      test    byte [es:di+23h], 80h ; [es:di+BDS.flags]
16052 ;      ;test word [es:di+23h], 80h ; [es:di+BDS.flags]
16053 ;      ;      ; set_dasd_true
16054 ;      jz      short DasdHasBeenSet
16055 ;      ; 24/12/2023
16056 ;      and     byte [es:di+3Fh], 7Fh
16057 ;      ; 10/12/2022
16058 ;      and     byte [es:di+23h], 7Fh
16059 ;      ;and word [es:di+23h], 0FF7Fh ; [es:di+BDS.flags]
16060 ;      ;      ; ~set_dasd_true
16061 0000125D C606[AA05]00 DoSetDasd:
16062 00001262 C606[3B01]50
16063 00001267 B004
16064 ;      ; 24/12/2023
16065 00001269 268A653E
16066 ;      mov     ah, [es:di+3Eh]
16067 ;      ; 02/09/2023
16068 ;      mov     ah, [es:di+22h] ; [es:di+BDS.formfactor]
16069 ;      cmp     ah, 2
16070 ;      cmp     byte [es:di+22h], 2 ; [es:di+BDS.formfactor]
16071 ;      ;      ; DEV_3INCH720KB
16072 ;      jz      short DoSet
16073 ;      ; 24/12/2023
16074 ;      mov     al, 1
16075 ;      cmp     ah, 1
16076 ;      cmp     ah, al ; 1
16077 ;      cmp     byte [es:di+22h], 1 ; [es:di+BDS.formfactor]
16078 ;      ;      ; DEV_5INCH96TPI
16079 ;      ;jz     short GotBig
16080 ;      ; 24/12/2023
16081 ;      mov     al, 1
16082 ;      jmp     short DoSet
16083 ;      ; 02/09/2023
16084 00001276 750E
16085 ;      jnz     short DoSet
16086 ;
16087 ; GotBig:
16088 ;      mov     al, 2          ; 160/320k in a      1.2 meg drive
16089 ;      ; 02/09/2023
16090 ;      inc     ax ; mov al, 2
16091 ;      cmp     byte [mediatype], 0
16092 ;      jnz     short DoSet
16093 ;      mov     al, 3          ; 1.2meg in a 1.2meg drive
16094 ;      ; 10/12/2022
16095 ;      inc     al ; al = 3
16096 ;      ; 18/12/2022
16097 ;      inc     ax ; al = 3
16098 00001280 40
16099 00001281 C606[3B01]54
16100 ;      mov     byte [gap_patch], 54h
16101 DoSet:
16102 ;      push    ds
16103 ;      push    si
16104 ;      mov     ds, [zeroseg] ; Point to interrupt vectors
16105 ;      ; 02/09/2023
16106 ;      xor     si, si
16107 ;      mov     ds, si ; 0
16108 ;
16109 ;      lds     si, [DSKADR]
16110 ;      lds     si, [78h]      ; [DSKADR] (Int 1Eh)
16111 ;      ;lds    si, ds:78h
16112 ;
16113 00001290 C644090F
16114 ;      mov     byte [si+9], 0Fh ;
16115 ;      ;      ; [si+DISK_PARAMS.DISK_HEAD_STTL]
16116 ;
16117 ;      pop     si
16118 ;      pop     ds
16119 ;      mov     ah, 17h
16120 ;      mov     dl, [es:di+4]
16121 ;      int     13h          ; DISK - DISK -      SET TYPE (AT,XT2,XT286,CONV,PS
16122 ;      ;      ; AL = disk type AL = 03h - high-capacity disk in high-capacity drive
16123 ;
16124 ; DasdHasBeenSet:
16125 ;      mov     ah, [es:di+13h] ; [es:di+BDS.secptrack]
16126 ;      mov     [fmt_eot], ah
16127 ;      retn
16128 ;
16129 ; ===== S U B   R O U T I N E =====
16130 ;
16131 ; 16/10/2022

```

```

16126 ; -----
16127 ;
16128 ; Set Media Type for Format
16129 ; Performs the int 13 with ah = 18h to see if the medium described in the
16130 ; BPB area in the BDS can be handled by the rom.
16131 ; On Input, ES:DI -> current BDS.
16132 ; The status of the operation is returned in AL
16133 ;
16134 ; - 0 - if the support is available, and the combination is valid.
16135 ; - 1 - no rom support
16136 ; - 2 - illegal combination
16137 ; - 3 - no media present (rom support exists but cannot determine now)
16138 ;
16139 ; Flags also may be altered. All other registers preserved.
16140 ; If the call to rom returns no error, then the current Dpt is "replaced" by
16141 ; the one returned by the rom. This is Done by changing the pointer in [Dpt]
16142 ; to the one returned. the original pointer to the disk base table is stored
16143 ; in TempDpt, until it is restored.
16144 ; -----
16145 ;
16146 ;
16147 ; 24/12/2023 - Retro DOS 5.0
16148 ;
16149 ; 19/10/2022
16150 SetMediaForFormat:
16151     000012A7 51     push    cx
16152     000012A8 52     push    dx
16153 ;
16154 ; If we have a format error, then do not change Dpt, TempDpt.
16155 ; but we need to call int 13h, ah=18h again.
16156 ;
16157     000012A9 803E[AA05]01    cmp     byte [had_format_error], 1
16158     000012AE 7425            jz      short SkipSaveDskAdr
16159     000012B0 30C0            xor     al, al ; If already done return 0
16160     000012B2 803E[A905]01    cmp     byte [media_set_for_format], 1
16161     000012B7 7502            jnz     short DoSetMediaForFormat
16162     000012B9 EB7D            jmp     SetMediaRet ; Media already set
16163 ; -----
16164 ;
16165 DoSetMediaForFormat:
16166     000012BB 06     push    es
16167     000012BC 56     push    si
16168 ;
16169 ; 02/09/2023
16170 ; mov     es, [zeroseg] ; Point to interrupt vectors
16171     000012BD 31F6     xor     si, si ; 0
16172     000012BF 8EC6     mov     es, si
16173 ;
16174     000012C1 26C4367800    les     si, [es:DSKADR]
16175 ; les     si, es:78h ; [es:DSKADR]
16176 ; ; Get pointer to disk base table
16177     000012C6 8936[2D01]    mov     [dpt], si
16178     000012CA 8C06[2F01]    mov     [dpt+2], es ; Save pointer to table
16179 ;
16180 ; Initialize the head settle time to 0Fh. See the offsets
16181 ; given in dskprm.inc.
16182 ;
16183     000012CE 26C644090F    mov     byte [es:si+9], 0Fh ; [es:si+DISK_PARAMS.DISK_HEAD_STTL]
16184     000012D3 5E             pop     si
16185     000012D4 07             pop     es
16186 SkipSaveDskAdr:
16187 ; 24/12/2023
16188     000012D5 268B4D41    mov     cx, [es:di+41h] ; (PCDOS 7.1 IBMBIO.COM)
16189 ; mov     cx, [es:di+25h] ; [es:di+BDS.cylinders]
16190     000012D9 49             dec     cx
16191     000012DA 80E503    and     ch, 3
16192     000012DD D0CD     ror     ch, 1
16193     000012DF D0CD     ror     ch, 1
16194     000012E1 86E9     xchg    ch, cl
16195     000012E3 260A4D13    or      cl, [es:di+13h] ; [es:di+BDS.sectortrack]
16196     000012E7 268A5504    mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
16197     000012EB 06             push    es
16198     000012EC 1E             push    ds
16199     000012ED 56             push    si
16200     000012EE 57             push    di
16201     000012EF B418     mov     ah, 18h
16202     000012F1 CD13     int     13h ; DISK - SET MEDIA TYPE FOR FORMAT (AT model 3x9, XT2, XT286, PS)
16203 ; ; DL = drive number, CH = lower 8 bits of number of tracks, CL = sectors
16204 ;
16205     000012F3 7231     jc      short FormaStatErr
16206     000012F5 803E[AA05]01    cmp     byte [had_format_error], 1
16207     000012FA 7423     jz      short skip_disk_base_setting
16208     000012FC 06             push    es ; Save segment returned by the rom
16209 ;
16210 ; 02/09/2023
16211 ; mov     es, [zeroseg] ; Point to interrupt vector segment
16212     000012FD 31F6     xor     si, si
16213     000012FF 8EC6     mov     es, si ; 0
16214     00001301 06             push    es ; * ; 02/09/2023
16215 ;
16216     00001302 26C4367800    les     si, [es:DSKADR]
16217 ; les     si, es:78h ; [es:DSKADR] (Int 1Eh)
16218 ; ; Get current disk base table
16219     00001307 8936[AB05]    mov     [tempdpt], si
16220     0000130B 8C06[AD05]    mov     [tempdpt+2], es ; Save it
16221 ;
16222 ; 02/09/2023
16223 ; mov     es, [zeroseg]
16224 ; xor     si, si ; 0
16225 ; mov     es, si
16226     0000130F 07             pop     es ; * ; 02/09/2023
16227 ;
16228     00001310 26893E7800    mov     es:78h, di
16229     00001312 268F067A00    mov     [es:DSKADR], di
16230     00001315 268F067A00    pop     word ptr es:7Ah ; replace with one returned by rom
16231     0000131A C606[A905]01    pop     word [es:DSKADR+2]
16232     0000131F 30C0     mov     byte [media_set_for_format], 1
16233 ; skip_disk_base_setting:
16234     0000131F 30C0     xor     al, al ; Legal combination + rom support code
16235     00001321 A2[AA05]    mov     ds:had_format_error, al ; Reset the flag
16236     00001324 EB0E     mov     [had_format_error], al
16237 ; jmp     short PopStatRet
16238 ; -----
16239 ;
16240 FormaStatErr:
16241 ; 10/12/2022
16242     00001326 B003     mov     al, 3
16243 ;
16244     00001328 80FC0C    cmp     ah, 0Ch ; DSK_ILLEGAL_COMBINATION
16245 ; ; illegal combination = 0Ch
16246     0000132B 7406     jz      short FormatStatIllegalComb
16247     0000132D 80FC80    cmp     ah, 80h ; DSK_TIMEOUT_ERR
16248     00001330 7402     jz      short FormatStatTimeout
16249 ; 10/12/2022

```

```

16249             ;dec     al
16250             ; 18/12/2022
16251 00001332 48   dec     ax
16252             ; al = 2
16253             ;mov     al, 1           ; Function not supported.
16254             ;jmp     short PopStatRet
16255             ; -----
16256
16257 FormatStatIllegalComb:
16258             ; 10/12/2022
16259             ;dec     al           ; 3 -> 2 or 2 -> 1
16260             ; 18/12/2022
16261 00001333 48   dec     ax
16262             ; al = 2
16263             ;mov     al, 2           ; Function supported, but
16264             ;                               ; Illegal sect/trk,trk combination.
16265             ; 10/12/2022
16266             ;jmp     short PopStatRet
16267             ; -----
16268
16269 FormatStatTimeOut:
16270             ; 10/12/2022
16271             ; al = 3
16272             ;mov     al, 3           ; Function supported, but
16273             ;                               ; Media not present.
16274
16275 00001334 5F   pop     di
16276 00001335 5E   pop     si
16277 00001336 1F   pop     ds
16278 00001337 07   pop     es
16279
16280 00001338 5A   SetMediaRet:
16281 00001339 59   pop     dx
16282 0000133A C3   pop     cx
16283             retn
16284
16285             ; ===== S U B   R O U T I N E =====
16286
16287             ; 16/10/2022
16288
16289             ; -----
16290             ;
16291             ; RESET THE DRIVE
16292             ;
16293             ; we also set [Step_Drv] to -1 to force the main disk routine to use the
16294             ; slow head settle time for the next operation. this is because the reset
16295             ; operation moves the head to cylinder 0,so we need to do a seek the next
16296             ; time around - there is a problem with 3.5" drives in that the head does
16297             ; not settle down in time,even for read operations!!
16298             ; -----
16299
16300 ResetDisk:
16301 0000133B 50   push    ax
16302
16303             ; 02/09/2023
16304             mov     ax, 1 ; PCDOS 7.1
16305 0000133C B80100 cmp     [media_set_for_format], al ; 1
16306             ;cmp     byte [media_set_for_format], 1
16307             ;                               ; Reset while formatting?
16308 00001343 7503   jnz     short ResetDisk_cont
16309             ;                               ; Then verify operation in "fmt & vrfy"
16310             ;mov     byte [had_format_error], 1 ; Might have failed.
16311 00001345 A2[AA05] mov     [had_format_error], al ; 1
16312
16313 ResetDisk_cont:
16314             ; 02/09/2023 (ah=0)
16315             xor     ah, ah           ; So signals that we had a format error
16316             int     13h             ; DISK - RESET DISK SYSTEM
16317             ;                               ; DL = drive (if bit 7 is set both hard disks and floppy disks reset)
16318             mov     byte [step_drv], 0FFh ; -1
16319             ;                               ; Zap up the speed
16320             pop     ax
16321             retn
16322
16323             ; ===== S U B   R O U T I N E =====
16324
16325             ; 16/10/2022
16326
16327             ; -----
16328             ;
16329             ; This routine sets up the drive parameter table with the values needed for
16330             ; format,does an int 13. values in Dpt are restored after a verify is done.
16331             ;
16332             ; on entry - ES:DI - points to bds for the drive
16333             ; Xfer_Seg:BX - points to trkbuf
16334             ; AL - number of sectors
16335             ; AH - int 13 function code
16336             ; CL - sector number for verify
16337             ; DS - Bios_Data
16338             ;
16339             ; ON EXIT - DS,DI,ES,BX remain unchanged.
16340             ; AX and flags are the results of the int 13
16341             ; -----
16342
16343             ; 24/12/2023 - Retro DOS 5.0
16344
16345             ; 19/10/2022
16346
16347 ToRom:
16348 00001351 53   push    bx
16349             push    si
16350
16351             ; Compaq bug fix - check whether we are using new ROM
16352             ; functionality to set up format, not merely if it exists.
16353             ; This was formerly a check against [new_rom]
16354             test     byte [media_set_for_format], 1
16355             jnz     short GotValidDpt
16356             push    ax
16357             push    es           ; Save bds segment
16358             ; 24/12/2023
16359 0000135C 26807D3E02 cmp     byte [es:di+3Eh], 2
16360             ;cmp     byte [es:di+22h], 2 ; [es:di+BDS.formfactor]
16361             ;                               ; ffSmall ; is it a 3.5" drive?
16362             ; 24/12/2023
16363             ;pushf ; not necessary           ; (Save the cmp result)
16364             mov     es, [zeroseg]
16365             les     si, es:78h ; Get pointer to disk base table
16366             les     si, [es:DSKADR]
16367             ;mov     word ptr ds:dpt, si
16368             ;mov     word ptr ds:dpt+2, es ; Save pointer to table
16369             mov     [dpt], si
16370             mov     [dpt+2], es ; Save pointer to table
16371
16372             mov     al, [formt_eot]

```



```

16373 00001375 26884404      mov     [es:si+4], al ; [es:si+DISK_PARMS.DISK_EOT]
16374 00001379 A0[3B01]      mov     al, [gap_patch]
16375 0000137C 26884407      mov     [es:si+7], al ; [es:si+DISK_PARMS.DISK_FORMAT_GAP]
16376                                     ; Important for format
16377 00001380 26C644090F      mov     byte [es:si+9], 0Fh ; [es:si+DISK_PARMS.DISK_HEAD_STTL]
16378                                     ; Assume we are doing a seek operation
16379                                     ; Setup motor start correctly for 3.5" drives
16380                                     ; 24/12/2023
16381                                     ; popf ; Get result of earlier cmp
16382 00001385 7505      jnz     short MotorStrtOK
16383 00001387 26C6440A04      mov     byte [es:si+0Ah], 4 ; [es:si+DISK_PARMS.DISK_MOTOR_STRT]
16384 MotorStrtOK:
16385 0000138C 07      pop     es ; Restore bds segment
16386 0000138D 58      pop     ax
16387 GotValidDpt:
16388 0000138E 8B16[3901]      mov     dx, [trknum] ; Set track number
16389 00001392 88D5      mov     ch, dl ; Set low 8 bits in ch
16390 00001394 268A5504      mov     dl, [es:di+4] ; Set drive number
16391 00001398 8A36[3801]      mov     dh, [hdnum] ; Set head number
16392 0000139C 06      push    es ; Save bds segment
16393 0000139D 8E06[A804]      mov     es, [xfer_seg]
16394 000013A1 CD13      int     13h ; DISK -
16395 000013A3 07      pop     es ; Restore bds segment
16396 000013A4 5E      pop     si
16397 000013A5 5B      pop     bx
16398 000013A6 C3      retn
16399
16400 ; -----
16401
16402 ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
16403 ; 24/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
16404
16405 ; BIOSCODE:1124h (MSDOS 6.21, IO.SYS)
16406 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1404h
16407
16408 ; =====
16409
16410 ; get the owner of the physical drive represented by the logical drive in al.
16411 ; the assumption is that we **always** keep track of the owner of a drive!!
16412 ; if this is not the case, the system may hang, just following the linked list.
16413 ;
16414 ; =====
16415
16416 ; 24/12/2023 - Retro DOS 5.0
16417
16418 ; 19/10/2022
16419 ioctl_getown:
16420 000013A7 E8FAF1      call    SetDrive
16421 000013AA 268A4504      mov     al, [es:di+4] ; [es:di+BDS.drivenum]
16422                                     ; Get physical drive number
16423 000013AE C43E[1901]      les     di, [start_bds] ; Get start of bds chain
16424
16425 000013B2 26384504      cmp     [es:di+4], al ; [es:di+BDS.drivenum]
16426 000013B6 7507      jnz     short getnextBDS
16427                                     ; 24/12/2023
16428 000013B8 26F6453F20      test    byte [es:di+3Fh], 20h ; (PCDOS 7.1 IBMBIO.COM)
16429                                     ; 10/12/2022
16430                                     ; test byte [es:di+23h], 20h
16431                                     ; ;test word [es:di+23h], 20h ; [es:di+BDS.flags]
16432                                     ; ;fi_own_physical
16433 000013BD 7514      jnz     short exitown
16434
16435 000013BF 26C43D      getnextBDS: les     di, [es:di] ; [es:di+BDS.link]
16436 000013C2 EBEE      jmp     short ownloop
16437
16438 ; -----
16439
16440 ; =====
16441
16442 ; set the ownership of the physical drive represented by the logical drive
16443 ; in al to al.
16444 ;
16445 ; =====
16446
16447 ; 24/12/2023 - Retro DOS 5.0
16448
16449 ; 19/10/2022
16450 000013C4 E8DDF1      ioctl_setown: call    SetDrive
16451 000013C7 C606[7A00]01      mov     byte [fsetowner], 1
16452                                     ; set flag for CheckSingle to look at.
16453 000013CC E8A8F5      call    checksingle
16454                                     ; 02/09/2023
16455 000013CF FE0E[7A00]      dec     byte [fsetowner] ; 0
16456                                     ; mov byte [fsetowner], 0
16457                                     ; set ownership of drive reset flag
16458                                     ; Fall into ExitOwn
16459
16460 ; =====
16461
16462 ; if there is only one logical drive assigned to this physical drive, return
16463 ; 0 to user to indicate this. Enter with ES:di -> the owner's bds.
16464 ;
16465 ; =====
16466
16467 ; 24/12/2023 - Retro DOS 5.0
16468
16469 000013D3 30C9      exitown: xor     cl, cl
16470                                     ; 24/12/2023
16471 000013D5 26F6453F10      test    byte [es:di+3Fh], 10h ; (PCDOS 7.1 IBMBIO.COM)
16472                                     ; 12/12/2022
16473                                     ; test byte [es:di+23h], 10h
16474                                     ; ;test word [es:di+23h], 10h ; [es:di+BDS.flags]
16475                                     ; ;fi_am_mult
16476 000013DA 7406      jz      short exitnomult
16477 000013DC 268A4D05      mov     cl, [es:di+5] ; [es:di+BDS.drivelet]
16478                                     ; Get logical drive number
16479                                     ; Get it 1-based
16480 000013E0 FEC1      inc     cl
16481
16482 000013E2 C51E[1200]      exitnomult: lds     bx, [ptrsav]
16483 000013E6 884F01      mov     [bx+1], cl ; [bx+unit]
16484                                     ; Exit normal termination
16485                                     ; 12/12/2022
16486                                     ; cf=0
16487                                     ; c!c
16488 000013E9 C3      retn
16489
16490 ; ===== S U B R O U T I N E =====
16491
16492 ; 16/10/2022
16493
16494 ; -----
16495
16496 ; moves the old Dpt that had been saved in TempDpt back to Dpt. this is done

```

```

16497 ; only if the first byte of TempDpt is not -1.
16498 ; all registers (including flags) are preserved.
16499 ;
16500 ; -----
16501 ;
16502 ; 24/12/2023
16503 ; 19/10/2022
16504 RestoreOldDpt:
16505 ; if we have already restored the disk base table earlier,
16506 ; do not do it again.
16507
16508 000013EA 50          push    ax
16509 000013EB 30C0        xor     al, al
16510 000013ED A2[AA05]     mov     [had_format_error], al; Reset flag and
16511 000013F0 8606[A905]   xchg    al, [media_set_for_format]; get current flag setting
16512 000013F4 08C0        or      al, al
16513 000013F6 7418        jz      short DontRestore
16514 000013F8 56          push    si
16515 000013F9 1E          push    ds
16516 000013FA 06          push    es
16517 000013FB C536[AB05]   lds     si, [tempdpt]
16518
16519 ; 17/10/2022
16520 ;mov     es, [cs:BIOSDATAWORD]
16521 ;;mov    es, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
16522 ;mov     es, [es:zéroseg]
16523 ;;mov    es, es:zéroseg ; CAS -- bleeeech!
16524
16525 ; 24/12/2023
16526 000013FF 31C0        xor     ax, ax
16527 00001401 8EC0        mov     es, ax ; 0
16528
16529 ;mov     es:78h, si ; [es:DSKADR] (Int 1Eh)
16530 00001403 2689367800  mov     [es:DSKADR], si
16531 ;mov     word ptr es:7Ah, ds ; [es:DSKADR+2]
16532 00001408 268C1E7A00  mov     [es:DSKADR+2], ds
16533 0000140D 07          pop     es
16534 0000140E 1F          pop     ds
16535 0000140F 5E          pop     si
16536 DontRestore:
16537 00001410 58          pop     ax
16538 ; 12/12/2022
16539 ; cf=0
16540 ;clc
16541 00001411 C3          retn    ; clear carry
16542
16543 ; -----
16544 ;
16545 ; 16/10/2022
16546
16547 ; =====
16548 ; get media id
16549 ; =====
16550 ;
16551 ; FUNCTION: get the volume label, the system id and the serial number from
16552 ; the media that has the extended boot record.
16553 ; for the conventional media, this routine will return "unknown
16554 ; media type" error to dos.
16555 ;
16556 ; INPUT : ES:di -> bds table for this drive.
16557 ;
16558 ; OUTPUT: the request packet filled with the information, if not carry.
16559 ; if carry set, then al contains the device driver error number
16560 ; that will be returned to dos.
16561 ; register DS, DX, AX, CX, DI, SI destroyed.
16562 ;
16563 ; SUBROUTINES TO BE CALLED:
16564 ; BootIo:NEAR
16565 ;
16566 ; LOGIC:
16567 ; to recognize the extended boot record, this logic will actually
16568 ; access the boot sector even if it is a hard disk.
16569 ; note: the valid extended bpb is recognized by looking at the mediabyte
16570 ; field of bpb and the extended boot signature.
16571 ;
16572 ; {
16573 ; get logical drive number from bds table;
16574 ; rFlag = read operation;
16575 ; BootIo; /*get the media boot record into the buffer
16576 ; if (no error) then
16577 ; if (extended boot record) then
16578 ; { set volume label, volume serial number and system id
16579 ; of the request packet to those of the boot record;
16580 ; };
16581 ; else /*not an extended bpb */
16582 ; { set register al to "unknown media.." error code;
16583 ; set carry bit;
16584 ; };
16585 ; else
16586 ; ret; /*already error code is set in the register al
16587 ;
16588 ; =====
16589 ;
16590 ;size_of_EXT_BOOT_SERIAL equ 4
16591 ;size_of_EXT_BOOT_VOL_LABEL equ 11
16592 ;size_of_EXT_SYSTEM_ID equ 8
16593 ;
16594 ; 24/12/2023 - Retro DOS 5.0
16595 ; (PCDOS 7.1 IBMBIO.COM - BIOSCODE:1478h)
16596 ;
16597 ; 19/10/2022
16598 GetMediaId:
16599 00001412 E8B000      call    ChangeLineChk
16600 00001415 268A4505    mov     al, [es:di+5] ; [es:di+BDS.drivelet] ; Logical drive number
16601 00001419 C606[2001]02  mov     byte [rflag], 2 ; Read operation
16602 0000141E E88C00      call    BootIo ; Read boot sector into DiskSector
16603 00001421 722E        jb      short IOct1_If1
16604 ; valid? (0F0h-0FFh?)
16605 00001423 803E[6701]F0    cmp     byte [disksector+15h], 0F0h
16606 ; [disksector+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
16607 ;jb      short IOct1_If2 ; brief not valid (0F0h - 0FFh)
16608 ; 24/12/2023
16609 00001428 7225        jb      short IOct1_If7
16610
16611 ; 24/12/2023
16612 ; 10/12/2022
16613 ;mov     si, disksector+26h
16614 ;;;
16615 ; 24/12/2023
16616 ;mov     si, disksector+43h ; BS_FAT32_VolID
16617 0000142A BE[9401]     mov     si, disksector+42h ; BS_FAT32_BootSig ; 24/12/2023
16618 0000142D 833E[6801]00  cmp     word [disksector+16h], 0 ; BPB.FATSz16
16619 00001432 7403        jz      short IOct1_If3 ; FAT32 fs
16620 00001434 83EE1C      sub     si, 1Ch ; FAT (12-16) fs ; 43h-1Ch = 27h ; BS_VolID

```



```

16621 ; si = disksector+26h = BS_BootSig ; 24/12/2023
16622
16623 IOct1_If3:
16624 ;cmp byte [si-1], 29h ; BS_BootSig or BS_FAT32_BootSig
16625 ;;;
16626 cmp byte [si], 29h
16627 ;cmp byte [disksector+26h], 29h ; [disksector+EXT_BOOT.SIG]
16628 ; EXT_BOOT_SIGNATURE
16629 jne short IOct1_If2 ; not extended boot record
16630 les di, [ptrsav] ; es:di points to request header
16631 les di, [es:bx+19] ; [es:bx+IOCTL_REQ.GENERICIOCTL_PACKET]
16632 ; 10/12/2022
16633 ;mov si, disksector+27h ; disksector+EXT_BOOT.SERIAL
16634 inc si
16635 ; 24/12/2023
16636 ; si = disksector+27h (BS_VolID)
16637 ; or disksector+43h (BS_FAT32_VolID)
16638 add di, 2 ; A_MEDIA_ID_INFO.MI_SERIAL
16639 ; 24/12/2023
16640 IOct1_If4: mov cx, 23 ; size_of_EXT_BOOT_SERIAL
16641 ; L+size_of_EXT_BOOT_VOL_LABEL
16642 ; +size_of_EXT_SYSTEM_ID
16643 rep movsb ; Move from Bios_Data into request packet
16644 ; 10/12/2022
16645 ; cf = 0
16646 ;clc
16647
16648 retn
16649 ; -----
16650 ; 24/12/2023
16651 IOct1_If2: stc
16652
16653 IOct1_If7:
16654 mov al, 7 ; error_unknown_media
16655 ;stc
16656
16657 IOct1_If6:
16658 IOct1_If1: retn
16659 ; -----
16660 ; 16/10/2022
16661 ; =====
16662 ; set media id
16663 ; =====
16664
16665 ; function: set the volume label, the system id and the serial number of
16666 ; the media that has the extended boot record.
16667 ; for the conventional media, this routine will return "unknown
16668 ; media.." error to dos.
16669 ; this routine will also set the corresponding informations in
16670 ; the bds table.
16671 ;
16672 ; input : ES:di -> bds table for this drive.
16673 ;
16674 ; output: the extended boot record in the media will be set according to
16675 ; the request packet.
16676 ; if carry set, then al contains the device driver error number
16677 ; that will be returned to dos.
16678 ;
16679 ; subroutines to be called:
16680 ; BootIo:NEAR
16681 ;
16682 ; logic:
16683 ; {
16684 ; get drive_number from bds;
16685 ; rFlag = "read operation";
16686 ; BootIo;
16687 ; if (no error) then
16688 ; if (extended boot record) then
16689 ; { set volume label,volume serial number and system id
16690 ; of the boot record to those of the request packet;
16691 ; rFlag = "write operation";
16692 ; get drive number from bds;
16693 ; BootIo; /*write it back*/
16694 ; }
16695 ; else /*not an extended bpb */
16696 ; { set register al to "unknown media.." error code;
16697 ; set carry bit;
16698 ; ret; /*return back to caller */
16699 ; }
16700 ; else
16701 ; ret; /*already error code is set */
16702 ; }
16703 ; =====
16704 ; 24/12/2023 - Retro DOS 5.0
16705
16706 SetMediaId:
16707 ; 19/10/2022
16708 call ChangeLineChk
16709 mov al, [es:di+5] ; [es:di+BDS.drivelet]
16710 ; Logical drive number
16711 mov dl, al
16712 mov byte [rflag], 2 ; romread
16713 push dx
16714 call BootIo ; Read boot sec to Bios_Data:DiskSector
16715 pop dx
16716 jb short IOct1_If6
16717 ; valid? (0F0h-0FFh?)
16718 cmp byte [disksector+15h], 0F0h
16719 ; [disksector+EXT_BOOT.BPB+EBPB.MEDIADESCRIPTOR]
16720 jb short IOct1_If7 ; Brif not
16721 ; 24/12/2023
16722 ;cmp byte [disksector+26h], 29h ; [disksector+EXT_BOOT.SIG]
16723 ; EXT_BOOT_SIGNATURE
16724 ;jnz short IOct1_If7 ; not extended boot record
16725
16726 push es ; Save BDS pointer
16727 push di
16728 push ds ; PointES To boot record
16729 pop es
16730 ; 24/12/2023
16731 ;;;
16732 ;mov di, disksector+43h ; disksector+EXT_BOOT.SERIAL
16733 mov di, disksector+42h ; BS_FAT32_BootSig ; 24/12/2023
16734 cmp word [disksector+16h], 0 ; BPB.FATSz16
16735 jz short IOct1_If5 ; FAT32 fs
16736 sub di, 1ch ; 67-28 ; offset disksektor+27h
16737
16738
16739
16740
16741
16742
16743
16744

```

```

16745 ; di = disksector+26h = BS_BootSig ; 24/12/2023
16746
16747 IOct1_If5: ;cmp byte [di-1], 29h ; BS_BootSig or BS_FAT32_BootSig
16748 0000147F 803D29 ;cmp byte [di], 29h
16749 00001482 7404 ;je short IOct1_If8
16750 00001484 5F ;pop di ; not extended boot record
16751 00001485 07 ;pop es
16752 ;jmp short IOct1_If7
16753 ; 24/12/2023
16754 00001486 EBC6 ;jmp short IOct1_If2
16755
16756 IOct1_If8: ;;;
16757 ; 24/12/2023
16758 ;mov di, disksector+27h ; disksector+EXT_BOOT.SERIAL
16759 00001488 47 ;inc di
16760 ; di = disksector+27h (BS_VolID)
16761 ; or disksector+43h (BS_FAT32_VolID)
16762
16763 00001489 C536[1200] ;lds si, [ptrsav] ; ds:si points to request header.
16764 0000148D C57413 ;lds si, [si+19] ; [si+IOCTL_REQ.GENERICIOCTL_PACKET]
16765 00001490 83C602 ;add si, 2 ; A_MEDIA_ID_INFO.MI_SERIAL
16766
16767 ; 24/12/2023
16768 ;mov cx, 23 ; size_of_EXT_BOOT_SERIAL
16769 ; ; +size_of_EXT_BOOT_VOL_LABEL
16770 ; ; +size_of_EXT_SYSTEM_ID
16771 ;rep movsb
16772 00001493 E8B2FF ;call IOct1_If4 ; copy volume serial, label and system id
16773
16774 00001496 06 ;push es ; point ds back to Bios_Data
16775 00001497 1F ;pop ds
16776 00001498 5F ;pop di ; restore bds pointer
16777 00001499 07 ;pop es
16778 0000149A E8B2F3 ;call mov_media_ids ; update the bds media id info.
16779 0000149D 88D0 ;mov al, dl
16780 0000149F C606[2001]03 ;mov byte [rflag], 3 ; romwrite
16781 000014A4 E80600 ;call BootIo ; write it back.
16782 000014A7 C606[1E01]FF ;mov byte [tim_drv], 0FFh
16783 ; ; make sure chk_media check the driver
16784 ; ; return with error code from BootIo
16785 000014AC C3 ;retn
16786 ; -----
16787 ; 24/12/2023
16788 ;IOct1_If7: ;mov al, 7 ; error_unknown_media
16789 ; ; stc
16790 ;IOct1_If6: ;retn
16791 ;
16792 ; ===== S U B R O U T I N E =====
16793 ;
16794 ; 16/10/2022
16795 ; -----
16796 ; BootIo
16797 ; -----
16800 ;
16801 ; function: read/write the boot record into boot sector.
16802 ;
16803 ; input :
16804 ; al=logical drive number
16805 ; rFlag = operation (read/write)
16806 ;
16807 ; output: for read operation, the boot record of the drive specified in bds
16808 ; be read into the DiskSector buffer.
16809 ; for write operation, the DiskSector buffer image will be written
16810 ; to the drive specified in bds.
16811 ; if carry set, then al contains the device driver error number
16812 ; that will be returned to dos.
16813 ; AX,CX,DX register destroyed.
16814 ; if carry set, then al will contain the error code from DiskIO.
16815 ;
16816 ; subroutines to be called:
16817 ; DiskIO:NEAR
16818 ;
16819 ; logic:
16820 ;
16821 ; {
16822 ; first_sector = 0; /*logical sector 0 is the boot sector */
16823 ; sectorcount = 1; /*read 1 sector only */
16824 ; buffer = DiskSector; /*read it into the DiskSector buffer */
16825 ; call DiskIO (rFlag,drive_number,first_sector,sectorcount,buffer);
16826 ; }
16827 ; =====
16828 ;
16829 ; 19/10/2022
16830 ; BootIo:
16831 ;
16832 ; push es
16833 000014AD 06 ; push di
16834 000014AE 57 ; push bx
16835 000014AF 53 ; push ds
16836 000014B0 1E ; pop es ; Point ES: to Bios_Data
16837 000014B1 07
16838 ;
16839 ; Call DiskIO to read/write the boot sec. The parameters which
16840 ; need to be initialized for this subroutine out here are
16841 ; - Transfer address to Bios_Data:DiskSector
16842 ; - Low sector needs to be initialized to 0. this is a reg. param
16843 ; - Hi sector in [Start_Sec_H] needs to be initialised to 0.
16844 ; - Number of sectors <-- 1
16845 ;
16846 000014B2 BF[5201] ;mov di, disksector ; es:di -> transfer address
16847 000014B5 31D2 ;xor dx, dx ; Firstsector (h) -> 0
16848 000014B7 8916[9C04] ;mov [start_sec_h], dx ; Start sector (h) -> 0
16849 000014BB B90100 ;mov cx, 1
16850 000014BE E850F5 ;call diskio
16851 000014C1 5B ;pop bx
16852 000014C2 5F ;pop di
16853 000014C3 07 ;pop es
16854 000014C4 C3 ;retn
16855 ;
16856 ; ===== S U B R O U T I N E =====
16857 ;
16858 ; 16/10/2022
16859 ; -----
16860 ; ChangelineChk
16861 ; -----
16862 ;
16863 ; when the user calls get/set media id call before dos establishes the media
16864 ; by calling "media_chk", the change line activity of the drive is going to be
16865 ; lost. this routine will check the change line activity and will save the
16866 ; history in the flags.
16867 ;
16868 ;

```

```

16869 ; FUNCTION: check the change line error activity
16870 ;
16871 ; INPUT : ES:di -> bds table.
16872 ;
16873 ; OUTPUT: flag in bds table will be updated if change line occurs.
16874 ;
16875 ; SUBROUTINES TO BE CALLED:
16876 ; Set_Changed_DL
16877 ;
16878 ; -----
16879 ;
16880 ; 24/12/2023 - Retro DOS 5.0
16881 ChangeLnChk:
16882 000014C5 268A5504 mov dl, [es:di+4] ; [es:di+BDS.drivenum]
16883 000014C9 08D2 or dl, dl ; Fixed disk?
16884 000014CB 7821 js short ChangeLnChkRet ; Yes, skip it.
16885 ; 24/12/2023
16886 000014CD 26F6453F04 test byte [es:di+3Fh], 4 ; [es:di+BDS.flags] ; PCDOS 7.1
16887 ; 12/12/2022
16888 ; test byte [es:di+23h], 4
16889 ; test word [es:di+23h], 4 ; [es:di+BDS.flags]
16890 ; return_fake_bpb
16891 000014D2 751A jnz short ChangeLnChkRet
16892 000014D4 803E[7700]01 cmp byte [fhave96], 1 ; This rom support change line?
16893 000014D9 7513 jnz short ChangeLnChkRet
16894 000014DB E8AD07 call haschange ; This drive support change line?
16895 000014DE 740E jz short ChangeLnChkRet ; Do nothing
16896 ;
16897 ; Execute the rom disk interrupt to check changeline activity.
16898 ;
16899 000014E0 B416 mov ah, 16h
16900 000014E2 CD13 int 13h ; DISK - FLOPPY DISK - CHANGE OF DISK STATUS (AT,XT2,XT286,CONV,PS)
16901 ; DL = drive to check
16902 ; Return: AH = disk change status
16903 000014E4 7308 jnb short ChangeLnChkRet
16904 000014E6 53 push bx
16905 000014E7 BB4000 mov bx, 40h ; fchanged
16906 ; Update flag in BDS for this
16907 ; physical drive
16908 000014EA E87707 call set_changed_dl
16909 000014ED 5B pop bx
16910 ChangeLnChkRet:
16911 000014EE C3 retn
16912 ; -----
16913 ;
16914 ; 16/10/2022
16915 ;
16916 ; =====
16917 ; GetAccessFlag
16918 ; =====
16919 ;
16920 ; FUNCTION: get the status of UNFORMATTED_MEDIA bit of flags in bds table
16921 ;
16922 ; INPUT :
16923 ; ES:di -> bds table
16924 ;
16925 ; OUTPUT: a_DiskAccess_Control.dac_access_flag = 0 if disk i/o not allowed.
16926 ; = 1 if disk i/o allowed.
16927 ; =====
16928 ;
16929 ; 24/12/2023 - Retro DOS 5.0
16930 ;
16931 ; 19/10/2022
16932 GetAccessFlag:
16933 000014EF C51E[1200] lds bx, [ptrsav] ; DS:BX points to request header
16934 000014F3 C55F13 lds bx, [bx+19] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
16935 ; mov al, 0 ; Assume result is unformatted
16936 ; 10/12/2022
16937 sub al, al
16938 000014F6 28C0 ; 24/12/2023
16939 test byte [es:di+40h], 02h ; (PCDOS 7.1 IBMBIO.COM)
16940 000014F8 26F6454002 test word ptr es:[di+3Fh], 200h
16941 ; 10/12/2022
16942 test byte [es:di+36], 02h
16943 ; test word [es:di+35], 200h ; [es:di+BDS.flags]
16944 ; unformatted_media
16945 ; Done if unformatted
16946 000014FD 7501 jnz short GafDone ; Return true for formatted
16947 ; inc al
16948 ; 24/12/2023
16949 000014FF 40 inc ax
16950 GafDone:
16951 00001500 884701 mov [bx+1], al ; [bx+A_DISKACCESS_CONTROL.DAC_ACCESS_FLAG]
16952 00001503 C3 retn
16953 ; -----
16954 ;
16955 ; 16/10/2022
16956 ;
16957 ; =====
16958 ; SetAccessFlag
16959 ; =====
16960 ;
16961 ; function: set/reset the UNFORMATTED_MEDIA bit of flags in bds table
16962 ;
16963 ; input :
16964 ; ES:di -> bds table
16965 ;
16966 ; output: unformtted_media bit modified according to the user request
16967 ; =====
16968 ;
16969 ; 24/12/2023 - Retro DOS 5.0
16970 ;
16971 ; 19/10/2022
16972 SetAccessFlag:
16973 00001504 C51E[1200] lds bx, [ptrsav] ; ES:BX points to request header
16974 00001508 C55F13 lds bx, [bx+19] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
16975 ; 24/12/2023
16976 0000150B 26806540FD and byte [es:di+40h], 0FDh ; (PCDOS 7.1 IBMBIO.COM)
16977 ; and word ptr es:[di+3Fh], 0FDFFh
16978 ; 10/12/2022
16979 ; and byte [es:di+36], 0FDh
16980 ; and word [es:di+35], 0FDFFh ; [es:di+BDS.flags]
16981 ; ~unformatted_media
16982 00001510 807F0100 cmp byte [bx+1], 0 ; [bx+A_DISKACCESS_CONTROL.DAC_ACCESS_FLAG]
16983 00001514 7505 jnz short saf_Done
16984 ; 24/12/2023
16985 00001516 26804D4002 or byte [es:di+40h], 02h ; (PCDOS 7.1 IBMBIO.COM)
16986 ; or word ptr es:[di+3Fh], 200h
16987 ; 15/04/2024
16988 ; 10/12/2022
16989 ; or byte [es:di+36], 02h
16990 ; or word [es:di+35], 200h ; [es:di+BDS.flags]
16991 ; unformatted_media
16992 saf_Done:

```

```

16993 0000151B C3
16994
16995
16996
16997
16998
16999
17000
17001
17002
17003
17004
17005
17006
17007
17008
17009
17010
17011
17012 0000151C 06
17013 0000151D C41E[1200]
17014 00001521 268B470D
17015
17016
17017
17018 00001525 3C48
17019
17020 00001527 7404
17021
17022 00001529 3C08
17023
17024 0000152B 7513
17025
17026 0000152D 0E
17027 0000152E 07
17028
17029
17030 0000152F B90E00
17031
17032
17033 00001532 BF[BD0E]
17034
17035
17036 00001535 86C4
17037 00001537 F2AE
17038 00001539 7505
17039 0000153B B80001
17040
17041
17042
17043
17044
17045
17046
17047 0000153E 07
17048
17049
17050
17051 0000153F C3
17052
17053
17054 00001540 07
17055 00001541 E991EB
17056
17057
17058
17059
17060
17061
17062
17063
17064
17065
17066
17067
17068
17069
17070
17071
17072
17073
17074
17075
17076
17077
17078
17079
17080
17081
17082
17083
17084
17085
17086
17087
17088
17089
17090
17091
17092
17093 00001544 C51E[1200]
17094 00001548 C55F13
17095
17096
17097 0000154B 31D2
17098 0000154D 8917
17099
17100 0000154F 268A5504
17101
17102
17103
17104 00001553 B420
17105
17106 00001555 CD13
17107 00001557 7216
17108 00001559 FE07
17109
17110 0000155B FEC8
17111 0000155D 3C02
17112 0000155F 740A
17113 00001561 0404
17114 00001563 3C07
17115 00001565 7404
17116 00001567 3C09

; -----
; retn
; -----
; 16/10/2022
; =====
; Ioctl_Support_Query
; =====
; New device command which was added in DOS 5.00 to allow a query of a
; specific GENERIC IOCTL to see if it is supported. Bit 7 in the
; device attributes specifies if this function is supported.
; =====
; 24/12/2023 - Retro DOS 5.0
; 19/10/2022
ioctl_support_query:
    push    es
    les     bx, [ptrsav] ; ES:BX Points to request header.
    mov     ax, [es:bx+13] ; [es:bx+IOCTL_REQ.MAJORFUNCTION]
                                ; AL == Major, AH == Minor
    ; 24/12/2023
    ; 02/09/2023 (PCDOS 7.1)
    cmp     al, 48h ; IOC_NEW_DC (PCDOS 7.1)
                                ; new generic ioctl function (FAT32)
    je      short ioctl_support
    cmp     al, 8 ; IOC_DC
                                ; See if major code is 8
    jne     short nosupport
ioctl_support:
    push    cs
    pop     es
    ; 24/12/2023
    ; 02/09/2023
    mov     cx, 14 ; (PCDOS 7.1) IOC_DC_TABLE_LEN
    ;mov     cx, 11 ; IOC_DC_TABLE_LEN
    ; 10/12/2022
    mov     di, IOC_DC_Table
    ;mov     di, 0C60h ; IOC_DC_Table
                                ; at 2C7h:0C60h = 70h:31D0h
    xchg     al, ah ; Put minor code in AL
    repne    scasb ; Scan for minor code in AL
    jnz     short nosupport ; it was not found
    mov     ax, 100h
    ; 10/12/2022
    ; (jump to ioctlsupexit is not required)
    jmp     short $+2 ; ioctlsupexit
                                ; Signal ioctl is supported
    ; jmp     short ioctlsupexit
; -----
ioctlsupexit:
    pop     es
    ; 10/12/2022
    ; cf = 0
    cld
    retn
; -----
nosupport:
    pop     es
    jmp     bc_cmderr
; -----
; 16/10/2022
; =====
; GetMediaSenseStatus
; =====
; FUNCTION: Will return the type of diskette media in the specified DOS
; diskette drive and whether the media is the default type
; for that drive. (default type means the max size for that
; drive)
; INPUT : ES:DI -> BDS table
; OUTPUT: If carry clear
; DS:BX -> Updated IOCTLPacket
;
; Special Function at offset 0:
; 0 - Media detected is not default type
; 1 - Media detected is default type
;
; Device Type at offset 1:
; 2 - 720K 3.5" 80 tracks
; 7 - 1.44M 3.5" 80 tracks
; 9 - 2.88M 3.5" 80 tracks
;
; Error Codes returned in AX if carry set:
; 8102 - Drive not ready - No disk is in the drive.
; 8107 - Unknown media type - Drive doesn't support this function or
; the media is really unknown, any error
; other than "media not present"
; =====
; 19/10/2022
SenseMediaType:
    lds     bx, [ptrsav] ; DS:BX points to request header.
    lds     bx, [bx+19] ; bx+IOCTL_REQ.GENERICIOCTL_PACKET]
    ; 10/10/2022
    mov     word [bx], 0 ; Initialize the 2 packet bytes
    xor     dx, dx
    mov     [bx], dx ; 0
    ;
    mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
                                ; Get int 13h drive number from BDS
    ; 10/12/2022
    xor     dh, dh ; DX = physical drive number
    mov     ah, 20h ; Get Media Type function
                                ; If no carry media type in AL
    int     13h ; DISK - QCACHE - DISMOUNT
    jc      short MediaSenseEr ; error code in AH
    inc     byte [bx] ; Signal media type is default (bit 1)
DetermineMediaType:
    dec     al
    cmp     al, 2 ; Chk for 720K ie: (3-1) = 2
    jz      short GotMediaType
    add     al, 4
    cmp     al, 7 ; Chk for 1.44M ie: (4-1+4) = 7
    jz      short GotMediaType
    cmp     al, 9 ; Chk for 2.88M ie: (6-1+4) = 9

```

```

17117 00001569 7510          jnz     short UnknownMediaType ; Just didn't recognize media type
17118
17119 0000156B 884701      GotMediaType:
17120                      mov     [bx+1], al      ; Save the return value
17121                      ; 10/12/2022
17122                      ; cf = 0
17123 0000156E C3          ; clc
17124                      ; Signal success
17125                      retn
17126
17127 0000156F 80FC32      MediaSenseEr:
17128 00001572 74E7          cmp     ah, 32h      ; See if not default media error
17129 00001574 B002          jz      short DetermineMediaType ; Not really an error
17130 00001576 80FC31      mov     al, 2      ; Now assume drive not ready
17131 00001579 7402          cmp     ah, 31h      ; See if media was present
17132                      jz      short SenseErrExit ; Return drive not ready
17133 0000157B B007          UnknownMediaType:
17134                      mov     al, 7      ; Just don't know the media type
17135 0000157D B481          SenseErrExit:
17136 0000157F F9          mov     ah, 81h      ; Signal error return
17137 00001580 C3          stc
17138                      retn
17139
17140                      ; -----
17141                      ; 10/12/2022
17142                      ; db 0
17143                      ; -----
17144
17145                      ; -----
17146                      ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:15F2h
17147                      ; -----
17148                      ; 26/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
17149
17150                      ; ===== S U B R O U T I N E =====
17151
17152 00001581 C51E[1200]      SetLockState:
17153 00001585 C55F13          lds     bx, [ptrsav] ; set media lock state
17154                      lds     bx, [bx+13h] ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
17155                      mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
17156                      ; call check_int13h_exts_present
17157                      ; 26/12/2023
17158                      call    check_int13h_exts_p
17159 0000158B 721C          ; mov     al, 3      ; unknown command error
17160 0000158D 8A07          jc      short setlockst_ret
17161 0000158F B445          mov     al, [bx]      ; [bx+A_LOCKSTATE_CONTROL.LOCKSTATE_FUNCTIONS]
17162 00001591 CD13          mov     ah, 45h
17163                      int     13h      ; DISK - IBM/MS Extension - LOCK/UNLOCK DRIVE
17164                      ; (DL - drive, [SI - disk address packet])
17165 00001593 884701      mov     [bx+1], al      ; 1 = locked, 0 = not locked
17166                      ; [bx+A_LOCKSTATE_CONTROL.LOCKSTATE_FLAG]
17167                      ; 26/12/2023
17168 00001596 EB0A          jmp     short sls_em
17169
17170                      ; jnc     short setlockst_ret
17171                      ; mov     al, ah
17172                      ; call    maperror
17173                      ; setlockst_ret:
17174                      ; mov     ah, 81h      ; Return this status in case of carry
17175                      ; retn
17176
17177                      ; ===== S U B R O U T I N E =====
17178
17179                      EjectMedia:
17180                      ; mov     dl, [es:di+4] ; eject media in drive
17181                      ; ; [es:di+BDS.drivenum]
17182                      ; call    check_int13h_exts_present
17183                      ; 26/12/2023
17184 00001598 E81100      call    check_int13h_exts_p
17185                      ; mov     al, 3      ; unknown command error
17186 0000159B 720C          jc      short ejectm_ret
17187 0000159D B80046      mov     ax, 4600h
17188 000015A0 CD13          int     13h      ; DISK - IBM/MS Extension - EJECT MEDIA
17189                      ; (DL - drive)
17190                      ; 26/12/2023
17191 000015A2 7305          sls_em: jnc     short ejectm_ret
17192 000015A4 88E0          mov     al, ah
17193 000015A6 E800F8      call    maperror
17194                      ; setlockst_ret:
17195                      ; 26/12/2023
17196 000015A9 B481          ejectm_ret: mov     ah, 81h      ; Return this status in case of carry
17197 000015AB C3          retn
17198
17199                      ; ===== S U B R O U T I N E =====
17200
17201                      ; 26/12/2023
17202                      check_int13h_exts_p:
17203 000015AC 268A5504      mov     dl, [es:di+4]
17204
17205                      check_int13h_exts_present:
17206 000015B0 B441          mov     ah, 41h
17207 000015B2 53          push    bx
17208 000015B3 BBAA55          mov     bx, 55AAh
17209 000015B6 CD13          int     13h      ; DISK - Check for INT 13h Extensions
17210                      ; BX = 55AAh, DL = drive number
17211                      ; Return: CF set if not supported
17212                      ; AH = extensions version
17213                      ; BX = AA55h
17214                      ; CX = Interface support bit map
17215 000015B8 81FB55AA      cmp     bx, 0AA55h
17216 000015BC 5B          pop     bx
17217 000015BD 7505          jnz     short exts_notsupported
17218 000015BF F6C102      test    cl, 2      ; bit 1 - drive locking and ejecting subset
17219 000015C2 7503          jnz     short exts_supported
17220                      exts_notsupported:
17221                      ; 26/12/2023
17222 000015C4 B003          mov     al, 3
17223                      ;
17224 000015C6 F9          stc
17225                      exts_supported:
17226                      retn
17227
17228                      ; ===== S U B R O U T I N E =====
17229
17230                      GetDrvMapInfo:
17231 000015C8 8CD9          mov     cx, ds      ; get drive map information
17232                      ;
17233                      ; es:di points to BDS which belongs to
17234                      ; the requested logical/dos drive number
17235                      ;
17236                      ; Format of parameter block:
17237                      ; Offset Description (Table 01570)
17238                      ; 00h (call) length of this buffer (in bytes)
17239                      ; 01h (ret) number of bytes in parameter block
17240                      ; actually used

```

```

17241                                     ; 02h (ret) drive flags
17242                                     ; 03h (ret) physical drive number
17243                                     ; 00h-7Fh floppy
17244                                     ; 80h-FEh hard
17245                                     ; FFh no physical drive
17246                                     ; 04h (ret) bitmap of logical drives associated with
17247                                     ; physical drive
17248                                     ; bit 0 = drive A:, etc.
17249                                     ; 08h (ret) relative block address of partition start
17250                                     ; qword
17251                                     ;
17252                                     ; Ref: Ralf Brown's Interrupt List, INTERRUPT.G
17253 000015CA C51E[1200] lds bx, [ptrsav]
17254 000015CE C55F13 lds bx, [bx+13h]
17255 000015D1 B80381 mov ax, 8103h
17256                                     ; [bx+IOCTL_REQ.GENERICIOCTL_PACKET]
17257 000015D4 803F10 cmp byte [bx], 10h
17258 000015D7 7251 jb short gdmi_4
17259 000015D9 268A5504 mov dl, [es:di+4]
17260 000015DD 885703 mov [bx+3], dl
17261 000015E0 C6470110 mov byte [bx+1], 10h
17262 000015E4 268B4517 mov ax, [es:di+17h]
17263 000015E8 894708 mov [bx+8], ax
17264 000015EB 268B4519 mov ax, [es:di+19h]
17265 000015EF 89470A mov [bx+0Ah], ax
17266 000015F2 31C0 xor ax, ax
17267 000015F4 884702 mov [bx+2], al
17268 000015F7 89470C mov [bx+0Ch], ax
17269 000015FA 89470E mov [bx+0Eh], ax
17270 000015FD 894704 mov [bx+4], ax
17271                                     ; drive flags = 0 (protected mode flags etc.)
17272 00001600 894706 mov [bx+6], ax
17273 00001603 8EC1 es, cx
17274                                     ; les di, dword ptr es:start_bds ; 1st BDS
17275 00001605 26C43E[1901] les di, [es:start_bds]
17276 0000160A B90100 mov cx, 1
17277 gdmi_1:                                     ; bit 0 (drive A:)
17278 0000160D 83FFFF cmp di, 0FFFFh
17279 00001610 7415 jz short gdmi_3
17280 00001612 26385504 cmp [es:di+4], dl
17281                                     ; [es:di+BDS.drivenum], dl
17282 00001616 7506 jnz short gdmi_2
17283 00001618 094F04 or [bx+4], cx
17284                                     ; is it same physical drive ?
17285 0000161B 094706 or [bx+6], ax
17286 gdmi_2:                                     ; no
17287 0000161E D1E1 shl cx, 1
17288 00001620 D1D0 rcl ax, 1
17289 00001622 26C43D les di, [es:di]
17290 00001625 EBE6 jmp short gdmi_1
17291 gdmi_3:                                     ; set high word of the bit select (set) value
17292 00001627 B80001 mov ax, 100h
17293 gdmi_4:                                     ; next BDS
17294 0000162A C3 retn
17295                                     ; loop until di = -1 (last BDS sign)
17296                                     ;
17297                                     ; -----
17298                                     ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
17299                                     ; 26/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
17300                                     ; -----
17301                                     ; MSINT13.ASM - MSDOS 6.0 - 1991
17302                                     ; -----
17303                                     ; 16/03/2019 - Retro DOS v4.0
17304                                     ;
17305                                     ; int 2f function 13h allows the user to change the orig13 int_13 vector
17306                                     ; after booting. this allows testing and implementation of custom int_13
17307                                     ; handlers, without giving up ms-dos error recovery
17308                                     ; entry: ds:dx == addr. of new int_13 handler
17309                                     ; es:bx == addr. of new int_13 vector used by warm boot (int19)
17310                                     ; exit: orig13 == address of new int_13 handler
17311                                     ; ds:dx == old orig13 value
17312                                     ; es:bx == old old13 value
17313                                     ;
17314                                     ; int 2f handler for external block drivers to communicate with the internal
17315                                     ; block driver in msdisk. the multiplex number chosen is 8. the handler
17316                                     ; sets up the pointer to the request packet in [ptrsav] and then jumps to
17317                                     ; dsk_entry, the entry point for all disk requests.
17318                                     ;
17319                                     ; on exit from this driver, we will return to the external driver
17320                                     ; that issued this int 2f, and can then remove the flags from the stack.
17321                                     ; this scheme allows us to have a small external device driver, and makes
17322                                     ; the maintainance of the various drivers (driver and msbio) much easier,
17323                                     ; since we only need to make changes in one place (most of the time).
17324                                     ;
17325                                     ; ax=800h - check for installed handler - reserved
17326                                     ; ax=801h - install the bds into the linked list
17327                                     ; ax=802h - dos request
17328                                     ; ax=803h - return bds table starting pointer in ds:di
17329                                     ; (ems device driver hooks int 13h to handle 16kb dma overrun
17330                                     ; problem. bds table is going to be used to get head/sector
17331                                     ; informations without calling generic ioctl get device parm call.)
17332                                     ;
17333                                     ; BIOSSEGMENT equ 70h
17334 DOSBIOSSEG equ 0070h ; 17/10/2022
17335                                     ;
17336                                     ; ;BIOSCODE:1302h (MSDOS 6.21, IO.SYS)
17337                                     ; ;BIOSCODE:16AAh (PCDOS 7.1, IBMBIO.COM) ; 26/12/2023
17338                                     ;
17339 i2f_handler:                                     ; here is 02C7h:1302h = 0070h:3872h
17340                                     ;
17341 0000162B 80FC13 cmp ah, 13h
17342 0000162E 7413 jz short int2f_replace_int13
17343 00001630 80FC08 cmp ah, 8
17344 00001633 7432 jz short mine
17345                                     ;
17346                                     ; Check for WIN386 startup and return the BIOS instance data
17347                                     ;
17348 00001635 80FC16 cmp ah, 16h
17349 00001638 746D jz short win386call
17350 0000163A 80FC4A cmp ah, 4Ah
17351 0000163D 7503 jnz short i2f_handler_iret
17352 0000163F E99800 jmp handle_multmult
17353                                     ; -----
17354 i2f_handler_iret:
17355 iret
17356 00001642 CF
17357                                     ; -----
17358 int2f_replace_int13:
17359 cli
17360 00001643 FA push ax
17361 00001644 50 mov ax, ds
17362 00001645 8CD8 mov ds, word [cs:0030h]
17363                                     ; then we can use ds: -> Bios_Data
17364                                     ; 15/10/2022
17365                                     ; mov ds, word ptr cs:BIOSDATAWORD ; [cs:0030h]

```



```

17365                                     ; = [02C7h:0030h] = [0070h:25A0h]
17366 00001647 2E8E1E[3000]      mov     ds, [cs:BIOSDATAWORD] ; 17/10/2022
17367                                     ; 19/10/2022
17368                                     ;push    word ptr ds:orig13      ; save old value of old13 and
17369                                     ;push    word ptr ds:orig13+2    ; orig13 so that we can
17370                                     ;push    word ptr ds:old13      ; return them to caller
17371                                     ;push    word ptr ds:old13+2
17372
17373                                     ; 02/09/2023 (PCDOS 7.1)
17374                                     ;push    word [orig13]
17375 0000164C FF36[B600]      push    word [orig13+2]
17376                                     ;push    word [old13]
17377 00001650 FF36[0801]      push    word [old13+2]
17378
17379                                     ;mov     word ptr ds:orig13, dx; orig13 := addr. of new int_13
17380                                     ;mov     word ptr ds:orig13+2, ax
17381                                     ;mov     word ptr ds:old13, bx ; old13 := addr. of new boot_13
17382                                     ;mov     word ptr ds:old13+2, es
17383
17384                                     ;mov     [orig13], dx
17385                                     ; 02/09/2023
17386 00001654 8716[B400]      xchg     dx, [orig13]
17387 00001658 A3[B600]      mov     [orig13+2], ax
17388                                     ;mov     [old13], bx
17389                                     ; 02/09/2023
17390 0000165B 871E[0601]      xchg     bx, [old13]
17391 0000165F 8C06[0801]      mov     [old13+2], es
17392
17393 00001663 07                                     pop     es                      ; es:bx := old old13 vector
17394                                     ; 02/09/2023
17395                                     ;pop     bx
17396 00001664 1F                                     pop     ds                      ; ds:dx := old orig13 vector
17397                                     ;pop     dx ; 02/09/2023
17398 00001665 58                                     pop     ax
17399
i2f_iret:
17400 00001666 CF      ired
17401
; -----
17402
mine:
17403
17404 00001667 3CF8      cmp     al, 0F8h                ; iret on reserved functions
17405 00001669 73FB      jnb     short i2f_iret
17406 0000166B 08C0      or      al, al                ; a get installed state request?
17407 0000166D 7503      jnz     short disp_func
17408 0000166F B0FF      mov     al, 0FFh
17409                                     ;jmp     short i2f_iret
17410                                     ; 02/09/2023
17411 00001671 CF      ired
17412
; -----
17413
disp_func:
17414
17415 00001672 3C01      cmp     al, 1                ; request for installing bds?
17416 00001674 7418      jz      short do_subfun_01
17417 00001676 3C03      cmp     al, 3                ; get bds vector?
17418 00001678 7423      jz      short do_get_bds_vector
17419
; set up pointer to request packet
17420
17421
17422 0000167A 1E      push    ds
17423 0000167B 2E8E1E[3000]  mov     ds, [cs:BIOSDATAWORD] ; 17/10/2022
17424                                     ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
17425                                     ; = [0070h:25A0h] = [02C7h:0030h]
17426                                     ; 19/10/2022
17427                                     ;mov     word ptr ds:ptrsav, bx
17428                                     ;mov     word ptr ds:ptrsav+2, es
17429 00001680 891E[1200]  mov     [ptrsav], bx
17430 00001684 8C06[1400]  mov     [ptrsav+2], es
17431 00001688 1F      pop     ds
17432                                     ;jmp     far ptr i2f_dskentry
17433                                     ; 07/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
17434                                     ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1708h
17435 00001689 EA[5E06]7000 jmp     DOSBIOSSEG:dsk_entry ; BIOSDATA:dsk_entry
17436                                     ; 17/10/2022
17437                                     ; jmp     far DOSBIOSSEG:dsk_entry
17438                                     ; jmp     DOSBIOSSEG:i2f_dskentry ; 70h:i2f_dskentry
17439                                     ; NOTE: jump to a FAR function, not an
17440                                     ; IRET type function. Callers of
17441                                     ; this int2f subfunction will have
17442                                     ; to be careful to do a popf
17443
; -----
17444
do_subfun_01:
17445
17446
17447 0000168E 06      push    es
17448 0000168F 1E      push    ds
17449 00001690 1E      push    ds
17450 00001691 07      pop     es
17451                                     ; 17/10/2022
17452 00001692 2E8E1E[3000]  mov     ds, [cs:BIOSDATAWORD]
17453                                     ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
17454                                     ; point ds: -> Bios_Data
17455 00001697 E8BC03      call    install_bds
17456 0000169A 1F      pop     ds
17457 0000169B 07      pop     es
17458                                     ;jmp     short i2f_iret
17459                                     ; 02/09/2023
17460 0000169C CF      ired
17461
; -----
17462
do_get_bds_vector:
17463
17464                                     ; 17/10/2022
17465 0000169D 2E8E1E[3000]  mov     ds, [cs:BIOSDATAWORD]
17466                                     ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
17467 000016A2 C53E[1901]  lds     di, [start_bds]
17468                                     ;lds     di, ds:start_bds
17469
; i2f_iret:
17470                                     ; 10/12/2022
17471                                     ; jmp     short i2f_iret
17472                                     ; 02/09/2023
17473                                     ired
17474
; -----
17475
; 17/10/2022
17476 ; 16/10/2022
17477
; WIN386 startup stuff is done here. If starting up we set our WIN386 present
17478 ; flag and return instance data. If exiting, we reset the WIN386 present flag
17479 ; NOTE: we assume that the BIOS int 2fh is at the bottom of the chain.
17480
win386call:
17481
17482
17483 000016A7 1E      push    ds
17484 000016A8 2E8E1E[3000]  mov     ds, [cs:BIOSDATAWORD]
17485                                     ;mov     ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
17486                                     ; at 2C7h:30h = 70h:25A0h
17487 000016AD 3C05      cmp     al, 5                ; win386_Init
17488                                     ; is it win386 initializing?

```

```

17489 000016AF 7410          jz      short win386Init
17490 000016B1 3C06          cmp     al, 6          ; win386_Exit
17491                                ; is itwin386 exiting?
17492 000016B3 7523          jnz     short win_iret ; if not, continue int2f chain
17493                                ; 12/12/2022
17494 000016B5 F6C201        test    dl, 1
17495                                ;test    dx, 1          ; is itwin386 or win286 dos extender?
17496 000016B8 751E          jnz     short win_iret ; if not win386, then continue
17497                                ;and     ds:Iswin386, 0 ; indicate that win386 is not present
17498 000016BA 8026[1208]00 and     byte [Iswin386], 0
17499 000016BF EB17          jmp     short win_iret
17500                                ; -----
17501
17502 win386Init:
17503                                ; 12/12/2022
17504 000016C1 F6C201        test    dl, 1
17505                                ;test    dx, 1          ; is it win386 or win286 dos extender?
17506 000016C4 7512          jnz     short win_iret ; if not win386, then continue
17507                                ;or      ds:Iswin386, 1 ; Indicate WIN386 present
17508 000016C6 800E[1208]01 or      byte [Iswin386], 1
17509                                ;mov     word ptr ds:SI_Next, bx ; Hook our structure into chain
17510                                ;mov     word ptr ds:SI_Next+2, es
17511 000016CB 891E[E007]    mov     [SI_Next], bx
17512 000016CF 8C06[E207]    mov     [SI_Next+2], es
17513                                ;mov     bx, offset win386_SI ; point ES:BX to Win386_SI
17514 000016D3 BB[DE07]      mov     bx, win386_SI ; 19/10/2022
17515 000016D6 1E          push    ds
17516 000016D7 07          pop     es
17517
17518 000016D8 1F          win_iret: pop     ds
17519                                ; 10/12/2022
17520                                ;jmp     short i2f_iret ; return back up the chain
17521                                ; 02/09/2023
17522 000016D9 CF          i2f_iret: iret
17523                                ; -----
17524
17525 handle_multmult:
17526 000016DA 3C01          cmp     al, 1
17527 000016DC 7514          jnz     short try_2
17528 000016DE 1E          push    ds
17529 000016DF E84500        call    HMAPtr          ; get offset of free HMA
17530                                ; 10/12/2022
17531                                ;xor     bx, bx
17532                                ;dec     bx
17533 000016E2 BBFFFF        mov     bx, 0FFFFh
17534 000016E5 8EC3          mov     es, bx          ; seg of HMA
17535 000016E7 89FB          mov     bx, di
17536 000016E9 F7D3          not     bx
17537 000016EB 09DB          or      bx, bx
17538 000016ED 7401          jz      short try_1
17539 000016EF 43          inc     bx
17540
17541 000016F0 1F          try_1:  pop     ds
17542                                ;jmp     short i2f_iret
17543                                ; 02/09/2023
17544 000016F1 CF          iret
17545                                ; -----
17546
17547 try_2:
17548 000016F2 3C02          cmp     al, 2          ; multMULTALLOCHMA
17549 000016F4 7530          jnz     short try_3
17550 000016F6 1E          push    ds
17551                                ; 10/12/2022
17552                                ;xor     di, di
17553                                ;dec     di
17554 000016F7 8FFFFF        mov     di, 0FFFFh      ; assume not enough space
17555 000016FA 8EC7          mov     es, di
17556 000016FC E82800        call    HMAPtr          ; get offset of free HMA
17557 000016FF 83FFFF        cmp     di, 0FFFFh
17558 00001702 7421          jz      short InsuffHMA
17559 00001704 F7DF          neg     di              ; free space in HMA
17560 00001706 39FB          cmp     bx, di
17561 00001708 7605          jbe     short try_4
17562                                ; 10/12/2022
17563                                ;sub     di, di
17564                                ;dec     di
17565 0000170A BFFFFF        mov     di, 0FFFFh
17566                                ;jmp     short InsuffHMA
17567                                ; 02/09/2023
17568 0000170D 1F          pop     ds
17569 0000170E CF          iret
17570                                ; -----
17571
17572 try_4:
17573                                ;mov     di, ds:FreeHMAPtr
17574 0000170F 8B3E[D707]    mov     di, [FreeHMAPtr]
17575 00001713 83C30F        add     bx, 15
17576                                ;and     bx, 0FFF0h
17577                                ; 10/12/2022
17578 00001716 80E3F0        and     bl, 0F0h
17579                                ;add     ds:FreeHMAPtr, bx ; update the free pointer
17580 00001719 011E[D707]    add     [FreeHMAPtr], bx
17581 0000171D 7506          jnz     short InsuffHMA
17582 0000171F C706[D707]FFFF    mov     word [FreeHMAPtr], 0FFFFh ; -1
17583                                ;mov     ds:FreeHMAPtr, 0FFFFh
17584                                ; no more HMA if we have wrapped
17585
17586 00001725 1F          InsuffHMA: pop     ds
17587                                ; 10/12/2022
17588
17589 try_3:
17590                                ;jmp     short i2f_iret
17591                                ; 02/09/2023
17592                                ; iret
17593                                ; -----
17594                                ; 10/12/2022
17595
17596 ;try_3:
17597                                ;jmp     i2f_iret
17598
17599 ; ===== S U B R O U T I N E =====
17600 ; 16/10/2022
17601
17602 ; -----
17603 ;
17604 ; procedure : HMAPtr
17605 ;
17606 ; Gets the offset of the free HMA area ( with respect to
17607 ; seg ffff )
17608 ; If DOS has not moved high, tries to move DOS high.
17609 ; In the course of doing this, it will allocate all the HMA
17610 ; and set the FreeHMAPtr to past the end of the BIOS and
17611 ; DOS code. The call to MoveDOSIntoHMA (which is a pointer)
17612 ; enters the routine in sysinit1 called FTryToMoveDOSHi.

```



```

17613 ;
17614 ; RETURNS : offset of free HMA in DI
17615 ; BIOS_DATA, seg in DS
17616 ;
17617 ;-----
17618 ;
17619 ; 17/10/2022
17620 HMAPtr:
17621 00001727 2E8E1E[3000] mov ds, [cs:BIOSDATAWORD]
17622 ;mov ds, word ptr cs:BIOSDATAWORD ; [cs:Bios_Data_Word]
17623 0000172C 8B3E[D707] mov di, [FreeHMAPtr]
17624 ;mov di, ds:FreeHMAPtr
17625 00001730 83FFFF cmp di, 0FFFFh
17626 00001733 750F jnz short HMAPtr_retn
17627 00001735 803E[DD07]00 cmp byte [SysinitPresent], 0
17628 ;cmp ds:SysinitPresent, 0
17629 0000173A 7408 jz short HMAPtr_retn
17630 0000173C FF1E[D907] call far [MoveDOSIntoHMA]
17631 ;call ds:MoveDOSIntoHMA ; call far [MoveDOSIntoHMA]
17632 00001740 8B3E[D707] mov di, [FreeHMAPtr]
17633 ;mov di, ds:FreeHMAPtr
17634 HMAPtr_retn:
17635 00001744 C3 retn
17636 ;
17637 ; ===== S U B R O U T I N E =====
17638 ;
17639 ; 16/10/2022
17640 ;
17641 ; move a 512 byte sector from ds:si to es:di, do not trash cx
17642 ; but go ahead and update direction flag, si, & di
17643 move_sector:
17644 ; The 80386 microprocessor considers an access to WORD 0FFFFh in
17645 ; any segment to be a fault. Theoretically, this could be handled
17646 ; by the fault handler and the behavior of an 8086 could be emulated
17647 ; by wrapping the high byte to offset 0000h. This would be a lot
17648 ; of work and was, indeed, blown off by the win386 guys. COMPAQ
17649 ; also handles the fault incorrectly in their ROM BIOS for real
17650 ; mode. Their fault handler was only designed to deal with one
17651 ; special case which occurred in a magazine benchmark, but didn't
17652 ; handle the general case worth beans.
17653 ;
17654 ; Simply changing this code to do a byte loop would work okay but
17655 ; would involve a general case performance hit. Therefore, we'll
17656 ; check for either source or destination offsets being within one
17657 ; sector of the end of their segments and only in that case fall
17658 ; back to a byte move.
17659 ;
17660 ;
17661 ;
17662 00001745 FC cld
17663 00001746 51 push cx
17664 00001747 B90001 mov cx, 256
17665 0000174A 81FE00FE cmp si, 0FE00h
17666 0000174E 770A ja short movsec_bytes
17667 00001750 81FF00FE cmp di, 0FE00h
17668 00001754 7704 ja short movsec_bytes
17669 00001756 F3A5 rep movsw
17670 00001758 59 pop cx
17671 00001759 C3 retn
17672 ;
17673 ;-----
17674 movsec_bytes:
17675 0000175A D1E1 shl cx, 1
17676 0000175C F3A4 rep movsb
17677 0000175E 59 pop cx
17678 0000175F C3 retn
17679 ;
17680 ; ===== S U B R O U T I N E =====
17681 ;
17682 ; 16/10/2022
17683 ;
17684 ; check_wrap is a routine that adjusts the starting sector, starting head
17685 ; and starting cylinder for an int 13 request that requests i/o of a lot
17686 ; of sectors. it only does this for fixed disks. it is used in the sections
17687 ; of code that handle ecc errors and dma errors. it is necessary, because
17688 ; ordinarily the rom would take care of wraps around heads and cylinders,
17689 ; but we break down a request when we get an ecc or dma error into several
17690 ; i/o of one or more sectors. in this case, we may already be beyond the
17691 ; number of sectors on a track on the medium, and the request would fail.
17692 ;
17693 ; input conditions:
17694 ; all registers set up for an int 13 request.
17695 ;
17696 ; output:
17697 ; dh - contains starting head number for request
17698 ; cx - contains starting sector and cylinder numbers
17699 ; (the above may or may not have been changed, and are 0-based)
17700 ; all other registers preserved.
17701 ;
17702 ; 26/12/2023 - Retro DOS 5.0
17703 check_wrap:
17704 00001760 50 push ax
17705 00001761 53 push bx
17706 00001762 06 push es
17707 00001763 57 push di
17708 00001764 E86C00 call find_bds ; get pointer to bds for drive in dl
17709 00001767 725E jb short no_wrap ; finished if DOS doesn't use it
17710 ; 26/12/2023
17711 00001769 26F6453F01 test byte [es:di+3Fh], 1
17712 ; 12/12/2022
17713 ; test byte [es:di+23h], 1
17714 ; test word [es:di+23h], 1 ; [es:di+BDS.flags], fnon_removable
17715 0000176E 7457 jz short no_wrap ; no wrapping for removable media
17716 00001770 268B5D13 mov bx, [es:di+13h] ; [es:di+BDS.sectorpertrack]
17717 00001774 89C8 mov ax, cx
17718 00001776 83E03F and ax, 3Fh ; extract sector number
17719 00001779 39D8 cmp ax, bx ; are we going to wrap?
17720 0000177B 764A jbe short no_wrap
17721 0000177D F6F3 div bl ; ah=new sector #, al=# of headwraps
17722 ;
17723 ; we need to be careful here. if the new sector # is 0, then we are on the
17724 ; last sector on that track.
17725 ;
17726 0000177F 08E4 or ah, ah
17727 00001781 7503 jnz short not_on_bound
17728 ; 18/12/2022
17729 00001783 48 dec ax ; *
17730 00001784 88DC mov ah, bl ; set sector=BDS_BPB.BPB_SECTORS PERTRACK
17731 ; if on boundary
17732 ; dec al ; * ; also decrement # of head wraps
17733 not_on_bound:
17734 00001786 80E1C0 and cl, 0C0h ; zero out sector #
17735 00001789 08E1 or cl, ah ; or in new sector #
17736 0000178B 30E4 xor ah, ah ; ax = # of head wraps

```

```

17737 0000178D 40          inc     ax
17738 0000178E 00F0        add     al, dh      ; add in starting head #
17739 00001790 80D400      adc     ah, 0       ; catch any carry
17740          ; 02/09/2023
17741 00001793 268B5D15    mov     bx, [es:di+15h] ; [es:di+BDS.heads]
17742 00001797 39D8        cmp     ax, bx
17743          ; cmp     ax, [es:di+15h] ; [es:di+BDS.heads]
17744          ; are we going to wrap around a head?
17745 00001799 7632        jbe     short no_wrap_head ; do not lose new head number!!
17746 0000179B 52          push    dx          ; preserve drive number and head number
17747 0000179C 31D2        xor     dx, dx
17748          ; mov     bx, [es:di+15h] ; [es:di+BDS.heads]
17749 0000179E F7F3        div     bx          ; dx=new head #, ax=# of cylinder wraps
17750
17751          ; careful here! if new head # is 0, then we are on the last head.
17752
17753          or     dx, dx
17754 000017A2 7507        jnz     short no_head_bound
17755 000017A4 89DA        mov     dx, bx      ; on boundary. set to BDS_BPB.BPB_HEADS
17756
17757          ; if we had some cylinder wraps, we need to reduce them by one!!
17758
17759          or     ax, ax
17760 000017A8 7401        jz      short no_head_bound
17761 000017AA 48          dec     ax          ; reduce number of cylinder wraps
17762
no_head_bound:
17763 000017AB 88D7        mov     bh, dl      ; bh has new head number
17764 000017AD 5A          pop     dx          ; restore drive number and head number
17765 000017AE FECF        dec     bh          ; get it 0-based
17766 000017B0 88FE        mov     dh, bh      ; set up new head number in dh
17767 000017B2 88CF        mov     bh, cl
17768 000017B4 80E73F      and     bh, 3Fh      ; preserve sector number
17769 000017B7 B306        mov     bl, 6
17770 000017B9 86CB        xchg    cl, bl
17771 000017BB D2EB        shr     bl, cl      ; get ms cylinder bits to ls end
17772 000017BD 00C5        add     ch, al      ; add in cylinder wrap
17773 000017BF 10E3        adc     bl, ah      ; add in high byte
17774 000017C1 D2E3        shl     bl, cl      ; move up to ms end
17775 000017C3 86D9        xchg    bl, cl      ; restore cylinder bits into cl
17776 000017C5 08F9        or      cl, bh      ; or in sector number
17777
no_wrap:
17778 000017C7 F8          cld
17779 000017C8 5F          pop     di
17780 000017C9 07          pop     es
17781 000017CA 5B          pop     bx
17782 000017CB 58          pop     ax
17783 000017CC C3          retn
17784
17785          ; -----
17786
no_wrap_head:
17787 000017CD 88C6        mov     dh, al      ; do not lose new head number
17788 000017CF FECE        dec     dh          ; get it 0-based
17789 000017D1 EBF4        jmp     short no_wrap
17790
17791          ; ===== S U B R O U T I N E =====
17792
17793          ; 16/10/2022
17794
17795          ; this is a special version of the bds lookup code which is
17796          ; based on physical drives rather than the usual logical drives
17797          ; carry is set if the physical drive in dl is found, es:di -> its bds
17798          ; otherwise carry is clear
17799          ;
17800          ; guaranteed to trash no registers except es:di
17801
17802          ; 19/10/2022
17803
find_bds:
17804 000017D3 C43E[1901]    les     di, [start_bds] ; point es:di to first bds
17805
fbds_1:
17806 000017D7 26385504    cmp     [es:di+4], dl ; [es:di+BDS.drivenum]
17807 000017DB 7409        jz      short fbds_2
17808 000017DD 26C43D      les     di, [es:di] ; [es:di+BDS.link]
17809          ; go to next bds
17810 000017E0 83FFFF      cmp     di, 0FFFFh
17811 000017E3 75F2        jnz     short fbds_1
17812 000017E5 F9          stc
17813
fbds_2:
17814 000017E6 C3          retn
17815
17816          ; ===== S U B R O U T I N E =====
17817
17818          ; 16/10/2022
17819          ; 17/10/2022
17820
doint:
17821          ; 10/12/2022
17822 000017E7 8A5608      mov     dl, [bp+8] ; [bp+INT13FRAME.olddx]
17823          ; get physical drive number
17824          ; 19/10/2022 - Temporary !
17825          ; db 8Ah, 96h, 8, 0 ; mov dl, [bp+8]
17826
17827 000017EA 30E4        xor     ah, ah
17828 000017EC 08C0        or      al, al
17829 000017EE 7410        jz      short dointdone ; if zero sectors, return ax=0
17830          ; 10/12/2022
17831 000017F0 8A6603      mov     ah, [bp+3] ; [bp+INT13FRAME.olddx+1]
17832          ; get request code
17833          ; db 8Ah, 0A6h, 3, 0 ; mov ah, [bp+3]
17834 000017F3 FF7610    push    word [bp+10h] ; [bp+INT13FRAME.olddx]
17835          ; db 0FFh, 0B6h, 10h, 0 ; push word [bp+10h]
17836 000017F6 9D          popf
17837          ; call far 70h:797h ; MSDOS 6.21 IO.SYS BIOSCODE:14EAh
17838          ; 17/10/2022
17839 000017F7 9A[0B07]7000    call    DOSBIOSSEG:call_orig13
17840          ; call call_orig13 ; call far 70h:797h
17841          ; call far KERNEL_SEGMENT:call_orig13
17842 000017FC 9C          pushf
17843          ; 10/12/2022
17844 000017FD 8F4610      pop     word [bp+10h] ; [bp+INT13FRAME.olddx]
17845          ; db 8Fh, 86h, 10h, 0 ; pop word [bp+10h]
17846
dointdone:
17847 00001800 C3          retn
17848
17849          ; -----
17850
17851          ; 16/10/2022
17852
17853          ; this is the true int 13 handler. we parse the request to see if there is
17854          ; a dma violation. if so, depending on the function, we:
17855          ; read/write break the request into three pieces and move the middle one
17856          ; into our internal buffer.
17857          ;
17858          ; format copy the format table into the buffer
17859          ; verify point the transfer address into the buffer
17860          ;

```

```

17861 ; this is the biggest bogosity of all. the ibm controller does not handle
17862 ; operations that cross physical 64k boundaries. in these cases, we copy
17863 ; the offending sector into the buffer below and do the i/o from there.
17864
17865 ;struc INT13FRAME
17866 ;.oldbp: resw
17867 ;.oldax: resw
17868 ;.oldbx: resw
17869 ;.oldcx: resw
17870 ;.olddx: resw
17871 ;.oldds: resw ; now we save caller's ds, too
17872 ;.olddi: resd
17873 ;.oldf: resw
17874 ;end struc
17875
17876 ; -----
17877
17878 ; entry conditions:
17879 ; ah = function
17880 ; al = number of sectors
17881 ; es:bx = dma address
17882 ; cx = packed track and sector
17883 ; dx = head and drive
17884 ; output conditions:
17885 ; no dma violation.
17886
17887 ; use extreme caution when working with this code. In general,
17888 ; all registers are hot at all times.
17889
17890 ; question: does this code handle cases where dma errors
17891 ; occur during ecc retries, and where ecc errors occur during
17892 ; dma breakdowns???? HMMMMM.
17893
17894 ; -----
17895
17896 ; -----
17897
17898 ; 26/12/2023 - Retro DOS v5.0
17899 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1889h
17900 dtype_array:
17901 00001801 90004000 dd 400090h ; 40h:90h is drive type array addr
17902
17903 ; 17/10/2022
17904 ;DTYPEARRAY equ dtype_array - DOSBIOSSEG_2C7h ; (14F5h for MSDOS 5.0 IO.SYS)
17905 ; 09/12/2022
17906 DTYPEARRAY equ dtype_array
17907
17908 ; -----
17909
17910 ; stick some special stuff out of mainline
17911
17912 ; we know we're doing a format command. if we have changeline
17913 ; support, then flag some special changed stuff and set changed
17914 ; by format bit for all logical drives using this physical drive
17915
17916 format_special_stuff:
17917 cmp byte [fhave96], 0 ; do we have changeline support?
17918 jz short format_special_stuff_done ; brif not
17919 push bx
17920 mov bx, 140h ; fchanged_by_format+fchanged
17921 call set_changed_d1 ; indicate that media changed by format
17922 pop bx
17923 jmp short format_special_stuff_done
17924
17925 ; -----
17926
17927 ; 16/10/2022
17928
17929 ; we know we've got ec35's on the system. Now see if we're doing
17930 ; a floppy. If so, create a mask and see if this particular
17931 ; drive is an ec35. If so, set dtype_array[drive]=93h
17932
17933 ; 19/10/2022
17934 ec35_special_stuff:
17935 test dl, dl ; floppy or hard disk?
17936 js short ec35_special_stuff_done ; if harddrive, we're done
17937 push ax ; see if this PARTICULAR drive is ec35
17938 push cx
17939 mov cl, dl ; turn drive number into bit map
17940 mov al, 1 ; assume drive 0
17941 shl al, cl ; shiftover correct number of times
17942 test [ec35flag], al ; electrically compatible 3.5 incher?
17943 pop cx
17944 pop ax
17945 jz short ec35_special_stuff_done
17946 ; done if this floppy is not an ec35
17947 push bx ; free up a far pointer (es:bx)
17948 push es
17949 ; 17/10/2022
17950 les bx, [cs:DTYPEARRAY]
17951 ;les bx, dword ptr cs:DTYPEARRAY ; [cs:dtype_array]
17952 ; 0070h:3A65h = 2C7h:14F5h
17953 add bl, dl
17954 adc bh, 0 ; find entry for this drive
17955 mov byte [es:bx], 93h ; establish drive type as:
17956 ; (360k disk in 360k drive,
17957 ; no double-stepping, 250 kbs transfer rate)
17958 pop es
17959 pop bx
17960 jmp short ec35_special_stuff_done
17961
17962 ; -----
17963
17964 ; 16/10/2022
17965
17966 ; ps2_30 machine has some problem with ah=8h (read drive parm), int 13h.
17967 ; this function does not reset the common buses after the execution.
17968 ; to solve this problem, when we detect ah=8h, then we will save the result and
17969 ; will issue ah=1 (read status) call to reset the buses.
17970
17971 ps2_special_stuff:
17972 cmp byte [prevoper], 8 ; (ps2_30)
17973 ; read driver parm ?
17974 jz short ps2_30_problem
17975 cmp byte [prevoper], 15h
17976 ; apparently function 15h fails, too
17977 jnz short ps2_special_stuff_done
17978 ps2_30_problem:
17979 push ax
17980 mov ah, 1
17981 ; 26/12/2023
17982 ; call 70h:70Bh ; PCDOS 7.1 IBMBIO.COM BIOSCODE:18D7h
17983 ; ; call BIOSDATA:call_orig13
17984 ; ; call far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:1543h
17985 ; 17/10/2022
17986 call DOSBIOSSEG:call_orig13

```

```

17985             ;call call_orig13 ; call far 70:797h
17986             ; call far KERNEL_SEGMENT:call_orig13
17987 00001854 58      pop     ax
17988 00001855 EB22     jmp     short ps2_special_stuff_done
17989 ; -----
17990 ; 17/10/2022
17991 ; 16/10/2022
17992
17993 ; here is the actual int13 handler
17994
17995 i13z:             ; 0070h:3ABh = 02C7h:154Bh
17996
17997 ; cas -- inefficient! could push ds and load ds-> Bios_Data before
17998 ; vectoring up here from Bios_Data
17999
18000             ; 19/10/2022
18001             push    ds ; save caller's ds register first thing
18002 00001857 1E      ;mov    ds, word [cs:0030h]
18003             ; and set up our own ds -> Bios_Data
18004             mov     ds, [cs:BIOSDATAWORD]
18005 00001858 2E8E1E[3000] ;mov    ds, word ptr cs:BIOSDATAWORD ; [cs:0030h]
18006             ; = [02C7h:0030h] = [0070h:25A0h]
18007
18008 ; let the operation proceed. if there is a dma violation, then we do things
18009
18010             mov     [prevoper], ax ; save request
18011 0000185D A3[1E00]  cmp     ah, 5 ; romformat
18012 00001860 80FC05   jz      short format_special_stuff
18013 00001863 74A0     ; go do special stuff for format
18014
18015 format_special_stuff_done:
18016 00001865 803E[A204]00 cmp     byte [ec35flag], 0 ; any electrically compat 3.5 inchers?
18017 0000186A 75AA     jnz     short ec35_special_stuff
18018             ; go handle it out of line if so
18019
18020 ec35_special_stuff_done:
18021             ; 26/12/2023
18022             ;call    70h:70Bh ; PCDOS 7.1 IBMBIO.COM BIOSCODE:18EDh
18023 0000186C 9A[0B07]7000 ;call    far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:1560h
18024             call    DOSBIOSSEG:call_orig13
18025             ;call    call_orig13 ; call far KERNEL_SEGMENT:call_orig13
18026 00001871 9C      pushf ; save result flags
18027
18028 00001872 803E[AF05]FA cmp     byte [model_byte], 0FAh ; is this a ps2/30?
18029             ; mdl_ps2_30
18030 00001877 74C5     jz      short ps2_special_stuff
18031             ; exit mainline to address special
18032             ; ps2/30 problem if so
18033
18034 ps2_special_stuff_done:
18035             popf
18036             jb      short goterr13 ; error on original orig13 call-thru?
18037
18038 ret_from_i13:
18039             pop     ds
18040             retf    2 ; restore ds & iret w/flags
18041 ; -----
18042 ; most of our code exits through here. If carry isn't set, then
18043 ; just do a simple exit. Else doublecheck that we aren't getting
18044 ; a changeline error.
18045
18046 i13ret_ck_chglinerr:
18047             jnb     short ret_from_i13 ; done if not an error termination
18048
18049 i13_ret_error:
18050             cmp     ah, 6 ; did i see a change event?
18051             jnz     short int13b ; skip if wrong error
18052             or      dl, dl ; is this for the hard disk?
18053             js      short int13b ; yes, ignore
18054             cmp     byte [fhav96], 0
18055             jz      short int13b ; just in case ROM returned this
18056             ; error even though it told us it
18057             ; never would
18058             push    bx
18059             mov     bx, 40h ; fchanged
18060             call    set_changed_dl
18061             pop     bx
18062
18063 int13b:
18064             stc ; now return the error
18065             jmp     short ret_from_i13
18066 ; -----
18067 ; some kind of error occurred. see if it is dma violation
18068
18069 goterr13:
18070             cmp     ah, 9 ; dma error?
18071             jz      short gotdmaerr
18072
18073 goterr13_xxxx:
18074             cmp     ah, 11h ; ecc error?
18075             jnz     short i13_ret_error ; other error. just return back.
18076             cmp     byte [media_set_for_format], 1 ; formatting?
18077             jz      short i13_ret_error
18078
18079             cmp     byte [prevoper+1], 2
18080             ;cmp     byte ptr ds:prevoper+1, 2 ; ecc-corrected error
18081             ; (2 = romread)
18082             ; ECC correction only applies to reads
18083             jnz     short i13_ret_error
18084
18085             xor     ah, ah
18086             ;call    far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:15ABh
18087             ; 17/10/2022
18088             call    DOSBIOSSEG:call_orig13
18089             ;call    call_orig13 ; call far KERNEL_SEGMENT:call_orig13
18090             ; call far 70:797h
18091             mov     ax, [prevoper]
18092             xor     ah, ah ; return code = no error
18093             cmp     al, 1 ; if request for one sector, assume ok
18094             jz      short ret_from_i13 ; return with carry clear
18095             push    bx
18096             push    cx
18097             push    dx
18098             mov     [number_of_sec], al
18099
18100 loop_ecc:
18101             mov     ax, 201h ; read one sector
18102
18103 ; we do reads one sector at a time. this ensures that we will eventually
18104 ; finish the request since ecc errors on one sector do read in that sector.
18105 ;
18106 ; we need to put in some "intelligence" into the ecc handler to handle reads
18107 ; that attempt to read more sectors than are available on a particular
18108 ; track.
18109 ;
18110 ; we call check_wrap to set up the sector #, head # and cylinder # for
18111 ; this request.
18112 ;
18113 ; at this point, all registers are set up for the call to orig13, except

```

```

18109 ; that there may be a starting sector number that is bigger than the number
18110 ; of sectors on a track.
18111 ;
18112 000018CE E88FFE      call    check_wrap      ; get correct parameters for int 13
18113                      ; call far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:15C5h
18114                      ; 17/10/2022
18115 000018D1 9A[0B07]7000 call    DOSBIOSSEG:call_orig13
18116                      ; call call_orig13 ; call far KERNEL_SEGMENT:call_orig13
18117 000018D6 730C      jnb     short ok11_op      ; DMA error during ECC read?
18118 000018D8 80FC09    cmp     ah, 9
18119 000018DB 741B      jz      short handle_dma_during_ecc
18120 000018DD 80FC11    cmp     ah, 11h ; only allow ecc errors
18121 000018E0 7510      jnz     short ok11_exit_err
18122                      ; 10/12/2022
18123                      ; xor ax ax -> ah = 0
18124                      ; mov     ah, 0 ; ecc error. reset the system again.
18125 000018E2 31C0      xor     ax, ax ; clear the error code so that if this
18126                      ; was the last sector, no error code
18127                      ; will be returned for the corrected
18128                      ; read. (clear carry too.)
18129
18130 000018E4 FE0E[2000] ok11_op: dec     byte [number_of_sec]
18131 000018E8 7409      jz      short ok11_exit ; all done?
18132 000018EA FEC1      inc     cl ; advance sector number
18133                      ; add 200h to address
18134 000018EC FEC7      inc     bh
18135 000018EE FEC7      inc     bh
18136 000018F0 EBD9      jmp     short loop_ecc
18137
18138 ; -----
18139 ; locate error returns centrally
18140
18141 ok11_exit_err:
18142 000018F2 F9          stc ; set carry bit again.
18143
18144 000018F3 5A          ok11_exit: pop     dx
18145 000018F4 59          pop     cx
18146 000018F5 5B          pop     bx
18147 000018F6 EB88      jmp     short i13ret_ck_chglinerr
18148
18149 ; -----
18150 ; do the single sector read again, this time into our temporary
18151 ; buffer, which is guaranteed not to have a DMA error, then
18152 ; move the data to its proper location and proceed
18153
18154 handle_dma_during_ecc:
18155 000018F8 06          push    es
18156 000018F9 53          push    bx
18157 000018FA BB[5201]   mov     bx, disksector
18158 000018FD 1E          push    ds
18159 000018FE 07          pop     es ; point es:bx to buffer
18160 000018FF B80102    mov     ax, 201h ; read one sector
18161                      ; call far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:15F8h
18162                      ; 17/10/2022
18163 00001902 9A[0B07]7000 call    DOSBIOSSEG:call_orig13
18164                      ; call call_orig13 ; call far KERNEL_SEGMENT:call_orig13
18165 00001907 5B          pop     bx
18166 00001908 07          pop     es
18167 00001909 7305      jnb     short handle_dma_during_ecc_noerr
18168 0000190B 80FC11    cmp     ah, 11h
18169 0000190E 75E2      jnz     short ok11_exit_err ; if anything but ecc error, bomb out
18170
18171 ; now we're kosher. Copy the data to where it belongs and resume
18172 ; the ECC looping code.
18173
18174 handle_dma_during_ecc_noerr:
18175 00001910 56          push    si
18176 00001911 57          push    di
18177 00001912 89DF      mov     di, bx
18178 00001914 BE[5201]   mov     si, disksector
18179 00001917 E82BFE    call    move_sector
18180 0000191A 5F          pop     di
18181 0000191B 5E          pop     si
18182 0000191C EBC6      jmp     short ok11_op
18183
18184 ; -----
18185 ; we truly have a dma violation. restore register ax and retry the
18186 ; operation as best we can.
18187
18188 goddmaerr:
18189 0000191E A1[1E00]   mov     ax, [prevoper] ; 19/10/2022
18190 00001921 FB          sti
18191 00001922 80FC02    cmp     ah, 2 ; romread
18192 00001925 723B      jb      short i13_done_dmaerr
18193                      ; just pass dma error thru for
18194                      ; functions we don't handle
18195 00001927 80FC04    cmp     ah, 4 ; romverify
18196 0000192A 743C      jz      short intverify
18197 0000192C 80FC05    cmp     ah, 5 ; romformat
18198 0000192F 7448      jz      short intformat
18199 00001931 772F      ja      short i13_done_dmaerr
18200
18201 ; we are doing a read/write call. check for dma problems
18202
18203 ; ***** set up stack frame here!!! *****
18204
18205 00001933 52          push    dx
18206 00001934 51          push    cx
18207 00001935 53          push    bx
18208 00001936 50          push    ax
18209 00001937 55          push    bp
18210 00001938 89E5      mov     bp, sp
18211 0000193A 8CC2      mov     dx, es ; check for 64k boundary error
18212                      ; 26/12/2023
18213                      ; add     dx, dx
18214                      ; add     dx, dx
18215                      ; add     dx, dx
18216                      ; add     dx, dx ; dx = dx*16
18217 0000193C D1E2      shl     dx, 1
18218 0000193E D1E2      shl     dx, 1
18219 00001940 D1E2      shl     dx, 1
18220 00001942 D1E2      shl     dx, 1 ; segment converted to absolute address
18221 00001944 01DA      add     dx, bx ; combine with offset
18222 00001946 81C2FF01 add     dx, 511 ; simulate a transfer
18223
18224 ; if carry is set, then we are within 512 bytes of the end of the segment.
18225 ; we skip the first transfer and perform the remaining buffering and transfer
18226
18227 0000194A 7303      jnb     short no_skip_first
18228 0000194C E98300    jmp     bufferx ; restore dh=head & do buffer
18229
18230 ; -----
18231 no_skip_first:
18232 0000194F D0EE      shr     dh, 1 ; dh = number of sectors before address

```

```

18233 00001951 B480          mov     ah, 128      ; ah = max number of sectors in segment
18234 00001953 28F4          sub      ah, dh
18235
18236          ; ah is now the number of sectors that we can successfully write in this
18237          ; segment. if this number is above or equal to the requested number, then we
18238          ; continue the operation as normal. otherwise, we break it into pieces.
18239
18240          ; wait a sec. this is goofy. the whole reason we got here in the
18241          ; first place is because we got a dma error. so it's impossible
18242          ; for the whole block to fit, unless the dma error was returned
18243          ; in error.
18244
18245 00001955 38C4          cmp      ah, al      ; can we fit it in?
18246 00001957 7236          jb      short doblock ; no, perform blocking.
18247
18248          ; yes, the request fits. let it happen.
18249
18250 00001959 8A7609          mov     dh, [bp+9]      ; [bp+INT13FRAME.olddx+1]
18251                                ; set up head number
18252 0000195C E888FE          call     doint
18253 0000195F E9D900          jmp      bad13          ; and return from this place
18254
18255          ; -----
18256
18257 00001962 B409          mov     ah, 9          ; pass dma error thru to caller
18258 00001964 F9          stc
18259 00001965 E914FF          jmp      ret_from_i13   ; return with error,
18260                                ; we know it's not a changeline error
18261
18262          ; -----
18263          ; verify the given sectors. place the buffer pointer into our space.
18264
18265          intverify:
18266 00001968 06          push     es          ; save caller's dma address
18267 00001969 53          push     bx
18268 0000196A 1E          push     ds          ; es:bx -> Bios_Data:disksector
18269 0000196B 07          pop      es
18270
18271 0000196C BB[5201]          dosimple: mov     bx, disksector
18272                                ; do the i/o from Bios_Data:disksector
18273                                ; ;call far 70:797h ; MSDOS 6.21 IO.SYS BIOSCODE:1665h
18274                                ; ; 17/10/2022
18275 0000196F 9A[0B07]7000          call     DOSBIOSSEG:call_orig13
18276                                ; call far KERNEL_SEGMENT:call_orig13
18277 00001974 5B          pop      bx
18278 00001975 07          pop      es
18279 00001976 E907FF          jmp      i13ret_ck_chglinerr
18280
18281          ; -----
18282          ; format operation. copy the parameter table into Bios_Data:disksector
18283
18284          intformat:
18285 00001979 06          push     es
18286 0000197A 53          push     bx
18287 0000197B 56          push     si
18288 0000197C 57          push     di
18289 0000197D 1E          push     ds
18290
18291          ; point ds to the caller's dma buffer, es to Bios_Data
18292          ; in other words, swap (ds, es)
18293
18294 0000197E 06          push     es
18295 0000197F 1E          push     ds
18296 00001980 07          pop      es
18297 00001981 1F          pop      ds
18298 00001982 89DE          mov     si, bx
18299 00001984 BF[5201]          mov     di, disksector
18300 00001987 E8BBFD          call     move_sector    ; user's data into Bios_Data:disksector
18301 0000198A 1F          pop      ds
18302 0000198B 5F          pop      di
18303 0000198C 5E          pop      si          ; do the i/o from
18304 0000198D EBDD          jmp      short dosimple ; Bios_Data:disksector
18305
18306          ; -----
18307          ; we can't fit the request into the entire block. perform the operation on
18308          ; the first block.
18309
18310          ; doblock is modified to correctly handle multi-sector disk i/o.
18311          ; old doblock had added the number of sectors i/oed (ah in old doblock) after
18312          ; the doint call to cl. observing only the lower 6 bits of cl(=max. 64) can
18313          ; represent a starting sector, if ah was big, then cl would be clobbered.
18314          ; by the way, we still are going to use cl for this purpose since checkwrap
18315          ; routine will use it as an input. to prevent cl from being clobbered, a
18316          ; safe number of sectors should be calculated like "63 - # of sectors/track".
18317          ; doblock will handle the first block of requested sectors within the
18318          ; boundary of this safe value.
18319
18320          ; 26/12/2023 - Retro DOS v5.0
18321          doblock:
18322
18323          ; try to get the # of sectors/track from bds via rom drive number.
18324          ; for any mini disks installed, here we have to pray that they have the
18325          ; same # of sector/track as the main dos partition disk drive.
18326
18327 0000198F 8B5608          mov     dx, [bp+8]      ; [bp+INT13FRAME.olddx]
18328                                ; get head #, drive #
18329
18330 00001992 51          push     cx
18331 00001993 06          push     es
18332 00001994 57          push     di          ; ah - # of sectors before dma boundary
18333                                ; al - requested # of sectors for i/o.
18334 00001995 E83BFE          call     find_bds
18335 00001998 268B4D13          mov     cx, [es:di+13h] ; [es:di+BDS.secperttrack]
18336                                ; 26/12/2023
18337 0000199C 26F6453F01          test     byte [es:di+3Fh], 1
18338                                ; 12/12/2022
18339                                ; test byte [es:di+23h], 1
18340                                ; ;test word [es:di+23h], 1 ; [es:di+BDS.flags],fnon_removable
18341 000019A1 5F          pop      di
18342 000019A2 07          pop      es
18343 000019A3 88E0          mov     al, ah          ; set al=ah for floppies
18344 000019A5 7404          jz      short doblockflop ; they are track by track operation
18345 000019A7 B43F          mov     ah, 63          ; ah = 63-secpt (# safe sectors??)
18346 000019A9 28CC          sub     ah, cl          ; al - # of sectors before dma boundary
18347
18348          doblockflop:
18349 000019AB 59          pop      cx
18350          doblockcontinue:
18351 000019AC 38C4          cmp     ah, al          ; if safe_# >= #_of_sectors_to_go_before dma,
18352 000019AE 7305          jnb     short doblocklast ; then #_of_sectors_to_go as it is for doint.
18353 000019B0 50          push     ax
18354 000019B1 88E0          mov     al, ah          ; otherwise, set al to ah to operate.
18355 000019B3 EB03          jmp     short doblockdoint
18356
18357          ; -----
18358          doblocklast:

```



```

18357 000019B5 88C4      mov     ah, al
18358 000019B7 50      push    ax
18359                      ; let ah = al =      # of sectors for this shot
18360 000019B8 E82CFE    doblockdoint: call    doint
18361 000019BB 727E      jb     short bad13    ; something happened, bye!
18362 000019BD 58      pop     ax
18363 000019BE 286602    sub     [bp+2], ah     ; sub [bp+INT13FRAME.oidax], ah
18364                      ; decrement by the successful operation
18365 000019C1 00E1      add     cl, ah         ; advance sector #. safety gauranteed.
18366 000019C3 00E7      add     bh, ah         ; advance dma address
18367 000019C5 00E7      add     bh, ah         ; twice for 512 byte sectors
18368 000019C7 38C4      cmp     ah, al         ; check the previous value
18369 000019C9 740A      jz      short buffer   ; if #_of_sectors_to_go < safe_#,
18370                      ; then we are done already.
18371 000019CB 28E0      sub     al, ah         ; otherwise,
18372                      ; #_sector_to_go = #_of_sector_to_go - safe_#
18373 000019CD E890FD    call    check_wrap     ; get new cx, dh for the next operation.
18374 000019D0 EBDA      jmp     short doblockcontinue ; handles next sectors left.
18375                      ; -----
18376
18377
18378 000019D2 8A7609    bufferx:  mov     dh, [bp+9]    ; [bp+INT13FRAME.olddx+1]
18379                      ; set up head number
18380
18381 000019D5 53      buffer:  push     bx
18382 000019D6 8A6603    mov     ah, [bp+3]     ; [bp+INT13FRAME.oidax+1]
18383 000019D9 80FC03    cmp     ah, 3          ; romwrite
18384 000019DC 7525      jnz     short doread   ;
18385
18386                      ; copy the offending sector into local buffer
18387
18388 000019DE 06      push     es
18389 000019DF 1E      push     ds
18390 000019E0 56      push     si
18391 000019E1 57      push     di
18392 000019E2 1E      push     ds            ; exchange segment registers
18393 000019E3 06      push     es
18394 000019E4 1F      pop      ds
18395 000019E5 07      pop      es
18396 000019E6 BF[5201]  mov     di, disksector ; where to move
18397 000019E9 57      push     di            ; save it
18398 000019EA 89DE      mov     si, bx         ; source
18399 000019EC E856FD    call    move_sector    ; move sector into local buffer
18400 000019EF 5B      pop      bx           ; new transfer address
18401                      ; (es:bx = Bios_Data:diskbuffer)
18402 000019F0 5F      pop      di            ; restore caller's di & si
18403 000019F1 5E      pop      si
18404 000019F2 1F      pop      ds            ; restore Bios_Data
18405
18406                      ; see if we are wrapping around a track or head
18407
18408 000019F3 B001      mov     al, 1          ; [bp+INT13FRAME.olddx]
18409                      ; get drive number
18410 000019F5 8A5608    mov     dl, [bp+8]
18411 000019F8 E865FD    call    check_wrap     ; sets up registers if wrap-around
18412                      ;
18413                      ; ah is function
18414                      ; al is 1 for single sector transfer
18415                      ; es:bx is local transfer address
18416                      ; cx is track/sector number
18417                      ; dx is head/drive number
18418                      ; si, di unchanged
18419 000019FB E8E9FD    call    doint
18420 000019FE 07      pop      es            ; restore caller's dma segment
18421 000019FF 723A      jb     short bad13     ; go clean up
18422 00001A01 EB22      jmp     short dotail
18423                      ; -----
18424
18425                      ; reading a sector. do int first, then move things around
18426
18427
18428 00001A03 06      doread:  push     es
18429 00001A04 53      push     bx
18430 00001A05 1E      push     ds            ; es = Bios_Code
18431 00001A06 07      pop      es
18432 00001A07 BB[5201]  mov     bx, disksector
18433 00001A0A B001      mov     al, 1
18434 00001A0C 8A5608    mov     dl, [bp+8]     ; [bp+INT13FRAME.olddx]
18435                      ; get drive number
18436 00001A0F E84EFD    call    check_wrap     ;
18437                      ; ah = function
18438                      ; al = 1 for single sector
18439                      ; es:bx points to local buffer
18440                      ; cx, dx are track/sector, head/drive
18441 00001A12 E8D2FD    call    doint
18442 00001A15 5B      pop      bx
18443 00001A16 07      pop      es
18444 00001A17 7222      jb     short bad13
18445 00001A19 56      push     si
18446 00001A1A 57      push     di
18447 00001A1B 89DF      mov     di, bx
18448 00001A1D BE[5201]  mov     si, disksector
18449 00001A20 E822FD    call    move_sector
18450 00001A23 5F      pop      di
18451 00001A24 5E      pop      si
18452
18453                      ; note the fact that we've done 1 more sector
18454
18455
18456 00001A25 5B      dotail:  pop      bx            ; retrieve new dma area
18457 00001A26 80C702    add     bh, 2          ; advance over sector
18458 00001A29 41      inc     cx
18459 00001A2A 8A4602    mov     al, [bp+2]     ; [bp+INT13FRAME.oidax]
18460 00001A2D F8      clc
18461 00001A2E FEC8      dec     al
18462 00001A30 7409      jz      short bad13    ; no more i/o
18463
18464                      ; see if we wrap around a track or head boundary with starting sector
18465                      ; we already have the correct head number to pass to check_wrap
18466
18467 00001A32 8A5608    mov     dl, [bp+8]     ; [bp+INT13FRAME.olddx]
18468 00001A35 E828FD    call    check_wrap
18469 00001A38 E8ACFD    call    doint
18470
18471                      ; we are done. ax has the final code; we throw away what we got before
18472
18473                      ; M046 -- okay gang. Now we've either terminated our DMA loop,
18474                      ; or we've finished. If carry is set now, our only
18475                      ; hope for salvation is that it was a read operation
18476                      ; and the error code is ECC error. In that case, we'll
18477                      ; just pop the registers and go do the old ECC thing.
18478                      ; when the DMA error that got us here in the first
18479                      ; place occurs, it'll handle it.
18480

```

```

18481
18482 00001A3B 89EC
18483 00001A3D 5D
18484 00001A3E 5B
18485 00001A3F 5B
18486 00001A40 59
18487 00001A41 5A
18488 00001A42 7203
18489 00001A44 E935FE
18490
18491
18492
18493 00001A47 E958FE
18494
18495
18496
18497
18498
18499
18500
18501
18502
18503
18504
18505
18506
18507
18508
18509
18510
18511 00001A4A 8A26[7500]
18512 00001A4E BF[3C05]
18513 00001A51 1E
18514 00001A52 07
18515 00001A53 E934EC
18516
18517
18518
18519
18520
18521
18522
18523
18524
18525
18526
18527
18528 00001A56 1E
18529 00001A57 BE[1901]
18530
18531
18532
18533
18534 00001A5A C534
18535
18536 00001A5C 268A4504
18537 00001A60 384404
18538
18539 00001A63 7518
18540 00001A65 B310
18541
18542 00001A67 26085D3F
18543
18544
18545 00001A6B 085C3F
18546
18547 00001A6E 2680653FDF
18548
18549
18550 00001A73 8A5C3F
18551
18552
18553 00001A76 80E302
18554 00001A79 26085D3F
18555
18556
18557
18558 00001A7D B8FFFF
18559 00001A80 3904
18560
18561
18562 00001A82 75D6
18563 00001A84 8C4402
18564
18565 00001A87 893C
18566 00001A89 268905
18567
18568
18569 00001A8C 1F
18570
18571
18572
18573
18574
18575
18576
18577
18578 00001A8D 268A4550
18579
18580
18581
18582
18583 00001A91 3A06[2C01]
18584 00001A95 7603
18585 00001A97 A2[2C01]
18586
18587 00001A9A C3
18588
18589
18590
18591
18592
18593
18594
18595
18596
18597
18598
18599
18600
18601
18602
18603
18604

bad13:
        mov     sp, bp
        pop     bp
        pop     bx
        pop     bx
        pop     cx
        pop     dx
        jnb     short xgoterr13_xxxx ; go handle ECC errors
        jmp     ret_from_i13 ; non-error exit
; -----
xgoterr13_xxxx:
        jmp     goterr13_xxxx
; -----
;         ; 10/12/2022
;         ;db     0
; -----
;Bios_Code ends
; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; -----
; MSBIO2.ASM - MSDOS 6.0 - 1991
; -----
; 17/03/2019 - Retro DOS v4.0
; -----
; 19/10/2022
dsk_init:
        mov     ah, [drvmax] ; 2C7h:1742h = 70h:3CB2h
        mov     di, dskdrvs
        push    ds ; pass result in es:di
        pop     es
        jmp     SetPtrSav
; ===== S U B R O U T I N E =====
; -----
; install_bds installs a bds at location es:di into the current linked list of
; bds maintained by this device driver. it places the bds at the end of the
; list. Trashes (at least) ax, bx, di, si
; -----
; 26/12/2023 - Retro DOS v5.0
; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1AE0h
install_bds:
        push    ds ; save Bios_Data segment
        mov     si, start_bds ; beginning of chain
; ds:si now points to link to first bds
; assume bds list is non-empty
loop_next_bds:
        lds     si, [si] ; [si+BDS.link]
; fetch next bds
        mov     al, [es:di+4] ; [es:di+BDS.drivenum]
        cmp     [si+4], al ; does this one share a physical
; drivewith new one?
        jnz     short next_bds
        mov     bl, 10h ; fi_am_mult
; 26/12/2023
        or      [es:di+3Fh], bl ; [es:di+BDS.flags]
        or      [es:di+23h], bl ; set both of them to i_am_mult if so
        or      [si+3Fh], bl ; [si+BDS.flags]
        or      [si+23h], bl ; [si+BDS.flags]
        and     byte [es:di+3Fh], 0DFh ; [es:di+BDS.flags], ~fi_own_physical
        and     byte [es:di+23h], 0DFh ; we don't own it
        mov     bl, [si+3Fh]
        mov     bl, [si+23h] ; [si+BDS.flags]
; determine if changeline available
        and     bl, 2 ; fchangeline
        or      [es:di+3Fh], bl
        or      [es:di+23h], bl ; [es:di+BDS.flags]
next_bds:
; 02/09/2023 (PCDOS 7.1)
        mov     ax, 0FFFFh ; -1
        cmp     [si], ax ; [si+BDS.link], -1
        cmp     word [si], 0FFFFh ; [si+BDS.link], -1
; are we at end of list?
        jnz     short loop_next_bds
        mov     [si+2], es ; [si+BDS.link+2], es
; install bds
        mov     [si], di
        mov     [es:di], ax ; [es:di+BDS.link], -1
        mov     word [es:di], 0FFFFh ; [es:di+BDS.link], -1
; set next pointer to null
        pop     ds
; 01/07/2023 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS - BIOSCODE:1785h)
; 16/10/2022 (MSDOS 6.0 Code)
; **** If the new drive has a higher EOT value, we must alter the
; 'eot' variable appropriately.
; 26/12/2023
        mov     al, [es:di+50h] ; [es:di+BDS.rsecpertrack]
; 01/06/2019
        mov     al, [es:di+52]
; 22/07/2023
        mov     al, [es:di+BDS.rsecpertrack]
        cmp     al, [eot]
        jbe     short _eot_ok
        mov     [eot], al
_eot_ok:
        retn
; -----
; 17/10/2022
; DRVLET equ drvlet - DOSBIOSEG_2C7h
; SNGMSG equ sngmsg - DOSBIOSEG_2C7h
; 09/12/2022
DRVLET equ drvlet
SNGMSG equ sngmsg
; 16/10/2022
; -----
; ask to swap the disk in drive a:
; es:di -> bds
; ds -> Bios_Data
; -----

```



```

18605
18606 ; 26/12/2023 - Retro DOS v5.0
18607
18608 ; 19/10/2022
18609 00001A9B F606[1208]01 swpdsk: test byte [IsWin386], 1
18610 ;test ds:IsWin386, 1 ; Is win386 present?
18611 00001AA0 7405 jz short no_win386 ; no, skip SetFocus
18612
18613 ; set focus to the correct VM
18614 ;call far ptr 70h:813h ; PCDOS 7.1 IBMBIO.COM BIOSCODE:1B2Ch
18615 ;;call far 70h:8D1h ; MSDOS 6.21 IO.SYS BIOSCODE:179Ah
18616 ; 17/10/2022
18617 00001AA2 9A[1308]7000 call DOSBIOSSEG:V86_Crit_SetFocus ; BIOSDATA:V86_Crit_SetFocus
18618 ;call far ptrV86_Crit_SetFocus ; call far 70h:8D1h
18619 ; ; call far KERNEL_SEGMENT:V86_Crit_SetFocus
18620
18621 00001AA7 51 no_win386: push cx
18622 00001AA8 52 push dx
18623 00001AA9 268A5505 mov dl, [es:di+5] ; [es:di+BDS.drivelet]
18624 ; get the drive letter
18625
18626 ; WARNING : next two instructions assume that if the new disk is for drive B
18627 ; then existing dsk is drive A & vice versa
18628
18629 00001AAD 88D6 mov dh, dl
18630 00001AAF 80F601 xor dh, 1
18631 00001AB2 29C9 sub cx, cx ; nobody has handled swap disk
18632 00001AB4 B8004A mov ax, 4A00h ; multMULT<<8)|multMULTSWPSK
18633 ; broadcast code for swap disk
18634 ; Broadcast it
18635 00001AB7 CD2F int 2Fh
18636 00001AB9 41 inc cx ; cx == -1 ?
18637 00001ABA 741E jz short swpdsk9 ; somebody has handled it
18638
18639 ; using a different drive in a one drive system so request the user change disks
18640
18641 00001ABC 80C241 add dl, 'A'
18642 ; 17/10/2022
18643 00001ABF 2E8816[F91A] mov [cs:DRVLET], dl ; "A: and press any key when ready\r\n\n"
18644 ; 16/10/2022
18645 ;;mov byte [cs:drvlet], dl
18646 ;mov byte ptr cs:17E4h, dl ; [cs:drvlet]
18647 ; 0070h:3D54h = 2C7h:17E4h
18648 00001AC4 BE[DD1A] mov si, SNGMSG ; "\r\nInsert diskette for drive "
18649 ;mov si, 17C8h ; sngmsg
18650 ; 0070h:3D38h = 2C7h:17C8h
18651 00001AC7 53 push bx
18652 00001AC8 2E cs
18653 00001AC9 AC lodsb ; get the next character of the message
18654 ;lods byte ptr cs:[si]
18655 wrmsg_loop: int 29h ; DOS 2+ internal - FAST PUTCHAR
18656 00001ACA CD29 ; AL = character to display
18657
18658 00001ACC 2E cs
18659 00001ACD AC lodsb
18660 ;lods byte ptr cs:[si] ; cs lodsb
18661 ; get the next character of the message
18662 00001ACE 08C0 or al, al
18663 00001AD0 75F8 jnz short wrmsg_loop
18664 00001AD2 E833E7 call con_flush ; flush out keyboard queue
18665 ; call rom-bios
18666 00001AD5 30E4 xor ah, ah
18667 00001AD7 CD16 int 16h ; KEYBOARD - READ CHAR FROM BUFFER, WAIT IF EMPTY
18668 ; Return: AH = scan code, AL = character
18669 00001AD9 5B pop bx
18670
18671 00001ADA 5A swpdsk9: pop dx
18672 00001ADB 59 pop cx
18673 00001ADC C3 retn
18674
18675 ; -----
18676 ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
18677
18678 ; -----
18679 ; include msbio.c12 (MSDOS 6.0, 1991)
18680 ; -----
18681 ; (MSDOS 6.21 IO.SYS BIOSCODE:17D5h)
18682 ; -----
18683 ; 17/03/2019 - Retro DOS v4.0
18684 ; 26/12/2023 - Retro DOS v5.0
18685
18686 ; MSDOS 5.0 IO.SYS offset 0070h:3D38h or 02C7h:17C8h
18687 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1B67h
18688
18689 00001ADD 0D0A sngmsg: db 0Dh,0Ah
18690 00001ADF 496E73657274206469- db 'Insert diskette for drive '
18691 00001AE8 736B6574746520666F-
18692 00001AF1 7220647269766520
18693
18694 ; MSDOS 5.0 IO.SYS offset 0070h:3D54h or 02C7h:17E4h
18695 00001AF9 413A20616E64207072- drvlet: db 'A: and press any key when ready',0Dh,0Ah
18696 00001B02 65737320616E79206B-
18697 00001B0B 6579207768656E2072-
18698 00001B14 656164790D0A
18699 00001B1A 0A00 db 0Ah,0
18700
18701 ; ===== S U B R O U T I N E =====
18702 ; -----
18703 ; input : es:di points to current bds for drive.
18704 ; return : zero set if no open files
18705 ; zero reset if open files
18706 ; -----
18707 ; 26/12/2023 - Retro DOS v5.0
18708
18709 00001B1C 26837D3C00 chkopcnt: cmp word [es:di+3Ch], 0
18710 ;cmp word [es:di+20h], 0 ; [es:di+BDS.opcnt]
18711 retn
18712
18713 ; ===== S U B R O U T I N E =====
18714 ; -----
18715 ; at media check time, we need to really get down and check what the change is.
18716 ; this is guaranteed to be expensive.
18717 ;
18718 ; es:di -> bds, ds -> Bios_Data
18719 ; -----
18720 ; 26/12/2023 - Retro DOS v5.0
18721 ; PCDOS 7.1 IBMBIO.COM - BIOSCODE:1BA6h
18722
18723 00001B22 E852EE mediacheck: call checksingle ; make sure correct disk is in place

```

```

18724 00001B25 31F6          xor     si, si
18725 00001B27 E86101        call    haschange
18726 00001B2A 742F          jz      short mediaret
18727                          ; 26/12/2023
18728                          ; test byte [es:di+3Fh], 40h ; [es:di+BDS.flags], fchanged ; 40h
18729 00001B2C E85001        call    checkromchange
18730 00001B2F 752B          jnz     short mediadovolid
18731 00001B31 50             push    ax
18732 00001B32 52             push    dx
18733 00001B33 268A5504        mov     dl, [es:di+4] ; [es:di+BDS.drivenum]
18734                          ; set logical drive number
18735 00001B37 B416          mov     ah, 16h
18736 00001B39 CD13          int     13h
18737                          ; DISK - FLOPPY          DISK - CHANGE OF DISK STATUS (AT,XT2,XT286,CONV,PS)
18738                          ; DL = drive to          check
18739                          ; Return: AH = disk change status
18739 00001B3B 5A             pop     dx
18740 00001B3C 58             pop     ax
18741 00001B3D 721D          jb      short mediadovolid
18742 00001B3F BE0100        mov     si, 1 ; signal no change
18743
18744 ; there are some drives with changeline that "lose" the changeline indication
18745 ; if a different drive is accessed after the current one. in order to avoid
18746 ; missing a media change, we return an "i don't know" to dos if the changeline
18747 ; is not active and we are accessing a different drive from the last one.
18748 ; if we are accessing the same drive, then we can safely rely on the changeline
18749 ; status.
18750                          ; 19/10/2022
18751 00001B42 8A1E[1E01]    mov     bl, [tim_drv] ; get last drive accessed
18752 00001B46 26385D04        cmp     [es:di+4], bl ; [es:di+BDS.drivenum]
18753                          ; (if the last drive accessed is not current drive
18754                          ; mediachange status may be incorrect. So,
18755                          ; "I don't now" will be returned even if it is indicated
18756                          ; as media is not changed.)
18757 00001B4A 740F          jz      short mediaret ; (same drive,
18758                          ; media changeline indication is reliable)
18759
18760 ; do the 2 second twiddle. if time >= 2 seconds, do a valid check.
18761 ; otherwise return "i don't know" (strictly speaking, we should return a
18762 ; "not changed" here since the 2 second test said no change.)
18763
18764 00001B4C 50             push    ax
18765 00001B4D 51             push    cx
18766 00001B4E 52             push    dx
18767 00001B4F E8D8EA        call    Check_Time_Of_Access
18768 00001B52 5A             pop     dx
18769 00001B53 59             pop     cx
18770 00001B54 58             pop     ax
18771 00001B55 09F6          or      si, si
18772 00001B57 7403          jz      short mediadovolid ; check_time says ">= 2 secs" passed"
18773                          ; (volume id will be checked)
18774 00001B59 31F6          xor     si, si ; return "i don't know"
18775 mediaret:
18776 00001B5B C3             retn
18777
18778 ; -----
18779 ; somehow the media was changed. look at vid to see. we do not look at fat
18780 ; because this may be different since we only set medbyt when doing a read
18781 ; or write.
18782
18783 mediadovolid:
18784 00001B5C E877EB        call    GetBp ; builda new bpb in current bds
18785 00001B5F 72FA          jb      short mediaret
18786 00001B61 E82D00        call    check_vid
18787 00001B64 73F5          jnb     short mediaret
18788 00001B66 E940F2        jmp     maperror ; fix up al for return to dos
18789
18790 ; -----
18791 ; simple, quick check of latched change. if no indication, then return
18792 ; otherwise do expensive check. if the expensive test fails, pop off the
18793 ; return and set al = 15 (for invalid media change) which will be returned to
18794 ; dos.
18795 ;
18796 ; for dos 3.3, this will work only for the drive that has changeline.
18797
18798 ; call with es:di -> bds, ds -> Bios_Data
18799 ; ***** warning: this routine will return one level up on the stack
18800 ; if an error occurs!
18801
18802 checklatchio:
18803
18804 ; if returning fake bpb then assume the disk has not changed
18805
18806 ; 26/12/2023
18807 ; cmp word [es:di+3Ch], 0 ; [es:di+BDS.opcnt]
18808 00001B69 E8B0FF        call    chkopcnt
18809 00001B6C 741B          jz      short checkret ; done if zero
18810
18811 ; check for past rom indications. if no rom change indicated, then return ok.
18812
18813 ; 26/12/2023
18814 ; test word [es:di+3Fh], 40h
18815 ; ; test [es:di+BDS.flags], fchanged ; 40h
18816 00001B6E E80E01        call    checkromchange
18817 00001B71 7416          jz      short checkret
18818
18819 ; we now see that a change line has been seen in the past. let's do the
18820 ; expensive verification.
18821
18822 00001B73 E860EB        call    GetBp ; buildbpb in current bds
18823 00001B76 720F          jb      short ret_no_error_map ; getbp has already called maperror
18824 00001B78 E81600        call    check_vid
18825 00001B7B 7207          jb      short checklatchret ; disk error trying to readin.
18826 00001B7D 09F6          or      si, si ; is changed for sure?
18827 00001B7F 7908          jns     short checkret
18828 00001B81 E88F00        call    returnvid
18829
18830 checklatchret:
18831 00001B84 E822F2        call    maperror ; fix up al for return to dos
18832 ret_no_error_map:
18833 00001B87 F9             stc
18834 00001B88 5E             pop     si ; pop off return address
18835 00001B89 C3             retn
18836
18837 ; -----
18838 ; check the fat and the vid. return in di -1 or 0. return with carry set
18839 ; only if there was a disk error. return that error code in ax.
18840 ;
18841 ; called with es:di -> bds, ds -> Bios_Data
18842
18843 checkfatvid:
18844 00001B8A E8D101        call    fat_check ; check the fat and the vid
18845 00001B8D 09F6          or      si, si
18846 00001B8F 7835          js      short changed_drv
18847

```

```

18848 ; the fat was the same. fall into check_vid and check volume id.
18849
18850 ; fall into check_vid
18851
18852 ; ===== S U B   R O U T I N E =====
18853
18854 ; now with the extended boot record, the logic should be enhanced.
18855
18856 ; if it is the extended boot record, then we check the volume serial
18857 ; number instead of volume id. if it is different, then set si to -1.
18858
18859 ; if it is same, then si= 1 (no change).
18860
18861 ; if it is not the extended boot record, then just follows the old
18862 ; logic. dos 4.00 will check if the # of fat in the boot record bpb
18863 ; is not 0. if it is 0 then it must be non_fat based system and
18864 ; should have already covered by extended boot structure checking.
18865 ; so, we will return "i don't know" by setting si to 0.
18866
18867 ; this routine assume the newest valid boot record is in cs:[disksector].
18868 ; (this will be gauranteed by a successful getbp call right before this
18869 ; routine.)
18870
18871 ; called with es:di -> bds, ds -> bds
18872
18873 ; 26/12/2023 - Retro DOS v5.0
18874 ; 19/10/2022
18875
18876 check_vid:
18877
18878 ; check the disksector.EXT_BOOT_SIG variable for the extended
18879 ; boot signature. if it is set then go to do the extended
18880 ; id check otherwise continue with code below
18881
18882 ; 26/12/2023
18883 ;;;
18884 cmp word [disksector+16h], 0 ; BPB_FATSz16
18885 jnz short chk_vid_1
18886 cmp byte [disksector+42h], 29h ; BS_FAT32_BootSig
18887 jmp short chk_vid_2 ; [disksector+EXT_BOOT.SIG],EXT_BOOT_SIGNATURE
18888
18889 chk_vid_1:
18890 ;;;
18891 cmp byte [disksector+26h], 29h
18892 ; [disksector+EXT_BOOT.SIG],
18893 ; EXT_BOOT_SIGNATURE
18894
18895 chk_vid_2:
18896 ; 26/12/2023
18897 jz short do_ext_check_id
18898 call haschange
18899 jz short checkret
18900 xor si, si
18901 cmp byte [disksector+10h], 0 ; BPB_NumFATS
18902 ; [disksector+EXT_BOOT.BPB+EBPB.NUMBEROFFATS]
18903 jz short checkfatret ; don't read vol id
18904 ; if not fat system
18905 call read_volume_id
18906 jb short checkfatret
18907 call check_volume_id
18908 mov si, 0FFFFh ; -1
18909 ; definitely changed
18910 jnz short changed_drv
18911
18912 inc si ; not changed
18913
18914 vid_no_changed:
18915 call resetchanged
18916 ; 12/12/2022
18917 ; cf=0 ('and' instruction in 'resetchanged' clears cf)
18918 ;clc
18919
18920 checkfatret:
18921 retn
18922
18923 ; -----
18924
18925 ; 12/12/2022
18926
18927 changed_drv:
18928 clc ; cas -- return no error
18929 mov byte [tim_drv], 0FFh
18930 ; ensure that we ask rom for media
18931 retn ; check next time round
18932
18933 ; -----
18934
18935 ; extended id check
18936
18937 ; 16/10/2022
18938
18939 ; the code to check extended id is basically a check to see if the
18940 ; volume serial number is still the same. the volume serial number
18941 ; previously read is in cs:disksector.EXT_BOOT_SERIAL
18942 ; ds:di points to the bds of the drive under consideration.
18943 ; the bds has fields containing the high and low words
18944 ; of the volume serial number of the media in the drive.
18945 ; compare these fields to the fields mentioned above. if these fields
18946 ; do not match the media has changed and so we should jump to the code
18947 ; starting at ext_changed else return "i don't know" status
18948 ; in the register used for the changeline status and continue executing
18949 ; the code given below. for temporary storage use the register which
18950 ; has been saved and restored around this block.
18951
18952 ; bds fields in inc\msbds.inc
18953
18954 ; 26/12/2023 - Retro DOS v5.0
18955 ; 19/10/2022
18956
18957 do_ext_check_id:
18958 ; 26/12/2023
18959 ;push ax
18960 ;mov ax, word ptr ds:disksector+27h
18961 ; [DiskSector+EXT_BOOT.SERIAL]
18962 ;mov ax, [disksector+27h]
18963
18964 ; 26/12/2023
18965 %if 1
18966 ;;;
18967 push di
18968 mov si, disksector+43h ; BS_FAT32_VolID
18969 ; [DiskSector+FAT32_EXT_BOOT.SERIAL]
18970 cmp word [disksector+16h], 0 ; BPB_FATSz16
18971 jz short chk_vid_3
18972 sub si, 28 ; BS_VolID
18973 ; si = disksector+27h ; [DiskSector+EXT_BOOT.SERIAL]
18974
18975 chk_vid_3:
18976 ; [es:di+89h] = [es:di+BDS.vol_serial]
18977 add di, 137 ; BDS.vol_serial
18978 cmpsw ; [DiskSector+EXT_BOOT.SERIAL] ; (or FAT32_EXT_BOOT)
18979 ; = [di+BDS.vol_serial] ?
18980 jnz short chk_vid_4
18981 cmpsw ; [DiskSector+EXT_BOOT.SERIAL+2] ; (or FAT32_EXT_BOOT)
18982 ; = [di+BDS.vol_serial+2] ?

```

```

18972 chk_vid_4:
18973 00001BE3 5F      pop     di
18974                ;pop     ax
18975 00001BE4 7504     jnz     short ext_changed ; not equal/same
18976 00001BE6 31F6     xor     si, si          ; 0 ; don't know
18977 00001BE8 EBD8     jmp     short vid_no_changed ; reset the flag
18978                ;;;
18979 %else
18980                ; 02/09/2023
18981                xor     si, si ; 0
18982                cmp     ax, [es:di+57h] ; [di+BDS.vol_serial]
18983                jnz     short ext_changed
18984                mov     ax, [disksector+29h] ; [DiskSector+EXT_BOOT.SERIAL+2]
18985                cmp     ax, [es:di+59h] ; [di+BDS.vol_serial+2]
18986                jnz     short ext_changed
18987                xor     si, si ; 0
18988                ; don't know
18989                pop     ax
18990                jmp     short vid_no_changed
18991                ; reset the flag
18992 %endif
18993
18994 ; -----
18995
18996 ext_changed:
18997                ; 26/12/2023
18998                ;pop     ax
18999                ; 02/09/2023
19000                dec     si ; mov si, 0FFFFh ; -1
19001 00001BEA BEFFFF   mov     si, 0FFFFh ; -1
19002                ; disk changed!
19003                ; 12/12/2022
19004                ; ('changed_drv' clears cf)
19005                cllc
19006 00001BED EBD7     jmp     short changed_drv
19007
19008 ; -----
19009
19010 ; at i/o time, we detected the error. now we need to determine whether the
19011 ; media was truly changed or not. we return normally if media change unknown.
19012 ; and we pop off the call and jmp to harderr if we see an error.
19013 ;
19014 ; es:di -> bds
19015
19016 checkio:
19017 00001BEF 80FC06    cmp     ah, 6
19018 00001BF2 75D1     jnz     short checkfatret
19019 00001BF4 E825FF    call    chkopcmt
19020 00001BF7 74CC     jz      short checkfatret
19021 00001BF9 E8DAEA    call    GetBp
19022 00001BFC 7212     jb      short no_error_map
19023 00001BFE E889FF    call    checkfatvid
19024 00001C01 7209     jb      short checkioerr ; disk error trying to read in.
19025 00001C03 09F6     or      si, si          ; is changed for sure?
19026 00001C05 7802     js      short checkioerr ; yes changed
19027 00001C07 45       inc     bp              ; allow a retry
19028 00001C08 C3       retn
19029
19030 ; -----
19031
19032 00001C09 E80700    call    returnvid
19033
19034 checkioerr:
19035 00001C0C F9       stc
19036 00001C0D E955F1    jmp     harderr
19037
19038 ; -----
19039
19040 00001C10 E955F1    jmp     harderr2
19041
19042 ; ===== S U B R O U T I N E =====
19043
19044 ; return vid sets up the vid for a return to dos.
19045 ; es:di -> bds, returns pointer in packet to bds_volid
19046 ; *** trashes si! ***
19047
19048 returnvid:
19049 00001C13 BE1600    mov     si, 22          ; extra
19050                ; offset into pointer to return value
19051 00001C16 E80700    call    vid_into_packet
19052 00001C19 B406     mov     ah, 6
19053 00001C1B F9       stc
19054 00001C1C C3       retn
19055
19056 ; -----
19057
19058 ; moves the pointer to the volid for the drive into the original request packet
19059 ; no attempt is made to preserve registers.
19060 ;
19061 ; assumes es:di -> bds
19062 ; **trashes si**
19063
19064 media_set_vid:
19065 00001C1D BE0F00    mov     si, 15          ; trans+1
19066                ; return the value here in packet
19067                ; fall into vid_into_packet
19068
19069 ; ===== S U B R O U T I N E =====
19070
19071 ; return pointer to vid in bds at es:di in packet[si]
19072                ; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
19073                ; 19/10/2022
19074 vid_into_packet:
19075 00001C20 1E       push    ds              ; return pointer to vid in bds at es:di in packet[si]
19076 00001C21 C51E[1200] lds     bx, [ptrsav]
19077                ; add di, 75 ; BDS.volid
19078                ; 14/04/2024
19079                add     di, 125 ; (PCDOS 7.1)
19080                mov     [bx+si], di
19081 00001C25 83C77D    sub     di, 75          ; BDS.volid
19082 00001C28 8938     sub     di, 125
19083                mov     [bx+si+2], es
19084 00001C2A 83EF7D    pop     ds
19085 00001C2D 8C4002    pop     ds
19086 00001C30 1F       dofloppy:
19087                ; 18/12/2022
19088 00001C31 C3       retn
19089
19090 ; -----
19091
19092 ; -----
19093 ; hidensity - examine a drive/media descriptor to set the media type. if
19094 ; the media descriptor is not f9 (not 96tpi or 3 1/2), we return and let the
19095 ; caller do the rest. otherwise, we pop off the return and jump to the tail

```

```

19096 ; of getbp. for 3.5" media, we just return.
19097 ;
19098 ; inputs: es:di point to correct bds for this drive
19099 ; ah has media byte
19100 ;
19101 ; outputs:      carry clear
19102 ;             no registers modified
19103 ;             carry set
19104 ;             al = sectors/fat
19105 ;             bh = number of root directory entries
19106 ;             bl = sectors per track
19107 ;             cx = number of sectors
19108 ;             dh = sectors per allocation unit
19109 ;             dl = number of heads
19110 ;
19111 ;-----
19112 ; 26/12/2023 - Retro DOS v5.0
19113
19114 hidensity:
19115 ; check for correct drive
19116 ;
19117 ; 26/12/2023
19118 test byte [es:di+3Fh], 2 ; is it special?
19119 00001C32 26F6453F02 ; 12/12/2022
19120 ; test byte [es:di+23h], 2
19121 ; test word [es:di+23h], 2 ; is it special?
19122 ; test word [es:di+23h], 2 ; [es:di+BDS.flags], fchangeline
19123 00001C37 74F8 jz short dofloppy ; no, do normal floppy test
19124 ;
19125 ; we have a media byte that is pretty complex. examine drive information
19126 ; table to see what kind it is.
19127 ;
19128 ; 26/12/2023
19129 cmp byte [es:di+3Eh], 2 ; is it single-media?
19130 00001C39 26807D3E02 cmp byte [es:di+22h], 2 ; is it single-media?
19131 ; jz short dofloppy ; [es:di+BDS.formfactor], ffSmall
19132 00001C3E 74F1 jz short dofloppy ; yes, use fatid...
19133 ;
19134 ; 96 tpi drive?
19135 cmp ah, 0F9h
19136 00001C43 75EC jnz short dofloppy
19137 ;
19138 ;----- If formfactor of drive = ff0ther or ff288 it has to be
19139 ;----- a 720K diskette
19140 ;
19141 ; 02/09/2023 (PCDOS 7.1)
19142 ; 26/12/2023
19143 00001C45 268A453E mov al, [es:di+3Eh] ; [es:di+BDS.formfactor]
19144 ; mov al, [es:di+22h] ; [es:di+BDS.formfactor]
19145 00001C49 3C07 cmp al, 7
19146 ; cmp byte [es:di+22h], 7 ; [es:di+BDS.formfactor]
19147 ; ff0ther
19148 00001C4B 7413 jz short Is720K
19149 00001C4D 3C09 cmp al, 9
19150 ; cmp byte [es:di+22h], 9 ; [es:di+BDS.formfactor]
19151 ; ff288
19152 00001C4F 740F jz short Is720K
19153 00001C51 B007 mov al, 7 ; seven sectors / fat
19154 00001C53 BB0FE0 mov bx, 57359 ; 224*256+0Fh
19155 ; 224 root dir entries
19156 ; & 0Fh sector max
19157 00001C56 B96009 mov cx, 2400 ; 80*15*2
19158 ; 80 tracks, 15 sectors/track,
19159 ; 2 sides
19160 ; 02/09/2023
19161 00001C59 5A pop dx ; pop off return address
19162 00001C5A BA0201 mov dx, 258 ; 1*256+2
19163 ; sectors/allocation unit
19164 ; & head max
19165 ; pop off return address
19166 00001C5D E9EAEA jmp Has1 ; return to tail of getbp
19167 ; -----
19168
19169 Is720K:
19170 ; 02/09/2023
19171 00001C60 5B pop bx ; pop off return address
19172 ; add sp, 2 ; pop off return address
19173 00001C61 E9A9EA jmp Has720K ; return to 720K code
19174 ; -----
19175 ; 18/12/2022
19176
19177 ; dofloppy:
19178 ; retn
19179
19180 ; ===== S U B R O U T I N E =====
19181 ;
19182 ; 16/10/2022
19183 ;
19184 ;-----
19185 ; set_changed_dl - sets flag bits according to bits set in bx.
19186 ; essentially used to indicate changeline, or format.
19187 ;
19188 ; inputs: dl contains physical drive number
19189 ; bx contains bits to set in the flag field in the bdss
19190 ; outputs: none
19191 ; registers modified: flags
19192 ;
19193 ; called from int13 hooker. Must preserve ALL registers!!!
19194 ;
19195 ; in the virtual drive system we *must* flag the other drives as being changed
19196 ;-----
19197 ;
19198 ; 26/12/2023 - Retro DOS v5.0
19199
19200 set_changed_dl:
19201 00001C64 06 push es
19202 00001C65 57 push di
19203 ; les di, ds:start_bds
19204 00001C66 C43E[1901] ; 19/10/2022
19205 ; les di, [start_bds]
19206 ;
19207 ; note: we assume that the list is non-empty
19208
19209 scan_bds:
19210 00001C6A 26385504 cmp [es:di+4], dl ; [es:di+BDS.drivenum]
19211 00001C6E 7504 jnz short get_next_bds
19212 ;
19213 ; someone may complain, but this *always* must be done when a disk change is
19214 ; noted. there are *no* other compromising circumstances.
19215 ; 26/12/2023
19216 00001C70 26095D3F or [es:di+3Fh], bx ; [es:di+BDS.flags]
19217 ; or [es:di+23h], bx ; [es:di+BDS.flags]
19218 ; signal change on other drive
19219 get_next_bds:

```

```

19220 00001C74 26C43D          les     di, [es:di]      ; [es:di+BDS.link]
19221                                ; go to next bds
19222 00001C77 83FFFF          cmp     di, 0FFFFh
19223 00001C7A 75EE            jnz     short scan_bds ; loop unless we hit end of chain
19224 00001C7C 5F              pop     di
19225 00001C7D 07              pop     es
19226 00001C7E C3              retn

; ===== S U B   R O U T I N E =====
; -----
; checkromchange - see if external program has diddled rom change line.
;
;   inputs: es:di points to current bds.
;   outputs: zero set - no change
;           zero reset - change
;   registers modified: none
; -----
;
;   ; 26/12/2023 - Retro DOS v5.0
checkromchange:
;   ; test word [es:di+BDS.flags], fchanged ; 40h
;   ; 26/12/2023
19241                                test    word [es:di+BDS.flags], 40h
19242                                ; 10/12/2022
19243 00001C7F 26F6453F40      test    byte [es:di+3Fh], 40h
19244                                ; 10/12/2022
19245                                test    byte [es:di+23h], 40h
19246                                ;; test word [es:di+23h], 40h ; [es:di+BDS.flags]
19247                                ; fchanged
19248 00001C84 C3              retn

; ===== S U B   R O U T I N E =====
; -----
; resetchanged - restore value of change line
;
;   inputs: es:di points to current bds
;   outputs: none
;   registers modified: none
; -----
;
;   ; 26/12/2023 - Retro DOS v5.0
resetchanged:
;   ; and word [es:di+BDS.flags], ~fchanged ; 0FFBFh
;   ; 26/12/2023
19263                                and     word [es:di+BDS.flags], 0BFh
19264 00001C85 2680653FBF      and     byte [es:di+3Fh], 0BFh
19265                                ; 10/12/2022
19266                                and     byte [es:di+23h], 0BFh
19267                                ;; and word [es:di+23h], 0FFBFh ; [es:di+BDS.flags]
19268                                ; ~fchanged
19269 00001C8A C3              retn

; ===== S U B   R O U T I N E =====
; -----
; haschange - see if drive can supply change line
;
;   inputs: es:di points to current bds
;   outputs: zero set - no change line available
;           zero reset - change line available
;   registers modified: none
; -----
;
;   ; 26/12/2023 - Retro DOS v5.0
haschange:
;   ; test word [es:di+BDS.flags], fchangeline ; 2
;   ; 26/12/2023
19286 00001C8B 26F6453F02      test    word [es:di+BDS.flags], 2
19287                                ; 10/12/2022
19288                                test    byte [es:di+23h], 2
19289                                ;; test word [es:di+23h], 2 ; [es:di+BDS.flags]
19290                                ; fchangeline
19291 00001C90 C3              retn

; -----
; 16/10/2022
; -----
;
; set_volume_id - main routine, calls other routines.
; read_volume_id - read the volume id and tells if it has been changed.
; transfer_volume_id - copy the volume id from tmp to special drive.
; check_volume_id - compare volume id in tmp area with one expected for drive.
; fat_check - see of the fatid has changed in the specified drive.
; -----
;
; set_volume_id
; if drive has changeline support, read in and set the volume_id
; and the last fat_id byte. if no change line support then do nothing.
;
; on entry:
; es:di points to the bds for this disk.
; ah contains media byte
;
; on exit:
; carry clear:
;   successful call
; carry set
; error and ax has error code
;
set_volume_id:
;   push    dx      ; save registers
;   push    ax
;   call    haschange ; does drive have changeline support?
;   jz      short setvret ; no, get out
;   call    read_volume_id
;   jb      short seterr
;   call    transfer_volume_id ; copy the volume id to special drive
;   call    resetchanged ; restore value of change line
;
setvret:
;   ; 10/12/2022
;   ; cf = 0
;   ; clc ; no error, clear carry flag
;   pop     ax      ; restore registers
;   pop     dx
;   retn

; -----
;
; seterr:
;   pop     dx      ; pop stack but don't overwrite ax
;   pop     dx      ; restore dx
;   retn

; -----
;
; root_sec: dw 0 ; root sector #

```



```

19344 ; 16/10/2022
19345 ; ROOTSEC equ root_sec - DOSBIOSEG_2C7h
19346 ; 09/12/2022
19347 ROOTSEC equ root_sec
19348
19349 ; ===== S U B   R O U T I N E =====
19350
19351 ; 16/10/2022
19352
19353 ; read_volume_id read the volume id and tells if it has been changed.
19354 ;
19355 ; on entry:
19356 ; es:di points to current bds for drive.
19357 ;
19358 ; on exit:
19359 ; carry clear
19360 ; si = 1 no change
19361 ; si = 0 ?
19362 ; si = -1 change
19363 ;
19364 ; carry set:
19365 ; error and ax has error code.
19366
19367 read_volume_id:
19368     push    dx                ; preserve registers
19369     push    cx
19370     push    bx
19371     push    ax
19372     push    es                ; stack the bds last
19373     push    di
19374     push    ds                ; point es to Bios_Data
19375     pop     es
19376     mov     di, tmp_vid        ; "NO NAME      "
19377     mov     si, nul_vid        ; "NO NAME      "
19378     ; 26/12/2023
19379     mov     cx, 11            ; PCDOS 7.1 - 02/09/2023
19380     ; mov     cx, 12            ; initialize tmp_vid to      null vi_id
19381
19382     ; rep     movsb
19383     ; 26/12/2023
19384     ; rep     movs byte ptr es:[di], byte ptr cs:[si]
19385     ; db 0FBh, 2Eh, 0A4h
19386     ; cs      ; nul_vid is in BIOSCODE segment
19387     ; rep     movsb
19388     rep     cs
19389     movsb
19390
19391     pop     di
19392     pop     es
19393     mov     al, [es:di+11] ; [es:di+BDS.fats]
19394     ; # of fats
19395     mov     cx, [es:di+17] ; [es:di+BDS.fatsecs]
19396     ; sectors / fat
19397     mul     cl              ; size taken by      fats
19398     add     ax, [es:di+9] ; [es:di+BDS.resectors]
19399     ; add on reserved sectors
19400     ;
19401     ; ax is now sector # (0      based)
19402
19403     ; 17/10/2022
19404     mov     [cs:ROOTSEC], ax
19405     ; mov     word ptr cs:198Fh, ax ; [cs:root_sec]
19406     ; 0070h:3EFFh = 2C7h:198Fh
19407     mov     ax, [es:di+12] ; [es:di+BDS.direntries]
19408     ; # root dir entries
19409     mov     cl, 4           ; 16 entries/sector
19410     shr     ax, cl          ; divide by 16
19411     ; mov     cx, ax          ; cx is # of sectors to      scan
19412     ; 02/09/2023 (PCDOS 7.1, one byte opcode)
19413     xchg    ax, cx          ; cx is # of sectors to      scan
19414
19415     next_sec:
19416     push    cx              ; save outer loop counter
19417     mov     ax, [cs:ROOTSEC]
19418     ; mov     ax, word ptr cs:198Fh ; [cs:root_sec]
19419     mov     cx, [es:di+19] ; [es:di+BDS.secptrack]
19420     ; sectors / track
19421     xor     dx, dx
19422     div     cx
19423
19424     ; set up registers for call to read_sector
19425
19426     inc     dx              ; dx= sectors into track
19427     ; ax= track count from 0
19428     mov     cl, dl          ; sector to read
19429     xor     dx, dx
19430     div     word [es:di+21] ; [es:di+BDS.heads]
19431     ; # heads on this disc
19432     mov     dh, dl          ; head number
19433     mov     ch, al          ; track #
19434     call    read_sector     ; get first sector of the root directory,
19435     ; ds:bx -> directory sector
19436     jb     short readviderr
19437     mov     cx, 16          ; # of dir entries in a      block of root
19438     mov     al, 8           ; volume label bit
19439
19440     fvid_loop:
19441     ; 02/09/2023 (PCDOS 7.1)
19442     cmp     [bx], ch ; 0
19443     ; cmp     byte [bx], 0 ; end of dir?
19444     jz     short no_vid     ; yes, no vol id
19445     cmp     byte [bx], 0E5h ; empty entry?
19446     jz     short ent_loop   ; yes, skip
19447     test    [bx+11], al      ; is volume label bit set in fcb?
19448     jnz     short found_vid  ; jmp yes
19449
19450     ent_loop:
19451     add     bx, 32           ; add length of      directory entry
19452     loop    fvid_loop
19453     pop     cx              ; outer loop
19454     inc     word [cs:ROOTSEC]
19455     ; inc     word ptr cs:198Fh ; inc word [root_sec]
19456     ; next_sector
19457     loop    next_sec        ; continue
19458
19459     notfound:
19460     ; 02/09/2023
19461     ; xor     si, si
19462     jmp     short fvid_ret
19463
19464     ; -----
19465
19466     found_vid:
19467     ; 02/09/2023
19468     ; cf = 0 ('test' instruction clears cf)
19469     pop     cx              ; clean stack of outer loop counter
19470     mov     si, bx          ; point to volume_id
19471     push    es              ; preserve current bds

```

```

19468 00001D21 57          push    di
19469 00001D22 1E          push    ds
19470 00001D23 07          pop     es          ; point es to Bios_Data
19471 00001D24 BF[4008]    mov     di, tmp_vid  ; "NO NAME"
19472 00001D27 B90B00    mov     cx, 11       ; VOLID_SIZ-1
19473                                ; length of string minus nul
19474 00001D2A F3A4          rep movsb            ; mov volume label to tmp_vid
19475                                ;xor    al, al
19476                                ; 02/09/2023
19477 00001D2C 91          xchg    ax, cx        ; ax = 0
19478 00001D2D AA          stosb                ; null terminate
19479                                ;;xor    si, si
19480                                ; 02/09/2023
19481                                ;xchg    ax, si        ; si = 0
19482 00001D2E 5F          pop     di            ; restore current bds
19483 00001D2F 07          pop     es
19484
fvid_ret:
19485                                ; 02/09/2023
19486 00001D30 31F6          xor     si, si ; 0
19487
19488                                pop     ax
19489                                ; 10/12/2022
19490                                ; cf = 0
19491                                ;clc
19492
rvidret:
19493                                pop     bx          ; restore registers
19494                                pop     cx
19495                                pop     dx
19496                                retn
19497
; -----
19498
no_vid:
19499
19500 00001D37 59          pop     cx          ; clean stack of outer loop counter
19501                                ;jmp     short notfound ; not found
19502                                ; 02/09/2023
19503 00001D38 EBF6          jmp     short fvid_ret
19504
; -----
19505
readviderr:
19506
19507 00001D3A 5E          pop     si          ; trash the outer loop counter
19508 00001D3B 5E          pop     si          ; caller's ax, return error code instead
19509 00001D3C EBF5          jmp     short rvidret
19510
; -----
19511
; 26/12/2023 - Retro DOS v5.0
19512 ; 02/09/2023 - Retro DOS v4.2 (IO.SYS optimization)
19513 ; PCDOS 7.1 - IBMBIO.COM - BIOSCODE:1DCFh
19514
preset_volid_addr:
19515                                mov     si, tmp_vid  ; "NO NAME"
19516 00001D3E BE[4008]    ; 26/12/2023
19517                                ; PCDOS 7.1
19518                                add     di, 125      ; BDS.volid
19519 00001D41 83C77D    mov     cx, 11       ; VOLID_SIZ (12 for MSDOS 5.0-6.22 versions)
19520 00001D44 B90B00    ; MSDOS 6.21 (MSDOS 5.0 & 6.?)
19521                                ;add     di, 75      ; BDS.volid
19522                                ;mov     cx, 12       ; VOLID_SIZ
19523                                ;
19524                                ;cld
19525 00001D47 FC          cld
19526 00001D48 C3          retn
19527
; ===== S U B   R O U T I N E =====
19528
; transfer_volume_id - copy the volume id from tmp to special drive
19529
;
; inputs:  es:di has current bds
; outputs: bds for drive has volume id from tmp
19530
; 27/12/2023 - Retro DOS v5.0
19531
transfer_volume_id:
19532                                push    di          ; copy the volume id from tmp to special drive
19533                                push    si
19534                                push    cx
19535                                ; 27/12/2023
19536                                push    si
19537 00001D49 57          ;mov     si, tmp_vid  ; "NO NAME"
19538                                ;;add     di, BDS.volid
19539 00001D4A 51          ;add     di, 75      ; BDS.volid
19540                                ;;mov     cx, VOLID_SIZ
19541 00001D4B 56          ;mov     cx, 12       ; VOLID_SIZ
19542                                ;cld
19543                                ; 02/09/2023 (PCDOS 7.1)
19544                                call    preset_volid_addr
19545
19546                                rep movsb
19547                                ; 27/12/2023
19548                                pop     si
19549
chk_volid_ok:
19550                                pop     cx
19551                                pop     si
19552                                pop     di
19553                                retn
19554
; ===== S U B   R O U T I N E =====
19555
; check_volume_id - compare volume id in tmp area with
19556 ; one expected for drive
19557
;
; inputs: es:di has current bds for drive
; outputs: zero true means it matched
19558
; 27/12/2023 - Retro DOS v5.0
19559
check_volume_id:
19560                                push    di
19561                                push    cx
19562
19563                                ;mov     si, tmp_vid  ; "NO NAME"
19564                                ;;add     di, BDS.volid
19565                                ;add     di, 75      ; BDS.volid
19566                                ;;mov     cx, VOLID_SIZ
19567                                ;mov     cx, 12       ; VOLID_SIZ
19568                                ;cld
19569                                ; 02/09/2023 (PCDOS 7.1)
19570                                call    preset_volid_addr
19571
19572                                repe cmpsb            ; are the 2 volume_ids the same?
19573                                ; 27/12/2023
19574                                pop     cx
19575                                pop     di
19576                                ;retn
19577                                jmp     short chk_volid_ok
19578
19579 00001D5C EBF4          jmp     short chk_volid_ok
19580
19581
19582 00001D57 E8E4FF    call    preset_volid_addr
19583
19584 00001D5A F3A6          repe cmpsb            ; are the 2 volume_ids the same?
19585
19586                                ; 27/12/2023
19587                                pop     cx
19588                                pop     di
19589                                ;retn
19590                                jmp     short chk_volid_ok
19591

```



```

19592 ; ===== S U B   R O U T I N E =====
19593 ;
19594 ; fat_check - see if the fatid has changed in the specified drive.
19595 ; - uses the fat id obtained from the boot sector.
19596 ;
19597 ; inputs: medbyt is expected fat id
19598 ; es:di points to current bds
19599 ;
19600 ; output: si = -1 if fat id different,
19601 ; si = 0 otherwise
19602 ;
19603 ; no other registers changed.
19604 ;
19605 fat_check:
19606 00001D5E 50      push    ax
19607 00001D5F 31F6     xor     si, si      ; say fat id's are same.
19608 00001D61 A0[1F01]  mov     al, [medbyt] ; 19/10/2022
19609 00001D64 263A4510 cmp     al, [es:di+10h] ; [es:di+BDS.media]
19610 ; compare it with the bds medbyte
19611 00001D68 7401     jz      short okret1 ; carry clear
19612 00001D6A 4E      dec     si
19613 okret1:
19614 00001D6B 58      pop     ax
19615 00001D6C C3      retn
19616 ;
19617 ; -----
19618 ; BIOSCODE:1DFEh (PCDOS 7.1 IBMBIO.COM) ; 27/12/2023
19619 ; times 2 db 0
19620 ;
19621 ; BIOSCODE:1A69h (MSDOS 6.21 IO.SYS) ((& MSDOS 6.22 IO.SYS))
19622 ; times 7 db 0
19623 ;
19624 ; BIOSCODE:180Bh (MSDOS 5.0 IO.SYS)
19625 ;
19626 ; 09/12/2022
19627 ; times 4 db 0 ; 17/10/2022
19628 ; db 4 dup(0) ; times 4 db 0
19629 ;
19630 ; -----
19631 ;
19632 ; 09/12/2022
19633 ; db 0
19634 ;
19635 number2div equ ($-BCODE_start)
19636 number2mod equ (number2div % 16)
19637 ;
19638 %if (number2mod>0) & (number2mod<16) ; 17/09/2023
19639 00001D6D 00<rep 3h> times (16-number2mod) db 0
19640 ;
19641 %endif
19642 ;align 16
19643 ;
19644 ; 09/12/2022
19645 BCODE_END equ $ - BCODE_start
19646 ; 29/09/2023
19647 BCODEEND:
19648 ;SYSINITSEG equ IOSYSCODESEG+(BCODE_END>>4)
19649 ; 13/12/2022
19650 SYSINITOFFSET equ BCODE_END
19651 ; 29/09/2023
19652 ;SYSINITOFFSET equ $-$$
19653 SYSINITSEG equ IOSYSCODESEG+(SYSINITOFFSET>>4)
19654 ;
19655 ; 28/09/2023
19656 S2SIZE equ $-$$
19657 ;
19658 ;--- End of DOSBIOS code segment -----
19659 ;
19660 ; 16/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
19661 ; 01/05/2019 - Retro DOS v4.0
19662 ; =====
19663 ; end of BIOSCODE
19664 ;
19665 ; -----
19666 ; %include sysinit5.s ; 09/12/2022
19667 ; -----
19668 ;
19669 ; =====
19670 ; (IO.SYS) SYSINIT SEGMENT
19671 ; =====
19672 ; 09/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
19673 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
19674 ;
19675 ; 27/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
19676 ;
19677 section .SYSINIT vstart=0
19678 ;
19679 ; *****
19680 ; SYSINIT.BIN (MSDOS 6.21 IO.SYS) - RETRO DOS v4.0 by ERDOGAN TAN - 21/10/2022
19681 ;
19682 ; Last Update: 04/01/2023 (Modified IO.SYS) ((Previous: 30/12/2022))
19683 ;
19684 ; Beginning: 03/06/2018 (Retro DOS 3.0), 21/03/2019 (Retro DOS 4.0)
19685 ;
19686 ; Assembler: NASM version 2.15
19687 ;
19688 ; ((nasm sysinit6.s -l sysinit6.lst -o SYSINIT6.BIN -Z error.txt))
19689 ;
19690 ; Modified from 'sysinit2.s' (SYSINIT2.BIN) file of Retro DOS v3.0 (6/7/2018)
19691 ;
19692 ; Derived from 'SYSINIT1.ASM' and 'SYSINIT2.ASM' files of MSDOS 6.0
19693 ; source code by Microsoft, 1991
19694 ;
19695 ; Derived from 'SYSINIT.ASM' file of MSDOS 2.0 (IBM PCDOS v2.0) source code
19696 ; by Microsoft, 12/10/1983
19697 ; *****
19698 ; main file: 'retrodos4.s'
19699 ; incbin 'SYSINIT3.BIN' ; (SYINITSEG)
19700 ;
19701 ; 30/12/2022 - Retro DOS v4.2
19702 ; Retro DOS v4.0 - 2019
19703 ; SYSINIT (MSDOS 6.21 IO.SYS) draft: 'sysinit3.s' (01/07/2019)
19704 ;
19705 ; 21/10/2022
19706 ;
19707 ; -----
19708 ; This source code (version) is based on SYSINIT source code of disassembled
19709 ; MSDOS 5.0 IO.SYS file (SYSINIT.BIN)
19710 ; Disassembler: Hex-Rays Interactive Disassembler (IDA)
19711 ;
19712 ; Binary file splitter & joiner: FFSJ v3.3
19713 ; -----
19714 ;
19715 ; SYSINIT.TXT (27/01/1983)

```

```

19716 ;-----
19717 ;   SYSINIT is a module linked behind the OEM bios. It takes
19718 ; over the system initialization after the OEM bios has
19719 ; performed any initialization it needs to do. Control is
19720 ; transfered with a long jump to the external variable SYSINIT
19721 ;
19722 ;
19723 ;   The OEM has the following variables declared external:
19724 ;
19725 ;   CURRENT_DOS_LOCATION      WORD
19726 ;
19727 ; This word contains the segment number of the DOS before it
19728 ; is relocated. The OEM bios must set this value.
19729 ;
19730 ;   FINAL_DOS_LOCATION      WORD
19731 ;
19732 ; This word contains the segment number of the DOS after SYSINIT
19733 ; moves it. The OEM bios must set this value.
19734 ;
19735 ;   DEVICE_LIST              DWORD
19736 ;
19737 ; This double word pointer points to the linked list of
19738 ; character and block device drivers. The OEM must set this
19739 ; value.
19740 ;
19741 ;   MEMORY_SIZE              WORD
19742 ;
19743 ; This word contains the number of RAM paragraphs. If the
19744 ; bios doesn't set this variable SYSINIT will automatically
19745 ; calculate it. NOTE: systems with PARITY checked memory must
19746 ; size memory in the BIOS. SYSINIT's method is to write memory
19747 ; and read it back until it gets a mismatch.
19748 ;
19749 ;   DEFAULT_DRIVE            BYTE
19750 ;
19751 ; This is the initial default drive when the system first comes
19752 ; up. drive a=0, drive b=1, etc. If the bios doesn't set
19753 ; it then drive a is assumed.
19754 ;
19755 ;   BUFFERS                  BYTE
19756 ;
19757 ; This is the default number of buffers for the system. This
19758 ; value may be overridden by the user in the CONFIG.SYS file.
19759 ; It is DBed to 2 in SYSINIT it should be greater than 1.
19760 ;
19761 ;   FILES                    BYTE
19762 ;
19763 ; This is the default number of files for the system. This
19764 ; value may be overridden by the user in the CONFIG.SYS file.
19765 ; It is DBed to 8 in SYSINIT, values less than 5 are ignored.
19766 ;
19767 ;   SYSINIT                  FAR
19768 ;
19769 ; The entry point of the SYSINIT module. OEM BIOS jumps to
19770 ; this label at the end of its INIT code.
19771 ;
19772 ;   The OEM has the following variables declared public:
19773 ;
19774 ;   RE_INIT                  FAR
19775 ;
19776 ; This is an entry point which allows the BIOS to do some INIT
19777 ; work after the DOS is initialized. ALL REGISTERS MUST BE
19778 ; PRESERVED. On entry DS points to the first available memory
19779 ; (after the DOS). DS:0 points to a 100H byte program header
19780 ; prefix which represents the "program" currently running.
19781 ; This program should be thought of as the OEM BIOS and
19782 ; SYSINIT taken together. This is not a normal program in
19783 ; that no memory is allocated to it, it is running in free
19784 ; memory.
19785 ; NOTES:
19786 ;   At the time this routine is called SYSINIT occupies the
19787 ; highest 10K of memory ("highest" is determined by the value
19788 ; of the MEMORY_SIZE variable), DO NOT DO WRITES THERE.
19789 ;   Since this is called AFTER DOS is initialized, you can
19790 ; make system calls. This also implies that the code for this
19791 ; routine CANNOT be thrown away by use of the
19792 ; FINAL_DOS_LOCATION since the DOS has already been moved.
19793 ;   If you don't want anything done just set this to point
19794 ; at a FAR RET instruction.
19795 ;
19796 ; -----
19797 ; TITLE   BIOS SYSTEM INITIALIZATION
19798 ; -----
19799 ;
19800 ;include version.inc
19801 ; -----
19802 ;
19803 ;FALSE    EQU    0
19804 ;TRUE     EQU    0FFFFh
19805 ;
19806 ;IBMVER    EQU    TRUE
19807 ;IBMCOPYRIGHT EQU    FALSE
19808 ;STACKSW   EQU    TRUE ;Include Switchable Hardware Stacks
19809 ;IBMJPAPER EQU    FALSE ; If TRUE set KANJI true also
19810 ;MSVER     EQU    FALSE
19811 ;ALTVECT   EQU    FALSE ; Switch to build ALTVECT version
19812 ;KANJI     EQU    FALSE
19813 ;
19814 ;(MSDOS 6.0, versiona.inc, 1991)
19815 ; -----
19816 ;MAJOR_VERSION EQU    6
19817 ;MINOR_VERSION EQU    0 ;6.00
19818 ;MINOR_VERSION EQU    21 ;6.21 ; 21/03/2019 - Retro DOS v4.0
19819 ;
19820 ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0)
19821 ; -----
19822 ;MAJOR_VERSION EQU    5
19823 ;MINOR_VERSION EQU    0
19824 ;
19825 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21)
19826 ;MAJOR_VERSION EQU    6
19827 ;MINOR_VERSION EQU    22
19828 ;
19829 ; 21/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1)
19830 ;MAJOR_VERSION EQU    7
19831 ;MINOR_VERSION EQU    10
19832 ;
19833 expected_version equ    (MINOR_VERSION<<8)+MAJOR_VERSION
19834 ;
19835 ;DOSREVN   equ    00000000b ; m037 - bits 0-2 = revision number of DOS
19836 ; ; currently 0.
19837 ;DOSREVN   equ    00000111b ; [[[ 7 for Retro DOS v4.0 ]]] (21/03/2019)
19838 ;DOSINROM  equ    00001000B ; bit 3 of ver flags returned in BH
19839 ;DOSINHMA  equ    00010000B ; bit 4 of ver flags

```

```

19840
19841
19842
19843
19844
19845
19846
19847
19848
19849
19850
19851
19852
19853
19854
19855
19856
19857
19858
19859
19860
19861
19862
19863
19864
19865
19866
19867
19868
19869
19870
19871
19872
19873
19874
19875
19876
19877
19878
19879
19880
19881
19882
19883
19884
19885
19886
19887
19888
19889
19890
19891
19892
19893
19894
19895
19896
19897
19898
19899
19900
19901
19902
19903
19904
19905
19906
19907
19908
19909
19910
19911
19912
19913
19914
19915
19916
19917
19918
19919
19920
19921
19922
19923
19924
19925
19926
19927
19928
19929
19930
19931
19932
19933
19934
19935
19936
19937
19938
19939
19940
19941
19942
19943
19944
19945
19946
19947
19948
19949
19950
19951
19952
19953
19954
19955
19956
19957
19958
19959
19960
19961
19962
19963

;
; if1
; %OUT ... for DOS Version 5.00 ...
; endif

;*****
;Each assembler program should:
; mov ah,030h ;DOS Get Version function
; int 021h ;Version ret. in AX,minor version first
; cmp ax,expected_version ;ALL utilities should check for an
; jne error_handler ; EXACT version match.
;*****

; -----
; device definitions

;Attribute bit masks
DEVTYP EQU 8000h ;Bit 15 - 1 if Char, 0 if block
DEVIOCTL EQU 4000h ;Bit 14 - CONTROL mode bit
ISFATBYDEV EQU 2000h ;Bit 13 - Device uses FAT ID bytes, comp media.
ISCIN EQU 0001h ;Bit 0 - This device is the console input.
ISCOU EQU 0002h ;Bit 1 - This device is the console output.
ISNULL EQU 0004h ;Bit 2 - This device is the null device.
ISCLOCK EQU 0008h ;Bit 3 - This device is the clock device.
ISIBM EQU 0010h ;Bit 4 - This device is special

; The device table list has the form:
struc SYSDEV
.NEXT: resd 1 ;Pointer to next device header
.ATT: resw 1 ;Attributes of the device
.STRAT: resw 1 ;Strategy entry point
.INT: resw 1 ;Interrupt entry point
.NAME: resb 8 ;Name of device (only first byte used for block)
.size:
endstruc

;Static Request Header
struc SRHEAD
.REQLEN: resb 1 ;Length in bytes of request block
.REQUNIT: resb 1 ;Device unit number
.REQFUNC: resb 1 ;Type of request
.REQSTAT: resw 1 ;Status word
.resb 8 ;Reserved for queue links
.size:
endstruc

;Status word masks
STERR EQU 8000H ;Bit 15 - Error
STBUI EQU 0200H ;Bit 9 - Buisy
STDON EQU 0100H ;Bit 8 - Done
STECODE EQU 00FFH ;Error code
WRECODE EQU 0

;Function codes
DEVINIT EQU 0 ;Initialization
DINITHL EQU 26 ;Size of init header
DEVMDCH EQU 1 ;Media check
DMEDHL EQU 15 ;Size of media check header
DEVBPB EQU 2 ;Get BPB
DEVDRDIOTL EQU 3 ;IOCTL read
DBPBHL EQU 22 ;Size of Get BPB header
DEVDR EQU 4 ;Read
DRDWRHL EQU 22 ;Size of RD/WR header
DEVDRND EQU 5 ;Non destructive read no wait (character devs)
DRDNDHL EQU 14 ;Size of non destructive read header
DEVIST EQU 6 ;Input status
DSTATHL EQU 13 ;Size of status header
DEVIFL EQU 7 ;Input flush
; 21/02/2024
DFLSHL EQU 15 ;Size of flush header
DFLSHL equ 13 ; PCDOS 7.1 IBMDOS.COM ; 21/02/2024
DEVWRT EQU 8 ;write
DEVWRTV EQU 9 ;write with verify
DEVOST EQU 10 ;Output status
DEVOFL EQU 11 ;Output flush
DEVWRIOTL EQU 12 ;IOCTL write

; -----
struc SYS_FCB
.fcb_drive: resb 1
.fcb_name: resb 8
.fcb_ext: resb 3
.fcb_EXTENT: resw 1
.fcb_RECSIZ: resw 1 ; Size of record (user settable)
.fcb_FLISIZ: resw 1 ; Size of file in bytes; used with the following
; word
.fcb_DRVBP: resw 1 ; BP for SEARCH FIRST and SEARCH NEXT
.fcb_FDATE: resw 1 ; Date of last writing
.fcb_FTIME: resw 1 ; Time of last writing
.fcb_DEVID: resb 1 ; Device ID number, bits 0-5 if file.
; bit 7=0 for file, bit 7=1 for I/O device
; if file, bit 6=0 if dirty
; if I/O device, bit 6=0 if EOF (input)
; Bit 5=1 if Raw mode
; Bit 0=1 if console input device
; Bit 1=1 if console output device
; Bit 2=1 if null device
; Bit 3=1 if clock device
.fcb_FIRCLUS: resw 1 ; First cluster of file
.fcb_CLUSPOS: resw 1 ; Position of last cluster accessed
.fcb_LSTCLUS: resw 1 ; Last cluster accessed and directory
.resb 1 ; pack 2 12 bit numbers into 24 bits...
.fcb_NR: resb 1 ; Next record
.fcb_RR: resb 4 ; Random record
.size:
endstruc

; -----
; Field definition for I/O buffer information

; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0, BUFFER.INC, 1991)
; 03/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)

struc BUFFINFO
.buf_next: resw 1 ; Pointer to next buffer in list
.buf_prev: resw 1 ; Pointer to prev buffer in list
.buf_ID: resb 1 ; Drive of buffer (bit 7 = 0)
; SFT table index (bit 7 = 1)
; = FFH if buffer free
; Bit 7 = 1 if Remote file buffer
; = 0 if Local device buffer
; Bit 6 = 1 if buffer dirty
; Bit 5 = Reserved

```

```

19964                                     ; Bit 4 = Search bit (bit 7 = 1)
19965                                     ; Bit 3 = 1 if buffer is DATA
19966                                     ; Bit 2 = 1 if buffer is DIR
19967                                     ; Bit 1 = 1 if buffer is FAT
19968                                     ; Bit 0 = Reserved
19969 00000006 ????????? .buf_sector:      resd 1      ; Sector number of buffer (flags bit 7 = 0)
19970                                     ; The next two items are often refed as a word (flags bit 7 = 0)
19971 0000000A ?? .buf_wrtcnt:      resb 1      ; For FAT sectors, # times sector written out
19972 0000000B ????.buf_wrtcntinc:  resw 1      ; " " " " , # sectors between each write
19973 0000000D ????.resw 1 ; * ; 03/01/2024 ; PCDOS 7.1
19974                                     ; hw of sectors per FAT
19975 0000000F ????????? .buf_DPB:      resd 1      ; Pointer to drive parameters
19976 00000013 ????.buf_fill:  resw 1      ; How full buffer is (flags bit 7 = 1)
19977 00000015 ?? .buf_reserved:  resb 1      ; make DWORD boundary for 386
19978 00000016 ????.resw 1 ; * ; 03/01/2024 ; PCDOS 7.1
19979                                     ; reserved word for dword boundary
19980 .size:      ; 20 bytes ; MSDOS 5.0 to 6.22
19981          ; 24 bytes ; PCDOS 7.1 ; 03/01/2024
19982 endstruc
19983
19984 %define buf_offset BUFFINFO.buf_sector ; 22/07/2019
19985                                     ; For buf_flags bit 7 = 1, this is the byte
19986                                     ; offset of the start of the buffer in
19987                                     ; the file pointed to by buf_ID. Thus
19988                                     ; the buffer starts at location
19989                                     ; buf_offset in the file and contains
19990                                     ; buf_fill bytes.
19991
19992 bufinsiz      equ      BUFFINFO.size ; Size of structure in bytes
19993
19994
19995 buf_Free      equ      0FFh          ; buf_id of free buffer
19996
19997 ; Flag byte masks
19998 buf_isnet     EQU      10000000B
19999 buf_dirty     EQU      01000000B
20000 ;***
20001 buf_visit     EQU      00100000B
20002 ;***
20003 buf_snbuf     EQU      00010000B
20004
20005 buf_isDATA    EQU      00001000B
20006 buf_isDIR     EQU      00000100B
20007 buf_isFAT     EQU      00000010B
20008 buf_type_0    EQU      11110001B      ; AND sets type to "none"
20009
20010 buf_NetID     EQU      bufinsiz
20011
20012 ; -----
20013
20014 ; -----
20015 ** DPB - Drive Parameter Block
20016
20017 ; 25/03/2019 - Retro DOS v4.0 (MSDOS 6.0, DPB.INC, 1991)
20018
20019 ; BUGBUG - this isn't authoritative - it's my probably incomplete and
20020 ; possibly inaccurate deductions from code study... - jgl
20021 ;
20022 ; The DPB is DOS's main structure for describing block devices.
20023 ; It contains info about the "Drive" intermingled with info about
20024 ; the FAT file system which is presumably on the drive. I don't know
20025 ; how those fields are used if it's not the FAT file system - BUGBUG
20026 ;
20027 ; The DPBs are statically allocated and chained off of DPBHead.
20028 ; Users scan this chain looking for a match on DPB_DRIVE.
20029 ; The DPBs are built at init time from info in the SYSDEV structure.
20030
20031 ; 08/06/2018 - Retro DOS v3.0 (MSDOS 3.3, DPB.INC, 24/07/1987)
20032
20033 ; 12/05/2019 - Retro DOS v4.0
20034
20035 ; 01/01/2024
20036 %if 0
20037
20038 struc          DPB
20039 .DRIVE:        resb 1      ; Logical drive # assoc with DPB (A=0,B=1,...)
20040 .UNIT:         resb 1      ; Driver unit number of DPB
20041 .SECTOR_SIZE:  resw 1      ; Size of physical sector in bytes
20042 .CLUSTER_MASK: resb 1      ; Sectors/cluster - 1
20043 .CLUSTER_SHIFT: resb 1     ; Log2 of sectors/cluster
20044 .FIRST_FAT:    resw 1      ; Starting record of FATs
20045 .FAT_COUNT:    resb 1      ; Number of FATs for this drive
20046 .ROOT_ENTRIES: resw 1      ; Number of directory entries
20047 .FIRST_SECTOR: resw 1      ; First sector of first cluster
20048 .MAX_CLUSTER:  resw 1      ; Number of clusters on drive + 1
20049 ; MSDOS 3.3
20050 ; .FAT_SIZE:    resb 1      ; Number of records occupied by FAT
20051 ; MSDOS 6.0
20052 .FAT_SIZE:     resw 1      ; Number of records occupied by FAT
20053 .DIR_SECTOR:   resw 1      ; Starting record of directory
20054 .DRIVER_ADDR:  resd 1      ; Pointer to driver
20055 .MEDIA:        resb 1      ; Media byte
20056 .FIRST_ACCESS: resb 1      ; This is initialized to -1 to force a media
20057                                     ; check the first time this DPB is used
20058 .NEXT_DPB:     resd 1      ; Pointer to next Drive parameter block
20059 .NEXT_FREE:     resw 1      ; Cluster # of last allocated cluster
20060 .FREE_CNT:      resw 1      ; Count of free clusters, -1 if unknown
20061 .size:
20062 endstruc
20063
20064 %else
20065
20066 ; 01/01/2024 - Retro DOS v5.0 (PCDOS 7.1)
20067
20068 struc          DPB
20069 .DRIVE:        resb 1 ; 0      ; Logical drive # assoc with DPB (A=0,B=1,...)
20070 .UNIT:         resb 1 ; 1      ; Driver unit number of DPB
20071 .SECTOR_SIZE:  resw 1 ; 2      ; Size of physical sector in bytes
20072 .CLUSTER_MASK: resb 1 ; 4      ; Sectors/cluster - 1
20073 .CLUSTER_SHIFT: resb 1 ; 5      ; Log2 of sectors/cluster
20074 .FIRST_FAT:    resw 1 ; 6      ; Starting record of FATs
20075 .FAT_COUNT:    resb 1 ; 8      ; Number of FATs for this drive
20076 .ROOT_ENTRIES: resw 1 ; 9      ; Number of directory entries
20077 .FIRST_SECTOR: resw 1 ; 11     ; First sector of first cluster
20078 .MAX_CLUSTER:  resw 1 ; 13     ; Number of clusters on drive + 1
20079 .FAT_SIZE:     resw 1 ; 15     ; Number of records occupied by FAT
20080 .DIR_SECTOR:   resw 1 ; 17     ; Starting record of directory
20081 .DRIVER_ADDR:  resd 1 ; 19     ; Pointer to driver
20082 .MEDIA:        resb 1 ; 23     ; Media byte
20083 .FIRST_ACCESS: resb 1 ; 24     ; This is initialized to -1 to force a media
20084                                     ; check the first time this DPB is used
20085 .NEXT_DPB:     resd 1 ; 25     ; Pointer to next Drive parameter block
20086 .NEXT_FREE:     resw 1 ; 29     ; Cluster # of last allocated cluster
20087 .FREE_CNT:      resw 1 ; 31     ; Count of free clusters, -1 if unknown

```

```

20088 ; FAT32 fs ; 01/01/2024
20089 ; ref: https://en.wikibooks.org/wiki/
20090 ; First_steps_towards_system_programming_under_MS-DOS_7/Appendix
20091 ; -- A.03-1. Structure of Drive Parameters Blocks (DPB) ---
20092 00000021 ???? .FREE_CNT_HW: resw 1 ; 33 ; High word of free cluster count
20093 00000023 ???? .EXT_FLAGS: resw 1 ; 35 ; FAT32 extended flags (active FAT number)
20094 00000025 ???? .FSINFO_SECTOR: resw 1 ; 37 ; (FAT32 fs) FSINFO structure sector address
20095 00000027 ???? .BKBOOT_SECTOR: resw 1 ; 39 ; (FAT32 fs) Backup Boot Sector address
20096 00000029 ????????? .FCLUS_FSECTOR: resd 1 ; 41 ; The first cluster's first sector address
20097 0000002D ????????? .LAST_CLUSTER: resd 1 ; 45 ; The last cluster number
20098 00000031 ????????? .FAT32_SIZE: resd 1 ; 49 ; Number of FAT sectors (for FAT32 fs)
20099 00000035 ????????? .ROOT_CLUSTER: resd 1 ; 53 ; Root directory's cluster number (FAT32 fs)
20100 ; 01/01/2024 - Retro DOS v5.0
20101 00000039 ????????? .FAT32_NXTFREE: resd 1 ; 57 ; The next free cluster (for FAT32 fs)
20102 .size: ; 61 bytes ; 01/01/2024 (PCDOS 7.1)
20103 endstruc
20104
20105 %endif
20106
20107 DPBSIZ EQU DPB.size ; Size of the structure in bytes
20108
20109 DSKSIZ EQU DPB.MAX_CLUSTER ; Size of disk (temp used during init only)
20110
20111 ; -----
20112 ; 26/03/2018
20113
20114 ; IOCTL SUB-FUNCTIONS
20115 IOCTL_GET_DEVICE_INFO EQU 0
20116 IOCTL_SET_DEVICE_INFO EQU 1
20117 IOCTL_READ_HANDLE EQU 2
20118 IOCTL_WRITE_HANDLE EQU 3
20119 IOCTL_READ_DRIVE EQU 4
20120 IOCTL_WRITE_DRIVE EQU 5
20121 IOCTL_GET_INPUT_STATUS EQU 6
20122 IOCTL_GET_OUTPUT_STATUS EQU 7
20123 IOCTL_CHANGEABLE? EQU 8
20124 IOCTL_SHARING_RETRY EQU 11
20125 GENERIC_IOCTL_HANDLE EQU 12
20126 GENERIC_IOCTL EQU 13
20127
20128 ; GENERIC IOCTL SUB-FUNCTIONS
20129 RAWIO EQU 8
20130
20131 ; RAWIO SUB-FUNCTIONS
20132 GET_DEVICE_PARAMETERS EQU 60H
20133 SET_DEVICE_PARAMETERS EQU 40H
20134 READ_TRACK EQU 61H
20135 WRITE_TRACK EQU 41H
20136 VERIFY_TRACK EQU 62H
20137 FORMAT_TRACK EQU 42H
20138
20139 ; DEVICETYPE VALUES
20140 MAX_SECTORS_IN_TRACK EQU 63
20141 DEV_5INCH EQU 0
20142 DEV_5INCH96TPI EQU 1
20143 DEV_3INCH720KB EQU 2
20144 DEV_8INCHSS EQU 3
20145 DEV_8INCHDS EQU 4
20146 DEV_HARDDISK EQU 5
20147 DEV_OTHER EQU 7
20148 ;DEV_3INCH1440KB EQU 7
20149 DEV_3INCH2880KB EQU 9
20150 ; Retro DOS v2.0 - 26/03/2018
20151 ;;DEV_TAPE EQU 6
20152 ;;DEV_ERIMO EQU 8
20153 ;DEV_3INCH2880KB EQU 9
20154 DEV_3INCH1440KB EQU 10
20155
20156 ;MAX_DEV_TYPE EQU 9 ; MAXIMUM DEVICE TYPE THAT WE
20157 ; CURRENTLY SUPPORT.
20158 MAX_DEV_TYPE EQU 10
20159
20160 struc A_SECTORTABLE
20161 00000000 ???? .ST_SECTORNUMBER: resw 1
20162 00000002 ???? .ST_SECTORSIZE: resw 1
20163 .size:
20164 endstruc
20165
20166 ; -----
20167 ; structure, equates for devmark for mem command.
20168
20169 ; 25/03/2019 - Retro DOS v4.0 (MSDOS 6.0, DEVMARK.INC, 1991)
20170
20171 struc devmark
20172 00000000 ?? .id: resb 1
20173 00000001 ???? .seg: resw 1
20174 00000003 ???? .size: resw 1
20175 00000005 ?????? .dum: resb 3
20176 00000008 ?????????????????? .filename: resb 8
20177 endstruc
20178
20179 devmark_stk equ 'S'
20180 devmark_device equ 'D'
20181 devmark_ifs equ 'I'
20182 devmark_buf equ 'B'
20183 devmark_cds equ 'L' ; lastdrive
20184 devmark_files equ 'F'
20185 devmark_fcbs equ 'X'
20186 devmark_inst equ 'T' ; used for sysinit base for install= command.
20187 devmark_ems_stub equ 'E'
20188
20189 setbrkdone equ 00000001b
20190 for_devmark equ 00000010b
20191 not_for_devmark equ 11111101b
20192
20193 ; -----
20194 ; Memory arena structure
20195
20196 ; 24/03/2019 - Retro DOS v4.0
20197 ; (MSDOS 6.0, ARENA.INC)
20198
20199 ;** Arena Header
20200
20201 struc ARENA
20202 00000000 ?? .SIGNATURE: resb 1 ; 4D for valid item, 5A for last item
20203 00000001 ???? .OWNER: resw 1 ; owner of arena item
20204 00000003 ???? .SIZE: resw 1 ; size in paragraphs of item
20205 00000005 ?????? .RESERVED resb 3 ; reserved
20206 00000008 ?????????????????? .NAME: resb 8 ; owner file name
20207 endstruc
20208
20209 ; 12/04/2019
20210
20211 arena_owner_system EQU 0 ; free block indication

```

```

20212
20213 arena_signature_normal EQU 4Dh ; valid signature, not end of arena
20214 arena_signature_end EQU 5Ah ; valid signature, last block in arena
20215
20216 ; -----
20217 ; Process data block (otherwise known as program header)
20218
20219 ; 23/03/2019 - Retro DOS v4.0
20220
20221 ; (MSDOS 6.0 - PDB.INC, 1991)
20222
20223 FILPERPROC EQU 20
20224
20225 struc PDB ; Process_data_block
20226 00000000 ???? .EXIT_CALL: resw 1 ; INT int_abort system terminate
20227 00000002 ???? .BLOCK_LEN: resw 1 ; size of execution block
20228 00000004 ?? resb 1
20229 00000005 ?????????? .CPM_CALL: resb 5 ; ancient call to system
20230 0000000A ?????????? .EXIT: resd 1 ; pointer to exit routine
20231 0000000E ?????????? .CTRL_C: resd 1 ; pointer to ^C routine
20232 00000012 ?????????? .FATAL_ABORT: resd 1 ; pointer to fatal error
20233 00000016 ???? .PARENT_PID: resw 1 ; PID of parent (terminate PID)
20234 00000018 <res 14h> .JFN_TABLE: resb FILPERPROC ; indices into system table
20235 0000002C ???? .ENVIRON: resw 1 ; seg addr of environment
20236 0000002E ?????????? .USER_STACK: resd 1 ; stack of self during system calls
20237 00000032 ???? .JFN_LENGTH: resw 1 ; number of handles allowed
20238 00000034 ?????????? .JFN_POINTER: resd 1 ; pointer to JFN table
20239 00000038 ?????????? .NEXT_PDB: resd 1 ; pointer to nested PDB's
20240 0000003C ?? .INTERCON: resb 1 ; *** jh-3/28/90 ***
20241 0000003D ?? .APPEND: resb 1 ; *** Not sure if still used ***
20242 0000003E ???? .NOVELL_USED: resb 2 ; Novell shell (redir) uses these
20243 00000040 ???? .VERSION: resw 1 ; DOS version reported to this app
20244 00000042 <res Eh> .PAD1: resb 14
20245 00000050 ???????????? .CALL_SYSTEM: resb 5 ; portable method of system call
20246 00000055 ???????????? .PAD2: resb 7 ; reserved so FCB 1 can be used as an extended FCB
20247 0000005C <res 10h> .FCB1: resb 16 ; default FCB 1
20248 0000006C <res 10h> .FCB2: resb 16 ; default FCB 2
20249 0000007C ?????????? .PAD3: resb 4 ; not sure if this is used by PDB_FCB2
20250 00000080 <res 80h> .TAIL: resb 128 ; command tail and default DTA
20251 ;.size:
20252 endstruc
20253
20254 ; -----
20255 ; <system call definitions>
20256
20257 ; 23/03/2019 - Retro DOS v4.0
20258
20259 ; (MSDOS 6.0 - SYSCALL.INC, 1991)
20260
20261 ABORT EQU 0 ; 0 0
20262 STD_CON_INPUT EQU 1 ; 1 1
20263 STD_CON_OUTPUT EQU 2 ; 2 2
20264 STD_AUX_INPUT EQU 3 ; 3 3
20265 STD_AUX_OUTPUT EQU 4 ; 4 4
20266 STD_PRINTER_OUTPUT EQU 5 ; 5 5
20267 RAW_CON_IO EQU 6 ; 6 6
20268 RAW_CON_INPUT EQU 7 ; 7 7
20269 STD_CON_INPUT_NO_ECHO EQU 8 ; 8 8
20270 STD_CON_STRING_OUTPUT EQU 9 ; 9 9
20271 STD_CON_STRING_INPUT EQU 10 ; 10 A
20272 STD_CON_INPUT_STATUS EQU 11 ; 11 B
20273 STD_CON_INPUT_FLUSH EQU 12 ; 12 C
20274 DISK_RESET EQU 13 ; 13 D
20275 SET_DEFAULT_DRIVE EQU 14 ; 14 E
20276 FCB_OPEN EQU 15 ; 15 F
20277 FCB_CLOSE EQU 16 ; 16 10
20278 DIR_SEARCH_FIRST EQU 17 ; 17 11
20279 DIR_SEARCH_NEXT EQU 18 ; 18 12
20280 FCB_DELETE EQU 19 ; 19 13
20281 FCB_SEQ_READ EQU 20 ; 20 14
20282 FCB_SEQ_WRITE EQU 21 ; 21 15
20283 FCB_CREATE EQU 22 ; 22 16
20284 FCB_RENAME EQU 23 ; 23 17
20285 GET_DEFAULT_DRIVE EQU 25 ; 25 19
20286 SET_DMA EQU 26 ; 26 1A
20287 GET_DEFAULT_DPB EQU 31 ; 31 1F
20288 FCB_RANDOM_READ EQU 33 ; 33 21
20289 FCB_RANDOM_WRITE EQU 34 ; 34 22
20290 GET_FCB_FILE_LENGTH EQU 35 ; 35 23
20291 GET_FCB_POSITION EQU 36 ; 36 24
20292 SET_INTERRUPT_VECTOR EQU 37 ; 37 25
20293 CREATE_PROCESS_DATA_BLOCK EQU 38 ; 38 26
20294 FCB_RANDOM_READ_BLOCK EQU 39 ; 39 27
20295 FCB_RANDOM_WRITE_BLOCK EQU 40 ; 40 28
20296 PARSE_FILE_DESCRIPTOR EQU 41 ; 41 29
20297 GET_DATE EQU 42 ; 42 2A
20298 SET_DATE EQU 43 ; 43 2B
20299 GET_TIME EQU 44 ; 44 2C
20300 SET_TIME EQU 45 ; 45 2D
20301 SET_VERIFY_ON_WRITE EQU 46 ; 46 2E
20302 ; Extended functionality group
20303 GET_DMA EQU 47 ; 47 2F
20304 GET_VERSION EQU 48 ; 48 30
20305 KEEP_PROCESS EQU 49 ; 49 31
20306 GET_DPB EQU 50 ; 50 32
20307 SET_CTRL_C_TRAPPING EQU 51 ; 51 33
20308 GET_INDOS_FLAG EQU 52 ; 52 34
20309 GET_INTERRUPT_VECTOR EQU 53 ; 53 35
20310 GET_DRIVE_FREESPACE EQU 54 ; 54 36
20311 CHAR_OPER EQU 55 ; 55 37
20312 INTERNATIONAL EQU 56 ; 56 38
20313 ; Directory Group
20314 MKDIR EQU 57 ; 57 39
20315 RMDIR EQU 58 ; 58 3A
20316 CHDIR EQU 59 ; 59 3B
20317 ; File Group
20318 CREAT EQU 60 ; 60 3C
20319 OPEN EQU 61 ; 61 3D
20320 CLOSE EQU 62 ; 62 3E
20321 READ EQU 63 ; 63 3F
20322 WRITE EQU 64 ; 64 40
20323 UNLINK EQU 65 ; 65 41
20324 LSEEK EQU 66 ; 66 42
20325 CHMOD EQU 67 ; 67 43
20326 IOCTL EQU 68 ; 68 44
20327 XDUP EQU 69 ; 69 45
20328 XDUP2 EQU 70 ; 70 46
20329 CURRENT_DIR EQU 71 ; 71 47
20330 ; Memory Group
20331 ALLOC EQU 72 ; 72 48
20332 DEALLOC EQU 73 ; 73 49
20333 SETBLOCK EQU 74 ; 74 4A
20334 ; Process Group
20335 EXEC EQU 75 ; 75 4B

```



```

20336 EXIT EQU 76 ; 76 4C
20337 WAITPROCESS EQU 77 ; 77 4D
20338 FIND_FIRST EQU 78 ; 78 4E
20339 ; Special Group
20340 FIND_NEXT EQU 79 ; 79 4F
20341 ; SPECIAL SYSTEM GROUP
20342 SET_CURRENT_PDB EQU 80 ; 80 50
20343 GET_CURRENT_PDB EQU 81 ; 81 51
20344 GET_IN_VARS EQU 82 ; 82 52
20345 SETDPB EQU 83 ; 83 53
20346 GET_VERIFY_ON_WRITE EQU 84 ; 84 54
20347 DUP_PDB EQU 85 ; 85 55
20348 RENAME EQU 86 ; 86 56
20349 FILE_TIMES EQU 87 ; 87 57
20350 ;
20351 ALLOCOPER EQU 88 ; 88 58
20352 ; Network extention system calls
20353 GetExtendedError EQU 89 ; 89 59
20354 CreateTempFile EQU 90 ; 90 5A
20355 CreateNewFile EQU 91 ; 91 5B
20356 LockOper EQU 92 ; 92 5C Lock and Unlock
20357 ServerCall EQU 93 ; 93 5D CommitAll, ServerDOSCall,
; CloseByName, CloseUser,
; CloseUserProcess,
; GetOpenFileList
20358 ;
20359 ;
20360 ;
20361 UserOper EQU 94 ; 94 5E Get and Set
20362 AssignOper EQU 95 ; 95 5F On, Off, Get, Set, Cancel
20363 xNameTrans EQU 96 ; 96 60
20364 PathParse EQU 97 ; 97 61
20365 GetCurrentPSP EQU 98 ; 98 62
20366 Hongeu1 EQU 99 ; 99 63
20367 ECS_CALL EQU 99 ; 99 63 ;; DBCS support
20368 Set_Printer_Flag EQU 100 ; 100 64
20369 GetExtCntry EQU 101 ; 101 65
20370 GetSetCdPg EQU 102 ; 102 66
20371 ExtHandle EQU 103 ; 103 67
20372 Commit EQU 104 ; 104 68
20373 GetSetMediaID EQU 105 ; 105 69
20374 IFS_IOCTL EQU 107 ; 107 6B
20375 ExtOpen EQU 108 ; 108 6C
20376 ;
20377 ;ifdef ROMEXEC
20378 ;ROM_FIND_FIRST EQU 109 ; 109 6D
20379 ;ROM_FIND_NEXT EQU 110 ; 110 6E
20380 ;ROM_EXCLUDE EQU 111 ; 111 6F
20381 ;endif
20382 ;
20383 Set_Oem_Handler EQU 248 ; 248 F8
20384 OEM_C1 EQU 249 ; 249 F9
20385 OEM_C2 EQU 250 ; 250 FA
20386 OEM_C3 EQU 251 ; 251 FB
20387 OEM_C4 EQU 252 ; 252 FC
20388 OEM_C5 EQU 253 ; 253 FD
20389 OEM_C6 EQU 254 ; 254 FE
20390 OEM_C7 EQU 255 ; 255 FF
20391 ;
20392 ; -----
20393 ; SYSCONF.ASM (MSDOS 3.3 - 24/07/1987)
20394 ; -----
20395 ;
20396 ;; IF STACKSW
20397 ;
20398 ;;
20399 ;; Internal Stack Parameters
20400 ;EntrySize equ 8
20401 ;
20402 ;MinCount equ 8
20403 ;DefaultCount equ 9
20404 ;MaxCount equ 64
20405 ;
20406 ;MinSize equ 32
20407 ;DefaultSize equ 128
20408 ;MaxSize equ 512
20409 ;
20410 ;; ENDIF
20411 ;
20412 ; -----
20413 ; BIOSTRUC.INC (MSDOS 3.3 - 24/07/1987)
20414 ; -----
20415 ;;;Rev 3.30 Modification
20416 ; ROM BIOS CALL PACKET STRUCTURES
20417 ;
20418 ;*****
20419 ;System Service call ( Int 15h )
20420 ;*****
20421 ;Function AH = 0C0h, Return system configuration
20422 ;For PC and PCJR on return:
20423 ; (AH) = 80h
20424 ; (CY) = 1
20425 ;For PCXT, PC PORTABLE and PCAT on return:
20426 ; (AH) = 86h
20427 ; (CY) = 1
20428 ;For all others:
20429 ; (AH) = 0
20430 ; (CY) = 0
20431 ; (ES:BX) = pointer to system descriptor vector in ROS
20432 ; System descriptor :
20433 ; DW xxxx length of descriptor in bytes,
20434 ; minimum length = 8
20435 ; DB xx model byte
20436 ; 0FFh = PC
20437 ; 0FEh = PC/XT, Portable
20438 ; 0FDh = PC/JR
20439 ; 0Fch = PC/AT
20440 ; 0F9h = Convertable
20441 ; 0F8h = Model 80
20442 ; 0E0 thru 0EFh = reserved
20443 ;
20444 ; DB xx secondary model byte
20445 ; 000h = PC1
20446 ; 000h = PC/XT, Portable
20447 ; 000h = PC/JR
20448 ; 000h = PC/AT
20449 ; 001h = PC/AT Model 339
20450 ; 003h = PC/RT
20451 ; 000h = Convertable
20452 ;
20453 ; DB xx bios revision level
20454 ; 00 for first release, subsequent release
20455 ; of code with same model byte and
20456 ; secondary model byte require revision level
20457 ; to increase by one.
20458 ;
20459 ; DB xx feature information byte 1

```

```

20460 ; X0000000 = 1, bios use DMA channel 3
20461 ; = 0, DMA channel 3 not used
20462 ;
20463 ; 0x000000 = 1, 2nd Interrupt chip present
20464 ; = 0, 2nd Interrupt chip not present
20465 ;
20466 ; 00x00000 = 1, Real Time Clock present
20467 ; = 0, Real Time Clock not present
20468 ;
20469 ; 000x0000 = 1, Keyboard escape sequence(INT 15h)
20470 ; called in keyboard interrupt
20471 ; (Int 09h).
20472 ; = 0, Keyboard escape sequence not
20473 ; called.
20474 ; 0000xxxx reserved
20475 ;
20476 ; DB xx feature information byte 2 - reserved
20477 ;
20478 ; DB xx feature information byte 2 - reserved
20479 ;
20480 ; DB xx feature information byte 2 - reserved
20481 ;
20482 ; DB xx feature information byte 2 - reserved
20483 ;
20484 ;
20485 ; 22/03/2019
20486 struct ROMBIOS_DESC ; BIOS_SYSTEM_DESCRIPTOR
20487 .bios_sd_leng: resw 1
20488 .bios_sd_modelbyte: resb 1
20489 .bios_sd_scnd_modelbyte: resb 1
20490 .bios_sd_scnd_modelbyte: resb 1
20491 .bios_sd_scnd_modelbyte: resb 1
20492 .bios_sd_featurebyte1: resb 1
20493 .bios_sd_featurebyte1: resb 4
20494 endstruc
20495 ;
20496 ;FeatureByte1 bit map equates
20497 DMAchannel3 equ 10000000b
20498 ScndIntController equ 01000000b
20499 RealTimeClock equ 00100000b
20500 KeyEscapeSeq equ 00010000b
20501 ;;End of Modification
20502 ;
20503 ; -----
20504 ; SYSVAR.INC (MSDOS 6.0 - 1991)
20505 ; -----
20506 ; 22/03/2019 - Retro DOS v4.0
20507 ;
20508 ; SCCSID = @(#)sysvar.asm 1.1 85/04/10
20509 ;
20510 struct SysInitVars
20511 ; MSDOS 3.3
20512 .SYSI_DPB: resd 1 ; DPB chain
20513 .SYSI_SFT: resd 1 ; SFT chain
20514 .SYSI_CLOCK: resd 1 ; CLOCK device
20515 .SYSI_CON: resd 1 ; CON device
20516 .SYSI_MAXSEC: resw 1 ; maximum sector size
20517 .SYSI_BUF: resd 1 ; buffer chain
20518 .SYSI_CDS: resd 1 ; CDS list
20519 .SYSI_FCB: resd 1 ; FCB chain
20520 .SYSI_KEEP: resw 1 ; keep count
20521 .SYSI_NUMIO: resb 1 ; number of block devices
20522 .SYSI_NCDS: resb 1 ; number of CDS's
20523 .SYSI_DEV: resd 1 ; device list
20524 ; MSDOS 6.0
20525 .SYSI_ATTR: resw 1 ; null device attribute word
20526 .SYSI_STRAT: resw 1 ; null device strategy entry point
20527 .SYSI_INTER: resw 1 ; null device interrupt entry point
20528 .SYSI_NAME: resb 8 ; null device name
20529 .SYSI_SPLICE: resb 0 ; TRUE -> splices being done
20530 .SYSI_IBMDOS_SIZE: resw 1 ; DOS size in paragraphs
20531 .SYSI_IFS_DOSCALL@: resd 1 ; IFS DOS service routine entry
20532 .SYSI_IFS: resd 1 ; IFS header chain
20533 .SYSI_BUFFERS: resw 2 ; BUFFERS= values (m,n)
20534 .SYSI_BOOT_DRIVE: resb 1 ; boot drive A=1 B=2,...
20535 .SYSI_DWMOVE: resb 1 ; 1 if 386 machine
20536 .SYSI_EXT_MEM: resw 1 ; Extended memory size in KB.
20537 .size:
20538 endstruc
20539 ;
20540 ;This is added for more information exchange between DOS, BIOS.
20541 ;DOS will give the pointer to SysInitTable in ES:DI. - J.K. 5/29/86
20542 ;
20543 ; 22/03/2019
20544 struct SysInitVars_Ext
20545 .SYSI_InitVars: resd 1 ; Points to the above structure.
20546 .SYSI_Country_Tab: resd 1 ; DOS_Country_cdpdg_info
20547 endstruc
20548 ;
20549 ; 09/06/2018
20550 ; 08/06/2018 - Retro DOS v3.0 (MSDOS 3.3)
20551 SYSI_DPB equ 0
20552 SYSI_SFT equ 4
20553 SYSI_CLOCK equ 8
20554 SYSI_CON equ 12
20555 SYSI_MAXSEC equ 16
20556 SYSI_BUF equ 18
20557 SYSI_CDS equ 22
20558 SYSI_FCB equ 26
20559 SYSI_KEEP equ 30
20560 SYSI_NUMIO equ 32
20561 SYSI_NCDS equ 33
20562 SYSI_DEV equ 34
20563 ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0)
20564 SYSI_ATTR equ 38
20565 SYSI_STRAT equ 40
20566 SYSI_INTER equ 42
20567 SYSI_NAME equ 44
20568 SYSI_SPLICE equ 52
20569 SYSI_IBMDOS_SIZE equ 53
20570 SYSI_IFS_DOSCALL@ equ 55
20571 SYSI_IFS equ 59
20572 SYSI_BUFFERS equ 63
20573 SYSI_BOOT_DRIVE equ 67
20574 SYSI_DWMOVE equ 68
20575 SYSI_EXT_MEM equ 69
20576 ;
20577 ;The SYSI_BUF of SysInitVars points to the following structure
20578 ;
20579 EMS_MAP_BUFF_SIZE EQU 12 ; EMS map buffer size
20580 ;
20581 struct BUFFINF ; BUFFINFO
20582 .Buff_Queue: resd 1 ; Head of list of buffers
20583 .Dirty_Buff_Count: resw 1 ; number of dirty buffers in list

```



```

20584 00000006 ???????? .Cache_ptr:    resd 1    ; pointer to secondary cache
20585 0000000A ????.      .Cache_count:    resw 1    ; number of secondary cache entries
20586
20587 0000000C ??        .Buff_In_HMA:    resb 1    ; flag to indicate that buffers
20588                        ; are in HMA
20589 0000000D ???????? .Lo_Mem_Buff:    resd 1    ; Ptr to scratch buff in Low Mem
20590                        ; used to read/write on disks
20591 00000011 ???????? .UU_EMS_FIRST_PAGE: resw 2
20592 00000015 ????.      .UU_EMS_NPA640:    resw 1
20593 00000017 ??        .UU_EMS_mode:    resb 1    ; no EMS = -1
20594 00000018 ????.      .UU_EMS_handle:    resw 1    ; EMS handle for buffers
20595 0000001A ????.      .UU_EMS_PageFrame_Number: resw 1 ; EMS page frame number
20596 0000001C ????.      .UU_EMS_Seg_Cnt:    resw 1    ; EMS segment count
20597 0000001E ????.      .UU_EMS_Page_Frame:    resw 1    ; EMS page frame segment address
20598 00000020 ????.      .UU_EMS_reserved: resw 1    ; EMS segment count
20599 00000022 ??        .UU_EMS_Map_Buff: resb 1    ; map buffer
20600 .size:
20601 endstruc
20602
20603 ; -----
20604 ; CURDIR.INC (MSDOS 6.0 - 1991)
20605 ; -----
20606 ; 22/03/2019 - Retro DOS v4.0
20607
20608 ;** CDS - Current Directory Structure
20609 ;
20610 ; CDS items are used by the internal routines to store cluster numbers and
20611 ; network identifiers for each logical name. The ID field is used dually,
20612 ; both as net ID and for a cluster number for local devices. In the case
20613 ; of local devices, the cluster number will be -1 if there is a potential
20614 ; of the disk being changed or if the path must be rechecked.
20615 ;
20616 ; Some pathnames have special preambles, such as
20617 ;
20618 ;     \\machine\sharename\...
20619 ; For these pathnames we can't allow ".." processing to back us
20620 ; up into the special front part of the name. The CURDIR_END field
20621 ; holds the address of the separator character which marks
20622 ; the split between the special preamble and the regular
20623 ; path list; ".." processing isn't allowed to back us up past
20624 ; (i.e., before) CURDIR_END
20625 ; For the root, it points at the leading /. For net
20626 ; assignments it points at the end (nul) of the initial assignment:
20627 ; A:/      \\foo\bar      \\foo\bar\blech\bozo
20628 ;   ^      ^              ^
20629
20630 DIRSTRLEN EQU 64+3    ; Max length in bytes of directory strings
20631 TEMPLEN EQU DIRSTRLEN*2
20632
20633 struc      curdir_list
20634 ; MSDOS 3.3
20635 00000000 <res 43h> .cdir_text resb DIRSTRLEN    ; text of assignment and curdir
20636 00000043 ????.      .cdir_flags resw 1    ; various flags
20637 00000045 ???????? .cdir_devptr resd 1    ; local pointer to DPB or net device
20638 00000049 ???????? .cdir_ID resw 2    ; cluster of current dir (net ID)
20639 0000004D ????.      .cdir_usr_word resw 1
20640 0000004F ????.      .cdir_end resw 1    ; end of assignment
20641 ; MSDOS 6.0
20642 00000051 ??        .cdir_type: resb 1    ; IFS drive (2=ifs, 4=netuse)
20643 00000052 ???????? .cdir_ifd_hdr: resd 1    ; Ptr to File System Header
20644 00000056 ????.      .cdir_fsda: resb 2    ; File System Dependent Data Area
20645 .size:
20646 endstruc
20647
20648 curdirilen EQU curdir_list.size    ; Needed for screwed up
20649 ; ASM87 which doesn't allow
20650 ; size directive as a macro
20651 ; argument
20652 %define curdir_netID    dword [curdir_list.cdir_ID]
20653
20654 ;** Flag values for CURDIR_FLAGS
20655 ;
20656 ; Flag word masks
20657 curdir_isnet EQU 1000000000000000B
20658 curdir_isifs EQU 1000000000000000B
20659 curdir_inuse EQU 0100000000000000B
20660 curdir_splice EQU 0010000000000000B
20661 curdir_local EQU 0001000000000000B
20662
20663 ; -----
20664 ; SF.INC (MSDOS 6.0 - 1991)
20665 ; -----
20666 ; 25/03/2019 - Retro DOS v4.0
20667
20668 ; 09/04/2024 - Retro DOS v4.2 (BugFix)
20669 ; 09/04/2024 - Retro DOS v5.0
20670
20671 ; system file table
20672
20673 ;** System File Table SuperStructure
20674 ;
20675 ; The system file table entries are allocated in contiguous groups.
20676 ; There may be more than one such groups; the SF "superstructure"
20677 ; tracks the groups.
20678
20679 struc      SF
20680 00000000 ???????? .SFLink:    resd 1
20681 00000004 ????.      .SFCount:    resw 1    ; number of entries
20682 00000006 ????.      .SFTable:    resw 1    ; beginning of array of the following
20683 .size:
20684 endstruc
20685
20686 ;** System file table entry
20687 ;
20688 ; These are the structures which are at SFTABLE in the SF structure.
20689
20690 struc      SF_ENTRY
20691 00000000 ????.      .sf_ref_count: resw 1    ; number of processes sharing entry
20692 ; if FCB then ref count
20693 00000002 ????.      .sf_mode:    resw 1    ; mode of access or high bit on if FCB
20694 00000004 ??        .sf_attr:    resb 1    ; attribute of file
20695 00000005 ????.      .sf_flags:    resw 1    ; Bits 8-15
20696 ; Bit 15 = 1 if remote file
20697 ; = 0 if local file or device
20698 ; Bit 14 = 1 if date/time is not to be
20699 ; set from clock at CLOSE. Set by
20700 ; FILETIMES and FCB_CLOSE. Reset by
20701 ; other reseters of the dirty bit
20702 ; (WRITE)
20703 ; Bit 13 = Pipe bit (reserved)
20704 ;
20705 ; Bits 0-7 (old FCB_devid bits)
20706 ; If remote file or local file, bit
20707 ; 6=0 if dirty Device ID number, bits

```

```

20708                                     ; 0-5 if local file.
20709                                     ; bit 7=0 for local file, bit 7
20710                                     ; =1 for local I/O device
20711                                     ; If local I/O device, bit 6=0 if EOF (input)
20712                                     ; Bit 5=1 if Raw mode
20713                                     ; Bit 0=1 if console input device
20714                                     ; Bit 1=1 if console output device
20715                                     ; Bit 2=1 if null device
20716                                     ; Bit 3=1 if clock device
20717 00000007 ????????? .sf_devptr: resd 1 ; Points to DPB if local file, points
20718                                     ; to device header if local device,
20719                                     ; points to net device header if
20720                                     ; remote
20721 0000000B ???? .sf_firclus: resw 1 ; First cluster of file (bit 15 = 0)
20722 ;.sf_lstclus: resw 1 ; *
20723 0000000D ???? .sf_time: resw 1 ; Time associated with file
20724 0000000F ???? .sf_date: resw 1 ; Date associated with file
20725 00000011 ????????? .sf_size: resd 1 ; Size associated with file
20726 00000015 ????????? .sf_position: resd 1 ; Read/write pointer or LRU count for FCBS
20727 ;
20728 ; Starting here, the next 7 bytes may be used by the file system to store an
20729 ; ID
20730 ;
20731 00000019 ???? .sf_cluspos: resw 1 ; Position of last cluster accessed
20732 0000001B ????????? .sf_dirsec: resd 1 ; 09/04/2024 ; Sector number of directory sector for this file
20733 0000001F ?? .sf_dirpos: resb 1 ; Offset of this entry in the above
20734 ;
20735 ; End of 7 bytes of file-system specific info.
20736 ;
20737 00000020 <res Bh> .sf_name: resb 11 ; 11 character name that is in the
20738                                     ; directory entry. This is used by
20739                                     ; close to detect file deleted and
20740                                     ; disk changed errors.
20741 ; SHARING INFO
20742 0000002B ????????? .sf_chain: resd 1 ; link to next SF
20743 0000002F ???? .sf_UID: resw 1
20744 00000031 ???? .sf_PID: resw 1
20745 00000033 ???? .sf_MFT: resw 1
20746 00000035 ???? .sf_lstclus: resw 1 ; * ; Last cluster accessed
20747 00000037 ????????? .sf_IFS_HDR: resd 1 ; **
20748 .size:
20749 endstruc
20750
20751 ; -----
20752 ; DOSCNTRY.INC (MSDOS 3.3 - 24/07/1987)
20753 ; -----
20754 ; 11/06/2018 - Retro DOS v3.0
20755
20756 ; Equates for COUNTRY INFORMATION.
20757 SetCountryInfo EQU 1 ; country info
20758 SetUcase EQU 2 ; uppercase table
20759 SetLcase EQU 3 ; lowercase table (Reserved)
20760 SetUcaseFile EQU 4 ; uppercase file spec table
20761 SetFileList EQU 5 ; valid file character list
20762 SetCollate EQU 6 ; collating sequence
20763 SetDBCS EQU 7 ; double byte character set
20764 SetALL EQU -1 ; all the entries
20765
20766 ; DOS country and code page information table structure.
20767 ; Internally, IBMDOS gives a pointer to this table.
20768 ; IBMBIO, MODE and NLSFUNC modules communicate with IBMDOS through
20769 ; this structure.
20770
20771 struc country_cdpq_info ; DOS_country_cdpq_info
20772 .ccInfo_reserved: resb 8 ; reserved for internal use
20773 .ccPath_CountrySys: resb 64 ; path and filename for country info
20774 .ccSysCodePage: resw 1 ; system code page id
20775 .ccNumber_of_entries: resw 1 ; dw 5
20776 .ccSetUcase: resb 1 ; db SetUcase ; = 2
20777 .ccUcase_ptr: resd 1 ; pointer to Ucase table
20778
20779 .ccSetUcaseFile: resb 1 ; db SetUcaseFile ; = 4
20780 .ccFileUcase_ptr: resd 1 ; pointer to File Ucase table
20781
20782 .ccSetFileList: resb 1 ; db SetFileList ; = 5
20783 .ccFileChar_ptr: resd 1 ; pointer to File char list table
20784
20785 .ccSetCollate: resb 1 ; db SetCollate ; = 6
20786 .ccCollate_ptr: resd 1 ; pointer to collate table
20787
20788 .ccSetCountryInfo: resb 1 ; db SetCountryInfo ; = 1
20789 .ccCountryInfoLen: resw 1 ; length of country info
20790 .ccDosCountry: resw 1 ; system country code id
20791 .ccDosCodePage: resw 1 ; system code page id
20792 .ccDFormat: resw 1 ; date format
20793 .ccCurSymbol: resb 5 ; db " ",0
20794 ; 5 byte of (currency symbol+0)
20795 .cc1000Sep: resb 2 ; db " ",0 ; 2 byte of (1000 sep. + 0)
20796 .ccDecSep: resb 2 ; db " ",0 ; 2 byte of (Decimal sep. + 0)
20797 .ccDateSep: resb 2 ; db " ",0 ; 2 byte of (date sep. + 0)
20798 .ccTimeSep: resb 2 ; db " ",0 ; 2 byte of (time sep. + 0)
20799 .ccCFormat: resb 1 ; currency format flags
20800 .ccSigDigits: resb 1 ; # of digits in currency
20801 .ccTFormat: resb 1 ; time format
20802 .ccMono_Ptr: resd 1 ; monospace routine entry point
20803 .ccListSep: resb 2 ; db " ",0 ; data list separator
20804 .ccReserved_area: resw 5 ; dw 5 dup(?) ; reserved
20805 .size:
20806 endstruc
20807
20808 NEW_COUNTRY_SIZE equ country_cdpq_info.size - country_cdpq_info.ccDosCountry
20809
20810 ; =====
20811 ; retrodos4.s (offset addresses in MSDOS.SYS or RETRODOS.SYS)
20812 ; =====
20813 ; 21/03/2019 - Retro DOS v4.0
20814 ; 21/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
20815
20816 ; KERNEL_SEGMENT equ 0070h ; (IO.SYS loading segment, BIOS_DATA segment)
20817 ; 21/10/2022
20818 DOSBIODATASEG equ 0070h ; (IO.SYS loading segment, BIOS_DATA segment)
20819 ; 22/10/2022
20820 DOSBIOCODESEG equ 02C7h ; (MSDOS 5.0 IO.SYS, BIOS_CODE segment)
20821 ; 09/12/2022
20822 DOSBIOCODESEG equ IOSYSCODESEG
20823
20824 ; Note: These offset addresses must be changed when the code
20825 ; in retrodos4.s (MSDOS.SYS) file will be changed.
20826
20827 ; (following addresses can be verified by searching them in retrodos4.lst)
20828
20829 ; 09/12/2022
20830 %if 0
20831

```

```

20832 ; 13/05/2019
20833
20834 ;Iswin386 equ 08CFh
20835 ;V86_Crit_SetFocus equ 08D0h
20836 ; 21/10/2022
20837 Iswin386 equ 08D0h
20838 V86_Crit_SetFocus equ 08D1h
20839
20840 ;seg_reinit equ 0772h ; not used in Retro DOS v4.0
20841 ; 21/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
20842 seg_reinit equ 0032h ; DOSBIOCODESEG:0032h
20843
20844 ;SysinitPresent equ 08FCh
20845 ; 21/10/2022
20846 SysinitPresent equ 08FDh
20847
20848 inHMA equ 000Dh
20849 xms equ 000Eh
20850 ;FreeHMAPtr equ 08F6h
20851 ;multrk_flag equ 0533h
20852 ;ec35_flag equ 0535h
20853 ;EOT equ 012Eh
20854 ; 21/10/2022
20855 FreeHMAPtr equ 08F7h
20856 multrk_flag equ 052Fh
20857 ec35_flag equ 0531h
20858 EOT equ 012Ch
20859
20860 ;NextStack equ 08BFh
20861 ;IT_StackLoc equ 08C5h
20862 ;IT_StackSize equ 08C9h
20863 ; 21/10/2022
20864 NextStack equ 08C0h
20865 IT_StackLoc equ 08C6h
20866 IT_StackSize equ 08CAh
20867
20868 ;MoveDOSIntoHMA equ 08F8h
20869 ; 21/10/2022
20870 MoveDOSIntoHMA equ 08F9h
20871
20872 ;INT19SEM equ 0644h ; 01/05/2019 - retrodos4.lst
20873 ;I19_LST equ 0645h ; 27/03/2019 - retrodos4.lst
20874 ; 21/10/2022
20875 INT19SEM equ 0640h ; (iosys5.txt)
20876 I19_LST equ 0641h ; (iosys5.txt)
20877
20878 %endif
20879
20880 ; 09/12/2022
20881 seg_reinit equ _seg_reinit
20882 ec35_flag equ ec35flag
20883 INT19SEM equ int19sem
20884 I19_LST equ i19_lst
20885
20886 INT19OLD02 equ I19_LST+1 ; 0642h ; 21/10/2022
20887 INT19OLD08 equ I19_LST+6
20888 INT19OLD09 equ I19_LST+11
20889 INT19OLD0A equ I19_LST+16
20890 INT19OLD0B equ I19_LST+21
20891 INT19OLD0C equ I19_LST+26
20892 INT19OLD0D equ I19_LST+31
20893 INT19OLD0E equ I19_LST+36
20894 INT19OLD70 equ I19_LST+41
20895 INT19OLD72 equ I19_LST+46
20896 INT19OLD73 equ I19_LST+51
20897 INT19OLD74 equ I19_LST+56
20898 INT19OLD76 equ I19_LST+61
20899 INT19OLD77 equ I19_LST+66 ; 0683h ; 21/10/2022
20900
20901 ; 09/12/2022
20902 %if 0
20903
20904 ;keyrd_func equ 04E9h
20905 ;keysts_func equ 04EAh
20906 ;t_switch equ 04F6h
20907 ; 21/10/2022
20908 keyrd_func equ 04E5h
20909 keysts_func equ 04E6h
20910 t_switch equ 04F2h
20911
20912 ; 22/10/2022
20913 SYSINITSEG equ 046Dh ; SYSINIT segment
20914 BCODE_END equ (SYSINITSEG-DOSBIOCODESEG)*16 ; = 1A60h
20915 BCODE_START equ 30h ; (offset BiosDataWord in DOSBIOCODESEG)
20916 RE_INIT equ 089Bh ; (re_init offset in DOSBIODATASEG)
20917
20918 %endif
20919
20920 ; 09/12/2022
20921 BCODESTART equ BIOSDATAWORD
20922 RE_INIT equ re_init
20923
20924 ; -----
20925 ; CONFIG.INC (MSDOS 6.0 - 1991)
20926 ; -----
20927 ; 15/04/2019 - Retro DOS v4.0
20928
20929 CONFIG_BEGIN equ '['
20930 CONFIG_BREAK equ 'C'
20931 CONFIG_BUFFERS equ 'B'
20932 CONFIG_COMMENT equ 'Y'
20933 CONFIG_COUNTRY equ 'Q'
20934 CONFIG_DEVICE equ 'D'
20935 CONFIG_DEVICEHIGH equ 'U'
20936 CONFIG_DOS equ 'H'
20937 CONFIG_DRIVPARM equ 'P'
20938 CONFIG_FCBS equ 'X'
20939 CONFIG_FILES equ 'F'
20940 CONFIG_INCLUDE equ 'J'
20941 CONFIG_INSTALL equ 'I'
20942 CONFIG_INSTALLHIGH equ 'W'
20943 CONFIG_LASTDRIVE equ 'L'
20944 CONFIG_MENUCOLOR equ 'R'
20945 CONFIG_MENUEFAULT equ 'A'
20946 CONFIG_MENUITEM equ 'E'
20947 CONFIG_MULTITRACK equ 'M'
20948 CONFIG_NUMLOCK equ 'N'
20949 CONFIG_REM equ 'O'
20950 CONFIG_SEMICOLON equ ';'
20951 CONFIG_SET equ 'V'
20952 CONFIG_SHELL equ 'S'
20953 CONFIG_STACKS equ 'K'
20954 CONFIG_SUBMENU equ 'O'
20955 CONFIG_SWITCHES equ 'I'

```

```

20956 CONFIG_UNKNOWN equ 'Z'
20957
20958 ; 13/05/2024 - Retro DOS v5.0 (PCDOS 71 IBMBIO.COM)
20959 CONFIG_DOSDATA equ 'T'
20960
20961 CONFIG_OPTION_QUERY equ 80h
20962
20963 ; -----
20964 ; SYSINIT1.ASM (MSDOS 6.0 - 1991)
20965 ; -----
20966 ; 21/03/2019 - Retro DOS v4.0
20967
20968 true equ 0FFFFh
20969 false equ 0
20970 cr equ 13
20971 lf equ 10
20972 tab equ 9
20973
20974 multMULT equ 4Ah
20975 multMULTGETHMAPTR equ 1
20976 multMULTALLOCHMA equ 2
20977
20978 ;NOEXEC equ FALSE
20979
20980 stacksw equ true ;include switchable hardware stacks
20981 mycds_size equ 88 ;size of curdir_list. if it is not
20982 ;the same, then will generate compile error.
20983
20984 entrysize equ 8
20985
20986 mincount equ 8
20987 defaultcount equ 9
20988 maxcount equ 64
20989
20990 minsize equ 32
20991 defaultsize equ 128
20992 maxsize equ 512
20993
20994 ;%define allocbyte byte [es:bp+0]
20995 ;%define intlevel byte [es:bp+1]
20996 ;%define savedsp word [es:bp+2]
20997 ;%define savedss word [es:bp+4]
20998 ;%define newsp word [es:bp+6]
20999
21000 allocbyte equ 0
21001 intlevel equ 1
21002 savedsp equ 2
21003 savedss equ 4
21004 newsp equ 6
21005
21006 free equ 0
21007 allocated equ 1
21008 overflowed equ 2
21009 clobbered equ 3
21010
21011 ;-----
21012 ; external variable defined in ibmbio module for multi-track
21013
21014 multrk_on equ 10000000b ;user specified mutitrack=on, or system turns
21015 ; it on after handling config.sys file as a
21016 ; default value, if multrk_flag = multrk_off1.
21017 multrk_off1 equ 00000000b ;initial value. no "multitrack=" command entered.
21018 multrk_off2 equ 00000001b ;user specified multitrack=off.
21019
21020 ; SYSINITSEG SEGMENT PUBLIC 'SYSTEM_INIT'
21021
21022 SYSINIT$:
21023 ;IF STACKSW
21024 ; include MSSTACK.INC ;Main stack program and data definitions
21025 ; include STKMES.INC ;Fatal stack error message
21026 ; public Endstackcode
21027 ;Endstackcode label byte
21028 ;ENDIF
21029
21030 ; 05/07/2018
21031 ; -----
21032 ; 04/06/2018 - Retro DOS v3.0
21033
21034 ; -----
21035 ; 21/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS - SYSINIT)
21036 ; -----
21037
21038 ; MSStack.inc
21039 ;
21040 ; Interrupt level 2, 3, 4, 5, 6, 7, (10, 11, 12, 14, 15 - AT level)
21041 ; should follow the standard Interrupt Sharing Scheme which has
21042 ; a standard header structure.
21043 ; Fyi, the following shows the relations between
21044 ; the interrupt vector and interrupt level.
21045 ;
21046 ; VEC(Hex) 2 8 9 A B C D E 70 72 73 74 76 77
21047 ; LVL(Deci) 9 0 1 2 3 4 5 6 8 10 11 12 14 15
21048 ; MSSTACK module modifies the following interrupt vectors
21049 ; to meet the standard Interrupt Sharing standard;
21050 ; A, B, C, D, E, 72, 73, 74, 76, 77.
21051 ; Also, for interrupt level 7 and 15, the FirstFlag in a standard header
21052 ; should be initialized to indicat whether this interrupt handler is
21053 ; the first (= 80h) or not. The FirstFlag entry of INT77h's
21054 ; program header is initialized in STKINIT.INC module.
21055 ; FirstFlag is only meaningful for interrupt level 7 and 15.
21056 ;
21057 ;
21058 ; User specifies the number of stack elements - default = 9
21059 ; minimum = 8
21060 ; maximum = 64
21061 ;
21062 ; Intercepts Asynchronous Hardware Interrupts only
21063 ;
21064 ; Picks a stack from pool of stacks and switches to it
21065 ;
21066 ; Calls the previously saved interrupt vector after pushing flags
21067 ;
21068 ; On return, returns the stack to the stack pool
21069 ;
21070 ;
21071 ; This is a modification of STACKS:
21072 ; 1. To fix a bug which was causing the program to take up too much space.
21073 ; 2. To dispense stack space from hi-mem first rather than low-mem first.
21074 ; . Clobbers the stack that got too big instead of innocent stack
21075 ; . Allows system to work if the only stack that got too big was the most
21076 ; deeply nested one
21077 ; 3. Disables NMI interrupts while setting the NMI vector.
21078 ; 4. Does not intercept any interrupts on a PCjr.
21079 ; 5. Double checks that a nested interrupt didn't get the same stack.

```

```

21080 ; 6. Intercepts Ints 70, 72-77 for PC-ATs and other future products
21081
21082 ;EVEN
21083 ;align 2
21084 ; 21/10/2022
21085
21086 dw 0 ; spare field but leave these in order
21087 stackcount: dw 0
21088 stackat: dw 0
21089 stacksize: dw 0
21090 stacks: dw 0
21091 dw 0
21092
21093 firstentry: dw stacks
21094 lastentry: dw stacks+(defaultcount*entrysize)-entrysize
21095 nextentry: dw stacks+(defaultcount*entrysize)-entrysize
21096
21097 ;*****
21098 ; THESE ARE THE INDIVIDUAL INTERRUPT HANDLERS
21099
21100 ; -----
21101
21102 old02: dd 0
21103
21104 int02:
21105 ; *****
21106 ;
21107 ; this is special support for the pc convertible / nmi handler
21108 ;
21109 ; on the pc convertible, there is a situation where an nmi can be
21110 ; caused by using the "out" instructions to certain ports. when this
21111 ; occurs, the pc convertible hardware *guarantees* that **nothing**
21112 ; can stop the nmi or interfere with getting to the nmi handler. this
21113 ; includes other type of interrupts (hardware and software), and
21114 ; also includes other type of nmi's. when any nmi has occurred,
21115 ; no other interrupt (hardware, software or nmi) can occur until
21116 ; the software takes specific steps to allow further interrupting.
21117 ;
21118 ; for pc convertible, the situation where the nmi is generated by the
21119 ; "out" to a control port requires "fixing-up" and re-attempting. in
21120 ; otherwords, it is actually a "restartable exception". in this
21121 ; case, the software handler must be able to get to the stack in
21122 ; order to figure out what instruction caused the problem, where
21123 ; it was "out"ing to and what value it was "out"ing. therefore,
21124 ; we will not switch stacks in this situation. this situation is
21125 ; detected by interrogating port 62h, and checking for a bit value
21126 ; of 80h. if set, *****do not switch stacks*****.
21127 ;
21128 ; *****
21129 ;
21130 ;
21131 push ax
21132 push es
21133 mov ax,0F000h
21134 mov es,ax
21135 ; 02/11/2022
21136 cmp byte [es:0FFFEh],0F9h ; mdl_convert ; check if convertible
21137 pop es
21138 jne short normal02
21139
21140 in al,62h ; PC/XT PPI port C. Bits:
21141 ; 0-3: values of DIP switches
21142 ; 5: 1=Timer 2 channel out
21143 ; 6: 1=I/O channel check
21144 ; 7: 1=RAM parity check error occurred.
21145 test al,80h
21146 jz short normal02
21147 special02:
21148 pop ax
21149 jmp far [cs:old02]
21150 normal02:
21151 pop ax
21152 call do_int_stacks
21153 dw old02
21154
21155 ; -----
21156
21157 old08: dd 0
21158
21159 int08:
21160 call do_int_stacks
21161 dw old08
21162
21163 ; -----
21164
21165 old09: dd 0
21166
21167 int09:
21168 ; keyboard interrupt must have a three byte jump, a nop and a zero byte
21169 ; as its first instruction for compatibility reasons
21170
21171 jmp short keyboard_1b1
21172 nop
21173 db 0
21174
21175 keyboard_1b1:
21176 call do_int_stacks
21177 dw old09
21178
21179 ; -----
21180
21181 old70: dd 0
21182
21183 int70:
21184 call do_int_stacks
21185 dw old70
21186
21187 ; -----
21188
21189 ; irp a,<0a,0b,0c,0d,0e,72,73,74,76,77>
21190 ;public int&a
21191 ;public old&a
21192 ;public firstflag&a
21193 ;int&a proc far
21194 ; jmp short entry_int&a&_stk
21195 ;old&a dd 0 ;forward pointer
21196 ; dw 424bh ;compatible signature for int. sharing
21197 ;firstflag&a db 0 ;the firstly hooked.
21198 ; jmp short intret_&a ;reset routine. we don't care this.
21199 ; db 7 dup (0) ;reserved for future.
21200 ;entry_int&a&_stk:
21201 ; call do_int_stacks
21202 ; dw old&a
21203

```

```

21204 ;intret_&a:
21205 ;      iret
21206 ;int&a      endp
21207 ;      endm
21208
21209 ; -----
21210
21211 int0A:
21212     jmp     short entry_int0A_stk
21213 old0A:     dd     0
21214     dw     424Bh
21215 firstflag0A:
21216     db     0
21217     jmp     short intret_0A
21218     times 7 db 0
21219
21220 entry_int0A_stk:
21221     call    do_int_stacks
21222     dw     old0A
21223 intret_0A:
21224     iret
21225
21226 ; -----
21227
21228 int0B:
21229     jmp     short entry_int0B_stk
21230 old0B:     dd     0
21231     dw     424Bh
21232 firstflag0B:
21233     db     0
21234     jmp     short intret_0B
21235     times 7 db 0
21236
21237 entry_int0B_stk:
21238     call    do_int_stacks
21239     dw     old0B
21240 intret_0B:
21241     iret
21242
21243 ; -----
21244
21245 int0C:
21246     jmp     short entry_int0C_stk
21247 old0C:     dd     0
21248     dw     424Bh
21249 firstflag0C:
21250     db     0
21251     jmp     short intret_0C
21252     times 7 db 0
21253
21254 entry_int0C_stk:
21255     call    do_int_stacks
21256     dw     old0C
21257 intret_0C:
21258     iret
21259
21260 ; -----
21261
21262 int0D:
21263     jmp     short entry_int0D_stk
21264 old0D:     dd     0
21265     dw     424Bh
21266 firstflag0D:
21267     db     0
21268     jmp     short intret_0D
21269     times 7 db 0
21270
21271 entry_int0D_stk:
21272     call    do_int_stacks
21273     dw     old0D
21274 intret_0D:
21275     iret
21276
21277 ; -----
21278
21279 int0E:
21280     jmp     short entry_int0E_stk
21281 old0E:     dd     0
21282     dw     424Bh
21283 firstflag0E:
21284     db     0
21285     jmp     short intret_0E
21286     times 7 db 0
21287
21288 entry_int0E_stk:
21289     call    do_int_stacks
21290     dw     old0E
21291 intret_0E:
21292     iret
21293
21294 ; -----
21295
21296 int72:
21297     jmp     short entry_int72_stk
21298 old72:     dd     0
21299     dw     424Bh
21300 firstflag72:
21301     db     0
21302     jmp     short intret_72
21303     times 7 db 0
21304
21305 entry_int72_stk:
21306     call    do_int_stacks
21307     dw     old72
21308 intret_72:
21309     iret
21310
21311 ; -----
21312
21313 int73:
21314     jmp     short entry_int73_stk
21315 old73:     dd     0
21316     dw     424Bh
21317 firstflag73:
21318     db     0
21319     jmp     short intret_73
21320     times 7 db 0
21321
21322 entry_int73_stk:
21323     call    do_int_stacks
21324     dw     old73
21325 intret_73:
21326     iret
21327

```



```

21328 ; -----
21329
21330 int74:
21331 jmp short entry_int74_stk
21332 old74: dd 0
21333 dw 424Bh
21334 firstflag74:
21335 db 0
21336 jmp short intret_74
21337 times 7 db 0
21338
21339 entry_int74_stk:
21340 call do_int_stacks
21341 dw old74
21342 intret_74:
21343 iret
21344 ; -----
21345
21346 int76:
21347 jmp short entry_int76_stk
21348 old76: dd 0
21349 dw 424Bh
21350 firstflag76:
21351 db 0
21352 jmp short intret_76
21353 times 7 db 0
21354
21355 entry_int76_stk:
21356 call do_int_stacks
21357 dw old76
21358 intret_76:
21359 iret
21360 ; -----
21361
21362 int77:
21363 jmp short entry_int77_stk
21364 old77: dd 0
21365 dw 424Bh
21366 firstflag77:
21367 db 0
21368 jmp short intret_77
21369 times 7 db 0
21370
21371 entry_int77_stk:
21372 call do_int_stacks
21373 dw old77
21374 intret_77:
21375 iret
21376 ; -----
21377
21378 ;*****
21379 ;common routines
21380 ;*****
21381
21382 ; do interrupt stack switching. the fake return address holds
21383 ; a pointer to the far-pointer of the actual interrupt
21384 ; service routine
21385
21386 ; 21/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 SYSINIT)
21387 ; 21/03/2019 - Retro DOS v4.0
21388
21389 ;allocbyte equ 0
21390 ;intlevel equ 1
21391 ;savedsp equ 2
21392 ;savedss equ 4
21393 ;newsp equ 6
21394
21395 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 SYSINIT)
21396 ; (MSDOS 6.21 IO.SYS, SYSINIT:0147h)
21397
21398 do_int_stacks:
21399 push ax
21400 push bp
21401 push es
21402 mov es,[cs:stacks+2] ; Get segment of stacks
21403 mov bp,[cs:nextentry] ; get most likely candidate
21404 mov al,allocated ; 1
21405 ; 21/10/2022
21406 ;xchg [es:bp+allocbyte],al
21407 ; 11/12/2022
21408 xchg [es:bp],al ; grab the entry
21409 cmp al,free ; 0 ; still avail?
21410 jne short notfree02
21411 sub word [cs:nextentry],entrysize ; set for next interrupt
21412
21413 found02:
21414 mov [es:bp+savedsp],sp ; save sp value
21415 mov [es:bp+savedss],ss ; save ss also
21416
21417 mov ax,bp ; temp save of table offset
21418
21419 mov bp,[es:bp+newsp] ; get new SP value
21420 ; 21/10/2022
21421 ;mov bp,[es:bp+6]
21422 ; 11/12/2022
21423 ;cmp [es:bp+0],ax
21424 cmp [es:bp],ax ; check for offset into table
21425 jne short foundbad02
21426
21427 ; 02/07/2023 (MSDOS 6.21 SYSINIT code)
21428 mov ax,es ; point ss,sp to the new stack
21429 mov es,bp
21430 mov bp,sp
21431 mov bp,[bp+6]
21432 mov ss,ax
21433 mov sp,es
21434 mov es,ax
21435 mov bp,[cs:bp]
21436
21437 ; 21/10/2022 (MSDOS 5.0 SYSINIT code)
21438 ;push bp
21439 ;mov bp,sp
21440 ;mov ax,[bp+8]
21441 ;pop bp
21442 ;push es
21443 ;pop ss
21444 ;mov sp,bp
21445 ;mov bp,ax
21446 ; 11/12/2022
21447 ;mov bp,[cs:bp+0]
21448
21449
21450
21451

```

```

21452             ;mov     bp,[cs:bp]
21453
21454 0000018B 9C      pushf             ; go execute the real interrupt handler
21455             ; 11/12/2022
21456 0000018C 2EFF5E00 call     far [cs:bp]             ; which will iret back to here
21457             ; 21/10/2022
21458             ;call    far [cs:bp+0]
21459
21460 00000190 89E5      mov     bp,sp             ; retrieve the table offset for us
21461             ; 11/12/2022
21462 00000192 268B6E00 mov     bp,[es:bp]             ; but leave it on the stack
21463             ; 21/10/2022
21464             ;mov     bp,[es:bp+0]
21465 00000196 268E5604 mov     ss,[es:bp+savedss]     ; get old stack back
21466 0000019A 268B6602 mov     sp,[es:bp+savedsp]
21467
21468             ; 11/12/2022
21469             ;mov     byte [es:bp+allocbyte],free ; free the entry
21470             ; 21/10/2022
21471 0000019E 26C6460000 mov     byte [es:bp],free ; 0
21472 000001A3 2E892E[1000] mov     [cs:nextentry],bp     ; setup to use next time
21473
21474 000001A8 07        pop     es
21475 000001A9 5D        pop     bp             ; saved on entry
21476 000001AA 58        pop     ax             ; saved on entry
21477 000001AB 83C402      add     sp,2
21478 000001AE CF        iret             ; done with this interrupt
21479
21480 notfree02:
21481 000001AF 3C01      cmp     al,allocated         ; error flag
21482 000001B1 7404      je      short findnext02     ; no, continue
21483             ; 11/12/2022
21484             ;xchg     [es:bp+allocbyte],al ; yes, restore error value
21485             ; 21/10/2022
21486 000001B3 26864600 xchg     [es:bp],al
21487
21488 findnext02:
21489 000001B7 E81200      call    longpath
21490 000001BA EBA8      jmp     short found02
21491
21492 foundbad02:
21493 000001BC 2E3B2E[0C00] cmp     bp,[cs:firstentry]
21494 000001C1 72F4      jc      short findnext02
21495 000001C3 89C5      mov     bp,ax             ; flag this entry
21496             ; 11/12/2022
21497             ;mov     byte [es:bp+allocbyte],clobbered
21498             ; 21/10/2022
21499 000001C5 26C6460003 mov     byte [es:bp],clobbered ; 3
21500 000001CA EBEB      jmp     short findnext02     ; keep looking
21501
21502 ; -----
21503
21504 ; Common routines
21505
21506 longpath:
21507             ; 21/03/2019
21508 000001CC 2E8B2E[0E00] mov     bp,[cs:lastentry]     ; start with last entry in table
21509
21510 lploopp:
21511             ; 11/12/2022
21512             ;cmp     byte [es:bp+allocbyte],free ; is entry free?
21513             ; 21/10/2022
21514 000001D1 26807E0000 cmp     byte [es:bp],free
21515 000001D6 7512      jne     short inuse         ; no, try next one
21516
21517 000001D8 B001      mov     al,allocated
21518             ; 11/12/2022
21519             ;xchg     [es:bp+allocbyte],al ; allocate entry
21520             ; 21/10/2022
21521 000001DA 26864600 xchg     [es:bp],al
21522 000001DE 3C00      cmp     al,free           ; is it still free?
21523 000001E0 7414      je      short found         ; yes, go use it
21524
21525 000001E2 3C01      cmp     al,allocated       ; is it other than Allocated or Free?
21526 000001E4 7404      je      short inuse         ; no, check the next one
21527
21528             ; 11/12/2022
21529             ;mov     [es:bp+allocbyte],al ; yes, put back the error state
21530             ; 21/10/2022
21531 000001E6 26884600 mov     [es:bp],al
21532
21533 inuse:
21534 000001EA 2E3B2E[0C00] cmp     bp,[cs:firstentry]
21535 000001EF 7406      je      short fatal
21536 000001F1 83ED08      sub     bp,entrysize
21537 000001F4 EBD8      jmp     short lploopp
21538
21539 found:
21540         retn
21541
21542 fatal:
21543 000001F7 1E        push    ds
21544 000001F8 B800F0      mov     ax,0F000h         ;look at the model byte
21545 000001FB 8ED8      mov     ds,ax
21546 000001FD 803EFEFF9 cmp     byte [0FFFEh],0F9h ; mdl_convert ; convertible?
21547 00000202 1F        pop     ds
21548 00000203 7504      jne     short skip_nmis
21549
21550 00000205 B007      mov     al,07h            ; disable pc convertible nmis
21551 00000207 E672      out     72h,al
21552
21553 skip_nmis:
21554 00000209 FA        cli             ; disable and mask
21555 0000020A B0FF      mov     al,0FFh           ; all other ints
21556 0000020C E621      out     021h,al
21557 0000020E E6A1      out     0A1h,al
21558
21559 00000210 8CCE      mov     si,cs
21560 00000212 8EDE      mov     ds,si
21561 00000214 BE[3B02] mov     si,fatal_msg
21562
21563 ;SR;
21564 ; we set all foci to this VM to issue the stack failure message
21565 ;
21566 00000217 50        push    ax
21567 00000218 1E        push    ds
21568             ;mov     ax,Bios_Data ; 0070h
21569             ;mov     ax,KERNEL_SEGMENT ; 0070h
21570             ; 21/10/2022
21571 00000219 B87000      mov     ax,DOSBIODATASEG
21572 0000021C 8ED8      mov     ds,ax
21573
21574             ;test     byte [08D0h],1 ; (MSDOS 6.21, IO.SYS - SYSINIT:021Eh)
21575 0000021E F606[1208]01 test     byte [IsWin386],1 ; (retrodos4.sys, offset: ****h)
21576 00000223 1F        pop     ds
21577 00000224 58        pop     ax
21578 00000225 7405      jz      short fatal_loop     ; win386 not present, continue
21579
21580 ;;call far ptr 0070h:08D1h ; (MSDOS 621, IO.SYS - SYSINIT:0227h)

```



```

21576             ;call  KERNEL_SEGMENT:V86_Crit_SetFocus ; set focus to this VM
21577             ; 21/10/2022
21578 00000227 9A[1308]7000      call  DOSBIODATASEG:V86_Crit_SetFocus ; 0070h:08D1h
21579             ;
21580             ;SR; we do not bother about the returned status of this call.
21581             ;
21582 fatal_loop:
21583 0000022C AC          lodsb
21584 0000022D 3C24        cmp     al,'$'
21585 0000022F 7408        je      short fatal_done
21586
21587             mov     bl,7
21588 00000233 B40E        mov     ah,14
21589 00000235 CD10        int     10h          ; whoops, this enables ints
21590 00000237 EBF3        jmp     short fatal_loop
21591
21592 fatal_done:
21593 00000239 EBFE        jmp     short fatal_done
21594
21595             ; 21/03/2019 - Retro DOS v4.0 (MSDOS 6.0, SYSINIT1.ASM, 1991)
21596             ; -----
21597             ; include msbio.c15          ; fatal stack error message
21598
21599             ; MSDOS 6.21, IO.SYS, SYSINIT:023Bh
21600
21601             ; STKMES.INC - MSDOS 3.3 (24/07/1987)
21602             ; -----
21603             ; 04/06/2018 - Retro DOS v3.0
21604
21605 fatal_msg:
21606             db      0Dh,0Ah
21607 0000023B 0D0A        db      7,0Dh,0Ah
21608 0000023D 070D0A      db      "Internal stack overflow",0Dh,0Ah
21609 00000240 496E7465726E616C20-
21609 00000249 7374616368206F7665-
21609 00000252 72666C6F770D0A
21610 00000259 53797374656D206861-
21610 00000262 6C7465640D0A24      db      "System halted",0Dh,0Ah,"$"
21611
21612 endstackcode:
21613             ; -----
21614             ; SYINIT1.ASM (MSDOS 6.0, 1991) 'SYSINIT' jump addr from 'MSINIT.ASM'
21615             ; -----
21616             ; 04/06/2018 - Retro DOS v3.0 (MSDOS 3.3, SYSINIT1.ASM, 24/07/1987)
21617
21618             ; 22/03/2019 - Retro DOS v4.0
21619
21620             ; SYSINIT:0269h (MSDOS 6.21 IO.SYS, SYSINIT segment, offset: 0269h)
21621
21622             ; ('SYSINIT:' location/address is used in 'retrodos4.s'. If following
21623             ; address will be changed, it must also be changed in 'retrodos4.s'.)
21624
21625             ; 21/10/2022- Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
21626             ; -----
21627             ; SYSINITSEG:0267h (MSDOS 5.0 IO.SYS, SYSINIT segment, offset: 0267h)
21628
21629             ; SYSINIT:0269h (MSDOS 6.22 IO.SYS, SYSINIT segment, offset: 0269h)
21630
21631             ; 29/12/2023- Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
21632             ; -----
21633             ; SYSINITSEG:0269h (PCDOS 7.1 IBMBIO.COM, SYSINIT segment, offset: 0269h)
21634
21635 SYSINIT:
21636             JMP GOINIT
21637 00000269 E9AD01      ;JMP     SYSIN ; 25/02/2018 - Retro DOS 2.0 modification
21638
21639             ; -----
21640
21641 struct DDHighInfo
21642             .ddhigh_CSegPtr resd 1 ; pointer to code segment to be relocated
21643 00000000 ?????????? .ddhigh_CSegLen resw 1 ; length of code segment to be relocated
21644 00000004 ?????      .ddhigh_CallBak resd 1 ; pointer to the call back routine
21645 00000006 ??????????
21646             endstruct
21647
21648             ; 22/03/2019 - Retro DOS v4.0
21649
21650 runhigh: db 0
21651
21652             ; 02/11/2022
21653             ;align 4
21654
21655 DOSINFO:
21656 0000026D 00000000      dd      0 ; address of the DOS Sysini Variables
21657
21658             ;MSDOS:
21659 dos_temp_location: ; dword ; MSDOS 6.0
21660 00000271 0000        dosinit: ; MSDOS 6.0
21661             dw      0
21662
21663             ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
21664             ;FINAL_DOS_LOCATION: ; 20/04/2019 - Retro DOS v4.0
21665             ; dw      0
21666             ;MSDOS 5.0 IO.SYS - SYSINIT:0271h
21667
21668 CURRENT_DOS_LOCATION:
21669             dw      0
21670
21671             ;DOSSIZE: ; Retro DOS 2.0 feature - 25/02/2018
21672             ; dw      0 ; 'MSDOS.BIN' kernel size in words
21673
21674             ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
21675             ; (MSDOS 5.0 MSDOS.SYS size is 37394 bytes)
21676             ;DOSSIZE equ 0A000h ; (MSDOS 6.0 - SYSINIT1.ASM - 1991)
21677             ; 30/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
21678             ; 03/09/2023 (PCDOS 7.1 IBMDOS.COM size is 42566 bytes, 04/12/2003)
21679             DOSSIZE equ 0B000h ; (PCDOS 7.1 - SYSINIT)
21680
21681 DEVICE_LIST:
21682             dd      0
21683
21684             ; 04/06/2018 - Retro DOS v3.0
21685             ; 28/03/2018
21686             ; ; MSDOS 3.3 - SYSINIT1.ASM - 24/07/1987
21687             ;
21688             sysi_country:
21689             dd      0 ; 5/29/86 Pointer to country table in DOS
21690
21691             ; MSDOS 6.0
21692 dos_segrenit: dw 0,0 ; room for dword
21693
21694             lo_doscod_size: dw 0 ; dos code size when in low mem
21695             hi_doscod_size: dw 0 ; dos code size when in HMA
21696
21697             def_php: dw 0

```

```

21697 ; M022--
21698 ; pointer for calling into Bios_Code for re-initializing segment values.
21699 ; call with ax = new segment for Bios_Code. Notice that we'll
21700 ; call it in its temporary home, cuz seg_reinit won't get moved to
21701 ; the new home.
21702
21703 ;Bios_Code equ    KERNEL_SEGMENT    ; 0070h
21704 ; 21/10/2022
21705 ;DOSBIOCODESEG    equ    02C7h ; (MSDOS 5.0 IO.SYS)
21706
21707 ; 22/10/2022
21708 seg_reinit_ptr:    ; label dword
21709 dw seg_reinit ; Bios_Code:0032h for MSDOS 6.21 IO.SYS
21710 00000287 [3200]
21711 temp_bcode_seg:
21712 dw Bios_Code ; 02CCh for MSDOS 6.21 IO.SYS
21713 ; 22/10/2022
21714 00000289 0203 dw DOSBIOCODESEG ; 02C7h for MSDOS 5.0 IO.SYS
21715 ; 364h for PCDOS 7.1 IBMBIO.COM - 29/12/2023
21716 fake_floppy_drv:
21717 0000028B 00 db 0 ; set to 1 if this machine
21718 ; does not have any floppies!!!
21719
21720 ; Internal Stack Parameters
21721
21722 0000028C 0900 stack_count: dw defaultcount ; 9
21723 0000028E 8000 stack_size: dw defaultsize ; 128
21724 00000290 00000000 stack_addr: dd 0
21725
21726 ; 05/06/2018 - Retro DOS v3.0
21727
21728 ; various default values
21729
21730 00000294 0100 MEMORY_SIZE: dw 1
21731
21732 ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0 source, MSDOS 6.21 disassembled src.)
21733
21734 00000296 0000 RPLMemTop: dw 0 ; 22/10/2022 (MSDOS 5.0 IO.SYS SYSINIT:0294h)
21735 00000298 00 DEFAULT_DRIVE: db 0 ; initialized by ibminit.
21736 00000299 FFFF buffers: dw 0FFFFh ; initialized during buffer allocation
21737 0000029B 0000 h_buffers: dw 0 ; # of the heuristic buffers. initially 0.
21738 0000029D 0000 singlebuffersize: dw 0 ; maximum sector size + buffer head
21739
21740 0000029F 08 FILES: db 8 ; enough files for pipe
21741 000002A0 04 FCBS: db 4 ; performance for recycling
21742 000002A1 00 KEEP: db 0 ; keep original set
21743 000002A2 05 NUM_CDS: db 5 ; 5 net drives
21744
21745 ; 22/10/2022 (MSDOS 5.0 SYSINIT)
21746 ;CONFBOT: dw 0
21747 ;ALLOCLIM: dw 0
21748 ;CONFBOT: ; 02/11/2022
21749 ;top_of_cdss: dw 0
21750
21751 ; 30/12/2022 - Retrodos v4.2 (MSDOS 6.21 SYSINIT)
21752 ; (SYSINIT:02A3h)
21753 000002A3 0000 CONFBOT: dw 0
21754 000002A5 0000 ALLOCLIM: dw 0
21755 000002A7 0000 top_of_cdss: dw 0
21756
21757 ; 02/11/2022 (MSDOS 5.0 SYSINIT)
21758 ; 30/12/2022 (MSDOS 6.21 SYSINIT)
21759 ;ALLOCLIM: dw 0 ; (SYSINIT:02A3h)
21760
21761 000002A9 413A5C00 DirStrng: db "A:\",0 ; string for the root directory of a drive
21762
21763 ; 30/12/2022 - Retro DOS v4.2 (MSDOS 6.21 SYSINIT)
21764 %if 0
21765 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
21766 ; (SYSINIT:02A9h)
21767
21768 command_line:
21769 db 2,0
21770 db 'P'
21771 db 0
21772 times 124 db 0 ; db 124 dup(0)
21773
21774 %endif
21775
21776 ; (SYSINIT:0329h)
21777 000002AD 00 ZERO: db 0
21778 000002AE 00 sepchr: db 0
21779 000002AF 0000 linecount: dw 0 ; line count in config.sys
21780 000002B1 20202020D0A24 showcount: db ' ',cr,lf,'$' ; used to convert linecount to ascii.
21781 000002B9 0000 buffer_linenum: dw 0 ; line count for "buffers=" command if entered.
21782
21783 000002BB FF sys_model_byte: db 0FFh ; model byte used in sysinit
21784 000002BC 00 sys_scnd_model_byte: db 0 ; secondary model byte used in sysinit
21785
21786 000002BD 0000 buf_prev_off: dw 0
21787
21788 ;IF NOT NOEXEC
21789 ;COMEXE EXEC0 <0,COMMAND_LINE,DEFAULT_DRIVE,ZERO>
21790 ;ENDIF
21791
21792 ; 29/12/2023
21793 ; 01/05/2018
21794 COMEXE:
21795 000002BF 0000 EXEC0.ENVIRON: dw 0 ; seg addr of environment
21796 000002C1 [B94B] EXEC0.COM_LINE: dw command_line ; pointer to asciz command line
21797 000002C3 0000 dw 0 ; SYSINIT segment (0473h for MSDOS 6.21 IO.SYS)
21798 ; SYSINIT segment (0544h for PCDOS 7.1 IBMBIO.COM)
21799 000002C5 [9802] EXEC0.5C_FCB: dw DEFAULT_DRIVE ; default fcb at 5C
21800 000002C7 0000 dw 0 ; SYSINIT segment (0473h for MSDOS 6.21 IO.SYS)
21801 000002C9 [AD02] EXEC0.6C_FCB: dw ZERO ; default fcb at 6C
21802 000002CB 0000 dw 0
21803
21804 ; variables for install= command.
21805
21806 000002CD 00 multi_pass_id: db 0 ; parameter passed to multi_pass
21807 ; indicating the pass number
21808 ; 0 - do scan for DOS=HIGH/LOW
21809 ; 1 - load device drivers
21810 ; 2 - was to load IFS
21811 ; now it is unused
21812 ; 3 - do install=
21813 ; >3 - nop
21814 000002CE 0000 install_flag: dw 0
21815
21816 00000001b have_install_cmd equ ; config.sys has install= commands
21817 00000010b has_installed equ ; sysinit_base installed.
21818
21819 000002D0 0000 config_size: dw 0 ; size of config.sys file. set by sysconf.asm
21820 000002D2 00000000 sysinit_base_ptr: dd 0 ; pointer to sysinit_base

```

```

21821 000002D6 00000000 sysinit_ptr: dd 0 ; returning addr. from sysinit_base
21822 000002DA 0000 checksum: dw 0 ; used by sum_up
21823
21824 000002DC 20<rep 14h> ldxec_fcb: times 20 db 20h ; db 20 dup (' ') ; big enough
21825 000002F0 00 ldxec_line: db 0 ; # of parm characters
21826 000002F1 20 ldxec_start: db 0
21827 000002F2 00<rep 50h> ldxec_parm: times 80 db 0 ; db 80 dup (0)
21828
21829 ; instexe exec0 <0,ldxec_line,ldxec_fcb,ldxec_fcb>
21830
21831 instexe:
21832 00000342 0000 iexec.enviro: dw 0 ; seg addr of environment
21833 00000344 [F002] iexec.ldxec_line: dw ldxec_line ; pointer to asciz command line
21834 00000346 0000 dw 0 ; SYSINIT segment (0473h for MSDOS 6.21 IO.SYS)
21835 ; SYSINIT segment (0544h for PCDOS 7.1 IBMBIO.COM)
21836 00000348 [DC02] iexec.ldxec_5c_fcb: dw ldxec_fcb ; default fcb at 5C
21837 0000034A 0000 dw 0 ; SYSINIT segment (0473h for MSDOS 6.22 IO.SYS)
21838 0000034C [DC02] iexec.ldxec_6c_fcb: dw ldxec_fcb ; default fcb at 6C
21839 0000034E 0000 dw 0
21840
21841 ; variables for comment=
21842
21843 00000350 00 com_level: db 0 ; level of " " in command line
21844 00000351 00 cmm1: db 0 ; length of comment string token
21845 00000352 00 cmm11: db 0 ; token
21846 00000353 00 cmm2: db 0 ; token
21847 00000354 00 cmd_indicator: db 0
21848 00000355 00 donotshownum: db 0
21849
21850 00000356 0000 count: dw 0
21851 00000358 0000 org_count: dw 0
21852 0000035A 0000 chrptr: dw 0
21853 0000035C 0000 cntryfilehandle: dw 0
21854 0000035E 0000 old_area: dw 0
21855 00000360 0000 impossible_owner_size: dw 0 ; paragraph
21856
21857 bucketptr: ; label dword
21858 bufptr: ; label dword ; leave this stuff in order!
21859 00000362 0000 memlo: dw 0
21860 prmb1k: ; label word
21861 00000364 0000 memhi: dw 0
21862 00000366 0000 ldoft: dw 0
21863 00000368 0000 area: dw 0
21864
21865 ; 29/12/2023 - PCDOS 7.1 IBMBIO.COM - SYSINIT:036Ah
21866 0000036A 0000 prev_memhi: dw 0
21867 0000036C 0000 prev_allocim: dw 0
21868 0000036E 00 dosdata_umb: db 0
21869
21870 ; Following is the request packet used to call INIT routines for
21871 ; all device drivers. Some fields may be accessed individually in
21872 ; the code, and hence have individual labels, but they should not
21873 ; be separated.
21874
21875 0000036F 19 packet: db 25 ; PCDOS 7.1 IBMBIO.COM
21876 ;db 24 ; was 22
21877 00000370 00 db 0
21878 00000371 00 db 0 ; initialize code
21879 00000372 0000 dw 0
21880 00000374 00<rep 8h> times 8 db 0 ; db 8 dup (?)
21881
21882 0000037C 00 unitcount: db 0
21883 0000037D 00000000 break_addr: dd 0
21884 00000381 00000000 bpb_addr: dd 0
21885 ; 22/10/2022
21886 00000385 00 devdrivenum: db 0
21887 00000386 0000 configmsgflag: dw 0 ; used to control "error in config.sys line #" message
21888
21889 ; end of request packet
21890
21891 ;drivenumber: db 0 ; 22/03/2019
21892
21893 toomanydrivesflag:
21894 00000388 00 db 0 ; >24 fixed disk partitions flag ; M029
21895 00000389 90 align 2
21896
21897 BCodeSeg: ; 21/10/2022
21898 0000038A 0203 dw DOSBIOCODESEG ; (02C7h for MSDOS 5.0 IO.SYS)
21899 ; 0364h for PCDOS 7.1 IBMBIO.COM - 29/12/2023
21900 ;dw Bios_Code ; = KERNEL_SEGMENT = 0070h (for Retro DOS v4.0)
21901 ; BCodeSeg = 2CCh (for MSDOS 6.21 IO.SYS)
21902
21903 ; 30/12/2022
21904 ; MSDOS 6.21 IO.SYS, SYSINIT:0387h
21905 ;
21906 ; Magicbackdoor: dd 0
21907 ; NullBackdoor:
21908 ; retf
21909
21910 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
21911 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
21912 ; 19/04/2019
21913 _timer_lw_:
21914 0000038C 0000 dw 0 ; MSDOS 6.21 IO.SYS - SYSINIT:038Ch
21915
21916 ; 29/12/2023 - Retro DOS v5.0
21917 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:038Eh
21918
21919 0000038E 00 F5_key: db 0
21920 0000038F 00 F8_key: db 0
21921 00000390 00000000 MagicBackdoor: dd 0
21922 NullBackdoor:
21923 00000394 CB retf
21924
21925 ;SR;
21926 ; This is the communication block between the DOS and the BIOS. It starts at
21927 ; the SysinitPresent flag. Any other data that needs to be communicated
21928 ; to the DOS should be added after SysinitPresent. The pointer to this block
21929 ; is passed to DOS as part of the DOSINIT call.
21930 ;
21931
21932 BiosComBlock:
21933 ;dd Bios_Data:SysinitPresent
21934 ; 0070h:08FDh for MSDOS 6.21 IO.SYS
21935 00000395 [DD07] dw SysinitPresent ; (retrodos4.sys, offset: ****h)
21936 ;dw KERNEL_SEGMENT ; 0070h
21937 ; 21/10/2022
21938 00000397 7000 dw DOSBIODATASEG ; 0070h
21939
21940 ;align 2
21941
21942 ; 22/10/2022 - (MSDOS 5.0 IO.SYS, SYSINIT:0406h)
21943 ; 30/12/2022 - (MSDOS 6.21 IO.SYS, SYSINIT:0392h)
21944 tempstack:

```

```

21945 00000399 00<rep 80h>          times 128 db 0 ; db 80h dup (?)
21946
21947 ; -----
21948 ; 29/12/2023 - Retro DOS v5.0
21949 ; 22/10/2022 - Retro DOS v4.0
21950 ; ; (MSDOS 5.0 IO.SYS, SYSINIT:0486h)
21951 GOINIT: ; ; (MSDOS 6.22 IO.SYS, SYSINIT:0412h)
21952 ; ; (PCDOS 7.1 IBMBIO.COM, SYSINIT:0419h)
21953 ; 12/12/2023
21954 push cs
21955 00000419 0E pop ds
21956 0000041A 1F
21957
21958 ; 12/12/2022
21959 ; 22/03/2019 - Retro DOS v4.0
21960 ; 06/07/2018
21961 ; 04/06/2018 - Retro DOS v3.0
21962 ; before doing anything else, let's set the model byte
21963 0000041B B4C0 mov ah,0C0h ; get system configuration
21964 0000041D CD15 int 15h ;
21965 0000041F 7214 jc short no_rom_config
21966
21967 ;cmp ah,0 ; double check
21968 ;jne short no_rom_config
21969 ; 03/09/2023
21970 00000421 08E4 or ah,ah
21971 00000423 7510 jnz short no_rom_config
21972
21973 ; 12/12/2023 ; *
21974 ; ds = cs
21975
21976 00000425 268A702 mov al,[es:bx+ROMBIOS_DESC.bios_sd_modelbyte]
21977 ;mov [cs:sys_model_byte],al
21978 00000429 A2[B802] mov [sys_model_byte],al ; *
21979 0000042C 268A703 mov al,[es:bx+ROMBIOS_DESC.bios_sd_scnd_modelbyte]
21980 ;mov [cs:sys_scnd_model_byte],al
21981 00000430 A2[BC02] mov [sys_scnd_model_byte],al ; *
21982 ;jmp short SYSIN
21983 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
21984 00000433 EB29 jmp short move_myself
21985
21986 no_rom_config: ; Old ROM
21987 ; 12/12/2023
21988 ;mov ax,0F000h
21989 ;mov ds,ax
21990 ;mov al,[0FFFEh]
21991 ;mov [cs:sys_model_byte],al; set the model byte.
21992 ; 12/12/2023
21993 ; ds = cs
21994 00000435 B800F0 mov ax,0F000h
21995 00000438 8EC0 mov es,ax
21996 0000043A 26A0FEFF mov al,[es:0FFFEh]
21997 0000043E A2[B802] mov [sys_model_byte],al ; set the model byte.
21998
21999 ; set fake_floppy_drv if there is no diskette drives in this machine.
22000 ; execute the equipment determination interrupt and then
22001 ; check the returned value to see if we have any floppy drives
22002 ; if we have no floppy drive we set cs:fake_floppy_drv to 1
22003 ; see the at tech ref bios listings for help on the equipment
22004 ; flag interrupt (11h)
22005
22006 ; 22/10/2022
22007 ;check_for_fake_floppy: ; entry point for rom_config above
22008 00000441 CD11 int 11h ; check equipment flag
22009
22010 ; 29/12/2023 - Retro DOS v5.0
22011 ;jmp short check_for_fake_floppy
22012 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0446h
22013 ;db 52h ; 'RPS' sign
22014 ;db 50h
22015 ;db 53h
22016
22017 check_for_fake_floppy:
22018 ; 29/12/2023
22019 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0449h
22020 ;or ax,1 ; (nonsense! this may be overwritten/disabled
22021 ; ; by using 'RPS' sign position)
22022 ; ; 03/07/2023 - Erdogan Tan
22023 ;test ax,1 ; have any floppies?
22024
22025 ; 12/12/2022
22026 00000443 A801 test al,1
22027 ;test ax,1 ; have any floppies?
22028 00000445 7517 jnz short move_myself ; yes,normal system
22029
22030 ; Some ROM BIOSs lie that there are no floppy drives. Lets find out
22031 ; whether it is an old ROM BIOS or a new one
22032 ;
22033 ; WARNING !!!
22034 ;
22035 ; This sequence of code is present in MSINIT.ASM also. Any modification
22036 ; here will require an equivalent modification in MSINIT.ASM also
22037
22038 ; 12/12/2023
22039 ;push es ; not necessary
22040
22041 00000447 30C9 xor cl,cl
22042 00000449 B408 mov ah,8 ; get disk parameters
22043 0000044B B200 mov dl,0 ; of drive 0
22044 0000044D CD13 int 13h
22045
22046 ;pop es ; 12/12/2023
22047
22048 0000044F 720D jc short move_myself ; if error lets assume that the
22049 ; ; ROM BIOS lied
22050 ;cmp cl,0 ; double check (max sec no cannot be 0)
22051 ;je short move_myself
22052 ; 03/09/2023
22053 00000451 08C9 or cl,cl
22054 00000453 7409 jz short move_myself
22055
22056 00000455 08D2 or dl,dl ; number of flp drvs == 0?
22057 00000457 7505 jnz short move_myself ; no
22058
22059 ;mov byte [cs:fake_floppy_drv],1 ; set fake flag.
22060 ; 12/12/2023
22061 ; ds = cs
22062 00000459 C606[8B02]01 mov byte [fake_floppy_drv],1 ; set fake flag.
22063
22064 move_myself:
22065 ; 12/12/2023
22066 ;cld ; not necessary ; set up move
22067 ;xor si,si
22068 ;mov di,si

```

```

22069
22070 ; 12/12/2023
22071 ; ds = cs
22072 ; 12/12/2022
22073 ;push cs
22074 ;pop ds
22075
22076 ;mov cx,[cs:MEMORY_SIZE]
22077 0000045E 8B0E[9402] mov cx,[MEMORY_SIZE] ; 12/12/2022
22078
22079 ; (MSDOS 6.0 - SYSINIT1.ASM - 1991)
22080 ;;; if msver
22081 ; cmp cx,1 ; 1 means do scan
22082 ; jnz short noscan
22083 ; mov cx,2048 ; start scanning at 32k boundary
22084 ; xor bx,bx
22085
22086 ;memscan:inc cx
22087 ; jz short setend
22088 ; mov ds,cx
22089 ; mov al,[bx]
22090 ; not al
22091 ; mov [bx],al
22092 ; cmp al,[bx]
22093 ; not al
22094 ; mov [bx],al
22095 ; jz short memscan
22096 ;setend:
22097 ; mov cs:[memory_size],cx
22098 ;;; endif
22099
22100 ;noscan: ; cx is mem size in para
22101 ;;;
22102 ; cas -- a) if we got our memory size from the ROM, we should test it
22103 ; before we try to run.
22104 ; b) in any case, we should check for sufficient memory and give
22105 ; an appropriate error diagnostic if there isn't enough
22106 ;
22107 ; push cs
22108 ; pop ds
22109 ;
22110 ; cas note: It would be better to put dos + bios_code BELOW sysinit
22111 ; that way it would be easier to slide them down home in a minimal
22112 ; memory system after sysinit. As it is, you need room to keep
22113 ; two full non-overlapping copies, since sysinit sits between the
22114 ; temporary home and the final one. the problem with doing that
22115 ; is that sys*.asm are filled with "mov ax,cs, sub ax,11h" type stuff.
22116 ;
22117 ; dec cx ; one para for an arena at end of mem
22118 ; ; in case of UMBS
22119
22120 ; 22/10/2022
22121 ; (MSDOS 5.0 IO.SYS SYSINIT:04DBh)
22122
22123 ; 12/12/2022
22124 ;push cs
22125 ;pop ds
22126
22127 00000462 49 dec cx
22128
22129 ;----- Check if an RPL program is present at TOM and do not tromp over it
22130
22131 00000463 31DB xor bx,bx
22132 00000465 8EC3 mov es,bx
22133 ;mov bx,[es:(2Fh*4)] ; INT 2Fh address (0:0BCh)
22134 ;mov es,[es:((2Fh*4)+2)] ; INT 2Fh segment (0:0BEh)
22135 ; 29/09/2023
22136 00000467 26C41EBC00 les bx,[es:(2Fh*4)]
22137 0000046C 26817F035250 cmp word [es:bx+3], 'RP'
22138 00000472 751B jne short NORPL
22139 00000474 26807F054C cmp byte [es:bx+5], 'L'
22140 00000479 7514 jne short NORPL
22141
22142 0000047B 89CA mov dx,cx ; get TOM into DX
22143 0000047D 52 push dx
22144 0000047E B8064A mov ax,4A06h
22145 ;mov ax,(multMULT<<8)+multMULTRPLTOM
22146 00000481 CD2F int 2Fh ; Get new TOM from any RPL
22147 00000483 58 pop ax
22148 00000484 89D1 mov cx,dx
22149 00000486 39C2 cmp dx,ax
22150 00000488 7405 je short NORPL
22151
22152 ; 11/12/2022
22153 ; ds = cs
22154 0000048A 8916[9602] mov [RPLMemTop],dx
22155 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
22156 ;mov [cs:RPLMemTop],dx
22157
22158 0000048E 49 dec cx
22159 NORPL:
22160 0000048F B8[1054] mov ax,SI_end ; need this much room for sysinit
22161 ; (SI_end == sysinit code size)
22162 ; 03/09/2023
22163 ; (58A0h for MSDOS 6.21 IO.SYS)
22164 ; (5B40h for PCDOS 7.1 IBMBIO.COM)
22165 00000492 E80509 call off_to_para
22166 00000495 29C1 sub cx,ax
22167
22168 ; we need to leave room for the DOS and (if not ROMDOS) for the BIOS
22169 ; code above sysinit in memory
22170 ;
22171 00000497 81E9000B sub cx,DOSSIZE/16 ; (0A00h) ; leave this much room for DOS
22172 ; (0B00h) ; (PCDOS 7.1 IBMBIO.COM) -03/09/2023-
22173
22174 0000049B B8701D mov ax,BCODE_END ; (1A60h for MSDOS 5.0 IO.SYS)
22175 ; (1A70h for MSDOS 6.21 IO.SYS)
22176 ; 03/09/2023
22177 ; (1E00h for PCDOS 7.1 IBMBIO.COM)
22178 0000049E E8F908 call off_to_para ; leave this much room for BIOS code
22179 000004A1 29C1 sub cx,ax
22180 000004A3 8EC1 mov es,cx ; segment where sysinit will be located
22181
22182 ; 12/12/2023
22183 000004A5 FC cld ; not necessary ; set up move
22184 000004A6 31F6 xor si,si
22185 000004A8 89F7 mov di,si
22186
22187 000004AA B9[1054] mov cx,SI_end ; (sysinit code size)
22188 000004AD D1E9 shr cx,1 ; divide by 2 to get words
22189 000004AF F3A5 rep movsw ; relocate sysinit
22190
22191 000004B1 06 push es ; push relocated segment
22192 000004B2 B8[B704] mov ax,SYSDIN

```

```

22193 000004B5 50          push    ax                ; push relocated entry point
22194
22195 000004B6 CB          retf                ; far jump to relocated sysinit
22196
22197 ; ===== S U B R O U T I N E =====
22198
22199 ; 30/12/2023
22200 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:04CEh
22201 %if 0
22202 get_cpu_type:
22203     pushf
22204     push    bx
22205     xor     bx,bx
22206     xor     ax,ax
22207     push    ax
22208     popf
22209     pushf
22210     pop     ax
22211     and     ax,0F000h
22212     cmp     ax,0F000h
22213     jz      short cpu_8086
22214     mov     ax,0F000h
22215     push    ax
22216     popf
22217     pushf
22218     pop     ax
22219     and     ax,0F000h
22220     jz      short cpu_286
22221 cpu_386:
22222     inc     bx
22223 cpu_286:
22224     inc     bx
22225 cpu_8086:
22226     mov     ax,bx
22227     pop     bx
22228     popf
22229     retn
22230 %endif
22231
22232 ; -----
22233
22234 ;   MOVE THE DOS TO ITS PROPER LOCATION
22235
22236 ; 22/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
22237 ; (SYSINIT:0533h)
22238 ; 30/12/2022 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS)
22239 ; (SYSINIT:04BFh)
22240 ; 03/09/2023 - Retro DOS 4.2 (5.0 - Modified PCDOS 7.1 IBMBIO.COM)
22241 ; (SYSINIT:04F3h)
22242 SYSIN:
22243 ; Retro DOS 5.0 - 30/12/2023
22244 ; Retro DOS 4.0 - 22/03/2019
22245 ; Retro DOS 2.0 - 25/02/2018
22246
22247 ; 23/04/2019
22248 ;;mov     ax,Bios_Data
22249 ;mov     ax,KERNEL_SEGMENT ; 0070h
22250 ; 21/10/2022
22251 mov     ax,DOSBIODATASEG ; 0070h
22252 mov     ds,ax
22253
22254 ; 30/12/2023 - Retro DOS v5.0
22255 ;;;
22256 ;push     es
22257 ;push     ax                ; not needed (*) E.TAN - 03/07/2023
22258 ;push     di
22259
22260 ;call     get_cpu_type     ; determine if 386 system
22261 ;
22262 get_cpu_type:
22263     pushf
22264     xor     ax,ax
22265     push    ax
22266     popf
22267     pushf
22268     pop     ax
22269     and     ax,0F000h
22270     cmp     ax,0F000h
22271     jz      short cpu_8086
22272     mov     ax,0F000h
22273     push    ax
22274     popf
22275     pushf
22276     pop     ax
22277     and     ax,0F000h
22278     jz      short cpu_286
22279 cpu_386:
22280     sub     ax,ax
22281 cpu_286:
22282     inc     ax
22283 cpu_8086:
22284 ; ax = 0
22285 ; 30/12/2023 - Retro DOS v5.0
22286 mov     [cs:cpu_type],al ; 07/04/2024
22287 popf
22288 ;
22289 ;cmp     ax,2                ; 0 = 8086, 1 = 286, 2 = 386
22290 cmp     al,2
22291 jnz     short not_386_system
22292 cld
22293 ; 80386
22294 push     ds
22295 pop     es                ; change A20 line on/off check code
22296 mov     di,cpu386_cmpsd
22297 mov     ax,04B9h          ; mov cx,4 ; B90400
22298 stosw
22299 mov     ax,0F300h          ; repz ; F3
22300 stosw
22301 mov     ax,0A766h          ; cmpsd ; 66A7
22302 stosw
22303 not_386_system:
22304 ;pop     di
22305 ;pop     ax
22306 ;pop     es
22307 ;;;
22308 mov     [MovedOSIntoHMA+2],cs ; set seg of routine to move DOS
22309 mov     byte [SysinitPresent],1 ; flag that MovedOSIntoHMA can be called
22310
22311 ; first move the MSDOS.SYS image up to a harmless place
22312 ; on top of our new sysinitseg
22313
22314 ; 22/10/2022
22315 mov     ax,SI_end          ; how big is sysinitseg?
22316 call    off_to_para
22317 mov     cx,cs              ; pick a buffer for msdos above us

```



```

22317 00000506 01C8      add     ax,cx
22318 00000508 8EC0      mov     es,ax
22319
22320 0000050A 31F6      xor     si,si
22321 0000050C 89F7      mov     di,si
22322
22323 0000050E 2E8E1E[7302]  mov     ds,[cs:CURRENT_DOS_LOCATION] ; where it is (set by msinit)
22324
22325      ;mov     ax,cs
22326      ;mov     ds,ax
22327
22328      ;;mov     cx,20480 ; MSDOS 6.21 IO.SYS - SYSINIT:04E2h
22329      ;;mov     cx,dossize/2 ; MSDOS 6.0
22330      ;mov     cx,[DOSSIZE] ; words (not bytes!) ; Retro DOS v4.0 (3.0, 2.0)
22331      ;mov     es,[FINAL_DOS_LOCATION] ; on top of SYSINIT code
22332      ;mov     ds,[CURRENT_DOS_LOCATION]
22333
22334      ; 22/10/2022
22335 00000513 B90058      mov     cx,DOSSIZE/2 ; 5000h
22336      ; 03/09/2023
22337      ; 5800h (PCDOS 7.1)
22338 00000516 F3A5      rep     movsw
22339 00000518 2E8C06[7302]  mov     [cs:CURRENT_DOS_LOCATION],es
22340
22341      ; The DOS code is ORGed at a non-zero value to allow it to be located in
22342      ; HIMEM. Thus, the DOS segment location must be adjusted accordingly.
22343      ; If this is ROMDOS, however, only the init code is loaded into RAM, so
22344      ; this ORG is not done. The entry point is at offset zero in the segment.
22345
22346      ; 22/04/2019 (MSDOS 6.0 & MSDOS 6.21 kernel address modification)
22347      ;mov     ax,cs
22348      ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
22349      ;mov     ds,ax
22350
22351      ; 29/04/2019 - Retro DOS v4.0 ! important MODIFICATION !
22352
22353      ; ; 24/04/2019
22354      ;;ifndef ROMDOS
22355      ; mov     ax,[es:3] ; get offset of dos
22356      ; ; ax = 3DE0h for MSDOS 6.21 kernel (MSDOS.SYS, offset 3)
22357      ; mov     [dosinit],ax ; that's the entry point offset
22358      ; call    off_to_para ; subtract this much from segment
22359      ; ; 23/04/2019
22360      ; ;sub     [CURRENT_DOS_LOCATION],ax
22361      ; ;sub     [FINAL_DOS_LOCATION],ax
22362      ;;else
22363      ; mov     word [dosinit],0 ; entry to init is at zero
22364      ; ;
22365      ; ;endif ; ROMDOS
22366
22367      ; 29/04/2019 - Retro DOS v4.0 ! important MODIFICATION !
22368      ; (! MSDOS6.BIN starts with DOSDATA ! - Retro DOS v4.0 modification)
22369
22370      ;mov     ax,[es:0] ; DOSCODE start address = DOSDATA size (= 136Ah)
22371      ; ; (Valid for Retro DOS v4.0 only!)
22372
22373      ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
22374      ; (SYSINIT:0563h for MSDOS 5.0 IO.SYS SYSINIT)
22375      ; 03/09/2023 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS)
22376      ; (SYSINIT:04ECh for MSDOS 6.21 IO.SYS SYSINIT)
22377      ; (SYSINIT:0540h for PCDOS 7.1 IBMBIO.COM SYSINIT)
22378 0000051D A10300      mov     ax,[3] ; mov ax, word ptr ds:3
22379      ; 30/12/2023
22380      ; ax = 3F10h for IBMDOS 7.1 kernel
22381      ; (IBMDOS.SYS, offset 3)
22382
22383 00000520 2EA3[7102]      mov     [cs:dosinit],ax ; (SYSINIT:0563h for MSDOS 5.0 IO.SYS SYSINIT)
22384      ; 02/11/2022
22385 00000524 E87308      call    off_to_para ; subtract this much from segment
22386 00000527 2E2906[7302]      sub     [cs:CURRENT_DOS_LOCATION],ax
22387
22388      ; Current DOSCODE start address = dword [dosinit]
22389
22390      ;; If this is not ROMDOS, then the BIOS code is moved to the top of memory
22391      ;; until it is determined whether it will be running in HIMEM or not.
22392
22393      ;ifndef ROMDOS
22394
22395      ; now put Bios_Code up on top of that. Assume Bios_Code + dossize < 64k
22396
22397      ; 22/10/2022
22398 0000052C 8CC0      mov     ax,es
22399 0000052E 05000B      add     ax,DOSSIZE/16 ; get paragraph of end of dos
22400 00000531 8EC0      mov     es,ax
22401 00000533 2E8706[8902]  xchg     ax,[cs:temp_bcode_seg] ; swap with original home of Bios_Code
22402 00000538 8ED8      mov     ds,ax ; point to loaded image of Bios_Code
22403
22404      ;mov     si,BCODE_START ; mov si,30h
22405      ; 09/12/2022
22406 0000053A BE[3000]      mov     si,BCODESTART
22407      ; 02/11/2022
22408 0000053D 89F7      mov     di,si
22409      ; 30/12/2023
22410      ;mov     cx,1E00h ; BCODE_END = (SYSINITSEG-DOSBIOCODESEG)*16
22411      ; ; (544h-364h)*10h = 1E00h (for PCDOS 7.1 IBMBIO.COM)
22412      ;mov     cx,BCODE_END ; mov cx,1A60h ; mov cx,1A70h ; 30/12/2022
22413      ;sub     cx,si
22414      ; 31/03/2024
22415      BCODESIZE equ BCODEEND-BCODESTART
22416 0000053F B9401D      mov     cx,BCODESIZE
22417 00000542 D1E9      shr     cx,1
22418 00000544 F3A5      rep     movsw ; move Bios_Code into place
22419
22420 00000546 8CC0      mov     ax,es ; tell it what segment it's in
22421 00000548 2EFF1E[8702]  call    far [cs:seg_reinit_ptr] ; far call to seg_reinit in Bios_Code (M022)
22422
22423      ;endif ; not ROMDOS
22424
22425      ; now call dosinit while it's in its temporary home
22426
22427      ;mov     ax,cs
22428      ;mov     ds,ax
22429
22430      ;mov     dx,[MEMORY_SIZE] ; set for call to dosinit
22431
22432      ; 22/10/2022
22433
22434 0000054D 2EC43E[9503]  les     di,[cs:BiosComBlock] ; ptr to BIOS communication block
22435      ; es = KERNEL_SEGMENT (70h), di = 'SysInitPresent' address
22436 00000552 2EC536[7502]  lds     si,[cs:DEVICE_LIST] ; set for call to dosinit
22437      ; ds = KERNEL_SEGMENT (70h), si = 'res_dev_list' address
22438
22439 00000557 2E8B16[9402]  mov     dx,[cs:MEMORY_SIZE] ; set for call to dosinit
22440

```

```

22441 0000055C FA      cli
22442 0000055D 8CC8    mov     ax,cs
22443 0000055F 8ED0    mov     ss,ax
22444
22445 ; 30/12/2023 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
22446 ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0, SYSINIT1.ASM)
22447 %define locstack ($ - SYSINIT$) & 0FFFFh ; 532h in MSDOS 6.21 IO.SYS
22448 ; 5A6h in MSDOS 5.0 IO.SYS SYSINIT
22449 ; 586h in PCDOS 7.1 IBMBIO.COM SYSINIT
22450
22451 ;SYSINIT:0532h:
22452
22453 ; 22/10/2022
22454 ;-----
22455 ;SYSINIT:05A6h:
22456 ;locstack: ; (at SYSINIT:05A6h for MSDOS 5.0 IO.SYS)
22457
22458 ; 03/09/2023
22459 ; (locstack at SYSINIT:0586h in PCDOS 7.1 IBMBIO.COM SYSINIT)
22460
22461 ;mov     sp,05A6h
22462 mov     sp,locstack ; set stack
22463
22464 sti
22465
22466 ;align 2
22467 ; 30/03/2018
22468 ;LOCSTACK:
22469 ;CALL     FAR [CS:MSDOS] ; FINAL_DOS_LOCATION:0
22470 ;('jmp DOSINIT' in 'MSHEAD.ASM')
22471 ;('DOSINIT:' is in 'MSINIT.ASM')
22472
22473 ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
22474 ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0, 6.21)
22475
22476 ; This call to DOSINIT will relocate the DOS data from its present location
22477 ; at the top of memory, to its final location in low memory just above the
22478 ; BIOS data. It will then build important DOS data structures in low
22479 ; memory following the DOS data. It returns (among many other things) the
22480 ; new starting address of free memory.
22481 00000565 2EFF1E[7102] call    far [cs:dosinit] ; call dosinit
22482 ; es:di -> sysinitvars_ext
22483
22484 0000056A 2E8C1E[8502] mov     [cs:def_php],ds ; save pointer to PSP
22485
22486 ; 11/12/2022
22487 ; 22/03/2019
22488 0000056F 0E      push    cs
22489 00000570 1F      pop     ds
22490 ; 22/10/2022
22491 00000571 A3[8302] mov     [hi_doscod_size],ax
22492 00000574 890E[8102] mov     [lo_doscod_size],cx
22493 00000578 8916[7D02] mov     [dos_segreinit],dx
22494
22495 ; 11/12/2022
22496 ; ds = cs
22497 ;mov     [cs:hi_doscod_size],ax; size of doscode (including exepatch)
22498 ;mov     [cs:lo_doscod_size],cx; (not including exepatch)
22499 ;mov     [cs:dos_segreinit],dx ; save offset of segreinit
22500
22501 ; 05/06/2018 - Retro DOS v3.0
22502 ; ES:DI = Address of pointer to SYSINITVARS structure (MSDOS 3.3)
22503
22504 ; 11/12/2022
22505 ; ds = cs
22506 ; 22/10/2022
22507 ;mov     ax,[es:di+SysInitVars_Ext.SYSI_InitVars] ; 5/29/86
22508 0000057C 268B05 mov     ax,[es:di] ; 22/03/2019
22509 ;mov     [cs:DOSINFO],ax
22510 0000057F A3[6D02] mov     [DOSINFO],ax
22511 ;mov     ax,[es:di+SysInitVars_Ext.SYSI_InitVars+2]
22512 00000582 268B4502 mov     ax,[es:di+2]
22513 ;mov     [cs:DOSINFO+2],ax
22514 00000586 A3[6F02] mov     [DOSINFO+2],ax ; set the sysvar pointer
22515
22516 ;mov     ax,[es:di+SysInitVars_Ext.SYSI_Country_Tab]
22517 00000589 268B4504 mov     ax,[es:di+4]
22518 ;mov     [cs:sysi_country],ax
22519 0000058D A3[7902] mov     [sysi_country],ax
22520 ;mov     ax,[es:di+SysInitVars_Ext.SYSI_Country_Tab+2]
22521 00000590 268B4506 mov     ax,[es:di+6]
22522 ;mov     [cs:sysi_country+2],ax
22523 00000594 A3[7B02] mov     [sysi_country+2],ax ; set the SYSI_Country pointer
22524
22525 ; 20/04/2019
22526 ;mov     ax,[CURRENT_DOS_LOCATION]
22527 ;mov     es,[CURRENT_DOS_LOCATION]
22528 ;mov     ax,[FINAL_DOS_LOCATION] ; give dos its temporary location
22529 ; 22/10/2022
22530 ;mov     ax,[cs:CURRENT_DOS_LOCATION]
22531 ;;;mov    [dos_segreinit+2],es
22532 ;;;mov    [dos_segreinit+2],ax
22533 ;mov     [cs:dos_segreinit+2],ax
22534 ; 11/12/2022
22535 ; ds = cs
22536 00000597 8E06[7302] mov     es,[CURRENT_DOS_LOCATION]
22537 0000059B 8C06[7F02] mov     [dos_segreinit+2],es
22538 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
22539 ;mov     es,[cs:CURRENT_DOS_LOCATION]
22540 ;mov     [cs:dos_segreinit+2],es
22541
22542 ; -----
22543
22544 ;SYSINIT:0577h:
22545 ; ... RPLArena ... MSDOS 6.21 IO.SYS (SYSINIT:0577h to SYSINIT:05D1h)
22546 ;SYSINIT:05D1h: ; NorRPLArena
22547
22548 ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS SYSINIT)
22549 ;----- Cover up RPL code with an arena
22550 ;SYSINIT:05EBh:
22551 ; 11/12/2022
22552 ; ds = cs
22553 0000059F 31DB      xor     bx,bx
22554 000005A1 391E[9602] cmp     [RPLMemTop],bx ; 0
22555 ;cmp     word [RPLMemTop],0
22556 ;;;cmp    word [cs:RPLMemTop],0
22557 000005A5 7450      je     short NorRPLArena
22558
22559 ;----- alloc all memory
22560
22561 ; 11/12/2022
22562 ;mov     bx,0FFFFh
22563 000005A7 4B      dec     bx
22564 ; bx = 0FFFFh

```



```

22565 000005A8 B448      mov     ah,48h
22566 000005AA CD21      int      21h
22567                      ; DOS - 2+ - ALLOCATE MEMORY
22568                      ; BX = number of 16-byte paragraphs desired
22569 000005AC B448      mov     ah,48h
22570 000005AE CD21      int      21h
22571
22572 000005B0 8EC0      mov     es,ax          ; get it into ES and save it
22573 000005B2 06        push    es
22574
22575 ;----- resize upto RPL mem
22576
22577 ; 11/12/2022
22578 ; ds = cs
22579 ;sub     ax,[cs:RPLMemTop]
22580 000005B3 2B06[9602] sub     ax,[RPLMemTop]
22581 000005B7 F7D8      neg     ax
22582 000005B9 48        dec     ax
22583 000005BA 89C3      mov     bx,ax
22584 000005BC B44A      mov     ah,4Ah
22585 000005BE CD21      int      21h
22586                      ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
22587                      ; ES = segment address of block to change
22588                      ; BX = new size in paragraphs
22589
22590 ;----- allocate the free (RPL MEM)
22591
22592 000005C0 BBFFFF      mov     bx,0FFFFh
22593 000005C3 B448      mov     ah,48h
22594 000005C5 CD21      int      21h
22595 000005C7 B448      mov     ah,48h
22596 000005C9 CD21      int      21h
22597
22598 ;----- mark that it belongs to RPL
22599
22600 000005CB 48        dec     ax
22601 000005CC 8EC0      mov     es,ax
22602 ;mov     word [es:arena_owner],8
22603 000005CE 26C70601000800 mov     word [es:1],8
22604 ;mov     word [es:arena_name],'RP'
22605 000005D5 26C70608005250 mov     word [es:8],'RP'
22606 ;mov     word [es:arena_name+2],'L'
22607 000005DC 26C7060A004C00 mov     word [es:10],'L'
22608 ;mov     word [es:arena_name+4],0
22609 000005E3 26C7060C000000 mov     word [es:12],0
22610 ;mov     word [es:arena_name+6],0
22611 000005EA 26C7060E000000 mov     word [es:14],0
22612
22613 000005F1 07        pop     es          ; get back ptr to first block
22614 000005F2 B449      mov     ah,49h          ; Dealloc      ; and free it
22615 000005F4 CD21      int      21h
22616                      ; DOS - 2+ - FREE MEMORY
22617                      ; ES = segment address of area to be freed
22618 ; 11/12/2022
22619 000005F6 F8        cld
22620
22621 ; -----
22622
22623 NORPLArena:
22624 ; 11/12/2022
22625 ; ds = cs
22626 ; 22/03/2019 - Retro DOS v4.0 (MSDOS 6.0, 6.21, IO.SYS)
22627 000005F7 C43E[6D02] les     di,[DOSINFO] ; es:di -> dosinfo
22628 ; 22/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS SYSINIT)
22629 ;les     di,[cs:DOSINFO] ; es:di -> dosinfo
22630
22631 ; 11/12/2022
22632 ;cld
22633 ; get the extended memory size
22634
22635 ; execute the get extended memory size subfunction in the bios int 15h
22636 ; if the function reports an error do nothing else store the extended
22637 ; memory size reported at the appropriate location in the dosinfo buffer
22638 ; currently pointed to by es:di. use the offsets specified in the
22639 ; definition of the sysinitvars struct in inc\sysvar.inc
22640 000005FB B488      mov     ah,88h
22641 000005FD CD15      int      15h          ; check extended memory size
22642 000005FF 720B      jc      short no_ext_memory
22643                      ; Get Extended Memory Size
22644                      ; Return: CF clear on success
22645                      ; AX = size of memory above 1M in K
22646 ;mov     [es:di+SYSI_EXT_MEM],ax ; save extended memory size
22647 ; 22/10/2022
22648 00000601 26894545 mov     [es:di+45h],ax ; save extended memory size
22649 00000605 09C0      or      ax,ax
22650 00000607 7403      jz      short no_ext_memory
22651 00000609 E8F006      call    C1rVDISKHeader
22652 no_ext_memory:
22653 ;mov     ax,[es:di+SYSI_MAXSEC]; get the sector size
22654 0000060C 268B4510 mov     ax,[es:di+10h]
22655 ;add     ax,bufinsiz
22656 ; 30/12/2023 - Retro DOS v5.0
22657 ;add     ax,20          ; size of buffer header
22658 00000610 83C018      add     ax,24          ; bufinsiz
22659                      ; size of buffer header = 24 (PCDOS v7.1 IBMBIO.COM)
22660                      ; (it was 20 in MSDOS 6.22 IO.SYS)
22661 ; 11/12/2022
22662 ; ds = cs
22663 00000613 A3[9D02] mov     [singlebuffersize],ax ; total size for a buffer
22664 ;mov     [cs:singlebuffersize],ax
22665 ; 11/12/2022
22666 00000616 A0[9802] mov     al,[DEFAULT_DRIVE] ; get the 1 based boot drive number set by msinit
22667 ;mov     al,[cs:DEFAULT_DRIVE]
22668 ;mov     [es:di+SYSI_BOOT_DRIVE],al ; set sysi_boot_drive
22669 00000619 26884543 mov     [es:di+43h],al
22670
22671 ; determine if 386 system...
22672
22673 ; 30/12/2023
22674 %if 0
22675 ;get_cpu_type          ; macro to determine cpu type
22676
22677 get_cpu_type:
22678 ; 11/12/2022
22679 pushf
22680 ;push    bx
22681 ;xor     bx,bx
22682 ; 11/12/2022
22683 ;xor     cx,cx
22684 ;
22685 ;xor     ax,ax
22686 ; ax = 0
22687 push    ax
22688 popf

```

```

22689      pushf
22690      pop     ax
22691      and     ax,0F000h
22692      ;cmp    ax,0F000h
22693      cmp     ah,0F0h
22694      je      short cpu_8086
22695      ;mov     ax,0F000h
22696      mov     ah,0F0h
22697      ; ax = 0F000h
22698      push    ax
22699      popf
22700      pushf
22701      pop     ax
22702      ;and     ax,0F000h
22703      and     ah,0F0h
22704      jz      short cpu_286
22705      cpu_386:
22706      ; 11/12/2022
22707      ;;;inc    bx
22708      ;;;inc    cx
22709      ; 11/12/2022
22710      ;mov     byte [es:di+SYSI_DWMOVE],1
22711      mov     byte [es:di+44h],1
22712
22713      ; 03/09/2023 - Retro DOS v5.0 (PCDOS 7.1 Modified SYSINIT)
22714      ; change A20 line on/off check code to the faster (for 32 bit cpu)
22715      ;push    es
22716      ;push    di
22717      ;mov     ax,DOSBIODATASEG ; 0070h
22718      ;mov     es,ax
22719      ;cld
22720      ;mov     di,cpu386_cmpsd ; (IsA200ff)
22721      ;mov     ax,489h          ; mov cx,4 ; B90400
22722      ;stosw
22723      ;mov     ax,0F300h        ; repz ; F3
22724      ;stosw
22725      ;mov     ax,0A766h        ; cmpsd ; 66A7
22726      ;stosw
22727      ;pop     di
22728      ;pop     es
22729
22730      cpu_286:
22731      ;;;inc    bx
22732      ;;;inc    cx
22733      cpu_8086:
22734      ; 11/12/2022
22735      ;;;mov    ax,bx
22736      ;pop     bx
22737      popf
22738      %endif
22739      ;...
22740
22741      ; 11/12/2022
22742      ;or      cl,cl
22743      ;jz      short not_386_system
22744      ; 11/12/202
22745      ;cmp     cl,2
22746      ;;;cmp    ax,2          ; is it a 386?
22747      ;jne     short not_386_system ; no: don't mess with flag
22748
22749      ; 30/12/2023 - Retro DOS v5.0
22750      cmp     byte [cpu_type], 2 ; is it a 386?
22751      jne     short _not_386_cpu ; no: don't mess with flag
22752
22753      ;mov     byte [es:di+SYSI_DWMOVE],1
22754      ; 11/12/2022
22755      ; 22/10/2022
22756      mov     byte [es:di+44h],1
22757      _not_386_cpu:
22758      ;mov     al,[es:di+SYSI_NUMIO]
22759      mov     al,[es:di+20h]
22760      ; 11/12/2022
22761      ; ds = cs
22762      mov     [drivenumber],al ; save start of installable block drvs
22763      ;mov     [cs:drivenumber],al
22764
22765      mov     ax,cs
22766      sub     ax,11h          ; room for PSP we will copy shortly
22767      ; 11/12/2022
22768      ;mov     cx,[singlebuffersize] ; temporary single buffer area
22769      ;;;mov    cx,[cs:singlebuffersize]
22770      ;shr     cx,1
22771      ;shr     cx,1          ; divide size by 16...
22772      ;shr     cx,1
22773      ;shr     cx,1          ; ...to get paragraphs...
22774      ;inc     cx          ; ... and round up
22775      ; 11/12/2022
22776      mov     bx,[singlebuffersize]
22777      mov     cl,4
22778      shr     bx,cl
22779      inc     bx
22780
22781      ; cas note: this unorthodox paragraph rounding scheme wastes a byte
22782      ; if [singlebuffersize] ever happens to be zero mod 16. Could this
22783      ; ever happen? Only if the buffer overhead was zero mod 16, since
22784      ; it is probably safe to assume that the sector size always will be.
22785      ;
22786      ; mohans also found a bug in CONFIG.SYS processing where it replaces
22787      ; EOF's with cr,lf's, without checking for collision with [confbot].
22788      ; perhaps the extra byte this code guarantees is what has kept that
22789      ; other code from ever causing a problem???
22790
22791      ; 11/12/2022
22792      sub     ax,bx
22793      ;sub     ax,cx
22794      mov     [top_of_cdss],ax ; temp "unsafe" location
22795      ; 22/10/2022
22796      ;mov     [cs:top_of_cdss],ax
22797
22798      ; chuckst -- 25 Jul 92 -- added code here to pre-allocate space
22799      ; for 26 temporary CDSSs, which makes it easier to use alloclim
22800      ; for allocating memory for MagicDrv.
22801
22802      ; 30/12/2023
22803      ;push    es ; not necessary (!*) ; preserve pointer to dosinfo
22804      ;push    di
22805
22806      ; 22/10/2022
22807      ; mov     cx,ax          ; save pointer for buffer
22808      ;
22809      ;;; now allocate space for 26 CDSSs
22810      ;
22811      sub     ax,((26*(curdirlen))+15)/16
22812      ; mov     [ALLOCLIM],ax ; init top of free memory pointer

```

```

22813      ; mov     [CONFBOT],ax          ; init this in case no CONFIG.SYS
22814
22815      ; 30/12/2022 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS)
22816      ; (SYSINIT:064Ch)
22817      mov     cx,ax ; (*)
22818      sub     ax,((26*(curdirilen))+15)/16 ; sub ax,143
22819      mov     [ALLOCLIM],ax          ; init top of free memory pointer
22820      mov     [CONFBOT],ax          ; init this in case no CONFIG.SYS
22821
22822      ; setup and initialize the temporary buffer at cx
22823
22824      ; les     di,[es:di+SYSI_BUF]    ; get the buffer chain entry pointer
22825      les     di,[es:di+12h]
22826      ; 11/12/2022
22827      xor     bx,bx
22828      ; xor     ax,ax
22829      ; mov     [es:di+BUFFINF.Dirty_Buff_Count],ax ; 0
22830      mov     word [es:di+4],0
22831      mov     [es:di+4],bx ; 0
22832      ; mov     [es:di+BUFFINF.Buff_Queue],ax ; 0
22833      mov     word [es:di],0
22834      mov     [es:di],bx ; 0
22835      ; mov     [es:di+BUFFINF.Buff_Queue+2],cx ; cx = [top_of_cdss] ; 6.21
22836      ; ; mov   [es:di+BUFFINF.Buff_Queue+2],ax ; ax = [top_of_cdss] ; 5.0
22837      ; mov     [es:di+2],ax
22838      ; mov     es,ax ; [top_of_cdss] = [CONFBOT]
22839      ; 30/12/2022 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS, SYSINIT)
22840      mov     [es:di+2],cx ; [top_of_cdss] ; (*)
22841      mov     es,cx
22842
22843      ; 11/12/2022
22844      ; xor     ax,ax
22845      ; mov     di,ax
22846      mov     di,bx
22847      ; di = 0
22848
22849      ; mov     [es:di+buffinfo.buf_next],ax ; points to itself
22850      ; 11/12/2022
22851      ; mov     [es:di],ax ; 0
22852      mov     [es:di],bx ; 0
22853      ; mov     [es:di+buffinfo.buf_prev],ax ; points to itself
22854      ; 11/12/2022
22855      ; mov     [es:di+2],ax ; 0
22856      mov     [es:di+2],bx ; 0
22857
22858      ; 22/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS SYINIT)
22859      ; MSDOS 5.0 IO.SYS - SYSINIT:06E0h
22860
22861      ; mov     word [es:di+buffinfo.buf_ID],00FFh ; free buffer,clear flag
22862      mov     word [es:di+4],00FFh
22863      ; SYSINIT:06E6h
22864      ; ; mov   [es:di+buffinfo.buf_sector],ax ; 0
22865      ; mov     word [es:di+6],0
22866      ; 11/12/2022
22867      ; mov     [es:di+buffinfo.buf_sector],bx ; 0
22868      mov     [es:di+6],bx ; 0
22869      ; ; mov   [es:di+buffinfo.buf_sector+2],ax ; 0
22870      ; mov     word [es:di+8],0
22871      ; 11/12/2022
22872      ; mov     [es:di+buffinfo.buf_sector+2],bx ; 0
22873      mov     [es:di+8],bx ; 0
22874
22875      ; 30/12/2023 (!*)
22876      ; pop     di
22877      ; pop     es
22878      ; restore pointer to DOSINFO data
22879
22880      ; 11/12/2022
22881      ; ds = cs
22882      ; 22/10/2022
22883      ; push    cs
22884      ; pop     ds
22885      call     TempCDS
22886      ; set up cdss so re_init and sysinit
22887      ; can make disk system calls
22888      ; tempcde trashes ds
22889      ; 10/05/2019
22890      mov     ds,[cs:def_php]
22891      ; retrieve pointer to PSP returned by DOSINIT
22892      ; if not ibmjapver
22893      ; call     far KERNEL_SEGMENT:re_init ; re-call the bios
22894      ; endif
22895
22896      ; 22/10/2022
22897      ; SYSINIT:06FEh: ; (MSDOS 5.0 IO.SYS, SYSINIT)
22898      ; 30/12/2022
22899      ; SYSINIT:0697h: ; (MSDOS 6.21 IO.SYS, SYSINIT)
22900      ; call     far ptr 70h:89Bh
22901      call     DOSBIODATASEG:RE_INIT
22902
22903      sti
22904      ; ints ok
22905      cld
22906      ; make sure
22907
22908      ; 23/03/2019
22909      ; SYSINIT:069Eh ; 30/12/2022
22910
22911      ; dosinit has set up a default "process" (php) at ds:0. we will move it out
22912      ; of the way by putting it just below sysinit at end of memory.
22913      mov     bx,cs
22914      sub     bx,10h
22915      mov     es,bx
22916      xor     si,si
22917      mov     di,si
22918      mov     cx,128
22919      rep     movsw
22920
22921      ; mov     [es:PDB.JFN_POINTER+2],es ; Relocate
22922      ; 22/10/2022
22923      mov     [es:36h],es
22924
22925      ; Set Process Data Block - Program Segment Prefix address
22926      ; BX = PDB/PSP segment
22927      mov     ah,50h ; SET_CURRENT_PDB
22928      int     21h
22929      ; tell DOS we moved it
22930      ; DOS - 2+ internal - SET PSP SEGMENT
22931      ; BX = segment address of new PSP
22932
22933      ; 22/10/2022
22934      ; 27/03/2019
22935      ; 30/12/2023
22936      ; push    ds ; */
22937      ; preserve DS returned by DOSINIT
22938      push    cs
22939      pop     ds

```

```

22937
22938 ; set up temp. critical error handler
22939 000006A2 BA[774A] mov dx,int24 ; set up int 24 handler
22940 ;;mov ax,(SET_INTERRUPT_VECTOR*256)+24h
22941 ;mov ax,(SET_INTERRUPT_VECTOR<<8)|24h
22942 000006A5 882425 mov ax,2524h
22943 000006A8 CD21 int 21h
22944
22945 000006AA 803E[8803]00 cmp byte [toomanydrivesflag],0 ; Q: >24 partitions? M029
22946 000006AF 7406 je short no_err ; N: continue M029
22947 000006B1 BA[9C53] mov dx,TooManyDrivesMsg ; Y: print error message M029
22948 ; 22/10/2022
22949 ;call print ; M029
22950 ; 12/12/2022
22951 000006B4 EB04 jmp short p_dosinit_msg ; 23/03/2019 - Retro DOS v4.0
22952
22953 ; 30/12/2023 - Retro DOS v5.0
22954 cpu_type:
22955 000006B6 FF db 0FFh ; db 0
22956
22957 no_err:
22958 ; 12/05/2019
22959 ; -----
22960 ; 27/06/2018 - Retro DOS v3.0 ; 23/03/2019 - Retro DOS v4.0
22961 ; 22/10/2022 - Retro DOS v4.0
22962 ; 12/12/2022
22963 ; 30/12/2023 - Retro DOS v5.0
22964 000006B7 BA[7B4A] mov dx,BOOTMES ; Display (fake) MSDOS version message
22965 p_dosinit_msg:
22966 000006BA E89543 call print ; Print message
22967 ; -----
22968 ; 11/12/2022
22969 ; 22/10/2022
22970 ; 23/03/2019 - Retro DOS v4.0
22971 ;pop ds ; start of free memory
22972 ;mov dl,[cs:DEFAULT_DRIVE]
22973
22974 ; 11/12/2022
22975 ; 27/03/2019
22976 000006BD 8A16[9802] mov dl,[DEFAULT_DRIVE]
22977 ; 30/12/2023
22978 ;pop ds ; */
22979
22980 or dl,dl
22981 ; 30/12/2023
22982 jz short nodrvset ; bios didn't say
22983 000006C3 7405 jz short ProcessConfig ; (Retro DOS v4.0 does not contain DBLSPACE code)
22984 ;dec dl ; A = 0
22985 ; 18/12/2022
22986 dec dx
22987 000006C5 4A mov ah,0Eh ; SET_DEFAULT_DRIVE
22988 000006C6 B40E int 21h ; select the disk
22989 000006C8 CD21 ; DOS - SELECT DISK
22990 ; DL = new default drive number (0 = A, 1 = B, etc.)
22991 ; Return: AL = number of logical drives
22992
22993 nodrvset:
22994 ; 04/01/2023 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS SYINIT)
22995 ; (SYSINIT:06DFh)
22996
22997 ; 30/12/2023 - Retro DOS 5.0
22998 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:0733h)
22999 000006CA 1E push ds
23000 000006CB 29C0 sub ax,ax
23001 000006CD 8ED8 mov ds,ax ; 0 ; ROM BIOS Data Area
23002 000006CF A16C04 mov ax,[46Ch] ; timer tick count (18.2 ticks per second)
23003 ;mov [cs:_timer_lw_],ax
23004 000006D2 1F pop ds
23005 ; ds = cs
23006 000006D3 A3[8C03] mov [_timer_lw_],ax
23007
23008 ; -----
23009 ;ifdef dblspace_hooks
23010 ; ....
23011 ; ....
23012 ;endif
23013
23014 ; -----
23015 ; 30/12/2023 - Retro DOS 5.0 (Modified MSDOS 7.1 IBMBIO.COM SYS SYINIT)
23016 ; -----
23017 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0740h
23018 ; -----
23019
23020 push cs
23021 000006D6 0E pop es
23022 000006D7 07
23023
23024 ; 07/04/2024
23025 ;mov word [cs:MagicBackdoor+2],cs
23026 ;mov word [cs:MagicBackdoor], NullBackdoor
23027 000006D8 8C0E[9203] mov word [MagicBackdoor+2],cs
23028 000006DC C706[9003][9403] mov word [MagicBackdoor], NullBackdoor
23029
23030 ; ds = es = cs = SYSINIT segment
23031
23032 000006E2 BE[AB16] set_drvspc_size:
23033 mov si,MagicDDName ; "\DBLSPACE.BIN"
23034 000006E5 E8772F set_dblspc_size:
23035 000006E8 732B call SizeDevice
23036 jnc short wait_for_key_2s
23037 ;cmp byte [cs:si], 'C'
23038 000006EA 803C43 cmp byte [si], 'C' ; "C:\STACKER.BIN"
23039 000006ED 740C je short set_drvspc_name
23040 ;cmp byte [cs:DEFAULT_DRIVE], 3
23041 000006EF 803E[9802]03 cmp byte [DEFAULT_DRIVE], 3
23042 000006F4 7405 je short set_drvspc_name
23043 000006F6 83EE02 sub si,2 ; "C:\DBLSPACE.BIN"
23044 000006F9 EBEA jmp short set_dblspc_size
23045
23046 set_drvspc_name:
23047 000006FB 803E[AD16]52 cmp byte [cs:MagicDDName+2], 'R' ; "BSPACE.BIN"
23048 00000700 7408 cmp byte [MagicDDName+2], 'R'
23049 je short set_stack_name
23050 ;mov word [cs:MagicDDName+2], 'RV' ; "DRVSPACE.BIN"
23051 00000702 C706[AD16]5256 mov word [MagicDDName+2], 'RV'
23052 00000708 EBD8 jmp short set_drvspc_size
23053
23054 set_stack_name:
23055 0000070A 81FE[B916] cmp si,StackerName ; "C:\STACKER.BIN"
23056 0000070E 734B jnb short wfk2s_4
23057 00000710 BE[BB16] mov si,StackerName+2 ; "\STACKER.BIN"
23058 00000713 EBD0 jmp short set_dblspc_size
23059
23060 wait_for_key_2s:
;mov [cs:MagicDDNamePtr], si

```

```

23061 00000715 8936[A716]      mov     [MagicDDNamePtr],si
23062 00000719 1E          push    ds
23063 0000071A 29C0       sub     ax,ax
23064 0000071C 8ED8       mov     ds,ax ; 0 ; ROMBIOS data area
23065 0000071E 8B166C04   mov     dx,[46Ch] ; Counter for Interrupt 1Ah
23066 wfk2s_1:
23067 00000722 B401       mov     ah,1
23068 00000724 CD16       int     16h ; KEYBOARD - CHECK BUFFER, DO NOT CLEAR
23069 ; Return: ZF clear if character in buffer
23070 ; AH = scan code, AL = character
23071 ; ZF set if no character in buffer
23072 00000726 7511       jnz     short wfk2s_2
23073 00000728 B402       mov     ah,2
23074 0000072A CD16       int     16h ; KEYBOARD - GET SHIFT STATUS
23075 ; AL = shift status bits
23076 0000072C A803       test    al,3
23077 0000072E 7509       jnz     short wfk2s_2
23078 00000730 A16C04   mov     ax,[46Ch] ; tick count
23079 00000733 29D0       sub     ax,dx
23080 00000735 3C25       cmp     al,37 ; 2 seconds
23081 00000737 72E9       jb      short wfk2s_1 ; wait for user's key press
23082 wfk2s_2:
23083 00000739 1F          pop     ds ; read/check the pressed key
23084 0000073A 29DB       sub     bx,bx ; bx = 0
23085 0000073C B402       mov     ah,2
23086 0000073E CD16       int     16h ; KEYBOARD - GET SHIFT STATUS
23087 ; AL = shift status bits
23088 00000740 A803       test    al,3 ; Left or Right SHIFT key pressed ?
23089 00000742 7402       jz      short wfk2s_3 ; no
23090 00000744 43          inc     bx
23091 00000745 43          inc     bx ; bx = 2
23092 wfk2s_3:
23093 00000746 B401       mov     ah,1
23094 00000748 CD16       int     16h ; KEYBOARD - CHECK BUFFER, DO NOT CLEAR
23095 ; Return: ZF clear if character in buffer
23096 ; AH = scan code, AL = character
23097 ; ZF set if no character in buffer
23098 0000074A 7418       jz      short wfk2s_6
23099 0000074C 80FC65     cmp     ah,65h ; F8 key pressed ?
23100 0000074F 740C       jz      short wfk2s_5
23101 00000751 80FC62     cmp     ah,62h ; F5 key pressed ?
23102 00000754 750E       jnz     short wfk2s_6
23103 ;mov     byte [cs:F5_key],1
23104 00000756 C606[8E03]01 mov     byte [F5_key],1
23105 wfk2s_4:
23106 0000075B EB49       jmp     short ProcessConfig ; continue (as normal/default state)
23107
23108 wfk2s_5:
23109 ;mov     byte [cs:F8_key],1
23110 0000075D C606[8F03]01 mov     byte [F8_key],1
23111 00000762 EB42       jmp     short ProcessConfig
23112
23113 wfk2s_6:
23114 00000764 E8AA02   call    AllocFreeMem ; get the largest free block from DOS
23115 00000767 E8700F   call    MagicPreload ; **** PRE-LOAD MAGICDRV!!! ****
23116
23117 ; 07/04/2024 - Retro DOS v5.0
23118 ; (DS may not be same with CS here!)
23119 0000076A 0E          push    cs
23120 0000076B 1F          pop     ds ; *
23121 0000076C 8E06[6803]   mov     es,[area]
23122
23123 00000770 09C0       or      ax,ax ; error?
23124 00000772 7406       jz      short wfk2s_7
23125 PreloadFailed:
23126 00000774 B449       mov     ah,49h ; Dealloc ; free the block if no load
23127 ;mov     es,[cs:area]
23128 ;mov     es,[area]
23129 00000776 CD21       int     21h ; DOS - 2+ - FREE MEMORY
23130 ; ES = segment address of area to be freed
23131 00000778 EB2C       jmp     short ProcessConfig
23132
23133 wfk2s_7:
23134 ;mov     bx,[cs:memhi]
23135 ;mov     es,[cs:area]
23136 ;sub     bx,[cs:area] ; get desired block size in paras
23137 ; 07/04/2024 - Retro DOS v5.0
23138 ; ds = cs ; *
23139 0000077A 8CC3       mov     bx,es
23140 0000077C F7DB       neg     bx ; bx = - [cs:area]
23141 0000077E 031E[6403]   add     bx,[memhi] ; bx = [cs:memhi] - [cs:area]
23142
23143 00000782 B44A       mov     ah,4Ah
23144 00000784 CD21       int     21h ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
23145 ; ES = segment address of block to change
23146 ; BX = new size in paragraphs
23147 00000786 8CC0       mov     ax,es
23148 00000788 48          dec     ax
23149 00000789 8EC0       mov     es,ax ; get Magicdrv arena
23150
23151 0000078B 26C70601000800 mov     word [es:1],8 ; [es:arena_owner]
23152 ;mov     word [es:ARENA.OWNER],8 ; set impossible owner
23153 00000792 26C70608005344 mov     word [es:8],4453h ; [es:arena_name],'SD' ; System Data
23154 ;mov     word [es:ARENA.NAME],'SD' ; 4453h
23155 00000799 2603060300 add     ax,[es:3] ; get MCB length
23156 ;add     ax,[es:ARENA.SIZE]
23157
23158 ;lds     si,[cs:DOSINFO] ; get to arena header
23159 0000079E C536[6D02]   lds     si,[DOSINFO]
23160 000007A2 40          inc     ax ; get addr of next MCB
23161 000007A3 8944FE     mov     [si-2], ax ; store that
23162
23163 ; -----
23164 ; MSDOS 6.21 IO.SYS, SYSINIT:0744h
23165 ;
23166 ; 23/03/2019 - Retro DOS v4.0 (MSDOS 6.0, SYSINIT1.ASM, 1991)
23167 ; -----
23168 ; 22/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS SYSINIT)
23169 ; -----
23170 ; 30/12/2022 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS SYSINIT)
23171 ; -----
23172 ; 07/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM SYSINIT)
23173 ;
23174 ; (MSDOS 6.22 IO.SYS - SYSINIT:0744h)
23175 ;
23176 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0820h
23177 ;
23178 ProcessConfig:
23179 ; ds = cs ; 27/03/2019
23180 ; 11/12/2022
23181 ; ds <> cs
23182
23183 ; (MSDOS 5.0 IO.SYS - SYSINIT:0746h)
23184

```

```

23185
23186 000007A6 E8BD1C      call    doconf          ; do pre-scan for dos=high/low
23187
23188      ; 11/12/2022
23189      ; 27/03/2019
23190      ; ds = cs (at return from doconf)
23191
23192      ; Now, if this is not romdos, we decide what to do with the DOS code.
23193      ; It will either be relocated to low memory, above the DOS data structures,
23194      ; or else it will be located in HiMem, in which case a stub with the DOS
23195      ; code entry points will be located in low memory. Dos_segreinit is used
23196      ; to tell the DOS data where the code has been placed, and to install the
23197      ; low memory stub if necessary. If the DOS is going to go into HiMem, we
23198      ; must first initialize it in its present location and load the installable
23199      ; device drivers. Then, if a HiMem driver has been located, we can actually
23200      ; relocate the DOS code into HiMem.
23201      ;
23202      ; For ROMDOS, if DOS=HIGH is indicated, then we need to call dos_segreinit
23203      ; to install the low memory stub (this must be done before allowing any
23204      ; device drivers to hook interrupt vectors). Otherwise, we don't need to
23205      ; call dos_segreinit at all, since the interrupt vector table has already
23206      ; been patched.
23207
23208      ; 22/10/2022 - Retro DOS v4.0
23209      ; (MSDOS 5.0 IO.SYS - SYSINIT:0749h)
23210      ; cmp byte [cs:runhigh],0 ; Did user choose to run low ?
23211      ; 11/12/2022
23212 000007A9 803E[6C02]00  cmp     byte [runhigh],0
23213 000007AE 7404          je      short dont_install_stub ; yes, don't install dos low mem stub
23214
23215      ;----- user chose to load high
23216
23217      ; 22/10/2022
23218      ; mov es,[cs:CURRENT_DOS_LOCATION] ; MSDOS 6.21 (& MSDOS 6.0)
23219      ; 11/12/2022
23220      ; ds = cs
23221      ; 13/04/2024
23222      %if 0
23223      mov     es,[CURRENT_DOS_LOCATION]
23224      %endif
23225      ; mov es,[cs:FINAL_DOS_LOCATION] ; Retro DOS v4.0
23226      ; 27/03/2019
23227      ; mov es,[FINAL_DOS_LOCATION]
23228
23229 000007B0 31C0          xor     ax,ax ; ax = 0 ---> install stub
23230
23231      ; 13/04/2024
23232      %if 0
23233      ; 11/12/2022
23234      ; ds = cs
23235      ; call far [cs:dos_segreinit]; call dos segreinit
23236      call far [dos_segreinit]
23237      %endif
23238 000007B2 EB08          jmp     short do_multi_pass
23239
23240      ;----- User chose to load dos low
23241
23242      dont_install_stub:
23243      ; 22/10/2022
23244 000007B4 31DB          xor     bx,bx ; M012
23245      ; don't use int 21 call to alloc mem
23246 000007B6 E80E03      call    MovDOSLo ; move it !
23247
23248 000007B9 B80100      mov     ax,1 ; dont install stub
23249
23250      ; 13/04/2024
23251      %if 1
23252      do_multi_pass:
23253      %endif
23254      ; 11/12/2022
23255      ; ds = cs
23256 000007BC 8E06[7302]      mov     es,[CURRENT_DOS_LOCATION]
23257      ; mov es,[cs:CURRENT_DOS_LOCATION] ; set_dos_final_position set it up
23258      ; mov es,[cs:FINAL_DOS_LOCATION] ; Retro DOS v4.0
23259      ; 27/03/2019
23260      ; do_multi_pass:
23261      ; mov es,[FINAL_DOS_LOCATION]
23262
23263      ; 11/12/2022
23264      ; ds =cs
23265      ; call far [cs:dos_segreinit]; inform dos about new seg
23266 000007C0 FF1E[7D02]      call far [dos_segreinit]
23267
23268      ; 13/04/2024
23269      %if 0
23270      do_multi_pass:
23271      %endif
23272
23273 000007C4 E84A02      call    AllocFreeMem ; allocate all the free mem
23274      ; & update [memhi] & [area]
23275      ; start of free memory.
23276
23277      ; ifdef dblspace_hooks
23278      ; mov bx,0 ; magic backdoor to place int hooks
23279      ; call cs:MagicBackdoor
23280      ; endif
23281
23282      ; 07/04/2024 - Retro DOS v5.0
23283      ; (PCDOS 7.1 IBMBIO.COM)
23284 000007C7 803E[8E03]01      cmp     byte [cs:F5_key],1
23285 000007CC 740D          je      short skip_magicbackdoor
23286      ; cmp byte [cs:F8_key],1
23287 000007CE 803E[8F03]01      cmp     byte [F8_key],1
23288 000007D3 7406          je      short skip_magicbackdoor
23289 000007D5 31DB          xor     bx,bx ; bx = 0 ; magic backdoor to place int hooks
23290      ; call far [cs:MagicBackdoor]
23291 000007D7 FF1E[9003]      call far [MagicBackdoor]
23292
23293      skip_magicbackdoor:
23294
23295      ; Now, process config.sys some more.
23296      ; Load the device drivers and install programs
23297
23298      ; 22/10/2022
23299      ; inc byte [cs:multi_pass_id] ; multi_pass_id = 1
23300      ; 11/12/2022
23301      ; ds = cs
23302 000007DB FE06[CD02]      inc     byte [multi_pass_id]
23303 000007DF E8201D      call    multi_pass ; load device drivers
23304 000007E2 E8E831      call    ShrinkUMB
23305 000007E5 E80C32      call    UnlinkUMB ; unlink all UMBS ;M002
23306      ; 02/11/2022
23307      ; inc byte [cs:multi_pass_id] ; multi_pass_id = 2
23308      ; 11/12/2022

```



```

23309      ; ds = cs
23310 000007E8 FE06[CD02] inc byte [multi_pass_id]
23311 000007EC E8131D call multi_pass ; was load ifs (now does nothing)
23312
23313 ;ifdef dblspace_hooks
23314 ;call MagicPostload ; make sure Magicdrv is final placed
23315 ;endif
23316
23317 ; ds = cs
23318
23319 ; 07/04/2024
23320 ;call endfile ; setup fcbs, files, buffers etc
23321
23322 ;ifdef dblspace_hooks
23323 ;call MagicSetCdss ; disable CDSS of reserved drives
23324 ;endif
23325
23326 ; 07/04/2024 - Retro DOS v5.0
23327 ; (PCDOS 7.1 IBMBIO.COM)
23328 ;cmp byte [cs:F5_key],1
23329 000007EF 803E[8E03]01 cmp byte [F5_key],1
23330 000007F4 7412 je short skip_magicpostload
23331 ;cmp byte [cs:F8_key],1
23332 000007F6 803E[8F03]01 cmp byte [F8_key],1
23333 000007FB 740B je short skip_magicpostload
23334 000007FD E8B710 call MagicPostload ; make sure Magicdrv is final placed
23335 ; 13/04/2024
23336 ; ds = cs
23337 00000800 E83E06 call endfile ; setup fcbs, files, buffers etc
23338 00000803 E81011 call MagicSetCdss ; disable CDSS of reserved drives
23339 ; ds = cs
23340 00000806 EB03 jmp short _@_
23341
23342 skip_magicpostload:
23343 ; 13/04/2024
23344 ; ds = cs
23345 00000808 E83606 call endfile ; setup fcbs, files, buffers etc
23346 _@_:
23347
23348 ;Reset SysinitPresent flag here. This is needed for the special fix for lying
23349 ;to device drivers. This has been moved up to this point to avoid problems
23350 ;with overlays called from installed programs
23351
23352 ; 11/12/2022
23353 ; ds = cs
23354
23355 ;;mov ax,Bios_Data ; 0070h
23356 ;mov ax,KERNEL_SEGMENT
23357 ; 21/10/2022
23358 0000080B B87000 mov ax,DOSBIODATASEG ; 0070h
23359 0000080E 8EC0 mov es,ax ; point ES to bios data
23360
23361 00000810 26C606[DD07]00 mov byte [es:SysinitPresent],0 ; clear SysinitPresent flag
23362
23363 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
23364 ;test word [cs:install_flag],have_install_cmd ; 1
23365 ;test byte [cs:install_flag],1
23366 ; 11/12/2022
23367 ; ds = cs
23368 00000816 F606[CE02]01 test byte [install_flag],1
23369 ;test byte [cs:install_flag],have_install_cmd
23370 ; are there install commands?
23371 0000081B 7407 jz short dolast ; no, no need for further processing
23372 ;inc byte [cs:multi_pass_id] ; mult_pass_id = 3
23373 ; 11/12/2022
23374 ; ds =cs
23375 0000081D FE06[CD02] inc byte [multi_pass_id]
23376 00000821 E8DE1C call multi_pass ; execute install= commands
23377
23378 dolast:
23379
23380 ; [area] has the segment address for the allocated memory of sysinit, confbot.
23381 ; free the confbot area used for config.sys and sysinit itself.
23382
23383 ; Now if DOS is supposed to run high, we actually move it into high memory
23384 ; (if HiMem manager is available). For ROMDOS, we don't actually move
23385 ; anything, but just set up the ROM area for suballocation (or print
23386 ; a message if HiMem is not available).
23387 ;
23388 ; There is also this little hack for CPM style DOS calls that needs to
23389 ; be done when A20 is set...
23390
23391 ; 11/12/2022
23392 ; ds = cs
23393
23394 ; 22/10/2022
23395 ;cmp byte [cs:runhigh],0FFh; are we still waiting to be moved?
23396 ; 11/12/2022
23397 00000824 803E[6C02]FF cmp byte [runhigh],0FFh
23398 00000829 7503 jne short _@@_ ; 09/12/2022 ; no, our job is over
23399 0000082B E84802 call LoadDOSHiOrLo
23400 _@@_:
23401 ;cmp byte [cs:runhigh],0 ; are we running low
23402 ; 11/12/2022
23403 ; ds = cs
23404 0000082E 803E[6C02]00 cmp byte [runhigh],0
23405 ;je short _@@@
23406 00000833 7403 je short ConfigDone ; yes, no CPM hack needed
23407 00000835 E84C05 call CPMHack ; make ffff:d0 same as 0:c0
23408 _@@@:
23409
23410 ; we are now done with CONFIG.SYS processing
23411
23412 ConfigDone:
23413 ; 12/12/2022
23414 ; 22/10/2022
23415 ;mov byte [cs:donotshownum],1
23416 ; done with config.sys.
23417 ; do not show line number message.
23418
23419 ;mov es,[cs:area]
23420 ; 12/12/2022
23421 ; ds = cs
23422 00000838 C606[5503]01 mov byte [donotshownum],1
23423 0000083D 8E06[6803] mov es,[area]
23424
23425 00000841 B449 mov ah,49h ; DEALLOC ; free allocated memory for command.com
23426 00000843 CD21 int 21h
23427 ; DOS - 2+ - FREE MEMORY
23428 ; ES = segment address of area to be freed
23429
23430 ; 22/10/2022
23431 ;test word [cs:install_flag],2
23432 ;test word [cs:install_flag],has_installed ; sysinit_base installed?

```

```

23433 ;test byte [cs:install_flag],has_installed
23434 ; 11/12/2022
23435 ; ds = cs
23436 00000845 F606[CE02]02 test byte [install_flag],2 ; has_installed
23437 ;test byte [install_flag],has_installed
23438 0000084A 741F jz short skip_free_sysinitbase ; no.
23439
23440 ; set block from the old_area with impossible_owner_size.
23441 ; this will free the unnecessary sysinit_base that had been put in memory to
23442 ; handle install= command.
23443
23444 ; 12/12/2022
23445 ;push es ; BUGBUG 3-30-92 JeffPar: no reason to save ES
23446 ;push bx
23447
23448 ; 22/10/2022
23449 ;mov es,[cs:old_area]
23450 ;mov bx,[cs:impossible_owner_size]
23451 ; 12/12/2022
23452 ; ds = cs
23453 0000084C 8E06[5E03] mov es,[old_area]
23454 00000850 8B1E[6003] mov bx,[impossible_owner_size]
23455
23456 00000854 B44A mov ah,4Ah ; SETBLOCK
23457 00000856 CD21 int 21h
23458 ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
23459 ; ES = segment address of block to change
23460 ; BX = new size in paragraphs
23461 00000858 8CC0 mov ax,es
23462 0000085A 48 dec ax
23463 0000085B 8EC0 mov es,ax ; point to arena
23464 ;mov word [es:ARENA.OWNER],8 ; set impossible owner
23465 0000085D 26C70601000800 mov word [es:1],8
23466 ;mov word [es:ARENA.NAME],'SD' ; 4453h ; System Data
23467 00000864 26C70608005344 mov word [es:8],'SD'
23468
23469 ; 12/12/2022
23470 ;pop bx
23471 ;pop es ; BUGBUG 3-30-92 JeffPar: no reason to save ES
23472
23473 skip_free_sysinitbase:
23474 ; 22/10/2022
23475 ;cmp byte [cs:runhigh],0
23476 ; 12/12/2022
23477 ; ds = cs
23478 0000086B 803E[6C02]00 cmp byte [runhigh],0
23479 00000870 7403 je short _@@@_ ; 04/07/2023
23480
23481 00000872 E8DF03 call InstVDiskHeader ; Install VDISK header (allocates some mem from DOS)
23482
23483 ; -----
23484
23485 _@@@_:
23486 ; 12/12/2022
23487 ; ds = cs
23488 ; 22/10/2022
23489 ; 27/03/2019
23490 ;push cs
23491 ;pop ds ; point DS to sysinitseg
23492
23493 ; set up the parameters for command
23494
23495 ; ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS SYSINIT)
23496 ;;ifdef MULTI_CONFIG
23497 ; mov byte [config_cmd],0 ; set special code for query_user
23498 ; call query_user ; to issue the AUTOEXEC prompt
23499 ; jnc short process_autoexec; we should process autoexec normally
23500 ; ; !!!
23501 ; or byte [bQueryOpt],4 ; MSDOS 6.21 IO.SYS - SYSINIT:081Fh
23502 ; ; !!!
23503 ; call disable_autoexec ; no, we should disable it
23504 ;process_autoexec:
23505 ;;endif ; !!!
23506 ; call CheckQueryOpt ; MSDOS 6.21 IO.SYS - SYSINIT:0827h
23507 ; ; !!!
23508
23509 ; 22/10/2022
23510 ;mov cl,[command_line]
23511 ;mov ch,0
23512 ;inc cx
23513 ;mov si,command_line
23514 ;add si,cx
23515 ;mov byte [si],cr ; cr-terminate command line
23516
23517 ; 22/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
23518 ; (SYSINIT:0809h)
23519
23520 ;;;
23521
23522 ; 30/12/2022 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS)
23523 ; (SYSINIT:0813h)
23524 ; ds = cs
23525 ; push cs
23526 ; pop ds
23527
23528 00000875 C606[6419]00 mov byte [config_cmd],0 ; set special code for query_user
23529 0000087A E89D3D call query_user ; to issue the AUTOEXEC prompt
23530 ; 07/04/2024
23531 ;jnc short process_autoexec; we should process autoexec normally
23532
23533 ; 07/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
23534 ;;;
23535 0000087D 9C pushf
23536 0000087E F606[7F4C]01 test byte [bDisableUI],1
23537 00000883 7507 jnz short _@@@_ ; F5 clean/interactive boot option (has been) disabled
23538 00000885 803E[8E03]01 cmp byte [F5_key],1
23539 0000088A 7405 je short _@@@@_ ; F5 key pressed, bypass AUTOEXEC.BAT (clean boot)
23540 _@@@@_:
23541 0000088C 9D popf
23542 0000088D 730B jnc short process_autoexec; we should process autoexec normally
23543 0000088F EB01 jmp short bypass_autoexec
23544 _@@@@_:
23545 00000891 9D popf ; cf status at the return from 'query_user' call
23546 bypass_autoexec:
23547 ;;;
23548
23549 ; !!!
23550 00000892 800E[834C]04 or byte [bQueryOpt],4 ; MSDOS 6.21 IO.SYS - SYSINIT:081Fh
23551 ; !!!
23552 00000897 E87B3E call disable_autoexec ; no, we should disable it
23553 process_autoexec:
23554 ; !!!
23555 0000089A E8C33E call CheckQueryOpt ; MSDOS 6.21 IO.SYS - SYSINIT:0827h
23556

```



```

23557             ;mov     cl,[command_line]
23558             ; 30/12/2022
23559 0000089D BE[B94B] mov     si,command_line
23560 000008A0 8A0C     mov     cl,[si]
23561 000008A2 B500     mov     ch,0
23562 000008A4 41       inc     cx
23563             ;mov     si,command_line
23564 000008A5 01CE     add     si,cx
23565 000008A7 C6040D   mov     byte [si],cr ; 0Dh ; cr-terminate command line
23566
23567             ;;;;
23568
23569 ; 30/12/2022 - Retro DOS v4.2
23570 %if 0
23571             ;mov     si,(offset command_line+1)
23572             mov     si,command_line+1
23573             push    ds
23574             pop     es
23575             mov     di,si
23576             mov     cl,0FFh ; -1
23577 _@_loop:
23578             inc     cl ; +1
23579             lodsb
23580             stosb
23581             or      al,al
23582             jnz     short @_loop
23583             dec     di
23584             mov     al,0Dh
23585             stosb ; cr-terminate command line
23586             mov     [command_line],cl ; command line length (except CR)
23587 %endif
23588
23589 ; -----
23590
23591 ; Once we get to this point, the above code, which is below "retry"
23592 ; in memory, can be trashed (and in fact is -- see references to retry
23593 ; which follow....)
23594
23595 retry: ; PCDOS 7.1 IBMBIO.COM - SYSINIT:094Ch ; 07/04/2024
23596 000008AA BA[2B4B] mov     dx,commnd ; now pointing to file description
23597
23598 ; we are going to open the command interpreter and size it as is done in
23599 ; ldfil. the reason we must do this is that sysinit is in free memory. if
23600 ; there is not enough room for the command interpreter,exec will probably
23601 ; overlay our stack and code so when it returns with an error sysinit won't be
23602 ; here to catch it. this code is not perfect (for instance .exe command
23603 ; interpreters are possible) because it does its sizing based on the
23604 ; assumption that the file being loaded is a .com file. it is close enough to
23605 ; correctness to be usable.
23606
23607 ; first, find out where the command interpreter is going to go.
23608
23609 000008AD 52       push    dx ; save pointer to name
23610 000008AE BBFFFF   mov     bx,0FFFFh
23611 000008B1 B448     mov     ah,48h ; ALLOC
23612 000008B3 CD21     int     21h ; get biggest piece
23613 000008B5 B448     mov     ah,48h ; ALLOC
23614 000008B7 CD21     int     21h ; second time gets it
23615 000008B9 726B     jc      short memerrjx ; oooops
23616
23617 000008BB 8EC0     mov     es,ax
23618 000008BD B449     mov     ah,49h ; DEALLOC
23619 000008BF CD21     int     21h ; give it right back
23620 000008C1 89DD     mov     bp,bx
23621
23622 ; es:0 points to block,and bp is the size of the block in para.
23623
23624 ; we will now adjust the size in bp down by the size of sysinit.
23625 ; we need to do this because exec might get upset if some of the exec
23626 ; data in sysinit is overlayed during the exec.
23627
23628             ; 22/10/2022
23629             ; (MSDOS 5.0 IO.SYS SYSINIT:083Bh)
23630 000008C3 8B1E[9402] mov     bx,[MEMORY_SIZE] ; get location of end of memory
23631 000008C7 8CC8     mov     ax,cs ; get location of beginning of sysinit
23632
23633 ; Note that the "config_wrkseg" environment data is a segment in
23634 ; unallocated memory (as of the Dealloc of [area], above). This is ideal
23635 ; in one sense, because Exec is going to make a copy of it for COMMAND.COM
23636 ; anyway, and no one has responsibility for freeing "config_wrkseg". But
23637 ; we need to make sure that there's no way Exec will stomp on that data
23638 ; before it can copy it, and one way to do that is to make the available
23639 ; memory calculation even more "paranoid", by subtracting "config_wrkseg"
23640 ; from the "memory_size" segment value (which is typically A000h) instead
23641 ; of the current sysinit CS....
23642 ;
23643 ; The reason I use the term "paranoid" is because this code should have
23644 ; slid the data required by Exec up to the very top of memory, because as
23645 ; it stands, you have to have sizeof(COMMAND.COM) PLUS 64K to load just
23646 ; COMMAND.COM (64k is about what sysinit, and all the goop above sysinit,
23647 ; consumes). Now it's just a little worse (65K or more, depending on
23648 ; the size of your CONFIG.SYS, since the size of the environment workspace
23649 ; is determined by the size of CONFIG.SYS.... -JTP
23650
23651             ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21, IO.SYS)
23652             ; (SYSINIT:0858h)
23653 000008C9 8B0E[6019] mov     cx,[config_envlen]
23654 000008CD E303     jcxz     no_env ; use config_wrkseg only if there's env data
23655 000008CF A1[6219] mov     ax,[config_wrkseg]
23656
23657             ; 22/10/2022
23658             ;mov     cx,[config_envlen]
23659             ;jcxz     no_env ; use config_wrkseg only if there's env data
23660             ;mov     ax,[config_wrkseg]
23661 ;no_env:
23662             ; 22/10/2022
23663             ; (MSDOS 5.0 IO.SYS SYSINIT:0841h)
23664 no_env:
23665             ; 30/12/2022
23666             ; (MSDOS 6.21 IO.SYS SYSINIT:0861h)
23667 000008D2 29C3     sub     bx,ax ; bx is size of sysinit in para
23668 000008D4 83C311   add     bx,11h ; add the sysinit php
23669 000008D7 29DD     sub     bp,bx ; sub sysinit size from amount of free memory
23670 000008D9 724B     jc      short memerrjx ; if there isn't even this much memory, give up
23671
23672             ;mov     ax,(OPEN<<8) ; open the file being execed
23673 000008DB B8003D   mov     ax,3D00h
23674 000008DE F9       stc ; in case of int 24
23675 000008DF CD21     int     21h
23676 000008E1 7271     jc      short comerr ; oooops
23677             ; DOS - 2+ - OPEN DISK FILE WITH HANDLE
23678             ; DS:DX -> ASCIZ filename
23679             ; AL = access mode
23680             ; 0 - read

```

```

23681 ; 22/10/2022
23682 ; (MSDOS 5.0 IO.SYS SYSINIT:0852h)
23683 000008E3 89C3 mov bx,ax ; handle in bx
23684
23685 ; If the standard command interpreter is being used, verify it is correct
23686
23687 ; 30/12/2022 - Retro DOS v4.2
23688 ; (MSDOS 6.21 IO.SYS, SYSINIT:0874h)
23689 000008E5 803E[284B]00 cmp byte [newcmd],0 ; was a new shell selected?
23690 000008EA 7518 jne short skip_validation ; yes
23691 ; 07/04/2024 - Retro DOS v5.0
23692 ; (PCDOS 7.1 IBMBIO.COM, SYSINIT:098Eh)
23693 000008EC BA[A608] mov dx,retry-4 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0948h
23694 000008EF B90400 mov cx,4 ;
23695 000008F2 B43F mov ah,READ ;
23696 000008F4 CD21 int 21h ;
23697 000008F6 803E[A608]E9 cmp byte [retry-4],0E9h
23698 000008FB 7557 jne short comerr
23699 ; 20/04/2019 - Retro DOS v4.0
23700 ; 30/12/2022
23701 ; cmp byte [retry-1],64h ; MSDOS 6.21 IO.SYS - SYSINIT:088Ch
23702 ; ; .. COMMAND.COM Version 6.20 (14h&0Fh)
23703 ; 07/04/2024 - Retro DOS v5.0
23704 ; ; cmp byte [retry-1],66h ; .. COMMAND.COM Version 6.22 (16h&0Fh)
23705 ; ; cmp byte [retry-1],7Ah ; PCDOS 7.1 IBMBIO.COM - SYSINIT:099Fh
23706 ; ; ; .. COMMAND.COM Version 7.10 (0Ah&0Fh)
23707 000008FD 803E[A908]7A cmp byte [retry-1],((MAJOR_VERSION&0Fh)<<4)|(MINOR_VERSION&0Fh)
23708 00000902 7550 jne short comerr ;
23709
23710 ; 22/10/2022
23711 ; cmp byte [newcmd],0 ; was a new shell selected?
23712 ; jne short skip_validation ; yes
23713 ; mov dx,retry-4 ;
23714 ; mov cx,4 ;
23715 ; mov ah,READ ;
23716 ; int 21h ;
23717 ; cmp byte [retry-4],0E9h
23718 ; jne short comerr
23719 ; ; 20/04/2019 - Retro DOS v4.0
23720 ; ; cmp byte [retry-1],64h ; MSDOS 6.21 IO.SYS - SYSINIT:088Ch
23721 ; ; ; cmp byte [retry-1],((MAJOR_VERSION&0Fh)<<4)|(MINOR_VERSION&0Fh)
23722 ; ; jne short comerr ;
23723
23724 ; skip_validation:
23725 ; 22/10/2022
23726 ; (MSDOS 5.0 IO.SYS SYSINIT:0854h)
23727 skip_validation:
23728 ; 30/12/2022
23729 ; (MSDOS 6.21 IO.SYS SYSINIT:0893h)
23730 00000904 31C9 xor cx,cx
23731 00000906 31D2 xor dx,dx
23732 ; mov ax,(LSEEK<<8)|2
23733 00000908 B80242 mov ax,4202h
23734 0000090B F9 stc ; in case of int 24
23735 0000090C CD21 int 21h ; get file size in dx:ax
23736 0000090E 7244 jc short comerr
23737
23738 00000910 83C00F add ax,15 ; convert size in dx:ax to para in ax
23739 00000913 83D200 adc dx,0 ; round up size for conversion to para
23740 00000916 E88104 call off_to_para
23741 00000919 B10C mov cl,12
23742 0000091B D3E2 shl dx,cl ; low nibble of dx to high nibble
23743 0000091D 09D0 or ax,dx ; ax is now # of para for file
23744 0000091F 83C010 add ax,10h ; 100h byte php
23745 00000922 39E8 cmp ax,bp ; will command fit in available mem?
23746 00000924 7208 jnb short okld ; jump if yes.
23747
23748 ; 30/12/2022
23749 %if 0
23750 ; 22/10/2022
23751 memerrjx: ; (MSDOS 5.0 IO.SYS SYSINIT:0876h)
23752 ; jmp memerr ; (MSDOS 5.0 IO.SYS SYSINIT:34D5h)
23753 ; 02/11/2022
23754 ; jmp mem_err
23755 ; 11/12/2022
23756 ; ds = cs
23757 jmp mem_err2
23758 %endif
23759 ; 30/12/2022
23760 ; (MSDOS 6.21, IO.SYS, SYSINIT:08B5h)
23761 memerrjx:
23762 00000926 BA[4751] mov dx,badmem ; "Configuration too large for memory"
23763 00000929 E82641 call print
23764 0000092C EB3A jmp short continue
23765
23766 okld:
23767 0000092E B43E mov ah,3Eh ; CLOSE
23768 00000930 CD21 int 21h ; close file
23769
23770 ; 22/10/2022
23771 00000932 5A pop dx ; (MSDOS 5.0 IO.SYS SYSINIT:087Dh)
23772
23773 ; 24/03/2019
23774
23775 00000933 0E push cs ; point es to sysinitseg
23776 00000934 07 pop es
23777 00000935 BB[BF02] mov bx,COMEXE ; point to exec block
23778 ; 22/10/2022
23779 ; pop dx ; recover pointer to name
23780
23781 ; ; ifdef MULTI_CONFIG
23782
23783 ; If there's any environment data in "config_wrkseg", pass it to shell;
23784 ; there will be data if there were any valid SET commands and/or if a menu
23785 ; selection was made (in which case the CONFIG environment variable will be
23786 ; set to that selection).
23787
23788 ; 23/10/2022
23789 ; mov cx,[config_envlen]
23790 ; jcxz no_envdata
23791 ; mov cx,[config_wrkseg]
23792 ; no_envdata:
23793 ; ; mov [bx+EXEC0.ENVIRON],cx
23794 ; ; mov [bx],cx
23795
23796 ; ; endif ;MULTI_CONFIG
23797
23798 ; 30/12/2022 - Retro DOS v4.2
23799 ; (MSDOS 6.21 IO.SYS SYSINIT:08C7h)
23800 00000938 8B0E[6019] mov cx,[config_envlen]
23801 0000093C E304 jcxz no_envdata
23802 0000093E 8B0E[6219] mov cx,[config_wrkseg]
23803 no_envdata:
23804 ; mov [bx+EXEC0.ENVIRON],cx

```

```

23805 00000942 890F      mov     [bx],cx
23806
23807      ; 23/10/2022
23808      ; (MSDOS 5.0 IO.SYS SYSINIT:0883h)
23809
23810      ;mov     [bx+EXEC0.COM_LINE+2],cs ; set segments
23811 00000944 8C4F04    mov     [bx+4],cs
23812      ;mov     [bx+EXEC0.5C_FCB+2],cs
23813 00000947 8C4F08    mov     [bx+8],cs
23814      ;mov     [bx+EXEC0.6C_FCB+2],cs
23815 0000094A 8C4F0C    mov     [bx+12],cs
23816
23817      ;mov     ax,(EXEC<<8) + 0
23818      ; 23/10/2022
23819      ;xor     ax,ax
23820      ;mov     ah,4Bh
23821      ; 04/07/2023
23822      ;mov     ax,4B00h
23823 0000094D B8004B    mov     ax,(EXEC<<8)
23824
23825 00000950 F9          stc
23826 00000951 CD21      int     21h      ; in case of int 24
23827      ; go start up command
23828      ; DOS - 2+ - LOAD OR EXECUTE (EXEC)
23829      ; DS:DX -> ASCIZ filename
23830      ; ES:BX -> parameter block
23831      ; AL = subfunc: load & execute program
23832      ;push    cs
23833      ;pop     ds
23834
23835      ; 13/04/2024
23836 00000953 52          push   dx      ; push to balance fall-through pop
23837
23838      ; note fall through if exec returns (an error)
23839      comerr:
23840      ; 23/10/2022
23841      ;;ifdef  MULTI_CONFIG
23842      ;cmp     byte [commnd4],0
23843      ;je      short comerr2 ; all defaults exhausted, print err msg
23844      ;cmp     byte [newcmd],0
23845      ;je      short continue ; don't print err msg for defaults just yet
23846      ;comerr2:
23847      ;;endif
23848
23849      ; 30/12/2022 - Retro DOS v4.2
23850      ;push    cs
23851      ;pop     ds
23852      ; 07/04/2024
23853      ; ds = cs
23854
23855 00000954 803E[9C4B]00    cmp     byte [commnd4],0
23856 00000959 7407          je      short comerr2 ; all defaults exhausted, print err msg
23857 0000095B 803E[284B]00    cmp     byte [newcmd],0
23858 00000960 7406          je      short continue ; don't print err msg for defaults just yet
23859      comerr2:
23860      ; 07/04/2024
23861      ;push    dx ; 30/12/2022
23862
23863      ; 23/10/2022
23864 00000962 BA[C350]    mov     dx,badcom ; want to print command error
23865 00000965 E8BE40    call    badfil
23866
23867      ; 07/04/2024
23868      ;pop     dx ; 30/12/2022
23869      continue:
23870      ; 13/04/2024
23871      ; 23/10/2022
23872 00000968 5A          pop     dx
23873
23874      ; 30/12/2022
23875      %if 0
23876
23877      ;;ifndef MULTI_CONFIG
23878      ;jmp     stall
23879      ; 24/10/2022
23880      stall:      ; (MSDOS 5.0 IO.SYS, SYSINIT:0899h)
23881      ;jmp     short stall
23882      ;;else
23883
23884      %endif
23885
23886      ; 30/12/2022 (MSDOS 6.21 SYSINIT, Retro DOS v4.2)
23887      ;%if 1
23888      ; 23/10/2022 (MSDOS 5.0 SYSINIT, RetroDOS v4.0)
23889      ;%if 0
23890      mov     ah,GET_DEFAULT_DRIVE ; 19h
23891      int     21h
23892      add     al,'A'
23893      mov     dl,al      ; DL == default drive letter
23894      mov     si,commnd2
23895      cmp     byte [newcmd],0 ; if a SHELL= was given
23896      jne     short do_def2 ; then try the 2nd alternate;
23897      mov     byte [si],0 ; otherwise, the default SHELL= was tried,
23898      jmp     short do_def3 ; which is the same as our 2nd alt, so skip it
23899      do_def2:
23900      cmp     byte [si],0 ; has 2nd alternate been tried?
23901      jne     short do_alt ; no
23902      do_def3:
23903      mov     si,commnd3
23904      cmp     byte [si],0 ; has 3rd alternate been tried?
23905      jne     short do_alt ; no
23906      mov     si,commnd4
23907      cmp     byte [si],0 ; has 4th alternate been tried?
23908      jne     short do_alt ; no
23909      push    dx
23910      mov     dx,badcomprmt
23911      call    print
23912      pop     dx      ; recover default drive letter in DL
23913      request_input:
23914      mov     ah,STD_CON_OUTPUT
23915      int     21h
23916      push    dx
23917      mov     dl,'>'
23918      int     21h
23919      mov     bl,[tmplate+1] ; [tmplate+1] = 12
23920      mov     bh,0
23921      mov     byte [commnd+bx],0Dh
23922      mov     dx,tmplate
23923      mov     ah,STD_CON_STRING_INPUT
23924      int     21h      ; read a line of input
23925      mov     dx,crlfm
23926      call    print
23927      pop     dx
23928      mov     bl,[tmplate+1] ;

```

```

23929 000009C3 08DB          or     bl,bl          ; was anything typed?
23930 000009C5 74D6          jz     short request_input ;
23931 000009C7 C606[284B]01        mov    byte [newcmd],1 ; disable validation for user-specified binaries
23932 000009CC C687[2B4B]00        mov    byte [commnd+bx],0 ; NULL-terminate it before execing it
23933 000009D1 C706[B94B]000D      mov    word [command_line],0D00h
23934 000009D7 EB35          jmp     short do_exec ;
23935
23936 000009D9 1E          do_alt:
23937 000009DA 07          push   ds
23938 000009DB C606[284B]00        pop     es
23939 000009E0 BF[2B4B]      mov    byte [newcmd],0 ; force validation for alternate binaries
23940                                mov    di,commnd ;
23941 000009E3 AC          do_alt1:
23942 000009E4 C644FF00        lodsb  byte [si-1],0 ; copy the alternate, zapping it as we go,
23943 000009E8 AA          mov     byte [si-1],0 ; so that we know it's been tried
23944 000009E9 08C0        stosb  ;
23945 000009EB 75F6        or     al,al ;
23946 000009ED BF[B94B]      jnz     short do_alt1 ;
23947 000009F0 807C023A      mov    di,command_line
23948 000009F4 7503        cmp    byte [si+2],':'
23949 000009F6 885401      jne     short do_alt2 ;
23950                                mov    [si+1],dl ; stuff default drive into alt. command line
23951 000009F9 AC          do_alt2:
23952 000009FA AA          lodsb  ;
23953 000009FB 08C0        stosb  ;
23954 000009FD 75FA        or     al,al ;
23955 000009FF C645FF0D      jnz     short do_alt2 ;
23956                                mov    byte [di-1],cr ;
23957
23958 ;; Last but not least, see if we need to call disable_autoexec
23959
23960 ; MSDOS 6.0 (SYSINIT1.ASM)
23961 ; cmp [command_line-1],0
23962 ; jne short do_exec ;
23963 ; mov [command_line-1], '/'
23964 ; call disable_autoexec ;
23965
23966 ; MSDOS 6.21 IO.SYS (SYSINIT:0994h)
23967 00000A03 C606[794C]00        mov    byte [dae_flag],0 ; 24/03/2019 - Retro DOS v4.0
23968 00000A08 E80A3D      call   disable_autoexec
23969 00000A0B E8523D      call   checkQueryOpt ; 24/03/2019 - Retro DOS v4.0
23970 00000A0E E999FE      do_exec:
23971                                jmp     retry ;
23972
23973 ;;endif ;MULTI_CONFIG
23974
23975 ;%endif ; 23/10/2022 (MSDOS 5.0 SYSINIT)
23976 ;%endif ; 30/12/2022 (MSDOS 6.21 SYSINIT)
23977
23978 ; 24/03/2019 - Retro DOS v4.0
23979
23980 ; -----
23981 ; procedure : AllocFreeMem
23982 ;
23983 ; Allocate Max memory from DOS to find out where to load DOS.
23984 ; DOS is at temporary location when this call is being made
23985 ;
23986 ; Inputs : None
23987 ; Outputs: The biggest chunk of memory is allocated (all mem at init time)
23988 ; [area] & [memhi] set to the para value of the start of the
23989 ; free memory.
23990 ;
23991 ; Uses : AX, BX
23992 ; -----
23993
23994 ; 30/12/2022 - Retro DOS v4.2
23995 ; (MSDOS 6.21 IO.SYS, SYSINIT:09A2h)
23996
23997 ; 08/04/2024 - Retro DOS v5.0
23998 ; (PCDOS 7.1 IBMBIO.COM, SYSINIT:0AB5h)
23999
24000 ; 23/10/2022
24001 AllocFreeMem:
24002 00000A11 BBFFFF      mov    bx,0FFFFh
24003 00000A14 B448      mov    ah,48h ; ALLOC
24004 00000A16 CD21      int     21h ; first time fails
24005 00000A18 B448      mov    ah,48h ; ALLOC
24006 00000A1A CD21      int     21h ; second time gets it
24007 ; 11/12/2022
24008 ; ds = cs
24009 ; mov [cs:area],ax
24010 ; mov [cs:memhi],ax ; memhi:memlo now points to
24011 00000A1C A3[6803]      mov    [area],ax
24012 00000A1F A3[6403]      mov    [memhi],ax ; memhi:memlo now points to
24013 00000A22 C3          retn ; start of free memory
24014
24015 ; include msbio.c16
24016 ; -----
24017 DOSLOMSG:
24018 00000A23 484D41206E6F742061- db 'HMA not available: Loading DOS low',0Dh,0Ah,'$'
24019 00000A2C 7661696C61626C653A-
24020 00000A35 204C6F6164696E6720-
24021 00000A3E 444F53206C6F770D0A-
24022 00000A47 24
24023
24024 FEmsg:
24025 db 'Fatal Error: Cannot allocate Memory for DOS',0Dh,0Ah,'$'
24026
24027 ; -----
24028 ;
24029 ; procedure : LoadDOSHiOrLo
24030 ;
24031 ; Tries to move DOS into HMA. If it fails then loads
24032 ; DOS into Low memory. For ROMDOS, nothing is actually
24033 ; moved; this just tries to allocate the HMA, and prints
24034 ; a message if this is not possible.
24035 ; -----
24036
24037 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24038 LoadDOSHiOrLo:
24039 ; 27/03/2019 - Retro DOS v4.0
24040 ; ds = cs
24041 00000A76 E81F00      call   TryToMovDOSHi ; Try moving it into HMA (M024)
24042 ; jc short LdngLo ; If that don't work...
24043 ; retn
24044 ; 18/12/2022
24045 jnc     short LoadDoshi_ok
24046 LdngLo:
24047 ; 23/10/2022

```

```

24044         ;push cs
24045         ;pop ds
24046         ; 11/12/2022
24047         ; ds = cs
24048 00000A7B B409      mov ah,9
24049 00000A7D BA[230A] mov dx,DOSLOMSG      ; inform user that we are
24050 00000A80 CD21      int 21h      ; loading low
24051
24052 ;ifndef ROMDOS
24053 ; actually move the dos, and reinitialize it.
24054
24055 00000A82 BB0100      mov bx,1      ; M012
24056                                     ; use int 21 alloc for mem
24057 00000A85 E83F00      call MovDOSLo
24058         ; 11/12/2022
24059         ; ds = cs
24060         ;mov es,[cs:CURRENT_DOS_LOCATION] ; give dos its temporary loc.
24061         ; 23/10/2022
24062 00000A88 8E06[7302] mov es,[CURRENT_DOS_LOCATION]
24063         ;mov es,[cs:FINAL_DOS_LOCATION] ; 24/03/2019 - Retro DOS v4.0
24064         ;mov es,[FINAL_DOS_LOCATION] ; 27/03/2019
24065 00000A8C 31C0      xor ax,ax      ; ax = 00 ---> install stub
24066         ; 11/12/2022
24067         ; ds = cs
24068         ;call far [cs:dosegreinit] ; call dos segreinit
24069 00000A8E FF1E[7D02] call far [dosegreinit] ; 27/03/2019
24070
24071 ;endif ; ROMDOS
24072         ; 23/10/2022
24073         ;mov byte [cs:runhigh],0      ; mark that we are running lo
24074         ; 11/12/2022
24075         ; ds = cs
24076 00000A92 C606[6C02]00 mov byte [runhigh],0 ; 27/03/2019
24077 LoadDoshi_ok:      ; 18/12/2022
24078 00000A97 C3        retn
24079
24080 ; -----
24081 ;
24082 ; procedure : TryToMovDOSHi
24083 ;
24084 ; This tries to move DOS into HMA.
24085 ; Returns CY if it failed.
24086 ; If it succeeds returns with carry cleared.
24087 ;
24088 ; For ROMDOS, dosegreinit must be called again to allow
24089 ; the A20 switching code in the low mem stub to be installed.
24090 ;
24091 ; -----
24092
24093         ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24094         ; (MSDOS 5.0 IO.SYS - SYSINIT:092Ah)
24095 TryToMovDOSHi:
24096         ; 11/12/2022
24097         ; 27/03/2019 - Retro DOS v4.0
24098         ; ds = cs
24099 00000A98 E81300      call MovDOSHi
24100 00000A9B 7210      jc short ttldhx
24101
24102 ;ifndef ROMDOS
24103         ; 23/10/2022
24104         ;mov es,[cs:CURRENT_DOS_LOCATION] ; give dos its temporary loc.
24105         ;mov es,[cs:FINAL_DOS_LOCATION] ; 24/03/2019 - Retro DOS v4.0
24106         ; 11/12/2022
24107         ; ds = cs
24108 00000A9D 8E06[7302] mov es,[CURRENT_DOS_LOCATION]
24109 ;else
24110 ;
24111 ;
24112 ;endif ; ROMDOS
24113
24114         ; 11/12/2022
24115         ; ds = cs
24116 00000AA1 31C0      xor ax,ax      ; ax = 00 ---> install stub
24117 00000AA3 FF1E[7D02] call far [cs:dosegreinit] ; call dos segreinit
24118         ;call far [dosegreinit]
24119 00000AA7 C606[6C02]01 mov byte [cs:runhigh],1
24120 00000AAC F8        mov byte [runhigh],1
24121         cll
24122 00000AAD C3        ttldhx:
24123         retn
24124
24125 ; -----
24126 ;
24127 ; procedure : MovDOSHi
24128 ;
24129 ; Tries to allocate HMA and Move DOS/BIOS code into HMA
24130 ; For ROMDOS, the code is not actually moved, but the
24131 ; HMA is allocated and prepared for sub-allocation.
24132 ;
24133 ; Returns : CY if it failed
24134 ;
24135 ; -----
24136
24137         ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24138 MovDOSHi:
24139         ; 14/05/2019
24140         ; 27/03/2019 - Retro DOS v4.0
24141         ; ds = cs
24142 00000AAE E8D600      call AllocHMA
24143 00000AB1 7213      jc short mdhx      ; did we get HMA?
24144 00000AB3 B8FFFF      mov ax,0FFFFh      ; yes, HMA seg = 0ffffh
24145         mov es,ax
24146
24147 ;ifndef ROMDOS
24148 ; actually move the BIOS and DOS
24149
24150         ; NOTE: Retro DOS v4.0 does not move BIOS (IO.SYS) to HMA
24151         ; 24/03/2019
24152         ; 23/10/2022
24153 00000AB8 E83200      call MovBIOS      ; First move BIOS into HMA
24154
24155         ; ES:DI points to free HMA after BIOS
24156
24157         ; 14/05/2019
24158         ; 24/03/2019 - Retro DOS v4.0
24159         ;xor di,di
24160
24161         ; 23/10/2022
24162         ;mov cx,[cs:hi_doscod_size] ; pass the code size of DOS
24163         ; 11/12/2022
24164         ; ds = cs
24165 00000ABB 8B0E[8302] mov cx,[hi_doscod_size] ; when it is in HMA
24166 00000ABF E81100      call MovDOS      ; and move it
24167

```

```

24168             ; ES:DI points to free HMA after DOS
24169 ;else
24170 ;             ; allocate space at beginning of HMA to allow for CPMHack
24171 ;
24172 ;     mov     di,0E0h                ; room for 5 bytes at ffff:d0
24173 ;
24174 ;endif ; ROMDOS
24175
24176 00000AC2 E87602    call     SaveFreeHMAPtr        ; Save the Free HMA ptr
24177 00000AC5 F8       cld
24178 mdhx:
24179 00000AC6 C3       retn
24180
24181 ; -----
24182 ;
24183 ; procedure : MovDOSLo
24184 ;
24185 ;             Allocates memory from DOS and moves BIOS/DOS code into it
24186 ; -----
24187 ;
24188 ;
24189 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24190
24191 ;ifndef ROMDOS
24192
24193 MovDOSLo:
24194 ; 14/05/2019
24195 ; 27/03/2019 - Retro DOS v4.0
24196 ; ds = cs
24197 00000AC7 E84500    call     AllocMemForDOS            ; incestuosly!!!
24198 ;
24199 ; 23/10/2022
24200 ; 14/05/2019
24201 ;inc     ax ; skip MCB
24202
24203 00000ACA 8EC0      mov     es,ax                ; pass the segment to MovBIOS
24204 ; NOTE: Retro DOS v4.0 does not move BIOS (IO.SYS) to HMA
24205 ; 24/03/2019
24206 ;
24207 ; 23/10/2022
24208 00000ACC E81E00    call     MovBIOS
24209
24210 ;----- ES:DI points memory immediately after BIOS
24211 ; 14/05/2019
24212 ; NOTE:
24213 ;     Order of (RETRO) DOS kernel sections at memory:
24214 ;     BIOSDATA+BIOSCODE+BIOSDATAINIT+DOSDATA+DOSCODE(LOW)
24215 ;
24216 ; 24/03/2019 - Retro DOS v4.0
24217 ;xor     di,di
24218
24219 ; 23/10/2022
24220 ;mov     cx,[cs:lo_doscod_size]      ; DOS code size when loaded
24221 ; 11/12/2022
24222 ; ds = cs
24223 00000ACF 8B0E[8102] mov     cx,[lo_doscod_size]      ; low
24224 ;call    MovDOS
24225 ;retn
24226 ; 11/12/2022
24227 ;jmp     short MovDOS
24228
24229 ;endif ; ROMDOS
24230
24231 ; 11/12/2022
24232
24233 ; -----
24234 ;
24235 ; procedure : MovDOS
24236 ;
24237 ;             Moves DOS code into requested area
24238 ;
24239 ; In : ES:DI - pointer to memory where DOS is to be moved
24240 ; CX       - size of DOS code to be moved
24241 ;
24242 ; Out : ES:DI - pointer to memory immediately after DOS
24243 ; -----
24244 ;
24245 ; 11/12/2022
24246 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24247
24248 ;ifndef ROMDOS
24249
24250 MovDOS:
24251 ; 14/05/2019
24252 ; 27/03/2019 - Retro DOS v4.0
24253 ;
24254 ; 11/12/2022
24255 ; ds = cs
24256 ;
24257 ; 23/10/2022
24258 ;push    ds ; *//
24259
24260 00000AD3 06        push    es
24261 00000AD4 57        push    di
24262
24263 ; 11/12/2022
24264 00000AD5 1E        push    ds ; *// ; 11/12/202
24265 ;
24266 ; 29/04/2019
24267 00000AD6 C536[7102] lds     si,[dosinit] ; 11/12/2022
24268 ; 23/10/2022
24269 ; lds     si,[cs:dosinit]
24270 ; 03/09/2023
24271 mov     ax,si
24272
24273 00000ADA 89F0      mov     ax,si
24274
24275 00000ADC F3A4      rep     movsb
24276
24277 00000ADE 1F        pop     ds ; *// ; 11/12/2022
24278
24279 00000ADF 5B        pop     bx                ; get back offset into which
24280 ;                                     ; DOS was moved
24281 ; 03/09/2023
24282 ;mov     ax,[cs:dosinit]            ; get the offset at which DOS
24283 ;                                     ; wants to run
24284 ; 03/09/2023
24285 ;mov     ax,[dosinit]
24286 ; ax = [dosinit]
24287
24288 00000AE0 29D8      sub     ax,bx
24289 00000AE2 E8B502    call    off_to_para
24290 00000AE5 5B        pop     bx                ; get the segment at which
24291 ;                                     ; we moved DOS into

```



```

24292 00000AE6 29C3      sub    bx,ax                ; Adjust segment
24293
24294      ; 11/12/2022
24295      ; 23/10/2022
24296      ;mov    [cs:CURRENT_DOS_LOCATION],bx ; and save it
24297      ;;mov    [cs:FINAL_DOS_LOCATION],bx
24298      ; 11/12/2022
24299 00000AE8 891E[7302] mov    [CURRENT_DOS_LOCATION],bx
24300
24301      ; 27/03/2019
24302      ;pop     ds ; *//
24303      ; ds = cs
24304      ;mov     [FINAL_DOS_LOCATION],bx
24305
24306 00000AEC C3          retn
24307
24308 ;endif ;ROMDOS
24309
24310 ; NOTE: Retro DOS v4.0 does not move BIOS (IO.SYS) to HMA
24311 ; 24/03/2019
24312 ; -----
24313 ;
24314 ; procedure : MovBIOS
24315 ;
24316 ;           Moves BIOS code into requested segment
24317 ;
24318 ; In : ES - segment to which BIOS is to be moved
24319 ;       ( it moves always into offset BCode_Start)
24320 ;
24321 ; Out : ES:DI - pointer to memory immediately after BIOS
24322 ; -----
24323 ;
24324 ;
24325 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24326 ; 08/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
24327
24328 ;ifndef ROMDOS
24329
24330 MovBIOS: ; proc    near
24331 ; 11/12/2022
24332 00000AED 1E      push    ds ; ds = cs
24333 ;
24334 ; 23/10/2022
24335 ;mov     ds,[cs:temp_bcode_seg] ; current BIOS code seg
24336 ; 17/09/2023 ; 08/04/2024
24337 00000AEE 8E1E[8902] mov     ds,[temp_bcode_seg]
24338 ;mov     si,BCODE_START ; mov si,30h
24339 ; 09/12/2022
24340 00000AF2 BE[3000] mov     si,BCODESTART ; 30h
24341 00000AF5 89F7      mov     di,si
24342 ;mov     cx,1E00h ; PCDOS 7.1 IBMBIO.COM ; 08/04/2024
24343 00000AF7 B9701D   mov     cx,BCODE_END ; mov cx,1A60h
24344 00000AFA 29F1      sub     cx,si ; size of BIOS
24345 00000AFC D1E9      shr     cx,1 ; Both the labels are para
24346 ; aligned
24347 00000AFE F3A5      rep     movsw
24348
24349 ; 11/12/2022
24350 00000B00 1F      pop     ds ; ds = cs
24351 ;
24352 00000B01 06      push    es
24353 00000B02 57      push    di ; save end of BIOS
24354 00000B03 8CC0      mov     ax,es
24355 ;
24356 ; 11/12/2022
24357 ;mov     [cs:BCodeSeg],ax ; save it for later use
24358 ;;call   dword ptr cs:_seg_reinit_ptr
24359 ;call   far [cs:seg_reinit_ptr] ; far call to seg_reinit (M022)
24360 ; ds = cs
24361 00000B05 A3[8A03] mov     [BCodeSeg],ax
24362 00000B08 FF1E[8702] call   far [seg_reinit_ptr]
24363 ;
24364 00000B0C 5F      pop     di
24365 00000B0D 07      pop     es ; get back end of BIOS
24366 00000B0E C3      retn
24367
24368 ;MovBIOS endp
24369
24370 ;endif ; ROMDOS
24371
24372 ; 11/12/2022
24373 %if 0
24374 ;
24375 ; 24/03/2019
24376 ; -----
24377 ;
24378 ; procedure : MovDOS
24379 ;
24380 ;           Moves DOS code into requested area
24381 ;
24382 ; In : ES:DI - pointer to memory where DOS is to be moved
24383 ;       CX - size of DOS code to be moved
24384 ;
24385 ; Out : ES:DI - pointer to memory immediately after DOS
24386 ; -----
24387 ;
24388 ;
24389 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24390
24391 ;ifndef ROMDOS
24392
24393 MovDOS:
24394 ; 14/05/2019
24395 ; 27/03/2019 - Retro DOS v4.0
24396
24397 ; 11/12/2022
24398 ; ds = cs
24399
24400 ; 23/10/2022
24401 ;push     ds ; *//
24402
24403 push     es
24404 push     di
24405
24406 ; 11/12/2022
24407 push     ds ; *// ; 11/12/202
24408
24409 ; 29/04/2019
24410 lds     si,[dosinit] ; 11/12/2022
24411 ; 23/10/2022
24412 ;lds     si,[cs:dosinit]
24413 ; 03/09/2023
24414 mov     ax,si
24415

```

```

24416 rep movsb
24417
24418 pop ds ; *// ; 11/12/2022
24419
24420 pop bx ; get back offset into which
24421 ; DOS was moved
24422
24423 ;mov ax,[dosinit] ; 03/09/2023
24424 ;;mov ax,[cs:dosinit] ; get the offset at which DOS
24425 ; wants to run
24426 sub ax,bx
24427 call off_to_para
24428 pop bx ; get the segment at which
24429 ; we moved DOS into
24430 sub bx,ax ; Adjust segment
24431
24432 ; 11/12/2022
24433 ; 23/10/2022
24434 ;mov [cs:CURRENT_DOS_LOCATION],bx ; and save it
24435 ;;mov [cs:FINAL_DOS_LOCATION],bx
24436 ; 11/12/2022
24437 mov [CURRENT_DOS_LOCATION],bx
24438
24439 ; 27/03/2019
24440 ;pop ds ; *//
24441 ; ds = cs
24442 ;mov [FINAL_DOS_LOCATION],bx
24443
24444 retn
24445
24446 ;endif ;ROMDOS
24447
24448 %endif
24449
24450 ; -----
24451 ;
24452 ; procedure : AllocMemForDOS
24453 ;
24454 ; Allocate memory for DOS/BIOS code from DOS !!!
24455 ;
24456 ; Out : AX - seg of allocated memoryblock
24457 ; -----
24458 ;
24459 ;
24460 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24461 ; 08/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
24462
24463 ;ifndef ROMDOS
24464
24465 AllocMemForDOS:
24466 ; 11/12/2022
24467 ; 14/05/2019
24468 ; 27/03/2019 - Retro DOS v4.0
24469 ; ds = cs
24470 ;mov ax,BCode_end
24471 ;sub ax,BCode_start ; BIOS code size
24472 ; 23/10/2022
24473 00000B0F B8701D mov ax,BCODE_END ; 1A60h ; 1A70h for MSDOS 6.21
24474 ; 30/12/2022
24475 ;mov ax,1E00h ; PCDOS 7.1 IBMBIO.COM ; 08/04/2024
24476 ;sub ax,BCODE_START ; 30h
24477 ; 09/12/2022
24478 00000B12 2D[3000] sub ax,BCODESTART ; sub ax,30h ; 08/04/2024
24479 ; 24/03/2019 - Retro DOS v4.0
24480 ; 02/11/2022
24481 ;add ax,[cs:lo_doscod_size]; DOS code size
24482 ; 11/12/2022
24483 ; ds = cs
24484 00000B15 0306[8102] add ax,[lo_doscod_size]
24485 00000B19 83C00F add ax,15
24486 00000B1C E87B02 call off_to_para ; convert to para
24487 ; 23/10/2022
24488 ; 14/05/2019
24489 ;inc ax ; + 1 paragraph for MCB
24490 00000B1F 09DB or bx,bx ; M012
24491 00000B21 89C3 mov bx,ax ; can we use int 21 for alloc
24492 00000B23 741A jz short update_arena ; M012
24493 00000B25 B448 mov ah,48h ; request DOS
24494 00000B27 CD21 int 21h
24495 00000B29 7250 jc short FatalErr ; IF ERR WE ARE HOSED
24496 ; 23/10/2022
24497 ; 24/03/2019 - Retro DOS v4.0 (ORG 0)
24498 00000B2B 83E803 sub ax,3 ; Take care ORG 30h of
24499 ; BIOS code
24500 00000B2E 8EC0 mov es,ax
24501 ;mov word [es:20h+ARENA.OWNER],08h ; mark it as system
24502 ;mov word [es:20h+ARENA.NAME],'SC' ; code area
24503 ; 14/05/2019
24504 ;mov word [es:ARENA.OWNER],08h ; mark it as system
24505 ;mov word [es:ARENA.NAME],'SC' ; code area
24506 ; 08/04/2024 (PCDOS 7.1 IBMBIO.COM)
24507 ; 23/10/2022
24508 00000B30 26C70621000800 mov word [es:20h+1],08h ; mark it as system
24509 00000B37 26C70628005343 mov word [es:20h+8],'SC' ; 4353h ; code area
24510
24511 00000B3E C3 retn
24512
24513 ; BUGBUG -- 5 Aug 92 -- chuckst -- Allocating space for DOS
24514 ; using DOS itself causes an arena to be generated.
24515 ; Unfortunately, certain programs (like PROTMAN$)
24516 ; assume that the device drivers are loaded into
24517 ; the first arena. For this reason, MagicDrv's
24518 ; main device driver header arena is manually
24519 ; truncated from the arena chain, and the space
24520 ; for DOS is allocated using the following
24521 ; simple code, which also assumes that the
24522 ; first arena is the free one where DOS's low
24523 ; stub will go.
24524 ;
24525 ; M012 : BEGIN
24526
24527 ; 23/10/2022
24528 update_arena:
24529 00000B3F 1E push ds ; ds = cs
24530 00000B40 57 push di
24531 00000B41 51 push cx
24532 00000B42 52 push dx
24533 ; 23/10/2022
24534 ;lds di,[cs:DOSINFO] ; get ptr to DOS var
24535 ; 11/12/2022
24536 ; ds = cs
24537 00000B43 C53E[6D02] lds di,[DOSINFO] ; 27/03/2019
24538 00000B47 4F dec di
24539 00000B48 4F dec di ; Arena head is immediately

```



```

24540                                     ; before sysvar
24541 00000B49 8E05                     mov     es,[di]                     ; es = arena head
24542                                     ;mov    cx,[es:ARENA.SIZE]           ; cx = total low mem size
24543 00000B4B 268B0E0300              mov     cx,[es:3]
24544 00000B50 39D9                     cmp     cx,bx                     ; is it sufficient ?
24545 00000B52 7227                     jb      short FatalErr           ; no, fatal error
24546
24547                                     ;mov    dl,[es:ARENA.SIGNATURE]
24548 00000B54 268A160000              mov     dl,[es:0]
24549 00000B59 8CC0                     mov     ax,es
24550 00000B5B 01D8                     add     ax,bx                     ; ax = new arena head
24551 00000B5D 8905                     mov     [di],ax                  ; store it in DOS data area
24552 00000B5F 8ED8                     mov     ds,ax
24553                                     ;mov    [ARENA.SIGNATURE],dl      ; type of arena
24554 00000B61 88160000              mov     [0],dl
24555                                     ;mov    word [ARENA.OWNER],0      ; free
24556 00000B65 C70601000000          mov     word [1],0
24557 00000B6B 29D9                     sub     cx,bx                     ; size of the new block
24558                                     ;mov    [ARENA.SIZE],cx          ; store it in the arena
24559 00000B6D 890E0300              mov     [3],cx
24560 00000B71 8CC0                     mov     ax,es                     ; return seg to the caller
24561                                     ; 23/10/2022
24562                                     ; 24/03/2019 - Retro DOS v4.0 (ORG 0) ; Take care ORG 30h of
24563 00000B73 83E803              sub     ax,3                     ; BIOS code
24564 00000B76 5A                     pop     dx
24565 00000B77 59                     pop     cx
24566 00000B78 5F                     pop     di
24567 00000B79 1F                     pop     ds ; ds = cs
24568 00000B7A C3                     retn
24569
24570                                     ;
24571                                     ; M012 : END
24572                                     ;
24573 00000B7B 0E                     FatalErr: push    cs
24574 00000B7C 1F                     pop     ds
24575 00000B7D BA[480A]              mov     dx,FEmsg
24576 00000B80 B409                     mov     ah,9
24577 00000B82 CD21                     int     21h                     ; DOS - PRINT STRING
24578                                     ; DS:DX -> string terminated by "$"
24579                                     ; 30/12/2022 (MSDOS 6.21 SYSINIT)
24580 00000B84 E9C707              jmp     stall
24581                                     ; 23/10/2022 (MSDOS 5.0 SYSINIT)
24582                                     ;cli
24583                                     ;hlt
24584
24585 ;endif ;ROMDOS
24586
24587 ; 25/03/2019 - Retro DOS v4.0
24588
24589 ; -----
24590 ;
24591 ; procedure : AllocHMA
24592 ;
24593 ; grab_the_hma tries to enable a20 and make sure there is memory
24594 ; up there. If it gets any sort of error, it will return with
24595 ; carry set so that we can resort to running low.
24596 ;
24597 ; It also returns ES: -> 0ffffh if it returns success
24598 ; -----
24599 ;
24600
24601 AllocHMA:
24602 ; cas note: The pre-286 check is no longer needed here since the
24603 ; presence of XMS is sufficient. However, this code hasn't
24604 ; been deleted because it can be recycled for skipping the
24605 ; extra pass of CONFIG.SYS and assuming we're running low
24606 ; in the case of a pre-286.
24607
24608 ;; see if we're running on a pre-286. If not, force low.
24609 ;
24610 ; xor     ax,ax
24611 ; pushf                                ; save flags (like int)
24612 ; push    ax
24613 ; popf
24614 ; pushf
24615 ; pop     ax
24616 ; popf                                ; restore original flags (like int)
24617 ; and     ax,0F000h
24618 ; cmp     ax,0F000h ; 8088/8086?
24619 ; jz      short grab_hma_error
24620
24621 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24622 ; (SYSINIT:0A26h)
24623
24624 ; 13/04/2024 - Retro DOS v5.0
24625 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:0C41h)
24626
24627 00000B87 1E                     push    ds
24628                                     ;mov    ax,Bios_Data
24629                                     ;mov    ax,KERNEL_SEGMENT
24630                                     ; 21/10/2022
24631 00000B88 B87000              mov     ax,DOSBIODATASEG ; 70h
24632 00000B8B 8ED8                     mov     ds,ax
24633
24634 00000B8D E84A00              call    IsXMSLoaded
24635 00000B90 7545              jnz     short grabhma_error
24636
24637 00000B92 B81043              mov     ax,4310h
24638 00000B95 CD2F              int     2Fh                     ; get the vector into es:bx
24639                                     ; - Multiplex - XMS - GET DRIVER ADDRESS
24640                                     ; Return: ES:BX -> driver entry point
24641
24642 00000B97 891E[0E00]          mov     [xms],bx
24643                                     ;mov    [0Eh], bx
24644 00000B9B 8C06[1000]          mov     [xms+2],es
24645                                     ;mov    [10h],es
24646
24647 00000B9F B401                     mov     ah,1                     ; request HMA
24648 00000BA1 BAF0FF              mov     dx,0FFFFh
24649                                     ;call   dword ptr ds:0Eh
24650 00000BA4 FF1E[0E00]          call    far [xms]
24651 00000BA8 48                     dec     ax
24652 00000BA9 7409              jz      short allocHMA_1 ; error if not able to allocate HMA
24653
24654 ;----- Himem may be lying because it has allocated mem for int 15
24655
24656 00000BAB B488                     mov     ah,88h
24657 00000BAD CD15              int     15h
24658                                     ; Get Extended Memory Size
24659                                     ; Return: CF clear on success
24660                                     ; AX = size of memory above 1M in K
24661 00000BAF 83F840              cmp     ax,64                     ; less than 64 K of hma ?
24662                                     ;jb     short grabhma_error
24663                                     ; 11/12/2022

```

```

24664 00000BB2 7224      jb      short grabhma_err ; cf=1
24665 allocHMA_1:
24666 00000BB4 B405      mov     ah,5           ; localenableA20
24667      ;call  dword ptr ds:0Eh
24668 00000BB6 FF1E[0E00]  call    far [xms]
24669 00000BBA 48        dec     ax
24670 00000BBB 751A      jnz     short grabhma_error ; error if couldn't enable A20
24671
24672 00000BBD E89D01     call    isvDiskInstalled
24673 00000BC0 7415      jz      short grabhma_error ; yes, we cant use HMA
24674
24675 00000BC2 B8FFFF      mov     ax,0FFFFh
24676 00000BC5 8EC0      mov     es,ax
24677 00000BC7 26C70610003412  mov     word [es:10h],1234h ; see if we can really read/write there
24678 00000BCE 26813E10003412  cmp     word [es:10h],1234h
24679      ;jne     short grabhma_error ; don't try to load there if XMS lied
24680      ; 11/12/2022
24681 00000BD5 7401      je      short allocHMA_ok
24682
24683      ; 11/12/2022
24684      ; 11/12/2022
24685      ; cf=0
24686      ; cld
24687      pop     ds
24688      retn
24689
24690 grabhma_error:
24691 00000BD7 F9        stc
24692      ; 11/12/022
24693 grabhma_err:      ; cf=1
24694 allocHMA_ok:      ; cf=0
24695 00000BD8 1F        pop     ds
24696 00000BD9 C3        retn
24697
24698 ; -----
24699 ;
24700 ; procedure : IsXMSLoaded
24701 ;
24702 ; Checks whether a XMS driver is loaded
24703 ;
24704 ; Returns : Z flag set if XMS driver loaded
24705 ;           Z flag reset if no XMS drivers are present
24706 ;
24707 ; -----
24708
24709 IsXMSLoaded:
24710 00000BDA B80043     mov     ax,4300h
24711 00000BDD CD2F      int     2Fh           ; - Multiplex - XMS - INSTALLATION CHECK
24712      ; Return: AL = 80h XMS driver installed
24713      ; AL <> 80h no driver
24714 00000BDF 3C80      cmp     al,80h         ; XMS installed?
24715 00000BE1 C3        retn
24716
24717 ; -----
24718 ;
24719 ; procedure : FTryToMovDOSHi
24720 ;
24721 ; Called from HMA suballoc calls
24722 ;
24723 ; -----
24724 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24725 ; (MSDOS 5.0 IO.SYS - SYSINIT:0A84h)
24726 ;
24727 ; 08/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
24728 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:0C9Fh)
24729 ;
24730 ; ((MSDOS 6.22 IO.SYS - SYSINIT:0B8Ch))
24731
24732 FTryToMovDOSHi:      ; proc far
24733
24734 00000BE2 50        push    ax
24735 00000BE3 53        push    bx
24736 00000BE4 51        push    cx
24737 00000BE5 52        push    dx
24738 00000BE6 56        push    si
24739 00000BE7 57        push    di
24740 00000BE8 1E        push    ds
24741 00000BE9 06        push    es
24742
24743      ; 23/10/2022
24744      ; 27/03/2019 - Retro DOS v4.0
24745      ; 11/12/2022
24746 00000BEA 0E        push    cs
24747 00000BEB 1F        pop     ds
24748
24749      ;cmp     byte [cs:runhigh],0FFh
24750      ; 11/12/2022
24751 00000BEC 803E[6C02]FF  cmp     byte [runhigh],0FFh
24752 00000BF1 7503      jne     short _ftymdh_1
24753
24754      ; ds = cs
24755 00000BF3 E8A2FE     call    TryToMovDOSHi
24756 _ftymdh_1:
24757 00000BF6 07        pop     es
24758 00000BF7 1F        pop     ds
24759 00000BF8 5F        pop     di
24760 00000BF9 5E        pop     si
24761 00000BFA 5A        pop     dx
24762 00000BFB 59        pop     cx
24763 00000BFC 5B        pop     bx
24764 00000BFD 58        pop     ax
24765
24766 00000BFE CB        retf
24767
24768 ; -----
24769 ;
24770 ; following piece of code will be moved into a para boundary. And the para
24771 ; address posted in seg of int 19h vector. Offset of int 19h will point to
24772 ; VDint19. This is to protect HMA from apps which use VDISK header method
24773 ; to determine free extended memory.
24774 ;
24775 ; For more details read "power programming" column by Ray Duncan in the
24776 ; May 30 1989 issue of PC Magazine (pp 377-388) [USING EXTENDED MEMORY,PART 1]
24777 ;
24778 ; -----
24779
24780      ; 30/12/2023 - Retro DOS 5.0
24781 00000BFF 00        db      0
24782
24783      ; 13/04/2024
24784 ;align 2
24785
24786      ; 30/12/2023
24787      ; PCDOS v7.1 IBMBIO.COM, SYSYINIT:0CBCh

```

```

24788
24789
24790 StartVDHead:
24791 ;----- what follows is a dummy device driver header (not used by DOS)
24792
24792 00000C00 00000000      dd      0          ; link to next device driver
24793 00000C04 0080        dw      8000h       ; device attribute
24794 00000C06 0000        dw      0          ; strategy routine offset
24795 00000C08 0000        dw      0          ; interrupt routine offset
24796 00000C0A 01         db      1          ; number of units
24797 ;db      7 dup(0)
24798 00000C0B 00<rep 7h>   times 7 db 0      ; reserved area
24799
24800 00000C12 564449534B    VDiskSig1:
24801 db      'VDISK'
24802
24803 VLEN1      equ      ($-VDiskSig1)
24804
24804 00000C17 202056332E33  db      ' V3.3'      ; vdisk label
24805 ;db      15 dup(0)      ; pad
24806 00000C1D 00<rep Fh>   times 15 db 0
24807 00000C2C 0000        dw      0          ; bits 0-15 of free HMA
24808 00000C2E 11         db      11h         ; bits 16-23 of free HMA (1M + 64K)
24809
24810 00000C2F EA          VInt19:
24811 db      0EAh          ; jmp to old vector
24812 00000C30 00000000    OldVDInt19:
24813 dd      0            ; saved int 19 vector
24814
24815 EndVDHead: ; label byte
24816
24817 00000C34 000000      VDiskHMAHead:
24818 db      0,0,0         ; non-bootable disk
24819 00000C37 564449534B  VDiskSig2:
24820 db      'VDISK'
24821
24822 VLEN2      equ      ($-VDiskSig2)
24823
24823 00000C3C 332E33      db      '3.3'      ; OEM - signature
24824 00000C3F 8000      dw      128         ; number of bytes/sector
24825 00000C41 01       db      1          ; sectors/cluster
24826 00000C42 0100     dw      1          ; reserved sectors
24827 00000C44 01       db      1          ; number of FAT copies
24828 00000C45 4000     dw      64         ; number of root dir entries
24829 00000C47 0002     dw      512        ; number of sectors
24830 00000C49 FE       db      0FEh       ; media descriptor
24831 00000C4A 0600     dw      6          ; number of sectors/FAT
24832 00000C4C 0800     dw      8          ; sectors per track
24833 00000C4E 0100     dw      1          ; number of heads
24834 00000C50 0000     dw      0          ; number of hidden sectors
24835 00000C52 4004     dw      440h        ; Start of free HMA in K (1M+64K)
24836
24837 EndVDiskHMAHead: ; label byte
24838
24839 ; -----
24840 ;
24841 ; procedure : InstVDiskHeader
24842 ;
24843 ;         Installs the VDISK header to reserve the 64k of HMA
24844 ;         It puts a 32 byte header at 10000:0 and
24845 ;         another header at (seg of int19):0
24846 ;
24847 ; Inputs : None
24848 ;
24849 ; Outputs : None
24850 ;
24851 ; USES : DS,SI,AX,CX,DX
24852 ;
24853 ; -----
24854
24855 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
24856
24857 InstVDiskHeader:
24858 00000C54 31C0      xor      ax,ax
24859 00000C56 8ED8      mov      ds,ax          ; seg of int vect table
24860
24861 ;----- save old int 19 vector
24862
24863 ; 23/10/2022
24864 00000C58 A16400    mov      ax,[19h*4]
24865 ;mov      [OldVDInt19],ax
24866 00000C5B 2EA3[300C] mov      [cs:OldVDInt19],ax
24867 00000C5F A16600    mov      ax,[19h*4+2]
24868 ;mov      [OldVDInt19+2],ax
24869 00000C62 2EA3[320C] mov      [cs:OldVDInt19+2],ax
24870
24871 ;----- calculate seg of new int 19 handler
24872
24873 00000C66 B448      mov      ah,48h          ; allocate memory
24874 ;mov      bx,(EndVDHead-StartVDHead+15)>>4
24875 ; 23/10/2022
24876 00000C68 BB0400    mov      bx,4
24877 00000C6B CD21      int      21h
24878
24879 ; if carry, fatal hanging error!!!!
24880
24881 00000C6D 48        dec      ax          ; point to arena
24882 00000C6E 8EC0      mov      es,ax
24883 ;mov      word [es:ARENA.OWNER],8      ; owner = System
24884 00000C70 26C70601000800 mov      word [es:1],8
24885 ;mov      word [es:ARENA.NAME],'SC' ; System Code
24886 00000C77 26C70608005343 mov      word [es:8],'SC' ; 4353h
24887 00000C7E 40        inc      ax
24888 00000C7F 8EC0      mov      es,ax          ; get back to allocated memory
24889
24890 ;----- install new int 19 vector
24891
24892 00000C81 FA        cli          ; no reboots at this time
24893 ;mov      word [19h*4],(VInt19-StartVDHead)
24894 00000C82 C70664002F00 mov      word [19h*4],47
24895 00000C88 A36600    mov      [19h*4+2],ax
24896
24897 ;----- move the code into proper place
24898
24899 ;mov      cx,(EndVDHead-StartVDHead)
24900 00000C8B B93400    mov      cx,52
24901 00000C8E BE[000C]  mov      si,StartVDHead
24902 00000C91 31FF      xor      di,di
24903 00000C93 0E        push     cs
24904 00000C94 1F        pop      ds
24905 00000C95 FC        cld
24906 00000C96 F3A4      rep      movsb
24907 00000C98 FB        sti          ; BUGBUG is sti OK now?
24908
24909 ;----- mov the HMA VDisk head into HMA
24910
24911 ; 23/10/2022

```

```

24912 00000C99 57      push    di
24913 00000C9A 06      push    es
24914
24915      ;mov    ax,0FFFFh
24916      ;mov    es,ax
24917      ; 03/09/2023
24918 00000C9B 49      dec     cx
24919      ; cx = 0FFFFh
24920 00000C9C 8EC1     mov     es,cx
24921
24922 00000C9E BF1000     mov     di,10h
24923      ;mov    cx,(EndVDiskHMAHead-VDiskHMAHead)
24924 00000CA1 B92000     mov     cx,32
24925 00000CA4 BE[340C]     mov     si,VDiskHMAHead
24926 00000CA7 F3A4      rep     movsb          ; ds already set to cs
24927
24928 00000CA9 5F      pop     di
24929 00000CAA 07      pop     es
24930
24931 00000CAB C3      retn
24932
24933      ; -----
24934      ; procedure : C1rVDISKHeader
24935      ;
24936      ;       Clears the first 32 bytes at 1MB boundary
24937      ;       So that DOS/HIMEM is not confused about the VDISK header
24938      ;       left by previous DOS=HIGH session
24939      ;
24940      ; -----
24941
24942      struc desc
24943      .seg_lim: resw    1          ; seg limit 64k
24944      .lo_word: resw    1          ; 24 bit seg physical
24945      .hi_byte: resb    1          ; address
24946      .acc_rights: resb    1      ; access rights ( CPL0 - R/W )
24947      .reserved: resw    1          ;
24948      .size:
24949      endstruc
24950
24951      ; 23/10/2022
24952      bmove:          ;label byte
24953
24954      dummy:          ;times desc.size db 0 ; desc <>
24955      times 8 db 0
24956      gdt:             ;times desc.size db 0 ; desc <>
24957      times 8 db 0
24958      src_desc:        dw     0FFFFh      ; desc <0ffffh,0,0,93h,0>
24959      dw     0
24960      db     0
24961      db     93h
24962      dw     0
24963      tgt_desc:        dw     0FFFFh      ; desc <0ffffh,0,10h,93h,0> ; 1MB
24964      dw     0
24965      db     10h
24966      db     93h
24967      dw     0
24968
24969      rombios_code:    ;times desc.size db 0 ; desc <>
24970      times 8 db 0
24971      temp_stack:      ;times desc.size db 0 ; desc <>
24972      times 8 db 0
24973
24974      C1rVDISKHead:    times 32 db 0      ; db 32 dup (0)
24975
24976      ; 25/03/2019 - Retro DOS v4.0 (MSDOS 6.21 IO.SYS, MSDOS 6.0 SYSINIT1.ASM)
24977
24978      ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
24979      ; (SYSINIT:0CA6h)
24980
24981      C1rVDISKHeader:  ; proc near
24982
24983      ;; 04/07/2023 - Retro DOS v4.2 IO.SYS (optimized)
24984
24985      ;-----;I070
24986      ; The following workaround get around a problem with the ;I070
24987      ; Tortugas and PS/2 30-286 BIOS when password server mode ;I070
24988      ; is set. On those machines the INT 15h block move code ;I070
24989      ; goes through the 8042 to twiddle A20 instead of port 92h. ;I070
24990      ; In password server mode the 8042 is disabled so the block ;I070
24991      ; move crashes the system. We can do this because these ;I070
24992      ; systems clear all of memory on a cold boot. ;I070
24993      ;
24994      ; in al,64h ; Test for password servr mode ;I070
24995      test al,10h ; Is keyboard inhibited? ;I070
24996      jnz short C1rVDISKok ; No, go do block move. ;I070
24997      ; Check for Tortugas... ;I070
24998      cmp word [cs:sys_model_byte],19F8h ;I070
24999      je short C1rVDISKno ;I070
25000      ; Check for mod 30-286 ;I070
25001      cmp word [cs:sys_model_byte],09FCh ;I070
25002      jne short C1rVDISKok ;I070
25003      C1rVDISKno: retn ; Return w/o block move. ;I070
25004      ;I070
25005      C1rVDISKok: ;I070
25006      ;-----;I070
25007
25008      ; 30/12/2023 - Retro DOS v5.0
25009      ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0DBAh
25010      C1rVDISKHeader:
25011      in al,64h ; 8042 keyboard controller status register
25012      ; 7: PERR 1=parity error in data received from keyboard
25013      ; +----- AT Mode -----+----- PS/2 Mode -----+
25014      ; 6: |RxTO receive (Rx) timeout | TO general timeout (Rx or Tx) |
25015      ; 5: |TxTO transmit (Tx) timeout | MOBF mouse output buffer full |
25016      ; +-----+
25017      ; 4: INH 0=keyboard communications inhibited
25018      ; 3: A2 0=60h was the port last written to, 1=64h was last
25019      ; 2: SYS distinguishes reset types: 0=cold reboot, 1=warm reboot
25020      ; 1: IBF 1=input buffer full (keyboard can't accept data)
25021      ; 0: OBF 1=output buffer full (data from keyboard is available)
25022      test al,10h ; test bit 4 - Is keyboard inhibited?
25023      jnz short C1rVDISKok ; No, go do block move
25024      ; 30/12/2023
25025      ; ds = cs
25026      cmp word [sys_model_byte],19F8h ; check for TORTUGA models
25027      jz short C1rVDISKno ; do not use INT 15h block move code
25028      ; (while 8042 is disabled)
25029      cmp word [sys_model_byte],9FCh ; check for PS/2 30-286 model
25030      jnz short C1rVDISKok
25031      C1rVDISKno:
25032      retn
25033      ; -----
25034      ; 30/12/2023
25035      C1rVDISKok:

```

```

25036 ; 12/12/2022
25037 ; ds = cs
25038
25039 ; 30/12/2022 - Retro DOS v4.2
25040 ; (MSDOS 6.21 IO.SYS SYSINIT:0CBFh)
25041
25042 00000D13 06      push    es
25043 00000D14 8CC8     mov     ax,cs
25044 00000D16 89C2     mov     dx,ax
25045 00000D18 B10C     mov     cl,12
25046 00000D1A D3EA     shr     dx,cl
25047 00000D1C B104     mov     cl,4
25048 00000D1E D3E0     shl     ax,cl
25049 00000D20 05[DC0C] add     ax,clrdVDISKHead
25050 00000D23 80D200    adc     dl,0
25051
25052 ;; 23/10/2022
25053 ;;mov [cs:src_desc+desc.lo_word],ax
25054 ;;mov [cs:src_desc+2],ax
25055 ;;mov [cs:src_desc+desc.hi_byte],dl
25056 ;;mov [cs:src_desc+4],dl
25057 ; 12/12/2022
25058 ;mov [src_desc+desc.lo_word],ax
25059 00000D26 A3[BE0C]  mov     [src_desc+2],ax
25060 ;mov [src_desc+desc.hi_byte],dl
25061 00000D29 8816[C00C]  mov     [src_desc+4],dl
25062
25063 00000D2D B91000    mov     cx,16 ; 16 words
25064 00000D30 0E      push    cs
25065 00000D31 07      pop     es
25066 00000D32 BE[AC0C]  mov     si,bmove
25067 00000D35 B487     mov     ah,87h
25068 00000D37 CD15     int     15h ; EXTENDED MEMORY - BLOCK MOVE (AT,XT286,PS)
25069 ; CX = number of words to move
25070 ; ES:SI -> global descriptor table
25071 ; Return: CF set on error, AH = status
25072 00000D39 07      pop     es
25073 00000D3A C3      retn
25074
25075 ; -----
25076 ;
25077 ; procedure : SaveFreeHMAPtr
25078 ;
25079 ; Save the Free HMA pointer in BIOS variable for later use.
25080 ; (INT 2f ax==4a01 call returns pointer to free HMA)
25081 ; Normalizes the pointer to ffff:xxxx format and stores only
25082 ; the offset.
25083 ;
25084 ; Inputs : ES:DI - pointer to free HMA
25085 ; Output : FreeHMAPtr in BIOS data segment updated
25086 ; -----
25087 ;
25088 ;
25089 SaveFreeHMAPtr:
25090 ; 03/09/2023
25091 00000D3B 1E      push    ds
25092 00000D3C B87000    mov     ax,DOSBIODATASEG ; 0070h
25093 00000D3F 8ED8     mov     ds,ax
25094 ;
25095 00000D41 8CC3     mov     bx,es
25096 00000D43 B8FFFF     mov     ax,0FFFFh ; HMA segment
25097 ; 03/09/2023
25098 00000D46 A2[0D00]  mov     [inHMA],al ; 0FFh ; (BIOSDATA:000Dh) ; 08/04/2024
25099 ;
25100 00000D49 29D8     sub     ax,bx
25101 00000D4B 83C70F    add     di,15 ; para round
25102 00000D4E 83E7F0    and     di,0FFF0h
25103 00000D51 B104     mov     cl,4
25104 00000D53 D3E0     shl     ax,cl
25105 00000D55 29C7     sub     di,ax
25106 ;
25107 ; 03/09/2023
25108 ;push ds
25109 ;;mov ax,Bios_Data ; 0070h
25110 ;mov ax,KERNEL_SEGMENT ; 0070h
25111 ; 21/10/2022
25112 ; 03/09/2023
25113 ;mov ax,DOSBIODATASEG ; 0070h
25114 ;mov ds,ax
25115 ; (BIOSDATA:07D7h for PCDOS 7.1 IBMBIO.COM) ; 08/04/2024
25116 00000D57 893E[D707]  mov     [FreeHMAPtr],di ; (ds:8F7h for MSDOS 6.21 IO.SYS)
25117 ;mov byte [inHMA],0FFh ; (ds:0Dh)
25118 00000D5B 1F      pop     ds
25119 00000D5C C3      retn
25120
25121 ; -----
25122 ;
25123 ; procedure : IsVDiskInstalled
25124 ;
25125 ; Checks for the presence of VDISK header at 1MB boundary
25126 ; & INT 19 vector
25127 ;
25128 ; Inputs : A20 flag should be ON
25129 ; Outputs : Zero set if VDISK header found else Zero cleared
25130 ; -----
25131 ;
25132 ;
25133 IsVDiskInstalled:
25134 00000D5D 31C0     xor     ax,ax
25135 00000D5F 8ED8     mov     ds,ax
25136 00000D61 8E1E4E00  mov     ds,[19*4+2]
25137 ;mov si,VDiskSig1-StartVDHead ; 12h
25138 ; 23/10/2022
25139 00000D65 BE1200    mov     si,12h ; 18
25140 ;mov cx,VLEN1 ; 5
25141 00000D68 B90500    mov     cx,5
25142 00000D6B 0E      push    cs
25143 00000D6C 07      pop     es
25144 00000D6D BF[120C]  mov     di,VDiskSig1
25145 00000D70 F3A6     rep     cmps
25146 00000D72 740F     je     short ivdins_retn
25147 00000D74 B8FFFF     mov     ax,0FFFFh
25148 00000D77 8ED8     mov     ds,ax
25149 ;mov si,10h+(VDiskSig2-VDiskHMAHead) ; 13h
25150 00000D79 BE1300    mov     si,13h
25151 00000D7C BF[370C]  mov     di,VDiskSig2
25152 ;;mov cx,VLEN2 ; 5
25153 ;mov cx,5
25154 ; 03/09/2023
25155 00000D7F B105     mov     cl,5
25156 00000D81 F3A6     rep     cmps
25157 ivdins_retn:
25158 00000D83 C3      retn ; returns the zero flag
25159

```

```

25160
25161
25162
25163
25164
25165
25166
25167
25168
25169
25170
25171 00000D84 1E
25172 00000D85 B9FFFF
25173 00000D88 8EC1
25174
25175
25176 00000D8A 41
25177 00000D8B 8ED9
25178 00000D8D BEC000
25179 00000D90 BFD000
25180
25181 00000D93 B105
25182 00000D95 FC
25183 00000D96 F3A4
25184 00000D98 1F
25185 00000D99 C3
25186
25187
25188
25189
25190
25191
25192
25193 00000D9A D1E8
25194 00000D9C D1E8
25195 00000D9E D1E8
25196 00000DA0 D1E8
25197 00000DA2 C3
25198
25199
25200
25201
25202
25203
25204
25205
25206
25207
25208
25209
25210
25211
25212 00000DA3 C43E[6D02]
25213 00000DA7 268A4D20
25214
25215
25216 00000DAB 30ED
25217
25218 00000DAD 26884D21
25219
25220
25221
25222
25223
25224
25225
25226 00000DB1 B058
25227
25228 00000DB3 F6E1
25229
25230 00000DB5 E85D05
25231 00000DB8 8B36[A702]
25232
25233
25234
25235
25236
25237
25238
25239 00000DBC 29C6
25240
25241
25242
25243
25244
25245
25246
25247
25248
25249
25250
25251
25252
25253
25254
25255 00000DBE 26897518
25256
25257 00000DC2 89F0
25258 00000DC4 26C745160000
25259
25260
25261 00000DCA 26C535
25262 00000DCD 8EC0
25263 00000DCF 31FF
25264
25265
25266
25267
25268
25269
25270
25271
25272
25273 00000DD1 2EA1[A902]
25274 00000DD5 AB
25275
25276
25277
25278
25279
25280
25281 00000DD6 E85200
25282
25283

; -----
;
; procedure : CPMHack
;
;       Copies the code from 0:c0 into ffff:0d0h
;       for CPM compatibility
;
; -----
;
; 11/12/2022
CPMHack:
    push    ds
    mov     cx,0FFFFh
    mov     es,cx          ; ES = FFFF
    ;xor     cx,cx
    ; 11/12/2022
    inc     cx ; cx = 0
    mov     ds,cx          ; DS = 0
    mov     si,0C0h
    mov     di,0D0h
    ;mov     cx,5
    mov     cl,5
    cld
    rep     movsb          ; move 5 bytes from 0:c0 to FFFF:D0
    pop     ds
    retn

; -----
;
; procedure : off_to_para
;
; -----
off_to_para:
    shr     ax,1
    shr     ax,1
    shr     ax,1
    shr     ax,1
    retn

; -----
;
; ** TempCDS - Create (Temporary?) CDS
;
; ENTRY    ?? BUGBUG
;          (DS) = SysInitSeg
; EXIT     ?? BUGBUG
; USES     ?? BUGBUG
;
; -----
;
; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; 30/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
TempCDS:
    les     di,[DOSINFO]
    mov     cl,[es:di+SYSI_NUMIO]

    ;mov     cl,[es:di+20h]
    xor     ch,ch          ; (cx) = # of block devices

    mov     [es:di+SYSI_NCDS],cl ; one CDS per device
    ;mov     [es:di+21h],cl

    ;mov     al,cl
    ;mov     ah,curdirilen ; curdir_list.size ; 88
    ;mov     ah,88
    ;mul     ah          ; (ax) = byte size for those CDSs
    ; 30/12/2023
    mov     al,curdirilen ; curdir_list.size ; 88
    ;mov     al,88
    mul     cl          ; (ax) = byte size for those CDSs

    call    ParaRound      ; (ax) = paragraph size for CDSs
    mov     si,[top_of_cdss] ; 31/12/2022

; BUGBUG - we don't update confbot - won't someone else use it?
; chunkst -- answer: no. Confbot is used to access the CDSs,
; 25 Jul 92 which are stored BELOW it. Alloclim is the
;          variable which has the top of free memory for
;          device driver loads, etc.

    sub     si,ax

; chunkst, 25 Jul 92 -- note: I'm removing the code here
; that automatically updates alloclim every time we
; set up some new CDSS. Instead, I've added code
; which pre-allocates space for 26 CDSS. This
; way we've got room for worst case CDSS before
; we place MagicDrv.sys

    mov     [ALLOCCLIM],si ; can't alloc past here!

; 30/12/2022
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; (SYSINIT:0C52h)
;mov     [ALLOCCLIM],si ; (MSDOS 5.0 SYSINIT)

    mov     [es:di+SYSI_CDS+2],si
    ;mov     [es:di+18h],si
    mov     ax,si
    mov     word [es:di+SYSI_CDS],0 ; set address of CDS list
    ;mov     [word es:di+16h],0
    lds     si,[es:di+SYSI_DPB] ; (ds:si) = address of first DPB
    lds     si,[es:di]
    mov     es,ax
    xor     di,di          ; (es:di) = address of 1st CDS

; * Initialize our temporary CDSs. We'll init each CDS with the
; info from the corresponding DPB.
;
; (cx) = count of CDSs left to process
; (es:di) = address of next CDS

fooset:
    ; 23/10/2022
    mov     ax,[cs:DirStrng] ; "A:"
    stosw
    ; setup the root as the curdir

    ; 23/10/2022 (MSDOS 5.0 SYSINIT)
    ;call    get_dpb_for_drive_al ; get dpb for drive in dpb

    ; 30/12/2022
    ; (MSDOS 6.21 SYSINIT:0D8Bh)
    call    get_dpb_for_drive_al ; get dpb for drive in dpb

    ; (ds:si) = address of DPB

```



```

25284 ; (si) = -1 if no drive
25285
25286 00000DD9 2EA1[AB02] mov ax,[cs:DirStrng+2] ; "\",0
25287 00000DDD AB stosw
25288 00000DDE 2EFE06[A902] inc byte [cs:DirStrng]
25289 00000DE3 31C0 xor ax,ax ; 0
25290 00000DE5 51 push cx
25291 ;mov cx,curdir_list.cdir_flags - 4 ; 63
25292 00000DE6 B93F00 mov cx,63 ; 23/10/2022
25293 00000DE9 F3AA rep stosb ; zero out rest of CURDIR_TEXTS
25294
25295 ; should handle the system that does not have any floppies.
25296 ; in this case,we are going to pretended there are two dummy floppies
25297 ; in the system. still they have dpb and cds,but we are going to
25298 ; 0 out curdir_flags,curdir_devptr of cds so ibmdos can issue
25299 ; "invalid drive specification" message when the user try to
25300 ; access them.
25301 ;
25302 ; (ax) = 0
25303 ; (es:di) = CURDIR_FLAGS in the CDS records
25304 ; (ds:si) = Next DPB (-1 if none)
25305
25306 00000DEB 83FEFF cmp si,-1 ; cmp si,0FFFFh
25307 00000DEE 740C je short fooset_zero ; don't have any physical drive.
25308
25309 ; check to see if we are faking floppy drives. if not go to normcds.
25310 ; if we are faking floppy drives then see if this cds being initialised
25311 ; is for drive a: or b: by checking the appropriate field in the dpb
25312 ; pointed to by ds:si. if not for a: or b: then go to normcds. if
25313 ; for a: or b: then execute the code given below starting at fooset_zero.
25314 ; for dpb offsets look at inc\dpb.inc.
25315
25316 ; 03/09/2023
25317 00000DF0 41 inc cx ; cx = 1
25318
25319 00000DF1 2E380E[8B02] cmp [cs:fake_floppy_drv],cl ; 1 ; 03/09/2023
25320 ;cmp byte [cs:fake_floppy_drv],1
25321 00000DF6 750A jne short normcds ; machine has floppy drives
25322 ;cmp byte [si+DPB.drive],1 ; if dpb_drive = 0 (a) or 1 (b).
25323 ;cmp byte [si],1
25324 00000DF8 380C cmp [si],cl ; 1 ; 03/09/2023
25325 00000DFA 7706 ja short normcds
25326
25327 ; 30/12/2023
25328 ; ax = 0
25329 fooset_zero:
25330 00000DFC B103 mov cl,3 ; the next dpb pointer
25331 ; AX should be zero here
25332 00000DFE F3AB rep stosw
25333 ; 30/12/2023
25334 ;pop cx
25335 00000E00 EB0F jmp short get_next_dpb ; findcds
25336
25337 ; (ax) = 0
25338
25339 ; 30/12/2023
25340 ;fooset_zero:
25341 ;mov cl,3
25342 ;rep stosw
25343 ;pop cx
25344 ;jmp short fincds
25345
25346 ;* We have a "normal" DPB and thus a normal CDS.
25347 ;
25348 ; (ax) = 0
25349 ; (es:di) = CURDIR_FLAGS in the CDS records
25350 ; (ds:si) = Next DPB (-1 if none)
25351
25352 normcds:
25353 ; 30/12/2023
25354 ;pop cx
25355
25356 ; if a non-fat based media is detected (by dpb.numberoffat == 0), then
25357 ; set curdir_flags to 0. this is for signaling ibmdos and ifsfunc that
25358 ; this media is a non-fat based one.
25359
25360 ;cmp byte [si+DPB.FAT_COUNT],0 ; non fat system?
25361 ; 23/10/2022
25362 ;cmp byte [si+8],0
25363 ; 03/09/2023 (ax=0)
25364 00000E02 384408 cmp [si+8],al ; 0
25365 00000E05 7403 je short setnormcds ; yes. set curdir_flags to 0. ax = 0 now.
25366 00000E07 B80040 mov ax,curdir_inuse ; 4000h ; else,fat system. set the flag to curdir_inuse.
25367 ;mov ax,4000h
25368 setnormcds:
25369 00000E0A AB stosw ; curdir_flags
25370 00000E0B 89F0 mov ax,si
25371 00000E0D AB stosw ; curdir_devptr
25372 00000E0E 8CD8 mov ax,ds
25373 00000E10 AB stosw
25374
25375 get_next_dpb: ; entry point for fake_fooset_zero
25376 ; 30/12/2022
25377 ; (MSDOS 6.21 SYSINIT:0DD1h)
25378 ; 23/10/2022
25379 ;lds si,[si+19h] ; (MSDOS 5.0 SYSINIT)
25380 ;lds si,[si+DPB.NEXT_DPB] ; [si+19h]
25381 fincds: ; get_next_dpb
25382 ; 30/12/2023
25383 00000E11 59 pop cx
25384 ; 30/12/2022
25385 ; (MSDOS 6.21 SYSINIT:0DD1h)
25386 00000E12 B8FFFF mov ax,-1 ; mov ax,0FFFFh
25387 00000E15 AB stosw ; curdir_id
25388 00000E16 AB stosw ; curdir_id
25389 00000E17 AB stosw ; curdir_user_word
25390 00000E18 B80200 mov ax,2
25391 00000E1B AB stosw ; curdir_end
25392 00000E1C B000 mov al,0 ; clear out 7 bytes (curdir_type,
25393 00000E1E AA stosb
25394 00000E1F AB stosw ; curdir_ifs_hdr,curdir_fsda)
25395 00000E20 AB stosw
25396 00000E21 AB stosw
25397
25398 00000E22 E2AD loop fooset
25399
25400 00000E24 2EC606[A902]41 mov byte [cs:DirStrng],"A"; "A:\"
25401
25402 00000E2A C3 retn
25403
25404 ; -----
25405 ;***get_dpb_for_drive_al -- lookup the DPB for drive in al
25406 ;
25407 ; entry:

```

```

25408 ;      al == ASCII CAPS drive letter
25409 ;
25410 ;      exit:
25411 ;      ds:si -> DPB, or si = -1 if not found
25412 ; -----
25413 ;
25414 ; 30/12/2023
25415 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:0EFEh
25416 ;
25417 ; 30/12/2022
25418 ; (MSDOS 6.21 SYSINIT:0DEAh)
25419 ; 23/10/2022
25420 get_dpb_for_drive_al:
25421     lds     si,[cs:DOSINFO]          ; point to first DPB
25422     ; lds     si,[si+SYSI_DPB]        ; (ds:si) = address of first DPB
25423     lds     si,[si]
25424     sub     al,'A'
25425
25426 get_dpb_for_drive_1:
25427     ; cmp     al,[si+DPB.DRIVE]        ; match?
25428     cmp     al,[si]
25429     je      short got_dpb_for_drive    ; done if so
25430
25431     lds     si,[si+DPB.NEXT_DPB] ; [si+19h]
25432     cmp     si,-1
25433     jne     short get_dpb_for_drive_1 ; loop until hit end of DPBs
25434
25435 got_dpb_for_drive:
25436     retn
25437
25438 ;=====
25439 ;** EndFile - Build DOS structures
25440 ;
25441 ; This procedure is called after the config.sys has been processed and
25442 ; installable device drivers have been loaded (but before "install="
25443 ; programs are loaded) to create the dos structures such as SFTS, buffers,
25444 ; FCBS, CDSS, etc. It also loads the sysinit_base module in low memory
25445 ; to allow for the safe EXECing of "install=" programs. All memory
25446 ; above these structures is deallocated back to DOS.
25447 ;
25448 ;
25449 ; ENTRY    ?? BUGBUG
25450 ; EXIT     ?? BUGBUG
25451 ; USES     ?? BUGBUG
25452 ;
25453 ;=====
25454 ; allocate files
25455 ; -----
25456 ;
25457 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
25458 ; (SYSINIT:0CCDh)
25459 ;
25460 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
25461 ; (SYSINIT:0E00h)
25462 ;
25463 ; 09/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
25464 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:0F14h)
25465 ;
25466 ; ((MSDOS 6.22 IO.SYS - SYSINIT:0E00h))
25467
25468 endfile:
25469 ; we are now setting up final cdss,buffers,files,fcss strings etc. we no
25470 ; longer need the space taken by the temp stuff below confbot,so set allocim
25471 ; to confbot.
25472
25473 ; if this procedure has been called to take care of install= command,
25474 ; then we have to save es,si registers.
25475
25476 ; 11/12/2022
25477 ; ds = cs
25478
25479 ; 23/10/2022
25480 ; 31/03/2019
25481     push    ds
25482
25483     ; mov     ax,Bios_Data ; 0070h
25484     ; mov     ax,KERNEL_SEGMENT ; 0070h
25485     ; 21/10/2022
25486     mov     ax,DOSBIODATASEG ; 0070h
25487     mov     ds,ax
25488
25489     ; cmp     word [052Fh],0
25490     cmp     word [multrk_flag],multrk_off1 ;=0,multrack= command entered?
25491     jne     short multrk_flag_done
25492     ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
25493     ; or      word [multrk_flag],multrk_on ; 80h ; default will be on.
25494     ; 12/12/2022
25495     or      byte [multrk_flag],multrk_on ; 80h
25496 multrk_flag_done:
25497     ; 23/10/2022
25498     ; 31/03/2019
25499     pop     ds
25500
25501 ; 11/12/2022
25502 ; ds = cs
25503 ; mov     ax,[top_of_cdss] ; mov ax,[CONFBOT]
25504 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
25505 ; (SYSINIT:0E14h)
25506     mov     ax,[CONFBOT]
25507     mov     [ALLOCLIM],ax
25508 ; 23/10/2022
25509     mov     ax,[cs:top_of_cdss]
25510     mov     [cs:ALLOCLIM],ax
25511
25512 ; 11/12/2022
25513 ; ds = cs
25514 ; push    cs
25515 ; pop     ds
25516
25517 ; mov     ax,[CONFBOT]
25518 ; mov     [ALLOCLIM],ax
25519
25520 ; 09/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
25521 ;;;
25522 ; mov     ax,[cs:ALLOCLIM]
25523 ; mov     ax,[ALLOCLIM]
25524 ; mov     [cs:prev_allocim],ax
25525     mov     [prev_allocim],ax
25526 ; mov     ax,[cs:memhi]
25527     mov     ax,[memhi]
25528 ; mov     [cs:prev_memhi],ax
25529     mov     [prev_memhi],ax
25530 dosfts:
25531 ;;;

```



```

25532
25533 00000E63 E88A39      call    round
25534
25535      ; 11/12/2022
25536      ; ds = cs
25537 00000E66 A0[9F02]    mov     al,[FILES]
25538      ; 23/10/2022
25539      ;mov    al,[cs:FILES]
25540 00000E69 2C05        sub     al,5
25541 00000E6B 764B        jbe     short dofcb
25542
25543 00000E6D 50           push    ax
25544      ;mov    al,devmark_files ; 'F'
25545 00000E6E B046        mov     al,'F'
25546 00000E70 E81808      call    setdevmark      ; set devmark for sfts (files)
25547 00000E73 58          pop     ax
25548 00000E74 30E4        xor     ah,ah            ; do not use cbw instruction!!!!
25549                                ; it does sign extend.
25550
25551      ; 11/12/2022
25552 00000E76 8B1E[6203]    ; ds = cs
25553 00000E7A 8B16[6403]    mov     bx,[memlo]
25554 00000E7E C53E[6D02]    mov     dx,[memhi]
25555                                ;get pointer to dos data
25556      ; 23/10/2022
25557      ;mov    bx,[cs:memlo]
25558      ;mov    dx,[cs:memhi]
25559      ;lds    di,[cs:DOSINFO]
25560                                ;ds:bp points to sft
25561 00000E82 C57D04        ;lds    di,[di+SYSI_SFT]
25562                                ;ds:bp points to sft
25563                                ;mov    [di+SF.SFLink],bx
25564 00000E85 891D        mov     [di],bx
25565 00000E87 895502        mov     [di+SF.SFLink+2],dx ;set pointer to new sft
25566
25567 00000E8A 0E          push    cs
25568 00000E8B 1F          pop     ds
25569
25570      ; 11/12/2022
25571      ; ds = cs
25572 00000E8C C43E[6203]    les     di,[memlo]      ;point to new sft
25573      ; 23/10/2022
25574      ;les    di,[cs:memlo]
25575
25576      ;mov    word [es:di+SF.SFLink],-1
25577 00000E90 26C705FFFF    mov     word [es:di],-1 ; 0FFFFh
25578      ;mov    [es:di+SF.SFCount],ax
25579 00000E95 26894504      mov     [es:di+4],ax
25580      ; 09/04/2024
25581 00000E99 B33B        mov     bl,SF_ENTRY.size ; 59
25582      ;mov    bl,59
25583 00000E9B F6E3        mul     bl              ;ax = number of bytes to clear
25584 00000E9D 89C1        mov     cx,ax
25585      ; 11/12/2022
25586      ; ds = cs
25587 00000E9F 0106[6203]    add     [memlo],ax      ;allocate memory
25588      ; 23/10/2022
25589      ;add     [cs:memlo],ax
25590 00000EA3 B80600      mov     ax,6
25591      ; 11/12/2022
25592 00000EA6 0106[6203]    add     [memlo],ax      ;remember the header too
25593      ;add     [cs:memlo],ax
25594      ; 11/12/2022
25595 00000EAA 800E[6919]02    or      byte [setdevmarkflag],for_devmark ; 2
25596      ; 23/10/2022
25597      ;or      byte [cs:setdevmarkflag],2
25598 00000EAF E83E39      call    round            ; check for mem error before the stosb
25599 00000EB2 01C7        add     di,ax
25600 00000EB4 31C0        xor     ax,ax
25601 00000EB6 F3AA        rep     stosb            ;clean out the stuff
25602
25603      ; allocate fcbs
25604      ; -----
25605
25606      ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
25607      ; (SYSINIT:0D48h)
25608 dofcb:
25609      ; 11/12/2022
25610      ; ds = cs
25611      ;push    cs
25612      ;pop     ds
25613 00000EB8 E83539      call    round
25614      ;mov    al,devmark_fcbs ; 'x' ;='x'
25615 00000EBB B058        mov     al,'x'
25616 00000EBD E8CB07      call    setdevmark
25617      ; 11/12/2022
25618      ; ds = cs
25619 00000EC0 A0[A002]    mov     al,[FCBS]
25620      ;mov    al,[cs:FCBS]
25621 00000EC3 30E4        xor     ah,ah            ; do not use cbw instruction!!!!
25622                                ; it does sign extend.
25623
25624 00000EC5 8B1E[6203]    ; 11/12/2022
25625 00000EC9 8B16[6403]    mov     bx,[memlo]
25626 00000ECD C53E[6D02]    mov     dx,[memhi]
25627                                ;get pointer to dos data
25628      ; 23/10/2022
25629      ;mov    bx,[cs:memlo]
25630      ;mov    dx,[cs:memhi]
25631      ;lds    di,[cs:DOSINFO]
25632                                ;mov    [di+SYSI_FCB],bx
25633      ;mov    [di+SYSI_FCB+2],dx ;set pointer to new table
25634      ; 23/10/2022
25635 00000ED1 895D1A      mov     [di+1Ah],bx      ; [di+SYSI_FCB]
25636 00000ED4 89551C      mov     [di+1Ch],dx      ; [di+SYSI_FCB+2]
25637
25638 00000ED7 2E8A1E[A102]    mov     bl,[cs:KEEP]
25639 00000EDC 30FF        xor     bh,bh
25640      ;mov    [di+SYSI_KEEP],bx
25641 00000EDE 895D1E      mov     [di+1Eh],bx      ; [di+SYSI_KEEP]
25642
25643 00000EE1 0E          push    cs
25644 00000EE2 1F          pop     ds
25645
25646 00000EE3 C43E[6203]    les     di,[memlo]      ;point to new table
25647      ;mov    word [es:di+SF.SFLink],-1
25648 00000EE7 26C705FFFF    mov     word [es:di],-1
25649      ;mov    [es:di+SF.SFCount],ax
25650      ; 02/11/2022
25651 00000EEC 26894504      mov     [es:di+4],ax
25652 00000EF0 B33B        mov     bl,SF_ENTRY.size ; 59
25653 00000EF2 89C1        mov     cx,ax
25654 00000EF4 F6E3        mul     bl              ;ax = number of bytes to clear
25655 00000EF6 0106[6203]    add     [memlo],ax      ;allocate memory

```

```

25656             ;mov     ax,6
25657 00000EFA B80600      mov     ax,SF.size-2 ; 6
25658 00000EFD 0106[6203] add     [memlo],ax           ;remember the header too
25659             ;or      byte [setdevmarkflag],for_devmark ; 2
25660 00000F01 800E[6919]02 or      byte [setdevmarkflag],2
25661 00000F06 E8E738      call    round           ; check for mem error before the stosb
25662 00000F09 01C7        add     di,ax           ;skip over header
25663 00000F0B B041        mov     al,'A'
25664             fillloop:
25665 00000F0D 51           push    cx           ; save count
25666 00000F0E B93B00      mov     cx,SF_ENTRY.size ; 59 ; number of bytes to fill
25667 00000F11 FC          cld
25668 00000F12 F3AA        rep     stosb           ; filled
25669
25670             ;mov     word [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_ref_count],0 ; [es:di-59]
25671             ;mov     word [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_position],0 ; [es:di-38]
25672             ;mov     word [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_position+2],0 ; [es:di-36]
25673
25674             ; 18/12/2022
25675             ; cx = 0
25676 00000F14 26894DC5      mov     [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_ref_count],cx ;0 ; [es:di-59]
25677 00000F18 26894DDA      mov     [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_position],cx ;0 ; [es:di-38]
25678 00000F1C 26894DDC      mov     [es:di-(SF_ENTRY.size)+SF_ENTRY.sf_position+2],cx ;0 ; [es:di-36]
25679
25680             ; 23/10/2022
25681             ;mov     word [es:di-3Bh],0
25682             ;mov     word [es:di-26h],0
25683             ;mov     word [es:di-24h],0
25684
25685 00000F20 59           pop     cx
25686 00000F21 E2EA        loop    fillloop
25687
25688             ; allocate buffers
25689             ; -----
25690
25691             ; search through the list of media supported and allocate 3 buffers if the
25692             ; capacity of the drive is > 360kb
25693
25694             ; 18/12/2022
25695             ; cx = 0
25696 00000F23 833E[9902]FF   cmp     word [buffers],-1      ; has buffers been already set?
25697 00000F28 7403          je      short dodefaultbuff
25698 00000F2A E98000      jmp     dobuff                 ; the user entered the buffers=.
25699
25700             dodefaultbuff:
25701             ; 18/12/2022
25702 00000F2D 890E[9B02]     mov     [h_buffers],cx ; 0
25703             ;inc     cx
25704             ;inc     cx
25705             ;mov     [buffers],cx ; 2
25706             ; 10/04/2024
25707 00000F31 C706[9902]0200 mov     word [buffers],2
25708
25709             ;mov     word [h_buffers],0      ; default is no heuristic buffers.
25710             ;mov     word [buffers],2        ; default to 2 buffers
25711
25712             ; 23/10/2022
25713             ; 04/09/2023
25714             ;push    ax
25715             ;push    ds ; 26/03/2019
25716
25717             ; 04/09/2023
25718             ; ds = cs
25719 00000F37 C42E[6D02]     les     bp,[DOSINFO]           ; search through the dpb's
25720             ;les     bp,[cs:DOSINFO]
25721             ;les     bp,[es:bp+SYSI_DPB]      ; get first dpb
25722             ; 11/12/2022
25723 00000F3B 26C46E00      les     bp,[es:bp]
25724             ; 23/10/2022
25725             ;les     bp,[es:bp+0]           ; ! (MSDOS 5.0 IO.SYS address compability) !
25726
25727             ; 04/09/2023
25728             ; ds = cs
25729             ;push    cs
25730             ;pop     ds
25731             ;SYSINIT:0DE2h:
25732             nextdpb:           ; test if the drive supports removeable media
25733             ;mov     b1,[es:bp+DPB.drive]
25734             ; 11/12/2022
25735 00000F3F 268A5E00      mov     b1,[es:bp]
25736             ; 23/10/2022
25737             ;mov     b1,[es:bp+0]           ; ! (MSDOS 5.0 IO.SYS address compability) !
25738
25739             ;inc     b1
25740             ; 18/12/2022
25741 00000F43 43           inc     bx
25742
25743             ;mov     ax,(IOCTL<<8)|8
25744 00000F44 B80844      mov     ax,4408h
25745 00000F47 CD21          int     21h                 ; DOS - 2+ - IOCTL -
25746
25747             ; ignore fixed disks
25748
25749             or      ax,ax           ; ax is nonzero if disk is nonremoveable
25750             jnz     short nosetbuf
25751
25752             ; get parameters of drive
25753
25754 00000F4D 31DB          xor     bx,bx
25755             ;mov     b1,[es:bp+DPB.drive]
25756             ; 11/12/2022
25757 00000F4F 268A5E00      mov     b1,[es:bp]
25758             ; 23/10/2022
25759             ;mov     b1,[es:bp+0]           ; ! (MSDOS 5.0 IO.SYS address compability) !
25760
25761             ;inc     b1
25762             ; 18/12/2022
25763 00000F53 43           inc     bx
25764
25765 00000F54 BA[BC4D]      mov     dx,deviceparameters
25766             ;mov     ax,(IOCTL<<8)|GENERIC_IOCTL
25767 00000F57 B80D44      mov     ax,440Dh
25768             ;mov     cx,(RAWIO<<8)|GET_DEVICE_PARAMETERS
25769 00000F5A 896008      mov     cx,860h
25770 00000F5D CD21          int     21h                 ; DOS - 2+ - IOCTL -
25771 00000F5F 7220          jc      short nosetbuf      ; get next dpb if driver doesn't support
25772             ; generic ioctl
25773
25774             ; determine capacity of drive
25775             ; media capacity = #sectors * bytes/sector
25776
25777             ;mov     bx,[deviceparameters+A_DEVICEPARAMETERS.DP_BPBP.A_BPBP.TOTALSECTORS]
25778             ; 23/10/2022
25779 00000F61 8B1E[CB4D]     mov     bx,[deviceparameters+15] ; total sectors (16 bit)

```

```

25780 ; to keep the magnitude of the media capacity within a word,
25781 ; scale the sector size
25782 ; (ie. 1 -> 512 bytes, 2 -> 1024 bytes,...)
25783
25784 ;mov ax,[deviceparameters+A_DEVICEPARAMETERS.DP_BPBA_A_BP.BYTESPERSECTOR]
25785 ; 23/10/2022
25786 00000F65 A1[C34D] mov ax,[deviceparameters+7] ; bytes per sector
25787 00000F68 31D2 xor dx,dx
25788 00000F6A B90002 mov cx,512
25789 00000F6D F7F1 div cx ; scale sector size in factor of
25790 ; 512 bytes
25791 00000F6F F7E3 mul bx ; ax = #sectors * size factor
25792 00000F71 09D2 or dx,dx ; just in case of large floppies
25793 00000F73 7505 jnz short setbuf
25794 00000F75 3DD002 cmp ax,720 ; 720 sectors * size factor of 1
25795 00000F78 7607 jbe short nosetbuf
25796 setbuf:
25797 ; 18/12/2022
25798 ; word [buffers] = 2
25799 00000F7A C606[9902]03 mov byte [buffers],3
25800 ;mov word [buffers],3
25801 00000F7F EB0D jmp short chk_memsize_for_buffers ; now check the memory size
25802 ; for default buffer count
25803 nosetbuf:
25804 ; 23/10/2022
25805 ;cmp word [es:bp+DPB.NEXT_DPB],-1
25806 00000F81 26837E19FF cmp word [es:bp+19h], -1 ; 0FFFFh
25807 00000F86 7406 je short chk_memsize_for_buffers
25808 ;les bp,[es:bp+DPB.NEXT_DPB] ; [es:bp+19h]
25809 00000F88 26C46E19 les bp,[es:bp+19h]
25810 00000F8C EBB1 jmp short nextdpb
25811
25812 ;from dos 3.3,the default number of buffers will be changed according to the
25813 ;memory size too.
25814 ; default buffers = 2
25815 ; if diskette media > 360 kb,then default buffers = 3
25816 ; if memory size > 128 kb (2000h para),then default buffers = 5
25817 ; if memory size > 256 kb (4000h para),then default buffers = 10
25818 ; if memory size > 512 kb (8000h para),then default buffers = 15.
25819
25820 chk_memsize_for_buffers:
25821 ; 18/12/2022
25822 ;cmp word [MEMORY_SIZE],2000h
25823 ;jbe short bufset
25824 ;mov word [buffers],5
25825 ;cmp word [MEMORY_SIZE],4000h
25826 ;jbe short bufset
25827 ;mov word [buffers],10
25828 ;cmp word [MEMORY_SIZE],8000h
25829 ;jbe short bufset
25830 ;mov word [buffers],15
25831
25832 ; 18/12/2022
25833 ; word [buffers] = 3 or 2
25834 00000F8E BB[9902] mov bx,buffers
25835 00000F91 A1[9402] mov ax,[MEMORY_SIZE]
25836 00000F94 48 dec ax ; [MEMORY_SIZE] - 1
25837
25838 cmp ah,20h ; ax >= 2000h ([MEMORY_SIZE] > 2000h) ; *
25839 00000F98 7213 jb short bufset
25840 00000F9A C6070F mov byte [bx],15 ; [buffers] = 15 ; ***
25841 00000F9D 80FC80 cmp ah,80h ; ax >= 8000h ([MEMORY_SIZE] > 8000h) ; ***
25842 00000FA0 730B jnb short bufset
25843 00000FA2 C6070A mov byte [bx],10 ; [buffers] = 10 ; **
25844 00000FA5 80FC40 cmp ah,40h ; ax >= 4000h ([MEMORY_SIZE] > 4000h) ; **
25845 00000FA8 7303 jnb short bufset
25846 00000FAA C60705 mov byte [bx],5 ; [buffers] = 5 ; *
25847 bufset:
25848 ; 23/10/2022
25849 ; 26/03/2019
25850 ; 04/09/2023
25851 ;pop ds
25852 ;pop ax
25853
25854 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
25855 ;j.k. here we should put extended stuff and new allocation scheme!!!
25856 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
25857
25858 ; 26/03/2019
25859
25860 ;*****
25861 ;
25862 ; function: actually allocate buffers in the memory and initialize it. *
25863 ; input : *
25864 ; memhi:memlo - start of the next available memory *
25865 ; buffers = number of buffers *
25866 ; h_buffers = number of secondary buffers *
25867 ;
25868 ; output: *
25869 ; buffinfo.cache_count - # of caches to be installed. *
25870 ; buffinfo set. *
25871 ; bufferqueue set. *
25872 ;
25873 ; subroutines to be called: *
25874 ;
25875 ;*****
25876
25877 ; 23/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
25878 ; (SYSINIT:0E60h)
25879
25880 ; ds = cs ; 31/03/2019
25881 ; 23/10/2022
25882 ;lds bx,[cs:DOSINFO] ; ds:bx -> sysinitvar
25883 ; 04/09/2023
25884 00000FAD A1[9902] mov ax,[buffers] ; 31/03/2019
25885 00000FB0 8B0E[9B02] mov cx,[h_buffers] ; *
25886 00000FB4 C51E[6D02] lds bx,[DOSINFO]
25887 ;mov ax,[cs:buffers] ; set sysi_buffers
25888 ;mov [bx+SYSI_BUFFERS],ax ; [bx+3Fh]
25889 00000FB8 89473F mov [bx+3Fh],ax
25890 ; 04/09/2023
25891 ;mov ax,[cs:h_buffers]
25892 ;mov [bx+SYSI_BUFFERS+2],ax ; [bx+41h]
25893 ;mov [bx+41h],ax
25894 ; 04/09/2023
25895 00000FBB 894F41 mov [bx+41h],cx ; *
25896 00000FBE C55F12 lds bx,[bx+12h]
25897 ;lds bx,[bx+SYSI_BUF] ; now,ds:bx -> buffinfo
25898 00000FC1 E82C38 call round ; get,[memhi]:[memlo]
25899 ;mov al,devmark_buf ; ='B'
25900 00000FC4 B042 mov al,'B'
25901 00000FC6 E8C206 call setdevmark
25902
25903 ;allocate buffers

```

```

25904
25905 0000FC9 1E          push    ds          ; save buffer info. ptr.
25906 0000FCA 53          push    bx
25907
25908 0000FCB E8D403       call    set_buffer
25909
25910 0000FCE 5B          pop     bx
25911 0000FCF 1F          pop     ds
25912
25913 ;now set the secondary buffer if specified.
25914
25915 0000FD0 2E833E[9B02]00      cmp     word [cs:h_buffers],0
25916 0000FD6 742D          je      short xif16
25917 0000FD8 E81538       call    round
25918 ; 23/10/2022
25919 0000FDB 2E8B0E[6203]       mov     cx,[cs:memlo]
25920 ;mov     [bx+BUFFINF.Cache_ptr],cx ; [bx+6]
25921 0000FE0 894F06       mov     [bx+6],cx
25922 0000FE3 2E8B0E[6403]       mov     cx,[cs:memhi]
25923 ;mov     [bx+BUFFINF.Cache_ptr+2],cx ; [bx+8]
25924 0000FE8 894F08       mov     [bx+8],cx
25925 0000FEB 2E8B0E[9B02]       mov     cx,[cs:h_buffers]
25926 ;mov     [bx+BUFFINF.Cache_count],cx ; [bx+10]
25927 0000FF0 894F0A       mov     [bx+10],cx
25928 0000FF3 B80002       mov     ax,512          ; 512 byte
25929 0000FF6 F7E1          mul     cx
25930 0000FF8 2EA3[6203]       mov     [cs:memlo],ax
25931 ;or      byte [cs:setdevmarkflag],for_devmark ; 2
25932 0000FFC 2E800E[6919]02     or      byte [cs:setdevmarkflag],2
25933 0001002 E8EB37       call    round
25934 xif16:
25935
25936 ; -----
25937 ; allocate cdss
25938 ; -----
25939
25940 buf1:
25941 0001005 E8E837       call    round
25942
25943 0001008 50          push    ax
25944 ; 23/10/2022
25945 ;mov     ax,devmark_cds      ;='L'
25946 0001009 B84C00       mov     ax,'L'
25947 000100C E87C06       call    setdevmark
25948 000100F 58          pop     ax
25949
25950 0001010 2EC43E[6D02]       les     di,[cs:DOSINFO]
25951 ;mov     cl,[es:di+SYSI_NUMIO]
25952 0001015 268A4D20      mov     cl,[es:di+20h]
25953 0001019 2E3A0E[A202]      cmp     cl,[cs:NUM_CDS]
25954 000101E 7305          jae     short gotncds      ; user setting must be at least numio
25955 0001020 2E8A0E[A202]      mov     cl,[cs:NUM_CDS]
25956 gotncds:
25957 0001025 30ED          xor     ch,ch
25958 ;mov     [es:di+SYSI_NCDS],cl ; [es:di+33]
25959 0001027 26884D21      mov     [es:di+21h],cl
25960 000102B 2EA1[6403]       mov     ax,[cs:memhi]
25961 ;mov     [es:di+SYSI_CDS+2],ax
25962 000102F 26894518      mov     [es:di+18h],ax
25963 0001033 2EA1[6203]       mov     ax,[cs:memlo]
25964 ;mov     [es:di+SYSI_CDS],ax
25965 0001037 26894516      mov     [es:di+16h],ax
25966 000103B 88C8          mov     al,cl
25967 ;mov     ah,curdirlen ; curdir_list.size
25968 000103D B458          mov     ah,88
25969 000103F F6E4          mul     ah
25970 0001041 E8D102       call    ParaRound
25971 0001044 2E0106[6403]     add     [cs:memhi],ax
25972
25973 ;or      byte [cs:setdevmarkflag],for_devmark ; 2
25974 0001049 2E800E[6919]02     or      byte [cs:setdevmarkflag],2
25975 000104F E89E37       call    round          ; check for mem error before initializing
25976 ;lds     si,[es:di+SYSI_DPB] ; [es:di+0]
25977 0001052 26C535       lds     si,[es:di]
25978 ;les     di,[es:di+SYSI_CDS] ; [es:di+22]
25979 0001055 26C47D16      les     di,[es:di+16h]
25980 0001059 E875FD       call    fooset
25981
25982 ; -----
25983 ; allocate space for internal stack
25984 ; -----
25985
25986 000105C 0E          push    cs
25987 000105D 1F          pop     ds
25988
25989 ; if the user did not entered stacks= command, as a default, do not install
25990 ; sytem stacks for pc1,pc xt,pc portable cases.
25991 ; otherwise,install it to the user specified value or to the default
25992 ; value of 9,128 for other systems.
25993
25994 000105E 833E[9002]FF      cmp     word [stack_addr],-1 ; has the user entered "stacks=" command?
25995 0001063 740E          je      short doinstallstack ; then install as specified by the user
25996 0001065 803E[BC02]00      cmp     byte [sys_scnd_model_byte],0 ; pc1,xt has the secondary model byte = 0
25997 000106A 7507          jne     short doinstallstack ; other model should have default stack of 9,128
25998 000106C 803E[BB02]FE      cmp     byte [sys_model_byte],0FEh ; pc1, pc/xt or pc portable ?
25999 0001071 736D          jae     short skipstack
26000 doinstallstack:
26001 0001073 A1[8C02]        mov     ax,[stack_count] ; stack_count = 0?
26002 0001076 09C0          or      ax,ax          ; then, stack size must be 0 too.
26003 0001078 7466          jz      short skipstack ; don't install stack.
26004
26005 ; dynamic relocation of stack code.
26006
26007 000107A E87337       call    round          ;[memhi] = seg. for stack code
26008 ;[memlo] = 0
26009
26010 ; set devmark block into memory for mem command
26011 ; devmark_id = 's' for stack
26012
26013 ;mov     al,devmark_stk ;='s'
26014 ; 23/10/2022
26015 000107D B053          mov     al,'s'
26016 000107F E80906       call    setdevmark
26017
26018 0001082 A1[6403]        mov     ax,[memhi]
26019 0001085 8EC0          mov     es,ax          ;es -> seg. the stack code is going to move.
26020 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
26021 ; 11/12/2022
26022 ; ds = cs
26023 ;push    cs
26024 ;pop     ds
26025 0001087 31F6          xor     si,si          ;!we know that stack code is at the beginning of sysinit.
26026 0001089 31FF          xor     di,di
26027 000108B B9[6902]        mov     cx,endstackcode

```

```

26028 0000108E 890E[6203]      mov     [memlo],cx
26029 00001092 E85B37      call    round                ;have enough space for relocation?
26030 00001095 F3A4      rep     movsb
26031
26032 00001097 1E      push    ds                  ; stick the location of the NextStack entry
26033                                     ;;mov ax,Bios_Data ; into the win386 Instance Data tables
26034                                     ;mov ax,KERNEL_SEGMENT ; 70h
26035                                     ; 21/10/2022
26036 00001098 B87000      mov     ax,DOSBIODATASEG ; 0070h
26037 0000109B 8ED8      mov     ds,ax
26038 0000109D C706[0208][1000]  mov     word [NextStack],nextentry ; (8C0h for MSDOS 6.21 IO.SYS)
26039 000010A3 8C06[0408]  mov     [NextStack+2],es      ; (8C2h for MSDOS 6.21 IO.SYS)
26040
26041 000010A7 2EA1[6203]      mov     ax,[cs:memlo]
26042 000010AB 2EA3[9002]      mov     [cs:stack_addr],ax ;set for stack area initialization
26043 000010AF A3[0808]      mov     [IT_stackLoc],ax ; pass it as Instance Data, too
26044 000010B2 2EA1[6403]      mov     ax,[cs:memhi] ;this will be used by stack_init routine.
26045 000010B6 2EA3[9202]      mov     [cs:stack_addr+2],ax
26046 000010BA A3[0A08]      mov     [IT_stackLoc+2],ax
26047
26048                                     ; space for internal stack area = stack_count(entrysize + stack_size)
26049
26050                                     ;mov ax,entrysize ; mov ax,8
26051                                     ; 23/10/2022
26052 000010BD B80800      mov     ax,8
26053 000010C0 2E0306[8E02]  add     ax,[cs:stack_size]
26054 000010C5 2EF726[8C02]  mul     word [cs:stack_count]
26055
26056 000010CA A3[0C08]      mov     [IT_stackSize],ax ; pass through to Instance Tables
26057
26058 000010CD 1F      pop     ds                  ; no more need to access Instance Table
26059
26060 000010CE E84402      call    ParaRound          ; convert size to paragraphs
26061
26062                                     ; 11/12/2022
26063                                     ; ds = cs
26064                                     ;add [cs:memhi],ax
26065 000010D1 0106[6403]  add     [memhi],ax
26066                                     ;or byte [cs:setdevmarkflag],for_devmark ; 2
26067                                     ;or byte [cs:setdevmarkflag],2
26068 000010D5 800E[6919]02 or     byte [setdevmarkflag],2
26069                                     ;or byte [setdevmarkflag],for_devmark ; 2
26070                                     ;to set the devmark_size for stack by round routine.
26071 000010DA E81337      call    round              ; check for memory error before
26072                                     ; continuing
26073 000010DD E87D03      call    stackinit          ; initialize hardware stack.
26074                                     ; cs=ds=sysinitseg,es=relocated stack code & data
26075 skipstack:
26076
26077                                     ; 10/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
26078                                     ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:11F0h)
26079                                     ;;;
26080                                     ;push cs
26081                                     ;pop ds
26082                                     ; ds = cs
26083 000010E0 803E[6E03]01 cmp     byte [dosdata_umb],1 ; PCDOS 7 feature - DOSDATA=UMB/NOUMB configuration
26084                                     ; 1 = DOSDATA=UMB, 2 = (UMB) done, 0 = NOUMB
26085 000010E5 7773      ja     short dosdata_umb_done; 2 - done
26086 000010E7 727D      jb     short dosdata_noumb ; 0 - DOSDATA=NOUMB
26087
26088 000010E9 803E[8B16]EA cmp     byte [setdevmark],0EAh
26089 000010EE 7476      je     short dosdata_noumb
26090
26091 000010F0 B80258      mov     ax,5802h
26092 000010F3 CD21      int     21h                ; DOS - 3+ - GET/SET MEMORY ALLOCATION STRATEGY
26093                                     ; AL = function code: (DOS 5beta) get UMB link state
26094 000010F5 98      cbw
26095 000010F6 89C7      mov     di,ax              ; al = 01h -> UMBs in DOS memory chain
26096                                     ; save current (previous) UMB link state
26097 000010F8 B80100      mov     bx,1              ; bx = 01h -> add UMBs to DOS memory chain
26098
26099 000010FB B80358      mov     ax,5803h
26100 000010FE CD21      int     21h
26101 00001100 7264      jc     short dosdata_noumb
26102
26103 00001102 B80058      mov     ax,5800h
26104 00001105 CD21      int     21h                ; DOS - 3+ - GET/SET MEMORY ALLOCATION STRATEGY
26105                                     ; AL = function code: get allocation strategy
26106
26107 00001107 89C6      mov     si,ax              ; ax = current strategy
26108                                     ; save current (previous) allocation strategy
26109 00001109 B84000      mov     bx,40h            ; bl = new strategy = 40h - high memory first fit
26110
26111 0000110C B80158      mov     ax,5801h
26112 0000110F CD21      int     21h
26113
26114 00001111 8B1E[6403]  mov     bx,[memhi]
26115 00001115 2B1E[6A03]  sub     bx,[prev_memhi]
26116
26117 00001119 B448      mov     ah,48h
26118 0000111B CD21      int     21h                ; DOS - 2+ - ALLOCATE MEMORY
26119                                     ; BX = number of 16-byte paragraphs desired
26120 0000111D 89C1      mov     cx,ax              ; ax = segment of allocated block
26121 0000111F 89FB      mov     bx,di              ; restore previous UMB link state
26122
26123 00001121 B80358      mov     ax,5803h
26124 00001124 CD21      int     21h                ; DOS - 3+ - GET/SET MEMORY ALLOCATION STRATEGY
26125                                     ; AL = function code: (DOS 5beta) set UMB link state
26126 00001126 89F3      mov     bx,si              ; restore previous allocation strategy
26127
26128 00001128 B80158      mov     ax,5801h
26129 0000112B CD21      int     21h                ; DOS - 3+ - GET/SET MEMORY ALLOCATION STRATEGY
26130                                     ; AL = function code: set allocation strategy
26131 0000112D 81F900A0  cmp     cx,0A000h          ; Is the allocated memory block (segment) a UMB?
26132 00001131 7233      jb     short dosdata_noumb ; no
26133
26134                                     ;mov word [ALLOCLIM],0FFFFh
26135                                     ;mov word [memlo],0
26136 00001133 890E[6403]  mov     [memhi],cx
26137 00001137 49      dec     cx
26138 00001138 8EC1      mov     es,cx              ; point to arena/mcb
26139                                     ; 10/04/2024
26140 0000113A 31C9      xor     cx,cx ; 0
26141 0000113C 890E[6203]  mov     [memlo],cx ; 0
26142 00001140 49      dec     cx
26143 00001141 890E[A502]  mov     [ALLOCLIM],cx ; 0FFFFh
26144
26145 00001145 26C70601000800 mov     word [es:1],8 ; [es:arena_owner], 8 ; set impossible owner
26146 0000114C 26C70608005344 mov     word [es:8],4453h ; [es:arena_name],'SD' ; System Data
26147 00001153 FE06[6E03]  inc     byte [dosdata_umb] ; 1 -> 2 ; DOSDATA=UMB done.
26148 00001157 E909FD      jmp     dosfts
26149
26150 dosdata_umb_done:
26151 0000115A A1[6A03]      mov     ax,[prev_memhi]      ; (recent memory block/segment before UMBs)

```

```

26152 0000115D A3[6403]      mov     [memhi],ax
26153 00001160 A1[6C03]      mov     ax,[prev_alloclim]
26154 00001163 A3[A502]      mov     [ALLOCLIM],ax
26155 dosdata_numb:
26156      ;;
26157
26158 ;skipstack:
26159 ; 24/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
26160 ; (SYSINIT:0F99h)
26161
26162 ; 11/12/2022
26163 ; ds = cs
26164 ;push  cs
26165 ;pop   ds
26166
26167 00001166 A0[9F02]      mov     al,[FILES]
26168 00001169 30E4      xor     ah,ah          ; do not use cbw instruction!!!!
26169                                ; it does sign extend.
26170 0000116B 89C1      mov     cx,ax
26171 0000116D 31DB      xor     bx,bx          ;close standard input
26172 0000116F B43E      mov     ah,3Eh ; CLOSE
26173 00001171 CD21      int     21h
26174 00001173 BB0200    mov     bx,2
26175 rccllloop:
26176 00001176 B43E      mov     ah,3Eh ; CLOSE ;close everybody but standard output
26177 00001178 CD21      int     21h ; need output so we can print message
26178 0000117A 43      inc     bx ; in case we can't get new one open.
26179 0000117B E2F9      loop    rccllloop
26180
26181 0000117D BA[C34A]     mov     dx,condev
26182 00001180 B002      mov     al,2
26183 00001182 B43D      mov     ah,3Dh ; OPEN ;open con for read/write
26184 00001184 F9      stc          ; set for possible int 24
26185 00001185 CD21      int     21h
26186 00001187 7305      jnc     short goaux
26187 00001189 E89A38    call    badfil
26188 0000118C EB13      jmp     short goaux2
26189 goaux:
26190 0000118E 50      push    ax
26191 0000118F BB0100    mov     bx,1          ;close standard output
26192 00001192 B43E      mov     ah,3Eh ; CLOSE
26193 00001194 CD21      int     21h
26194 00001196 58      pop     ax
26195
26196 00001197 89C3      mov     bx,ax          ;new device handle
26197 00001199 B445      mov     ah,45h ; XDUP
26198 0000119B CD21      int     21h          ;dup to 1,stdout
26199 0000119D B445      mov     ah,45h ; XDUP
26200 0000119F CD21      int     21h          ;dup to 2,stderr
26201 goaux2:
26202 000011A1 BA[C74A]     mov     dx,auxdev
26203 000011A4 B002      mov     al,2          ;read/write access
26204 000011A6 E8AE38    call    open_dev
26205
26206 000011A9 BA[CB4A]     mov     dx,prndev
26207 000011AC B001      mov     al,1          ;write only
26208 000011AE E8A638    call    open_dev
26209
26210 ;global rearm command for shared interrupt devices attached in the system;
26211 ;shared interrupt attachment has some problem when it issues interrupt
26212 ;during a warm reboot. once the interrupt is presented by the attachment,
26213 ;no further interrupts on that level will be presented until a global rearm
26214 ;is issued. by the request of the system architecture group, msbio will
26215 ;issue a global rearm after every device driver is loaded.
26216 ;to issue a global rearm: ;for pci,xt,palace
26217 ;
26218 ;
26219 ; out 02f2h,xx ; interrupt level 2
26220 ; out 02f3h,xx ; interrupt level 3
26221 ; out 02f4h,xx ; interrupt level 4
26222 ; out 02f5h,xx ; interrupt level 5
26223 ; out 02f6h,xx ; interrupt level 6
26224 ; out 02f7h,xx ; interrupt level 7
26225 ;
26226 ; for pc at,in addition to the above commands,
26227 ; need to handle the secondary interrupt handler
26228 ;
26229 ; out 06f2h,xx ; interrupt level 10
26230 ; out 06f3h,xx ; interrupt level 11
26231 ; out 06f4h,xx ; interrupt level 12
26232 ; out 06f6h,xx ; interrupt level 14
26233 ; out 06f7h,xx ; interrupt level 15
26234 ;
26235 ; for round-up machine
26236 ;
26237 ; none.
26238 ;
26239 ; where xx stands for any value.
26240 ;
26241 ; for your information,after naples level machine,the system service bios
26242 ; call (int 15h),function ah=0c0h returns the system configuration parameters
26243 ; 24/10/2022
26244
26245 000011B1 50      push    ax
26246 000011B2 53      push    bx
26247 000011B3 52      push    dx
26248 000011B4 06      push    es
26249
26250 000011B5 B0FF      mov     al,0FFh          ;reset h/w by writing to port
26251 000011B7 BAF202    mov     dx,2F2h          ;get starting address
26252 000011BA EE      out     dx,al          ; out 02f2h,0ffh
26253 000011BB 42      inc     dx
26254 000011BC EE      out     dx,al          ; out 02f3h,0ffh
26255 000011BD 42      inc     dx
26256 000011BE EE      out     dx,al          ; out 02f4h,0ffh
26257 000011BF 42      inc     dx
26258 000011C0 EE      out     dx,al          ; out 02f5h,0ffh
26259 000011C1 42      inc     dx
26260 000011C2 EE      out     dx,al          ; out 02f6h,0ffh
26261 000011C3 42      inc     dx
26262 000011C4 EE      out     dx,al          ; out 02f7h,0ffh
26263
26264 ;sb secondary global rearm
26265
26266 000011C5 B800F0    mov     ax,0F000h          ;get machine type
26267 000011C8 8EC0      mov     es,ax
26268 000011CA 26803EFEFFFC cmp     byte [es:0FFFEh],0Fch ;q:is it a at type machine
26269 000011D0 740D      je      short startrearm ; *if at no need to check
26270
26271 000011D2 B4C0      mov     ah,0C0h          ;get system configuration
26272 000011D4 CD15      int     15h          ; *
26273 000011D6 7216      jc      short finishrearm ; *jmp if old rom
26274
26275 ; test feature byte for secondary interrupt controller

```



```

26276
26277 000011D8 26F6470540      test    byte [es:bx+5],40h
26278                          ; 24/10/2022
26279                          ; test    byte [es:bx+ROMBIOS_DESC.bios_sd_featurebyte1],ScndIntController
26280 000011DD 740F           je      short finishrearm      ;jmp if it is there
26281
26282
26283 000011DF B0FF           startrearm:
26284 000011E1 BAF206        mov     al,0FFh                ;write any pattern to port
26285 000011E4 EE           mov     dx,6F2h              ;get starting address
26286 000011E5 42           out     dx,al                ;out 06f2h,0ffh
26287 000011E6 EE           inc     dx                    ;bump address
26288 000011E7 42           out     dx,al                ;out 06f3h,0ffh
26289 000011E8 EE           inc     dx                    ;bump address
26290 000011E9 42           out     dx,al                ;out 06f4h,0ffh
26291 000011EA 42           inc     dx                    ;bump address
26292 000011EB EE           out     dx,al                ;out 06f5h,0ffh
26293 000011EC 42           inc     dx                    ;bump address
26294 000011ED EE           out     dx,al                ;out 06f6h,0ffh
26295
26296
26297 000011EE 07           finishrearm:
26298 000011EF 5A           pop     es
26299 000011F0 5B           pop     dx
26300 000011F1 58           pop     bx
26301                          pop     ax
26302
26303
26304
26305
26306
26307
26308
26309
26310
26311
26312
26313
26314
26315
26316
26317
26318
26319
26320
26321
26322
26323
26324
26325
26326 000011F2 50           ; 24/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS, SYSINIT)
26327 000011F3 A1[6403]      ; (SYSINIT:1028h)
26328 000011F6 2B06[6803]
26329 000011FA A3[6003]
26330
26331 000011FD B054
26332 000011FF E88904
26333 00001202 58
26334
26335 00001203 8B3E[6403]
26336 00001207 8EC7
26337 00001209 893E[D402]
26338 0000120D 31FF
26339 0000120F 893E[D202]
26340 00001213 BE[2113]
26341 00001216 B98100
26342
26343
26344 00001219 010E[6203]
26345
26346
26347
26348
26349 0000121D 800E[6919]02
26350
26351 00001222 E8CB35
26352 00001225 F3A4
26353
26354 00001227 C706[D602][0813]
26355 0000122D 8C0E[D802]
26356
26357
26358
26359 00001231 800E[CE02]02
26360
26361
26362
26363
26364
26365
26366
26367
26368
26369 00001236 E8B735
26370 00001239 8B1E[6403]
26371 0000123D A1[6803]
26372 00001240 A3[5E03]
26373 00001243 8EC0
26374 00001245 29C3
26375
26376 00001247 B44A
26377 00001249 CD21
26378
26379
26380
26381 0000124B 06
26382 0000124C 8CC0
26383 0000124E 48
26384 0000124F 8EC0
26385
26386
26387
26388
26389 00001251 26C70601000800
26390 00001258 26C70608005344
26391 0000125F 07
26392
26393 00001260 BBFFFF
26394 00001263 B448
26395 00001265 CD21
26396 00001267 B448
26397 00001269 CD21
26398
26399

```

```

; global rearm end *****
; -----
; allocate sysinit_base for install= command
; -----
; sysinit_base allocation.
; check if endfile has been called to handle install= command.
;
set_sysinit_base:
; -----
; sysinit_base will be established in the secure area of
; lower memory when it handles the first install= command.
; sysinit_base is the place where the actual exec function will be called and
; will check sysinit module in high memory if it is damaged by the application
; program. if sysinit module has been broken, then "memory error..." message
; is displayed by sysinit_base.
; -----
;
; 24/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS, SYSINIT)
; (SYSINIT:1028h)
;
; 11/12/2022
; ds = cs
push    ax
mov     ax,[memhi]          ; set devmark for mem command
mov     ax,[area]
sub     ax,[area]
mov     [impossible_owner_size],ax ;remember the size in case.
;mov    al,devmark_inst ; 'T'
mov     al,'T'
call    setdevmark
pop     ax

mov     di,[memhi]
mov     es,di
mov     [sysinit_base_ptr+2],di ; save this entry for the next use.
xor     di,di
mov     [sysinit_base_ptr],di ; es:di -> destination.
mov     si,sysinit_base      ;ds:si -> source code to be relocated.
mov     cx,end_sysinit_base-sysinit_base ; 129
; 24/10/2022
;mov    cx,128 ; 11DCh-115Ch ; (MSDOS 5.0 IO.SYS, SYSINIT)
add     [memlo],cx
;or     byte cs:[setdevmarkflag],for_devmark ; 2
; 11/12/2022
; ds = cs
;or     byte [cs:setdevmarkflag],2
;or     byte [setdevmarkflag],2
;or     byte [setdevmarkflag],for_devmark
call    round                ; check mem error. also, readjust memhi for the next use.
rep     movsb                ; reallocate it.

mov     word [sysinit_ptr],sysinitptr ; returning address from
mov     [sysinit_ptr+2],cs          ; sysinit_base back to sysinit.
;or     word [install_flag],has_installed ; set the flag.
;or     byte [install_flag],has_installed ; 2
; 11/12/2022
;or     byte [install_flag],2
; 24/10/2022
;or     word [install_flag],2
;
; free the rest of the memory from memhi to confbot. still from confbot to
; the top of the memory will be allocated for sysinit and config.sys if
; have_install_cmd.
; -----
;
call    round
mov     bx,[memhi]
mov     ax,[area]
mov     [old_area],ax          ; save [area]
mov     es,ax                  ; calc what we needed
sub     bx,ax
; 24/10/2022
mov     ah,4Ah ; SETBLOCK
int     21h                    ; give the rest back
; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
; ES = segment address of block to change
; BX = new size in paragraphs

push    es
mov     ax,es
dec     ax
mov     es,ax
;mov    word [es:ARENA.OWNER],8 ; set impossible owner
;mov    word [es:ARENA.NAME],4453h ; System Data
;mov    word [es:ARENA.NAME], 'SD' ; System Data
; 24/10/2022
mov     word [es:1],8          ; set impossible owner
mov     word [es:8], 'SD'      ; System Data
pop     es

mov     bx,0FFFFh
mov     ah,48h ; ALLOC
int     21h
mov     ah,48h ; ALLOC
int     21h                    ; allocate the rest of the memory
; DOS - 2+ - ALLOCATE MEMORY
; BX = number of 16-byte paragraphs desired

```

```

26400 0000126B A3[6403]      mov     [memhi],ax      ; start of the allocated memory
26401 0000126E C706[6203]0000 mov     word [memlo],0    ; to be used next.
26402
26403 ;;;; at this moment,memory from [memhi]:0 to top-of-the memory is
26404 ;;;; allocated.
26405 ;;;; to protect sysinit,confbot module (from confbot (or =alloclim at
26406 ;;;; this time) to the top-of-the memory),here we are going to
26407 ;;;; 1). "setblock" from memhi to confbot.
26408 ;;;; 2). "alloc" from confbot to the top of the memory.
26409 ;;;; 3). "free alloc memory" from memhi to confbot.
26410
26411 ;memory allocation for sysinit,confbot module.
26412
26413 00001274 8EC0      mov     es,ax
26414      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
26415      ; (SYSINIT:11DFh)
26416 00001276 8B1E[A302]  mov     bx,[CONFBOT]
26417      ; 24/10/2022
26418      ;mov     bx,[top_of_cdss] ; mov bx,[confbot]
26419 0000127A 29C3      sub     bx,ax      ; confbot - memhi
26420 0000127C 4B      dec     bx      ; make a room for the memory block id.
26421 0000127D 4B      dec     bx      ; make sure!!!.
26422 0000127E B44A      mov     ah,4Ah ; SETBLOCK
26423 00001280 CD21      int     21h      ; this will free (confbot to top of memory)
26424      ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
26425      ; ES = segment address of block to change
26426      ; BX = new size in paragraphs
26427 00001282 BBFFFF      mov     bx,0FFFFh
26428 00001285 B448      mov     ah,48h ; ALLOC
26429 00001287 CD21      int     21h
26430 00001289 B448      mov     ah,48h ; ALLOC
26431 0000128B CD21      int     21h      ; allocate (confbot to top of memory)
26432      ; DOS - 2+ - ALLOCATE MEMORY
26433      ; BX = number of 16-byte paragraphs desired
26434 0000128D A3[6803]      mov     [area],ax      ; save allocated memory segment.
26435      ; need this to free this area for command.com.
26436 00001290 8E06[6403]  mov     es,[memhi]
26437 00001294 B449      mov     ah,49h      ; free allocated memory.
26438 00001296 CD21      int     21h      ; free (memhi to confbot(=area))
26439      ; DOS - 2+ - FREE MEMORY
26440      ; ES = segment address of area to be freed
26441 endfile_ret:
26442 00001298 C3      retn
26443
26444 ; End of "EndFile" DOS structure configuration.
26445
26446 ; -----
26447 ; 26/03/2019 - Retro DOS v4.0
26448 ; 24/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
26449 ; -----
26450 ; Do_Install_Exec
26451 ;
26452 ; This procedure is used to EXEC a program being loaded via the
26453 ; "install=" mechanism in config.sys. It does this by setting up
26454 ; the parameters, and then jumping to sysinit_base, which has been
26455 ; setup in low memory. When complete, sysinit_base will jump back
26456 ; up to this procedure (if sysinit remains uncorrupted by the installed
26457 ; program).
26458
26459 ;SYSINIT:10CFh:
26460
26461 do_install_exec:      ; now,handles install= command.
26462
26463 00001299 56      push     si      ; save si for config.sys again.
26464
26465 ; we are going to call load/exec function.
26466 ; set es:bx to the parameter block here;;;;;
26467 ; set ds:dx to the asciiz string. remember that we already has 0
26468 ; after the filename. so parameter starts after that. if next
26469 ; character is a line feed (i.e. 10),then assume that the 0
26470 ; we already encountered used to be a carriage return. in this
26471 ; case,let's set the length to 0 which will be followed by
26472 ; carriage return.
26473
26474 ; es:si -> command line in config.sys. points to the first non blank
26475 ; character after =.
26476
26477 0000129A 06      push     es
26478 0000129B 1E      push     ds
26479 0000129C 07      pop      es
26480 0000129D 1F      pop      ds      ; es->sysinitseg,ds->confbot seg
26481 0000129E 89F2      mov     dx,si      ; ds:dx->file name,0 in config.sys image.
26482
26483 000012A0 31C9      xor     cx,cx
26484 000012A2 FC      cld
26485 000012A3 2EC606[F102]20  mov     byte [cs:ldexec_start],' ' ; clear out the parm area
26486 000012A9 BF[F202]  mov     di,ldexec_parm
26487
26488 000012AC AC      installfilename:      ; skip the file name
26489      lodsb      ; al = ds:si; si++
26490      ; 05/09/2023
26491      or     al,al
26492      cmp     al,0
26493      je     short got_installparm
26494      jmp     short installfilename
26495      ; 10/04/2024
26496      jnz     short installfilename
26497 000012B1 AC      got_installparm:      ; copy the parameters to ldexec_parm
26498      lodsb
26499      mov     [es:di],al
26500      cmp     al,1f ; cmp al,0Ah ; line feed?
26501      je     short done_installparm
26502      inc     cl      ; # of char. in the parm.
26503      inc     di
26504      jmp     short got_installparm
26505 000012BE 2E880E[F002]  done_installparm:
26506      mov     byte [cs:ldexec_line],cl ; length of the parm.
26507      ; 05/09/2023
26508      or     cl,cl
26509      cmp     cl,0      ; if no parm,then
26510      jne     short install_seg_set ; let the parm area
26511      mov     byte [cs:ldexec_start],cr ; 0Dh
26512      ; starts with cr.
26513      install_seg_set:
26514      ; 05/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
26515      xor     bx,bx
26516      mov     word [cs:0],0      ; make a null environment segment
26517      mov     [cs:bx],bx ; 05/09/2023
26518      mov     ax,cs      ; by overlap jmp instruction of sysinitseg.
26519
26520 ; -----M067-----
26521 ;
26522 ; the environment pointer is made 0. so the current environment ptr.
26523 ; will be the same as pdb_environ which after dosinit is 0.

```



```

26524      ;      mov      cs:[instexe.exec0_environ],0 ; set the environment seg.
26525      ;
26526      ;      instexe.exec0_environ need not be initialized to 0 above. It was
26527      ;      done as a fix for bug #529. The actual bug was in NLSFUNC and
26528      ;      was fixed.
26529      ;
26530      ; -----
26531      ;
26532      ;;ifdef MULTI_CONFIG
26533      ;
26534      ; If there's any environment data in "config_wrkseg", pass to app
26535      ;
26536      ; 30/12/2022 - Retro DOS v4.0 (Modified MSDOS 6.21 IO.SYS SYSINIT)
26537      ; 24/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS SYSINIT)
26538      ;%if 0
26539      000012D4 89C1      mov      cx,ax ; *
26540      ; 05/09/2023
26541      000012D6 2E391E[6019]      cmp      [cs:config_envlen],bx ; 0
26542      ;cmp      word [cs:config_envlen],0
26543      000012DB 7405      je      short no_envdata2
26544      000012DD 2E8B0E[6219]      mov      cx,[cs:config_wrkseg] ; *
26545      no_envdata2:
26546      ;;endif ;MULTI_CONFIG
26547      ;
26548      ;%endif      ; 24/10/2022
26549      ;
26550      ;mov      [cs:instexe.exec0_environ],cx ; set the environment seg.
26551      ; 05/09/2023 (BugFix)
26552      ; 24/10/2022
26553      000012E2 2E890E[4203]      mov      [cs:iexec.environ],cx ; *
26554      ; 02/11/2022
26555      ;mov      [cs:iexec.environ],ax ; 05/09/2023
26556      ;
26557      ;mov      [cs:instexe.exec0_com_line+2],ax ; set the seg.
26558      000012E7 2EA3[4603]      mov      [cs:iexec.lindex_line+2],ax
26559      ;mov      [cs:instexe.exec0_5c_fcb+2],ax
26560      000012EB 2EA3[4A03]      mov      [cs:iexec.lindex_5c_fcb+2],ax
26561      ;mov      [cs:instexe.exec0_6c_fcb+2],ax
26562      000012EF 2EA3[4E03]      mov      [cs:iexec.lindex_6c_fcb+2],ax
26563      000012F3 E86000      call     sum_up
26564      000012F6 26A3[DA02]      mov      [es:checksum],ax      ; save the value of the sum
26565      000012FA 31C0      xor      ax,ax
26566      000012FC B44B      mov      ah,4Bh ; EXEC      ; load/exec
26567      000012FE BB[4203]      mov      bx,instexe      ; es:bx -> parm block.
26568      00001301 06      push     es      ; save es,ds for load/exec
26569      00001302 1E      push     ds      ; these registers will be restored in sysinit_base.
26570      00001303 2EFF2E[D202]      jmp      far [cs:sysinit_base_ptr] ; jmp to sysinit_base to execute
26571      ; load/exec function and check sum.
26572      ;
26573      ; -----
26574      ;
26575      ;j.k. this is the returning address from sysinit_base.
26576      ;
26577      ; 24/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS SYSINIT)
26578      ;
26579      sysinitptr:      ; returning far address from sysinit_base
26580      00001308 5E      pop      si      ; restore si for config.sys file.
26581      00001309 06      push     es
26582      0000130A 1E      push     ds
26583      0000130B 07      pop      es
26584      0000130C 1F      pop      ds      ; now ds - sysinitseg, es - confbot
26585      0000130D 7305      jnc      short install_exit_ret
26586      ;
26587      0000130F 56      push     si      ; error in loading the file for install=.
26588      00001310 E81737      call     badload      ; es:si-> path,filename,0.
26589      00001313 5E      pop      si
26590      ;
26591      ; 24/10/2022
26592      ;jmp      short sysinitptr_retn ; (MSDOS 5.0 IO.SYS, SYSINIT:1140h)
26593      ; 11/12/2022
26594      ; ds = cs
26595      ;
26596      ; 30/12/2022 - Retro DOS v4.2
26597      ; (MSDOS 6.21 IO.SYS, SYSINIT:1283h)
26598      ;
26599      install_exit_ret:
26600      00001314 C3      retn
26601      ;
26602      ; 30/12/2022 - Retro DOS v4.2
26603      %if 0
26604      install_exit_ret:
26605      ;retn      ; retn (MSDOS 6.21 IO.SYS, SYSINIT:1283h) ; 18/12/2022
26606      ;
26607      ; 24/10/2022 (MSDOS 5.0 IO.SYS SYSINIT)
26608      ;SYSINIT:1142h:
26609      mov      ah,4Dh
26610      int      21h      ; DOS - 2+ - GET EXIT CODE OF SUBPROGRAM (WAIT)
26611      cmp      ah,3
26612      jz      short sysinitptr_retn
26613      call     error_line
26614      stc
26615      sysinitptr_retn:      ; (SYSINIT:114Fh)
26616      retn
26617      ;
26618      %endif ; 24/10/2022
26619      ;
26620      ; -----
26621      ;
26622      ;** ParaRound - Round Up length to paragraph multiple
26623      ;
26624      ; ParaRound rounds a byte count up to a multiple of 16, then divides
26625      ; by 16 yielding a "length in paragraphs" value.
26626      ;
26627      ; ENTRY (ax) = byte length
26628      ; EXIT (ax) = rounded up length in paragraphs
26629      ; USES ax, flags
26630      ;
26631      ParaRound:
26632      00001315 83C00F      add      ax,15
26633      00001318 D1D8      rcr      ax,1
26634      0000131A D1E8      shr      ax,1
26635      0000131C D1E8      shr      ax,1
26636      0000131E D1E8      shr      ax,1
26637      00001320 C3      retn
26638      ;
26639      ; -----
26640      ; sysinit_base module.
26641      ;
26642      ; This module is relocated by the routine EndFile to a location in low
26643      ; memory. It is then called by SYSINIT to perform the EXEC of programs
26644      ; that are being loaded by the "install=" command. After the EXEC call
26645      ; completes, this module performs a checksum on the SYSINIT code (at the
26646      ; top of memory) to be sure that the EXECed program did not damage it.
26647      ; If it did, then this module will print an error message and stop the

```

```

26648 ; system. Otherwise, it returns control to SYSINIT.
26649 ;
26650 ;in: after relocation,
26651 ; ax = 4b00h - load and execute the program dos function.
26652 ; ds = confbot. segment of config.sys file image
26653 ; es = sysinitseg. segment of sysinit module itself.
26654 ; ds:dx = pointer to asciiz string of the path,filename to be executed.
26655 ; es:bx = pointer to a parameter block for load.
26656 ; SI_end (byte) - offset vaule of end of sysinit module label
26657 ; bigsize (word) - # of word from confbot to SI_end.
26658 ; chksum (word) - sum of every byte from confbot to SI_end in a
26659 ; word boundary modular form.
26660 ; sysinit_ptr (dword ptr) - return address to sysinit module.
26661 ;
26662 ;note: sysinit should save necessary registers and when the control is back
26663 ;
26664 ; 24/10/2022
26665 ; (SYSINIT:115Ch for MSDOS 5.0 SYSINIT)
26666 sysinit_base:
26667 00001321 2E8C166200 mov [cs:sysinit_base_ss],ss ; save stack
26668 00001326 2E89266400 mov [cs:sysinit_base_sp],sp
26669 0000132B CD21 int 21h ; load/exec dos call.
26670 0000132D 2E8E166200 mov ss,[cs:sysinit_base_ss] ; restore stack
26671 00001332 2E8B266400 mov sp,[cs:sysinit_base_sp]
26672 00001337 1F pop ds ; restore confbot seg
26673 00001338 07 pop es ; restore sysinitseg
26674 00001339 7216 jc short sysinit_base_end; load/exec function failed.
26675 ; at this time,i don't have to worry about
26676 ; that sysinit module has been broken or not.
26677 0000133B E81800 call sum_up ; otherwise,check if it is good.
26678 0000133E 263906[DA02] cmp [es:checksum],ax
26679 00001343 740C je short sysinit_base_end
26680 ;
26681 ; memory broken. show "memory allocation error" message and stall.
26682 ;
26683 00001345 B409 mov ah,9
26684 00001347 0E push cs
26685 00001348 1F pop ds
26686 ; 30/12/2022
26687 ; (MSDOS 6.21 IO.SYS, SYSINIT:12B8h)
26688 ;mov dx,102
26689 00001349 BA6600 mov dx,mem_alloc_err_msgx-sysinit_base ; 65h (for MSDOS 5.0 SYSINIT)
26690 ; 66h (for MSDOS 6.21 SYSINIT)
26691 0000134C CD21 int 21h
26692 ; DOS - PRINT STRING
26693 ; DS:DX -> string terminated by "$"
26694 ;
26695 ; 30/12/2022 - Retro DOS v4.2
26696 stall:
26697 ; 24/10/2022
26698 _stall:
26699 ; 11/12/2022
26700 0000134E F4 hlt
26701 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
26702 ;hlt ;use HLT to minimize energy consumption
26703 0000134F EBFD jmp short _stall
26704 ;
26705 sysinit_base_end:
26706 00001351 26FF2E[D602] jmp far [es:sysinit_ptr] ;return back to sysinit module
26707 ;
26708 ;-----
26709 sum_up:
26710 ;
26711 ;in: es - sysinitseg.
26712 ;out: ax - result
26713 ;
26714 ;
26715 ;remark: since this routine will only check starting from "locstack" to the end of
26716 ; sysinit segment,the data area, and the current stack area are not
26717 ; covered. in this sense,this check sum routine only gives a minimal
26718 ; gaurantee to be safe.
26719 ;
26720 ;first sum up confbot seg.
26721 ;
26722 00001356 1E push ds
26723 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
26724 ; (SYSINIT:12C6h)
26725 00001357 26A1[A302] mov ax,[es:CONFBOT]
26726 ; 24/10/2022
26727 ;mov ax,[es:top_of_cdss]
26728 0000135B 8ED8 mov ds,ax
26729 0000135D 31F6 xor si,si
26730 0000135F 31C0 xor ax,ax
26731 00001361 268B0E[D002] mov cx,[es:config_size] ; if config_size has been broken,then this
26732 ; whole test better fail.
26733 00001366 D1E9 shr cx,1 ; make it a word count
26734 00001368 7406 jz short sum_sys_code ; when config.sys file not exist.
26735 ;
26736 0000136A 0304 sum1: add ax,[si]
26737 0000136C 46 inc si
26738 0000136D 46 inc si
26739 0000136E E2FA loop sum1
26740 ;now,sum up sysinit module.
26741 sum_sys_code:
26742 ; 24/10/2022
26743 00001370 BE7013 mov si,locstack ;; 5A6h (MSDOS 5.0 IO.SYS, SYSINIT)
26744 ;; 532h (MSDOS 6.21 IO.SYS, SYSINIT)
26745 ; 10/04/2024
26746 ; 586h (PCDOS 7.1 IBMBIO.COM, SYSINIT)
26747 ; starting after the stack. M069
26748 ; this does not cover the possible stack code!!!
26749 ;
26750 ; 02/11/2022
26751 ;mov cx,3D20h ; (15648) for MSDOS 5.0 IO.SYS (SYSINIT)
26752 ; 10/04/2024
26753 ;mov cx,5B40h ; (23360) for PCDOS 7.1 IBMBIO.COM (SYSINIT)
26754 ; 30/12/2022
26755 00001373 B9[1054] mov cx,SI_end ; (22688) ; SI_end is the label at the end of sysinit
26756 00001376 29F1 sub cx,si ; from after_checksum to SI_end
26757 00001378 D1E9 shr cx,1
26758 ;
26759 0000137A 260304 sum2: add ax,[es:si]
26760 0000137D 46 inc si
26761 0000137E 46 inc si
26762 0000137F E2F9 loop sum2
26763 00001381 1F pop ds
26764 00001382 C3 retn
26765 ;
26766 ; 24/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
26767 ; 30/12/2022 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS, SYSINIT)
26768 ; (SYSINIT:12F2h)
26769 ; 10/04/2024 - Retro DOS 5.0 (Modified PCDOS 7.1 IBMBIO.COM, SYSINIT)
26770 ; (SYSINIT:149Dh)
26771

```

```

26772 sysinit_base_ss equ $-sysinit_base ; = 61 (MSDOS 5.0 IO.SYS, SYSINIT:115Ch)
26773 ;SYSINIT:11BDh: ; = 62 (MSDOS 6.21 IO.SYS, SYSINIT:1290h)
26774 ; = 62 (PCDOS 7.1 IBMBIO.COM, SYSINIT:143Bh)
26775 sysinit_base_ssx:
26776 dw 0
26777 sysinit_base_sp equ $-sysinit_base ; = 63 (MSDOS 5.0 IO.SYS, SYSINIT:1161h)
26778 ;SYSINIT:11BFh: ; = 64 (MSDOS 6.21 IO.SYS, SYSINIT:1295h)
26779 ; = 64 (PCDOS 7.1 IBMBIO.COM, SYSINIT:1440h)
26780 sysinit_base_spx:
26781 dw 0
26782
26783 mem_alloc_err_msgx:
26784
26785 ;include msbio.c14 ; memory allocation error message
26786
26787 ;(SYSINIT:12F6h: ; MSDOS 6.21 IO.SYS)
26788 ;SYSINIT:14A1h: ; PCDOS 7.1 IBMBIO.COM
26789 db 0Dh,0Ah
26790 db 'Memory allocation error $'
26791
26792 end_sysinit_base: ; label byte
26793 ; 24/10/2022
26794 ; (SYSINIT:11DCh for MSDOS 5.0 SYSINIT)
26795
26796 ; -----
26797 ; Set_Buffer
26798
26799 ;function: set buffers in the real memory.
26800 ; lastly set the memhi,memlo for the next available free address.
26801
26802 ;input: ds:bx -> buffinfo.
26803 ; [memhi]:[memlo = 0] = available space for the hash bucket.
26804 ; singlebuffersize = buffer header size + sector size
26805
26806 ;output: buffers Queue established.
26807 ; [memhi]:[memlo] = address of the next available free space.
26808 ; -----
26809
26810 ; 25/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
26811 ; (SYSINIT:11DCh)
26812
26813 ; 11/04/2024 - Retro DOS 5.0 (Modified PCDOS 7.1 IBMBIO.COM)
26814 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:14BCh)
26815
26816 set_buffer:
26817 xor di,di ; assume buffers not in HMA
26818 call GetBufferAddr
26819 jz short set_buff_1
26820 mov di,1 ; buffers in HMA
26821
26822 set_buff_1:
26823 ; 25/10/2022
26824 mov [bx+BUFFINF.Buff_Queue],di ; head of Buff Q
26825 mov [bx],di
26826 mov [bx+BUFFINF.Buff_Queue+2],es
26827 mov [bx+2],es
26828 mov word [bx+BUFFINF.Dirty_Buff_Count],0 ; set dirty_count to 0.
26829 mov word [bx+4],0
26830
26831 mov ax,di
26832 mov cx,[cs:buffers]
26833 push di ; remember first buffer
26834
26835 ; for each buffer
26836
26837 nxt_buff:
26838 call set_buffer_info ; set buf_link,buf_id...
26839 mov di,ax
26840 loop nxt_buff
26841
26842 sub di,[cs:singlebuffersize] ; point to last buffer
26843
26844 pop cx ; get first buffer
26845 mov [es:di+buffinfo.buf_next],cx ; last->next = first
26846 mov [es:di],cx
26847 xchg cx,di
26848 mov [es:di+buffinfo.buf_prev],cx ; first->prev = last
26849 ; 25/10/2022
26850 mov [es:di+2],cx
26851
26852 or di,di ; In HMA ?
26853 jz short set_buff_2 ; no
26854 mov byte [bx+BUFFINF.Buff_In_HMA],1
26855 mov ax,[cs:memhi] ; seg of scratch buff
26856 mov word [bx+BUFFINF.Lo_Mem_Buff],0 ; offset of sctarch buff is 0
26857 mov word [bx+13],0
26858 mov [bx+BUFFINF.Lo_Mem_Buff+2],ax
26859 mov word [bx+15],ax
26860 mov ax,[cs:singlebuffersize] ; size of scratch buff
26861 ; 11/04/2024 - Retro DOS v5.0
26862 ; 05/09/2023
26863 ; sub ax,bufinsiz ; 20 ; buffer head not required
26864 ; sub ax,20
26865 sub ax,24 ; bufinsiz ; (bufinsiz is 24 in PCDOS 7.1)
26866
26867 set_buff_2:
26868 add [cs:memlo],ax
26869 or byte [cs:setdevmarkflag],for_devmark ; 2
26870 or byte [cs:setdevmarkflag],2
26871 ; call round
26872 ; retn
26873 ; 12/12/2022
26874 jmp round
26875
26876 ; -----
26877 ; procedure : GetBufferAddr
26878 ;
26879 ; Gets the buffer address either in HMA or in Lo Mem
26880 ;
26881 ; returns in es:di the buffer address
26882 ; returns NZ if allocated in HMA
26883 ; -----
26884
26885 ; 25/10/2022
26886 GetBufferAddr:
26887 push bx
26888 push dx
26889
26890 ; 11/04/2024 - Retro DOS v5.0
26891 ; PCDOS 7.1 IBMBIO.COM
26892 ;;;
26893 cmp byte [cs:dosdata_umb],2

```

```

26894                                     ; is dosdata moved to UMB ? (DOSDATA=UMB done)
26895 00001404 7506                     jne     short gba_1      ; no
26896 00001406 837F02FF                 cmp     word [bx+2],0FFFFh ; is the buffer (already) in HMA ?
26897 0000140A 7423                     je      short gba_2      ; yes
26898 gba_1:
26899 ;;;
26900
26901 0000140C 2EA1[9D02]                 mov     ax, [cs:singlebuffersize]
26902 00001410 2EF726[9902]                 mul     word [cs:buffers]
26903                                     ;add     ax,0Fh
26904 00001415 83C00F                     add     ax,15
26905                                     ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
26906                                     ;and     ax,~15 ; 0FFFF0h ; para round
26907                                     ; 12/12/2022
26908 00001418 24F0                     and     al,~15 ; 0F0h
26909 0000141A 89C3                     mov     bx,ax
26910 0000141C B8024A                     mov     ax,4A02h
26911                                     ;mov     ax,((multMULT<<8)+multMULTALLOCHMA)
26912 0000141F CD2F                     int     2Fh ; DOS 5+ - ALLOCATE HMA SPACE
26913                                     ;         AX = 4A02h
26914                                     ;         BX = number of bytes
26915                                     ; Return:
26916                                     ;         ES:DI -> start of allocated HMA block or FFFFh:FFFFh
26917                                     ;         BX = number of bytes actually allocated
26918                                     ;         (rounded up to next paragraph)
26919                                     ; Notes:
26920                                     ;         this call is not valid unless DOS is loaded in the HMA
26921                                     ;         (DOS=HIGH)
26922
26923 00001421 83FFFF                 cmp     di,0FFFFh
26924 00001424 7506                     jne     short got_hma
26925
26926                                     ;mov     di,0 ; dont xor di,di Z flag needed
26927                                     ; 05/09/2023
26928                                     ; zf=1
26929 00001426 47                     inc     di ; 0FFFFh -> 0
26930                                     ; zf=1
26931
26932                                     ;zf=1
26933                                     ;xor     di,di ; 25/10/2022
26934                                     ;zf=1
26935 00001427 2E8E06[6403]                 mov     es,[cs:memhi]
26936 got_hma:
26937 0000142C 5A                     pop     dx
26938 0000142D 5B                     pop     bx
26939 0000142E C3                     retn
26940
26941                                     ; 11/04/2024 - Retro DOS v5.0
26942                                     ; PCDOS 7.1 IBMBIO.COM
26943                                     ;;;
26944 gba_2:
26945 0000142F C43F                     les     di,[bx]
26946 00001431 09FF                     or      di,di
26947                                     ;pop     dx
26948                                     ;pop     bx
26949                                     ;retn
26950                                     ; 11/04/2024 - Retro DOS v5.0
26951 00001433 EBF7                     jmp     short got_hma
26952 ;;;
26953
26954 ; -----
26955
26956 set_buffer_info:
26957
26958 ;function: set buf_link,buf_id,buf_sector
26959 ;
26960 ;in: es:di -> buffer header to be set.
26961 ;     ax = di
26962 ;
26963 ;out:
26964 ;     above entries set.
26965
26966                                     ; 25/10/2022
26967 00001435 2EFF36[BD02]                 push    word [cs:buf_prev_off]
26968                                     ;pop     word [es:di+buffinfo.buf_prev]
26969 0000143A 268F4502                 pop     word [es:di+2]
26970 0000143E 2EA3[BD02]                 mov     [cs:buf_prev_off],ax
26971 00001442 2E0306[9D02]                 add     ax,[cs:singlebuffersize] ;adjust ax
26972                                     ;mov     [es:di+buffinfo.buf_next],ax
26973 00001447 268905                 mov     [es:di],ax
26974                                     ;mov     word [es:di+buffinfo.buf_ID],00FFh ; new buffer free
26975 0000144A 26C74504FF00                 mov     word [es:di+4],00FFh
26976                                     ;mov     word [es:di+buffinfo.buf_sector],0 ; to compensate the masm 3 bug
26977 00001450 26C745060000                 mov     word [es:di+6],0
26978                                     ;mov     word [es:di+buffinfo.buf_sector+2],0 ; to compensate the masm 3 bug
26979 00001456 26C745080000                 mov     word [es:di+8],0
26980 0000145C C3                     retn
26981
26982 ; =====
26983 ; MSSTACK initialization routine - MSDOS 6.0 - SYSDINIT1.ASM - 1991
26984 ; -----
26985 ; 27/03/2019 - Retro DOS v4.0
26986
26987 ; -----
26988 ; ibmstack initialization routine.
26989 ;
26990 ; to follow the standard interrupt sharing scheme, msstack.asm
26991 ; has been modified. this initialization routine also has to
26992 ; be modified because for the interrupt level 7 and 15, firstflag
26993 ; should be set to signal that this interrupt handler is the
26994 ; first handler hooked to this interrupt vector.
26995 ; we determine this by looking at the instruction pointed by
26996 ; this vector. if it is iret, then this handler should be the
26997 ; first one. in our case, only the interrupt vector 77h is the
26998 ; interrupt level 15. (we don't hook interrupt level 7.)
26999 ;
27000 ; the followings are mainly due to m.r.t; ptm fix of p886 12/3/86
27001 ; some design changes are needed to the above interrupt sharing
27002 ; method. the above sharing scheme assumes that 1). interrupt
27003 ; sharing is never done on levels that have bios support. 2). "phantom"
27004 ; interrupts would only be generated on levels 7 and 15.
27005 ; these assumptions are not true any more. we have to use the firstflag
27006 ; for every level of interrupt. we will set the firstflag on the following
27007 ; conditions:
27008 ;
27009 ; a. if the cs portion of the vector is 0000, then "first"
27010 ; b. else if cs:ip points to valid shared header, then not "first"
27011 ; c. else if cs:ip points to an iret, then "first"
27012 ; d. else if cs:ip points to dummy, then "first"
27013 ;
27014 ; where dummy is - the cs portion must be f000, and the ip portion must
27015 ; be equal to the value at f000:ff01. this location is the initial value
27016 ; from vector table for interrupt 7, one of the preserved addresses in all
27017 ; the bioses for all of the machines.

```

```

27018 ;
27019 ; system design group requests bios to handle the phantom interrupts.
27020 ;
27021 ; the "phantom" interrupt is an illegal interrupt such as an interrupt
27022 ; produced by the bogus adapter card even without interrupt request is
27023 ; set. more specifically, 1). the 8259 has a feature when running in
27024 ; edge triggered mode to latch a pulse and present the interrupt when
27025 ; the processor indicates interrupt acknowledge (inta). the interrupt
27026 ; pulse was exist at the time of inta to get a "phantom" interrupt.
27027 ; 2). or, this is caused by adapter cards placing a glitch on the
27028 ; interrupt line.
27029 ;
27030 ; to handle those "phantom" interrupts, the main stack code will check
27031 ; the own firstflag, and if it is not "first" (which means the forward
27032 ; pointer points to the legal shared interrupt handler), then pass the
27033 ; control. if it is the first, then the following action should be
27034 ; taken. we don't have to implement skack logic in this case.
27035 ;
27036 ; to implement this logic, we rather choose a simple method.
27037 ; if ont of the above "firstflag" conditions is met, we are not
27038 ; going to hook this interrupt vector. the reason is if the original
27039 ; vector points to "iret" and do nothing, we don't need
27040 ; to implement the stack logic for it. this will simplify implementation
27041 ; while maintaining compatibility with the old version of dos.
27042 ; this implies that in the main stack code, there might be a stack code
27043 ; that will never be used, a dead code.
27044 ;
27045 ;in - cs, ds -> sysinitseg, es -> relocated stack code & data.
27046 ;
27047 ; 25/10/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
27048 ; (SYSINIT:1287h)
27049 ;
27050 ; 11/04/2024 - Retro DOS 5.0 (Modified PCDOS 7.1 IBMBIO.COM)
27051 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:157Ch)
27052 ;
27053 ; 14/12/2022
27054 stackinit:
27055     push    ax
27056     push    ds
27057     push    es
27058     push    bx
27059     push    cx
27060     push    dx
27061     push    di
27062     push    si
27063     push    bp
27064 ;
27065 ;currently es -> stack code area
27066 ;
27067 ; 12/12/2022
27068 ; ds = cs
27069     mov     ax,[stack_count]
27070     mov     cx,ax ; *!*
27071 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
27072 ; (MSDOS 5.0 IO.SYS - SYSINIT:1290h)
27073     mov     ax,[cs:stack_count] ; !! ;defined in cs
27074     mov     [es:stackcount],ax ;defined in stack code area
27075 ; (MSDOS 5.0 IO.SYS - SYSINIT:1298h)
27076     mov     ax,[stack_size] ; !! ;in cs
27077     mov     [es:stacksize],ax
27078 ; 12/12/2022
27079     mov     ax,[stack_addr] ; offset
27080 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
27081 ; (MSDOS 5.0 IO.SYS - SYSINIT:129Fh)
27082     mov     ax,[cs:stack_addr] ; !!
27083     mov     [es:stacks],ax
27084 ; 12/12/2022
27085     mov     bp,ax ; *!*
27086     mov     ax,[stack_addr+2]
27087 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
27088 ; (MSDOS 5.0 IO.SYS - SYSINIT:129Fh)
27089     mov     ax,[cs:stack_addr+2] ; !! ; segment
27090     mov     [es:stacks+2],ax
27091 ;
27092 ; initialize the data fields with the parameters
27093 ;
27094 ; "firstentry" will always be at stacks
27095 ;
27096     mov     bp,[es:stacks] ; get offset of stack
27097 ; 12/12/2022
27098     mov     bp = [es:stacks] ; *!*
27099     mov     [es:firstentry],bp
27100 ;
27101 ; the stacks will always immediately follow the table entries
27102 ;
27103     mov     ax,entrysize ; 8
27104     mov     cx,[es:stackcount]
27105 ; 12/12/2022
27106     mov     cx = [es:stackcount] ; *!*
27107     mul     cx
27108     add     ax,bp
27109     mov     [es:stackat],ax
27110     mov     bx,ax
27111     sub     bx,2
27112 ;
27113 ; zero the entire stack area to start with
27114 ;
27115     mov     di,[es:stackat]
27116     mov     ax,[es:stacksize]
27117     mul     cx
27118     mov     cx,ax
27119     xor     ax,ax
27120     push    es
27121     pop     ds ;ds = relocated stack code seg.
27122 ;
27123 ;now, ds -> stack code area
27124 ;
27125     mov     es,[stacks+2] ; get segment of stack area.
27126     cld
27127     rep     stosb
27128 ;
27129     mov     cx,[stackcount]
27130 ;
27131 ; loop for "count" times, building a table entry
27132 ; cs = sysinitseg, ds = relocated stack code seg, es = segment of stack space
27133 ; cx = number of entries
27134 ; es:bp => base of stacks - 2
27135 ; es:bx => first table entry
27136 ;
27137 buildloop:
27138     ; 11/12/2022
27139     mov     byte [es:bp+allocbyte],free ; mov [es:bp+0],0
27140     ; 25/10/2022
27141     mov     byte [es:bp],free

```

```

27142      ; 06/07/2023
27143      mov     [es:bp],al ; 0 ; free
27144      mov     [es:bp+intlevel],al ; ax = 0
27145      ;mov     [es:bp+1],al
27146      mov     [es:bp+savesp],ax
27147      ;mov     [es:bp+2],ax
27148      mov     [es:bp+savesdss],ax
27149      ;mov     [es:bp+4],ax
27150      add     bx,[stacksize]
27151      mov     [es:bp+newsd],bx ; mov [es:bp+6],bx
27152      ;mov     [es:bp+6],bx
27153      mov     [es:bx],bp
27154      add     bp,entrysize ; 8
27155
27156      loop     buildloop
27157
27158      sub     bp,entrysize ; 8
27159      mov     [lastentry],bp
27160      mov     [nextentry],bp
27161
27162      push     ds
27163      ;mov     ax,0F000h ;look at the model byte
27164      ; 05/09/2023
27165      mov     ah,0F0h ; ax = 0F000h
27166      mov     ds,ax
27167      cmp     byte [0FFFFh],0F9h ; mdl_convert ; convertible?
27168      pop     ds
27169      jne     short skip_disablenmis
27170
27171      mov     al,07h ; disable convertible nmis
27172      out     72h,al
27173
27174      skip_disablenmis:
27175      xor     ax,ax
27176      mov     es,ax ;es - segid of vector table at 0
27177      ;ds - relocated stack code segment
27178      cli
27179
27180      ;irp     aa,<02,08,09,70>
27181      ;
27182      ;mov     si,aa&h*4 ;pass where vector is to be adjusted
27183      ;mov     di,offset int19old&aa ;we have to set old&aa for int19 handler too.
27184      ;mov     bx,offset old&aa ;pass where to save original owner pointer
27185      ;mov     dx,offset int&aa ;pass where new handler is
27186      ;call    new_init_loop ;adjust the vector to new handler,
27187      ; ;saving pointer to original owner
27188      ;endm
27189
27190      stkinit_02:
27191      mov     si,02h*4 ; 8
27192      mov     di,INT19OLD02
27193      mov     bx,old02
27194      mov     dx,int02
27195      call    new_init_loop
27196
27197      stkinit_08:
27198      mov     si,08h*4 ; 32
27199      mov     di,INT19OLD08
27200      mov     bx,old08
27201      mov     dx,int08
27202      call    new_init_loop
27203
27204      stkinit_09:
27205      mov     si,09h*4 ; 36
27206      mov     di,INT19OLD09
27207      mov     bx,old09
27208      mov     dx,int09
27209      call    new_init_loop
27210
27211      stkinit_70:
27212      mov     si,70h*4 ; 448
27213      mov     di,INT19OLD70
27214      mov     bx,old70
27215      mov     dx,int70
27216      call    new_init_loop
27217
27218      ;irp     aa,<0a,0b,0c,0d,0e,72,73,74,76,77> ;shared interrupts
27219      ;
27220      ;mov     si,aa&h*4 ;pass where vector is to be adjusted
27221      ;push     ds ;save relocated stack code segment
27222      ;lds     bx, es:[si] ;ds:bx -> original interrupt handler
27223      ;push     ds
27224      ;pop     dx ;dx = segment value
27225      ;
27226      ;cmp     dx,0
27227      ;jz      int&aa&_first
27228      ;
27229      ;cmp     byte ptr ds:[bx],0cfh ;does vector point to an iret?
27230      ;jz      int&aa&_first
27231      ;
27232      ;cmp     word ptr ds:[bx.6],424bh ;magic offset (see int&aa, msstack.inc)
27233      ;jz      int&aa&_not_first
27234      ;
27235      ;cmp     dx,0f000h ;rom bios segment
27236      ;jnz     int&aa&_not_first
27237      ;
27238      ;push     es
27239      ;push     dx
27240      ;mov     dx,0f000h
27241      ;mov     es,dx
27242      ;cmp     bx,word ptr es:0ff01h
27243      ;pop     dx
27244      ;pop     es
27245      ;jz      int&aa&_first
27246      ;
27247      ;int&aa&_not_first: ;not the first. we are going to hook vector.
27248      ;pop     ds
27249      ;mov     di, offset int19old&aa ;we have to set old&aa for int19 handler too.
27250      ;mov     bx, offset old&aa ;pass where to save original owner pointer
27251      ;mov     dx, offset int&aa ;pass where new handler is
27252      ;call    new_init_loop ;adjust the vector to new handler, saving
27253      ; ;pointer to original owner.
27254      ;jmp     short int&aa&_end
27255      ;int&aa&_first: ;the first. don't have to hook stack code.
27256      ;pop     ds
27257      ;int&aa&_end:
27258      ;
27259      ;endm
27260
27261      stkinit_0A:
27262      mov     si,0Ah*4 ; 40
27263
27264      ; 14/12/2022
27265      %if 0
27266      ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27267      push     ds

```



```

27266      lds     bx,[es:si]
27267      push    ds
27268      pop     dx
27269
27270      cmp     dx,0
27271      je      short int_0A_first
27272
27273      cmp     byte [bx],0CFh
27274      je      short int_0A_first
27275
27276      cmp     word [bx+6],424Bh
27277      je      short int_0A_not_first
27278
27279      cmp     dx,0F000h
27280      jne     short int_0A_not_first
27281
27282      push    es
27283      push    dx
27284      mov     dx,0F000h
27285      mov     es,dx
27286      cmp     bx,[es:0FF01h]
27287      pop     dx
27288      pop     es
27289      je      short int_0A_first
27290      %endif
27291
27292      ; 14/12/2022
27293      ; 25/10/2022
27294      00001537 E8EB00      call     int_xx_first_check ; 27/03/2019 - Retro DOS v4.0
27295      0000153A 730C      jnc      short int_0A_first
27296
27297      int_0A_not_first:
27298      ; 14/12/2022
27299      ; 25/10/2022
27300      ;pop     ds
27301      0000153C BF[C205]    mov     di,INT19OLD0A
27302      0000153F BB[5900]    mov     bx,old0A
27303      00001542 BA[5700]    mov     dx,int0A
27304      00001545 E80701      call     new_init_loop
27305
27306      ; 14/12/2022
27307      ;jmp     short int_0A_end
27308      ;int_0A_first:
27309      ; 25/10/2022
27310      ;pop     ds
27311
27312      ; 14/12/2022
27313      int_0A_first:
27314      int_0A_end:
27315
27316      stkinit_0B:
27317      00001548 BE2C00      mov     si,0Bh*4 ; 44
27318
27319      ; 14/12/2022
27320      ; 25/10/2022
27321      0000154B E8D700      call     int_xx_first_check ; 27/03/2019 - Retro DOS v4.0
27322      0000154E 730C      jnc      short int_0B_end ; int_0B_first
27323
27324      ; 14/12/2022
27325      %if 0
27326      ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27327      push    ds
27328      lds     bx,[es:si]
27329      push    ds
27330      pop     dx
27331
27332      cmp     dx,0
27333      je      short int_0B_first
27334
27335      cmp     byte [bx],0CFh
27336      je      short int_0B_first
27337
27338      cmp     word [bx+6],424Bh
27339      je      short int_0B_not_first
27340
27341      cmp     dx,0F000h
27342      jne     short int_0B_not_first
27343
27344      push    es
27345      push    dx
27346      mov     dx,0F000h
27347      mov     es,dx
27348      cmp     bx,[es:0FF01h]
27349      pop     dx
27350      pop     es
27351      je      short int_0B_first
27352      %endif
27353
27354      int_0B_not_first:
27355      ; 14/12/2022
27356      ; 25/10/2022
27357      ;pop     ds
27358      00001550 BF[C705]    mov     di,INT19OLD0B
27359      00001553 BB[7100]    mov     bx,old0B
27360      00001556 BA[6F00]    mov     dx,int0B
27361      00001559 E8F300      call     new_init_loop
27362
27363      ; 14/12/2022
27364      ;jmp     short int_0B_end
27365      ;int_0B_first:
27366      ; 25/10/2022
27367      ;pop     ds
27368
27369      int_0B_end:
27370
27371      stkinit_0C:
27372      0000155C BE3000      mov     si,0Ch*4 ; 48
27373
27374      ; 14/12/2022
27375      ; 25/10/2022
27376      0000155F E8C300      call     int_xx_first_check
27377      00001562 730C      jnc      short int_0C_end ; int_0C_first
27378
27379      ; 14/12/2022
27380      %if 0
27381      ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27382      push    ds
27383      lds     bx,[es:si]
27384      push    ds
27385      pop     dx
27386
27387      cmp     dx,0
27388      je      short int_0C_first
27389

```

```

27390      cmp     byte [bx],0CFh
27391      je      short int_0C_first
27392
27393      cmp     word [bx+6],424Bh
27394      je      short int_0C_not_first
27395
27396      cmp     dx,0F000h
27397      jne     short int_0C_not_first
27398
27399      push    es
27400      push    dx
27401      mov     dx,0F000h
27402      mov     es,dx
27403      cmp     bx,[es:0FF01h]
27404      pop     dx
27405      pop     es
27406      je      short int_0C_first
27407 %endif
27408
27409 int_0C_not_first:
27410     ; 14/12/2022
27411     ; 25/10/2022
27412     ;pop     ds
27413 00001564 BF[CC05]    mov     di,INT19OLD0C
27414 00001567 BB[8900]    mov     bx,old0C
27415 0000156A BA[8700]    mov     dx,int0C
27416 0000156D E8DF00     call    new_init_loop
27417
27418     ; 14/12/2022
27419     ;jmp     short int_0C_end
27420 ;int_0C_first:
27421     ; 25/10/2022
27422     ;pop     ds
27423
27424 int_0C_end:
27425
27426 stkinit_0D:
27427 00001570 BE3400      mov     si,0Dh*4 ; 52
27428
27429     ; 14/12/2022
27430     ; 25/10/2022
27431 00001573 E8AF00      call    int_xx_first_check
27432 00001576 730C       jnc     short int_0D_end ; int_0D_first
27433
27434 ; 14/12/2022
27435 %if 0
27436     ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27437     push    ds
27438     lds     bx,[es:si]
27439     push    ds
27440     pop     dx
27441
27442     cmp     dx,0
27443     je      short int_0D_first
27444
27445     cmp     byte [bx],0CFh
27446     je      short int_0D_first
27447
27448     cmp     word [bx+6],424Bh
27449     je      short int_0D_not_first
27450
27451     cmp     dx,0F000h
27452     jne     short int_0D_not_first
27453
27454     push    es
27455     push    dx
27456     mov     dx,0F000h
27457     mov     es,dx
27458     cmp     bx,[es:0FF01h]
27459     pop     dx
27460     pop     es
27461     je      short int_0D_first
27462 %endif
27463
27464 int_0D_not_first:
27465     ; 14/12/2022
27466     ; 25/10/2022
27467     ;pop     ds
27468 00001578 BF[D105]    mov     di,INT19OLD0D
27469 0000157B BB[A100]    mov     bx,old0D
27470 0000157E BA[9F00]    mov     dx,int0D
27471 00001581 E8CB00     call    new_init_loop
27472
27473     ; 14/12/2022
27474     ;jmp     short int_0D_end
27475     ; 02/11/2022
27476 ;int_0D_first:
27477     ;pop     ds
27478
27479 int_0D_end:
27480
27481 stkinit_0E:
27482 00001584 BE3800      mov     si,0Eh*4 ; 56
27483
27484     ; 14/12/2022
27485     ; 25/10/2022
27486 00001587 E89B00      call    int_xx_first_check
27487 0000158A 730C       jnc     short int_0E_end ; int_0E_first
27488
27489 ; 14/12/2022
27490 %if 0
27491     ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27492     push    ds
27493     lds     bx,[es:si]
27494     push    ds
27495     pop     dx
27496
27497     cmp     dx,0
27498     je      short int_0E_first
27499
27500     cmp     byte [bx],0CFh
27501     je      short int_0E_first
27502
27503     cmp     word [bx+6],424Bh
27504     je      short int_0E_not_first
27505
27506     cmp     dx,0F000h
27507     jne     short int_0E_not_first
27508
27509     push    es
27510     push    dx
27511     mov     dx,0F000h
27512     mov     es,dx
27513     cmp     bx,[es:0FF01h]

```



```

27514         pop     dx
27515         pop     es
27516         je      short int_0E_first
27517     %endif
27518
27519     int_0E_not_first:
27520         ; 14/12/2022
27521         ; 25/10/2022
27522         ; pop     ds
27523     0000158C BF[D605]    mov     di,INT19OLD0E
27524     0000158F BB[B900]    mov     bx,old0E
27525     00001592 BA[B700]    mov     dx,int0E
27526     00001595 E8B700     call    new_init_loop
27527
27528         ; 14/12/2022
27529         ; jmp     short int_0E_end
27530     ;int_0E_first:
27531         ; 25/10/2022
27532         ; pop     ds
27533
27534     int_0E_end:
27535
27536     stkinit_72:
27537     00001598 BEC801     mov     si,72h*4 ; 456
27538
27539         ; 14/12/2022
27540         ; 25/10/2022
27541     0000159B E88700     call    int_xx_first_check
27542     0000159E 730C     jnc     short int_72_end ; int_72_first
27543
27544     ; 14/12/2022
27545     %if 0
27546         ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27547         push     ds
27548         lds     bx,[es:si]
27549         push     ds
27550         pop      dx
27551
27552         cmp     dx,0
27553         je      short int_72_first
27554
27555         cmp     byte [bx],0CFh
27556         je      short int_72_first
27557
27558         cmp     word [bx+6],424Bh
27559         je      short int_72_not_first
27560
27561         cmp     dx,0F000h
27562         jne     short int_72_not_first
27563
27564         push     es
27565         push     dx
27566         mov     dx,0F000h
27567         mov     es,dx
27568         cmp     bx,[es:0FF01h]
27569         pop      dx
27570         pop      es
27571         je      short int_72_first
27572     %endif
27573
27574     int_72_not_first:
27575         ; 14/12/2022
27576         ; 25/10/2022
27577         ; pop     ds
27578     000015A0 BF[E005]    mov     di,INT19OLD72
27579     000015A3 BB[D100]    mov     bx,old72
27580     000015A6 BA[CF00]    mov     dx,int72
27581     000015A9 E8A300     call    new_init_loop
27582
27583         ; 14/12/2022
27584         ; jmp     short int_72_end
27585     ;int_72_first:
27586         ; 25/10/2022
27587         ; pop     ds
27588
27589     int_72_end:
27590
27591     stkinit_73:
27592     000015AC BECC01     mov     si,73h*4 ; 460
27593
27594         ; 14/12/2022
27595         ; 25/10/2022
27596     000015AF E87300     call    int_xx_first_check
27597     000015B2 730C     jnc     short int_73_end ; int_73_first
27598
27599     ; 14/12/2022
27600     %if 0
27601         ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27602         push     ds
27603         lds     bx,[es:si]
27604         push     ds
27605         pop      dx
27606
27607         cmp     dx,0
27608         je      short int_73_first
27609
27610         cmp     byte [bx],0CFh
27611         je      short int_73_first
27612
27613         cmp     word [bx+6],424Bh
27614         je      short int_73_not_first
27615
27616         cmp     dx,0F000h
27617         jne     short int_73_not_first
27618
27619         push     es
27620         push     dx
27621         mov     dx,0F000h
27622         mov     es,dx
27623         cmp     bx,[es:0FF01h]
27624         pop      dx
27625         pop      es
27626         je      short int_73_first
27627     %endif
27628
27629     int_73_not_first:
27630         ; 14/12/2022
27631         ; 25/10/2022
27632         ; pop     ds
27633     000015B4 BF[E505]    mov     di,INT19OLD73
27634     000015B7 BB[E900]    mov     bx,old73
27635     000015BA BA[E700]    mov     dx,int73
27636     000015BD E88F00     call    new_init_loop
27637

```

```

27638             ; 14/12/2022
27639             ; jmp     short int_73_end
27640 ;int_73_first:
27641             ; 25/10/2022
27642             ; pop     ds
27643
27644 int_73_end:
27645
27646 stkinit_74:
27647 000015C0 BED001
27648             mov     si,74h*4 ; 464
27649
27650             ; 14/12/2022
27651             ; 25/10/2022
27652 000015C3 E85F00
27653             call    int_xx_first_check
27654 000015C6 730C
27655             jnc     short int_74_end ; int_74_first
27656
27657 ; 14/12/2022
27658 %if 0
27659             ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27660             push    ds
27661             lds     bx,[es:si]
27662             push    ds
27663             pop     dx
27664
27665             cmp     dx,0
27666             je      short int_74_first
27667
27668             cmp     byte [bx],0CFh
27669             je      short int_74_first
27670
27671             cmp     word [bx+6],424Bh
27672             je      short int_74_not_first
27673
27674             cmp     dx,0F000h
27675             jne     short int_74_not_first
27676
27677             push    es
27678             push    dx
27679             mov     dx,0F000h
27680             mov     es,dx
27681             cmp     bx,[es:0FF01h]
27682             pop     dx
27683             pop     es
27684             je      short int_74_first
27685 %endif
27686
27687 int_74_not_first:
27688             ; 14/12/2022
27689             ; 25/10/2022
27690             ; pop     ds
27691             mov     di,INT19OLD74
27692             mov     bx,old74
27693             mov     dx,int74
27694             call    new_init_loop
27695
27696             ; 14/12/2022
27697             ; jmp     short int_74_end
27698 ;int_74_first:
27699             ; 25/10/2022
27700             ; pop     ds
27701
27702 int_74_end:
27703
27704 stkinit_76:
27705 000015D4 BED801
27706             mov     si,76h*4 ; 472
27707
27708             ; 14/12/2022
27709             ; 25/10/2022
27710 000015D7 E84B00
27711 000015DA 730E
27712             call    int_xx_first_check
27713             jnc     short int_76_end ; int_76_first
27714
27715 ; 14/12/2022
27716 %if 0
27717             ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27718             push    ds
27719             lds     bx,[es:si]
27720             push    ds
27721             pop     dx
27722
27723             cmp     dx,0
27724             je      short int_76_first
27725
27726             cmp     byte [bx],0CFh
27727             je      short int_76_first
27728
27729             cmp     word [bx+6],424Bh
27730             je      short int_76_not_first
27731
27732             cmp     dx,0F000h
27733             jne     short int_76_not_first
27734
27735             push    es
27736             push    dx
27737             mov     dx,0F000h
27738             mov     es,dx
27739             cmp     bx,[es:0FF01h]
27740             pop     dx
27741             pop     es
27742             je      short int_76_first
27743 %endif
27744
27745 int_76_not_first:
27746             ; 14/12/2022
27747             ; 25/10/2022
27748             ; pop     ds
27749             mov     di,INT19OLD76
27750             mov     bx,old76
27751             mov     dx,int76
27752             call    new_init_loop
27753
27754             ; 14/12/2022
27755             ; jmp     short int_76_end
27756 ;int_76_first:
27757             ; 25/10/2022
27758             ; pop     ds
27759
27760 int_76_end:
27761
27762 stkinit_77:
27763 000015EA BEDC01
27764             mov     si,77h*4 ; 476
27765
27766             ; 14/12/2022
27767             ; 25/10/2022
27768 000015ED E83500
27769             call    int_xx_first_check

```

```

27762 000015F0 730C          jnc     short int_77_end ; int_77_first
27763
27764 ; 14/12/2022
27765 %if 0
27766 ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
27767     push    ds
27768     lds     bx,[es:si]
27769     push    ds
27770     pop     dx
27771
27772     cmp     dx,0
27773     je      short int_77_first
27774
27775     cmp     byte [bx],0CFh
27776     je      short int_77_first
27777
27778     cmp     word [bx+6],424Bh
27779     je      short int_77_not_first
27780
27781     cmp     dx,0F000h
27782     jne     short int_77_not_first
27783
27784     push    es
27785     push    dx
27786     mov     dx,0F000h
27787     mov     es,dx
27788     cmp     bx,[es:0FF01h]
27789     pop     dx
27790     pop     es
27791     je      short int_77_first
27792 %endif
27793
27794 int_77_not_first:
27795 ; 14/12/2022
27796 ; 25/10/2022
27797     ;pop     ds
27798 000015F2 BF[F405]      mov     di,INT19OLD77
27799 000015F5 BB[3101]      mov     bx,old77
27800 000015F8 BA[2F01]      mov     dx,int77
27801 000015FB E85100       call    new_init_loop
27802
27803 ; 14/12/2022
27804 ;jmp     short int_77_end
27805 ;int_77_first:
27806 ; 25/10/2022
27807 ;pop     ds
27808
27809 int_77_end:
27810     push    ds
27811 000015FF B800F0       mov     ax,0F000h          ; look at the model byte
27812 00001602 8ED8       mov     ds,ax
27813 00001604 803EFEFF9   cmp     byte [0FFFEh],0F9h ; mdl_convert ; pc convertible?
27814 00001609 1F         pop     ds
27815 0000160A 7504       jne     short skip_enablenmis
27816
27817 0000160C B027       mov     al,27h          ; enable convertible nmis
27818 0000160E E672       out     72h,al
27819
27820 ; 25/10/2022
27821 ; (MSDOS 5.0 SYSINIT:15FBh)
27822
27823 skip_enablenmis:
27824 00001610 FB         sti
27825 ;;mov     ax,Bios_Data ; 70h
27826 ;mov     ax,KERNEL_SEGMENT ; 70h
27827 ; 21/10/2022
27828 00001611 B87000      mov     ax,DOSBIODATASEG ; 0070h
27829 00001614 8ED8       mov     ds,ax
27830
27831 ;mov     [640h],1 ; SYSINIT:1736h for MSDOS 6.21 IO.SYS
27832
27833 00001616 C606[B105]01 mov     byte [INT19SEM],1 ; indicate that int 19
27834 ; initialization is complete
27835
27836 0000161B 5D         pop     bp          ; restore all
27837 0000161C 5E         pop     si
27838 0000161D 5F         pop     di
27839 0000161E 5A         pop     dx
27840 0000161F 59         pop     cx
27841 00001620 5B         pop     bx
27842 00001621 07         pop     es
27843 00001622 1F         pop     ds
27844 00001623 58         pop     ax
27845 00001624 C3         retn
27846
27847 ; 14/12/2022
27848 ; -----
27849
27850 ; 14/12/2022
27851 ; 25/10/2022
27852 %if 0
27853 ; 27/03/2019 - Retro DOS v4.0
27854 int_xx_first_check:
27855 00001625 1E         push    ds
27856 00001626 26C51C      lds     bx,[es:si]
27857 00001629 1E         push    ds
27858 0000162A 5A         pop     dx
27859
27860 ;cmp     dx,0
27861 ;je      short int_xx_first
27862 ; 05/09/2023
27863 0000162B 21D2      and     dx,dx
27864 0000162D 741E      jz      short int_xx_first
27865
27866 0000162F 803FCF      cmp     byte [bx],0CFh
27867 00001632 7419      je      short int_xx_first
27868
27869 00001634 817F064B42     cmp     word [bx+6],424Bh
27870 00001639 7411      je      short int_xx_not_first
27871
27872 0000163B 81FA00F0     cmp     dx,0F000h
27873 0000163F 750B      jne     short int_xx_not_first
27874
27875 00001641 06         push    es
27876 ;push    dx
27877 ;mov     dx,0F000h
27878 00001642 8EC2      mov     es,dx
27879 00001644 263B1E01FF   cmp     bx,[es:0FF01h]
27880 ;pop     dx
27881 00001649 07         pop     es
27882 0000164A 7401      je      short int_xx_first
27883
27884 int_xx_not_first:
27885 0000164C F9         stc

```

```

27886 int_xx_first:
27887 0000164D 1F      pop     ds
27888 0000164E C3      retn
27889
27890 ;%endif
27891
27892 ; -----
27893 ; 27/03/2019 - Retro DOS v4.0
27894
27895 ; 25/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
27896 ; (SYSINIT:1610h)
27897
27898 ; 13/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM, SYSINIT)
27899 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1905h)
27900
27901 new_init_loop:
27902 ;;; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
27903 0000164F 2E803E[6E03]02  cmp     byte [cs:dosdata_umb],2
27904 ;;; is DOSDATA=UMB done ? (DOSDATA is in UMB)
27905 00001655 7510      jne     short new_init_loop_1st
27906 00001657 1E      push    ds
27907 ; restore original/previous interrupt handler
27908 ; (from int19old?? field in BIOSDATA)
27909 00001658 B87000    ;mov     ax,70h
27910 0000165B 8ED8      mov     ax,DOSBIODATASEG
27911 0000165D C505      mov     ds,ax
27912 0000165F 268904    lds     ax,[di] ; restore original Int ?? handler addr from int19old?? field
27913 00001662 268C5C02  mov     [es:si],ax ; copy the original int handler addr to its int vector addr
27914 00001666 1F      mov     [es:si+2],ds
27915 pop     ds
27916 new_init_loop_1st:
27917 ;;;
27918 ;input: si=offset into vector table of the particular int vector being adjusted
27919 ; bx=ds:offset of oldxx, where will be saved the pointer to original owner
27920 ; dx=ds:offset of intxx, the new interrupt handler
27921 ; di=offset value of int19old&aa variable in bios.
27922 ; es=zero, segid of vector table
27923 ; ds=relocated stack code segment
27924
27925 ; 13/04/2024
27926 %if 0
27927 mov     ax,[es:si] ;remember offset in vector
27928 mov     [bx],ax ; to original owner in ds
27929 mov     ax,[es:si+2] ;remember segid in vector
27930 mov     [bx+2],ax ; to original owner in ds
27931
27932 push    ds
27933 ;mov     ax,Bios_Data ; 70h
27934 ;mov     ax,KERNEL_SEGMENT ; 70h
27935 ; 21/10/2022
27936 mov     ax,DOSBIODATASEG ; 0070h
27937 mov     ds,ax ;set int19oldxx value in bios for
27938 mov     ax,[es:si] ;int 19 handler
27939 mov     [di],ax
27940 mov     ax,[es:si+2]
27941 mov     [di+2],ax
27942 pop     ds
27943 %else
27944 ; 13/04/2024 - Retro DOS v5.0
27945 00001667 1E      push    ds
27946 00001668 268B4402  mov     ax,[es:si+2] ;remember segid in vector
27947 0000166C 894702    mov     [bx+2],ax ; to original owner in ds
27948 0000166F 50      push    ax
27949 00001670 268B04    mov     ax,[es:si] ;remember offset in vector
27950 00001673 8907      mov     [bx],ax ; to original owner in ds
27951 00001675 50      push    ax
27952 00001676 B87000    mov     ax,DOSBIODATASEG ; 0070h
27953 00001679 8ED8      mov     ds,ax ;set int19oldxx value in bios for
27954 0000167B 58      pop     ax ;int 19 handler
27955 0000167C 8905      mov     [di],ax
27956 0000167E 58      pop     ax
27957 0000167F 894502    mov     [di+2],ax
27958 00001682 1F      pop     ds
27959 %endif
27960 00001683 268914    mov     [es:si],dx ;set vector to point to new int handler
27961 00001686 268C5C02  mov     [es:si+2],ds
27962 0000168A C3      retn
27963
27964 ; End of STACK initialization routine
27965 ; -----
27966
27967 ; -----
27968 ;set the devmark for mem command.
27969 ;in: [memhi] - the address to place devmark
27970 ; [memlo] = 0
27971 ; al = id for devmark_id
27972 ;out: devmark established.
27973 ; the address saved in cs:[devmark_addr]
27974 ; [memhi] increase by 1.
27975 ; -----
27976
27977 ; 25/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS, SYSINIT)
27978 ; (SYSINIT:1637h)
27979 ; 04/09/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS, SYSINIT)
27980 ; (SYSINIT:176Ch)
27981
27982 ; 04/09/2023 - PCDOS 7.1 - IBMBIO.COM (SYSINIT:1944h)
27983
27984 setdevmark:
27985 ; 04/09/2023
27986 ;push    es
27987 ;push    cx
27988
27989 0000168B 2E8B0E[6403]  mov     cx,[cs:memhi]
27990 00001690 2E890E[6719]  mov     [cs:devmark_addr],cx
27991 00001695 8EC1      mov     es,cx
27992 ; 25/10/2022
27993 ;mov     [es:devmark.id],al
27994 ;mov     [es:0],al
27995 00001697 26A20000  inc     cx
27996 0000169B 41      ;mov     [es:devmark.seg],cx
27997 ;mov     [es:1],cx
27998 0000169C 26890E0100
27999
28000 ; 04/09/2023
28001 ;pop     cx
28002 ;pop     es
28003
28004 000016A1 2EFF06[6403]  inc     word [cs:memhi]
28005 000016A6 C3      retn
28006
28007 ; -----
28008 ; SYSPRE.ASM - MSDOS 6.0 - 1992
28009 ; -----

```

```

28010 ;; pre-load and final placement of dblspace.bin
28011 ;
28012 ; 08/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
28013 ; =====
28014 ;
28015 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1964h)
28016 ;;; -----
28017 000016A7 [AB16] MagicDDNamePtr: dw MagicDDName ; "\DBLSPACE.BIN"
28018 000016A9 433A db 'C:'
28019 000016AB 5C44424C5350414345- MagicDDName: db '\DBLSPACE.BIN',0
28020 000016B4 2E42494E00
28021 000016B8 433A5C535441434B45- StackerName: db 'C:\STACKER.BIN',0
28022 000016C2 522E42494E00
28023 000016C8 FFFF tiny_stub_start:
28024 000016CA FFFF dw 0FFFFh ; phony device driver link
28025 000016CC 0080 dw 0FFFFh ; dw -1, -1
28026 000016CE 00000000 dw 8000h ; mark as character device for MEM display
28027 000016D2 44424C5342494E24 dw 2 dup(0) ; strategy and interrupt
28028 db 'DBLSBIN$' ; magic default load
28029 ; (tiny_stub_end-tiny_stub_start = 18)
28030 ; ===== S U B R O U T I N E =====
28031 ; 08/04/2024 - Retro DOS v5.0
28032 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1997h)
28033 ;
28034 ;***MagicPreload - pre-load dblspace.bin
28035 ;
28036 ; EXIT ax = error code, 00 means none.
28037 ; ZF = true if ax == 0
28038 ;
28039 MagicPreload:
28040 ; 13/04/2024 - Retro DOS v5.0
28041 ; ds = cs
28042 ;mov byte [cs:setdevmarkflag],0 ; not for devmark
28043 000016DA C606[6919]00 mov byte [setdevmarkflag],0
28044 000016DF E80E31 call round
28045 000016E2 0E push cs
28046 000016E3 07 pop es
28047 ;mov byte [cs:DeviceHi],0 ; not to be loaded in UMB
28048 ; 13/04/2024
28049 000016E4 C606[4F24]00 mov byte [DeviceHi],0
28050 000016E9 E8771E call InitDevLoad ; set up sub-arena, DevLoadAddr,
28051 ; DevLoadEnd, and DevEntry
28052 ; gets arena name from bpb_addr
28053 ; 13/04/2024
28054 ; ds = cs
28055 ;
28056 ; check to make sure device driver fits our available space.
28057 ;
28058 ;mov ax,[cs:DevLoadAddr]
28059 000016EC A1[3324] mov ax,[DevLoadAddr]
28060 ;add ax,[cs:DevSize] ; calculate seg after DD load
28061 000016EF 0306[3124] add ax,[DevSize]
28062 000016F3 725E jc short pre_exit_err ; choke if overflows address space
28063 ;cmp ax,[cs:DevLoadEnd] ; does it overflow available space?
28064 000016F5 3B06[3524] cmp ax,[DevLoadEnd]
28065 000016F9 7758 ja short pre_exit_err
28066 ;
28067 _LoadDev: ; we're golden if not
28068 ; 13/04/2024
28069 ; ds = cs
28070 ;push cs
28071 ;pop ds
28072 ;mov dx,[cs:MagicDDNamePtr]
28073 000016FB 8B16[A716] mov dx,[MagicDDNamePtr]
28074 000016FF E8A21F call ExecDev ; load device driver using exec call
28075 00001702 724F jb short pre_exit_err
28076 ;
28077 ; 13/04/2024
28078 ; ds = cs
28079 ;les bx,[cs:DevEntry] ; point to the Magic DD header
28080 00001704 C41E[3724] les bx,[DevEntry]
28081 00001708 26817F122C2E cmp word [es:bx+12h],2E2Ch; is it our stamp? ; ',.'
28082 0000170E 7543 jnz short pre_exit_err
28083 ;mov word [cs:MagicBackdoor],14h ; save the backdoor entry.
28084 ; ; (initial IP -EXE header offset 20-)
28085 ;mov [cs:MagicBackdoor+2],es
28086 00001710 C706[9003]1400 mov word [MagicBackdoor],14h
28087 00001716 8C06[9203] mov [MagicBackdoor+2],es
28088 ;
28089 0000171A 0E push cs
28090 0000171B 07 pop es
28091 0000171C BB[6F03] mov bx,packet
28092 ;
28093 ;mov word [cs:break_addr],0
28094 ;mov ax,[cs:DevLoadEnd]
28095 ;mov [cs:break_addr+2],ax
28096 ;mov al,[cs:drivenumber] ; pass drive number to DBLSPACE as if
28097 ;mov [cs:devdrivenum],al ; it is a normal block device driver
28098 0000171F C706[7D03]0000 mov word [break_addr],0
28099 00001725 A1[3524] mov ax,[DevLoadEnd]
28100 00001728 A3[7F03] mov [break_addr+2],ax
28101 0000172B A0[8503] mov al,[drivenumber] ; pass drive number to DBLSPACE as if
28102 0000172E A2[8503] mov [devdrivenum],al ; it is a normal block device driver
28103 ;
28104 00001731 B80A00 mov ax,10 ; DS_INTERNAL_REVISION
28105 ; tell it what revision we expect
28106 ;call far [cs:MagicBackdoor]; first time call is init entry point
28107 00001734 FF1E[9003] call far [MagicBackdoor]
28108 ; with a standard device driver
28109 ; init packet at es:bx
28110 00001738 731D jnb short no_driver_version_fail ; skip if not a version failure
28111 0000173A B80600 mov ax,6 ; DS_INTERNAL_REVISION_6 ; (Stacker ?)
28112 ; tell it what revision we expect
28113 ;call far [cs:MagicBackdoor]
28114 0000173D FF1E[9003] call far [MagicBackdoor]
28115 00001741 7314 jnb short no_driver_version_fail
28116 ;
28117 ; In this case, we're going to display a message
28118 ;
28119 ;push cs
28120 ;pop ds
28121 ; 13/04/2024
28122 ; ds = cs
28123 00001743 BA[D853] mov dx,baddblspace ; "Required system component is not instal"...
28124 00001746 E80933 call print ; display the message
28125 ;
28126 ; point backdoor call back to safe far return
28127 ;
28128 fail_driver_load:
28129 ;mov [cs:MagicBackdoor+2],cs
28130 ;mov word [cs:MagicBackdoor],NullBackdoor
28131 00001749 8C0E[9203] mov [MagicBackdoor+2],cs

```

```

28132 0000174D C706[9003][9403]      mov     word [MagicBackdoor],NullBackdoor
28133                                pre_exit_err:
28134 00001753 B84000      mov     ax,40h                ; SYSPRE_BADFILE_ERROR
28135                                ; (problem loading dblspace.bin)
28136 00001756 C3          retn
28137
28138                                no_driver_version_fail:
28139 00001757 09C0      or     ax,ax                ; error code returned?
28140 00001759 75EE      jnz     short fail_driver_load
28141
28142                                magic_is_resident:
28143                                ; 13/04/2024
28144                                ; ds = cs
28145                                ;mov     ax,[cs:break_addr]
28146 0000175B A1[7D03]    mov     ax,[break_addr]
28147 0000175E E8B4FB      call    ParaRound                ; convert to paragraphs
28148                                ;add     ax,[cs:break_addr+2]
28149                                ;mov     [cs:DevBrkAddr+2],ax
28150                                ;mov     word [cs:DevBrkAddr],0; store normalized end here
28151 00001761 0306[7F03]    add     ax,[break_addr+2]
28152 00001765 A3[3D24]    mov     [DevBrkAddr+2],ax
28153 00001768 C706[3B24]0000 mov     word [DevBrkAddr],0
28154 0000176E BB0400      mov     bx,4                ; inquire how many paragraphs it wants
28155                                ;call    far [cs:MagicBackdoor]
28156 00001771 FF1E[9003]    call    far [MagicBackdoor]
28157                                ;mov     bx,[cs:ALLOCLIM]                ; get top of free memory
28158 00001775 8B1E[A502]    mov     bx,[ALLOCLIM]
28159 00001779 29C3      sub     bx,ax                ; see how much we'll lower it
28160                                ;cmp     bx,[cs:DevBrkAddr+2]                ; is there that much room free?
28161 0000177B 3B1E[3D24]    cmp     bx,[DevBrkAddr+2]
28162 0000177F 7212      jb     short cant_move_driver
28163                                ;;sub     [cs:ALLOCLIM],ax                ; (mov [cs:ALLOCLIM],bx)
28164                                ;mov     [cs:ALLOCLIM],bx ; Retro DOS v5.0 ; 08/04/2024
28165                                ; 13/04/2024
28166 00001781 891E[A502]    mov     [ALLOCLIM],bx
28167                                ;mov     es,[cs:ALLOCLIM]
28168 00001785 8E06[A502]    mov     es,[ALLOCLIM]
28169 00001789 BB0600      mov     bx,6                ; tell the driver to move itself
28170                                ;call    far [cs:MagicBackdoor]
28171 0000178C FF1E[9003]    call    far [MagicBackdoor]
28172                                ;mov     [cs:DevBrkAddr+2],ax                ; save end of low stub
28173 00001790 A3[3D24]    mov     [DevBrkAddr+2],ax                ; save end of low stub
28174
28175                                cant_move_driver:
28176                                ;mov     ax,[cs:DevBrkAddr+2]                ; get terminate segment
28177 00001793 A1[3D24]    mov     ax,[DevBrkAddr+2]
28178                                ;cmp     ax,[cs:DevLoadEnd]                ; terminate size TOO big?
28179 00001796 3B06[3524]    cmp     ax,[DevLoadEnd]
28180 0000179A 77B7      ja     short pre_exit_err                ; error out if so
28181
28182                                ;----- deal with block device drivers
28183
28184                                _isblock:                                ; if no units found,erase the device
28185                                ; 13/04/2024
28186                                ; ds = cs
28187                                ;mov     al,[cs:unitcount]
28188 0000179C A0[7C03]    mov     al,[unitcount]
28189 0000179F 08C0      or     al,al
28190 000017A1 74B0      jz     short pre_exit_err
28191 000017A3 30E4      xor     ah,ah
28192                                ;lds     si,[cs:DevEntry]                ; set ds:si to header
28193 000017A5 C536[3724]    lds     si,[DevEntry]
28194 000017A9 88440A      mov     [si+10],al                ; mov [si+SYSDEV.NAME],al
28195                                ; number of units in name field
28196                                ; device drivers are *supposed*
28197                                ; to do this for themselves.
28198 000017AC 89C1      mov     cx,ax
28199 000017AE 2EC43E[6D02] les     di,[cs:DOSINFO]                ; es:di point to dos info
28200 000017B3 268A6520 mov     ah,[es:di+20h]                ; [es:di+SYSI_NUMIO]
28201                                ; get number of devices
28202 000017B7 88E2      mov     dl,ah
28203 000017B9 00C4      add     ah,al                ; check for too many devices
28204 000017BB 80FC1A      cmp     ah,26                ; 'A' - 'Z' is 26 devices
28205 000017BE 7793      ja     short pre_exit_err
28206 000017C0 2E800E[6919]02 or     byte [cs:setdevmarkflag],2
28207 000017C6 E83B1F      call    DevSetBreak
28208                                ;jnc     short _ok_block
28209                                ;jmp     pre_exit_err
28210                                ; 13/04/2024
28211 000017C9 7288      jc     short pre_exit_err ; ds <> cs
28212
28213                                _ok_block:
28214 000017CB 26886520 mov     [es:di+20h],ah                ; [es:di+SYSI_NUMIO] ; update the amount
28215
28216 000017CF 2EC51E[8103]    lds     bx,[cs:bpb_addr]                ; point to bpb array (*)
28217 000017D4 30F6      xor     dh,dh
28218
28219                                _perunit:
28220 000017DB 2EC42E[6D02] les     bp,[cs:DOSINFO]
28221 000017DB 26C46E00 les     bp,[es:bp+0]                ; [es:bp.sysi_dpb]
28222                                ; get first dpb
28223                                ; [es:bp+SysInitvars.SYSI_DPB] ; [es:bp+0]
28224
28225 000017DF 26837E19FF _scandpb: cmp     word [es:bp+19h],0FFFFh ; -1 ; [es:bp.dpb_next_dpb]
28226 000017E4 7406      jz     short _foundpb
28227 000017E6 26C46E19 les     bp,[es:bp+19h]                ; les bp,[es:bp.dpb_next_dpb]
28228                                ; [es:bp+DPB.NEXT_DPB]
28229 000017EA EBF3      jmp     short _scandpb
28230
28231                                ; We've found the end of the DPB chain. Now extend it.
28232
28233                                _foundpb:
28234 000017EC 2EA1[3B24]    mov     ax,[cs:DevBrkAddr]
28235 000017F0 26894619 mov     [es:bp+19h],ax                ; [es:bp.dpb_next_dpb] ; DPB.NEXT_DPB
28236 000017F4 2EA1[3D24]    mov     ax,[cs:DevBrkAddr+2]
28237 000017F8 2689461B mov     [es:bp+18h],ax                ; [es:bp.dpb_next_dpb+2] ; DPB.NEXT_DPB+2
28238 000017FC 2EC42E[3B24]    les     bp,[cs:DevBrkAddr]
28239 00001801 26C74619FFFF mov     word [es:bp+19h],0FFFFh ; -1
28240 00001807 26C64618FF mov     byte [es:bp+18h],0FFh ; [es:bp.dpb_first_access],-1
28241                                ; DPB.FIRST_ACCESS
28242 0000180C 2E8306[3B24]3D add     word [cs:DevBrkAddr],61 ; DPBSIZ ; 3Dh
28243 00001812 E8CE1E      call    RoundBreakAddr
28244 00001815 8B37      mov     si,[bx]                ; ds:si points to bpb (*)
28245                                ; (mov si,[bx] ..and then.. add bx,2)
28246                                ; Note: If unit count > 1,bx points to a BPB in the BPB array,
28247                                ; the array address is in [bpb_addr] (*)
28248                                ; Erdogan Tan - 07/07/2023
28249 00001817 26885600 mov     [es:bp+0],dl                ; mov word [es:bp.dpb_drive],dx
28250                                ; [es:bp+DPB.DRIVE],dl
28251 0000181B 26887601 mov     [es:bp+1],dh                ; [es:bp+DPB.UNIT],dh
28252 0000181F 52      dx
28253 00001820 51      push cx
28254 00001821 BA5241      mov     dx,4152h                ; DX = signature 4152h ('AR') for FAT32 extended BPB/DPB
28255 00001824 31C9      xor     cx,cx                ; 0

```



```

28256 00001826 26894E1D      mov     [es:bp+10h],cx      ; DPB.NEXT_FREE ; last allocated cluster #
28257 0000182A 394C0B      cmp     [si+0Bh],cx        ; BPB.fatsecs16 ; [si+A_BPB.BPB_SECTORS PER FAT]
28258 0000182D 7514      jnz     short _setdpb      ; FAT DPB (33 bytes)
28259                                     ; FAT32 DPB (61 bytes)
28260 0000182F 26894E39      mov     [es:bp+39h],cx      ; DPB.FAT32_NXTFREE = 0
28261 00001833 26894E3B      mov     [es:bp+3Bh],cx      ; DPB.FAT32_NXTFREE+2 = 0
28262 00001837 49          dec     cx                  ; 0FFFFh ; -1
28263 00001838 26894E1F      mov     [es:bp+1Fh],cx      ; DPB.FREE_CNT (-1 = unknown)
28264 0000183C 26894E21      mov     [es:bp+21h],cx      ; DPB.FREE_CNT+2 (-1 = unknown)
28265 00001840 B95845      mov     cx,4558h           ; CX = signature 4558h ('EX') for FAT32 extended BPB/DPB
28266
28267 _setdpb:
28268 00001843 B453      mov     ah,53h             ; SETDPB ; hidden system call
28269 00001845 CD21      int     21h               ; DOS - 2+ internal - TRANSLATE BIOS PARAMETER BLOCK
28270                                     ; DS:SI -> BPB (BIOS Parameter Block)
28271                                     ; ES:BP -> buffer for DOS Drive Parameter Block
28272                                     ; (if CX=4558h & DX=4152h,FAT32 Extended DPB will be set)
28273 00001847 59          pop     cx
28274 00001848 5A          pop     dx
28275 00001849 268B4602     mov     ax,[es:bp+2]        ; [es:bp.dpb_sector_size] ; [es:bp+DPB.SECTOR_SIZE]
28276 0000184D 06          push    es
28277 0000184E 2EC43E[6D02] les     di,[cs:DOSINFO]
28278 00001853 263B4510     cmp     ax,[es:di+10h]      ; [es:di.sysi_maxsec] ; [es:di+SysInitvars.SYSI_MAXSEC]
28279 00001857 07          pop     es
28280 00001858 7604      jbe     short _iblk_1
28281                                     ; 13/04/2024
28282                                     ; ds <= cs
28283 0000185A B84000     mov     ax,40h             ; SYSPRE_BADFILE_ERROR ; (pre_exit_err)
28284                                     ; (problem loading dblspace.bin)
28285 0000185D C3          retn
28286
28287 _iblk_1:
28288 0000185E 1E          push    ds
28289 0000185F 2EC506[3724] lds     ax,[cs:DevEntry]
28290 00001864 26894613     mov     [es:bp+13h],ax      ; [es:bp+DPB.DRIVER_ADDR]
28291 00001868 268C5E15     mov     [es:bp+15h],ds      ; [es:bp+DPB.DRIVER_ADDR+2]
28292 0000186C 1F          pop     ds
28293 0000186D FEC2      inc     dl                 ; increment drive number
28294 0000186F FEC6      inc     dh                 ; increment unit number
28295 00001871 43          inc     bx
28296 00001872 43          inc     bx
28297                                     ; point to next BPB
28298 00001873 49          dec     cx                 ; (in the BPB array) (*) -add bx,2-
28299 00001874 7403      jz      short _linkit      ; loop _foundpb
28300 00001876 E973FF     jmp     _foundpb
28301
28302 _linkit:
28303 00001879 0E          push    cs
28304 0000187A 1F          pop     ds
28305 0000187B E825F5     call    TempCDS             ; set cds for new drives
28306                                     ; 13/04/2024
28307                                     ; (DS may not be same with CS here)
28308 0000187E 2EC43E[6D02] les     di,[cs:DOSINFO]      ; es:di = dos table (SysInitvars)
28309 00001883 268B4522     mov     ax,[es:di+22h]      ; [es:di+SYSI_DEV] ; dx:cx = head of list
28310 00001887 268B5D24     mov     bx,[es:di+24h]      ; [es:di+SYSI_DEV+2]
28311 00001888 2EC536[3724] lds     si,[cs:DevEntry]     ; ds:si = device location
28312 00001890 8904      mov     [si],ax            ; link in the driver
28313 00001892 895C02     mov     [si+2],bx
28314 00001895 26897522     mov     [es:di+22h],si      ; [es:di+SYSI_DEV] ; set head of list in dos
28315 00001899 268C5D24     mov     [es:di+24h],ds      ; [es:di+SYSI_DEV+2]
28316 0000189D E8861E     call    DevBreak           ; mark successful install
28317                                     ; 13/04/2024
28318                                     ; ds = cs
28319                                     ; mov     cx,[cs:DevBrkAddr+2] ; pass it a work buffer
28320                                     ; mov     dx,[cs:ALLOCLIM]     ; address in cx (segment)
28321 000018A0 8B0E[3D24] mov     cx,[cs:DevBrkAddr+2]
28322 000018A4 8B16[A502] mov     dx,[ALLOCLIM]
28323 000018A8 29CA      sub     dx,cx              ; for len dx (paragraphs)
28324 000018AA B80055     mov     ax,5500h           ; we're shuffle aware,but don't move
28325                                     ; any drives at this point.
28326 000018AD BB0200     mov     bx,2               ; switch what we can now
28327                                     ; call    far [cs:MagicBackdoor]
28328 000018B0 FF1E[9003] call    far [MagicBackdoor]
28329
28330 000018B4 31C0     pre_exit: xor     ax,ax              ; no errors!
28331 no_magic: ; 13/04/2024          ; zf=1
28332 000018B6 C3          retn
28333
28334 ; ===== S U B R O U T I N E =====
28335
28336 ; 08/04/2024 - Retro DOS v5.0
28337 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1B9Fh)
28338
28339 ;***MagicPostload -- called to clean up and make sure Magic is final placed
28340
28341 MagicPostload:
28342 ; 13/04/3024
28343 ; ds = cs
28344 000018B7 E89C00     call    get_dblspace_version ; is it there?
28345 000018BA 75FA      jnz     short no_magic      ; done if not
28346 000018BC F7C20080     test    dx,8000h           ; is it already permanent?
28347 000018C0 74F4      jz      short no_magic      ; no,done if so (not in final position)
28348 000018C2 BBFFFF     mov     bx,0FFFFh ; -1     ; how much space does it want?
28349 000018C5 B8114A     mov     ax,4A11h           ; multMagicdrv
28350                                     ; DBLSPACE.BIN - GET RELOCATION SIZE
28351 000018C8 CD2F      int     2Fh               ; get paragraphs into ax
28352 000018CA 40          inc     ax                 ; extra 2 paragraphs for the stub
28353 000018CB 40          inc     ax                 ; ((tiny_stub_end-tiny_stub_start)+15)/16
28354                                     ; (18+15)/16 = 2
28355                                     ; mov     [cs:DevSize],ax      ; store that (**)
28356                                     ; mov     byte [cs:DeviceHi],0 ; not to be loaded in UMB
28357                                     ; mov     [cs:bbp_addr+2],cs   ; pass name so that
28358                                     ; ; arena header can be set.
28359                                     ; mov     word [cs:bbp_addr],MagicDDName ; "\DBLSPACE.BIN"
28360                                     ; 13/04/2024
28361                                     ; ds = cs
28362 000018CC A3[3124]     mov     [DevSize],ax
28363 000018CF C606[4F24]00 mov     byte [DeviceHi],0
28364 000018D4 8C0E[8303] mov     [bbp_addr+2],cs
28365 000018D8 C706[8103][AB16] mov     word [bbp_addr],MagicDDName
28366
28367 000018DE E80F2F     call    round              ; normalize memhi:memlo
28368 000018E1 E87F1C     call    InitDevLoad        ; set up sub-arena,DevLoadAddr,
28369                                     ; DevLoadEnd,and DevEntry
28370                                     ; gets arena name from bbp_addr
28371                                     ; 13/04/2024
28372                                     ; ds = cs
28373                                     ; mov     es,[cs:DevLoadAddr] ; (**) (InitDevload sets this)
28374 000018E4 8E06[3324] mov     es,[DevLoadAddr]
28375
28376 ; First, move a little header in place so that this looks
28377 ; to the mem command like a legitimate driver load. Otherwise,
28378 ; it will display garbage for the device name
28379

```

```

28380 000018E8 31FF          xor     di,di          ; move a little header in place
28381                                     ; so that this looks to the mem command
28382                                     ; like a legitimate driver load
28383 000018EA BE[C816]        mov     si,tiny_stub_start
28384                                     ; (tiny_stub_end-tiny_stub_start)
28385 000018ED B91200        mov     cx,18
28386 000018F0 F3A4          rep movsb          ; move it!
28387 000018F2 8CC0          mov     ax,es          ; advance es appropriately
28388 000018F4 40            inc     ax
28389 000018F5 40            inc     ax
28390 000018F6 8EC0          mov     es,ax
28391 000018F8 BBFEFF        mov     bx,0FFFEh ; -2          ; final placement!
28392 000018FB B8114A        mov     ax,4A11h      ; multMagicdrv
28393 000018FE CD2F          int     2Fh           ; DBLSPACE.BIN - RELOCATE
28394                                     ; es = segment to which to relocate DBLSPACE.BIN
28395                                     ; mov ax,[cs:DevLoadAddr] ; (*)
28396                                     ; add ax,[cs:DevSize]          ; calculate seg after DD load
28397                                     ; mov [cs:DevBrkAddr+2],ax ; save as ending address!
28398                                     ; mov word [cs:DevBrkAddr],0
28399                                     ; 13/04/2024
28400                                     ; ds = cs
28401 00001900 A1[3324]        mov     ax,[DevLoadAddr]
28402 00001903 0306[3124]    add     ax,[DevSize]
28403 00001907 A3[3D24]        mov     [DevBrkAddr+2],ax
28404 0000190A C706[3B24]0000 mov     word [DevBrkAddr],0
28405
28406 00001910 E8F11D        call    DevSetBreak      ; go ahead and alloc mem for device
28407                                     ; call DevBreak
28408 ;no_magic:
28409 ;retn
28410 ; 13/04/2024 - Retro DOS v5.0
28411 00001913 E9101E        jmp     DevBreak
28412
28413 ; ===== S U B R O U T I N E =====
28414
28415 ; 08/04/2024 - Retro DOS v5.0
28416 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1C08h)
28417
28418 ;***MagicSetCdss -- disable CDSS for still unmounted DbLspace drives
28419 ;
28420 ; entry:
28421 ; CDSS are now persistent and in their final place
28422
28423 MagicSetCdss:
28424 ; 13/04/2024 - Retro DOS v5.0
28425 ; ds = cs
28426 00001916 E83D00        call    get_dbLspace_version ; is it there?
28427 00001919 753A          jnz     short magic_set_exit ; done if not
28428                                     ; cl = first DbLspace drive in ASCII
28429                                     ; ch = number of DbLspace drive letters
28430 0000191B 2EC536[6D02]    lds     si,[cs:DOSINFO]      ; point to DOS data area (SysInitVars)
28431 00001920 C57416        lds     si,[si+16h]          ; lds si,[si+SYSI_CDSS] ; fetch CDSS
28432 00001923 B458          mov     ah,88              ; curdirLen
28433 00001925 80E941        sub     cl,'A'              ; make it zero based.
28434 00001928 88C8          mov     al,cl              ; get first DbLspace drive letter
28435 0000192A F6E4          mul     ah                 ; find first DbLspace CDS
28436 0000192C 01C6          add     si,ax               ; cds pointer
28437 0000192E 88CA          mov     dl,cl              ; save for drive testing loop
28438 00001930 88E9          mov     cl,ch              ; get DbLspace drive count into cx
28439 00001932 30ED          xor     ch,ch
28440
28441 ; We know cx > 0, or else the driver wouldn't have stayed resident
28442
28443 magic_set_cdss_1:
28444 00001934 51            push    cx
28445 00001935 52            push    dx
28446 00001936 1E            push    ds
28447 00001937 56            push    si
28448 00001938 B8114A        mov     ax,4A11h          ; multMagicdrv
28449 0000193B B80100        mov     bx,1              ; MD_DRIVE_MAP ; inquire drive map
28450 0000193E CD2F          int     2Fh              ; DBLSPACE.BIN - "GetDriveMapping"
28451                                     ; see if this is an unused DbLspace drive
28452 00001940 5E            pop     si
28453 00001941 1F            pop     ds
28454 00001942 5A            pop     dx
28455 00001943 59            pop     cx
28456 00001944 38DA        cmp     dl,b1              ; if mapped to itself,it is vacant
28457 00001946 7504          jnz     short magic_set_cdss_2 ; skip if used
28458 00001948 806444BF    and     byte [si+44h],0BFh ; Retro DOS v5.0 ; 08/04/2024
28459                                     ; and word [si+43h],0BFFFh
28460                                     ; reset the bit in flags (curdir_inuse bit)
28461                                     ; [si+curdir_list.cdLr_flags],~curdir_inuse ; word
28462                                     ; [.. [si+1+curdir_list.cdLr_flags],0BFh ; byte)
28463
28464 0000194C 83C658        add     si,88              ; curdirLen
28465 0000194F FEC2          inc     dl                ; next drive
28466 00001951 E2E1          loop   magic_set_cdss_1
28467                                     ; 13/04/2024
28468 00001953 0E            push    cs
28469 00001954 1F            pop     ds
28470
28471 00001955 C3            retn
28472
28473 ; ===== S U B R O U T I N E =====
28474
28475 ; 08/04/2024 - Retro DOS v5.0
28476 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1C47h)
28477
28478 get_dbLspace_version:
28479 00001956 B8114A        mov     ax,4A11h          ; multMagicdrv
28480                                     ; DBLSPACE.BIN - "GetVersion" - INSTALLATION CHECK
28481                                     ; (BX = 0)
28482 00001959 31DB          xor     bx,bx              ; MD_VERSION = 0
28483 0000195B CD2F          int     2Fh              ; Return:
28484                                     ; AX = 0000h (successful)
28485                                     ; BX = 444Dh ("DM")
28486                                     ; CL = first drive letter used by DBLSPACE (41h = A:)
28487                                     ; CH = number of drive letters used by DBLSPACE
28488                                     ; DX = internal DBLSPACE.BIN version number (bits 14-0)
28489                                     ; bit 15 set if DBLSPACE.BIN has not yet been relocated
28490                                     ; to final position in memory (i.e. DBLSPACE.SYS /MOVE)
28491 0000195D 09C0          or      ax,ax              ; ax = 0 (successful,zf=1)
28492 0000195F C3            retn
28493
28494 ; -----
28495 ; SYSCONF.ASM - MSDOS 6.0 - 1991
28496 ; -----
28497 ; 27/03/2019 - Retro DOS v4.0
28498
28499 ;MULTI_CONFIG equ 1
28500
28501 HIGH_FIRST equ 080h          ; from ARENA.INC - modifier for
28502                                     ; allocation strategy call
28503

```



```

28504 ;have_install_cmd equ 00000001b ; config.sys has install= commands
28505 ;has_installed equ 00000010b ; sysinit_base installed.
28506
28507 default_filenum equ 8
28508
28509 ;stacksw equ true ; include switchable hardware stacks
28510
28511 ; external variable defined in ibmbio module for multi-track
28512
28513 ;multtrk_on equ 10000000b ;user spcified mutitrack=on,or system turns
28514 ; it on after handling config.sys file as a
28515 ; default value,if multtrk_flag = multtrk_off1.
28516 ;multtrk_off1 equ 00000000b ;initial value. no "multitrack=" command entered.
28517 ;multtrk_off2 equ 00000001b ;user specified multitrack=off.
28518
28519 ; if stacksw
28520
28521 ; internal stack parameters
28522
28523 ;entrysize equ 8
28524
28525 ;mincount equ 8
28526 ;defaultcount equ 9
28527 ;maxcount equ 64
28528
28529 ;minsize equ 32
28530 ;defaultsize equ 128
28531 ;maxsize equ 512
28532
28533 DOS_FLAG_OFFSET equ 86h
28534
28535 ;ifdef MULTI_CONFIG
28536 ;
28537 ; config_envlen must immediately precede config_wrkseg, because they
28538 ; may be loaded as a dword ptr
28539
28540 ; 30/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
28541 ; 25/10/2022
28542 00001960 0000 config_envlen: dw 0 ; when config_wrkseg is being used as
28543 ; a scratch env, this is its length
28544 00001962 0000 config_wrkseg: dw 0 ; config work area (above confbot)
28545 ; segment of work area
28546
28547 00001964 00 config_cmd: db 0 ; current config cmd
28548 ; (with CONFIG_OPTION_QUERY bit intact)
28549 00001965 00 config_multi: db 0 ; non-zero if multi-config config.sys
28550
28551 ;endif ; MULTI_CONFIG
28552
28553 00001966 00 multdeviceflag: db 0
28554
28555 00001967 0000 devmark_addr: dw 0 ;segment address for devmark.
28556
28557 00001969 00 setdevmarkflag: db 0 ;flag used for devmark
28558
28559 ; 30/12/2022
28560 ; 12/12/2022
28561 0000196A 00 driver_units: db 0 ;total unitcount for driver
28562
28563 ; 12/12/2022
28564 ;ems_stub_installed:
28565 ; db 0
28566
28567 ; 12/12/2022
28568 ;align 2
28569
28570 badparm_ptr: ; label dword
28571 0000196B 0000 badparm_off: dw 0
28572 0000196D 0000 badparm_seg: dw 0
28573
28574 ;*****
28575 ;take care of config.sys file.
28576 ;system parser data and code.
28577 ;*****
28578
28579 ;*****
28580 ; parser options set for msbio sysconf module
28581 ;*****
28582
28583 ;*** default assemble swiches definition *****
28584
28585 ;farsw equ 0 ; near call expected
28586 ;datesw equ 0 ; check date format
28587 ;timesw equ 0 ; check time format
28588 ;filesw equ 1 ; check file specification
28589 ;capsw equ 0 ; perform caps if specified
28590 ;cmpxsw equ 0 ; check complex list
28591 ;numsw equ 1 ; check numeric value
28592 ;keysw equ 0 ; support keywords
28593 ;swsw equ 1 ; support switches
28594 ;val1sw equ 1 ; support value definition 1
28595 ;val2sw equ 0 ; support value definition 2
28596 ;val3sw equ 1 ; support value definition 3
28597 ;drvsw equ 1 ; support drive only format
28598 ;qussw equ 0 ; support quoted string format
28599
28600 ; psdata_seg equ cs
28601
28602 ;.xlist
28603 ;include parse.asm ;together with psdata.inc
28604 ;.list
28605
28606 ; PSDATA.INC - MSDOS 6.0 - 1991
28607 ; =====
28608 ; 27/03/2019 - Retro DOS v4.0
28609
28610 ; 30/03/2019
28611 ; VERSION.INC (MSDOS 6.0)
28612 ; Set DBCS Blank constant
28613
28614 ; ifndef DBCS
28615 DB_SPACE EQU 2020h
28616 DB_SP_HI EQU 20h
28617 DB_SP_LO EQU 20h
28618 ; else
28619
28620 ;*****
28621 ; Parser include file
28622 ;*****
28623
28624 ;**** Equation field
28625 ;----- Character code definition
28626
28627 _$P_DBSP1 equ DB_SP_HI ;AN000; 1st byte of DBCS blank

```

```

28628 _$P_DBS2 equ DB_SP_LO ;AN000; 2nd byte of DBCS blank
28629 _$P_Period equ "." ;AN020;
28630 _$P_Slash equ "/" ;AN020;
28631 _$P_Space equ " " ;AN000; SBCS blank
28632 _$P_Comma equ "," ;AN000;
28633 _$P_Switch equ "/" ;AN000;
28634 _$P_Keyword equ "=" ;AN000;
28635 _$P_Colon equ ":" ;AN000;
28636 _$P_Plus equ "+" ;AN000;
28637 _$P_Minus equ "-" ;AN000;
28638 _$P_Rparen equ ")" ;AN000;
28639 _$P_Lparen equ "(" ;AN000;
28640 _$P_SQuote equ "'" ;AN025; deleted
28641 _$P_DQuote equ "" ;AN000;
28642 _$P_NULL equ 0 ;AN000;
28643 _$P_TAB equ 9 ;AN000;
28644 _$P_CR equ 0bh ;AN000;
28645 _$P_LF equ 0Ah ;AN000;
28646 _$P_ASCII80 equ 80h ;AN000; ASCII 80h character code
28647
28648 ;----- Masks
28649 _$P_Make_Lower equ 20h ;AN000; make lower case character
28650 _$P_Make_Upper equ 0FFh-_$P_Make_Lower ;AN000; make upper case character
28651
28652 ;----- DOS function call related equs
28653
28654 _$P_DOS_Get_CDI equ 3800h ;AN000; get country dependent information
28655 ; by this call, following information
28656
28657 00000000 ????.
28658 00000002 ??????????.
28659 00000007 ????.
28660 00000009 ????.
28661 0000000B ????.
28662 0000000D ????.
28663 0000000F ??
28664 00000010 ??
28665 00000011 ??
28666 00000012 ?????????.
28667 00000016 ????.
28668 00000018 <res Ah>
28669
28670 .size:
28671 endstruc
28672
28673 _$P_Date_MDY equ 0 ;AN000;
28674 _$P_Date_DMY equ 1 ;AN000;
28675 _$P_Date_YMD equ 2 ;AN000;
28676
28677 _$P_DOS_GetEV equ 6300h ;AN000; get DBCS EV call
28678 ;AN000; DS:SI will points to DBCS EV
28679
28680 _$P_DOS_Get_TBL equ 65h ;AN000; get uppercase table call
28681 ;AN000; following parameters are set
28682 ;AN000; to get casemap table.
28683 _$P_DOSTBL_Def equ -1 ;AN000; get default
28684 _$P_DOSTBL_BL equ 5 ;AN000; buffer length for Tbl pointer
28685 _$P_DOSTBL_File equ 4 ;AN000; get file uppercase table
28686 _$P_DOSTBL_Char equ 2 ;AN000; get character uppercase table
28687 ; By this call following information
28688 ; is returned.
28689 00000000 ??
28690 00000001 ????.
28691 00000003 ????.
28692
28693 endstruc
28694
28695 ; -----
28696 ; PARMS LABEL BYTE
28697 ; DW PARMSX
28698 ; DB 2 ; NUMBER OF STRINGS (0, 1, 2)
28699 ; DB length ; LENGTH OF THE NEXT LIST, 0 IF NONE
28700 ; DB " .. " ; EXTRA DELIMITER LIST,
28701 ; ; TYPICAL ARE ";", "=",
28702 ; ; "& WHITESPACE ALWAYS
28703 ; DB length ; LENGTH OF THE NEXT LIST, 0 IF NONE
28704 ; DB " .. " ; EXTRA END OF LINE LIST, CR, LF OR 0 ALWAYS
28705 ; -----
28706
28707 ;----- PARMS block structure
28708 struc _$P_PARMS_Blk
28709 .PARMSX_Address: resw 1 ;AN000; Address of PARMSX
28710 .Num_Extra: resb 1 ;AN000; Number of extra stuff
28711 .Len_Extra_Delim: resb 1 ;AN000; Length of extra delimiter
28712 endstruc
28713
28714 _$P_Len_PARMS equ 4 ;AN000;
28715 _$P_I_Use_Default equ 0 ;AN000; no extra stuff specified
28716 _$P_I_Have_Delim equ 1 ;AN000; extra delimiter specified
28717 _$P_I_Have_EOL equ 2 ;AN000; extra EOL specified
28718
28719 ; -----
28720 ; PARMSX LABEL BYTE
28721 ; DB minp,maxp ; MIN, MAX POSITIONAL OPERANDS ALLOWED
28722 ; ; DESCRIPTION OF POSITIONAL 1
28723 ; : REPEATS maxp-1 TIMES
28724 ; DB maxs
28725 ; DW CONTROL ; # OF SWITCHES
28726 ; : DESCRIPTION OF SWITCH 1
28727 ; DB maxk
28728 ; DW CONTROL ; REPEATS maxs-1 TIMES
28729 ; : # OF KEYWORD
28730 ; : DESCRIPTION OF KEYWORD 1
28731 ; : REPEATS maxk-1 TIMES
28732
28733 ;----- PARMSX block structure
28734 struc _$P_PARMSX_Blk
28735 .MinP: resb 1 ;AN000; Minimum positional number
28736 .MaxP: resb 1 ;AN000; Maximum positional number
28737 .1st_Control: resw 1 ;AN000; Address of the 1st CONTROL block
28738 endstruc
28739
28740 ; -----
28741 ; << Control field definition >>
28742
28743 ; CONTROL LABEL BYTE
28744 ; DW MATCH_FLAGS ; CONTROLS TYPE MATCHED
28745 ; ; 8000H=NUMERIC VALUE, (VALUE LIST WILL BE CHECKED)
28746 ; ; 4000H=SIGNED NUMERIC VALUE (VALUE LIST WILL BE CHECKED)
28747 ; ; 2000H=SIMPLE STRING(VALUE LIST WILL BE CHECKED)
28748 ; ; 1000H=DATE STRING (VALUE LIST WON'T BE CHECKED)
28749 ; ; 0800H=TIME STRING (VALUE LIST WON'T BE CHECKED)
28750 ; ; 0400H=COMPLEX LIST (VALUE LIST WON'T BE CHECKED)
28751 ; ; 0200H=FILE SPEC (VALUE LIST WON'T BE CHECKED)
28752 ; ; 0100H=DRIVE ONLY (VALUE LIST WON'T BE CHECKED)

```

```

28752 ; ; 0080H=QUOTED STRING (VALUE LIST WON'T BE CHECKED)
28753 ; ; 0010H=IGNORE ":" AT END IN MATCH
28754 ; ; 0002H=REPEATS ALLOWED
28755 ; ; 0001H=OPTIONAL
28756 ;
28757 ; DW FUNCTION_FLAGS ;
28758 ; ; 0001H=CAP RESULT BY FILE TABLE
28759 ; ; 0002H=CAP RESULT BY CHAR TABLE
28760 ; ; 0010H=REMOVE ":" AT END
28761 ; (tm10) ; 0020H=colon is not necessary for switch
28762 ; DW RESULT ; RESULT BUFFER
28763 ; DW VALUES ; VALUE LISTS
28764 ; DB nld ; NUMBER OF KEYWORD/SWITCH SYNONYMS IN FOLLOWING LIST
28765 ; DB "...",0 ; IF n > 0, KEYWORD 1
28766 ;
28767 ;
28768 ; Note:
28769 ; - The MATCH_FLAG is bit significant. You can set, for example, TIME bit and
28770 ; DATE bit simalteniously.
28771 ;
28772 ; The parser examines each bit along with the following priority.
28773 ;
28774 ; COMPLEX -> DATE -> TIME -> NUMERIC VAL -> SIGNED NUMERIC VAL -> DRIVE ->
28775 ; FILE SPEC -> SIMPLE STRING.
28776 ;
28777 ; - When the FUNCTION_FLAG is 0001 or 0002, the STRING pointed to by a pointer
28778 ; in the result buffer is capitalized.
28779 ;
28780 ; - Match_Flags 0001H and 0002H have meaning only for the positional.
28781 ;
28782 ; - The "...",0 (bottom most line) does require '=' or '/'. When you need a
28783 ; switch, for example, '/A', then STRING points to;
28784 ;
28785 ; DB 1 ; number of following synonyms
28786 ; DB '/A',0
28787 ;
28788 ; when you need a keyword, for example, 'CODEPAGE=', then "...",0 will be;
28789 ;
28790 ; DB 1 ; number of following synonyms
28791 ; DB 'CODEPAGE=',0
28792 ;
28793 ; - "...",0 must consist of upper case characters only because the parser
28794 ; performs pattern matching after converting input to upper case (by
28795 ; using the current country upper case table)
28796 ;
28797 ; - One "...",0 can contain only one switch or keyword. If you need, for
28798 ; example /A and /B, the format will be;
28799 ;
28800 ; DB 2 ; number of following synonyms
28801 ; DB '/A',0
28802 ; DB '/B',0
28803 ; -----
28804 ;
28805 ; **** Match_Flags
28806 ;
28807 ; _P_Num_Val equ 8000h ;AN000; Numeric value
28808 ; _P_SNum_Val equ 4000h ;AN000; Signed numeric value
28809 ; _P_Simple_S equ 2000h ;AN000; Simple string
28810 ; _P_Date_S equ 1000h ;AN000; Date string
28811 ; _P_Time_S equ 0800h ;AN000; Time string
28812 ; _P_Cmpx_S equ 0400h ;AN000; Complex string
28813 ; _P_File_Spc equ 0200h ;AN000; File Spec
28814 ; _P_Drv_Only equ 0100h ;AN000; Drive Only
28815 ; _P_Qu_String equ 0080h ;AN000; Quoted string
28816 ; _P_Ig_Colon equ 0010h ;AN000; Ignore colon at end in match
28817 ; _P_Repeat equ 0002h ;AN000; Repeat allowed
28818 ; _P_Optional equ 0001h ;AN000; Optional
28819 ;
28820 ; **** Function flags
28821 ;
28822 ; _P_CAP_File equ 0001h ;AN000; CAP result by file table
28823 ; _P_CAP_Char equ 0002h ;AN000; CAP result by character table
28824 ; _P_Rm_Colon equ 0010h ;AN000; Remove ":" at the end
28825 ; _P_colon_is_not_necessary equ 0020h ;AN000; (tm10) /+10 and /+:10
28826 ;
28827 ; ----- Control block structure
28828 ;
28829 ; struct _P_Control_Blk
28830 ; .Match_Flag: resw 1 ;AN000; Controls type matched
28831 ; .Function_Flag: resw 1 ;AN000; Function should be taken
28832 ; .Result_Buf: resw 1 ; Result buffer address
28833 ; .Value_List: resw 1 ;AN000; Value list address
28834 ; .nid: resb 1 ;AN000; # of keyword/SW synonyms
28835 ; .KEYorSW: resb 1 ;AN000; keyword or sw
28836 ; endstruct
28837 ;
28838 ; -----
28839 ; << Value List Definition >>
28840 ;
28841 ; VALUES LABEL BYTE
28842 ; DB nval ; NUMBER OF VALUE DEFINITIONS (0 - 3)
28843 ;
28844 ; +-
28845 ; DB nrng ; NUMBER OF RANGES
28846 ; +DB ITEM_TAG ; RETURN VALUE IF RANGE MATCHED
28847 ; +DD X,Y ; RANGE OF VALUES
28848 ; :
28849 ; DB nnval ; NUMBER OF CHOICES
28850 ; +DB ITEM_TAG ; RETURN VALUE IF NUMBER CHOICE MATCHED
28851 ; +DD VALUE ; SPECIFIC CHOICE IF NUMBER
28852 ; :
28853 ; DB nstrval ; NUMBER OF CHOICES
28854 ; +DB ITEM_TAG ; RETURN VALUE IF STRING CHOICE MATCHED
28855 ; +DW STRING ; SPECIFIC CHOICE IF STING
28856 ; +- :
28857 ;
28858 ; STRING DB "...",0 ; ASCII STRING IMAGE
28859 ;
28860 ; Note:
28861 ; - ITEM_TAG must not be OFFH, which will be used in the result buffer
28862 ; when no choice lists are provided.
28863 ;
28864 ; - STRING must consist of upper case characters only because the parser
28865 ; performs pattern matching after converting input to upper case (by
28866 ; using the current country upper case table)
28867 ; -----
28868 ;
28869 ; _P_nval_None equ 0 ;AN000; no value list ID
28870 ; _P_nval_Range equ 1 ;AN000; range list ID
28871 ; _P_nval_Value equ 2 ;AN000; value list ID
28872 ; _P_nval_String equ 3 ;AN000; string list ID
28873 ; _P_Len_Range equ 9 ;AN000; Length of a range choice(two DD plus one DB)
28874 ; _P_Len_Value equ 5 ;AN000; Length of a value choice(one DD plus one DB)
28875 ; _P_Len_String equ 3 ;AN000; Length of a string choice(one DW plus one DB)
28876 ; _P_No_nrng equ 0 ;AN000; (tm07) no nrng. nnval must not be 0.
28877 ;
28878 ; struct _P_Val_List

```

```

28876 00000000 ??      .NumofList: resb 1      ;AN000; number of following choice
28877 00000001 ????.   .val_XL:   resw 1      ;AN000; lower word of value
28878 00000003 ????.   .val_XH:   resw 1      ;AN000; higher word of value
28879 00000005 ????.   .val_YL:   resw 1      ;AN000; lower word of another value
28880 00000007 ????.   .val_YH:   resw 1      ;AN000; higher word of another value
28881
28882
28883
28884
28885
28886
28887
28888
28889
28890
28891
28892
28893
28894
28895
28896
28897
28898
28899
28900
28901
28902
28903
28904
28905
28906
28907
28908
28909
28910
28911
28912
28913
28914
28915
28916
28917
28918
28919
28920
28921
28922
28923
28924
28925
28926
28927
28928 00000000 ??      .Type:      resb 1      ;AN000; Type returned
28929 00000001 ??      .Item_Tag:   resb 1      ;AN000; Matched item tag
28930 00000002 ????.   .SYNONYM_Ptr: resw 1      ;AN000; pointer to Synonym list returned
28931 00000004 ???????? .Picked_Val: resb 4      ;AN000; value
28932
28933
28934
28935
28936
28937
28938
28939
28940
28941
28942
28943
28944
28945
28946
28947
28948
28949
28950
28951
28952
28953
28954
28955
28956
28957
28958
28959
28960
28961
28962
28963
28964
28965
28966
28967
28968
28969
28970
28971
28972
28973 0000196F 0000    .P_ORDINAL: dw 0      ;AN000; Operand ordinal save area
28974 00001971 0000    .P_RC:      dw 0      ;AN000; Return code from parser
28975 00001973 0000    .P_SI_Save: dw 0      ;AN000; Pointer of command buffer
28976 00001975 0000    .P_DX:      dw 0      ;AN000; Return result buffer address
28977 00001977 00      .P_Terminator: db 0    ;AN000; Terminator code (ASCII)
28978 00001978 0000    .P_DBCSEV_OFF: dw 0    ;AN000; Offset of DBCS EV
28979 0000197A 0000    .P_DBCSEV_SEG: dw 0    ;AN000; Segment of DBCS EV
28980 0000197C 0000    .P_Flags:   dw 0      ;AN000; Parser internal flags
28981
28982 %define _P_Flags1 _P_Flags ;AN038; to reference first byte flags
28983 %define _P_Flags2 _P_Flags+1 ;AN038; to reference second byte flags only
28984
28985 ;in second byte of _P_Flags, referenced as _P_Flags2:
28986 _P_equ equ 01h      ;AN000; "=" packed in string buffet
28987 _P_Neg equ 02h      ;AN000; Negative value
28988 _P_Time12 equ 04h    ;AN000; set when PM is specified
28989 _P_Key_Cmp equ 08h    ;AN000; set when keyword compare
28990 _P_SW_Cmp equ 10h     ;AN000; set when switch compare
28991 _P_Extra equ 20h      ;AN000; set when extra delimiter found
28992 _P_SW equ 40h         ;AN000; set when switch found (tm08)
28993 _P_Signed equ 80h     ;AN000; signed numeric specified
28994
28995 ;in first byte of _P_Flags, referenced as _P_Flags1:
28996 _P_time12am equ 01h   ;AN038; set when AM is specified on time
28997 _P_TIME_AGAIN equ 02h ;AN039; SET WHEN READY TO RE-PARSE TIME
28998 0000197E 0000    _P_SaveSI_Cmpx: dw 0    ;AN000; save si for later use by complex
28999 00001980 0000    _P_KEYorSW_Ptr: dw 0    ;AN000; points next to "=" or ":" code

```

```

29000 00001982 0000      _$P_Save_EOB:      dw  0          ;AN000; save pointer to EOB
29001 00001984 0000      _$P_Found_SYNONYM: dw  0          ;AN000; es:@ points to found synonym
29002
29003 00001986 00<rep 80h>  _$P_STRING_BUF:      times 128 db 0      ;AN000; Pick a operand from command line
29004      _$P_STRING_BUF_END equ  $          ;AN000;
29005
29006      ; 25/10/2022
29007      ; (MSDOS 5.0 IO.SYS, SYSINIT:16F8h)
29008
29009 00001A06 FF      _$P_Char_CAP_Ptr:  db  0FFh          ;AN000; info id
29010      dw  0          ;AN000; offset of char case map table
29011 00001A09 0000      dw  0          ;AN000; segment of char case map table
29012
29013      ; 25/10/2022
29014      ; IF CAPSW
29015      _$P_File_CAP_Ptr:  db  0FFh          ;AN000; info id
29016      dw  0          ;AN000; offset of file case map table
29017      dw  0          ;AN000; segment of file case map table
29018      ; ENDIF
29019
29019      ; (tm06) IF FileSW          ;AN000;(check if file spec is supported)
29020
29021
29022      ;M029
29023      ;!!!WARNING!!!
29024      ; In routine SYSPARSE (parse.asm), _$P_FileSp_Char is reinitialized using
29025      ; hardcoded strings. If the chars in the string are changed here, corresponding
29026      ; changes need to be made in SYSPARSE
29027
29028      ; IF FileSW+DrvSW          ;AN000;(check if file spec is supported)
29029
29030      ; 25/10/2022
29031      ; (MSDOS 5.0 IO.SYS, SYSINIT:16FDh)
29032
29033 00001A0B 5B5D7C3C3E2B3D3B22  _$P_FileSp_Char      db  '['<=>+;"'      ;AN000; delimiter of file spec
29034      _$P_FileSp_Len      equ  $-_$P_FileSp_Char ;AN000;
29035
29036      ; ENDIF          ;AN000;(of FileSW)
29037
29038      ; delimiter parsing
29039      _$P_colon_period      equ  01h          ;AN032; check for colon & period
29040      _$P_period_only      equ  02h          ;AN032; check only for period
29041
29042      ; filespec error flag
29043 00001A14 00      _$P_err_flag:      db  0          ;AN033; flag set if filespec parsing error
29044      ;AN033; was detected.
29045      _$P_error_filespec      equ  01h          ;AN033; mask to set flag
29046
29047
29048      ; PARSE.ASM - MSDOS 6.0 - 1991
29049      ; =====
29050      ; 27/03/2019 - Retro DOS v4.0
29051
29052      ; *****
29053      ; SysParse;
29054
29055      ; Function : Parser Entry
29056
29057      ; Input: DS:SI -> command line
29058      ; ES:DI -> parameter block
29059      ; CS -> psdata.inc
29060      ; CX = operand ordinal
29061
29062      ; Note: ES is the segment containing all the control blocks defined
29063      ; by the caller, except for the DOS COMMAND line parms, which
29064      ; is in DS.
29065
29066      ; Output: CY = 1 error of caller, means invalid parameter block or
29067      ; invalid value list. But this parser does NOT implement
29068      ; this feature. Therefore CY always zero.
29069
29070      ; CY = 0 AX = return code
29071      ; BL = terminated delimiter code
29072      ; CX = new operand ordinal
29073      ; SI = set past scanned operand
29074      ; DX = selected result buffer
29075
29076      ; Use:      _$P_Skip_Delim, _$P_Chk_EOL, _$P_Chk_Delim, _$P_Chk_DBCS
29077      ;      _$P_Chk_Swtch, _$P_Chk_Pos_Control, _$P_Chk_Key_Control
29078      ;      _$P_Chk_Sw_Control, _$P_Fill_Result
29079
29080      ; Vars: _$P_Ordinal(RW), _$P_RC(RW), _$P_SI_Save(RW), _$P_DX(R), _$P_Terminator(R)
29081      ;      _$P_SaveSI_Cmpx(W), _$P_Flags(RW), _$P_Found_SYNONYM(R), _$P_Save_EOB(W)
29082
29083      ; ----- Modification History -----
29084
29085      ; 4/04/87 : Created by K. K,
29086      ; 4/28/87 : _$P_Val_YH assemble error (tm01)
29087      ; : JMP SHORT assemble error (tm02)
29088      ; 5/14/87 : Someone doesn't want to include psdata (tm03)
29089      ; 6/12/87 : _$P_Bridge is missing when TimesW equ 0 and (CmpxSw equ 1 or
29090      ; DateSw equ 1) (tm04)
29091      ; 6/12/87 : _$P_SorD_Quote is missing when QusSw equ 0 and CmpxSw equ 1
29092      ; (tm05) in PSDATA.INC
29093      ; 6/12/87 : _$P_FileSp_Char and _$P_FileSp_Len are missing
29094      ; when FileSW equ 0 and DrvSW equ 1 (tm06) in PSDATA.INC
29095      ; 6/18/87 : $VAL1 and $VAL3, $VAL2 and $VAL3 can be used in the same
29096      ; value-list block (tm07)
29097      ; 6/20/87 : Add _$P_SW to check if there's an omiting parameter after
29098      ; switch (keyword) or not. If there is, backup si for next call
29099      ; (tm08)
29100      ; 6/24/87 : Complex Item checking does not work correctly when CmpSw equ 1
29101      ; and DateSw equ 0 and TimesW equ 0 (tm09)
29102      ; 6/24/87 : New function flag _$P_colon_is_not_necessary for switch
29103      ; /+15 and /+:15 are allowed for user (tm10)
29104      ; 6/29/87 : ECS call changes DS register but it causes the address problem
29105      ; in user's routines. _$P_Chk_DBCS (tm11)
29106      ; 7/10/87 : Switch with no_match flag (0x0000H) does not work correctly
29107      ; (tm12)
29108      ; 7/10/87 : Invalid switch/keyword does not work correctly
29109      ; (tm13)
29110      ; 7/10/87 : Drive_only breaks 3 bytes after the result buffer
29111      ; (tm14)
29112      ; 7/12/87 : Too_Many_Operands sets DX=0 as the PARSE result
29113      ; (tm15)
29114      ; 7/24/87 : Negative lower bound on numeric ranges cause trouble
29115
29116      ; 7/24/87 : Quoted strings being returned with quotes.
29117
29118      ; 7/28/87 : Kerry S (;AN018;)
29119      ; Non optional value on switch (match flags<>0 and <>1) not flagged
29120      ; as an error when missing. Solution: return error 2. Modules
29121      ; affected: _$P_Chk_SW_Control.
29122
29123      ; 7/29/87 : Kerry S (;AN019;)

```

```

29124 ; Now allow the optional bit in match flags for switches. This
29125 ; allows the switch to be encountered with a value or without a
29126 ; value and no error is returned.
29127 ;
29128 ;
29129 ; 8/28/87 : Ed K, Kerry S (;AN020;)
29130 ; 9/14/87 In PROC _$P_Get_DecNum, when checking for field separators
29131 ; within a date response, instead of checking just for the one
29132 ; character defined by the COUNTRY DEPENDENT INFO, check for
29133 ; all three chars, "-", "/", and ".". Change _$P_Chk_Switch to allow
29134 ; slashes in date strings when DateSw (assembler switch) is set.
29135 ;
29136 ; 9/1/87 : Kerry S (;AN021;)
29137 ; In PROC _$P_String_Comp, when comparing the switch or keyword on
29138 ; the command line with the string in the control block the
29139 ; comparing was stopping at a colon (switch) or equal (keyword)
29140 ; on the command line and assuming a match. This allowed a shorter
29141 ; string on the command line than in the synonym list in the control
29142 ; block. I put in a test for a null in the control block so the
29143 ; string in the control block must be the same length as the string
29144 ; preceding the colon or equal on the command line.
29145 ;
29146 ; 8/28/87 : Kerry S (;AN022;)
29147 ; All references to data in PSDATA.INC had CS overrides. This caused
29148 ; problems for people who included it themselves in a segment other
29149 ; than CS. Added switch to allow including PSDATA.INC in any
29150 ; segment.
29151 ;
29152 ; 9/16/87 : Ed K (;AN023;) PTM1040
29153 ; in _$P_set_cdi PROC, it assumes CS points to psdata. Change Push CS
29154 ; into PUSH CS. In _$P_Get_DecNum PROC, fix AN020
29155 ; forced both TIME and DATE to use the delims, "-", "/", ".".
29156 ; Created FLAG, in _$P_time_Format PROC, to request the delim in
29157 ; BL be used if TIME is being parsed.
29158 ;
29159 ; 9/24/87 : Ed K
29160 ; Removed the include to STRUC.INC. Replaced the STRUC macro
29161 ; invocations with their normally expanded code; made comments
29162 ; out of the STRUC macro invocation statements to maintain readability.
29163 ;
29164 ; 9/24/87 : Ed K (;AN024;) PTM1222
29165 ; When no CONTROL for a keyword found, tried to fill in RESULT
29166 ; pointed to by non-existent CONTROL.
29167 ;
29168 ; 10/15/87 : Ed K (;AN025;) PTM1672
29169 ; A quoted text string can be framed only by double quote. Remove
29170 ; support to frame quoted text string with single quote.
29171 ; (apostrophe) _$P_Sord_Quote is removed from PSDATA.INC.
29172 ; _$P_SQuote EQU also removed from PSDATA.INC. Any references to
29173 ; single quote in PROC prologues are left as is for history reasons.
29174 ;
29175 ; This fixes another bug, not mentioned in p1672, in that two
29176 ; quote chars within a quoted string is supposed to be reported as
29177 ; one quote character, but is reported as two quotes. This changed
29178 ; two instructions in PROC _$P_Quoted_Str.
29179 ;
29180 ; Also fixed are several JMP that caused a NOP, these changed to
29181 ; have the SHORT operator to avoid the unneeded NOP.
29182 ;
29183 ; The code and PSDATA.INC have been aligned for ease of reading.
29184 ;
29185 ; 10/26/87 : Ed K (;AN026;) PTM2041, DATE within SWITCH, BX reference to
29186 ; psdata buffer should have cs.
29187 ;
29188 ; 10/27/87 : Ed K (;AN027;) PTM2042 comma between keywords implies
29189 ; positional missing.
29190 ;
29191 ; 11/06/87 : Ed K (;AN028;) PTM 2315 Parser should not use line feed
29192 ; as a line delimiter, should use carriage return.
29193 ; Define switch: LFEOLSW, if on, accept LF as end of line char.
29194 ;
29195 ; 11/11/87 : Ed K (;AN029;) PTM 1651 GET RID OF WHITESPACE AROUND "=".
29196 ;
29197 ; 11/18/87 : Ed K (;AN030;) PTM 2551 If filename is just "", then
29198 ; endless loop since SI is returned still pointing to start
29199 ; of that parm.
29200 ;
29201 ; 11/19/87 : Ed K (;AN031;) PTM 2585 date & time getting bad values.
29202 ; Vector to returned string has CS instead of cs, but
29203 ; when tried to fix it on previous version, changed similar
29204 ; but wrong place.
29205 ;
29206 ; 12/09/87 : Bill L (;AN032;) PTM 2772 colon and period are now valid
29207 ; delimiters between hours, minutes, seconds for time. And period
29208 ; and comma are valid delimiters between seconds and 100th second.
29209 ;
29210 ; 12/14/87 : Bill L (;AN033;) PTM 2722 if illegal delimiter characters
29211 ; in a filespec, then flag an error.
29212 ;
29213 ; 12/22/87 : Bill L (;AN034;) All local data to parser is now
29214 ; indexed off of the cs equate instead of the DS register.
29215 ; Using this method, DS can point to the segment of PSP or to psdata
29216 ; --> local parser data. Why were some references to local data changed
29217 ; to do this before, but not all ???
29218 ;
29219 ; 02/02/88 : Ed K (;AC035;) INSPECT utility, suggests optimizations.
29220 ;
29221 ; 02/05/88 : Ed K (;AN036;) P3372-UPPERCASE TRANSLATION, CS HOSED.
29222 ;
29223 ; 02/08/88 : Ed K (;AN037;) P3410-AVOID POP OF CS, CHECK BASESW FIRST.
29224 ;
29225 ; 02/19/88 : Ed K (;AN038;) p3524 above noon and "am" should be error
29226 ;
29227 ; 02/23/88 : Ed K (;AN039;) p3518 accept "comma" and "period" as decimal
29228 ; separator in TIME before hundredths field.
29229 ;
29230 ; 08/09/90 : SA M005 Prevented parser from recognizing '=' signs within
29231 ; strings as keywords.
29232 ;
29233 ; *****
29234 ;
29235 ; IF FarSw ;AN000;(Check if need far return)
29236 ; SysParse proc far ;AN000;
29237 ; ELSE ;AN000;
29238 ; SysParse proc near ;AN000;
29239 ; ENDIF ;AN000;(of FarSw)
29240 ;
29241 ; 27/03/2019 - Retro DOS v4.0
29242 ; (MSDOS 6.21 IO.SYS - SYSINIT:1842h)
29243 ;
29244 ; 25/10/2022 - Retro DOS v4.0
29245 ; (MSDOS 5.0 IO.SYS - SYSINIT:1707h)
29246 ;
29247 ; 06/09/2023 - Retro DOS v4.2 IO.SYS Optimization (& Retro DOS v5.0 pre-work)

```



```

29248 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1D08h)
29249
29250 SysParse:
29251 ; 06/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
29252 ; dx = 0
29253 00001A15 1E push ds ; *!*
29254 00001A16 0E push cs
29255 00001A17 1F pop ds
29256
29257 ;mov word [cs:$_P_Flags],0 ;AC034; Clear all internal flags
29258 ;cld ;AN000; confirm forward direction
29259 ;mov word [cs:$_P_ORDINAL],cx ;AC034; save operand ordinal
29260 ;mov word [cs:$_P_RC],$_P_No_Error ;AC034; Assume no error
29261 ;mov word [cs:$_P_Found_SYNONYM],0 ;AC034; initialize synonym pointer
29262 ;
29263 ;
29264 ;mov word [cs:$_P_DX],0 ;AC034; (tm15)
29265
29266 ; 06/09/2023
29267 00001A18 8916[7C19] mov [_P_Flags],dx ; 0 ;AC034; Clear all internal flags
29268 00001A1C FC cld ;AN000; confirm forward direction
29269 00001A1D 890E[6F19] mov [_P_ORDINAL],cx ;AC034; save operand ordinal
29270 00001A21 8916[7119] mov [_P_RC],dx ; $_P_No_Error ;AC034; Assume no error
29271 00001A25 8916[8419] mov [_P_Found_SYNONYM],dx ; 0 ;AC034; initialize synonym pointer
29272 00001A29 8916[7519] mov [_P_DX],dx ; 0 ;AC034; (tm15)
29273
29274 ;M029 -- Begin changes
29275 ; The table of special chars $_P_FileSp_Char should be initialized on every
29276 ; entry to SysParse. This is in the non-checksum region and any program that
29277 ; corrupts this table but does not corrupt the checksum region will leave
29278 ; command.com parsing in an inconsistent state.
29279 ; NB: The special characters string has been hardcoded here. If any change
29280 ; is made to it in psdata.inc, a corresponding change needs to be made here.
29281
29282 ;IF FileSW + DrvSW
29283 ; 14/04/2024 (NASM syntax BugFix) .. '[' (MASM) -> '[' (NASM)
29284
29285 ;mov word [cs:$_P_FileSp_Char], '['
29286 ;mov word [cs:$_P_FileSp_Char+2], '|<'
29287 ;mov word [cs:$_P_FileSp_Char+4], '>+'
29288 ;mov word [cs:$_P_FileSp_Char+6], '='
29289
29290 ; 14/04/2024
29291 ; 06/09/2023
29292 00001A2D C706[0B1A]5B5D mov word [_P_FileSp_Char], '[' ; mov word [_P_FileSp_Char],5D5Bh
29293 00001A33 C706[0D1A]7C3C mov word [_P_FileSp_Char+2], '|<' ; 3C7Ch
29294 00001A39 C706[0F1A]3E2B mov word [_P_FileSp_Char+4], '>+' ; 2B3Eh
29295 00001A3F C706[111A]3D3B mov word [_P_FileSp_Char+6], '=' ; 3B3Dh
29296
29297 00001A45 1F ;ENDIF
29298 pop ds ; *!*
29299
29300 ;M029 -- End of changes
29301
29302 00001A46 E87806 call $_P_Skip_Delim ;AN000; Move si to 1st non white space
29303 00001A49 7313 jnc short $_P_Start ;AN000; If EOL is not encountered, do parse
29304
29305 ----- End of Line
29306 mov ax,$_P_RC_EOL ;AN000; set exit code to -1
29307 push bx ;AN000;
29308 ;mov bx,[es:di+$_P_PARAMS_Blk.PARMSX_Address] ;AN000; Get the PARMSX address to
29309 mov bx,[es:di]
29310 ;cmp cl,[es:bx+$_P_PARAMSX_Blk.MinP] ;AN000; check ORDINAL to see if the minimum
29311 cmp cl,[es:bx]
29312 jae short $_P_Fin ;AN000; positional found.
29313
29314 mov ax,$_P_Op_Missing ;AN000; If no, set exit code to missing operand
29315 $_P_Fin: ;AN000;
29316 00001A5A 5B pop bx ;AN000;
29317 00001A5B E90A01 jmp $_P_Single_Exit ;AN000; return to the caller
29318
29319 -----
29320 $_P_Start: ;AN000;
29321 00001A5E 2E8936[7E19] mov [cs:$_P_SavesI_Cmpx],si ;AN000;AC034; save ptr to command line for later use by complex,
29322 00001A63 53 push bx ;AN000; quoted string or file spec.
29323 00001A64 57 push di ;AN000;
29324 00001A65 55 push bp ;AN000;
29325 ;lea bx,[cs:$_P_STRING_BUF] ;AC034; set buffer to copy from command string
29326 ; 02/11/2022
29327 ;lea bx,[$_P_STRING_BUF]
29328 ; 07/09/2023
29329 00001A66 BB[8619] mov bx,$_P_STRING_BUF
29330 00001A69 2EF606[7D19]20 test byte [cs:$_P_Flags2],$_P_Extra ;AC034; 3/9 extra delimiter encountered ?
29331 00001A6F 7543 jnz short $_P_Pack_End ;AN000; 3/9 if yes, no need to copy
29332
29333 $_P_Pack_Loop: ;AN000;
29334 00001A71 AC lodsb ;AN000; Pick a operand from buffer
29335 00001A72 E8F106 call $_P_Chk_Switch ;AN000; Check switch character
29336 00001A75 723C jc short $_P_Pack_End_BY_EOL ;AN020; if carry set found delimiter type slash, need backup si, else
29337
29338 continue
29339 00001A77 E86906 call $_P_Chk_EOL ;AN000; Check EOL character
29340 00001A7A 7437 je short $_P_Pack_End_BY_EOL ;AN000; need backup si
29341
29342 00001A7C E89906 call $_P_Chk_Delim ;AN000; Check delimiter
29343 00001A7F 7518 jne short $_P_PL01 ;AN000; If no, process next byte
29344
29345 test byte [cs:$_P_Flags2],$_P_Extra ;AC034; 3/9 If yes and white spec,
29346 ; (tm08)jne short $_P_Pack_End ;AN000; 3/9 then
29347 jnz short $_P_Pack_End_backup_si ;AN000; (tm08)
29348
29349 call $_P_Skip_Delim ;AN000; skip subsequent white space,too
29350 jmp short $_P_Pack_End ;AN000; finish copy by placing NUL at end
29351
29352 $_P_Pack_End_backup_si: ;AN000; (tm08)
29353 00001A8E 2EF606[7D19]41 test byte [cs:$_P_Flags2],$_P_Sw+$_P_equ ;AN000;AC034; (tm08)
29354 00001A94 741E jz short $_P_Pack_End ;AN000; (tm08)
29355
29356 dec si ;AN000; (tm08)
29357 jmp short $_P_Pack_End ;AN025; (tm08)
29358
29359 $_P_PL01: ;AN000;
29360 00001A99 2E8807 mov [cs:bx],al ;AN000; move byte to STRING_BUF
29361 00001A9C 3C3D cmp al,$_P_Keyword ;AN000; if it is equal character,
29362 00001A9E 7506 jne short $_P_PL00 ;AN000; then
29363
29364 or byte [cs:$_P_Flags2],$_P_equ ;AC034; remember it in flag
29365 $_P_PL00: ;AN000;
29366 00001AA6 43 inc bx ;AN000; ready to see next byte
29367 00001AA7 E8D506 call $_P_Chk_DBCS ;AN000; was it 1st byte of DBCS ?
29368 00001AAA 73C5 jnc short $_P_Pack_Loop ;AN000; if no, process to next byte
29369
29370 lodsb ;AN000; if yes, store
29371 00001AAC AC mov [cs:bx],al ;AN000; 2nd byte of DBCS
29372 00001AAD 2E8807 mov [cs:bx],al ;AN000; update pointer
29373 00001AB0 43 inc bx ;AN000;

```

```

29371 00001AB1 EBBE      jmp     short _$P_Pack_Loop      ;AN000; process to next byte
29372
29373 _$P_Pack_End_BY_EOL:      ;AN000;
29374 00001AB3 4E      dec     si      ;AN000; backup si pointer
29375 _$P_Pack_End:      ;AN000;
29376 00001AB4 2E8936[7319]  mov     [cs:_$P_SI_Save],si      ;AC034; save next pointer, SI
29377      ; 07/09/2023
29378      ;mov     byte [cs:bx],_$P_NULL ;AN000; put NULL at the end
29379 00001AB9 30E4      xor     ah,ah ; 0 ; *
29380 00001ABB 2E8827      mov     [cs:bx],ah ; _$P_NULL ;AN000; put NULL at the end
29381      ;
29382 00001ABE 2E891E[8219]  mov     [cs:_$P_Save_EOB],bx      ;AC034; 3/17/87 keep the address for later use of complex
29383      ;mov     bx,[es:di+_$P_PARAMS_Blk.PARMSX_Address] ;AN000; get PARMSX address
29384 00001AC3 268B1D      mov     bx,[es:di]
29385      ;lea     si,[cs:_$P_STRING_BUF];AC034;
29386      ; 02/11/2022
29387      ;lea     si,[_$P_STRING_BUF]
29388      ; 07/09/2023
29389 00001AC6 BE[8619]      mov     si,_$P_STRING_BUF
29390 00001AC9 2E803C2F      cmp     byte [cs:si],_$P_Switch ;AN000; the operand begins w/ switch char ?
29391 00001ACD 7440      je      short _$P_SW_Manager ;AN000; if yes, process as switch
29392
29393 00001ACF 2E803C22      cmp     byte [cs:si],_$P_DQuote ;M005;is it a string?
29394 00001AD3 7408      je      short _$P_Positional_Manager ;M005;if so, process as one!
29395
29396 00001AD5 2EF606[7D19]01  test    byte [cs:_$P_Flags2],_$P_equ ;AC034; the operand includes equal char ?
29397 00001ADB 7552      jnz     short _$P_Key_Manager ;AN000; if yes, process as keyword
29398
29399 _$P_Positional_Manager:      ;AN000; else process as positional
29400 00001ADD 268A4701      mov     al,[es:bx+_$P_PARAMS_Blk.MaxP] ;AN000; get maxp
29401      ; 07/09/2023
29402      ;xor     ah,ah      ;AN000; ax = maxp
29403 00001AE1 2E3906[6F19]  cmp     [cs:_$P_ORDINAL],ax      ;AC034; too many positional ?
29404 00001AE6 7312      jae     short _$P_Too_Many_Error ;AN000; if yes, set exit code to too many
29405
29406 00001AE8 2EA1[6F19]      mov     ax,[cs:_$P_ORDINAL]      ;AC034; see what the current ordinal
29407 00001AEC D1E0      shl     ax,1      ;AN000; ax = ax*2
29408 00001AEE 43      inc     bx      ;AC035; add '2' to
29409 00001AEF 43      inc     bx      ;AC035; BX reg
29410      ;AN000; now bx points to 1st CONTROL
29411 00001AF0 01C3      add     bx,ax      ;AN000; now bx points to specified CONTROL address
29412 00001AF2 268B1F      mov     bx,[es:bx]      ;AN000; now bx points to specified CONTROL itself
29413 00001AF5 E87200      call    _$P_Chk_Pos_Control ;AN000; do process for positional
29414 00001AF8 EB53      jmp     short _$P_Return_to_Caller ;AN000; and return to the caller
29415
29416 _$P_Too_Many_Error:      ;AN000;
29417 00001AFA 2EC706[7119]0100  mov     word [cs:_$P_RC],_$P_Too_Many ;AC034; set exit code
29418 00001B01 EB4A      jmp     short _$P_Return_to_Caller ;AN000; and return to the caller
29419
29420      ; 07/09/2023 - Retro DOSD v4.2 IO.SYS (Optimization)
29421      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1E06h)
29422 get_maxp:
29423      ;mov     al,[es:bx+1]
29424 00001B03 268A4701      mov     al,[es:bx+_$P_PARAMS_Blk.MaxP] ;AN000; get maxp
29425      ; 07/09/2023
29426      ; ah=0 ; *
29427      ;xor     ah,ah ; 0      ;AN000; ax = maxp
29428 00001B07 30ED      xor     ch,ch ; **
29429 00001B09 40      inc     ax      ;AN000;
29430 00001B0A D1E0      shl     ax,1      ;AN000; ax = (ax+1)*2
29431 00001B0C 01C3      add     bx,ax      ;AN000; now bx points to maxs
29432 00001B0E C3      retn
29433
29434 _$P_SW_Manager:      ;AN000;
29435      ; 07/09/2023
29436      ;mov     al,[es:bx+_$P_PARAMS_Blk.MaxP] ;AN000; get maxp
29437      ;xor     ah,ah      ;AN000; ax = maxp
29438      ;inc     ax      ;AN000;
29439      ;shl     ax,1      ;AN000; ax = (ax+1)*2
29440      ;add     bx,ax      ;AN000; now bx points to maxs
29441 00001B0F E8F1FF      call    get_maxp ; 07/09/2023
29442
29443 00001B12 268A0F      mov     cl,[es:bx]      ;AN000;
29444      ; 07/09/2023
29445      ;xor     ch,ch ; ** (ch=0)      ;AN000; cx = maxs
29446      ;or     cx,cx      ;AN000; at least one switch ?
29447      ;jz     short _$P_SW_Not_Found ;AN000;
29448      ; 07/07/2023
29449 00001B15 E30F      jcxz    _$P_SW_Not_Found      ; no
29450
29451 00001B17 43      inc     bx      ;AN000; now bx points to 1st CONTROL address
29452
29453 _$P_SW_Mgr_Loop:      ;AN000;
29454 00001B18 53      push    bx      ;AN000;
29455 00001B19 268B1F      mov     bx,[es:bx]      ;AN000; bx points to Switch CONTROL itself
29456 00001B1C E8A900      call    _$P_Chk_SW_Control ;AN000; do process for switch
29457 00001B1F 5B      pop     bx      ;AN000;
29458 00001B20 732B      jnc     short _$P_Return_to_Caller ;AN000; if the CONTROL is for the switch, exit
29459
29460 00001B22 43      inc     bx      ;AC035; add '2' to
29461 00001B23 43      inc     bx      ;AC035; BX reg
29462      ;AN000; else bx points to the next CONTROL
29463 00001B24 E2F2      loop    _$P_SW_Mgr_Loop      ;AN000; and loop
29464
29465 _$P_SW_Not_Found:      ;AN000;
29466 00001B26 2EC706[7119]0300  mov     word [cs:_$P_RC],_$P_Not_In_SW ;AC034; here no CONTROL for the switch has
29467 00001B2D EB1E      jmp     short _$P_Return_to_Caller ;AN000; not been found, means error.
29468
29469 _$P_Key_Manager:      ;AN000;
29470      ; 07/09/2023
29471      ;mov     al,[es:bx+_$P_PARAMS_Blk.MaxP] ;AN000; get maxp
29472      ;xor     ah,ah      ;AN000; ax = maxp
29473      ;inc     ax      ;AN000;
29474      ;shl     ax,1      ;AN000; ax = (ax+1)*2
29475      ;add     bx,ax      ;AN000; now bx points to maxs
29476 00001B2F E8D1FF      call    get_maxp ; 07/09/2023
29477
29478 00001B32 268A07      mov     al,[es:bx]      ;AN000;
29479 00001B35 30E4      xor     ah,ah ; 0      ;AN000; ax = maxs
29480 00001B37 D1E0      shl     ax,1      ;AN000;
29481 00001B39 40      inc     ax      ;AN000; ax = ax*2+1
29482 00001B3A 01C3      add     bx,ax      ;AN000; now bx points to maxk
29483 00001B3C 268A0F      mov     cl,[es:bx]      ;AN000;
29484      ; 07/09/2023
29485      ;xor     ch,ch ; ** (ch=0)      ;AN000; cx = maxk
29486      ;or     cx,cx      ;AN000; at least one keyword ?
29487      ;jz     short _$P_Key_Not_Found ;AN000;
29488      ; 07/07/2023
29489 00001B3F E305      jcxz    _$P_Key_Not_Found      ; no
29490
29491 00001B41 43      inc     bx      ;AN000; now bx points to 1st CONTROL
29492
29493 _$P_Key_Mgr_Loop:      ;AN000;
29494      ; 07/09/2023

```



```

29495 ; ('_P_Chk_Key_Control' contains only 'stc' instruction)
29496 ; (always returns with cf=1)
29497 ;push    bx ;AN000;
29498 ;mov     bx,[es:bx] ;AN000; bx points to keyword CONTROL itself
29499 ;call    _P_Chk_Key_Control ;AN000; do process for keyword
29500 ;pop     bx ;AN000;
29501 ;jnc     short _P_Return_to_Caller ;AN000; if the CONTROL is for the keyword, exit
29502 ; 07/09/2023
29503 ; cf=1 (after 'call _P_Chk_Key_Control')
29504
29505 00001B42 43 inc     bx ;AC035; add '2' to
29506 00001B43 43 inc     bx ;AC035; BX reg
29507 ;AN000; else bx points to the next CONTROL
29508 00001B44 E2FC loop    _P_Key_Mgr_Loop ;AN000; and loop
29509
29510 ;_P_Key_Not_Found: ;AN000;
29511 00001B46 2EC706[7119]0400 mov     word [cs:_P_RC],_P_Not_In_Key ;AC034; here no CONTROL for the keyword has
29512 ;_P_Return_to_Caller: ;AN000;
29513 00001B4D 5D pop     bp ;AN000;
29514 00001B4E 5F pop     di ;AN000;
29515 00001B4F 5B pop     bx ;AN000;
29516 00001B50 2E8B0E[6F19] mov     cx,[cs:_P_ORDINAL] ;AC034; return next ordinal
29517 00001B55 2EA1[7119] mov     ax,[cs:_P_RC] ;AC034; return exit code
29518 00001B59 2E8B36[7319] mov     si,[cs:_P_SI_Save] ;AC034; return next operand pointer
29519 00001B5E 2E8B16[7519] mov     dx,[cs:_P_DX] ;AC034; return result buffer address
29520 00001B63 2E8A1E[7719] mov     bl,[cs:_P_Terminator] ;AC034; return delimiter code found
29521 ;_P_Single_Exit: ;AN000;
29522 00001B68 F8 cld ;AN000;
29523 00001B69 C3 retn ;AN000;
29524
29525 ;*****
29526 ; _P_Chk_Pos_Control
29527 ;
29528 ; Function: Parse CONTROL block for a positional
29529 ;
29530 ; Input: ES:BX -> CONTROL block
29531 ;        cs:SI -> _P_STRING_BUF
29532 ;
29533 ; Output: None
29534 ;
29535 ; Use: _P_Fill_Result, _P_Check_Match_Flags
29536 ;
29537 ; Vars: _P_Ordinal(w), _P_RC(w)
29538 ;*****
29539
29540 ;_P_Chk_Pos_Control:
29541 00001B6A 50 push     ax ;AN000;
29542 ;mov     ax,[es:bx+_P_Control_Blk.Match_Flag] ;AN000;
29543 00001B6B 268B07 mov     ax,[es:bx]
29544 ; 12/12/2022
29545 00001B6E A802 test     al,_P_Repeat
29546 ;test    ax,_P_Repeat ;AN000; repeat allowed ?
29547 00001B70 7505 jnz     short _P_CPC00 ;AN000; then do not increment ORDINAL
29548
29549 00001B72 2EFF06[6F19] inc     word [cs:_P_ORDINAL] ;AC034; update the ordinal
29550 ;_P_CPC00: ;AN000;
29551 00001B77 2E803C00 cmp     byte [cs:si],_P_NULL ;AN000; no data ?
29552 00001B7B 7517 jne     short _P_CPC01 ;AN000;
29553
29554 ; 12/12/2022
29555 00001B7D A801 test     al,_P_Optional
29556 ;test    ax,_P_Optional ;AN000; yes, then is it optional ?
29557 00001B7F 7509 jnz     short _P_CPC02 ;AN000;
29558
29559 00001B81 2EC706[7119]0200 mov     word [cs:_P_RC],_P_Op_Missing ;AC034; no, then error 3/17/87
29560 00001B88 EB0D jmp     short _P_CPC_Exit ;AN000;
29561
29562 ;_P_CPC02: ;AN000;
29563 00001B8A 50 push     ax ;AN000;
29564 ;mov     al,_P_String ;AN000; if it is optional return NULL
29565 ;mov     ah,_P_No_Tag ;AN000; no item tag indication
29566 ; 07/07/2023
29567 00001B8B B803FF mov     ax,(_P_No_Tag<<8)|_P_String
29568 00001B8E E89600 call    _P_Fill_Result ;AN000;
29569 00001B91 58 pop     ax ;AN000;
29570 00001B92 EB03 jmp     short _P_CPC_Exit ;AN000;
29571
29572 ;_P_CPC01: ;AN000;
29573 00001B94 E81101 call    _P_Check_Match_Flags ;AN000;
29574 ;_P_CPC_Exit: ;AN000;
29575 00001B97 58 pop     ax ;AN000;
29576 00001B98 C3 retn ;AN000;
29577
29578 ;*****
29579 ; _P_Chk_Key_Control
29580 ;
29581 ; Function: Parse CONTROL block for a keyword
29582 ;
29583 ; Input: ES:BX -> CONTROL block
29584 ;        cs:SI -> _P_STRING_BUF
29585 ;
29586 ; Output: CY = 1 : not match
29587 ;
29588 ; Use: _P_Fill_Result, _P_Search_KEYorSW, _P_Check_Match_Flags
29589 ;
29590 ; Vars: _P_RC(w), _P_SaveSI_Cmpx(w), _P_KEYorSW_Ptr(R), _P_Flags(w)
29591 ;*****
29592
29593 ; 07/09/2023
29594 ;_P_Chk_Key_Control:
29595 ; stc ;AN000; this logic works when the keySW
29596 ; retn ;AN000; is reset.
29597
29598 ;*****
29599 ; _P_Search_KEYorSW:
29600 ;
29601 ; Function: Search specified keyword or switch from CONTROL
29602 ;
29603 ; Input: ES:BX -> CONTROL block
29604 ;        cs:SI -> _P_STRING_BUF
29605 ;
29606 ; Output: CY = 1 : not match
29607 ;
29608 ; Use: _P_String_Comp, _P_MoveBP_NUL, _P_Found_SYNONYM
29609 ;*****
29610
29611 ; 25/10/2022 - Retro DOS v4.0
29612 ; (MSDOS 5.0 IO.SYS - SYSINIT:18B6h)
29613
29614 ;_P_Search_KEYorSW: ;AN000;
29615 00001B99 55 push     bp ;AN000;
29616 00001B9A 51 push     cx ;AN000;
29617 00001B9B 268A4F08 mov     cl,[es:bx+_P_Control_Blk.nid] ;AN000; Get synonym count
29618 00001B9F 30ED xor     ch,ch ;AN000; and set it to cx

```

```

29619             ;or      cx,cx                ;AN000; No synonyms specified ?
29620             ;jz      short _$P_KEYorSW_Not_Found ;AN000; then indicate not found by CY
29621             ; 07/07/2023
29622 00001BA1 E30D      jcxz     _$P_KEYorSW_Not_Found
29623
29624             ;lea     bp,[es:bx+_$P_Control_Blk.KEYorSW] ;AN000; BP points to the 1st synonym
29625             ; 25/10/2022
29626 00001BA3 8D6F09     lea     bp,[bx+_$P_Control_Blk.KEYorSW]
29627             ;lea     bp,[bx+9]
29628 _$P_KEYorSW_Loop:      ;AN000;
29629 00001BA6 E8B503     call    _$P_String_Comp      ;AN000; compare string in buffer w/ the synonym
29630 00001BA9 7308      jnc     short _$P_KEYorSW_Found      ;AN000; If match, set it to synonym pointer
29631
29632 00001BAB E80E00     call    _$P_MoveBP_NUL      ;AN000; else, bp points to the next string
29633 00001BAE E2F6      loop    _$P_KEYorSW_Loop      ;AN000; loop nld times
29634 _$P_KEYorSW_Not_Found: ;AN000;
29635 00001BB0 F9         stc     ;AN000; indicate not found in synonym list
29636 00001BB1 EB06      jmp     short _$P_KEYorSW_Exit;AN000; and exit
29637
29638 _$P_KEYorSW_Found:    ;AN000;
29639 00001BB3 2E892E[8419] mov     [cs:_$P_Found_SYNONYM],bp ;AC034; set synonym pointer
29640 00001BB8 F8         clc     ;AN000; indicate found
29641 _$P_KEYorSW_Exit:    ;AN000;
29642 00001BB9 59         pop     cx      ;AN000;
29643 00001BBA 5D         pop     bp      ;AN000;
29644 00001BBB C3         retn    ;AN000;
29645
29646 ;*****
29647 ; _$P_MoveBP_NUL
29648 ;*****
29649
29650 _$P_MoveBP_NUL:
29651 _$P_MBP_Loop:        ;AN000;
29652             ; 11/12/2022
29653 00001BBC 26807E0000   cmp     byte [es:bp],_$P_NULL ;AN000; Increment BP that points
29654             ; 25/10/2022 (MSDOS 5.0 IO.SYS compatibility)
29655             ; (SYSINIT:18DBh)
29656             ;cmp     byte [es:bp+0],0
29657 00001BC1 7403      je      short _$P_MBP_Exit      ;AN000; to the synonym list
29658
29659             inc     bp      ;AN000; until
29660 00001BC4 EBF6      jmp     short _$P_MBP_Loop      ;AN000; NULL encountered.
29661
29662 _$P_MBP_Exit:        ;AN000;
29663 00001BC6 45         inc     bp      ;AN000; bp points to next to NULL
29664 00001BC7 C3         retn    ;AN000;
29665
29666 ;*****
29667 ; _$P_Chk_SW_Control
29668 ;
29669 ; Function: Parse CONTROL block for a switch
29670 ;
29671 ; Input:      ES:BX -> CONTROL block
29672 ;            cs:SI -> _$P_STRING_BUF
29673 ;
29674 ; Output:     CY = 1 : not match
29675 ;
29676 ; Use:        _$P_Fill_Result, _$P_Search_KEYorSW, _$P_Check_Match_Flags
29677 ;
29678 ; Vars:       _$P_SaveSI_Cmpx(W), _$P_KEYorSW_Ptr(R), _$P_Flags(W)
29679 ;*****
29680
29681 _$P_Chk_SW_Control:
29682
29683 ;IF SwSW           ;AN000;(Check if switch is supported)
29684             ;or      byte [cs:_$P_Flags+1],10h
29685 00001BC8 2E800E[7D19]10 or      byte [cs:_$P_Flags2],_$P_SW_Cmp ;AC034; Indicate switch for later string comparison
29686 00001BCE E8C8FF     call    _$P_Search_KEYorSW ;AN000; Search the switch in the CONTROL block
29687 00001BD1 7248      jc      short _$P_Chk_SW_Err0 ;AN000; not found, then try next CONTROL
29688
29689             ;and     [cs:_$P_Flags+],0EFh
29690 00001BD3 2E8026[7D19]EF and     byte [cs:_$P_Flags2],0FFh-_$P_SW_Cmp
29691             ;AC034; reset the indicator previously set
29692 00001BD9 50         push    ax      ;AN000; /switch:
29693 00001BDA 2EA1[8019]   mov     ax,[cs:_$P_KEYorSW_Ptr] ;AC034; ^
29694 00001BDE 29F0      sub     ax,si      ;AN000; SI KEYorSW
29695 00001BE0 2E0106[7E19] add     [cs:_$P_SaveSI_Cmpx],ax ;AC034; update for complex list
29696 00001BE5 58         pop     ax      ;AN000;
29697
29698 00001BE6 2E8B36[8019] mov     si,[cs:_$P_KEYorSW_Ptr] ;AC034; set si at the end or colon
29699 00001BEB 2E803C00   cmp     byte [cs:si],_$P_NULL ;AN000; any data after colon
29700 00001BEF 7525      jne     short _$P_CSW00      ;AN000; if yes, process match flags
29701
29702 00001BF1 2E807CFF3A   cmp     byte [cs:si-1],_$P_Colon ;AN000; if no, the switch terminated by colon ?
29703 00001BF6 7509      jne     short _$P_Chk_if_data_required ;AN000; if yes,
29704
29705 00001BF8 2EC706[7119]0900 mov     word [cs:_$P_RC],_$P_Syntax ;AC034; return syntax error
29706 00001BFF EB1C      jmp     short _$P_Chk_SW_Exit ;AN000;
29707
29708 _$P_Chk_if_data_required: ;AN018; no data, no colon
29709             ;cmp     word [es:bx+_$P_Control_Blk.Match_Flag],0
29710 00001C01 26833F00   cmp     word [es:bx],0      ;AN018; should have data? zero match flag means switch followed by nothing is
29711 00001C05 7416      je      short _$P_Chk_SW_Exit ;AN018; match flags not zero so should have something if optional bit is not
29712 on
29713             ;test     word [es:bx+_$P_Control_Blk.Match_Flag],_$P_Optional
29714             ; 02/11/2022 (MSDOS 5.0 IO.SYS SYINIT compatibility)
29715             ;test     word [es:bx],1
29716             ; 12/12/2022
29717             ;test     word [es:bx],_$P_Optional ;AN019; see if no value is valid
29718 00001C07 26F60701   test     byte [es:bx],_$P_Optional
29719 00001C0B 7510      jnz     short _$P_Chk_SW_Exit ;AN019; if so, then leave, else yell
29720
29721 00001C0D 2EC706[7119]0200 mov     word [cs:_$P_RC],_$P_Op_Missing ;AC034; return required operand missing
29722 00001C14 EB07      jmp     short _$P_Chk_SW_Exit ;AN018;
29723
29724 _$P_CSW00:          ;AN000;
29725 00001C16 E88F00     call    _$P_Check_Match_Flags ;AN000; process match flag
29726 00001C19 F8         clc     ;AN000; indicate match
29727             ;jmp     short _$P_Chk_SW_Single_Exit ;AN000;
29728             ; 12/12/2022
29729 00001C1A C3         retn
29730
29731 _$P_Chk_SW_Err0:    ;AN000;
29732 00001C1B F9         stc     ;AN000; not found in switch synonym list
29733             ;jmp     short _$P_Chk_SW_Single_Exit ;AN000;
29734             ; 12/12/2022
29735 00001C1C C3         retn
29736
29737 _$P_Chk_SW_Exit:    ;AN000;
29738 00001C1D 50         push    ax      ;AN000;
29739             ;mov     al,_$P_String      ;AN000;
29740             ;mov     ah,_$P_No_Tag     ;AN000;

```

```

29741      ; 07/07/2023
29742      mov     ax,(_$P_No_Tag<8)|_$P_String
29743      call    _$P_Fill_Result      ;AN000; set result buffer
29744      pop     ax
29745      cld
29746      _$P_Chk_SW_Single_Exit:
29747      retn
29748      ;ELSE
29749      ; stc
29750      ; retn
29751      ;AN000; (of IF SwSw)
29752      ;AN000; this logic works when the SwSw
29753      ;AN000; is reset.
29754      ;*****
29755      ;_$P_Fill_Result
29756      ;
29757      ; Function: Fill the result buffer
29758      ;
29759      ; Input:  AH = Item tag
29760      ;         AL = type
29761      ;         AL = 1: CX,DX has 32bit number (CX = high)
29762      ;         AL = 2: DX has index(offset) into value list
29763      ;         AL = 6: DL has driver # (1-A, 2-B, ... , 26 - Z)
29764      ;         AL = 7: DX has year, CL has month and CH has date
29765      ;         AL = 8: DL has hours, DH has minutes, CL has seconds,
29766      ;             and CH has hundredths
29767      ;         AL = else: cs:SI points to returned string buffer
29768      ;         ES:BX -> CONTROL block
29769      ;
29770      ; Output:  None
29771      ;
29772      ; Use:     _$P_Do_CAPS_String, _$P_Remove_Colon, _$P_Found_SYNONYM
29773      ;
29774      ; Vars:    _$P_DX(w)
29775      ;*****
29776      _$P_Fill_Result:
29777      push     di
29778      mov     di,[es:bx+_$P_Control_Blk.Result_Buf]
29779      mov     [cs:_$P_DX],di
29780      ;mov     [es:di+_$P_Result_Blk.Type],al ;AN000; store type
29781      ;mov     [es:di+_$P_Result_Blk.Item_Tag],ah ;AN000; store item tag
29782      ; 07/09/2023
29783      ;mov     [es:di+_$P_Result_Blk.Type],ax
29784      mov     [es:di],ax
29785      ; store type (al) and item tag (ah)
29786      push     ax
29787      mov     ax,[cs:_$P_Found_SYNONYM] ;AC034; if yes,
29788      mov     [es:di+_$P_Result_Blk.SYNONYM_Ptr],ax
29789      ;AN000; then set it to the result
29790      pop     ax
29791      _$P_RLT04:
29792      cmp     al,_$P_Number
29793      jne     short _$P_RLT00
29794      ;AN000; if number
29795      ;AN000;
29796      _$P_RLT02:
29797      mov     [es:di+_$P_Result_Blk.Picked_Val],dx ;AN000; then store 32bit
29798      mov     [es:di+_$P_Result_Blk.Picked_Val+2],cx ;AN000; number
29799      jmp     short _$P_RLT_Exit
29800      ;AN000;
29801      _$P_RLT00:
29802      cmp     al,_$P_List_Idx
29803      jne     short _$P_RLT01
29804      ;AN000; if list index
29805      ;AN000;
29806      mov     [es:di+_$P_Result_Blk.Picked_Val],dx
29807      ;AN000; then store list index
29808      jmp     short _$P_RLT_Exit
29809      ;AN000;
29810      _$P_RLT01:
29811      cmp     al,_$P_Date_F
29812      je      short _$P_RLT02
29813      ;AN000; Date format ?
29814      ;AN000;
29815      cmp     al,_$P_Time_F
29816      je      short _$P_RLT02
29817      ;AN000; Time format ?
29818      ;AN000;
29819      cmp     al,_$P_Drive
29820      jne     short _$P_RLT03
29821      ;AN000; drive format ?
29822      ;AN000;
29823      mov     [es:di+_$P_Result_Blk.Picked_Val],dl ;AN000; store drive number
29824      jmp     short _$P_RLT_Exit
29825      ;AN000;
29826      _$P_RLT03:
29827      cmp     al,_$P_Complex
29828      jne     short _$P_RLT05
29829      ;AN000; complex format ?
29830      ;AN000;
29831      mov     ax,[cs:_$P_SaveSI_Cmpx] ;AC034; then get pointer in command buffer
29832      inc     ax
29833      ;AN000; skip left Parentheses
29834      mov     [es:di+_$P_Result_Blk.Picked_Val],ax ;AN000; store offset
29835      mov     [es:di+_$P_Result_Blk.Picked_Val+2],ds ;AN000; store segment
29836      jmp     short _$P_RLT_Exit
29837      ;AN000;
29838      _$P_RLT05:
29839      ;AN000;
29840      ;----- AL = 3, 5, or 9
29841      mov     [es:di+_$P_Result_Blk.Picked_Val],si
29842      ;AN000; store offset of STRING_BUF
29843      mov     [es:di+_$P_Result_Blk.Picked_Val+2],cs
29844      ;AN031; store segment of STRING_BUF
29845      push     ax
29846      test    byte [es:bx+_$P_Control_Blk.Function_Flag],_$P_CAP_File
29847      jz      short _$P_RLT_CAP00
29848      ;AN000; need CAPS by file table?
29849      ;AN000;
29850      mov     al,_$P_DOSTBL_File
29851      jmp     short _$P_RLT_CAP02
29852      ;AN000; use file upper case table
29853      ;AN000;
29854      _$P_RLT_CAP00:
29855      test    byte [es:bx+_$P_Control_Blk.Function_Flag],_$P_CAP_Char
29856      jz      short _$P_RLT_CAP01
29857      ;AN000; need CAPS by char table ?
29858      ;AN000;
29859      mov     al,_$P_DOSTBL_Char
29860      ;AN000; use character upper case table
29861      ;AN000;
29862      call    _$P_Do_CAPS_String
29863      ;AN000; process CAPS along the table
29864      _$P_RLT_CAP01:
29865      pop     ax
29866      test    byte [es:bx+_$P_Control_Blk.Function_Flag],_$P_Rm_Colon
29867      jz      short _$P_RLT_Exit
29868      ;AN000; removing colon at end ?
29869      ;AN000;
29870      call    _$P_Remove_Colon
29871      ;AN000; then process it.
29872      ;AN000;
29873      _$P_RLT_Exit:
29874      pop     di
29875      retn
29876      ;AN000;
29877      ;*****

```

```

29865 ; _$P_Check_Match_Flags
29866 ;
29867 ; Function: Check the mutch_flags and make the exit code and set the
29868 ; result buffer
29869 ;
29870 ; Check for types in this order:
29871 ; Complex
29872 ; Date
29873 ; Time
29874 ; Drive
29875 ; Filespec
29876 ; Quoted String
29877 ; Simple String
29878 ;
29879 ; Input: cs:SI -> _$P_STRING_BUF
29880 ; ES:BX -> CONTROL block
29881 ;
29882 ; Output: None
29883 ;
29884 ; Use: _$P_Value, P$_SValue, _$P_Simple_String, _$P_Date_Format
29885 ; _$P_Time_Format, _$P_Complex_Format, _$P_File_Foemat
29886 ; _$P_Drive_Format
29887 ; *****
29888 ;
29889 ; 25/10/2022 - Retro DOS v4.0
29890 ; (MSDOS 5.0 IO.SYS - SYSINIT:19CFh)
29891 ;
29892 ; 14/04/2024 - Retro DOS v5.0
29893 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:1FC3h)
29894 ;
29895 ; 12/12/2022
29896 ; _$P_Check_Match_Flags:
29897 00001CA8 2EC606[141A]00 mov byte [cs:_$P_err_flag],_$P_NULL
29898 ; AN033;AC034;; clear filespec error flag.
29899 00001CAE 50 push ax ; AN000;
29900 ;mov ax,[es:bx+_$P_Control_Blk.Match_Flag] ; AN000;
29901 00001CAF 268B07 mov ax,[es:bx] ; AN000; load match flag(16bit) to ax
29902 00001CB2 09C0 or ax,ax ; AC035; test ax for zero
29903 00001CB4 7517 jnz short _$P_Mat ; AN000; (tm12)
29904 00001CB6 50 push ax ; AN000; (tm12)
29905 00001CB7 53 push bx ; AN000; (tm12)
29906 00001CB8 52 push dx ; AN000; (tm12)
29907 00001CB9 57 push di ; AN000; (tm12)
29908 00001CBA 2EC706[7119]0900 mov word [cs:_$P_RC],_$P_Syntax ; AC034; (tm12)
29909 ;mov ah,_$P_No_Tag ; AN000; (tm12)
29910 ;mov al,_$P_String ; AN000; (tm12)
29911 ; 07/07/2023
29912 00001CC1 B803FF mov ax,(_$P_No_Tag<<8)|_$P_String
29913 00001CC4 E860FF call _$P_Fill_Result ; AN000; (tm12)
29914 00001CC7 5F pop di ; AN000; (tm12)
29915 00001CC8 5A pop dx ; AN000; (tm12)
29916 00001CC9 5B pop bx ; AN000; (tm12)
29917 00001CCA 58 pop ax ; AN000; (tm12)
29918 ; 12/12/2022
29919 ; jmp short _$P_Bridge ; AC035; (tm12)
29920 ; 12/12/2022
29921 ; _$P_Mat: ; AN000; (tm12)
29922 ; jmp short _$P_Match03 ; AN025; (tm09)
29923 ; _$P_Bridge:
29924 00001CCB EB6E jmp short _$P_Match_Exit ; AN000; (tm02)
29925 ;
29926 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
29927 ; (SYSINIT:19F9h)
29928 ; 12/12/2022
29929 ; nop ; db 90h
29930 ;
29931 ; 12/12/2022
29932 ; _$P_Mat:
29933 ; _$P_Match03: ; AN000;
29934 ; test ax,_$P_Num_Val ; 8000h;AN000; Numeric value
29935 ; 07/07/2023
29936 00001CCD F6C480 test ah,(_$P_Num_Val>>8) ; 80h
29937 00001CD0 7412 jz short _$P_Match04 ; AN000;
29938 ;
29939 00001CD2 2EC706[7119]0000 mov word [cs:_$P_RC],_$P_No_Error ; AC034; assume no error
29940 00001CD9 E81E01 call _$P_Value ; AN000; do process
29941 00001CDC 2E833E[7119]09 cmp word [cs:_$P_RC],_$P_Syntax ; AC034; if error, examine the next type
29942 00001CE2 7557 jne short _$P_Match_Exit ; AN000;
29943 ; _$P_Match04: ; AN000;
29944 ; test ax,_$P_SNum_Val ; 4000h ; AN000; signed numeric value
29945 ; 07/07/2023
29946 00001CE4 F6C440 test ah,(_$P_SNum_Val>>8) ; 40h
29947 00001CE7 7412 jz short _$P_Match05 ; AN000;
29948 ;
29949 00001CE9 2EC706[7119]0000 mov word [cs:_$P_RC],_$P_No_Error ; AC034; assume no error
29950 00001CF0 E8E300 call _$P_SValue ; AN000; do process
29951 00001CF3 2E833E[7119]09 cmp word [cs:_$P_RC],_$P_Syntax ; AC034; if error, examine the next type
29952 00001CF9 7540 jne short _$P_Match_Exit ; AN000;
29953 ; _$P_Match05: ; AN000;
29954 ; test ax,_$P_Drv_Only ; 100h;AN000; Drive only
29955 ; 07/07/2023
29956 00001CFB F6C401 test ah,(_$P_Drv_Only>>8) ; 1
29957 00001CFE 7415 jz short _$P_Match06 ; AN000;
29958 ;
29959 00001D00 2EC706[7119]0000 mov word [cs:_$P_RC],_$P_No_Error ; AC034; assume no error
29960 00001D07 E8F202 call _$P_File_Format ; AN000; 1st, call file format
29961 00001D0A E87203 call _$P_Drive_Format ; AN000; check drive format, next
29962 00001D0D 2E833E[7119]09 cmp word [cs:_$P_RC],_$P_Syntax ; AC034; if error, examine the next type
29963 00001D13 7526 jne short _$P_Match_Exit ; AN000;
29964 ; _$P_Match06: ; AN000;
29965 ; test ax,_$P_File_Spc ; 200h;AN000; File spec
29966 ; 07/07/2023
29967 00001D15 F6C402 test ah,(_$P_File_Spc>>8) ; 2
29968 00001D18 7412 jz short _$P_Match07 ; AN000;
29969 ;
29970 00001D1A 2EC706[7119]0000 mov word [cs:_$P_RC],_$P_No_Error ; AC034; assume no error
29971 00001D21 E8D802 call _$P_File_Format ; AN000; do process
29972 00001D24 2E833E[7119]09 cmp word [cs:_$P_RC],_$P_Syntax ; AC034; if error, examine the next type
29973 00001D2A 750F jne short _$P_Match_Exit ; AN000;
29974 ; _$P_Match07: ; AN000;
29975 ; test ax,_$P_Simple_S ; 2000h;AN000; Simple string
29976 ; 07/07/2023
29977 00001D2C F6C420 test ah,(_$P_Simple_S>>8) ; 20h
29978 00001D2F 740A jz short _$P_Match09 ; AN000;
29979 ;
29980 00001D31 2EC706[7119]0000 mov word [cs:_$P_RC],_$P_No_Error ; AC034; assume no error
29981 00001D38 E8BA01 call _$P_Simple_String ; AN000; do process
29982 ; _$P_Match09: ; AN000;
29983 ; _$P_Match_Exit: ; AN000;
29984 00001D3B 2E833E[141A]01 cmp word [cs:_$P_err_flag],_$P_error_filespec ; AC034; bad filespec ?
29985 00001D41 750F jne short _$P_Match2_Exit ; AN033; no, continue
29986 00001D43 2E833E[7119]00 cmp word [cs:_$P_RC],_$P_No_Error ; AN033;AC034;; check for other errors ?
29987 00001D49 7507 jne short _$P_Match2_Exit ; AN033; no, continue
29988 00001D4B 2EC706[7119]0900 mov word [cs:_$P_RC],_$P_Syntax ; AN033;AC034;; set error flag

```

```

29989 _$P_Match2_Exit: ;AN033;
29990 pop ax ;AN000;
29991 retn ;AN000;
29992
29993 ;*****
29994 ; _$P_Remove_Colon;
29995 ;
29996 ; Function: Remove colon at end
29997 ;
29998 ; Input: cs:SI points to string buffer to be examined
29999 ;
30000 ; Output: None
30001 ;
30002 ; Use: _$P_Chk_DBCS
30003 ;*****
30004
30005 _$P_Remove_Colon:
30006 push ax ;AN000;
30007 push si ;AN000;
30008 _$P_RCOL_Loop: ;AN000;
30009 mov al,[cs:si] ;AN000; get character
30010 or al,al ;AN000; end of string ?
30011 jz short _$P_RCOL_Exit ;AN000; if yes, just exit
30012
30013 cmp al,_$P_Colon ;AN000; is it colon ?
30014 jne short _$P_RCOL00 ;AN000;
30015
30016 cmp byte [cs:si+1],_$P_NULL ;AN000; if so, next is NULL ?
30017 jne short _$P_RCOL00 ;AN000; no, then next char
30018
30019 mov byte [cs:si],_$P_NULL ;AN000; yes, remove colon
30020 jmp short _$P_RCOL_Exit ;AN000; and exit.
30021
30022 _$P_RCOL00: ;AN000;
30023 call _$P_Chk_DBCS ;AN000; if not colon, then check if
30024 jnc short _$P_RCOL01 ;AN000; DBCS leading byte.
30025
30026 inc si ;AN000; if yes, skip trailing byte
30027 _$P_RCOL01: ;AN000;
30028 inc si ;AN000; si points to next byte
30029 jmp short _$P_RCOL_Loop ;AN000; loop until NULL encountered
30030
30031 _$P_RCOL_Exit: ;AN000;
30032 pop si ;AN000;
30033 pop ax ;AN000;
30034 retn ;AN000;
30035
30036 ;*****
30037 ; _$P_Do_CAPS_String;
30038 ;
30039 ; Function: Perform capitalization along with the file case map table
30040 ; or character case map table.
30041 ;
30042 ; Input: AL = 2 : Use character table
30043 ; AL = 4 : Use file table
30044 ; cs:SI points to string buffer to be capitalized
30045 ;
30046 ; Output: None
30047 ;
30048 ; Use: _$P_Do_CAPS_Char, _$P_Chk_DBCS
30049 ;*****
30050
30051 _$P_Do_CAPS_String:
30052 push si ;AN000;
30053 push dx ;AN000;
30054 mov dl,al ;AN000; save info id
30055
30056 _$P_DCS_Loop: ;AN000;
30057 mov al,[cs:si] ;AN000; load charater and
30058 call _$P_Chk_DBCS ;AN000; check if DBCS leading byte
30059 jc short _$P_DCS00 ;AN000; if yes, do not need CAPS
30060
30061 or al,al ;AN000; end of string ?
30062 jz short _$P_DCS_Exit ;AN000; then exit.
30063
30064 call _$P_Do_CAPS_Char ;AN000; Here a SBCS char need to be CAPS
30065 mov [cs:si],al ;AN000; stored upper case char to buffer
30066 jmp short _$P_DCS01 ;AN000; process next
30067 _$P_DCS00: ;AN000;
30068 inc si ;AN000; skip DBCS leading and trailing byte
30069 _$P_DCS01: ;AN000;
30070 inc si ;AN000; si point to next byte
30071 jmp short _$P_DCS_Loop ;AN000; loop until NULL encountered
30072 _$P_DCS_Exit: ;AN000;
30073 pop dx ;AN000;
30074 pop si ;AN000;
30075 retn
30076
30077 ;*****
30078 ; _$P_Do_CAPS_Char;
30079 ;
30080 ; Function: Perform capitalization along with the file case map table
30081 ; or character case map table.
30082 ;
30083 ; Input: DL = 2 : Use character table
30084 ; DL = 4 : Use file table
30085 ; AL = character to be capitalized
30086 ;
30087 ; Output: None
30088 ;
30089 ; Use: INT 21h /w AH=65h
30090 ;*****
30091
30092 _$P_Do_CAPS_Char:
30093 cmp al,_$P_ASCII80 ;80h ;AN000; need upper case table ?
30094 jae short _$P_DCC_Go ;AN000;
30095
30096 cmp al,"a" ;AN000; if no,
30097 jb short _$P_CAPS_Ret ;AN000; check if "a" <= AL <= "z"
30098
30099 cmp al,"z" ;AN000;
30100 ja short _$P_CAPS_Ret ;AN000; if yes, make CAPS
30101
30102 and al,_$P_Make_Upper ;0DFh ;AN000; else do nothing.
30103 jmp short _$P_CAPS_Ret ;AN000;
30104 ; 07/07/2023
30105 retn
30106
30107 _$P_DCC_Go: ;AN000;
30108 push bx ;AN000;
30109 push es ;AN000;
30110 push di ;AN000;
30111
30112 ;lea di,[cs:_$P_Char_CAP_Ptr] ;AC034; or use char CAPS table ?

```

```

30113         ;lea di,[_$P_Char_CAP_Ptr]
30114         ; 07/09/2023
30115 00001DAB BF[061A]
30116         mov di,_$P_Char_CAP_Ptr
30117 00001DAE 2E3815
30118 00001DB1 7415
30119         _$P_DCC00:
30120         cmp [cs:di],dl ;AN000; already got table address ?
30121         je short _$P_DCC01 ;AN000; if no,
30122
30123         ;In this next section, ES will be used to pass a 5 byte workarea to INT 21h,
30124         ; the GET COUNTRY INFO call. This usage of ES is required by the function
30125         ; call, regardless of what base register is currently be defined as cs.
30126
30127         push ax ;AN000; get CAPS table thru DOS call
30128         push cx ;AN000;
30129         push dx ;AN000;
30130
30131         push cs ;AC036; pass current base seg into
30132         ;(Note: this used to push CS. BUG...
30133         pop es ;AN000; ES reg, required for
30134         ;get extended country information
30135         ;mov al,dl ; function ;AN000; upper case table
30136         ; 07/07/2023
30137         xchg ax,dx
30138         mov ah,_$P_DOS_Get_TBL ; 65h ;AN000; get extended CDI
30139         mov bx,_$P_DOSTBL_Def ; -1;AN000; get active CON
30140         mov cx,_$P_DOSTBL_BL ; 5 ;AN000; buffer length
30141         ;mov dx,_$P_DOSTBL_Def ;AN000; get for default code page
30142         ; 07/07/2023
30143         mov dx,bx ; 0FFFFh
30144
30145         int 21h ;DI already set to point to buffer
30146         ;AN000; es:di point to buffer that
30147         ;now has been filled in with info
30148         pop dx ;AN000;
30149         pop cx ;AN000;
30150         pop ax ;AN000;
30151
30152         _$P_DCC01: ;AN000;
30153
30154         ;In this next section, ES will be used as the base of the XLAT table, provided
30155         ; by the previous GET COUNTRY INFO DOS call. This usage of ES is made
30156         ; regardless of which base reg is currently the cs reg.
30157
30158         ; 14/04/2024
30159         ;mov bx,[cs:di+_$P_DOS_TBL.Off] ;AN000; get offset of table
30160         ;mov es,[cs:di+_$P_DOS_TBL.Seg] ;AN000; get segment of table
30161         ; 07/07/2023
30162         les bx,[cs:di+_$P_DOS_TBL.Off]
30163         inc bx ;AC035; add '2' to
30164         inc bx ;AC035; BX reg
30165         sub al,_$P_ASCII80 ; 80h ;AN000; make char to index
30166         ;xlat es:[bx] ;AN000; perform case map
30167         es
30168         xlat
30169         pop di ;AN000;
30170         pop es ;AN000;
30171         pop bx ;AN000;
30172         _$P_CAPS_Ret:
30173         retn ;AN000;
30174
30175         ;*****
30176         ; _$P_Value / _$P_SValue
30177         ;
30178         ; Function: Make 32bit value from cs:SI and see value list
30179         ; and make result buffer.
30180         ; _$P_SValue is an entry point for the signed value
30181         ; and this will simply call _$P_Value after the handling
30182         ; of the sign character, "+" or "-"
30183         ;
30184         ; Input: cs:SI -> _$P_STRING_BUF
30185         ; ES:BX -> CONTROL block
30186         ;
30187         ; Output: None
30188         ;
30189         ; Use: _$P_Fill_Result, _$P_Check_OVF
30190         ;
30191         ; Vars: _$P_RC(W), _$P_Flags(RW)
30192         ;*****
30193
30194         ; 26/10/2022 - Retro DOS v4.0
30195         ; (MSDOS 5.0 IO.SYS - SYSINIT:1B0Bh)
30196
30197         ; 07/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
30198         ; (MSDOS 6.21 IO.SYS - SYSINIT:1C46h)
30199         _$P_SValue:
30200         push ax ;AN000; when signed value here
30201         or byte [cs:_$P_Flags2],_$P_Signed ;AC034; indicate a signed numeric
30202         and byte [cs:_$P_Flags2],0FFh-_$P_Neg ;AC034; assume positive value
30203         ;and byte [cs:_$P_Flags2],~_$P_Neg ; 07/07/2023
30204         mov al,[cs:si]
30205         cmp al,_$P_Plus ;AN000; get sign
30206         je short _$P_SVal00 ;AN000; "+" ?
30207
30208         cmp al,_$P_Minus ;AN000; "-" ?
30209         jne short _$P_SVal01 ;AN000; else
30210
30211         or byte [cs:_$P_Flags2],_$P_Neg ;AC034; set this is negative value
30212         _$P_SVal00:
30213         inc si ;AN000; skip sign char
30214         _$P_SVal01:
30215         call _$P_Value ;AN000; and process value
30216         pop ax ;AN000;
30217         retn
30218
30219         ;*****
30220         ; 07/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
30221         ; (MSDOS 6.21 IO.SYS - SYSINIT:1C6Ah)
30222
30223         ; 26/10/2022
30224         _$P_Value:
30225         push ax ;AN000;
30226         push cx ;AN000;
30227         push dx ;AN000;
30228         push si ;AN000;
30229         xor cx,cx ;AN000; cx = higher 16 bits
30230         xor dx,dx ;AN000; dx = lower 16 bits
30231         push bx ;AN000; save control pointer
30232         _$P_Value_Loop:
30233         mov al,[cs:si] ;AN000; get character
30234         or al,al ;AN000; end of line ?
30235         jz short _$P_Value00 ;AN000;
30236
30237         call _$P_0099 ;AN000; make asc(0..9) to bin(0..9)
30238         jc short _$P_Value_Err0 ;AN000;

```



```

30237
30238 00001E0F 30E4      xor     ah,ah          ;AN000;
30239 00001E11 89C5      mov     bp,ax          ;AN000; save binary number
30240
30241      ; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
30242      ; Ref: Disassembled PCDOS 7.1 IBMBIO.COM SYSINIT code
30243      ;
30244      ; Erdogan Tan - July 2023
30245      %if 0
30246      shl     dx,1          ;AN000; to have 2*x
30247      rcl     cx,1          ;AN000; shift left w/ carry
30248      call    _$P_Check_OVF ;AN000; Overflow occurred ?
30249      jc      short _$P_Value_Err0 ;AN000; then error, exit
30250
30251      mov     bx,dx          ;AN000; save low(2*x)
30252      mov     ax,cx          ;AN000; save high(2*x)
30253      shl     dx,1          ;AN000; to have 4*x
30254      rcl     cx,1          ;AN000; shift left w/ carry
30255      call    _$P_Check_OVF ;AN000; Overflow occurred ?
30256      jc      short _$P_Value_Err0 ;AN000; then error, exit
30257
30258      shl     dx,1          ;AN000; to have 8*x
30259      rcl     cx,1          ;AN000; shift left w/ carry
30260      call    _$P_Check_OVF ;AN000; Overflow occurred ?
30261      jc      short _$P_Value_Err0 ;AN000; then error, exit
30262
30263      add     dx,bx          ;AN000; now have 10*x
30264      adc     cx,ax          ;AN000; 32bit ADD
30265      call    _$P_Check_OVF ;AN000; Overflow occurred ?
30266      jc      short _$P_Value_Err0 ;AN000; then error, exit
30267
30268      add     dx,bp          ;AN000; Add the current one degree decimal
30269      adc     cx,0           ;AN000; if carry, add 1 to high 16bit
30270      call    _$P_Check_OVF ;AN000; Overflow occurred ?
30271      jc      short _$P_Value_Err0 ;AN000; then error, exit
30272
30273      inc     si             ;AN000; update pointer
30274      jmp     short _$P_Value_Loop ;AN000; loop until NULL encountered
30275      ;_$_P_Value_Err0:
30276      %endif
30277      ;****
30278      %if 1
30279      ; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
30280      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:2130h)
30281
30282      ; 14/04/2024 - Retro DOS v5.0
30283      xor     ah,ah
30284      mov     bp,ax          ; save binary number
30285      call    _$P_Value_2x_OVF ; multiply cx:dx by 2 and then check overflow
30286      mov     bx,dx          ; ax:bx = 2*(cx:dx)
30287      mov     ax,cx
30288      call    _$P_Value_2x_OVF ; multiply cx:dx by 2 and then check overflow
30289      call    _$P_Value_2x_OVF ; multiply cx:dx by 2 and then check overflow
30290      add     dx,bx          ; 8*(cx:dx)+2*(cx:dx) = 10*(cx:dx)
30291      adc     cx,ax
30292      call    _$P_Value_Chk_Add_OVF
30293      add     dx,bp          ; Add the current one degree decimal
30294      ; if carry, add 1 to high 16bit
30295      adc     cx,0
30296      call    _$P_Value_Chk_Add_OVF ; overflow occurred ?
30297      ; then error, exit (without return here)
30298      inc     si             ; update pointer
30299      jmp     short _$P_Value_Loop
30300
30301      _$P_Value_2x_OVF:
30302      shl     dx,1          ; to have 2*x
30303      rcl     cx,1          ; shift left w/ carry
30304      _$P_Value_Chk_Add_OVF:
30305      call    _$P_Check_OVF ; check overflow (for the last shift or add)
30306      jc      short _$P_Value_OVF
30307      retn
30308      _$P_Value_OVF:
30309      inc     sp             ; skip "call" return address to the caller
30310      inc     sp
30311
30312      ;_$_P_Value_Err0:
30313      %endif
30314      ;****
30315
30316      _$P_Value_Err0:
30317      pop     bx             ;AN000;
30318      jmp     _$P_Value_Err ;AN000; Bridge
30319
30320      _$P_Value00:
30321      pop     bx             ;AN000;
30322      test    byte [cs:$_P_Flags2],$_P_Neg ;AN000; restore control pointer
30323      jz      short _$P_Value01 ;AC034; here cx,dx = 32bit value
30324      ;AN000; was it negative ?
30325      not     cx             ;AN000; +
30326      not     dx             ;AN000; |- Make 2's complement
30327      add     dx,1           ;AN000; |
30328      adc     cx,0           ;AN000; +
30329
30330      _$P_Value01:
30331      mov     si,[es:bx+$_P_Control_Blk.Value_List] ;AN000; / nval = 0
30332      mov     al,[es:si]     ;AN000; si points to value list
30333      ; 07/09/2023
30334      cmp     al,_$P_nval_None ; 0 ;AN000; no value list ?
30335      ;*jne short _$P_Value02 ;AN000;
30336      ;* 07/07/2023
30337      je      short _$P_Value05
30338      ; 07/09/2023
30339      or      al,al
30340      jz      short _$P_Value05 ; _$P_nval_None
30341
30342      mov     al,_$P_Number   ;AN000; Set type
30343      mov     ah,_$P_No_Tag   ;AN000; No ITEM_TAG set
30344      ; 07/07/2023
30345      ;*mov     ax,(_$P_No_Tag<<8)|$_P_Number
30346      ;*jmp     short _$P_Value_Exit ;AN000;
30347
30348      ; 26/10/2022 (MSDOS 5.0 IO.SYS, SYSINIT compatibility)
30349      ; (SYSINIT:1BA5h)
30350      ; 12/12/2022
30351      nop                     ; db 90h
30352
30353      _$P_Value02:
30354      ;IF Val1SW ;AN000; / nval = 1
30355      ;(tm07) cmp al,_$P_nval_Range ;AN000; (Check if value list id #1 is supported)
30356      ;(tm07) jne short _$P_Value03 ;AN000; have range list ?
30357
30358      inc     si             ;AN000;
30359      mov     al,[es:si]     ;AN000; al = number of range
30360

```

```

30361         ; 07/09/2023
30362         ;cmp     al,_$P_No_nrng         ;AN000; (tm07)
30363         ;je      short _$P_Value03     ;AN000; (tm07)
30364 00001E64 08C0         or      al,al
30365 00001E66 745D         jz      short _$P_Value03 ; _$P_No_nrng
30366
30367 00001E68 46           inc     si ;AN000; si points to 1st item_tag
30368         _$P_val02_Loop: ;AN000;
30369 00001E69 2EF606[7D19]80 test    byte [cs:_$P_Flags2],_$P_Signed ;AC034;
30370 00001E6F 751E         jnz     short _$P_Val02_Sign ;AN000;
30371
30372 00001E71 263B4C03      cmp     cx,[es:si+_$P_Val_List.Val_XH] ;AN000; comp cx with XH
30373 00001E75 7234         jb      short _$P_Val02_Next ;AN000;
30374 00001E77 7706         ja      short _$P_Val_In ;AN000;
30375
30376 00001E79 263B5401      cmp     dx,[es:si+_$P_Val_List.Val_XL] ;AN000; comp dx with XL
30377 00001E7D 722C         jb      short _$P_Val02_Next ;AN000;
30378
30379         _$P_Val_In: ;AN000;
30380 00001E7F 263B4C07      cmp     cx,[es:si+_$P_Val_List.Val_YH] ;AN000; comp cx with YH (tm01)
30381 00001E83 7726         ja      short _$P_Val02_Next ;AN000;
30382 00001E85 7237         jb      short _$P_Val_Found ;AN000;
30383
30384 00001E87 263B5405      cmp     dx,[es:si+_$P_Val_List.Val_YL] ;AN000; comp dx with YL
30385 00001E8B 771E         ja      short _$P_Val02_Next ;AN000;
30386
30387 00001E8D EB2F         jmp     short _$P_Val_Found ;AN000;
30388
30389         _$P_val02_Sign: ;AN000;
30390 00001E8F 263B4C03      cmp     cx,[es:si+_$P_Val_List.Val_XH] ;AN000; comp cx with XH
30391 00001E93 7C16         jl      short _$P_Val02_Next ;AN000;
30392 00001E95 7F06         jg      short _$P_SVal_In ;AN000;
30393
30394 00001E97 263B5401      cmp     dx,[es:si+_$P_Val_List.Val_XL] ;AN000; comp dx with XL
30395 00001E9B 7C0E         jl      short _$P_Val02_Next ;AN000;
30396
30397         _$P_SVal_In: ;AN000;
30398 00001E9D 263B4C07      cmp     cx,[es:si+_$P_Val_List.Val_YH] ;AN000; comp cx with YH
30399 00001EA1 7F08         jg      short _$P_Val02_Next ;AN000;
30400
30401 00001EA3 7C19         jl      short _$P_Val_Found ;AN000;
30402
30403 00001EA5 263B5405      cmp     dx,[es:si+_$P_Val_List.Val_YL] ;AN000; comp dx with YL
30404         ;jg      short _$P_Val02_Next ;AN000;
30405         ;jmp     short _$P_Val_Found ;AN000;
30406         ; 07/07/2023
30407 00001EA9 7E13         jng     short _$P_Val_Found
30408
30409         _$P_Val02_Next: ;AN000;
30410 00001EAB 83C609      add     si,_$P_Len_Range ;AN000;
30411 00001EAE FEC8         dec     al ;AN000; loop nrng times in AL
30412 00001EB0 75B7         jne     short _$P_Val02_Loop ;AN000;
30413         ; / Not found
30414 00001EB2 2EC706[7119]0600 mov     word [cs:_$P_RC],_$P_Out_Of_Range ;AC034;
30415         ;mov     al,_$P_Number ;AN000;
30416         ;mov     ah,_$P_No_Tag ;AN000; No ITEM_TAG set
30417         _$P_Value05: ;* 07/07/2023
30418         ; 07/07/2023
30419 00001EB9 B801FF      mov     ax,(_$P_No_Tag<<8)|_$P_Number
30420 00001EBC EB11         jmp     short _$P_Value_Exit ;AN000;
30421
30422         _$P_Val_Found: ;AN000;
30423 00001EBE B001         mov     al,_$P_Number ;AN000;
30424 00001EC0 268A24      mov     ah,[es:si] ;AN000; found ITEM_TAG set
30425 00001EC3 EB0A         jmp     short _$P_Value_Exit ;AN000;
30426
30427         _$P_Value03: ;AN000; / nval = 2
30428
30429         ;IF val2sw ;AN000;(Check if value list id #2 is supported)
30430         ;;;cmp     al,$P_nval_Value ;AN000; have match list ? ASSUME nval=2,
30431         ;;;jne     $P_Value04 ;AN000; even if it is 3 or more.
30432         ;(tm07) inc si ;AN000;
30433         ;(tm07) mov al,es:[si] ;AN000; al = nrng
30434         ; mov     ah,$P_Len_Range ;AN000;
30435         ; mul     ah ;AN000; Skip nrng field
30436         ; inc     ax ;AN000;
30437         ; add     si,ax ;AN000; si points to nval
30438         ; mov     al,es:[si] ;AN000; get nval
30439         ; inc     si ;AN000; si points to 1st item_tag
30440         _$P_val03_Loop: ;AN000;
30441         ; cmp     cx,es:[si+$P_Val_XH] ;AN000; comp cx with XH
30442         ; jne     $P_val03_Next ;AN000;
30443         ;
30444         ; cmp     dx,es:[si+$P_Val_XL] ;AN000; comp dx with XL
30445         ; je      $P_Val_Found ;AN000;
30446         ;
30447         _$P_val03_Next: ;AN000;
30448         ; add     si,$P_Len_Value ;AN000; points to next value choice
30449         ; dec     al ;AN000; loop nval times in AL
30450         ; jne     $P_val03_Loop ;AN000;
30451         ; / Not found
30452         ; mov     psdata_seg:$P_RC,$P_Not_in_val ;AC034;
30453         ; mov     al,$P_Number ;AN000;
30454         ; mov     ah,$P_No_Tag ;AN000; No ITEM_TAG set
30455         ; jmp     short $P_Value_Exit ;AN000;
30456         ;
30457         ;ENDIF ;AN000;(of val2sw)
30458         _$P_Value04:
30459
30460         _$P_Value_Err: ;AN000;
30461 00001EC5 2EC706[7119]0900 mov     word [cs:_$P_RC],_$P_Syntax ;AC034;
30462         ;mov     al,_$P_String ;AN000; Set type
30463         ;mov     ah,_$P_No_Tag ;AN000; No ITEM_TAG set
30464         ; 07/09/2023
30465         ; 07/07/2023
30466 00001ECC B803FF      mov     ax,(_$P_No_Tag<<8)|_$P_String
30467         _$P_Value_Exit: ;AN000;
30468 00001ECF E855FD      call    _$P_Fill_Result ;AN000;
30469 00001ED2 5E         pop     si ;AN000;
30470 00001ED3 5A         pop     dx ;AN000;
30471 00001ED4 59         pop     cx ;AN000;
30472 00001ED5 58         pop     ax ;AN000;
30473 00001ED6 C3         retn    ;AN000;
30474
30475         ; 28/03/2019 - Retro DOS v4.0
30476
30477         ;*****
30478         ; _$P_Check_OVF
30479         ;
30480         ; Function: Check if overflow is occurred with consideration of
30481         ; signed or un-signed numeric value
30482         ;
30483         ; Input: Flag register
30484         ;

```



```

30485 ; Output: CY = 1 : Overflow
30486 ;
30487 ; Vars: _$P_Flags(R)
30488 ;*****
30489 ;
30490 ; 26/10/2022
30491 _$P_Check_OVF:
30492 00001ED7 9C pushf ;AN000;
30493 00001ED8 2EF606[7D19]02 test byte [cs:_$P_Flags2],_$P_Neg ;AC034; is it negative value ?
30494 00001EDE 7502 jnz short _$P_COVF ;AN000; if no, check overflow
30495 ;
30496 00001EE0 9D popf ;AN000; by the CY bit
30497 00001EE1 C3 retn ;AN000;
30498 ;
30499 _$P_COVF:
30500 00001EE2 9D popf ;AN000;
30501 00001EE3 7002 jo short _$P_COVF00 ;AN000; else,
;AN000; check overflow by the OF
30502 ;
30503 00001EE5 F8 clc ;AN000; indicate it with CY bit
30504 00001EE6 C3 retn ;AN000; CY=0 means no overflow
30505 ;
30506 _$P_COVF00:
30507 00001EE7 F9 stc ;AN000;
30508 00001EE8 C3 retn ;AN000; and CY=1 means overflow
30509 ;
30510 ;*****
30511 ; _$P_0099;
30512 ;
30513 ; Function: Make ASCII 0-9 to Binary 0-9
30514 ;
30515 ; Input: AL = character code
30516 ;
30517 ; Output: CY = 1 : AL is not number
30518 ; CY = 0 : AL contains binary value
30519 ;*****
30520 ; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
30521 %if 0
30522 _$P_0099:
30523 cmp al,"0" ;AN000;
30524 ;jb short _$P_0099Err ;AN000; must be 0 =< al =< 9
30525 ; 12/12/2022
30526 ;jb short _$P_0099Err2 ; cf=1
30527 ;
30528 cmp al,"9" ;AN000;
30529 ;ja short _$P_0099Err ;AN000; must be 0 =< al =< 9
30530 ;
30531 sub al,"0" ;AN000; make char -> bin
30532 ; 12/12/2022
30533 ; cf=0
30534 ;clc
30535 ;retn ;AN000; indicate no error
30536 ;AN000;
30537 _$P_0099Err:
30538 stc ;AN000;
30539 _$P_0099Err2: ; 12/12/2022
30540 retn ;AN000;
30541 %endif
30542 ;
30543 ; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
30544 %if 1
30545 _$P_0099:
30546 cmp al,"0" ; cmp al,30h
30547 00001EE9 3C30 ; must be 0 =< al =< 9
30548 00001EEB 7207 ; cmp al,3Ah
30549 00001EED 3C3A ; cf=0 -> cf=1
30550 00001EEF F5 cmc
30551 00001EF0 7202 ;jb short _$P_0099Err
30552 00001EF2 2C30 sub al,"0" ; sub al,30h ; make char -> bin
30553 ; cf=0
30554 _$P_0099Err: ; cf=1
30555 00001EF4 C3 retn
30556 %endif
30557 ;
30558 ;*****
30559 ; _$P_Simple_String
30560 ;
30561 ; Function: See value list for the simple string
30562 ; and make result buffer.
30563 ;
30564 ; Input: CS:SI -> _$P_STRING_BUF
30565 ; ES:BX -> CONTROL block
30566 ;
30567 ; Output: None
30568 ;
30569 ; Use: _$P_Fill_Result, _$P_String_Comp
30570 ;
30571 ; Vars: _$P_RC(W)
30572 ;*****
30573 ;
30574 _$P_Simple_String:
30575 00001EF5 50 push ax ;AN000;
30576 00001EF6 53 push bx ;AN000;
30577 00001EF7 52 push dx ;AN000;
30578 00001EF8 57 push di ;AN000;
30579 00001EF9 268B7F06 mov di,[es:bx+_$P_Control_Blk.value_List] ;AN000; di points to value list
30580 00001EFD 268A05 mov al,[es:di] ;AN000; get nval
30581 00001F00 08C0 or al,al ;AN000; no value list ?
30582 00001F02 7504 jnz short _$P_Sim00 ;AN000; then
30583 ;
30584 00001F04 B4FF mov ah,_$P_No_Tag ;AN000; No ITEM_TAG set
30585 00001F06 EB4C jmp short _$P_Sim_Exit ;AN000; and set result buffer
30586 ;
30587 _$P_Sim00:
30588 ;IF val3SW+keySW ;AN000;
30589 00001F08 3C03 cmp al,_$P_nval_String ;AN000;(Check if keyword or value list id #3 is supported)
30590 00001FOA 753F jne short _$P_Sim01 ;AN000; String choice list provided ?
30591 ;AN000; if no, syntax error
30592 00001F0C 47 inc di ;AN000;
30593 00001F0D 268A05 mov al,[es:di] ;AN000; al = nrng
30594 00001F10 B409 mov ah,_$P_Len_Range ;AN000;
30595 00001F12 F6E4 mul ah ;AN000; Skip nrng field
30596 00001F14 40 inc ax ;AN000; ax = (nrng*9)+1
30597 00001F15 01C7 add di,ax ;AN000; di points to nval
30598 00001F17 268A05 mov al,[es:di] ;AN000; get nnval
30599 00001F1A B405 mov ah,_$P_Len_Value ;AN000;
30600 00001F1C F6E4 mul ah ;AN000; Skip nnval field
30601 00001F1E 40 inc ax ;AN000; ax = (nnval*5)+1
30602 00001F1F 01C7 add di,ax ;AN000; di points to nstrval
30603 00001F21 268A05 mov al,[es:di] ;AN000; get nstrval c
30604 00001F24 47 inc di ;AC035; add '2' to
30605 00001F25 47 inc di ;AC035; DI reg
30606 ;AN000; di points to 1st string in list
30607 _$P_Sim_Loop:
30608 00001F26 268B2D mov bp,[es:di] ;AN000; get string pointer

```

```

30609 00001F29 E83200      call    _$P_String_Comp      ;AN000; compare it with operand
30610 00001F2C 7312      jnc     short _$P_Sim_Found  ;AN000; found on list ?
30611
30612 00001F2E 83C703      add     di,_$P_Len_String ; 3 ;AN000; if no, point to next choice
30613 00001F31 FEC8      dec     al                  ;AN000; loop nstval times in AL
30614 00001F33 75F1      jne     short _$P_Sim_Loop  ;AN000;
30615                                ;AN000; / Not found
30616 00001F35 2EC706[7119]0800      mov     word [cs:$_P_RC],$_P_Not_In_Str ;AC034;
30617 00001F3C B4FF      mov     ah,$_P_No_Tag       ;AN000; No ITEM_TAG set
30618 00001F3E EB14      jmp     short _$P_Sim_Exit  ;AN000;
30619
30620      _$P_Sim_Found:
30621 00001F40 268A65FF      mov     ah,[es:di-1]       ;AN000; set item_tag
30622 00001F44 B002      mov     al,$_P_List_Idx    ;AN000;
30623 00001F46 268B15      mov     dx,[es:di]         ;AN000; get address of STRING
30624 00001F49 EB0B      jmp     short _$P_Sim_Exit0 ;AN000;
30625 ;ENDIF
30626      _$P_Sim01:
30627 00001F4B 2EC706[7119]0900      mov     word [cs:$_P_RC],$_P_Syntax ;AC034;
30628 00001F52 B4FF      mov     ah,$_P_No_Tag       ;AN000; No ITEM_TAG set
30629      _$P_Sim_Exit:
30630 00001F54 B003      mov     al,$_P_String      ;AN000; Set type
30631      _$P_Sim_Exit0:
30632 00001F56 E8CEFC      call    _$P_Fill_Result    ;AN000;
30633 00001F59 5F      pop     di                 ;AN000;
30634 00001F5A 5A      pop     dx                 ;AN000;
30635 00001F5B 5B      pop     bx                 ;AN000;
30636 00001F5C 58      pop     ax                 ;AN000;
30637 00001F5D C3      retn                      ;AN000;
30638
30639 ;*****
30640 ; _$P_String_Comp:
30641 ;
30642 ; Function: Compare two string
30643 ;
30644 ; Input:      cs:SI -> 1st string
30645 ;            ES:BP -> 2nd string (Must be upper case)
30646 ;            ES:BX -> CONTROL block
30647 ;
30648 ; Output:     CY = 1 if not match
30649 ;
30650 ; Use:        _$P_Chk_DBCS, _$P_Do_CAPS_Char
30651 ;
30652 ; Vars:       _$P_Keyor_SW_Ptr(W), _$P_Flags(R), _$P_KeyorSW_Ptr
30653 ;*****
30654
30655      _$P_String_Comp:
30656 00001F5E 50      push    ax                ;AN000;
30657 00001F5F 55      push    bp                ;AN000;
30658 00001F60 52      push    dx                ;AN000;
30659 00001F61 56      push    si                ;AN000;
30660 00001F62 B202      mov     di,$_P_DOSTBL_Char ;AN000; use character case map table
30661      _$P_SCOM00:
30662 00001F64 2E8A04      mov     al,[cs:si]         ;AN000; get command character
30663 00001F67 E81502      call    _$P_Chk_DBCS      ;AN000; DBCS ?
30664 00001F6A 723A      jc      short _$P_SCOM00   ;AN000; yes, DBCS
30665
30666 00001F6C E82AFE      call    _$P_Do_CAPS_Char   ;AN000; else, upper case map before comparison
30667 ;IF KeySw+SwSw
30668 00001F6F 2EF606[7D19]08      test    byte [cs:$_P_Flags2],$_P_Key_Cmp ;AC034; keyword search ?
30669 00001F75 740D      jz      short _$P_SCOM04   ;AN000;
30670
30671 00001F77 3C3D      cmp     al,$_P_Keyword      ;AN000; "=" is delimiter
30672 00001F79 751F      jne     short _$P_SCOM03    ;AN000; IF "=" on command line AND (bp+1=> char after the "=" in synonym
list)
30673
30674 00001F7B 26807E0100      cmp     byte [es:bp+1],$_P_NULL ;AN021; at end of keyword string in the control block THEN
30675 00001F80 756D      jne     short _$P_SCOM_Differ ;AN021;
30676
30677 00001F82 EB13      jmp     short _$P_SCOM05    ;AN000; keyword found in synonym list
30678
30679      _$P_SCOM04:
30680 00001F84 2EF606[7D19]10      test    byte [cs:$_P_Flags2],$_P_SW_Cmp ;AC034; switch search ?
30681 00001F8A 740E      jz      short _$P_SCOM03    ;AN000;
30682
30683 00001F8C 3C3A      cmp     al,$_P_Colon        ;AN000; ":" is delimiter, at end of switch on command line
30684 00001F8E 750A      jne     short _$P_SCOM03    ;AN000; continue compares
30685
30686 ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30687 ; cmp byte [es:bp+0],$_P_NULL
30688 ; 11/12/2022
30689 00001F90 26807E0000      cmp     byte [es:bp],$_P_NULL ;AN021; IF at end of switch on command AND
30690 00001F95 7558      jne     short _$P_SCOM_Differ ;AN021; at end of switch string in the control block THEN
30691
30692      _$P_SCOM05:
30693 00001F97 46      inc     si                 ;AN000; found a match
30694 00001F98 EB58      jmp     short _$P_SCOM_Same ;AN000; si points to just after "=" or ":"
30695
30696      _$P_SCOM03:
30697 ;ENDIF
30698 ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30699 ; cmp al,[es:bp+0]
30700 ; 11/12/2022
30701 00001F9A 263A4600      cmp     al,[es:bp]         ;AN000; compare operand w/ a synonym
30702 00001F9E 751B      jne     short _$P_SCOM_Differ0 ;AN000; if different, check ignore colon option
30703
30704 00001FA0 08C0      or      al,al              ;AN000; end of line
30705 00001FA2 744E      jz      short _$P_SCOM_Same ;AN000; if so, exit
30706
30707 ; 12/12/2022
30708 ; inc si
30709 ; inc bp
30710 ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30711 00001FA4 EB11      jmp     short _$P_SCOM01    ;AN000; loop until NULL or "=" or ":" found in case
30712
30713      _$P_SCOM00:
30714 ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30715 ; cmp al,[es:bp+0]
30716 ; 11/12/2022
30717 00001FA6 263A4600      cmp     al,[es:bp]         ;AN000; compare leading byte
30718 00001FAA 7543      jne     short _$P_SCOM_Differ ;AN000; if not match, say different
30719
30720 00001FAC 46      inc     si                 ;AN000; else, load next byte
30721 00001FAD 2E8A04      mov     al,[cs:si]         ;AN000; and
30722 00001FB0 45      inc     bp                 ;AN000;
30723 ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30724 ; cmp al,[es:bp+0]
30725 ; 11/12/2022
30726 00001FB1 263A4600      cmp     al,[es:bp]         ;AN000; compare 2nd byte
30727 00001FB5 7538      jne     short _$P_SCOM_Differ ;AN000; if not match, say different, too
30728
30729 ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30730 ; 12/12/2022
30731      _$P_SCOM01:

```

```

30732 00001FB7 46      inc     si                ;AN000; else update operand pointer
30733 00001FB8 45      inc     bp                ;AN000; and synonym pointer
30734                                     ;_$_SCOM01:
30735 00001FB9 EBA9     jmp     short $_P_SCOM_Loop ;AN000; loop until NULL or "=" or "/" found in case
30736                                     ;_$_SCOM_Differ0:
30737                                     ;AN000;
30738                                     ;IF SWSW
30739 00001FBB 2EF606[7D19]40 test    byte [cs:$_P_Flags2],$_P_SW ;AC034;(tm10)
30740 00001FC1 740E     jz      short $_P_not_applicable ;AN000;(tm10)
30741                                     ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30742                                     ;test word [es:bx+$_P_Control_Blk.Function_Flag],$_P_colon_is_not_necessary ;AN000;(tm10)
30743                                     ; 12/12/2022
30744 00001FC3 26F6470220 test    byte [es:bx+$_P_Control_Blk.Function_Flag],$_P_colon_is_not_necessary
30745 00001FC8 7407     jz      short $_P_not_applicable ;AN000;(tm10)
30746                                     ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30747                                     ;cmp byte [es:bp+0],$_P_NULL
30748                                     ; 11/12/2022
30749 00001FCA 26807E0000 cmp     byte [es:bp],$_P_NULL ;AN000;(tm10)
30750                                     ;(deleted ;AN025;) jne short $_P_not_applicable ;AN000;(tm10)
30751 00001FCF 7421     je      short $_P_SCOM_Same ;AN025;(tm10)
30752                                     ;_$_P_not_applicable:
30753                                     ;AN000;(tm10)
30754                                     ;ENDIF
30755                                     ;test word [es:bx+$_P_Control_Blk.Match_Flag],$_P_Ig_Colon
30756                                     ;AN000; ignore colon option specified ?
30757                                     ;test byte [es:bx+$_P_Control_Blk.Match_Flag],$_P_Ig_Colon
30758                                     ; 12/12/2022
30759 00001FD1 26F60710 test    byte [es:bx],$_P_Ig_Colon
30760                                     ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30761                                     ;test word [es:bx],$_P_Ig_Colon ; 10h
30762 00001FD5 7418     jz      short $_P_SCOM_Differ ;AN000; if no, say different.
30763                                     ;cmp al,$_P_Colon
30764                                     ;AN000; End up with ":" and
30765 00001FD7 3C3A     jne     short $_P_SCOM02 ;AN000; subsequently
30766                                     ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30767 00001FD9 7509     cmp     byte [es:bp+0],$_P_NULL
30768                                     ; 11/12/2022
30769 00001FDB 26807E0000 cmp     byte [es:bp],$_P_NULL ;AN000; NULL ?
30770 00001FE0 750D     jne     short $_P_SCOM_Differ ;AN000; if no, say different
30771 00001FE2 EB0E     jmp     short $_P_SCOM_Same ;AN000; else, say same
30772                                     ;_$_SCOM02:
30773                                     ;AN000;
30774 00001FE4 3C00     cmp     al,$_P_NULL ;AN000; end up NULL and :
30775 00001FE6 7507     jne     short $_P_SCOM_Differ ;AN000;
30776                                     ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30777                                     ;cmp byte [es:bp+0],$_P_Colon
30778                                     ; 11/12/2022
30779 00001FE8 26807E003A cmp     byte [es:bp],$_P_Colon;AN000; if no, say different
30780 00001FED 7403     je      short $_P_SCOM_Same ;AN000; else, say same
30781                                     ;_$_SCOM_Differ:
30782                                     ;AN000;
30783 00001FEF F9      stc     ;AN000; indicate not found
30784 00001FF0 EB05     jmp     short $_P_SCOM_Exit ;AN000;
30785                                     ;_$_SCOM_Same:
30786                                     ;AN000;
30787                                     ; 12/12/2022
30788                                     ; cf=0
30789 00001FF2 2E8936[8019] mov     [cs:$_P_KEYorSW_Ptr],si ;AC034; for later use by keyword or switch
30790                                     ; 12/12/2022
30791                                     ;clc
30792                                     ;AN000; indicate found
30793                                     ;_$_SCOM_Exit:
30794                                     ;AN000;
30795 00001FF7 5E      pop     si ;AN000;
30796 00001FF8 5A      pop     dx ;AN000;
30797 00001FF9 5D      pop     bp ;AN000;
30798 00001FFA 58      pop     ax ;AN000;
30799 00001FFB C3      retn
30800                                     ; 30/03/2019
30801                                     ;IF FileSW+DrvSW
30802                                     ;AN000;(check if file spec or drive only is supported)
30803                                     ;*****
30804                                     ;_$_File_Format;
30805                                     ;
30806                                     ; Function: Check if the input string is valid file spec format.
30807                                     ; And set the result buffer.
30808                                     ;
30809                                     ; Input: cs:SI -> $_P_STRING_BUF
30810                                     ; ES:BX -> CONTROL block
30811                                     ;
30812                                     ; Output: None
30813                                     ;
30814                                     ; Use: $_P_Fill_Result, $_P_Chk_DBCS, $_P_FileSp_Chk
30815                                     ;
30816                                     ; Vars: $_P_RC(w), $_P_SI_Save(w), $_P_Terminator(w), $_P_SaveSI_Cmpx(R)
30817                                     ; $_P_SaveSI_Cmpx(R)
30818                                     ;*****
30819                                     ;_$_File_Format:
30820                                     ;push ax
30821                                     ;AN000;
30822                                     ;push di
30823                                     ;AN000;
30824                                     ;push si
30825                                     ;AN000;
30826 00001FFC 50      mov     di,[cs:$_P_SaveSI_Cmpx] ;AC034; get user buffer address
30827 00001FFD 57      mov     al,[cs:si] ;AN000; load character
30828 00001FFE 56      or      al,al ;AN000; end of line ?
30829 00001FFF 2E8B3E[7E19] jz      short $_P_FileF_Err ;AN000; if yes, error exit
30830                                     ;call $_P_FileSp_Chk
30831                                     ;AN000; else, check if file special character
30832 00002004 2E8A04     jne     short $_P_FileF03 ;AN000; if yes,
30833 00002007 08C0     mov     byte [cs:$_P_err_flag],$_P_error_filespec
30834 00002009 7413     ;AN033;AC034;; set error flag- bad char.
30835                                     ;pop si
30836                                     ;AN033;
30837 0000200B E85D00     mov     byte [cs:si],$_P_NULL ;AN033;
30838 0000200E 7523     pop     di ;AN033;
30839 00002010 2EC606[141A]01 jmp     short $_P_FileF02 ;AN033;
30840                                     ;_$_FileF_Err:
30841                                     ;AN000;
30842 00002016 5E      pop     si ;AN000;
30843 00002017 2EC60400     mov     byte [cs:si],$_P_NULL ;AN000;
30844 0000201B 5F      pop     di ;AN000;
30845 0000201C EB3E     ;
30846                                     ;test word [es:bx+$_P_Control_Blk.Match_Flag],$_P_Optional ;AN000; is it optional ?
30847 0000201E 5E      test    byte [es:bx+$_P_Control_Blk.Match_Flag],$_P_Optional
30848 0000201F 2EC60400     ; 12/12/2022
30849 00002023 5F      test    byte [es:bx],$_P_Optional
30850                                     ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
30851                                     ;
30852                                     ;
30853                                     ;
30854 00002024 26F60701     ;
30855                                     ;

```

```

30856          ;test word [es:bx],_$_P_Optional
30857 00002028 7532      jnz short $_P_FileF02 ;AN000;
30858
30859 0000202A 2EC706[7119]0200      mov word [cs:$_P_RC],$_P_Op_Missing ;AC034; 3/17/87
30860 00002031 EB29      jmp short $_P_FileF02 ;AN000;
30861
30862      $_P_FileF03: ;AN000;
30863 00002033 58          pop ax ;AN000; discard save si
30864 00002034 56          push si ;AN000; save new si
30865      $_P_FileF_Loop1: ;AN000;
30866 00002035 2E8A04      mov al,[cs:si] ;AN000; load character (not special char)
30867 00002038 08C0      or al,al ;AN000; end of line ?
30868 0000203A 741E      jz short $_P_FileF_RLT ;AN000;
30869
30870 0000203C E82C00      call $_P_FileSp_Chk ;AN000; File special character ?
30871 0000203F 740B      je short $_P_FileF00 ;AN000;
30872
30873 00002041 E83B01      call $_P_Chk_DBCS ;AN000; no, then DBCS ?
30874 00002044 7302      jnc short $_P_FileF01 ;AN000;
30875 00002046 47          inc di ;AN000; if yes, skip next byte
30876 00002047 46          inc si ;AN000;
30877      $_P_FileF01: ;AN000;
30878 00002048 47          inc di ;AN000;
30879 00002049 46          inc si ;AN000;
30880 0000204A EBE9      jmp short $_P_FileF_Loop1 ;AN000;
30881
30882      ;
30883 0000204C 2EA2[7719]      mov [cs:$_P_Terminator],al ;AC034;
30884 00002050 2EC60400      mov byte [cs:si],$_P_NULL ;AN000; update end of string
30885 00002054 47          inc di ;AN000;
30886 00002055 2E893E[7319]      mov [cs:$_P_SI_Save],di ;AC034; update next pointer in command line
30887      $_P_FileF_RLT: ;AN000;
30888 0000205A 5E          pop si ;AN000;
30889 0000205B 5F          pop di ;AN000;
30890      $_P_FileF02: ;AN000;
30891 0000205C 58          pop ax ;AN000; (tm14)
30892          ;test ax,$_P_File_Spc ; 200h ;AN000; (tm14)
30893          ; 08/07/2023
30894 0000205D F6C402      test ah,($_P_File_Spc>>8) ; 2
30895 00002060 7408      jz short $_P_Drv_Only_Exit ;AN000; (tm14)
30896
30897 00002062 50          push ax ;AN000; (tm14)
30898          ;mov ah,$_P_No_Tag ;AN000; set
30899          ;mov al,$_P_File_Spec ;AN000; result
30900          ; 08/07/2023
30901 00002063 B805FF      mov ax,($_P_No_Tag<<8)|$_P_File_Spec ; 0FF05h
30902          ; set result
30903 00002066 E8BEFB      call $_P_Fill_Result ;AN000; buffer to file spec
30904 00002069 58          pop ax ;AN000;
30905
30906      $_P_Drv_Only_Exit: ;AN000; (tm14)
30907 0000206A C3          retn ;AN000;
30908
30909      ;*****
30910      ; $_P_FileSp_Chk
30911      ;
30912      ; Function: Check if the input byte is one of file special characters
30913      ;
30914      ; Input: cs:SI -> $_P_STRING_BUF
30915      ; AL = character code to be examined
30916      ;
30917      ; Output: ZF = 1 , AL is one of special characters
30918      ;*****
30919
30920      $_P_FileSp_Chk:
30921 0000206B 53          push bx ;AN000;
30922 0000206C 51          push cx ;AN000;
30923          ;lea bx,[cs:$_P_FileSp_Char] ;AC034; special character table
30924          ;lea bx,[_$_P_FileSp_Char] ; "[]|<>+=;\" at
30925          ; 07/09/2023 ; MSDOS 6.21 IO.SYS - SYSINIT:1838h
30926          mov bx,$_P_FileSp_Char
30927 0000206D BB[0B1A]      mov cx,$_P_FileSp_Len ; 9 ;AN000; load length of it
30928 00002070 B90900      ;
30929      $_P_FileSp_Loop: ;AN000;
30930 00002073 2E3A07      cmp al,[cs:bx] ;AN000; is it one of special character ?
30931 00002076 7404      je short $_P_FileSp_Exit ;AN000;
30932
30933 00002078 43          inc bx ;AN000;
30934 00002079 E2F8      loop $_P_FileSp_Loop ;AN000;
30935
30936 0000207B 41          inc cx ;AN000; reset ZF
30937      $_P_FileSp_Exit: ;AN000;
30938 0000207C 59          pop cx ;AN000;
30939 0000207D 5B          pop bx ;AN000;
30940 0000207E C3          retn
30941
30942      ;ENDIF ;AN000;(of FileSw+DrvSw)
30943
30944      ;IF DrvSw ;AN000;(Check if drive only is supported)
30945
30946      ;*****
30947      ; $_P_Drive_Format;
30948      ;
30949      ; Function: Check if the input string is valid drive only format.
30950      ; And set the result buffer.
30951      ;
30952      ; Input: cs:SI -> $_P_STRING_BUF
30953      ; ES:BX -> CONTROL block
30954      ;
30955      ; Output: None
30956      ;
30957      ; Use: $_P_Fill_Result, $_P_Chk_DBCS
30958      ;
30959      ; Vars: $_P_RC(w)
30960      ;*****
30961
30962      $_P_Drive_Format:
30963 0000207F 50          push ax ;AN000;
30964 00002080 52          push dx ;AN000;
30965 00002081 2E8A04      mov al,[cs:si] ;AN000;
30966 00002084 08C0      or al,al ;AN000; if null string
30967 00002086 7436      je short $_P_Drv_Exit ;AN000; do nothing
30968
30969 00002088 E8F400      call $_P_Chk_DBCS ;AN000; is it leading byte ?
30970 0000208B 722A      jc short $_P_Drv_Err ;AN000;
30971
30972 0000208D 2E837C013A      cmp word [cs:si+1],$_P_Colon ;AN000; "d", ":", 0 ?
30973 00002092 740D      je short $_P_DrvF00 ;AN000;
30974
30975          ;test word [es:bx+$_P_Control_Blk.Match_Flag],$_P_Ig_Colon
30976          ;test byte [es:bx+$_P_Control_Blk.Match_Flag],$_P_Ig_Colon ;AN000; colon can be ignored?
30977          ; 12/12/2022
30978 00002094 26F60710      test byte [es:bx],$_P_Ig_Colon
30979          ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)

```

```

30980             ;test word [es:bx],_$_P_Ig_Colon
30981 00002098 741D jz short $_P_Drv_Err ;AN000;
30982
30983 0000209A 2E807C0100 cmp byte [cs:si+1],_$_P_NULL ;AN000; "d", 0 ?
30984 0000209F 7516 jne short $_P_Drv_Err ;AN000;
30985
30986 $_P_DrvF00: ;AN000;
30987 000020A1 0C20 or al,_$_P_Make_Lower ;AN000; lower case
30988 000020A3 3C61 cmp al,"a" ;AN000; drive letter must
30989 000020A5 7210 jb short $_P_Drv_Err ;AN000; in range of
30990
30991 000020A7 3C7A cmp al,"z" ;AN000; "a"-"z"
30992 000020A9 770C ja short $_P_Drv_Err ;AN000; if no, error
30993
30994 000020AB 2C60 sub al,"a"-1 ;AN000; make text drive to binary drive
30995 000020AD 88C2 mov dl,al ;AN000; set
30996 ;mov ah,_$_P_No_Tag ;AN000; result
30997 ;mov al,_$_P_Drive ;AN000; buffer
30998 ; 08/07/2023
30999 000020AF B806FF mov ax,(_$_P_No_Tag<<8)|_$_P_Drive ; 0FF06h
31000 ; set result buffer
31001 000020B2 E872FB call $_P_Fill_Result ;AN000; to drive
31002 000020B5 EB07 jmp short $_P_Drv_Exit ;AN000;
31003
31004 $_P_Drv_Err: ;AN000;
31005 000020B7 2EC706[7119]0900 mov word [cs:_$_P_RC],_$_P_Syntax ;AC034;
31006 $_P_Drv_Exit: ;AN000;
31007 000020BE 5A pop dx ;AN000;
31008 000020BF 58 pop ax ;AN000;
31009 000020C0 C3 retn ;AN000;
31010
31011 ;ENDIF ;AN000;(of DrvSW)
31012
31013 ;*****
31014 ; $_P_Skip_Delim;
31015 ;
31016 ; Function: Skip delimiters specified in the PARMS list, white space
31017 ; and comma.
31018 ;
31019 ; Input: DS:SI -> Command String
31020 ; ES:DI -> Parameter List
31021 ;
31022 ; Output: CY = 1 if the end of line encountered
31023 ; CY = 0 then SI move to 1st non-delimiter character
31024 ; AL = Last examined character
31025 ;
31026 ; Use: $_P_Chk_EOL, $_P_Chk_Delim,
31027 ;
31028 ; Vars: $_P_Flags(R)
31029 ;*****
31030
31031 $_P_Skip_Delim:
31032 $_P_Skip_Delim_Loop: ;AN000;
31033 000020C1 AC lodsb ;AN000;
31034 000020C2 E81E00 call $_P_Chk_EOL ;AN000; is it EOL character ?
31035 000020C5 7416 jz short $_P_Skip_Delim_CY ;AN000; if yes, exit w/ CY on
31036
31037 000020C7 E84E00 call $_P_Chk_Delim ;AN000; is it one of delimiters ?
31038 000020CA 7514 jnz short $_P_Skip_Delim_NCY ;AN000; if no, exit w/ CY off
31039
31040 000020CC 2EF606[7D19]20 test byte [cs:_$_P_Flags2],_$_P_Extra ;AC034; extra delim or comma found ?
31041 000020D2 74ED jz short $_P_Skip_Delim_Loop ;AN000; if no, loop
31042
31043 000020D4 2EF606[7D19]41 test byte [cs:_$_P_Flags2],_$_P_SW+_$_P_equ ;AC034; /x , or xxx=zzz , (tm08)
31044 ;jz short $_P_Exit_At_Extra ;AN000; no switch, no keyword (tm08)
31045 ; 08/07/2023
31046 ; cf=0
31047 000020DA 7505 jnz short $_P_Skip_Delim_Exit
31048 000020DC C3 retn
31049
31050 ;dec si ;AN000; backup si for next call (tm08)
31051 ;jmp short $_P_Exit_At_Extra ;AN000; else exit w/ CY off
31052 ; 12/12/2022
31053 ; cf=0
31054 ; 08/07/2023
31055 ;jmp short $_P_Skip_Delim_Exit
31056
31057 $_P_Skip_Delim_CY: ;AN000;
31058 000020DD F9 stc ;AN000; indicate EOL
31059 000020DE EB01 jmp short $_P_Skip_Delim_Exit ;AN000;
31060
31061 $_P_Skip_Delim_NCY: ;AN000;
31062 000020E0 F8 clc ;AN000; indicate non delim
31063 $_P_Skip_Delim_Exit: ;AN000; in this case, need
31064 000020E1 4E dec si ;AN000; backup index pointer
31065 ; 08/07/2023
31066 ; 12/12/2022
31067 ;$_P_Exit_At_Extra: ; cf=0
31068 000020E2 C3 retn ;AN000;
31069
31070 ; 12/12/2022
31071 ;$_P_Exit_At_Extra: ;AN000;
31072 ;clc ;AN000; indicate extra delim
31073 ;retn ;AN000;
31074
31075 ;*****
31076 ; $_P_Chk_EOL;
31077 ;
31078 ; Function: Check if AL is one of End of Line characters.
31079 ;
31080 ; Input: AL = character code
31081 ; ES:DI -> Parameter List
31082 ;
31083 ; Output: ZF = 1 if one of End of Line characters
31084 ;*****
31085
31086 $_P_Chk_EOL:
31087 000020E3 53 push bx ;AN000;
31088 000020E4 51 push cx ;AN000;
31089 000020E5 3C0D cmp al,_$_P_CR ;AN000; Carriage return ?
31090 000020E7 742C je short $_P_Chk_EOL_Exit ;AN000;
31091 000020E9 3C00 cmp al,_$_P_NULL ;AN000; zero ?
31092 000020EB 7428 je short $_P_Chk_EOL_Exit ;AN000;
31093 ;IF LFEOLSW ;AN028; IF LF TO BE ACCEPTED AS EOL
31094 000020ED 3C0A cmp al,_$_P_LF ;AN000; Line feed ?
31095 000020EF 7424 je short $_P_Chk_EOL_Exit ;AN000;
31096 ;ENDIF ;AN028;
31097 000020F1 26807D0202 cmp byte [es:di+_$_P_PARMS_Blk.Num_Extra],_$_P_I_Have_EOL
31098 ;AN000; EOL character specified ?
31099 000020F6 721D jb short $_P_Chk_EOL_Exit ;AN000;
31100 000020F8 31DB xor bx,bx ;AN000;
31101 000020FA 268A5D03 mov bl,[es:di+_$_P_PARMS_Blk.Len_Extra_Delim]
31102 ;AN000; get length of delimiter list
31103 000020FE 83C304 add bx,_$_P_Len_PARMS ;AN000; skip it

```

```

31104      ; 08/07/2023
31105      xor     cx,cx ; *
31106      cmp     byte [es:bx+di],_$_P_I_Use_Default ;AN000; No extra EOL character ?
31107      je      short _$_P_Chk_EOL_NZ ;AN000;
31108      ; 08/07/2023
31109      ;xor     cx,cx ;AN000; Get number of extra character
31110      ;xor     ch,ch ; *
31111      mov     cl,[es:bx+di] ;AN000;
31112      _$_P_Chk_EOL_Loop: ;AN000;
31113      inc     bx ;AN000;
31114      cmp     al,[es:bx+di] ;AN000; Check extra EOL character
31115      je      short _$_P_Chk_EOL_Exit ;AN000;
31116      loop    _$_P_Chk_EOL_Loop ;AN000;
31117      ; 08/07/2023
31118      ; cx=0
31119      _$_P_Chk_EOL_NZ: ;AN000;
31120      ;cmp     al,_$_P_CR ;AN000; reset ZF
31121      ; 08/07/2023
31122      inc     cx ; zf=0 (cx=1) ; *
31123      _$_P_Chk_EOL_Exit: ;AN000;
31124      pop     cx ;AN000;
31125      pop     bx ;AN000;
31126      retn
31127
31128      ;*****
31129      ; _$_P_Chk_Delim;
31130      ;
31131      ; Function: Check if AL is one of delimiter characters.
31132      ; if AL+[si] is DBCS blank, it is replaced with two SBCS
31133      ; blanks.
31134      ;
31135      ; Input: AL = character code
31136      ; DS:SI -> Next Character
31137      ; ES:DI -> Parameter List
31138      ;
31139      ; Output: ZF = 1 if one of delimiter characters
31140      ; SI points to the next character
31141      ; Vars: _$_P_Terminator(W), _$_P_Flags(W)
31142      ;*****
31143
31144      ; 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
31145      ; MSDOS 6.21 IO.SYS - SYSINIT:1FAEh
31146      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:2451h) ; (Retro DOS v5.0)
31147
31148      _$_P_Chk_Delim:
31149      push     bx ;AN000;
31150      push     cx ;AN000;
31151      mov     byte [cs:_$_P_Terminator],_$_P_Space
31152      ;AC034; Assume terminated by space
31153      ;and     byte [cs:_$_P_Flags20,0DFh
31154      and     byte [cs:_$_P_Flags2],0FFh-_$_P_Extra ;AC034;
31155      cmp     al,_$_P_Space ; 20h ;AN000; Space ?
31156      je      short _$_P_Chk_Delim_Exit ;AN000;
31157
31158      cmp     al,_$_P_TAB ;AN000; TAB ?
31159      je      short _$_P_Chk_Delim_Exit ;AN000;
31160
31161      cmp     al,_$_P_Comma ;AN000; Comma ?
31162      je      short _$_P_Chk_Delim_Exit0 ;AN000;
31163
31164      ; Note: _$_P_Chk_Delim00 part of code is nonsense here
31165      ; because _$_P_Space = _$_P_DBSPl = 20h
31166      ; Erdogan Tan - 08/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
31167      ;_$_P_Chk_Delim00:
31168      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:246Bh)
31169      ; (MSDOS 6.21 IO.SYS - SYSINIT:1FC8h)
31170      %if 0
31171      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
31172      _$_P_Chk_Delim00: ;AN000;
31173      cmp     al,_$_P_DBSPl ; 20h ;AN000; 1st byte of DBCS Space ?
31174      jne     short _$_P_Chk_Delim01 ;AN000;
31175
31176      cmp     byte [si],_$_P_DBSPl ; 20h ;AN000; 2nd byte of DBCS Space ?
31177      jne     short _$_P_Chk_Delim01 ;AN000;
31178
31179      mov     al,_$_P_Space ;AN000;
31180      inc     si ;AN000; make si point to next character
31181      cmp     al,al ;AN000; Set ZF
31182      jmp     short _$_P_Chk_Delim_Exit ;AN000;
31183      %endif
31184
31185      _$_P_Chk_Delim01: ;AN000;
31186      cmp     byte [es:di-_$_P_PARAMS_Blk.Num_Extra],_$_P_I_Have_Delim
31187      ;AN000; delimiter character specified ?
31188      jnb     short _$_P_Chk_Delim_Exit ;AN000;
31189
31190      ;xor     cx,cx ;AN000;
31191      xor     ch,ch
31192      ;mov     cl,[es:di+3]
31193      mov     cl,[es:di+_$_P_PARAMS_Blk.Len_Extra_Delim]
31194      ;AN000; get length of delimiter list
31195      ;or     cx,cx ;AN000; No extra Delim character ?
31196      ;jz     short _$_P_Chk_Delim_NZ ;AN000;
31197      ; 08/07/2023
31198      jcxz    _$_P_Chk_Delim_NZ
31199
31200      mov     bx,_$_P_Len_PARAMS-1 ; 3;AN000; set bx to 1st extra delimiter
31201      _$_P_Chk_Delim_Loop: ;AN000;
31202      inc     bx ;AN000;
31203      cmp     al,[es:bx+di] ;AN000; Check extra Delim character
31204      je      short _$_P_Chk_Delim_Exit0 ;AN000;
31205
31206      loop    _$_P_Chk_Delim_Loop ;AN000; examine all extra delimiter
31207
31208      _$_P_Chk_Delim_NZ: ;AN000;
31209      ;cmp     al,_$_P_Space ;AN000; reset ZF
31210      ; 08/07/2023
31211      ; cx=0 here
31212      inc     cx ; cx=1, zf=0
31213      _$_P_Chk_Delim_Exit: ;AN000;
31214      _$_P_ChkDfin: ;AN000;
31215      pop     cx ;AN000;
31216      pop     bx ;AN000;
31217      retn ;AN000;
31218
31219      _$_P_Chk_Delim_Exit0: ;AN000;
31220      mov     [cs:_$_P_Terminator],al ;AC034; keep terminated delimiter
31221      test    byte [cs:_$_P_Flags2],_$_P_equ ;AN027;AC034;; if terminating a key=
31222      jnz     short _$_P_No_Set_Extra ;AN027; then do not set the EXTRA bit
31223
31224      or     byte [cs:_$_P_Flags2],_$_P_Extra
31225      ;AC034; flag terminated extra delimiter or comma
31226      _$_P_No_Set_Extra: ;AN027;
31227      cmp     al,al ;AN000; set ZF

```



```

31228 00002164 EBE7      jmp     short _$P_Chk_Delim_Exit ;AN000;
31229
31230      ;*****
31231      ; _$P_Chk_Switch;
31232
31233      ; Function: Check if AL is the switch character not in first position of
31234      ; _$P_STRING_BUF
31235
31236      ; Input:      AL = character code
31237      ;             BX = current pointer within _$P_String_Buf
31238      ;             SI =>next char on command line (following the one in AL)
31239
31240      ; Output:     CF = 1 (set)if AL is switch character, and not in first
31241      ;             position, and has no chance of being part of a date string,
31242      ;             i.e. should be treated as a delimiter.
31243
31244      ;             CF = 0 (reset, cleared) if AL is not a switch char, is in the first
31245      ;             position, or is a slash but may be part of a date string, i.e.
31246      ;             should not be treated as a delimiter.
31247
31248      ; Vars:      _$P_Terminator(W)
31249
31250      ; Use:       _$P_0099
31251      ;*****
31252
31253      _$P_Chk_Switch:
31254      ;;lea bp,[cs:_$P_STRING_BUF];AN020;AC034
31255      ;;lea bp,[_$P_STRING_BUF] ;BP=OFFSET of _$P_String_Buf even in group addressing
31256      ; 08/07/2023
31257 00002166 BD[8619]    mov     bp,_$P_STRING_BUF
31258
31259      ; .IF <BX NE BP> THEN ;AN020;IF not first char THEN
31260 00002169 39EB        cmp     bx,bp ;AN000;
31261 0000216B 7406        je      short _$P_STRUC_L2 ;AN000;
31262
31263      ; .IF <AL EQ _$P_Switch> THEN ;AN020;otherwise see if a slash
31264 0000216D 3C2F        cmp     al,_$P_Switch ;AN000;
31265 0000216F 750C        jne     short _$P_STRUC_L5 ;AN000;
31266
31267 00002171 F9          stc      ;AN020;not in first position and is slash
31268      jmp     short _$P_STRUC_L1 ;AN000;
31269      ; 12/12/2022
31270 00002172 C3          retn
31271
31272      ; 12/12/2022
31273      _$P_STRUC_L5: ;AN000;
31274      CLC ;AN020;not a slash
31275      ; .ENDIF ;AN020;
31276      ; .ELSE ;AN020;is first char in the buffer, ZF=0
31277      jmp     short _$P_STRUC_L1 ;AN000;
31278
31279      _$P_STRUC_L2: ;AN000;
31280      ; .IF <AL EQ _$P_Switch> THEN ;AN020;
31281 00002173 3C2F        cmp     al,_$P_Switch ;AN000;
31282 00002175 7506        jne     short _$P_STRUC_L12 ;AN000;
31283
31284 00002177 2E800E[7D19]40 or     byte [cs:_$P_Flags2],_$P_SW ;AN020 ;AC034;;could be valid switch, first char and is slash
31285      ; .ENDIF ;AN020;
31286
31287      ; 12/12/2022
31288      ; cf=0
31289      ;retn
31290
31291      _$P_STRUC_L5:
31292      ; 12/12/2022
31293      _$P_STRUC_L12: ;AN000;
31294 0000217D F8          clc      ;AN020;CF=0 indicating first char
31295      ; .ENDIF ;AN020;
31296      _$P_STRUC_L1: ;AN000;
31297 0000217E C3          retn ;AN000;
31298
31299      ;*****
31300      ; _$P_Chk_DBCS:
31301
31302      ; Function: Check if a specified byte is in ranges of the DBCS lead bytes
31303
31304      ; Input:
31305      ; AL = Code to be examined
31306
31307      ; Output:
31308      ; If CF is on then a lead byte of DBCS
31309
31310      ; Use: INT 21h w/AH=63
31311
31312      ; Vars: _$P_DBCSEV_Seg(RW), _$P_DBCSEV_Off(RW)
31313      ;*****
31314
31315      _$P_Chk_DBCS:
31316 0000217F 1E          push     ds ;AN000;
31317 00002180 56          push     si ;AN000;
31318 00002181 53          push     bx ;AN000; (tm11)
31319      ;cmp word [cs:_$P_DBCSEV_SEG],0 ;AC034; ALREADY SET ?
31320      ;jne short _$P_DBCS00 ;AN000;
31321      ; 08/07/2023
31322 00002182 2E8B36[7A19]    mov     si,[cs:_$P_DBCSEV_SEG]
31323 00002187 21F6        and     si,si ; 0 ?
31324 00002189 7525        jnz     short _$P_DBCS00 ; already set
31325 0000218B 50          push     ax ;AN000;
31326 0000218C 1E          push     ds ;AN000; (tm11)
31327 0000218D 51          push     cx ;AN000;
31328 0000218E 52          push     dx ;AN000;
31329 0000218F 57          push     di ;AN000;
31330 00002190 55          push     bp ;AN000;
31331 00002191 06          push     es ;AN000;
31332      ; si = 0 ; 08/07/2023
31333      ;xor si,si ;AN000;
31334 00002192 8EDE        mov     ds,si ; 0 ;AN000;
31335 00002194 B80063      mov     ax,_$P_DOS_GetEV ; 6300h ;AN000; GET DBCS EV CALL
31336 00002197 CD21        int     21h ;AN000;
31337      ; DOS - 3.2+ only - GET DOUBLE BYTE CHARACTER SET LEAD TABLE
31338 00002199 8CDB        mov     bx,ds ;AN000; (tm11)
31339 0000219B 09DB        or      bx,bx ;AN000; (tm11)
31340 0000219D 07          pop      es ;AN000;
31341 0000219E 5D          pop      bp ;AN000;
31342 0000219F 5F          pop      di ;AN000;
31343 000021A0 5A          pop      dx ;AN000;
31344 000021A1 59          pop      cx ;AN000;
31345 000021A2 1F          pop      ds ;AN000; (tm11)
31346 000021A3 58          pop      ax ;AN000;
31347 000021A4 741E        jz      short _$P_NON_DBCS ;AN000;
31348
31349 000021A6 2E8936[7819]    mov     [cs:_$P_DBCSEV_OFF],si ;AC034; save EV offset
31350 000021AB 2E891E[7A19]    mov     [cs:_$P_DBCSEV_SEG],bx ;AC034; save EV segment (tm11)
31351      _$P_DBCS00: ;AN000;

```

```

31352             ;mov     si,[cs:$_P_DBCSEV_OFF];AC034; load EV offset
31353             ;mov     ds,[cs:$_P_DBCSEV_SEG];AC034; and segment
31354             ; 08/07/2023
31355 000021B0 2EC536[7819]             lds     si,[cs:$_P_DBCSEV_OFF]
31356 _P_DBCS_LOOP:             ;AN000;
31357             cmp     word [si],0             ;AN000; zero vector ?
31358             je      short _P_NON_DBCS       ;AN000; then exit
31359             cmp     al,[si]                 ;AN000;
31360             jnb     short _P_DBCS01         ;AN000; Check if AL is in
31361             cmp     al,[si+1]               ;AN000; range of
31362             ja      short _P_DBCS01         ;AN000; the vector
31363             stc                             ;AN000; if yes, indicate DBCS and exit
31364             ; 04/02/2026
31365             jmp     short _P_DBCS_EXIT     ;AN000;
31366
31367 _P_NON_DBCS:             ;AN000;
31368             ; 12/12/2022
31369             ; cf=0
31370             ;clc                             ;AN000; indicate SBCS
31371 _P_DBCS_EXIT:             ;AN000;
31372             pop     bx                     ;AN000; (tm11)
31373             pop     si                     ;AN000;
31374             pop     ds                     ;AN000;
31375             retn                          ;AN000;
31376
31377 _P_DBCS01:             ;AN000;
31378             inc     si                     ;AC035; add '2' to
31379             inc     si                     ;AC035; SI reg
31380             ;AN000; get next vector
31381             jmp     short _P_DBCS_LOOP     ;AN000; loop until zero vector found
31382
31383 ; SYSCONF.ASM - MSDOS 6.0 - 1991
31384 ; =====
31385 ; 27/03/2019 - Retro DOS v4.0
31386
31387 ;control block definitions for parser.
31388 ;-----
31389 ; buffer = [n | n,m] {/e}
31390
31391 ; 30/03/2019
31392
31393 struc p_parms
31394     resw 1             ; dw ?
31395     resb 1             ; db 1 ; an extra delimiter list
31396     resb 1             ; db 1 ; length is 1
31397     resb 1             ; db ',' ; delimiter
31398 .size:
31399 endstruc
31400
31401 struc p_pos
31402     resw 1             ; dw ? ; numeric value??
31403     resw 1             ; dw ? ; function
31404     resw 1             ; dw ? ; result value buffer
31405
31406 ; note: by defining result_val before this structure, we could remove
31407 ; the "result_val" from every structure invocation
31408
31409     resw 1             ; dw ? ; value list
31410     resb 1             ; db 0 ; no switches/keywords
31411 .size:
31412 endstruc
31413
31414 struc p_range
31415     resb 1             ; db 1 ; range definition
31416     resb 1             ; db 1 ; 1 definition of range
31417     resb 1             ; db 1 ; item tag for this range
31418     resd 1             ; dd ? ; numeric min
31419     resd 1             ; dd ? ; numeric max
31420 .size:
31421 endstruc
31422
31423 ;-----
31424
31425             ; 26/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
31426             ; (SYSINIT:1F48h)
31427
31428             ; 08/07/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
31429             ; MSDOS 6.21 IO.SYS - SYSINIT:2083h
31430             ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:251Dh) ; (Retro DOS v5.0)
31431
31432 ; buffer = [n | n,m] {/e}
31433
31434 ;buf_parms p_parms <buf_parmsx>
31435 buf_parms:
31436     dw     buf_parmsx
31437     db 1             ; an extra delimiter list
31438     db 1             ; length is 1
31439     db ','           ; delimiter
31440
31441 buf_parmsx:
31442     dw 201h,buf_pos1,buf_pos2; min 1, max 2 positionals
31443     db 1             ; one switch
31444     dw sw_x_ctr1
31445     db 0             ; no keywords
31446
31447 ;buf_pos1 p_pos <8000h,0,result_val,buf_range_1> ; numeric
31448 buf_pos1:
31449     dw 8000h         ; numeric value??
31450     dw 0             ; function
31451     dw result_val    ; result value buffer
31452     dw buf_range_1   ; value list
31453     db 0             ; no switches/keywords
31454
31455 ;buf_range_1 p_range <,,,1,99> ; M050
31456 buf_range_1:
31457     db 1             ; range definition
31458     db 1             ; 1 definition of range
31459     db 1             ; item tag for this range
31460     dd 1             ; numeric min
31461     dd 99            ; numeric max
31462
31463 ;buf_pos2 p_pos <8001h,0,result_val,buf_range_2> ; optional num.
31464 buf_pos2:
31465     dw 8001h
31466     dw 0
31467     dw result_val
31468     dw buf_range_2
31469     db 0
31470
31471 ;buf_range_2 p_range <,,,0,8>
31472 buf_range_2:
31473     db 1
31474     db 1
31475     db 1

```



```

31476 000021FB 00000000      dd      0
31477 000021FF 08000000      dd      8
31478
31479 ;sw_x_ctrl p_pos <0,0,result_val,noval,1> ; followed by one switch
31480 sw_x_ctrl:
31481      dw      0
31482      dw      0
31483      dw      result_val
31484      dw      noval
31485      db      1          ; 1 switch
31486
31487 switch_x:
31488      db      '/x',0          ; M016
31489
31490 p_buffers:
31491      dw      0          ; local variables
31492 p_h_buffers:
31493      dw      0
31494      ; 26/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
31495 p_buffer_slash_x:
31496      db      0 ; 31/03/2019
31497
31498 ;-- common definitions -----
31499
31500 noval:      db      0
31501
31502 result_val:      ;label byte
31503      db      0          ; type returned
31504 result_val_itag:
31505      db      0          ; item tag returned
31506 result_val_swoff:
31507      dw      0          ; es:offset of the switch defined
31508 rv_byte:      ;label byte
31509 rv_dword: dd      0          ; value if number,or seg:offset to string.
31510
31511 ;-----
31512      ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
31513      ; (SYSINIT:1F99h)
31514
31515 ; break = [ on | off ]
31516
31517 ;brk_parms p_parms <brk_parmsx>
31518 brk_parms:
31519      dw      brk_parmsx
31520      db      1          ; an extra delimiter list
31521      db      1          ; length is 1
31522      db      ','        ; delimiter
31523
31524 brk_parmsx:
31525      dw      101h,brk_pos ; min,max = 1 positional
31526      db      0          ; no switches
31527      db      0          ; no keywords
31528
31529 ;brk_pos p_pos <2000h,0,result_val,on_off_string> ; simple string
31530 brk_pos:
31531      dw      2000h
31532      dw      0
31533      dw      result_val
31534      dw      on_off_string
31535      db      0
31536
31537 on_off_string:      ;label byte
31538      db      3          ; signals that there is a string choice
31539      db      0          ; no range definition
31540      db      0          ; no numeric values choice
31541      db      2          ; 2 strings for choice
31542      db      1          ; the 1st string tag
31543      dw      on_string
31544      db      2          ; the 2nd string tag
31545      dw      off_string
31546
31547 on_string:
31548      db      "ON",0
31549 off_string:
31550      db      "OFF",0
31551
31552 p_ctrl_break:
31553      db      0          ; local variable
31554
31555 ;-----
31556      ; 27/10/2022
31557
31558 ; country = n {m {path}}
31559 ; or
31560 ; country = n,,path
31561
31562 ;cntry_parms p_parms <cntry_parmsx>
31563 cntry_parms:
31564      dw      cntry_parmsx
31565      db      1
31566      db      1
31567      db      ','
31568
31569 cntry_parmsx:
31570      dw      301h,cntry_pos1,cntry_pos2,cntry_pos3 ; min 1, max 3 pos.
31571
31572      db      0          ; no switches
31573      db      0          ; no keywords
31574
31575 ;cntry_pos1 p_pos <8000h,0,result_val,cc_range> ; numeric value
31576 cntry_pos1:
31577      dw      8000h
31578      dw      0
31579      dw      result_val
31580      dw      cc_range
31581      db      0
31582
31583 ;cc_range p_range <,,1,999>
31584 cc_range:
31585      db      1
31586      db      1
31587      db      1
31588      db      1
31589      dd      1
31590      dd      999
31591
31592 ;cntry_pos2 p_pos <8001h,0,result_val,cc_range> ; optional num.
31593 cntry_pos2:
31594      dw      8001h
31595      dw      0
31596      dw      result_val
31597      dw      cc_range
31598      db      0

```

```

31599
31600 ;cntry_pos3 p_pos <201h,0,result_val,noval> ; optional filespec
31601 cntry_pos3:
31602 dw 201h
31603 dw 0
31604 dw result_val
31605 dw noval
31606 db 0
31607
31608 p_cntry_code:
31609 dw 0 ; local variable
31610 p_code_page:
31611 dw 0 ; local variable
31612
31613 ;-----
31614 ; 27/10/2022
31615
31616 ; files = n
31617
31618 ;files_parms p_parms <files_parmsx>
31619 files_parms:
31620 dw files_parmsx
31621 db 1
31622 db 1
31623 db 1
31624 db ','
31625
31626 files_parmsx:
31627 dw 101h,files_pos ; min,max 1 positional
31628 dw 0 ; no switches
31629 dw 0 ; no keywords
31630
31631 ;files_pos p_pos <8000h,0,result_val,files_range,0> ; numeric value
31632 files_pos:
31633 dw 8000h
31634 dw 0
31635 dw result_val
31636 dw files_range
31637 db 0
31638
31639 ;files_range p_range <,,,8,255>
31640 files_range:
31641 db 1
31642 db 1
31643 db 1
31644 dd 8
31645 dd 255
31646
31647 p_files:
31648 db 0 ; local variable
31649
31650 ;-----
31651 ; 27/10/2022
31652
31653 ; fcbs = n,m
31654
31655 ;fcbs_parms p_parms <fcbs_parmsx>
31656 fcbs_parms:
31657 dw fcbs_parmsx
31658 db 1
31659 db 1
31660 db 1
31661 db ','
31662
31663 fcbs_parmsx:
31664 dw 201h,fcbs_pos_1,fcbs_pos_2 ; min,max = 2 positional
31665 dw 0 ; no switches
31666 dw 0 ; no keywords
31667
31668 ;fcbs_pos_1 p_pos <8000h,0,result_val,fcbs_range> ; numeric value
31669 fcbs_pos_1:
31670 dw 8000h
31671 dw 0
31672 dw result_val
31673 dw fcbs_range
31674 db 0
31675
31676 ;fcbs_range p_range <,,,1,255>
31677 fcbs_range:
31678 db 1
31679 db 1
31680 db 1
31681 dd 1
31682 dd 255
31683
31684 ;fcbs_pos_2 p_pos <8000h,0,result_val,fcbs_keep_range> ; numeric value
31685 fcbs_pos_2:
31686 dw 8000h
31687 dw 0
31688 dw result_val
31689 dw fcbs_keep_range
31690 db 0
31691
31692 ;fcbs_keep_range p_range <,,,0,255>
31693 fcbs_keep_range:
31694 db 1
31695 db 1
31696 db 1
31697 dd 0
31698 dd 255
31699
31700 p_fcbs: db 0 ; local variable
31701 p_keep: db 0 ; local variable
31702
31703 ;-----
31704 ; 27/10/2022
31705
31706 ; lastdrive = x
31707
31708 ;ldrv_parms p_parms <ldrv_parmsx>
31709 ldrv_parms:
31710 dw ldrv_parmsx
31711 db 1
31712 db 1
31713 db 1
31714 db ','
31715
31716 ldrv_parmsx:
31717 dw 101h,ldrv_pos ; min,max = 1 positional
31718 dw 0 ; no switches
31719 dw 0 ; no keywords
31720
31721 ;ldrv_pos p_pos <110h,10h,result_val,noval> ; drive only, ignore colon
31722 ldrv_pos: ; remove colon at end

```

```

31723 000022DE 1001      dw      110h
31724 000022E0 1000      dw      10h
31725 000022E2 [1522]    dw      result_val
31726 000022E4 [1422]    dw      noval
31727 000022E6 00        db      0
31728
31729 000022E7 00        p_ldrv:   db      0          ; local variable
31730
31731 ;-----
31732
31733 ; 27/10/2022
31734
31735 ; stacks = n,m
31736
31737 ;stks_parms p_parms <stks_parmsx>
31738 stks_parms:
31739 000022E8 [ED22]      dw      stks_parmsx
31740 000022EA 01          db      1
31741 000022EB 01          db      1
31742 000022EC 3B          db      ','
31743
31744 stks_parmsx:
31745 000022ED 0202[F522][0923] dw      202h,stks_pos_1,stks_pos_2 ; min,max = 2 positionals
31746 000022F3 00          db      0          ; no switches
31747 000022F4 00          db      0          ; no keywords
31748
31749 ;stks_pos_1 p_pos <8000h,0,result_val,stks_range> ; numeric value
31750 stks_pos_1:
31751 000022F5 0080        dw      8000h
31752 000022F7 0000        dw      0
31753 000022F9 [1522]      dw      result_val
31754 000022FB [FE22]      dw      stks_range
31755 000022FD 00          db      0
31756
31757 ;stks_range p_range <,,,0,64>
31758 stks_range:
31759 000022FE 01          db      1
31760 000022FF 01          db      1
31761 00002300 01          db      1
31762 00002301 00000000    dd      0
31763 00002305 40000000    dd      64
31764
31765 ;stks_pos_2 p_pos <8000h,0,result_val,stk_size_range> ; numeric value
31766 stks_pos_2:
31767 00002309 0080        dw      8000h
31768 0000230B 0000        dw      0
31769 0000230D [1522]      dw      result_val
31770 0000230F [1223]      dw      stk_size_range
31771 00002311 00          db      0
31772
31773 ;stk_size_range p_range <,,,0,512>
31774 stk_size_range:
31775 00002312 01          db      1
31776 00002313 01          db      1
31777 00002314 01          db      1
31778 00002315 00000000    dd      0
31779 00002319 00020000    dd      512
31780
31781 p_stack_count:
31782 0000231D 0000        dw      0          ; local variable
31783
31784 p_stack_size:
31785 0000231F 0000        dw      0          ; local variable
31786
31787 ;-----
31788
31789 ; 27/10/2022
31790
31791 ; multitrack = [ on | off ]
31792
31793 ;mtrk_parms p_parms <mtrk_parmsx>
31794 mtrk_parms:
31795 00002321 [2623]      dw      mtrk_parmsx
31796 00002323 01          db      1
31797 00002324 01          db      1
31798 00002325 3B          db      ','
31799
31800 mtrk_parmsx:
31801 00002326 0101[2C23] dw      101h,mtrk_pos ; min,max = 1 positional
31802 0000232A 00          db      0          ; no switches
31803 0000232B 00          db      0          ; no keywords
31804
31805 ;mtrk_pos p_pos <2000h,0,result_val,on_off_string> ; simple string
31806 mtrk_pos:
31807 0000232C 0020        dw      2000h
31808 0000232E 0000        dw      0
31809 00002330 [1522]      dw      result_val
31810 00002332 [3122]      dw      on_off_string
31811 00002334 00          db      0
31812
31813 p_mtrk:   db      0          ; local variable
31814
31815 ;-----
31816
31817 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
31818 ; (SYSINIT:20B2h)
31819
31820 ; switches=/k
31821
31822 ;swit_parms p_parms <swit_parmsx>
31823 swit_parms:
31824 00002336 [3B23]      dw      swit_parmsx
31825 00002338 01          db      1
31826 00002339 01          db      1
31827 0000233A 3B          db      ','
31828
31829 swit_parmsx:
31830 0000233B 0000        dw      0          ; no positionals
31831 ; 08/07/2023 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS, SYSINIT)
31832 ;db      5          ; # of switches
31833 0000233D 06          db      6          ; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM, SYSINIT)
31834 ; 27/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS, SYSINIT)
31835 ;db      3          ;
31836 0000233E [4B23]      dw      swit_k_ctrl ; /k control
31837 ; 01/01/2023 - Retro DOS v4.2 ; *
31838 00002340 [5723]      dw      swit_n_ctrl ; * ; /n control (for MULTI_CONFIG only)
31839 00002342 [6323]      dw      swit_f_ctrl ; * ; /f control (for MULTI_CONFIG only)
31840 00002344 [6F23]      dw      swit_t_ctrl ; ; /t control
31841 00002346 [7B23]      dw      swit_w_ctrl ; ; /w control
31842 ; 14/04/2024 - Retro DOS v5.0 ; **
31843 00002348 [8723]      dw      swit_i_ctrl ; ; /i control
31844 0000234A 00          db      0          ; no keywords
31845
31846 ;swit_k_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows

```

```

31847
31848 0000234B 00000000[1522]-
31848 00002351 [1422]
31849 00002353 01
31850 00002354 2F4B00
31851
31852
31853
31854
31855
31856
31857
31858
31859 00002357 00000000[1522]-
31859 0000235D [1422]
31860 0000235F 01
31861 00002360 2F4E00
31862
31863
31864
31865 00002363 00000000[1522]-
31865 00002369 [1422]
31866 0000236B 01
31867 0000236C 2F4600
31868
31869
31870
31871
31872
31873 0000236F 00000000[1522]-
31873 00002375 [1422]
31874 00002377 01
31875 00002378 2F5400
31876
31877
31878 0000237B 00000000[1522]-
31878 00002381 [1422]
31879 00002383 01
31880 00002384 2F5700
31881
31882
31883
31884
31885
31886 00002387 0000
31887 00002389 0000
31888 0000238B [1522]
31889 0000238D [1422]
31890 0000238F 01
31891 00002390 2F4900
31892
31893
31894
31895
31896
31897
31898 00002393 00
31899 00002394 00
31900 00002395 00
31901
31902 00002396 00
31903
31904
31905
31906
31907
31908
31909
31910
31911
31912
31913 00002397 [9C23]
31914 00002399 01
31915 0000239A 01
31916 0000239B 3B
31917
31918 0000239C 01
31919 0000239D 02
31920 0000239E [A423]
31921 000023A0 [A423]
31922 000023A2 00
31923 000023A3 00
31924
31925
31926
31927
31928 000023A4 00200000[1522]-
31928 000023AA [B623]
31929 000023AC 00
31930 000023AD 00200000[1522]-
31930 000023B3 [B623]
31931 000023B5 00
31932
31933
31934 000023B6 03
31935 000023B7 00
31936 000023B8 00
31937 000023B9 04
31938 000023BA 01
31939 000023BB [E423]
31940 000023BD 02
31941 000023BE [E923]
31942 000023C0 03
31943 000023C1 [ED23]
31944 000023C3 04
31945 000023C4 [F123]
31946
31947
31948
31949
31950
31951 000023C6 [CB23]
31952 000023C8 01
31953 000023C9 01
31954 000023CA 3B
31955
31956 000023CB 01
31957 000023CC 01
31958 000023CD [D123]
31959 000023CF 00
31960 000023D0 00
31961
31962
31963

swit_k_ctrl:
    dw    0,0,result_val,noval

    db    1
swit_k:    db    '/K',0

; 01/01/2023 - Retro DOS v4.2 (MSDOS 6.21 IO.SYS)
; (SYSINIT:220Ch) ; *

; 27/10/2022 - Retro DOS v4.0 (MSDOS 5.0 IO.SYS, SYSINIT)
;
;
;swit_n_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows
swit_n_ctrl: ; *
    dw    0,0,result_val,noval

    db    1
swit_n: db    '/N',0

;swit_f_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows
swit_f_ctrl: ; *
    dw    0,0,result_val,noval

    db    1
swit_f: db    '/F',0

; 27/10/2022

;swit_t_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows M059
swit_t_ctrl:
    dw    0,0,result_val,noval

    db    1
swit_t:    db    '/T',0
;swit_w_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows M059
swit_w_ctrl: M063
    dw    0,0,result_val,noval

    db    1
swit_w:    db    '/W',0
;

; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM, SYSINIT)
;;;
;swit_i_ctrl p_pos <0,0,result_val,noval,1> ; switch string follows
swit_i_ctrl:
    dw    0
    dw    0
    dw    result_val
    dw    noval
    db    1
swit_i:    db    '/I',0
;;;

; There doesn't need to be p_swit_n or p_swit_f because /N and /F are
; acted upon during MULTI_CONFIG processing; we only needed entries
; in the above table to prevent the parsing code from complaining about them

p_swit_k: db    0 ; local variable
p_swit_t: db    0 ; local variable M059
p_swit_w: db    0 ; local variable M063
; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
p_swit_i: db    0

;-----

; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; (SYSINIT:20E8h)

; DOS = [ high | low ]

;dos_parms p_parms <dos_parmsx>
dos_parms:
    dw    dos_parmsx
    db    1
    db    1
    db    ','
dos_parmsx:
    db    1 ; min parameters
    db    2 ; max parameters
    dw    dos_pos
    dw    dos_pos
    db    0 ; no switches
    db    0 ; no keywords

;dos_pos p_pos <2000h,0,result_val,dos_strings> ; simple string
; p_pos <2000h,0,result_val,dos_strings> ; simple string
dos_pos:
    dw    2000h,0,result_val,dos_strings

    db    0
    dw    2000h,0,result_val,dos_strings

    db    0

dos_strings: ;label byte
    db    3 ; signals that there is a string choice
    db    0 ; no range definition
    db    0 ; no numeric values choice
    db    4 ; 4 strings for choice
    db    1 ; the 1st string tag
    dw    hi_string
    db    2 ; the 2nd string tag
    dw    lo_string
    db    3
    dw    umb_string
    db    4
    dw    noumb_string

; 14/04/2024 - Retro DOS v5.0
; (PCDOS 7.1 IBMDOS.COM - SYSINIT:273Eh)
;;;
dosdata_parms:
    dw    dosdata_parmsx ; DOSDATA = UMB|NOUMB
    db    1
    db    1
    db    ','
dosdata_parmsx:
    db    1
    db    1 ; min,max = 1 positional
    dw    dosdata_pos
    db    0 ; no switches
    db    0 ; no keywords

; dosdata_pos p_pos <2000h,0,result_val,dosdata_strings>
dosdata_pos:

```

```

31964 000023D1 0020      dw      2000h      ; simple string
31965 000023D3 0000      dw      0
31966 000023D5 [1522]    dw      result_val
31967 000023D7 [DA23]    dw      dosdata_strings
31968 000023D9 00      db      0
31969
31970 000023DA 03      db      3          ; signals that there is a string choice
31971 000023DB 00      db      0          ; no range definition
31972 000023DC 00      db      0          ; no numeric values choice
31973 000023DD 02      db      2          ; 2 strings for choice
31974 000023DE 01      db      1          ; the 1st string tag
31975 000023DF [ED23]    dw      umb_string      ; "UMB"
31976 000023E1 02      db      2          ; the 2nd string tag
31977 000023E2 [F123]    dw      noumb_string     ; "NOUMB"
31978
31979
31980 000023E4 4849474800    hi_string: db      "HIGH",0
31981 000023E9 4C4F5700    lo_string: db      "LOW",0
31982 000023ED 554D4200    umb_string: db      "UMB",0
31983 000023F1 4E4F554D4200    noumb_string: db      "NOUMB",0
31984
31985
31986 000023F7 00      p_dos_hi: db      0          ; local variable
31987                                     ; BUGBUG : I dont know whether PARSE uses
31988                                     ; this variable or not
31989                                     ; 14/04/2024 (PCDOS 7.1 IBMBIO.COM)
31990 000023F8 00      db      0
31991
31992 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
31993 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
31994 ;%if 0
31995
31996 ;***** RICHID ****
31997
31998 ;include highvar.inc ; devicehigh variables (used by loadhigh also)
31999
32000 ; 30/03/2019 - Retro DOS v4.0
32001 ;-----
32002
32003 ; Module: HIGHVAR.INC - Data common to LOADHIGH and DEVICEHIGH, res seg
32004 ;
32005 ; Date: May 14, 1992
32006 ;
32007 ;*****
32008 ;
32009 ; Modification log:
32010 ;
32011 ; DATE WHO DESCRIPTION
32012 ; -----
32013 ; 05/14/92 t-richj Original
32014 ; 06/21/92 t-richj Final revisions before check-in
32015 ;
32016 ;*****
32017 ;
32018 ; There are two primary definitions which need to be made, selectively, before
32019 ; this include file should be used. These are:
32020 ; HV_Extern - If this has been defined, variables for this module will be
32021 ; declared as external. Otherwise, variables will be declared
32022 ; public, as well as defined, here. LoadHigh declares HV_Extern
32023 ; in stub.asm and loadhi.asm, and does not declare it in
32024 ; rdata.asm... DeviceHigh does not declare HV_Extern anywhere
32025 ; (as only one module, sysconf.asm, includes this file).
32026 ; HV_LoadHigh - This should be defined when this module is going into
32027 ; command.com, for LoadHigh. All of loadhi.asm, stub.asm and
32028 ; rdata.asm define this, while io.sys' sysconf.asm does not.
32029 ;
32030 ;*****
32031 ;
32032 ; To keep track of which UMBs were specified on the DH/LH command lines, and
32033 ; to keep track of the minimum sizes given for each, there're two arrays kept
32034 ; in { IO.SYS: sysinitseg / COMMAND.COM: DATARES }... each is MAXUMB elements
32035 ; big. 16 should be around 14 too many for most users, so there's no expected
32036 ; space problem (it's just such a nice round number, eh?).
32037
32038 MAXUMB equ 16
32039
32040 ; Memory elements owned by the system are marked as PSP address 8 in both the
32041 ; USA and Japan; Japanese systems also use 9 under more bizzarre conditions.
32042
32043 FreePSPowner equ 0 ; Free MCBs all have an owner PSP address of 0
32044 SystemPSPowner equ 8
32045 JapanPSPowner equ 9
32046
32047 ; for LoadHigh and DeviceHigh:
32048 ;
32049 ; fInHigh - Is set to 1 during HideUMBs(), and back to zero in
32050 ; UnHideUMBs().
32051 ; fUmbTiny - Is set to 1 iff the user has specified /S on the command-
32052 ; line.
32053 ; SegLoad - Segment address for first UMB specified; set automatically.
32054 ; UmbLoad - The load UMB number; for example, this is 3 if the user has
32055 ; given a command-line like "/L:3,500;4"
32056 ; UmbUsed - An array of characters, each of which is 1 iff the UMB
32057 ; matching its index number was specified on the command-line;
32058 ; for example, after "/L:3,500;4;7", UmbUsed[3], [4] and [7]
32059 ; will be set to 1. All others will be set to 0.
32060 ; UmbSize - An array of words, each of which is interpreted as a size
32061 ; specified by the user for a UMB (in the above example, all
32062 ; elements would be zero save UmbSize[3], which would be 500.
32063 ; fm_umb - Set to the old UMB link-state (0x80 or 0x00)
32064 ; fm_strat - Set to the old memory-allocation strategy (0$00000???)
32065 ; fm_argc - Number of arguments received by ParseVar() (see ParseVar()
32066 ; for details).
32067
32068 000023F9 00      fInHigh: db      0
32069 000023FA 00      fUmbTiny: db      0
32070 000023FB 0000    SegLoad: dw      0
32071 000023FD 00      UmbLoad: db      0
32072 000023FE 00<rep 10h> UmbUsed: times MAXUMB db 0 ; times 16 db 0 ; db 16 dup(?)
32073 0000240E 0000<rep 10h> UmbSize: times MAXUMB dw 0 ; times 16 dw 0 ; dw 16 dup(?)
32074 0000242E 00      fm_umb: db      0
32075 0000242F 00      fm_strat: db      0
32076 00002430 00      fm_argc: db      0
32077
32078 ; UmbLoad is set to UNSPECIFIED, below, until /L:umb is read; at which point
32079 ; UmbLoad is set to the UMB number given.
32080
32081 UNSPECIFIED equ -1
32082
32083 ;%endif ; 27/10/2022
32084
32085 ;***** RICHID ****
32086
32087 ; 30/03/2019 - Retro DOS v4.0 (MSDOS 6.0, SYSCONF.ASM)

```

```

32088 ; ((MSDOS 6.21 IO.SYS -> SYNINIT:22BAh))
32089
32090 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32091 ; (SYSINIT:212Bh)
32092
32093 ;public DevEntry
32094
32095 00002431 0000 DevSize: dw 0 ; size of the device driver being loaded (paras)
32096 00002433 0000 DevLoadAddr: dw 0 ; Mem addr where the device driver is 2 b loaded
32097 00002435 0000 DevLoadEnd: dw 0 ; MaxAddr to which device can be loaded
32098 00002437 00000000 DevEntry: dd 0 ; Entry point to the device driver
32099 0000243B 00000000 DevBrkAddr: dd 0 ; Break address of the device driver
32100 ; 30/12/2022
32101 ; 27/10/2022
32102 0000243F 00 ConvLoad: db 0 ; Use conventional (dos 5 -style) InitDevLoad?
32103 ;
32104 00002440 00 DevUMB: db 0 ; byte indicating whether to load DDs in UMBS
32105 00002441 0000 DevUMBAddr: dw 0 ; cuurent UMB used fro loading devices (paras)
32106 00002443 0000 DevUMBSize: dw 0 ; Size of the current UMB being used (paras)
32107 00002445 0000 DevUMBFree: dw 0 ; Start of free are in the current UMB (paras)
32108 ;
32109 00002447 00000000 DevXMSAddr: dd 0
32110 ;
32111 0000244B 0000 DevExecAddr: dw 0 ; Device load address parameter to Exec call
32112 0000244D 0000 DevExecReloc: dw 0 ; Device load relocation factor
32113 ;
32114 0000244F 00 DeviceHi: db 0 ; Flag indicating whther the current device
32115 ; is being loaded into UMB
32116 00002450 0000 DevSizeOption: dw 0 ; SIZE= option
32117 ;
32118 00002452 00 Int12Lied: db 0 ; did we trap int 12 ?
32119 00002453 0000 OldInt12Mem: dw 0 ; value in 40:13h (int 12 ram)
32120 00002455 50524F544D414E24 ThreeComName: db 'PROTMAN$' ; 3Com Device name
32121 ;
32122 0000245D 00 FirstUMBLinked: db 0
32123 0000245E 0000 DevDOSData: dw 0 ; segment of DOS Data
32124 00002460 00000000 DevCmdLine: dd 0 ; Current Command line
32125 00002464 00 DevSavedDelim: db 0 ; The delimiter which was replaced with null
32126 ; to use the file name in the command line
32127 ; 13/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
32128 ; ifdef dblspace_hooks
32129 00002465 00 MagicHomeFlag: db 0 ; set non-zero when MagicDrv is final placed
32130 ; endif
32131 ;
32132 ; =====
32133 ;
32134 ; 31/03/2019 - Retro DOS v4.0
32135 ;
32136 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32137 ; (SYSINIT:215Eh)
32138 ;
32139 ; -----
32140 ;
32141 ; procedure : doconf
32142 ;
32143 ; Config file is parsed initially with this routine. For the
32144 ; Subsequent passes 'multi_pass' entry is used .
32145 ;
32146 ; -----
32147 ;
32148 ; 27/10/2022
32149 doconf:
32150 00002466 0E push cs
32151 00002467 1F pop ds
32152
32153 00002468 B80037 mov ax,3700h
32154 ;mov ax,(CHAR_OPER<<8) ; get switch character
32155 0000246B CD21 int 21h
32156 0000246D 8816[BA4B] mov [command_line+1],dl ; set in default command line
32157
32158 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32159 ; 27/10/2022
32160 ;;ifdef MULTI_CONFIG
32161 ; ;mov [command_line-1],dl ; save default switchchar
32162 00002471 8816[B84B] mov [def_swchr],dl ; 31/03/2019
32163 ;;endif MULTI_CONFIG
32164
32165 00002475 BA[CF4A] mov dx,config ;'\CONFIG.SYS' ;now pointing to file description
32166 00002478 B8003D mov ax,3D00h
32167 ;mov ax,OPEN<<8 ;open file "config.sys"
32168 0000247B F9 stc ;in case of int 24
32169 0000247C CD21 int 21h ;function request
32170 0000247E 7309 jnc short noprob ;brif opened okay
32171
32172 ; 31/12/2022
32173 ; 27/10/2022
32174 ;;ifdef MULTI_CONFIG
32175 00002480 E8A019 call kbd_read ; we still want to give the guy
32176 ; ; a chance to select clean boot!
32177 ;;endif ; (ie, no autoexec.bat processing)
32178 00002483 C606[CD02]0B mov byte [multi_pass_id],11 ; set it to unreasonable number
32179 00002488 C3 retn
32180 noprob:
32181 00002489 89C3 mov bx,ax ; File handle
32182 0000248B 31C9 xor cx,cx ; 0
32183 0000248D 31D2 xor dx,dx ; 0
32184 ;mov ax,4202h
32185 0000248F B80242 mov ax,(LSEEK<<8)|2
32186 00002492 CD21 int 21h
32187 00002494 A3[5603] mov [count],ax ; dx:ax = file size ; 08/09/2023
32188 ; 08/09/2023 - Erdogan Tan - Note:
32189 00002497 31D2 xor dx,dx ; dx already must be 0 here ; 08/09/2023
32190 ; I am not removing 'xor dx,dx' here
32191 ; for MSDOS compatibility.
32192 ; ((Also PCDOS 7.1 has 'xor dx,dx' here))
32193 ; (Error will be same if CONGIG.SYS file
32194 ; size > 64KB)
32195 ;mov ax,4200h
32196 00002499 B80042 mov ax,LSEEK<<8 ;reset pointer to beginning of file
32197 0000249C CD21 int 21h
32198
32199 ; 31/12/2022 - Retro DOS v4.2
32200 0000249E 8B16[A502] mov dx,[ALLOCLIM] ;use current alloclim value
32201 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32202 ;mov dx,[top_of_cdss]
32203
32204 000024A2 A1[5603] mov ax,[count]
32205 000024A5 A3[D002] mov [config_size],ax ;save the size of config.sys file.
32206 000024A8 E86AEE call ParaRound
32207 000024AB 29C2 sub dx,ax
32208
32209 ; 31/12/2022
32210 ; 27/10/2022
32211 ;ifdef MULTI_CONFIG

```



```

32212 ;
32213 ; The size of the CONFIG.SYS workspace (for recreating the in-memory
32214 ; CONFIG.SYS image, and later for building the initial environment) need
32215 ; not be any larger than CONFIG.SYS itself, EXCEPT for the fact that
32216 ; we (may) add a variable to the environment that does not explicitly appear
32217 ; in CONFIG.SYS, and that variable is CONFIG (as in CONFIG=COMMON).
32218 ; The default setting for CONFIG cannot result in more than 1 paragraph
32219 ; of extra space, so here we account for it (the worst case of course is
32220 ; when CONFIG.SYS is some very small size, like 0 -JTP)
32221 ;
32222 000024AD 4A      dec     dx          ;reserve 1 additional paragraph
32223 000024AE 8916[6219] mov     [config_wrkseg],dx ;this is the segment to be used for
32224 000024B2 29C2    sub     dx,ax          ;rebuilding the config.sys memory image
32225 ;;endif ;MULTI_CONFIG
32226
32227 000024B4 83EA11    sub     dx,11h          ;room for header
32228
32229 ; 31/12/2022
32230 000024B7 8916[A502] mov     [ALLOCLIM],dx      ;config starts here. new alloclim value.
32231 000024BB 8916[A302] mov     [CONFBOT],dx
32232
32233 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32234 ;mov     [top_of_cdss],dx
32235 ;call    TempCDS
32236 ; 31/12/2022
32237 ; 11/12/2022
32238 ; ds <= cs
32239 ;mov     dx,[cs:top_of_cdss]
32240
32241 ; 08/09/2023
32242 ; ds = cs
32243 000024BF 8B0E[5603] mov     cx,[count]
32244
32245 000024C3 8EDA    mov     ds,dx
32246 000024C5 8EC2    mov     es,dx
32247
32248 000024C7 31D2    xor     dx,dx
32249 ; 08/09/2023
32250 ;mov     cx,[cs:count]
32251 000024C9 B43F    mov     ah,3Fh
32252 ;mov     ah,READ ; 3Fh
32253 000024CB F9      stc
32254 000024CC CD21    int     21h          ;in case of int 24
32255 000024CE 9C      pushf          ;function request
32256
32257 ; find the eof mark in the file. if present,then trim length.
32258
32259 000024CF 50      push     ax
32260 000024D0 57      push     di
32261 000024D1 51      push     cx
32262 000024D2 B01A    mov     al,1Ah          ; eof mark
32263 000024D4 89D7    mov     di,dx          ; point to buffer
32264 000024D6 E305    jcxz    puteol          ; no chars
32265 000024D8 F2AE    repnz   scasb          ; find end
32266 000024DA 7501    jnz     short puteol    ; none found and count exhausted
32267
32268 ; we found a 1a. back up
32269
32270 000024DC 4F      dec     di          ; backup past 1Ah
32271
32272 ; just for the halibut, stick in an extra eol
32273
32274 puteol:
32275 000024DD B00D    mov     al,cr ; 0Dh
32276 000024DF AA      stosb
32277 000024E0 B00A    mov     al,lf ;0Ah
32278 000024E2 AA      stosb
32279 000024E3 29D7    sub     di,dx          ; difference moved
32280 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32281 ;mov     [cs:count],di ; new count
32282
32283 ; 11/12/2022
32284 ; 31/03/2019 - Retro DOS v4.0
32285 000024E5 0E      push    cs
32286 000024E6 1F      pop     ds
32287
32288 000024E7 893E[5603] mov     [count],di      ; new count
32289
32290 000024EB 59      pop     cx
32291 000024EC 5F      pop     di
32292 000024ED 58      pop     ax
32293
32294 ; 11/12/2022
32295 ; 27/10/2022
32296 ;push    cs
32297 ;pop     ds
32298
32299 000024EE 50      push    ax
32300 ;mov     ah,CLOSE
32301 000024EF B43E    mov     ah,3Eh
32302 000024F1 CD21    int     21h
32303 000024F3 58      pop     ax
32304 000024F4 9D      popf
32305 000024F5 7204    jc      short conferr   ;if not we've got a problem
32306 000024F7 39C1    cmp     cx,ax
32307 000024F9 742C    jz      short getcom     ;couldn't read the file
32308 conferr:
32309 000024FB BA[CF4A] mov     dx,config        ;want to print config error
32310 000024FE E82525 call    badfil
32311 ; 14/04/2024
32312 endconv: ; 01/01/2023
32313 retn
32314
32315 ;-----
32316 ;
32317 ; entry : multi_pass
32318 ;
32319 ;         called to execute device=,install= commands
32320 ;
32321 ;-----
32322
32323 ; 27/10/2022
32324 multi_pass:
32325 00002502 0E      push    cs
32326 00002503 1F      pop     ds
32327
32328 00002504 803E[CD02]0A cmp     byte [multi_pass_id],10
32329 ;jae_endconv:
32330 00002509 73F6    jae     short endconv    ; do nothing. just return.
32331
32332 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32333 0000250B FF36[A302] push    word [CONFBOT]
32334 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32335 ;push    word [top_of_cdss]

```

```

32336 0000250F 07          pop     es                ; es -> confbot
32337
32338 00002510 8B36[5803]   mov     si,[org_count]
32339 00002514 8936[5603]   mov     [count],si          ; set count
32340 00002518 31F6        xor     si,si ; 0
32341 0000251A 8936[5A03]   mov     [chrptr],si        ; reset chrptr
32342 0000251E 8936[AF02]   mov     [linecount],si     ; reset linecount
32343
32344 00002522 E88822     call    getchr
32345 00002525 EB06        jmp     short conflp
32346
32347          ; 14/04/2024
32348          ; 01/01/2023
32349 ;endconv:
32350          ;retn
32351
32352 getcom:
32353          ; 03/01/2023
32354          ; ds = cs
32355 00002527 E8C016     call    organize          ; organize the file
32356 0000252A E88022     call    getchr
32357 conflp:
32358 0000252D 72D2        jc      short endconv
32359
32360 0000252F FF06[AF02]   inc     word [linecount] ; increase linecount
32361
32362          ; 08/09/2023
32363 00002533 30E4        xor     ah,ah ; 0
32364          ;mov    byte [multideviceflag],0      ; reset multideviceflag.
32365          ;mov    byte [setdevmarkflag],0      ; reset setdevmarkflag.
32366 00002535 8826[6619]   mov     [multideviceflag],ah ; 0
32367 00002539 8826[6919]   mov     [setdevmarkflag],ah ; 0
32368
32369 0000253D 3C0A        cmp     al,1f             ; linefeed?
32370 0000253F 7448        je      short blank_line  ; then ignore this line.
32371
32372          ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32373          ; (SYSINIT:23CCh)
32374          ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32375          ;%if 0
32376
32377          ;ifdef    MULTI_CONFIG
32378
32379          ; If this is a genuine CONFIG.SYS command, then there should be a line
32380          ; number immediately following it....
32381
32382 00002541 A2[6419]   mov     [config_cmd],al    ; save original command code
32383          ;and    al,NOT CONFIG_OPTION_QUERY
32384 00002544 247F        and     al,~CONFIG_OPTION_QUERY ; and al,7Fh
32385
32386          ; 08/09/2023
32387 00002546 3826[6519]   cmp     [config_multi],ah ; 0
32388          ;cmp    byte [config_multi],0 ; is this a multi-config config.sys?
32389 0000254A 7427        je      short not_final  ; no, line number is not embedded
32390
32391 0000254C 50          push    ax
32392 0000254D E85D22     call    getchr            ; ignore end-of-image errors,
32393 00002550 88C4        mov     ah,al            ; because if there's an error
32394 00002552 E85822     call    getchr            ; fetching the line number that's
32395 00002555 86C4        xchg    al,ah            ; supposed to be there, the next
32396 00002557 A3[AF02]   mov     [linecount],ax    ; getchr call will get the same error
32397 0000255A 58          pop     ax
32398
32399          ;
32400          ; HACK: when 4DOS.COM is the shell and it doesn't have an environment from
32401          ; which to obtain its original program name, it grovels through all of
32402          ; memory to find the filename that was used to exec it; it wants to find
32403          ; the SHELL= line in the in-memory copy of CONFIG.SYS, and it knows that
32404          ; sysinit converts the SHELL= keyword to an 'S', so it expects to find an 'S'
32405          ; immediately before the filename, but since we are now storing line # info
32406          ; in the config.sys memory image, 4DOS fails to find the 'S' in the right
32407          ; spot.
32408
32409          ; So, on the final pass of CONFIG.SYS, copy the command code (eg, 'S')
32410          ; over the line number info, since we no longer need that info anyway. This
32411          ; relies on the fact that getchr leaves ES:SI pointing to the last byte
32412          ; retrieved.
32413          ;
32414 0000255B 803E[CD02]02 cmp     byte [multi_pass_id],2; final pass?
32415 00002560 7211        jb     short not_final  ; no
32416          ;test    word [install_flag],have_install_cmd
32417 00002562 F606[CE02]01 test    byte [install_flag],have_install_cmd ; 1
32418 00002567 7407        jz     short final      ; no install cmds, so yes it is
32419 00002569 803E[CD02]03 cmp     byte [multi_pass_id],3; final pass?
32420 0000256E 7203        jb     short not_final  ; no
32421 final:
32422 00002570 268804     mov     [es:si],al        ; save backward-compatible command code
32423 not_final:
32424          ;
32425          ;
32426          ; 31/12/2022
32427          ;%endif ; 27/10/2022
32428 00002573 88C4        mov     ah,al
32429 00002575 E83522     call    getchr
32430 00002578 7314        jnc     short tryi
32431
32432 0000257A 803E[CD02]02 cmp     byte [multi_pass_id],2
32433          ;jae    short jae_endconv          ; do not show badop again for multi_pass.
32434          ; 27/10/2022
32435 0000257F 7380        jnb     short endconv
32436 00002581 E90009     jmp     badop
32437
32438 coff:
32439          ; 11/12/2022
32440          ; ds = cs
32441          ;push    cs
32442          ;pop     ds
32443 00002584 E81D22     call    newline
32444 00002587 EBA4        jmp     short conflp ; 13/05/2019
32445
32446 blank_line:
32447 00002589 E82122     call    getchr
32448 0000258C EB9F        jmp     short conflp
32449
32450          ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32451          ; 11/12/2022
32452          ; (there is not a jump or call to here from anywhere!)
32453          ;coff_p:
32454          ;push    cs
32455          ;pop     ds
32456
32457          ;to handle install= commands,we are going to use multi-pass.
32458          ;the first pass handles the other commands and only set install_flag when
32459          ;it finds any install command. the second pass will only handle the

```



```

32460 ;install= command.
32461
32462 ;-----
32463 ;install command
32464 ;-----
32465
32466 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32467 ; (SYSINIT:2250h)
32468
32469 0000258E 803E[CD02]00 tryi:
32470 00002593 7503 cmp byte [multi_pass_id],0; the initial pass for DOS=HI
32471 00002595 E97F01 jne short not_init_pass
32472 not_init_pass: jmp multi_try_doshi
32473 00002598 803E[CD02]02 cmp byte [multi_pass_id],2; the second pass was for ifs=
32474 ; 11/12/2022
32475 ;je short multi_pass_coff2; now it is NOPs
32476 0000259D 74E5 je short coff
32477 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32478 ;je short multi_pass_coff
32479 ; This pass can be made use of if
32480 ; we want do some config.sys process
32481 ; after device drivers are loaded
32482 ; and before install= commands
32483 ; are processed
32484
32485 0000259F 803E[CD02]03 cmp byte [multi_pass_id],3; the third pass for install= ?
32486 000025A4 741D je short multi_try_i
32487 000025A6 80FC48 cmp ah, CONFIG_DOS ; 'H'
32488 ; 11/12/2022
32489 ;je short multi_pass_coff2
32490 000025A9 74D9 je short coff
32491 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32492 ;je short multi_pass_coff
32493
32494 ; make note of any INSTALL= or INSTALLHIGH= commands we find,
32495 ; but don't process them now.
32496
32497 000025AB 80FC49 cmp ah,CONFIG_INSTALL ; 'I' ; install= command?
32498 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32499 000025AE 7507 jne short precheck_installhigh ; the first pass is for normal operation.
32500 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32501 ;jne short tryb
32502
32503 ;or word [install_flag],have_install_cmd ; set the flag
32504 000025B0 800E[CE02]01 or byte [install_flag],have_install_cmd ; 1
32505 multi_pass_coff2:
32506 000025B5 EBCD jmp short coff ; 13/05/2019 ; and handles the next command
32507
32508 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32509 ; (SYSINIT:2448h)
32510 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32511 ;%if 0
32512 precheck_installhigh:
32513 000025B7 80FC57 cmp ah,CONFIG_INSTALLHIGH ; 'w' ; signifier for INSTALLHIGH
32514 000025BA 756B jne short tryb ; carry on with normal processing
32515 ;or word [install_flag],have_install_cmd
32516 000025BC 800E[CE02]01 or byte [install_flag],have_install_cmd ; 1
32517 000025C1 EBC1 jmp short coff
32518 ;%endif ; 27/10/2022
32519
32520 multi_try_i:
32521 000025C3 80FC49 cmp ah,CONFIG_INSTALL ; 'I' ; install= command?
32522 ; 31/12/2022 - Retro DOS v4.2
32523 000025C6 750A jne short multi_try_n ; no, check for installhigh
32524 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32525 ;jne short multi_pass_filter
32526
32527 ; 31/12/2022
32528 ;%if 1
32529 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32530 ;%if 0
32531 ;ifdef MULTI_CONFIG
32532 000025C8 E84F20 call query_user ; query the user if config_cmd
32533 000025CB 7241 jc short multi_pass_filter ; has the CONFIG_OPTION_QUERY bit set
32534 ;endif
32535 ;%endif ; 27/10/2022
32536
32537 000025CD E8C9EC call do_install_exec ;install it.
32538 000025D0 EBB2 jmp short coff ;to handle next install= command.
32539
32540 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32541 ; (SYSINIT:2463h)
32542 ;%if 1
32543 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32544 ;%if 0
32545
32546 multi_try_n:
32547 000025D2 80FC57 cmp ah,CONFIG_INSTALLHIGH ; installhigh= command?
32548 000025D5 7537 jne short multi_pass_filter ; no. ignore this.
32549 ;ifdef MULTI_CONFIG
32550 000025D7 E84020 call query_user ; query the user if config_cmd
32551 000025DA 7232 jc short multi_pass_filter ; has the CONFIG_OPTION_QUERY bit set
32552 ;endif
32553
32554 ; The memory environment is in its normal DOS state, so do
32555 ; the standard calls to set the alloc strategy for loading high
32556
32557 000025DC B80058 mov ax,(ALLOCOPER<<8)|0 ; 5800h
32558 000025DF CD21 int 21h ;get alloc strategy
32559 000025E1 89C3 mov bx,ax
32560 000025E3 53 push bx ; save for the return
32561
32562 000025E4 81CB8000 or bx,HIGH_FIRST ; 80h ;set alloc to HighFirst
32563 000025E8 B80158 mov ax,(ALLOCOPER<<8)|1 ; 5801h
32564 000025EB CD21 int 21h ;set alloc strategy
32565
32566 000025ED B80258 mov ax,(ALLOCOPER<<8)|2 ; 5802h
32567 000025F0 CD21 int 21h ; get link state
32568 000025F2 30E4 xor ah,ah ; clear top byte
32569 000025F4 50 push ax ; save for return
32570
32571 000025F5 B80358 mov ax,(ALLOCOPER<<8)|3 ; 5803h
32572 000025F8 BB0100 mov bx,1
32573 000025FB CD21 int 21h ;link in UMBS
32574
32575 000025FD E899EC call do_install_exec ;install it.
32576
32577 00002600 B80358 mov ax,(ALLOCOPER<<8)|3
32578 00002603 5B pop bx ; recover original link state
32579 00002604 CD21 int 21h
32580 00002606 5B pop bx ; recover original alloc strategy
32581 00002607 B80158 mov ax,(ALLOCOPER<<8)|1
32582 0000260A CD21 int 21h
32583

```

```

32584             ;jmp     short coff             ;to handle next install= command.
32585             ; 01/01/2023
32586 0000260C EBA7             jmp     short multi_pass_coff2
32587
32588             ;%endif ; 27/10/2022
32589
32590 multi_pass_filter:
32591             cmp     ah,CONFIG_COMMENT ; 'Y' ; comment?
32592             je      short multi_pass_adjust
32593             cmp     ah,CONFIG_UNKNOWN ; 'Z' ; bad command?
32594             je      short multi_pass_adjust
32595             cmp     ah,CONFIG_REM ; '0' ; rem?
32596             jne     short multi_pass_coff ; ignore the rest of the commands.
32597
32598 multi_pass_adjust:
32599             dec     word [chrptr] ; these commands need to
32600             inc     word [count] ; adjust chrptr,count
32601             ; for newline proc.
32602
32603 multi_pass_coff:
32604             ; 11/12/2022
32605             ;jmp     short coff             ; to handle next install= commands.
32606             ; 01/01/2023
32607             jmp     short multi_pass_coff2
32608
32609             ;-----
32610             ; buffer command
32611             ;-----
32612             ;*****
32613             ;
32614             ; function: parse the parameters of buffers= command.
32615             ;
32616             ; input :
32617             ; es:si -> parameters in command line.
32618             ; output:
32619             ; buffers set
32620             ; buffer_slash_x flag set if /x option chosen.
32621             ; h_buffers set if secondary buffer cache specified.
32622             ;
32623             ; subroutines to be called:
32624             ; sysinit_parse
32625             ; logic:
32626             ; {
32627             ;     set di points to buf_parms; /*parse control definition*/
32628             ;     set dx,cx to 0;
32629             ;     reset buffer_slash_x;
32630             ;     while (end of command line)
32631             ;     { sysinit_parse;
32632             ;       if (no error) then
32633             ;       {
32634             ;         if (result_val._$P_synonym_ptr == slash_e) then /*not a switch *
32635             ;         {
32636             ;           buffer_slash_x = 1
32637             ;         }
32638             ;         else if (cx == 1) then /* first positional */
32639             ;         {
32640             ;           buffers = result_val._$P_picked_val;
32641             ;           else h_buffers = result_val._$P_picked_val;
32642             ;           else {show error message;error exit}
32643             ;         }
32644             ;       }
32645             ;       if (buffer_slash_x is off & buffers > 99) then show_error;
32646             ;     }
32647             ; }
32648             ;*****
32649             ;
32650             ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32651             ; (SYSINIT:229Ch)
32652             tryb:
32653             cmp     ah,CONFIG_BUFFERS ; 'B'
32654             jne     short tryc
32655
32656             ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32657             ; (SYSINIT:24BFh)
32658             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32659             ;%if 0
32660             ;ifdef MULTI_CONFIG
32661             call    query_user ; query the user if config_cmd
32662             jc      short tryc ; has the CONFIG_OPTION_QUERY bit set
32663             ;endif
32664             ;%endif ; 27/10/2022
32665
32666             ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32667             ; 18/12/2022
32668             xor     cx,cx
32669             mov     byte [p_buffer_slash_x],0 ; 31/03/2019
32670             mov     [p_buffer_slash_x],cl ; 0
32671
32672             mov     di,buf_parms
32673             ;xor     cx,cx ; 18/12/2022
32674             ; 03/01/2023
32675             mov     dx,cx
32676
32677 do7:
32678             call    sysinit_parse
32679             jnc     short if7 ; parse error,
32680             call    badparm_p ; and show messages and end the search loop.
32681             ;jmp     short sr7
32682             ; 31/12/2022
32683             ;sr7:
32684             jmp     coff
32685             ; 03/01/2023
32686             jmp     badparm_p_coff
32687
32688             if7:
32689             cmp     ax,_$P_RC_EOL ; 0FFFFh; end of line?
32690             je      short en7 ; then jmp to $endloop for semantic check
32691             cmp     word [result_val_swoff],switch_x
32692             cmp     word [result_val+_$P_Result_Blk.SYNONYM_Ptr],switch_x
32693             jne     short if11
32694             ; 31/12/2022
32695             je      short do7 ;je short en11
32696
32697             ; mov     byte [p_buffer_slash_x],1 ; set the flag M016
32698             ; jmp     short en11 ; 31/12/2022
32699             if11:
32700             mov     ax,[rv_dword]
32701             mov     ax,[result_val+_$P_Result_Blk.Picked_val]
32702             cmp     cx,1
32703             jne     short if13
32704
32705             mov     [p_buffers],ax
32706             jmp     short en11
32707             ; 31/12/2022
32708             jmp     short do7
32709
32710             if13:
32711             mov     [p_h_buffers],ax
32712             en11:
32713             jmp     short do7
32714
32715             en7:
32716             cmp     word [p_buffers],99

```

```

32708 00002666 760B      jbe     short if18
32709
32710      ; cmp     byte [p_buffer_slash_x],0 ; M016
32711      ; jne     short if18
32712
32713 00002668 E82508      call    badparm_p
32714 0000266B C706[1122]0000 mov     word [p_h_buffers],0
32715 00002671 EB12      jmp     short sr7
32716
32717 00002673 A1[0F22]     mov     ax,[p_buffers] ; we don't have any problem.
32718 00002676 A3[9902]     mov     [buffers],ax ; now,let's set it really.
32719
32720 00002679 A1[1122]     mov     ax,[p_h_buffers]
32721 0000267C A3[9B02]     mov     [h_buffers],ax
32722
32723      ; mov     al,[p_buffer_slash_x] ; M016
32724      ; mov     [buffer_slash_x],al
32725
32726 0000267F A1[AF02]     mov     ax,[linecount]
32727 00002682 A3[B902]     mov     [buffer_linenum],ax ; save the line number for the future use.
32728      ; 31/12/2022
32729      ; jmp     short sr7
32730      ; 03/01/2023
32731
32732 00002685 E9FCFE     sr7:    jmp     coff
32733
32734      ;-----
32735      ; break command
32736      ;-----
32737
32738      ;*****
32739      ; function: parse the parameters of break = command.
32740      ;
32741      ; input :
32742      ; es:si -> parameters in command line.
32743      ; output:
32744      ; turn the control-c check on or off.
32745      ;
32746      ; subroutines to be called:
32747      ; sysinit_parse
32748      ; logic:
32749      ; {
32750      ; {
32751      ;     set di to brk_parms;
32752      ;     set dx,cx to 0;
32753      ;     while (end of command line)
32754      ;     { sysinit_parse;
32755      ;       if (no error) then
32756      ;       { if (result_val._$P_item_tag == 1) then /*on */ *
32757      ;       { set p_ctrl_break,on;
32758      ;       { else /*off */ *
32759      ;       { set p_ctrl_break,off;
32760      ;       { else {show message;error_exit};
32761      ;       ;
32762      ;       if (no error) then
32763      ;       { dos function call to set ctrl_break check according to
32764      ;       ;
32765      ;       ;
32766      ;*****
32767
32768      ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32769      ; (SYSINIT:22FFh)
32770
32771 00002688 80FC43     tryc:   cmp     ah,CONFIG_BREAK ; 'C'
32772 0000268B 7539      jne     short trym
32773
32774      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32775      ; (SYSINIT:2527h)
32776      ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32777      ;%if 0
32778      ;ifdef MULTI_CONFIG
32779      ; call     query_user ; query the user if config_cmd
32780      ; jc      short trym ; has the CONFIG_OPTION_QUERY bit set
32781      ;endif
32782      ;%endif ; 27/10/2022
32783
32784 00002692 BF[1D22]     mov     di,brk_parms
32785 00002695 31C9      xor     cx,cx
32786      ; 03/01/2023
32787      ;mov     dx,cx
32788
32789 00002697 E8CB07     do22:   call    sysinit_parse
32790 0000269A 7303      jnc     short if22 ; parse error
32791      ; call    badparm_p ; show message and end the search loop.
32792      ; jmp     short sr22
32793      ; 31/12/2022
32794
32795      ;sr22:
32796      ; jmp     coff
32797      ; 03/01/2023
32798      ; jmp     badparm_p_coff
32799
32800 0000269F 83F8FF     if22:   cmp     ax,_$P_RC_EOL ; end of line?
32801 000026A2 7415      je      short en22 ; then end the $endloop
32802
32803      ; cmp     byte [result_val_itag],1
32804 000026A4 803E[1622]01     cmp     byte [result_val+_$P_Result_Blk.Item_Tag],1
32805 000026A9 7507      jne     short if26
32806
32807 000026AB C606[4222]01     mov     byte [p_ctrl_break],1 ; turn it on
32808      ; jmp     short en26
32809      ; 31/12/2022
32810      ; jmp     short do22
32811
32812 000026B2 C606[4222]00     if26:   mov     byte [p_ctrl_break],0 ; turn it off
32813      ; jmp     short do22 ; we actually set the ctrl break
32814
32815 000026B7 EBDE     en22:   jmp     short do22
32816
32817 000026B9 B433      mov     ah,SET_CTRL_C_TRAPPING ; if we don't have any parse error.
32818 000026BB B001      mov     al,1
32819 000026BD 8A16[4222]     mov     dl,[p_ctrl_break]
32820 000026C1 CD21      int     21h
32821      ; 31/12/2022
32822      ; jmp     short sr22
32823      ; 03/01/2023
32824
32825 000026C3 E9BEFE     sr22:   jmp     coff
32826
32827      ;-----
32828      ; multitrack command
32829      ;-----
32830
32831      ;*****
32832      ; function: parse the parameters of multitrack= command.
32833      ;

```

```

32832 ;
32833 ; input :
32834 ; es:si -> parameters in command line.
32835 ; output:
32836 ; turn multrk_flag on or off.
32837 ;
32838 ; subroutines to be called:
32839 ; sysinit_parse
32840 ; logic:
32841 ; {
32842 ;     set di to brk_parms;
32843 ;     set dx,cx to 0;
32844 ;     while (end of command line)
32845 ;     { sysinit_parse;
32846 ;         if (no error) then
32847 ;             if (result_val._$P_item_tag == 1) then          /*on      */
32848 ;                 set p_mtrk,on;
32849 ;             else                                           /*off      */
32850 ;                 set p_mtrk,off;
32851 ;             else {show message;error_exit};
32852 ;         };
32853 ;         if (no error) then
32854 ;             dos function call to set multrk_flag according to p_mtrk.
32855 ;         };
32856 ;     };
32857 ;
32858 ;*****
32859 ;
32860 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32861 trym:
32862     cmp     ah,CONFIG_MULTITRACK ; 'M'
32863     jne     short tryu
32864 ;
32865 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32866 ; (SYSINIT:2569h)
32867 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32868 ;%if 0
32869 ;ifdef MULTI_CONFIG
32870     call    query_user      ; query the user if config_cmd
32871     jc      short tryu      ; has the CONFIG_OPTION_QUERY bit set
32872 ;endif
32873 ;%endif ; 27/10/2022
32874 ;
32875     mov     di,mtrk_parms
32876     xor     cx,cx
32877     ; 03/01/2023
32878     ;mov     dx,cx
32879 do31:
32880     call    sysinit_parse
32881     jnc     short if31      ; parse error
32882     ;call    badparm_p      ; show message and end the search loop.
32883     ;;jmp     short sr31
32884     ; 31/12/2022
32885 ;sr31:
32886     ;jmp     coff
32887     ; 03/01/2023
32888     jmp     badparm_p_coff
32889 ;if31:
32890     cmp     ax,_$P_RC_EOL ; end of line?
32891     je      short en31      ; then end the $endloop
32892 ;
32893     ;cmp     byte [result_val_itag],1
32894     ;cmp     byte [result_val+_$P_Result_Blk.Item_Tag],1
32895     ;jne     short if35
32896 ;
32897     mov     byte [p_mtrk],1 ; turn it on temporarily.
32898     ;jmp     short en35
32899     ; 31/12/2022
32900     jmp     short do31
32901 ;if35:
32902     mov     byte [p_mtrk],0 ; turn it off temporarily.
32903 ;en35:
32904     jmp     short do31      ; we actually set the multrk_flag here
32905 ;en31:
32906     push    ds
32907     ;;mov     ax,Bios_Data ; 70h
32908     ;mov     ax,KERNEL_SEGMENT ; 70h
32909     ; 21/10/2022
32910     mov     ax,DOSBIODATASEG ; 0070h
32911     mov     ds,ax
32912 ;
32913     cmp     byte [cs:p_mtrk],0
32914     jne     short if39
32915 ;
32916     mov     word [multrk_flag],multrk_off2 ; 0001h
32917     jmp     short en39
32918 ;if39:
32919     mov     word [multrk_flag],multrk_on ; 0080h
32920 ;en39:
32921     pop     ds
32922     ; 31/12/2022
32923     ;jmp     short sr31
32924     ; 03/01/2023
32925 ;sr31:
32926     jmp     coff
32927 ;
32928 ;-----
32929 ; DOS=HIGH/LOW command
32930 ;-----
32931 ;
32932 ; 27/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32933 multi_try_doshi:
32934     cmp     ah,CONFIG_DOS ; 'H'
32935     je      short it_is_h
32936 ;skip_it:
32937     jmp     multi_pass_filter
32938 ;it_is_h:
32939     ; M003 - removed initing DevUMB
32940     ; & runhigh
32941 ;
32942 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32943 ; (SYSINIT:25C1h)
32944 ; 27/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32945 ;%if 0
32946 ;ifdef MULTI_CONFIG
32947     call    query_user      ; query the user if config_cmd
32948     jc      short skip_it    ; has the CONFIG_OPTION_QUERY bit set
32949 ;endif
32950 ;%endif ; 27/10/2022
32951 ;
32952     mov     di,dos_parms
32953     xor     cx,cx
32954     ; 03/01/2023
32955     ;mov     dx,cx
32956 h_do_parse:
32957     call    sysinit_parse

```

```

32956 0000272C 7303      jnc     short h_parse_ok      ; parse error
32957 h_badparm:
32958      ; 03/01/2023
32959      ; call badparm_p          ; show message and end the search loop.
32960      ; jmp short h_end
32961      ; 11/12/2022
32962 ;h_end:
32963      ; jmp coff
32964      ; 03/01/2023
32965 0000272E E92306      jmp     badparm_p_coff
32966 h_parse_ok:
32967      cmp     ax, _P_RC_EOL      ; end of line?
32968      je      short h_end        ; then end the $endloop
32969      call    ProcDOS
32970      jmp     short h_do_parse
32971      ; 11/12/2022
32972      ; 03/01/2023
32973 h_end:
32974      jmp     coff
32975
32976 ;-----
32977 ; devicehigh command
32978 ;-----
32979
32980      ; 28/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
32981 tryu:
32982      cmp     ah, CONFIG_DEVICEHIGH ; 'U'
32983      jne     short tryd
32984
32985      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
32986      ; (SYSINIT:25E9h)
32987      ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32988      ; if 0
32989      ; ifdef MULTI_CONFIG
32990      call    query_user          ; query the user if config_cmd
32991      jc      short tryd          ; has the CONFIG_OPTION_QUERY bit set
32992      ; endif
32993      ; endif ; 28/10/2022
32994
32995      ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
32996      ; if 0
32997      ; 01/01/2023
32998      ; ds = cs
32999
33000      call    InitVar
33001      call    ParseSize          ; process the size= option
33002      ; jnc short tryu_0
33003      ; 31/12/2022
33004 0000274E 720C      jc      short tryu_1 ; 31/03/2019 - Retro DOS v4.0
33005
33006      ; endif ; 28/10/2022
33007
33008      ; 31/12/2022
33009      ; if 0
33010      ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33011      ; mov [cs:badparm_off], si ; stash it there in case of an error
33012      ; mov [cs:badparm_seg], es
33013      ; 11/12/2022
33014      ; ds = cs
33015      mov     [badparm_off], si
33016      mov     [badparm_seg], es
33017
33018      ; 31/12/2022
33019      ; call ParseSize
33020      ; jnc short tryu_2 ; 28/10/2022
33021
33022      ; call badparm_p
33023      ; jmp coff
33024      ; 03/01/2023
33025      jmp     badparm_p_coff
33026      ; endif
33027
33028      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33029      ; (SYSINIT:2606h)
33030      ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33031      ; if 0
33032      tryu_0:
33033      ; mov ax, [cs:DevSizeOption]
33034      ; 31/12/2022
33035      mov     ax, [DevSizeOption] ; ds = cs
33036      or      ax, ax
33037      jnz     short tryu_2
33038
33039      call    ParseVar
33040      jnc     short tryu_2
33041      tryu_1:
33042      ; 31/12/2022
33043      ; ds = cs
33044      mov     [badparm_off], si
33045      mov     [badparm_seg], es
33046      ; mov [cs:badparm_off], si ; If ParseVar up there failed, then
33047      ; mov [cs:badparm_seg], es ; ES:SI points to its problem area...
33048
33049      ; call badparm_p
33050      ; jmp coff
33051      ; 03/01/2023
33052      jmp     badparm_p_coff
33053
33054      ; endif ; 28/10/2022
33055
33056      tryu_2:
33057      push    si
33058      push    es
33059
33060      ; 08/09/2023 - Retro DOS 4.2 IO.SYS (Optimization)
33061      ; MSDOS 6.21 IO.SYS - SYSINIT:2623h
33062      ; PCDOS 7.1 IBMBIO.COM - SYSINIT:2B17h
33063      tryu_3:
33064      mov     al, [es:si]
33065      cmp     al, cr
33066      ; 14/04/2024
33067      ; je short tryu_4
33068      ; 08/09/2023 - Retro DOS 5.0 (PCDOS 7.1 IBMBIO.COM)
33069      je      short tryu_5
33070      cmp     al, lf
33071      je      short tryu_4
33072      call    delim
33073      jz      short tryu_4
33074      inc     si
33075      jmp     short tryu_3
33076
33077      ; 14/04/2024
33078      ; 08/09/2023 - Retro DOS 5.0 (PCDOS 7.1 IBMBIO.COM)
33079      tryu_5:

```

```

33080 0000277C B020      mov     al,20h ; ' ' ; blank instead of cr
33081
33082
33083
33084
33085 0000277E A2[6424]   tryu_4:
33086                      ; 11/12/2022
33087                      ; ds = cs
33088                      mov     [DevSavedDelim],al
33089                      ;mov    [cs:DevSavedDelim],al ; Save the delimiter before replacing
33090                      ; it with null
33091                      ; 18/12/2022
33092                      sub     bx,bx
33093                      mov     [es:si],bl ; 0
33094                      ;mov    byte [es:si],0
33095
33096                      pop     es
33097                      pop     si ; 14/04/2024
33098
33099
33100                      ;-----
33101                      ; BEGIN PATCH TO CHECK FOR NON-EXISTANT UMBS -- t-richj 7-21-92
33102                      ;-----
33103                      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33104                      ; (SYSINIT:2642h)
33105                      ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33106                      ;%if 0
33107                      ; 10/07/2023 - Retro DOS v4.2 IO.SYS (optimized)
33108                      ; MSDOS 6.21 IO.SYS - SYSINIT:2642h
33109                      %if 1
33110                      ; 01/01/2023
33111                      ; ds = cs
33112                      call     UmbTest ; See if UMBS are around...
33113                      ; 01/01/2023
33114                      ;jnc     short NrmTst ; ...yep. So do that normal thang.
33115                      ;mov     byte [cs:DeviceHi],0 ; ...nope... so load low.
33116                      ; 31/12/2022
33117                      ; ds = cs, bx = 0
33118                      ;mov     byte [DeviceHi],bl ; 0
33119                      ;jmp     short LoadDevice
33120                      ; 01/01/2023
33121                      ;jc       short LoadDevice ; bl = 0
33122                      %endif
33123                      ;%endif
33124                      ;-----
33125                      ; END PATCH TO CHECK FOR NON-EXISTANT UMBS -- t-richj 7-21-92
33126                      ;-----
33127                      NrmTst:
33128                      ; 11/12/2022
33129                      ; ds = cs
33130                      ;mov     byte [cs:DeviceHi],0
33131                      ;mov     byte [DeviceHi],0
33132                      ; 18/12/2022
33133                      ; bx = 0
33134                      cmp     [DevUMB],bl ; 0
33135                      ;cmp     byte [DevUMB],0
33136                      ;;cmp    byte [cs:DevUMB],0 ; do we support UMBS
33137                      ;je       short LoadDevice ; no, we don't
33138                      ;mov     byte [cs:DeviceHi],1
33139                      ; 11/12/2022
33140                      ;mov     byte [DeviceHi],1
33141                      ; 18/12/2022
33142                      inc     bl ; mov bl,1 ; (*)
33143                      ; 11/12/2022
33144                      ;jmp     short LoadDevice2 ; 11/12/2022
33145                      jmp     short LoadDevice
33146
33147                      ;-----
33148                      ; device command
33149                      ;-----
33150                      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33151                      ; (SYSINIT:2665h)
33152                      ; 28/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33153                      ; (SYSINIT:2401h)
33154                      tryd:
33155                      ; 11/12/2022
33156                      ;xor     bx,bx ; 31/12/2022
33157                      ;
33158                      ;
33159                      cmp     ah,CONFIG_DEVICE ; 'D'
33160                      je       short gotd
33161                      skip_it2:
33162                      jmp     tryq
33163                      gotd:
33164
33165                      ; 31/12/2022 - Retro DOS v4.2
33166                      ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33167                      ;%if 0
33168                      ;ifdef    MULTI_CONFIG
33169                      call     query_user ; query the user if config_cmd
33170                      jc       short skip_it2 ; has the CONFIG_OPTION_QUERY bit set
33171                      ;endif
33172                      ;%endif ; 28/10/2022
33173
33174                      ; 31/12/2022
33175                      sub     bx,bx
33176                      ; bx = 0
33177                      ; 11/12/2022
33178                      ; ds = cs
33179                      ;mov     byte [DeviceHi],0
33180                      ;mov     word [DevSizeOption],0
33181                      mov     [DevSizeOption],bx ; 0
33182                      mov     byte [DevSavedDelim],' '
33183                      ;mov     byte [cs:DeviceHi],0 ; not to be loaded in UMB ;M007
33184                      ;mov     word [cs:DevSizeOption],0
33185                      ;mov     byte [cs:DevSavedDelim],' ' ; In case of DEVICE= the null has to
33186                      ; be replaced with a ' '
33187                      LoadDevice:
33188                      ; 11/12/2022
33189                      ;mov     byte [DeviceHi],0
33190                      mov     byte [DeviceHi],bl ; 0 or 1 (*)
33191                      LoadDevice2:
33192                      ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33193                      ;
33194                      ;push     cs
33195                      ;pop     ds
33196                      ;
33197                      ;mov     [bpb_addr],si ; pass the command line to the dvice
33198                      ;mov     [bpb_addr+2],es
33199                      ;
33200                      ;mov     [DevCmdLine],si ; save it for ourself
33201                      ;mov     [DevCmdLine+2],es
33202                      ;
33203                      ;mov     byte [driver_units],0 ; clear total block units for driver

```



```

33204
33205 ; 11/12/2022
33206 ; ds = cs
33207 ;mov bx,cs
33208 ;mov ds,bx
33209
33210 ;mov [cs:bpb_addr],si ; pass the command line to the device
33211 000027B3 8936[8103] mov [bpb_addr],si
33212 ;mov [cs:bpb_addr+2],es
33213 000027B7 8C06[8303] mov [bpb_addr+2],es
33214
33215 ;mov [cs:DevCmdLine],si ; save it for ourself
33216 000027BB 8936[6024] mov [DevCmdLine],si
33217 ;mov [cs:DevCmdLine+2],es
33218 000027BF 8C06[6224] mov [DevCmdLine+2],es
33219
33220 ; 31/12/2022 - Retro DOS v4.2
33221 000027C3 C606[6A19]00 mov byte [driver_units],0 ; clear total block units for driver
33222
33223 000027C8 E82520 call round
33224
33225 000027CB E8910E call SizeDevice
33226 000027CE 723F jc short BadFile
33227
33228 ; 11/12/2022
33229 ; ds = cs
33230
33231 ; - Begin DeviceHigh primary logic changes -----
33232
33233 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33234 ; (SYSINIT:26A4h)
33235
33236 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33237 ;%if 0
33238 000027D0 C606[3F24]01 mov byte [ConvLoad],1 ; Doesn't matter if DeviceHi==0
33239
33240 ; 22/07/2023
33241 ;mov al,[DeviceHi] ; If not using upper memory,
33242 000027D5 800E[4F24]00 or byte [DeviceHi],0 ; Skip all this and go on to
33243 ; 10/07/2023
33244 ;or al,al
33245 000027DA 741E jz short DevConvLoad ; the actual load.
33246
33247 ;call GetLoadUMB ; Returns first UMB spec'ed in AX
33248 000027DC A0[FD23] mov al,[UmbLoad] ; 19/04/2019 - Retro DOS v4.0
33249
33250 000027DF 3CFF cmp al,-1 ; If umb0 not specified, it's old style
33251 000027E1 7417 jz short DevConvLoad ; so load high even if SIZE= is smaller
33252
33253 000027E3 FE0E[3F24] dec byte [ConvLoad] ; 0 ; They specified /L, so use new loader
33254
33255 000027E7 E8590A call GetLoadSize ; Returns size of first UMB specified
33256 000027EA 09C0 or ax,ax
33257 000027EC 7406 jz short tryd_1 ; If size1 not specified, nada to do:
33258
33259 000027EE 3B06[3124] cmp ax,[DevSize] ; /L:...,Size < DevSize?
33260 000027F2 7B06 jge short DevConvLoad
33261 tryd_1:
33262 000027F4 A1[3124] mov ax,[DevSize] ; Size < DevSize, so write DevSize as
33263 000027F7 E8550A call StoLoadSize ; minsize for load UMB.
33264
33265 ;%endif ; 28/10/2022
33266
33267 ; - End DeviceHigh primary logic changes -----
33268
33269 DevConvLoad:
33270 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33271 000027FA E8660D call InitDevLoad
33272
33273 ; 11/12/2022
33274 ; ds = cs
33275 000027FD A1[3324] mov ax,[DevLoadAddr]
33276 00002800 0306[3124] add ax,[DevSize]
33277 00002804 7206 jc short NoMem
33278 00002806 3906[3524] cmp [DevLoadEnd],ax
33279 0000280A 7315 jae short LoadDev
33280
33281 ; 11/12/2022
33282 ;mov ax,[cs:DevLoadAddr]
33283 ;add ax,[cs:DevSize]
33284 ;jc short NoMem
33285 ;cmp [cs:DevLoadEnd],ax
33286 ;jae short LoadDev
33287 NoMem:
33288 ; 11/12/2022
33289 ; ds = cs
33290 ;jmp mem_err
33291 0000280C E92020 jmp mem_err2
33292
33293 BadFile:
33294 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33295 ;;call RetFromUM ; Does nothing if didn't call HideUMBS
33296 ;;cmp byte [es:si], ' '
33297 ;;jae short tryd_2
33298 ; 31/12/2022
33299 ;cmp byte [es:si],0Dh ; cr
33300 ;jne short tryd_2
33301 ;jmp badop
33302 ; 31/12/2022
33303 ; ds = cs
33304 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33305 ; (SYSINIT:26E6h)
33306 0000280F E8AE0E call RetFromUM ; Does nothing if didn't call HideUMBS
33307 00002812 26803C20 cmp byte [es:si], ' '
33308 ;cmp byte [es:si],20h ; space
33309 00002816 7303 jnb short tryd_2
33310 00002818 E96906 jmp badop
33311 tryd_2:
33312 0000281B E80C22 call badload
33313 0000281E E963FD jmp coff
33314
33315 LoadDev:
33316 00002821 06 push es
33317 00002822 1F pop ds
33318
33319 00002823 89F2 mov dx,si ;ds:dx points to file name
33320 00002825 E87C0E call ExecDev ; load device driver using exec call
33321 badldreset:
33322 00002828 1E push ds
33323 00002829 07 pop es ;es:si back to config.sys
33324 0000282A 0E push cs
33325 0000282B 1F pop ds ;ds back to sysinit
33326 0000282C 72E1 jc short BadFile
33327 goodld:

```

```

33328 ; 11/12/2022
33329 ; ds = cs
33330
33331 0000282E 06 push es ; + ; 31/12/2022
33332 0000282F 56 push si ; ++
33333 00002830 E89E0E call RemoveNull
33334 00002833 06 push es
33335 00002834 56 push si
33336
33337 00002835 0E push cs
33338 00002836 07 pop es
33339
33340 00002837 1E push ds ; ** ; ds = cs
33341 00002838 56 push si
33342
33343 ;lds si,[cs:DevEntry] ; peeks the header attribute
33344 ; 31/12/2022
33345 ; ds = cs
33346 00002839 C536[3724] lds si,[DevEntry]
33347
33348 ;test word [si+4],8000h
33349 ; 11/12/2022
33350 0000283D F6440580 test byte [si+SYSDEV.ATT+1],DEVTYP>>8
33351 ;test word [si+SYSDEV.ATT],DEVTYP ; block device driver?
33352 00002841 7514 jnz short got_device_com_cont ; no.
33353
33354 00002843 2EC536[6D02] lds si,[cs:DOSINFO] ; ds:si -> sys_var
33355 ;cmp byte [si+32],26
33356 00002848 807C201A cmp byte [si+SYSI_NUMIO],26 ; no more than 26 drive number
33357 0000284C 7209 jb short got_device_com_cont
33358
33359 0000284E 5E pop si
33360 0000284F 1F pop ds ; **
33361
33362 00002850 5E pop si ; clear the stack
33363 00002851 07 pop es
33364
33365 ; 28/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33366 ;call RetFromUM
33367 ; 31/12/2022
33368 ; ds = cs ; **
33369 00002852 E86B0E call RetFromUM ; Do this before we leave
33370
33371 ;jmp short badnumblock
33372 ; 31/12/2022
33373 00002855 EB73 jmp short badnumblock2 ; ds = cs
33374
33375 got_device_com_cont:
33376 00002857 5E pop si
33377 00002858 1F pop ds
33378
33379 ; 11/12/2022
33380 ; ds = cs
33381
33382 00002859 E8AE06 call LieInt12Mem
33383 0000285C E80B07 call UpdatePDB ; update the PSP:2 value M020
33384
33385 ; 11/12/2022
33386 ; ds = cs
33387 ; 08/09/2023
33388 0000285F 31C0 xor ax, ax ; 0
33389 00002861 3806[6619] cmp byte [multdeviceflag],al ; 0
33390 ;cmp byte [multdeviceflag],0
33391 ;cmp byte [cs:multdeviceflag],0 ; Pass limit only for the 1st device
33392 ; driver in the file ; M027
33393 00002865 750B jne short skip_pass_limit ; M027
33394
33395 ; 11/12/2022
33396 ; ds = cs
33397 ;mov word [cs:break_addr],0; pass the limit to the DD
33398 ;mov bx,[cs:DevLoadEnd]
33399 ;mov [cs:break_addr+2],bx
3400
3401 ;mov word [break_addr],0
3402 ; 08/09/2023
3403 00002867 A3[7D03] mov [break_addr],ax ; 0
3404 0000286A 8B1E[3524] mov bx,[DevLoadEnd]
3405 0000286E 891E[7F03] mov [break_addr+2],bx
3406
3407 skip_pass_limit:
3408 ; Note: sysi_numio (in DOS DATA) currently reflects the REAL
3409 ; number of installed devices (including Dblspace drives) where
3410 ; "drivenumber" is the number that the next block device will
3411 ; be assigned to. Because some naughty device drivers (like
3412 ; interlnk) look at the internal DOS variable instead of the
3413 ; value we pass it, we'll temporarily stick our value into
3414 ; DOS DATA while we're initializing the device drivers.
3415 ;
3416 ; Note that this will make it impossible for this device
3417 ; driver to access the Dblspace drive letters, whether
3418 ; they are swapped-hosts or unswapped compressed drives,
3419 ; during its initialization phase.
3420
3421 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
3422 ; (SYSINIT:2752h)
3423 ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
3424 ;%if 0
3425 ; 31/12/2022
3426 ;push ds
3427
3428 ;lds bx,[cs:DOSINFO] ; ds:bx -> sys_var
3429 ; 31/12/2022
3430 ; ds = cs
3431 ; 08/09/2023
3432 ;lds bx,[DOSINFO] ; ds:bx -> sys_var
3433
3434 ;mov al,[cs:drivenumber] ; temporarily use this next drv value
3435 ;mov [cs:devdrivenum],al ; pass drive number in packet to driver
3436 ;mov ah,al
3437
3438 ; 08/09/2023
3439 ; ds = cs
3440 00002872 A0[8503] mov al,[drivenumber] ; temporarily use this next drv value
3441 00002875 A2[8503] mov [devdrivenum],al ; pass drive number in packet to driver
3442 00002878 88C4 mov ah,al
3443 0000287A C51E[6D02] lds bx,[DOSINFO] ; ds:bx -> sys_var
3444
3445 0000287E 874720 xchg ax,[bx+SYSI_NUMIO] ; swap with existing values
3446 ; 31/12/2022
3447 ;pop ds
3448
3449 00002881 50 push ax ; save real sysi_numio/ncds in ax
3450
3451 ;%endif ; 29/10/2022

```



```

33452
33453 ; 29/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33454 ; (SYSINIT:24B9h)
33455
33456 00002882 BB0600 mov     bx,SYSDEV.STRAT ; 6
33457 00002885 E8B01F call    calldev          ; calldev (sdevstrat);
33458 00002888 BB0800 mov     bx,SYSDEV.INT ; 8
33459 0000288B E8AA1F call    calldev          ; calldev (sdevint);
33460
33461 ; 11/12/2022
33462 ; ds <= cs (from calldev) ; 31/12/2022
33463
33464 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33465 ; (SYSINIT:2773h)
33466 ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33467 ;%if 0
33468 0000288E 58      pop     ax                ; get real sysi_numio value
33469 ; 31/12/2022
33470 ;push     ds
33471 0000288F 2EC51E[6D02] lds     bx,[cs:DOSINFO]          ; ds:bx -> sys_var
33472 00002894 894720 mov     [bx+SYSI_NUMIO],ax      ; swap with existing values
33473 ; 31/12/2022
33474 ;pop      ds
33475
33476 ;%endif ; 29/10/2022
33477
33478 ; 11/12/2022
33479 00002897 0E      push    cs
33480 00002898 1F      pop     ds
33481
33482 00002899 E89C06 call    TrueInt12Mem
33483
33484 ; 11/12/2022
33485 ; ds = cs
33486 ;mov     ax,[cs:break_addr]      ; move break addr from the req packet
33487 ;mov     [cs:DevBrkAddr],ax
33488 ;mov     ax,[cs:break_addr+2]
33489 ;mov     [cs:DevBrkAddr+2],ax
33490 0000289C A1[7D03] mov     ax,[break_addr]
33491 0000289F A3[3B24] mov     [DevBrkAddr],ax
33492 000028A2 A1[7F03] mov     ax,[break_addr+2]
33493 000028A5 A3[3D24] mov     [DevBrkAddr+2],ax
33494
33495 ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33496 ;call    RetFromUM              ; There we go... all done.
33497 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33498 ; (SYSINIT:2791h)
33499 000028A8 E8150E call    RetFromUM              ; There we go... all done.
33500
33501 ; 31/12/2022
33502 ; ds = cs
33503
33504 ; 11/12/2022
33505 000028AB 803E[4024]00 cmp     byte [DevUMB],0
33506 ;cmp     byte [cs:DevUMB],0
33507 000028B0 7403 je      short tryd_3
33508 000028B2 E80010 call    AllocUMB
33509 ; 31/12/2022
33510 ; ds = cs
33511 tryd_3:
33512
33513 ;ifndef ROMDOS
33514 ;----- If we are waiting to be moved into hma lets try it now !!!
33515
33516 ; 11/12/2022
33517 ; ds = cs
33518
33519 ;cmp     byte [cs:runhigh],0FFh
33520 000028B5 803E[6C02]FF cmp     byte [runhigh],0FFh ; 11/12/2022
33521 000028BA 7503 jne     short tryd_4
33522
33523 ; 11/12/2022
33524 ; ds = cs
33525 000028BC E8D9E1 call    TryToMovDOSHi          ; move DOS into HMA if reqd
33526 tryd_4:
33527 ;endif ; ROMDOS
33528
33529 000028BF 5E      pop     si
33530 000028C0 1F      pop     ds
33531 000028C1 C60400 mov     byte [si],0          ; *p = 0;
33532
33533 000028C4 0E      push    cs
33534 000028C5 1F      pop     ds
33535
33536 000028C6 EB1F jmp     short was_device_com
33537
33538 ;-----
33539
33540 ; 02/04/2019 - Retro DOS v4.0
33541
33542 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33543 ; (SYSINIT:27B3h)
33544
33545 badnumblock:
33546 000028C8 0E      push    cs
33547 000028C9 1F      pop     ds
33548 badnumblock2:
33549 000028CA BA[6E51] mov     dx,badblock
33550 000028CD E88221 call    print
33551
33552 ;----- fall thru -----
33553
33554 ; 31/12/2022 - Retro DOS v4.2
33555
33556 erase_dev_do:
33557 ; modified to show message "error in config.sys..."
33558
33559 ;call    CheckDoubleSpace ; MSDOS 6.21 IO.SYS SYSINIT:27BBh
33560 ; (Note: 'call CheckDoubleSpace'
33561 ; has been removed at 'erase_dev_do:' pos
33562 ; in PCDOS 7.1 IBMBIO.COM - SYSINIT:2CBAh)
33563 ; Erdogan Tan - 10/07/2023
33563 000028D0 5E      pop     si ; ++
33564 000028D1 07      pop     es ; + ; 31/12/2022
33565
33566 000028D2 0E      push    cs
33567 000028D3 1F      pop     ds
33568
33569 skip1_resetmemhi:
33570 ; 11/12/2022
33571 ; ds = cs
33572 000028D4 833E[8603]00 cmp     word [configmsgflag],0
33573 ;cmp     word [cs:configmsgflag],0
33574 000028D9 7409 je      short no_error_line_msg
33575

```

```

33576 000028DB E8DA05      call error_line          ; no "error in config.sys" msg for device driver. dcr d493
33577                          ; 11/12/2022
33578                          ; ds = cs
33579                          ;mov word [cs:configmsgflag],0
33580 000028DE C706[8603]0000 mov word [configmsgflag],0; set the default value again.
33581
33582 no_error_line_msg:
33583 000028E4 E99DFC      jmp      coff
33584
33585 ;-----
33586
33587 was_device_com:
33588 ; 14/12/2022
33589 ; ds = cs
33590 000028E7 A1[3D24]      mov     ax,[DevBrkAddr+2]
33591                          ;mov ax,[cs:DevBrkAddr+2] ; 13/05/2019
33592 000028EA 3B06[3524]      cmp     ax,[DevLoadEnd]
33593                          ;cmp ax,[cs:DevLoadEnd]
33594 000028EE 7605          jbe     short breakok
33595
33596 000028F0 5E            pop     si
33597 000028F1 07            pop     es
33598 000028F2 E91AFF      jmp     BadFile
33599
33600 breakok:
33601 ; 14/12/2022
33602 ; ds = cs
33603 000028F5 C43E[6D02]      les     di,[DOSINFO]
33604 000028F9 C516[3724]      lds     dx,[DevEntry]
33605                          ;lds dx,[cs:DevEntry] ;set ds:dx to header
33606 000028FD 89D6          mov     si,dx
33607
33608 ; 14/11/2022
33609 ;les di,[cs:DOSINFO]      ;es:di point to dos info
33610
33611 ; 14/12/2022
33612 ; ds <> cs
33613
33614 ;mov ax,[si+4]
33615 000028FF 8B4404      mov     ax,[si+SYSDEV.ATT] ;get attributes
33616 ; 12/12/2022
33617 00002902 F6C480      test    ah,DEVTYPE>>8 ; 80h
33618 ;test ax,DEVTYPE ; 8000h ;test if block dev
33619 00002905 7426          jz      short isblock
33620
33621 ;----- lets deal with character devices
33622
33623 00002907 2E800E[6919]02 or      byte [cs:setdevmarkflag],for_devmark ; 2
33624 0000290D E8F40D      call   DevSetBreak      ;go ahead and alloc mem for device
33625
33626 00002910 72BE          jc      short erase_dev_do ;device driver's init routine failed.
33627
33628 ; 12/12/2022
33629 00002912 A801          test    al,ISCIN
33630 ;test ax,ISCIN ; 1      ;is it a console in?
33631 00002914 7408          jz      short tryclk
33632
33633 00002916 2689550C      mov     [es:di+SYSI_CON],dx ; es:di+12
33634 0000291A 268C5D0E      mov     [es:di+SYSI_CON+2],ds ; es:di+14
33635
33636 tryclk:
33637 ; 12/12/2022
33638 0000291E A808          test    al,ISCLOCK
33639 ;test ax,ISCLOCK ; 8    ;is it a clock device?
33640 00002920 7408          jz      short golink
33641
33642 00002922 26895508      mov     [es:di+SYSI_CLOCK],dx ; es:di+8
33643 00002926 268C5D0A      mov     [es:di+SYSI_CLOCK+2],ds ; es:di+10
33644 0000292A E9DF00      jmp     linkit
33645
33646 ;----- deal with block device drivers
33647
33648 isblock:
33649 0000292D 2EA0[7C03]      mov     al,[cs:unitcount] ;if no units found,erase the device
33650 00002931 08C0          or      al,al
33651 00002933 749B          jz      short erase_dev_do
33652 ;mov [si+10],al
33653 00002935 88440A      mov     [si+SYSDEV.NAME],al ; number of units in name field
33654 ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33655 ;add [cs:driver_units],al
33656 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33657 00002938 2E0006[6A19] add     [cs:driver_units],al ; keep total for all drivers in file
33658
33659 0000293D 98            cbw     ; warning no device > 127 units
33660 0000293E 89C1          mov     cx,ax
33661 00002940 88E6          mov     dh,ah
33662 ;mov dl,[es:di+32]
33663 00002942 268A5520      mov     dl,[es:di+SYSI_NUMIO] ;get number of devices
33664 00002946 88D4          mov     ah,dl
33665 00002948 00C4          add     ah,al ; check for too many devices
33666 0000294A 80FC1A      cmp     ah,26 ; 'A' - 'Z' is 26 devices
33667 0000294D 7603          jbe     short ok_block
33668 0000294F E976FF      jmp     badnumblock
33669
33670 ok_block:
33671 00002952 2E800E[6919]02 or      byte [cs:setdevmarkflag],for_devmark ; 2
33672 00002958 E8A90D      call   DevSetBreak      ; alloc the device
33673 0000295B 72B3          jc      short jc_edd
33674 0000295D 26004520      add     [es:di+SYSI_NUMIO],al ; update the amount
33675
33676 00002961 2E0006[8503]      add     [cs:drivenumber],al ; remember amount for next device
33677 00002966 2EC51E[8103]      lds     bx,[cs:bpb_addr] ; point to bpb array
33678
33679 0000296B 2EC42E[6D02]      perunit:
33680 ;les bp,[cs:DOSINFO]
33681 ;les bp,[es:bp+SYSI_DPB] ; get first dpb
33682 ; 11/12/2022
33683 00002970 26C46E00      les     bp,[es:bp]
33684 ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33685 ;les bp,[es:bp+0] ; [es:bp+SYSI_DPB]
33686
33687 00002974 26837E19FF      scandpb:
33688 00002979 7406          ;cmp word [es:bp+25],-1
33689 ;cmp word [es:bp+DPB.NEXT_DPB],-1
33690 0000297B 26C46E19      je      short foundpb
33691 0000297F EBF3          ;les bp,[es:bp+25]
33692 ;les bp,[es:bp+DPB.NEXT_DPB]
33693 00002981 2EA1[3B24]      jmp     short scandpb
33694 00002985 26894619      foundpb:
33695 00002989 2EA1[3D24]      mov     ax,[cs:DevBrkAddr]
33696 0000298D 2689461B      mov     [es:bp+DPB.NEXT_DPB],ax
33697 ;mov ax,[cs:DevBrkAddr+2]
33698 ;mov [es:bp+DPB.NEXT_DPB+2],ax
33699 00002991 2EC42E[3B24]      les     bp,[cs:DevBrkAddr]
33699 00002996 2E8306[3B24]3D add     word [cs:DevBrkAddr],DPBSIZ ; 33

```

```

33700                                     ; 08/09/2023
33701                                     ; (61 in PCDOS 7.1 IBMBIO.COM)
33702 0000299C E8440D      call      RoundBreakAddr
33703
33704 0000299F 26C74619FFFF  mov     word [es:bp+DPB.NEXT_DPB],-1
33705 000029A5 26C64618FF  mov     byte [es:bp+DPB.FIRST_ACCESS],-1
33706
33707 000029AA 8B37          mov     si,[bx]          ;ds:si points to bpb
33708 000029AC 43          inc     bx
33709 000029AD 43          inc     bx          ;point to next guy
33710                                     ;mov     [es:bp+DPB.DRIVE],dx
33711                                     ; 11/12/2022
33712 000029AE 26895600      mov     [es:bp],dx ; 13/05/2019
33713                                     ; 29/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33714                                     ;mov     [es:bp+0],dx          ; [es:bp+DPB.DRIVE]
33715
33716                                     ; 13/04/2024 - Retro DOS v5.0
33717                                     ; PCDOS 7.1 IBMBIO.COM
33718                                     ;;;
33719 000029B2 52          push    dx
33720 000029B3 51          push    cx          ; initialize FAT32 extended DPB parameters/fields
33721 000029B4 BA5241      mov     dx,4152h          ; 'AR' signature for FAT32 extended DPB
33722 000029B7 31C9      xor     cx,cx          ; 0
33723                                     ;mov     [es:bp+1Dh],cx
33724 000029B9 26894E1D      mov     [es:bp+DPB.NEXT_FREE],cx ; last allocated cluster #
33725                                     ;cmp     [si+0Bh],cx          ; BPB.fatsecs16
33726 000029BD 394C0B      cmp     [si+A_BPB.SECTORSPERFAT],cx ; 0
33727 000029C0 7514          jnz     short set_dpb ; FAT DPB (33 bytes) -jnz-
33728                                     ; FAT32 DPB (61 bytes) -jz-
33729
33730 000029C2 26894E39      ;mov     [es:bp+39h],cx
33731 mov     [es:bp+DPB.FAT32_NXTFREE],cx ; 0
33732                                     ;mov     [es:bp+3Bh],cx
33733 000029C6 26894E3B      mov     [es:bp+DPB.FAT32_NXTFREE+2],cx ; 0
33734 000029CA 49          dec     cx          ; 0FFFFh ; -1
33735                                     ;mov     [es:bp+1Fh],cx
33736 000029CB 26894E1F      mov     [es:bp+DPB.FREE_CNT],cx ; -1 = unknown
33737                                     ;mov     [es:bp+21h],cx
33738 000029D3 B95845      mov     [es:bp+DPB.FREE_CNT+2],cx ; -1 = unknown
33739 mov     cx,4558h          ; 'EX' signature for FAT32 extended DPB
33740 set_dpb:
33741                                     ;;;
33742 000029D6 B453          mov     ah,SETDPB ; 53h          ;hidden system call
33743 000029D8 CD21          int     21h
33744                                     ; DOS - 2+ internal - TRANSLATE BIOS PARAMETER BLOCK
33745                                     ; DS:SI -> BPB (BIOS Parameter Block)
33746                                     ; ES:BP -> buffer for DOS Drive Parameter Block
33747                                     ; 13/04/2024
33748                                     ;;;
33749 000029DA 59          pop     cx
33750 000029DB 5A          pop     dx
33751                                     ;;;
33752
33753                                     ;mov     ax,[es:bp+2]
33754 000029DC 268B4602      mov     ax,[es:bp+DPB.SECTOR_SIZE]
33755 000029E0 06          push    es
33756 000029E1 2EC43E[6D02]  les     di,[cs:DOSINFO]          ;es:di point to dos info
33757                                     ;cmp     ax,[es:di+10h]
33758 000029E6 263B4510      cmp     ax,[es:di+SYSI_MAXSEC]
33759 000029EA 07          pop     es
33760                                     ; 13/04/2024
33761                                     ;jna     short iblk_1
33762                                     ;jmp     bad_bpb_size_sector
33763                                     ; 29/10/2022
33764 000029EB 777F          ja     short bad_bpb_size_sector
33765 iblk_1:
33766 000029ED 1E          push    ds
33767 000029EE 52          push    dx
33768
33769 000029EF 2EC516[3724]  lds     dx,[cs:DevEntry]
33770                                     ;mov     [es:bp+13h],dx
33771 000029F4 26895613      mov     [es:bp+DPB.DRIVER_ADDR],dx
33772                                     ;mov     [es:bp+15h],ds
33773 000029F8 268C5E15      mov     [es:bp+DPB.DRIVER_ADDR+2],ds
33774
33775 000029FC 5A          pop     dx
33776 000029FD 1F          pop     ds
33777
33778 000029FE 42          inc     dx
33779 000029FF FEC6          inc     dh
33780                                     ;loop   perunit
33781                                     ; 13/04/2024
33782                                     ;;;
33783 00002A01 49          dec     cx          ; cx = cx - 1
33784                                     ; cx = remain count from [cs:unitcount]
33785 00002A02 7403          jz     short iblk_2          ; cx = 0 -> done
33786 00002A04 E964FF      jmp     perunit          ; loop until cx is 0
33787 iblk_2:
33788                                     ;;;
33789
33790 00002A07 0E          push    cs
33791 00002A08 1F          pop     ds
33792
33793 00002A09 E897E3      call    TempCDS          ; set cds for new drives
33794                                     ; 31/12/2022
33795                                     ; ds <> cs
33796 linkit:
33797 00002A0C 2EC43E[6D02]  les     di,[cs:DOSINFO]          ;es:di = dos table
33798 00002A11 268B4D22      mov     cx,[es:di+SYSI_DEV]      ;dx:cx = head of list
33799 00002A15 268B5524      mov     dx,[es:di+SYSI_DEV+2]
33800
33801 00002A19 2EC536[3724]  lds     si,[cs:DevEntry]          ;ds:si = device location
33802 00002A1E 26897522      mov     [es:di+SYSI_DEV],si      ;set head of list in dos
33803 00002A22 268C5D24      mov     [es:di+SYSI_DEV+2],ds
33804 00002A26 8B04          mov     ax,[si]
33805 00002A28 2EA3[3724]    mov     [cs:DevEntry],ax          ;get pointer to next device
33806                                     ;and save it
33807 00002A2C 890C          mov     [si],cx
33808 00002A2E 895402      mov     [si+2],dx          ;link in the driver
33809 enddev:
33810 00002A31 5E          pop     si
33811 00002A32 07          pop     es
33812 00002A33 40          inc     ax
33813 00002A34 740B          jz     short coeffj3          ;ax = ffff (no more devs if yes)?
33814
33815 00002A36 2EFE06[6619]  inc     byte [cs:multdeviceflag] ; possibly multiple device driver.
33816 00002A3B E8E80C      call    DevBreak          ; M009
33817                                     ; 11/12/2022
33818                                     ; ds = cs (DevBreak)
33819
33820                                     ; 03/04/2019 - Retro DOS v4.0
33821                                     ; MSDOS 6.21 IO.SYS - SYSINIT:290Dh
33822 00002A3E E9EDFD      jmp     goodld          ; otherwise pretend we loaded it in
33823 coeffj3:

```

```

33824 ; 18/12/2022
33825 ; ax = 0
33826 00002A41 2EA2[6619] mov [cs:multdeviceflag],al ; 0
33827 ;mov byte [cs:multdeviceflag],0 ; reset the flag
33828 00002A45 E8DE0C call DevBreak
33829 ; 11/12/2022
33830 ; ds = cs (DevBreak)
33831
33832 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33833 ; (SYSINIT:2919h)
33834 ; 11/07/2023
33835 00002A48 E80204 call CheckProtmanArena
33836
33837 ; 02/11/2022 (MSDOS 5.0 IO.SYS compatibility)
33838 ;;call CheckProtmanArena ; adjust alloclim if Protman$ just
33839 ; ; created a bogus arena to try
33840 ; ; to protect some of its resident-
33841 ; ; init code.
33842 ; 13/04/2024 - Retro DOS v5.0
33843 ; PCDOS 7.1 IBMBIO.COM
33844 ;;call CheckDoubleSpace
33845 ;jmp coff
33846
33847 ;-----
33848
33849 ; 13/04/2024 - Retro DOS v5.0
33850 ; PCDOS 7.1 IBMBIO.COM
33851 ;;
33852
33853 CheckDoubleSpace:
33854
33855 ;; ifdef dblspace_hooks
33856
33857 ; Now check for two special MagicDrv cases:
33858 ;
33859 ; a) the last driver load was MagicDrv final placement:
33860 ; -> add number of MagicDrv reserved drives to drivenumber
33861 ;
33862 ; b) MagicDrv is currently in temporary home:
33863 ; -> call it to give it a chance to mount and shuffle drives
33864 ;
33865 ;cmp byte [cs:MagicHomeFlag],0 ; already home?
33866 ; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
33867 00002A4B 2EF606[6524]01 test byte [cs:MagicHomeFlag],1 ; already home?
33868 00002A51 7545 jnz short no_more_magic_calls ; nothing more to do if so
33869
33870 ; Now inquire of driver whether it is present, and final located
33871
33872 ;mov ax,multMagicdrv ; 4A11h
33873 ;mov bx,MD_VERSION ; 0
33874 ;int 2fh ; ch = number of MagicDrv drive letters
33875 ;or ax,ax ; is it there?
33876 ; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
33877 ;;
33878 00002A53 E800EF call get_dblspace_version ; is it there?
33879 ;jnz short no_more_magic_calls ; done if not
33880 00002A56 750B jnz short set_magic_homeflag
33881 ;;
33882
33883 00002A58 F7C20080 test dx,8000h ; is it final placed?
33884 00002A5C 751C jnz short magic_not_yet_home ; skip if not
33885
33886 ; Okay, now the driver is final placed! Set the flag so we
33887 ; don't keep checking it, and add its number of drive letters
33888 ; to drivenumber.
33889
33890 ; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
33891 ;mov byte [cs:MagicHomeFlag],0ffh ; set the flag!
33892 00002A5E 2E002E[8503] add [cs:drivenumber],ch ; add number of MagicDrv volumes to
33893 ; the drive number we'll pass to the
33894 ; next loadable block device.
33895 ;jmp short no_more_magic_calls ; and finished.
33896
33897 ;;
33898 set_magic_homeflag:
33899 00002A63 2EC606[6524]01 mov byte [cs:MagicHomeFlag],1 ; set the flag!
33900 00002A69 E918FB jmp coff
33901 ;;
33902
33903 ; 03/04/2019 - Retro DOS v4.0
33904
33905 bad_bpb_size_sector:
33906 00002A6C 5E pop si
33907 00002A6D 07 pop es
33908 00002A6E BA[9050] mov dx,badsiz_pre
33909 00002A71 B8[6E50] mov bx,crlfm
33910 00002A74 E8B91F call prnerr
33911
33912 00002A77 E90AFB jmp coff
33913
33914 magic_not_yet_home:
33915 00002A7A 06 push es
33916 00002A7B 56 push si
33917
33918 00002A7C 2E8B0E[6403] mov cx,[cs:memhi] ; pass it a work buffer
33919 00002A81 2E8B16[A502] mov dx,[cs:ALLOCLIM] ; address in cx (segment)
33920 00002A86 29CA sub dx,cx ; for len dx (paragraphs)
33921
33922 00002A88 BB0200 mov bx,2
33923 00002A8B 2EA0[6A19] mov al,[cs:driver_units] ; shuffle magicdrives and new drives
33924 ; by this many units
33925
33926 ;BUGBUG 29-Oct-1992 bens Take this 55h out after Beta 4
33927 00002A8F B455 mov ah,55h ; backdoor won't shuffle unless it
33928 ; sees this, to prevent bad things
33929 ; from happening if people run the
33930 ; new driver with an old BIOS
33931 00002A91 2EFF1E[9003] call far [cs:MagicBackdoor]
33932
33933 00002A96 5E pop si
33934 00002A97 07 pop es
33935
33936 ;no_more_magic_calls:
33937 ;
33938 ;; endif
33939 ; retn
33940
33941 ; 13/04/2024
33942 ;;
33943 no_more_magic_calls:
33944 00002A98 E9E9FA jmp coff
33945 ;;
33946
33947 ;-----

```

```

33948 ; country command
33949 ; the syntax is:
33950 ; country=country id {,codepage {,path}}
33951 ; country=country id {,,path} :default codepage id in dos
33952 ;-----
33953
33954 ; 30/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
33955 ; (SYSINIT:2663h)
33956
33957 00002A9B 80FC51      tryq:      cmp     ah,CONFIG_COUNTRY ; 'Q'
33958 00002A9E 7403      je      short tryq_cont
33959
33960 00002AA0 E90D01      skip_it3:  jmp     tryf
33961 tryq_cont:
33962
33963 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
33964 ; (SYSINIT:297Eh)
33965 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
33966 ;%if 0
33967 ;ifdef MULTI_CONFIG
33968 00002AA3 E8741B      call    query_user      ; query the user if config_cmd
33969 00002AA6 72F8      jc      short skip_it3  ; has the CONFIG_OPTION_QUERY bit set
33970 ;endif
33971 ;%endif ; 02/11/2022
33972
33973 ; 31/12/2022
33974 ;xor     bx,bx
33975 00002AA8 31C9      xor     cx,cx
33976 ; 14/12/2022
33977 ; ds = cs
33978 ; bx = 0
33979 ;mov     byte [cs:centry_drv],0 ; reset the drive,path to default value.
33980 ;mov     word [cs:p_code_page],0
33981 ; 31/12/2022
33982 ; cx = 0
33983 ;mov     [centry_drv],b1 ; 0
33984 ;mov     [p_code_page],bx ; 0
33985 00002AAA 880E[DB4A]  mov     [centry_drv],cl ; 0
33986 00002AAE 890E[7A22]  mov     [p_code_page],cx ; 0
33987
33988 00002AB2 BF[4322]      mov     di,centry_parms
33989 ;xor     cx,cx ; 31/12/2022
33990 ; 03/01/2023
33991 ;mov     dx,cx
33992
33993 00002AB5 E8AD03      do52:    call    sysinit_parse
33994 00002AB8 730B      jnc     short if52      ; parse error,check error code and
33995
33996 00002ABA E8E000      call    centry_error    ; show message and end the search loop.
33997 ; 14/12/2022
33998 ; ds = cs
33999 00002ABD C706[7822]FFFF  mov     word [p_centry_code],-1
34000 ;mov     word [cs:p_centry_code],-1 ; signals that parse error.
34001 00002AC3 EB34      jmp     short sr52
34002
34003 00002AC5 83F8FF      if52:    cmp     ax,_$_RC_EOL ; 0FFFFh; end of line?
34004 00002AC8 742F      jz      short sr52      ; then end the search loop
34005
34006 ;cmp     byte [cs:result_val+$_P_Result_Blk.Type],$_P_number ; numeric?
34007 ; 14/12/2022
34008 ; ds = cs
34009 00002ACA 803E[1522]01  cmp     byte [result_val],$_P_number
34010 ;cmp     byte [cs:result_val],$_P_number
34011 00002ACF 7512      jnz     short if56
34012
34013 ;mov     ax,[cs:rv_dword]
34014 ;mov     ax,[cs:result_val+$_P_Result_Blk.Picked_Val]
34015 ; 14/12/2022
34016 00002AD1 A1[1922]      mov     ax,[result_val+$_P_Result_Blk.Picked_Val]
34017 00002AD4 83F901      cmp     cx,1
34018 00002AD7 7505      jne     short if57
34019
34020 ;mov     [cs:p_centry_code],ax
34021 ; 14/12/2022
34022 00002AD9 A3[7822]      mov     [p_centry_code],ax
34023
34024 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34025 ;jmp     short en57
34026 ; 12/12/2022
34027 ;jmp     short en56
34028 00002ADC EBD7      jmp     short do52
34029
34030 if57:
34031 ;mov     [cs:p_code_page],ax
34032 ; 14/12/2022
34033 ; ds = cs
34034 00002ADE A3[7A22]      mov     [p_code_page],ax
34035
34036 en57:
34037 ;jmp     short en56      ; path entered
34038 ; 12/12/2022
34039 00002AE1 EBD2      jmp     short do52
34040
34041 if56:
34042 push     ds
34043 push     es
34044 push     si
34045 push     di
34046
34047 push     cs
34048 pop      es
34049 ;lds     si,[cs:rv_dword] ; move the path to known place.
34050 ; 14/12/2022
34051 00002AE9 C536[1922]  lds     si,[rv_dword]
34052 00002AED BF[DB4A]  mov     di,centry_drv
34053 00002AF0 E82C1F      call    move_asciiz
34054
34055 pop      di
34056 pop      si
34057 pop      es
34058 pop      ds
34059
34060 en56:
34061 jmp     short do52
34062
34063 sr52:
34064 ; 14/12/2022
34065 ; ds = cs
34066 00002AF9 833E[7822]FF  cmp     word [p_centry_code],-1
34067 ;cmp     word [cs:p_centry_code],-1 ; had a parse error?
34068 ;jne     short tryq_open
34069 ;mov     dx,badcountry
34070 00002B00 E981FA      jmp     coff
34071
34072 tryqbad:
34073 ;mov     dx,badcountry
34074 ;jmp     tryqchkerr
34075
34076 ;"invalid country code or code page"

```

```

34072      tryq_open:
34073      ; 14/12/2022
34074      ; ds = cs
34075      cmp     byte [centry_drv],0
34076      ;cmp     byte [cs:centry_drv],0
34077      je      short tryq_def
34078      mov     dx,centry_drv
34079      jmp     short tryq_openit
34080
34081      tryq_def:
34082      mov     dx,centry_root
34083      tryq_openit:
34084      mov     ax,3D00h          ;open a file
34085      stc
34086      int     21h
34087      jc      short tryqfilebad  ;open failure
34088
34089      ; 14/12/2022
34090      ; ds = cs
34091      mov     [centryfilehandle],ax
34092      ;mov     [cs:centryfilehandle],ax      ;save file handle
34093      mov     bx,ax
34094      mov     ax,[p_centry_code]
34095      mov     dx,[p_code_page]
34096      ;mov     ax,[cs:p_centry_code]
34097      ;mov     dx,[cs:p_code_page]      ;now,ax=country id,bx=filehandle
34098      ;mov     cx,[cs:memhi]
34099      mov     cx,[memhi]
34100      add     cx,384            ;need 6k buffer to handle country.sys
34101      ;M023
34102
34103      ; 14/12/2022
34104      ; ds = cs
34105      cmp     cx,[ALLOCLIM]
34106      ;cmp     cx,[cs:ALLOCLIM]
34107      ja      short tryqmemory    ;cannot allocate the buffer for country.sys
34108      mov     si,centry_drv      ;ds:si -> centry_drv
34109      cmp     byte [si],0        ;default path?
34110      jne     short tryq_set_for_dos
34111
34112      inc     si
34113      inc     si                ;ds:si -> centry_root
34114
34115      tryq_set_for_dos:
34116      ; 14/12/2022
34117      ; ds = cs
34118      les     di,[sysi_country]
34119      ;les     di,[cs:sysi_country]      ;es:di -> country info tab in dos
34120      push    di                ;save di
34121      ;add     di,8
34122      add     di,country_cdpj_info.ccPath_CountrySys ; 8
34123      call    move_asciiz        ;set the path to country.sys in dos.
34124      pop     di                ;es:di -> country info tab again.
34125
34126      ; 14/12/2022
34127      mov     cx,[memhi]
34128      ;mov     cx,[cs:memhi]
34129      mov     ds,cx
34130      xor     si,si
34131      call    setdoscountryinfo  ;ds:si -> 2k buffer to be used.
34132      ; ds <> cs ; 14/12/2022      ;now do the job!!!
34133      jnc     short tryqchkerr   ;read error or could not find country,code page combination
34134
34135      cmp     cx,-1
34136      je      short tryqbad      ;could not find matching country_id,code page?
34137      ;then "invalid country code or code page"
34138
34139      tryqfilebad:
34140      push    cs
34141      pop     es
34142      cmp     byte [cs:centry_drv],0 ;is the default file used?
34143      je      short tryqdefbad
34144      mov     si,centry_drv
34145      jmp     short tryqbadload
34146
34147      tryqdefbad:
34148      mov     si,centry_root      ;default file has been used.
34149      ;es:si -> \country.sys in sysinit_seg
34150      tryqbadload:
34151      call    badload             ;ds will be restored to sysinit_seg
34152      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34153      ; (SYSINIT:2A69h)
34154      mov     cx,[CONFBOT] ; ds = cs (from badload)
34155      ;mov     cx,[cs:CONFBOT]
34156      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34157      ;mov     cx,[cs:top_of_cdss]
34158      ; 11/12/2022
34159      ; ds = cs
34160      ;mov     cx,[top_of_cdss] ; mov cx,[CONFBOT]
34161      mov     es,cx              ;restore es -> confbot.
34162      jmp     short coeffj4
34163
34164      tryqmemory:
34165      mov     dx,insufmemory
34166      tryqchkerr:
34167      ;mov     cx,[cs:CONFBOT]
34168      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34169      ;mov     cx,[cs:top_of_cdss]
34170      ; 12/12/2022
34171      push    cs
34172      pop     ds
34173      ; 31/12/2022 - Retro DOS v4.2
34174      mov     cx,[CONFBOT] ; (MSDOS 6.21 IO.SYS, SYSINIT)
34175      ;mov     cx,[top_of_cdss] ; mov cx,[CONFBOT]
34176      mov     es,cx              ;restore es -> confbot seg
34177      ;push    cs
34178      ;pop     ds                ;restore ds to sysinit_seg
34179      jnc     short coeffj4      ;if no error,then exit
34180      call    print              ;else show error message
34181      call    error_line
34182      coeffj4:
34183      ;mov     bx,[cs:centryfilehandle]
34184      ; 11/12/2022
34185      ; ds = cs
34186      mov     bx,[centryfilehandle]
34187      mov     ah,3Eh
34188      int     21h                ;close a file. don't care even if it fails.
34189      jmp     coeff
34190
34191      ;-----
34192      centry_error:
34193      ;function: show "invalid country code or code page" messages,or

```



```

34196 ; "error in country command" depending on the error code
34197 ; in ax returned by sysparse;
34198 ;in:ax - error code
34199 ; ds - sysinitseg
34200 ; es - confbot
34201 ;out: show message. dx destroyed.
34202
34203 00002B9D 83F806 cmp ax,_$P_Out_Of_Range ; 6
34204 00002BA0 7505 jne short if64
34205 00002BA2 BA[D750] mov dx,badcountry ;"invalid country code or code page"
34206 00002BA5 EB03 jmp short en64
34207 if64:
34208 00002BA7 BA[FD50] mov dx,badcountrycom ;"error in contry command"
34209 en64:
34210 00002BAA E8A51E call print
34211 ;call error_line
34212 ;retn
34213 ; 11/12/2022
34214 00002BAD E90803 jmp error_line
34215
34216 ;-----
34217 ; files command
34218 ;-----
34219
34220 ;*****
34221 ; function: parse the parameters of files= command. *
34222 ; *
34223 ; input : *
34224 ; es:si -> parameters in command line. *
34225 ; output: *
34226 ; variable files set. *
34227 ; *
34228 ; subroutines to be called: *
34229 ; sysinit_parse *
34230 ; logic: *
34231 ; { *
34232 ; set di points to files_parms; *
34233 ; set dx,cx to 0; *
34234 ; while (end of command line) *
34235 ; { sysinit_parse; *
34236 ; if (no error) then *
34237 ; files = result_val._$P_picked_val *
34238 ; else *
34239 ; error exit; *
34240 ; }; *
34241 ; }; *
34242 ; *
34243 ;*****
34244
34245 tryf:
34246 00002BB0 80FC46 cmp ah,CONFIG_FILES ; 'F'
34247 00002BB3 7528 jne short tryl
34248
34249 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34250 ; (SYSINIT:2AABh)
34251 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34252 ;%if 0
34253 ;ifdef MULTI_CONFIG
34254 call query_user ; query the user if config_cmd
34255 00002BB8 7223 jc short tryl ; has the CONFIG_OPTION_QUERY bit set
34256 ;endif
34257 ;%endif ; 30/10/2022
34258
34259 ; 14/12/2022
34260 ; ds = cs
34261
34262 00002BBA BF[7C22] mov di,files_parms
34263 00002BBD 31C9 xor cx,cx
34264 ; 03/01/2023
34265 ;mov dx,cx
34266 do67:
34267 call sysinit_parse
34268 00002BC2 7303 jnc short if67 ; parse error
34269 ;call badparm_p ; and show messages and end the search loop.
34270 ;jmp short sr67
34271 ; 03/01/2023
34272 00002BC4 E98D01 jmp badparm_p_coff
34273 if67:
34274 00002BC7 83F8FF cmp ax,_$P_RC_EOL ; end of line?
34275 00002BCA 7408 je short en67 ; then end the $endloop
34276
34277 ; 14/12/2022
34278 ; ds = cs
34279 ;;mov al,[cs:rv_dword]
34280 ;mov al,[cs:result_val+_$P_Result_Blk.Picked_Val]
34281 ;mov [cs:p_files],al ; save it temporarily
34282 ;mov al,[rv_dword]
34283 00002BCC A0[1922] mov al,[result_val+_$P_Result_Blk.Picked_Val]
34284 00002BCF A2[9B22] mov [p_files],al
34285
34286 00002BD2 EBEB jmp short do67
34287 en67:
34288 ; 14/12/2022
34289 ; ds = cs
34290 00002BD4 A0[9B22] mov al,[p_files]
34291 00002BD7 A2[9F02] mov [FILES],al
34292 ;mov al,[cs:p_files]
34293 ;mov [cs:FILES],al ; no error. really set the value now.
34294 sr67:
34295 00002BDA E9A7F9 jmp coff
34296
34297 ; 04/04/2019 - Retro DOS v4.0
34298
34299 ;-----
34300 ; lastdrive command
34301 ;-----
34302
34303 ;*****
34304 ; function: parse the parameters of lastdrive= command. *
34305 ; *
34306 ; input : *
34307 ; es:si -> parameters in command line. *
34308 ; output: *
34309 ; set the variable num_cds. *
34310 ; *
34311 ; subroutines to be called: *
34312 ; sysinit_parse *
34313 ; logic: *
34314 ; { *
34315 ; set di points to ldrv_parms; *
34316 ; set dx,cx to 0; *
34317 ; while (end of command line) *
34318 ; { sysinit_parse; *
34319 ; if (no error) then *

```

```

34320 ; set num_cds to the returned value; *
34321 ; else /*error exit*/ *
34322 ; error exit; *
34323 ; }; *
34324 ; }; *
34325 ; }; *
34326 ; *****
34327
34328
34329 00002BDD 80FC4C tryl: cmp ah,CONFIG_LASTDRIVE ; 'L'
34330 00002BE0 7528 jne short tryp
34331
34332 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34333 ; (SYSINIT:2AE0h)
34334 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34335 ;%if 0
34336 00002BE2 E8351A call query_user ; query the user if config_cmd
34337 00002BE5 7223 jc short tryp ; has the CONFIG_OPTION_QUERY bit set
34338 ;endif
34339 ;%endif ; 30/10/2022
34340
34341 ; 14/12/2022
34342 ; ds = cs
34343
34344 00002BE7 BF[D322] mov di,ldrv_parms
34345 00002BEA 31C9 xor cx,cx
34346 ; 03/01/2023
34347 ;mov dx,cx
34348
34349 00002BEC E87602 do73: call sysinit_parse
34350 00002BEF 7303 jnc short if73 ; parse error
34351 ;call badparm_p ; and show messages and end the search loop.
34352 ;jmp short sr73
34353 ; 03/01/2023
34354 00002BF1 E96001 jmp badparm_p_coff
34355
34356 00002BF4 83F8FF if73: cmp ax,_P_RC_EOL ; end of line?
34357 00002BF7 7408 je short en73 ; then end the $endloop
34358
34359 ; 14/12/2022
34360 ; ds = cs
34361 ;mov al,[cs:rv_dword]
34362 ;mov al,[cs:rv_byte] ; pick up the drive number
34363 ;mov [cs:p_ldrv],al ; save it temporarily
34364
34365 ;mov al,[rv_dword]
34366 00002BF9 A0[1922] mov al,[rv_byte]
34367 00002BFC A2[E722] mov [p_ldrv],al
34368
34369 00002BFF EBEB jmp short do73
34370
34371 en73: ; 14/12/2022
34372 ; ds = cs
34373 00002C01 A0[E722] mov al,[p_ldrv]
34374 00002C04 A2[A202] mov [NUM_CDS],al
34375 ;mov al,[cs:p_ldrv]
34376 ;mov [cs:NUM_CDS],al ; no error. really set the value now.
34377
34378 00002C07 E97AF9 sr73: jmp coff
34379
34380 ;-----
34381 ; setting drive parameters
34382 ;-----
34383
34384
34385 00002C0A 80FC50 tryp: cmp ah,CONFIG_DRIVPARM ; 'P'
34386 00002C0D 7516 jne short tryk
34387
34388 ; 31/12/2022 - Retro DOS v4.2
34389 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34390 ;%if 0
34391 ;ifdef MULTI_CONFIG
34392 00002C0F E8081A call query_user ; query the user if config_cmd
34393 00002C12 7211 jc short tryk ; has the CONFIG_OPTION_QUERY bit set
34394 ;endif
34395 ;%endif ; 30/10/2022
34396
34397 00002C14 E8EC0E call parseline
34398 00002C17 7209 jc short trybad
34399 00002C19 E8050E call setparms
34400 00002C1C E8470E call diddleback
34401
34402 ; No error check here, because setparms and diddleback have no error
34403 ; returns, and setparms as coded now can return with carry set.
34404 ; jc short trybad
34405
34406 ; 12/12/2022
34407 ; cf = 0
34408 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34409 ;jc short trybad
34410
34411 00002C1F E962F9 jmp coff
34412
34413 00002C22 E95F02 trybad: jmp badop
34414
34415 ;-----
34416 ; setting internal stack parameters
34417 ; stacks=m,n where
34418 ; m is the number of stacks (range 8 to 64,default 9)
34419 ; n is the stack size (range 32 to 512 bytes,default 128)
34420 ; j.k. 5/5/86: stacks=0,0 implies no stack installation.
34421 ; any combinations that are not within the specified limits will
34422 ; result in "unrecognized command" error.
34423 ;-----
34424
34425 ;*****
34426 ;
34427 ; function: parse the parameters of stacks= command. *
34428 ; the minimum value for "number of stacks" and "stack size" is *
34429 ; 8 and 32 each. in the definition of sysparse value list,they *
34430 ; are set to 0. this is for accepting the exceptional case of *
34431 ; stacks=0,0 case (,which means do not install the stack.) *
34432 ; so,after sysparse is done,we have to check if the entered *
34433 ; values (stack_count,stack_size) are within the actual range, *
34434 ; (or if "0,0" pair has been entered.) *
34435 ; input : *
34436 ; es:si -> parameters in command line. *
34437 ; output: *
34438 ; set the variables stack_count,stack_size. *
34439 ; *
34440 ; subroutines to be called: *
34441 ; sysinit_parse *
34442 ; logic: *
34443 ; {

```



```

34444 ; set di points to stks_parms; *
34445 ; set dx,cx to 0; *
34446 ; while (end of command line) *
34447 { sysinit_parse; *
34448 ; if (no error) then *
34449 ; { if (cx == 1) then /* first positional = stack count */ *
34450 ; p_stack_count = result_val._$P_picked_val; *
34451 ; if (cx == 2) then /* second positional = stack size */ *
34452 ; p_stack_size = result_val._$P_picked_val; *
34453 ; } *
34454 ; else /*error exit*/ *
34455 ; error exit; *
34456 ; } *
34457 ; here check p_stack_count,p_stack_size if it meets the condition; *
34458 ; if o.k.,then set stack_count,stack_size; *
34459 ; else error_exit; *
34460 ; } *
34461 ; *****
34462
34463 tryk:
34464 ;if stacksw
34465 ;cmp ah,CONFIG_STACKS ; 'K'
34466 00002C25 80FC4B je short do_tryk
34467 00002C28 7402 skip_it4:
34468 jmp short trys ; 15/12/2022
34469 00002C2A EB79 do_tryk:
34470
34471 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34472 ; (SYSINIT:2B33h)
34473 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34474 ;%if 0
34475 ;ifdef MULTI_CONFIG
34476 00002C2C E8EB19 call query_user ; query the user if config_cmd
34477 00002C2F 72F9 jc short skip_it4 ; has the CONFIG_OPTION_QUERY bit set
34478 ;endif
34479 ;%endif ; 30/10/2022
34480
34481 ; 14/12/2022
34482 ; ds = cs
34483
34484 00002C31 BF[E822] mov di,stks_parms
34485 00002C34 31C9 xor cx,cx
34486 ; 03/01/2023
34487 ;mov dx,cx
34488
34489 do79:
34490 00002C36 E82C02 call sysinit_parse
34491 00002C39 730B jnc short if79 ; parse error
34492
34493 00002C3B BA[8951] mov dx,badstack ; "invalid stack parameter"
34494 00002C3E E8111E call print ; and show messages and end the search loop.
34495 00002C41 E87402 call error_line
34496 ;jmp sr79
34497 ; 11/12/2022
34498 00002C44 EB39 jmp short sr79
34499
34500 00002C46 83F8FF if79:
34501 00002C49 7412 cmp ax,_$P_RC_EOL ; end of line?
34502 je short en79 ; then end the $endloop
34503
34504 ; 14/12/2022
34505 ; ds = cs
34506
34507 ;mov ax,[cs:rv_dword]
34508 ;mov ax,[cs:result_val+_$P_Result_Blk.Picked_Val]
34509 00002C4B A1[1922] mov ax,[rv_dword]
34510 mov ax,[result_val+_$P_Result_Blk.Picked_Val]
34511 00002C4E 83F901 cmp cx,1
34512 00002C51 7505 jne short if83
34513
34514 ; 14/12/2022
34515 ;mov [cs:p_stack_count],ax
34516 ;jmp short en83
34517 00002C53 A3[1D23] mov [p_stack_count],ax
34518 00002C56 EBDE jmp short do79
34519
34520 if83:
34521 ; 14/12/2022
34522 00002C58 A3[1F23] mov [cs:p_stack_size],ax
34523 mov [p_stack_size],ax
34524 00002C5B EBD9 en83:
34525 jmp short do79
34526
34527 en79:
34528 ; 14/12/2022
34529 00002C5D A1[1D23] ; ds = cs
34530 00002C60 09C0 mov ax,[p_stack_count]
34531 00002C62 741E or ax,ax
34532 jz short if87
34533
34534 ; 14/12/2022
34535 ;cmp word [p_stack_count],0
34536 ;cmp word [cs:p_stack_count],0
34537 ;je short if87
34538
34539 ; 14/12/2022
34540 00002C64 83F808 cmp ax,mincount ; 8
34541 cmp word [cs:p_stack_count],mincount ; 8
34542 ; 15/12/2022
34543 00002C67 721F jb short en87
34544 00002C69 833E[1F23]20 cmp word [p_stack_size],minsize ; 32
34545 cmp word [cs:p_stack_size],minsize ; 32
34546 ; 15/12/2022
34547 00002C6E 7218 jb short en87
34548
34549 if94:
34550 ; 14/12/2022
34551 ; ds = cs
34552 ; ax = [p_stack_count]
34553 ;mov ax,[p_stack_count]
34554 ;mov ax,[cs:p_stack_count]
34555 00002C70 A3[8C02] mov [stack_count],ax
34556 mov [cs:stack_count],ax
34557 00002C73 A1[1F23] mov ax,[cs:p_stack_size]
34558 mov ax,[p_stack_size]
34559 00002C76 A3[8E02] mov [cs:stack_size],ax
34560 mov [stack_size],ax
34561 00002C79 C706[9002]FFFF mov word [cs:stack_addr],-1 ; stacks= been accepted.
34562 mov word [stack_addr],-1
34563 sr79:
34564 jmp coff
34565
34566 if87:
34567 ; 14/12/2022
34568 00002C82 3906[1F23] cmp [p_stack_size],ax ; 0
34569 00002C86 74E8 je short if94 ; ax = [p_stack_count] = 0
34570 cmp word [cs:p_stack_size],0

```

```

34568             ;je      short if94
34569 en87:
34570             ; 15/12/2022
34571             ; ([p_stack_count] is invalid, use default values)
34572             ; 14/12/2022
34573             ; ds = cs
34574 00002C88 C706[8C02]0900 mov     word [stack_count],defaultcount ; 9
34575 00002C8E C706[8E02]8000 mov     word [stack_size],defaultsize ; 128
34576 00002C94 C706[9002]0000 mov     word [stack_addr],0
34577             ;mov     word [cs:stack_count],defaultcount ; 9
34578             ;
34579             ;mov     word [cs:stack_size],defaultsize ; 128
34580             ;mov     word [cs:stack_addr],0
34581
34582 00002C9A BA[8951] mov     dx,badstack
34583 00002C9D E8B21D call    print
34584 00002CA0 E81502 call    error_line
34585 00002CA3 EBDA jmp     short sr79
34586
34587             ; 15/12/2022
34588 %if 0
34589             mov     di,stks_parms
34590             xor     cx,cx
34591             ; 03/01/2023
34592             ;mov     dx,cx
34593 do79:
34594             call    sysinit_parse
34595             jnc     short if79             ; parse error
34596
34597             mov     dx,badstack             ; "invalid stack parameter"
34598             call    print             ; and show messages and end the search loop.
34599             call    error_line
34600             ;jmp     sr79
34601             ; 11/12/2022
34602             jmp     short sr79
34603 if79:
34604             cmp     ax,_$P_RC_EOL             ; end of line?
34605             je      short en79             ; then end the $endloop
34606
34607             ;mov     ax,[cs:rv_dword]
34608             mov     ax,[cs:result_val+_$P_Result_Blk.Picked_Val]
34609             cmp     cx,1
34610             jne     short if83
34611
34612             mov     [cs:p_stack_count],ax
34613             jmp     short en83
34614 if83:
34615             mov     [cs:p_stack_size],ax
34616 en83:
34617             jmp     short do79
34618 en79:
34619             cmp     word [cs:p_stack_count],0
34620             je      short if87
34621
34622             cmp     word [cs:p_stack_count],mincount ; 8
34623             jnb     short 1188
34624             cmp     word [cs:p_stack_size],minsize ; 32
34625             jnb     short if88
34626 1188:
34627             mov     word [cs:p_stack_count],-1 ; invalid
34628 if88:
34629             jmp     short en87
34630
34631             ; 11/12/2022
34632 if94:
34633             mov     ax,[cs:p_stack_count]
34634             mov     [cs:stack_count],ax
34635             mov     ax,[cs:p_stack_size]
34636             mov     [cs:stack_size],ax
34637             mov     word [cs:stack_addr],-1             ; stacks= been accepted.
34638 sr79:
34639             jmp     coff
34640
34641 if87:
34642             cmp     word [cs:p_stack_size],0
34643             je      short en87
34644             mov     word [cs:p_stack_count],-1 ; invalid
34645 en87:
34646             cmp     word [cs:p_stack_count],-1 ; invalid?
34647             jne     short if94
34648
34649             mov     word [cs:stack_count],defaultcount ; 9
34650             ;reset to default value.
34651             mov     word [cs:stack_size],defaultsize ; 128
34652             mov     word [cs:stack_addr],0
34653
34654             mov     dx,badstack
34655             call    print
34656             call    error_line
34657             jmp     short sr79
34658
34659 %endif
34660
34661             ; 11/12/2022
34662 %if 0
34663 if94:
34664             mov     ax,[cs:p_stack_count]
34665             mov     [cs:stack_count],ax
34666             mov     ax,[cs:p_stack_size]
34667             mov     [cs:stack_size],ax
34668             mov     word [cs:stack_addr],-1             ; stacks= been accepted.
34669 sr79:
34670             jmp     coff
34671 %endif
34672             ;endif
34673
34674             ;-----
34675             ; shell command
34676             ;-----
34677
34678 trys:
34679             cmp     ah,CONFIG_SHELL ; 'S'
34680             jne     short tryx
34681
34682             ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34683             ; (SYSINIT:2BE1h)
34684             ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34685             ;%if 0
34686             ;ifdef MULTI_CONFIG
34687             call    query_user             ; query the user if config_cmd
34688             jc      short tryx             ; has the CONFIG_OPTION_QUERY bit set
34689             ; 14/04/2024
34690             ; ds = cs
34691             ;mov     byte [cs:newcmd],1

```

```

34692 00002CAF C606[284B]01      mov     byte [newcmd],1
34693                          ;endif
34694                          ;%endif ; 30/10/2022
34695
34696                          ;;mov word [cs:command_line],0 ; zap length,first byte of command-line
34697                          ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34698                          ;mov byte [cs:command_line+1],0
34699                          ; 15/12/2022
34700                          ; ds = cs
34701                          ; 08/09/2023
34702                          ;mov byte [command_line+1],0
34703 00002CB4 C706[B94B]0000      mov     word [command_line],0 ; zap length,first byte of command-line
34704
34705 00002CBA BF[2C4B]           mov     di,commnd+1          ; we already have the first char
34706 00002CBD 8845FF           mov     [di-1],al          ; of the new shell in AL, save it now
34707
34708 00002CC0 E8EA1A           storeshell:
34709 00002CC3 08C0             call    getchr
34710 00002CC5 741C             or      al,al          ; this is the normal case: "organize"
34711                          ;jz short getshparms          ; put a ZERO right after the filename
34712 00002CC7 3C20             cmp     al," "          ; this may happen if there are no args
34713 00002CC9 7209             jb     short endofshell ; I suppose...
34714 00002CCB 8805           mov     [di],al
34715 00002CCD 47             inc     di
34716                          ;cmp di,commnd+63          ; this makes sure we don't overflow
34717                          ;jb short storeshell          ; commnd (the filename)
34718                          ;jmp short endofshell
34719                          ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34720                          ;jmp short storeshell
34721                          ; 03/01/2023
34722 00002CCE 81FF[6A4B]       cmp     di,commnd+63          ; this makes sure we don't overflow
34723 00002CD2 72EC             jb     short storeshell ; commnd (the filename)
34724                          ;jmp short endofshell
34725
34726                          ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34727                          ;getshparms:
34728                          ; mov     byte [di],0          ; zero-terminate the filename
34729                          ; mov     di,command_line+1    ; prepare to process the command-line
34730
34731                          ;parmloop:
34732                          ; call    getchr
34733                          ; cmp     al," "
34734                          ; jb     short endofparms
34735                          ; mov     [di],al
34736                          ; inc     di
34737                          ; cmp     di,command_line+126
34738                          ; jb     short parmloop
34739                          ;endofparms:
34740                          ; mov     cx,di
34741                          ; sub     cx,command_line+1
34742                          ; mov     [cs:command_line],cx
34743
34744                          ;endofshell:
34745                          ; mov     byte [di],0          ; zero-terminate the filename (or
34746                          ;                          ; the command-line as the case may be)
34747
34748                          ;skipline:
34749                          ; cmp     al,lf          ; 0Ah          ; the safest way to eat the rest of
34750                          ; je     short endofline    ; the line: watch for ever-present LF
34751                          ;call    getchr
34752                          ; jnc     short skipline    ; keep it up as long as there are chars
34753
34754                          ;endofline:
34755                          ; jmp     conflp
34756
34757                          ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34758 00002CD4 C60500          endofshell:
34759                          mov     byte [di],0          ; zero-terminate the filename (or
34760                          ;                          ; the command-line as the case may be)
34761                          ; 11/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
34762                          ; MSDOS 6.21 IO.SYS - SYSINIT:2C33h
34763                          ; PCDOS 7.1 IBMBIO.COM - SYSINIT:314Eh
34764                          ;call    getchr
34765 00002CD7 3C0A           skipline:
34766 00002CD9 7405           cmp     al,lf          ; 0Ah          ; the safest way to eat the rest of
34767 00002CDB E8CF1A           je     short endofline    ; the line: watch for ever-present LF
34768                          ;call    getchr
34769                          ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.1 IO.SYS)
34770                          ; (SYSINIT:2C3Ah)
34771                          ;jnb short skipline
34772
34773 00002CE0 E94AF8          endofline:
34774                          jmp     conflp
34775
34776                          ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34777                          ;getshparms:
34778                          ; 18/12/2022
34779                          ; al = 0
34780                          ;mov     [di],al ; 0
34781 00002CE5 BF[BA4B]       ;mov     byte [di],0          ; zero-terminate the filename
34782                          ;mov     di,command_line+1    ; prepare to process the command-line
34783 00002CE8 E8C21A           parmloop:
34784 00002CEB 3C20             call    getchr
34785                          ; cmp     al," " ; 20h
34786                          ; jb     short endofshell
34787                          ; 03/01/2023
34788                          ; jb     short endofparms
34789                          ;
34790 00002CEF 8805           mov     [di],al
34791 00002CF1 47             inc     di
34792                          ; jmp     short parmloop
34793 00002CF2 81FF[374C]       ; 03/01/2023 - Retro DOS v4.2
34794 00002CF6 72F0             cmp     di,command_line+126
34795                          ; jb     short parmloop
34796
34797                          ; 03/01/2023 - Retro DOS v4.2
34798 00002CF8 89F9           endofparms:
34799 00002CFA 81E9[BA4B]       mov     cx,di
34800                          sub     cx,command_line+1
34801                          ;mov     [cs:command_line],cx
34802                          ; 03/01/2023
34803 00002CFE 880E[B94B]       mov     [command_line],cx
34804 00002D02 EBD0             jmp     short endofshell
34805
34806                          ;-----
34807                          ; fcbs command
34808                          ;-----
34809
34810                          ;*****
34811                          ; function: parse the parameters of fcbs= command.          *
34812                          ;
34813                          ; input :          *
34814                          ; es:si -> parameters in command line.          *
34815                          ; output:         *
34816                          ; set the variables fcbs,keep.          *

```

```

34816 ;
34817 ; subroutines to be called:
34818 ; sysinit_parse
34819 ; logic:
34820 ; {
34821 ;     set di points to fcbs_parms;
34822 ;     set dx,cx to 0;
34823 ;     while (end of command line)
34824 ;     { sysparse;
34825 ;         if (no error) then
34826 ;         { if (cx == 1) then /* first positional = fcbs */
34827 ;             fcbs = result_val._$P_picked_val;
34828 ;             if (cx == 2) then /* second positional = keep */
34829 ;                 keep = result_val._$P_picked_val;
34830 ;             }
34831 ;         else /*error exit*/
34832 ;             error exit;
34833 ;         };
34834 ;     };
34835 ; }
34836 ;*****
34837
34838 00002D04 80FC58
34839 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34840 00002D07 7534
34841 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34842 ;jne short try ; comment command
34843
34844 ; 31/12/2022 - Retro DOS v4.2
34845 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34846 ;%if 0
34847 ;ifdef MULTI_CONFIG
34848 call query_user ; query the user if config_cmd
34849 00002D0C 722F jnc short try1 ; has the CONFIG_OPTION_QUERY bit set
34850 ;endif
34851 ;%endif ; 30/10/2022
34852
34853 00002D0E BF9C22 mov di,fcbs_parms
34854 00002D11 31C9 xor cx,cx
34855 ; 03/01/2023
34856 ;mov dx,cx
34857 do98:
34858 call sysinit_parse
34859 ; 03/01/2023
34860 ;jnc short if98 ; parse error
34861 ;call badparm_p ; and show messages and end the search loop.
34862 ;jmp short sr98
34863 ;-----
34864 ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34865 00002D16 723C jc short badparm_p_coff
34866 if98:
34867 cmp ax,_$P_RC_EOL ; end of line?
34868 00002D1B 7412 je short en98 ; then end the $endloop
34869
34870 ;mov al,[cs:rv_dword]
34871 ;mov al,[cs:result_val+_$P_Result_Blk.Picked_val]
34872 ; 15/12/2022
34873 ; ds = cs
34874 mov al,[result_val+_$P_Result_Blk.Picked_val]
34875 00002D20 83F901 cmp cx,1
34876 00002D23 7505 jne short if102 ; the first positional?
34877 ;mov [cs:p_fcbs],al
34878 ; 15/12/2022
34879 00002D25 A2D122 mov [p_fcbs],al
34880 ;jmp short en102
34881 00002D28 EBE9 jmp short do98
34882 if102:
34883 ;mov [cs:p_keep],al
34884 ; 15/12/2022
34885 00002D2A A2D222 mov [p_keep],al
34886 en102:
34887 jmp short do98
34888 en98:
34889 ; 15/12/2022
34890 ; ds = cs
34891 00002D2F A0D122 mov al,[p_fcbs]
34892 00002D32 A2A002 mov [FCBS],al
34893 00002D35 C606A10200 mov byte [KEEP],0
34894 ;mov al,[cs:p_fcbs] ; M017
34895 ;mov [cs:FCBS],al ; M017
34896 ;mov byte [cs:KEEP],0 ; M017
34897 sr98:
34898 00002D3A E947F8 jmp coff
34899
34900 ; 31/12/2022 - Retro DOS v4.2
34901 ;%if 0
34902
34903 ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34904 ;-----
34905 ; comment= do nothing. just decrease chrptr,and increase count for correct
34906 ; line number
34907 ;-----
34908
34909 tryy:
34910 cmp ah,CONFIG_COMMENT ; 'Y'
34911 jne short try0
34912
34913 donothing:
34914 ; 15/12/2022
34915 ; ds = cs
34916 dec word [chrptr]
34917 inc word [count]
34918 ; 02/11/2022
34919 ;dec word [cs:chrptr]
34920 ;inc word [cs:count]
34921
34922 jmp coff
34923
34924 ;-----
34925 ; rem command
34926 ;-----
34927
34928 try0:
34929 cmp ah,CONFIG_REM ; '0' ; do nothing with this line.
34930 je short donothing
34931
34932 %endif
34933
34934 ; 07/04/2019 - Retro DOS v4.0
34935 ;-----
34936 ; switches command
34937 ;-----
34938
34939

```

```

34940 ;*****
34941 ;
34942 ; function: parse the option switches specified.
34943 ; note - this command is intended for the future use also.
34944 ; when we need to set system data flag,use this command.
34945 ;
34946 ; input :
34947 ; es:si -> parameters in command line.
34948 ; output:
34949 ; p_swit_k set if /k option chosen.
34950 ;
34951 ; subroutines to be called:
34952 ; sysinit_parse
34953 ; logic:
34954 ; {
34955 ;     set di points to swit_parms; /*parse control definition*/
34956 ;     set dx,cx to 0;
34957 ;     while (end of command line)
34958 ;     { sysinit_parse;
34959 ;       if (no error) then
34960 ;       { if (result_val._$P_synonym_ptr == swit_k) then
34961 ;         p_swit_k = 1
34962 ;       }
34963 ;       else {show error message;error exit}
34964 ;     }
34965 ; };
34966 ;
34967 ;*****
34968
34969 SUPPRESS_WINA20 EQU 00000010b ; M025 ; (DOSSYM.INC, MSDOS 6.0)
34970
34971 try1:
34972     cmp     ah,CONFIG_SWITCHES ; '1'
34973     je      short do_try1 ; switches= command entered?
34974 skip_it5:
34975     ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
34976     ; (SYSINIT:2C8Ah)
34977     jmp     tryv
34978     ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34979     jmp     tryz
34980
34981 do_try1:
34982
34983     ; 31/12/2022 - Retro DOS v4.2
34984     ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
34985     ;%if 0
34986     ;ifdef MULTI_CONFIG
34987     call    query_user ; query the user if config_cmd
34988     jc      short skip_it5 ; has the CONFIG_OPTION_QUERY bit set
34989     ;endif
34990     ;%endif ; 30/10/2022
34991
34992     mov     di,swit_parms
34993     xor     cx,cx
34994     ; 03/01/2023
34995     mov     dx,cx
34996 do110:
34997     call    sysinit_parse
34998     jnc     short if110 ; parse error
34999     call    badparm_p ; and show messages and end the search loop.
35000     jmp     short sr110
35001     ;-----
35002     ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35003 badparm_p_coff:
35004     call    badparm_p
35005     jmp     coff
35006     ;-----
35007 if110:
35008     cmp     ax,_$P_RC_EOL ; end of line?
35009     je      short en110 ; then jmp to $endloop for semantic check
35010
35011     ; 15/12/2022
35012     ds = cs
35013     ;cmp word [cs:result_val_swoff],swit_k
35014     ;cmp word [cs:result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_k
35015     cmp     word [result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_k
35016     jne     short if115 ; M059
35017     ; 15/12/2022
35018     mov     byte [p_swit_k],1
35019     mov     byte [cs:p_swit_k],1 ; set the flag
35020     jmp     short do110
35021 if115:
35022     ; 15/12/2022 ;M059
35023     ;cmp word [cs:result_val_swoff],swit_t
35024     ;cmp word [cs:result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_t ;M059
35025     cmp     word [result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_t ;M059
35026     jne     short if116 ;M059 M063
35027     ; 14/04/2024
35028     ;;;
35029     jne     short if118 ; (PCDOS 7.1 IBMBIO.COM)
35030     ;;;
35031     ; 15/12/2022
35032     mov     byte [p_swit_t],1
35033     mov     byte [cs:p_swit_t],1 ;M059
35034     jmp     short do110 ;M059
35035
35036     ; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
35037     ;;;
35038 if118:
35039     ;cmp word [cs:result_val_swoff],swit_i ; offset "/"
35040     ;cmp word [cs:result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_i
35041     cmp     word [result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_i
35042     jne     short if116
35043     mov     byte [cs:p_swit_i],1 ; set the flag
35044     mov     byte [p_swit_i],1
35045     jmp     short do110
35046     ;;;
35047 if116:
35048     ; 15/12/2022
35049     ;cmp word [cs:result_val_swoff],swit_w
35050     ;cmp word [cs:result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_w ;M063
35051     cmp     word [result_val+_$P_Result_Blk.SYNONYM_Ptr],swit_w ;M063
35052     jne     short do110 ;M063
35053     ; 15/12/2022
35054     mov     byte [p_swit_w],1
35055     mov     byte [cs:p_swit_w],1 ;M063
35056     jmp     short do110 ;M063
35057 en110:
35058     ; 15/12/2022
35059     ds = cs
35060     cmp     byte [p_swit_k],1
35061     cmp     byte [cs:p_swit_k],1 ; if /k entered,
35062     push     ds
35063     ;mov     ax,Bios_Data

```

```

35064             ;mov     ax,KERNEL_SEGMENT ; 0070h
35065             ; 21/10/2022
35066             mov     ax,DOSBIODATASEG ; 0070h
35067             mov     ds,ax
35068             jne     short if117
35069             ; 14/04/2024
35070             mov     byte [keyrd_func],0 ; 4E5h ; use the conventional keyboard functions
35071             ; BIOSDATA:047Eh for PCDOS 7.1 IBMBIO.COM
35072             mov     byte [keysts_func],1 ; 4E6h (for MSDOS 6.21 IO.SYS)
35073             ; BIOSDATA:047Fh for PCDOS 7.1 IBMBIO.COM
35074
35075             if117:
35076             ; 15/12/2022
35077             ; ds <> cs
35078             mov     al,[cs:p_swit_t] ;M059
35079             mov     [t_switch],al ; 4F2h (for MSDOS 6.21 IO.SYS) ;M059
35080             cmp     byte [cs:p_swit_w],0 ; BIOSDATA:048Bh for PCDOS 7.1 IBMBIO.COM
35081             je      short skip_dos_flag ;M063
35082             push    es ;M063
35083             push    bx
35084             mov     ah,GET_IN_VARS ; 52h ;M063
35085             int     21h ;M063
35086             ; DOS - 2+ internal - GET LIST OF LISTS
35087             ; Return: ES:BX -> DOS list of lists
35088             ;or     bytes [es:86h],2
35089             or      byte [es:DOS_FLAG_OFFSET],SUPPRESS_WINA20 ; 2 ;M063
35090             pop     bx
35091             pop     es
35092             skip_dos_flag: ;M063
35093             pop     ds
35094             sr110:
35095             jmp     coff
35096
35097             ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35098             ; (SYSINIT:2D14h)
35099             ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35100             %if 0
35101
35102             tryv:
35103
35104             ;ifdef     MULTI_CONFIG
35105             ;-----
35106             ; set command (as in "set var=value<cr>lf")
35107             ;-----
35108
35109             cmp     ah,CONFIG_SET ; 'v'
35110             jne     short tryn
35111             call    query_user ; query the user if config_cmd
35112             jc      short tryn ; has the CONFIG_OPTION_QUERY bit set
35113             call    copy_envvar ; copy var at ES:SI to "config_wrkseg"
35114             jnc     short sr110 ; no error
35115             err:
35116             call    error_line ; whoops, display error in line xxx
35117             jmp     short sr110 ; jump to coff (to skip to next line)
35118
35119             ;-----
35120             ; numlock command (as in "numlock=on|off")
35121             ;-----
35122
35123             tryn:
35124             cmp     ah,CONFIG_NUMLOCK ; 'N'
35125             jne     short tryy
35126             ; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
35127             jne     short tryt
35128
35129             call    query_user ; query the user if config_cmd
35130             jc      short tryy ; has the CONFIG_OPTION_QUERY bit set
35131             call    set_numlock
35132             jc      short err
35133             jmp     short sr110 ; all done
35134
35135             ;endif ;MULTI_CONFIG
35136
35137             ; 14/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMBIO.COM)
35138             ;-----
35139             ; dosdata command
35140             ;-----
35141             tryt:
35142             ;cmp     ah,54h ; 'T'
35143             cmp     ah,CONFIG_DOSDATA ; 'T' ; PCDOS 7 new config cmd
35144             jne     short tryy
35145
35146             call    query_user
35147             jc      short tryy
35148
35149             mov     di,dosdata_parms
35150             xor     cx,cx
35151             ; 14/04/2024 - Retro DOS v5.0
35152             ;mov     dx,cx ; 0
35153             do120:
35154             call    sysinit_parse
35155             jnc     short if120
35156
35157             ;call    badparm_p
35158             ;jmp     short en120
35159             ; 14/04/2024 - Retro DOS v5.0
35160             jmp     badparm_p_coff
35161             if120:
35162             ;cmp     ax,0FFFFh
35163             cmp     ax,_$P_RC_EOL ; -1 ; end of line?
35164             jz      short en120
35165             cmp     byte [result_val_itag],1 ; tag 1 (UMB)
35166             ; [result_val+$_P_Result_Blk.Item_Tag]
35167             jnz     short if121
35168             mov     byte [dosdata_umb],1 ; DOSDATA=UMB (1) NOUMB (0)
35169             jmp     short sr120
35170             ; 14/04/2024
35171             jmp     short do120
35172             if121:
35173             mov     byte [dosdata_umb],0 ; DOSDATA=UMB (1) NOUMB (0)
35174             sr120:
35175             jmp     short do120
35176             ; 14/04/2024
35177             ;en120:
35178             jmp     coff
35179
35180             ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35181             ;-----
35182             ; comment= do nothing. just decrease chrptr,and increase count for correct
35183             ; line number
35184             ;-----
35185
35186             ; 31/12/2022
35187             tryy:
35188             cmp     ah,CONFIG_COMMENT ; 'Y'

```



```

35188 00002E2C 750B      jne      short try0
35189
35190 donothing:
35191      ; 15/12/2022
35192      ; ds = cs
35193 00002E2E FF0E[5A03]  dec     word [chrptr]
35194 00002E32 FF06[5603]  inc     word [count]
35195      ; 02/11/2022
35196      ;dec     word [cs:chrptr]
35197      ;inc     word [cs:count]
35198 en120:
35199 00002E36 E94BF7      jmp     coff      ; 14/04/2024
35200
35201      ;-----
35202      ; rem command
35203      ;-----
35204
35205 try0:
35206 00002E39 80FC30      cmp     ah,CONFIG_REM ; '0'      ; do nothing with this line.
35207 00002E3C 74F0      je      short donothing
35208
35209 ;%endif      ; 30/10/2022
35210
35211      ; 30/10/2022
35212      ; (MSSOS 5.0 IO.SYS - SYSINIT:29D7h)
35213
35214      ;-----
35215      ; bogus command
35216      ;-----
35217
35218 tryz:
35219 00002E3E 80FCFF      cmp     ah,0FFh      ;null command? (BUGBUG - who sets FFh anyway?)
35220      ; 31/12/2022
35221 00002E41 74EB      je      short donothing
35222      ; 02/11/2022
35223      ;je      short tryz_donothing
35224
35225 00002E43 FF0E[5A03]  dec     word [chrptr]
35226 00002E47 FF06[5603]  inc     word [count]
35227 00002E4B EB37      jmp     short badop
35228
35229      ; 31/12/2022
35230      ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
35231 ;tryz_donothing:
35232      ; jmp     donothing
35233
35234      ; 07/04/2019 - Retro DOS v4.0
35235
35236      ;-----
35237
35238      ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35239      ; (SYSINIT:2D5Dh)
35240
35241      ; 11/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
35242
35243      ; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35244      ;
35245      ;***CheckProtmanArena -- special hack for adjusting alloclim with Protman$
35246      ;
35247      ; adjusts alloclim if Protman$ reduced our arena through a manual hack.
35248      ;
35249 CheckProtmanArena:
35250      ; 08/09/2023
35251      ; ds = cs
35252 00002E4D 06      push    es
35253      ;mov     ax,[cs:area]      ; get our arena header
35254 00002E4E A1[6803]  mov     ax,[area] ; 08/09/2023
35255 00002E51 48      dec     ax
35256 00002E52 8EC0      mov     es,ax
35257      ;add     ax,[es:ARENA.SIZE]
35258 00002E54 2603060300 add     ax,[es:3]      ; find end of arena
35259 00002E59 40      inc     ax
35260      ; 08/09/2023
35261 00002E5A 3B06[A502]  cmp     ax,[ALLOCLIM]
35262      ;cmp     ax,[cs:ALLOCLIM] ; is it less than alloclim?
35263 00002E5E 7703      ja      short CheckProtmanDone
35264
35265      ;mov     [cs:ALLOCLIM],ax ; reduce alloclim then
35266      ; 08/09/2023
35267 00002E60 A3[A502]  mov     [ALLOCLIM],ax
35268 CheckProtmanDone:
35269 00002E63 07      pop     es
35270 00002E64 C3      retn
35271
35272      ;-----
35273
35274 sysinit_parse:
35275
35276      ;-----
35277 ;set up registers for sysparse
35278 ;in)es:si -> command line in confbot
35279 ; di -> offset of the parse control definition.
35280 ;
35281 ;out)      calls sysparse.
35282 ; carry will set if parse error.
35283 ; *** the caller should check the eol condition by looking at ax
35284 ; *** after each call.
35285 ; *** if no parameters are found,then ax will contain a error code.
35286 ; *** if the caller needs to look at the synonym@ of the result,
35287 ; *** the caller should use cs:@ instead of es:@.
35288 ; cx register should be set to 0 at the first time the caller calls this
35289 ; procedure.
35290 ; ax - exit code
35291 ; bl - terminated delimiter code
35292 ; cx - new positional ordinal
35293 ; si - set to pase scanned operand
35294 ; dx - selected result buffer
35295      ;-----
35296
35297      ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35298      ; (SYSINIT:2D78h)
35299
35300      ; 14/04/2024 - Retro DOS v5.0
35301      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:32F3h)
35302
35303      ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
35304      ; ds = cs
35305 00002E65 8C06[6D19]  mov     [badparm_seg],es      ;save the pointer to the parm
35306 00002E69 8936[6B19]  mov     [badparm_off],si      ;we are about to parse for badparm msg.
35307
35308      ; 24/10/2022
35309 00002E6D 06      push    es      ;save es,ds
35310 00002E6E 1E      push    ds
35311

```

```

35312 00002E6F 06      push    es
35313 00002E70 1F      pop     ds                ;now ds:si -> command line
35314
35315 00002E71 0E      push    cs
35316 00002E72 07      pop     es                ;now es:di -> control definition
35317
35318      ; 09/09/2023
35319      ;mov     [cs:badparm_seg],ds    ;save the pointer to the parm
35320      ;mov     [cs:badparm_off],si   ;we are about to parse for badparm msg.
35321
35322      ;mov     dx,0
35323      ; 04/01/2023
35324 00002E73 29D2      sub     dx,dx ; 0
35325 00002E75 E89DEB      call    SysParse
35326      ;cmp     ax,_$P_No_Error        ; 0      ;no error
35327      ; 06/09/2023
35328 00002E78 21C0      and     ax,ax
35329
35330      ;**cas note: when zero true after cmp, carry clear
35331
35332      ;je      short 114
35333      ; 24/10/2022 (MSDOS 5.0 IO.SYS compatibility, SYSINIT:2A02h)
35334      ; 12/12/2022
35335 00002E7A 7405      je      short en4 ; cf=0
35336 00002E7C 83F8FF      cmp     ax,_$P_RC_EOL ; 0FFFFh;or the end of line?
35337      ;jne     short if4
35338      ; 12/12/2022
35339 00002E7F 7400      je      short en4 ; cf=0
35340      ; 06/09/2023
35341      ; cf=1
35342
35343      ; 12/12/2022
35344      ;114:
35345      ;      ; 12/12/2022
35346      ;      ; cf=0
35347      ;      ; clc
35348      ;      jmp     short en4
35349
35350      if4:
35351      ; 24/10/2022
35352      ; 06/09/2023 (cf=1)
35353      ;stc
35354
35355 00002E81 1F      pop     ds
35356 00002E82 07      pop     es
35357 00002E83 C3      retn
35358
35359      ; 11/12/2022
35360      %if 0
35361
35362      ;-----
35363      ;
35364      ; procedure : badop_p
35365      ;
35366      ; same thing as badop,but will make sure to set ds register back
35367      ; to sysinitseg and return back to the caller.
35368      ;
35369      ;-----
35370
35371      badop_p:
35372      push    cs
35373      pop     ds                ;set ds to configsys seg.
35374      mov     dx,badopm
35375      call    print
35376      ;call    error_line
35377      ;retn
35378      ; 11/12/2022
35379      jmp     error_line
35380
35381      %endif
35382
35383      ;-----
35384      ;
35385      ; label : badop
35386      ;
35387      ;-----
35388
35389      badop:
35390 00002E84 BA[4A50]      mov     dx,badopm        ;want to print command error "unrecognized command..."
35391 00002E87 E8C81B      call    print
35392 00002E8A E82B00      call    error_line        ;show "error in config.sys ..." .
35393 00002E8D E9F4F6      jmp     coff
35394
35395      ;-----
35396      ;
35397      ; procedure : badparm_p
35398      ;
35399      ; show "bad command or parameters - xxxxxx"
35400      ; in badparm_seg,badparm_off -> xxxxx
35401      ;
35402      ;-----
35403
35404      ; 24/10/2022
35405      badparm_p:
35406      ; 11/12/2022
35407      ; ds = cs
35408      ; 11/12/2022
35409      ;push    ds ; *
35410 00002E90 52      push    dx
35411 00002E91 56      push    si
35412
35413      ; 11/12/2022
35414      ; ds = cs
35415      ;push    cs
35416      ;pop     ds
35417
35418 00002E92 BA[7150]      mov     dx,badparm
35419 00002E95 E8BA1B      call    print                ; "bad command or parameters - "
35420 00002E98 C536[6B19]      lds     si,[badparm_ptr]
35421
35422      ; print "xxxx" until cr.
35423
35424      do1:
35425 00002E9C 8A14      mov     dl,[si]                ; get next character
35426 00002E9E 80FA0D      cmp     dl,cr ; 0dh            ; is a carriage return?
35427 00002EA1 7407      je      short en1                ; exit loop if so
35428
35429 00002EA3 B402      mov     ah,2 ; STD_CON_OUTPUT    ; function 2
35430 00002EA5 CD21      int     21h                    ; display character
35431 00002EA7 46      inc     si                        ; next character
35432 00002EA8 EBF2      jmp     short do1
35433
35434 00002EAA 0E      push    cs
35435 00002EAB 1F      pop     ds

```



```

35436
35437 00002EAC BA[6E50]      mov     dx,crlfm
35438 00002EAF E8A01B      call    print
35439 00002EB2 E80300      call    error_line
35440
35441 00002EB5 5E          pop     si
35442 00002EB6 5A          pop     dx
35443                      ; 11/12/2022
35444                      ;pop     ds ; *
35445 badparmp_ret:
35446 00002EB7 C3          retn
35447
35448 ; 10/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
35449 %if 0
35450
35451 ;-----
35452 ;
35453 ; procedure : getchr
35454 ;
35455 ;-----
35456
35457 ; 24/10/2022
35458 getchr:
35459 ; 12/12/2022
35460 ;push    cx
35461 ;mov     cx,[count]
35462 ;jcxz    nochar
35463 ; 12/12/2022
35464 cmp     word [count],1
35465 jb      short nochar ; cf=1 ([count] = 0)
35466
35467 mov     si,[chrptr]
35468 mov     al,[es:si]
35469 dec     word [count]
35470 inc     word [chrptr]
35471 ; 12/12/202
35472 ; cf=0
35473 ;clc
35474 ;get_ret:
35475 ;pop     cx
35476 ;retn
35477 nochar:
35478 ; 12/12/2022
35479 ; cf=1
35480 ;stc
35481 ;jmp     short get_ret
35482
35483 retn
35484 %endif
35485
35486 ; 11/12/2022
35487 %if 0
35488
35489 ;-----
35490 ;
35491 ; procedure : incorrect_order
35492 ;
35493 ; show "incorrect order in config.sys ..." message.
35494 ;
35495 ;-----
35496
35497 incorrect_order:
35498 mov     dx,badorder
35499 call    print
35500 call    showlinenum
35501 retn
35502
35503 %endif
35504
35505 ;-----
35506 ;
35507 ; procedure : error_line
35508 ;
35509 ; show "error in config.sys ..." message.
35510 ;
35511 ;-----
35512
35513 ; 11/12/2022
35514 ; 24/10/2022
35515 error_line:
35516 ; 11/12/2022
35517 ; ds = cs
35518 ;push    cs
35519 ;pop     ds
35520
35521 00002EB8 BA[A651]      mov     dx,errorcmd
35522 00002EBB E8941B      call    print
35523                      ;call    showlinenum
35524                      ;retn
35525                      ; 11/12/2022
35526                      ;jmp     short shortlinenum
35527
35528 ;-----
35529 ;
35530 ; procedure : showlinenum
35531 ;
35532 ; convert the binary linecount to decimal ascii string in showcount
35533 ; and display showcount at the current curser position.
35534 ; in.) linecount
35535 ;
35536 ; out) the number is printed.
35537 ;
35538 ;-----
35539
35540 ; 11/12/2022
35541 ; ds = cs
35542 ; 24/10/2022
35543 showlinenum:
35544 00002EBE 06          push    es
35545                      ; 11/12/2022
35546                      ;push    ds
35547 00002EBF 57          push    di
35548
35549                      push    cs
35550 00002EC0 0E          pop     es ; es=cs
35551
35552 ; 11/12/2022
35553 ;push    cs
35554 ;pop     ds
35555
35556 00002EC2 BF[B502]      mov     di,showcount+4 ; di -> the least significant decimal field.
35557 00002EC5 B90A00      mov     cx,10 ; decimal divide factor
35558                      ;mov     ax,[cs:linecount]
35559                      ; 11/12/2022

```

```

35560 00002EC8 A1[AF02]      mov     ax,[linecount]
35561                          s1n_loop:
35562                          ; 11/12/2022
35563 00002ECB 39C8          cmp     ax,cx ; < 10 ?
35564                          ;cmp    ax,10      ; < 10?
35565 00002ECD 720C          jnb     short s1n_last
35566                          xor     dx,dx
35567 00002ECF 31D2          div     cx      ; cx = 10
35568 00002ED1 F7F1          or      dl,30h      ; add "0" (= 30h) to make it an ascii.
35569 00002ED3 80CA30        mov     [di],dl
35570 00002ED6 8815          dec     di
35571 00002ED8 4F            jmp     short s1n_loop
35572 00002ED9 EBF0
35573
35574                          s1n_last:
35575 00002EDB 0C30          or      al,30h ; "0"
35576 00002EDD 8805          mov     [di],al
35577 00002EDF 89FA          mov     dx,di
35578 00002EE1 E86E1B        call    print      ; show it.
35579 00002EE4 5F            pop     di
35580                          ; 11/12/2022
35581                          ;pop    ds
35582 00002EE5 07            pop     es
35583 00002EE6 C3            retn
35584
35585                          ; 07/04/2019 - Retro DOS v4.0
35586                          ; (MSDOS 6.21 IO.SYS, SYSINIT:2E44h)
35587
35588                          ;-----
35589                          ;
35590                          ; procedure : ProcDOS
35591                          ;
35592                          ; Process the result of DOS= parsing
35593                          ;
35594                          ; result_val._$P_item_tag      = 1 for DOS=HIGH
35595                          ;                          = 2 for DOS=LOW
35596                          ;                          = 3 for DOS=UMB
35597                          ;                          = 4 for DOS=NOUMB
35598                          ;-----
35599
35600                          ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
35601                          ; (SYTSINIT:2AB5h)
35602
35603                          ProcDOS:
35604                          ; 01/01/2023
35605 00002EE7 30E4          xor     ah,ah
35606                          ;;mov    al,[cs:result_val_itag]
35607                          ;mov     al,[cs:result_val+_$P_Result_Blk.Item_Tag]
35608                          ; 01/01/2023
35609 00002EE9 A0[1622]      mov     al,[result_val+_$P_Result_Blk.Item_Tag]
35610 00002EEC 48            dec     ax
35611 00002EED 7415          jz      short pd_hi
35612 00002EEF 48            dec     ax
35613 00002EF0 740E          jz      short pd_lo
35614 00002EF2 48            dec     ax
35615 00002EF3 7405          jz      short pd_umb
35616                          ;;mov    byte [cs:DevUMB],0
35617                          ; 18/12/2022
35618                          ;mov     byte [cs:DevUMB],ah ; 0
35619                          ; 01/01/2023
35620 00002EF5 8826[4024]      mov     byte [DevUMB],ah ; 0
35621 00002EF9 C3            retn
35622
35623                          pd_umb:
35624 00002EFA C606[4024]FF    mov     byte [DevUMB],0FFh
35625                          ;mov     byte [cs:DevUMB],0FFh
35626 00002EFF C3            retn
35627
35628                          pd_lo:
35629 00002F00 A2[6C02]      mov     [runhigh],al ; 0
35630                          ; 18/12/2022
35631                          ;mov     [cs:runhigh],al ; 0
35632                          ;;mov    byte [cs:runhigh],0
35633 00002F03 C3            retn
35634
35635                          pd_hi:
35636 00002F04 C606[6C02]FF    mov     byte [runhigh],0FFh
35637                          ;mov     byte [cs:runhigh],0FFh
35638
35639 00002F09 C3            limx:      ; 11/12/2022
35640                          retn
35641
35642                          ;-----
35643                          ;
35644                          ; procedure : LieInt12Mem
35645                          ;
35646                          ; Input : DevEntry points to Device Start address (offset == 0)
35647                          ;         alloclim set to the limit of low memory.
35648                          ;
35649                          ; Output : none
35650                          ;
35651                          ; Changes the ROM BIOS variable which stores the total low memory
35652                          ; If a 3com device driver (any character device with name 'PROTMAN$')
35653                          ; is being loaded alloclim is converted into Ks and stored in 40:13h
35654                          ; Else if a device driver being loaded into UMB the DevLoadEnd is
35655                          ; converted into Ks and stored in 40:13h
35656                          ;-----
35657
35658                          LieInt12Mem:
35659                          ; 11/12/2022
35660                          ; ds = cs
35661 00002F0A A1[A502]      mov     ax,[ALLOCLIM]
35662                          ;mov     ax,[cs:ALLOCLIM]      ; lie INT 12 as alloclim
35663                          ; assuming that it is 3Com
35664 00002F0D E84200        call    IsIt3Com      ; Is it 3Com driver?
35665 00002F10 740A          jz      short lim_set      ; yes, lie to him differently
35666                          ; 13/05/2019
35667                          ;cmp     byte [cs:DeviceHi],0 ; Is the DD being loaded in UMB
35668                          ;je      short limx          ; no, don't lie
35669                          ;mov     ax,[cs:DevLoadEnd]    ; lie INT 12 as end of UMB
35670                          ; 11/12/2022
35671                          ; ds = cs
35672 00002F12 803E[4F24]00    cmp     byte [DeviceHi],0
35673 00002F17 74F0          je      short limx
35674 00002F19 A1[3524]      mov     ax,[DevLoadEnd]
35675                          lim_set:
35676                          ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
35677                          ; 11/12/2022
35678                          ;call    SetInt12Mem
35679                          ;limx:
35680                          ;retn
35681
35682                          ;jmp     short SetInt12Mem
35683

```

```

35684
35685
35686
35687
35688
35689
35690
35691
35692
35693
35694
35695
35696
35697
35698
35699
35700 00002F1C 1E
35701 00002F1D 8B4000
35702 00002F20 8EDB
35703 00002F22 8B1E1300
35704
35705 00002F26 B106
35706 00002F28 D3E8
35707 00002F2A A31300
35708
35709 00002F2D 1F
35710
35711
35712 00002F2E 891E[5324]
35713 00002F32 C606[5224]FF
35714
35715 00002F37 C3
35716
35717
35718
35719
35720
35721
35722
35723
35724
35725
35726
35727
35728
35729
35730
35731
35732
35733
35734
35735 00002F38 803E[5224]00
35736
35737
35738
35739 00002F3D 7412
35740
35741 00002F3F 884000
35742 00002F42 8826[5224]
35743
35744
35745 00002F46 1E
35746
35747 00002F47 8ED8
35748 00002F49 2EA1[5324]
35749 00002F4D A31300
35750 00002F50 1F
35751
35752 00002F51 C3
35753
35754
35755
35756
35757
35758
35759
35760
35761
35762
35763
35764
35765
35766
35767 00002F52 1E
35768 00002F53 06
35769 00002F54 56
35770
35771 00002F55 C536[3724]
35772
35773 00002F59 83C60A
35774 00002F5C 0E
35775 00002F5D 07
35776 00002F5E BF[5524]
35777 00002F61 B90800
35778 00002F64 F3A6
35779 00002F66 5E
35780 00002F67 07
35781 00002F68 1F
35782 00002F69 C3
35783
35784
35785
35786
35787
35788 00002F6A 1E
35789 00002F6B B462
35790 00002F6D CD21
35791 00002F6F 8EDB
35792 00002F71 2E8B1E[A502]
35793
35794 00002F76 891E0200
35795 00002F7A 1F
35796 00002F7B C3
35797
35798
35799
35800
35801
35802
35803
35804
35805
35806
35807

```

```

;-----
;
; procedure : SetInt12Mem
;
; Input : AX = Memory size to be set (in paras)
; Output : none
;
; Sets the variable 40:13 to the memory size passed in AX
; It saves the old value in 40:13 in OldInt12Mem,
; It also sets a flag Int12Lied to 0FFh, which is checked before
; restoring the value of 40:13
;-----
;
; 01/11/2022
SetInt12Mem:
    push    ds
    mov     bx,40h
    mov     ds,bx
    mov     bx,[13h]
    ; ROM BIOS Data Segment
    ; INT 12 memory variable
    ; save it
    mov     [cs:OldInt12Mem],bx
    mov     cx,6
    shr     ax,cx
    ; convert paras into Ks
    mov     [13h],ax
    ; Lie
    mov     byte [cs:Int12Lied],0FFh ; mark that we are lying
    pop     ds
    ; 14/04/2024
    ; ds = cs
    mov     [OldInt12Mem],bx
    mov     byte [Int12Lied],0FFh
;limx:
    retn

;-----
;
; procedure : TrueInt12Mem
;
; Input : Int12Lied = 0 if we are not lying currently
;         = 0FFh if we are lying
;         OldInt12Mem = Saved value of 40:13h
;
; Output : none
;
; Resets the INT 12 Memory variable if we were lying about int 12
; and resets the flag which indicates that we were lying
;-----
;
TrueInt12Mem:
    ; 11/12/2022
    ; ds = cs
    cmp     byte [Int12Lied],0
    ;cmp     byte [cs:Int12Lied],0 ; were we lying so far?
    ; 01/11/2022 (MSDOS 5.0 IO.SYS, SYS.INIT:2B1Dh)
    ;mov     byte [cs:Int12Lied],0 ; reset it anyway
    je      short timx
    ; 18/12/2022
    mov     ax,40h
    mov     [Int12Lied],ah ; 0
    ;mov     byte [Int12Lied],0
    ;mov     byte [cs:Int12Lied],0
    push    ds
    ;mov     ax,40h
    mov     ds,ax
    mov     ax,[cs:OldInt12Mem]
    mov     [13h],ax
    ; restore INT 12 memory
    pop     ds
timx:
    retn

;-----
;
; procedure : IsIt3Com?
;
; Input : DevEntry = Seg:0 of device driver
; Output : Zero flag set if device name is 'PROTMAN$'
;         else Zero flag is reset
;-----
;
IsIt3Com:
    ; 11/12/2022
    ; ds = cs
    push    ds
    push    es
    push    si
    ; 11/12/2022
    lds     si,[DevEntry]
    ;lds     si,[cs:DevEntry]
    ; ptr to device header
    add     si,SYSDEV.NAME ; 10 ; ptr device name
    push    cs
    pop     es
    mov     di,ThreeComName
    mov     cx,8
    ; name length
    rep     cmpsb
    pop     si
    pop     es
    pop     ds
    retn

;M020 : BEGIN
;-----
;
UpdatePDB:
    push    ds
    mov     ah,62h
    int     21h ; DOS - 3+ - GET PSP ADDRESS
    mov     ds,bx
    mov     bx,[cs:ALLOCLIM]
    ;mov     [2],bx
    mov     [PDB.BLOCK_LEN],bx
    pop     ds
    retn

; M020 : END
;-----
;
; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; %if 0
;
; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:2EEH)

```

```

35808 ;include highload.inc      ; Routines for devicehigh parsing, control of HIDDEN
35809 ;include highexit.inc      ; umb's, etc
35810
35811 ; -----
35812 ; HIGHLOAD.INC (MSDOS 6.0 - 1991)
35813 ; -----
35814 ; 07/04/2019 - Retro DOS v4.0
35815
35816 ;*****
35817 ;
35818 ; This file contains routines needed to parse and implement user-given
35819 ; command-line options of the form "/S/L:3,0x500;2;7,127;0x0BE4". InitVar()
35820 ; and Parsevar() are used to parse this data and place it in encoded form into
35821 ; the variables in highvar.inc, for use by the rest of the routines.
35822 ;
35823 ; DeviceHigh accepts this command-line (handled in sysconf.asm, not here):
35824 ;   DEVICEHIGH SIZE=hhhhh module opts
35825 ; Or, DeviceHigh and LoadHigh accept any of the following:
35826 ;   DH/LH module opts
35827 ;   DH/LH [/S][/L:umb[,size][;umb[,size]]*] module opts
35828 ;   DH/LH [/L:umb[,size][;umb[,size]]*][/S] module opts
35829 ; The initial UMB,SIZE pair designates the module's load address; the remainder
35830 ; of the UMB and SIZE pairs are used to indicate specific UMBs to be left
35831 ; available during the load.
35832 ;
35833 ; When an actual load is ready to be performed, a call to HideUMBs() will
35834 ; temporarily allocate (as owner 8+"HIDDEN ") all free elements in any
35835 ; upper-memory block which was not specified by the user... in addition, if
35836 ; UMBs were marked to shrink (/S option) to a certain size ("umb,size"), any
35837 ; elements in that umb SAVE the lower-half of the newly-shrunk one are also
35838 ; allocated. After the load, the function UnHideUMBs() (in highexit.inc) will
35839 ; free any UMBs so allocated.
35840 ;
35841 ; When a device driver loads, there is the additional problem of allocating its
35842 ; initial load site; this should be restricted to the first UMB specified on
35843 ; the command-line. The function FreezeUM temporarily allocates all remaining
35844 ; free upper-memory elements (as owner 8+"FROZEN "), except those in the load
35845 ; UMB. Then the initial allocation may be made, and a call to UnFreeze will
35846 ; return any so-allocated memory elements to FREE, for the true load. Note
35847 ; that UnFreeze leaves HIDDEN elements allocated; it only frees FROZEN ones.
35848 ;
35849 ;*****
35850
35851 SWITCH      equ      '/'      ; Switch character
35852
35853 DOS_CHECK_STRATEGY equ 5800h  ; Int 21h, Func 58h, Svc 0 = check alloc strat
35854 DOS_SET_STRATEGY   equ 5801h  ; Int 21h, Func 58h, Svc 1 = set alloc strategy
35855 DOS_CHECK_UMBLINK  equ 5802h  ; Int 21h, Func 58h, Svc 2 = check link state
35856 DOS_GET_UMBLINK    equ 5802h  ; 20/04/2019
35857 DOS_SET_UMBLINK    equ 5803h  ; Int 21h, Func 58h, Svc 3 = set link state
35858 DOS_GET_DOS_LISTS  equ 52h    ; Int 21h, Func 52h = return list of lists
35859 DOS_UMB_HEAD       equ 8ch    ; Offset from ES (after func52h) to get UMBHead
35860
35861 CR equ 0dh          ; Carriage Return
35862 LF equ 0ah          ; Line Feed
35863 TAB equ 09h         ; Tab character (^I)
35864
35865 ; -----
35866 ;*** InitVar - initializes all the variables used in ParseVar and HideUMBs
35867 ; -----
35868 ; ENTRY:      None
35869 ; EXIT:       Variables listed in highvar.inc are initialized
35870 ; ERROR EXIT: None
35871 ; USES:       Flags, variables in highvar.inc
35872 ; -----
35873 ; Note that element 0 references UMB 0 (conventional), not UMB 1. Its contents
35874 ; are largely ignored, but it is initialized nonetheless.
35875 ; -----
35876
35877 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35878 ; (SYSINIT:2EEH)
35879
35880 InitVar:
35881 ; 01/01/2023
35882 ; ds = cs
35883
35884 ;pushreg <ax, cx, di, es>
35885 ; 03/01/2023
35886 ;push ax
35887 ;push cx
35888 ;push di
35889 00002F7C 06      push es
35890
35891 ;dataseg es          ;Point ES into appropriate data segment
35892 00002F7D 0E      push cs
35893 00002F7E 07      pop  es
35894
35895 00002F7F 31C0     xor  ax,ax
35896 ;mov [es:fUmbTiny],al ;Shrink UMBs? (made 1 if /S given)
35897 ;mov [es:fInHigh],al  ;Set to 1 when DH/LH has been called
35898 ;mov [es:SegLoad],ax  ;Load Address (seg), used for DH only
35899 ;mov byte [es:UmbLoad],UNSPECIFIED ; 0FFh
35900 ; ;Later is the # of the 1st spec'd UMB
35901 ;mov [es:fm_argc], al ;Start with zero args having been read
35902
35903 ; 01/01/2023
35904 ; ds = cs
35905 00002F81 A2[FA23] mov [fUmbTiny],al ;Shrink UMBs? (made 1 if /S given)
35906 00002F84 A2[F923] mov [fInHigh],al ;Set to 1 when DH/LH has been called
35907 00002F87 A3[FB23] mov [SegLoad],ax ;Load Address (seg), used for DH only
35908 00002F8A C606[FD23]FF mov byte [UmbLoad],UNSPECIFIED ; 0FFh
35909 ; ;Later is the # of the 1st spec'd UMB
35910 00002F8F A2[3024] mov [fm_argc], al ;Start with zero args having been read
35911
35912 00002F92 FC      cld
35913
35914 00002F93 B91000   mov cx,MAXUMB ; 16 ;For each entry
35915 00002F96 BF[FE23] mov di,UmbUsed ;on the UmbUsed array,
35916 00002F99 F3AA     rep stosb ; Store 0
35917
35918 ;mov cx,MAXUMB ; 16 ;Okay... for each entry
35919 ; 01/01/2023
35920 00002F9B B110     mov cl,MAXUMB ; 16
35921 00002F9D BF[0E24] mov di,UmbSize ;on the UmbSize array,
35922 00002FA0 F3AB     rep stosw ; Store 0
35923
35924 ;normseg es          ; Return ES
35925
35926 ;popreg <es, di, cx, ax>
35927 00002FA2 07      pop  es
35928 ; 03/01/2023
35929 ;pop di
35930 ;pop cx
35931 ;pop ax

```

```

35932
35933 00002FA3 C3      retn
35934
35935
35936 ;-----
35937 ;*** FixMem - scans the upper memory chain and concatenates adjacent free MCBs
35938 ;-----
35939 ; ENTRY   : None
35940 ; EXIT    : None
35941 ; ERROR   : None
35942 ; USES    : Flags, fm_umb, fm_strat
35943 ;-----
35944 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
35945 ; (SYSINIT:2F22h)
35946 FixMem:
35947 ; 01/01/2023
35948 ;push  ax
35949 ;push  bx
35950 ;push  cx
35951 ;push  dx
35952 00002FA4 06      push  es
35953
35954 00002FA5 E84900    call  fm_link      ; Link in UMBS
35955
35956 00002FA8 E80002    call  UmbHead      ; Get first upper-memory MCB address (0x9FFF)
35957 00002FAB 723F      jc      short fmX      ; (if couldn't get it, leave now).
35958
35959 00002FAD 8EC0      mov     es,ax          ; It returns in AX, so move it to ES.
35960
35961 ; - walk MCB Chain -----
35962
35963 00002FAF 31D2      xor     dx,dx          ; We're keeping the address of the last MCB
35964 00002FB1 89D1      mov     cx,dx          ; in CX... and the last owner
35965 00002FB3 42        inc     dx          ; in dx as we go through the loop:
35966
35967 ;-----
35968 ; FM10--DX = last MCB's owner's PSP address
35969 ; CX = last MCB's address (segment)
35970 ;-----
35971
35972 00002FB4 26A00000  fm10:   mov     al,[es:ARENA.SIGNATURE] ; if 'Z', don't repeat loop
35973 00002FB8 268B1E0100 mov     bx,[es:ARENA.OWNER] ; if not zero, do nothing
35974 00002FBD 09D3      or      bx,dx          ; dx was owner of previous MCB
35975 00002FBF 7516      jnz     short fm30      ; If not both zero, don't cat.
35976
35977 ; - Coalesce memory blocks at ES:00 and CX:00 -----
35978
35979 00002FC1 268B1E0300 fm20:   mov     bx,[es:ARENA.SIZE] ; Grab this block's Size,
35980 00002FC6 8EC1      mov     es,cx          ; Go back to prev MCB's address
35981 00002FC8 26A20000    mov     [es:ARENA.SIGNATURE],al ; & move the SECOND sig here
35982
35983 00002FCC 26031E0300 add     bx,[es:ARENA.SIZE] ; Size += first MCB's size
35984 add     bx,1          ; And add one for the header
35985 ; 11/07/2023
35986 00002FD1 43        inc     bx
35987 00002FD2 26891E0300 mov     [es:ARENA.SIZE],bx ; write the size
35988
35989 ; -----
35990
35991 00002FD7 8CC1      fm30:   mov     cx,es          ; Put this address on the stack
35992 00002FD9 268B160100 mov     dx,[es:ARENA.OWNER] ; And remember its owner
35993
35994 00002FDE 8CC3      mov     bx,es          ; Move to the next MCB
35995 00002FE0 26031E0300 add     bx,[es:ARENA.SIZE]
35996 00002FE5 43        inc     bx
35997 00002FE6 8EC3      mov     es,bx
35998
35999 ;cmp    al,'Z'
36000 00002FE8 3C5A      cmp     al,arena_signature_end
36001 00002FEA 75C8      jne     short fm10      ; If signature != 'Z', there are more.
36002
36003 00002FEC E81300    fmX:   call  fm_unlink      ; Unlink UMBS
36004
36005 00002FEF 07        pop     es
36006 ; 01/01/2023
36007 ;pop    dx
36008 ;pop    cx
36009 ;pop    bx
36010 ;pop    ax
36011
36012 00002FF0 C3      retn
36013
36014 ;-----
36015 ;*** fm_link - links UMBS not already linked in
36016 ;-----
36017 ; ENTRY:   None
36018 ; EXIT:    fm_umb == 0 if not linked in previously, 1 if already linked in
36019 ; ERROR:   None
36020 ; USES:    AX, BX, fm_umb
36021 ;-----
36022
36023 ; 01/01/2023 - Retro DOS v4.2
36024 fm_link:
36025 mov     ax,DOS_CHECK_UMBLINK ; 5802h
36026 00002FF4 CD21      int     21h          ; Current link-state is now in al
36027
36028 ;putdata fm_umb,al ; So store it in fm_umb for later
36029 ;
36030 ;push   es
36031 ;push   cs
36032 ;pop    es
36033 ;mov    [es:fm_umb],al
36034 ;pop    es
36035
36036 ; 01/01/2023
36037 ; ds = cs
36038 ;mov    [cs:fm_umb],al
36039 00002FF6 A2[2E24]  mov     [fm_umb],al
36040
36041 mov     ax,DOS_SET_UMBLINK ; 5803h
36042 mov     bx,1
36043 int     21h
36044 00003001 C3      retn
36045
36046 ;-----
36047 ;*** fm_unlink - unlinks UMBS if fm_umb is set to 0
36048 ;-----
36049 ; ENTRY:   fm_umb == 1 : leave linked, else unlink
36050 ; EXIT:    None
36051 ; ERROR:   None
36052 ; USES:    AX, BX
36053 ;-----
36054
36055 ; 01/01/2023 - Retro DOS v4.2

```

```

36056 fm_unlink:
36057 00003002 31DB xor     bx,bx
36058
36059         ;getdata bl,fm_umb          ; fm_umb already has the old link-state
36060
36061         ;
36062         ;push  ds
36063         ;push  cs
36064         ;pop   ds
36065         ;mov   bl,[fm_umb]
36066         ;pop   ds
36067
36068         ; 01/01/2023
36069         ; ds = cs
36070 00003004 8A1E[2E24] mov     bl,[cs:fm_umb]
36071         mov     bl,[fm_umb]
36072 00003008 B80358 mov     ax,DOS_SET_UMBLINK ; 5803h
36073 0000300B CD21 int     21h                ; so just use that, and call int 21h
36074 0000300D C3 retn
36075
36076 ; 08/04/2019 - Retro DOS v4.0
36077
36078 ;-----
36079 ;*** ParseVar - parses [/S][L:umb[,size][;umb[,size]]*] and builds the table
36080 ; laid out in highvar.inc
36081 ;-----
36082 ; ENTRY: ES:SI points to command tail of LoadHigh/DeviceHigh (whitespace ok)
36083 ; EXIT: ES:SI points to first character in child program name
36084 ; ERROR: ES:SI points to character which caused error, carry set, AX == code
36085 ; USES: ES:SI, AX, flags, variables in highvar.inc
36086 ;-----
36087 ; Error codes (in AX if carry set on return):
36088
36089 PV_InvArg equ 1 ; Invalid argument passed
36090 PV_BadUMB equ 2 ; Bad UMB number passed (duplicate?)
36091 PV_InvSwT equ 3 ; Unrecognized switch passed
36092
36093 ; This routine expects ES:SI to point to a string much like the following:
36094 ; "/S/L:1,200;2 module options"
36095 ; Optionally, the string can begin with whitespace; neither /S nor /L is
36096 ; required, though that's what this routine is supposed to parse.
36097
36098 optS equ 'S' ; /S
36099 optL equ 'L' ; /L:...
36100
36101 ;-----
36102 ; LoadHigh has a list of arguments, returned by cparse, which is used to create
36103 ; a command-line for spawning a child process. For a typical LH command, say,
36104 ; lh /l:1,1000;2 print/d:lpt2
36105 ; the arguments would look like (one per line):
36106 ; lh
36107 ; /l
36108 ; 1
36109 ; 1000
36110 ; 2
36111 ; print
36112 ; /d
36113 ; :lpt2
36114 ; In short, if "print" were, say, "43", there'd be no way to determine which
36115 ; arg was the filename. So, inside this routine, we keep a running counter
36116 ; of the number of arguments LH will need to skip in order to get to the
36117 ; program name. The "lh" is implicit--it'll always have to skip that. So if
36118 ; there's no "/l" or "/s", fm_argc will be 0 ... other than that, 1 is added
36119 ; for:
36120 ; Each /L
36121 ; Each /S (there should be only one)
36122 ; Each UMB number (they follow ":" or ";")
36123 ; Each UMB size (they follow ",")
36124 ; So, in the above example, fm_argc would be 4-- and LH would skip right to
36125 ; "print". Note that InitVar initializes fm_argc to zero.
36126 ;-----
36127
36128 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36129 ; (SYSINIT:2F9Fh)
36130
36131 ParseVar:
36132 ;pushreg <di, ds, es>
36133 ; 01/01/2023
36134 ;push  di ; * ; (not required) ; 01/01/2023
36135 0000300E 1E push    ds
36136 0000300F 06 push    es
36137
36138 push    es ; Make DS:SI point to it, as well as ES:SI
36139 pop     ds ; (regardless if we're in devhigh or loadhigh)
36140 cld
36141
36142 ;-----
36143 ; PV10--ES:SI = any whitespace on the command-line
36144 ;-----
36145
36146 00003013 AC pv10: lodsb ; here, ES:SI==" /L..."--must eat whitespace
36147 00003014 E8A200 call    iswhite
36148 00003017 74FA jz     short pv10 ; ES:SI==" /L..."--keep eating.
36149 ;cmp  al,'/'
36150 00003019 3C2F cmp    al,SWTCH
36151 0000301B 7404 je     short pv20 ; ES:SI==" /L..."--go process a switch
36152
36153 0000301D 4E dec     si ; Backup--it's now "odule options", and we need
36154 0000301E F8 clc ; that "m" we just read (or whatever it is).
36155 0000301F EB2B jmp     short pvx ; Then return with carry clear == we're done.
36156
36157 00003021 AC pv20: lodsb ; Just read 'S' or 'L', hopefully
36158 ;toUpper al ; So we make it upper-case, and...
36159 00003022 24DF and     al,0DFh
36160 ;cmp  al,'S'
36161 00003024 3C53 cmp    al,optS ; just read 'S'?
36162 00003026 750D jne     short pv30
36163
36164 ;call  incArgc ; If it's /S, it's another arg for LH to skip.
36165 00003028 2EFE06[3024] inc     byte [cs:fm_argc] ; 19/04/2019
36166
36167 ;putdata fUmbTiny,1 ; /S, so ES:SI==" /L..." or " module opts", or
36168 ;
36169 ;push  es
36170 ;push  cs
36171 ;pop   es
36172 ;mov   [es:fUmbTiny],1
36173 ;pop   es
36174
36175 0000302D 2EC606[FA23]01 mov     byte [cs:fUmbTiny],1
36176
36177 00003033 EBDE jmp     short pv10 ; possibly even "/L...".
36178
36179 pv30: ;cmp  al,'L'

```



```

36180 00003035 3C4C      cmp     al,optL      ; If it's not 'L' either, then 'tis a bad
36181 00003037 750D      jne     short pvE1      ; switch!
36182
36183      ;call    incArgc      ; If it's /L, it's another arg for LH to skip.
36184 00003039 2EFE06[3024] inc     byte [cs:fm_argc] ; 19/04/2019
36185
36186 0000303E E80E00      call    parseL
36187 00003041 73D0      jnc     short pv10      ; If no carry, go back and look for more
36188
36189 00003043 4E        dec     si              ; Else, back up and exit.
36190 00003044 EB03      jmp     short pvErr      ; AX has already been set by parseL
36191
36192 pvE1:      ;mov     ax,3
36193 00003046 B80300      mov     ax,PV_InvSwT      ; Unrecognized switch passed
36194 00003049 4E        dec     si
36195 0000304A 4E        dec     si
36196 0000304B F9        stc
36197 pvX: ;popreg<es, ds, di>
36198 0000304C 07        pop     es
36199 0000304D 1F        pop     ds
36200      ; 01/01/2023
36201      ;pop     di ; * ; (not required) ; 01/01/2023
36202 0000304E C3        retn
36203
36204
36205      ;-----
36206      *** parseL - parses ":nnnn[,nnnn][;nnnn[,nnnn]]*" for ParseVar
36207      ;-----
36208      ENTRY:      ES:SI points to colon
36209      EXIT:      ES:SI points to first character not parsed
36210      ERROR:      Carry set; rewind three characters and return (see ParseVar)
36211      USES:      ES:SI, flags, AX, CX, DX, variables in highvar.inc
36212      ;-----
36213      ; If the string here is terminated with anything other than whitespace or a
36214      ; switchchar (perhaps it's /S or another /L:... ), then we return with carry
36215      ; set, indicating that they've screwed up the syntax. The 3-character rewind
36216      ; makes sure the app /L: is reported as being the culprit.
36217      ;-----
36218
36219 0000304F AC        lodsb
36220 00003050 3C3A      cmp     al,':'        ; Make sure they did /L:
36221 00003052 754E      jne     short pLE1      ; If they didn't, return with carry set.
36222
36223      ;-----
36224      ; PL10--ES:SI = a UMB number, after /L: or ;
36225      ;-----
36226
36227 00003054 E8DB00      call    GetXNum      ; After this, 'tis "size" or ";umb" or " mod"
36228 00003057 724F      jc     short pLE2      ; And error if it's a bad number.
36229 00003059 E89D01      call    convUMB      ; Convert any address to a UMB number
36230
36231 0000305C 88C1      mov     cl,al          ; Remember the UMB number
36232 0000305E E87600      call    stowUMB      ; Mark this UMB # as used;
36233 00003061 7245      jc     short pLE2      ; If it was already marked, it'll error
36234
36235      ;call    incArgc      ; Each UMB number is another arg for LH to skip
36236 00003063 2EFE06[3024] inc     byte [cs:fm_argc] ; 08/04/2019 - Retro DOS v4.0
36237
36238 00003068 AC        lodsb
36239 00003069 3C3B      cmp     al,':'        ; Did "umb;" ?
36240 0000306B 74E7      je     short pL10      ; Yep: go back and get another UMB.
36241
36242 0000306D E84900      call    iswhite      ; Did "umb " ?
36243 00003070 743B      jz     short plX      ; Yep: return (it'll go back to whitespace)
36244
36245 00003072 E83900      call    isEOL        ; Did "umb" ?
36246 00003075 7435      jz     short pLSwX     ; If so, backup and exit like everything's ok
36247
36248      ;cmp     al, '/'
36249 00003077 3C2F      cmp     al,SWTCH      ; Did "umb/" ? (as in, "/L:1,100;2/S")
36250 00003079 7431      je     short pLSwX     ; If so, back up ES:SI one character and return
36251
36252 0000307B 3C2C      cmp     al,','        ; Did "umb," ?
36253 0000307D 7523      jne     short pLE1      ; Just what the heck DID they do? Return error.
36254
36255      ; --- Read a size -----
36256
36257 0000307F E8B000      call    GetXNum      ; Stop on "size;" or "size " or anything else
36258 00003082 721E      jc     short pLE1      ; And error if it's a bad size.
36259
36260 00003084 E81601      call    toPara        ; Convert from bytes to paragraphs
36261
36262 00003087 E8CE01      call    stowSiz       ; CL still has the UMB number for this routine
36263
36264      ;call    incArgc      ; Each UMB size is another arg for LH to skip
36265 0000308A 2EFE06[3024] inc     byte [cs:fm_argc] ; 08/04/2019 - Retro DOS v4.0
36266
36267 0000308F AC        lodsb
36268 00003090 3C3B      cmp     al,':'        ; They did "umb,size;", so get another UMB.
36269 00003092 74C0      je     short pL10      ;
36270
36271 00003094 E82200      call    iswhite      ; Did it end with whitespace?
36272 00003097 7414      jz     short plX      ; If so, we're done here--go back.
36273
36274 00003099 E81200      call    isEOL        ; Did they do "umb,size" and end??? (stupid)
36275 0000309C 740E      jz     short pLSwX     ; If so, backup and exit like everything's ok
36276
36277      ;cmp     al, '/'
36278 0000309E 3C2F      cmp     al,SWTCH      ; Did they do "umb,size/" ?
36279 000030A0 740A      je     short pLSwX     ; If so, again, we're done here.
36280
36281 pvE1:      ;mov     ax,1
36282 000030A2 B80100      mov     ax,PV_InvArg      ; If not, we don't know WHAT they did...
36283 000030A5 4E        dec     si
36284 000030A6 F9        stc
36285 000030A7 C3        retn
36286
36287 pvE2:      ;mov     ax,2
36288 000030A8 B80200      mov     ax,PV_BadUMB      ; In this case, they've specified a UMB twice
36289      ; 12/12/2022
36290      ; cf=1
36291      ;stc
36292      retn
36293
36294 pLSwX:      dec     si              ; If we hit a '/' character, back up one char
36295      ; so the whitespace checker will see it too.
36296
36297 plX: ; 12/12/2022
36298      ; cf=0
36299      ;clc
36300      ; Then just return with carry clear, so
36301      ; ParseVar will go about its business.
36302      retn
36303
36304      ;-----
36305      *** incArgc - increments fm_argc, for use with LoadHigh command-line parsing
36306      ;-----

```

```

36304 ; ENTRY:      None
36305 ; EXIT:       None
36306 ; ERROR:      None
36307 ; USES:       fm_argc, flags
36308 ; -----
36309
36310 ;incArgc:
36311 ;push  ax
36312
36313 ;;;getdata al, fm_argc ; Obtain previous value of fm_argc,
36314
36315 ;mov  al,[cs:fm_argc]
36316
36317 ;inc  al ; Increment it,
36318
36319 ;;;putdata fm_argc, al ; And store it right back.
36320
36321 ;mov  [cs:fm_argc],al
36322
36323 ;pop  ax
36324 ;retn
36325
36326 ;-----
36327 ;*** isEOL - returns with ZF set if AL contains CR or LF, or 0
36328 ;-----
36329 ; ENTRY:      AL contains character to test
36330 ; EXIT:       ZF set iff AL contains CR or LF, or 0
36331 ; ERROR:      None
36332 ; USES:       ZF
36333 ;-----
36334
36335 isEOL:
36336 cmp    al,0 ; Null-terminator
36337 je     short iex
36338 cmp    al,CR ; 0Dh ; Carriage Return
36339 je     short iex
36340 cmp    al,LF ; 0Ah ; LineFeed
36341 iex:
36342 retn
36343
36344 ;-----
36345 ;*** iswhite - returns with ZF set if AL contains whitespace (or "=")
36346 ;-----
36347 ; ENTRY:      AL contains character to test
36348 ; EXIT:       ZF set iff AL contains space, tab, or equals
36349 ; ERROR:      None
36350 ; USES:       ZF
36351 ;-----
36352
36353 iswhite:
36354 cmp    al,' ' ; Space
36355 je     short iwX
36356 cmp    al,'=' ; Equals (treat as whitespace)
36357 je     short iwX
36358 cmp    al,tab ; 9 ; Tab
36359 iwX:
36360 retn
36361
36362 ;-----
36363 ;*** unMarkUMB - marks a given UMB as unused, even if previously marked used
36364 ;-----
36365 ; ENTRY:      AL contains UMB number
36366 ; EXIT:       None
36367 ; ERROR:      None
36368 ; USES:       Flags, variables in highvar.inc
36369 ;-----
36370
36371 ; 02/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
36372
36373 unMarkUMB:
36374 ; 02/01/2023
36375 ;push  ax
36376 ;push  bx
36377 ;push  di
36378 ;push  es
36379 ;
36380 ;push  cs
36381 ;pop   es
36382
36383 xor    ah,ah
36384 mov    bx,ax
36385
36386 ; 19/04/2019
36387
36388 ;;;mov  byte [es:bx+UmbUsed],0
36389 ;mov    [es:bx+UmbUsed],ah ; 0
36390 ; 02/01/2023
36391 ; ds= cs
36392 ;mov    [cs:bx+UmbUsed],ah ; 0
36393 mov     [bx+UmbUsed],ah ; 0
36394
36395 cmp     [UmbLoad],al
36396 ;cmp    [cs:UmbLoad],al
36397 ;;;cmp  [es:UmbLoad],al
36398 jne     short umu10
36399
36400 ;;;mov  [es:UmbLoad],0 ; If unmarked the load UMB, load into convent.
36401 ;mov    [es:UmbLoad],ah ; 0
36402 ; 02/01/2023
36403 ; ds = cs
36404 ;mov    [cs:UmbLoad],ah ; 0
36405 mov     [UmbLoad],ah ; 0
36406 umu10:
36407 ;pop    es
36408 ;pop    di
36409 ;pop    bx
36410 ;pop    ax
36411 retn
36412
36413 ;-----
36414 ;*** stowUMB - marks a given UMB as used, if it hasn't been so marked before
36415 ; -- accepts a UMB # in AL, and makes sure it hasn't yet been
36416 ; listed in the /L:... chain. If it's the first one specified, it sets UmbLoad
36417 ; to that UMB #... and in any case, it marks the UMB as specified.
36418 ;-----
36419 ; ENTRY:      AL contains UMB number, as specified by the user
36420 ; EXIT:       None
36421 ; ERROR:      Carry set if UMB # is less than 0 or >= MAXUMB (see highvar.inc)
36422 ; USES:       AX, Flags, variables in highvar.inc
36423 ;-----
36424
36425 ; 01/01/2023 - Retro DOS v4.2
36426 stowUMB:
36427 cmp     al,MAXUMB ; 16

```



```

36428 000030D9 7202      jb      short su10
36429 000030DB F9        stc
36430 000030DC C3        retn          ; Ooops-- UMB>=MAXUMB
36431 su10:
36432      ; 01/01/2023
36433      ;push  bx
36434      ;push  di
36435      ;push  si
36436      ;push  ds
36437      ;push  es
36438      ;push  cs
36439      ;pop   es
36440      ;push  cs
36441      ;pop   ds
36442
36443      ; 01/01/2023
36444      ; ds <> cs
36445      ;cmp   byte [cs:UmbLoad],0FFh
36446 000030DD 2E803E[FD23]FF cmp     byte [cs:UmbLoad],UNSPECIFIED
36447      ; If this, we haven't been here before
36448 000030E3 7504      jne     short su20
36449 000030E5 2EA2[FD23] mov     [cs:UmbLoad],al      ; So remember this UMB as the load UMB slot.
36450
36451      ;;cmp   byte [UmbLoad],0FFh
36452      ;cmp   byte [UmbLoad],UNSPECIFIED ; If this, we haven't been here before
36453      ;jne     short su20
36454      ;mov     [UmbLoad],al      ; So remember this UMB as the load UMB slot.
36455 su20:
36456 000030E9 08C0      or      al,al      ; If they gave UMB 0, there's really nothing
36457 000030EB 740E      jz      short su30      ; that we should do here.
36458
36459      ;mov     bl,al
36460      ;xor     bh,bh
36461      ;mov     ax,1          ; Now, AX = 1, and BX = UMB Number
36462      ; 01/01/2023
36463 000030ED 30E4      xor     ah,ah
36464 000030EF 89C3      mov     bx,ax
36465 000030F1 B001      mov     al,1
36466
36467      ;xchg    [es:bx+Umbused],al
36468      ; 01/01/2023
36469 000030F3 2E8687[FE23] xchg    [cs:bx+Umbused],al
36470
36471      ;or      al,al      ; If it was already 1, then al==1... and that
36472      ;jz      short su30      ; means an error.
36473      ;
36474      ;stc          ; OOPS! This one's been used before. :(
36475
36476      ; 01/01/2023
36477 000030F8 3C01      cmp     al,1
36478 000030FA F5        cmc      ; if al > 0 -> cf = 1
36479 su30:
36480      ; 01/01/2023
36481      ;pop     es
36482      ;pop     ds
36483      ;pop     si
36484      ;pop     di
36485      ;pop     bx
36486 000030FB C3        retn
36487
36488      ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
36489 %if 0
36490      ;
36491      ;-----
36492      ;*** stowsiz - marks a given UMB as having a given minimum size
36493      ;-----
36494      ; ENTRY:    CL contains UMB number, AX contains size
36495      ; EXIT:      None
36496      ; ERROR:     None
36497      ; USES:      AX, DX, Flags, variables in highvar.inc
36498      ;-----
36499
36500      ; 13/05/2019
36501
36502      ; 01/01/2023 - Retro DOS v4.2
36503 stowsiz:
36504      ; 01/01/2023
36505      ;push  bx
36506      ;;push  di ; ?
36507      ;push  es
36508
36509      ;push  cs
36510      ;pop   es
36511
36512      mov     bl,cl          ; Now bl==UMB number, AX==size
36513      mov     bh,0          ;      bx==UMB number, AX==size
36514      shl     bl,1          ;      bx==offset into array, AX=size
36515      ;mov     [es:bx+UmbSize],ax      ; Store the size
36516      ; 01/01/2023
36517      mov     [cs:bx+UmbSize],ax      ; Store the size
36518
36519      ; 01/01/2023
36520      ;pop     es
36521      ;;pop  di ; ?
36522      ;pop     bx
36523      retn
36524 %endif
36525
36526      ; -----
36527      ;*** toDigit - converts a character-digit to its binary counterpart
36528      ; -- verifies that CL contains a valid character-digit; if so, it
36529      ; changes CL to its counterpart binary digit ((CL-'0') or (CL-'A'+10)).
36530      ; A-F are considered valid iff gnradox is 16.
36531      ; -----
36532      ; ENTRY:    CL contains a digit ('0' to '9' or, if gnradox==16, 'A' to 'F')
36533      ; EXIT:      CL contains digit in binary (0 to 9 or, if gnradox==16, 0 to 15)
36534      ; ERROR:     Carry set indicates invalid digit; carry clear indicates good digit
36535      ; USES:      CL, Flags
36536      ; -----
36537      ; If the string is preceeded with "0x", the value is read as hexadecimal; else,
36538      ; as decimal. After a read, you may check the radix by examining gnradox--it
36539      ; will be 10 or 16.
36540      ; -----
36541
36542 gnradox:
36543 000030FC 0000      dw      0          ; Must be a word--16x16 multiplication
36544
36545 toDigit:
36546 000030FE 2E833E[FC30]10 cmp     word [cs:gnradix],16
36547 00003104 751C      jne     short td20      ; Don't check hex digits if radix isn't 16
36548
36549 toDigit_hex:
36550 00003106 80F961      cmp     cl,'a'      ; 61h
36551 00003109 7209      jb      short td10

```

```

36552 0000310B 80F966      cmp     cl,'f' ; 66h
36553 0000310E 7720      ja      short tdE      ; Nothing valid above 'z' at all...
36554 00003110 80E957      sub     cl,'a'-10 ; 57h      ; Make 'a'==10 and return.
36555                      ;clc      ; <- CLC is implicit from last SUB
36556 00003113 C3      retn
36557
36558 00003114 80F941      cmp     cl,'A' ; 41h
36559 00003117 7209      jb      short td20      ; Below 'A'? Not a letter...
36560 00003119 80F946      cmp     cl,'F' ; 46h
36561 0000311C 7712      ja      short tdE      ; Above 'F'? Not a digit.
36562 0000311E 80E937      sub     cl,'A'-10 ; 37h      ; Make 'A'==10 and return.
36563                      ;clc      ; <- CLC is implicit from last SUB
36564 00003121 C3      retn
36565
36566
36567 00003122 80F930      cmp     cl,'0'      ; If less than zero,
36568                      ;jb      short tdE      ; Done.
36569 00003125 720A      jb      short tdEr ; 08/04/2019
36570 00003127 80F939      cmp     cl,'9'      ; Or, if greater than nine,
36571 0000312A 7704      ja      short tdE      ; Done.
36572 0000312C 80E930      sub     cl,'0' ; 30h      ; Okay--make '0'==0 and return.
36573                      ;clc      ; <- CLC is implicit from last SUB
36574 0000312F C3      retn
36575
36576 00003130 F9      stc
36577
36578 00003131 C3      tdEr:      ; 08/04/2019 - Retro DOS v4.0
36579                      retn
36580
36581
36582
36583
36584
36585
36586
36587
36588
36589
36590
36591
36592
36593
36594
36595
36596
36597
36598
36599
36600
36601
36602 00003132 51
36603
36604
36605 00003133 FC
36606 00003134 31C0
36607 00003136 31DB
36608 00003138 31C9
36609 0000313A 31D2
36610
36611 0000313C 2EC706[FC30]0A00      mov     word [cs:gnradix],10 ; And default to a radix of 10 (dec)
36612
36613 00003143 268A0C      mov     cl,[es:si]      ; Now AX=0, BX=0, CH=0/CL=char, DX=0
36614                      ;call    toDigit
36615 00003146 E8D9FF      call    toDigit_dec
36616                      ;jc      short gxnE      ; If it's not a digit, leave now.
36617                      ; 01/01/2023
36618 00003149 7233      jc      short gxnX
36619
36620 0000314B 08C9      or      cl,cl
36621 0000314D 7517      jnz     short gxn20      ; Doesn't have '0x'
36622 0000314F 268A4C01      mov     cl,[es:si+1]
36623 00003153 80F978      cmp     cl,'x'      ; Either 'x'...
36624 00003156 7405      je      short gxn10
36625 00003158 80F958      cmp     cl,'X'      ; ...or 'X' means it's hexadecimal
36626 0000315B 7509      jne     short gxn20
36627
36628
36629 0000315D 2EC706[FC30]1000      gxn10:      mov     word [cs:gnradix], 16
36630 00003164 46      inc     si      ; Since we read "0x", march over it.
36631 00003165 46      inc     si
36632
36633
36634
36635
36636
36637
36638
36639
36640
36641 00003166 268A0C
36642 00003169 46
36643
36644 0000316A E891FF
36645 0000316D 720D
36646
36647 0000316F E80E00
36648 00003172 720A
36649
36650 00003174 01C8      add     ax,cx      ; Add the digit
36651 00003176 11DA      adc     dx,bx      ; (BX is 0!)--Adds 1 iff last add wrapped
36652                      ;jc      short gxnX      ; If _that_ wrapped, it's too big.
36653                      ;jmp     short gxn20
36654 00003178 73EC      jnc     short gxn20
36655
36656
36657 0000317A EB02      gxnE:      ;stc      ; In this case, we need to set the carry
36658                      jmp     short gxnX      ; and leave--there were no digits given.
36659 0000317C 4E      gxnQ:      dec     si      ; Don't read in the offensive character.
36660 0000317D F8      clc      ; And clear carry, so they know it's okay.
36661
36662
36663
36664 0000317E 59
36665
36666 0000317F C3
36667
36668
36669
36670
36671
36672
36673
36674
36675

```

```

;-----
;*** GetXNum - reads a 32-bit ASCII number at ES:SI and returns it in DX:AX
;-----
; ENTRY:  ES:SI points to an ascii string to scan
; EXIT:   ES:SI moved to first invalid digit, DX:AX contains value read
; ERROR:  Carry set if # is too big, or has no digits (EOL possibly)
; USES:   ES:SI, DX, AX, Flags, gnradox
;-----
; If the string is preceeded with "0x", the value is read as hexadecimal; else,
; as decimal. After a read, you may check the radix by examining gnradox--it
; will be 10 or 16.
;-----

; 08/04/2019 - Retro DOS v4.0

; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:3109h)

GetXNum:
;pushreg <bx, cx, ds>
; 01/01/2023
;push  bx
;push  cx ; *
;push  ds

    cld
    xor  ax,ax
    xor  bx,bx
    xor  cx,cx
    xor  dx,dx      ; Start with 0 (makes sense)

    mov  word [cs:gnradix],10 ; And default to a radix of 10 (dec)

    mov  cl,[es:si]      ; Now AX=0, BX=0, CH=0/CL=char, DX=0
    ;call toDigit
    call toDigit_dec
    ;jc  short gxnE      ; If it's not a digit, leave now.
    ; 01/01/2023
    jc   short gxnX

    or   cl,cl
    jnz  short gxn20      ; Doesn't have '0x'
    mov  cl,[es:si+1]
    cmp  cl,'x'      ; Either 'x'...
    je   short gxn10
    cmp  cl,'X'      ; ...or 'X' means it's hexadecimal
    jne  short gxn20

gxn10:
    mov  word [cs:gnradix], 16
    inc  si      ; Since we read "0x", march over it.
    inc  si

;-----
; GxN20--ES:SI = a digit in a number; if not, we're done
; DX:AX = current total
; BX = 0
; CH = 0
;-----

gxn20:
    mov  cl,[es:si]      ; Now DX:AX=current total, CH=0/CL=char
    inc  si

    call toDigit      ; Accepts only valid digits, A-F -> 10-16
    jc   short gxnQ      ; <- Ah... wasn't a digit. Stop.

    call mul32      ; Multiply DX:AX by gnradox
    jc   short gxnX      ; (if it's too big, error out)

    add  ax,cx      ; Add the digit
    adc  dx,bx      ; (BX is 0!)--Adds 1 iff last add wrapped
    ;jc  short gxnX      ; If _that_ wrapped, it's too big.
    ;jmp short gxn20
    jnc  short gxn20

gxnE:
    ;stc      ; In this case, we need to set the carry
    jmp  short gxnX      ; and leave--there were no digits given.

gxnQ:
    dec  si      ; Don't read in the offensive character.
    clc      ; And clear carry, so they know it's okay.

gxnX:
    ; 01/01/2023
    ;pop  ds
    pop  cx ; *
    pop  bx
    retn

;-----
;*** mul32 - multiplies the number in DX:AX by gnradox
;-----
; ENTRY:  DX:AX = the number to be multiplied, BX = 0, gnradox = multiplier
; EXIT:   DX:AX has been multiplied by gnradox if carry clear; BX still 0
; ERROR:  Carry set if number was too large
; USES:   Flags, AX, DX
;-----

```

```

36676
36677
36678 00003180 50
36679 00003181 89D0
36680 00003183 2EF726[FC30]
36681 00003188 7211
36682
36683 0000318A 89C2
36684 0000318C 58
36685
36686 0000318D 87D3
36687 0000318F 2EF726[FC30]
36688 00003194 87D3
36689 00003196 01DA
36690 00003198 31DB
36691 0000319A C3
36692
36693 0000319B 58
36694 0000319C C3
36695
36696
36697
36698
36699
36700
36701
36702
36703
36704
36705
36706
36707
36708
36709
36710 0000319D 51
36711
36712 0000319E B104
36713 000031A0 D3E8
36714 000031A2 92
36715 000031A3 B10C
36716 000031A5 D3E0
36717 000031A7 09D0
36718
36719 000031A9 59
36720 000031AA C3
36721
36722
36723
36724
36725
36726
36727
36728
36729
36730
36731
36732
36733
36734
36735
36736
36737
36738
36739
36740
36741
36742
36743
36744
36745
36746
36747
36748 000031AB B452
36749 000031AD CD21
36750
36751 000031AF 26A18C00
36752
36753
36754 000031B3 83F8FF
36755 000031B6 F5
36756
36757 000031B7 C3
36758
36759
36760
36761
36762
36763
36764
36765
36766
36767
36768
36769
36770
36771
36772
36773
36774
36775
36776
36777
36778
36779
36780
36781
36782
36783
36784
36785
36786
36787
36788
36789
36790
36791
36792
36793
36794
36795
36796 000031B8 26833E010008
36797 000031BE 7507
36798
36799

mul32:
    push    ax                ; DX=old:hi, AX=old:lo, TOS=old:lo, BX=0
    mov     ax,dx             ; DX=old:hi, AX=old:hi, TOS=old:lo, BX=0
    mul     word [cs:gnradix] ; DX=?, AX=new:hi, TOS=old:lo, BX=0
    jc      short m32E        ; Too big?

    mov     dx,ax             ; DX=new:hi, AX=new:hi, TOS=old:lo, BX=0
    pop     ax                ; DX=new:hi, AX=old:lo, TOS=orig, BX=0

    xchg    dx,bx             ; DX=0, AX=old:lo, TOS=orig, BX=new:hi
    mul     word [cs:gnradix] ; DX=carry, AX=new:lo, TOS=orig, BX=new:hi
    xchg    dx,bx             ; DX=new:hi, AX=new:lo, TOS=orig, BX=carry
    add     dx,bx             ; DX=new:hi, AX=new:lo, TOS=orig, BX=carry
    xor     bx,bx             ; DX=new:hi, AX=new:lo, TOS=orig, BX=0
    retn

m32E:
    pop     ax
    retn

;-----
;*** toPara - divides DX:AX by 16; result in AX only (discards extra DX data)
;-----
; ENTRY:  DX:AX = the number to be divided
; EXIT:   Interpereting DX:AX as bytes, AX=paragraph equivalent, 0xFFFF max
; ERROR:  None
; USES:   Flags, AX, DX
;-----
; Note: The 386 has a 32-bit SHR, which would work perfectly for this... but we
;       can't ensure a 386 host machine. Sorry.
;-----

; 01/01/2023 - Retro DOS v4.2
toPara:
    push    cx                ; DX:AX=HHHH hhhh hhhh hhhh:LLLL 1111 1111 1111
    mov     cl,4              ;
    shr     ax,cl             ; DX:AX=HHHH hhhh hhhh hhhh:0000 LLLL 1111 1111
    xchg    ax,dx             ; DX:AX=0000 LLLL 1111 1111:HHHH hhhh hhhh hhhh
    mov     cl,12             ;
    shl     ax,cl             ; DX:AX=0000 LLLL 1111 1111:hhhh 0000 0000 0000
    or      ax,dx             ; AX=hhhh LLLL 1111 1111
    pop     cx
    retn

;-----
;*** UmbHead - returns in AX the address of the first UMB block (0x9FFF)
;-----
; ENTRY:  Nothing
; EXIT:   AX contains 0x9FFF for most systems
; ERROR:  Carry set if pointer is 0xFFFF (if not set up yet--DH runs into this)
; USES:   Flags, AX
;-----
; Early in the boot-cycle, the pointer used to obtain this value isn't set up;
; to be precise, before a UMB provider is around. In this event, the pointer
; is always set to 0xFFFF; it changes once a provider is around. On most
; machines (all of 'em I've seen), it changes to 0x9FFF at that point.
;-----

; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
UmbHead:
; 13/05/2019 (because of callers, pushes & pops are not needed here)

;push    si ; ?
;push    ds ; ?
;push    es
;push    bx ; *

; 09/04/2019
; !!! No need to save es,bx,ds,si above !!! (es,bx are changed here)

    mov     ah,GET_IN_VARS    ; Call int 21h, function 52h...
    int     21h

    mov     ax,[es:DOS_UMB_HEAD] ; And read what's in ES:[008C]

; 01/01/2023
    cmp     ax,0FFFFh
    cmc
; if AX=0FFFFh -> CF=1
    retn

; 01/01/2023
; %if 0
;     cmp     ax,0FFFFh
;     je      short uhE          ; If it's 0xFFFF, it's an error...
;
;     cllc
;     jmp     short uhX          ; Else, it isn't (CLC done by prev cmp)
;
;     ; 12/12/2022
;     retn
; uhE:
;     stc
; uhX:
;     pop     bx ; *
;     pop     es
;     pop     ds ; ?
;     pop     si ; ?
;     retn
; %endif

;-----
;*** isSysMCB - sets ZF if ES points to an MCB owned by "SC" + (8 or 9)
;-----
; ENTRY:  ES:0 should point to a valid MCB
; EXIT:   ZF set if owned by SC+8 or SC+9 (for japan)
; USES:   Flags
;-----

isSysMCB:
    push    ax

    mov     ax,[es:ARENA.OWNER] ; Check the owner...
    cmp     ax,SystemPSPOwner   ; 8 (for US OR Japan) is valid
    je      short ism10
    cmp     ax,JapanPSPOwner    ; 9 (for Japan) is valid
    je      short ism10
    jmp     short ismX          ; Anything else isn't.
    jne     short ismX
    cmp     word [es:ARENA.OWNER],SystemPSPOwner ; 8 ; 09/04/2019
    jne     short ismX
ism10:
    mov     ax,[es:ARENA.NAME] ; Check the name...

```

```

36800      ;cmp    ax,'SC' ; 4353h
36801 000031C0 26813E08005343      cmp    word [es:ARENA.NAME],'SC'
36802 ismX:
36803      ;pop     ax
36804 000031C7 C3      retn
36805
36806      ; 09/04/2019 - Retro DOS v4.0
36807
36808      ; -----
36809      ; *** AddrToUmb - converts a segment address in AX to its appropriate UMB number
36810      ; -----
36811      ; ENTRY:  AX contains a segment address
36812      ; EXIT:   AX will contain the UMB number which contains the address (0==conv)
36813      ; ERROR:  If the address is above UM Range, AX will return as FFFF.
36814      ; USES:   Flags, AX
36815      ; -----
36816      ; An address in the following areas is treated as:
36817      ; 0 <-> umbhead (0x9FFF) = Conventional memory
36818      ; 0x9FFF <-> addr of first UM sys MCB = UMB #1
36819      ; ...
36820      ; addr of last UM sys MCB <-> TOM = invalid; returns #0xFFFF
36821      ; -----
36822
36823      ; 01/01/2023 - Retro DOS v4.2
36824 AddrToUmb:
36825      ; 01/01/2023
36826      ;push    cx
36827      ;push    dx
36828 000031C8 06      push    es
36829
36830 000031C9 89C2      mov     dx,ax      ; DX = address to search for
36831
36832 000031CB E8DDFF      call    UmbHead    ; AX = first segment
36833 000031CE 7222      jc     short atuE    ; If it couldn't get it, error out.
36834
36835      ; 22/07/2023
36836      ;mov     es,ax ; *      ; ES = first UMB segment
36837 000031D0 31C9      xor     cx,cx ; 0      ; Pretend we're on UMB 0 for now... (cx = UMB#)
36838
36839      ; 22/07/2023
36840 atu10:
36841 000031D2 8EC0      mov     es,ax ; * ; ** ; 22/07/2023
36842      ; -----
36843      ; ATU10--ES - Current MCB address
36844      ; DX - Address given for conversion
36845      ; CX - Current UMB #
36846      ; -----
36847
36848      ;atu10:
36849      ;mov     ax,es ; * ; 18/07/2023
36850 000031D4 39D0      cmp     ax,dx      ; Present segment >= given segment?
36851 000031D6 731D      jae     short atuX    ; Yep--done.
36852
36853 000031D8 E8DDFF      call    isSysMCB    ; Returns with ZF set if this is a system MCB
36854 000031DB 7501      jnz     short atu20
36855
36856 000031DD 41      inc     cx      ; If it _was_ a system MCB, we're in a new UMB.
36857 atu20:
36858      ;mov     al,[es:ARENA.SIGNATURE]
36859      ;cmp     al,arena_signature_end ; 'z'
36860      ; 22/07/2023
36861      ; ax = es
36862      ;mov     ax,es ; **
36863 000031DE 2603060300      add     ax,[es:ARENA.SIZE]
36864 000031E3 26803E00005A      cmp     byte [es:ARENA.SIGNATURE],arena_signature_end
36865 000031E9 7403      je     short atu30    ; 'z' means this was the last MCB... that's it.
36866
36867      ;NextMCB es,ax
36868
36869      ;mov     ax,es ; **
36870      ;add     ax,[es:3]
36871      ;add     ax,[es:ARENA.SIZE]
36872 000031EB 40      inc     ax
36873      ; 22/07/2023
36874      ;mov     es,ax ; *
36875 000031EC EBE4      jmp     short atu10
36876
36877      ; -----
36878      ; if we get to atu30, they specified a number that was past the last MCB.
36879      ; make sure it's not _inside_ that MCB before we return an error condition.
36880      ; -----
36881
36882 atu30:
36883      ; 22/07/2023
36884      ; ax = es + [es:ARENA.SIZE]
36885      ;mov     ax,es ; **
36886      ;add     ax,[es:ARENA.SIZE] ; **
36887 000031EE 39D0      cmp     ax,dx      ; Present >= given?
36888 000031F0 7303      jae     short atuX    ; Yep! It _was_ inside.
36889
36890 atuE:
36891 000031F2 31C9      xor     cx,cx ; 0      ; Else, fall through with UMB # == -1
36892 000031F4 49      dec     cx      ; (that makes it return 0xFFFF and sets CF)
36893 atuX:
36894 000031F5 89C8      mov     ax,cx      ; Return the UMB number in AX
36895
36896      pop     es
36897      ; 01/01/2023
36898      ;pop     dx
36899 000031F8 C3      pop     cx
36900      retn
36901
36902      ; -----
36903      ; *** convUMB - checks after GetXNum to convert an address to a UMB number
36904      ; -- if GetXNum read a hex number, we interperete that as a segment
36905      ; address rather than a UMB number... and use that address to look up a UMB.
36906      ; This routine checks for that condition and calls AddrToUmb if necessary.
36907      ; -----
36908      ; ENTRY:  AX contains a UMB number or segment, gnradiX has been set by GetXNum
36909      ; EXIT:   AX will contain a UMB number
36910      ; ERROR:  None
36911      ; USES:   Flags, AX
36912      ; -----
36913
36914      ; 01/01/2023 - Retro DOS v4.2
36915 convUMB:
36916 000031F9 2E833E[FC30]10      cmp     word [cs:gnradix],16
36917 000031FF 7507      jne     short cu10    ; If it didn't read in hex, it's not an address
36918 00003201 E8C4FF      call    AddrToUmb    ; Else, convert the address to a UMB number
36919      ;cmp     ax,0FFFFh
36920      ;jne     short cu10
36921      ;inc     ax      ; If too high, ignore it (make it conventional)
36922      ; 01/01/2023
36923 00003204 40      inc     ax
36924 00003205 7401      jz     short cu10    ; If too high, ignore it (make it conventional)

```

```

36924 00003207 48      dec     ax
36925 cu10:
36926 00003208 c3      retn
36927
36928 ; 01/01/2023 - Retro DOS v4.2
36929 ;%if 0
36930
36931 ;-----
36932 ;*** setUMBs - links umbs and sets allocation strategy for a load
36933 ;-- if LoadHigh, the allocation strategy MAY be LOW_FIRST instead
36934 ;of the usual HIGH_FIRST. See the code.
36935 ;-----
36936 ; ENTRY:  None
36937 ; EXIT:   None
36938 ; ERROR:  None
36939 ; USES:   Flags, fm_umb, fm_strat
36940 ;-----
36941
36942 ;setUMBs:
36943 ; push    ax
36944 ; push    bx
36945 ; call    fm_link
36946 ; pop     bx
36947 ; pop     ax
36948 ; retn
36949
36950 ;%endif
36951
36952 ; 18/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
36953 ; loadLow subroutine is not used anywhere of IO.SYS 6.22 (& 5.0)
36954 ;%if 0
36955
36956 ;-----
36957 ;*** loadLow - returns AL==0 if UMB0 == 0, else AL==1
36958 ;-----
36959 ; ENTRY:  None
36960 ; EXIT:   AL==0 if mem strategy should be set to LOW_FIRST, else AL==1
36961 ; Carry set if UMB0 not specified (_NOT_ an error)
36962 ; ERROR:  None
36963 ; USES:   Flags, fm_strat, fm_umb
36964 ;-----
36965 ; We want to set the memory strategy to LOW_FIRST if the user specified a
36966 ; load UMB, and it is 0. That 0 can be either from the user having _specified_
36967 ; zero (/L:0;...), or from having specified a too-big min size (/L:1,99999999)
36968 ; such that the load UMB is too small, and shouldn't be used.
36969 ;-----
36970
36971 loadLow:
36972 ; push    ds
36973 ; push    cs          ; Point DS into appropriate data segment
36974 ; pop     ds
36975
36976 ; mov     al,[UmbLoad]
36977 ; mov     al,[cs:UmbLoad]
36978 ; cmp     al,UNSPECIFIED ; 0FFh, -1
36979 ; jne     short 1110
36980
36981 ; stc
36982 1115:
36983 ; mov     al,1          ; Return with AL==1 && STC if no UMBs specified
36984 ; stc
36985 ; jmp     short 11X
36986 ; retn
36987
36988 1110:
36989 ; or      al,al          ; AL=the load UMB: Is it == 0?
36990 ; jz      short 11X      ; Yep... CF==0 (from OR) && AL=0, so just exit
36991
36992 ; jnz     short 1115     ; 09/04/2019 - Retro DOS v4.0
36993 ; retn
36994
36995 ; mov     al,1
36996 ; clc
36997 ; 11X:
36998 ; pop     ds          ; Return DS to where it was
36999 ; retn
37000
37001 ;%endif
37002
37003 ;-----
37004 ;*** HideUMBs - links UMBs and hides upper-memory as appropriate
37005 ;-----
37006 ; ENTRY:  None
37007 ; EXIT:   None
37008 ; ERROR:  None
37009 ; USES:   Flags, fm_strat, fm_umb
37010 ;-----
37011
37012 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37013 ; (SYSINIT:322Fh)
37014 HideUMBs:
37015 ; 01/01/2023
37016 ; push    ax
37017 ; push    cx
37018 ; push    ds
37019 ; push    es
37020
37021 ; 01/01/2023
37022 ; ds = cs
37023
37024 call    UmbTest      ; See if we REALLY linked in anything...
37025 jc      short husX    ; ...if not, there's nothing for us to do.
37026
37027 call    FixMem        ; Concatenate adjacent free MCBs in upper mem
37028
37029 ; call    setUMBs      ; Link UMBs and set memory-allocation strategy
37030 ; 01/01/2023
37031 call    fm_link
37032
37033 ; putdata fInHigh,1    ; Remember that we're now running high
37034 ; mov     byte [cs:fInHigh],1
37035 ; 01/01/2023
37036 mov     byte [fInHigh],1
37037
37038 ; call    GetLoadUMB    ; See if they gave us a list to leave free
37039 ; mov     al,[cs:UmbLoad] ; 09/04/2019 - Retro DOS v4.0
37040 ; 01/01/2023
37041 mov     al,[UmbLoad]
37042
37043 ; cmp     al,UNSPECIFIED ; If they didn't,
37044 ; je      short husX    ; then we shouldn't do this loop:
37045
37046 ; xor     cx,cx
37047
37048 ; -----

```

```

37048 ; HUS10-CX - UMB number (after inc, 1==first UMB)
37049 ; -----
37050
37051 00003223 41      hus10:      inc      cx          ; For each UMB:
37052 ; 01/01/2023
37053 00003224 80F910  cmp      cl,MAXUMB
37054 ;cmp      cx,MAXUMB ; 16
37055 00003227 730E      jae      short hus20
37056
37057 00003229 88C8      mov      al,cl          ; (stopping as soon as we're outside of the
37058 0000322B 06      push     es
37059 0000322C E8A200  call     findUMB      ; valid range of UMBs)
37060 0000322F 07      pop      es          ; push/pop: trash what findumb finds. :-)
37061 00003230 7205      jc       short hus20
37062
37063 ; 02/01/2023
37064 ;push     cx ; *
37065 00003232 E84F01  call     _hideUMB_    ; hide what we need to hide.
37066 ;pop      cx ; *
37067
37068 00003235 EBEC      jmp      short hus10
37069
37070 hus20:      ;call    GetLoadUMB      ; Now check if they offered /L:0
37071 ; 01/01/2023
37072 ; ds = cs
37073 ;mov      al,[UmbLoad]
37074 ;mov      al,[cs:UmbLoad] ; 09/04/2019 - Retro DOS v4.0
37075 00003237 800E[FD23]00 or       byte [UmbLoad],0
37076 ;or       al,al      ; --Is the load UMB 0? (-1==unspecified)
37077 0000323C 7503      jnz      short husX    ; If not, we're done.
37078
37079 0000323E E86802  call     hl_unlink    ; If so, however, fix UMBs and strategy.
37080
37081 00003241 07      husX:      pop      es
37082 ; 01/01/2023
37083 ;pop      ds
37084 ;pop      cx
37085 ;pop      ax
37086 00003242 C3      retn
37087
37088 ; -----
37089 *** GetLoadUMB - Returns the load UMB number in AL (-1 if not specified)
37090 ; -----
37091 ; ENTRY:   None
37092 ; EXIT:    AL == load UMB
37093 ; ERROR:   None
37094 ; USES:    Flags, AX
37095 ; -----
37096
37097 ;GetLoadUMB:
37098 ; ;getdata al, UmbLoad
37099 ; push     ds
37100 ; push     cs
37101 ; pop      ds
37102 ; mov      al,[UmbLoad]
37103 ; pop      ds
37104 ; retn
37105
37106 ; -----
37107 *** GetLoadSize - Returns the load UMB minimum size (0 if not specified)
37108 ; -----
37109 ; ENTRY:   None
37110 ; EXIT:    AX == load UMB minimum size
37111 ; ERROR:   None
37112 ; USES:    Flags, AX
37113 ; -----
37114
37115 ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
37116 %if 0
37117 ; 01/01/2023 - Retro DOS v4.2
37118 GetLoadSize:
37119 ; 09/04/2019 - Retro DOS v4.0
37120 ;mov      al,[cs:UmbLoad]
37121 ; 01/01/2023
37122 ; ds = cs
37123 mov      al,[UmbLoad]
37124 ;jmp      short GetSize
37125
37126 ;push     bx
37127 ;push     si
37128 ;push     ds
37129 ;push     cs
37130 ;pop      ds
37131
37132 ;mov      al,[UmbLoad]
37133
37134 ;xor      ah,ah      ; ax==UMB
37135 ;mov      bx,UmbSize ; bx==array
37136 ;shl      al,1       ; ax==offset
37137 ;add      ax,bx      ; ax==element index
37138 ;mov      si,ax      ; ds:si==element index
37139
37140 ;lodsw                      ; hh
37141
37142 ;add      bx,ax
37143 ;mov      ax,[bx]
37144
37145 ;pop      ds
37146 ;pop      si
37147 ;pop      bx
37148 ;retn
37149 %endif
37150
37151 ; -----
37152 *** GetSize - Returns the UMB in AL's minimum size (0 if not specified)
37153 ; -----
37154 ; ENTRY:   AL == a UMB number
37155 ; EXIT:    AX == UMB minimum size, as specified by the user
37156 ; ERROR:   None
37157 ; USES:    Flags, AX
37158 ; -----
37159
37160 ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
37161 GetLoadSize:
37162 ; ds = cs
37163 ;mov      al,[UmbLoad]
37164 ; al = [UmbLoad]
37165 ; ....
37166
37167 ; 01/01/2023 - Retro DOS v4.2
37168 GetSize:
37169 ; 09/04/2019 - Retro DOS v4.0
37170
37171 ;push     bx ; 01/01/2023

```



```

37172         ;push    si
37173         ;push    ds
37174         ;push    cs
37175         ;pop     ds
37176
37177 00003243 30E4        xor     ah,ah           ; ax==UMB
37178 00003245 BB[0E24]   mov     bx,UmbSize      ; bx==array
37179 00003248 D0E0        shl     al,1           ; ax==offset
37180         ;add     ax,bx           ; ax==element index
37181         ;mov     si,ax           ; ds:si==element index
37182
37183         ;lodsw                   ; ax==size
37184
37185 0000324A 01C3        add     bx,ax
37186         ; 01/01/2023
37187         ; ds = cs
37188 0000324C 8B07        mov     ax,[bx]
37189         ;mov     ax,[cs:bx]
37190
37191         ;pop     ds
37192         ;pop     si
37193         ;pop     bx ; 01/01/2023
37194 s1s10:      ; 08/09/2023
37195 0000324E C3         retn
37196
37197 ; -----
37198 ; *** StoLoadUMB - Overrides the load UMB number with what's in AL
37199 ; -----
37200 ; ENTRY:  AL == new load UMB
37201 ; EXIT:   None
37202 ; ERROR:  None
37203 ; USES:   Flags, AX
37204 ; -----
37205 ; CAUTION: Should only be used if /L:... was used. Logically, that is the only
37206 ;           time you would ever need this, so that's okay.
37207 ; -----
37208
37209 ; StoLoadUMB subroutine is not used anywhere
37210 ; of PCDOS 7.1 IBMBIO.COM (& MSDOS 6.21 IO.SYS)
37211 ; Erdogan Tan - 18/07/2023
37212
37213 ;StoLoadUMB:
37214 ; ;putdata UmbLoad, al
37215 ; ;push    es
37216 ; ;push    cs
37217 ; ;pop     es           ; mov [cs:UmbLoad], al !!!! ; 08/09/2023
37218 ; ;mov     [es:UmbLoad],al
37219 ; ;pop     es
37220 ; ;retn
37221
37222 ; -----
37223 ; *** StoLoadSize - Overrides the load UMB minimum size with what's in AX
37224 ; -----
37225 ; ENTRY:  AL == new load size
37226 ; EXIT:   None
37227 ; ERROR:  None
37228 ; USES:   Flags, AX
37229 ; -----
37230 ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
37231 ; 01/01/2023 - Retro DOS v4.2
37232 StoLoadSize:
37233 ; 01/01/2023
37234 ;push    dx
37235
37236 ;getdata dl, UmbLoad      ; Put UMB# in DL and size in AX
37237 ;
37238 ;push    ds
37239 ;push    cs
37240 ;pop     ds
37241 ;mov     dl,[UmbLoad]
37242 ;pop     ds
37243
37244 ; 08/09/2023
37245 ; MSDOS 6.21 IO.SYS - SYSINIT:32B6h
37246 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:3831h
37247
37248 ; ;mov     dl,[UmbLoad]           ; BUG ! CL would/must be used here
37249 ; ;                               ; instead of DL (*) ; 18/07/2023
37250 ;mov     dl,[cs:UmbLoad] ; Retro DOS v4.0, v4.1, v4.2
37251 ;cmp     dl,UNSPECIFIED ; 0FFh
37252 ;je      short s1s10
37253
37254 ; ; BUG ! stowSiz uses CL instead of DL !
37255 ; ; (CL is set in ParseL which calls stowSiz)
37256 ; ; (This BUG existing in PCDOS 7.1 IBMBIO.COM also)
37257 ; ; Erdogan Tan - 18/07/2023
37258
37259 ; 08/09/2023 (BugFix)
37260 ;mov     cl,[cs:UmbLoad]
37261 ; 08/09/2023
37262 ; ds = cs
37263 0000324F 8A0E[FD23]   mov     cl,[UmbLoad]
37264 00003253 80F9FF     cmp     cl,UNSPECIFIED ; 0FFh
37265 00003256 74F6         je      short s1s10
37266
37267 ; 08/09/2023
37268 ; call     stowSiz           ; we've got a function to do just this
37269 ; s1s10:
37270 ; ; 01/01/2023
37271 ; ;pop     dx
37272 ; ;retn
37273
37274 ; 08/09/2023
37275 ; ;jmp     stowSiz
37276 ; ;jmp     short stowSiz
37277
37278 ; 08/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
37279 %if 1
37280 ; -----
37281 ; *** stowSiz - marks a given UMB as having a given minimum size
37282 ; -----
37283 ; ENTRY:  CL contains UMB number, AX contains size
37284 ; EXIT:   None
37285 ; ERROR:  None
37286 ; USES:   AX, DX, Flags, variables in highvar.inc
37287 ; -----
37288
37289 ; 13/05/2019
37290
37291 ; 01/01/2023 - Retro DOS v4.2
37292 stowSiz:
37293 ; 01/01/2023
37294 ;push    bx
37295 ; ;push    di ; ?

```

```

37296             ;push  es
37297
37298             ;push  cs
37299             ;pop   es
37300
37301 00003258 88CB      mov     bl,cl          ; Now bl==UMB number, AX==size
37302 0000325A B700      mov     bh,0           ;          bx==UMB number, AX==size
37303 0000325C D0E3      shl     bl,1           ;          bx==offset into array, AX=size
37304             ;mov    [es:bx+UmbSize],ax    ; Store the size
37305             ; 01/01/2023
37306 0000325E 2E8987[0E24] mov    [cs:bx+UmbSize],ax    ; Store the size
37307
37308             ; 01/01/2023
37309             ;pop   es
37310             ;;pop  di ; ?
37311             ;pop   bx
37312
37313 00003263 C3         retn
37314 %endif
37315
37316 ; -----
37317 ; *** hideUMB - marks as HIDDEN all FREE elements in UMB passed as AL
37318 ; -----
37319 ; ENTRY:    AL must indicate a valid UMB; 0==conv && is invalid.
37320 ; EXIT:     None; free elements in UMB marked as hidden
37321 ; ERROR:    None
37322 ; USES:     Flags
37323 ; -----
37324
37325             ; 01/01/2023 - Retro DOS v4.2
37326 hideUMB:
37327             ; 02/01/2023
37328 00003264 52         push    dx ; (*)
37329             ; 01/01/2023
37330             ;push    ax
37331 00003265 06         push    es
37332
37333 00003266 E86800      call    findUMB ; (*) ; Returns with carry if err, else ES == MCB
37334 00003269 7224      jc      short hux
37335
37336 ; -----
37337 ; HU10--ES - MCB inside UMB; if it's a system MCB,
37338 ;           we're not in the same UMB, so exit.
37339 ; -----
37340
37341 0000326B E84AFF      hu10:   call    isSysMCB ; Returns with ZF set if owner is SYSTEM
37342 0000326E 741F      jz      short hux ; If it is, we've finished the UMB.
37343             ;call    isFreeMCB ; Returns with ZF set if owner is 0
37344 00003270 26830E010000 or     word [es:ARENA.OWNER],0
37345 00003276 7503      jnz     short hu20
37346
37347 00003278 E81700      call    hideMCB
37348 hu20:
37349             ;mov     al,[es:ARENA.SIGNATURE]
37350             ;cmp     al,arena_signature_end ; 'z'
37351             ; 19/07/2023
37352 0000327B 26803E00005A cmp     byte [es:ARENA.SIGNATURE],'z'
37353 00003281 740C      jz      short hux ; 'z' means this was the last MCB... that's it.
37354
37355             ;NextMCB es,ax ; Go on forward.
37356 00003283 8CC0      mov     ax,es
37357             ;add     ax,[es:3]
37358 00003285 2603060300 add     ax,[es:ARENA.SIZE]
37359 0000328A 40         inc     ax
37360 0000328B 8EC0      mov     es,ax
37361
37362 0000328D EBDC      jmp     short hu10
37363 huX:
37364 0000328F 07         pop     es
37365             ; 01/01/2023
37366             ;pop     ax
37367             ; 02/01/2023
37368 00003290 5A         pop     dx ; (*)
37369 00003291 C3         retn
37370
37371             ; 02/01/2023
37372 %if 0
37373
37374 ; -----
37375 ; *** isTiny - returns with ZF set if user didn't specify /s
37376 ; -----
37377 ; ENTRY:    None
37378 ; EXIT:     ZF set if user DIDN'T specify /s
37379 ; ERROR:    None
37380 ; USES:     Flags
37381 ; -----
37382
37383             ; 02/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37384 istiny:
37385             ; 02/01/2023
37386             ;push    ax
37387
37388             ;getdata al,fUmbTiny
37389             ;
37390             ;push    ds
37391             ;push    cs
37392             ;pop     ds
37393             ;mov     al,[fUmbTiny]
37394             ;pop     ds
37395
37396             ; 09/09/2023
37397             ;mov     al,[cs:fUmbTiny]
37398             ; 02/01/2023
37399             ; ds = cs
37400             mov     al,[fUmbTiny]
37401
37402             or     al,al
37403             ; 02/01/2023
37404             ;pop     ax
37405             retn
37406
37407 %endif
37408
37409 ; -----
37410 ; *** isFreeMCB - returns with ZF set if current MCB (ES:0) is FREE
37411 ; -----
37412 ; ENTRY:    ES:0 should point to an MCB
37413 ; EXIT:     ZF set if MCB is free, else !ZF
37414 ; ERROR:    None
37415 ; USES:     Flags
37416 ; -----
37417
37418 ;isFreeMCB:
37419             ; or     word [es:ARENA.OWNER],0

```



```

37420 ; retn
37421
37422 ; -----
37423 ; *** hideMCB - marks as HIDDEN the MCB at ES:0
37424 ; -----
37425 ; ENTRY: ES:0 should point to an MCB
37426 ; EXIT: None; MCB marked as HIDDEN
37427 ; ERROR: None
37428 ; USES: None
37429 ; -----
37430
37431 hideMCB:
37432 mov word [es:ARENA.OWNER],SystemPSPOwner ; 8
37433 mov word [es:ARENA.NAME+0], 'HI' ; 4948h
37434 mov word [es:ARENA.NAME+2], 'DD' ; 4444h
37435 mov word [es:ARENA.NAME+4], 'EN' ; 4E45h
37436 mov word [es:ARENA.NAME+6], ' ' ; 2020h
37437 retn
37438
37439 ; -----
37440 ; *** unHideMCB - marks as FREE the MCB at ES:0
37441 ; -----
37442 ; ENTRY: ES:0 should point to an MCB
37443 ; EXIT: None; MCB marked as FREE
37444 ; ERROR: None
37445 ; USES: None
37446 ; -----
37447
37448 ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37449
37450 unHideMCB:
37451 ; 03/01/2023
37452 ; push ax
37453 mov word [es:ARENA.OWNER],FreePSPOwner ; 0
37454 mov ax, ' ' ; 2020h
37455 mov [es:ARENA.NAME+0],ax
37456 mov [es:ARENA.NAME+2],ax
37457 mov [es:ARENA.NAME+4],ax
37458 mov [es:ARENA.NAME+6],ax
37459 ; 03/01/2023
37460 ; pop ax
37461 retn
37462
37463 ; -----
37464 ; *** findUMB - makes ES:0 point to the first MCB in UMB given as AL
37465 ; -- returns UmbHEAD pointer (0x9FFF) if passed AL==0
37466 ; -----
37467 ; ENTRY: AL should be to a valid UMB number
37468 ; EXIT: ES:0 points to first MCB in UMB (_not_ the 8+SC MCB that heads it)
37469 ; ERROR: Carry set if couldn't reach UMB (too high)
37470 ; USES: Flags, ES
37471 ; -----
37472
37473 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37474 ; (SYSINIT:3344h)
37475
37476 findUMB:
37477 ; 01/01/2023
37478 ; push ax
37479 ; 02/01/2023
37480 push cx ; *
37481 push dx
37482 xor ah,ah ; Zap ah, so al==ax
37483
37484 mov dx,ax ; Store the to-be-found UMB number in DX
37485
37486 call UmbHead ; Returns first UMB segment in AX
37487 ; 22/07/2023
37488 ; mov es,ax ; *
37489 xor cx,cx ; Pretend we're on UMB 0 for now...
37490
37491 ; 22/07/2023
37492
37493 fu10:
37494 mov es,ax ; * ; **
37495 ; -----
37496 ; FU10--CX - This UMB number; 0 == conventional
37497 ; DX - The UMB number they're looking for
37498 ; ES - The current MCB address
37499 ; -----
37500
37501 ; fu10:
37502 cmp cx,dx ; If CX==DX, we've found the UMB we're
37503 je short fux ; searching for--so exit.
37504
37505 call issysMCB ; Returns with ZF set if owner is SYSTEM
37506 jnz short fu20
37507
37508 inc cx ; If it _was_ SYSTEM, we're in a new UMB.
37509
37510 fu20:
37511 ; mov al,[es:ARENA.SIGNATURE]
37512 ; cmp al,arena_signature_end ; 'z'
37513 ; 19/07/2023
37514 cmp byte [es:ARENA.SIGNATURE],arena_signature_end
37515 je short fuE ; 'Z' means this was the last MCB... that's it.
37516 ; NextMCB es,ax ; Go on forward.
37517 ; 22/07/2023
37518 ; ax = es
37519 ; mov ax,es ; * ; 22/07/2023
37520 ; add ax,[es:3]
37521 add ax,[es:ARENA.SIZE]
37522 inc ax
37523 ; 22/07/2023
37524 ; mov es,ax ; **
37525 jmp short fu10
37526
37527 fuE:
37528 stc
37529
37530 fux:
37531 ; 01/01/2023
37532 ; pop dx
37533 ; 02/01/2023
37534 pop cx ; *
37535 ; pop ax
37536 ; The address is already in ES.
37537 retn
37538
37539 ; -----
37540 ; *** BigFree - makes ES:0 point to the largest free MCB in UMB given as AL
37541 ; -----
37542 ; ENTRY: AL should be to a valid UMB number
37543 ; EXIT: ES:0 points to largest free MCB in UMB, AX returns its size
37544 ; ERROR: Carry set if couldn't reach UMB (0 or too high)
37545 ; USES: Flags, ES
37546 ; -----

```

```

37544         ; 01/01/2023 - Retro DOS v4.2
37545 BigFree:
37546         ; 01/01/2023
37547         ;push  bx
37548 000032FA 51      push  cx
37549
37550         call  findUMB          ; Returns with CF if err, else ES==MCB
37551 000032FE 723A     jc      short bfx          ; (would be "jc bFE"; it just does stc)
37552
37553 00003300 31DB     xor     bx,bx          ; Segment address of largest free MCB
37554 00003302 31C9     xor     cx,cx          ; Size of largest free MCB
37555
37556         ; -----
37557         ; BF10--ES - Current MCB address
37558         ; BX - Address of largest free MCB so far
37559         ; CX - Size of largest free MCB so far
37560         ; -----
37561
37562 bf10:
37563 00003304 E8B1FE   call  isSysMCB          ; If we've left the MCB, we're done.
37564 00003307 7428     jz      short bf30
37565
37566         ;call  isFreeMCB        ; Returns with ZF set if owner is 0
37567 00003309 26830E010000 or     word [es:ARENA.OWNER],0
37568 0000330F 750C     jnz     short bf20
37569
37570 00003311 26A10300 mov     ax,[es:ARENA.SIZE]
37571         ;cmp     cx,[es:ARENA.SIZE]      ; Compare sizes...
37572 00003315 39C1     cmp     cx,ax
37573         ;jg      short bf20          ; Unless we're bigger,
37574         ; 19/07/2023
37575 00003317 7D04     jge     short bf20
37576
37577 00003319 8CC3     mov     bx,es          ; Store this new element's address,
37578         ;mov     cx,[es:ARENA.SIZE]      ; and its size.
37579 0000331B 89C1     mov     cx,ax
37580
37581 bf20:
37582         ;mov     al,[es:ARENA.SIGNATURE]
37583         ;cmp     al,arena_signature_end; 'z'
37584         ; 19/07/2023
37585         ;cmp     byte [es:0],'z'
37586 0000331D 26803E00005A cmp     byte [es:ARENA.SIGNATURE],arena_signature_end
37587 00003323 740C     jz      short bf30          ; 'z' means this was the last MCB.
37588
37589         ;NextMCB es,ax          ; Go on forward.
37590 00003325 8CC0     mov     ax,es
37591         ;add     ax,[es:3]
37592 00003327 2603060300 add     ax,[es:ARENA.SIZE]
37593 0000332C 40       inc     ax
37594 0000332D 8EC0     mov     es,ax
37595
37596 0000332F EBD3     jmp     short bf10
37597
37598 00003331 8EC3     bf30:  mov     es,bx          ; Return the address
37599 00003333 89C8     mov     ax,cx          ; Return the size
37600 00003335 09DB     or      bx,bx
37601 00003337 7501     jnz     short bfx          ; (if size==0, there's nothing free)
37602
37603 00003339 F9       bFE:  stc
37604
37605 0000333A 59       bFX:  pop     cx
37606         ; 01/01/2023
37607         ;pop     bx
37608 0000333B C3       retn
37609
37610         ; -----
37611         ; *** isSpecified - sets ZF if UMB in AL wasn't specified in DH/LH line.
37612         ; -----
37613         ; ENTRY:    AL should be to a valid UMB number
37614         ; EXIT:     ZF set if UMB wasn't specified, ZF clear if it was
37615         ; ERROR:    None
37616         ; USES:     Flags
37617         ; -----
37618
37619         ; 02/01/2023 - Retro DOS v4.2
37620
37621 isSpecified:
37622         ; 02/01/2023
37623         ;push  ax
37624
37625 0000333C 30FF     xor     bh,bh
37626 0000333E 88C3     mov     bl,al
37627
37628         ;getdata al,DS:Umbused[bx]
37629         ;
37630         ;push  ds
37631         ;push  cs
37632         ;pop   ds
37633         ;mov   al,[bx+Umbused]
37634         ;pop   ds
37635
37636         ;mov   al,[cs:bx+Umbused]
37637         ; 02/01/2023
37638         ; ds = cs
37639 00003340 8A87[FE23] mov     al,[bx+Umbused]
37640
37641 00003344 08C0     or      al,al          ; Sets ZF if al==0 (ie, if unspecified)
37642
37643         ; 09/09/2023
37644         ; 02/01/2023
37645         ;pop   ax
37646
37647 00003346 C3       retn
37648
37649         ; -----
37650         ; *** shrinkMCB - breaks an MCB into two pieces, the lowest one's size==AX
37651         ; -----
37652         ; ENTRY:    AX == new size, ES:0 == current MCB
37653         ; EXIT:     None; MCB broken if carry clear
37654         ; ERROR:    Carry set if MCB isn't as large as AX+0x20 (not a useful split)
37655         ; USES:     Flags
37656         ; -----
37657         ; If the size of the to-be-split MCB isn't at least 0x20 bytes greater than
37658         ; the specified new size, the split is useless; if it's only 0x10 bytes, that
37659         ; 0x10 will be used to make a header that mentions a 0-byte free space, and
37660         ; that just sucks up 0x10 bytes for nothing. So we make 0x20 bytes the
37661         ; minimum for performing a split.
37662         ; -----
37663
37664 MIN_SPLIT_SIZE equ 20h
37665
37666         ; 02/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37667

```

```

37668 shrinkMCB:
37669     ;pushreg <bx,cx,es>
37670     ; 02/01/2023
37671     ;push bx
37672 00003347 51     push cx
37673 00003348 06     push es
37674
37675 00003349 89C3    mov bx,ax           ; Move things around... and
37676     ; 02/01/2023
37677     ;mov ax,es           ; save this one for later.
37678
37679 0000334B 268B0E0300 mov cx,[es:ARENA.SIZE]
37680     ; 02/01/2023
37681 00003350 89C8    mov ax,cx
37682
37683 00003352 83E820    sub ax,MIN_SPLIT_SIZE ; 32
37684     ;sub cx,MIN_SPLIT_SIZE ; 32
37685     ;cmp bx,cx           ; {New size} vs {Current Size-20h}
37686     ;ja short smE         ; if wanted_size > cur-20h, abort.
37687     ; 18/12/2022
37688     ;cmp cx,bx
37689     ; 02/01/2023
37690 00003355 39D8    cmp ax,bx
37691 00003357 7228    jnb short smE ; (*)
37692
37693 00003359 268A160000 mov dl,[es:ARENA.SIGNATURE]
37694
37695     ;mov cx,[es:ARENA.SIZE]
37696     ; 02/01/2023
37697 0000335E 8CC0    mov ax,es
37698
37699 00003360 26891E0300 mov [es:ARENA.SIZE],bx
37700 00003365 26C60600004D mov byte [es:ARENA.SIGNATURE],'M'
37701
37702 0000336B 01D8    add ax,bx
37703 0000336D 40     inc ax
37704 0000336E 8EC0    mov es,ax           ; Move to new arena area
37705
37706 00003370 89C8    mov ax,cx
37707 00003372 29D8    sub ax,bx
37708     ; 12/12/2022
37709     ; ax > 0
37710 00003374 48     dec ax           ; And prepare the new size
37711
37712     ; 18/12/2022
37713 00003375 2688160000 mov [es:ARENA.SIGNATURE],dl
37714     ;mov word [es:ARENA.OWNER],0 ; (**)
37715 0000337A 26A30300 mov [es:ARENA.SIZE],ax
37716     ;mov ax,' ' ; 2020h
37717     ;mov [es:ARENA.NAME+0],ax ; (**)
37718     ;mov [es:ARENA.NAME+2],ax ; (**)
37719     ;mov [es:ARENA.NAME+4],ax ; (**)
37720     ;mov [es:ARENA.NAME+6],ax ; (**)
37721
37722     ; 18/12/2022
37723 0000337E E8A801    call freeMCB ; (**)
37724
37725     ; 12/12/2022
37726     ; cf=0
37727     ;clc
37728     ; 18/12/2022
37729     ;jmp short smX
37730
37731 smE:
37732     ; 18/12/2022
37733     ; cf=1 (*)
37734     ;stc
37735
37736 smX:
37737     ;popreg <es,cx,bx>
37738     pop es
37739     pop cx
37740     ; 02/01/2023
37741     pop bx
37742     retn
37743
37744 ; -----
37745 ; *** hideUMB? - hides as appropriate the UMB in CL
37746 ; -----
37747 ; ENTRY: CL should be to a valid UMB number, and AX to its address (findUMB)
37748 ; EXIT: None; UMB is hidden as necessary
37749 ; ERROR: None
37750 ; USES: Flags, AX, CX
37751 ; -----
37752 ; PRIMARY LOGIC:
37753 ;
37754 ; If the UMB is specified in the DH/LH statement, then:
37755 ; If the largest free segment is too small (check specified size), then:
37756 ; Pretend it wasn't ever specified, and fall out of this IF.
37757 ; Else, if largest free segment is LARGER than specified size, then:
37758 ; If /S was given on the command-line, then:
37759 ; Break that element into two pieces
37760 ; Set a flag that we're shrinking
37761 ; Endif
37762 ; Endif
37763 ; If the UMB is NOT specified (or was removed by the above):
37764 ; Hide all free elements in the UMB
37765 ; If the flag that we're shrinking was set, then:
37766 ; UN-hide the lower portion of the shrunken UMB
37767 ; ENDIF
37768 ; ENDIF
37769 ; -----
37770     ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37771     ; (SYSINIT:3426h)
37772 _hideUMB:
37773     ; 02/01/2023
37774     ; ds = cs
37775
37776     ; 01/01/2023
37777     ;push bx
37778     ;push dx
37779 00003384 06     push es
37780
37781 00003385 88C8    mov al,cl
37782 00003387 E8B2FF    call isSpecified ; Returns ZF set if al's umb was NOT specified
37783 0000338A 742D    jz short hu_20
37784
37785 0000338C 88C8    mov al,cl           ; Retrieve the size of the largest
37786 0000338E E869FF    call BigFree       ; free element in AX; put its address in ES
37787 00003391 7226    jc short hu_20     ; Oops. Errors mean skip this part.
37788
37789 00003393 50     push ax           ; TOS==size of BigFree in UMB (popped as BX)
37790 00003394 88C8    mov al,cl         ; Retrieve the user's specified
37791 00003396 E8AAFE    call GetSize       ; minimum size for this umb (into AX)

```

```

37792 00003399 5B          pop     bx          ; Now BX==BigFree, AX==Specified Size
37793
37794 0000339A 09C0       or      ax,ax          ; If they didn't specify one,
37795 0000339C 741B       jz      short hu_20     ; Skip over all this.
37796
37797 0000339E 39D8       cmp     ax,bx          ; Ah... if (specified > max free)
37798 000033A0 7607       jbe     short hu_10
37799
37800 000033A2 88C8       mov     al,cl          ; Then mark that UMB as unused. Nya nya.
37801 000033A4 E81DFD     call    unMarkUMB
37802 000033A7 EB10       jmp     short hu_20
37803
37804          hu_10:
37805          ;call    isTiny          ; Returns ZF clear if user specified /s
37806          ;jz      short hu_20
37807          ; 02/01/2023
37808          ;isTiny:
37809          ;mov     al,[fUmbTiny] ; ds = cs
37810          ;or      al,al
37811 000033A9 800E[FA23]00 or      byte [fUmbTiny],0
37812 000033AE 7409       jz      short hu_20
37813
37814 000033B0 E894FF     call    shrinkMCB      ; They specified /S, so shrink the MCB to AX
37815 000033B3 7204       jc      short hu_20     ; Ah... if didn't shrink after all, skip this:
37816
37817 000033B5 8CC2       mov     dx,es
37818 000033B7 EB09       jmp     short hu_30      ; Skip the spec check.. we wanna hide this one.
37819
37820 000033B9 89C8       hu_20: mov     ax,cx
37821 000033BB E87EFF     call    isSpecified     ; If they specified this UMB, we're done...
37822 000033BE 7510       jnz     short hu_X       ; so leave.
37823
37824          xor     dx,dx
37825          hu_30:
37826          mov     al,cl
37827          call    hideUMB      ; Hides everything in UMB #al
37828
37829 000033C7 09D2       or      dx,dx          ; Did we shrink a UMB? If not, DX==0,
37830 000033C9 7405       jz      short hu_X       ; So we should leave.
37831
37832 000033CB 8EC2       mov     es,dx          ; Ah, but if it isn't, DX==the MCB's address;
37833 000033CD E8E6FE     call    unHideMCB       ; Un-hides the lower portion of that MCB.
37834
37835          hu_X:
37836          pop     es
37837          ; 01/01/2023
37838          ;pop     dx
37839          ;pop     bx
37840          retn
37841
37842          ; -----
37843          ; *** UnFreeze - Marks FROZEN elements as FREE
37844          ; -----
37845          ; Entry:  None
37846          ; Exit:   None; all 8+FROZEN elements are marked as FREE, from any UMB.
37847          ; Error:  None
37848          ; Uses:   Flags
37849          ; -----
37850
37851          ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
37852          UnFreeze:
37853          ; 03/01/2023
37854          ;push    ax
37855          ;push    es
37856          call    UmbHead      ; Returns with carry if err, else ES == MCB
37857 000033D6 721C       jc      short ufx
37858
37859          ; 22/07/2023
37860          uf10:
37861          mov     es,ax ; *
37862
37863          ; -----
37864          ; UF10--ES - Current MCB address
37865          ; -----
37866
37867          ;uf10:
37868          call    isFrozMCB     ; Returns with ZF set if MCB is FROZEN
37869 000033DD 7505       jnz     short uf20
37870 000033DF E8D4FE     call    unHideMCB
37871          ; 09/09/2023
37872          ; ax <> es
37873 000033E2 8CC0       mov     ax,es ; *
37874
37875          uf20:
37876          ;mov     al,[es:ARENA.SIGNATURE]
37877          ;cmp     al,arena_signature_end ; 'Z'
37878          ; 22/07/2023
37879 000033E4 26803E00005A cmp     byte [es:ARENA.SIGNATURE],arena_signature_end ; 'Z'
37880 000033EA 7408       jz      short ufx        ; 'Z' means this was the last MCB.. that's it.
37881
37882          ;NextMCB es,ax          ; Go on forward.
37883          ; 22/07/2023
37884          ; ax = es
37885          ;mov     ax,es ; *
37886          ;add     ax,[es:3]
37887 000033EC 2603060300 add     ax,[es:ARENA.SIZE]
37888 000033F1 40         inc     ax
37889          ; 22/07/2023
37890          ;mov     es,ax
37891          jmp     short uf10
37892
37893          ufx:
37894          pop     es
37895          ; 03/01/2023
37896          ;pop     ax
37897          retn
37898
37899          ; -----
37900          ; *** isFrozMCB - returns with ZF set if current MCB (ES:0) is FROZEN
37901          ; -----
37902          ; ENTRY:   ES:0 should point to an MCB
37903          ; EXIT:    ZF set if MCB is frozen, else !ZF
37904          ; ERROR:   None
37905          ; USES:    Flags
37906          ; -----
37907          isFrozMCB:
37908          ;push    ax
37909
37910          ;mov     ax,[es:ARENA.OWNER] ; Check the owner...
37911 000033F6 26833E010008 cmp     ax,SystemPSPOwner ; 8 (for US OR Japan) is valid
37912 000033FC 7522       cmp     word [es:ARENA.OWNER],SystemPSPOwner
37913          ;jne     short ifmX
37914
37915          ;mov     ax,[es:ARENA.NAME+0]
37916          ;cmp     ax,'FR' ; 5246h

```

```

37916 000033FE 26813E08004652      cmp     word [es:ARENA.NAME+0], 'FR'
37917 00003405 7519                jne     short ifmX
37918                        ;mov     ax, [es:ARENA.NAME+2]
37919                        ;cmp     ax, 'OZ' ; 5A4Fh
37920 00003407 26813E0A004F5A      cmp     word [es:ARENA.NAME+2], 'OZ'
37921 0000340E 7510                jne     short ifmX
37922                        ;mov     ax, [es:ARENA.NAME+4]
37923                        ;cmp     ax, 'EN' ; 4E45h
37924 00003410 26813E0C00454E      cmp     word [es:ARENA.NAME+4], 'EN'
37925 00003417 7507                jne     short ifmX
37926                        ;mov     ax, [es:ARENA.NAME+6]
37927                        ;cmp     ax,      ; 2020h
37928 00003419 26813E0E002020      cmp     word [es:ARENA.NAME+6], ' '
37929 ifmX:
37930                        ;pop     ax
37931 00003420 C3                retn
37932
37933 ; -----
37934 ; *** frezMCB - marks as 8+FROZEN the MCB at ES:0
37935 ; -----
37936 ; ENTRY:      ES:0 should point to an MCB
37937 ; EXIT:       None; MCB frozen
37938 ; ERROR:      None
37939 ; USES:       None
37940 ; -----
37941
37942 frezMCB:
37943 00003421 26C70601000800      mov     word [es:ARENA.OWNER], SystemPSPOwner ; 8
37944 00003428 26C70608004652      mov     word [es:ARENA.NAME+0], 'FR'
37945 0000342F 26C7060A004F5A      mov     word [es:ARENA.NAME+2], 'OZ'
37946 00003436 26C7060C00454E      mov     word [es:ARENA.NAME+4], 'EN'
37947 0000343D 26C7060E002020      mov     word [es:ARENA.NAME+6], ' '
37948 00003444 C3                retn
37949
37950 ; -----
37951 ; *** FreezeUM - Marks FROZEN all UM elements now FREE, save those in load UMB
37952 ; -----
37953 ; Entry:      None
37954 ; Exit:       None; all free elements not in load UMB marked as 8+FROZEN
37955 ; Error:      None
37956 ; Uses:       Flags
37957 ; -----
37958
37959 ; 01/01/2023 - Retro DOS v4.2
37960 FreezeUM:
37961 ; 01/01/2023
37962 ; push ax
37963 ; push cx
37964 ; push dx
37965 00003445 06                push    es
37966
37967 ; call GetLoadUMB
37968 ; 01/01/2023
37969 ; ds = cs
37970 ; mov al, [cs:UmbLoad] ; 19/04/2019 - Retro DOS v4.0
37971 00003446 A0[FD23]      mov     al, [UmbLoad]
37972
37973 00003449 30E4                xor     ah, ah ; Zap ah, so al==ax
37974 0000344B 89C2                mov     dx, ax ; Store the load UMB in DX, so we can skip it
37975
37976 0000344D E85BFD      call    UmbHead ; Returns first UMB segment in AX
37977 ; 22/07/2023
37978 ; mov es, ax ; *
37979 00003450 31C9                xor     cx, cx ; Pretend we're on UMB 0 for now...
37980
37981 ; 22/07/2023
37982 fum10:
37983 00003452 8EC0                mov     es, ax ; *
37984
37985 ; -----
37986 ; FUM10--ES - Current MCB address
37987 ; CX - Current UMB number
37988 ; DX - UMB number to skip (load UMB)
37989 ; -----
37990
37991 ; fum10:
37992 00003454 E861FD      call    issysMCB ; Returns with ZF set if owner is SYSTEM
37993 00003457 7501                jnz     short fum20
37994
37995 00003459 41                inc     cx ; If it _was_ SYSTEM, we're in a new UMB.
37996 fum20:
37997 0000345A 39D1                cmp     cx, dx ; If this is the load UMB, we don't want to
37998 0000345C 740B                je      short fum30 ; freeze anything... so skip that section.
37999
38000 ; call isFreeMCB ; Oh. If it's not free, we can't freeze it
38001 0000345E 26830E010000      or      word [es:ARENA.OWNER], 0
38002 00003464 7503                jnz     short fum30 ; either.
38003
38004 00003466 E8B8FF      call    frezMCB
38005 fum30:
38006 ; mov al, [es:ARENA.SIGNATURE]
38007 ; cmp al, arena_signature_end ; 'Z'
38008 ; 22/07/2023
38009 00003469 26803E00005A      cmp     byte [es:ARENA.SIGNATURE], arena_signature_end ; 'Z'
38010 0000346F 7408                je      short fumX ; 'Z' means this was the last MCB.. that's it.
38011
38012 ; NextMCB es, ax ; Go on forward.
38013 ; 22/07/2023
38014 ; ax = es
38015 ; mov ax, es
38016 ; add ax, [es:3]
38017 00003471 2603060300      add     ax, [es:ARENA.SIZE]
38018 00003476 40                inc     ax
38019 ; 22/07/2023
38020 ; mov es, ax ; *
38021 00003477 EBD9                jmp     short fum10
38022
38023 00003479 07                fumX:   pop     es
38024 ; 01/01/2023
38025 ; pop dx
38026 ; pop cx
38027 ; pop ax
38028 0000347A C3                retn
38029
38030 ; -----
38031 ; *** UmbTest - returns with carry set if UMBs are not available, else CF==false
38032 ; -----
38033 ; ENTRY:      None
38034 ; EXIT:       Carry is clear if UMBs are available, or set if they are not
38035 ; ERROR:      None
38036 ; USES:       CF (AX,BX,DS,ES pushed 'cause they're used by others)
38037 ; -----
38038
38039 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)

```

```

38040 UmbTest:
38041 ; 01/01/2023
38042 ;push ax
38043 0000347B 53 ;push bx ; *
38044 ;push ds
38045 0000347C 06 ;push es ; **
38046
38047 ; 01/01/2023
38048 ; ds = cs
38049
38050 call fm_link ; Link in UMBS (if not already linked)
38051 call walkMem ; Check to see if they're really linked
38052 pushf ; And remember what we found out
38053 call fm_unlink ; Unlink UMBS (if we have linked 'em)
38054 00003487 9D ; And restore what we found out.
38055 popf
38056 00003488 07 pop es ; **
38057 ; 01/01/2023
38058 ;pop ds
38059 00003489 5B pop bx ; *
38060 ;pop ax
38061 0000348A C3 retn
38062
38063 ; -----
38064 ; *** walkMem - travels memory chain and returns carry clear iff UMBS are linked
38065 ; -----
38066 ENTRY: None
38067 EXIT: Carry SET if MCB chain stops before 9FFF, CLEAR if stops >= 9FFF.
38068 ERROR: None
38069 USES: Flags
38070 ; -----
38071
38072 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38073 ; (SYSINIT:3541h)
38074
38075 walkMem:
38076 ;push ax ; ?
38077 ;push bx ; ?
38078 ;push ds ; ? ; 01/01/2023 (MSDOS 6.21 IO.SYS, SYSINIT:352Fh)
38079 ;push es ; ? no need to save contents of these registers ?
38080
38081 0000348B B452 mov ah,GET_IN_VARS ; Call int 21h, function 52h...
38082 0000348D CD21 int 21h
38083
38084 0000348F 268B47FE mov ax,[es:bx-2]
38085 ; 22/07/2023
38086 um10:
38087 00003493 8EC0 mov es,ax ; * ; **
38088
38089 ; -----
38090 ; UM10: ES = Current MCB pointer
38091 ; -----
38092
38093 ;um10:
38094 ;mov al,[es:ARENA.SIGNATURE]
38095 ;cmp al,arena_signature_end ; 'z'
38096 ; 22/07/2023
38097 00003495 26803E00005A cmp byte [es:ARENA.SIGNATURE],arena_signature_end ; 'z'
38098 0000349B 7408 je short um20 ; If signature == 'Z', hay no more.
38099
38100 ;NextMCB es,bx ; Move to the next MCB
38101
38102 ;mov bx,es
38103 ;add bx,[es:3]
38104 ;add bx,[es:ARENA.SIZE]
38105 ;inc bx
38106 ;mov es,bx
38107 ; 22/07/2023
38108 ; ax = es
38109 ;mov ax,es ; *
38110 0000349D 2603060300 add ax,[es:ARENA.SIZE]
38111 000034A2 40 inc ax
38112 ;mov es,ax ; **
38113
38114 000034A3 EBEE jmp short um10 ; And restart the loop.
38115 um20:
38116 ; 22/07/2023
38117 ; ax = es
38118 ;mov ax,es
38119
38120 000034A5 3DFF9F cmp ax,9FFFh ; This sets CF if ax < 9FFF.
38121
38122 ;pop es ; ?
38123 ;pop ds ; ? ; 01/01/2023 (MSDOS 6.21 IO.SYS, SYSINIT:353Dh)
38124 ;pop bx ; ?
38125 ;pop ax ; ?
38126
38127 000034A8 C3 retn
38128
38129 ; -----
38130 ; *** hl_unlink - unlinks UMBS if fm_umb is set to 0; restores strategy too
38131 ; -----
38132 ENTRY: fm_umb == 1 : leave linked, else unlink
38133 EXIT: None
38134 ERROR: None
38135 USES: AX, BX
38136 ; -----
38137
38138 ; 01/01/2023 - Retro DOS v4.2
38139 hl_unlink:
38140 000034A9 30FF xor bh,bh
38141
38142 ;getdata bl,fm_umb ; Restore original link-state
38143 ;
38144 ;push ds
38145 ;push cs
38146 ;pop ds
38147 ;mov bl,[fm_umb]
38148 ;pop ds
38149
38150 ; 01/01/2023
38151 ; ds = cs
38152 ;mov bl,[cs:fm_umb]
38153 000034AB 8A1E[2E24] mov bl,[fm_umb]
38154
38155 000034AF B80358 mov ax,DOS_SET_UMBLINK ; 5803h
38156 000034B2 CD21 int 21h
38157 000034B4 C3 retn
38158
38159 ; -----
38160 ; HIGHEXIT.INC (MSDOS 6.0 - 1991)
38161 ; -----
38162 ; 09/04/2019 - Retro DOS v4.0
38163

```



```

38164 ; Module:  HIGHEXIT.INC - Code executed after LoadHigh or DeviceHigh
38165 ; Date:    May 14, 1992
38166
38167 ; Modification log:
38168 ;
38169 ; DATE      WHO      DESCRIPTION
38170 ; -----
38171 ; 05/14/92  t-richj  Original
38172 ; 06/21/92  t-richj  Final revisions before check-in
38173
38174 UMB_HeadIdx equ    8Ch    ; Offset from ES (after func52h) to get UMBHead
38175
38176 ; -----
38177 ; *** UnHideUMBs - Marks HIDDEN elements as FREE
38178 ; -----
38179 ; ENTRY:  None; perhaps, earlier, HideUMBs was called... if not, we have
38180 ;         very little to do, as no elements will be marked as HIDDEN.
38181 ; EXIT:   Sets InHigh to zero; carry clear if HideUMBs was called earlier.
38182 ; ERROR:  None
38183 ; USES:   fInHigh (from highvar.inc), carry flag
38184 ; -----
38185
38186 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38187 ; (SYSINIT:357Bh)
38188
38189 UnHideUMBs:
38190 000034B5 50      push    ax          ; Save ax for what we're about to do
38191
38192 ; -----
38193 ; BUGBUG t-richj 11-8-92: The following six lines were commented out for a good
38194 ; length of time. Those six constitute a check of whether or not we should
38195 ; indeed clean up the upper-memory chain; without such a check, COMMAND.COM
38196 ; will destroy the current link-state and memory-allocation strategy after
38197 ; every command execution.
38198 ; -----
38199
38200 ;getdata al,fInHigh ; Get InHigh from data segment
38201 ;
38202 ;push    ds
38203 ;push    cs
38204 ;pop     ds
38205 ;mov     al,[fInHigh]
38206 ;pop     ds
38207
38208 ;mov     al,[cs:fInHigh]
38209 ; 31/12/2022
38210 ; ds = cs
38211 000034B6 A0[F923] mov     al,[fInHigh]
38212
38213 000034B9 08C0      or      al,al
38214 000034BB 7503      jnz     short uhu10    ; If didn't call loadhigh/devicehigh earlier,
38215
38216 000034BD 58          pop     ax          ; then there's nothing to do here... so
38217 000034BE F9          stc          ; restore everything and return. Just like
38218 000034BF C3          retn         ; that.
38219
38220 000034C0 E88E00   uhu10:  call    linkumb      ; Make sure UMBS are linked in.
38221 000034C3 E81200   call    FreeUMBs
38222
38223 ;putdata fInHigh,0 ; We're leaving, so update fInHigh.
38224 ;
38225 ;push    es
38226 ;push    cs
38227 ;pop     es
38228 ;mov     byte [es:fInHigh],0
38229 ;pop     ds
38230
38231 ; 31/12/2022
38232 ; ds = cs
38233 ;mov     byte [cs:fInHigh],0
38234 000034C6 C606[F923]00 mov     byte [fInHigh],0
38235
38236 ;call    he_unlink ; Unlink UMBS
38237 ; 31/12/2022
38238 ;;he_unlink:
38239 000034CB 30FF      xor     bh,bh
38240
38241 ;getdata bl,fm_umb ; Restore original link-state
38242 ;mov     bl,[cs:fm_umb]
38243 000034CD 8A1E[2E24] mov     bl,[fm_umb]
38244
38245 000034D1 B80358      mov     ax,DOS_SET_UMBLINK ; 5803h
38246 000034D4 CD21      int     21h
38247 ;;retn
38248
38249 000034D6 58          pop     ax
38250 ; 12/12/2022
38251 ; clc ; 12/12/2022 (this clc may not be necessary!?)
38252 000034D7 C3          retn
38253
38254 ; 31/12/2022
38255 ;%if 0
38256 ;
38257 ; -----
38258 ; *** he_unlink - unlinks UMBS if fm_umb is set to 0
38259 ; -----
38260 ; ENTRY:   fm_umb == 1 : leave linked, else unlink
38261 ; EXIT:    None
38262 ; ERROR:   None
38263 ; USES:    AX, BX
38264 ; -----
38265 ;
38266 ;he_unlink:
38267 ; xor     bh, bh
38268 ;
38269 ;getdata bl, fm_umb ; Restore original link-state
38270 ;mov     bl,[cs:fm_umb]
38271 ;
38272 ; mov     ax,DOS_SET_UMBLINK ; 5803h
38273 ; int     21h
38274 ; retn
38275 ;
38276 ;%endif
38277
38278 ; -----
38279 ; *** freeUMBs - frees all HIDDEN memory elements in upper-memory.
38280 ; -----
38281 ; ENTRY:   None
38282 ; EXIT:    None; HIDDEN memory elements returned to FREE
38283 ; ERROR:   None (ignore CF)
38284 ; USES:    Flags
38285 ; -----
38286
38287 FreeUMBs:

```

```

38288 000034D8 50      push    ax
38289 000034D9 06      push    es
38290
38291 000034DA E86700    call    HeadUmb      ; Returns with carry if err, else ES == MCB
38292 000034DD 721C      jc      short fusX
38293 fus10:
38294 000034DF 8EC0      mov     es,ax        ; Prepare for the loop; ES = current MCB addr.
38295 ;fus10:
38296 000034E1 E81A00    call    isHideMCB    ; Returns with ZF set if owner is 0
38297 000034E4 7505      jnz     short fus20
38298 000034E6 E84000    call    freeMCB
38299      ; 09/09/2023
38300      ; ax <> es
38301 000034E9 8CC0      mov     ax,es
38302 fus20:
38303      ;mov     al,[es:ARENA.SIGNATURE]
38304      ;cmp     al,arena_signature_end ; 'z'
38305      ; 22/07/2023
38306 000034EB 26803E00005A cmp     byte [es:ARENA.SIGNATURE],arena_signature_end ; 'z'
38307 000034F1 7408      jz      short fusX    ; That means this was the last MCB--that's it.
38308
38309      ; 22/07/2023
38310      ; ax = es
38311      ;mov     ax,es
38312 000034F3 2603060300 add     ax,[es:ARENA.SIZE]
38313 000034F8 40      inc     ax
38314      ; 22/07/2023
38315      ;mov     es,ax
38316 000034F9 EBE4      jmp     short fus10    ; Go on forward.
38317 fusX:
38318 000034FB 07      pop     es
38319 000034FC 58      pop     ax
38320 000034FD C3      retn
38321
38322 ; -----
38323 ; *** isHideMCB - returns with ZF set if current MCB (ES:0) is HIDDEN
38324 ; -----
38325 ; ENTRY:      ES:0 should point to an MCB
38326 ; EXIT:      ZF set if MCB is hidden, else !ZF
38327 ; ERROR:      None
38328 ; USES:      Flags
38329 ; -----
38330
38331 isHideMCB:
38332      ;push    ax
38333
38334 000034FE 26833E010008 cmp     word [es:ARENA.OWNER],SystemPSPowner ; If the owner's SYSTEM
38335 00003504 7522      jne     short ihm_x    ; then check for HIDDEN
38336
38337      ;mov     ax,[es:ARENA.NAME]
38338      ;cmp     ax,'HI' ; 4948h
38339 00003506 26813E08004849 cmp     word [es:ARENA.NAME+0],'HI'
38340 0000350D 7519      jne     short ihm_x
38341      ;mov     ax,[es:ARENA.NAME+2]
38342      ;cmp     ax,'DD' ; 4444h
38343 0000350F 26813E0A004444 cmp     word [es:ARENA.NAME+2],'DD'
38344 00003516 7510      jne     short ihm_x
38345      ;mov     ax,[es:ARENA.NAME+4]
38346      ;cmp     ax,'EN' ; 4E45h
38347 00003518 26813E0C00454E cmp     word [es:ARENA.NAME+4],'EN'
38348 0000351F 7507      jne     short ihm_x
38349      ;mov     ax,[es:ARENA.NAME+6]
38350      ;cmp     ax,' ' ; 2020h
38351 00003521 26813E0E002020 cmp     word [es:ARENA.NAME+6],' '
38352 ihm_x:
38353      ;pop     ax
38354 00003528 C3      retn
38355
38356 ; -----
38357 ; *** freeMCB - marks as free the MCB at ES:0
38358 ; -----
38359 ; ENTRY:      ES:0 should point to an MCB
38360 ; EXIT:      None; MCB free'd
38361 ; ERROR:      None
38362 ; USES:      AX
38363 ; -----
38364
38365 freeMCB:
38366 00003529 26C70601000000 mov     word [es:ARENA.OWNER],0
38367 00003530 B82020      mov     ax,' ' ; mov ax,2020h ; 31/12/2022
38368 00003533 26A30800      mov     [es:ARENA.NAME+0],ax
38369 00003537 26A30A00      mov     [es:ARENA.NAME+2],ax
38370 0000353B 26A30C00      mov     [es:ARENA.NAME+4],ax
38371 0000353F 26A30E00      mov     [es:ARENA.NAME+6],ax
38372 00003543 C3      retn
38373
38374 ; -----
38375 ; *** HeadUmb - returns in AX the address of the first UMB block (0x9FFF)
38376 ; -----
38377 ; ENTRY:      Nothing
38378 ; EXIT:      AX contains 0x9FFF for most systems
38379 ; ERROR:      Carry set if pointer is 0xFFFF (if not set up yet--DH runs into this)
38380 ; USES:      Flags, AX
38381 ; -----
38382
38383 HeadUmb:
38384      ; 13/05/2019
38385
38386      ;push    si ; ?
38387      ;push    ds ; ?
38388      ;push    es
38389      ;push    bx ; *
38390
38391      ; 09/04/2019
38392      ; !!! No need to save es,bx,ds,si above !!! (es,bx are changed here)
38393
38394 00003544 B452      mov     ah,GET_IN_VARS      ; Call int 21h, function 52h...
38395 00003546 CD21      int     21h
38396      ; DOS - 2+ internal - GET LIST OF LISTS
38397      ; Return: ES:BX -> DOS list of lists
38398      ;mov     ax,[es:8Ch]
38399 00003548 26A18C00      mov     ax,[es:UMB_HeadIdx] ; And read what's in ES:008C
38400 0000354C 83F8FF      cmp     ax,0FFFFh
38401      ;je      short xhu_e    ; If it's 0xFFFF, it's an error...
38402
38403      ;clc
38404      ;jmp     short xhu_x    ; Else, it isn't.
38405 xhu_e:
38406      ;stc
38407 0000354F F5      cmc
38408 xhu_x:
38409      ;pop     bx ; *
38410      ;pop     es
38411      ;pop     ds ; ?

```



```

38412             ;pop     si ; ?
38413 00003550 c3      retn
38414
38415             ;-----
38416             ;*** linkumb - links UMBS not already linked in; updates fm_umb as needed
38417             ;-----
38418 ENTRY:      None
38419 EXIT:       fm_umb == 0 if not linked in previously, 1 if already linked in
38420 ERROR:      None
38421 USES:       AX, BX, fm_umb
38422             ;-----
38423
38424 linkumb:
38425 00003551 B80258      mov     ax,DOS_GET_UMBLINK ; 5802h
38426 00003554 CD21      int      21h                ; Current link-state is now in al
38427
38428             or      al,al                ; BUGBUG: proper check?
38429 00003558 7508      jnz     short lumbx        ; Jumps if UMBS already linked in
38430
38431             mov     ax,DOS_SET_UMBLINK ; 5803h
38432 0000355D BB0100      mov     bx,1
38433 00003560 CD21      int      21h
38434
38435 00003562 C3      lumbx:
38436             retn
38437
38438 ;%endif
38439
38440 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
38441 ; (SYSINIT:2B5Fh)
38442
38443             ;-----
38444             ; SYSCONF.ASM (MSDOS 6.0 - 1991)
38445             ;-----
38446             ; 09/04/2019 - Retro DOS v4.0
38447
38448             ;-----
38449             ; procedure : InitDevLoad
38450             ;
38451             ; Input : DeviceHi = 0 indicates load DD in low memory
38452             ;               = 1 indicates load in UMB:
38453             ;               ConvLoad = 0 indicates a new-style load (see below)
38454             ;               = 1 indicates a DOS 5-style load
38455             ;               DevSize = Size of the device driver file in paras
38456             ;
38457             ; Output : none
38458             ;
38459             ; Initializes DevLoadAddr, DevLoadEnd & DevEntry.
38460             ; Also sets up a header for the Device driver entry for mem utility
38461             ;
38462             ;-----
38463             ; For a "new-style load", we break off the current DevEntry and link the umbs
38464             ; as we see fit, using HideUMBS (and UnHideUMBS at exit, though _it_ decides
38465             ; whether it's entitled to do anything). HideUMBS uses the chart built by
38466             ; Parsevar to determine which UMBS to leave FREE, and which not.
38467             ;-----
38468
38469             ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
38470             ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38471             ; (SYSINIT:364Ah)
38472 InitDevLoad:
38473             ; 01/01/2023
38474             ;push     es ; *
38475
38476             ; 11/12/2022
38477             ; ds = cs
38478 00003563 803E[4F24]00      cmp     byte [DeviceHi],0
38479             ;cmp     byte [cs:DeviceHi],0 ; Are we loading in UMB ?
38480             ;je      short InitForLo ; no, init for lo mem
38481 00003568 7439      je      short initforlo_x ; 09/04/2019
38482
38483             ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38484             ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
38485             ; %if 0
38486             ; 01/01/2023
38487 0000356A 803E[3F24]01      cmp     byte [ConvLoad],1 ; Are we loading as per DOS 5?
38488             ;cmp     byte [cs:ConvLoad],1 ; Are we loading as per DOS 5?
38489 0000356F 7413      je      short InitForConv
38490
38491             ; There are two stages to preparing upper-memory; first, we mark as 8+HIDDEN
38492             ; any areas not specified on the /L:... chain. Second, we mark as 8+FROZEN
38493             ; any areas left in upper-memory, except for elements in the load UMB...
38494             ; we then malloc space as per Dos-5 style, and mark as free any spaces which
38495             ; are 8+FROZEN (but leave 8+HIDDEN still hidden). The load is performed,
38496             ; and UnHideUMBS later on marks all 8+HIDDEN as free.
38497
38498 00003571 E85904      call     ShrinkUMB ; Stop using the old device arena
38499
38500             call     HideUMBS ; Mark up the UM area as we see fit
38501             call     FreezeUM ; Hide everything BUT the load area
38502             call     GetUMBForDev ; And grab that load area as needed
38503             pushf
38504             call     UnFreeze ; Then unhide everything frozen
38505             popf
38506             ;jc      short InitForLo ; (if carry, it's loading low)
38507             ;jmp     short InitForHi
38508             ; 06/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
38509 00003582 EB0B      jmp     short idl0
38510
38511 ;%endif ; 01/11/2022
38512
38513             ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
38514             ; (SYSINIT:2B67h)
38515 InitForConv:
38516             ; 11/12/2022
38517             ; ds = cs
38518 00003584 E83700      call     SpaceInUMB ; Do we have space left in the
38519             ; current UMB ?
38520             jnc     short InitForHi ; yes, we have
38521             call     ShrinkUMB ; shrink the current UMB in use
38522             call     GetUMBForDev ; else try to allocate new UMB
38523
38524 0000358F 720D      idl0: ; 06/07/2023
38525             jc      short InitForLo ; we didn't succeed, so load
38526             ; in low memory
38527 InitForHi:
38528             ; 11/12/2022
38529             ; ds = cs
38530             ;mov     ax,[cs:DevUMBFree] ; get Para addr of free mem
38531             ;mov     dx,[cs:DevUMBAddr] ; UMB start addr
38532             ;add     dx,[cs:DevUMBSize] ; DX = UMB End addr
38533             mov     ax,[DevUMBFree]
38534             mov     dx,[DevUMBAddr]
38535             add     dx,[DevUMBSize]
38536             jmp     short idl1

```

```

38536
38537
38538
38539
38540
38541 0000359E C606[4F24]00
38542
38543
38544
38545
38546
38547
38548
38549 000035A3 A1[6403]
38550 000035A6 8B16[A502]
38551
38552 000035AA E86600
38553
38554
38555
38556
38557
38558
38559 000035AD A3[3324]
38560 000035B0 8916[3524]
38561 000035B4 C706[3724]0000
38562 000035BA A3[3924]
38563
38564
38565 000035BD C3
38566
38567
38568
38569
38570
38571
38572
38573
38574
38575
38576
38577
38578
38579
38580
38581
38582
38583
38584 000035BE A1[4324]
38585 000035C1 0306[4124]
38586 000035C5 2B06[4524]
38587
38588
38589
38590
38591
38592
38593 000035C9 83F801
38594 000035CC 7205
38595
38596 000035CE 48
38597
38598
38599 000035CF 3B06[3124]
38600
38601
38602 000035D3 C3
38603
38604
38605
38606
38607
38608
38609
38610
38611
38612
38613
38614
38615
38616
38617
38618
38619
38620
38621
38622
38623
38624
38625
38626
38627
38628
38629
38630
38631
38632
38633
38634
38635
38636
38637
38638
38639
38640
38641
38642
38643
38644
38645
38646
38647
38648
38649
38650
38651
38652 000035D4 BBFFFF
38653 000035D7 B80048
38654 000035DA CD21
38655
38656
38657
38658 000035DC 09DB
38659

InitForLo:
; 11/12/2022
; ds = cs
;mov byte [cs:DeviceHi],0 ; in case we failed to load
mov byte [DeviceHi],0
initforlo_x:
; 11/12/2022
; ds = cs
; into UMB indicate that
; we are loading low
; AX = start of Low memory
; DX = End of Low memory
;mov ax,[cs:memhi]
;mov dx,[cs:ALLOCLIM]
mov ax,[memhi]
mov dx,[ALLOCLIM]
idl1:
call DevSetMark ; setup a sub-arena for DD
; 11/12/2022
; ds = cs
;mov [cs:DevLoadAddr],ax ; init the Device load address
;mov [cs:DevLoadEnd],dx ; init the limit of the block
;mov word [cs:DevEntry],0 ; init Entry point to DD
;mov [cs:DevEntry+2],ax
mov [DevLoadAddr],ax
mov [DevLoadEnd],dx
mov word [DevEntry],0
mov [DevEntry+2],ax
; 01/01/2023
;pop es ; *
retn

;-----
; procedure : SpaceInUMB?
;
; Input : DevUMBAddr, DevUMBSize, DevUMBFree & DevSize
; Output : Carry set if no space in UMB
; Carry clear if Space is available for the device in
; current UMB
;-----

SpaceInUMB:
; 11/12/2022
; ds = cs
;mov ax,[cs:DevUMBSize]
;add ax,[cs:DevUMBAddr] ; End of UMB
;sub ax,[cs:DevUMBFree] ; - Free = Remaining space
mov ax,[DevUMBSize]
add ax,[DevUMBAddr] ; End of UMB
sub ax,[DevUMBFree] ; - Free = Remaining space
; 11/12/2022
;or ax,ax ; Nospace ?
;jnz short spcinumb1
;stc
;retn
; 11/12/2022
cmp ax,1
jb short spcinumb2 ; cf=1
spcinumb1:
dec ax ; space for sub-arena
; 11/12/2022
; ds = cs
cmp ax,[DevSize]
;cmp ax,[cs:DevSize] ; do we have space ?
spcinumb2:
retn

;-----
; procedure : PrepareMark
;
; Input : AX==Address of MCB (not addr of free space), BX==Size
; Output : None; MCB marked appropriately and DevUMB* set as needed.
;-----

; ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
;
;PrepareMark:
; push ds
; mov ds,ax
; mov word [ARENA.OWNER],8
; mov word [ARENA.NAME],'SD' ; 4453h
; pop ds
;
; inc ax
; mov [cs:DevUMBAddr],ax
; mov [cs:DevUMBFree],ax
; mov [cs:DevUMBSize],bx ; update the UMB variables
; retn

;-----
; procedure : GetUMBForDev
;
; Input : DevSize
; Output : Carry set if couldn't allocate a UMB to fit the
; the device.
; If success carry clear
;
; Allocates the biggest UMB for loading devices and updates
; DevUMBSize, DevUMBAddr & DevUMBFree if it succeeded in allocating
; UMB.
;
; This routine relies on the fact that all of the low memory
; is allocated, and any DOS alloc calls should return memory
; from the UMB pool.
;-----
; ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; ; (SYSINIT:2BC6h)

GetUMBForDev:
; 11/12/2022
; ds = cs
mov bx,0FFFFh
mov ax,4800h
int 21h
; DOS - 2+ - ALLOCATE MEMORY
; BX = number of 16-byte paragraphs desired
or bx,bx
;jz short gufd_err

```

```

38660          ; 09/09/2023
38661 000035DE 742E      jz      short gufd_error ; bx = 0
38662
38663 000035E0 4B        dec      bx
38664          ; 11/12/2022
38665          ; ds = cs
38666 000035E1 391E[3124] cmp     [DevSize],bx
38667          ; cmp     [cs:DevSize],bx
38668 000035E5 7725      ja      short gufd_err
38669
38670 000035E7 43        inc      bx
38671
38672 000035E8 B80048      mov     ax,4800h
38673 000035EB CD21      int     21h
38674 000035ED 721D      jc      short gufd_err
38675
38676          ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
38677          ; dec     ax
38678          ; call    PrepareMark
38679          ;
38680 PrepareMark:
38681 000035EF 1E        push     ds
38682 000035F0 48        dec     ax
38683 000035F1 8ED8      mov     ds,ax
38684 000035F3 C70601000800 mov     word [ARENA.OWNER],8
38685 000035F9 C70608005344 mov     word [ARENA.NAME],'SD' ; 4453h
38686 000035FF 40        inc     ax
38687 00003600 1F        pop     ds
38688          ; 11/12/2022
38689          ; ds = cs
38690          ; mov     [cs:DevUMBSize],bx      ; update the UMB Variables
38691          ; mov     [cs:DevUMBAddr],ax
38692          ; mov     [cs:DevUMBFree],ax
38693 gufd_x:          ; 09/09/2023
38694 00003601 891E[4324] mov     [DevUMBSize],bx      ; update the UMB Variables
38695 00003605 A3[4124]   mov     [DevUMBAddr],ax
38696 00003608 A3[4524]   mov     [DevUMBFree],ax
38697          ;
38698          ; 11/12/2022
38699          ; cf=0
38700          ; cld
38701 0000360B C3        ret     ; mark no error
38702
38703          ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
38704 %if 1
38705 gufd_err:
38706 0000360C 31DB      xor     bx,bx ; 0
38707 gufd_error:
38708          xor     ax,ax ; 0
38709          stc     ; cf=1
38710          jmp     short gufd_x
38711 %endif
38712
38713          ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
38714 %if 0
38715 gufd_err:
38716          xor     ax,ax ; 0
38717          ; 11/12/2022
38718          ; ds = cs
38719          ; mov     [cs:DevUMBSize],ax      ; erase the previous values
38720          ; mov     [cs:DevUMBAddr],ax
38721          ; mov     [cs:DevUMBFree],ax
38722          mov     [DevUMBSize],ax      ; erase the previous values
38723          mov     [DevUMBAddr],ax
38724          mov     [DevUMBFree],ax
38725          stc
38726          ret     ;
38727 %endif
38728
38729 ;-----
38730 ;
38731 ; procedure : DevSetMark
38732 ;
38733 ; Input : AX - Free segment were device is going to be loaded
38734 ; Output : AX - Segment at which device can be loaded (AX=AX+1)
38735 ;
38736 ; Creates a sub-arena for the device driver
38737 ; puts 'D' marker in the sub-arena
38738 ; Put the owner of the sub-arena as (AX+1)
38739 ; Copies the file name into sub-arena name field
38740 ;
38741 ; Size field of the sub-arena will be set only at succesful
38742 ; completion of Device load.
38743 ;
38744 ;-----
38745
38746          ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
38747          ; (SYSINIT:2C13h)
38748
38749 DevSetMark:
38750 00003613 06        push     es
38751          ; 03/01/2023
38752          ; push     di
38753 00003614 1E        push     ds
38754 00003615 56        push     si
38755 00003616 8EC0      mov     es,ax
38756 00003618 26C606000044 mov     byte [es:devmark.id],devmark_device ; 'D'
38757 0000361E 40        inc     ax
38758 0000361F 26A30100 mov     [es:devmark.seg],ax
38759
38760 ;----- Copy file name
38761
38762 00003623 50        push     ax      ; save load addr
38763
38764          ; 09/09/2023
38765          ; ds = cs
38766          ; lds     si,[cs:bpb_addr]      ; command line is still there
38767 00003624 C536[8103] lds     si,[bpb_addr]
38768
38769          mov     di,si
38770          cld
38771 dsm_again:
38772          lodsb
38773          cmp     al,':'
38774          jne     short isit_slash
38775          mov     di,si
38776          jmp     short dsm_again
38777 isit_slash:
38778          cmp     al,'\'
38779          jne     short isit_null
38780          mov     di,si
38781          jmp     short dsm_again
38782 isit_null:
38783          or      al,al

```

```

38784 0000363E 75EB      jnz     short dsm_again
38785 00003640 89FE      mov     si,di
38786
38787 00003642 BF0800     mov     di,devmark.filename ; 8
38788 00003645 B90800     mov     cx,8                ; maximum 8 characters
38789
38790 00003648 AC      dsm_next_char:
38791 00003649 08C0      lodsb
38792 0000364B 7407      or      al,al
38793 0000364D 3C2E      jz      short blankout
38794 0000364F 7403      cmp     al,' '
38795 00003651 AA      je      short blankout
38796 00003652 E2F4      stosb
38797      loop   dsm_next_char
38798 00003654 E304     blankout:
38799 00003656 B020      jcxz    dsm_exit
38800 00003658 F3AA      mov     al,' '
38801      rep     stosb                ; blank out the rest
38802 0000365A 58      dsm_exit:
38803 0000365B 5E      pop     ax                    ; restore load addr
38804 0000365C 1F      pop     si
38805      pop     ds
38806      ; 03/01/2023
38807 0000365D 07      ;pop     di
38808 0000365E C3      pop     es
38809      retn
38810
38811
38812
38813
38814
38815
38816
38817
38818
38819
38820
38821
38822
38823
38824
38825
38826
38827 0000365F 1E
38828 00003660 06
38829 00003661 1F
38830 00003662 89F2
38831 00003664 B8003D
38832 00003667 CD21
38833 00003669 7237
38834
38835 0000366B 89C3
38836 0000366D B80242
38837 00003670 31C9
38838 00003672 89CA
38839 00003674 CD21
38840 00003676 7223
38841 00003678 83C00F
38842 0000367B 83D200
38843 0000367E F7C2F0FF
38844
38845
38846 00003682 7409
38847 00003684 2EC706[3124]FFFF
38848
38849 0000368B EB0E
38850
38851
38852
38853 0000368D B104
38854 0000368F D3E8
38855 00003691 B10C
38856 00003693 D3E2
38857 00003695 09D0
38858
38859
38860
38861
38862
38863
38864
38865
38866
38867
38868 00003697 2EA3[3124]
38869
38870
38871
38872
38873
38874
38875
38876
38877
38878 0000369B 9C
38879
38880 0000369C B8003E
38881 0000369F CD21
38882 000036A1 9D
38883
38884
38885
38886 000036A2 1F
38887 000036A3 C3
38888
38889
38890
38891
38892
38893
38894
38895
38896
38897
38898
38899
38900
38901
38902
38903
38904
38905 000036A4 2E8B1E[3324]
38906 000036A9 2E891E[4B24]
38907

```

```

;-----
; procedure : SizeDevice
;
; Input : ES:SI - points to device file to be sized
;
; Output : Carry set if file cannot be opened or if it is an OS2EXE file
;
; Calculates the size of the device file in paras and stores it
; in DevSize
;-----
; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
SizeDevice:
; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; 11/12/2022 ; *
push     ds ; *
push     es
pop      ds
mov      dx,si          ; ds:dx -> file name
mov      ax,3D00h       ; open
int      21h
jc       short sd_err    ; open failed

mov      bx,ax           ; BX - file handle
mov      ax,4202h        ; seek
xor      cx,cx
mov      dx,cx           ; to end of file
int      21h
jc       short sd_close  ; did seek fail (impossible)
add      ax,15           ; para convert
adc      dx,0
test     dx,0FFF0h       ; size > 0ffffh paras ?
;jz      short szdev1    ; no
; 22/07/2023
jz       short sd_ctp
mov      word [cs:DevSize],0FFFFh ; invalid device size
; assuming that we fail later
jmp      short sd_close

sd_ctp:
; 22/07/2023
;szdev1:
mov      cl,4            ; convert it to paras
shr      ax,cl
mov      cl,12
shl      dx,cl
or       ax,dx ; * ; cf=0
;
; 22/07/2023 - Retro DOS v4.2 IO.SYS (optimized)
; MSDOS 6.21 IO.SYS - SYSINIT:37A6h
; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
;cmp     ax,[cs:DevSizeOption]
;ja      short szdev2
;mov     ax,[cs:DevSizeOption]
; 12/12/2022
;clc
;szdev2:
mov      [cs:DevSize],ax ; save file size (in paragraphs)
; 22/07/2023
;clc ; cf=0 ; * ; CLC is not needed here
; (OR instruction clears CF) - E.TAN 22/07/2023
;
; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; 12/12/2022
; cf=0
;clc
sd_close:
pushf
; let close not spoil our
; carry flag
mov      ax,3E00h        ; close
int      21h             ; we are not checking for err
popf
sd_err:
; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; 11/12/2022 ; *
pop      ds ; *
retn
;-----
; procedure : ExecDev
;
; Input : ds:dx -> device to be executed
; DevLoadAddr - contains where device has to be loaded
;
; Output : Carry if error
; Carry clear if no error
;
; Loads a device driver using the 4b03h function call
;-----
; 01/11/2022
ExecDev:
mov      bx,[cs:DevLoadAddr]
mov      [cs:DevExecAddr],bx ; Load the parameter block
; block for exec with

```

```

38908                                     ; load address
38909 000036AE 2E891E[4D24]      mov     [cs:DevExecReloc],bx
38910 000036B3 8CCB             mov     bx,cs
38911 000036B5 8EC3             mov     es,bx
38912 000036B7 BB[4B24]         mov     bx,DevExecAddr      ; es:bx points to parameters
38913                                     ;mov     al,3      ; (load program only)
38914                                     ;mov     ah,EXEC ; 4Bh
38915                                     ; 04/07/2023
38916 000036BA B8034B           mov     ax,(EXEC<<8)|03h
38917 000036BD CD21             int     21h      ; load in the device driver
38918                                     ; DOS - 2+ - LOAD OR EXECUTE (EXEC)
38919                                     ; DS:DX -> ASCIZ filename
38920                                     ; ES:BX -> parameter block
38921                                     ; AL = subfunction
38922 000036BF C3               retn
38923
38924 -----
38925 ;
38926 ; procedure : RetFromUM
38927 ;
38928 ; Input : None
38929 ; Output : ConvLoad set if didn't previously call HideUMBs
38930 ; ConvLoad clear if did.
38931 ;
38932 ; Prepares memory for more devices after returning from loading one
38933 ; using the DOS 6 options (/L:... etc).
38934 -----
38935 ;
38936 ; 31/12/2022 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
38937 ; (SYSINIT:37D1h)
38938 ;
38939 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
38940 ;%if 0
38941 RetFromUM:
38942 ; 31/12/2022
38943 ; ds = cs
38944 pushf
38945 000036C0 9C               ;mov     byte [cs:ConvLoad],1
38946                                     mov     byte [ConvLoad],1
38947 000036C1 C606[3F24]01     call    UnHideUMBs
38948 000036C6 E8ECFD         jc     short rfUM1      ; Skip this if didn't HideUMBs
38949 000036C9 7204             ; 31/12/2022
38950                                     ; ds = cs
38951                                     ;mov     byte [cs:ConvLoad],0
38952                                     mov     byte [ConvLoad],0
38953                                     ; 09/09/2023
38954 000036CB FE0E[3F24]     dec     byte [ConvLoad] ; -> 0
38955 rfUM1:
38956 popf
38957 000036CF 9D               retn
38958 000036D0 C3
38959
38960 ;%endif ; 01/11/2022
38961 -----
38962 ;
38963 ; procedure : RemoveNull
38964 ;
38965 ; Input : ES:SI points to a null terminated string
38966 ;
38967 ; Output : none
38968 ;
38969 ; Replaces the null at the end of a string with blank
38970 -----
38971 ;
38972 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
38973 ; (SYSINIT:2CCEh)
38974 RemoveNull:
38975 ; 11/12/2022
38976 ; ds = cs
38977 rn_next:
38978 mov     bl,[es:si]
38979 or      bl,bl      ; null ?
38980 jz      short rn_gotnull
38981 inc     si      ; advance the pointer
38982 jmp     short rn_next
38983 rn_gotnull:
38984 ; 11/12/2022
38985 mov     bl,[DevSavedDelim]
38986 ;mov     bl,[cs:DevSavedDelim]
38987 000036DB 8A1E[6424]     mov     [es:si],bl      ; replace null with blank
38988                                     ; 02/11/2022
38989 ; 11/12/2022
38990 rba_ok:      ; 10/04/2019
38991 retn
38992 -----
38993 ;
38994 ; procedure : RoundBreakAddr
38995 ;
38996 ; Input : DevBrkAddr
38997 ; Output : DevBrkAddr
38998 ;
38999 ; Rounds DevBrkAddr to a para address so that it is of the form xxxx:0
39000 -----
39001 ;
39002 RoundBreakAddr:
39003 mov     ax,[cs:DevBrkAddr]
39004 call    ParaRound
39005 add     [cs:DevBrkAddr+2],ax
39006 mov     word [cs:DevBrkAddr],0
39007 000036E3 2EA1[3B24]     mov     ax,[cs:DevLoadEnd]
39008 000036E7 E82BDC         cmp     [cs:DevBrkAddr+2],ax
39009 000036EA 2E0106[3D24]   jbe     short rba_ok
39010 000036EF 2EC706[3B24]0000 jmp     mem_err
39011 000036F6 2EA1[3524]     ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39012 000036FA 2E3906[3D24]   ; 11/12/2022
39013 000036FF 76E1             rba_ok:
39014 00003701 E92911         retn
39015
39016 -----
39017 ;
39018 ; procedure : DevSetBreak
39019 ;
39020 ; Input : DevBrkAddr
39021 ; Output : Carry set if Device returned Init failed
39022 ; Else carry clear
39023 -----
39024 ;
39025 DevSetBreak:
39026 push    ax
39027
39028
39029
39030
39031 00003704 50

```

```

39032
39033 00003705 2EA1[3D24]      mov     ax,[cs:DevBrkAddr+2] ;remove the init code
39034 00003709 2E803E[6619]00    cmp     byte [cs:multdeviceflag],0
39035 0000370F 750F      jne     short set_break_continue ;do not check it.
39036 00003711 2E3B06[3324]    cmp     ax,[cs:DevLoadAddr]
39037 00003716 7508      jne     short set_break_continue ;if not same, then o.k.
39038
39039      ;cmp     word [cs:DevBrkAddr],0
39040      ;je     short break_failed ;[DevBrkAddr+2]=[memhi] & [DevBrkAddr]=0
39041      ; 12/12/2022
39042 00003718 2E833E[3B24]01    cmp     word [cs:DevBrkAddr],1
39043 0000371E 7204      jb     short break_failed
39044
39045 set_break_continue:
39046 00003720 E8C0FF    call    RoundBreakAddr
39047      ; 12/12/2022
39048 00003723 F8      cld
39049 break_failed:
39050 00003724 58      pop     ax
39051      ;cld
39052 00003725 C3      retn
39053
39054      ; 12/12/2022
39055 ;break_failed:
39056      ;pop     ax
39057      ;stc
39058      ;retn
39059
39060 ;-----
39061 ;
39062 ; procedure : DevBreak
39063 ;
39064 ; Input : DevLoadAddr & DevBrkAddr
39065 ; Output : none
39066 ;
39067 ; Marks a succesful install of a device driver
39068 ; Sets device size field in sub-arena &
39069 ; Updates Free ptr in UMB or adjusts memhi
39070 ;
39071 ;-----
39072
39073      ; 11/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
39074 DevBreak:
39075      ;push     ds ; 11/12/2022
39076
39077      ; 11/12/2022
39078 00003726 0E      push    cs
39079 00003727 1F      pop     ds
39080      ;mov     ax,[cs:DevLoadAddr]
39081      ;mov     bx,[cs:DevBrkAddr+2]
39082 00003728 A1[3324]    mov     ax,[DevLoadAddr]
39083 0000372B 8B1E[3D24]    mov     bx,[DevBrkAddr+2]
39084      ; 11/12/2022
39085 0000372F 1E      push    ds
39086
39087 00003730 48      dec     ax ; seg of sub-arena
39088 00003731 8ED8    mov     ds,ax
39089 00003733 40      inc     ax ; Back to Device segment
39090 00003734 29D8    sub     ax,bx
39091 00003736 F7D8    neg     ax ; size of device in paras
39092 00003738 A30300    mov     [devmark.size],ax ; store it in sub-arena
39093
39094      ; 11/12/2022
39095 0000373B 1F      pop     ds
39096      ; ds = cs
39097
39098 0000373C 803E[4F24]00    cmp     byte [DeviceHi],0
39099      ;cmp     byte [cs:DeviceHi],0
39100 00003741 7405      je     short db_lo
39101      ;mov     [cs:DevUMBFree],bx ; update Free ptr in UMB
39102      ;jmp     short db_exit
39103      ; 11/12/2022
39104 00003743 891E[4524]    mov     [DevUMBFree],bx
39105 00003747 C3      retn
39106 db_lo:
39107      ; 11/12/2022
39108      ; ds = cs
39109      ;mov     [cs:memhi],bx
39110      ;mov     word [cs:memlo],0
39111 00003748 891E[6403]    mov     [memhi],bx
39112 0000374C C706[6203]0000    mov     word [memlo],0 ; 18/12/2022
39113
39114 db_exit:
39115      ;pop     ds ; 11/12/2022
39116 00003752 C3      sd_ret: ; 09/09/2023
39117      retn
39118
39119 ; 10/04/2019 - Retro DOS v4.0
39120
39121 ;-----
39122 ;
39123 ; procedure : ParseSize
39124 ;
39125 ; Parses the command line for SIZE= command
39126 ;
39127 ; ES:SI = command line to parsed
39128 ;
39129 ; returns ptr to command line after SIZE= option in ES:SI
39130 ; updates the DevSizeOption variable with value supplied
39131 ; in SIZE=option
39132 ; Returns carry if the SIZE option was invalid
39133 ;
39134 ;-----
39135      ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
39136      ; (SYSINIT:2D5Ah)
39137
39138      ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization) (&BugFix))
39139      ; (MSDOS 6.21 IO.SYS - SYSINIT:3871h) - Retro DOS v4.2 -
39140      ; (PCDOS 7.1 IO.SYS - SYSINIT:3D6Eh) - Retro DOS v5.0 -
39141 ParseSize:
39142      ;push     bx
39143      ;mov     bx,si
39144
39145      ; 09/09/2023
39146 00003753 56      push    si ; * ; mov bx,si
39147
39148      ; 11/12/2022
39149      ; ds = cs
39150      ;mov     word [cs:DevSizeOption],0 ; init the value
39151      ;mov     [cs:DevCmdLine],si
39152      ;mov     [cs:DevCmdLine+2],es
39153 00003754 C706[5024]0000    mov     word [DevSizeOption],0 ; init the value
39154 0000375A 8936[6024]    mov     [DevCmdLine],si
39155 0000375E 8C06[6224]    mov     [DevCmdLine+2],es

```



```

39156 00003762 E82400      call    SkipDelim
39157 00003765 26813C5349    cmp     word [es:si], 'SI' ; 4953h
39158 0000376A 7528      jne     short ps_no_size
39159 0000376C 26817C025A45    cmp     word [es:si+2], 'ZE' ; 455Ah
39160 00003772 7520      jne     short ps_no_size
39161 00003774 268A4404    mov     al, [es:si+4]
39162 00003778 E80D10      call    delim
39163      ;jne     short ps_no_size
39164      ; 22/07/2023
39165 0000377B 7518      jne     short ps_no_size_2 ; cf=0 here
39166 0000377D 83C605    add     si, 5
39167 00003780 E81400      call    GetHexNum
39168 00003783 7210      jc      short ps_err
39169      ; 11/12/2022
39170      ; ds = cs
39171      ;mov     [cs:DevSizeOption], ax
39172 00003785 A3[5024]    mov     [DevSizeOption], ax
39173
39174      ; 09/09/2023
39175 00003788 58      pop     ax ; * (discard previous si value on top of stack)
39176
39177      ; call    SkipDelim ; **
39178      ;
39179      ; 22/07/2023
39180      ;ps_no_size_2:
39181      ; cf = 0
39182      ; retn
39183
39184      ; 09/09/2023
39185      ;jmp     short SkipDelim
39186
39187      ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
39188      %if 1
39189      ; 01/11/2022
39190      SkipDelim:
39191      sd_next_char:
39192 00003789 268A04    mov     al, [es:si]
39193 0000378C E8F90F    call    delim
39194 0000378F 75C1      jnz     short sd_ret ; cf=0 ; 09/09/2023
39195 00003791 46      inc     si
39196 00003792 EBF5      jmp     short sd_next_char ; 01/11/2022
39197      ; 11/12/2022
39198      ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39199      ;sd_ret:
39200      ;retn
39201      %endif
39202
39203      ;;;call SkipDelim ; **
39204      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39205      ;mov     bx, si
39206      ps_no_size:
39207      ;mov     si, bx
39208      ;pop     bx
39209 00003794 F8      clc     ; cf=0
39210      ;retn
39211      ; 11/12/2022
39212      ps_err:      ; cf=1
39213      ps_no_size_2: ; 09/09/2023 (cf=0)
39214      ; 09/09/2023
39215 00003795 5E      pop     si ; * ; mov si, bx
39216      ;sd_ret:      ; cf=?
39217 00003796 C3      retn
39218
39219      ;ps_err:
39220      ; 02/11/2022
39221      ;pop     bx
39222      ;stc
39223      ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39224      ; 11/12/2022
39225      ; cf=1
39226      ;stc
39227      ; 11/12/2022
39228      ;sd_ret:
39229      ; 22/07/2023
39230      ; 12/04/2019
39231      ;retn
39232
39233      ; 12/04/2019 - Retro DOS v4.0
39234
39235      ;-----
39236      ;
39237      ; procedure : SkipDelim
39238      ;
39239      ; Skips delimiters in the string pointed to by ES:SI
39240      ; Returns ptr to first non-delimiter character in ES:SI
39241      ;
39242      ;-----
39243
39244      ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
39245      %if 0
39246      ; 01/11/2022
39247      SkipDelim:
39248      sd_next_char:
39249      mov     al, [es:si]
39250      call    delim
39251      jnz     short sd_ret
39252      inc     si
39253      jmp     short sd_next_char ; 01/11/2022
39254      ; 11/12/2022
39255      ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39256      ;sd_ret:
39257      ;retn
39258      %endif
39259
39260      ;-----
39261      ;
39262      ; procedure : GetHexNum
39263      ;
39264      ; Converts an ascii string terminated by a delimiter into binary.
39265      ; Assumes that the ES:SI points to a Hexadecimal string
39266      ;
39267      ; Returns in AX the number number of paras equivalent to the
39268      ; hex number of bytes specified by the hexadecimal string.
39269      ;
39270      ; Returns carry in case it encountered a non-hex character or
39271      ; if it encountered crlf
39272      ;
39273      ;-----
39274
39275      ; 13/05/2019
39276
39277      ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
39278      ; (SYSINIT:38C5h)
39279

```

```

39280             ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
39281             ; (SYSINIT:2DA5h)
39282 GetHexNum:
39283     xor     ax,ax
39284     xor     dx,dx
39285 ghn_next:
39286     mov     bl,[es:si]
39287     cmp     bl,cr    ; 0Dh
39288     je      short ghn_err
39289     cmp     bl,lf    ; 0Ah
39290     je      short ghn_err
39291     push    ax
39292     mov     al,bl
39293     call    delim
39294     pop     ax
39295     ; 03/01/2023
39296     mov     cx,4
39297     jz      short ghn_into_paras
39298     call    GetNibble
39299     ;jc      short ghn_err
39300     ; 11/12/2022
39301     jc      short ghn_ret ; cf=1
39302     ; 03/01/2023
39303     ;mov     cx,4
39304 ghn_shift1:
39305     shl     ax,1
39306     rcl     dx,1
39307     loop    ghn_shift1
39308     or      al,bl
39309     inc     si
39310     jmp     short ghn_next
39311 ghn_into_paras:
39312     add     ax,15
39313     adc     dx,0
39314     test    dx,0FFF0h
39315     jnz     short ghn_err
39316     ; 03/01/2023
39317     ;mov     cx,4
39318 ghn_shift2:
39319     cll
39320     rcr     dx,1
39321     rcr     ax,1
39322     loop    ghn_shift2
39323     cll
39324     retn
39325     ; 11/12/2022
39326 ghn_err:
39327 gnib_err:
39328     stc
39329 ghn_ret:
39330 gnib_ret:
39331     retn
39332
39333 ;-----
39334 ;
39335 ; procedure : GetNibble
39336 ;
39337 ; Convert one nibble (hex digit) in BL into binary
39338 ;
39339 ; Returns binary value in BL
39340 ;
39341 ; Returns carry if BL contains non-hex digit
39342 ;
39343 ;-----
39344
39345 GetNibble:
39346     cmp     bl,'0'
39347     ;jb      short gnib_err
39348     ; 11/12/2022
39349     jb      short gnib_ret ; cf=1
39350     cmp     bl,'9'
39351     ja      short is_it_hex
39352     sub     bl,'0'    ; cll
39353     retn
39354 is_it_hex:
39355     cmp     bl,'A'
39356     ;jb      short gnib_err
39357     ; 11/12/2022
39358     jb      short gnib_ret ; cf=1
39359     cmp     bl,'F'
39360     ja      short gnib_err ; 11/12/2022
39361     sub     bl,'A'-10    ; cll
39362     retn
39363     ; 11/12/2022
39364 ;gnib_err:
39365 ;
39366 ; stc
39367 ;gnib_ret:
39368 ;
39369 ; retn
39370 ;=====
39371
39372 ; 12/04/2019 - Retro DOS v4.0
39373
39374 ; umb.inc (MSDOS 6.0, 1991)
39375 DOS_ARENA     equ 24h    ; offset of arena_head var in DOS data segm.
39376 UMB_ARENA     equ 8Ch    ; offset of umb_head in DOS data
39377
39378 XMM_REQUEST_UMB     equ 10h
39379 XMM_RELEASE_UMB     equ 11h
39380
39381 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
39382
39383 ;-----
39384 ;
39385 ; Procedure Name : umb_insert
39386 ;
39387 ; Inputs
39388 ; : DOSDATA:UMB_HEAD = start of umb chain
39389 ; : BX = seg address of UMB to be linked in
39390 ; : DX = size of UMB to be linked in paras
39391 ; : DS = data
39392 ;
39393 ; Outputs
39394 ; : links the UMB into the arena chain
39395 ;
39396 ; Uses
39397 ; : AX, CX, ES, DX, BX
39398 ;-----
39399 umb_insert:
39400     push    ds
39401     ; 31/12/2022
39402     ; ds = cs
39403

```



```

39404             ;mov     ds,[cs:DevDOSData]
39405 000037F8 8E1E[5E24]    mov     ds,[DevDOSData] ; 31/12/2022
39406             ;mov     ds,[8Ch]
39407 000037FC 8E1E8C00    mov     ds,[UMB_ARENA]          ; es = UMB_HEAD
39408 00003800 8CD8        mov     ax,ds
39409 00003802 8EC0        mov     es,ax
39410 ui_next:
39411 00003804 39D8        cmp     ax,bx                ; Q: is current block above
39412                                     ; new block
39413 00003806 770F        ja     short ui_insert          ; Y: insert it
39414                                     ; Q: is current block the
39415                                     ; last
39416 00003808 26803E00005A    cmp     byte [es:ARENA.SIGNATURE],arena_signature_end ; 'Z'
39417 0000380E 745C        je     short ui_append          ; Y: append new block to chain
39418                                     ; N: get next block
39419 00003810 8ED8        mov     ds,ax                ; M005
39420             ;call    get_next          ; ax = es = next block
39421 00003812 E83B01    call    _get_next_ ; 13/04/2019 - Retro DOS v4.0
39422 00003815 EBED        jmp     short ui_next
39423
39424 ui_insert:
39425             mov     cx,ds                ; ds = previous arena
39426 00003819 41          inc     cx                ; top of previous block
39427
39428             sub     cx,bx
39429 0000381C F7D9        neg     cx                ; cx = size of used block
39430             ;mov     byte [0],'M'
39431 0000381E C60600004D    mov     byte [ARENA.SIGNATURE],arena_signature_normal ; 'M'
39432             ;mov     word [1],8
39433 00003823 C70601000800    mov     word [ARENA.OWNER],8 ; mark as system owned
39434             ;mov     [3],cx
39435 00003829 890E0300    mov     [ARENA.SIZE],cx
39436             ;mov     word [8],4353h ; 'SC'
39437 0000382D C70608005343    mov     word [ARENA.NAME],'SC' ; 4353h
39438
39439             ; prepare the arena at start of new block
39440
39441             mov     es,bx
39442 00003835 26C60600004D    mov     byte [es:ARENA.SIGNATURE],arena_signature_normal ; 'M'
39443 0000383B 26C70601000000    mov     word [es:ARENA.OWNER],arena_owner_system ; 0
39444             ; mark as free
39445 00003842 83EA02        sub     dx,2                ; make room for arena at
39446                                     ; start & end of new block
39447 00003845 2689160300    mov     [es:ARENA.SIZE],dx
39448
39449             ; prepare arena at end of new block
39450
39451             add     bx,dx
39452 0000384C 43          inc     bx
39453 0000384D 8EC3        mov     es,bx                ; es=arena at top of new block
39454 0000384F 43          inc     bx                ; bx=top of new block
39455
39456                                     ; ax contains arena just above
39457                                     ; this block
39458 00003850 29D8        sub     ax,bx                ; ax = size of used block
39459
39460             mov     byte [es:ARENA.SIGNATURE],arena_signature_normal
39461 00003858 26C70601000800    mov     word [es:ARENA.OWNER],8 ; mark as system owned
39462 0000385F 26A30300    mov     [es:ARENA.SIZE],ax
39463 00003863 26C70608005343    mov     word [es:ARENA.NAME],'SC' ; 4353h
39464
39465             jmp     short ui_done
39466
39467 ui_append:
39468                                     ; es = arena of last block
39469 0000386C 2603060300    add     ax,[es:ARENA.SIZE] ; ax=top of last block-1 para
39470 00003871 26832E030001    sub     word [es:ARENA.SIZE],1; reflect the space we are
39471                                     ; going to rsrv on top of this
39472                                     ; block for the next arena.
39473             ; 13/05/2019
39474 00003877 26C60600004D    mov     byte [es:ARENA.SIGNATURE],arena_signature_normal
39475
39476             mov     cx,ax                ; cx=top of prev block-1
39477 0000387F 40          inc     ax
39478 00003880 29D8        sub     ax,bx                ; ax=top of prev block -
39479                                     ; seg. address of new block
39480             neg     ax
39481
39482 00003884 8EC1        mov     es,cx                ; ds = arena of unused block
39483
39484             mov     byte [es:ARENA.SIGNATURE],arena_signature_normal
39485 0000388C 26C70601000800    mov     word [es:ARENA.OWNER],8 ; mark as system owned
39486 00003893 26A30300    mov     [es:ARENA.SIZE],ax
39487 00003897 26C70608005343    mov     word [es:ARENA.NAME],'SC'
39488
39489             ; prepare the arena at start of new block
39490             mov     es,bx
39491 000038A0 26C60600005A    mov     byte [es:ARENA.SIGNATURE],arena_signature_end
39492 000038A6 26C70601000000    mov     word [es:ARENA.OWNER],arena_owner_system
39493             ; mark as free
39494 000038AD 4A          dec     dx                ; make room for arena
39495 000038AE 2689160300    mov     [es:ARENA.SIZE],dx
39496
39497 ui_done:
39498 000038B3 1F          uc_done: ; 31/12/2022 ; *!
39499             pop     ds
39500             ; ds = cs ; 31/12/2022
39501             ;uc_done: ; 18/12/2022
39502 000038B4 C3          au_exit: ; 09/09/2023
39503             retn
39504
39505             ;-----
39506             ; procedure : AllocUMB
39507             ;
39508             ; Allocate all UMBs and link it to DOS arena chain
39509             ;
39510             ;-----
39511
39512 AllocUMB:
39513             ; 31/12/2022
39514             ; ds = cs
39515 000038B5 E84700    call    InitAllocUMB          ; link in the first UMB
39516 000038B8 72FA        jc     short au_exit          ; quit on error
39517
39518 ui_next:
39519             call    umb_allocate          ; allocate
39520             jc     short au_coalesce
39521             call    umb_insert            ; & insert till no UMBs
39522             jmp     short au_next
39523
39524 au_coalesce:
39525             ; 09/09/2023
39526             ; call    umb_coalesce          ; coalesce all UMBs
39527             ; au_exit:
39528             ; ; 31/12/2022
39529             ; ; ds = cs

```

```

39528 ;   retn
39529
39530 ;   ; 09/09/2023
39531 ;   jmp     short umb_coalesce
39532
39533 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
39534
39535 ; 13/04/2019 - Retro DOS v4.0
39536
39537 ;-----
39538 ;
39539 ;** umb_coalesce - Combine free blocks ahead with current block
39540 ;
39541 ;   Coalesce adds the block following the argument to the argument block,
39542 ;   if it's free. Coalesce is usually used to join free blocks, but
39543 ;   some callers (such as $setblock) use it to join a free block to it's
39544 ;   preceeding allocated block.
39545 ;
39546 ;   EXIT   'C' clear if OK
39547 ;           (ds) unchanged, this block updated
39548 ;           (ax) = address of next block, IF not at end
39549 ;           'C' set if arena trashed
39550 ;   USES   cx, di, ds, es
39551 ;-----
39552
39553 umb_coalesce:
39554 ; 31/12/2022
39555 ; ds = cs
39556 ; push     ds ; *!
39557 000038C4 1E
39558
39559 000038C5 31FF
39560
39561 ;mov     es,[cs:DevDOSData]
39562 ; 31/12/2022
39563 000038C7 8E06[5E24]
39564 000038CB 268E068C00
39565
39566 000038D0 8CC0
39567 000038D2 8ED8
39568
39569 000038D4 26393E0100
39570 000038D9 7407
39571
39572 000038DB E86B00
39573 000038DE 72D3
39574 000038E0 EBEE
39575
39576 000038E2 E86400
39577 000038E5 72CC
39578
39579 000038E7 26393E0100
39580 000038EC 75E2
39581
39582 000038EE 268B0E0300
39583 000038F3 41
39584
39585 000038F4 010E0300
39586 000038F8 268A0D
39587 000038FB 880D
39588 000038FD EBE3
39589
39590 ; 18/12/2022
39591 ;uc_done:
39592 ;retn
39593
39594 ;-----
39595 ;
39596 ; procedure : InitAllocUMB
39597 ;
39598 ;-----
39599
39600 InitAllocUMB:
39601 ; 31/12/2022
39602 ; ds = cs
39603 000038FF E8D8D2
39604 00003902 7527
39605 00003904 B452
39606 00003906 CD21
39607
39608 ; 31/12/2022
39609 ; ds = cs
39610 00003908 8C06[5E24]
39611 0000390C B81043
39612 0000390F CD2F
39613
39614 ;mov     [cs:DevXMSAddr],bx ; get XMS driver address
39615 00003911 891E[4724]
39616 00003915 8C06[4924]
39617
39618 00003919 803E[5D24]00
39619
39620 ;cmp     byte [cs:FirstUMBLinked],0 ; have we already linked a UMB?
39621 ;jne     short ia_1 ; quit if we already did it
39622 0000391E 770A
39623 00003920 E83900
39624
39625 ;ja      short ia_1 ; cf=0
39626 00003923 7207
39627
39628 ;call    LinkFirstUMB ; else link the first UMB
39629 00003925 C606[5D24]FF
39630
39631 ;mov     byte [FirstUMBLinked],0FFh ; mark that 1st UMB linked
39632 ;mov     byte [cs:FirstUMBLinked],0FFh ; mark that 1st UMB linked
39633
39634 ; 12/12/2022
39635 0000392A C3
39636
39637 ;retn
39638
39639 ;ia_1:
39640 ; 12/12/2022
39641 ; cf=0
39642 ;clc
39643 ;retn
39644
39645 ;ia_err:
39646 ;stc
39647
39648 ;ia_err2:
39649 ;retn
39650
39651 ;-----
39652 ;
39653 ; Procedure Name : umb_allocate
39654 ;
39655 ; Inputs : DS = data
39656 ;
39657 ; Outputs : if UMB available
39658 ;           Allocates the largest available UMB and
39659 ;           BX = segment of allocated block
39660 ;           DX = size of allocated block
39661 ;           NC

```

```

39652 ;                                     else
39653 ;                                     CY
39654 ;
39655 ; Uses                               : BX, DX
39656 ;
39657 ;-----
39658
39659 umb_allocate:
39660 ; 31/12/2022
39661 ; ds = cs
39662 0000392D 50      push    ax
39663 0000392E B410    mov     ah,XMM_REQUEST_UMB ; 16
39664 00003930 BAFfff mov     dx,0FFFFh ; try to allocate largest
39665 ;                                     ; possible
39666 ; 31/12/2022
39667 00003933 FF1E[4724] call   far [DevXMSAddr]
39668 ; call   far [cs:DevXMSAddr]
39669 ; dx now contains the size of
39670 ; the largest UMB
39671 00003937 09D2    or      dx,dx
39672 00003939 740B    jz      short ua_err
39673
39674 0000393B B410    mov     ah,XMM_REQUEST_UMB ; 16
39675
39676 ; 31/12/2022
39677 0000393D FF1E[4724] call   far [DevXMSAddr]
39678 ; call   far [cs:DevXMSAddr]
39679
39680 00003941 83F801   cmp     ax,1 ; Q: was the reqst successful
39681 ; jne     short ua_err ; N: error
39682 ; 27/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
39683 00003944 7601   jna     short ua_done ; if ax=1 then cf=0, else cf=1 (ax=0)
39684 ua_err:
39685 00003946 F9      stc
39686
39687 ; clc
39688 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39689 ; 12/12/2022
39690 ; cf=0
39691 ; clc
39692 ua_done:
39693 00003947 58      pop     ax
39694 00003948 C3      retn
39695 ; 27/07/2023
39696 ; ua_err:
39697 ; stc
39698 ; jmp     short ua_done
39699
39700 ;-----
39701 ;
39702 ; ** get_next - Find Next item in Arena
39703 ;
39704 ; ENTRY ds - pointer to block head
39705 ; EXIT  AX,ES - pointers to next head
39706 ; 'C' set if arena damaged
39707 ;
39708 ;-----
39709
39710 ; 01/11/2022
39711 get_next:
39712 00003949 803E00005A cmp     byte [0],arena_signature_end ; 'z'
39713 0000394E 740A    je      short gn_err
39714
39715 00003950 8CD8    mov     ax,ds ; ax=current block
39716 00003952 03060300 add     ax,[ARENA.SIZE] ; ax=ax + current block length
39717 00003956 40      inc     ax ; remember that header!
39718 00003957 8EC0    mov     es,ax
39719 ; clc
39720 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39721 ; 11/12/2022
39722 ; cf=0
39723 ; clc
39724 00003959 C3      retn
39725
39726 0000395A F9      gn_err:
39727 ; 11/12/2022
39728 ; cf=1
39729 0000395B C3      lfu_err:
39730 retn
39731
39732 ;-----
39733 ;
39734 ; procedure : LinkFirstUMB
39735 ;-----
39736
39737 ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
39738 ; (SYSINIT:2F81h)
39739 LinkFirstUMB:
39740 ; 31/12/2022
39741 ; ds = cs
39742 0000395C E8CEFF   call   umb_allocate
39743 0000395F 72FA    jc      short lfu_err ; ds = cs ; 31/12/2022
39744
39745 ; bx = segment of allocated UMB
39746 ; dx = size of UMB
39747
39748 ; 31/12/2022
39749 ; ds = cs
39750
39751 00003961 CD12    int     12h ; ax = size of memory
39752 00003963 B106    mov     cl,6
39753 00003965 D3E0    shl     ax,cl ; ax = size in paragraphs
39754
39755 00003967 89C1    mov     cx,ax ; cx = size in paras
39756 00003969 29D8    sub     ax,bx ; ax = - size of unused block
39757
39758 0000396B F7D8    neg     ax
39759
39760 ; sub     cx,1 ; cx = first umb_arena
39761 ; 09/09/2023
39762 0000396D 49      dec     cx
39763 0000396E 8EC1    mov     es,cx ; es = first umb_arena
39764
39765 00003970 26C60600004D mov     byte [es:ARENA.SIGNATURE],arena_signature_normal ; 'M'
39766 00003976 26C70601000800 mov     word [es:ARENA.OWNER],8 ; mark as system owned
39767
39768 0000397D 26A30300    mov     [es:ARENA.SIZE],ax
39769 00003981 26C70608005343 mov     word [es:ARENA.NAME],'SC' ; 4353h
39770
39771 ; put in the arena for the first UMB
39772
39773 00003988 8EC3    mov     es,bx ; es has first free umb seg
39774 0000398A 26C60600005A mov     byte [es:ARENA.SIGNATURE],arena_signature_end ; 'z'
39775 00003990 26C70601000000 mov     word [es:ARENA.OWNER],arena_owner_system ; 0

```

```

39776                                     ; mark as free
39777 00003997 4A                         dec     dx             ; make room for arena
39778 00003998 2689160300                mov     [es:ARENA.SIZE],dx
39779
39780                                     ;mov     es,[cs:DevDOSData]
39781                                     ; 31/12/2022
39782 0000399D 8E06[5E24]                mov     es,[DevDOSData] ; ds = cs
39783 000039A1 BF8C00                     mov     di,UMB_ARENA ; 8Ch
39784 000039A4 26890D                     mov     [es:di],cx      ; initialize umb_head in DOS
39785                                     ; data segment with the arena
39786                                     ; just below Top of Mem
39787
39788                                     ; we must now scan the arena chain and update the size of the last arena
39789
39790 000039A7 BF2400                     mov     di,DOS_ARENA ; 24h
39791 000039AA 268E05                     mov     es,[es:di]      ; es = start arena
39792 000039AD 31FF                     xor     di,di
39793                                     ;scan_next
39794                                     ; 09/12/2022
39795 scannext:
39796 000039AF 26803D5A                   cmp     byte [es:di],arena_signature_end ; 'Z'
39797 000039B3 740C                     je      short got_last
39798
39799 000039B5 8CC0                     mov     ax,es
39800 000039B7 2603060300                add     ax,[es:ARENA.SIZE]
39801 000039BC 40                     inc     ax
39802 000039BD 8EC0                     mov     es,ax
39803                                     ;jmp     short scan_next
39804                                     ; 09/12/2022
39805 000039BF EBEE                     jmp     short scannext
39806 got_last:
39807                                     ;sub     word [es:ARENA.SIZE],1
39808                                     ; 09/09/2023
39809 000039C1 26FF0E0300                dec     word [es:ARENA.SIZE]
39810
39811 000039C6 26C60600004D                mov     byte [es:ARENA.SIGNATURE],arena_signature_normal ; 'M'
39812                                     ;clc
39813                                     ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39814                                     ; 11/12/2022
39815                                     ; cf=0
39816                                     ;clc
39817 000039CC C3                         retn
39818
39819                                     ; 11/12/2022
39820                                     ;;lfu_err:
39821                                     ; ;stc
39822                                     ; ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
39823                                     ; ; 11/12/2022
39824                                     ; ; cf=1
39825                                     ; ;stc
39826                                     ; ;retn
39827
39828                                     ;-----
39829                                     ;
39830                                     ; procedure : ShrinkUMB
39831                                     ;
39832                                     ; Shrinks the current UMB in use, so that the unused portions
39833                                     ; of the UMB is given back to the DOS free mem pool
39834                                     ;-----
39835                                     ;
39836
39837 shrinkUMB:
39838                                     ; 12/12/2022
39839                                     ; ds = cs
39840 000039CD 833E[4124]00                cmp     word [DevUMBAddr],0
39841                                     ;cmp     word [cs:DevUMBAddr],0
39842 000039D2 741F                     je      short su_exit
39843 000039D4 06                     push    es
39844                                     ; 01/01/2023
39845                                     ;push    bx
39846                                     ; 12/12/2022
39847                                     ;mov     bx,[cs:DevUMBFree]
39848                                     ;sub     bx,[cs:DevUMBAddr]
39849                                     ;mov     es,[cs:DevUMBAddr]
39850 000039D5 8B1E[4524]                mov     bx,[DevUMBFree]
39851 000039D9 2B1E[4124]                sub     bx,[DevUMBAddr]
39852 000039DD 8E06[4124]                mov     es,[DevUMBAddr]
39853
39854 000039E1 B8004A                     mov     ax,4A00h
39855 000039E4 CD21                     int     21h
39856                                     ; DOS - 2+ - ADJUST MEMORY BLOCK SIZE (SETBLOCK)
39857                                     ; ES = segment address of block to change
39858                                     ; BX = new size in paragraphs
39859 000039E6 8CC0                     mov     ax,es
39860 000039E8 48                     dec     ax
39861 000039E9 8EC0                     mov     es,ax
39862 000039EB 26C70601000800                mov     word [es:ARENA.OWNER],8
39863                                     ; 01/01/2023
39864                                     ;pop     bx
39865 000039F2 07                     pop     es
39866 su_exit:
39867 000039F3 C3                         retn
39868
39869                                     ;-----
39870                                     ;
39871                                     ; procedure : UnlinkUMB
39872                                     ;
39873                                     ; Unlinks the UMBs from the DOS arena chain
39874                                     ;-----
39875                                     ;
39876
39877 unlinkUMB:
39878                                     ; 12/12/2022
39879                                     ; ds = cs
39880 000039F4 1E                     push    ds
39881 000039F5 06                     push    es
39882                                     ; 12/12/2022
39883 000039F6 803E[5D24]00                cmp     byte [FirstUMBLinked],0
39884                                     ;cmp     byte [cs:FirstUMBLinked],0
39885 000039FB 7420                     je      short ulu_x      ; nothing to unlink
39886                                     ; 12/12/2022
39887 000039FD 8E06[5E24]                mov     es,[DevDOSData]
39888                                     ;mov     es,[cs:DevDOSData] ; get DOS data seg
39889 00003A01 268E1E2400                mov     ds,[es:DOS_ARENA]
39890 00003A06 268B3E8C00                mov     di,[es:UMB_ARENA]
39891 ulu_next:
39892 00003A0B E83BFF                     call    get_next
39893 00003A0E 720D                     jc      short ulu_x
39894 00003A10 39C7                     cmp     di,ax            ; is the next one UMB ?
39895 00003A12 7404                     je      short ulu_found
39896 00003A14 8ED8                     mov     ds,ax
39897 00003A16 EBF3                     jmp     short ulu_next
39898 ulu_found:
39899                                     ;mov     byte [0], 'Z'

```

```

39900 00003A18 C60600005A      mov     byte [ARENA.SIGNATURE],arena_signature_end ; 'z'
39901                          ulu_x:
39902 00003A1D 07                pop     es
39903 00003A1E 1F                pop     ds
39904 00003A1F C3                retn
39905
39906                          ; -----
39907                          ; SYSINIT2.ASM - MSDOS 6.0 - 1991
39908                          ; -----
39909                          ; 14/04/2019 - Retro DOS v4.0
39910
39911                          ; Multiple configuration block support Created 16-Mar-1992 by JeffPar
39912
39913                          ; Summary:
39914                          ;
39915                          ; The procedure "organize" crunches the in-memory copy of config.sys
39916                          ; into lines delimited by CR/LF (sometimes no CR, but *always* an LF)
39917                          ; with the leading "keyword=" replaced by single character codes (eg, B
39918                          ; for BUFFERS, D for DEVICE, Z for any unrecognized keyword); see comtab
39919                          ; and/or config.inc for the full list.
39920                          ;
39921                          ; [blockname] and INCLUDE are the major syntactical additions for multi-
39922                          ; configuration support. blockname is either MENU, which contains one
39923                          ; or more MENUITEM lines, an optional MENUEFAULT (which includes optional
39924                          ; time-out), or any user-defined keyword, such as NETWORK, CD-ROM, etc.
39925                          ; INCLUDE allows the current block to name another block for inclusion
39926                          ; during the processing phase of CONFIG.SYS. An INCLUDE is only honored
39927                          ; once, precluding nasty infinite-loop scenarios. If blocks are present
39928                          ; without a MENU block, then only lines inside COMMON blocks are processed.
39929                          ;
39930                          ; Example:
39931                          ;
39932                          ; [menu]
39933                          ; menuitem=misc,Miscellaneous
39934                          ; menuitem=network,Network Configuration
39935                          ; menudefault=network,15
39936                          ;
39937                          ; [network]
39938                          ; include misc
39939                          ; device=foo
39940                          ;
39941                          ; [misc]
39942                          ; device=bar
39943                          ; include alternate
39944                          ;
39945                          ; [alternate]
39946                          ; device=tar
39947                          ;
39948                          ;
39949                          ; when the menu is displayed
39950                          ;
39951                          ; 1. Miscellaneous
39952                          ; 2. Network Configuration
39953                          ;
39954                          ; #2 is highlighted as the default option, and will be automatically
39955                          ; selected after 15 seconds. It will invoke the following lines in the
39956                          ; following order:
39957                          ;
39958                          ;     DEVICE=BAR
39959                          ;     DEVICE=TAR
39960                          ;     DEVICE=FOO
39961                          ;
39962
39963                          ;MULTI_CONFIG equ 1
39964
39965                          ; the following depend on the positions of the various letters in switchlist
39966
39967                          switchnum equ 11111000b ; 0F8h ; which switches require number
39968
39969                          flagec35 equ 00000100b ; 4 ; electrically compatible 3.5 inch disk drive
39970                          flagdrive equ 00001000b ; 8
39971                          flagcylin equ 00010000b ; 16
39972                          flagseclim equ 00100000b ; 32
39973                          flagheads equ 01000000b ; 64
39974                          flagfff equ 10000000b ; 128
39975
39976                          ; -----
39977                          ; 19/04/2019 - Retro DOS v4.0
39978
39979                          ; MSDOS 6.21 IO.SYS - SYSINIT:3E78h
39980
39981                          ; 01/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
39982                          ; MSDOS 5.0 IO.SYS - SYSINIT:3054h
39983
39984 00003A20 00      insert_blank: db 0
39985
39986                          ; -----
39987                          ;
39988                          ; procedure : setparms
39989                          ;
39990                          ; the following set of routines is used to parse the drivparm = command in
39991                          ; the config.sys file to change the default drive parameters.
39992                          ;
39993                          ; -----
39994
39995                          setparms:
39996                          push     ds
39997                          push     ax
39998                          push     bx
39999                          push     cx
40000                          push     dx
40001
40002                          push     cs
40003                          pop      ds
40004
40005                          xor     bx,bx
40006 00003A2A 8A1E[1A4F]      mov     bl,[drive]
40007                          ; 18/12/2022
40008 00003A2E 43                inc     bx
40009                          inc     bl ; get it correct for ioctl call
40010                          ; (1=a,2=b...)
40011 00003A2F BA[BC4D]      mov     dx,deviceparameters
40012                          ;mov     ah,IOCTL ; 44h
40013                          ;mov     al,GENERIC_IOCTL ; 0Dh
40014                          ; 04/07/2023
40015 00003A32 B80D44      mov     ax,(IOCTL<<8)|GENERIC_IOCTL
40016                          ;mov     ch,RAWIO ; 8
40017                          ;mov     cl,SET_DEVICE_PARAMETERS ; 40h
40018                          ; 04/07/2023
40019 00003A35 B94008      mov     cx,(RAWIO<<8)|SET_DEVICE_PARAMETERS
40020 00003A38 CD21                int     21h
40021
40022                          ; 27/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
40023 00003A3A 8A26[1B4F]      mov     ah,[switches]

```

```

40024      ;mov     al,[deviceparameters+20]
40025 00003A3E A0[D04D]      mov     al,[deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK]
40026 00003A41 8A0E[1A4F]    mov     cl,[drive]
40027      ;
40028      ; ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40029      ; ;mov     ax,Bios_Data      ; get Bios_Data segment
40030      ; ;mov     ax,KERNEL_SEGMENT ; 70h
40031      ; ; 21/10/2022
40032      ; ;mov     ax,DOSBIODATASEG ; 0070h
40033      ; ;mov     ds,ax      ; set Bios_Data segment
40034      ;
40035      ; ; 27/07/2023
40036      ; ;test word [cs:switches],flagec35 ; 4
40037      ; ;test byte [cs:switches],flagec35
40038      ; ;jz      short not_ec35
40039      ;
40040      ; ; 27/07/2023
40041      ; ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40042      ; ;test word [switches],flagec35 ; 4
40043      ; ; 12/12/2022
40044      ; ;test byte [switches],flagec35 ; 4
40045      ; ;jz      short eot_ok
40046      ;
40047      ; ;mov     cl,[cs:drive]      ; which drive was this for?
40048      ; ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40049      ; ;mov     cl,[drive]
40050      ; ; 27/07/2023
40051      ; ;mov     ax,DOSBIODATASEG ; 0070h
40052      ; ;mov     ds,ax
40053      ;
40054 00003A45 BA7000      mov     dx,DOSBIODATASEG
40055 00003A48 8EDA      mov     ds,dx
40056      ;
40057 00003A4A F6C404      test     ah,flagec35 ; test byte [cs:switches],flagec35
40058 00003A4D 7408      jz      short not_ec35
40059      ;
40060      ; ;mov     al,1      ; assume drive 0
40061      ; ;shl     al,cl      ; set proper bit depending on drive
40062      ; ;or     [531h],al ; (MSDOS 6.21 IO.SYS Offset SYINIT:3EACH)
40063      ; ;or     [ec35_flag],al ; set the bit in the permanent flags
40064      ; ; 27/07/2023
40065 00003A4F B401      mov     ah,1
40066 00003A51 D2E4      shl     ah,cl
40067 00003A53 0826[A204] or      [ec35_flag],ah
40068      ;
40069      ; ; 07/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
40070      ; ; MSDOS 6.21 IO.SYS - SYINIT:3EB0h
40071      ; ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40072 not_ec35:
40073      ; Now adjust the BIOS's EOT variable if our new drive has more
40074      ; sectors per track than any old ones.
40075      ;
40076      ; ; 27/07/2023
40077      ; ;mov     al,[cs:deviceparameters+20]
40078      ; ;mov     al,[cs:deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK]
40079      ;
40080      ; ;cmp     al,[12Ch] ; (MSDOS 6.21 IO.SYS Offset SYINIT:3EB4h)
40081 00003A57 3A06[2C01]    cmp     al,[eot]
40082 00003A5B 7603      jbe     short eot_ok
40083 00003A5D A2[2C01]    mov     [eot],al
40084      ;
40085 00003A60 5A      eot_ok:
40086 00003A61 59      pop     dx ; fix up all the registers
40087 00003A62 5B      pop     cx
40088 00003A63 58      pop     bx
40089 00003A64 1F      pop     ax
40090 00003A65 C3      pop     ds ; 13/05/2019
40091      ;
40092      ;
40093      ;
40094      ; procedure : diddleback
40095      ;
40096      ; replace default values for further drivparm commands
40097      ;
40098      ;
40099      ;
40100      ; diddleback:
40101 00003A66 1E      push    ds
40102 00003A67 0E      push    cs
40103 00003A68 1F      pop     ds
40104      ;
40105 00003A69 C706[C04D]5000    mov     word [deviceparameters+4],80
40106      ;
40107 00003A6F C606[BD4D]02    mov     word [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],80
40108      ;
40109      ;
40110 00003A74 C706[BE4D]0000    mov     byte [deviceparameters+1],2
40111 00003A7A C706[1B4F]0000    mov     byte [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICE_TYPE],DEV_3INCH720KB ; 2
40112 00003A80 1F      ;mov     word [deviceparameters+2],0
40113 00003A81 C3      mov     word [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICE_ATTRIBUTES],0
40114      ;
40115      ;
40116      ;
40117      ;
40118      ;
40119      ;
40120      ;
40121      ;
40122      ;
40123      ; entry point is parseline. al contains the first character in command line.
40124      ;
40125      ;
40126      ;
40127      ; ; 03/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
40128      ; ; (SYSINIT:3EDFh)
40129      ;
40130      ; ; 01/11/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40131      ; ; (SYSINIT:30ACh)
40132      ;
40133      ;
40134      ;
40135      ;
40136      ;
40137      ;
40138      ;
40139      ;
40140      ;
40141      ;
40142      ;
40143      ;
40144      ;
40145      ;
40146      ;
40147      ;

```



```

40148
40149
40150      cmp     al,' '           ; space?
40151      jbe     short getnext    ; skip over space
40152      cmp     al,'/'
40153      je      short getparm    ; mark error invalid-character-in-input
40154      stc
40155      jmp     short exitpl      ; 03/01/2023
40156      ; 03/01/2023
40157      ; swterr:
40158      retn
40159
40160      getparm:
40161      call    check_switch
40162      mov     [switches],bx     ; save switches read so far
40163      jc      short swterr
40164      getnext:
40165      call    getchr
40166      jc      short done_line
40167      jmp     short nextswtch
40168      ; 03/01/2023
40169      jnc     short nextswtch
40170      ; swterr:
40171      jmp     short exitpl      ; exit if error
40172
40173      done_line:
40174      ; 12/12/2022
40175      test    byte [switches],flagdrive ; 8
40176      ; test    word [switches],flagdrive ; 8 ; see if drive specified
40177      jnz     short okay
40178      stc
40179      jmp     short exitpl      ; mark error no-drive-specified
40180      ; 03/01/2023
40181      retn
40182
40183      okay:
40184      mov     ax,[switches]
40185      and     ax,0003h          ; get flag bits for changeline and non-rem
40186      mov     [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES],ax
40187      mov     word [deviceparameters+A_DEVICEPARAMETERS.DP_TRACKTABLEENTRIES],0
40188      ; cllc
40189      ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40190      ; 12/12/2022
40191      ; cf=0
40192      ; cllc
40193      call    setdeviceparameters
40194      ; 03/01/2023
40195      jmp     setdeviceparameters
40196      ; exitpl:
40197      ; 03/01/2023
40198      ; ds = cs
40199      pop     ds ; *
40200      retn
40201      put_back:
40202      inc     word [count]      ; one more char to scan
40203      dec     word [chrptr]     ; back up over linefeed
40204      jmp     short done_line
40205
40206      %endif
40207
40208      ; -----
40209      ; procedure : check_switch
40210      ;
40211      ; processes a switch in the input. it ensures that the switch is valid, and
40212      ; gets the number, if any required, following the switch. the switch and the
40213      ; number *must* be separated by a colon. carry is set if there is any kind of
40214      ; error.
40215      ;
40216      ; -----
40217
40218      ; 09/09/2023
40219
40220      err_swtrch:
40221      xor     bx,cx             ; remove this switch from the records
40222
40223      err_check:
40224      stc
40225      err_chk:
40226      done_swtrch: ; 09/09/2023 (cf=0)
40227      retn
40228
40229      ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
40230
40231      check_switch:
40232      call    getchr
40233      jc      short err_check
40234      jc      short err_chk
40235      and     al,0DFh           ; convert it to upper case
40236      cmp     al,'A'
40237      jb      short err_check
40238      jb      short err_chk ; 15/04/2019 - Retro DOS v4.0
40239      cmp     al,'Z'
40240      ja      short err_check
40241
40242      push    es
40243      push    cs
40244      pop     es
40245
40246      ; mov     cl,[switchlist]           ; get number of valid switches
40247      ; mov     ch,0
40248      ; mov     di,1+switchlist         ; point to string of valid switches
40249      ; 09/09/2023
40250      mov     di,switchlist
40251      mov     cl,[di]
40252      mov     ch,0
40253      inc     di ; 1+switchlist
40254
40255      repne   scasb
40256
40257      pop     es
40258      jnz     short err_check
40259
40260      mov     ax,1
40261      shl     ax,cl              ; set bit to indicate switch
40262      mov     bx,[switches]      ; get switches so far
40263      or      bx,ax              ; save this with other switches
40264      mov     cx,ax
40265      ; 12/12/2022
40266      test    al,switchnum ; 0F8h
40267      ; test    ax,switchnum ; 0F8h ; test against switches that require number to follow
40268      jz      short done_swtrch
40269
40270      call    getchr
40271      jc      short err_swtrch

```

```

40272
40273 00003ABB 3C3A      cmp     al,':'
40274 00003ABD 75C3      jne     short err_swch
40275
40276 00003ABF E8EB0C      call    getchr
40277 00003AC2 53              push    bx                ; preserve switches
40278                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40279                                ;mov     byte [cs:sepchr], ' ' ; allow space separators
40280                                ; 12/12/2022
40281                                ; ds = cs
40282 00003AC3 C606[AE02]20      mov     byte [sepchr], ' '
40283 00003AC8 E8980D      call    getnum
40284                                ;mov     byte [cs:sepchr], 0
40285                                ; 12/12/2022
40286 00003ACB C606[AE02]00      mov     byte [sepchr], 0
40287 00003AD0 5B              pop     bx                ; restore switches
40288
40289                                ; because getnum does not consider carriage-return or line-feed as ok, we do
40290                                ; not check for carry set here. if there is an error, it will be detected
40291                                ; further on (hopefully).
40292
40293                                ; 09/09/2023
40294                                ;call    process_num
40295                                ;jmp     short process_num
40296
40297 ;done_swch:
40298                                ;clc
40299                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40300                                ; 12/12/2022
40301                                ; cf=0
40302                                ;clc
40303                                ;retn
40304
40305 -----
40306
40307 ; procedure : process_num
40308
40309 ; this routine takes the switch just input, and the number following (if any),
40310 ; and sets the value in the appropriate variable. if the number input is zero
40311 ; then it does nothing - it assumes the default value that is present in the
40312 ; variable at the beginning. zero is ok for form factor and drive, however.
40313
40314 -----
40315
40316                                ; 02/11/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40317                                ; (SYSINIT:3156h)
40318 process_num:
40319 00003AD1 850E[1B4F]      test     [switches],cx      ; if this switch has been done before,
40320 00003AD5 752B      jnz     short done_ret      ; ignore this one.
40321                                ; 12/12/2022
40322 00003AD7 F6C108      test     cl,flagdrive ; 8
40323                                ;test    cx,flagdrive ; 8
40324 00003ADA 7404      jz      short try_f
40325 00003ADC A2[1A4F]      mov     byte [drive],al
40326                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40327                                ;jmp     short done_ret
40328                                ; 12/12/2022
40329                                ; cf=0
40330 00003ADF C3              retn     ; 13/05/2019
40331
40332 try_f:
40333                                ; 12/12/2022
40334 00003AE0 F6C180      test     cl,flagff ; 80h
40335 00003AE3 7404      jz      short try_t
40336
40337 ; ensure that we do not get bogus form factors that are not supported
40338
40339                                ;mov     [deviceparameters+1],al
40340 00003AE5 A2[BD4D]      mov     [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICETYPE],al
40341                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40342                                ;jmp     short done_ret
40343                                ; 12/12/2022
40344                                ; cf=0
40345 00003AE8 C3              retn     ; 13/05/2019
40346
40347 00003AE9 09C0      try_t:
40348 00003AEB 7415      or      ax,ax
40349                                ; 12/12/2022
40350 00003AED F6C110      jz      short done_ret      ; if number entered was 0, assume default value
40351                                ; 12/12/2022
40352 00003AF0 7404      test     cl,flagcyl ; 10h
40353                                ;test    cx,flagcyl ; 10h
40354                                ;jz      short try_s
40355                                ;mov     [deviceparameters+4],ax
40356 00003AF2 A3[C04D]      mov     [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],ax
40357                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40358                                ;jmp     short done_ret
40359                                ; 12/12/2022
40360                                ; cf=0
40361 00003AF5 C3              retn     ; 13/05/2019
40362
40363 00003AF6 F6C120      try_s:
40364                                ; 12/12/2022
40365                                ;test    cl,flagseclim ; 20h
40366                                ;test    cx,flagseclim ; 20h
40367                                ;jz      short try_h
40368 00003AFB A3[184F]      mov     [slim],ax
40369                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40370                                ;jmp     short done_ret
40371                                ; 12/12/2022
40372                                ; cf=0
40373 00003AFE C3              retn     ; 13/05/2019
40374
40375 ; must be for number of heads
40376
40377 try_h:
40378                                mov     [hlim],ax
40379 done_ret:
40380                                ;clc
40381                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40382                                ; 12/12/2022
40383                                ; cf=0 (test instruction resets cf)
40384                                ;clc
40385                                ;retn
40386
40387 ; 16/04/2024 - Retro DOS v5.0
40388 ; 03/01/2023 - Retro DOS v4.2
40389 %if 1
40390                                ; 15/04/2019 - Retro DOS v4.0
40391
40392 -----
40393
40394 ; procedure : parseline
40395

```



```

40396 ; entry point is parseline. al contains the first character in command line.
40397 ;
40398 ;-----
40399 ;
40400 ; 16/04/2024 - RetroDOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
40401 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4151h)
40402 ;
40403 ; 03/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
40404 ; (SYSINIT:3EDFh)
40405 ;
40406 ; 01/11/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40407 ; (SYSINIT:30ACh)
40408
40409 parseline:
40410 ; 03/01/2023
40411 ; ds = cs ; *
40412
40413 ;push ds ; *
40414
40415 ;push cs ; *
40416 ;pop ds ; *
40417
40418 nextswtch:
40419 cmp al,cr ; carriage return?
40420 je short done_line
40421 cmp al,lf ; linefeed?
40422 je short put_back ; put it back and done
40423
40424 ; anything less or equal to a space is ignored.
40425
40426 cmp al,' ' ; space?
40427 jbe short getnext ; skip over space
40428 cmp al,'/'
40429 je short getparm
40430 stc ; mark error invalid-character-in-input
40431 jmp short exitpl
40432 ; 03/01/2023
40433 swterr:
40434 retn
40435
40436 getparm:
40437 call check_switch
40438 mov [switches],bx ; save switches read so far
40439 jc short swterr
40440
40441 getnext:
40442 call getchr
40443 jc short done_line
40444 jmp short nextswtch
40445 ; 03/01/2023
40446 jnc short nextswtch
40447 ;swterr:
40448 jmp short exitpl ; exit if error
40449
40450 done_line:
40451 ; 12/12/2022
40452 test byte [switches],flagdrive ; 8
40453 ;test word [switches],flagdrive ; 8 ; see if drive specified
40454 jnz short okay
40455 stc ; mark error no-drive-specified
40456 jmp short exitpl
40457 ; 03/01/2023
40458 retn
40459
40460 ;exitpl:
40461 ; 03/01/2023
40462 ; ds = cs
40463 ;pop ds ; *
40464 ;retn
40465
40466 put_back:
40467 inc word [count] ; one more char to scan
40468 dec word [chrptr] ; back up over linefeed
40469 jmp short done_line
40470
40471 okay:
40472 mov ax,[switches]
40473 and ax,0003h ; get flag bits for changeline and non-rem
40474 mov [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES],ax
40475 ; 16/04/2024
40476 ;mov word [deviceparameters+A_DEVICEPARAMETERS.DP_TRACKTABLEENTRIES],0
40477 ;
40478 mov word [deviceparameters+92],0 ; PCDOS 7.1 IBMBIO.COM
40479 ;
40480 ;clc ; everything is fine
40481 ; 01/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40482 ; 12/12/2022
40483 ; cf=0
40484 ;clc
40485 ;call setdeviceparameters
40486 ; 03/01/2023
40487 jmp short setdeviceparameters
40488
40489 %endif
40490 ; M047 -- Begin modifications (too numerous to mark specifically)
40491 ;
40492 ;-----
40493 ;
40494 ; procedure : setdeviceparameters
40495 ;
40496 ; setdeviceparameters sets up the recommended bpb in each bds in the
40497 ; system based on the form factor. it is assumed that the bpbs for the
40498 ; various form factors are present in the bpbtable. for hard files,
40499 ; the recommended bpb is the same as the bpb on the drive.
40500 ; no attempt is made to preserve registers since we are going to jump to
40501 ; sysinit straight after this routine.
40502 ;
40503 ; if we return carry, the DRIVPARM will be aborted, but presently
40504 ; we always return no carry
40505 ;
40506 ; note: there is a routine by the same name in msdioc1.asm
40507 ;
40508 ;-----
40509 ;
40510 ; 15/04/2019 - Retro DOS v4.0
40511 ;
40512 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
40513 ;
40514 ; 03/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
40515 ; (SYSINIT:3FC4h)
40516 ;
40517 ; 09/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
40518 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4236h)
40519

```

```

40520 setdeviceparameters:
40521 ; 03/01/2023
40522 ; ds = cs
40523
40524 00003B45 06      push     es
40525
40526 00003B46 0E      push     cs
40527 00003B47 07      pop      es
40528
40529 00003B48 31DB     xor      bx,bx
40530 00003B4A 8A1E[BD4D]   mov     b1,[deviceparameters+A_DEVICEPARAMETERS.DP_DEVICETYPE]
40531 00003B4E 80FB00     cmp     b1,DEV_5INCH ; 0
40532 00003B51 7506     jne     short got_80
40533
40534 00003B53 C706[C04D]2800   mov     word [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],40
40535 ; 48 tpi=40 cyl
40536
40537 00003B59 D1E3     shl     bx,1          ; get index into bpb table
40538 00003B5B 8BB7[2C50]   mov     si,[bpbtable+bx] ; get address of bpb
40539
40540 ;mov     di,deviceparameters+7
40541 ; 02/11/2022
40542 00003B5F BF[C34D]     mov     di,deviceparameters+A_DEVICEPARAMETERS.DP_BPB ; es:di -> bpb
40543 00003B62 B93B00     mov     cx,A_BPB.size ; 31
40544 ; 09/09/2023
40545 ;mov     cx,59 ; PCDOS 7.1 IBMBIO.COM A_BPB.size
40546 00003B65 FC     cld
40547 ;repe     movsb
40548 ; 02/11/2022
40549 00003B66 F3A4     rep     movsb
40550
40551 00003B68 07      pop     es
40552
40553 ; 12/12/2022
40554 00003B69 F606[1B4F]20   test    byte [switches],flagseclim ; 20h
40555 ;test     word [switches],flagseclim ; 20h
40556 00003B6E 7406     jz      short see_heads
40557
40558 00003B70 A1[184F]     mov     ax,[slim]
40559 ;mov     [deviceparameters+20],ax
40560 00003B73 A3[D04D]     mov     [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK],ax
40561
40562 see_heads:
40563 ; 12/12/2022
40564 00003B76 F606[1B4F]40   test    byte [switches],flagheads ; 40h
40565 ;test     word [switches],flagheads ; 40h
40566 00003B7B 7406     jz      short heads_not_altered
40567
40568 00003B7D A1[164F]     mov     ax,[hlim]
40569 ;mov     [deviceparameters+22],ax
40570 00003B80 A3[D24D]     mov     [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS],ax
40571
40572 heads_not_altered:
40573 ; set up correct media descriptor byte and sectors/cluster
40574 ; sectors/cluster is always 2 except for any one sided disk or 1.44M
40575
40576 ;mov     byte [deviceparameters+9],2
40577 ; 02/11/2022
40578 ;mov     byte [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER],2
40579 ; 03/01/2023
40580 mov     ax,2
40581 00003B83 B80200   mov     [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER],al ; 2
40582 00003B86 A2[C54D]
40583
40584 00003B89 B3F0     mov     b1,0F0h          ; get default mediabyte
40585
40586 ; preload the mediadescriptor from the bpb into bh for convenient access
40587
40588 ;mov     bh,[deviceparameters+17]
40589 ; 02/11/2022
40590 00003B8B 8A3E[CD4D]   mov     bh,[deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.MEDIADESCRIPTOR]
40591
40592 ; 03/01/2023
40593 ; ax = 2
40594 00003B8F 3906[D24D]   cmp     [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS],ax ; >2 heads?
40595 ;cmp     word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS],2 ; >2 heads?
40596 00003B93 773C     ja      short got_correct_mediad ; just use default if heads>2
40597
40598 00003B95 7524     jne     short only_one_head ; one head, do one head stuff
40599
40600 ; two head drives will use the mediadescriptor from the bpb
40601
40602 00003B97 88FB     mov     b1,bh          ; get mediadescriptor from bpb
40603
40604 ; two sided drives have two special cases to look for. One is
40605 ; a 320K diskette (40 tracks, 8 secs per track). It uses
40606 ; a mediaid of 0fch. The other is 1.44M, which uses only
40607 ; one sector/cluster.
40608
40609 ; any drive with 18secs/trk, 2 heads, 80 tracks, will be assumed
40610 ; to be a 1.44M and use only 1 sector per cluster. Any other
40611 ; type of 2 headed drive is all set.
40612
40613 00003B99 833E[D04D]12   cmp     word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK],18
40614 00003B9E 7509     jne     short not_144m
40615 00003BA0 833E[C04D]50   cmp     word [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],80
40616 00003BA5 7502     jne     short not_144m
40617
40618 ; we've got cyl=80, heads=2, secpertrack=18. Set cluster size to 1.
40619
40620 00003BA7 EB24     jmp     short got_one_secperclus_drive
40621
40622 ; check for 320K
40623
40624 not_144m:
40625 00003BA9 833E[C04D]28   cmp     word [deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS],40
40626 00003BAE 7521     jne     short got_correct_mediad
40627 00003BB0 833E[D04D]08   cmp     word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK],8
40628 00003BB5 751A     jne     short got_correct_mediad
40629
40630 00003BB7 B3FC     mov     b1,0FCh
40631 00003BB9 EB16     jmp     short got_correct_mediad
40632
40633 only_one_head:
40634
40635 ; if we don't have a 360K drive, then just go use 0f0h as media descr.
40636
40637 00003BBB 803E[BD4D]00   cmp     byte [deviceparameters+A_DEVICEPARAMETERS.DP_DEVICETYPE],DEV_5INCH ; 0
40638 00003BC0 740B     je      short got_one_secperclus_drive
40639
40640 ; single sided 360K drive uses either 0fch or 0feh, depending on
40641 ; whether sectorspertrack is 8 or 9. For our purposes, anything
40642 ; besides 8 will be considered 0fch
40643

```

```

40644 00003BC2 B3FC          mov     b1,0FCh          ; single sided 9 sector media id
40645 00003BC4 833E[D04D]08  cmp     word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK],8
40646          ; 12/12/2022
40647 00003BC9 7502          jne     short got_one_secperclus_drive ; okay if anything besides 8
40648
40649 00003BCB B3FE          mov     b1,0FEh          ; 160K mediaid
40650
40651          ; we've either got a one sided drive, or a 1.44M drive
40652          ; either case we'll use 1 sector per cluster instead of 2
40653
40654 got_one_secperclus_drive:
40655          ; 03/01/2023
40656          ; ax = 2
40657 00003BCD 48          dec     ax ; ax = 1
40658 00003BCE A2[C54D]      mov     [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER],al ; 1
40659          ;mov     byte [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER],1
40660
40661 got_correct_mediad:
40662 00003BD1 881E[CD4D]      mov     [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.MEDIADESCRIPTOR],b1
40663
40664          ; Calculate the correct number of Total Sectors on medium
40665
40666 00003BD5 A1[C04D]      mov     ax,[deviceparameters+A_DEVICEPARAMETERS.DP_CYLINDERS]
40667 00003BD8 F726[D24D]      mul     word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS]
40668 00003BDC F726[D04D]      mul     word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK]
40669 00003BE0 A3[CB4D]      mov     word [deviceparameters+A_DEVICEPARAMETERS.DP_BPB+A_BPB.TOTALSECTORS],ax
40670 00003BE3 F8          clc          ; we currently return no errors
40671
40672 00003BE4 C3          retn
40673
40674          ; M047 -- end rewritten routine
40675
40676          ;-----
40677          ;
40678          ; procedure : organize
40679          ;
40680          ;-----
40681
40682          ; 09/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
40683 %if 1
40684 end_commd_line:
40685 00003BE5 AA          stosb          ; store line feed char in buffer for the linecount.
40686          ;mov     byte [cs:com_level],0 ; reset the command level.
40687          ; 03/01/2023
40688          ; ds = cs
40689          ;mov     byte [com_level],0
40690          ;jmp     short org1
40691          ; 09/09/2023
40692 00003BE6 EB0E          jmp     short org0
40693
40694 00003BE8 F9          nochar1:
40695 00003BE9 C3          stc
40696          retn
40697 %endif
40698          ; 02/11/2022 - Retrodos v4.0 (Modified MSDOS 5.0 IO.SYS)
40699          ; (SYSINIT:3234h)
40700
40701          ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
40702          ; (SYSINIT:4067h)
40703
40704          ; 09/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
40705          ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:42D9h)
40706
40707 organize:
40708          ; 03/01/2023
40709          ; ds = cs
40710 00003BEA 8B0E[5603]    mov     cx,[count]
40711          ;mov     cx,[cs:count]
40712          jcxz     nochar1
40713
40714          ;ifndef     MULTI_CONFIG
40715          ;
40716          ; In MULTI_CONFIG, we map to upper case on a line-by-line basis,
40717          ; because we the case of values in SET commands preserved
40718          ;
40719          ; call     mapcase
40720          ;endif
40721          ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40722          ; 03/01/2023 - Retro DOS v4.2
40723          ;call     mapcase
40724 00003BF0 31F6          xor     si,si
40725 00003BF2 89F7          mov     di,si
40726 00003BF4 31C0          xor     ax,ax
40727          ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40728          ;mov     byte [cs:com_level],0
40729          ; 12/12/2022
40730          ;mov     [cs:com_level],al ; 0
40731          ; 03/01/2023
40732          ; ds = cs
40733          ; 09/09/2023
40734          ;mov     [com_level],al ; 0
40735
40736 00003BF6 C606[5003]00  org0:
40737          mov     byte [com_level],0 ; 09/09/2023
40738          org1:
40739          call     skip_comment
40740          jz       short end_commd_line ; found a comment string and skipped.
40741          call     get2 ; not a comment string. then get a char.
40742          cmp     al,1f ; 0Ah
40743          je       short end_commd_line ; starts with a blank line.
40744          cmp     al,', ' ; 20h
40745          jbe     short org1 ; skip leading control characters
40746          ; 09/09/2023
40747          ;jmp     short findit
40748
40749          ; 09/09/2023
40750 %if 0
40751 end_commd_line:
40752          stosb          ; store line feed char in buffer for the linecount.
40753          ;mov     byte [cs:com_level],0 ; reset the command level.
40754          ; 03/01/2023
40755          ; ds = cs
40756          ;mov     byte [com_level],0
40757          ;jmp     short org1
40758
40759          nochar1:
40760          stc
40761          retn
40762 %endif
40763
40764          findit:
40765          push     cx
40766          push     si
40767          push     di
40768          mov     bp,si

```

```

40768 00003C10 4D          dec     bp
40769 00003C11 BE[D04C]    mov     si,comtab      ; prepare to search command table
40770 00003C14 B500      mov     ch,0
40771 findcom:
40772 00003C16 89EF      mov     di,bp
40773 00003C18 8A0C      mov     cl,[si]
40774 00003C1A 46          inc     si
40775 00003C1B E345      jcxz    nocom
40776
40777          ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40778
40779          ; 03/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
40780
40781 ;ifdef     MULTI_CONFIG
40782
40783 ;   Simplify future parsing by collapsing ";" onto "REM", and at the same
40784 ;   time skip the upcoming delimiter test (since ";" need not be followed by
40785 ;   anything in particular)
40786
40787 00003C1D 26803D3B  cmp     byte [es:di],CONFIG_SEMICOLON ; ';'
40788 00003C21 7430      je      short semicolon
40789 loopcom:
40790          ;mov     al,[es:di]
40791          ;inc     di
40792          ;and     al,~20h ; 0DFh          ; force upper case
40793          ;inc     si                      ; compare to byte @es:di
40794          ;cmp     al,[si-1]
40795          ; 28/07/2023 - Retro DOS v4.2 IO.SYS (optimization)
40796 00003C23 268A25  mov     ah,[es:di]
40797 00003C26 47          inc     di
40798 00003C27 80E4DF  and     ah,~20h ; 0DFh
40799 00003C2A AC          lodsb
40800          ; mov al,[si]
40801          ; inc si
40802          ;cmp     al,ah
40803          ;loope  loopcom
40804 00003C2B 30C4      xor     ah,al          ; result: ah = 0 (*) if ah = al
40805 00003C2D E1F4      loopz   loopcom
40806 ;else
40807 ;   repe     cmpsb
40808 ;endif
40809          ; 02/11/2022
40810          ; 03/01/2023 - Retro DOS v4.2
40811          ;repe     cmpsb
40812
40813          ; 28/07/2023
40814          ;lahf
40815 00003C2F 01CE      add     si,cx          ; bump to next position without affecting flags
40816          ;sahf
40817 00003C31 AC          lodsb
40818          ;jnz     short findcom
40819          ; 28/07/2023
40820 00003C32 08E4      or      ah,ah          ; (*)
40821 00003C34 75E0      jnz     short findcom
40822
40823 00003C36 26803D0D  cmp     byte [es:di],cr          ; the next char might be cr,lf
40824 00003C3A 7421      je      short gotcom0          ; such as in "rem",cr,lf case.
40825 00003C3C 26803D0A  cmp     byte [es:di],lf
40826 00003C40 741B      je      short gotcom0
40827
40828          ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40829
40830          ; 03/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
40831
40832 ;ifdef     MULTI_CONFIG
40833
40834 ; Skip the delimiter test for the BEGIN identifier (it doesn't have one).
40835
40836 00003C42 3C5B      cmp     al,CONFIG_BEGIN ; '['
40837 00003C44 7417      je      short gotcom0
40838 ;endif
40839 00003C46 50          push    ax
40840 00003C47 268A05  mov     al,[es:di]      ; now the next char. should be a delim.
40841
40842 ;ifdef     MULTI_CONFIG
40843
40844 ;   If keyword is *immediately* followed by a question mark (?), then
40845 ;   set the high bit of the ASCII command code (CONFIG_OPTION_QUERY) that is
40846 ;   stored in the CONFIG.SYS memory image.
40847
40848 00003C4A 3C3F      cmp     al, '?'          ; explicit interactive command?
40849 00003C4C 7509      jne     short no_query    ; no
40850 00003C4E 58          pop     ax                ; yes, so retrieve the original code
40851          ;or      al,80h ; 03/01/2023
40852 00003C4F 0C80      or      al,CONFIG_OPTION_QUERY ; and set the QUERY bit
40853 00003C51 E80A      jmp     short gotcom0
40854 semicolon:
40855 00003C53 B030      mov     al,CONFIG_REM ; '0'
40856 00003C55 E806      jmp     short gotcom0
40857 no_query:
40858 ;endif ;MULTI_CONFIG
40859
40860          ; 02/11/2022
40861          ; 03/01/2023 - Retro DOS v4.2
40862          ;push    ax
40863          ;mov     al,[es:di]      ; now the next char. should be a delim.
40864
40865 00003C57 E82E0B  call    delim
40866 no_delim:
40867 00003C5A 58          pop     ax
40868 00003C5B 75B9      jnz     short findcom
40869 gotcom0:
40870          pop     di
40871 00003C5E 5E          pop     si
40872 00003C5F 59          pop     cx
40873 00003C60 E810      jmp     short gotcom
40874 nocom:
40875          pop     di
40876 00003C63 5E          pop     si
40877 00003C64 59          pop     cx
40878 00003C65 B05A      mov     al,CONFIG_UNKNOWN ; 'Z'
40879 00003C67 AA          stosb          ; save indicator char.
40880 _skipline:
40881 00003C68 E86801  call    get2
40882 00003C6B 3C0A      cmp     al,lf ; 0Ah          ; skip this bad command line
40883 00003C6D 75F9      jne     short _skipline
40884          ;jmp     short end_commd_line ; handle next command line
40885          ; 09/09/2023
40886 00003C6F E973FF  jmp     end_commd_line
40887 gotcom:
40888          stosb          ; save indicator char in buffer
40889
40890          ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40891

```

```

40892 ; 03/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
40893
40894 ;ifdef MULTI_CONFIG
40895
40896 ; Don't pollute "cmd_indicator" with the CONFIG_OPTION_QUERY bit though;
40897 ; it screws up the direct comparisons below.
40898
40899 00003C73 247F and al,~CONFIG_OPTION_QUERY ; 7Fh
40900 ;endif
40901 ;mov [cs:cmd_indicator],al ; save it for the future use.
40902 ; 03/01/2023
40903 ; ds = cs
40904 00003C75 A2[5403] mov [cmd_indicator],al ; save it for the future use.
40905
40906 ;ifdef MULTI_CONFIG
40907
40908 ; There is no whitespace/delimiter between the "begin block" character
40909 ; ([) and the name of block (eg, [menu]), therefore skip this delimiter
40910 ; skipping code
40911
40912 00003C78 3C5B cmp al,CONFIG_BEGIN
40913 00003C7A 7455 je short org31
40914 00003C7C 3C4F cmp al,CONFIG_SUBMENU ; 'O'
40915 00003C7E 740F je short no_mapcase
40916 00003C80 3C45 cmp al,CONFIG_MENUITEM ; 'E'
40917 00003C82 740B je short no_mapcase
40918 00003C84 3C41 cmp al,CONFIG_MENUEFAULT ; 'A'
40919 00003C86 7407 je short no_mapcase
40920 00003C88 3C4A cmp al,CONFIG_INCLUDE ; 'J'
40921 00003C8A 7403 je short no_mapcase
40922 00003C8C E8350B call mapcase ; map case of rest of line to UPPER
40923 no_mapcase:
40924 ;endif
40925 ; 02/11/2022
40926 ;mov [cs:cmd_indicator],al ; save it for the future use.
40927 ; 03/01/2023
40928 ; ds = cs
40929 ;mov [cmd_indicator],al
40930 org2:
40931 00003C8F E84101 call get2 ; skip the command name until delimiter
40932 00003C92 3C0A cmp al,lf ; 0Ah
40933 00003C94 740F je short org21
40934 00003C96 3C0D cmp al,cr ; 0Dh
40935 00003C98 740B je short org21
40936 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40937 ; 03/01/2023 - Retro DOS v4.2
40938 00003C9A 3C2F cmp al,'/' ; T-RICHJ: Added to allow DEVHIGH/L:...
40939 00003C9C 7407 je short org21 ; T-RICHJ: to be parsed properly.
40940
40941 00003C9E E8E70A call delim
40942 00003CA1 75EC jnz short org2
40943 00003CA3 EB02 jmp short org3
40944
40945 00003CA5 4E org21: ;if cr or lf then
40946 00003CA6 41 dec si ; undo si, cx register
40947 inc cx ; and continue
40948 org3:
40949 ;cmp byte [cs:cmd_indicator],CONFIG_COMMENT ; 'Y'
40950 ;je short get_cmt_token
40951 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40952 ; 03/01/2023 - Retro DOS v4.2
40953 ;cmp byte [cs:cmd_indicator],CONFIG_DEVICE ; 'D'
40954 ;je short org_file
40955 ;cmp byte [cs:cmd_indicator],CONFIG_INSTALL ; 'I'
40956 ;je short org_file
40957 ;cmp byte [cs:cmd_indicator],CONFIG_INSTALLHIGH ; 'W'
40958 ;je short org_file
40959 ; 02/11/2022
40960 ; 03/01/2023 - Retro DOS v4.2
40961 ;cmp byte [cs:cmd_indicator],CONFIG_DEVICE ; 'D'
40962 ;je short org_file
40963 ;cmp byte [cs:cmd_indicator],CONFIG_SHELL ; 'S'
40964 ;je short org_file
40965 ;cmp byte [cs:cmd_indicator],CONFIG_SWITCHES ; '1'
40966 ;je short org_switch
40967
40968 ; 03/01/2023
40969 ; ds = cs
40970 00003CA7 803E[5403]59 cmp byte [cmd_indicator],CONFIG_COMMENT ; 'Y'
40971 00003CAC 745D je short get_cmt_token
40972 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
40973 ; 03/01/2023 - Retro DOS v4.2
40974 00003CAE 803E[5403]44 cmp byte [cmd_indicator],CONFIG_DEVICE ; 'D'
40975 00003CB3 7430 je short org_file
40976 00003CB5 803E[5403]49 cmp byte [cmd_indicator],CONFIG_INSTALL ; 'I'
40977 00003CBA 7429 je short org_file
40978 00003CBC 803E[5403]57 cmp byte [cmd_indicator],CONFIG_INSTALLHIGH ; 'W'
40979 00003CC1 7422 je short org_file
40980 ; 02/11/2022
40981 ; 03/01/2023 - Retro DOS v4.2
40982 ;cmp byte [cmd_indicator],CONFIG_DEVICE ; 'D'
40983 ;je short org_file
40984 00003CC3 803E[5403]53 cmp byte [cmd_indicator],CONFIG_SHELL ; 'S'
40985 00003CC8 741B je short org_file
40986 00003CCA 803E[5403]31 cmp byte [cmd_indicator],CONFIG_SWITCHES ; '1'
40987 00003CCF 7403 je short org_switch
40988
40989 org31:
40990 00003CD1 E99500 jmp org4
40991
40992 org_switch:
40993 00003CD4 E81601 call skip_comment
40994 00003CD7 7472 jz short end_commd_line_brdg
40995
40996 00003CD9 E8F700 call get2
40997 00003CDC E8B10A call org_delim
40998 00003CDF 74F3 jz short org_switch
40999
41000 00003CE1 AA stosb
41001 00003CE2 E99300 jmp org5
41002
41003 org_file: ; get the filename and put 0 at end
41004 00003CE5 E80501 call skip_comment
41005 00003CE8 7464 jz short org_put_zero
41006
41007 00003CEA E8E600 call get2 ; not a comment
41008 00003CED E8980A call delim
41009 00003CF0 74F3 jz short org_file ; skip the possible delimiters
41010
41011 00003CF2 AA stosb ; copy the first non delim char found in buffer
41012
41013 org_copy_file:
41014 00003CF3 E8F700 call skip_comment ; comment char in the filename?
41015 00003CF6 7456 jz short org_put_zero ; then stop copying filename at that point

```

```

41016
41017 00003CF8 E8D800      call    get2
41018 00003CFB 3C2F      cmp     al,'/' ; a switch char? (device=filename/xxx)
41019 00003CFD 7457      je      short end_file_slash ; this will be the special case.
41020
41021 00003CFF AA          stosb    ; save the char. in buffer
41022 00003D00 E8850A      call    delim
41023 00003D03 7459      jz      short end_copy_file
41024
41025 00003D05 3C20      cmp     al, ' '
41026 00003D07 77EA      ja      short org_copy_file ; keep copying
41027 00003D09 EB53      jmp     short end_copy_file ; otherwise, assume end of the filename.
41028
41029 get_cmt_token:          ; get the token. just max. 2 char.
41030 00003D0B E8C500      call    get2
41031 00003D0E 3C20      cmp     al, ' ' ; skip white spaces or "=" char.
41032 00003D10 74F9      je      short get_cmt_token ; (we are allowing the other special
41033 00003D12 3C09      cmp     al,tab ; 9 ; characters can used for comment id.
41034 00003D14 74F5      je      short get_cmt_token ; character.)
41035 00003D16 3C3D      cmp     al,'=' ; = is special in this case.
41036 00003D18 74F1      je      short get_cmt_token
41037 00003D1A 3C0D      cmp     al,cr
41038 00003D1C 7426      je      short get_cmt_end ; cannot accept the carriage return
41039 00003D1E 3C0A      cmp     al,lf
41040 00003D20 7422      je      short get_cmt_end
41041
41042 ; 03/01/2023
41043 ; ds = cs
41044 ;mov     [cs:cmmt1],al ; store it
41045 ;mov     byte [cs:cmmt],1 ; 1 char. so far.
41046 00003D22 A2[5203]   mov     [cmmt1],al ; store it
41047 00003D25 C606[5103]01 mov     byte [cmmt],1 ; 1 char. so far.
41048 00003D2A E8A600      call    get2
41049 00003D2D 3C20      cmp     al, ' ' ; 20h
41050 00003D2F 7413      je      short get_cmt_end
41051 00003D31 3C09      cmp     al,tab ; 9
41052 00003D33 740F      je      short get_cmt_end
41053 00003D35 3C0D      cmp     al,cr ; 0Dh
41054 00003D37 740B      je      short get_cmt_end
41055 00003D39 3C0A      cmp     al,lf ; 0Ah
41056 00003D3B 740E      je      short end_commd_line_brdg
41057
41058 ;mov     [cs:cmmt2],al
41059 ;inc     byte [cs:cmmt]
41060 ; 03/01/2023
41061 00003D3D A2[5303]   mov     [cmmt2],al
41062 00003D40 FE06[5103] inc     byte [cmmt]
41063
41064 get_cmt_end:
41065 00003D44 E88C00      call    get2
41066 00003D47 3C0A      cmp     al,lf
41067 00003D49 75F9      jne     short get_cmt_end ; skip it.
41068
41069 end_commd_line_brdg:
41070 jmp     end_commd_line ; else jmp to end_commd_line
41071
41072 org_put_zero:          ; make the filename in front of
41073 00003D4E 26C60500   mov     byte [es:di],0 ; the comment string to be an asciiz.
41074 00003D52 47          inc     di
41075 00003D53 E98FFE      jmp     end_commd_line ; (maybe null if device=/*)
41076
41077 end_file_slash:        ; al = "/" option char.
41078 00003D56 26C60500   mov     byte [es:di],0 ; make a filename an asciiz
41079 00003D5A 47          inc     di ; and
41080 00003D5B AA          stosb    ; store "/" after that.
41081 00003D5C EB1A      jmp     short org5 ; continue with the rest of the line
41082
41083 end_copy_file:
41084 00003D5E 26C645FF00 mov     byte [es:di-1],0 ; make it an asciiz and handle the next char.
41085 00003D63 3C0A      cmp     al,lf
41086 00003D65 74E4      je      short end_commd_line_brdg
41087 00003D67 EB0F      jmp     short org5
41088
41089 org4:                  ; org4 skips all delimiters after the command name except for '/'
41090 00003D69 E88100      call    skip_comment
41091 00003D6C 74DD      jz      short end_commd_line_brdg
41092
41093 call    get2
41094 call    org_delim ; skip delimiters except '/' (mrw 4/88)
41095 jz      short org4
41096 jmp     short org51
41097
41098 org5:                  ; rest of the line
41099 00003D78 E87200      call    skip_comment ; comment?
41100 00003D7B 74CE      jz      short end_commd_line_brdg
41101 00003D7D E85300      call    get2 ; not a comment.
41102
41103 org51:
41104 stosb    ; copy the character
41105 cmp     al,'"'; 22h ; a quote ?
41106 je      short at_quote
41107 cmp     al,' ' ; 20h
41108 ja      short org5
41109
41110 ; 09/09/2023
41111 ; (Note: PCDOS 7.1 IBMBIO.COM does not contain M051 modification)
41112
41113 ; 03/01/2023 ; M051 - Start
41114 ; ds = cs
41115 00003D89 803E[5403]55 cmp     byte [cmd_indicator],CONFIG_DEVICEHIGH
41116 ;cmp     byte [cs:cmd_indicator],CONFIG_DEVICEHIGH ; Q: is this devicehigh
41117 jne     short not_dh ; N:
41118 00003D90 3C0A      cmp     al,lf ; Q: is this line feed
41119 00003D92 7416      je      short org_dhlf ; Y: stuff a blank before the lf
41120 00003D94 3C0D      cmp     al,cr ; Q: is this a cr
41121 00003D96 75E0      jne     short org5 ; N:
41122 00003D98 26C645FF20 mov     byte [es:di-1],' ' ; overwrite cr with blank
41123 00003D9D AA          stosb    ; put cr after blank
41124 00003D9E FE06[203A] inc     byte [insert_blank]
41125 ;inc     byte [cs:insert_blank]; indicate that blank has been
41126 ;inserted
41127 00003DA2 EBD4      jmp     short org5
41128 not_dh:                  ; M051 - End
41129
41130 cmp     al,lf ; line feed?
41131 00003DA6 740F      je      short org1_brdg ; handles the next command line.
41132 00003DA8 EBCE      jmp     short org5 ; handles next char in this line.
41133
41134 org_dhlf:              ; M051 - Start
41135 ; 03/01/2023
41136 ; ds = cs
41137 00003DAA 803E[203A]01 cmp     byte [insert_blank],1
41138 ;cmp     byte [cs:insert_blank],1 ; Q:has a blank already been inserted
41139 00003DAF 7406      je      short org1_brdg ; Y:

```



```

41140 00003DB1 26C645FF20      mov     byte [es:di-1], ' ' ; overwrite lf with blank
41141 00003DB6 AA                stosb    ; put lf after blank
41142                                ; M051 - End
41143
41144 00003DB7 C606[203A]00      org1_brdg:
41145                                mov     byte [insert_blank],0
41146                                ;mov    byte [cs:insert_blank],0 ; M051: clear blank indicator for
41147 00003DBC E93CFE            jmp     org1 ; M051: devicehigh
41148
41149
41150 00003DBF 803E[5003]00      at_quote:
41151                                cmp     byte [com_level],0
41152 00003DC4 7407                ;cmp    byte [cs:com_level],0
41153                                je      short up_level
41154 00003DC6 C606[5003]00      ;mov    byte [cs:com_level],0 ; reset it.
41155 00003DCB EBAB                mov     byte [com_level],0
41156                                jmp     short org5
41157
41158                                up_level:
41159 00003DCD FE06[5003]        ;inc    byte [cs:com_level] ; set it.
41160 00003DD1 EBA5                inc     byte [com_level]
41161                                jmp     short org5
41162
41163                                ;-----
41164                                ; procedure : get2
41165                                ;-----
41166
41167                                ; 02/11/2022 - Retrodos v4.0 (Modified MSDOS 5.0 IO.SYS)
41168                                ; (SYSINIT:33FAh)
41169
41170                                ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
41171                                ; (SYSINIT:4270h)
41172                                get2:
41173                                jcxz    noget
41174 00003DD3 E304                ;
41175                                ;
41176                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41177                                ;;lods byte ptr es:[si]
41178                                ; 12/12/2022
41179 00003DD5 26                es
41180 00003DD6 AC                lodsb
41181                                ;mov    al, [es:si]
41182                                ;inc    si
41183                                ;
41184 00003DD7 49                dec     cx
41185 00003DD8 C3                retn
41186                                noget:
41187 00003DD9 59                pop     cx
41188                                ; 03/01/2023
41189                                ; ds = cs
41190                                ;mov    [cs:count],di ; 13/05/2019
41191                                ;mov    [cs:org_count],di
41192 00003DDA 893E[5603]        mov     [count],di
41193 00003DDE 893E[5803]        mov     [org_count],di
41194 00003DE2 31F6                xor     si,si
41195                                ;mov    [cs:chrptr],si
41196 00003DE4 8936[5A03]        mov     [chrptr],si
41197                                ;
41198                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41199
41200                                ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
41201
41202                                ;ifndef MULTI_CONFIG
41203                                ; retn
41204                                ;else
41205
41206                                ; This was the rather kludgy way out of procedure "organize", but instead
41207                                ; of returning to doconf, we now want to check config.sys BEGIN/END blocks
41208                                ; and the new boot menu stuff...
41209
41210 00003DE8 89F9                mov     cx,di
41211 00003DEA E9E300            jmp     menu_check
41212
41213                                ;endif
41214                                ; 02/11/2022
41215                                ; 03/01/2023 - Retro DOS v4.2
41216                                ;retn
41217
41218                                ;-----
41219                                ;
41220                                ; procedure : skip_comment
41221                                ;
41222                                ;skip the commented string until lf, if current es:si-> a comment string.
41223                                ;in) es:si-> string
41224                                ; cx -> length.
41225                                ;out) zero flag not set if not found a comment string.
41226                                ; zero flag set if found a comment string and skipped it. al will contain
41227                                ; the line feed character at this moment when return.
41228                                ; ax register destroyed.
41229                                ; if found, si, cx register adjusted accordingly.
41230                                ;
41231                                ;-----
41232
41233                                ; 03/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
41234                                ; (SYSINIT:428Dh)
41235
41236                                skip_comment:
41237 00003DED E3EA                jcxz    noget ; get out of the organize routine.
41238                                ;
41239                                ; 03/01/2023
41240                                ; ds = cs
41241
41242 00003DEF 803E[5003]00      cmp     byte [com_level],0
41243                                ;cmp    byte [cs:com_level],0 ; only check it if parameter level is 0.
41244 00003DF4 752C                jne     short no_commt ; (not inside quotations)
41245
41246 00003DF6 803E[5103]01      cmp     byte [cmmt],1
41247                                ;cmp    byte [cs:cmmt],1
41248 00003DFB 7225                jb      short no_commt
41249
41250 00003DFD 268A04            mov     al,[es:si]
41251
41252 00003E00 3806[5203]        cmp     [cmmt1],al
41253                                ;cmp    [cs:cmmt1],al
41254 00003E04 751C                jne     short no_commt
41255
41256 00003E06 803E[5103]02      cmp     byte [cmmt],2
41257                                ;cmp    byte [cs:cmmt],2
41258 00003E0B 750A                jne     short skip_cmmt
41259
41260 00003E0D 268A4401        mov     al,[es:si+1]
41261
41262 00003E11 3806[5303]        cmp     [cmmt2],al
41263                                ;cmp    [cs:cmmt2],al

```

```

41264 00003E15 750B      jne      short no_commt
41265 skip_commt:
41266 00003E17 E3C0      jcxz     noget          ; get out of organize routine.
41267 00003E19 268A04    mov     al,[es:si]
41268 00003E1C 46          inc     si
41269 00003E1D 49          dec     cx
41270 00003E1E 3C0A      cmp     al,1f          ; line feed?
41271 00003E20 75F5      jne     short skip_commt
41272 no_commt:
41273 00003E22 C3          retn
41274
41275 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
41276 ; (SYSINIT:42C8h)
41277
41278 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
41279 ;%if 0
41280
41281 ;ifdef      MULTI_CONFIG
41282
41283 ;-----
41284 ;
41285 ;   kbd_read: wait for keystroke
41286 ;
41287 ;   INPUT
41288 ;       DS == CS == sysinitseg
41289 ;
41290 ;   OUTPUT
41291 ;       Carry SET to clean boot, CLEAR otherwise
41292 ;
41293 ;   OTHER REGS USED
41294 ;       All
41295 ;
41296 ;   HISTORY
41297 ;       Created 16-Nov-1992 by JeffPar
41298 ;-----
41299 ;
41300
41301 kbd_read:
41302 00003E23 F606[7F4C]02    test    byte [bDisableUI],2
41303 00003E28 7520      jnz     short kbd_nodelay
41304
41305 00003E2A 1E          push    ds              ; the bios timer tick count is incremented
41306 00003E2B 29C0      sub     ax,ax           ; 18.2 times per second;
41307 00003E2D 8ED8      mov     ds,ax           ; watch the timer tick count for 37 transitions
41308 ;mov     dx,[046Ch]      ; get initial value
41309 kbd_loop:
41310 00003E2F B401      mov     ah,1            ;
41311 00003E31 CD16      int     16h            ; peek the keyboard
41312 00003E33 7514      jnz     short kbd_loopdone ; something's there, get out
41313 00003E35 B402      mov     ah,2            ; peek the shift states
41314 00003E37 CD16      int     16h            ;
41315 00003E39 A803      test    al,03h         ; either right or left shift key bits set?
41316 00003E3B 750C      jnz     short kbd_loopdone ; yes
41317 00003E3D A16C04    mov     ax,[046Ch]
41318 ;sub     ax,dx           ; get difference
41319 ; 15/04/2019 - Retro DOS v4.0
41320 00003E40 2E2B06[8C03]    sub     ax,[cs:_timer_lw_] ; MSDOS 6.21 IO.SYS - SYSINIT:42E5h
41321
41322 00003E45 3C25      cmp     al,37           ; reached limit? ; (2 seconds)
41323 00003E47 72E6      jb     short kbd_loop ; not yet
41324 kbd_loopdone:
41325 00003E49 1F          pop     ds              ; delay complete!
41326 kbd_nodelay:
41327 00003E4A 29DB      sub     bx,bx           ; assume clean boot
41328 00003E4C B402      mov     ah,2            ; peek the shift states
41329 00003E4E CD16      int     16h            ;
41330 00003E50 A803      test    al,03h         ; either right or left shift key bits set?
41331 00003E52 7407      jz      short kbd_notshift ; no
41332 00003E54 43          inc     bx              ; yes
41333 00003E55 43          inc     bx
41334 ; MSDOS 6.21 IO.SYS - SYSINIT:4301h
41335 00003E56 800E[834C]04    or      byte [bQueryOpt],4
41336 kbd_notshift:
41337 00003E5B B401      mov     ah,1            ; peek the keyboard
41338 00003E5D CD16      int     16h            ;
41339 00003E5F 743E      jz      short kbd_test ; no key present
41340 00003E61 08C0      or      al,al           ; is it a function key?
41341 00003E63 753A      jnz     short kbd_test ; no
41342
41343 ; MSDOS 6.21 IO.SYS - SYSINIT:430Bh
41344 00003E65 80FC62    cmp     ah,62h          ; CTRL F5
41345 00003E68 7405      je     short kbd_cfg_bypass
41346
41347 00003E6A 80FC3F    cmp     ah,3Fh          ; F5 function key?
41348 00003E6D 750D      jne     short kbd_notf5 ; no
41349 kbd_cfg_bypass:
41350 00003E6F BA[FA51]    mov     dx,$CleanMsg
41351 00003E72 E8DD0B    call    print
41352 ; MSDOS 6.21 IO.SYS - SYSINIT:431Bh
41353 00003E75 800E[834C]04    or      byte [bQueryOpt],4
41354 00003E7A EB16      jmp     short kbd_eat ; yes, clean boot selected
41355 kbd_notf5:
41356 ; MSDOS 6.21 IO.SYS - SYSINIT:4322h
41357 00003E7C 80FC65    cmp     ah,65h          ; CTRL F8
41358 00003E7F 7405      je     short kbd_cfg_confirm
41359
41360 00003E81 80FC42    cmp     ah,42h          ; F8 function key?
41361 00003E84 7523      jne     short kbd_exit ; no
41362 kbd_cfg_confirm:
41363 00003E86 BA[3852]    mov     dx,$InterMsg
41364 00003E89 E8C60B    call    print
41365 00003E8C B301      mov     bl,1            ; yes, interactive-boot option enabled
41366 00003E8E 881E[834C]    mov     [bQueryOpt],bl ; change default setting
41367 kbd_eat:
41368 00003E92 B400      mov     ah,0            ;
41369 00003E94 CD16      int     16h            ; eat the key we assumed was a signal
41370 00003E96 C606[894C]FF    mov     byte [secElapsed],-1
41371 00003E9B 09DB      or      bx,bx           ;
41372 00003E9D 7405      jz      short kbd_clean ;
41373 kbd_test:
41374 00003E9F 80FB02    cmp     bl,2            ;
41375 00003EA2 7205      jb     short kbd_exit ;
41376 kbd_clean:
41377 00003EA4 E86E08    call    disable_autoexec ; yes, tell COMMAND to skip autoexec.bat
41378 00003EA7 F9          stc                    ; set carry to indicate abort
41379 00003EA8 C3          retn
41380 kbd_exit:
41381 00003EA9 F8          clc                    ; clear carry to indicate success
41382 00003EAA C3          retn
41383
41384 ;-----
41385 ;
41386 ;   set_numlock: set numlock LED
41387 ;

```



```

41388 ; INPUT
41389 ; ES:SI -> numlock setting (ie, "ON" or "OFF")
41390 ;
41391 ; OUTPUT
41392 ; None
41393 ;
41394 ; OTHER REGS USED
41395 ; None
41396 ;
41397 ; HISTORY
41398 ; Created 16-Nov-1992 by JeffPar
41399 ;
41400 ;-----
41401 ;
41402 ; 04/01/2023 - Retro DOS v4.2
41403 ;
41404 set_numlock:
41405 ; 04/01/2023
41406 ;push ax
41407 push ds
41408 sub ax,ax
41409 mov ds,ax
41410 mov ax,[es:si] ; get 1st 2 bytes of value (ON or OF)
41411 cmp ax,[cs:OnOff+2] ; should we turn it off?
41412 jne short not_off ; no
41413 and byte [0417h],~20h ; 0DFh
41414 jmp short set_done
41415 not_off:
41416 cmp ax,[cs:OnOff] ; should we turn it on?
41417 stc
41418 jne short set_done ; no
41419 or byte [0417h],20h
41420 set_done:
41421 pop ds
41422 ; 04/01/2023
41423 ;pop ax
41424 retn
41425 ;
41426 ; 16/04/2019 - Retro DOS v4.0
41427 ;
41428 ;-----
41429 ;
41430 menu_check: check for presence of menu (and other) configuration blocks
41431 ;
41432 ; INPUT
41433 ; CX == "organized" config.sys memory image length
41434 ; ES:SI -> "organized" config.sys memory image
41435 ; DS == CS == sysinitseg
41436 ;
41437 ; OUTPUT
41438 ; Same as above; the idea is that menu_check simply transforms
41439 ; a block-structured config.sys image into a conventional image,
41440 ; based on the user's block selection and any other boot-time options
41441 ; the user may have employed...
41442 ;
41443 ; OTHER REGS USED
41444 ; All
41445 ;
41446 ; NOTES
41447 ; [count] and [org_count] are set to the new config.sys image length
41448 ;
41449 ; HISTORY
41450 ; Created 16-Mar-1992 by JeffPar
41451 ;
41452 ;-----
41453 ;
41454 ; 04/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
41455 ; (SYSINIT:4378h)
41456 ;
41457 menu_check:
41458 ;
41459 ; Search for SWITCHES, determine if /N or /F are present; if so, then
41460 ; disable clean/interactive boot options
41461 ;
41462 push cx ;
41463 push si ;
41464 sub bx,bx ; remains ZERO until first block
41465 swchk_loop:
41466 call get_char ; get first char of current line
41467 jc short swchk_end ; hit eof
41468 cmp al,CONFIG_BEGIN ; '['
41469 jne short swchk_next1 ;
41470 inc bx ; remember that we've seen a block
41471 jmp short swchk_nextline
41472 swchk_next1:
41473 cmp al,CONFIG_NUMLOCK
41474 jne short swchk_next2 ;
41475 or bx,bx ; only do NUMLOCK commands that exist
41476 jnz short swchk_nextline ; before the first block
41477 call set_numlock ; REM it out so we don't act on it later, too
41478 mov byte [es:si-1],CONFIG_REM
41479 jmp short swchk_nextline
41480 swchk_next2:
41481 cmp al,CONFIG_SWITCHES
41482 jne short swchk_nextline ; this line ain't it
41483 swchk_scan:
41484 call get_char ; look for /N or /F
41485 swchk_scan1:
41486 cmp al,LF ; end of line?
41487 je short swchk_nextline ; yes
41488 cmp al,'/' ; switch-char?
41489 jne short swchk_scan ; no
41490 call get_char ;
41491 and al,~20h ; 0DFh ; convert to upper case
41492 cmp al,[swit_n+1] ; 'N'
41493 jne short swchk_scan2 ; no
41494 or byte [bDisableUI],1
41495 jmp short swchk_scan ; continue looking for switches of interest
41496 swchk_scan2:
41497 cmp al,[swit_f+1] ; 'F'
41498 jne short swchk_scan1 ; no
41499 or byte [bDisableUI],2
41500 jmp short swchk_scan ; continue looking for switches of interest
41501 swchk_nextline:
41502 call skip_opt_line ;
41503 jmp short swchk_loop ;
41504 swchk_end:
41505 pop si ;
41506 pop cx ;
41507 ;
41508 ; Do the keyboard tests for clean/interactive boot now, but only if
41509 ; the DisableUI flag is still clear
41510 ;
41511 test byte [bDisableUI],1

```

```

41512 00003F2C 7508      jnz short menu_search
41513
41514      ;
41515      ; wait for 2 seconds first, UNLESS the /F bit was set in bDisableUI, or
41516      ; there is anything at all in the keyboard buffer
41517      ;
41517 00003F2E E8F2FE      call kbd_read
41518 00003F31 7303      jnc short menu_search
41519 00003F33 E9EE01      jmp menu_abort
41520
41521      ; Search for MENU block; it is allowed to be anywhere in config.sys
41522
41523 menu_search:
41524 00003F36 29DB      sub bx,bx      ; if no MENU, default to zero for no_selection
41525 00003F38 BFC44C      mov di,szMenu ;
41526 00003F3B E80304      call find_block ; find the MENU block
41527 00003F3E 7337      jnc short menu_found ;
41528 00003F40 C606BC4C00    mov byte [szBoot],0
41529 00003F45 E90C02      jmp no_selection ; not found
41530
41531      ; Process the requested menu color(s)
41532
41533 menu_color:
41534 00003F48 51      push cx
41535 00003F49 52      push dx
41536      ; mov dx,0007h ; default color setting
41537      ; 10/09/2023
41538      ; mov dl,7 ; !!
41539 00003F4A E89E06      call get_number ; get first number
41540 00003F4D 80E30F      and bl,0Fh ; !!! ; first # is foreground color (for low nibble)
41541 00003F50 88DD      mov ch,bl ; save it in CH
41542      ; 01/08/2023 - Retro DOS v4.2 IO.SYS (optimization) by Erdogan Tan
41543      ; (high nibble of dl is 0)
41544      ; and dl,0F0h ; !!! ; (low nibble of dl would be zero)
41545      ; or dl,bl ; (low nibble of dl is 7) ! 14/08/2023
41546 00003F52 88DA      mov dl,bl ; 14/08/2023
41547 00003F54 E83108      call delim ; did we hit a delimiter
41548 00003F57 750E      jne short check_color ; no, all done
41549 00003F59 E88F06      call get_number ; get next number
41550 00003F5C 80E30F      and bl,0Fh ; second # is background color (for high nibble)
41551 00003F5F 88DE      mov dh,bl ; save it in DH
41552      ; 10/09/2023
41553      ; and dl,0Fh ; !!! ;
41554 00003F61 B104      mov cl,4 ;
41555 00003F63 D2E3      shl bl,cl ;
41556 00003F65 08DA      or dl,bl ;
41557
41558 00003F67 38F5      cmp ch,dh ; are foreground/background the same?
41559 00003F69 7503      jne short set_color ; no
41560 00003F6B 80F208      xor dl,08h ; yes, so modify the fgnd intensity
41561
41562 00003F6E 88167A4C      mov [bMenuColor],dl ;
41563 00003F72 5A      pop dx ;
41564 00003F73 59      pop cx ;
41565 00003F74 E9A900      jmp menu_nextitem
41566
41567      ; Back to our regularly scheduled program (the COLOR and other goop)
41568      ; above is there simply to alleviate short jump problems)
41569
41570 menu_found:
41571 00003F77 C606844C01    mov byte [bDefBlock],1
41572      ; mov word [offDefBlock],0
41573 00003F7C C606884CFF    mov byte [secTimeout],-1
41574 00003F81 8026834CFD    and byte [bQueryOpt],~2 ; 0FDh
41575      ; 10/09/2023
41576 00003F86 29D2      sub dx,dx
41577 00003F88 8916864C      mov [offDefBlock],dx ; 0
41578
41579 00003F8C E85706      call skip_opt_line ; skip to next line
41580      ; 10/09/2023
41581      ; sub dx,dx ; initialize total block count (0 => none yet)
41582
41583      ; Process the menu block now
41584
41585 menu_process:
41586 00003F8F E87A06      call get_char ; get first char of current line
41587 00003F92 722E      jc short to_menu_getdefault ; could happen if menu block at end (rare)
41588 00003F94 247F      and al,~CONFIG_OPTION_QUERY ; 7Fh
41589 00003F96 3C5B      cmp al,CONFIG_BEGIN ; BEGIN implies END
41590 00003F98 7428      je short to_menu_getdefault
41591 00003F9A 3C4F      cmp al,CONFIG_SUBMENU
41592 00003F9C 744D      je short menu_item ; go process sub-menu
41593 00003F9E 3C45      cmp al,CONFIG_MENUITEM
41594 00003FA0 7449      je short menu_item ; go process menu item
41595 00003FA2 3C41      cmp al,CONFIG_MENUDEFAULT
41596 00003FA4 741E      je short menu_default ; go process menu default
41597 00003FA6 3C52      cmp al,CONFIG_MENUCOLOR
41598 00003FA8 749E      je short menu_color ; go process menu color
41599 00003FAA 3C4E      cmp al,CONFIG_NUMLOCK
41600 00003FAC 740F      je short menu_numlock ;
41601 00003FAE 3C30      cmp al,CONFIG_REM ; allow remarks in menu block
41602 00003FB0 746E      je short menu_nextitem ;
41603 00003FB2 E8C307      call any_delim ; allow blank lines and such
41604 00003FB5 7469      je short menu_nextitem ;
41605 00003FB7 F9      stc ;
41606 00003FB8 E82607      call print_error ; non-MENU command!
41607 00003FBB EB63      jmp short menu_nextitem
41608
41609 00003FBD E8EBFE      call set_numlock
41610 00003FC0 EB5E      jmp short menu_nextitem
41611
41612 00003FC2 EB62      to_menu_getdefault: jmp short menu_getdefault
41613
41614      ; Save the offset of the default block name, we'll need it later
41615
41616 menu_default:
41617 00003FC4 8936864C      mov [offDefBlock],si ; save address of default block name
41618 00003FC8 803E894C00    cmp byte [secElapsed],0
41619 00003FCD 751A      jne short timeout_skip ; secElapsed is only zero for the FIRST menu,
41620 00003FCF E8EA05      call skip_token ; and for subsequent menus IF nothing was typed;
41621 00003FD2 724C      jc short menu_nextitem ; secElapsed becomes -1 forever as soon as
41622 00003FD4 E8FB05      call skip_delim ; something is typed
41623 00003FD7 7247      jc short menu_nextitem ;
41624 00003FD9 89DE      mov si,bx ;
41625 00003FDB E80D06      call get_number ; get number (of seconds for timeout)
41626 00003FDE 80FB5A      cmp bl,90 ; limit it to a reasonable number
41627      ; jb short timeout_ok ; (besides, 99 is the largest # my simple
41628 00003FE1 7602      jna short timeout_ok ; 01/08/2023
41629 00003FE3 B35A      mov bl,90 ; display function can handle)
41630
41631 00003FE5 881E884C      timeout_ok: mov [secTimeout],bl ;
41632
41633 00003FE9 EB35      timeout_skip: jmp short menu_nextitem
41634
41635      ; verify that this is a valid menu item by searching for the named block

```

```

41636
41637
41638 menu_item:
41639 ;cmp dl,9 ; 04/01/2023
41640 cmp dl,MAX_MULTI_CONFIG ; have we reached the max # of items yet?
41641 jae short menu_nextitem ;
41642 mov di,si ; DS:DI -> block name to search for
41643 call srch_block ;
41644 je short menu_itemfound ;
41645 stc ;
41646 call print_error ; print error and pause
41647 jmp short menu_nextitem ; if not found, ignore this menu item
41648
41649 ; srch_block, having succeeded, returns DI -> past the token that it
41650 ; just matched, which in this case should be a descriptive string; ES:SI
41651 ; and CX are unmodified
41652
41653 menu_itemfound:
41654 inc dx ; otherwise, increment total block count
41655 mov bx,dx ; and use it to index the arrays of offsets
41656 mov [abBlockType+bx],al
41657 add bx,bx ; of recorded block names and descriptions
41658
41659 ; There should be a description immediately following the block name on
41660 ; MENUITEM line; failing that, we'll just use the block name as the
41661 ; description...
41662 mov [aoffBlockName+bx],si
41663 mov [aoffBlockDesc+bx],si
41664 mov di,bx ; skip_delim modifies BX, so stash it in DI
41665 call skip_token ;
41666 jc short menu_nextitem ; hit eol/eof
41667 call skip_delim ;
41668 jc short menu_nextitem ; hit eol/eof
41669 xchg bx,di ;
41670 mov [aoffBlockDesc+bx],di
41671
41672 menu_nextitem:
41673 call skip_opt_line ;
41674 jmp menu_process ; go back for more lines
41675
41676 ; Display menu items now, after determining which one is default
41677
41678 menu_getdefault:
41679 or dl,dl ; where there any valid blocks at all?
41680 jnz short menu_valid ; yes
41681 sub bx,bx ; no, so force autoselect of 0
41682 jmp menu_autoselect ; (meaning: process common blocks only)
41683
41684 menu_valid:
41685 sub bx,bx ;
41686 mov [bMaxBlock],dl ; first, record how many blocks we found
41687 mov di,[offDefBlock] ;
41688 or di,di ; does a default block exist?
41689 jz short menu_noddefault ; no
41690 inc bx ; yes, walk name table, looking for default
41691
41692 menu_chkdefault:
41693 push bx ;
41694 add bx,bx ;
41695 mov si,[aoffBlockName+bx] ;
41696 mov cx,128 ; arbitrary maximum length of a name
41697 push ds ;
41698 push es ;
41699 pop ds ;
41700 call comp_names ; is this block the same as the default?
41701 pop ds ;
41702 pop bx ;
41703 je short menu_setdefault ; yes
41704 inc bx ;
41705 cmp bl,[bMaxBlock] ; all done searching?
41706 jbe short menu_chkdefault ; not yet
41707
41708 menu_noddefault:
41709 mov bl,1 ; if no default, force default to #1
41710
41711 menu_setdefault:
41712 mov [bDefBlock],bl ; yes, this will be the initial current block
41713
41714 ; If the timeout was explicitly set to 0 (or technically, anything that
41715 ; failed to resolve to a number, like "NONE" or "EAT POTATOES"), then we're
41716 ; supposed to skip menu display and run with the specified default block;
41717 ; however, if the user hit Enter prior to boot, thereby requesting fully
41718 ; INTERACTIVE boot, then we shall display the menu block anyway (though still
41719 ; with no timeout)
41720 cmp byte [secTimeout],0 ; is timeout zero? (ie, assume default)
41721 jne short menu_display ; no
41722 test byte [bQueryOpt],1 ; yes, but was INTERACTIVE requested?
41723 jnz short menu_display ; yes, so *don't* assume default after all
41724 jmp not_topmenu ;
41725
41726 ; Reset the mode, so that we know screen is clean and cursor is home
41727
41728 menu_display:
41729 mov ah,0Fh ; get current video mode
41730 int 10h ;
41731 mov ah,00h ; just re-select that mode
41732 int 10h ;
41733 push es ;
41734 mov ax,40h ; reach down into the ROM BIOS data area
41735 mov es,ax ; and save the current (default) video page
41736 mov ax,[es:004Eh] ; start address and page #, in case the
41737 [wCRTStart],ax ; undocumented QUIET option was enabled
41738 mov al,[es:0062h] ;
41739 mov [bCRTPage],al ;
41740 mov ax,[bMenuPage] ; select new page for menu
41741 int 10h ;
41742 mov ax,0600h ; clear entire screen
41743 mov bh,[bMenuColor] ; using this color
41744 sub cx,cx ; upper left row/col
41745 ;mov d1,[es:CRT_Cols]
41746 mov d1,[es:4Ah]
41747 dec d1 ;
41748 ;mov dh,[es:CRT_Rows];
41749 mov dh,[es:84h]
41750 or dh,dh ; # of rows valid?
41751 jnz short menu_clear ; hopefully
41752 mov dh,[bLastRow] ; no, use a default
41753
41754 menu_clear:
41755 int 10h ; clear the screen using the req. attribute
41756 pop es ;
41757 mov [bLastRow],dh ; save DH
41758 mov dx,$MenuHeader
41759 call print ; cursor now on row 3 (numbered from 0)
41760
41761 test byte [bDisableUI],1
41762 jnz short menu_nostatus
41763 mov bh,[bMenuPage] ;

```

```

41760 000040C6 8A36[7E4C]      mov     dh,[bLastRow]      ; restore DH
41761 000040CA B200      mov     dl,0              ; print the status line on row DH, col 0,
41762 000040CC B402      mov     ah,02h           ; now that we can trash the cursor position
41763 000040CE CD10      int     10h              ;
41764 000040D0 BA[C152]      mov     dx,_%StatusLine    ;
41765 000040D3 E87C09      call    print             ;
41766 000040D6 B403      mov     ah,3             ; get cursor position
41767 000040D8 CD10      int     10h              ;
41768 000040DA 80EA02      sub     dl,2              ;
41769 000040DD 8816[7D4C]      mov     [bLastCol],dl     ; save column where status char will go
41770
41771
41772 000040E1 BB0100      menu_nostatus:
41773      mov     bx,1          ; now prepare to display all the menu items
41774 000040E4 E8B002      menu_disploop:
41775 000040E7 43          call    print_item; print item #BL
41776 000040E8 3A1E[854C]      inc     bx                ; why "inc bx"? because it's a 1-byte opcode
41777 000040EC 76F6      cmp     bl,[bMaxBlock]    ; all done?
41778      jbe short menu_disploop ; not yet
41779
41780      ; Set cursor position to just below the menu items
41781 000040EE B200      mov     dl,0              ; select column
41782 000040F0 88DE      mov     dh,b1             ;
41783 000040F2 80C604      add     dh,4              ; select row below menu
41784 000040F5 8A3E[7B4C]      mov     bh,[bMenuPage]    ;
41785 000040F9 B402      mov     ah,02h           ; set cursor position beneath the block list
41786 000040FB CD10      int     10h              ;
41787
41788 000040FD BA[AE52]      mov     dx,_%MenuPrmpt    ;
41789 00004100 E84F09      call    print             ;
41790 00004103 E82903      call    select_item       ; make a selection, return # in BX
41791 00004106 BA[6E50]      mov     dx,crlfm          ;
41792 00004109 E84609      call    print             ;
41793 0000410C FF36[7F4C]      push    word [bDisableUI] ;
41794 00004110 800E[7F4C]01      or      byte [bDisableUI],1
41795 00004115 E86704      call    show_status       ; clear the status line now
41796 00004118 8F06[7F4C]      pop     word [bDisableUI] ;
41797
41798      ; Now begins the "re-organization" process...
41799
41800      menu_autoselect:
41801 0000411C 83FBFF      cmp     bx,-1 ; 0FFFFh    ; clean boot requested?
41802 0000411F 7508      jne short normal_boot ; no
41803 00004121 E8F105      call    disable_autoexec ; basically, add a /D to the command.com line
41804
41805 00004124 29C9      menu_abort:
41806 00004126 E9E400      sub     cx,cx             ; then immediately exit with 0 config.sys image
41807      jmp menu_exit         ;
41808
41809 00004129 83FBFE      normal_boot:
41810 0000412C 7509      cmp     bx,-2 ; 0FFFEh    ; back to top-level menu?
41811 0000412E 8B0E[5603]      jne short not_topmenu ; no
41812 00004132 29F6      mov     cx,[count]        ; yes, start all over
41813 00004134 E9FFFD      sub     si,si             ;
41814      jmp     menu_search    ;
41815
41816 00004137 80BF[8A4C]4F      not_topmenu:
41817 0000413C 7510      cmp     byte [abBlockType+bx],CONFIG_SUBMENU
41818 0000413E 01DB      jne short not_submenu    ;
41819 00004140 8BBF[944C]      add     bx,bx             ;
41820 00004144 E8E101      mov     di,[aoffBlockName+bx]
41821 00004147 89FE      call    srch_block        ; THIS CANNOT FAIL!
41822 00004149 89D9      mov     si,di             ;
41823 0000414B E929FE      mov     cx,bx             ; ES:SI and CX are ready for another round
41824      jmp     menu_found    ;
41825
41826 0000414E 01DB      not_submenu:
41827 00004150 8B9F[944C]      add     bx,bx             ; get BX -> name of selected block
41828      mov     bx,[aoffBlockName+bx]
41829
41830      ; BX should now either be ZERO (meaning no block has been selected) or
41831      ; the offset relative to ES of the block name to be processed (along with
41832      ; all the "common" lines of course)
41833
41834 00004154 891E[864C]      no_selection:
41835 00004158 8B0E[5603]      mov     [offDefBlock],bx ; save selection
41836 0000415C 29F6      mov     cx,[count]        ; reset ES:SI and CX for reprocessing
41837 0000415E 1E          sub     si,si             ;
41838 0000415F 8E1E[6219]      push    ds                ;
41839 00004163 29FF      mov     ds,[config_wrkseg]; this is where we'll store new config.sys image
41840      sub     di,di          ;
41841
41842      ; ES:SI-> config.sys, DS:DI -> new config.sys workspace
41843
41844      ; work our way through the config.sys image again, this time copying
41845      ; all lines that are (A) "common" lines outside any block or (B) lines
41846      ; within the requested block. Lines inside INCLUDED blocks are transparently
41847      ; copied by copy_block in a recursive fashion; the amount of recursion is
41848      ; limited by the fact INCLUDE statements are REMed by copy_block as they are
41849      ; processed and by the number of unique INCLUDE stmts in config.sys...
41850
41851      ; BUGBUG 20-Mar-1992 JeffPar: If we can figure out the lower bound of the
41852      ; stack we're running on, then we should check it inside copy_block
41853
41854 00004165 53          copyblock_loop:
41855 00004166 E82F01      push    bx                ; save selected block name
41856 00004169 5B          call    copy_block        ; process (named or common) block
41857 0000416A 7232      pop     bx                ;
41858      jc short move_config ; hit eof
41859
41860      ; copy_block can only return for two reasons: it hit eof or a new block
41861
41862      copyblock_begin:
41863      ; 10/09/2023
41864      %if 0
41865      push    ax             ;
41866      push    cx             ;
41867      push    si             ;
41868      push    di             ; always do "common" blocks
41869      mov     di,szCommon    ;
41870      push    ds             ;
41871      push    cs             ;
41872      pop     ds             ;
41873      call    comp_names     ;
41874      pop     ds             ;
41875      pop     di             ;
41876      pop     si             ;
41877      pop     cx             ;
41878      pop     ax             ;
41879      je short copyblock_check
41880      %endif
41881      ; 10/09/2023
41882 0000416C 57          push    di
41883 0000416D BF[C94C]      mov     di,szCommon      ; always do "common" blocks

```

```

41884 00004170 E81602      call    comp_names_x    ; (comp_names_safe)
41885 00004173 5F          pop     di
41886 00004174 740F         je      short copyblock_check
41887
41888 00004176 09DB         or      bx,bx          ; is there a block name to check?
41889 00004178 7414         jz      short copyblock_skip ; no
41890 0000417A 57          push    di
41891 0000417B 89DF         mov     di,bx          ; check block against given block name
41892 0000417D 1E          push    ds
41893 0000417E 06          push    es
41894 0000417F 1F          pop     ds
41895 00004180 E8E501       call    comp_names      ; is this the block we really want to do?
41896 00004183 1F          pop     ds
41897 00004184 5F          pop     di
41898
41899 00004185 7217       copyblock_check:      jc      short move_config ; hit eof
41900 00004187 7505       jne     short copyblock_skip ;
41901 00004189 E85A04       call    skip_opt_line  ;
41902 0000418C EBD7       jmp     short copyblock_loop
41903
41904
41905 0000418E E85504       copyblock_skip:      call    skip_opt_line  ; this ain't the block we wanted, so skip it
41906 00004191 E87804       call    get_char      ;
41907 00004194 7208       jc      short move_config ; hit eof
41908 00004196 247F       and     al,~CONFIG_OPTION_QUERY ; 7Fh
41909 00004198 3C5B       cmp     al,CONFIG_BEGIN ;
41910 0000419A 74D0       je      short copyblock_begin
41911 0000419C EBF0       jmp     short copyblock_skip ; anything else is just skipped
41912
41913 ;
41914 ; To create as little risk to the rest of SysInit as little as possible,
41915 ; and to free the workspace at "config_wrkseg" for creating an environment,
41916 ; copy the new config.sys image to "confbot"
41917 ;
41918 0000419E 89F9       move_config:        mov     cx,di          ; now copy workspace at DS:DI to "confbot"
41919 000041A0 51          push    cx
41920
41921 ;
41922 ; But first, copy the CONFIG=<configuration><0> string to the workspace,
41923 ; since the configuration name only currently exists in the "confbot" area
41924 ;
41925 000041A1 B90700       ;mov     cx,7
41926 000041A4 BE[BC4C]    mov     cx,szMenu-szBoot-1
41927 000041A7 47          mov     si,szBoot ; first copy the CONFIG= part
41928 ;          inc     di          ; skip a byte, in case absolutely nothing
41929 ;          ; was copied to the workspace, because we always
41930 ;          ; zero the first byte of the workspace (below)
41931
41932 000041A8 2E          copy_boot:        ;lods     byte ptr cs:[si];
41933 000041A9 AC          ;cs
41934 000041AA 8805       lodsb
41935 000041AC 47          mov     [di],al
41936 000041AD E2F9       inc     di
41937 ;          loop    copy_boot
41938 000041AF 06          push     es
41939 ;          ; then copy the configuration name
41940 ;          ;mov     cx,128-7 ; put an upper limit on the name, to be safe
41941 000041B0 B179       ; 04/01/2023
41942 000041B2 2E8B36[864C] mov     cl,128-7
41943 000041B7 09F6       mov     si,[cs:offDefBlock]; ES:SI -> default block name
41944 000041B9 7505       or      si,si          ; valid?
41945 000041BB 0E          jnz     short l1       ; yes
41946 000041BC 07          push    cs
41947 000041BD BE[C94C]    pop     es
41948 000041C0 268A04       mov     si,szCommon
41949 000041C3 E8B205       mov     al,[es:si]
41950 000041C6 7406       call    any_delim
41951 000041C8 8805       je      short l2
41952 000041CA 46          mov     [di],al
41953 000041CB 47          inc     si
41954 000041CC E2F2       inc     di
41955 000041CE C6050A       loop    l1
41956 000041D1 07          mov     byte [di],lf ; terminate the configuration string
41957 ;          pop     es
41958 ;
41959 ; Now we can copy "config_wrkseg" (DS) to "confbot" (ES)
41960 000041D2 29FF       sub     di,di
41961 000041D4 2E893E[6019] mov     [cs:config_envlen],di
41962 000041D9 29F6       sub     si,si
41963 000041DB 59          pop     cx ; recover the size of "config_wrkseg"
41964
41965 000041DC 51          push    cx
41966 000041DD F3A4       rep     movsb ; moved!
41967 000041DF 59          pop     cx
41968 000041E0 8CD8       mov     ax,ds
41969 000041E2 1F          pop     ds
41970
41971 ;
41972 ; Now that the config_wrkseg is available once again, we shall
41973 ; use it to create an environment. The first thing to go in will be
41974 ; the "CONFIG=configuration" thing. It is also important to zero
41975 ; the first byte of the workspace, so that copy_envvar knows the buffer
41976 ; is empty.
41977 000041E3 06          push     es
41978 000041E4 8EC0       mov     es,ax
41979 000041E6 46          inc     si ; ES:SI -> "CONFIG=configuration"
41980 000041E7 26C606000000 mov     byte [es:0],0 ; empty the environment block
41981 000041ED E82600       call    copy_envvar ; copy envvar at ES:SI to "config_wrkseg"
41982 000041F0 07          pop     es
41983
41984 ;
41985 ; Before returning, restore the default video page setting but do NOT
41986 ; do it using INT 10h's Set Active Page function, because if the menu was
41987 ; displayed on a different page, then it's because we don't want to see
41988 ; all the device driver/TSR goop (which goes to the default page)
41989
41990 000041F1 803E[7B4C]00 menu_done:      cmp     byte [bMenuPage],0
41991 000041F6 7415       je      short menu_exit ;
41992 000041F8 06          push    es
41993 000041F9 B84000       mov     ax,40h
41994 000041FC 8EC0       mov     es,ax
41995 000041FE A1[814C]    mov     ax,[wCRTstart]
41996 00004201 26A34E00   mov     [es:004Eh],ax
41997 00004205 A0[804C]    mov     al,[bCRTPage]
41998 00004208 26A26200   mov     [es:0062h],al
41999 0000420C 07          pop     es
42000
42001 0000420D 890E[5603] menu_exit:      mov     [count],cx ; set new counts
42002 00004211 890E[5803] mov     [org_count],cx
42003 ; 10/09/2023 (*) - Erdogan Tan
42004 ; MSDOS 6.21 IO.SYS - SYSINIT:46D3h
42005 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:491Ah
42006 ;sub     si,si ; always return ES:SI pointing to config.sys
42007 00004215 C3          retn

```

```

42008
42009 ; (*) NOTE: MSDOS 6.0 source code (SYSINIT2.ASM) contains 'sub si,si' at this
42010 ; position (then 'retn' just after it)
42011 ; but MSDOS 6.21 and PCDOS 7.1 SYSINITs contain only 'retn' here.
42012
42013 -----
42014 ;
42015 ; copy_envvar: copy the envvar at ES:SI to "config_wrkseg"
42016 ;
42017 ; INPUT
42018 ; ES:SI -> environment variable (in the form "var=string<cr/lf>")
42019 ;
42020 ; OUTPUT
42021 ; config_envlen (ie, where to put next envvar) updated appropriately
42022 ; carry set if error (eg, missing =); clear otherwise
42023 ;
42024 ; OTHER REGS USED
42025 ; None
42026 ;
42027 ; NOTES
42028 ; None
42029 ;
42030 ; HISTORY
42031 ; Created 29-Mar-1992 by JeffPar
42032 ;
42033 -----
42034
42035 ; 04/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
42036 ; (SYSINIT:46D4h)
42037
42038 copy_envvar:
42039     push    cx
42040     push    si
42041     push    ds
42042     push    es
42043     push    es
42044     mov     es,[config_wrkseg] ; ES:DI to point to next available byte
42045     pop     ds                 ; DS:SI to point to envvar
42046
42047 ; Have to calculate the length of the variable name (and if we hit
42048 ; the end of the line before we hit '=', then it's curtains for this
42049 ; config.sys line)
42050 ;
42051 ; The check for NULL is important because copy_envvar is also used to copy
42052 ; the initial CONFIG= setting, which will have been zapped by a NULL if no
42053 ; menu block existed (in order to prevent the creation of an environment)
42054
42055     sub     cx,cx
42056 copy_varlen:
42057     lodsb
42058     or      al,al
42059     ;stc    ; 10/09/2023 (x)
42060     jz      short copy_envexit ; yes, abort
42061     cmp     al,cr
42062     ;stc    ; 10/09/2023 (x)
42063     je      short copy_envexit
42064     cmp     al,lf
42065     ;stc    ; 10/09/2023 (x)
42066     je      short copy_envexit
42067     inc     cx
42068     cmp     al,'='
42069     jne     short copy_varlen
42070     mov     al,0
42071     mov     ah,[si]
42072     sub     si,cx
42073     dec     cx
42074     sub     di,di
42075     ; save char after '='
42076     ; back up to given varname
42077     ; CX == # of bytes in varname
42078     ; start looking for DS:SI at ES:0
42079 copy_varsrch:
42080     cmp     byte [es:di],al
42081     je      short copy_envprep ; search failed, just copy var
42082     mov     bx,di
42083     push    cx
42084     push    si
42085     repe    cmpsb
42086     pop     si
42087     pop     cx
42088     jne     short copy_varnext ; no match, skip to next varname
42089     cmp     byte [es:di],'='
42090     jne     short copy_varnext ; no match, there's more characters
42091
42092 ; Previous occurrence of variable has been found; determine the
42093 ; entire length and then destroy it
42094
42095     mov     cx,-1
42096     repne   scasb
42097     ; guaranteed to get null (since we put it there)
42098     push    si
42099     mov     si,di
42100     mov     di,bx
42101     mov     cx,[cs:config_envlen]
42102     sub     cx,si
42103     ; destroy variable now
42104     ; rep movs byte ptr es:[di],byte ptr es:[si]
42105     ; db 0F3h,26h,0A4h ; MSDOS 6.21 IO.SYS - SYSINIT:4724h
42106
42107     rep     ; 0F3h
42108     es     ; 26h
42109     movsb  ; 0A4h
42110
42111     pop     si
42112 copy_envprep:
42113     cmp     ah,cr
42114     ; if there is nothing after the '='
42115     je      short copy_envdel ; then just exit with variable deleted
42116     cmp     ah,lf
42117     ;
42118     je      short copy_envdel
42119     ; jmp     short copy_envloop
42120 ; 04/01/2023
42121 copy_envloop:
42122     ;
42123     lodsb
42124     cmp     al,cr
42125     ;
42126     je      short copy_envdone
42127     cmp     al,lf
42128     ;
42129     je      short copy_envdone
42130     stosb
42131     jmp     short copy_envloop
42132
42133 copy_varnext:
42134     ;
42135     push    cx
42136     mov     cx,-1
42137     repne   scasb
42138     pop     cx
42139     jmp     short copy_varsrch
42140
42141 ; 04/01/2023
42142 ; copy_envloop:
42143 ; lodsb
42144 ;

```



```

42132      ;      cmp      al,cr      ;
42133      ;      je       short copy_envdone
42134      ;      cmp      al,lf      ;
42135      ;      je       short copy_envdone
42136      ;      stosb     ;
42137      ;      jmp      short copy_envloop
42138
42139      copy_envdone:
42140      sub      al,al      ; do SUB to clear carry as well
42141      stosb     ; always null-terminate these puppies
42142
42143      copy_envdel:
42144      mov      [es:di],al    ; and stick another null to terminate the env.
42145      mov      [cs:config_envlen],di
42146      ; 10/09/2023 (x) - Erdogan Tan
42147      stc      ; in order to clear carry flag via cmc (compact code trick!)
42148      copy_envexit:
42149      cmc      ; (x) ; reverse carry flag status (je -> cf=1)
42150      pop      es          ;
42151      pop      ds          ;
42152      pop      si          ;
42153      pop      cx          ;
42154
42155      copy_done: ; 18/12/2022
42156      retn
42157
42158      ;-----
42159      ; copy_block: copy the current block to the new config.sys workspace
42160      ;
42161      ; INPUT
42162      ; CX == remaining bytes in "organized" config.sys memory image
42163      ; ES:SI -> remaining bytes in "organized" config.sys memory image
42164      ; DS:DI -> new config.sys workspace (equal in size to the original
42165      ; config.sys image) where the current block is to be copied
42166      ;
42167      ; OUTPUT
42168      ; Same as above
42169      ; AL also equals the last character read from the organized image
42170      ;
42171      ; OTHER REGS USED
42172      ; All
42173      ;
42174      ; NOTES
42175      ; None
42176      ;
42177      ; HISTORY
42178      ; Created 16-Mar-1992 by JeffPar
42179      ;-----
42180
42181      ; 04/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
42182      ; (SYSINIT:4759h)
42183
42184      copy_block:
42185      call     get_char      ; check for include
42186      jc      short copy_done
42187      and     al,~CONFIG_OPTION_QUERY ; 7Fh
42188      cmp     al,CONFIG_BEGIN ; another BEGIN implies END as well
42189      je      short copy_done ;
42190
42191      cmp     al,CONFIG_INCLUDE ; 'J'
42192      mov     al,ah          ; AL == the original line code
42193      jne     short copy_line ; not an "include" line
42194
42195      ; We have hit an "INCLUDE" line; first, REM out the line so that we
42196      ; never try to include the block again (no infinite include loops please),
42197      ; then search for the named block and call copy_block again.
42198
42199      mov     byte [es:si-1],CONFIG_REM ; '0'
42200      push    di            ;
42201
42202      mov     di,szMenu
42203      call    comp_names_safe ; don't allow INCLUDE MENU
42204      je      short copy_skip ;
42205
42206      mov     di,szCommon
42207      call    comp_names_safe ; don't allow INCLUDE COMMON
42208      je      short copy_skip ;
42209
42210      mov     di,si          ; try to find the block
42211      call    srch_block
42212      mov     dx,di          ;
42213
42214      ; 10/09/2023
42215      ; pop     di            ;
42216      jne     short copy_error ; no such block
42217      pop     di            ; 10/09/2023
42218
42219      push    cx             ;
42220      mov     cx,bx          ;
42221      push    si             ;
42222      dec     dx             ;
42223      mov     si,dx          ;
42224      call    skip_line      ; skip the rest of the "block name" line
42225      call    copy_block     ; and copy in the rest of that block
42226      pop     si             ;
42227      pop     cx             ;
42228      sub     al,al          ; force skip_opt_line to skip...
42229      jmp     short copy_nextline
42230
42231      copy_error:
42232      ; 10/09/2023
42233      cld
42234      copy_skip:
42235      pop     di
42236      ; copy_error:
42237      ; 10/09/2023 (cf=0)
42238      ; cld
42239      call    print_error    ; note that carry is clear, no pause
42240      jmp     short copy_nextline
42241
42242      ; Copy the line at ES:SI to the current location at DS:DI
42243
42244      copy_line:
42245      mov     [di],al        ;
42246      inc     di             ;
42247      cmp     al,' '         ; is this is a "real" line with a "real" code?
42248      jb      short copy_nextline ; no
42249      cmp     byte [cs:config_multi],0
42250      je      short copy_loop ; not a multi-config config.sys, don't embed #s
42251      call    get_linenum    ; BX == line # of line @ES:SI
42252      mov     [di],bx        ; stash it immediately following the line code
42253      inc     di             ;
42254      inc     di             ;
42255      jmp     short copy_next ;
42256      copy_loop:

```

```

42256 000042FB E80E03      call    get_char      ;
42257 000042FE 7297      jc     short copy_done ; end of file
42258 00004300 8805      mov     [di],al      ;
42259 00004302 47        inc     di          ;
42260
42261 00004303 3C0A      cmp     al,lf ; 0Ah    ; done with line?
42262 00004305 75F4      jne short copy_loop   ; nope
42263
42264
42265 00004307 E8DC02      call    skip_opt_line ;
42266 0000430A EB8C      jmp     short copy_block
42267
42268 ; 18/12/2022
42269 ;copy_done:
42270 ;retn
42271
42272
42273
42274
42275
42276
42277
42278
42279
42280
42281
42282
42283
42284
42285
42286
42287
42288
42289
42290
42291
42292
42293
42294 0000430C 50
42295 0000430D 29DB
42296 0000430F 51
42297 00004310 89F2
42298 00004312 56
42299 00004313 2E8B0E[5603]
42300 00004318 29F6
42301
42302 0000431A E8C402
42303 0000431D 7205
42304 0000431F 43
42305 00004320 39D6
42306 00004322 72F6
42307
42308 00004324 5E
42309 00004325 59
42310 00004326 58
42311 00004327 C3
42312
42313
42314
42315
42316
42317
42318
42319
42320
42321
42322
42323
42324
42325
42326
42327
42328
42329
42330
42331
42332
42333
42334
42335
42336
42337
42338
42339
42340
42341 00004328 50
42342 00004329 51
42343 0000432A 2E8B0E[5603]
42344 0000432F 56
42345 00004330 29F6
42346 00004332 1E
42347 00004333 06
42348 00004334 1F
42349 00004335 E80900
42350 00004338 89F7
42351 0000433A 89CB
42352 0000433C 1F
42353 0000433D 5E
42354 0000433E 59
42355 0000433F 58
42356
42357 00004340 C3
42358
42359
42360
42361
42362
42363
42364
42365
42366
42367
42368
42369
42370
42371
42372
42373
42374
42375
42376
42377
42378
42379

```

```

;-----
;
; get_linenum:  return line # (in BX) of current line (@ES:SI)
;
; INPUT
;   ES:SI -> some line in the config.sys memory image
;
; OUTPUT
;   BX == line # (relative to 1)
;
; OTHER REGS USED
;   DX
;
; NOTES
;   None
;
; HISTORY
;   Created 16-Mar-1992 by JeffPar
;-----
;
get_linenum:
    push    ax                ;
    sub     bx,bx             ; BX == line # (to be returned)
    push    cx                ;
    mov     dx,si             ; DX == the offset we're looking for
    push    si                ;
    mov     cx,[cs:count]     ;
    sub     si,si             ; prepare to scan entire file
get_linenum_loop:
    call    skip_line         ;
    jc     short get_linenum_done
    inc     bx                ;
    cmp     si,dx             ; have we exceeded the desired offset yet?
    jb     short get_linenum_loop ; no
get_linenum_done:
    pop     si                ;
    pop     cx                ;
    pop     ax                ;
    retn
;-----
;
; srch_block:  searches entire config.sys for block name @ES:DI
;
; INPUT
;   ES -> config.sys image
;   ES:DI -> block name to find
;
; OUTPUT
;   ZF flag set, if found
;   ES:DI -> just past the name in the block heading, if found
;   BX == # bytes remaining from that point, if found
;
; OTHER REGS USED
;   None
;
; NOTES
;   This differs from "find_block" in that it searches the ENTIRE
;   config.sys image, not merely the remaining portion, and that it
;   takes a pointer to block name that is *elsewhere* in the image
;   (ie, ES) as opposed to some string constant in our own segment (DS).
;
; HISTORY
;   Created 16-Mar-1992 by JeffPar
;-----
;
srch_block:
;   ; returns BX -> named block in CONFIG.SYS
    push    ax                ;
    push    cx                ;
    mov     cx,[cs:count]     ;
    push    si                ;
    sub     si,si             ;
    push    ds                ;
    push    es                ;
    pop     ds                ;
    call    find_block        ;
    mov     di,si             ;
    mov     bx,cx             ;
    pop     ds                ;
    pop     si                ;
    pop     cx                ;
    pop     ax                ;
find_exit:
;   ; 16/04/2019
    retn
;-----
;
; find_block:  searches rest of config.sys for block name @DS:DI
;
; INPUT
;   DS:DI -> block name to find
;   ES:SI -> remainder of config.sys image
;   CX == remaining size of config.sys image
;
; OUTPUT
;   ZF flag set, if found (also, CF set if EOF)
;   ES:SI -> where the search stopped (at end of block name or EOF)
;   CX == # bytes remaining from that point
;
; OTHER REGS USED
;   AX
;
; NOTES
;   This differs from "srch_block" in that it searches only the
;   remaining portion of the config.sys image and leaves SI and CX
;   pointing to where the search left off, and that it takes a pointer

```



```

42380 ; to search string in our own segment (DS:DI instead of ES:DI).
42381 ;
42382 ; HISTORY
42383 ; Created 16-Mar-1992 by JeffPar
42384 ;
42385 ;-----
42386
42387 find_block:
42388 00004341 E8C802 call get_char ; get line code
42389 00004344 72FA jc short find_exit ; end of file
42390 00004346 247F and al,~CONFIG_OPTION_QUERY
42391 00004348 3C5B cmp al,CONFIG_BEGIN ; beginning of a block?
42392 0000434A 740C je short check_line ; no
42393 0000434C 3C4A cmp al,CONFIG_INCLUDE
42394 0000434E 7513 jne short next_line
42395 00004350 2E800E[6519]01 or byte [cs:config_multi],1
42396 00004356 EB0B jmp short next_line ;
42397
42398 00004358 2E800E[6519]01 check_line:
42399 0000435E E80700 or byte [cs:config_multi],1
42400 00004361 76DD call comp_names ; compare block names
42401 ; jbe short find_exit ; end of file, or names matched
42402 00004363 E88002 next_line:
42403 00004366 EBD9 call skip_opt_line ; no, so skip to next line
42404 ; jmp short find_block ;
42405 ; find_exit:
42406 ; retn
42407 ;-----
42408 ;
42409 ; comp_names: compares keyword @DS:DI to position in config.sys @ES:SI
42410 ;
42411 ; INPUT
42412 ; DS:DI -> keyword to compare
42413 ; ES:SI -> position in config.sys
42414 ; CX == remaining bytes in config.sys
42415 ;
42416 ; OUTPUT
42417 ; ZF flag set, if match (also, CF set if EOF)
42418 ; ES:SI -> where the comparison stopped (at end of block name or EOF)
42419 ; CX == # bytes remaining from that point
42420 ;
42421 ; OTHER REGS USED
42422 ; AX
42423 ;
42424 ; NOTES
42425 ; None
42426 ;
42427 ; HISTORY
42428 ; Created 16-Mar-1992 by JeffPar
42429 ;
42430 ;-----
42431
42432 comp_names:
42433 00004368 57 push di ;
42434 comp_loop: ;
42435 00004369 E8A002 call get_char ;
42436 0000436C 7210 jc short comp_exit ;
42437 0000436E E80704 call any_delim ; is next character a delimiter?
42438 00004371 8A25 mov ah,[di] ; (get next character we're supposed to match)
42439 00004373 740B je short comp_almost ; yes, it *could* be a match
42440 00004375 47 inc di ;
42441 00004376 25DFDF and ax,~2020h ; 0DFDFh
42442 ; BUGBUG -- assumes both names are alphanumeric -JTP
42443 00004379 38E0 cmp al,ah ; match?
42444 0000437B 74EC je short comp_loop ; yes, keep looking at the characters
42445 0000437D F8 clc ; prevent erroneous eof indication: clear carry
42446 comp_exit:
42447 0000437E 5F pop di ;
42448 0000437F C3 retn ;
42449 comp_almost:
42450 00004380 86C4 xchg al,ah ; we don't know for sure if it's a match
42451 00004382 E8F303 call any_delim ; until we verify that the second string has
42452 00004385 86C4 xchg al,ah ; been exhausted also...
42453 00004387 EBF5 jmp short comp_exit ; if we are, this call to any_delim will tell...
42454 ;-----
42455 ;
42456 ; 10/09/2023 - RetroDOS v4.2 IO.SYS (Optimization)
42457 comp_names_x:
42458 ;
42459 ; comp_names_safe:
42460 ;
42461 00004389 50 push ax
42462 0000438A 51 push cx
42463 0000438B 56 push si
42464 0000438C 1E push ds
42465 0000438D 0E push cs
42466 0000438E 1F pop ds
42467 0000438F E8D6FF call comp_names
42468 00004392 1F pop ds
42469 00004393 5E pop si
42470 00004394 59 pop cx
42471 00004395 58 pop ax
42472 00004396 C3 retn
42473 ;-----
42474 ;
42475 ; print_item: display menu item #BL
42476 ;
42477 ; INPUT
42478 ; BL == menu item # to display
42479 ;
42480 ; OUTPUT
42481 ; Menu item displayed, with appropriate highlighting if BL == bDefBlock
42482 ;
42483 ; OTHER REGS USED
42484 ; None
42485 ;
42486 ; NOTES
42487 ; This function saves/restores the current cursor position, so you
42488 ; needn't worry about it.
42489 ;
42490 ; HISTORY
42491 ; Created 16-Mar-1992 by JeffPar
42492 ;
42493 ;-----
42494 ;
42495 ; 04/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
42496 ; (SYSINIT:485Ah)
42497
42498 print_item: ; prints menu item #BL (1 to N)
42499 00004397 50 push ax ;
42500 00004398 53 push bx ;
42501 00004399 51 push cx ;
42502 0000439A 52 push dx ;

```

```

42504 0000439B 56          push    si          ;
42505 0000439C B403       mov     ah,03h        ; get cursor position
42506 0000439E 8A3E[7B4C]  mov     bh,[bMenuPage] ; always page zero
42507 000043A2 CD10       int     10h         ; DH/DL = row/column
42508 000043A4 52          push    dx          ; save it
42509 000043A5 B402       mov     ah,02h        ; set cursor position
42510 000043A7 88DE       mov     dh,b1         ;
42511 000043A9 80C603    add     dh,3          ;
42512 000043AC B205       mov     dl,5          ;
42513 000043AE CD10       int     10h         ; set cursor position for correct row/col
42514 000043B0 88D8       mov     al,b1         ;
42515 000043B2 0430       add     al,'0'        ; convert menu item # to ASCII digit
42516 000043B4 8A26[7A4C]  mov     ah,[bMenuColor] ; normal attribute
42517 000043B8 3A1E[844C]  cmp     bl,[bDefBlock] ; are we printing the current block?
42518 000043BC 7510       jne short print_other ; no
42519 000043BE 80CC70    or      ah,70h        ; yes, set bgnd color to white
42520 000043C1 88E5       mov     ch,ah         ;
42521 000043C3 B104       mov     cl,4          ;
42522 000043C5 D2C5       rol     ch,cl        ;
42523 000043C7 38E5       cmp     ch,ah         ; are fgnd/bgnd the same?
42524 000043C9 7503       jne short print_other ; no
42525 000043CB 80F408    xor     ah,08h        ; yes, so modify the fgnd intensity
42526          print_other:
42527 000043CE B700       mov     bh,0          ;
42528 000043D0 01DB       add     bx,bx         ;
42529 000043D2 88BF[A84C]  mov     di,[aoffBlockDesc+bx]
42530 000043D6 88E3       mov     bl,ah         ; put the attribute in the correct register now
42531 000043D8 8A3E[7B4C]  mov     bh,[bMenuPage] ; get correct video page #
42532 000043DC B409       mov     ah,09h        ; write char/attr
42533 000043DE B90100    mov     cx,1          ;
42534 000043E1 CD10       int     10h         ;
42535 000043E3 FEC2       inc     dl            ; increment column
42536 000043E5 B402       mov     ah,02h        ;
42537 000043E7 CD10       int     10h         ;
42538          ;mov     ax,0900h+'.' ;
42539 000043E9 B82E09    mov     ax,092Eh      ;
42540 000043EC CD10       int     10h         ; display '.'
42541 000043EE FEC2       inc     dl            ; increment column
42542 000043F0 B402       mov     ah,02h        ;
42543 000043F2 CD10       int     10h         ;
42544          ;mov     ax,0900h+' ' ;
42545 000043F4 B82009    mov     ax,0920h      ;
42546 000043F7 CD10       int     10h         ; display ' '
42547 000043F9 FEC2       inc     dl            ; increment column
42548 000043FB B402       mov     ah,02h        ;
42549 000043FD CD10       int     10h         ;
42550 000043FF 06          push    es          ;
42551          print_loop:
42552 00004400 268A05    mov     al,[es:di] ; get a character of the description
42553 00004403 47          inc     di            ;
42554 00004404 3C09       cmp     al,TAB ; 9; substitute spaces for tabs
42555 00004406 7502       jne short print_nontab ;
42556 00004408 B020       mov     al,' '        ;
42557          print_nontab:
42558 0000440A 3C20       cmp     al,' '        ;
42559 0000440C 7215       jb short print_done ; stop at the 1st character < space
42560 0000440E 3C24       cmp     al,'$'        ;
42561 00004410 7411       je short print_done ; also stop on $
42562 00004412 B409       mov     ah,09h        ; display function #
42563 00004414 CD10       int     10h         ;
42564 00004416 FEC2       inc     dl            ; increment column
42565 00004418 80FA4E    cmp     dl,78         ; far enough?
42566 0000441B 7306       jae short print_done ; yes
42567 0000441D B402       mov     ah,02h        ;
42568 0000441F CD10       int     10h         ;
42569 00004421 EBDD       jmp short print_loop
42570          print_done:
42571 00004423 07          pop     es          ;
42572 00004424 5A          pop     dx          ;
42573 00004425 B402       mov     ah,02h        ;
42574 00004427 CD10       int     10h         ; restore previous row/col
42575 00004429 5E          pop     si          ;
42576 0000442A 5A          pop     dx          ;
42577 0000442B 59          pop     cx          ;
42578 0000442C 5B          pop     bx          ;
42579 0000442D 58          pop     ax          ;
42580 0000442E C3          retn           ;
42581
42582          ;-----
42583          ;
42584          ; select_item: wait for user to select menu item, with time-out
42585          ;
42586          ; INPUT
42587          ;     None
42588          ;
42589          ; OUTPUT
42590          ;     BX == menu item # (1-N), or -1 for clean boot
42591          ;     Selected menu item highlighted
42592          ;     Cursor positioned beneath menu, ready for tty-style output now
42593          ;
42594          ; OTHER REGS USED
42595          ;     None
42596          ;
42597          ; NOTES
42598          ;     None
42599          ;
42600          ; HISTORY
42601          ;     Created 16-Mar-1992 by JeffPar
42602          ;-----
42603
42604
42605          select_item:
42606 0000442F 8A1E[844C]  mov     bl,[bDefBlock] ; BL will be the default block #
42607 00004433 88D8       mov     al,b1         ;
42608 00004435 E83701    call    disp_num      ;
42609 00004438 E84401    call    show_status   ; display current interactive status
42610 0000443B 803E[884C]FF  cmp     byte [secTimeout],-1 ; no time-out, just go to input
42611 00004440 7452       je short input_key
42612 00004442 B42C       mov     ah,GET_TIME ; 2ch
42613 00004444 CD21       int     21h         ;
42614 00004446 88F7       mov     bh,dh         ; BH = initial # of seconds
42615          check_time:
42616 00004448 A0[884C]    mov     al,[secTimeout] ;
42617 0000444B 2A06[894C]  sub     al,[secElapsed] ;
42618 0000444F 730D       jae short show_time
42619 00004451 800E[834C]02  or      byte [bQueryOpt],2 ; disable all further prompting
42620 00004456 C606[894C]00  mov     byte [secElapsed],0
42621 0000445B E9F600     jmp select_done ; time's up!
42622          show_time:
42623 0000445E 53          push    bx          ;
42624 0000445F 88C3       mov     bl,al         ; save # in BL
42625 00004461 8A3E[7B4C]  mov     bh,[bMenuPage] ;
42626 00004465 B403       mov     ah,03h        ; get cursor position
42627 00004467 CD10       int     10h         ;

```

```

42628 00004469 52          push    dx          ;
42629 0000446A 80C208      add     dl,8          ; move cursor to the right
42630 0000446D B402          mov     ah,02h       ; set cursor position
42631 0000446F CD10          int     10h         ;
42632 00004471 BA[2553]    mov     dx,$Timeout    ;
42633 00004474 E8DB05    call    print          ; print the "Time remaining: " prompt
42634 00004477 88D8          mov     al,bl          ; recover # from BL
42635 00004479 98           cbw           ; this works because AL is always <= 90
42636 0000447A B10A          mov     cl,10          ;
42637 0000447C F6F1          div     cl           ; AL = tens digit, AH = ones digit
42638 0000447E 88E1          mov     cl,ah          ;
42639 00004480 0430          add     al,'0'         ;
42640 00004482 B40E          mov     ah,0Eh        ;
42641 00004484 CD10          int     10h         ; write TTY tens digit
42642 00004486 88C8          mov     al,cl          ;
42643 00004488 0430          add     al,'0'         ;
42644 0000448A B40E          mov     ah,0Eh        ;
42645 0000448C CD10          int     10h         ; write TTY ones digit
42646 0000448E 5A           pop     dx            ;
42647 0000448F B402          mov     ah,02h       ; set cursor position back to where it was
42648 00004491 CD10          int     10h         ;
42649 00004493 5B           pop     bx            ;
42650
input_key:
42651 00004494 B406          mov     ah,RAW_CON_IO ; 6
42652 00004496 B2FF          mov     dl,0FFh      ; input request
42653 00004498 CD21          int     21h         ;
42654 0000449A 751F          jnz short got_key     ;
42655 0000449C 803E[884C]FF  cmp     byte [secTimeout],-1; is there a time-out?
42656 000044A1 74F1          je      short input_key ; no, just go back to input
42657 000044A3 B42C          mov     ah,GET_TIME  ;
42658 000044A5 CD21          int     21h         ; DH = seconds
42659 000044A7 88F4          mov     ah,dh         ;
42660 000044A9 28FE          sub     dh,bh         ; should generally be zero or one
42661 000044AB 88E7          mov     bh,ah         ;
42662 000044AD 7302          jnc short got_time    ;
42663 000044AF B601          mov     dh,1         ; it wrapped back to zero, so assume one
42664
got_time:
42665 000044B1 08F6          or      dh,dh         ; any change?
42666 000044B3 74DF          jz      short input_key ; no
42667 000044B5 0036[894C]  add     [secElapsed],dh ;
42668 000044B9 EB8D          jmp short check_time  ;
42669
got_key:
42670 000044BB 50           push    ax            ;
42671 000044BC B8FFFF          mov     ax,-1         ; zap both secTimeout and secElapsed
42672 000044BF 8706[884C]  xchg    [secTimeout],ax ;
42673 000044C3 3CFF          cmp     al,-1         ; was time-out already disabled?
42674 000044C5 740E          je      short timeout_disabled ; yes
42675 000044C7 53           push    bx            ; let's disable # seconds display
42676 000044C8 B8200A       mov     ax,0A20h      ; write multiple spaces
42677 000044CB 8B1E[7A4C]  mov     bx,[bMenuColor] ;
42678 000044CF B95000       mov     cx,80         ; 80 of them, to be safe
42679 000044D2 CD10          int     10h         ; to completely obliterate # seconds display
42680 000044D4 5B           pop     bx            ;
42681
timeout_disabled:
42682
42683 000044D5 58           pop     ax            ;
42684 000044D6 08C0          or      al,al         ; extended key pressed?
42685 000044D8 755A          jnz short normal_key ; no
42686 000044DA CD21          int     21h         ; get the next part of the key then
42687 000044DC 74B6          jz      short input_key ; hmmm, what happened to the second part?
42688
42689 000044DE 3C48          cmp     al,48h        ; up arrow?
42690 000044E0 7510          jne short not_up      ; no
42691 000044E2 80FB01       cmp     bl,1          ; are we as up as up can get?
42692 000044E5 76AD          jbe short input_key    ; yes, ignore it
42693 000044E7 FE0E[844C]  dec     byte [bDefBlock] ;
42694 000044EB E8A9FE       call    print_item     ; re-print the current item
42695 000044EE FECB          dec     bl            ; and then print the new current item
42696 000044F0 EB12          jmp     short print1    ;
42697
not_up:
42698 000044F2 3C50          cmp     al,50h        ; down arrow?
42699 000044F4 7518          jne short not_down    ; no
42700 000044F6 3A1E[854C]  cmp     bl,[bMaxBlock] ; are we as down as down can get?
42701 000044FA 7310          jae short to_input_key ; yes, ignore it
42702 000044FC FE06[844C]  inc     byte [bDefBlock] ;
42703 00004500 E894FE       call    print_item     ; re-print the current item
42704 00004503 43           inc     bx            ; and then print the new current item
42705
print1:
42706 00004504 88D8          mov     al,bl          ;
42707
print2:
42708 00004506 E88EFE       call    print_item     ;
42709 00004509 E86300       call    disp_num       ;
42710
to_input_key:
42711 0000450C EB86          jmp short input_key ; 10/09/2023
42712
not_down:
42713 0000450E F606[7F4C]01 test     byte [bDisableUI],1
42714 00004513 75F7          jnz short to_input_key ; don't allow F8 or F5
42715 00004515 3C42          cmp     al,42h        ; F8 function key?
42716 00004517 750B          jne short not_f8      ; no
42717 00004519 8036[834C]01 xor     byte [bQueryOpt],1
42718 0000451E E85E00       call    show_status    ;
42719 00004521 E970FF       jmp     input_key      ;
42720
not_f8:
42721 00004524 3C3F          cmp     al,3Fh        ; F5 function key?
42722 00004526 75E4          jne short to_input_key ; no
42723 ; 02/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
42724 ; MSDOS 6.21 IO.SYS - SYSINIT:49EBh
42725 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4C32h)
42726 00004528 800E[834C]04 or      byte [bQueryOpt],4 ; no more queries
42727 0000452D BBFFFF          mov     bx,-1         ; special return code (-1) indicating clean boot
42728 00004530 B020          mov     al,' '        ; don't want to display anything really;
42729 00004532 EB26          jmp     short disp_input ; just want to display the cr/lf sequence...
42730
normal_key:
42731
42732 00004534 3C0D          cmp     al,0Dh        ; Enter?
42733 00004536 741C          je      short select_done ; yes
42734 00004538 3C08          cmp     al,08h        ; backspace?
42735 0000453A 7504          jne short not_backspace ; no
42736 0000453C BBFEFF          mov     bx,-2 ; 0FFFEh ; yes, special return code
42737 0000453F C3           retn                 ;
42738
not_backspace:
42739 00004540 2C30          sub     al,'0'        ; is greater than '0'?
42740 00004542 76C8          jbe short to_input_key ; no
42741 00004544 3A06[854C]  cmp     al,[bMaxBlock] ; is less than or equal to the maximum digit?
42742 00004548 77C2          ja      short to_input_key ; no
42743 0000454A A2[844C]    mov     [bDefBlock],al ;
42744 0000454D E847FE       call    print_item     ; redisplay the current selection
42745 00004550 88C3          mov     bl,al         ; set new selection
42746 00004552 EBB2          jmp short print2
42747
select_done:
42748
42749 00004554 B700          mov     bh,0          ; return a full 16-bit value (for indexing)
42750 00004556 88D8          mov     al,bl         ;
42751 00004558 0430          add     al,'0'        ; convert it into a digit, then display it

```

```

42752
42753
42754
42755
42756
42757
42758
42759
42760
42761
42762
42763
42764
42765
42766
42767
42768
42769
42770
42771
42772
42773
42774
42775
42776
42777
42778
42779
42780
42781 0000455A 50
42782
42783
42784
42785
42786 0000455B B220
42787 0000455D 38D0
42788 0000455F 7602
42789
42790 00004561 88C2
42791
42792 00004563 B402
42793 00004565 CD21
42794 00004567 BA[6E50]
42795 0000456A E8E504
42796 0000456D 58
42797 0000456E C3
42798
42799
42800
42801
42802 0000456F 53
42803 00004570 0430
42804 00004572 B40A
42805 00004574 8B1E[7A4C]
42806 00004578 B90100
42807 0000457B CD10
42808 0000457D 5B
42809 0000457E C3
42810
42811
42812
42813
42814
42815
42816
42817
42818
42819
42820
42821
42822
42823
42824
42825
42826
42827
42828
42829
42830
42831
42832
42833 0000457F 53
42834 00004580 8B1E[7A4C]
42835 00004584 B403
42836 00004586 CD10
42837 00004588 52
42838 00004589 B402
42839 0000458B 8B1E[7D4C]
42840 0000458F F606[7F4C]01
42841 00004594 740C
42842 00004596 B200
42843 00004598 CD10
42844 0000459A B8200A
42845 0000459D B95000
42846
42847
42848 000045A0 EB11
42849
42850 000045A2 CD10
42851
42852
42853
42854
42855
42856 000045A4 A0[2153]
42857 000045A7 803E[834C]01
42858 000045AC 7503
42859 000045AE A0[1D53]
42860
42861 000045B1 B40E
42862
42863 000045B3 CD10
42864
42865 000045B5 5A
42866 000045B6 B402
42867 000045B8 CD10
42868 000045BA 5B
42869 000045BB C3
42870
42871
42872
42873
42874
42875

; fall into disp_input
; 16/04/2019 - Retro DOS v4.0

-----
;
; disp_input: display a single character + cr/lf
;
; INPUT
;     AL == character to display
;
; OUTPUT
;     None
;
; OTHER REGS USED
;     None
;
; NOTES
;     This function is used not only for the menu input selection but
;     also for the interactive line prompting (the y/n/a thing).
;
; HISTORY
;     Created 16-Mar-1992 by JeffPar
;
-----

disp_input:
    push    ax
    ;cmp    al,' '
    ;jae    short disp_ok
    ;mov     al,' '
    ; 10/09/2023 - Retro DOS v4.2 IO:SYS (Optimization)
    mov     dl,' ' ; 20h
    cmp     al,dl
    jna     short disp_input_ok
disp_ok:
    mov     dl,al
disp_input_ok:
    mov     ah,STD_CON_OUTPUT ; 2
    int     21h
    mov     dx,crlfm
    call    print
    pop     ax
    retn

-----

disp_num:
    push    bx
    add     al,'0'
    mov     ah,0Ah
    mov     bx,[bMenuColor]
    mov     cx,1
    int     10h
    pop     bx
    retn

-----

;
; show_status: display current interactive mode setting (on/off/none)
;
; INPUT
;     None
;
; OUTPUT
;     None
;
; OTHER REGS USED
;     None
;
; NOTES
;     None
;
; HISTORY
;     Created 16-Mar-1992 by JeffPar
;
-----

show_status:
    push    bx
    mov     bx,[bMenuColor] ; BL = video page #
    mov     ah,03h
    int     10h ; get cursor position
    push    dx
    mov     ah,02h ; save it
    mov     dx,[bLastCol] ; set cursor position
    mov     byte [bDisableUI],1 ; set correct row/col
    test    byte [bDisableUI],1
    jz      short show_onoff ; just show on/off
    mov     dl,0
    int     10h
    mov     ax,0A20h ; write multiple spaces
    mov     cx,80 ; 80 of them, to be exact
    ; 10/09/2023
    ;int     10h ; to obliterate the status line
    jmp     short show_done ;
show_onoff:
    int     10h
    ; - VIDEO - WRITE CHARACTERS ONLY AT CURSOR POSITION
    ; AL = character, BH = display page - alpha mode
    ; BL = color of character (graphics mode, PCjr only)
    ; CX = number of times to write character
    mov     al,[_$NO] ; assume OFF
    cmp     byte [bQueryOpt],1 ; is interactive mode on?
    jne     short show_noton ; no
    mov     al,[_$YES] ; yes
show_noton:
    mov     ah,0Eh ; write TTY
show_done: ; 10/09/2023
    int     10h
;show_done:
    pop     dx
    mov     ah,02h
    int     10h ; restore original cursor position
    pop     bx
    retn

; 16/04/2019 - Retro DOS v4.0

-----
;
; skip_token: advances ES:SI/CX past the current token
;

```

```

42876 ;
42877 ; INPUT
42878 ; ES:SI -> position in config.sys
42879 ; CX == remaining bytes in config.sys
42880 ;
42881 ; OUTPUT
42882 ; CF set if EOL/EOF hit
42883 ; AL == 1st char of delimiter
42884 ; ES:SI -> just past the delimiter
42885 ; CX == # bytes remaining from that point
42886 ;
42887 ; OTHER REGS USED
42888 ; AX
42889 ;
42890 ; NOTES
42891 ; None
42892 ;
42893 ; HISTORY
42894 ; Created 16-Mar-1992 by JeffPar
42895 ;
42896 ;-----
42897
42898 skip_token:
42899 000045BC E84D00 call get_char
42900 000045BF 7210 jc short skip_token_done
42901 000045C1 E8B401 call any_delim
42902 000045C4 75F6 jne short skip_token
42903 skip_check_eol:
42904 000045C6 3C0D cmp al,cr ; 0Dh
42905 000045C8 7406 je short skip_token_eol
42906 000045CA 3C0A cmp al,lf ; 0Ah
42907 000045CC 7402 je short skip_token_eol
42908 000045CE F8 clc
42909 ; jmp short skip_token_done
42910 000045CF C3 retn
42911 skip_token_eol:
42912 000045D0 F9 stc
42913 skip_token_done:
42914 000045D1 C3 retn
42915 ;
42916 ;-----
42917 ;
42918 ; skip_delim: advances ES:SI/CX past the current delimiter
42919 ;
42920 ; INPUT
42921 ; ES:SI -> position in config.sys
42922 ; CX == remaining bytes in config.sys
42923 ;
42924 ; OUTPUT
42925 ; CF set if EOF hit
42926 ; AL == 1st char of token
42927 ; ES:SI -> just past the token
42928 ; CX == # bytes remaining from that point
42929 ; ES:BX -> new token (since ES:SI is already pointing 1 byte past token)
42930 ;
42931 ; OTHER REGS USED
42932 ; AX
42933 ;
42934 ; NOTES
42935 ; None
42936 ;
42937 ; HISTORY
42938 ; Created 16-Mar-1992 by JeffPar
42939 ;
42940 ;-----
42941
42942 skip_delim: ; returns carry set if eol/eof
42943 000045D2 E83700 call get_char ;
42944 000045D5 8D5CFF lea bx,[si-1] ; also returns BX -> next token
42945 000045D8 72F7 jc short skip_token_done ;
42946 000045DA E8AB01 call delim ;
42947 000045DD 74F3 je short skip_delim ;
42948 000045DF EBE5 jmp short skip_check_eol ; 13/05/2019
42949 ;
42950 ;-----
42951 ;
42952 ; skip_opt_line: same as skip_line provided AL != LF
42953 ;
42954 ; INPUT
42955 ; AL == last character read
42956 ; ES:SI -> position in config.sys
42957 ; CX == remaining bytes in config.sys
42958 ;
42959 ; OUTPUT
42960 ; CF set if EOF hit
42961 ; AL == 1st char of new line
42962 ; ES:SI -> just past 1st char of new line
42963 ; CX == # bytes remaining from that point
42964 ;
42965 ; OTHER REGS USED
42966 ; AX
42967 ;
42968 ; NOTES
42969 ; In other words, the purpose here is to skip to the next line,
42970 ; unless ES:SI is already sitting at the front of the next line (which
42971 ; it would be if the last character fetched -- AL -- was a linefeed)
42972 ;
42973 ; HISTORY
42974 ; Created 16-Mar-1992 by JeffPar
42975 ;
42976 ;-----
42977
42978 ; 03/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
42979 ; skip_opt_line:
42980 ; cmp al,lf ; 0Ah
42981 ; je short skip_line_done
42982 ;
42983 ; fall into skip_line
42984 ;
42985 ;-----
42986 ;
42987 ; skip_line: skip to the next line
42988 ;
42989 ; INPUT
42990 ; ES:SI -> position in config.sys
42991 ; CX == remaining bytes in config.sys
42992 ;
42993 ; OUTPUT
42994 ; CF set if EOF hit
42995 ; ES:SI -> just past 1st char of new line
42996 ; CX == # bytes remaining from that point
42997 ;
42998 ; OTHER REGS USED
42999 ; AX

```

```

43000 ;
43001 ; NOTES
43002 ; None
43003 ;
43004 ; HISTORY
43005 ; Created 16-Mar-1992 by JeffPar
43006 ;
43007 ;-----
43008 ;
43009 skip_line:
43010 call get_char
43011 jc short skip_line_done
43012 skip_opt_line: ; 03/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
43013 cmp al,1f ; 0Ah
43014 jne short skip_line
43015 skip_line_done:
43016 num_done: ; 18/12/2022
43017 retn
43018 ;
43019 ;-----
43020 ;
43021 ; get_number: return binary equivalent of numeric string
43022 ;
43023 ; INPUT
43024 ; ES:SI -> position in config.sys
43025 ; CX == remaining bytes in config.sys
43026 ;
43027 ; OUTPUT
43028 ; AL == non-digit encountered
43029 ; BX == binary #
43030 ; ES:SI -> just past 1st non-digit
43031 ; CX == # bytes remaining from that point
43032 ;
43033 ; OTHER REGS USED
43034 ; AX
43035 ;
43036 ; NOTES
43037 ; None
43038 ;
43039 ; HISTORY
43040 ; Created 16-Mar-1992 by JeffPar
43041 ;
43042 ;-----
43043 ; 13/05/2019
43044 ;
43045 ;
43046 get_number:
43047 sub bx,bx ; BX = result
43048 num_loop:
43049 call get_char ;
43050 jc short num_done ;
43051 cmp al,'0' ; convert to value
43052 jb short num_done ; no more number
43053 cmp al,'9' ;
43054 ja short num_done ;
43055 push ax ;
43056 mov ax,10 ;
43057 push dx ;
43058 mul bx ;
43059 pop dx ;
43060 mov bx,ax ;
43061 pop ax ;
43062 sub al,'0' ;
43063 cbw ;
43064 add bx,ax ;
43065 jmp short num_loop ;
43066 ;
43067 ; 18/12/2022
43068 ;num_done:
43069 ;retn
43070 ;
43071 ;-----
43072 ;
43073 ; get_char: return next character, advance ES:SI, and decrement CX
43074 ;
43075 ; INPUT
43076 ; ES:SI -> position in config.sys
43077 ; CX == remaining bytes in config.sys
43078 ;
43079 ; OUTPUT
43080 ; AL == next character
43081 ; ES:SI -> just past next character
43082 ; CX == # bytes remaining from that point
43083 ;
43084 ; OTHER REGS USED
43085 ; AX
43086 ;
43087 ; NOTES
43088 ; None
43089 ;
43090 ; HISTORY
43091 ; Created 16-Mar-1992 by JeffPar
43092 ;
43093 ;-----
43094 ;
43095 get_char:
43096 sub cx,1 ; use SUB to set carry,zero
43097 jb short get_fail ; out of data
43098 ;lods byte ptr es:[si] ;
43099 es
43100 lodsb
43101 mov ah,al ;
43102 retn ;
43103 get_fail:
43104 mov cx,0 ; restore CX to zero
43105 nearby_ret: ; leave carry set, zero not set
43106 retn
43107 ;
43108 ;-----
43109 ;
43110 ; query_user: ask user whether to execute current config.sys command
43111 ;
43112 ; INPUT
43113 ; AL == current command code
43114 ; ES:SI -> current command line in config.sys
43115 ; config_cmd == current command code, but with QUERY bit intact
43116 ; (00h used to generate "Process AUTOEXEC.BAT" prompt)
43117 ;
43118 ; OUTPUT
43119 ; CF set if command should be ignored (it is also REM'ed out)
43120 ;
43121 ; OTHER REGS USED
43122 ; BX, CX, DX, DI
43123 ;

```



```

43124 ; NOTES
43125 ; None
43126 ;
43127 ; HISTORY
43128 ; Created 16-Mar-1992 by JeffPar
43129 ;
43130 ;-----
43131 ;
43132 ; 31/12/2022 - Retro UNIX 386 v4.2 (Modified MSDOS 6.21 IO.SYS)
43133 ; (SYSINIT:4AE5h)
43134 ;
43135 ; 12/12/2022
43136 query_user:
43137     test     byte [bQueryOpt],4      ; answer no to everything?
43138     ; 01/01/2023
43139     jz      short qu_1              ;
43140     jmp     skip_all                ;
43141     ; 12/12/2022
43142     ;;jmp short skip_all            ;
43143     ;jnz short skip_all            ;
43144 qu_1:
43145     test     byte [bQueryOpt],2      ; answer yes to everything?
43146     jnz short nearby_ret           ; yes (and return carry clear!)
43147     push     ax                     ;
43148     mov      al,[config_cmd]         ;
43149     test     byte [bQueryOpt],1      ; query every command?
43150     jnz short query_all             ; yes
43151     test     al,CONFIG_OPTION_QUERY ;
43152     ; 01/01/2023
43153     jnz short query_all             ;
43154     ; 12/12/2022
43155     ;;jmp short do_cmd              ;
43156     ;jz      short do_cmd ; cf=0    ;
43157     ;
43158     ; 01/01/2023
43159     pop      ax
43160     retn
43161
43162 query_all:
43163 ; Search for the command code (AL) in "comtab", and then print
43164 ; out the corresponding keyword, followed by the rest of the actual
43165 ; line pointed to by ES:SI
43166
43167     push     si                     ; save pointer to rest of CONFIG.SYS line
43168     mov      dx,_$AutoPrmpt         ;
43169     and      al,~CONFIG_OPTION_QUERY ; 7Fh
43170     jz      short generic_prompt     ; config_cmd must have been 0
43171
43172     mov      dh,al                  ; save config_cmd in DH
43173     sub      bx,bx                  ;
43174     mov      di,comtab              ;
43175
43176 find_match:
43177     mov      bl,[di]                ; get size of current keyword
43178     or       bl,bl                  ;
43179     jz      short line_print         ; end of table
43180     inc      di                     ;
43181     cmp      al,[di+bx]              ; match?
43182     je      short cmd_match          ; yes
43183     lea      di,[di+bx+1]            ; otherwise, skip this command code
43184     ; 13/05/2019
43185     jmp short find_match            ; loop
43186 cmd_match:
43187     mov      cl,[di-1]              ;
43188     mov      ch,0                    ;
43189     mov      ah,STD_CON_OUTPUT ; 2
43190
43191 cmd_print:
43192     mov      al,[di]                ;
43193     inc      di                     ;
43194     mov      dl,al                  ;
43195     int      21h                    ;
43196     loop     cmd_print              ;
43197     mov      dl,'='                 ;
43198     cmp      dh,CONFIG_SET ; 'V'    ; for SET commands, don't display a '='
43199     jne short cmd_notset            ;
43200     mov      dl,''                  ;
43201 cmd_notset:
43202     int      21h                    ; '=' looks funny on SET commands
43203
43204 line_print:
43205     ;lods byte ptr es:[si]          ;
43206     es
43207     lodsb                             ;
43208     or       al,al                  ;
43209     jnz short non_null              ;
43210     mov      al,' '                 ;
43211 non_null:
43212     cmp      al,' '                 ; control code?
43213     jb      short prompt_user        ; yes, assume end of line
43214     jne short non_space              ;
43215     ; 10/09/2023
43216     cmp      [es:si],al ; 20h        ;
43217     ;cmp     byte [es:si],''         ;
43218     jb      short prompt_user        ;
43219 non_space:
43220     mov      dl,al                  ;
43221     mov      ah,STD_CON_OUTPUT ; 2    ;
43222     int      21h                    ;
43223     jmp short line_print            ;
43224
43225 prompt_user:
43226     mov      dx,_$InterPrmpt        ;
43227
43228 generic_prompt:
43229     call     print                   ;
43230 input_loop:
43231     mov      ah,0                    ; read a key
43232     int      16h                    ;
43233     or       al,al                  ; is it a function key?
43234     jnz short not_func              ; no
43235     cmp      ah,3Fh                  ; F5 function key?
43236     jne short input_loop            ; no
43237     mov      al,[_$NO]               ;
43238     or       byte [bQueryOpt],4      ; no more queries
43239     jmp      short legal_char        ;
43240 not_func:
43241     and      al,~20h ; 0DFh          ; converting to upper case
43242     cmp      al,[_$NO]               ; verify character is legal
43243     je      short legal_char         ;
43244     cmp      al,[_$YES]              ;
43245     je      short legal_char         ;
43246     cmp      byte [config_cmd],0     ;
43247     je      short input_loop         ; don't allow Esc on this query
43248     cmp      al,1Bh                  ; Esc?
43249     jne short input_loop            ;

```

```

43248 000046C7 800E[834C]02          or     byte [bQueryOpt],2      ; no more interactive boot prompts
43249 000046CC A0[1D53]          mov     al,[_$YES]
43250                                legal_char:
43251 000046CF E888FE          call    disp_input            ;
43252 000046D2 5E              pop     si              ; restore pointer to rest of CONFIG.SYS line
43253                                ;
43254 000046D3 3A06[2153]        cmp     al,[_$NO]            ; process line?
43255 000046D7 7403          je      short skip_cmd      ; no
43256                                ; 12/12/2022
43257 000046D9 F8              cllc
43258                                do_cmd:
43259 000046DA 58              pop     ax              ;
43260                                ; 12/12/2022
43261                                ; cf=0
43262                                ;clic
43263 000046DB C3              retn              ; just do the command
43264
43265                                skip_cmd:
43266 000046DC 58              pop     ax              ;
43267                                skip_all:
43268 000046DD B430          mov     ah,CONFIG_REM ; '0' ; fake out the rest of sysinit's processing
43269 000046DF F9              stc
43270 000046E0 C3              retn
43271
43272                                ;-----
43273                                ;
43274                                ; print_error: displays multi-config error conditions
43275                                ;
43276                                ; INPUT
43277                                ; Carry set to pause, clear to not
43278                                ; ES:SI -> current command line in config.sys
43279                                ;
43280                                ; OUTPUT
43281                                ; None
43282                                ;
43283                                ; OTHER REGS USED
43284                                ; None
43285                                ;
43286                                ; NOTES
43287                                ; None
43288                                ;
43289                                ; HISTORY
43290                                ; Created 16-Mar-1992 by JeffPar
43291                                ;-----
43292
43293                                print_error:
43294                                push     ax
43295 000046E1 50              push     bx
43296 000046E2 53              push     cx
43297 000046E3 51              push     dx
43298 000046E4 52              push     ds
43299 000046E5 1E              push     cs
43300 000046E6 0E              pop      ds
43301 000046E7 1F              pushf
43302 000046E8 9C              call     get_linenum
43303 000046E9 E820FC        mov     [linecount],bx
43304 000046EC 891E[AF02]    call     error_line
43305 000046F0 E8C5E7        popf
43306 000046F3 9D              jnc short pe_ret
43307 000046F4 7319          mov     dx,_$PauseMsg
43308 000046F6 BA[DB51]      call     print
43309 000046F9 E85603        mov     ax,0C07h          ; flush input buffer, then wait for key
43310 000046FC B8070C          int     21h              ; wait for a key
43311 000046FF CD21          or      al,al            ; extended key?
43312 00004701 08C0          jnz short pe_1           ; no
43313 00004703 7504          mov     ah,07h          ; yes
43314 00004705 B407          int     21h              ; eat it too
43315 00004707 CD21
43316
43317 00004709 BA[6E50]      mov     dx,crlfm
43318 0000470C E84303      call    print
43319
43320 0000470F 1F              pop     ds
43321 00004710 5A              pop     dx
43322 00004711 59              pop     cx
43323 00004712 5B              pop     bx
43324 00004713 58              pop     ax
43325 00004714 C3              retn
43326
43327                                ;-----
43328                                ;
43329                                ; This function is very simple: it merely prepends a "/D" to the
43330                                ; command-line for the shell; this (undocumented) switch disables
43331                                ; AUTOEXEC.BAT processing and the date/time prompt that is usually
43332                                ; displayed when there's no AUTOEXEC.BAT.
43333                                ;
43334                                ;
43335                                ; disable_autoexec:
43336                                ; MSDOS 6.21 IO.SYS - SYSINIT:4BE2h
43337                                ; 17/04/2019 - Retro DOS v4.0
43338 00004715 F606[834C]04    test     byte [bQueryOpt],4
43339 0000471A 7443          jz      short disable_exit
43340 0000471C F606[794C]01    test     byte [dae_flag],1
43341 00004721 753C          jnz     short disable_exit
43342 00004723 800E[794C]01    or      byte [dae_flag],1
43343                                ;or byte [bQueryOpt],2 ; MSDOS 6.0
43344 00004728 810E[834C]0201  or      word [bQueryOpt],102h ; [bDefBlock] = 1
43345 0000472E BA4420        mov     dx,'D ' ; 2044h
43346                                dae_1:
43347                                ; 03/08/2023 - Retro DOS v4.2 IO.SYS (optimization)
43348 00004731 A0[B84B]      mov     al,[def_swchr]
43349                                ;mov al,[command_line-1] ; get default switchchar
43350 00004734 08C0          or      al,al            ; anything there?
43351 00004736 7427          jz      short disable_exit ; no, disable_autoexec already called
43352 00004738 8A1E[B94B]    mov     bl,[command_line]
43353 0000473C B700          mov     bh,0            ; BX == command-line length
43354 0000473E 89D9          mov     cx,bx
43355 00004740 80C303        add     bl,3
43356 00004743 80FB7E        cmp     bl,126
43357 00004746 7717          ja      short disable_exit ;
43358 00004748 881E[B94B]    mov     [command_line],bl ; update length
43359 0000474C 81C3[BA4B]    add     bx,command_line+1 ; make sure we move the NULL too
43360 00004750 41              inc     cx              ; (just for consistency sake)
43361                                ;
43362 00004751 8A67FD        mov     ah,[bx-3]
43363 00004754 8827          mov     [bx],ah
43364 00004756 4B              dec     bx
43365 00004757 E2F8          loop   disable_loop
43366 00004759 8847FE        mov     [bx-2],al
43367                                ;mov word [bx-1],'D ' ; 2044h ; /D is stuffed into place now
43368 0000475C 8957FF        mov     [bx-1],dx ; MSDOS 6.21 IO.SYS - SYSINIT:4C29h
43369                                ;mov byte [command_line-1],0 ;
43370                                disable_exit:
43371 0000475F C3              retn

```



```

43372
43373
43374 00004760 803E[834C]01      CheckQueryOpt:      ; MSDOS 6.21 IO.YSYS - SYSINIT:4C2Dh
43375 00004765 75F8                cmp     byte [bQueryOpt],1
43376 00004767 F606[794C]02      jnz     short disable_exit
43377 0000476C 75F1                test    byte [dae_flag],2
43378 0000476E 800E[794C]02      jnz     short disable_exit
43379                                or      byte [dae_flag],2
43380                                ;mov     dx,'Y' ; (MASM syntax) ; 2059h
43381 00004773 BA5920                ; 10/09/2023 (BugFix)
43382 00004776 EBB9                mov     dx,'Y' ; (NASM syntax) ; 2059h
43383                                jmp     short dae_1
43384
43385                                ;endif ;MULTI_CONFIG
43386                                ;%endif      ; 02/11/2022
43387
43388
43389                                ; 19/04/2019 - Retro DOS v4.0
43390
43391                                ;-----
43392                                ;
43393                                ; procedure : delim
43394                                ;
43395                                ;-----
43396
43397                                ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
43398                                ; (SYSINIT:4C45h)
43399
43400                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
43401                                ;%if 0
43402                                ;;ifdef      MULTI_CONFIG
43403                                ;
43404                                any_delim:
43405 00004778 3C0D                cmp     al,cr
43406 0000477A 7427                je      short delim_ret
43407 0000477C 3C0A                cmp     al,lf
43408 0000477E 7423                je      short delim_ret
43409 00004780 3C5B                cmp     al,[' '
43410 00004782 741F                je      short delim_ret
43411 00004784 3C5D                cmp     al,']'
43412 00004786 741B                je      short delim_ret
43413                                ;
43414                                ;;endif ;MULTI_CONFIG
43415                                ;%endif ; 02/11/2022
43416
43417                                ; 02/11/2022 - Retrodos v4.0 (Modified MSDOS 5.0 IO.SYS)
43418                                ; (SYSINIT:3450h)
43419                                delim:
43420 00004788 3C2F                cmp     al,'/' ; ibm will assume "/" as an delimiter.
43421 0000478A 7417                je      short delim_ret
43422
43423 0000478C 3C00                cmp     al,0 ; special case for sysinit!!!
43424 0000478E 7413                je      short delim_ret
43425
43426                                org_delim: ; used by organize routine except for getting
43427 00004790 3C20                cmp     al,' ' ; the filename.
43428 00004792 740F                je      short delim_ret
43429 00004794 3C09                cmp     al,tab ; 9
43430 00004796 740B                je      short delim_ret
43431 00004798 3C3D                cmp     al,'='
43432 0000479A 7407                je      short delim_ret
43433 0000479C 3C2C                cmp     al,', '
43434 0000479E 7403                je      short delim_ret
43435 000047A0 3C3B                cmp     al,';'
43436
43437                                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
43438
43439                                ; 04/01/2023 - Retro DOS v4.2
43440                                ;ifdef      MULTI_CONFIG
43441                                ; Make sure there's no chance of a false EOF indication
43442 000047A2 F8                clc
43443                                ;endif
43444                                ; 02/11/2022
43445                                delim_ret:
43446                                ; 04/01/2023
43447                                ; cf = 0
43448                                nl_ret: ; 10/09/2023
43449 000047A3 C3                retn
43450
43451                                ;-----
43452                                ;
43453                                ; procedure : newline
43454                                ;
43455                                ; newline returns with first character of next line
43456                                ;
43457                                ;-----
43458
43459                                newline:
43460                                call     getchr ;skip non-control characters
43461 000047A7 72FA                jc      short nl_ret
43462 000047A9 3C0A                cmp     al,lf ;look for line feed
43463 000047AB 75F7                jne     short newline
43464
43465                                ; 10/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
43466                                ;call     getchr
43467                                ;nl_ret:
43468                                ;retn
43469                                ; 10/09/2023
43470                                ;jmp     short getchr
43471
43472                                ; 10/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
43473                                %if 1
43474
43475                                ;-----
43476                                ;
43477                                ; procedure : getchr
43478                                ;
43479                                ;-----
43480
43481                                ; 24/10/2022
43482                                getchr:
43483                                ; 12/12/2022
43484                                ;push     cx
43485                                ;mov     cx,[count]
43486                                ;jcxz     nochar
43487                                ; 12/12/2022
43488 000047AD 833E[5603]01      cmp     word [count],1
43489 000047B2 720F                jb      short nochar ; cf=1 ([count] = 0)
43490
43491 000047B4 8B36[5A03]      mov     si,[chrptr]
43492 000047B8 268A04      mov     al,[es:si]
43493 000047BB FF0E[5603]      dec     word [count]
43494 000047BF FF06[5A03]      inc     word [chrptr]
43495                                ; 12/12/2022

```

```

43496         ; cf=0
43497         ;clc
43498 ;get_ret:
43499         ;pop     cx
43500         ;retn
43501 nochar:
43502         ; 12/12/2022
43503         ; cf=1
43504         ;stc
43505         ;jmp     short get_ret
43506
43507 000047C3 C3      retn
43508 %endif
43509
43510 ;-----
43511 ;
43512 ; procedure : mapcase
43513 ;
43514 ;-----
43515
43516         ; 02/11/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 IO.SYS)
43517
43518         ; 04/01/2023 - Retro DOS 4.2 (Modified MSDOS 6.21 IO.SYS)
43519         ; (SYSINIT:4C7Eh)
43520 mapcase:
43521 000047C4 51      push     cx
43522 000047C5 56      push     si
43523 000047C6 1E      push     ds
43524
43525         push     es
43526 000047C8 1F      pop      ds
43527
43528         ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
43529
43530 ; 04/01/2023 - Retro DOS 4.2
43531
43532 ;ifdef     MULTI_CONFIG
43533 000047C9 88C3    mov     bl,al          ; same cmd code this line
43534
43535 ;else
43536 ; xor     si,si
43537 ;endif
43538         ; 02/11/2022
43539         ; 04/01/2023 - Retro DOS 4.2
43540         ;xor     si, si
43541
43542 000047CB AC      convloop:
43543 000047CC 3C61    lodsb
43544 000047CE 7209    cmp     al,'a'
43545 000047D0 3C7A    jnb     short noconv
43546 000047D2 7705    cmp     al,'z'
43547 000047D4 2C20    ja      short noconv
43548 000047D6 8844FF  sub     al,20h
43549         mov     [si-1],al
43550 noconv:
43551         ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
43552
43553 ; 04/01/2023 - Retro DOS 4.2
43554 ;ifdef     MULTI_CONFIG
43555
43556 ; When MULTI_CONFIG enabled, "mapcase" is used to map everything to
43557 ; upper-case a line at a time, after we've been able to figure out whether
43558 ; the line is a SET command or not (since we don't want to upper-case
43559 ; anything after the "=" in a SET)
43560 ;
43561 000047D9 80FB56  cmp     bl,CONFIG_SET ; 'V' ; preserve case for part of the line?
43562 000047DC 7504    jne     short check_eol ; no, just check for end-of-line
43563 000047DE 3C3D    cmp     al,'=' ; separator between SET var and value?
43564 000047E0 740A    je      short convdone ; yes
43565
43566 000047E2 3C0D    check_eol:
43567 000047E4 7406    cmp     al,cr
43568 000047E6 3C0A    je      short convdone
43569 000047E8 7402    cmp     al,lf
43570         je      short convdone
43571 ;endif
43572 000047EA E2DF    ; 02/11/2022
43573         loop convloop
43574 000047EC 1F      convdone:
43575 000047ED 5E      pop     ds
43576 000047EE 59      pop     si
43577 000047EF C3      pop     cx
43578         retn
43579
43580 ;-----
43581 ;
43582 ; procedure : round
43583 ;
43584 ; round the values in memlo and memhi to paragraph boundary.
43585 ; perform bounds check.
43586 ;
43587 ;-----
43588
43589 round:
43590         ; 10/09/2023 - Retro DOS v4.2 IO.SYS (Optimization)
43591 000047F0 1E      push     ds
43592 000047F1 0E      push     cs
43593 000047F2 1F      pop      ds
43594
43595 000047F3 50      push     ax
43596 000047F4 A1[6203] ;mov     ax,[cs:memlo]
43597         mov     ax,[memlo]
43598 000047F7 E81BCB  call    ParaRound          ; para round up
43599
43600         ;add     [cs:memhi],ax
43601 000047FA 0106[6403] ;add     [memhi],ax
43602         ;mov     word [cs:memlo],0
43603 000047FE C706[6203]0000 ;mov     word [memlo],0
43604         ;mov     ax,[cs:memhi] ; ax = new memhi
43605 00004804 A1[6403] ;mov     ax,[memhi]
43606         ;cmp     ax,[cs:ALLOCLIM] ; if new memhi >= alloclim, error
43607 00004807 3B06[A502] ;cmp     ax,[ALLOCLIM]
43608         ;jae     short mem_err
43609         ; 13/04/2024
43610 0000480B 7322    ;jae     short mem_err2 ; ds = cs
43611         ;test    byte [cs:setdevmarkflag],for_devmark ; 2
43612 0000480D F606[6919]02 ;test    byte [setdevmarkflag],for_devmark ; 2
43613 00004812 7416    ;jz      short skip_set_devmarksize
43614 00004814 06      push     es
43615 00004815 56      push     si
43616         ;mov     si,[cs:devmark_addr]
43617 00004816 8B36[6719] ;mov     si,[devmark_addr]
43618 0000481A 8EC6    mov     es,si
43619 0000481C 29F0    sub     ax,si

```

```

43620 0000481E 48      dec     ax
43621                ;mov     [es:3],ax
43622 0000481F 26A30300 mov     [es:devmark.size],ax ; paragraph
43623                ;and     byte [cs:setdevmarkflag],not_for_devmark ; 0FDh
43624 00004823 8026[6919]FD and     byte [setdevmarkflag],not_for_devmark ; 0FDh
43625 00004828 5E      pop     si
43626 00004829 07      pop     es
43627                skip_set_devmarksize:
43628 0000482A 58      pop     ax
43629
43630                ; 10/09/2023
43631 0000482B 1F      pop     ds
43632
43633                ; 11/12/2022
43634                ; cf = 0
43635                ; 02/11/2022
43636                ;clc     ; ? (not needed here) ; clear carry
43637 0000482C C3      retn
43638
43639                ;-----
43640
43641                mem_err:
43642                ; 11/12/2022
43643 0000482D 0E      push    cs
43644 0000482E 1F      pop     ds
43645                mem_err2:
43646 0000482F BA[4751] mov     dx,badmem
43647                ;push    cs
43648                ;pop     ds
43649 00004832 E81D02 call    print
43650 00004835 E916CB jmp     stall
43651
43652                ;-----
43653                ;
43654                ; procedure : calldev
43655                ;
43656                ;-----
43657
43658                ; 02/11/2022 - Retrodos v4.0 (Modified MSDOS 5.0 IO.SYS)
43659                ; (SYSINIT:34E0h)
43660
43661                ; 13/04/2024 - Retrodos v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
43662                ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4F3Eh)
43663
43664                calldev:
43665 00004838 2E8E1E[3924] mov     ds,[cs:DevEntry+2]
43666 0000483D 2E031E[3724] add     bx,[cs:DevEntry] ; do a little relocation
43667 00004842 8B07      mov     ax,[bx]
43668
43669 00004844 2EFF36[3724] push    word [cs:DevEntry]
43670 00004849 2EA3[3724] mov     [cs:DevEntry],ax
43671 0000484D BB[6F03] mov     bx,packet
43672 00004850 2EFF1E[3724] call    far [cs:DevEntry]
43673 00004855 2E8F06[3724] pop     word [cs:DevEntry]
43674 0000485A C3      retn
43675
43676                ;-----
43677                ;
43678                ; procedure : todigit
43679                ;
43680                ;-----
43681
43682                todigit:
43683 0000485B 2C30      sub     al,'0'
43684                ;jb     short notdig ; 02/11/2022
43685                ; 12/12/2022
43686 0000485D 7203      jb     short notdig2
43687                ;cmp    al,9
43688                ;ja     short notdig
43689                ;clc
43690                ;retn
43691                ; 12/12/2022
43692 0000485F 3C0A      cmp     al,10
43693 00004861 F5      cmc
43694                notdig:
43695                ;stc
43696                notdig2:
43697 00004862 C3      retn
43698
43699                ;-----
43700                ;
43701                ; procedure : getnum
43702                ;
43703                ; getnum parses a decimal number.
43704                ; returns it in ax, sets zero flag if ax = 0 (may be considered an
43705                ; error), if number is bad carry is set, zero is set, ax=0.
43706                ;-----
43707
43708                getnum:
43709                push    bx
43710 00004863 53      xor     bx,bx ; running count is zero
43711 00004864 31DB      b2:
43712                call    todigit ; do we have a digit ?
43713 00004866 E8F2FF      jc     short badnum ; no, bomb
43714 00004869 7247      xchg    ax,bx ; put total in ax
43715                push    bx ; save digit (0 to 9)
43716 0000486B 93      ;mov    bx,10 ; base of arithmetic
43717 0000486C 53      ; 12/12/2022
43718                mov     bl,10
43719                mul     bx ; shift by one decimal digit
43720 0000486D B30A      pop     bx ; get back digit (0 to 9)
43721 0000486F F7E3      add     al,bl ; get total
43722 00004871 5B      adc     ah,0 ; make that 16 bits
43723 00004872 00D8      jc     short badnum ; too big a number
43724 00004874 80D400      xchg    ax,bx ; stash total
43725 00004877 7239
43726                call    getchr ;get next digit
43727 00004879 93      jc     short b1 ; no more characters
43728                cmp     al,' ' ; space?
43729 0000487A E830FF      je     short b15 ; then end of digits
43730 0000487D 722D      cmp     al',' ; ',' is a seperator!!!
43731 0000487F 3C20      je     short b15 ; then end of digits.
43732 00004881 741F      cmp     al,' ' ; tab
43733 00004883 3C2C      je     short b15 ; tab
43734 00004885 741B      cmp     al,tab ; 9
43735 00004887 3C09      je     short b15 ; allow 0 or special separators
43736 00004889 7417      cmp     al,[cs:sepchr]
43737 0000488B 2E3A06[AE02] je     short b15 ; see if another switch follows
43738 00004890 7410      cmp     al,'/'
43739 00004892 3C2F      ; 12/12/2022
43740                ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
43741                ;nop
43742                ;cas - remnant of old bad code
43743                ;nop

```

```

43744 00004894 740C      je      short b15
43745 00004896 3C0A      cmp     al,lf          ; line-feed?
43746 00004898 7408      je      short b15
43747 0000489A 3C0D      cmp     al,cr          ; carriage return?
43748 0000489C 7404      je      short b15
43749 0000489E 08C0      or      al,al          ; end of line separator?
43750 000048A0 75C4      jnz     short b2       ; no, try as a valid char...
43751
43752 000048A2 2EFF06[5603] b15:      inc     word [cs:count] ; one more character to s...
43753 000048A7 2EFF0E[5A03]      dec     word [cs:chrptr] ; back up over separator
43754
43755 000048AC 89D8      b1:      mov     ax,bx          ; get proper count
43756 000048AE 09C0      or      ax,ax          ; clears carry, sets zero accordingly
43757 000048B0 5B      pop     bx
43758 000048B1 C3      retn
43759
43760      badnum:
43761      ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
43762 000048B2 31C0      ;mov     byte [cs:sepchr],0
43763      xor     ax,ax          ; set zero flag, and ax = 0
43764 000048B4 2EA2[AE02] ; 12 /12/2022
43765 000048B8 5B      mov     [cs:sepchr],al ; 0
43766 000048B9 F9      pop     bx
43767 000048BA C3      stc
43768      retn          ; and carry set
43769
43770      ;*****
43771
43772      setdoscountryinfo:
43773
43774      ;-----
43775      ;input: es:di -> pointer to dos_country_cdpd_info
43776      ; ds:0 -> buffer.
43777      ; si = 0
43778      ; ax = country id
43779      ; dx = code page id. (if 0, then use ccscodespage as a default.)
43780      ; bx = file handle
43781      ; this routine can handle maximum 438 country_data entries.
43782
43783      ;output: dos_country_cdpd_info set.
43784      ; carry set if any file read failure or wrong information in the file.
43785      ; carry set and cx = -1 if cannot find the matching country_id,
43786      ; codepage_id in the file.
43787      ;-----
43788
43789      ; 04/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
43790      ; (SYSINIT:4D83h)
43791
43792      ; 10/09/2023 - RetroDOS v4.2 IO.SYS (Optimization)
43793      ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:4FCAh)
43794 000048BB 57      push    di
43795 000048BC 50      push    ax
43796 000048BD 52      push    dx
43797
43798 000048BE 31C9      xor     cx,cx
43799 000048C0 31D2      xor     dx,dx
43800 000048C2 B80002      mov     ax,512          ;read 512 bytes
43801 000048C5 E84301      call    readincontrolbuffer ;read the file header
43802 000048C8 724A      jc      short setdosdata_fail
43803
43804 000048CA 06      push    es
43805 000048CB 56      push    si
43806
43807 000048CC 0E      push    cs
43808 000048CD 07      pop     es
43809
43810 000048CE BF1E4B[ ] mov     di,country_file_signature ; db 0FFh,'COUNTRY'
43811 000048D1 B90800      mov     cx,8             ;length of the signature
43812 000048D4 F3A6      repz    cmpsb
43813
43814 000048D6 5E      pop     si
43815 000048D7 07      pop     es
43816 000048D8 753A      jnz     short setdosdata_fail ;signature mismatch
43817
43818 000048DA 83C612      add     si,18             ;si -> county info type
43819 000048DD 803C01      cmp     byte [si],1       ;only accept type 1 (currently only 1 header type)
43820 000048E0 7532      jne     short setdosdata_fail ;cannot proceed. error return
43821
43822 000048E2 46      inc     si                 ;si -> file offset
43823 000048E3 8B14      mov     dx,[si]           ;get the info file offset.
43824 000048E5 8B4C02      mov     cx,[si+2]
43825 000048E8 B80018      mov     ax,6144          ;read 6144 bytes.
43826 000048EB E81D01      call    readincontrolbuffer ;read info
43827 000048EE 7224      jc      short setdosdata_fail
43828
43829 000048F0 8B0C      mov     cx,[si]           ;get the # of country, codepage combination entries
43830 000048F2 81F9B601  cmp     cx,438           ;cannot handle more than 438 entries.
43831 000048F6 771C      ja      short setdosdata_fail
43832
43833 000048F8 46      inc     si
43834 000048F9 46      inc     si                 ;si -> entry information packet
43835 000048FA 5A      pop     dx                 ;restore code page id
43836 000048FB 58      pop     ax                 ;restore country id
43837 000048FC 5F      pop     di
43838
43839      setdoscntry_find:      ;search for desired country_id,codepage_id.
43840 000048FD 3B4402      cmp     ax,[si+2]         ;compare country_id
43841 00004900 7509      jne     short setdoscntry_next
43842
43843      ;cmp     dx,0          ;no user specified code page ?
43844      ;je      short setdoscntry_any_codepage ;then no need to match code page id.
43845      ; 10/09/2023
43846 00004902 09D2      or      dx,dx ; cmp dx,0
43847 00004904 7413      jz      short setdoscntry_any_codepage
43848 00004906 3B5404      cmp     dx,[si+4]         ;compare code page id
43849 00004909 7411      je      short setdoscntry_got_it
43850
43851      setdoscntry_next:
43852 0000490B 0334      add     si,[si]           ;next entry
43853 0000490D 46      inc     si
43854 0000490E 46      inc     si                 ;take a word for size of entry itself
43855 0000490F E2EC      loop    setdoscntry_find
43856
43857      ;mov     cx,-1          ;signals that bad country id entered.
43858      ; 10/09/2023
43859 00004911 49      dec     cx ; 0 -> -1
43860
43861      setdoscntry_fail:
43862 00004913 F9      stc
43863      retn
43864
43865      setdosdata_fail:
43866 00004914 5E      pop     si
43867 00004915 59      pop     cx
43868 00004916 5F      pop     di

```

```

43868 00004917 EBF9      jmp      short setdoscntry_fail
43869
43870 setdoscntry_any_codepage:      ;use the code_page_id of the country_id found.
43871 00004919 8B5404      mov      dx,[si+4]
43872
43873 setdoscntry_got_it:      ;found the matching entry
43874 0000491C 2E8916[264B]      mov      [cs:cntrycodepage_id],dx ;save code page id for this country.
43875 00004921 8B540A      mov      dx,[si+10]      ;get the file offset of country data
43876 00004924 8B4C0C      mov      cx,[si+12]
43877 00004927 880002      mov      ax,512      ;read 512 bytes
43878 0000492A E8DE00      call     readincontrolbuffer
43879 0000492D 72E3      jc       short setdoscntry_fail
43880
43881 0000492F 880C      mov      cx,[si]      ;get the number of entries to handle.
43882 00004931 46      inc      si
43883 00004932 46      inc      si      ;si -> first entry
43884
43885 setdoscntry_data:
43886 00004933 57      push     di      ;es:di -> dos_country_cdpdpg_info
43887 00004934 51      push     cx      ;save # of entry left
43888 00004935 56      push     si      ;si -> current entry in control buffer
43889
43890 00004936 8A4402      mov      al,[si+2]      ;get data entry id
43891 00004939 E8A400      call     getcountrydestination ;get the address of destination in es:di
43892 0000493C 727C      jc       short setdoscntry_data_next ;no matching data entry id in dos
43893
43894 0000493E 8B5404      mov      dx,[si+4]      ;get offset of data
43895 00004941 8B4C06      mov      cx,[si+6]
43896 00004944 880042      mov      ax,4200h
43897 00004947 F9      stc
43898 00004948 CD21      int      21h      ;move pointer
43899 0000494A 72C8      jc       short setdosdata_fail
43900
43901 0000494C BA0002      mov      dx,512      ;start of data buffer
43902 0000494F B91400      mov      cx,20      ;read 20 bytes only. we only need to
43903 00004952 B43F      mov      ah,3Fh      ;look at the length of the data in the file.
43904 00004954 F9      stc
43905 00004955 CD21      int      21h      ;read the country.sys data
43906 00004957 72BB      jc       short setdosdata_fail ;read failure
43907
43908 00004959 39C8      cmp      ax,cx
43909 0000495B 75B7      jne      short setdosdata_fail ; 13/05/2019
43910
43911 0000495D 8B5404      mov      dx,[si+4]      ;get offset of data again.
43912 00004960 8B4C06      mov      cx,[si+6]
43913 00004963 880042      mov      ax,4200h
43914 00004966 F9      stc
43915 00004967 CD21      int      21h      ;move pointer back again
43916 00004969 72A9      jc       short setdosdata_fail
43917
43918 0000496B 56      push     si
43919 0000496C BE0802      mov      si,(512+8)      ;get length of the data from the file
43920 0000496F 8B0C      mov      cx,[si]
43921 00004971 5E      pop      si
43922 00004972 BA0002      mov      dx,512      ;start of data buffer
43923 00004975 83C10A      add      cx,10      ;signature + a word for the length itself
43924 00004978 B43F      mov      ah,3Fh      ;read the data from the file.
43925 0000497A F9      stc
43926 0000497B CD21      int      21h
43927 0000497D 7295      jc       short setdosdata_fail
43928
43929 0000497F 39C8      cmp      ax,cx
43930 00004981 7591      jne      short setdosdata_fail
43931
43932 00004983 8A4402      mov      al,[si+2]      ;save data id for future use.
43933 00004986 BE0802      mov      si,(512+8)      ;si-> data buffer + id tag field
43934 00004989 8B0C      mov      cx,[si]      ;get the length of the file
43935 0000498B 41      inc      cx      ;take care of a word for lenght of tab
43936 0000498C 41      inc      cx      ;itself.
43937 0000498D 81F9F805      cmp      cx,(2048-512-8) ; 1528 ;fit into the buffer?
43938 00004991 7781      ja       short setdosdata_fail
43939
43940 ;if      bugfix
43941 00004993 E83100      call     setdbcs_before_copy
43942 ;endif
43943
43944 00004996 3C01      cmp      al,SetCountryInfo ; 1 ;is the data for setcountryinfo table?
43945 00004998 7511      jne      short setdoscntry_mov ;no, don't worry
43946
43947 0000499A 26FF7518      push     word [es:di+country_cdpdpg_info.ccMono_Ptr-country_cdpdpg_info.ccCountryInfoLen]
43948 ;push     word [es:di+24] ;cannot destroy ccmmono_ptr address. save them.
43949 0000499E 26FF751A      push     word [es:di+country_cdpdpg_info.ccMono_Ptr-country_cdpdpg_info.ccCountryInfoLen+2]
43950 ;push     word [es:di+26] ;at this time di -> cccountryinfoLen
43951
43952 000049A2 57      push     di      ;save di
43953
43954 ;push     ax
43955 ;mov      ax,[cs:cntrycodepage_id] ;do not use the code page info in country_info
43956 ;mov      [si+4],ax      ;use the saved one for this !!!!
43957 ;pop      ax
43958 ; 10/09/2023
43959 000049A3 2EFF36[264B]      push     word [cs:cntrycodepage_id]
43960 000049A8 8F4404      pop      word [si+4]
43961
43962 setdoscntry_mov:
43963 000049AB F3A4      rep      movsb      ;copy the table into dos
43964 000049AD 3C01      cmp      al,SetCountryInfo ;was the ccmmono_ptr saved?
43965 000049AF 7509      jne      short setdoscntry_data_next
43966
43967 000049B1 5F      pop      di      ;restore di
43968 000049B2 268F451A      pop      word [es:di+country_cdpdpg_info.ccMono_Ptr-country_cdpdpg_info.ccCountryInfoLen+2]
43969 ;pop      word [es:di+26] ;restore
43970 000049B6 268F4518      pop      word [es:di+country_cdpdpg_info.ccMono_Ptr-country_cdpdpg_info.ccCountryInfoLen]
43971 ;pop      word [es:di+24]
43972
43973 setdoscntry_data_next:
43974 000049BA 5E      pop      si      ;restore control buffer pointer
43975 000049BB 59      pop      cx      ;restore # of entries left
43976 000049BC 5F      pop      di      ;restore pointer to dso_country_cdpdpg
43977 000049BD 0334      add      si,[si]      ;try to get the next entry
43978 000049BF 46      inc      si
43979 000049C0 46      inc      si      ;take a word of entry length itself
43980 000049C1 49      dec      cx
43981 ; 10/09/2023
43982 000049C2 741B      jz       short setdoscntry_ok
43983 ;cmp      cx,0
43984 ;je       short setdoscntry_ok
43985 000049C4 E96CFF      jmp      setdoscntry_data
43986
43987 ; 18/12/2022
43988 ;setdoscntry_ok:
43989 ;retn
43990
43991 ;-----

```

```

43992
43993
43994
43995
43996
43997
43998 000049C7 3C07
43999 000049C9 7514
44000
44001
44002 000049CB 50
44003 000049CC 31C0
44004 000049CE 263905
44005 000049D1 740B
44006
44007
44008
44009
44010 000049D3 57
44011
44012
44013 000049D4 51
44014 000049D5 268B0D
44015
44016 000049D8 47
44017 000049D9 47
44018
44019 000049DA F3AA
44020 000049DC 59
44021
44022 000049DD 5F
44023
44024
44025 000049DE 58
44026
44027
44028 000049DF C3
44029
44030
44031
44032
44033
44034
44035
44036
44037
44038
44039
44040
44041
44042
44043
44044
44045
44046
44047
44048
44049
44050
44051
44052 000049E0 51
44053
44054 000049E1 83C74A
44055
44056 000049E4 268B0D
44057 000049E7 47
44058 000049E8 47
44059
44060
44061 000049E9 263805
44062 000049EC 7413
44063
44064 000049EE 26803D01
44065 000049F2 7405
44066
44067 000049F4 83C705
44068 000049F7 EB03
44069
44070
44071
44072 000049F9 83C729
44073
44074 000049FC E2EB
44075 000049FE F9
44076
44077
44078
44079 000049FF 59
44080 00004A00 C3
44081
44082
44083
44084 00004A01 47
44085
44086
44087
44088
44089
44090
44091
44092
44093 00004A02 3C01
44094 00004A04 74F9
44095
44096
44097
44098
44099 00004A06 26C43D
44100
44101 00004A09 59
44102 00004A0A C3
44103
44104
44105
44106
44107
44108
44109
44110
44111
44112
44113
44114
44115

; if bugfix
; 10/09/2023 - RetroDOS v4.2 IO.SYS (Optimization)

setdbcs_before_copy:
    cmp     al,SetDBCS ; 7 ; dbcs vector set?
    jne     short sdbcsbc ; jump if not

; 10/09/2023
    push    ax
    xor     ax,ax
    cmp     [es:di],ax ; 0
    je      short sdbcsbc_pop

; cmp word [es:di],0 ; zero byte data block?
; je short sdbcsbc ; jump if so

    push    di
; 10/09/2023
    ; push ax
    push    cx
    mov     cx,[es:di] ; load block length
    ; add di,2 ; points actual data
    inc     di
    inc     di
    ; xor al,al ; fill bytes
    rep     stosb ; clear data block
    pop     cx
    pop     ax
    pop     di

sdbcsbc_pop: ; 10/09/2023
    pop     ax
sdbcsbc:
setdoscntry_ok: ; 18/12/2022
    retn

; endif

;-----
getcountrydestination:
;-----
; get the destination address in the dos country info table.
;
; input: al - data id
; es:di -> dos_country_cdpdpg_info
; on return:
; es:di -> destination address of the matching data id
; carry set if no matching data id found in dos.
;-----

; 04/01/2023 - RetroDOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:4EB2h)

; 10/09/2023 - RetroDOS v4.2 IO.SYS (Optimization)
; (PCDOS 7.1 IBMBIO.COM - SYSINIT:50F9h)

    push    cx
    ; add di,74
    add     di,country_cdpdpg_info.ccNumber_of_entries
; skip the reserved area, syscodepage etc.
    mov     cx,[es:di] ; get the number of entries
    inc     di
    inc     di ; si -> the first start entry id

getcntrydest:
    cmp     byte [es:di],al
    je      short getcntrydest_ok

    cmp     byte [es:di],SetCountryInfo ; was it setcountryinfo entry?
    je      short getcntrydest_1

    add     di,5 ; next data id
    jmp     short getcntrydest_loop

getcntrydest_1:
    ; add di,41
    add     di,NEW_COUNTRY_SIZE+3 ; next data id
getcntrydest_loop:
    loop    getcntrydest
    stc
    jmp     short getcntrydest_exit

getcntrydest_exit:
; 10/09/2023
    pop     cx
    retn

getcntrydest_ok:
; 10/09/2023
    inc     di

; cmp al,SetCountryInfo ; 1 ; select country info?
; jne short getcntrydest_ok1
;
; ; inc di ; now di -> cccountryinfoflen
; jmp short getcntrydest_exit

; 10/09/2023
    cmp     al,SetCountryInfo ; 1 ; select country info?
    je      short getcntrydest_exit

getcntrydest_ok1:
; les di,[es:di+1] ; get the destination in es:di
; 10/09/2023
    les     di,[es:di]
; getcntrydest_exit:
    pop     cx
    retn

;-----
readincontrolbuffer:
;-----
; move file pointer to cx:dx
; read ax bytes into the control buffer. (should be less than 2 kb)
; si will be set to 0 hence ds:si points to the control buffer.
;
; entry: cx, dx offset from the start of the file where the read/write pointer
; be moved.
; ax - # of bytes to read

```



```

44116 ; bx - file handle
44117 ; ds - buffer seg.
44118 ;return: the control data information is read into ds:0 - ds:0200.
44119 ; cx,dx value destroyed.
44120 ; carry set if error in reading file.
44121 ;-----
44122
44123 00004A0B 50      push    ax          ;# of bytes to read
44124 00004A0C B80042 mov     ax,4200h
44125 00004A0F F9      stc
44126 00004A10 CD21    int     21h          ;move pointer
44127 00004A12 59      pop     cx          ;# of bytes to read
44128 00004A13 7209    jc     short ricb_exit
44129
44130 00004A15 31D2    xor     dx,dx          ;ds:dx -> control buffer
44131 00004A17 31F6    xor     si,si
44132 00004A19 B43F    mov     ah,3Fh        ;read into the buffer
44133 00004A1B F9      stc
44134 00004A1C CD21    int     21h          ;should be less than 1024 bytes.
44135 ricb_exit:
44136 00004A1E C3      retn
44137
44138 ;-----
44139
44140 ;! set_country_path procedure is not called from anywhere !
44141 ; Erdogan Tan - 04/08/2023
44142 %if 0
44143
44144 set_country_path:
44145
44146 ;-----
44147 ;in: ds - sysinitseg, es - confbot, si -> start of the asciiz path string
44148 ; dosinfo_ext, cntry_drv, cntry_root, cntry_path
44149 ; assumes current directory is the root directory.
44150 ;out: ds:di -> full path (cntry_drv).
44151 ; set the cntry_drv string from the country=,path command.
44152 ; ds, es, si value saved.
44153 ;-----
44154
44155 ; 04/01/2023 - Retrodos v4.2 (Modified MSDOS 6.21 IO.SYS)
44156 ; (SYSINIT:4EF4h)
44157
44158 ; 10/09/2023 - Retrodos v4.2 IO.SYS (Optimization)
44159 ; (Retrodos v5.0 Pre-Works)
44160 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:513Bh)
44161
44162 push    si
44163
44164 push    ds          ;switch ds, es
44165 push    es
44166 pop     ds
44167 pop     es          ;now ds -> confbot, es -> sysinitseg
44168
44169 call    chk_drive_letter ;current ds:[si] is a drive letter?
44170 jc     short scp_default_drv ;no, use current default drive.
44171
44172 mov     al,[si]
44173 inc     si
44174 inc     si          ;si -> next char after ":"
44175 jmp     short scp_setdrv
44176
44177 scp_default_drv:
44178 mov     ah,19h
44179 int     21h
44180 add     al,"A"      ;convert it to a character.
44181
44182 scp_setdrv:
44183 mov     [cs:cntry_drv],al ;set the drive letter.
44184 mov     di,cntry_path
44185 mov     al,[si]
44186 cmp     al,"\"
44187 je     short scp_root_dir
44188
44189 cmp     al,"/"      ;let's accept "/" as an directory delim
44190 ;je     short scp_root_dir
44191 ;jmp     short scp_path
44192 ; 04/01/2023
44193 jne     short scp_path
44194
44195 scp_root_dir:
44196 dec     di          ;di -> cntry_root
44197
44198 scp_path:
44199 call    move_asciiz  ;copy it
44200
44201 mov     di,cntry_drv
44202
44203 scpath_exit:
44204 push    ds          ;switch ds, es
44205 push    es
44206 pop     ds
44207 pop     es          ;ds, es value restored
44208
44209 pop     si
44210 retn
44211
44212 ;-----
44213
44214 chk_drive_letter:
44215 ;check if ds:[si] is a drive letter followed by ":".
44216 ;assume that every alpha character is already converted to upper case.
44217 ;carry set if not.
44218
44219 ; 04/01/2023 - Retrodos v4.2
44220
44221 push    ax
44222 cmp     byte [si],"A"
44223 ;jb     short cdletter_no
44224 jb     short cdletter_exit
44225 cmp     byte [si],"Z"
44226 ja     short cdletter_no
44227 cmp     byte [si+1],":"
44228 ;jne     short cdletter_no
44229 ;jmp     short cdletter_exit
44230 ; 04/01/2023
44231 je     short cdletter_exit
44232
44233 cdletter_no:
44234 stc
44235 cdletter_exit:
44236 pop     ax
44237 retn
44238
44239 %endif

```

```

44240
44241
44242
44243
44244
44245
44246
44247
44248
44249
44250
44251
44252
44253
44254
44255 00004A1F F604FF
44256 00004A22 A4
44257
44258
44259 00004A23 75FA
44260 00004A25 C3
44261
44262
44263
44264
44265
44266
44267
44268
44269 00004A26 0E
44270 00004A27 07
44271
44272 00004A28 89D6
44273
44274 00004A2A BA[B150]
44275 00004A2D BB[6E50]
44276
44277 00004A30 0E
44278 00004A31 1F
44279 00004A32 E81D00
44280
44281 00004A35 268A14
44282 00004A38 08D2
44283 00004A3A 7407
44284 00004A3C B402
44285 00004A3E CD21
44286 00004A40 46
44287 00004A41 EBF2
44288
44289 00004A43 89DA
44290 00004A45 E80A00
44291
44292
44293 00004A48 803E[5503]01
44294
44295
44296 00004A4D 7407
44297
44298
44299
44300 00004A4F E966E4
44301
44302
44303
44304
44305
44306
44307 00004A52 B409
44308 00004A54 CD21
44309
44310 00004A56 C3
44311
44312
44313
44314
44315
44316
44317
44318
44319
44320 00004A57 E80500
44321 00004A5A 7309
44322
44323
44324 00004A5C BA[BF4A]
44325
44326
44327
44328
44329
44330
44331
44332 00004A5F B43D
44333 00004A61 F9
44334 00004A62 CD21
44335
44336 00004A64 C3
44337
44338
44339 00004A65 89C3
44340
44341
44342
44343
44344
44345
44346
44347 00004A67 B80044
44348
44349 00004A6A CD21
44350
44351 00004A6C F6C280
44352 00004A6F 75F3
44353
44354 00004A71 B43E
44355 00004A73 CD21
44356 00004A75 EBE5
44357
44358
44359
44360
44361
44362
44363

;-----
move_asciiz:
;in: ds:si -> source es:di -> target
;out: copy the string until 0.
;assumes there exists a 0.
; 10/09/2023 - RetroDOS v4.2 IO.SYS (Optimization)
; (MSDOS 6.21 IO.SYS - SYSINIT:4F40h)
; (PCDOS 7.1 IBMBIO.COM - SYSINIT:5187h)
masciiz_loop:
; 10/09/2023
test byte [si],0FFh
movsb
;cmp byte [si-1],0 ; was it 0?
;jne short masciiz_loop
;jnz short masciiz_loop ; 10/09/2023
retn

;-----
; ds:dx points to string to output (asciz)
;
; prints <badld_pre> <string> <badld_post>
badfil:
push cs
pop es
mov si,dx
badload:
mov dx,badld_pre ; want to print config error
mov bx,crlfm
prnerr:
push cs
pop ds ; *
call print
prn1:
mov dl,[es:si]
or dl,dl
jz short prn2
mov ah,STD_CON_OUTPUT ; 2
int 21h
inc si
jmp short prn1
prn2:
mov dx,bx
call print
; 11/12/2022
; ds = cs ; *
cmp byte [donotshownum],1
; suppress line number when handling command.com
;cmp byte [cs:donotshownum],1
je short prnexit
; 18/12/2022
;call error_line
jmp error_line
;prnexit:
;retn

;-----
print:
mov ah,STD_CON_STRING_OUTPUT ; 9
int 21h
prnexit: ; 18/12/2022
retn

;-----
; open device pointed to by dx, al has access code
; if unable to open do a device open null device instead
; 02/11/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; (SYSINIT:3764h)
open_dev:
call open_file
jnc short open_dev3
open_dev1:
mov dx,nuldev
; 18/12/2022
;call open_file
;of_retn:
;retn
; 18/12/2022
;jmp short open_file
open_file:
mov ah,OPEN ; 3Dh
stc
int 21h
of_retn: ; 18/12/2022
retn
open_dev3:
mov bx,ax ; handle from open to bx
;xor ax,ax ; get device info
;mov ah,IOCTL ; 44h
;mov ax,(IOCTL<<8) ; 13/05/2019
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
;xor ax,ax
;mov ah,44h ; IOCTL
; 11/12/2022
mov ax,4400h ; IOCTL<<8
int 21h
test dl,10000000b ; 80h
jnz short of_retn
mov ah,CLOSE ; 3Eh
int 21h
jmp short open_dev1

;-----
; 18/12/2022
%if 0
open_file:
mov ah,OPEN ; 3Dh

```



```

44364         stc
44365         int     21h
44366         retn
44367 %endif
44368
44369 ;-----
44370
44371 ; test int24. return back to dos with the fake user response of "fail"
44372
44373 int24:
44374     mov     al,3           ; fail the system call
44375     00004A79 CF          ; return back to dos.
44376     iret
44377
44378 ; 19/04/2019 - Retro DOS v4.0
44379
44380 ;-----
44381 ; DATA
44382 ;-----
44383
44384 ;include copyrigh.inc           ; copyright statement
44385
44386 ; MSDOS 6.21 IO.SYS - SYSINIT:4FA3h
44387
44388 ;MsDosVersion6Copyr:
44389 ;     db     'MS DOS Version 6 (C)Copyright 1981-1993 Microsoft Corp '
44390 ;     db     'Licensed Material - Property of Microsoft All rights reserved '
44391
44392 ; 22/10/2022
44393 ; MSDOS 5.0 IO.SYS - SYSINIT:378ch
44394
44395 ; 28/12/2022
44396 %if 0
44397 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
44398 MsDosVersion5Copyr:
44399 ;     db     'MS DOS Version 5.00 (C)Copyright 1981-1991 Microsoft Corp '
44400 ;     db     'Licensed Material - Property of Microsoft All rights reserved '
44401 %endif
44402
44403 ; 13/04/2024 - Retro DOS v5.0
44404 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:51EAh (IBMBIO.COM offset 42266)
44405 %if 0
44406 IBMDOSV71COPYR:
44407 ;     db     'IBM DOS Version 7.1 (C)Copyright 1981-2002 IBM Corporation '
44408 ;     db     'Licensed Material - Property of IBM All rights reserved '
44409 %endif
44410
44411 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
44412 ; 22/10/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
44413 ; 20/04/2019 - Retro DOS v4.0
44414 ;BOOTMES:
44415 ;     db     13
44416 ;     db     10
44417 ;     db     "MS-DOS version "
44418 ;     db     MAJOR_VERSION + "0"
44419 ;     db     "."
44420 ;     db     (MINOR_VERSION / 10) + "0"
44421 ;     db     (MINOR_VERSION % 10) + "0"
44422 ;     db     13,10
44423 ;     db     "Copyright 1981-1993 Microsoft Corp.",13,10,"$"
44424 ;     ; 22/10/2022
44425 ;     db     "Copyright 1981-1991 Microsoft Corp.",13,10,"$"
44426 ;     ;
44427 ;     db     0
44428
44429 ; 01/01/2023 - Retro DOS v4.2
44430
44431 ; 28/12/2022 - Retro DOS v4.1
44432 ;MsDosVersion5Copyr:
44433 ;     db     13,10,"MS DOS Version 5.0"
44434 ;     db     13,10,"Copyright 1981-1991 Microsoft Corp.",13,10,"$",0
44435
44436 ; 12/12/2022
44437 db     0
44438 ; 12/12/2022
44439 ;BOOTMES:
44440 db     13,10
44441 ;;;db "Retro DOS v4.0 (Modified MSDOS 5.0) "
44442 ; 28/12/2022
44443 ;;;db "Retro DOS v4.1 (Modified MSDOS 5.0) "
44444 ; 01/01/2023
44445 ;db "Retro DOS v4.2 (Modified MSDOS 6.22) "
44446 ; 30/12/2023
44447 db     "Retro DOS v5.0 (Modified PCDOS 7.1) "
44448
44449 db     13,10
44450 ;db "by Erdogan Tan [2024] " ; 01/01/2024
44451 ;db "by Erdogan Tan [2026] " ; 19/01/2026
44452
44453 db     13,10
44454 db     13,10,"$",0
44455
44456 nuldev: db "NUL",0
44457 condev: db "CON",0
44458 auxdev: db "AUX",0
44459 prndev: db "PRN",0
44460
44461 ;IFDEF CONFIGPROC
44462 config: db "\"CONFIG.SYS",0
44463
44464 cntry_drv: db "A:"
44465 cntry_root: db "\"
44466 cntry_path: db "COUNTRY.SYS",0
44467
44468 ;db 52 dup (0)
44469 times 52 db 0
44470
44471 country_file_signature:
44472 db     0FFh,'COUNTRY'
44473
44474 cntrycodepage_id:
44475 dw     0
44476
44477 ;ENDIF ; CONFIGPROC
44478
44479 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
44480 ; (SYSINIT:5081h)
44481
44482 ; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
44483 %ifdef MULTI_CONFIG

```

```

44481 00004B28 00
44482 00004B29 40
44483
44484
44485
44486
44487
44488
44489
44490
44491 00004B2A 0C
44492 00004B2B 5C434F4D4D414E442E-
44493 00004B34 434F4D00
44494 00004B38 00<rep 33h>
44495
44496
44497
44498
44499 00004B6B 5C434F4D4D414E442E-
44500 00004B74 434F4D00
44501 00004B78 022F5000
44502 00004B7C 5C4D53444F535C434F-
44503 00004B85 4D4D414E442E434F4D-
44504 00004B8E 00
44505 00004B8F 0B413A5C4D53444F53-
44506 00004B98 202F5000
44507 00004B9C 5C444F535C434F4D4D-
44508 00004BA5 414E442E434F4D00
44509 00004BAD 09413A5C444F53202F-
44510 00004BB6 5000
44511 00004BB8 00
44512 00004BB9 022F50
44513 00004BBC 00<rep 7Dh>
44514
44515
44516 00004C39 00<rep 40h>
44517
44518
44519
44520
44521
44522
44523
44524 00004C79 00
44525
44526
44527
44528
44529
44530
44531
44532
44533 00004C7A 07
44534 00004C7B 00
44535 00004C7C 05
44536 00004C7D 00
44537 00004C7E 18
44538 00004C7F 00
44539
44540 00004C80 00
44541 00004C81 0000
44542 00004C83 00
44543 00004C84 01
44544 00004C85 00
44545 00004C86 0000
44546 00004C88 FF
44547 00004C89 00
44548 00004C8A 00<rep Ah>
44549 00004C94 0000<rep Ah>
44550 00004CA8 0000<rep Ah>
44551
44552 00004CBC 434F4E4649473D00
44553 00004CC4 4D454E5500
44554 00004CC9 434F4D4D4F4E00
44555
44556
44557
44558
44559
44560
44561
44562
44563
44564
44565
44566
44567
44568 00004CD0 015B5B
44569
44570 00004CD3 05425245414B43
44571 00004CDA 074255464645525342
44572 00004CE3 07434F4D4D454E5459
44573 00004CEC 07434F554E54525951
44574 00004CF5 0644455649434544
44575 00004CFD 0A4445564943454849-
44576 00004D06 474855
44577 00004D09 03444F5348
44578 00004D0E 08445249565041524D-
44579 00004D17 50
44580 00004D18 044643425358
44581 00004D1E 0546494C455346
44582
44583 00004D2E 07494E5354414C4C49
44584 00004D37 0B494E5354414C4C48-
44585 00004D40 49474857
44586 00004D44 094C41535444524956-
44587 00004D4D 454C
44588 00004D4F 075355424D454E554F
44589 00004D58 094D454E55434F4C4F-
44590 00004D61 5252
44591 00004D63 0B4D454E5544454641-
44592 00004D6C 554C5441
44593 00004D70 084D454E554954454D-
44594 00004D79 45

newcmd: db 0 ; non-zero if non-std shell specified
template: db 64 ; must precede command
;endif

;ifdef ROMEXEC
; db 7 ; size of command line (excl. null)
;command: db "COMMAND",0
; db 56 dup (0)
;else
; 02/11/2022
; db 12 ; size of command line (excl. null)
command: db "\COMMAND.COM",0
; db 51 dup (0)
; times 51 db 0
;endif

; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
;ifdef MULTI_CONFIG
command2: db "\COMMAND.COM",0 ; alternate commands to exec,
; db 2,"/P",0 ; followed by their respective alternate
command3: db "\MSDOS\COMMAND.COM",0; command lines
; db 11,"A:\MSDOS /P",0 ;(the drive letter are dynamically replaced)
command4: db "\DOS\COMMAND.COM",0 ;
; db 9,"A:\DOS /P",0 ;

def_switch:
; db 0 ; default switchchar (referenced as command_line-1)
;endif
; 30/10/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
command_line:
; db 2,"/P" ; default command.com args
; db 125 dup (0)
; times 125 db 0

pathstring:
; db 64 dup (0)
; times 64 db 0

; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:51D3h)
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; %if 0

dae_flag:
; db 0 ; MSDOS 6.21 IO.SYS - SYSINIT:51D2h

;ifdef MULTI_CONFIG

; 04/03/2022- Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS, SYSINIT)
MAX_MULTI_CONFIG equ 9 ; max # of multi-config menu items supported

; Beware of byte pairs accessed as words (see all "KEEP AFTER" notes below)

bMenuColor: db 07h ; 1Fh ; default fgnd/bgnd color
bMenuPage: db 0 ; menu video page (KEEP AFTER bMenuColor)
; db 5 ; video page function # (KEEP AFTER bMenuPage)
bLastCol: db 0 ; ending column on status line
bLastRow: db 24 ; row # of status line (KEEP AFTER bLastCol)
bDisableUI: db 0 ; 1=disable clean/interactive
; 2=disable default 2-second delay
; value saved from BIOS data area
bCRTPage: db 0 ; value saved from BIOS data area
wCRTStart: dw 0 ; value saved from BIOS data area
bQueryOpt: db 0 ; 0=off, 1=prompt all, 2=prompt none, 4=skip all
bDefBlock: db 1 ; default block #
bMaxBlock: db 0 ; maximum block #
offDefBlock: dw 0 ; offset of name of default block (if any)
secTimeout: db -1 ; 0FFh ; # of seconds for timeout (-1 == indefinite)
; # of seconds elapsed so far (KEEP AFTER secTimeout)
secElapsed: db 0
abBlockType: times MAX_MULTI_CONFIG+1 db 0 ; array of block types
aoffBlockName: times MAX_MULTI_CONFIG+1 dw 0 ; array of offsets of block names
aoffBlockDesc: times MAX_MULTI_CONFIG+1 dw 0 ; array of offsets of block descriptions

szBoot: db "CONFIG=",0
szMenu: db "MENU",0
szCommon: db "COMMON",0

;endif ;MULTI_CONFIG

; 10/09/2023
; MSDOS 6.21 IO.SYS - SYSINIT:5229h
; (PCDOS 7.1 IBMBIO.COM - SYSINIT:546Eh)

comtab: ; label byte
; cmd len command cmd code
; -----
;

;ifdef MULTI_CONFIG
; db 1, "[", CONFIG_BEGIN
;endif
; db 5, "BREAK", CONFIG_BREAK
; db 7, "BUFFERS", CONFIG_BUFFERS
; db 7, "COMMENT", CONFIG_COMMENT
; db 7, "COUNTRY", CONFIG_COUNTRY
; db 6, "DEVICE", CONFIG_DEVICE
; db 10, "DEVICEHIGH", CONFIG_DEVICEHIGH
; db 3, "DOS", CONFIG_DOS
; db 8, "DRIVPARM", CONFIG_DRIVPARM
; db 4, "FCBS", CONFIG_FCBS
; db 5, "FILES", CONFIG_FILES
;ifdef MULTI_CONFIG
; db 7, "INCLUDE", CONFIG_INCLUDE
;endif
; db 7, "INSTALL", CONFIG_INSTALL
; db 11, "INSTALLHIGH", CONFIG_INSTALLHIGH
; db 9, "LASTDRIVE", CONFIG_LASTDRIVE
;ifdef MULTI_CONFIG
; db 7, "SUBMENU", CONFIG_SUBMENU
; db 9, "MENCOLOR", CONFIG_MENCOLOR
; db 11, "MENUDEFAULT", CONFIG_MENUDEFAULT
; db 8, "MENUITEM", CONFIG_MENUITEM

```

```

44591
44592 00004D7A 0A4D554C5449545241-
44592 00004D83 434B4D
44593
44594 00004D86 074E554D4C4F434B4E
44595
44596 00004D8F 0352454D30
44597
44598 00004D94 0353455456
44599
44600 00004D99 055348454C4C53
44601
44602 00004DA0 06535441434B534B
44603
44604 00004DA8 085357495443484553-
44604 00004DB1 31
44605
44606
44607
44608
44609
44610
44611 00004DB2 07444F534441544154
44612 00004DBB 00
44613
44614
44615
44616
44617
44618
44619
44620
44621
44622
44623
44624
44625
44626
44627
44628
44629
44630
44631
44632
44633
44634
44635
44636
44637
44638
44639
44640
44641
44642
44643
44644
44645
44646
44647
44648
44649
44650
44651
44652 00004DBC 00
44653
44654 00004DBD 02
44655
44656 00004DBE 0000
44657
44658 00004DC0 5000
44659
44660
44661
44662
44663
44664 00004DC2 00
44665
44666
44667 00004DC3 0000
44668
44669 00004DC5 00
44670 00004DC6 0000
44671 00004DC8 0000
44672 00004DC9 0000
44673
44674 00004DCB 0000
44675
44676 00004DCD 00
44677 00004DCE 0000
44678
44679 00004DD0 0000
44680
44681 00004DD2 0000
44682
44683
44684
44685 00004DD4 00<rep 44h>
44686
44687
44688
44689
44690
44691
44692
44693
44694 00004E18 0000
44695
44696 00004E1A 00<rep FCh>
44697
44698
44699
44700
44701
44702
44703
44704
44705 00004F16 0200
44706 00004F18 0900
44707
44708 00004F1A 00
44709
44710
44711 00004F1B 0000
44712

;endif
db 10, "MULTITRACK", CONFIG_MULTITRACK

;ifdef MULTI_CONFIG
db 7, "NUMLOCK", CONFIG_NUMLOCK
;endif
db 3, "REM", CONFIG_REM
;ifdef MULTI_CONFIG
db 3, "SET", CONFIG_SET
;endif
db 5, "SHELL", CONFIG_SHELL
;if STACKSW
db 6, "STACKS", CONFIG_STACKS
;endif
db 8, "SWITCHES", CONFIG_SWITCHES

; 18/03/2025 (BugFix)
;db 0

; 10/09/2023
;adosdata: ; PCDOS 7.1 IBMBIO.COM - SYSINIT:5550h
; 13/04/2024 - Retro DOS v5.0
db 7, "DOSDATA", CONFIG_DOSDATA ; 'T'
db 0

;%endif ; 02/11/2022

; 01/01/2023 - Retro DOS v4.2
%if 0

comtab:
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; (SYSINIT:38EDh)
db 7, "BUFFERS", CONFIG_BUFFERS
db 5, "BREAK", CONFIG_BREAK
db 6, "DEVICE", CONFIG_DEVICE
db 10, "DEVICEHIGH", CONFIG_DEVICEHIGH
db 5, "FILES", CONFIG_FILES
db 4, "FCBS", CONFIG_FCBS
db 9, "LASTDRIVE", CONFIG_LASTDRIVE
db 10, "MULTITRACK", CONFIG_MULTITRACK
db 8, "DRIVPARM", CONFIG_DRIVPARM
db 6, "STACKS", CONFIG_STACKS
db 7, "COUNTRY", CONFIG_COUNTRY
db 5, "SHELL", CONFIG_SHELL
db 7, "INSTALL", CONFIG_INSTALL
db 7, "COMMENT", CONFIG_COMMENT
db 3, "REM", CONFIG_REM
db 8, "SWITCHES", CONFIG_SWITCHES
db 3, "DOS", CONFIG_DOS
db 0

%endif

; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:530Ch)

; 13/04/2024 - Retro DOS v5.0
; (PCDOS 7.1 IBMBIO.COM - SYSINIT:555Ah)

deviceparameters:
; A_DEVICEPARAMETERS <0,dev_3inch720kb,0,80>
devp.specialfunc: ; deviceparameters +
db 0 ; A_DEVICEPARAMETERS.DP_SPECIALFUNCTIONS
devp.devtype:
db 2 ; A_DEVICEPARAMETERS.DP_DEVICETYPE
devp.devattr:
dw 0 ; A_DEVICEPARAMETERS.DP_DEVICEATTRIBUTES
devp.cylinders:
dw 80 ; A_DEVICEPARAMETERS.DP_CYLINDERS

; 04/08/2023 - Retro DOS v4.2 IO.SYS (optimization)

;times 286 db 0
devp.mediatype: ; A_DEVICEPARAMETERS.DP_MEDIATYPE
db 0
devp.bpb: ; A_DEVICEPARAMETERS.DP_BPB
devp.bps: ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.BYTESPERSECTOR
dw 0
devp.secpersclus: ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERCLUSTER
db 0
dw 0 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.RESERVEDSECTORS
db 0 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.NUMBEROFFATS
dw 0 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.ROOTENTRIES
devp.totalsecs: ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.TOTALSECTORS
dw 0
devp.mediaid: ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.MEDIADESCRIPTOR
db 0
dw 0 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERFAT
devp.spt: ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.SECTORSPERTRACK
dw 0
devp.heads: ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.HEADS
dw 0

; 13/04/2024 - Retro DOS v5.0
; (PCDOS 7.1 IBMBIO.COM)
times 68 db 0 ; PCDOS 7.1 (FAT32 BPB)
;times 14 db 0 ; MSDOS 6.21
;dw 0 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.HIDDENSECTORS
;dw 0
;dw 0 ; A_DEVICEPARAMETERS.DP_BPB+A_BPB.BIGTOTALSECTORS
;dw 0
;times 6 db 0

devp.trktblents:
dw 0 ; A_DEVICEPARAMETERS.DP_TRACKTABLEENTRIES
devp.secttbl: ; A_DEVICEPARAMETERS.DP_SECTORTABLE
times 252 db 0 ; MAX_SECTORS_IN_TRACK * A_SECTORTABLE.size
; 63*4 bytes

; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:5430h)

; 13/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
; (PCDOS 7.1 IBMBIO.COM - SYSINIT:56B4h)

hlim: dw 2
slim: dw 9

drive: db 0

switches:
dw 0

```

```

44713 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
44714 ; (SYSINIT:5437h)
44715
44716 ; the following are the recommended bpbs for the media that
44717 ; we know of so far.
44718
44719 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
44720 ; MSDOS 5.0 IO.SYS - SYSINIT:3AA9h
44721
44722 ; 27/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
44723 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:56BBh
44724
44725 ; 48 tpi diskettes
44726
44727 bpb48t: dw 512
44728 db 2
44729 dw 1
44730 db 2
44731 dw 112
44732 dw 2*9*40 ; 720
44733 db 0FDh
44734 dw 2
44735 dw 9
44736 dw 2
44737 dd 0
44738 ; 27/12/2023
44739 times 28 db 0 ; FAT32 extensions (to BDS)
44740 db 90h
44741
44742 ; 96tpi diskettes
44743
44744 bpb96t: dw 512
44745 db 1
44746 dw 1
44747 db 2
44748 dw 224
44749 dw 2*15*80 ; 2400
44750 db 0F9h
44751 dw 7
44752 dw 15
44753 dw 2
44754 dd 0
44755 ; 27/12/2023
44756 times 28 db 0 ; FAT32 extensions (to BDS)
44757 db 90h
44758
44759 ; 3 1/2 inch diskette bpb
44760
44761 bpb35: dw 512
44762 db 2
44763 dw 1
44764 db 2
44765 dw 112
44766 dw 2*9*80 ; 1440
44767 db 0F9h
44768 dw 3
44769 dw 9
44770 dw 2
44771 dd 0
44772 ; 27/12/2023
44773 times 28 db 0 ; FAT32 extensions (to BDS)
44774 db 90h
44775
44776 bpb35h: dw 512
44777 db 1
44778 dw 1
44779 db 2
44780 dw 224
44781 dw 2*18*80 ; 2880
44782 db 0F0h
44783 dw 9
44784 dw 18
44785 dw 2
44786 dd 0
44787 ; 27/12/2023
44788 times 28 db 0 ; FAT32 extensions (to BDS)
44789 db 90h
44790
44791 ; m037 - BEGIN
44792
44793 bpb288: dw 512
44794 db 2
44795 dw 1
44796 db 2
44797 dw 240
44798 dw 2*36*80 ; 5760
44799 db 0F0h
44800 dw 9
44801 dw 36
44802 dw 2
44803 dd 0
44804 ; 27/12/2023
44805 times 28 db 0 ; FAT32 extensions (to BDS)
44806 db 90h
44807
44808 ; m037 - END
44809
44810 ; 12/05/2019
44811 align 2
44812
44813 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
44814 ; MSDOS 5.0 IO.SYS - SYSINIT:3B26h
44815
44816 ; 13/04/2024 - Retro DOS v5.0
44817 ; (PCDOS 7.1 IBMBIO.COM - SYSINIT:5738h)
44818
44819 bpbtable: dw bpb48t ; 48tpi drives
44820 dw bpb96t ; 96tpi drives
44821 dw bpb35 ; 3.5" drives
44822 ; the following are not supported, so default to 3.5" media layout
44823 dw bpb35 ; not used - 8" drives
44824 dw bpb35 ; not used - 8" drives
44825 dw bpb35 ; not used - hard files
44826 dw bpb35 ; not used - tape drives
44827 dw bpb35h ; 3-1/2" 1.44mb drive
44828 dw bpb35 ; ERIMO m037
44829 dw bpb288 ; 2.88 MB diskette drives m037
44830
44831 0000502C [1D4F]
44832 0000502E [534F]
44833 00005030 [894F]
44834
44835 00005032 [894F]
44836 00005034 [894F]
44837 00005036 [894F]
44838 00005038 [894F]
44839 0000503A [BF4F]
44840 0000503C [894F]
44841 0000503E [F54F]
44842

```

```

44837 switchlist:
44838 db      8,"FHSTDICN"          ; preserve the positions of n and
44839
44840 ;-----
44841 ; Messages
44842 ;-----
44843
44844 ; 19/04/2019 - Retro DOS v4.0
44845
44846 ; MSDOS 6.21 IO.SYS - SYSINIT:54D1h
44847
44848 00005049 00
44849 db      0
44850
44851 ; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
44852 ; MSDOS 5.0 IO.SYS - SYSINIT:3B44h
44853
44854 ; 13/04/2024
44855 ; MSDOS 6.22 IO.SYS - SYSINIT:559Eh
44856
44857 ; 13/04/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
44858 ; PCDOS 7.1 IBMBIO.COM - SYSINIT:5756h
44859
44860 badopm:
44861 db      0Dh,0Ah
44862 db      'Unrecognized command in CONFIG.SYS'
44863
44864
44865 crlfm:
44866 db      0Dh,0Ah,'$'
44867
44868 badparm:
44869 db      0Dh,0Ah
44870 db      'Bad command or parameters - $'
44871
44872
44873 badsiz_pre:
44874 db      0Dh,0Ah
44875 db      'Sector size too large in file $'
44876
44877
44878 badld_pre:
44879 db      0Dh,0Ah
44880 db      'Bad or missing $'
44881
44882
44883 badcom:
44884 db      'Command Interpreter',0
44885
44886
44887 badcountry:
44888 db      0Dh,0Ah
44889 db      'Invalid country code or code page',0Dh,0Ah,'$'
44890
44891
44892 badcountrycom:
44893 db      0Dh,0Ah
44894 db      'Error in COUNTRY command',0Dh,0Ah,'$'
44895
44896
44897 insufmemory:
44898 db      0Dh,0Ah
44899 db      'Insufficient memory for COUNTRY.SYS file',0Dh,0Ah,'$'
44900
44901
44902 badmem:
44903 db      0Dh,0Ah
44904 db      'Configuration too large for memory',0Dh,0Ah,'$'
44905
44906
44907 badblock:
44908 db      0Dh,0Ah
44909 db      'Too many block devices',0Dh,0Ah,'$'
44910
44911
44912 badstack:
44913 db      0Dh,0Ah
44914 db      'Invalid STACK parameters',0Dh,0Ah,'$'
44915
44916
44917 ; 18/12/2022
44918 ;badorder:
44919 ;db      0Dh,0Ah
44920 ;db      'Incorrect order in CONFIG.SYS line $'
44921 errorcmd:
44922 db      'Error in CONFIG.SYS line $'
44923
44924
44925 ; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
44926 ; (SYSINIT:566Eh)
44927
44928 ; 02/11/2023 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
44929 ;%if 0
44930
44931 OnOff: db      'ON'
44932 OnOff2: db      'OFF'
44933
44934 ; 04/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
44935 ; (SYSINIT:5673h)
44936 ;startmsg:
44937 ; db      'Starting MS-DOS...',0Dh,0Ah
44938 ; db      0Ah,0
44939
44940 ; 17/12/2023 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMBIO.COM)
44941 ; (SYSINIT:58F7h)
44942 startmsg:
44943 db      'Starting PC DOS...',0Dh,0Ah
44944
44945 db      0Ah,0
44946
44947 _$PauseMsg:
44948 ; 17/12/2023
44949 ;db      'Press any key to continue . . .',0Dh,0Ah,'$'
44950 ; 04/08/2023 (PCDOS 7.10 - IBMBIO.COM SYSINIT:590Dh)
44951 db      'Press any key to continue...',0Dh,0Ah,'$'
44952
44953 000051DB 507265737320616E79-
44954 000051E4 206B657920746F2063-
44955 000051ED 6F6E74696E75652E2E-

```

```

44925 000051F6 2E0D0A24
44926
44927
44928
44929 000051FA 504320444F53206973-
44929 00005203 20627970617373696E-
44929 0000520C 6720796F757220434F-
44929 00005215 4E4649472E53595320-
44929 0000521E 616E64204155544F45-
44929 00005227 5845432E4241542066-
44929 00005230 696C65732E0D0A24
44930
44931
44932
44933 00005238 504320444F53207769-
44933 00005241 6C6C2070726F6D7074-
44933 0000524A 20796F7520746F2063-
44933 00005253 6F6E6669726D206561-
44933 0000525C 636820434F4E464947-
44933 00005265 2E53595320636F6D6D-
44933 0000526E 616E642E0D0A24
44934
44935 00005275 0D0A
44936
44937
44938
44939
44940
44941
44942 00005277 2020504320444F5320-
44942 00005280 372E31205374617274-
44942 00005289 7570204D656E750D0A
44943 00005292 2020
44944 00005294 CD<rep 17h>
44945 000052AB 0D0A24
44946
44947 000052AE 2020456E7465722061-
44947 000052B7 2063686F6963653A20-
44947 000052C0 24
44948
44949 000052C1 46353D427970617373-
44949 000052CA 207374617274757020-
44949 000052D3 66696C65732046383D-
44949 000052DC 436F6E6669726D2065-
44949 000052E5 616368206C696E6520-
44949 000052EE 6F6620434F4E464947-
44949 000052F7 2E53595320
44950 000052FC 616E64204155544F45-
44950 00005305 5845432E424154205B-
44950 0000530E 205D24
44951
44952
44953
44954
44955 00005311 205B592C4E2C455343-
44955 0000531A 5D3F24
44956 0000531D 59455324
44957 00005321 4E4F2024
44958
44959 00005325 54696D652072656D61-
44959 0000532E 696E696E673A2024
44960
44961
44962
44963 00005336 456E74657220636F72-
44963 0000533F 72656374206E616D65-
44963 00005348 206F6620436F6D6D61-
44963 00005351 6E6420496E74657270-
44963 0000535A 72657465722028666F-
44963 00005363 72206578616D706C65-
44963 0000536C 2C20433A5C434F4D4D-
44963 00005375 414E442E434F4D29
44964 0000537D 0D0A24
44965
44966 00005380 50726F636573732041-
44966 00005389 55544F455845432E42-
44966 00005392 4154205B592C4E5D3F-
44966 0000539B 24
44967
44968
44969
44970
44971
44972
44973
44974
44975
44976
44977 0000539C 5741524E494E472120-
44977 000053A5 4C6F676963616C2064-
44977 000053AE 726976657320706173-
44977 000053B7 74205A3A2065786973-
44977 000053C0 7420616E642077696C-
44977 000053C9 6C2062652069676E6F-
44977 000053D2 7265640D0A24
44978
44979
44980
44981
44982
44983
44984
44985
44986
44987
44988
44989
44990
44991
44992 000053D8 526571756972656420-
44992 000053E1 73797374656D20636F-
44992 000053EA 6D706F6E656E742069-
44992 000053F3 73206E6F7420696E73-
44992 000053FC 74616C6C65640D0A24-
44992 00005405 00
44993
44994
44995
44996
44997
44998
44999
45000
45001

_cleanMsg:
;db 'MS-DOS is bypassing your CONFIG.SYS and AUTOEXEC.BAT files.',0dh,0ah,'$'
; 17/12/2023
db 'PC DOS is bypassing your CONFIG.SYS and AUTOEXEC.BAT files.',0dh,0ah,'$'

_interMsg:
;db 'MS-DOS will prompt you to confirm each CONFIG.SYS command.',0dh,0ah,'$'
; 17/12/2023
db 'PC DOS will prompt you to confirm each CONFIG.SYS command.',0dh,0ah,'$'

_menuHeader:
db 0dh,0ah
; 17/12/2023
;db ' MS-DOS 6.2 Startup Menu',0dh,0ah
;db ' '
;times 23 db (0cdh) ; ALT 205 ; '===== ' ; 06/08/2023
;db 0dh,0ah,'$'
; 04/08/2023 (PCDOS 7.10 - IBMBIO.COM SYSINIT:59A7h)
db ' PC DOS 7.1 Startup Menu',0dh,0ah

db ' '
times 23 db (0cdh) ; ALT 205 ; '===== ' ; 06/08/2023
db 0dh,0ah,'$'
_menuPrmpt:
db ' Enter a choice: $'

_statusLine:
db 'F5=Bypass startup files F8=Confirm each line of CONFIG.SYS '

db 'and AUTOEXEC.BAT [ ]$'

_interPrmpt:
;db '[Y,N]?$'
; 13/04/2024
; 04/08/2023
db '[Y,N,ESC]?$' ; PC DOS 7.1 - IBMBIO.COM

_$YES: db 'YES$'
_$NO: db 'NO $'
_$TimeOut:
db 'Time remaining: $'

badcomprompt:
;db 'Enter correct name of Command Interpreter (eg, C:\COMMAND.COM)'
; 13/04/2024 (PCDOS 7.1 IBMBIO.COM)
db 'Enter correct name of Command Interpreter (for example, C:\COMMAND.COM)'

db 0dh,0ah,'$'
_$AutoPrmpt:
db 'Process AUTOEXEC.BAT [Y,N]?$'

;endif ; 02/11/2022

; 01/01/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 IO.SYS)
; (SYSINIT:5840h)

; 02/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 IO.SYS)
; MSDOS 5.0 IO.SYS - SYSINIT:3CE0h

TooManyDrivesMsg:
db 'WARNING! Logical drives past Z: exist and will be ignored',0dh,0ah,'$'

; MSDOS 6.21 IO.SYS - SYSINIT:587Ch
;db 'wrong DBLSPACE.BIN version',0dh,0ah,'$'
;db 7 dup(0)

;times 7 db 0
; 02/11/2022 (MSDOS 5.0 IO.SYS SYSINIT compatibility)
; MSDOS 5.0 IO.SYS - SYSINIT:3D1Ch
; 09/12/2022
;times 4 db 0

; 08/04/2024 - Retro DOS v5.0
; PC DOS 7.1 IBMBIO.COM - SYSINIT:5B0Bh
baddblspace:
db 'Required system component is not installed',0dh,0ah,'$',0

;db 7 dup(0)

;-----
; 09/12/2022
;db 0

number3div equ ($-SYSINIT$)
number3mod equ (number3div % 16)

```



```

45002 %if (number3mod>0) & (number3mod<16) ; 17/09/2023
45003 00005406 00<rep Ah> times (16-number3mod) db 0
45004 %endif
45005 ;
45006 ;-----
45007 ; 09/12/2022 - MSDOS 5.0 IO.SYS:3D20h ;;; SI_end = 3D20h for MSDOS 5.0 IO.SYS
45008 ;-----
45009 ;
45010 ;MSDOS 6.21 IO.SYS - SYSINIT:5899h
45011 ;
45012 ;-----
45013 ; 20/04/2019 - Retro DOS v4.0
45014 ;
45015 ; 09/12/2022
45016 ;
45017 ;bss_start:
45018 ;
45019 ;ABSOLUTE bss_start
45020 ;
45021 ;alignb 16
45022 ;
45023 SI_end: ; SI_end equ $
45024 ;
45025 ;-----
45026 ;
45027 ;sysinitseg ends
45028 ;
45029 ; *****
45030 ;
45031 ; 04/01/2023 - MSDOS 6.21 SYSINIT:SI_end = SYSINIT:58A0h (IOSYS:9F46h)
45032 ; 09/12/2022 - MSDOS 5.0 SYSINIT:SI_end = SYSINIT:3D20h
45033 ;
45034 SYSINITSIZE equ SI_end - SYSINIT$
45035 DOSLOADSEG equ SYSINITSEG+((SYSINITSIZE+15)/16)
45036 ;
45037 ;-----
45038 ; End of Retro DOS v5.0 IBMBIO.COM (IO.SYS) source by Erdogan Tan (2023)
45039 ;-----
45040 ;
45041 ; 21/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0)
45042 ; 02/10/2023 - Retro DOS v5.0 (Modified PCDOS 7.1)
45043 ;-----
45044 ;
45045 ;-----
45046 ;
45047 ; START OF PCDOS 7.1 -IBMDOS.COM- KERNEL CODE (MSDOS.SYS) -will be relocated-
45048 ;-----
45049 ;
45050 ; 02/10/2023 - Retro DOS v5.0
45051 ; 04/01/2023 - Retro DOS v4.2
45052 ; 29/12/2022 - Retro DOS v4.1
45053 ; 18/03/2019 - Retro DOS v4.0
45054 ; 11/06/2018 - Retro DOS v3.0
45055 ;
45056 ;MSDOS_BIN_OFFSET:
45057 IBMDOS_BIN_OFFSET: ; this offset must be paragraph aligned
45058 ; ; 28/06/2019 ('msdos6.s')
45059 ; incbin 'MSDOS6.BIN' ; Retro DOS 4.0 - MSDOS 6.21 KERNEL
45060 ;
45061 ; 29/12/2022
45062 ; incbin 'MSDOS51.BIN' ; Retro DOS 4.1 - MSDOS 5.0+ KERNEL
45063 ;
45064 ; 29/09/2023 (PARASTART=3DE0h)
45065 ; 27/09/2023 (BugFix) ((PARASTART=3DD0h))
45066 ; 04/01/2023
45067 ; incbin 'MSDOS6.BIN' ; Retro DOS 4.2 - MSDOS 6.21+ KERNEL
45068 ;
45069 ; 04/02/2026
45070 ; 29/01/2026
45071 ; 06/08/2025
45072 ; 10/07/2024
45073 ; 07/07/2024
45074 ; 08/05/2024
45075 ; 14/04/2024
45076 ; 02/10/2023 - Retro DOS v5.0 - PCDOS 7.1 KERNEL
45077 incbin 'IBMDOS7.BIN'
45078 ;
45079 ; ; 28/12/2022 (BugFix)
45080 ; ; 22/12/2022
45081 ; ; 21/12/2022 ('msdos5.s')
45082 ; incbin 'MSDOS5.BIN' ; Retro DOS 4.0 - MSDOS 5.0+ KERNEL
45083 ;
45084 ; 28/09/2023
45085 ;msdos_bin_size equ $ - MSDOS_BIN_OFFSET
45086 ;
45087 align 2
45088 ;
45089 ; 21/12/2022
45090 ;;END_OF_KERNEL:
45091 ;END_OF_KERNEL equ $
45092 ;
45093 ; 28/09/2023
45094 S3SIZE equ $-$$
45095 KERNEL_SIZE equ S1SIZE+S2SIZE+S3SIZE
45096 ;=====
45097 ; END
45098 ;=====
45099 ; Retro DOS v5.0 by Erdogan Tan (Redevelopment of PC-DOS 7.1 KERNEL via NASM)
45100 ;-----
45101 ; APRIL 2024, ISTANBUL - TURKIYE.

```